

The Cloud Database Engineer

AWS RDS, Azure SQL, and Google Cloud for Production DBAs

Rajesh Guntupalli

Contents

AWS RDS, Azure SQL, and Google Cloud for Production DBAs	1
Chapter 1: Why Cloud Databases Changed Everything	3
1.1 The Shift from Infrastructure Owner to Service Consumer	3
1.2 What Managed Services Actually Manage	4
1.3 The Configuration Model: Parameter Groups, Server Parameters, and Database Flags	6
1.4 Networking and Security: The New pg_hba.conf	7
1.5 Operational Observability: From pg_stat to CloudWatch and Beyond	9
1.6 The Economics of Cloud Databases: What You Pay For and What It Costs You . . .	10
1.7 What Stayed the Same: The Fundamentals Don't Move	12
1.8 Choosing a Platform: RDS, Aurora, Azure SQL, Cloud SQL, and AlloyDB	14
1.9 The DBA's New Toolkit	15
Key Takeaways	15
Chapter 2: Managed vs Self-Managed: The Real Tradeoffs	17
What "Managed" Actually Means at the Infrastructure Layer	17
The Access Boundary: What You Can and Cannot Touch	18
Parameter Tuning: From postgresql.conf to Parameter Groups	19
High Availability: What You Delegate and What You Still Own	20
Backup, Restore, and the Point-in-Time Recovery Model	22
Terraform Provisioning: Encoding the Decisions You Cannot Delegate	23
Key Takeaways	25
Chapter 3: Cloud Database Landscape: AWS, Azure, GCP Side-by-Side	27
3.1 The Service Taxonomy: What Each Cloud Actually Offers	27
3.2 Compute and Storage Architecture: What Changes Under the Hood	29
3.3 Networking: VPCs, Private Link, and the End of pg_hba.conf	30
3.4 Authentication and Access Control: IAM Is the New pg_hba.conf	33
3.5 High Availability and Disaster Recovery: Comparing the Failover Mechanics	35
3.6 Backup and Point-in-Time Recovery: What "Managed" Actually Means	37
3.7 Performance Monitoring: What You Get, What You Have to Build	39
3.8 The Operational Cost Model: Thinking in Cloud Units	41
Key Takeaways	43
Chapter 4: The Cloud DBA Role: What Changes, What Stays	45
4.1 The Managed Service Contract: What AWS, Azure, and GCP Actually Own	45
4.2 Access Patterns and Permissions: From pg_hba.conf to IAM	46

4.3 Configuration Management: Parameter Groups, Server Parameters, and What You Cannot Touch	47
4.4 High Availability and Failover: Understanding What the Cloud Gives You	49
4.5 Monitoring: CloudWatch, Azure Monitor, and Cloud Logging vs. What You Had Before	51
4.6 The New DBA Work: Cost Management, IaC, and Architecture Ownership	53
4.7 What Genuinely Does Not Change	56
4.8 Organizational Changes: The DBA in Cloud-Native Teams	58
Key Takeaways	58
Chapter 5: Cloud Database Architecture Patterns	61
5.1 High Availability and Failover Architecture	61
5.2 Read Scaling and Replica Patterns	64
5.3 Storage Architecture and Auto-Scaling	66
5.4 Network Architecture: VPC, Private Link, and Connectivity	67
5.5 Parameter Tuning in Managed Environments	69
5.6 Serverless and Auto-Scaling Database Patterns	72
5.7 Multi-Region and Disaster Recovery Architecture	74
5.8 Observability Architecture: Metrics, Logs, and Tracing	76
5.9 Cost Architecture: Designing for Efficiency	78
5.10 Extension and Feature Constraints by Cloud Provider	80
Key Takeaways	82
Chapter 6: RDS Deep Dive: Engines, Instance Types, and Storage	83
6.1 Engine Selection and Version Strategy	83
6.2 Instance Families and Sizing	85
6.3 RDS Storage: gp2, gp3, io1, io2, and Magnetic	86
6.4 Parameter Groups: The Replacement for postgresql.conf	90
6.5 Multi-AZ, Read Replicas, and High Availability Architecture	92
6.6 Backup, Restore, and PITR	95
6.7 Upgrade Paths and Maintenance Windows	96
6.8 RDS Operational Monitoring and Alerting	98
Key Takeaways	101
Chapter 7: Aurora PostgreSQL vs RDS PostgreSQL: When to Choose Which	103
The Architectural Divide: What's Actually Different Under the Hood	103
When RDS PostgreSQL Is the Right Answer	104
When Aurora PostgreSQL Is the Right Answer	105
Provisioning and Configuration: Hands-On Comparison	106
Failover, Replication, and Connection Management	108
Monitoring, Limits, and Production Gotchas	110
Migration Path: RDS PostgreSQL to Aurora PostgreSQL	112
The Decision Framework	114
Key Takeaways	115
Chapter 8: Aurora MySQL vs RDS MySQL	117
8.1 Architectural Differences: Storage Layer and Replication Model	117
8.2 Provisioning Aurora MySQL vs RDS MySQL with Terraform and AWS CLI	118
8.3 Parameter Groups: What You Can and Cannot Tune	120

8.4 Connection Management: RDS Proxy and Aurora	122
8.5 Failover, Backups, and Aurora-Specific Features	124
8.6 Monitoring Aurora MySQL vs RDS MySQL	127
8.7 Schema Changes and Online DDL in Aurora vs RDS MySQL	129
8.8 Choosing Between Aurora MySQL and RDS MySQL	130
Key Takeaways	132
Chapter 9: Multi-AZ vs Read Replicas: Architecture Decisions	133
9.1 Multi-AZ: What It Is, What It Isn't, and How It Actually Works	133
9.2 Read Replicas: Asynchronous Replication and the Lag Problem	134
9.3 Configuring Multi-AZ and Read Replicas Across Providers	135
9.4 Failover Behavior: What Actually Happens During the Event	139
9.5 Architectural Decision Framework	141
9.6 Testing Failover in Production-Like Environments	145
9.7 Cost Implications of Multi-AZ vs Read Replica Choices	147
Key Takeaways	148
Chapter 10: RDS Proxy: Connection Pooling at Scale	149
10.1 Why Connection Pooling Is Different in the Cloud	149
10.2 Creating and Configuring RDS Proxy	151
10.3 IAM Authentication Through the Proxy	155
10.4 Monitoring Proxy Performance and Connection Pinning	158
10.5 Failover Behavior: What RDS Proxy Changes (and What It Doesn't)	162
10.6 Sizing and Scaling Considerations	164
10.7 Limitations and When Not to Use RDS Proxy	165
10.8 Practical Runbook: Diagnosing a Connection Pool Exhaustion Event	166
Key Takeaways	169
Chapter 11: Aurora Serverless v2: Auto-Scaling in Practice	171
11.1 Aurora Capacity Units: What You're Actually Scaling	171
11.2 Provisioning Aurora Serverless v2 with the AWS CLI and Terraform	172
11.3 Scaling Behavior: What Triggers It, How Fast It Responds	174
11.4 RDS Proxy Integration and the Connection Management Layer	175
11.5 Monitoring, Alerting, and Cost Governance	178
11.6 Writer/Reader Asymmetry and Failover Behavior	180
11.7 Practical Capacity Sizing: From Workload Profile to ACU Bounds	181
11.8 Limitations and When Serverless v2 Is Not the Right Choice	182
Key Takeaways	183
Chapter 12: Parameter Groups, Option Groups, and Tuning on RDS	185
12.1 How Parameter Groups Work: The RDS Configuration Model	185
12.2 Creating and Managing Parameter Groups	186
12.3 Option Groups: SQL Server and MySQL Extensions	188
12.4 Production Tuning Patterns for RDS PostgreSQL	190
12.5 Tuning Limits: What RDS Won't Let You Do	192
12.6 RDS MySQL and Aurora MySQL Parameter Tuning	193
12.7 RDS SQL Server Parameter Tuning	195
12.8 Using RDS Performance Insights with Parameter Tuning	196

12.9 Parameter Group Change Management in Production	198
12.10 Monitoring Parameter Group Effectiveness	200
Key Takeaways	201
Chapter 13: Azure SQL Database vs Managed Instance vs SQL Server on VM	203
13.1 The Three Deployment Models: Mapping the Landscape	203
13.2 Provisioning Each Model: Infrastructure as Code Patterns	204
13.3 Networking: Private Endpoints, VNet Integration, and the Public Internet	207
13.4 High Availability and Business Continuity	209
13.5 Security: Authentication, Encryption, and Compliance Patterns	211
13.6 Performance Tuning: Service Tiers, Resource Limits, and the Query Store	214
13.7 Migration Patterns: Moving to Each Deployment Model	216
13.8 Cost Optimization: Choosing the Right Tier and Right-Sizing	219
13.9 Production Gotchas: What the Documentation Does Not Tell You	220
13.10 Choosing Your Model: A Decision Framework	224
Key Takeaways	225
Chapter 14: Elastic Pools: Multi-Tenant Cost Optimization	227
14.1 The Multi-Tenant Database Problem: Why Dedicated Instances Fail at Scale	227
14.2 Azure SQL Elastic Pools: The Reference Implementation	229
14.3 Aurora Serverless v2: Fractional Scaling for AWS Multi-Tenancy	231
14.4 RDS Proxy: Connection Pooling as Cost Optimization Infrastructure	233
14.5 Google Cloud SQL and AlloyDB: Consolidation Strategies	236
14.6 Per-Tenant Resource Governance: Preventing the Noisy Neighbor	239
14.7 Tenant Tier Management: Mapping SaaS Plans to Resource Configurations	242
14.8 Cost Attribution: Knowing What Each Tenant Actually Costs	244
Key Takeaways	245
Chapter 15: Azure SQL Hyperscale: When You Need It	247
What Hyperscale Actually Is (And What It Isn't)	247
Provisioning Hyperscale: Configuration That Matters	249
Named Replicas: Read Scale-Out Without the Traditional Overhead	252
Migrating Into Hyperscale: Patterns That Work in Production	253
Production Gotchas and Operational Patterns	256
Monitoring Hyperscale in Production	259
Migrating Out of Hyperscale	261
Cost Modeling: When Hyperscale Is the Right Economic Choice	263
Key Takeaways	263
Chapter 16: Active Geo-Replication and Failover Groups	265
16.1 The Geography Problem: What You're Actually Solving	265
16.2 Aurora Global Database: Architecture and Configuration	266
16.3 Azure SQL Active Geo-Replication and Failover Groups	269
16.4 Google Cloud Spanner and Cloud SQL: Multi-Region Strategies	272
16.5 Terraform Patterns for Multi-Region Database Infrastructure	274
16.6 Failover Automation and DR Drill Patterns	279
16.7 RPO Measurement and Continuous Validation	281
16.8 Common Failure Modes and Operational Traps	283

16.9 Geo-Replication Checklist for Production Readiness	284
Key Takeaways	285
Chapter 17: Azure SQL Security: TDE, Always Encrypted, Ledger	287
17.1 Transparent Data Encryption: What It Does (and Doesn't) Protect	287
17.2 Always Encrypted: Column-Level Encryption the Right Way	289
17.3 Always Encrypted with Secure Enclaves	292
17.4 Azure SQL Ledger: Tamper-Evident Tables for Compliance	292
17.5 Layering the Security Stack: TDE + Always Encrypted + Ledger Together	295
17.6 Operational Monitoring and Alerting for Security Features	297
17.7 Migration Scenarios: Adding Security to Existing Databases	300
Key Takeaways	301
Chapter 18: Azure Arc-Enabled SQL Server: Hybrid Cloud	303
18.1 Understanding Azure Arc Architecture for SQL Server	303
18.2 Onboarding SQL Server Instances to Arc	304
18.3 Azure Monitor and Log Analytics Integration	307
18.4 Microsoft Defender for SQL via Arc	309
18.5 Azure Policy for Hybrid SQL Server Governance	311
18.6 Arc as a Migration Bridge	313
18.7 Azure AD Authentication for Arc SQL Server	315
18.8 Operational Runbooks for Arc-Managed SQL Server	317
18.9 Limitations and When Not to Use Arc	319
Key Takeaways	320
Chapter 19: Cloud SQL: PostgreSQL and SQL Server on GCP	323
19.1 Cloud SQL Architecture: What GCP Manages and What You Still Own	323
19.2 Provisioning Cloud SQL Instances: PostgreSQL and SQL Server	324
19.3 Database Flags: Configuring PostgreSQL and SQL Server Parameters	327
19.4 High Availability, Replication, and Failover	328
19.5 IAM Authentication and Network Security	330
19.6 Backup, Recovery, and PITR	331
19.7 Performance Tuning: PostgreSQL on Cloud SQL	334
19.8 Performance Tuning: SQL Server on Cloud SQL	336
19.9 Monitoring, Alerting, and Operational Visibility	338
19.10 Migrations to Cloud SQL: Practical Patterns	340
Key Takeaways	342
Chapter 20: AlloyDB: PostgreSQL-Compatible with ML Built In	345
20.1 Architecture: What AlloyDB Actually Is Under the Hood	345
20.2 Provisioning AlloyDB Clusters and Instances	348
20.3 IAM Authentication and Access Control	350
20.4 ML Inference with <code>google_ml_integration</code>	353
20.5 Migrating PostgreSQL Workloads to AlloyDB	356
20.6 AlloyDB vs Cloud SQL for PostgreSQL: When to Use Which	359
20.7 Monitoring AlloyDB in Production	360
20.8 High Availability and Backup in AlloyDB	362
Key Takeaways	364

Chapter 21: Spanner: When You Need Global Consistency	365
21.1 TrueTime and External Consistency: What It Actually Means	365
21.2 Spanner’s Data Model: Interleaving, Keys, and Schema Design	366
21.3 Provisioning and Configuration: Instances, Databases, and Processing Units	368
21.4 Transactions, Mutations, and the Application Layer	370
21.5 Observability, Operational Patterns, and Migration Realities	373
21.6 Query Execution and the Spanner Query Optimizer	376
Key Takeaways	378
Chapter 22: BigQuery for Analytics: When to Push Workloads Out	379
22.1 Understanding Why OLTP Engines Break Under Analytics Load	379
22.2 BigQuery Architecture: What a DBA Needs to Know	380
22.3 Loading Data from RDS, Cloud SQL, and AlloyDB into BigQuery	382
22.4 Configuring BigQuery for Production Use	384
22.5 Query Patterns, Optimization, and What Surprises Production DBAs	386
22.6 Monitoring, Alerting, and Operational Runbooks	389
22.7 When Not to Use BigQuery: Recognizing the Antipatterns	391
22.8 Integrating BigQuery with Your Broader Data Architecture	392
Key Takeaways	393
Chapter 23: AlloyDB Omni: Running Google’s Database On-Premises	395
23.1 What AlloyDB Omni Actually Is — and What It Is Not	395
23.2 Deploying AlloyDB Omni on Bare Metal and Virtual Machines	396
23.3 Connecting AlloyDB Omni to Google Cloud Services	399
23.4 Backup, HA, and the Operational Gap	401
23.5 Performance Tuning the Columnar Engine	403
23.6 Upgrading AlloyDB Omni Without Downtime	406
23.7 Edge Deployment Patterns	408
23.8 Security Hardening for Production Omni Deployments	410
Key Takeaways	412
Chapter 24: Multi-Cloud Database Strategy: Benefits and Traps	413
24.1 Why Organizations Go Multi-Cloud (And Why Some Shouldn’t)	413
24.2 Architecture Patterns That Actually Work in Production	414
24.3 Cross-Cloud Networking: The Layer That Breaks Everything	417
24.4 Data Consistency and the CAP Theorem in Practice	419
24.5 Identity, Access, and the Security Boundary Problem	422
24.6 Schema Management Across Cloud Boundaries	423
24.7 Observability Across Cloud Boundaries	426
24.8 Failover Runbooks and the Discipline of Practice	428
24.9 When to Walk Away from Multi-Cloud	430
Key Takeaways	431
Chapter 25: Cross-Cloud Replication Architectures	433
25.1 Understanding the Cross-Cloud Replication Landscape	433
25.2 Logical Replication: PostgreSQL Across AWS, Azure, and GCP	435
25.3 AWS DMS for Cross-Cloud CDC Replication	437
25.4 Azure SQL as a Cross-Cloud Replication Target	440

25.5 Handling Schema Changes and DDL Propagation	443
25.6 Failover and Switchover in Cross-Cloud Architectures	445
25.7 Conflict Resolution in Bidirectional Cross-Cloud Replication	447
25.8 Cost Modeling for Cross-Cloud Replication	448
Key Takeaways	449
Chapter 26: Data Sovereignty and Residency Compliance	451
26.1 Understanding the Regulatory Landscape for Cloud DBAs	451
26.2 AWS RDS and Aurora: Region, Replication, and Key Custody	452
26.3 Azure SQL: Geo-Replication, Paired Regions, and Data Boundary	455
26.4 Google Cloud SQL, AlloyDB, and Spanner: Residency Controls	457
26.5 Audit Logging for Residency Compliance	459
26.6 Data Classification and Residency-Aware Schema Design	463
26.7 Right to Erasure and the Database Engineering Challenge	465
26.8 Operationalizing Sovereignty: Runbooks and Governance	468
Key Takeaways	470
Chapter 27: Hybrid Cloud: On-Premises and Cloud Together	473
27.1 Network Architecture: Connecting Your Data Center to the Cloud	473
27.2 Data Synchronization Patterns Across Boundaries	475
27.3 Hybrid Failover and Disaster Recovery	478
27.4 Hybrid Identity and Access Management	480
27.5 Cross-Boundary Query Patterns and Their Performance Implications	482
27.6 Monitoring a Hybrid Database Environment	485
27.7 Managing Schema Changes Across Hybrid Environments	488
27.8 Cost Engineering in Hybrid Architectures	490
Key Takeaways	491
Chapter 28: Avoiding Vendor Lock-In Without Over-Engineering	493
28.1 Understanding What Lock-In Actually Means in Practice	493
28.2 The Abstraction Layer Trap — and Where Abstractions Actually Help	494
28.3 Choosing Which Proprietary Features Are Worth the Lock-In	496
28.4 Portable Infrastructure Patterns That Earn Their Complexity	498
28.5 Realistic Migration Paths — Planning the Exit You May Never Take	501
28.6 When Multi-Cloud Database Architectures Are Actually Justified	505
28.7 Practical Portability Audit: What to Document Right Now	507
Key Takeaways	509
Chapter 29: Backup and Restore in Cloud: Automated Snapshots vs Manual	511
29.1 How Automated Snapshots Work Under the Hood	511
29.2 Configuring Automated Backup Windows and Retention	512
29.3 Manual Snapshots and On-Demand Backups	515
29.4 Point-in-Time Recovery: Configuration and Execution	516
29.5 Cross-Region and Cross-Account Backup Strategies	519
29.6 Restore Testing and Validation Frameworks	522
29.7 Logical Backups as a Supplement to Snapshots	529
29.8 Backup Encryption, Access Control, and Compliance	531
29.9 Backup Cost Management	534

Key Takeaways	536
Chapter 30: Monitoring Cloud Databases: CloudWatch, Azure Monitor, Cloud Logging	539
30.1 What You Lost and What You Gained: Monitoring Philosophy for Managed Databases	539
30.2 CloudWatch: Monitoring AWS RDS and Aurora in Production	540
30.3 Azure Monitor: Metrics, Diagnostics, and Log Analytics for Azure SQL	543
30.4 Google Cloud Monitoring and Cloud Logging for Cloud SQL and AlloyDB	547
30.5 Building Effective Dashboards Across All Three Platforms	550
30.6 Alerting Strategy: Avoiding Alert Fatigue Without Missing Real Incidents	553
30.7 Common Instrumentation Gaps and How to Close Them	557
30.8 Integrating Cloud Database Monitoring with Existing Observability Stacks	560
Key Takeaways	562
Chapter 31: IAM-Based Database Authentication	563
31.1 Why IAM Authentication Changes the Security Model	563
31.2 IAM Authentication on AWS RDS and Aurora	564
31.3 Azure SQL and Entra ID (Azure Active Directory) Authentication	567
31.4 Google Cloud SQL and AlloyDB IAM Authentication	569
31.5 Connection Pooling and Token Renewal in Production	571
31.6 Auditing and Monitoring IAM-Authenticated Connections	574
31.7 Troubleshooting IAM Authentication Failures	577
31.8 Security Hardening Beyond Basic IAM Authentication	579
Key Takeaways	580
Chapter 32: VPC, Private Endpoints, and Network Security	583
32.1 Virtual Networks: The Foundation of Cloud Database Isolation	583
32.2 Private Endpoints and Private Link: Eliminating Public Exposure	585
32.3 Security Groups, Firewall Rules, and the Principle of Least Privilege	589
32.4 Cross-VPC Connectivity: Peering, Transit Gateway, and PrivateLink	590
32.5 IAM Authentication vs. Network Authentication: Combining Database and Network Security	592
32.6 TLS Enforcement and Certificate Management	595
32.7 Auditing Network Access and Detecting Anomalies	597
32.8 Network Security Automation and Infrastructure as Code	599
32.9 Common Misconfigurations and How to Find Them	602
Key Takeaways	604
Chapter 33: Database Migration to Cloud: Practical Runbook	605
33.1 Migration Assessment: Knowing What You're Moving Before You Move It	605
33.2 AWS DMS: Full Load and Change Data Capture	608
33.3 Azure Database Migration Service and SQL Server Migration	611
33.4 GCP Database Migration Service	615
33.5 Cutover Runbook: The Final 60 Minutes	617
33.6 Rollback Plan	622
33.7 Post-Migration Optimization	623
Key Takeaways	625

Chapter 34: Zero-Downtime Cloud Migrations	627
34.1 Pre-Migration Assessment: What You Must Know Before You Touch Anything . . .	627
34.2 Continuous Replication with AWS DMS	629
34.3 Schema Conversion and Heterogeneous Migrations	632
34.4 Azure and GCP Migration Paths	634
34.5 Cutover Orchestration	636
34.6 Post-Migration Validation	642
34.7 Rollback Strategy and Post-Migration Cleanup	644
Key Takeaways	646
Chapter 35: Query Performance Tuning on Managed Services	647
35.1 Capturing and Identifying Slow Queries	647
35.2 Reading Execution Plans on Managed Services	649
35.3 Index Tuning in the Cloud: What Works and What Doesn't	651
35.4 Wait Event Analysis and Resource Bottleneck Identification	653
35.5 Parameter Tuning on Managed Services	656
35.6 Query Rewrites and Application-Level Optimizations	659
35.7 Monitoring Query Performance Over Time	661
Key Takeaways	664
Chapter 36: Storage Tiers: gp2 vs gp3 vs io1 vs io2	667
Understanding the Storage Landscape: What Replaced Your SAN	667
io1 vs io2: When You Need Guaranteed IOPS	669
Configuring Storage: CLI, Terraform, and Modification Patterns	670
Storage Autoscaling, Modification Limits, and the Gotchas Nobody Tells You	674
Benchmarking Your Storage Tier Before Migrating Production	675
Workload-Specific Recommendations	679
Decision Framework: Picking the Right Tier	682
Monitoring Storage Health in Production	682
Migration Playbook: gp2 to gp3 at Scale	685
Key Takeaways	686
Chapter 37: Read Replica Economics: When They Pay Off	689
37.1 How Cloud Read Replicas Differ from On-Premises Streaming Replication	689
37.2 The Real Cost Model: What You're Actually Paying For	690
37.3 When Replicas Actually Pay Off: Workload Patterns That Justify the Cost	692
37.4 Provisioning and Routing: Getting the Architecture Right	694
37.5 Replica Lag: Measuring, Managing, and Designing Around It	700
37.6 Operational Patterns: Replica Promotion, Failover, and Maintenance	702
37.7 Cost Optimization Strategies for Replica Fleets	704
37.8 Cross-Cloud Comparisons: Choosing the Right Read Scaling Tool	706
Key Takeaways	707
Chapter 38: Reserved Instances vs On-Demand: The Math	709
38.1 How Cloud Database Pricing Works Under the Hood	709
38.2 Running the Break-Even Analysis	710
38.3 Reserved Instance Mechanics: What the Contracts Actually Say	713
38.4 Sizing Commitments Against Real Workload Data	715

38.5 Cross-Cloud Comparison: Where Each Provider’s Model Has Edges	718
38.6 Auditing Existing Commitments for Waste	719
38.7 The RI Purchase Workflow for Production Teams	721
Key Takeaways	724
Chapter 39: Right-Sizing Cloud Database Instances	727
39.1 Understanding the Cloud Instance Taxonomy	727
39.2 Identifying Sizing Mismatches from Cloud Metrics	728
39.3 Vertical Scaling: Resizing the Instance	730
39.4 Storage Right-Sizing: IOPS, Throughput, and Autoscaling	732
39.5 Horizontal Scaling Patterns and Read Replica Sizing	734
39.6 Automated Right-Sizing with Scheduled Scaling	736
39.7 Cost Allocation and Showback Reports	740
39.8 Right-Sizing During Migration Waves	741
Key Takeaways	744
Chapter 40: The Cloud DBA’s Cost and Performance Dashboard	745
40.1 The Cost-Performance Relationship in Cloud Databases	745
40.2 Building the Unified Monitoring Foundation	746
40.3 Query-Level Cost Attribution	748
40.4 Cost Governance with Terraform and Tagging	751
40.5 The Production Dashboard: Queries, Views, and Composite Metrics	756
40.6 Reserved Capacity, Right-Sizing, and the 30-Day Review Cadence	759
Key Takeaways	763

AWS RDS, Azure SQL, and Google Cloud for Production DBAs

By Rajesh Guntupalli

Last updated: July 30, 2026

Chapter 1: Why Cloud Databases Changed Everything

If you have spent years managing PostgreSQL clusters on bare metal, tuning SQL Server on VMware, or babysitting Oracle RAC through a 2 AM failover, this chapter is your orientation briefing. The cloud did not simply move databases to someone else’s hardware — it fundamentally restructured what database engineering means, what DBAs are responsible for, and where the real leverage points are in a modern production system. Before diving into AWS RDS parameter groups or Azure SQL Elastic Pools, you need to understand what changed, what stayed the same, and what you have actually traded away in exchange for managed infrastructure.

1.1 The Shift from Infrastructure Owner to Service Consumer

When you managed an on-premises PostgreSQL 14 cluster, you owned the entire stack. You decided the kernel version, the filesystem (XFS over ext4 for most serious shops), the storage controller queue depth, the NIC bonding configuration, and the `vm.swappiness` setting. You wrote the `postgresql.conf`. You managed `pg_hba.conf` line by line. Failure meant your pager went off. Victory meant the database stayed up without anyone noticing.

Cloud managed databases make a different bargain. AWS RDS, Azure SQL Database, and Google Cloud SQL take ownership of the infrastructure layer — hardware provisioning, OS patching, storage volume management, and replication topology — and hand you back a connection string and a set of configuration knobs that are smaller than what you had before, but much faster to operate.

This is not a demotion. It is a role shift. The DBA who spent 40% of their week on storage provisioning tickets and OS-level tuning can redirect that time toward query optimization, schema design, connection pooling strategy, and multi-region availability architecture. The work that was invisible to the business becomes visible. The work that was visible to the business — keeping the database running — gets absorbed into a service-level agreement with a cloud provider.

The practical consequence is that you need to internalize the boundary between what you control and what the managed service controls. That boundary is different on every platform, and it shifts with every major release. On RDS for PostgreSQL, you cannot modify kernel parameters like `kernel.shmmax` or use `pg_prewarm` to manually warm the buffer cache after a failover — the OS is not yours. On Azure SQL Database, you cannot access the underlying SQL Server instance with a `sysadmin` login the way you would on a self-managed instance. On Cloud SQL, you cannot install

custom extensions not on Google’s approved list.

Knowing where the wall is before you hit it at 2 AM is the entire point of Chapter 1.

1.2 What Managed Services Actually Manage

The phrase “managed database” gets used loosely. Here is a concrete breakdown of what each major cloud platform actually takes off your plate, and where the line is drawn.

Infrastructure management is fully handled across all three platforms: AWS, Azure, and GCP provision the underlying compute, attach storage, configure the network interfaces, and run the hypervisor or bare-metal allocation. You never SSH into the host running your RDS instance. You never run `fdisk` on an Azure SQL storage volume. This is non-negotiable and non-configurable.

OS patching is managed by the provider on a schedule you can influence but not fully control. RDS maintenance windows let you specify a preferred day and time for minor version upgrades and OS patches, but AWS reserves the right to apply security patches outside that window when the vulnerability is severe enough. Azure SQL uses a similar maintenance window configuration. If your compliance posture requires knowing every patch that touches a database host, cloud managed services require additional audit tooling — the provider’s patch logs are available through CloudTrail on AWS and Azure Monitor on Azure, but they are not in the same format as a traditional change management ticket.

High availability and replication is where managed services deliver their most significant operational value. A Multi-AZ RDS deployment synchronously replicates your data to a standby in a different Availability Zone. Failover is automatic and AWS states it typically completes within one to two minutes for standard RDS Multi-AZ. Compare that to a manual Pacemaker/Corosync cluster failover on-premises, where timing depends heavily on how well-rehearsed the runbook is — a well-tested setup might land in a similar window, while an untested one can turn into an hour on the phone with a vendor.

Backups are automated, with point-in-time recovery available up to the retention period you configure (1–35 days on RDS, 1–35 days on Azure SQL, 1–35 days on Cloud SQL). The backup mechanism itself is managed — on RDS, automated backups use storage-level snapshots combined with continuous WAL archiving to S3. You do not run `pg_basebackup` manually. You do not manage WAL archiving with `archive_command`. You configure a retention period and trust the service.

What managed services do **not** manage: your schema design, your index strategy, your query plans, your connection pool sizing, your application-level caching logic, your data lifecycle policies, and your disaster recovery architecture above the single-region level. These remain entirely your responsibility, and they represent the highest-leverage work a cloud DBA performs.

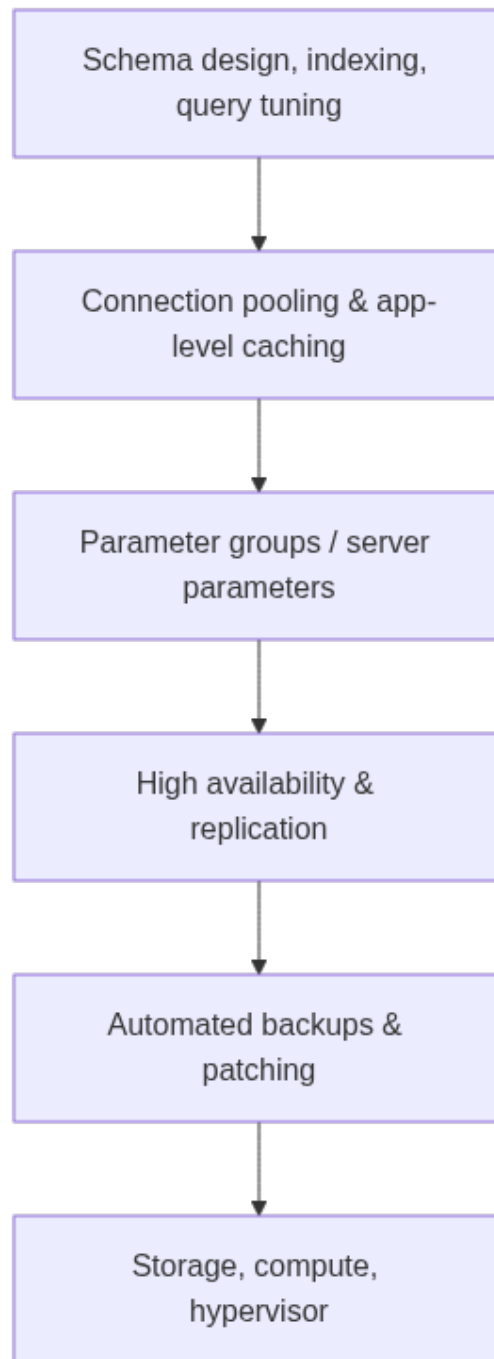


Figure 1: The layers a cloud DBA still owns (top) versus the layers the provider takes over (bottom). Parameter configuration sits in between: you set the values, but the provider enforces which ones are exposed.

1.3 The Configuration Model: Parameter Groups, Server Parameters, and Database Flags

On-premises PostgreSQL is configured through `postgresql.conf` and `pg_hba.conf`. SQL Server is configured through `sp_configure`, trace flags, and the SQL Server Configuration Manager. When you move to managed databases, these familiar mechanisms are replaced by provider-specific configuration abstractions.

AWS RDS Parameter Groups are collections of engine parameters that you apply to one or more DB instances. They map almost directly to `postgresql.conf` for RDS PostgreSQL, or to SQL Server instance-level configuration for RDS SQL Server. Some parameters are static (requiring a reboot to apply), and some are dynamic (applied immediately). The critical difference from on-premises is that some parameters are locked entirely — you cannot set `wal_level = logical` on standard RDS PostgreSQL without enabling logical replication at the instance level through a separate parameter, and even then, the implementation is managed rather than raw.

Here is how you create and apply a custom parameter group for RDS PostgreSQL using the AWS CLI:

```
# Create a custom parameter group for PostgreSQL 15
aws rds create-db-parameter-group \
  --db-parameter-group-name prod-pg15-tuned \
  --db-parameter-group-family postgres15 \
  --description "Production PostgreSQL 15 tuned parameters" \
  --region us-east-1

# Set shared_buffers - note: on RDS this is set as a percentage
# of instance memory using DBInstanceClassMemory formula
aws rds modify-db-parameter-group \
  --db-parameter-group-name prod-pg15-tuned \
  --parameters \

  ↪ "ParameterName=shared_buffers,ParameterValue={DBInstanceClassMemory/32768},ApplyMethod=pending-reboot"
  ↪ \
  "ParameterName=work_mem,ParameterValue=65536,ApplyMethod=immediate" \
  "ParameterName=max_connections,ParameterValue=500,ApplyMethod=pending-reboot" \
  "ParameterName=checkpoint_completion_target,ParameterValue=0.9,ApplyMethod=immediate" \
  "ParameterName=log_min_duration_statement,ParameterValue=1000,ApplyMethod=immediate" \
  --region us-east-1

# Apply the parameter group to an existing RDS instance
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-primary \
  --db-parameter-group-name prod-pg15-tuned \
  --apply-immediately \
  --region us-east-1
```

Notice the `{DBInstanceClassMemory/32768}` formula. RDS does not let you set `shared_buffers` in absolute kilobytes the way you would in `postgresql.conf`. Instead, it uses a formula language that resolves against the actual memory of whatever instance class the parameter group is applied to. This is actually a feature — the parameter group becomes portable across instance sizes without manual recalculation — but it is a mental model shift if you are used to typing `shared_buffers = 8GB` directly.

Azure SQL Server Parameters live in the Azure Portal under “Server parameters” for Azure Database for PostgreSQL Flexible Server, or as database-scoped configurations for Azure SQL Database (the managed SQL Server service). The CLI equivalent:

```
# Azure CLI: Configure parameters on Azure Database for PostgreSQL Flexible Server
az postgres flexible-server parameter set \
```

```
--resource-group prod-rg \  
--server-name prod-postgres-flex \  
--name work_mem \  
--value 65536  
  
az postgres flexible-server parameter set \  
--resource-group prod-rg \  
--server-name prod-postgres-flex \  
--name log_min_duration_statement \  
--value 1000  
  
# For Azure SQL Database (SQL Server-compatible), use database-scoped configs  
# Connect with sqlcmd or SSMS and run:  
# ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 4;  
# ALTER DATABASE SCOPED CONFIGURATION SET LEGACY_CARDINALITY_ESTIMATION = OFF;
```

Google Cloud SQL and AlloyDB use database flags, which are passed at instance creation or modification time:

```
# GCP CLI: Set database flags on a Cloud SQL PostgreSQL instance  
gcloud sql instances patch prod-postgres-instance \  
--database-flags \  
  max_connections=500,\ \  
  work_mem=65536,\ \  
  log_min_duration_statement=1000,\ \  
  checkpoint_completion_target=0.9 \  
--project my-prod-project  
  
# Some flags require an instance restart - gcloud will warn you  
# Check which flags are supported and whether restart is needed  
gcloud sql instances describe prod-postgres-instance \  
--project my-prod-project \  
--format="json(settings.databaseFlags)"
```

A production gotcha shared across all three platforms: you will encounter parameters that exist on-premises but are either locked, unavailable, or behave differently in managed environments. On RDS PostgreSQL, `max_wal_size` cannot be set below a minimum enforced by AWS. On Cloud SQL, `fsync` cannot be disabled even if you wanted to (which you do not). On Azure SQL Database (the SQL Server service), you cannot use `DBCC FREEPROCCACHE` without specific permissions, and `DBCC DROPCLEANBUFFERS` is not available at all in the serverless tier.

1.4 Networking and Security: The New `pg_hba.conf`

The most jarring transition for on-premises DBAs is often not the configuration model — it is the networking and authentication model. On-premises, your security perimeter is relatively straightforward: `pg_hba.conf` controls who can connect to PostgreSQL, firewall rules at the OS or network level control which IPs can reach port 5432, and SSL is configured in `postgresql.conf` with certificates you manage yourself.

Cloud managed databases replace this with layered abstractions: VPCs, security groups, private endpoints, IAM authentication, and certificate authorities managed by the cloud provider. Each layer is more powerful and more auditable than what you had before — but only if you understand how they compose.

VPC placement is the first decision. Every RDS instance, Azure SQL server, or Cloud SQL instance should live inside a private subnet with no public accessibility. Public accessibility should be off by default. On RDS, this means creating the DB subnet group from private subnets only:

```

# Create an RDS subnet group using private subnets only
aws rds create-db-subnet-group \
  --db-subnet-group-name prod-db-subnet-group \
  --db-subnet-group-description "Private subnets for production RDS" \
  --subnet-ids subnet-0a1b2c3d4e5f subnet-0f5e4d3c2b1a \
  --region us-east-1

# Create the RDS instance with no public access
aws rds create-db-instance \
  --db-instance-identifier prod-postgres-primary \
  --db-instance-class db.r6g.2xlarge \
  --engine postgres \
  --engine-version 15.4 \
  --master-username dbadmin \
  --master-user-password "$(aws secretsmanager get-secret-value \
    --secret-id prod/postgres/master-password \
    --query SecretString --output text)" \
  --db-subnet-group-name prod-db-subnet-group \
  --vpc-security-group-ids sg-0123456789abcdef0 \
  --no-publicly-accessible \
  --multi-az \
  --allocated-storage 500 \
  --storage-type gp3 \
  --storage-encrypted \
  --region us-east-1

```

IAM authentication replaces the password-in-config-file pattern that was acceptable on-premises and is a liability in cloud environments. With RDS IAM authentication, database users are mapped to IAM roles, and the authentication token is generated on the fly with a 15-minute validity window. No long-lived passwords stored in application config files. What you used to manage in `pg_hba.conf` with `md5` or `scram-sha-256` authentication lines, you now manage with IAM policies.

```

# Python: Generate an IAM auth token and connect to RDS PostgreSQL
import boto3
import psycopg2
from botocore.exceptions import ClientError

def get_rds_iam_token(hostname: str, port: int, username: str, region: str) -> str:
    """Generate a short-lived IAM authentication token for RDS."""
    client = boto3.client('rds', region_name=region)
    token = client.generate_db_auth_token(
        DBHostname=hostname,
        Port=port,
        DBUsername=username,
        Region=region
    )
    return token

def connect_with_iam_auth():
    hostname = "prod-postgres-primary.cluster-xxxxxxxxxxx.us-east-1.rds.amazonaws.com"
    port = 5432
    username = "app_readonly"
    region = "us-east-1"
    database = "production"

    # Token is valid for 15 minutes - generate fresh per connection
    token = get_rds_iam_token(hostname, port, username, region)

    conn = psycopg2.connect(
        host=hostname,
        port=port,
        user=username,
        password=token,
        database=database,
        sslmode="verify-full",
        sslrootcert="/etc/ssl/certs/rds-ca-2019-root.pem"
    )
    return conn

```

```
# The IAM role attached to the EC2/Lambda/ECS task must have:
# rds-db:connect permission on the specific DB user resource ARN
# arn:aws:rds-db:us-east-1:123456789012:dbuser:prod-postgres-primary/app_readonly
```

Private Link and Private Endpoints eliminate the exposure of database traffic to the public internet entirely, even within a VPC. Azure SQL Private Link creates a private IP in your VNet that maps to the Azure SQL server — traffic never leaves Microsoft’s backbone. AWS PrivateLink for RDS works similarly. On GCP, Cloud SQL’s Private IP option keeps all traffic within the VPC.

The mental model shift: you are no longer thinking about firewall rules as a flat list of IP-to-port mappings. You are thinking about IAM principals, VPC routing, and service endpoints as a layered security architecture. The good news is that this architecture is auditable in ways that a text file on a server never was.

1.5 Operational Observability: From pg_stat to CloudWatch and Beyond

On a self-managed PostgreSQL server, your observability toolkit is `pg_stat_activity`, `pg_stat_statements`, `pg_stat_bgwriter`, the PostgreSQL log files, and whatever you have wired up with Prometheus and Grafana. On SQL Server, it is DMVs, Extended Events, and SQL Server Profiler. You own the data collection pipeline end to end.

Cloud managed databases provide observability through their native monitoring platforms — Amazon CloudWatch for RDS, Azure Monitor for Azure SQL, and Cloud Monitoring for Cloud SQL — and these platforms are significantly more powerful at the infrastructure level than anything you ran on-premises. They are also more limited at the database engine internals level, because you do not own the host.

CloudWatch for RDS exposes metrics at 1-minute granularity by default (10-second granularity with Enhanced Monitoring enabled, which adds a small cost). Key metrics you should always alarm on in production:

```
# Create a CloudWatch alarm for RDS CPU exceeding 80% for 10 minutes
aws cloudwatch put-metric-alarm \
  --alarm-name "prod-rds-cpu-high" \
  --alarm-description "RDS CPU above 80% for 10 minutes" \
  --metric-name CPUUtilization \
  --namespace AWS/RDS \
  --statistic Average \
  --period 300 \
  --evaluation-periods 2 \
  --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-primary \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:prod-dba-alerts \
  --ok-actions arn:aws:sns:us-east-1:123456789012:prod-dba-alerts \
  --region us-east-1

# Create alarm for replica lag exceeding 30 seconds on a read replica
aws cloudwatch put-metric-alarm \
  --alarm-name "prod-rds-replica-lag-high" \
  --alarm-description "Read replica lag above 30 seconds" \
  --metric-name ReplicaLag \
  --namespace AWS/RDS \
  --statistic Maximum \
  --period 60 \
```

```
--evaluation-periods 3 \  
--threshold 30 \  
--comparison-operator GreaterThanThreshold \  
--dimensions Name=DBInstanceIdentifier,Value=prod-postgres-replica-1 \  
--alarm-actions arn:aws:sns:us-east-1:123456789012:prod-dba-alerts \  
--region us-east-1
```

The metrics that matter most in production cloud databases — `FreeStorageSpace`, `DatabaseConnections`, `ReadLatency`, `WriteLatency`, `ReplicaLag`, `FreeableMemory` — map directly to the concerns you had on-premises, but the collection and alerting infrastructure is provided rather than self-built.

What you lose is raw access to the operating system metrics underneath. Enhanced Monitoring on RDS gives you per-process CPU and memory breakdowns through an agent that runs on the DB host and ships to CloudWatch Logs, but it is read-only telemetry — you cannot act on it by tuning kernel parameters. You observe what the managed service gives you, and you tune within the parameters it exposes.

Performance Insights, available on RDS and Aurora, is the closest cloud equivalent to `pg_stat_statements` combined with a wait event analysis tool. It shows you top SQL by load, broken down by wait events (CPU, I/O, lock, etc.), across a 7-day free window or up to 2 years with the paid tier. For most production workload analysis, it replaces the combination of `pg_stat_statements` reset cycles and manual wait event querying you ran on-premises.

1.6 The Economics of Cloud Databases: What You Pay For and What It Costs You

No chapter on why cloud databases changed everything would be complete without an honest discussion of cost, because cost is where cloud database decisions most often go wrong. On-premises infrastructure is a capital expense (CapEx) that depreciates over a hardware lifecycle. Cloud databases are an operating expense (OpEx) that runs indefinitely and scales directly with usage. This changes how you make architectural decisions.

An `db.r6g.4xlarge` RDS instance (16 vCPU, 128 GB RAM) carries a meaningful on-demand hourly rate — check current AWS pricing for your region and engine rather than relying on a number printed in a book, since rates change over time. A Multi-AZ deployment roughly doubles the compute cost compared to single-AZ, because you are paying for a synchronous standby that stays warm and ready. Adding several terabytes of `gp3` storage adds a proportional monthly storage cost on top of compute. Adding one read replica at the same instance class effectively doubles your compute spend again.

Before you reach for the largest instance class, understand what you are buying. An `r6g.4xlarge` with 128 GB RAM is appropriate when your working set — the data actively accessed by your queries — genuinely approaches that size. If your total database is 200 GB but your hot working set is 20 GB, a smaller instance such as an `r6g.xlarge` (32 GB RAM, a fraction of the `4xlarge`'s cost) with appropriate indexing will outperform an `r6g.4xlarge` with poor query patterns.

Reserved Instances can meaningfully reduce RDS costs compared to on-demand pricing, with the exact discount depending on the term (1-year vs. 3-year) and payment option — check current AWS pricing rather than budgeting off a fixed percentage. For production databases with stable

workloads, Reserved Instances (or Savings Plans) are not optional — they are the responsible operational choice.

Aurora Serverless v2 introduces a different cost model: you pay per Aurora Capacity Unit (ACU) per second, with each ACU representing approximately 2 GB of memory and proportional CPU. For workloads with unpredictable spikes, this model avoids paying for peak capacity around the clock. For steady, predictable workloads, provisioned Aurora at Reserved pricing is almost always cheaper.

The cost comparison with on-premises is more complex than the cloud vendors' calculators suggest. You need to include fully loaded infrastructure costs: hardware amortization, data center space and power, network equipment, OS licensing (for Windows-based databases), DBA time spent on patching and hardware maintenance, and the cost of unplanned downtime. When all of these are included honestly, cloud managed databases are frequently cost-competitive or cheaper for mid-sized organizations that lack economies of scale in hardware procurement. For very large deployments — hundreds of nodes, petabytes of data — the calculus often favors hybrid or private cloud approaches, which is why this book exists rather than one that simply tells you to migrate everything.

The Terraform pattern below represents the production baseline for an RDS deployment — the minimum configuration a production DBA should require before any database goes live:

```
# Terraform: Production-grade RDS PostgreSQL baseline
resource "aws_db_instance" "prod_postgres" {
  identifier      = "prod-postgres-primary"
  engine          = "postgres"
  engine_version  = "15.4"
  instance_class  = "db.r6g.2xlarge"

  allocated_storage      = 500
  max_allocated_storage  = 2000 # Enable storage autoscaling up to 2TB
  storage_type           = "gp3"
  storage_encrypted      = true
  kms_key_id             = aws_kms_key.rds_encryption_key.arn

  db_name = "production"
  username = "dbadmin"
  password = data.aws_secretsmanager_secret_version.db_master_password.secret_string

  multi_az          = true
  db_subnet_group_name = aws_db_subnet_group.prod.name
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
  publicly_accessible = false

  parameter_group_name = aws_db_parameter_group.prod_pg15.name
  option_group_name     = aws_db_option_group.prod.name

  backup_retention_period = 14
  backup_window           = "03:00-04:00"
  maintenance_window     = "sun:04:00-sun:05:00"
  auto_minor_version_upgrade = false # Control upgrades explicitly in production

  performance_insights_enabled      = true
  performance_insights_retention_period = 731 # 2 years
  monitoring_interval               = 60    # Enhanced Monitoring every 60 seconds
  monitoring_role_arn               = aws_iam_role.rds_enhanced_monitoring.arn

  enabled_cloudwatch_logs_exports = ["postgresql", "upgrade"]

  deletion_protection = true
  skip_final_snapshot = false
  final_snapshot_identifier = "prod-postgres-final-snapshot"

  tags = {
```

```
Environment = "production"
Team        = "platform-data"
CostCenter  = "engineering"
}
}
```

Every parameter in that Terraform block represents a decision that a production DBA needs to make consciously and own. `auto_minor_version_upgrade = false` means you control when minor version upgrades happen — the default is true, which lets AWS apply them during your maintenance window without further approval. In a regulated environment or one with strict change management, that default will cause you problems. `deletion_protection = true` means you cannot accidentally destroy a production database with a `terraform destroy` or a misclick in the console. `performance_insights_retention_period = 731` keeps two years of query performance history, which matters when you are debugging a regression that appeared after a deployment six weeks ago.

These are not configuration details — they are operational commitments. Infrastructure as code forces you to make them explicit.

1.7 What Stayed the Same: The Fundamentals Don't Move

After four sections on what changed, it is worth being direct about what did not change, because this is where many cloud migration projects go wrong. Engineers assume that because the infrastructure layer is managed, the database layer is somehow simpler. It is not.

Query optimization is unchanged. A table scan on 500 million rows costs the same in RDS PostgreSQL as it does on a bare-metal server running the same PostgreSQL version with the same data. The query planner has the same logic, the same cost model, the same sensitivity to stale statistics. `EXPLAIN (ANALYZE, BUFFERS)` produces the same output. The fix is the same: appropriate indexing, updated statistics, rewritten queries. The cloud provider does not optimize your queries.

```
-- PostgreSQL: Identifying sequential scans on large tables (works on RDS, Cloud SQL, AlloyDB)
↳ identically)
SELECT
  schemaname,
  relname AS table_name,
  seq_scan,
  seq_tup_read,
  idx_scan,
  idx_tup_fetch,
  n_live_tup,
  CASE
    WHEN seq_scan + idx_scan = 0 THEN NULL
    ELSE ROUND(100.0 * seq_scan / (seq_scan + idx_scan), 2)
  END AS seq_scan_pct,
  pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM
  pg_stat_user_tables
WHERE
  n_live_tup > 100000
  AND seq_scan > 0
ORDER BY
  seq_tup_read DESC
LIMIT 20;

-- SQL Server: Equivalent on RDS SQL Server or Azure SQL - missing index DMV analysis
```



```
SELECT
  DB_NAME(mid.database_id) AS database_name,
  OBJECT_NAME(mid.object_id, mid.database_id) AS table_name,
  migs.avg_total_user_cost * migs.avg_user_impact * (migs.user_seeks + migs.user_scans) AS
↪ improvement_measure,
  'CREATE INDEX [IX_' + OBJECT_NAME(mid.object_id, mid.database_id)
    + '_' + REPLACE(REPLACE(ISNULL(mid.equality_columns, ''), '['), ')', ']') +
    + CASE WHEN mid.inequality_columns IS NOT NULL THEN '_' +
      ↪ REPLACE(REPLACE(mid.inequality_columns, '['), ')', ']') ELSE '' END
    + ']'
    + ' ON ' + mid.statement
    + ' (' + ISNULL(mid.equality_columns, '')
    + CASE WHEN mid.equality_columns IS NOT NULL AND mid.inequality_columns IS NOT NULL THEN ','
      ↪ ELSE '' END
    + ISNULL(mid.inequality_columns, '') + ') '
    + ISNULL(' INCLUDE (' + mid.included_columns + ')', '') AS create_index_statement,
  migs.user_seeks,
  migs.user_scans,
  migs.last_user_seek,
  mid.equality_columns,
  mid.inequality_columns,
  mid.included_columns
FROM
  sys.dm_db_missing_index_groups mig
  INNER JOIN sys.dm_db_missing_index_group_stats migs
    ON migs.group_handle = mig.index_group_handle
  INNER JOIN sys.dm_db_missing_index_details mid
    ON mig.index_handle = mid.index_handle
WHERE
  migs.avg_total_user_cost * migs.avg_user_impact * (migs.user_seeks + migs.user_scans) > 10
ORDER BY
  improvement_measure DESC;
```

Both queries work without modification against the equivalent managed service. The DMV and catalog views are the same engine, running in the same way, just on infrastructure you do not own.

Transaction isolation and locking behavior is unchanged. Deadlocks happen in RDS PostgreSQL for exactly the same reasons they happen on-premises. Multi-version concurrency control works identically. Autovacuum runs on the same schedule logic. If your application generates bloat by doing large bulk deletes without a `VACUUM`, RDS will accumulate bloat exactly like an on-premises instance. The managed service does not vacuum on your behalf any more aggressively than a default-configured on-premises server.

Connection management is unchanged — and arguably more important. PostgreSQL's process-per-connection model means that 1,000 concurrent connections consume substantial memory and scheduler overhead regardless of whether PostgreSQL is running on RDS, Cloud SQL, or bare metal. On-premises, the instinct was often to tune `max_connections` high and hope for the best. In cloud environments, especially when running many Lambda functions or microservices that each hold database connections, this problem becomes acute. PgBouncer (or RDS Proxy, which we cover in Chapter 4) is not optional for application architectures with more than a few dozen concurrent clients.

Backup recovery time must be tested. Automated backups are only valuable if you have verified, under conditions resembling production, that you can actually restore from them in the time your SLA requires. This does not change because backups are managed. What changes is the restoration mechanism — you are restoring from RDS snapshots rather than running `pg_restore` manually — but the discipline of testing restores remains entirely your responsibility.

1.8 Choosing a Platform: RDS, Aurora, Azure SQL, Cloud SQL, and AlloyDB

This book covers all three major clouds, and readers who are evaluating where to run their workloads deserve a direct framework for that decision. This is not a comprehensive feature comparison — those go stale within months — but a durable architectural framing.

Stick with your existing ecosystem first. If your organization is deeply invested in AWS — IAM, VPCs, CloudFormation, existing RDS instances — the switching cost to Azure or GCP is significant and the marginal benefit is usually small. Conversely, if your analytics workload already runs on BigQuery, Cloud SQL or AlloyDB for transactional data keeps your data gravity in GCP. Platform consistency reduces operational complexity, and operational complexity is where production incidents are born.

Match the service tier to your operational maturity. Aurora PostgreSQL and Aurora MySQL are AWS’s managed distributed database engines, purpose-built for cloud scale. They offer faster failover than standard RDS Multi-AZ (typically under thirty seconds), up to fifteen read replicas, and global database capabilities for multi-region active-active patterns. They also cost more and have behavioral differences from standard PostgreSQL that matter for some workloads — the storage engine is Aurora’s own distributed log-structured storage, not PostgreSQL’s standard heap files. If you are running a greenfield application with high availability requirements and AWS is your platform, Aurora is worth the evaluation. If you are migrating an existing on-premises PostgreSQL workload and want the most straightforward compatibility, standard RDS PostgreSQL reduces the number of unknowns.

Azure SQL Database vs. Azure Database for PostgreSQL Flexible Server is a choice between SQL Server compatibility and PostgreSQL compatibility, not between capabilities per se. If your application was written for SQL Server — T-SQL stored procedures, SQL Server-specific features, existing SSRS or SSIS dependencies — Azure SQL Database is the natural target. If your workload is PostgreSQL, Azure Database for PostgreSQL Flexible Server (which replaced the older single-server offering) provides a well-integrated managed PostgreSQL environment with good integration into Azure Monitor and Azure Active Directory.

Google Cloud SQL vs. AlloyDB follows a similar pattern to RDS vs. Aurora. Cloud SQL is standard PostgreSQL (or MySQL, or SQL Server) with managed infrastructure — the most compatible option for existing workloads. AlloyDB is Google’s PostgreSQL-compatible managed database with a custom storage layer, columnar caching, and query acceleration through Google’s infrastructure — aimed at workloads that need PostgreSQL compatibility but are pushing the limits of what standard PostgreSQL can deliver. AlloyDB is a newer service and the operational patterns around it are still maturing compared to Cloud SQL’s long track record.

The honest answer is: for most production workloads that are not at extreme scale, standard RDS PostgreSQL, Azure Database for PostgreSQL Flexible Server, or Cloud SQL PostgreSQL are excellent choices that will not limit you. Evaluate Aurora, AlloyDB, or Azure SQL Hyperscale when you have specific requirements — storage growth beyond what standard managed services handle cost-effectively, read scale beyond what a few replicas provide, or sub-thirty-second failover RTO — and when your team has the operational capacity to understand the differences.

1.9 The DBA's New Toolkit

The tools change when the platform changes. A production cloud DBA in 2024 operates with a toolkit that is meaningfully different from the on-premises DBA toolkit, and the difference is not just the addition of cloud consoles.

Infrastructure as code (IaC) is not optional. Every database instance, parameter group, subnet group, security group, and IAM policy should be defined in Terraform, Pulumi, or CloudFormation. Clicking through the AWS console to create a production database is the equivalent of hand-editing `postgresql.conf` on a server that has no change management — it will work until it doesn't, and you will have no record of what you did. Chapter 2 covers Terraform patterns for RDS in depth.

The CLI is your command line. `aws`, `az`, and `gcloud` replace the SSH session. Being fluent in the CLI for your primary platform is as important as knowing `psql` commands. The examples throughout this book use CLI commands intentionally — they are scriptable, auditable, and version-controllable in ways that console clicks are not.

Secrets management replaces config files. AWS Secrets Manager, Azure Key Vault, and GCP Secret Manager are the production-grade replacements for the `DATABASE_URL` environment variable that contains a plaintext password. Rotation policies, audit logs, and IAM-controlled access make them categorically more secure. We cover integration patterns in Chapter 5.

Cost monitoring is a DBA function. On-premises, the DBA rarely owned infrastructure costs — that was procurement and finance's problem. In cloud environments where database spend is metered hourly and can surprise you at month-end, monitoring your RDS or Azure SQL spend against budget is part of database operations. AWS Cost Explorer, Azure Cost Management, and GCP Billing all provide service-level breakdowns that let you see exactly what your database fleet is costing. Setting budget alerts before they are needed, not after the first unexpected invoice, is the professional approach.

Documentation lives in the provider's release notes. PostgreSQL major version 15 to 16 release notes are the same whether you are on-premises or on RDS. But RDS-specific behavior, parameter limitations, and maintenance policies live in AWS's RDS documentation and change independently of PostgreSQL versions. Subscribing to AWS RDS release notes, Azure SQL Blog, and Cloud SQL release notes is operational discipline — these are the channels where you learn that a new behavior will affect your production instances before it affects your production instances.

Key Takeaways

- Cloud managed databases shift the DBA's responsibility from infrastructure ownership to service consumption. The infrastructure layer — OS, storage, replication, patching — is managed by the provider. Schema design, query optimization, connection management, and cross-region availability architecture remain entirely your responsibility.
- The configuration boundary is platform-specific and changes over time. RDS parameter groups, Azure Server Parameters, and Cloud SQL database flags map to familiar on-premises configuration mechanisms but enforce restrictions that do not exist on self-managed instances. Know which parameters are locked before you need them in an incident.

- Networking and authentication are more layered in cloud environments than on-premises. VPC placement, IAM authentication, and private endpoints replace flat firewall rules and `pg_hba.conf`. This architecture is more secure and more auditable, but it requires understanding how the layers compose to debug connectivity failures.
 - The database engine fundamentals do not change on managed infrastructure. Query plans, locking behavior, MVCC, autovacuum, and connection overhead work identically. Performance problems that exist on-premises follow you to cloud managed databases unless you address them at the query and schema level.
 - Infrastructure as code, CLI fluency, secrets management, and cost monitoring are non-negotiable components of cloud database operations. Clicking through a console to manage production databases is not a sustainable operational practice, and treating cloud database cost as someone else's problem will produce unwelcome surprises at billing time.
-

Chapter 2: Managed vs Self-Managed: The Real Tradeoffs

Every DBA who moves to the cloud arrives with the same question: what exactly am I giving up, and what am I getting in return? The marketing answer is “you get managed infrastructure.” The production answer is more nuanced. Managed database services abstract away the operating system, storage subsystem, replication mechanics, and backup scheduling — but they do not abstract away the need to understand them. This chapter maps the specific boundaries between what AWS RDS, Azure SQL, and Google Cloud SQL control versus what you still own, so you can make architecture decisions with your eyes open rather than discovering limits during an incident at 2 a.m.

What “Managed” Actually Means at the Infrastructure Layer

When you ran PostgreSQL on bare metal or VMware, your responsibility surface looked like this: hardware procurement, OS patching, filesystem layout, network configuration, PostgreSQL installation, parameter tuning, backup scheduling, replication setup, failover scripting, and monitoring. In a managed service, roughly the bottom half of that stack disappears — but it disappears in specific ways that matter to how you work.

AWS RDS runs your database inside an EC2 instance that AWS owns. You never SSH into it. The OS is there — Amazon Linux 2 or a variant depending on the engine — but it is not your OS. You cannot install extensions that are not on the approved list, you cannot modify `pg_hba.conf` directly, and you cannot change the PostgreSQL binary. What you can do is control almost everything above that layer: parameter groups, option groups, subnet groups, security groups, IAM authentication, automated backup retention up to 35 days, and Multi-AZ deployment.

Azure SQL Database is further abstracted than RDS. Microsoft runs SQL Server inside a service fabric that you have zero visibility into. There is no concept of an underlying VM in the standard tiers. You configure DTUs or vCores, storage limits, firewall rules, Active Directory authentication, and Transparent Data Encryption. The SQL Server instance itself — its memory settings, its tempdb layout, its trace flags — is managed by Microsoft according to tier.

Google Cloud SQL sits closer to the RDS model than to Azure SQL Database. It runs PostgreSQL, MySQL, or SQL Server on Compute Engine VMs that Google manages. You get database flags (the equivalent of `postgresql.conf` parameters), authorized networks, private IP configuration, and automated backups. Like RDS, the OS is not yours.

The practical consequence for an experienced DBA: your tuning vocabulary shifts from OS-level and

filesystem-level settings toward database-level configuration objects — parameter groups, database flags, elastic pool settings — and cloud-native controls like IAM policies and VPC routing.

The Access Boundary: What You Can and Cannot Touch

This is where most on-prem DBAs first hit friction. The following table summarizes the access boundaries that matter most in production:

Capability	Self-Managed PostgreSQL	AWS RDS PostgreSQL	Azure SQL	Cloud SQL PostgreSQL
OS access	Full root	None	None	None
Install arbitrary extensions	Yes	No (approved list only)	No	No (approved list)
Modify <code>pg_hba.conf</code>	Yes	No (use IAM/security groups)	N/A	No (use authorized networks)
Access WAL files directly	Yes	No (can stream via logical replication)	N/A	No
<code>pg_basebackup</code> to external target	Yes	No (use native backup export to S3)	N/A	No (use export to GCS)
Custom superuser access	Full	<code>rds_superuser</code> role (limited)	<code>db_owner</code> (limited)	<code>cloudsqlsuperuser</code> (limited)
Set <code>max_connections</code> freely	Yes	Yes (via parameter group)	Tier-dependent ceiling	Yes (via database flags)
Tablespace on custom paths	Yes	No	No	No

The `rds_superuser` point deserves emphasis. On RDS PostgreSQL, you receive a user with the `rds_superuser` role. This gives you the ability to manage other users, replication slots, logical replication, and most administrative functions. But it does not give you `pg_read_all_settings` on internal parameters, and it cannot create C-language extensions. If you have built any tooling that depends on superuser-level OS interaction — for example, a custom archiving script that writes directly to the filesystem — that tooling requires redesign.

Azure SQL is a different situation entirely. The concept of a SQL Server instance owner with `sysadmin` rights does not exist in Azure SQL Database. The closest equivalent is `db_owner` on individual databases, plus server-level roles like `loginmanager` and `dbmanager` on the logical server. If your on-prem SQL Server processes rely on SQL Server Agent jobs with `xp_cmdshell`, those processes need to be rebuilt as Azure Functions, Azure Automation runbooks, or Elastic Jobs.

Parameter Tuning: From postgresql.conf to Parameter Groups

On a self-managed PostgreSQL instance, tuning `work_mem`, `shared_buffers`, `checkpoint_completion_target`, and `effective_cache_size` means editing a file and reloading or restarting. In RDS, that workflow translates to parameter groups.

A **parameter group** in RDS is a named configuration object that you attach to one or more DB instances. AWS provides default parameter groups (read-only) for each engine version. You create a custom parameter group, modify the parameters you need, and associate it with your instance. Parameter changes fall into two categories: static (require a reboot) and dynamic (apply immediately). Knowing which is which before you plan a change window matters.

```
# Create a custom RDS parameter group for PostgreSQL 15
aws rds create-db-parameter-group \
  --db-parameter-group-name prod-pg15-custom \
  --db-parameter-group-family postgres15 \
  --description "Production PostgreSQL 15 parameter group" \
  --region us-east-1

# Modify key parameters
aws rds modify-db-parameter-group \
  --db-parameter-group-name prod-pg15-custom \
  --parameters \
    "ParameterName=work_mem,ParameterValue=65536,ApplyMethod=immediate" \
    "ParameterName=checkpoint_completion_target,ParameterValue=0.9,ApplyMethod=immediate" \
    "ParameterName=max_connections,ParameterValue=500,ApplyMethod=pending-reboot" \
    "ParameterName=log_min_duration_statement,ParameterValue=1000,ApplyMethod=immediate" \
  --region us-east-1

# Apply the parameter group to an existing instance
aws rds modify-db-instance \
  --db-instance-identifier prod-pg-primary \
  --db-parameter-group-name prod-pg15-custom \
  --apply-immediately \
  --region us-east-1
```

On Google Cloud SQL, the equivalent concept is **database flags**. The syntax and application mechanism differ, but the intent is the same. Note that Cloud SQL does not support all PostgreSQL parameters — some low-level settings are fixed by Google.

```
# Set database flags on a Cloud SQL PostgreSQL instance
gcloud sql instances patch prod-pg-instance \
  --database-flags \
    max_connections=500,\
    work_mem=65536,\
    checkpoint_completion_target=0.9,\
    log_min_duration_statement=1000 \
  --project my-gcp-project

# Verify current flags
gcloud sql instances describe prod-pg-instance \
  --project my-gcp-project \
  --format="json(settings.databaseFlags)"
```

On Azure SQL, the tuning model is fundamentally different. You do not set `max_connections` because connection limits are determined by the service tier. For an 8-vCore General Purpose instance, Azure SQL allows approximately 800 concurrent connections. For Business Critical, limits are higher. You tune at the query level using Query Store, Automatic Plan Correction, and resource governor equivalents — but you do not manage server memory allocation directly. Azure SQL's Resource Governor equivalent is elastic pool resource governance, and it operates at the pool level rather than the instance level.

One real gotcha with RDS parameter groups: if you create a Multi-AZ deployment and modify a static parameter, the pending reboot applies to both primary and standby simultaneously during the next maintenance window — or immediately if you force it. This means you need to treat static parameter changes with the same care as a planned failover, because that is effectively what AWS does to apply them.

High Availability: What You Delegate and What You Still Own

In a self-managed PostgreSQL environment, high availability required you to configure streaming replication, set `primary_conninfo`, manage `recovery.conf` (or `standby.signal` in PostgreSQL 12+), build a failover mechanism with Patroni, Repmgr, or a custom script, and test it regularly. That is a significant operational surface.

RDS Multi-AZ reduces that surface dramatically. When you enable Multi-AZ on an RDS PostgreSQL instance, AWS provisions a synchronous standby in a different Availability Zone, manages the replication, monitors the primary, and executes automatic failover — typically completing within 60 to 120 seconds for standard RDS. You do not configure any of this. The connection string uses the RDS endpoint, which automatically resolves to whichever instance is currently primary.

What you still own: **application reconnection logic**. RDS failover changes the underlying IP that the DNS endpoint points to. DNS TTL for RDS endpoints is 5 seconds, but client-side DNS caching in the JVM, in certain connection poolers, and in some OS-level resolvers can hold stale entries for minutes. If your application opens a connection and holds it across the failover boundary, that connection will break and must be reestablished. This is not RDS failing to do its job — it is the boundary between the managed service and the application tier.

For Aurora, the HA story changes again. Aurora separates compute from storage. The Aurora storage layer replicates 6 copies of data across 3 Availability Zones synchronously before acknowledging a write. Failover from a primary to a read replica in Aurora typically completes in under 30 seconds — faster than standard RDS Multi-AZ — because there is no data volume to synchronize; the replicas already share the storage layer.

```
# Python example: RDS-aware connection handling with retry logic
# This pattern is necessary because RDS/Aurora failover breaks existing connections

import boto3
import psycopg2
import time
import logging
from psycopg2 import OperationalError

logger = logging.getLogger(__name__)

def get_rds_connection(host, port, dbname, user, password,
                       max_retries=5, retry_delay=2):
    """
    Establish a connection to RDS with retry logic suitable for
    Multi-AZ failover scenarios. On failover, DNS propagation
    may take 5-30 seconds; this loop handles the gap.
    """
    attempt = 0
    last_error = None

    while attempt < max_retries:
```



```
try:
    conn = psycopg2.connect(
        host=host,
        port=port,
        dbname=dbname,
        user=user,
        password=password,
        connect_timeout=5,
        # options=-c statement_timeout=30000 for 30s query timeout
        options="-c statement_timeout=30000"
    )
    conn.set_session(autocommit=False)
    logger.info(f"Connected to RDS at {host} on attempt {attempt + 1}")
    return conn

except OperationalError as e:
    last_error = e
    attempt += 1
    logger.warning(
        f"RDS connection attempt {attempt} failed: {e}. "
        f"Retrying in {retry_delay}s..."
    )
    time.sleep(retry_delay)
    retry_delay = min(retry_delay * 2, 30) # exponential backoff, cap at 30s

raise RuntimeError(
    f"Could not connect to RDS after {max_retries} attempts. "
    f"Last error: {last_error}"
)

def execute_with_reconnect(host, port, dbname, user, password, query, params=None):
    """
    Execute a query and handle connection failures that can occur
    mid-session during failover events.
    """
    conn = get_rds_connection(host, port, dbname, user, password)
    try:
        with conn.cursor() as cur:
            cur.execute(query, params)
            conn.commit()
            return cur.fetchall() if cur.description else None
    except OperationalError as e:
        logger.error(f"Query failed, connection likely lost during failover: {e}")
        conn.close()
        # Re-raise so the caller can decide whether to retry at the business logic level
        raise
    finally:
        if not conn.closed:
            conn.close()
```

Azure SQL Business Critical tier uses an Always On Availability Group under the hood, with a hidden read-only secondary exposed via the `ApplicationIntent=ReadOnly` connection string parameter. Business Critical carries the highest published availability SLA of the Azure SQL tiers — check current Microsoft documentation for the exact figure, since it has changed over time and varies with zone redundancy. For General Purpose tier, Azure uses remote storage (Premium SSD) and failover involves attaching that storage to a new compute node — typically 20 to 30 seconds but potentially longer under high storage load.

For Cloud SQL, high availability within a region uses a standby instance with synchronous replication. Cross-region read replicas can be promoted manually but do not participate in automatic failover. For multi-region automatic failover in Google Cloud, the answer is **AlloyDB** or **Cloud Spanner**, not Cloud SQL.

Backup, Restore, and the Point-in-Time Recovery Model

On a self-managed PostgreSQL instance, your backup strategy involved `pg_basebackup` for full backups, WAL archiving to a shared storage target, and periodic validation through `pg_restore` test runs. You controlled the retention schedule, the storage location, and the restore procedure.

In RDS, automated backups are configured with a retention period (1 to 35 days) and a backup window. RDS takes daily snapshots and continuously archives transaction logs to S3 — storage you never see but that AWS uses to serve Point-in-Time Recovery (PITR). Restoring to a point in time always creates a **new DB instance**; you cannot overwrite the existing one. This is one of the most operationally significant differences from self-managed recovery. If you need to restore to 30 minutes ago, you restore to a new endpoint and redirect traffic, or you use the restored instance to extract specific data with `pg_dump` and load it into the production instance.

```
# Initiate a point-in-time restore on RDS to a new instance
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier prod-pg-primary \
  --target-db-instance-identifier prod-pg-pitr-restore-20240115 \
  --restore-time 2024-01-15T14:30:00Z \
  --db-instance-class db.r6g.2xlarge \
  --db-subnet-group-name prod-subnet-group \
  --vpc-security-group-ids sg-0abc123def456789 \
  --no-publicly-accessible \
  --region us-east-1

# Monitor the restore status
aws rds describe-db-instances \
  --db-instance-identifier prod-pg-pitr-restore-20240115 \
  --query 'DBInstances[0].{Status:DBInstanceStatus,Endpoint:Endpoint.Address}' \
  --region us-east-1
```

The storage costs for automated backups in RDS are worth understanding before you enable 35-day retention on large instances. AWS provides backup storage equal to the total provisioned storage of your RDS instances at no additional charge; storage beyond that is billed per GB-month at a rate you should confirm against current AWS pricing. For a 10 TB RDS instance with 35-day retention and high write volume, backup storage costs can exceed the compute costs of the instance itself. Audit your backup retention policies as deliberately as you would on-prem.

Azure SQL automated backups use a similar model: full weekly, differential daily, transaction log every 5 to 12 minutes. Retention for Azure SQL Database is 1 to 35 days for PITR, with long-term retention (LTR) available for up to 10 years. PITR restores, like RDS, create a new database — they do not overwrite the source.

One genuine improvement in managed services over self-managed setups for most teams: **backup validation is handled for you**. AWS verifies that automated snapshots are restorable. On-prem shops frequently discover corrupted backup chains during actual recovery events. That said, this does not replace the need to periodically test restore procedures yourself — automated backup verification confirms the backup file is intact, not that your application functions correctly on the restored data.

Terraform Provisioning: Encoding the Decisions You Cannot Delegate

Infrastructure as Code is not optional in a production cloud database environment. The decisions you make at provisioning time — instance class, storage type, Multi-AZ enablement, encryption, parameter group, deletion protection — are significantly harder to change after the fact than on-prem settings. Terraform locks these decisions into version-controlled code and makes them reproducible.

```
# Terraform configuration for a production-grade RDS PostgreSQL instance
# This encodes the decisions that matter most at provisioning time.

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# KMS key for RDS encryption at rest
resource "aws_kms_key" "rds_key" {
  description      = "RDS encryption key for production PostgreSQL"
  deletion_window_in_days = 30
  enable_key_rotation = true

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
  }
}

# Custom parameter group for PostgreSQL 15
resource "aws_db_parameter_group" "pg15_prod" {
  name       = "prod-pg15-custom"
  family     = "postgres15"
  description = "Production PostgreSQL 15 parameters"

  parameter {
    name      = "work_mem"
    value     = "65536"
    apply_method = "immediate"
  }

  parameter {
    name      = "checkpoint_completion_target"
    value     = "0.9"
    apply_method = "immediate"
  }

  parameter {
    name      = "log_min_duration_statement"
    value     = "1000"
    apply_method = "immediate"
  }

  parameter {
    name      = "log_connections"
    value     = "1"
    apply_method = "immediate"
  }

  parameter {
    name      = "shared_preload_libraries"
    value     = "pg_stat_statements,auto_explain"
  }
}
```

```
    apply_method = "pending-reboot"
  }
}

# RDS subnet group
resource "aws_db_subnet_group" "prod" {
  name          = "prod-rds-subnet-group"
  subnet_ids    = var.private_subnet_ids
  description    = "Private subnets for RDS - no public route"

  tags = {
    Environment = "production"
  }
}

# Production RDS PostgreSQL instance
resource "aws_db_instance" "prod_pg" {
  identifier      = "prod-pg-primary"
  engine          = "postgres"
  engine_version  = "15.4"
  instance_class  = "db.r6g.2xlarge" # 8 vCPU, 64 GB RAM

  # Storage: gp3 gives 3000 IOPS baseline, upgradeable independently of storage size
  allocated_storage      = 500
  max_allocated_storage  = 2000 # autoscaling ceiling in GB
  storage_type            = "gp3"
  iops                    = 12000 # provisioned IOPS for gp3
  storage_throughput      = 500 # MB/s throughput for gp3
  storage_encrypted       = true
  kms_key_id              = aws_kms_key.rds_key.arn

  db_name = "appdb"
  username = "dbadmin"
  # In production: use aws_secretsmanager_secret and reference here
  password = var.db_master_password

  # HA and networking
  multi_az      = true
  db_subnet_group_name = aws_db_subnet_group.prod.name
  vpc_security_group_ids = [var.db_security_group_id]
  publicly_accessible = false

  # Parameter group
  parameter_group_name = aws_db_parameter_group.pg15_prod.name

  # Backup configuration
  backup_retention_period = 14
  backup_window           = "03:00-04:00" # UTC
  maintenance_window      = "sun:04:30-sun:05:30"
  copy_tags_to_snapshot    = true
  delete_automated_backups = false

  # Protection and logging
  deletion_protection = true
  skip_final_snapshot = false
  final_snapshot_identifier = "prod-pg-primary-final-snapshot"

  enabled_cloudwatch_logs_exports = [
    "postgresql",
    "upgrade"
  ]

  # Performance Insights for query-level visibility
  performance_insights_enabled = true
  performance_insights_retention_period = 7 # days; 731 for long-term (additional cost)

  tags = {
    Environment = "production"
    Owner       = "database-team"
    ManagedBy   = "terraform"
  }
}
```

```
}  
}
```

Two provisioning decisions in this template deserve attention for DBAs coming from on-prem.

First, `storage_type = "gp3"`. On RDS, gp3 replaced gp2 as the default general-purpose storage type. gp2 tied IOPS to storage size (3 IOPS per GB, bursting to 3000 for volumes under 1 TB). gp3 provides 3000 baseline IOPS and 125 MB/s throughput regardless of volume size, with independent scaling up to 16,000 IOPS and 1000 MB/s. For any database workload where you are not running io1 or io2 (the provisioned IOPS types), gp3 is almost always the right choice and is generally cheaper per GB than equivalent gp2 configurations — check current AWS pricing for the exact difference.

Second, `max_allocated_storage`. RDS Storage Autoscaling can expand your volume automatically when free space drops below 10% of allocated storage (or 5 GB, whichever is larger). Setting a ceiling with `max_allocated_storage` prevents runaway storage growth from a data incident — a bulk insert gone wrong, a logging misconfiguration — from producing a surprise billing event.

Key Takeaways

- Managed services eliminate OS, storage, and replication management, but replace those responsibilities with cloud-native configuration objects — parameter groups, database flags, security groups, IAM policies — that require the same deliberate attention as the files they replace.
- The `rds_superuser`, `cloudsqlsuperuser`, and Azure SQL `db_owner` roles are not equivalent to OS-level superuser access. Any automation, extension, or process that depends on superuser filesystem or binary access requires architectural redesign before it can run on a managed service.
- PITR restores on RDS, Azure SQL, and Cloud SQL always create a new database instance, not an in-place overwrite. Recovery procedures must account for the time to provision a new instance, redirect connections, and validate data — typically 15 to 45 minutes beyond the recovery point itself.
- Aurora's separated storage architecture makes it faster to fail over than standard RDS Multi-AZ (typically under 30 seconds versus 60 to 120 seconds) but introduces a different cost model and different read-scaling behavior through reader endpoints — understanding the storage layer is prerequisite to sizing Aurora correctly.
- Infrastructure as Code is not a convenience in cloud database environments — it is the mechanism through which provisioning-time decisions (encryption keys, storage type, Multi-AZ, deletion protection) become auditable, reproducible, and reviewable before a change reaches production.

Chapter 3: Cloud Database Landscape: AWS, Azure, GCP Side-by-Side

Every cloud provider sells you the same promise: managed databases that eliminate the undifferentiated heavy lifting of patching, backups, and hardware provisioning. The reality is that AWS, Azure, and GCP have taken fundamentally different architectural philosophies to reach that promise, and those differences shape every decision you make — from instance sizing to failover behavior to how you debug a slow query at 2 a.m. This chapter builds the mental map you need before writing a single line of Terraform or clicking a single console button. You will see how each platform organizes its database services, where their managed offerings genuinely replace on-prem work, and where the seams show.

3.1 The Service Taxonomy: What Each Cloud Actually Offers

Before comparing specifics, you need to understand the organizational logic of each platform. They do not map cleanly onto each other, and conflating them causes real architectural mistakes.

AWS Database Services

AWS built its database portfolio as a collection of purpose-built, largely independent services. For relational workloads, the primary offerings are:

- **Amazon RDS:** The foundational managed relational service. Supports MySQL, PostgreSQL, MariaDB, Oracle, and SQL Server. You pick an engine, pick an instance class, and AWS handles patching, backups, and failover. RDS is the closest thing to “take your on-prem database and move it to a managed host.”
- **Amazon Aurora:** AWS’s rewritten storage layer, available in MySQL-compatible and PostgreSQL-compatible variants. Aurora separates compute from storage in a way standard RDS does not. The storage layer automatically grows in 10 GB increments up to 128 TB, replicates across three Availability Zones at the storage level, and allows up to 15 read replicas without the replication lag penalty you see on standard RDS. Aurora is not just “faster RDS” — it is a different internal architecture.
- **Aurora Serverless v2:** Compute that scales in Aurora Capacity Units (ACUs), with a floor as low as 0.5 ACU and a ceiling that AWS has raised more than once (check current documentation for the exact maximum), with sub-second scaling response. One ACU is

approximately 2 GB of memory with proportional CPU. Useful for variable workloads, but the cold-start behavior when scaling from near-zero can surprise you in production.

- **RDS Proxy:** A fully managed connection pooler that sits in front of RDS or Aurora. It maintains a warm pool of connections to the database and presents a single endpoint to your application. Relevant when you have Lambda functions or containers generating thousands of short-lived connections.

Azure Database Services

Azure organizes its relational offerings around the concept of deployment models — how much of the infrastructure stack you control:

- **Azure SQL Database:** A fully managed PaaS service built on SQL Server. No OS access, no SQL Server Agent (limited version available), but a high published availability SLA (the exact figure varies by tier and zone redundancy — check current Microsoft documentation) and automatic tuning available. The purchasing model adds complexity: you choose between DTU-based pricing (a bundled compute/IO unit AWS has no equivalent for) and vCore-based pricing (which maps more cleanly to familiar instance sizing).
- **Azure SQL Managed Instance:** The “nearly full SQL Server” option. You get SQL Agent, cross-database queries, linked servers, and CLR — features that disappear in Azure SQL Database. The tradeoff is a longer provisioning time (up to four hours for some operations) and a higher cost floor.
- **Azure Database for PostgreSQL / MySQL:** Managed open-source database services, analogous to RDS PostgreSQL/MySQL. The Flexible Server deployment model is the current recommended path; the older Single Server model is being retired.
- **Azure SQL Hyperscale:** A distributed storage architecture for Azure SQL Database that allows databases up to 100 TB. The page server architecture separates compute from storage and enables near-instant database restores regardless of database size — a restore of a 10 TB database takes roughly the same time as restoring a 10 GB database.
- **Elastic Pools:** A resource-sharing model where multiple databases share a pool of DTUs or vCores. This is Azure’s answer to database consolidation — comparable to running multiple schemas on one PostgreSQL instance, but with more granular resource governance.

Google Cloud Database Services

GCP’s philosophy leans toward horizontal scale and global distribution more heavily than the other two:

- **Cloud SQL:** The managed MySQL, PostgreSQL, and SQL Server service. Functionally similar to RDS — it handles patching, backups, and HA failover. Storage scales automatically up to 64 TB. Cloud SQL HA uses a standby instance in a different zone with synchronous replication, promoted automatically on failure.
- **AlloyDB for PostgreSQL:** Google’s reengineered PostgreSQL-compatible service, analogous to how Aurora relates to RDS. AlloyDB separates log processing from storage, uses a distributed storage layer with columnar caching, and claims 4x faster transactional throughput than standard Cloud SQL PostgreSQL. AlloyDB also integrates the Vertex AI embedding generation directly within the database, which becomes relevant in Book 4 of this series.
- **Cloud Spanner:** A globally distributed, horizontally scalable relational database with external consistency (a stronger guarantee than serializable isolation). Spanner is in a category of its own — it uses TrueTime to order transactions across geographic regions. The SQL dialect is ANSI-compatible but not identical to PostgreSQL or SQL Server. Spanner is not a

drop-in replacement for anything; it requires application-level rethinking.

- **BigQuery:** Analytical, serverless, columnar. Not covered in this book except where it intersects with operational database exports.
- **Firestore / Bigtable:** NoSQL services outside the scope of this book.

The practical implication of this taxonomy: when a product manager says “we’re going multi-cloud,” you are not mapping AWS RDS to Azure SQL Database to Cloud SQL as equivalent services. Each has architectural behaviors that will affect your SLA, your cost model, and your operational runbooks differently.

3.2 Compute and Storage Architecture: What Changes Under the Hood

As an on-prem DBA, you have a mental model of “database server”: CPU, RAM, disks, and a file system you can inspect with `ls`. That model breaks in specific ways on each cloud platform. Understanding what breaks — and what remains — determines how you approach performance tuning and capacity planning.

AWS RDS: EBS-Based Single Instance

Standard RDS runs a single database process on an EC2-backed instance. Storage is Amazon EBS (Elastic Block Store). The key operational parameters:

- **gp3 storage:** General purpose SSD, baseline 3,000 IOPS up to 16,000 IOPS, throughput up to 1,000 MiB/s. You configure IOPS and throughput independently. For most OLTP workloads under 16,000 IOPS, gp3 is the right choice and costs less than provisioned IOPS.
- **io1/io2 storage:** Provisioned IOPS SSD, up to 256,000 IOPS for io2 Block Express. You pay per IOPS-month provisioned, so overprovisioning is expensive. Rule of thumb: if your peak IOPS demand exceeds 16,000 sustainably, move to io2.
- **Storage autoscaling:** You can enable autoscaling with a maximum threshold. RDS will expand storage automatically but will never shrink it. A common gotcha: a runaway job fills the disk, autoscaling kicks in, and you end up paying for 10 TB of storage you no longer need because you cannot scale down.
- **Instance classes:** RDS uses the `db.` prefix on EC2 instance families. The `db.r7g` family (Graviton3, memory-optimized) is the current sweet spot for most PostgreSQL workloads — a `db.r7g.4xlarge` gives you 16 vCPUs and 128 GB RAM. SQL Server workloads requiring Windows licensing have fewer Graviton options and typically run on `db.m7i` or `db.r7i`.

Aurora: Shared Distributed Storage

Aurora’s storage layer is the architectural departure that matters most. The Aurora storage is a distributed volume that spans six storage nodes across three AZs. Your Aurora cluster’s “writer” instance does not write to local disk — it writes log records to the distributed storage layer, which handles redo application independently of the compute layer. This means:

- Failover to a reader within the same region promotes without storage copy, typically completing in under 30 seconds. Aurora Global Database’s managed planned failover across regions is slower — AWS documents it as typically under a minute — since it involves cross-region coordination that a same-region reader promotion does not.

- Read replicas share the same storage volume. There is no binary log replication lag in the traditional sense — replicas apply the cache invalidation messages from the storage layer. Replica lag on a lightly loaded Aurora cluster is typically 10–20 milliseconds, not the 100ms–several seconds you may have seen on RDS read replicas under write pressure.
- You cannot SSH to the storage nodes. You cannot run `iostat` on the underlying storage. You lose the low-level observability you had on-prem and gain instead Aurora-specific CloudWatch metrics like `AuroraReplicaLag` and `VolumeWriteIOPs`.

Azure SQL: DTUs vs. vCores

The DTU (Database Transaction Unit) purchasing model is Azure’s attempt to simplify capacity selection by bundling CPU, memory, and IO into a single unit. The problem is that DTUs are opaque — a DTU is defined relative to performance on a specific Microsoft benchmark, not to hardware specs. You cannot predict whether a workload that runs on 8 CPU cores and 64 GB RAM on-prem will fit in 500 DTUs without testing.

The vCore model is more familiar. An 8-vCore General Purpose Azure SQL Database gives you 8 vCores and 40.8 GB memory. The Business Critical tier adds local SSD storage and a built-in readable secondary replica included in the cost. For new deployments, prefer vCore unless the workload is small and predictable enough that DTU bundles save money.

Azure SQL Hyperscale’s page server architecture is the most aggressive storage separation on the market. Page servers store the database pages, log service writes the transaction log, and the compute replica only caches the pages it needs. A 50 TB Hyperscale database restores in minutes because restore means redirecting pointers to a new snapshot, not copying bytes.

GCP Cloud SQL: Zonal with Optional HA

Cloud SQL instances run on GCP Compute Engine VMs with Persistent Disk (PD) storage. A standard Cloud SQL PostgreSQL instance is single-zone; if that zone fails, your database is unavailable until it recovers. Enabling High Availability provisions a standby instance in a different zone with synchronous replication through a regional Persistent Disk. Failover is automatic but takes 60–120 seconds in practice — significantly slower than Aurora’s sub-30-second failover.

Storage on Cloud SQL scales automatically up to 64 TB for PostgreSQL, but like RDS, it does not scale down. The `--storage-auto-increase-limit` flag lets you cap automatic growth.

AlloyDB uses a disaggregated storage design similar to Aurora but adds a columnar storage layer that caches frequently accessed data in a columnar format on the compute node. The practical effect: analytical queries that would require a separate OLAP system in traditional deployments can run acceptably on AlloyDB without a full scan of row-based storage.

3.3 Networking: VPCs, Private Link, and the End of `pg_hba.conf`

On-prem, you controlled database access through a combination of network topology (firewall rules, VLANs) and database-level access control (`pg_hba.conf` for PostgreSQL, `logins/users` for SQL Server). Cloud managed services extend and complicate this with additional abstraction layers. Getting networking wrong in cloud databases does not just cause connection failures — it creates security vulnerabilities and unexpected egress costs.

AWS VPC and RDS

Every RDS and Aurora instance lives inside a VPC subnet. You define which subnets the instance can use via a **DB Subnet Group**, which must span at least two AZs. Security Groups control inbound access — they are stateful firewall rules at the instance level.

The critical operational point: RDS instances in private subnets cannot be reached from the internet by default. To connect from your workstation, you need either a bastion host, AWS Systems Manager Session Manager tunneling, or a VPN/Direct Connect. This is the right posture for production, but it surprises engineers used to opening a pgAdmin connection directly to a server.

```
# Create a DB Subnet Group spanning three AZs
aws rds create-db-subnet-group \
  --db-subnet-group-name prod-postgres-subnets \
  --db-subnet-group-description "Production PostgreSQL subnets across 3 AZs" \
  --subnet-ids subnet-0a1b2c3d4e5f00001 subnet-0a1b2c3d4e5f00002 subnet-0a1b2c3d4e5f00003 \
  --region us-east-1

# Create Security Group allowing PostgreSQL access only from application subnet
aws ec2 create-security-group \
  --group-name rds-postgres-sg \
  --description "RDS PostgreSQL access from app tier" \
  --vpc-id vpc-0a1b2c3d4e5f00001

aws ec2 authorize-security-group-ingress \
  --group-id sg-0a1b2c3d4e5f12345 \
  --protocol tcp \
  --port 5432 \
  --source-group sg-0a1b2c3d4e5f67890 # app tier security group
```

RDS Proxy and Connection Management

If you have ever tuned `max_connections` in PostgreSQL and dealt with the “too many connections” error under load, RDS Proxy addresses exactly this problem for managed services. Lambda functions are the primary driver — each Lambda invocation can open its own database connection, and a spike to 1,000 concurrent invocations means 1,000 simultaneous connections, which will overwhelm a `db.r6g.large` long before you’d want to rely on raw connection count (check the instance’s default `max_connections` for the actual ceiling, and note that performance degrades well before that ceiling is hit).

RDS Proxy maintains a persistent connection pool to the database and multiplexes application connections against it. The proxy endpoint replaces your database endpoint in the application connection string. The proxy also integrates with AWS Secrets Manager and IAM for authentication, so your application never handles a plaintext database password.

```
# Create RDS Proxy for Aurora PostgreSQL cluster
aws rds create-db-proxy \
  --db-proxy-name prod-aurora-proxy \
  --engine-family POSTGRES \
  --auth '[{
    "AuthScheme": "SECRETS",
    "SecretArn": "arn:aws:secretsmanager:us-east-1:123456789012:secret:prod/aurora/pgcreds",
    "IAMAuth": "REQUIRED"
  }]' \
  --role-arn arn:aws:iam::123456789012:role/rds-proxy-role \
  --vpc-subnet-ids subnet-0a1b2c3d4e5f00001 subnet-0a1b2c3d4e5f00002 \
  --vpc-security-group-ids sg-0a1b2c3d4e5f12345 \
  --require-tls
```

Azure Private Link and Service Endpoints

Azure SQL Database has a public endpoint by default (in older deployments). For production, you

want a **Private Endpoint** — an Azure Private Link resource that assigns a private IP from your VNet to the Azure SQL logical server. Once a private endpoint exists, you can deny all public access via the firewall.

The Azure SQL firewall is conceptually similar to `pg_hba.conf` in that it operates at the server level before the database authentication step. But unlike `pg_hba.conf` where you write IP ranges directly into a config file, the Azure SQL firewall rules are API-managed objects attached to the logical server.

```
# Create Azure SQL Private Endpoint
az network private-endpoint create \
  --name prod-sql-private-endpoint \
  --resource-group prod-rg \
  --vnet-name prod-vnet \
  --subnet prod-db-subnet \
  --private-connection-resource-id \

  ↪ "/subscriptions/12345678-abcd-1234-abcd-123456789012/resourceGroups/prod-rg/providers/Microsoft.Sql/servers/prod-
  ↪ \
  --group-id sqlServer \
  --connection-name prod-sql-pe-connection

# Disable public network access after private endpoint is confirmed active
az sql server update \
  --name prod-sql-server \
  --resource-group prod-rg \
  --enable-public-network false
```

GCP VPC and Cloud SQL Authorized Networks

Cloud SQL can use two connectivity models: **Authorized Networks** (an IP allowlist — the simplest but least secure approach, similar to `pg_hba.conf` host-based entries) and **Private Service Access** (a VPC peering-based model that gives Cloud SQL a private IP in your VPC).

The recommended production approach uses Private Service Access combined with the **Cloud SQL Auth Proxy**, a sidecar process that handles IAM authentication and TLS tunneling without requiring direct IP allowlisting. The proxy runs alongside your application and presents a local socket that your application connects to as if it were a local PostgreSQL instance.

```
# Assign private IP to Cloud SQL instance
gcloud sql instances patch prod-postgres \
  --network=projects/my-project/global/networks/prod-vpc \
  --no-assign-ip \
  --project my-project

# Run Cloud SQL Auth Proxy (typically as a sidecar in Kubernetes)
# Download: https://cloud.google.com/sql/docs/postgres/sql-proxy
./cloud-sql-proxy \
  --address 127.0.0.1 \
  --port 5432 \
  my-project:us-central1:prod-postgres &
```

The Cloud SQL Auth Proxy is analogous to AWS RDS Proxy in that it handles authentication abstraction, but it does not provide connection pooling. For pooling with Cloud SQL, you run PgBouncer or pgpool-II separately — either as a sidecar or as a dedicated VM.

3.4 Authentication and Access Control: IAM Is the New pg_hba.conf

The phrase “IAM is the new pg_hba.conf” captures the conceptual shift without being entirely accurate. IAM controls which entities can perform management-plane operations (create instances, modify parameter groups, take snapshots). Database-level authentication — which users can log in to the database itself — remains a separate concern on every cloud platform, but IAM is increasingly integrated into that layer too.

AWS IAM Database Authentication

RDS and Aurora support IAM database authentication for MySQL and PostgreSQL. Instead of password-based login, an IAM entity (user or role) generates a short-lived authentication token using the AWS SDK. The token is valid for 15 minutes. On the database side, the user is created with the `rds_iam` authentication plugin enabled. The workflow:

1. Application assumes an IAM role.
2. Application calls `generate_db_auth_token` to get a 15-minute token.
3. Application connects using the token as the password.
4. RDS validates the token against IAM.

```
import boto3
import psycopg2

def get_iam_auth_token(hostname, port, username, region):
    client = boto3.client('rds', region_name=region)
    token = client.generate_db_auth_token(
        DBHostname=hostname,
        Port=port,
        DBUsername=username,
        Region=region
    )
    return token

def connect_rds_iam():
    hostname = "prod-aurora.cluster-xyz.us-east-1.rds.amazonaws.com"
    port = 5432
    username = "app_service_user"
    region = "us-east-1"
    database = "appdb"

    token = get_iam_auth_token(hostname, port, username, region)

    # SSL is required for IAM auth
    conn = psycopg2.connect(
        host=hostname,
        port=port,
        database=database,
        user=username,
        password=token,
        sslmode="require"
    )
    return conn
```

The database user still needs to exist as a PostgreSQL role — IAM auth does not bypass PostgreSQL’s internal user system. The integration point is the authentication method: the password field carries the IAM token, and RDS validates it before PostgreSQL processes the login.

Azure SQL: Azure Active Directory Integration

Azure SQL supports both SQL authentication (username/password stored in the database) and

Azure Active Directory authentication. AAD authentication is the recommended path for production because it enables conditional access policies, MFA for human users, and managed identity authentication for applications.

An Azure Managed Identity — a service principal automatically managed by Azure for a VM, App Service, or container — can authenticate to Azure SQL without any credentials in the application configuration. This is functionally equivalent to the IAM role-based RDS authentication above:

```
# Enable AAD admin for Azure SQL Server
az sql server ad-admin create \
  --server prod-sql-server \
  --resource-group prod-rg \
  --display-name "DBA-Team-AAD-Group" \
  --object-id "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx"

# Create contained database user for a Managed Identity
# Run this SQL as the AAD admin inside the target database

-- Create database user for App Service Managed Identity
-- Run as AAD admin inside the target database
CREATE USER [prod-app-service] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [prod-app-service];
ALTER ROLE db_datawriter ADD MEMBER [prod-app-service];

-- Verify AAD users
SELECT name, type_desc, authentication_type_desc
FROM sys.database_principals
WHERE type IN ('E', 'X'); -- E = external user, X = external group
```

GCP Cloud IAM and Cloud SQL

Cloud SQL integrates with GCP IAM through a feature called **Cloud SQL IAM database authentication** for PostgreSQL. A Cloud IAM user or service account is mapped to a PostgreSQL role using a login trigger. The user authenticates with an OAuth2 access token rather than a password. The role name in the database matches the email address of the IAM identity (truncated to 63 characters, which can cause collisions you need to test for in advance).

The conceptual alignment: in `pg_hba.conf`, you would write an `ident` or `peer` line to map OS users to database roles. Cloud SQL IAM auth does the same thing, replacing OS identity with GCP IAM identity.

Always Encrypted (Azure SQL)

Azure SQL offers Always Encrypted — a client-side encryption mechanism where encryption and decryption happen in the application layer, never in the database engine. The database server stores ciphertext and cannot read the plaintext values, even with administrative access. This is a fundamentally different threat model than TDE (Transparent Data Encryption), which only encrypts at-rest storage and is transparent to anyone with database access.

Always Encrypted requires application-level changes: the connection string must include the column encryption setting, and the client driver handles the cryptographic operations using keys stored in Azure Key Vault, Windows Certificate Store, or other supported keystores. The operational trade-off: you cannot filter on encrypted columns with inequality operators (a search for salaries between \$50,000 and \$100,000 on an encrypted column requires decrypting all rows client-side). Always Encrypted with Randomized Encryption supports no server-side filtering at all; Deterministic Encryption supports equality comparisons only.

3.5 High Availability and Disaster Recovery: Comparing the Failover Mechanics

HA is where managed services diverge most sharply from on-prem setups, and where the “managed” abstraction creates the most operational surprises. You need to understand what each platform does automatically, what you still configure, and what the failure scenarios look like.

AWS RDS Multi-AZ

Standard RDS Multi-AZ maintains a synchronous standby replica in a second AZ. The standby is not readable — it is purely a failover target. Failover happens automatically when RDS detects instance failure, AZ outage, or network partitioning. The DNS endpoint for the instance is updated to point to the promoted standby. Expected failover time: 60–120 seconds. Applications that cache DNS or use long connection timeouts may not recover within this window without explicit reconnect logic.

RDS Multi-AZ DB Cluster (a newer feature, not to be confused with Aurora) provides two readable standby instances in separate AZs with a stated failover time under 35 seconds. It uses semi-synchronous replication — different from Aurora’s storage-level replication. Choose this over standard Multi-AZ when you need the read capacity of the standbys and can tolerate the additional cost.

Aurora Global Database

Aurora Global Database extends the Aurora cluster across AWS Regions for disaster recovery and global read distribution. The primary Region hosts the writer and up to 15 readers. Secondary Regions each receive the storage log stream from the primary with typical replication lag under 1 second (the actual observed lag varies by workload and region distance — confirm against your own replication latency metrics before committing to an RPO based on vendor documentation).

Failover to a secondary Region — called a **managed planned failover** — promotes the secondary Region to primary. The process detects that the secondary has caught up to the primary, then redirects writes. For disaster scenarios where the primary Region is unavailable, you perform an **unplanned detach and promote**, which may lose the transactions that had not yet replicated. The RPO in a true regional outage is therefore bounded by that replication lag, not zero.

```
# Initiate managed planned failover for Aurora Global Database
# (use when primary region is healthy - for maintenance or DR testing)
aws rds failover-global-cluster \
  --global-cluster-identifier prod-global-aurora \
  --target-db-cluster-identifier \
    arn:aws:rds:eu-west-1:123456789012:cluster:prod-aurora-eu-secondary \
  --region us-east-1

# For unplanned failover when primary region is unreachable:
# First, remove the secondary from the global cluster (detach)
aws rds remove-from-global-cluster \
  --db-cluster-identifier \
    arn:aws:rds:eu-west-1:123456789012:cluster:prod-aurora-eu-secondary \
  --global-cluster-identifier prod-global-aurora \
  --region eu-west-1

# The secondary is now an independent cluster - promote it
# (it is already writable once detached; update your application endpoint)
```

Azure SQL Business Critical and Geo-Replication

Azure SQL Business Critical tier automatically provisions an Always On availability group across

nodes within a single region. You get a built-in readable secondary at no additional replica cost, with failover typically completing in under 30 seconds. This is the closest Azure analog to Aurora's standard HA.

For cross-region DR, Azure SQL offers two mechanisms:

- **Active Geo-Replication:** Creates up to four readable secondary databases in other regions. Failover is manual or application-triggered. Secondary databases are readable, making them useful for read offloading in addition to DR.
- **Auto-Failover Groups:** A layer above Active Geo-Replication that provides a single listener endpoint (a DNS alias) which automatically redirects to the current primary. On failover, the endpoint updates transparently. You configure the failover policy — automatic or manual — and the grace period before automatic failover triggers.

```
# Create Auto-Failover Group between two Azure SQL logical servers
az sql failover-group create \
  --name prod-fog \
  --partner-server prod-sql-server-dr \
  --resource-group prod-rg \
  --server prod-sql-server \
  --failover-policy Automatic \
  --grace-period 1 \
  --add-db prod-database-1 prod-database-2

# Check replication health
az sql failover-group show \
  --name prod-fog \
  --resource-group prod-rg \
  --server prod-sql-server \
  --query "{role:replicationRole, state:replicationState, partnerServer:partnerServers[0].id}"

# Manual failover to secondary (for DR testing or planned maintenance)
az sql failover-group set-primary \
  --name prod-fog \
  --resource-group prod-rg \
  --server prod-sql-server-dr
```

One subtle operational point with Auto-Failover Groups: the read-write listener always points to the current primary, and the read-only listener always points to the current secondary. If your application uses a single connection string for both reads and writes, it will work after failover. If your application uses separate connection strings for the read-only replica, verify those strings use the read-only listener endpoint (`<fog-name>.secondary.database.windows.net`) rather than the secondary server's direct hostname — the direct hostname stops being the secondary after failover.

GCP Cloud SQL HA and Cross-Region Replicas

Cloud SQL HA uses a primary instance and a standby instance in separate zones within the same region, connected by a shared regional Persistent Disk. When the primary fails, the standby mounts the disk and resumes. Because the disk is regional (synchronously replicated across zones), no data is lost — the RPO is zero for zone failures. The RTO is approximately 60–120 seconds, including time to detect the failure and complete the promotion.

For cross-region DR, Cloud SQL uses **Cross-Region Read Replicas**. These are asynchronous replicas in a different region. Unlike Aurora Global Database, these replicas cannot be promoted with a single API call that preserves the global endpoint — promoting a cross-region replica to a standalone instance requires updating your application's connection string manually or through automation you build yourself.

```
# Create cross-region read replica for DR
```



```
gcloud sql instances create prod-postgres-dr \
  --master-instance-name=prod-postgres \
  --region=us-west1 \
  --database-version=POSTGRES_15 \
  --tier=db-custom-8-32768 \
  --availability-type=REGIONAL \
  --project=my-project

# Check replication lag on cross-region replica
gcloud sql instances describe prod-postgres-dr \
  --project=my-project \
  --format="value(replicaConfiguration.replicationLagMaxSeconds)"

# Promote replica to standalone (DR failover - point of no return)
gcloud sql instances promote-replica prod-postgres-dr \
  --project=my-project
```

AlloyDB introduces a different HA model. AlloyDB clusters are regional by default, and the read pool instances within a region can absorb traffic during a primary failure while a new primary is provisioned from the distributed storage layer — similar to how Aurora promotes a reader. AlloyDB cross-region DR currently relies on backups or third-party replication tools, though Google continues to extend AlloyDB’s capabilities.

3.6 Backup and Point-in-Time Recovery: What “Managed” Actually Means

On-prem, you owned the backup strategy: take a base backup with `pg_basebackup` or SQL Server backup to disk, ship transaction logs to a secondary location, test restores quarterly (or, regrettably, never). Cloud managed services automate the base backup and log archiving, but they do not automate the testing, and the restore mechanics differ in ways that catch DBAs off guard.

AWS RDS and Aurora Backups

RDS automated backups capture a daily snapshot and stream transaction logs (PostgreSQL WAL, MySQL binary logs) to S3 throughout the day. Point-in-time recovery allows restoration to any second within the backup retention window (1–35 days, configurable). PITR always restores to a new instance — you cannot restore in-place over the existing instance. This is the correct behavior from a safety perspective but means your DR runbook must account for updating application connection strings after restore.

Aurora backups are stored in S3 in the Aurora storage subsystem — not as traditional filesystem snapshots. This is what enables the fast restore behavior: Aurora restores by making the backup’s storage blocks available to a new compute instance, then replaying only the WAL records since the last backup as the instance warms up. The practical result is that Aurora PITR to a large database is significantly faster than RDS PITR to the same effective dataset size.

```
# Restore Aurora cluster to a specific point in time
aws rds restore-db-cluster-to-point-in-time \
  --db-cluster-identifier prod-aurora-restored-20241115 \
  --source-db-cluster-identifier prod-aurora \
  --restore-to-time "2024-11-15T03:45:00Z" \
  --vpc-security-group-ids sg-0a1b2c3d4e5f12345 \
  --db-subnet-group-name prod-postgres-subnets \
  --region us-east-1

# After cluster restore, create a writer instance in the restored cluster
```

```
aws rds create-db-instance \  
  --db-instance-identifier prod-aurora-restored-writer \  
  --db-cluster-identifier prod-aurora-restored-20241115 \  
  --db-instance-class db.r7g.2xlarge \  
  --engine aurora-postgresql \  
  --region us-east-1
```

Manual snapshots exist outside the automated backup retention window. If you need to preserve the state of a database before a major schema migration, take a manual snapshot. Manual snapshots persist until explicitly deleted — they do not expire with the automated backup window.

Azure SQL Backup

Azure SQL Database performs automated full backups weekly, differential backups every 12–24 hours, and transaction log backups every 5–12 minutes. The retention period for PITR is 1–35 days on most tiers. Hyperscale uses a different mechanism — snapshots stored on Azure Blob Storage — which is why Hyperscale restores are fast regardless of database size.

The PITR restore in Azure SQL creates a new database on the same logical server (or a different server in the same region). You cannot restore over an existing database.

```
# Restore Azure SQL Database to point in time  
az sql db restore \  
  --dest-name prod-database-restored-20241115 \  
  --edition GeneralPurpose \  
  --service-objective GP_Gen5_8 \  
  --name prod-database-1 \  
  --resource-group prod-rg \  
  --server prod-sql-server \  
  --time "2024-11-15T03:45:00Z"  
  
# Check restore progress  
az sql db show \  
  --name prod-database-restored-20241115 \  
  --resource-group prod-rg \  
  --server prod-sql-server \  
  --query "{status:status, createMode:createMode}"
```

Long-term retention (LTR) in Azure SQL allows you to keep full backups for up to 10 years — useful for compliance scenarios where you need to restore data from two years ago to answer a legal hold. LTR backups are stored in geo-redundant Azure Blob Storage and count against your storage costs separately.

GCP Cloud SQL and AlloyDB Backups

Cloud SQL automated backups take daily snapshots and retain transaction logs for PITR within the retention window (up to 7 days by default, configurable up to 35 days for PostgreSQL 14+). PITR restores to a new instance, consistent with the behavior on AWS and Azure.

```
# List available backups for a Cloud SQL instance  
gcloud sql backups list \  
  --instance=prod-postgres \  
  --project=my-project \  
  --limit=10  
  
# Restore to point in time - creates a new instance  
gcloud sql instances clone prod-postgres prod-postgres-pitr-20241115 \  
  --point-in-time="2024-11-15T03:45:00.000Z" \  
  --project=my-project  
  
# Export a specific database to GCS for long-term retention  
gcloud sql export sql prod-postgres \  
  gs://my-backup-bucket/exports/prod-postgres-20241115.sql.gz \  
  --database=appdb
```

```
--project=my-project
```

AlloyDB automated backups operate similarly, but AlloyDB’s distributed storage layer makes the backup process non-blocking — backup I/O does not contend with production workload I/O in the same way it can on single-instance Cloud SQL.

A discipline that every DBA should build, regardless of cloud platform: **schedule regular restore tests**. AWS, Azure, and GCP all automate the backup creation, but none of them verify that the backup is restorable and that the restored database is consistent. Monthly restore tests to a non-production environment, with a validation query that checks row counts against known good values, should be part of every production runbook. The managed service handles the mechanics; the verification remains your responsibility.

3.7 Performance Monitoring: What You Get, What You Have to Build

On-prem, a PostgreSQL DBA reaches for `pg_stat_activity`, `pg_stat_bgwriter`, `pg_locks`, and the slow query log. A SQL Server DBA reaches for DMVs, SQL Server Profiler (or Extended Events), and the Query Store. Cloud platforms wrap these native tools with additional abstraction layers and, in some cases, surface insights that would take significant tooling to produce on-prem.

AWS Performance Insights

Performance Insights is AWS’s answer to the question “why is my database slow right now?” It exposes the database’s active session data — the same information underlying `pg_stat_activity` — through a time-series visualization of **Average Active Sessions (AAS)** broken down by wait event, SQL statement, host, user, and database.

The key metric is the **DB Load** line: the number of sessions actively executing or waiting at any given second. When DB Load exceeds the number of vCPUs on your instance, you have more demand than supply — the database is CPU-bound or has a wait bottleneck worth investigating. Performance Insights retains 7 days of data on the free tier and up to 2 years with the paid tier.

```
# Retrieve Performance Insights metrics via CLI
aws pi get-resource-metrics \
  --service-type RDS \
  --identifier db-ABCDEFGHIJKLMNQRSTUWXYZ \
  --metric-queries '[
    {
      "Metric": "db.load.avg",
      "GroupBy": {
        "Group": "db.wait_event",
        "Dimensions": ["db.wait_event.name", "db.wait_event.type"],
        "Limit": 5
      }
    }
  ]' \
  --start-time "2024-11-15T00:00:00Z" \
  --end-time "2024-11-15T01:00:00Z" \
  --period-in-seconds 60 \
  --region us-east-1
```

Enhanced Monitoring is a separate feature that captures OS-level metrics (CPU steal, file system I/O, process list) from the host at intervals as low as 1 second. This is the closest cloud-managed equivalent to running `top`, `iostat`, and `vmstat` on the database host. Enhanced Monitoring data

flows to CloudWatch Logs, not CloudWatch Metrics, so querying it programmatically requires Log Insights queries rather than standard GetMetricData calls.

Azure Query Performance Insight and Query Store

Azure SQL Database automatically enables **Query Store**, the SQL Server feature that captures query execution statistics and plan history. Query Store data is queryable through the standard DMVs:

```
-- Top 10 queries by total CPU time over the last 24 hours (Azure SQL / SQL Server)
SELECT TOP 10
    qsq.query_id,
    qsqt.query_sql_text,
    SUM(qsrs.avg_cpu_time * qsrs.count_executions) AS total_cpu_time_us,
    SUM(qsrs.count_executions) AS total_executions,
    MAX(qsrs.last_execution_time) AS last_execution
FROM sys.query_store_query qsq
JOIN sys.query_store_query_text qsqt ON qsq.query_text_id = qsqt.query_text_id
JOIN sys.query_store_plan qsp ON qsq.query_id = qsp.query_id
JOIN sys.query_store_runtime_stats qsrs ON qsp.plan_id = qsrs.plan_id
JOIN sys.query_store_runtime_stats_interval qsrsi
    ON qsrs.runtime_stats_interval_id = qsrsi.runtime_stats_interval_id
WHERE qsrsi.start_time >= DATEADD(HOUR, -24, GETUTCDATE())
GROUP BY qsq.query_id, qsqt.query_sql_text
ORDER BY total_cpu_time_us DESC;

-- Identify queries with plan regressions (new plan is slower than historical best)
SELECT
    qsq.query_id,
    qsqt.query_sql_text,
    rsp.plan_id AS regressed_plan_id,
    rsp.avg_duration AS regressed_avg_duration_us,
    bsp.plan_id AS best_plan_id,
    bsp.avg_duration AS best_avg_duration_us
FROM sys.dm_db_tuning_recommendations tr
CROSS APPLY OPENJSON(tr.details, '$.queryId') WITH (query_id INT '$') qid
JOIN sys.query_store_query qsq ON qsq.query_id = qid.query_id
JOIN sys.query_store_query_text qsqt ON qsq.query_text_id = qsqt.query_text_id
CROSS APPLY OPENJSON(tr.details, '$.regressedPlanId') WITH (plan_id BIGINT '$') rpid
CROSS APPLY OPENJSON(tr.details, '$.recommendedPlanId') WITH (plan_id BIGINT '$') bpid
JOIN sys.query_store_plan rsp ON rsp.plan_id = rpid.plan_id
JOIN sys.query_store_plan bsp ON bsp.plan_id = bpid.plan_id
WHERE tr.reason = N'PlanChange'
    AND tr.state = 1; -- Active recommendation
```

Query Performance Insight in the Azure portal is a GUI layer over Query Store that highlights the top CPU-consuming and long-running queries. It is useful for initial triage but limited for deep investigation — experienced DBAs will drop into the DMVs directly.

Azure SQL also offers **Automatic Tuning**, which can automatically create missing indexes and force the last known good query plan when a regression is detected. Automatic index creation is worth enabling in non-production environments to surface index recommendations, but treat it carefully in production: automatically created indexes consume storage and write overhead, and the automated decision logic may not account for your maintenance windows or storage constraints.

GCP Cloud SQL Insights and AlloyDB

Query Insights for Cloud SQL provides a sampling-based view of query activity — similar conceptually to Performance Insights but built around the PostgreSQL `pg_stat_statements` extension and sampling of active queries. It surfaces the top queries by execution count, mean latency, and rows processed.

For PostgreSQL, the underlying native tooling remains accessible through standard catalog queries:

```
-- Top queries by total execution time in PostgreSQL (Cloud SQL, AlloyDB, RDS, Aurora)
-- Requires pg_stat_statements extension
SELECT
    substring(query, 1, 80) AS query_snippet,
    calls,
    round(total_exec_time::numeric, 2) AS total_exec_time_ms,
    round(mean_exec_time::numeric, 4) AS mean_exec_time_ms,
    round((100.0 * total_exec_time / sum(total_exec_time) OVER ())::numeric, 2) AS pct_total
FROM pg_stat_statements
WHERE query NOT LIKE '%pg_stat_statements%'
ORDER BY total_exec_time DESC
LIMIT 20;

-- Identify high lock wait activity
SELECT
    pid,
    now() - pg_stat_activity.query_start AS duration,
    query,
    state,
    wait_event_type,
    wait_event
FROM pg_stat_activity
WHERE wait_event_type = 'Lock'
    AND state != 'idle'
ORDER BY duration DESC;
```

The SQL Server equivalent for lock wait analysis uses DMVs:

```
-- Active lock waits in SQL Server / Azure SQL
SELECT
    r.session_id,
    r.blocking_session_id,
    r.wait_type,
    r.wait_time / 1000.0 AS wait_time_seconds,
    r.status,
    substring(st.text, (r.statement_start_offset / 2) + 1,
        CASE r.statement_end_offset
            WHEN -1 THEN datalength(st.text)
            ELSE (r.statement_end_offset - r.statement_start_offset) / 2
        END) AS statement_text,
    db_name(r.database_id) AS database_name
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) st
WHERE r.blocking_session_id > 0
    OR r.wait_type LIKE 'LCK%'
ORDER BY r.wait_time DESC;
```

AlloyDB for PostgreSQL surfaces standard `pg_stat_*` views plus AlloyDB-specific metrics for its columnar cache. The `google_columnar_cache` schema provides insight into which tables and columns are cached in the columnar store and the cache hit rate — useful for understanding whether analytical queries are benefiting from the columnar acceleration layer.

3.8 The Operational Cost Model: Thinking in Cloud Units

The final mental model shift in this chapter is economic. On-prem, you amortize hardware costs over three to five years and pay for licenses, DBA headcount, and data center costs. In the cloud, you pay for every resource-hour consumed. This changes how you make sizing decisions, how you think about idle capacity, and how you evaluate feature tradeoffs.

AWS RDS Pricing Dimensions

An Aurora PostgreSQL deployment has multiple cost components:

- **Instance hours:** Charged per second for the time each compute instance (writer and readers) runs. Reserved Instances (1-year or 3-year commitments) reduce the effective hourly rate substantially for stable workloads.
- **Aurora I/O:** For Aurora Standard, each million read and write I/O operations to the storage layer is charged separately. I/O costs can dominate total spend for write-heavy OLTP workloads. **Aurora I/O-Optimized** is a pricing tier that eliminates the per-I/O charge in exchange for a higher instance and storage base rate. At roughly 25% or more of your Aurora cost coming from I/O charges, the I/O-Optimized tier becomes worth evaluating.
- **Storage:** Charged per GB-month for the Aurora storage volume, which grows automatically and never shrinks. Data you deleted six months ago still occupies storage until Aurora's background reclamation completes.
- **Backup storage:** Automated backups within the retention window are included. Storage beyond the database size is charged per GB-month.
- **Data transfer:** Intra-region data transfer between RDS and EC2 in the same AZ is free. Cross-AZ data transfer (which happens when your application is in AZ-a and your Aurora reader endpoint resolves to AZ-b) is charged per GB. Using the Aurora cluster endpoint reader with AZ affinity routing reduces this cost.

Azure SQL Pricing Dimensions

Azure SQL vCore pricing separates compute and storage:

- **vCores:** Charged per vCore-hour. The Serverless compute tier bills per vCore-second when active and a flat rate per GB-hour when paused — useful for development databases that are idle overnight.
- **Storage:** Charged per GB-month for the allocated storage. General Purpose tier uses remote storage; Business Critical tier uses local SSD included in the compute pricing.
- **Backup storage:** Locally redundant backup storage up to the database size is included. Geo-redundant backup storage is charged extra.
- **Elastic Pools:** Share a vCore or DTU allocation across multiple databases. If you have 20 databases where peak activity is staggered, pooling can cut compute costs compared to provisioning each database independently.

The Azure Hybrid Benefit allows you to apply existing on-premises SQL Server licenses with active Software Assurance to reduce the Azure SQL compute cost. For organizations migrating from on-prem SQL Server, this can make a significant difference in total cost — verify the current discount percentages with Microsoft's licensing terms, as they are updated periodically.

GCP Cloud SQL Pricing Dimensions

Cloud SQL pricing includes:

- **Instance type:** Charged per vCPU and GB of RAM per hour. Custom machine types allow precise sizing — you are not locked into predefined instance families.
- **Storage:** Charged per GB-month for SSD or HDD persistent disk.
- **High Availability:** Enabling HA doubles the compute cost (you pay for the standby instance).
- **Read replicas:** Charged at the same rate as the primary instance per replica.
- **Egress:** Data transferred out of GCP to the internet is charged per GB. Data transferred within the same region between Cloud SQL and Cloud Run or GKE is typically free.

The persistent-disk storage model on Cloud SQL means that provisioned storage counts against

your bill whether you use it or not. If you provision 2 TB and use 200 GB, you pay for 2 TB. Monitor storage utilization and adjust provisioning accordingly — or enable storage autoscaling with a reasonable cap.

Right-sizing in Practice

The most common cost optimization mistake for cloud databases is running on the same instance size chosen during initial provisioning without revisiting it. The right workflow:

1. Enable Performance Insights (AWS), Query Performance Insight (Azure), or Query Insights (GCP).
 2. After two to four weeks of representative traffic, look at the sustained CPU utilization and memory pressure metrics.
 3. If average CPU stays below 20% and you never see memory pressure, consider sizing down one instance tier.
 4. If you see sustained CPU above 60–70%, investigate query tuning before scaling up — you may be paying for compute to execute inefficient queries.
 5. Use Reserved Instances (AWS) or Committed Use Discounts (GCP) for your baseline capacity once you have a stable sizing baseline. Paying on-demand rates for a production database running continuously is one of the most visible and correctable cloud cost inefficiencies.
-

Key Takeaways

- **The three cloud platforms are not equivalent at the service layer.** Aurora’s distributed storage model, Azure SQL’s DTU vs. vCore models, and Cloud SQL’s zonal-vs-regional HA have real architectural consequences that affect failover time, replica behavior, and storage costs — mapping them as interchangeable is an architectural mistake.
 - **Networking has become a first-class database configuration concern.** Private endpoints, VPC peering, Security Groups, and connection proxies (RDS Proxy, Cloud SQL Auth Proxy) replace the IP-based rules of `pg_hba.conf` and require deliberate architecture decisions before your first production deployment.
 - **IAM integration reduces credential sprawl but does not eliminate database-level access control.** IAM tokens, Azure Managed Identity, and GCP service account authentication control who can connect, but PostgreSQL roles and SQL Server logins still govern what they can do inside the database — both layers must be managed.
 - **HA failover times and RPO characteristics differ substantially across platforms.** Aurora’s sub-30-second failover from shared storage contrasts with Cloud SQL’s 60–120 second zone failover and Azure SQL Business Critical’s Always On behavior. Design application reconnect logic and SLA commitments around the actual failover mechanics of your chosen platform, not a generic “managed = highly available” assumption.
 - **Backup automation covers creation, not validation.** Every managed database service handles scheduled backups automatically, but restore testing remains a manual operational responsibility. Monthly restore drills to a non-production environment are the only reliable way to know that your backup retention window represents actual recoverability.
-

Chapter 4: The Cloud DBA Role: What Changes, What Stays

If you have spent years tuning PostgreSQL `shared_buffers`, managing SQL Server Always On Availability Groups, and writing runbooks for failover procedures at 2 AM, the shift to managed cloud databases can feel disorienting. AWS, Azure, and Google Cloud take over large portions of the operational work you once owned completely — OS patching, storage provisioning, replication wiring — while introducing entirely new categories of responsibility that did not exist in your on-premises world. This chapter maps that transition precisely. It identifies what the cloud genuinely removes from your plate, what it hands back to you in a different form, and what it adds as net-new work. Understanding this boundary is the foundation for everything else in this book.

4.1 The Managed Service Contract: What AWS, Azure, and GCP Actually Own

When you move from self-managed PostgreSQL to Amazon RDS, you are entering into an implicit contract. The cloud provider owns the hypervisor, the operating system, the storage subsystem, the replication mechanism, and the software patching lifecycle. You own the database configuration, the schema design, the query performance, the access control model, and the backup retention policy. The line between those two domains is firm, and understanding it prevents both false assumptions and wasted effort.

What the provider owns:

On RDS for PostgreSQL, you will never run `apt-get upgrade postgresql`, restart the OS, or access `/var/lib/postgresql` directly. AWS handles minor version patching on a schedule you influence but do not fully control unless you disable auto minor version upgrades explicitly. The storage layer is EBS-backed with automatic replication within an Availability Zone. Multi-AZ deployments add a synchronous standby in a second AZ, and failover is handled automatically — typically completing in 60–120 seconds for RDS, and 30 seconds or less for Aurora.

On Azure SQL Database, Microsoft manages the underlying SQL Server engine, the Always On availability group that provides HA, and the Intelligent Query Processing features that ship with each compatibility level upgrade. Google Cloud SQL similarly abstracts OS management and handles read replica replication lag monitoring internally.

What you still own:

The provider's ownership does not extend to logical database design, index strategy, connection pooling, application-layer retry logic, or query plans. A poorly written query that causes a full table scan on a 2 TB table will perform exactly as badly on RDS as it did on your on-premises server. Cloud-managed does not mean self-healing at the query level. Your expertise in execution plans, wait events, lock escalation, and statistics management transfers directly and remains essential.

The practical implication is this: you spend less time on infrastructure operations and more time on database architecture decisions, performance engineering, and cost optimization. That is a genuine shift in how a senior DBA allocates time, not a reduction in the need for DBA expertise.

4.2 Access Patterns and Permissions: From `pg_hba.conf` to IAM

On-premises PostgreSQL access control flows through two files: `postgresql.conf` controls what the engine does, and `pg_hba.conf` controls who can connect, from where, and with what authentication method. SQL Server uses logins, server roles, and Windows Authentication through Active Directory. In the cloud, these familiar mechanisms are extended — not replaced — by identity and access management layers that sit above the database engine.

IAM Authentication on RDS

With IAM database authentication enabled on RDS PostgreSQL, a user connects using a short-lived authentication token generated by the AWS IAM service rather than a static password. The token is valid for 15 minutes and is generated using the AWS credentials of the calling identity — an EC2 instance profile, a Lambda execution role, or a developer's IAM user. The database-side configuration still uses `pg_hba.conf`, but the method becomes `md5` or `scram-sha-256` is bypassed in favor of the IAM token validation path.

```
# Enable IAM authentication on an existing RDS PostgreSQL instance
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --enable-iam-database-authentication \
  --apply-immediately

# Generate an authentication token for a database user named "app_user"
aws rds generate-db-auth-token \
  --hostname prod-postgres-01.cxyz1234.us-east-1.rds.amazonaws.com \
  --port 5432 \
  --region us-east-1 \
  --username app_user
```

On the database side, the user must exist with the `rds_iam` role granted:

```
-- Create a database user that authenticates via IAM
CREATE USER app_user WITH LOGIN;
GRANT rds_iam TO app_user;

-- The user has no password - authentication is entirely token-based
-- Grant object-level privileges as you normally would
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO app_user;
```

This pattern replaces the practice of embedding static database passwords in application configuration files. The calling application retrieves a fresh token before each connection attempt (or connection pool initialization), using the AWS SDK. The conceptual analog is Kerberos authentication with Active Directory — the application proves its identity to a central authority and receives a time-limited credential.

Azure SQL and Azure Active Directory

Azure SQL Database integrates with Azure Active Directory using contained database users. The model is familiar if you have used Windows Authentication with SQL Server, but the administration happens through the Azure portal, Azure CLI, or Entra ID (formerly Azure AD) rather than through local Windows group policy.

```
# Set an Azure AD admin for an Azure SQL logical server
az sql server ad-admin create \
  --resource-group prod-rg \
  --server prod-sql-server-01 \
  --display-name "DBA-Team-Admin" \
  --object-id "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx"
```

Inside the database, Entra-backed contained users are created without passwords:

```
-- Create an Azure AD-backed user in Azure SQL Database
CREATE USER [dba-engineer@company.com] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [dba-engineer@company.com];

-- For a managed identity (e.g., an Azure App Service)
CREATE USER [my-app-service] FROM EXTERNAL PROVIDER;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::dbo TO [my-app-service];
```

The Core Shift

What changes is not the concept of authentication and authorization — it is the locus of control. You now manage database access through the same IAM, RBAC, and identity systems that govern access to every other cloud resource. Permissions are auditable through CloudTrail (AWS), Azure Monitor Activity Log (Azure), and Cloud Audit Logs (GCP). Rotating credentials means updating IAM policies, not editing `pg_hba.conf` and running `pg_reload_conf()`. This is a genuine improvement in auditability and a reduction in credential sprawl, but it requires the DBA to become conversant in cloud IAM, which is a new skill domain.

4.3 Configuration Management: Parameter Groups, Server Parameters, and What You Cannot Touch

On self-managed PostgreSQL, you edit `postgresql.conf` directly and control every engine parameter from `max_connections` to `wal_level` to `autovacuum_vacuum_cost_delay`. On RDS, that file is owned by AWS. You configure the engine through Parameter Groups — a named collection of engine parameters that you apply to one or more DB instances.

RDS Parameter Groups

The default parameter group for any RDS engine version is managed by AWS and is not modifiable. You create a custom parameter group, copy defaults into it, modify what you need, and associate it with your instance. Changes to dynamic parameters take effect immediately; changes to static parameters require an instance reboot.

```
# Create a custom parameter group for RDS PostgreSQL 15
aws rds create-db-parameter-group \
  --db-parameter-group-name prod-pg15-custom \
  --db-parameter-group-family postgres15 \
  --description "Production PostgreSQL 15 tuning"

# Modify key parameters
aws rds modify-db-parameter-group \
```

```
--db-parameter-group-name prod-pg15-custom \  
--parameters \  
  "ParameterName=shared_buffers,ParameterValue={DBInstanceClassMemory/4},ApplyMethod=pending-reboot" \  
  "ParameterName=work_mem,ParameterValue=65536,ApplyMethod=immediate" \  
  "ParameterName=autovacuum_vacuum_scale_factor,ParameterValue=0.01,ApplyMethod=immediate" \  
  "ParameterName=log_min_duration_statement,ParameterValue=1000,ApplyMethod=immediate"  
  
# Associate with an instance  
aws rds modify-db-instance \  
  --db-instance-identifier prod-postgres-01 \  
  --db-parameter-group-name prod-pg15-custom \  
  --apply-immediately
```

Note the formula syntax for `shared_buffers`. RDS supports arithmetic expressions using `{DBInstanceClassMemory}` as a variable — this is how AWS recommends setting memory-relative parameters so the configuration remains valid if you resize the instance class. For a `db.r6g.2xlarge` with 64 GB RAM, `{DBInstanceClassMemory/4}` evaluates to 16 GB, which is 25% of RAM — the standard PostgreSQL recommendation.

What You Cannot Configure

This is where experienced DBAs encounter real friction. Several parameters are locked in RDS because they control behaviors that AWS manages on your behalf:

- `wal_level` is not directly settable on RDS PostgreSQL — it defaults to `replica` and only moves to `logical` when you enable the `rds.logical_replication` parameter (which requires a reboot); you cannot control it independently of that flag
- `archive_mode` is managed by AWS; you cannot disable it without understanding it breaks PITR
- `listen_addresses` is managed by AWS
- `ssl` is forced on — you cannot disable SSL on RDS connections
- `superuser` access does not exist on RDS; the `rds_superuser` role is a functional substitute but does not grant OS-level access
- Extensions must be whitelisted by AWS; you cannot install arbitrary shared libraries

The absence of a true superuser is a significant operational change. Tasks that previously required superuser — like terminating backend connections owned by other users, accessing `pg_authid`, or installing custom extensions — require either the `rds_superuser` role (which handles most cases) or a support ticket to AWS (which handles almost none of the rest gracefully). Plan your extension requirements before migration. If your application depends on an extension that is not in the RDS supported list, you may need AlloyDB on GCP, Aurora, or a self-managed EC2 deployment.

Azure SQL Server-Level vs. Database-Level Parameters

Azure SQL Database presents a different model. Many parameters that SQL Server DBAs manage at the server level — max server memory, MAXDOP, cost threshold for parallelism — are controlled through database-scoped configurations or are set by Microsoft at the service tier level.

```
-- Azure SQL: set database-scoped configuration  
ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 4;  
ALTER DATABASE SCOPED CONFIGURATION SET LEGACY_CARDINALITY_ESTIMATION = OFF;  
ALTER DATABASE SCOPED CONFIGURATION SET QUERY_OPTIMIZER_HOTFIXES = ON;  
  
-- Check current database-scoped configuration  
SELECT name, value, value_for_secondary  
FROM sys.database_scoped_configurations  
ORDER BY name;
```

Maximum memory is determined by the service tier and compute size — there is no `sp_configure 'max server memory'` you can run. Business Critical generally provides a higher memory-to-vCore ratio than General Purpose, since it runs on local SSD-backed hardware rather than remote storage, but the exact figures vary by hardware generation — check current Microsoft documentation rather than assuming a fixed number. What matters operationally is that these ceilings are fixed for the tier and not tunable.

4.4 High Availability and Failover: Understanding What the Cloud Gives You

HA in on-premises environments is expensive to build and maintain. A PostgreSQL streaming replication setup with Patroni, HAProxy, and etcd requires significant operational investment. SQL Server AlwaysOn Availability Groups require Windows Server Failover Clustering, shared storage or synchronous replication, and careful listener configuration. In the cloud, HA is a checkbox during instance creation, and the complexity of maintaining it disappears — but understanding what you got is essential because the behavior is not identical to what you built yourself.

RDS Multi-AZ: What It Is and Is Not

RDS Multi-AZ deploys a synchronous standby in a second Availability Zone. Writes are committed to both the primary and standby before returning to the application. Failover is automatic and DNS-based — the endpoint DNS name resolves to the standby after promotion, typically within 60–120 seconds including DNS TTL propagation. The standby is not readable. It is a hot standby only for failover purposes.

This surprises many DBAs coming from Patroni setups where the replica serves read traffic. On RDS Multi-AZ (non-Aurora), you pay for the standby instance in full and receive zero read scaling from it. Read scaling on RDS requires separate Read Replicas, which use asynchronous replication and carry replication lag that is your responsibility to monitor.

Aurora: A Different Architecture

Aurora changes this model substantially. Aurora separates compute from storage. All Aurora instances — primary and replicas — share the same distributed storage volume, which replicates data six ways across three Availability Zones. Aurora replicas apply redo log directly from the shared storage layer rather than receiving WAL over the network. This reduces replica lag dramatically — typically under 20 milliseconds in normal operation — and makes reads from Aurora replicas a reliable pattern for production read scaling.

Aurora also has a faster failover model. Because replicas already have access to the same storage volume as the primary, a failover involves promoting a replica to writer status rather than waiting for WAL replay to complete. Aurora typically fails over in under 30 seconds for replicas in the same Region, and some configurations achieve under 15 seconds with cluster-aware drivers.

```
# Create an Aurora PostgreSQL cluster with two replicas in different AZs
aws rds create-db-cluster \
  --db-cluster-identifier prod-aurora-cluster \
  --engine aurora-postgresql \
  --engine-version 15.4 \
  --master-username dbadmin \
  --master-user-password "$(aws secretsmanager get-secret-value \
```

```
--secret-id prod/aurora/master-password \
--query SecretString --output text)" \
--db-subnet-group-name prod-subnet-group \
--vpc-security-group-ids sg-0abc123def456 \
--storage-encrypted \
--kms-key-id alias/prod-rds-key \
--backup-retention-period 14 \
--preferred-backup-window "03:00-04:00" \
--preferred-maintenance-window "sun:04:00-sun:05:00"

# Add a writer instance
aws rds create-db-instance \
--db-cluster-identifier prod-aurora-cluster \
--db-instance-identifier prod-aurora-writer \
--db-instance-class db.r6g.2xlarge \
--engine aurora-postgresql

# Add a reader instance in a different AZ
aws rds create-db-instance \
--db-cluster-identifier prod-aurora-cluster \
--db-instance-identifier prod-aurora-reader-1 \
--db-instance-class db.r6g.xlarge \
--engine aurora-postgresql \
--availability-zone us-east-1b
```

Azure SQL Business Critical vs. General Purpose

Azure SQL HA behavior depends on the service tier. General Purpose uses remote storage (Azure Premium SSD) with a database availability SLA achieved through Azure infrastructure-level redundancy. Failover in the General Purpose tier can take 25–30 seconds as data files are remounted on a new compute node. Business Critical deploys an Always On availability group across three replicas using local SSD storage. One of those replicas is readable and accessible to your application — unlike RDS Multi-AZ standby, the Business Critical secondary serves read workloads without additional configuration.

The read-scale secondary in Business Critical is accessed via a connection string that includes `ApplicationIntent=ReadOnly`:

```
-- Connect string for Azure SQL Business Critical read-only replica
-- Server=prod-sql-server-01.database.windows.net;
-- Database=prod-db;
-- Authentication=Active Directory Default;
-- ApplicationIntent=ReadOnly;

-- Verify you are on the secondary
SELECT DATABASEPROPERTYEX(DB_NAME(), 'Updateability') AS replica_role;
-- Returns: READ_ONLY when connected to secondary
-- Returns: READ_WRITE when connected to primary
```

The HA tiers, failover behaviors, and read-scale capabilities across cloud providers have direct cost implications. Choosing between RDS Multi-AZ, Aurora, Azure SQL General Purpose, and Business Critical is as much an architectural decision as a financial one, and it belongs in the DBA's scope, not just the infrastructure team's.

4.5 Monitoring: CloudWatch, Azure Monitor, and Cloud Logging vs. What You Had Before

On-premises DBAs typically own their full monitoring stack — Prometheus with `pg_exporter`, Datadog agents, custom scripts querying `pg_stat_activity`, `pg_stat_bgwriter`, and `pg_locks`. In the cloud, the provider delivers a baseline monitoring layer automatically, and the DBA's job shifts from building that layer to interpreting it, extending it, and setting appropriate thresholds.

CloudWatch and Enhanced Monitoring on RDS

Every RDS instance publishes a set of CloudWatch metrics automatically with no configuration required: `CPUUtilization`, `DatabaseConnections`, `FreeableMemory`, `ReadIOPS`, `WriteIOPS`, `ReadLatency`, `WriteLatency`, `ReplicaLag`. These metrics refresh at 60-second intervals by default.

Enhanced Monitoring is an additional feature that installs an agent on the DB instance OS (which you cannot access directly) and publishes 50+ OS-level metrics at 1-second to 60-second granularity to CloudWatch Logs. These include per-process CPU, memory, disk I/O broken out by device, and file system metrics. Enable it for production instances — the cost is negligible compared to the diagnostic value during an incident.

```
# Enable Enhanced Monitoring with 15-second granularity
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --monitoring-interval 15 \
  --monitoring-role-arn arn:aws:iam::123456789012:role/rds-monitoring-role \
  --apply-immediately

# Query CloudWatch for replica lag over the last hour
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name ReplicaLag \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-reader-1 \
  --start-time $(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 60 \
  --statistics Maximum \
  --output table
```

Performance Insights

Performance Insights is the cloud analog to Oracle's Active Session History and a significant improvement over what most PostgreSQL on-premises deployments have out of the box. It records database load in terms of active sessions broken down by wait event, SQL statement, host, and user. The default retention is 7 days free; extending to up to 2 years incurs an additional per-vCPU-hour charge — check current AWS pricing.

For a production workload investigation, Performance Insights replaces the manual practice of sampling `pg_stat_activity` every few seconds and correlating the results. The UI presents a visual timeline of database load against the `max_vCPUs` line — anything above that line indicates the database is CPU-bound or waiting on a bottleneck that requires investigation.

```
# Retrieve Performance Insights data via boto3
import boto3
from datetime import datetime, timedelta, timezone

pi_client = boto3.client('pi', region_name='us-east-1')

response = pi_client.get_resource_metrics(
    ServiceType='RDS',
    Identifier='db-ABCDEFGHIJ1234567', # DBI resource ID, not instance ID
```



```
MetricQueries=[
    {
        'Metric': 'db.load.avg',
        'GroupBy': {
            'Group': 'db.wait_event',
            'Limit': 10
        }
    }
],
StartTime=datetime.now(timezone.utc) - timedelta(hours=1),
EndTime=datetime.now(timezone.utc),
PeriodInSeconds=60
)

for result in response['MetricList']:
    print(f"Metric: {result['Key']}")
    for dp in result['DataPoints'][-5:]: # last 5 data points
        print(f"    {dp['Timestamp']}: {dp.get('Value', 'N/A'):.4f}")
```

Azure Monitor and Query Performance Insight

Azure SQL Database surfaces equivalent functionality through Azure Monitor metrics (DTU percentage, CPU percentage, data I/O percentage, log write percentage, connection counts) and the Query Performance Insight blade, which aggregates data from Query Store — a feature that has been running automatically in Azure SQL since it was introduced in SQL Server 2016.

Query Store is particularly valuable because it captures query plan changes over time. A query that performed well for three months and suddenly regressed after an auto-statistics update will show the plan change in Query Store with the before and after execution statistics.

```
-- Azure SQL: find the top 10 queries by total CPU consumption in the last 24 hours
SELECT TOP 10
    q.query_id,
    qt.query_sql_text,
    SUM(rs.avg_cpu_time * rs.count_executions) AS total_cpu_ms,
    SUM(rs.count_executions) AS total_executions,
    SUM(rs.avg_duration * rs.count_executions) AS total_duration_ms
FROM sys.query_store_query_text qt
JOIN sys.query_store_query q ON qt.query_text_id = q.query_text_id
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
JOIN sys.query_store_runtime_stats_interval rsi
    ON rs.runtime_stats_interval_id = rsi.runtime_stats_interval_id
WHERE rsi.start_time >= DATEADD(HOUR, -24, GETUTCDATE())
GROUP BY q.query_id, qt.query_sql_text
ORDER BY total_cpu_ms DESC;
```

The monitoring shift is real: you spend less time building monitoring infrastructure and more time interpreting richer, already-collected data. The tradeoff is that cloud monitoring tools have gaps — they do not expose lock wait graphs as clearly as `pg_locks` queries do on self-managed PostgreSQL, and correlating application traces with database wait events across CloudWatch, X-Ray, and RDS Performance Insights requires more configuration than running a single monitoring agent that sees both layers.

4.6 The New DBA Work: Cost Management, IaC, and Architecture Ownership

The reduction in operational work does not leave a vacuum. Cloud DBAs acquire a meaningful set of responsibilities that did not exist in their on-premises roles: database infrastructure as code, cost engineering, and cross-service architecture design.

Infrastructure as Code for Databases

In on-premises environments, provisioning a PostgreSQL server involved coordinating with a server team, a storage team, and a network team, filling out a change request, and waiting. In the cloud, a DBA with appropriate IAM permissions can provision a multi-AZ Aurora cluster with encrypted storage, automated backups, and VPC-isolated networking in under 10 minutes using Terraform. The DBA who understands both the database requirements and the infrastructure configuration is in a stronger position than one who waits for an infrastructure team to provision resources.

```
# Terraform: production Aurora PostgreSQL cluster
resource "aws_rds_cluster" "prod" {
  cluster_identifier      = "prod-aurora-pg"
  engine                 = "aurora-postgresql"
  engine_version         = "15.4"
  availability_zones      = ["us-east-1a", "us-east-1b", "us-east-1c"]
  database_name          = "appdb"
  master_username        = "dbadmin"
  manage_master_user_password = true # Secrets Manager integration

  vpc_security_group_ids = [aws_security_group.rds.id]
  db_subnet_group_name   = aws_db_subnet_group.prod.name

  storage_encrypted      = true
  kms_key_id             = aws_kms_key.rds.arn

  backup_retention_period = 14
  preferred_backup_window = "03:00-04:00"
  deletion_protection    = true

  enabled_cloudwatch_logs_exports = ["postgresql"]

  db_cluster_parameter_group_name = aws_rds_cluster_parameter_group.prod.name

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
    Owner       = "database-team"
  }
}

resource "aws_rds_cluster_instance" "writer" {
  identifier      = "prod-aurora-pg-writer"
  cluster_identifier = aws_rds_cluster.prod.id
  instance_class   = "db.r6g.2xlarge"
  engine           = aws_rds_cluster.prod.engine
  engine_version   = aws_rds_cluster.prod.engine_version

  performance_insights_enabled      = true
  performance_insights_retention_period = 731 # 2 years
  monitoring_interval               = 15
  monitoring_role_arn               = aws_iam_role.rds_monitoring.arn

  db_parameter_group_name = aws_db_parameter_group.prod.name

  auto_minor_version_upgrade = false # control upgrades explicitly in production
}

resource "aws_rds_cluster_instance" "reader" {
```

```
count            = 2
identifier        = "prod-aurora-pg-reader-${count.index + 1}"
cluster_identifier = aws_rds_cluster.prod.id
instance_class    = "db.r6g.xlarge"
engine            = aws_rds_cluster.prod.engine
engine_version    = aws_rds_cluster.prod.engine_version

performance_insights_enabled = true
performance_insights_retention_period = 731
monitoring_interval          = 15
monitoring_role_arn          = aws_iam_role.rds_monitoring.arn

db_parameter_group_name = aws_db_parameter_group.prod.name
}

resource "aws_rds_cluster_parameter_group" "prod" {
  name     = "prod-aurora-pg15"
  family   = "aurora-postgresql15"

  parameter {
    name      = "shared_preload_libraries"
    value     = "pg_stat_statements,auto_explain"
    apply_method = "pending-reboot"
  }

  parameter {
    name      = "log_min_duration_statement"
    value     = "1000"
    apply_method = "immediate"
  }

  parameter {
    name      = "autovacuum_vacuum_scale_factor"
    value     = "0.01"
    apply_method = "immediate"
  }

  parameter {
    name      = "autovacuum_analyze_scale_factor"
    value     = "0.005"
    apply_method = "immediate"
  }
}
```

Treating database infrastructure as code gives you version history, peer review, repeatable deployments across environments, and the ability to reconstruct a destroyed environment from source control. These are practices that are standard in software engineering and are now expected of cloud DBAs. If your team does not already do this, starting with a simple Terraform module for a non-production cluster is the practical entry point.

Cost Engineering

On-premises database costs are largely fixed — server hardware depreciates, storage is bought in bulk, and the marginal cost of running a query is invisible. In the cloud, every instance-hour, every GB of storage, every I/O operation (on gp2 storage), and every GB of data transferred across AZs has a line-item cost. A cloud DBA who understands the cost model can make architecture recommendations that have direct financial impact.

The most common cost levers for RDS and Aurora are:

Instance sizing and Reserved Instances. On-demand pricing for a `db.r6g.2xlarge` running Aurora PostgreSQL in us-east-1 is meaningful at monthly scale. Purchasing a one-year no-upfront Reserved Instance for the same size produces a substantial discount compared to on-demand — consult the

current AWS pricing calculator for exact figures, as prices change. Most production workloads with stable compute requirements should run on Reserved Instances. A common failure pattern is running development and staging instances on on-demand because they were provisioned ad hoc, then leaving them running indefinitely.

Aurora I/O-Optimized vs. Standard. Aurora standard pricing includes a per-million I/O charge. Aurora I/O-Optimized pricing eliminates the I/O charge at a higher fixed storage rate. For workloads where I/O cost exceeds a certain percentage of total Aurora cost (check current AWS guidelines for the threshold), I/O-Optimized is the less expensive option. This decision requires pulling the actual I/O metrics from CloudWatch and calculating both pricing models against your current usage.

Storage autoscaling. RDS and Aurora both support storage autoscaling — storage grows automatically when free space falls below a threshold. This prevents the 2 AM call about a full disk but can mask unexpected data growth. Set a `MaxAllocatedStorage` limit and alert on storage growth rate, not just absolute free space.

Read Replica sizing. Read Replicas do not need to match the writer's instance class. If your read queries are primarily reporting workloads with high memory requirements for sort and hash operations, sizing replicas differently from the writer may make sense. If the replica is primarily serving simple key-value lookups, a smaller instance class may be sufficient.

```
# List all running RDS instances and their costs-relevant attributes
aws rds describe-db-instances \
  --query 'DBInstances[*].{
    ID:DBInstanceIdentifier,
    Class:DBInstanceClass,
    Engine:Engine,
    EngineVersion:EngineVersion,
    MultiAZ:MultiAZ,
    Storage:AllocatedStorage,
    StorageType:StorageType,
    Status:DBInstanceStatus
  }' \
  --output table

# Identify instances without Reserved Instance coverage using Cost Explorer API
aws ce get-reservation-coverage \
  --time-period Start=2024-01-01,End=2024-01-31 \
  --granularity MONTHLY \
  --filter '{
    "Dimensions": {
      "Key": "SERVICE",
      "Values": ["Amazon Relational Database Service"]
    }
  }' \
  --query 'CoveragesByTime[*].Groups[*].{
    Instance:Attributes.instanceType,
    CoverageHours:Coverage.CoverageHours.CoverageHoursPercentage
  }' \
  --output table
```

Architecture Ownership

The cloud DBA's most expansive new responsibility is architecture ownership — the ability to design and advocate for database solutions that span multiple cloud services rather than a single database engine. An on-premises DBA might optimize a single PostgreSQL cluster indefinitely. A cloud DBA is frequently asked whether data should live in RDS, Aurora, DynamoDB, Redshift, or S3-plus-Athena, and the answer requires understanding access patterns, query shapes, consistency requirements, and cost at scale.

This does not mean the cloud DBA becomes a generalist who knows everything shallowly. It means the DBA's deep expertise in relational database behavior is the anchor for evaluating other options. Understanding why a particular workload is poorly suited for a key-value store, or why an analytical query pattern belongs in a columnar store rather than a row store, is exactly the kind of judgment that deep database expertise develops.

4.7 What Genuinely Does Not Change

It is worth being direct about this, because hype around cloud services can create the impression that managed databases are fundamentally different systems that require entirely new skills.

Query optimization does not change. EXPLAIN and EXPLAIN ANALYZE on RDS PostgreSQL produce the same output as on self-managed PostgreSQL. The same principles govern index selection, join ordering, statistics quality, and plan regression. Query Store on Azure SQL and Performance Insights on RDS give you better visibility into query behavior than most on-premises deployments had, but interpreting that data requires the same foundational knowledge of execution plans and cost estimation.

```
-- PostgreSQL: execution plan analysis works identically on RDS and self-managed
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
    o.order_id,
    o.created_at,
    c.email,
    SUM(oi.quantity * oi.unit_price) AS order_total
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.created_at >= NOW() - INTERVAL '30 days'
    AND o.status = 'completed'
GROUP BY o.order_id, o.created_at, c.email
ORDER BY o.created_at DESC;

-- SQL Server: same analysis approach on Azure SQL
SET STATISTICS IO, TIME ON;
SELECT
    o.order_id,
    o.created_at,
    c.email,
    SUM(oi.quantity * oi.unit_price) AS order_total
FROM dbo.orders o
JOIN dbo.customers c ON o.customer_id = c.customer_id
JOIN dbo.order_items oi ON o.order_id = oi.order_id
WHERE o.created_at >= DATEADD(DAY, -30, GETUTCDATE())
    AND o.status = 'completed'
GROUP BY o.order_id, o.created_at, c.email
ORDER BY o.created_at DESC;
```

Schema design does not change. Normalization, denormalization tradeoffs, partitioning strategies, and foreign key design are engine-level concerns that the cloud provider does not touch. A poorly designed schema that causes table bloat through inefficient updates will bloat just as effectively on Aurora as on a bare-metal PostgreSQL server.

Backup and recovery reasoning does not change. The mechanics of backup change — you do not run `pg_basebackup` manually on RDS, and you do not schedule SQL Server maintenance plan jobs on Azure SQL. But the reasoning about RPO, RTO, backup retention, and recovery testing does not change. The cloud DBA must still define backup retention windows, understand

what PITR can recover and what it cannot, and test restore procedures regularly.

Transaction management and isolation levels do not change. READ COMMITTED, REPEATABLE READ, SERIALIZABLE — these are engine behaviors that the cloud wrapper does not modify. Application developers who write code with implicit assumptions about transaction boundaries will cause the same data integrity problems on RDS that they would cause on your on-premises server.

Locking and concurrency design does not change. Advisory locks, row-level locking, table-level locking, deadlock detection — these behave identically. The cloud DBA who knows how to diagnose a lock contention issue using `pg_locks` and `pg_stat_activity` on self-managed PostgreSQL will apply exactly the same skills on RDS.

```
-- PostgreSQL: identify lock contention (works on RDS, Aurora, Cloud SQL identically)
SELECT
    blocked.pid AS blocked_pid,
    blocked.query AS blocked_query,
    blocking.pid AS blocking_pid,
    blocking.query AS blocking_query,
    blocked_activity.wait_event_type,
    blocked_activity.wait_event,
    now() - blocked_activity.query_start AS blocked_duration
FROM pg_catalog.pg_locks blocked_lock
JOIN pg_catalog.pg_stat_activity blocked_activity
    ON blocked_activity.pid = blocked_lock.pid
JOIN pg_catalog.pg_locks blocking_lock
    ON blocking_lock.locktype = blocked_lock.locktype
    AND blocking_lock.DATABASE IS NOT DISTINCT FROM blocked_lock.DATABASE
    AND blocking_lock.relation IS NOT DISTINCT FROM blocked_lock.relation
    AND blocking_lock.page IS NOT DISTINCT FROM blocked_lock.page
    AND blocking_lock.tuple IS NOT DISTINCT FROM blocked_lock.tuple
    AND blocking_lock.virtualxid IS NOT DISTINCT FROM blocked_lock.virtualxid
    AND blocking_lock.transactionid IS NOT DISTINCT FROM blocked_lock.transactionid
    AND blocking_lock.classid IS NOT DISTINCT FROM blocked_lock.classid
    AND blocking_lock.objid IS NOT DISTINCT FROM blocked_lock.objid
    AND blocking_lock.objsubid IS NOT DISTINCT FROM blocked_lock.objsubid
    AND blocking_lock.pid != blocked_lock.pid
JOIN pg_catalog.pg_stat_activity blocking_activity
    ON blocking_activity.pid = blocking_lock.pid
WHERE NOT blocked_lock.granted
ORDER BY blocked_duration DESC;

-- SQL Server equivalent (Azure SQL)
SELECT
    blocked_req.session_id AS blocked_session,
    blocked_req.wait_type,
    blocked_req.wait_time / 1000.0 AS wait_seconds,
    blocked_req.blocking_session_id AS blocking_session,
    blocked_sql.text AS blocked_query,
    blocking_sql.text AS blocking_query,
    blocked_req.status
FROM sys.dm_exec_requests blocked_req
CROSS APPLY sys.dm_exec_sql_text(blocked_req.sql_handle) blocked_sql
JOIN sys.dm_exec_sessions blocking_sess
    ON blocking_sess.session_id = blocked_req.blocking_session_id
CROSS APPLY sys.dm_exec_sql_text(blocking_sess.most_recent_sql_handle) blocking_sql
WHERE blocked_req.blocking_session_id > 0
ORDER BY blocked_req.wait_time DESC;
```

The cloud changes where your database runs and who manages the infrastructure around it. It does not change how relational databases work or what makes them perform well. The DBA who has mastered the fundamentals carries those skills directly into the cloud environment and adds cloud-specific operational knowledge on top of that foundation.

4.8 Organizational Changes: The DBA in Cloud-Native Teams

Beyond the technical changes, the cloud DBA role shifts in organizational context. In many organizations that move to cloud-managed databases, the traditional operations-focused DBA headcount decreases while the need for database engineers embedded in development teams or platform teams increases.

The distinction matters. An operations DBA monitors a fleet of databases, responds to incidents, performs maintenance, and keeps the lights on. A database engineer (the role that cloud-managed services tend to create demand for) contributes to architectural decisions, reviews schema changes, automates database provisioning and migrations, builds the observability layer, and advises development teams on data model and query design choices. If you are currently an operations DBA in an organization moving to managed cloud databases, understanding this shift is important for positioning your career.

The practical advice is to develop skills in three areas that complement your existing database expertise and increase your value in a cloud context:

1. **Infrastructure as Code fluency.** Terraform or Pulumi for database infrastructure is a baseline expectation on mature cloud teams. If you can write and review Terraform for an Aurora cluster, security groups, parameter groups, and Secrets Manager integration, you can contribute directly to platform work rather than waiting for infrastructure teams to provision what you need.
2. **Observability and alerting design.** Building CloudWatch alarms, Azure Monitor alert rules, and dashboard configurations that accurately surface real problems without generating noise is a skill. Knowing which thresholds matter (replica lag exceeding a threshold appropriate for your RPO, connection pool saturation, storage growth rate), and knowing how to page on the right signals, is a DBA-specific contribution to the observability stack.
3. **Database migration engineering.** Moving data between environments — from on-premises to cloud, between cloud providers, or between database engines — is a domain where deep database knowledge is essential and where mistakes have serious consequences. AWS Database Migration Service, Azure Database Migration Service, and tools like `pglogical` and `pg_dump` in combination are all in scope. Chapter 9 covers this in depth.

Key Takeaways

- The cloud provider owns the OS, storage subsystem, replication wiring, and patching lifecycle; the DBA owns configuration, schema design, query performance, access control policy, and backup retention — this division of responsibility is firm and understanding it prevents both misplaced effort and dangerous assumptions.
- Authentication and authorization move from engine-native mechanisms (`pg_hba.conf`, SQL Server logins) to cloud IAM layers (AWS IAM database authentication, Azure Entra ID

contained users), requiring the DBA to become conversant in cloud identity systems while retaining all existing knowledge of database-level privilege management.

- Configuration management shifts from editing flat files to managing Parameter Groups (RDS) or database-scoped configurations (Azure SQL), with certain parameters locked because the provider manages them on your behalf — most notably, true superuser access does not exist on RDS, which affects extension management and some administrative operations.
- Core DBA expertise — query optimization, execution plan analysis, schema design, locking and concurrency management, transaction isolation reasoning, and backup/recovery strategy — transfers without modification to managed cloud databases; cloud wrappers change where the database runs, not how relational engines work.
- Cloud DBAs acquire meaningful net-new responsibilities in infrastructure as code, cost engineering (Reserved Instances, storage tier selection, I/O optimization), and cross-service architecture design, while the organizational context shifts toward database engineering roles embedded in development and platform teams rather than purely operations-focused roles.

Chapter 5: Cloud Database Architecture Patterns

Cloud databases don’t just move your workload to someone else’s hardware — they introduce architectural patterns that simply don’t exist in on-premises environments. Multi-AZ failover that completes automatically within minutes, read replicas that spin up in minutes, serverless capacity that scales from zero to 128 ACUs without a DBA touching a single configuration file: these capabilities change how you design systems from the ground up. This chapter covers the core architectural patterns you’ll implement on AWS RDS, Azure SQL, and Google Cloud SQL, maps them to what you already know from on-premises work, and shows you exactly where managed services expand your options and where they constrain them.

5.1 High Availability and Failover Architecture

From Manual Standbys to Managed HA

On premises, you built HA with Pacemaker/Corosync clusters, SQL Server Always On Availability Groups with manual witness configuration, or PostgreSQL streaming replication with Patroni. You managed VIPs, fencing, and split-brain scenarios yourself. In the cloud, the managed HA layer abstracts most of this — but understanding what’s happening underneath matters when things go wrong at 2 AM.

AWS RDS Multi-AZ deploys a synchronous standby replica in a separate Availability Zone. The standby is not a readable replica — it exists solely for failover. RDS uses a DNS endpoint; when failover occurs, the CNAME flips from primary to standby. Client applications must handle the DNS TTL-based reconnect window — AWS states standard RDS Multi-AZ failover typically completes within 60–120 seconds. The standby is on different physical hardware, a different power domain, and in some cases a different facility within the same AWS region. Your application connection string doesn’t change; only the IP behind the DNS name does.

Aurora goes further. Aurora Multi-AZ isn’t traditional synchronous replication — the storage layer itself is shared across six copies in three AZs. The “replica” Aurora creates is a hot standby that already has the data; failover is a matter of promoting the reader endpoint to writer, not copying data. Aurora failover typically completes in under 30 seconds with no data loss because Aurora replicas share the storage layer with the primary. This is meaningfully different from RDS Multi-AZ, which is block-level replication of the EBS volume.

Azure SQL offers multiple HA tiers. The General Purpose tier uses remote storage with a hot

standby; failover takes 20–30 seconds. Business Critical tier maintains three synchronous replicas locally (similar in concept to Always On AG with three synchronous partners) and typically fails over in well under 30 seconds. Hyperscale uses a page server architecture with compute and storage fully separated, similar in philosophy to Aurora, and supports rapid replica addition.

Cloud SQL (GCP) uses regional instances with a standby in a second zone within the same region. Failover is automatic and typically completes in 60–120 seconds — meaningfully slower than Aurora or Azure Business Critical. AlloyDB for PostgreSQL, Google’s premium offering, uses a Paxos-based distributed storage layer similar to Aurora, with failover under 60 seconds and readable replicas that share the underlying storage.

Configuring Multi-AZ on RDS

```
# Create a Multi-AZ RDS PostgreSQL instance
aws rds create-db-instance \
  --db-instance-identifier prod-postgres-primary \
  --db-instance-class db.r6g.4xlarge \
  --engine postgres \
  --engine-version 15.4 \
  --master-username dbadmin \
  --master-user-password "${DB_PASSWORD}" \
  --allocated-storage 500 \
  --storage-type gp3 \
  --iops 12000 \
  --storage-throughput 500 \
  --multi-az \
  --vpc-security-group-ids sg-0a1b2c3d4e5f \
  --db-subnet-group-name prod-db-subnet-group \
  --backup-retention-period 7 \
  --preferred-backup-window "03:00-04:00" \
  --preferred-maintenance-window "sun:05:00-sun:06:00" \
  --deletion-protection \
  --enable-performance-insights \
  --performance-insights-retention-period 7 \
  --no-publicly-accessible

# Verify Multi-AZ status and get endpoint
aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-primary \
  --query
  ↪ 'DBInstances[0].{MultiAZ:MultiAZ,Endpoint:Endpoint.Address,SecondaryAZ:SecondaryAvailabilityZone,Status:DBInstancesS
```

Configuring Aurora Cluster HA with Terraform

```
# Aurora PostgreSQL cluster with Multi-AZ writer and readers
resource "aws_rds_cluster" "prod_aurora" {
  cluster_identifier = "prod-aurora-postgres"
  engine             = "aurora-postgresql"
  engine_version     = "15.4"
  database_name      = "proddb"
  master_username    = "dbadmin"
  master_password    = var.db_password

  # Storage is automatically distributed across 3 AZs
  availability_zones = ["us-east-1a", "us-east-1b", "us-east-1c"]

  backup_retention_period = 7
  preferred_backup_window = "03:00-04:00"
  preferred_maintenance_window = "sun:05:00-sun:06:00"

  vpc_security_group_ids = [aws_security_group.aurora_sg.id]
  db_subnet_group_name   = aws_db_subnet_group.prod.name

  deletion_protection = true
  skip_final_snapshot = false
}
```

```
final_snapshot_identifier = "prod-aurora-final-${formatdate("YYYY-MM-DD", timestamp())}"

enabled_cloudwatch_logs_exports = ["postgresql"]

db_cluster_parameter_group_name = aws_rds_cluster_parameter_group.prod.name

tags = {
    Environment = "production"
    ManagedBy   = "terraform"
}
}

# Writer instance - placed in us-east-1a
resource "aws_rds_cluster_instance" "writer" {
    identifier      = "prod-aurora-writer"
    cluster_identifier = aws_rds_cluster.prod_aurora.id
    instance_class  = "db.r6g.4xlarge"
    engine          = aws_rds_cluster.prod_aurora.engine
    engine_version  = aws_rds_cluster.prod_aurora.engine_version

    availability_zone      = "us-east-1a"
    performance_insights_enabled = true
    performance_insights_retention_period = 7

    auto_minor_version_upgrade = false
}

# Reader instances - one each in 1b and 1c for AZ redundancy
resource "aws_rds_cluster_instance" "readers" {
    count          = 2
    identifier      = "prod-aurora-reader-${count.index + 1}"
    cluster_identifier = aws_rds_cluster.prod_aurora.id
    instance_class  = "db.r6g.2xlarge"
    engine          = aws_rds_cluster.prod_aurora.engine
    engine_version  = aws_rds_cluster.prod_aurora.engine_version

    availability_zone = count.index == 0 ? "us-east-1b" : "us-east-1c"

    performance_insights_enabled = true
    performance_insights_retention_period = 7
}
```

What You Can't Control

On a self-managed PostgreSQL cluster, you control the exact failover timing: `synchronous_commit`, `synchronous_standby_names`, and the Patroni TTL settings. In RDS Multi-AZ, you don't choose the replication protocol — it's block-level synchronous replication of EBS volumes managed by AWS. You can't query the replication lag on the standby because it isn't a readable PostgreSQL replica; it's a shadow volume. You can't apply patches independently to the standby before promoting it. You accept these constraints in exchange for not managing the HA stack yourself.

The one genuine gotcha: **RDS Multi-AZ failover requires DNS TTL propagation**. If your application has hardcoded IP addresses or DNS caches with long TTLs, failover won't be transparent. Always configure your JDBC, psycopg2, or .NET connection strings to use the RDS DNS endpoint and set `connect_timeout` values that allow for reconnection. In Java, set `socketTimeout=30` and implement retry logic — don't assume the managed service makes reconnection unnecessary.

5.2 Read Scaling and Replica Patterns

Shifting the Mental Model

On premises, adding a read replica meant provisioning a server, configuring streaming replication, managing replication slots, and worrying about `max_replication_slots` and `max_wal_senders`. In the cloud, replicas are a billing line item — you add them with a CLI call. The architectural question shifts from “can we afford the operational overhead?” to “how do we route traffic, manage replica lag, and avoid read-your-own-writes consistency bugs?”

RDS Read Replicas are asynchronous replicas using native PostgreSQL streaming replication or SQL Server log shipping. They’re readable, each with their own endpoint, and can be promoted to standalone instances. You can have up to five read replicas per RDS instance. Replication lag is the same concern you had on-premises: under normal conditions you’ll see sub-second lag, but long-running transactions on the primary that generate heavy WAL, or large bulk operations, can push lag to minutes. Monitor `ReplicaLag` in CloudWatch and alert at 30 seconds; investigate at 60 seconds.

Aurora Replicas work differently. Because all Aurora instances share the same storage layer, a reader instance doesn’t need to ship or apply WAL — it reads the same storage pages the writer uses, with a lag that reflects only page cache propagation. Aurora replica lag is typically under 20ms for reads. You can have up to 15 Aurora replicas. This is a genuinely different operational experience from PostgreSQL streaming replication, and it means Aurora replicas are appropriate for latency-sensitive reporting workloads that RDS read replicas are not.

Azure SQL Hyperscale supports named replicas (up to 30) that also share the underlying page server storage. This is Microsoft’s architectural answer to Aurora’s storage disaggregation. The General Purpose and Business Critical tiers support geo-replicas for disaster recovery, but they have higher replication lag than Hyperscale named replicas.

RDS Proxy for Connection Pooling

One on-premises pattern that doesn’t translate cleanly to RDS is PgBouncer or pgpool-II sitting in front of your replica pool. RDS Proxy is the managed alternative. It maintains a connection pool to your RDS or Aurora instances and exposes a single endpoint to applications. When you have Lambda functions or containerized microservices opening thousands of short-lived connections — a workload that would exhaust `max_connections` on PostgreSQL — RDS Proxy pins connections to backend instances and handles multiplexing.

```
# Create RDS Proxy in front of Aurora cluster
aws rds create-db-proxy \
  --db-proxy-name prod-aurora-proxy \
  --engine-family POSTGRESQL \
  --auth '[
    {
      "AuthScheme": "SECRETS",
      "SecretArn": "arn:aws:secretsmanager:us-east-1:123456789:secret:prod/aurora/dbadmin",
      "IAMAuth": "REQUIRED"
    }
  ]' \
  --role-arn arn:aws:iam::123456789:role/rds-proxy-role \
  --vpc-subnet-ids subnet-0a1b2c3d subnet-0e5f6g7h subnet-0i9j0k1l \
  --vpc-security-group-ids sg-0proxy1234 \
  --require-tls

# Register the Aurora cluster as the proxy target
```

```
aws rds register-db-proxy-targets \
  --db-proxy-name prod-aurora-proxy \
  --db-cluster-identifiers prod-aurora-postgres

# Create a separate proxy endpoint for read replicas
aws rds create-db-proxy-endpoint \
  --db-proxy-name prod-aurora-proxy \
  --db-proxy-endpoint-name prod-aurora-proxy-reader \
  --vpc-subnet-ids subnet-0a1b2c3d subnet-0e5f6g7h subnet-0i9j0k1l \
  --vpc-security-group-ids sg-0proxy1234 \
  --target-role READ_ONLY
```

RDS Proxy has a real limitation that catches DBAs by surprise: it doesn't support all PostgreSQL client features. COPY commands, LISTEN/NOTIFY, `pg_terminate_backend()`, and SET LOCAL within certain transaction modes behave unexpectedly through the proxy. If your application relies on advisory locks or NOTIFY-based event queuing, test against the proxy endpoint explicitly before deploying to production. The proxy also pins connections (holds a dedicated backend connection for a client) when it detects certain session-state changes, which reduces the multiplexing benefit. Keep transactions short and avoid session-level SET commands that force connection pinning.

Aurora Global Database for Cross-Region Reads

For applications serving multiple geographic regions, Aurora Global Database adds cross-region replication with typical lag under one second. This is different from RDS cross-region read replicas, which use the native PostgreSQL replication stream and typically show 2–10 seconds of lag depending on bandwidth.

```
# boto3: describe Aurora Global Cluster replication lag
import boto3

rds = boto3.client('rds', region_name='us-east-1')

response = rds.describe_global_clusters(
    GlobalClusterIdentifier='prod-global-cluster'
)

for cluster in response['GlobalClusters']:
    print(f"Global Cluster: {cluster['GlobalClusterIdentifier']}")
    print(f"Status: {cluster['Status']}")
    print(f"Engine: {cluster['EngineVersion']}")

    for member in cluster['GlobalClusterMembers']:
        print(f"  Member: {member['DBClusterArn']}")
        print(f"  Is Writer: {member['IsWriter']}")

    if not member['IsWriter']:
        # Fetch replication lag from the secondary region's CloudWatch
        secondary_region = member['DBClusterArn'].split(':')[3]
        cw = boto3.client('cloudwatch', region_name=secondary_region)

        lag_response = cw.get_metric_statistics(
            Namespace='AWS/RDS',
            MetricName='AuroraGlobalDBReplicationLag',
            Dimensions=[{'Name': 'DBClusterIdentifier',
                          'Value': member['DBClusterArn'].split(':')[1]}],
            StartTime='2024-01-15T00:00:00Z',
            EndTime='2024-01-15T01:00:00Z',
            Period=60,
            Statistics=['Average', 'Maximum']
        )

        for point in lag_response['Datapoints'][-5:]:
            print(f"  Lag: avg={point['Average']:.1f}ms max={point['Maximum']:.1f}ms")
```

5.3 Storage Architecture and Auto-Scaling

Cloud Storage Is Not a SAN

On premises, your DBA team provisioned LUNs from a SAN, set RAID levels, tuned stripe sizes, and called the storage team when you needed more space — a process that took days. In AWS RDS, storage is gp3 or io2 backed by networked EBS volumes. The performance characteristics are fundamentally different from local NVMe, and understanding those differences prevents over-provisioning and unexpected I/O bottlenecks.

RDS gp3 provides a baseline of 3,000 IOPS and 125 MB/s throughput for any volume, regardless of size. Unlike gp2 (which was 3 IOPS/GB with a burst bucket), gp3 decouples IOPS from capacity. You can have a 100GB volume with 12,000 IOPS if you need it. Maximum gp3 performance is 16,000 IOPS and 1,000 MB/s. For PostgreSQL workloads with heavy random I/O — OLTP with small row updates, index-heavy queries — gp3 at 12,000–16,000 IOPS covers the majority of production workloads on instances up to db.r6g.8xlarge.

RDS io2 is the choice for I/O-intensive OLTP — think 50,000–256,000 IOPS, 4,000 MB/s throughput. At these levels you’re typically also running db.r6g.16xlarge or db.r6i.32xlarge, where the network bandwidth to the EBS volume (25–50 Gbps) becomes the binding constraint, not the storage tier itself.

Aurora storage auto-scales in 10GB increments up to 128TB and you pay only for what you use. There is no pre-provisioning step; Aurora handles storage growth automatically. However, Aurora storage doesn’t shrink automatically — once expanded to 50TB, the billing floor is 50TB even if you delete data. Run `VACUUM` and monitor `aurora_storage_utilization` to understand actual versus allocated space.

Cloud SQL storage auto-increases by default: when free space runs low, Cloud SQL automatically adds capacity in an incremental step, up to the configured maximum (check current GCP documentation for the exact trigger threshold and increment size, as these have changed over time). You can disable this and manage storage manually, but for production workloads the auto-increase behavior is generally preferable.

```
-- Query Aurora storage utilization (run from psql connected to Aurora)
SELECT
    current_setting('server_version') AS pg_version,
    pg_size_pretty(sum(pg_database_size(datname))) AS total_logical_size,
    -- Aurora-specific storage stats via aurora_stat_utils extension
    pg_size_pretty(
        (SELECT sum(allocated_storage)
         FROM aurora_storage_object_stats()
         WHERE object_type = 'data')
    ) AS aurora_allocated_storage;

-- Monitor bloat that inflates Aurora storage costs
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS total_size,
    pg_size_pretty(
        pg_total_relation_size(schemaname||'.'||tablename) -
        pg_relation_size(schemaname||'.'||tablename)
    ) AS bloat_estimate,
    n_dead_tup,
```

```

last_autovacuum
FROM pg_stat_user_tables
WHERE n_dead_tup > 100000
ORDER BY n_dead_tup DESC
LIMIT 20;

```

Storage Scaling Gotchas

RDS storage scaling is **one-directional and time-bounded**. You can increase allocated storage, but you cannot decrease it. Once you scale from 500GB to 1TB, your minimum billing is 1TB from that point on. More critically, during an RDS storage scale operation on Multi-AZ instances, the process scales the standby first, then performs a failover, then scales the former primary (now standby). The entire operation can take 30–60 minutes for large volumes, during which a maintenance window effectively exists for any workload that can't tolerate the failover window.

For Azure SQL, storage scales online within the same service tier with no downtime for Business Critical and General Purpose. The Hyperscale tier is unique: Hyperscale storage auto-scales to 100TB with no size pre-configuration required, and this is genuinely transparent to the application. The Hyperscale architecture separates log service, page servers, and compute nodes so that storage scaling doesn't touch the compute layer at all.

5.4 Network Architecture: VPC, Private Link, and Connectivity

The `pg_hba.conf` Replacement

On premises, `pg_hba.conf` controlled which hosts could connect to PostgreSQL and with what authentication method. In the cloud, that control is split across multiple layers, and understanding which layer does what determines whether you have a security gap.

VPC Security Groups (AWS) act as stateful host-based firewalls at the instance level. You configure inbound rules that specify source CIDR ranges, specific security group IDs, or individual IPs. For a production RDS instance, you should never open port 5432 to 0.0.0.0/0. The pattern is:

1. RDS instance lives in a **private subnet** with no internet gateway route
2. Application servers have a security group (e.g., `sg-app-tier`)
3. RDS security group allows inbound 5432 only from `sg-app-tier`
4. Any DBA access uses a bastion host, VPN, or AWS Systems Manager Session Manager tunneling

```

# Create an RDS security group that allows only from the app tier security group
aws ec2 create-security-group \
  --group-name prod-rds-sg \
  --description "Production RDS - allow from app tier only" \
  --vpc-id vpc-0a1b2c3d4e5f6789

export RDS_SG_ID=$(aws ec2 describe-security-groups \
  --filters Name=group-name,Values=prod-rds-sg \
  --query 'SecurityGroups[0].GroupId' \
  --output text)

# Allow PostgreSQL port from app security group only
aws ec2 authorize-security-group-ingress \
  --group-id ${RDS_SG_ID} \
  --protocol tcp \

```

```

--port 5432 \
--source-group sg-0app12345678

# Verify no public access rule exists
aws ec2 describe-security-groups \
  --group-ids ${RDS_SG_ID} \
  --query 'SecurityGroups[0].IpPermissions'

# Create DB subnet group using private subnets only
aws rds create-db-subnet-group \
  --db-subnet-group-name prod-private-subnets \
  --db-subnet-group-description "Private subnets for production RDS" \
  --subnet-ids subnet-private-1a subnet-private-1b subnet-private-1c

```

AWS PrivateLink solves a specific architectural problem: you need to access an RDS instance from a different VPC or from an on-premises network via Direct Connect without exposing the database to the public internet or setting up full VPC peering. PrivateLink creates an interface VPC endpoint with a private IP in your VPC that proxies to the RDS endpoint. This is the right pattern for:

- SaaS applications that need to query a customer’s database
- Analytics platforms in a centralized VPC querying databases in spoke VPCs
- On-premises applications connecting through Direct Connect to RDS

Azure SQL network isolation uses service endpoints (simple, lower complexity) or Private Endpoint (production recommended). With a Private Endpoint, Azure SQL gets a private IP in your VNet. The Azure equivalent of VPC security groups is Network Security Groups (NSGs), and Azure SQL also supports a firewall rule layer at the server level — you configure allowed IP ranges and VNet rules. Unlike AWS, Azure SQL has a server-level “Allow Azure services and resources to access this server” toggle that is enabled by default and should be disabled in production environments where all connectivity is via Private Endpoint.

```

# Create Azure SQL Private Endpoint
az network private-endpoint create \
  --name prod-sql-private-endpoint \
  --resource-group prod-rg \
  --vnet-name prod-vnet \
  --subnet private-endpoints-subnet \
  --private-connection-resource-id \
    $(az sql server show --name prod-sql-server --resource-group prod-rg \
      --query id --output tsv) \
  --group-ids sqlServer \
  --connection-name prod-sql-private-connection

# Create private DNS zone for Azure SQL resolution
az network private-dns zone create \
  --resource-group prod-rg \
  --name "privatelink.database.windows.net"

az network private-dns link vnet create \
  --resource-group prod-rg \
  --zone-name "privatelink.database.windows.net" \
  --name prod-vnet-dns-link \
  --virtual-network prod-vnet \
  --registration-enabled false

# Create DNS record for the private endpoint
az network private-endpoint dns-zone-group create \
  --resource-group prod-rg \
  --endpoint-name prod-sql-private-endpoint \
  --name prod-sql-dns-zone-group \
  --private-dns-zone "privatelink.database.windows.net" \
  --zone-name sql

```


Cloud SQL uses Private IP (VPC peering between your VPC and Google’s service producer VPC) or Private Service Connect (analogous to AWS PrivateLink). For new deployments, Private Service Connect is preferred: it doesn’t require VPC peering, avoids IP range overlap issues that plague large organizations, and gives you a single IP in your VPC subnet that routes to Cloud SQL. The older Private IP approach with VPC peering works but creates a transitive routing constraint — you can’t reach Cloud SQL through a VPN or Interconnect from on-premises without additional configuration via Cloud DNS and route export settings.

```
# Create Cloud SQL instance with Private Service Connect
gcloud sql instances create prod-cloudsql-pg \
  --database-version=POSTGRES_15 \
  --tier=db-custom-8-32768 \
  --region=us-central1 \
  --no-assign-ip \
  --enable-private-service-connect \
  --allowed-psc-projects=my-project-id \
  --storage-type=SSD \
  --storage-size=500GB \
  --storage-auto-increase \
  --backup-start-time=03:00 \
  --maintenance-window-day=SUN \
  --maintenance-window-hour=5 \
  --availability-type=REGIONAL

# Get the PSC service attachment URI
gcloud sql instances describe prod-cloudsql-pg \
  --format="value(settings.ipConfiguration.pscConfig.serviceAttachmentLink)"

# Create a forwarding rule in your VPC pointing to the PSC attachment
# (Use the service attachment URI from the previous command)
gcloud compute forwarding-rules create prod-cloudsql-psc-endpoint \
  --region=us-central1 \
  --network=prod-vpc \
  --subnet=private-subnet-us-central1 \
  --address=prod-cloudsql-psc-ip \
  --target-service-attachment=<SERVICE_ATTACHMENT_URI>
```

5.5 Parameter Tuning in Managed Environments

What You Can Change vs. What Is Fixed

On premises, you had full authority over `postgresql.conf` — every GUC was yours to set. In managed cloud environments, you work within parameter groups (AWS), server parameters (Azure), and database flags (GCP), and some parameters that matter most to performance are fixed or bounded by the managed service.

The parameters you **cannot change** in RDS PostgreSQL: - `max_connections` — configurable but bounded by instance class (db.r6g.large forces a lower ceiling than on-premises because RDS manages memory for its own processes) - `wal_level` — always `logical` in RDS (which is actually more permissive than on-premises defaults) - `archive_mode` — always `on` in RDS (RDS manages WAL archival for backups and Point-in-Time Recovery) - `superuser` access — you get `rds_superuser` role, not true PostgreSQL superuser; certain system catalog writes and extensions requiring superuser are unavailable

The parameters you **should** tune in RDS PostgreSQL parameter groups:

```
-- Check current settings and their sources (run from psql)
SELECT
```

```

name,
setting,
unit,
source,
context,
short_desc
FROM pg_settings
WHERE name IN (
    'shared_buffers',
    'effective_cache_size',
    'work_mem',
    'maintenance_work_mem',
    'max_worker_processes',
    'max_parallel_workers',
    'max_parallel_workers_per_gather',
    'checkpoint_completion_target',
    'wal_buffers',
    'default_statistics_target',
    'random_page_cost',
    'effective_io_concurrency',
    'autovacuum_vacuum_scale_factor',
    'autovacuum_analyze_scale_factor',
    'autovacuum_vacuum_cost_delay',
    'log_min_duration_statement',
    'log_autovacuum_min_duration'
)
ORDER BY name;

```

In AWS, you apply these through a parameter group. For RDS, changes to parameters with `context = postmaster` require an instance reboot; `context = sighup` parameters apply without a reboot.

```

# Create a custom RDS parameter group for PostgreSQL 15
aws rds create-db-parameter-group \
    --db-parameter-group-name prod-postgres15-params \
    --db-parameter-group-family postgres15 \
    --description "Production PostgreSQL 15 parameters"

# Apply key performance parameters
aws rds modify-db-parameter-group \
    --db-parameter-group-name prod-postgres15-params \
    --parameters \
        "ParameterName=effective_cache_size,ParameterValue=75%,ApplyMethod=pending-reboot" \
        "ParameterName=work_mem,ParameterValue=65536,ApplyMethod=pending-reboot" \
        "ParameterName=maintenance_work_mem,ParameterValue=2097152,ApplyMethod=pending-reboot" \
        "ParameterName=checkpoint_completion_target,ParameterValue=0.9,ApplyMethod=immediate" \
        "ParameterName=default_statistics_target,ParameterValue=200,ApplyMethod=immediate" \
        "ParameterName=random_page_cost,ParameterValue=1.1,ApplyMethod=immediate" \
        "ParameterName=effective_io_concurrency,ParameterValue=200,ApplyMethod=immediate" \
        "ParameterName=autovacuum_vacuum_scale_factor,ParameterValue=0.02,ApplyMethod=immediate" \
        "ParameterName=autovacuum_analyze_scale_factor,ParameterValue=0.01,ApplyMethod=immediate" \
        "ParameterName=autovacuum_vacuum_cost_delay,ParameterValue=2,ApplyMethod=immediate" \
        "ParameterName=log_min_duration_statement,ParameterValue=1000,ApplyMethod=immediate" \
        "ParameterName=log_autovacuum_min_duration,ParameterValue=0,ApplyMethod=immediate"

```

Note on `shared_buffers` in RDS: AWS manages `shared_buffers` automatically based on instance class and sets it to 25% of total instance RAM, which is the standard PostgreSQL recommendation. You can override this in the parameter group, but AWS will not let you set it above a ceiling they manage — and exceeding 40% of available RAM in a managed environment where RDS also has memory reserved for its own processes can trigger OOM conditions. Accept the managed default here unless you have profiled evidence of a specific benefit.

Azure SQL Parameter Equivalent

Azure SQL (SQL Server) exposes server properties rather than a flat parameter file. The equivalent configuration surface area is `sp_configure`, Query Store settings, and database-level compatibility settings. Azure SQL manages memory (`max server memory`) automatically based on your vCore tier — you don't set it. What you do control:

```
-- Azure SQL: Check current configuration options
SELECT
    name,
    value,
    value_in_use,
    description,
    is_dynamic,
    is_advanced
FROM sys.configurations
WHERE name IN (
    'max degree of parallelism',
    'cost threshold for parallelism',
    'optimize for ad hoc workloads',
    'backup compression default',
    'remote query timeout (s)',
    'fill factor (%)'
)
ORDER BY name;

-- Set MAXDOP and cost threshold (Business Critical tier example)
-- Azure SQL allows sp_configure changes for non-memory parameters
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;

EXEC sp_configure 'max degree of parallelism', 4;
RECONFIGURE;

EXEC sp_configure 'cost threshold for parallelism', 50;
RECONFIGURE;

EXEC sp_configure 'optimize for ad hoc workloads', 1;
RECONFIGURE;

-- Database-level settings
ALTER DATABASE CURRENT SET QUERY_STORE = ON
(OPTION_MODE = READ_WRITE,
CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS = 30),
DATA_FLUSH_INTERVAL_SECONDS = 900,
MAX_STORAGE_SIZE_MB = 1000,
QUERY_CAPTURE_MODE = AUTO);

-- Check Query Store is capturing data
SELECT
    actual_state_desc,
    desired_state_desc,
    current_storage_size_mb,
    max_storage_size_mb,
    query_capture_mode_desc
FROM sys.database_query_store_options;
```

GCP Cloud SQL Flags

Cloud SQL exposes PostgreSQL configuration through database flags in the instance settings. The syntax and behavior mirror standard PostgreSQL GUCs; the constraint is that flags requiring a postmaster restart trigger a brief instance restart automatically when applied via `gcloud` or the Console.

```
# Apply PostgreSQL performance flags to Cloud SQL instance
gcloud sql instances patch prod-cloudsql-pg \
```

```

--database-flags \
  work_mem=65536,\
  maintenance_work_mem=2097152,\
  effective_cache_size=25165824,\
  checkpoint_completion_target=0.9,\
  default_statistics_target=200,\
  random_page_cost=1.1,\
  effective_io_concurrency=200,\
  autovacuum_vacuum_scale_factor=0.02,\
  autovacuum_analyze_scale_factor=0.01,\
  autovacuum_vacuum_cost_delay=2,\
  log_min_duration_statement=1000,\
  log_autovacuum_min_duration=0

# Verify applied flags
gcloud sql instances describe prod-cloudsql-pg \
  --format="yaml(settings.databaseFlags)"

```

A Cloud SQL-specific constraint worth noting: `max_connections` in Cloud SQL is tied to instance memory, and Cloud SQL sets a default that is lower than you might configure on a self-managed instance of the same RAM size. This is because Cloud SQL reserves memory for its HA and proxy infrastructure. If your application opens large connection pools against Cloud SQL without an intermediate connection pooler (PgBouncer on a GCE VM, or the Cloud SQL Auth Proxy with connection limiting), you'll hit `max_connections` errors under load. Plan for this upfront and size connection pools against the actual `max_connections` value, not the RAM-derived estimate from your on-premises experience.

5.6 Serverless and Auto-Scaling Database Patterns

When Serverless Makes Architectural Sense

Serverless databases — Aurora Serverless v2, Azure SQL Serverless, Neon, AlloyDB Omni — solve a specific problem: workloads with highly variable or intermittent demand where the cost of provisioning for peak is prohibitive and the operational overhead of manual scaling is unacceptable. The trade-offs are real: cold start latency, capacity limits, and pricing models that favor sustained workloads less than provisioned instances do.

Aurora Serverless v2 scales ACUs (Aurora Capacity Units, where 1 ACU = 2GB RAM and proportional CPU) between a configured minimum and maximum. Scaling is continuous — it doesn't round to whole numbers or cause connection drops. Unlike Serverless v1, v2 can maintain state during scaling; it doesn't drop connections on a scale event. For a development database that runs overnight jobs but sits idle during business hours, a min of 0.5 ACU and max of 8 ACU might reduce costs substantially versus a provisioned `db.r6g.large`. For production OLTP with a sustained 10,000 TPS, a provisioned instance is almost always the right choice — the per-ACU pricing of Serverless v2 at steady state typically exceeds provisioned pricing above a moderate utilization threshold.

```

# Aurora Serverless v2 - mixed cluster: serverless writer, provisioned readers
resource "aws_rds_cluster" "serverless_v2" {
  cluster_identifier = "staging-aurora-serverless"
  engine             = "aurora-postgresql"
  engine_version     = "15.4"
  database_name      = "stagingdb"
  master_username    = "dbadmin"
  master_password    = var.db_password
}

```

```

serverlessv2_scaling_configuration {
    min_capacity = 0.5
    max_capacity = 16
}

vpc_security_group_ids = [aws_security_group.aurora_sg.id]
db_subnet_group_name   = aws_db_subnet_group.prod.name

backup_retention_period = 3
skip_final_snapshot     = true # staging only
}

resource "aws_rds_cluster_instance" "serverless_writer" {
    identifier      = "staging-aurora-serverless-writer"
    cluster_identifier = aws_rds_cluster.serverless_v2.id
    instance_class  = "db.serverless"
    engine          = aws_rds_cluster.serverless_v2.engine
    engine_version  = aws_rds_cluster.serverless_v2.engine_version
}

```

Azure SQL Serverless applies to single databases in the General Purpose tier only. It auto-pauses after a configurable idle period (minimum 1 hour) and resumes on the next connection — the resume latency is typically several seconds to tens of seconds, making it unsuitable for production latency-sensitive applications. It's well-suited to development and test environments, infrequently accessed reporting databases, or any workload where the first query of the day can tolerate an initial delay.

```

# Create Azure SQL Serverless database
az sql db create \
  --resource-group dev-rg \
  --server dev-sql-server \
  --name dev-reporting-db \
  --edition GeneralPurpose \
  --family Gen5 \
  --compute-model Serverless \
  --min-capacity 0.5 \
  --capacity 4 \
  --auto-pause-delay 60 \
  --zone-redundant false

# Check current database state (running vs paused)
az sql db show \
  --resource-group dev-rg \
  --server dev-sql-server \
  --name dev-reporting-db \
  --query '{status:status,currentServiceObjectiveName:currentServiceObjectiveName}'

```

Autoscaling for Read Capacity: Aurora Auto Scaling

For Aurora, you can attach an Application Auto Scaling policy to the number of Aurora reader instances. This adds or removes reader instances based on `CPUUtilization` or `DatabaseConnections` CloudWatch metrics. The scale-out event adds a new Aurora reader — which takes 1–3 minutes to become available because, while the storage is already populated, the new instance needs to warm its buffer cache. Plan for this lag in your capacity planning: reactive auto-scaling can't respond instantly to a sudden traffic spike.

```

# Register Aurora reader as an auto-scaling target
aws application-autoscaling register-scalable-target \
  --service-namespace rds \
  --resource-id cluster:prod-aurora-postgres \
  --scalable-dimension rds:cluster:ReadReplicaCount \
  --min-capacity 1 \
  --max-capacity 5

# Create a target tracking scaling policy based on CPU

```

```
aws application-autoscaling put-scaling-policy \
  --policy-name aurora-reader-cpu-scaling \
  --service-namespace rds \
  --resource-id cluster:prod-aurora-postgres \
  --scalable-dimension rds:cluster:ReadReplicaCount \
  --policy-type TargetTrackingScaling \
  --target-tracking-scaling-policy-configuration '{
    "TargetValue": 70.0,
    "PredefinedMetricSpecification": {
      "PredefinedMetricType": "RDSReaderAverageCPUUtilization"
    },
    "ScaleInCooldown": 300,
    "ScaleOutCooldown": 120
  }'
```

5.7 Multi-Region and Disaster Recovery Architecture

RPO and RTO in Managed Cloud Environments

The cloud doesn't change what RPO and RTO mean — it changes what's achievable at a given operational complexity and cost. On premises, achieving sub-minute RPO typically meant synchronous replication to a co-located DR site with dedicated hardware. In the cloud, the same capability exists across geographic regions, but the network latency between regions (typically 30–80ms between US regions, 80–180ms between continents) means synchronous replication across regions is latency-additive for every write transaction.

AWS cross-region strategy options (RDS PostgreSQL):

Option	RPO	RTO	Notes
Cross-region read replica	Seconds to minutes	20–40 min (manual promote)	Async; lag depends on write rate
Aurora Global Database	< 1 second	< 1 minute (managed failover)	Shared storage; fast recovery
RDS automated backups + S3 cross-region copy	Hours	Hours	Restore from snapshot
AWS Backup cross-region	Hours	Hours	Policy-driven, integrates with RDS

For RPO near zero and RTO under one minute, Aurora Global Database is the only native AWS solution. For RTO of 30–60 minutes and RPO of seconds to a few minutes, cross-region read replicas with manual promotion is the standard approach.

Aurora Global Database Failover Procedure

Aurora Global Database supports two failover modes: managed planned failover (for maintenance, testing) and unmanaged failover (for disaster recovery when the primary region is unavailable). The difference is operationally important.

Managed planned failover gracefully promotes a secondary region to primary, drains the primary, and reconfigures the global cluster. This is safe and reversible.

Unmanaged failover (detach-and-promote) is used when the primary region is inaccessible. The secondary cluster is detached from the global cluster and promoted independently. After recovery, you must manually re-establish the global cluster or rebuild the replication topology — the old primary cannot simply re-join as a secondary without a rebuild.

```
# Initiate Aurora Global Database managed failover (boto3)
import boto3
import time

rds = boto3.client('rds', region_name='us-east-1')

def initiate_global_failover(global_cluster_id, target_db_cluster_arn):
    """
    Perform managed planned failover of Aurora Global Database.
    target_db_cluster_arn: the secondary cluster to promote to primary.
    """
    print(f"Initiating managed failover to {target_db_cluster_arn}")

    response = rds.failover_global_cluster(
        GlobalClusterIdentifier=global_cluster_id,
        TargetDbClusterIdentifier=target_db_cluster_arn
    )

    print(f"Failover initiated. Status: {response['GlobalCluster']['Status']}")

    # Poll for completion
    while True:
        status_response = rds.describe_global_clusters(
            GlobalClusterIdentifier=global_cluster_id
        )
        status = status_response['GlobalClusters'][0]['Status']
        members = status_response['GlobalClusters'][0]['GlobalClusterMembers']

        writer = next((m for m in members if m['IsWriter']), None)
        print(f"Status: {status} | Writer: {writer['DBClusterArn']} if writer else 'None'")

        if status == 'available':
            print("Failover complete.")
            break

        time.sleep(15)

# Example: fail over to us-west-2 secondary
initiate_global_failover(
    global_cluster_id='prod-global-cluster',
    target_db_cluster_arn='arn:aws:rds:us-west-2:123456789:cluster:prod-aurora-secondary'
)
```

Azure SQL Geo-Replication and Failover Groups

Azure SQL Business Critical supports active geo-replication (up to four readable secondary databases in different regions) and auto-failover groups (a layer on top of geo-replication that provides a single listener endpoint and automatic failover). Failover groups are the preferred production pattern because they provide a stable connection endpoint that doesn't change during failover.

```
# Create a failover group for Azure SQL
az sql failover-group create \
    --name prod-sql-failover-group \
    --partner-server prod-sql-server-westus2 \
    --resource-group prod-rg \
    --server prod-sql-server-eastus \
    --add-db prod-database \
    --failover-policy Automatic \
    --grace-period 1
```

```
# Check failover group status and replication state
az sql failover-group show \
  --name prod-sql-failover-group \
  --resource-group prod-rg \
  --server prod-sql-server-eastus \
  --query
  ↪ '{role:replicationRole,state:replicationState,partnerServers:partnerServers[]}.{location:location,role:replicationRo

# Initiate manual planned failover (to secondary)
az sql failover-group set-primary \
  --name prod-sql-failover-group \
  --resource-group prod-rg \
  --server prod-sql-server-westus2
```

When using Azure SQL failover groups, your application connects to the failover group listener endpoint (e.g., `prod-sql-failover-group.database.windows.net`) rather than the individual server endpoint. The listener endpoint routes to whichever server is currently primary. After failover, your connection string doesn't change. This is the Azure equivalent of the Aurora cluster endpoint — one DNS name regardless of which physical node is primary.

GCP Cloud SQL Cross-Region Replicas and AlloyDB

Cloud SQL cross-region read replicas are straightforward: you create a read replica in a different region, and GCP handles the asynchronous replication. Promoting a cross-region replica to a standalone instance for DR is a manual operation — there is no equivalent to Aurora Global Database's managed cross-region failover with automatic reconfiguration.

AlloyDB's approach to HA and DR is more sophisticated: within a region, AlloyDB uses Paxos-based distributed storage with automatic failover under 60 seconds. For cross-region DR, AlloyDB supports cross-region replicas. At the time of writing, AlloyDB cross-region replica promotion is also a manual operation requiring DBA intervention. If your RPO/RTO requirements demand automated cross-region failover with sub-minute recovery, Aurora Global Database remains the most mature managed option across the three major cloud providers.

5.8 Observability Architecture: Metrics, Logs, and Tracing

The Monitoring Stack You Inherit

On premises you assembled your own monitoring stack — Prometheus + Grafana + postgres_exporter, or Zabbix, or a commercial monitoring product. In the cloud, each provider ships integrated monitoring that is “good enough” for most situations and has specific gaps that matter at scale.

AWS CloudWatch automatically collects ~40 RDS metrics: `CPUUtilization`, `DatabaseConnections`, `FreeableMemory`, `ReadIOPS`, `WriteIOPS`, `ReadLatency`, `WriteLatency`, `ReplicaLag`, `DiskQueueDepth`. These are sufficient for high-level health monitoring but insufficient for query-level performance work. **RDS Performance Insights** fills that gap — it samples `pg_stat_activity`, wait events, and top SQL every second and presents a visual breakdown of what the database was doing. Performance Insights is the single most valuable monitoring tool in the RDS ecosystem, and the 7-day free retention tier is adequate for most incident analysis. Extend to 2 years for regulatory environments.

```
# Query Performance Insights API for top SQL by load (db-load)
```



```

import boto3
from datetime import datetime, timedelta, timezone

pi = boto3.client('pi', region_name='us-east-1')

# Get the resource identifier for your RDS instance
rds = boto3.client('rds', region_name='us-east-1')
instance = rds.describe_db_instances(
    DBInstanceIdentifier='prod-postgres-primary'
)
resource_id = instance['DBInstances'][0]['DbiResourceId']

end_time = datetime.now(timezone.utc)
start_time = end_time - timedelta(hours=1)

response = pi.get_resource_metrics(
    ServiceType='RDS',
    Identifier=resource_id,
    MetricQueries=[
        {
            'Metric': 'db.load.avg',
            'GroupBy': {
                'Group': 'db.sql',
                'Dimensions': ['db.sql.statement'],
                'Limit': 10
            }
        }
    ],
    StartTime=start_time,
    EndTime=end_time,
    PeriodInSeconds=300
)

print("Top SQL by average DB load (last hour):")
for metric in response['MetricList']:
    for key, val in metric.get('Keys', {}).items():
        print(f"\nSQL: {val.get('Dimensions', {}).get('db.sql.statement', 'N/A')[:120]}")
        # Sum up the data points for this SQL
        total_load = sum(
            dp['Value'] for dp in metric.get('DataPoints', [])
            if dp.get('Value') is not None
        )
        print(f"Total load contribution: {total_load:.2f}")

```

Azure SQL provides Query Store natively (enabled by default, unlike on-premises SQL Server where it's opt-in). Query Store captures execution statistics, plan history, and wait statistics automatically. The Azure Portal's Query Performance Insight surface Query Store data with minimal configuration. For deeper analysis, Azure Monitor Log Analytics can ingest SQL Server DMV output via the Azure Monitor extension.

Cloud SQL integrates with Cloud Monitoring (formerly Stackdriver). Key metrics are available out of the box under the `cloudsql.googleapis.com` metric namespace. For query-level visibility, Cloud SQL's Query Insights feature provides statement-level analysis similar to RDS Performance Insights. Enable it via the Console or CLI:

```

# Enable Query Insights on Cloud SQL
gcloud sql instances patch prod-cloudsql-pg \
  --insights-config-query-insights-enabled \
  --insights-config-record-application-tags \
  --insights-config-record-client-address \
  --insights-config-query-plans-per-minute=5

```

Log-Based Alerting Patterns

Slow query logs are your first line of defense against performance regressions. In all three cloud environments, log export to a centralized logging service is the correct pattern — don't rely on the console's inline log viewer for anything you'll need to search across hours of data.

```
# AWS: Export RDS PostgreSQL logs to CloudWatch Logs
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-primary \
  --cloudwatch-logs-export-configuration '{"EnableLogTypes":["postgresql","upgrade"]}' \
  --apply-immediately

# Create a CloudWatch Metric Filter for slow queries (> 1 second threshold)
aws logs put-metric-filter \
  --log-group-name /aws/rds/instance/prod-postgres-primary/postgresql \
  --filter-name slow-queries \
  --filter-pattern "[date, time, tz, user, host, process, session, msg, duration, statement, query]" \
  --metric-transformations \
    metricName=SlowQueryCount,metricNamespace=Custom/RDS,metricValue=1

# Create an alarm when slow queries exceed 10 per minute
aws cloudwatch put-metric-alarm \
  --alarm-name prod-postgres-slow-queries \
  --metric-name SlowQueryCount \
  --namespace Custom/RDS \
  --statistic Sum \
  --period 60 \
  --evaluation-periods 5 \
  --threshold 10 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789:prod-db-alerts

# GCP: Export Cloud SQL logs to Cloud Logging and create a log-based alert
# First, ensure log_min_duration_statement is set to 1000 (via database flags as shown earlier)

# Create a log-based metric for slow queries in Cloud Logging
gcloud logging metrics create cloudsql-slow-queries \
  --description="Count of Cloud SQL queries exceeding log_min_duration_statement" \
  --log-filter='resource.type="cloudsql_database" \
    textPayload=~"duration:" \
    resource.labels.database_id="my-project:prod-cloudsql-pg"'

# Create an alerting policy via gcloud monitoring
# (Complex policies are more practical via Terraform or the Console)
gcloud alpha monitoring policies create \
  --policy-from-file=slow-query-alert-policy.yaml
```

5.9 Cost Architecture: Designing for Efficiency

The Bill Is Part of the Architecture

On premises, infrastructure cost was largely sunk — the servers existed whether queries ran or not. In the cloud, every ACU, IOPS, byte of storage, and GB of data transfer is a billing event. Designing a cloud database architecture without modeling the cost profile is incomplete design.

RDS cost drivers, in rough order of impact for a typical production deployment: 1. Instance class (db.r6g.4xlarge Multi-AZ 2x single-AZ hourly rate) 2. Storage (gp3 vs. io2 is a large cost difference at high IOPS provisioning) 3. Data transfer (same-region same-AZ is free; cross-AZ data transfer is billed; cross-region is billed at a higher rate) 4. Snapshot storage (retained backups beyond the automated window; cross-region snapshot copies) 5. RDS Proxy (flat hourly charge per vCPU) 6. Performance Insights extended retention (beyond 7 days)

Immediate cost-reduction patterns that don't require re-architecting:

- Use **Reserved Instances** (1-year or 3-year) for production databases with predictable workloads. The discount versus on-demand pricing is substantial and consistent across instance families. Convertible Reserved Instances allow you to exchange instance family within the same region, which is valuable as you right-size.
- **Right-size with Performance Insights data** before committing to Reserved Instances. Look at DBLoad over 4–6 weeks; if average DBLoad is consistently below 2 on a db.r6g.4xlarge (which has 16 vCPUs), a db.r6g.2xlarge may be appropriate.
- **Aurora I/O-Optimized pricing** (available since mid-2023) bundles I/O costs into the instance price. If your Aurora cluster's I/O charges exceed roughly 25% of your total cluster cost, I/O-Optimized pricing reduces the total bill.
- Use **gp3 over gp2** for RDS if you haven't migrated yet — gp3 is the same or lower price per GB and allows IOPS configuration independent of capacity.

```
# Simple cost estimation helper for RDS instance sizing decision
# Uses boto3 to fetch current running config and CloudWatch metrics

import boto3
from datetime import datetime, timedelta, timezone

def estimate_rightsizing_opportunity(instance_id, region='us-east-1'):
    rds = boto3.client('rds', region_name=region)
    cw = boto3.client('cloudwatch', region_name=region)

    instance = rds.describe_db_instances(
        DBInstanceIdentifier=instance_id
    )['DBInstances'][0]

    instance_class = instance['DBInstanceClass']
    multi_az = instance['MultiAZ']

    print(f"Instance: {instance_id}")
    print(f"Class: {instance_class} | MultiAZ: {multi_az}")

    # Pull 7-day average CPU and memory metrics
    end_time = datetime.now(timezone.utc)
    start_time = end_time - timedelta(days=7)

    def get_metric_avg(metric_name):
        response = cw.get_metric_statistics(
            Namespace='AWS/RDS',
            MetricName=metric_name,
            Dimensions=[{'Name': 'DBInstanceIdentifier', 'Value': instance_id}],
            StartTime=start_time,
            EndTime=end_time,
            Period=3600,
            Statistics=['Average']
        )
        if response['Datapoints']:
            values = [dp['Average'] for dp in response['Datapoints']]
            return sum(values) / len(values)
        return None

    avg_cpu = get_metric_avg('CPUUtilization')
    avg_freeable_memory_mb = (get_metric_avg('FreeableMemory') or 0) / (1024 * 1024)
    avg_connections = get_metric_avg('DatabaseConnections')

    print(f"\n7-Day Averages:")
    print(f" CPU Utilization: {avg_cpu:.1f}%" if avg_cpu else " CPU: No data")
    print(f" Freeable Memory: {avg_freeable_memory_mb:.0f} MB" if avg_freeable_memory_mb else "
    ↪ Memory: No data")
    print(f" Connections: {avg_connections:.0f}" if avg_connections else " Connections: No data")
```

```
# Heuristic recommendations
if avg_cpu and avg_cpu < 20:
    print(f"\n >> CPU consistently below 20% - consider downsizing instance class")
if avg_freeable_memory_mb and avg_freeable_memory_mb > 8192:
    print(f"\n >> Over 8GB consistently free - memory may be over-provisioned")
if avg_cpu and avg_cpu > 75:
    print(f"\n >> CPU consistently above 75% - consider upsizing or read replica offload")

estimate_rightsizing_opportunity('prod-postgres-primary')
```

Cross-AZ Data Transfer Costs

A hidden cost that regularly surprises teams moving from on-premises is **cross-AZ data transfer**. In AWS, traffic between Availability Zones is billed in both directions. If your application tier in `us-east-1a` queries an RDS instance in `us-east-1b` (common in Multi-AZ setups where the primary has failed over), every query response traverses AZ boundaries. The per-GB rate is modest, but at high query volumes it accumulates.

The architectural mitigation is to ensure your application's Auto Scaling Group or ECS tasks are AZ-aware and prefer the same AZ as the current database primary when possible. This is practical only when you can detect which AZ the current primary is in (available via the RDS `AvailabilityZone` attribute from the API) and route accordingly. Most teams accept the cross-AZ transfer cost as the price of AZ-resilient architecture and focus cost optimization efforts on instance sizing and Reserved Instance coverage instead.

Azure equivalent: Azure charges for data transfer between Availability Zones within a region similarly. Azure SQL with Private Endpoint generally keeps traffic within the VNet and minimizes unexpected egress charges, but inter-zone traffic between application VMs and Azure SQL in a different zone is still billed.

GCP equivalent: Cloud SQL cross-zone traffic within a region follows GCP's standard inter-zone pricing, with the same general pattern: intra-region cross-zone is billed at a lower rate than inter-region, but it accumulates at scale.

5.10 Extension and Feature Constraints by Cloud Provider

Not All PostgreSQL Is the Same PostgreSQL

A common misconception when migrating from self-managed PostgreSQL is that any PostgreSQL extension that compiles will work on RDS, Cloud SQL, or AlloyDB. The managed service controls which extensions are available, and the list differs meaningfully across providers. This affects schema migration, PostGIS deployments, and custom procedural language usage.

RDS PostgreSQL has one of the most permissive extension lists among managed providers. Extensions are installed with `CREATE EXTENSION` and the available set is documented per engine version. Key extensions available on RDS: `pg_stat_statements`, `pgaudit`, `pg_cron`, `PostGIS`, `uuid-oss`, `btree_gin`, `btree_gist`, `pg_partman`, `aws_s3` (AWS-specific, for S3 import/export), `aws_lambda` (for invoking Lambda from PostgreSQL), `vector` (pgvector, for ML embedding similarity search).

Extensions **not available** on RDS that require a self-managed PostgreSQL: anything requiring `CREATE EXTENSION` with `superuser` privileges not delegated to `rds_superuser`, custom shared

libraries loaded via `shared_preload_libraries` that aren't on the approved list (you can't load your own `.so` file), and extensions that require OS-level access.

```
-- Check available extensions on RDS PostgreSQL
SELECT
    name,
    default_version,
    installed_version,
    comment
FROM pg_available_extensions
WHERE installed_version IS NOT NULL
    OR name IN (
        'pg_stat_statements',
        'pgaudit',
        'pg_cron',
        'postgis',
        'vector',
        'pg_partman',
        'aws_s3'
    )
ORDER BY name;

-- Install pgvector for ML embedding storage (available in RDS PostgreSQL 15+)
CREATE EXTENSION IF NOT EXISTS vector;

-- Verify installation
SELECT extname, extversion
FROM pg_extension
WHERE extname = 'vector';

-- Create a table using the vector type
CREATE TABLE document_embeddings (
    id          BIGSERIAL PRIMARY KEY,
    document_id BIGINT NOT NULL,
    content     TEXT,
    embedding   vector(1536), -- OpenAI Ada-002 embedding dimension
    created_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Create an IVFFlat index for approximate nearest-neighbor search
CREATE INDEX ON document_embeddings
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

Cloud SQL PostgreSQL has a somewhat more restricted extension list than RDS. PostGIS is fully supported. `pg_cron` is not available as of current versions — scheduled jobs must use Cloud Scheduler calling a Cloud Run function or a direct API call. `pgaudit` is available. `pg_partman` is not available; partition management must be handled through application logic or Cloud Scheduler-triggered stored procedures. The `vector` extension (pgvector) was added to Cloud SQL's extension support; check the current GCP documentation for the exact minimum Cloud SQL version that supports it, as availability was added after the initial pgvector release.

AlloyDB for PostgreSQL is worth distinguishing from Cloud SQL: AlloyDB has a broader extension set and includes `google_ml_integration`, a proprietary extension that enables calling Vertex AI models directly from SQL — a feature that has no direct equivalent on RDS or Cloud SQL.

```
-- AlloyDB: Call a Vertex AI model from SQL using google_ml_integration
-- (AlloyDB-specific; not available on Cloud SQL or RDS)
CREATE EXTENSION IF NOT EXISTS google_ml_integration;

-- Generate embeddings for text stored in a PostgreSQL table
SELECT
    id,
    content,
```

```
embedding('textembedding-gecko@003', content) AS text_embedding
FROM documents
LIMIT 10;
```

Azure SQL (SQL Server) extension semantics are different — SQL Server doesn't use PostgreSQL's extension system. The Azure SQL equivalent constraint is which CLR assemblies are permitted, which linked server providers are available, and which features require SQL Server Agent (which runs in a limited form in Azure SQL). Full-text search, In-Memory OLTP, and columnstore indexes are fully available in Azure SQL. R and Python external scripts (`sp_execute_external_script`) are available in SQL Server on VMs but not in Azure SQL Database PaaS; for ML workloads in Azure SQL, you use Azure ML integration instead.

Key Takeaways

- **Cloud HA is managed abstraction, not magic:** Aurora's shared-storage architecture gives sub-30-second failover with no data loss, while RDS Multi-AZ relies on DNS TTL propagation and requires application-side reconnect logic. Understanding the mechanism determines how you write your connection handling code.
 - **Read replica lag is architecturally different across services:** Aurora replicas share the storage layer and show sub-20ms lag; RDS and Cloud SQL read replicas use asynchronous streaming replication and can show meaningful lag under write pressure. Use the right replica type for the consistency requirement of each read workload.
 - **Storage scaling is one-directional and has operational windows:** RDS storage increases are permanent, and scaling a Multi-AZ instance triggers a failover cycle. Aurora storage auto-scales transparently but doesn't shrink. Model your storage growth trajectory before provisioning and prefer Aurora for workloads with unpredictable data volume growth.
 - **Network isolation requires multiple layers:** VPC security groups (or NSGs in Azure) are necessary but not sufficient. Private Endpoint or PrivateLink is the production-standard connectivity pattern — public endpoints with IP allowlisting are not an equivalent security control. Validate that your application and DBA tooling can operate without public database access before deploying.
 - **Extension and feature availability varies significantly across managed providers:** An extension-dependent architecture built on self-managed PostgreSQL may not port directly to any managed service. Audit your `pg_extension` list early in a cloud migration, check availability against the target provider's current documentation, and design mitigation patterns — application-layer alternatives, Cloud Scheduler-based replacements for `pg_cron`, or a provider switch — before the migration cutover.
-

Chapter 6: RDS Deep Dive: Engines, Instance Types, and Storage

Amazon RDS is where most DBAs begin their cloud journey, and for good reason — it takes the engines you already know (PostgreSQL, MySQL, SQL Server, Oracle) and wraps them in a managed layer that handles provisioning, patching, and backups. But “managed” does not mean “transparent.” The choices you make about engine version, instance family, and storage type have cascading effects on performance, cost, and operational flexibility that cannot be undone without downtime or data migration. This chapter breaks down those choices with the specificity that production systems demand: which instance families to use and why, how RDS storage behaves differently from what you know on-premises, where the hard limits are, and what you give up when AWS takes over the OS layer.

6.1 Engine Selection and Version Strategy

RDS supports six database engines: MySQL, PostgreSQL, MariaDB, Oracle, Microsoft SQL Server, and Amazon Aurora (which gets its own chapter, but shares the RDS console). Each engine runs on a managed EC2 instance underneath, and the distinction between “RDS for PostgreSQL” and “self-managed PostgreSQL on EC2” is not just operational convenience — it represents a fundamental trade-off in control.

What you lose versus self-managed PostgreSQL on EC2:

- No OS-level access. You cannot SSH into the underlying host.
- No `pg_hba.conf` direct editing. Authentication rules are enforced through security groups, IAM database authentication, and SSL certificate requirements set via parameter groups.
- No superuser. The `rds_superuser` role grants most capabilities, but it is not true superuser — you cannot load arbitrary shared libraries, and you cannot access system catalogs in ways that require true OS-level privilege.
- No WAL archive access. You can enable WAL archiving to S3 for logical replication, but you cannot read the raw WAL files directly.
- No `pg_basebackup` from the instance. Backups are managed by RDS using EBS snapshots.

What you gain:

- Automated backups with point-in-time recovery (PITR) up to 35 days.
- Automated minor version upgrades on a configurable maintenance window.
- Multi-AZ failover handled by the platform without you managing Pacemaker or Patroni.

- Monitoring metrics surfaced through CloudWatch without running exporters.

Engine version selection is not just about features — it determines your upgrade path. RDS maintains a list of supported minor versions, and AWS periodically deprecates older ones, issuing mandatory upgrades. PostgreSQL 11 reached end of life on AWS in 2023 and instances still running it received forced upgrades. You should never run more than one major version behind current in production.

Check available versions before provisioning:

```
# List all available PostgreSQL engine versions in us-east-1
aws rds describe-db-engine-versions \
  --engine postgres \
  --query 'DBEngineVersions[*].[EngineVersion,DBEngineVersionDescription]' \
  --output table \
  --region us-east-1

# Check which versions support a specific feature (e.g., Kerberos auth)
aws rds describe-db-engine-versions \
  --engine postgres \
  --query 'DBEngineVersions[?contains(SupportedFeatureNames, `kerberos`) == `true`].[EngineVersion]' \
  --output text \
  --region us-east-1
```

SQL Server licensing deserves separate attention. RDS SQL Server offers two licensing models: License Included (LI) and Bring Your Own License (BYOL). License Included is available for Standard and Enterprise editions, with the license cost embedded in the hourly rate. Enterprise edition on a large instance class adds a substantial license-included premium to the hourly rate — check current AWS pricing for the specific instance class and edition. If your organization already has SQL Server Enterprise Agreement licenses, BYOL reduces that cost significantly — but BYOL requires Software Assurance, and AWS audits eligibility.

Oracle on RDS follows the same LI vs. BYOL model. Oracle Enterprise Edition is only available as BYOL. Standard Edition 2 is available as License Included. The moment you need features like Partitioning, Advanced Compression, or Real Application Testing, you're in Enterprise Edition territory, which means BYOL and your own Oracle licensing agreement.

Version pinning strategy: In production, pin your minor version explicitly. Do not enable auto minor version upgrade on database instances that have application dependencies on specific behavior (pg_catalog views changed significantly between PostgreSQL 13 and 14, for example). Enable auto minor version upgrade only on development and QA instances.

```
# Disable auto minor version upgrade on an existing instance
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --no-auto-minor-version-upgrade \
  --apply-immediately \
  --region us-east-1

# Verify the change
aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-01 \
  --query 'DBInstances[0].AutoMinorVersionUpgrade' \
  --region us-east-1
```


6.2 Instance Families and Sizing

The RDS instance naming convention follows the same pattern as EC2: `db.{family}{generation}.{size}`. Understanding the family is more important than the size, because each family has different CPU architecture, memory ratios, and network characteristics.

Current generation families:

Family	vCPU:Memory Ratio	Use Case	Example
db.t4g	Burstable, 1:2	Dev/test, small workloads	db.t4g.medium: 2 vCPU, 4 GB
db.m7g	General purpose, 1:4	Web applications, moderate OLTP	db.m7g.4xlarge: 16 vCPU, 64 GB
db.r7g	Memory optimized, 1:8	Large buffers, analytics	db.r7g.8xlarge: 32 vCPU, 256 GB
db.r7i	Memory optimized, Intel	SQL Server (no ARM support)	db.r7i.16xlarge: 64 vCPU, 512 GB
db.x2idn	Memory intensive, 1:32	SAP HANA, large in-memory	db.x2idn.16xlarge: 64 vCPU, 2048 GB

The `g` suffix means Graviton (ARM-based). Graviton instances offer 10–20% better price-performance for PostgreSQL and MySQL workloads compared to equivalent Intel or AMD instances. The catch: SQL Server and Oracle are not supported on Graviton because AWS cannot run those binaries on ARM. PostgreSQL and MySQL have native ARM builds that run well on Graviton.

T-series instances (burstable) in production: The T-series uses CPU credits. When the instance is idle, it accumulates credits. Under sustained load, it burns through credits and throttles to its baseline CPU (which is 20–40% of nominal). This behavior is catastrophic for production OLTP workloads that have sustained query pressure. The only legitimate production use of T-series is for small read replicas with light traffic, or internal tooling databases with infrequent queries.

You can detect credit exhaustion in CloudWatch:

```
# Check CPU credit balance for a burstable instance
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name CPUCreditBalance \
  --dimensions Name=DBInstanceIdentifier,Value=dev-postgres-t4g \
  --start-time 2024-01-15T00:00:00Z \
  --end-time 2024-01-15T23:59:59Z \
  --period 3600 \
  --statistics Minimum \
  --output table

# Also check CPU credit usage rate
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name CPUCreditUsage \
  --dimensions Name=DBInstanceIdentifier,Value=dev-postgres-t4g \
  --start-time 2024-01-15T00:00:00Z \
  --end-time 2024-01-15T23:59:59Z \
  --period 3600 \
```

```
--statistics Sum \
--output table
```

Sizing recommendations for PostgreSQL migration: When migrating a self-managed PostgreSQL instance, the most important metric from your source system is `shared_buffers` utilization — specifically, what fraction of your working set fits in the buffer cache. On RDS, `shared_buffers` defaults to 25% of instance memory for PostgreSQL. A `db.r7g.4xlarge` has 128 GB of RAM, giving you ~32 GB of `shared_buffers`. If your on-premises system has a 40 GB buffer pool with reasonable hit rates, move to `db.r7g.8xlarge` (256 GB RAM, ~64 GB `shared_buffers`) with headroom.

The right way to size using actual database metrics:

```
-- Run this on your SOURCE PostgreSQL instance before migration
-- to understand buffer cache behavior

-- Buffer hit rate (should be >95% in production)
SELECT
    sum(heap_blks_hit) AS heap_hits,
    sum(heap_blks_read) AS heap_reads,
    round(
        sum(heap_blks_hit)::numeric /
        nullif(sum(heap_blks_hit) + sum(heap_blks_read), 0) * 100, 2
    ) AS buffer_hit_rate_pct
FROM pg_statio_user_tables;

-- Working set estimate: tables + indexes that fit in cache
SELECT
    schemaname,
    relname AS table_name,
    pg_size_pretty(pg_total_relation_size(schemaname||'.'||relname)) AS total_size,
    round(
        heap_blks_hit::numeric / nullif(heap_blks_hit + heap_blks_read, 0) * 100, 1
    ) AS table_hit_rate_pct
FROM pg_statio_user_tables
WHERE heap_blks_hit + heap_blks_read > 10000
ORDER BY pg_total_relation_size(schemaname||'.'||relname) DESC
LIMIT 20;

-- Identify tables causing physical reads (candidates for shared_buffers increase)
SELECT
    schemaname,
    relname,
    heap_blks_read AS physical_reads,
    heap_blks_hit AS cache_hits
FROM pg_statio_user_tables
WHERE heap_blks_read > 1000
ORDER BY heap_blks_read DESC
LIMIT 10;
```

Changing instance class: RDS instance class changes require a reboot. On Single-AZ, this means downtime of 3–10 minutes. On Multi-AZ, AWS performs the change on the standby first, then fails over — total visible downtime is typically 30–120 seconds for the failover. Plan instance resizing during low-traffic windows regardless of Multi-AZ, because the failover itself causes connection drops.

6.3 RDS Storage: gp2, gp3, io1, io2, and Magnetic

Storage in RDS is EBS-backed, and the storage type you choose determines IOPS characteristics, throughput ceilings, and cost. Getting this wrong is the most common cause of RDS performance

problems, because storage bottlenecks in managed services are much harder to diagnose than on-premises where you can run `iostat` directly.

Available storage types:

gp2 (General Purpose SSD): The legacy default. Delivers 3 IOPS per GB of storage, with a burst pool for volumes under 1 TB. A 100 GB gp2 volume gets 300 IOPS baseline with burst to 3,000 IOPS using credits (same credit model as T-series CPU). Above roughly 5.3 TB (where 3 IOPS/GB reaches the 16,000 IOPS ceiling), you get the full ceiling without burst dependency. Maximum throughput: 250 MB/s. Do not use gp2 for new deployments — it has been superseded entirely by gp3.

gp3 (General Purpose SSD v2): The current recommended default. Provides a baseline of 3,000 IOPS and 125 MB/s throughput regardless of volume size. You can independently provision additional IOPS up to 16,000 and throughput up to 1,000 MB/s at additional cost. A 100 GB gp3 volume gets 3,000 IOPS — the same as a 1 TB gp2 volume, at one-tenth the storage cost. For most OLTP workloads under 16,000 IOPS, gp3 is the right choice.

io1 (Provisioned IOPS SSD): Allows IOPS provisioning independent of storage size, up to 64,000 IOPS on supported instance types. Maximum throughput: 1,000 MB/s. More expensive than gp3 for the same IOPS level. io1 made sense before gp3 existed; today, it's primarily for workloads needing more IOPS than gp3 provides but that don't require io2's higher durability or ceiling.

io2 (Provisioned IOPS SSD v2): Higher durability (99.999% vs 99.8% for io1), same IOPS ceiling. For SQL Server and Oracle, io2 supports up to 256,000 IOPS. For most PostgreSQL and MySQL workloads, gp3 with provisioned IOPS is sufficient and cheaper.

Magnetic (standard): Legacy spinning disk. Do not use this for any workload. It exists only for backward compatibility.

Storage autoscaling: RDS supports automatic storage scaling. When free storage drops below 10% of allocated storage (or below 5 GB, whichever is smaller), RDS automatically increases storage. Autoscaling only increases storage — it never shrinks it. You cannot reduce allocated storage on an RDS instance without creating a new instance and migrating data. Set your initial storage thoughtfully; you will pay for every GB permanently.

Configure a new PostgreSQL instance with gp3 and autoscaling via Terraform:

```
# terraform/rds_instance.tf
# Production PostgreSQL RDS with gp3 storage and appropriate settings

resource "aws_db_instance" "prod_postgres" {
  identifier      = "prod-postgres-primary"
  engine         = "postgres"
  engine_version = "16.3"
  instance_class = "db.r7g.4xlarge"

  # Storage configuration
  storage_type            = "gp3"
  allocated_storage      = 500          # GB - initial allocation
  max_allocated_storage  = 2000        # GB - autoscaling ceiling
  iops                   = 12000        # Additional provisioned IOPS above gp3 baseline (3000)
  storage_throughput     = 500          # MB/s - above gp3 baseline (125 MB/s)
  storage_encrypted      = true
  kms_key_id            = aws_kms_key.rds_encryption.arn

  # Engine configuration
```

```

db_name = "appdb"
username = "dbadmin"
password = var.db_password # Use Secrets Manager in production

# High availability
multi_az = true
availability_zone = "us-east-1a" # for single-AZ; ignored when multi_az=true

# Backup and maintenance
backup_retention_period = 14
backup_window = "03:00-04:00"
maintenance_window = "sun:04:30-sun:05:30"
deletion_protection = true

# Network
db_subnet_group_name = aws_db_subnet_group.private.name
vpc_security_group_ids = [aws_security_group.rds_postgres.id]
publicly_accessible = false

# Parameter and option groups
parameter_group_name = aws_db_parameter_group.postgres16_prod.name

# Monitoring
monitoring_interval = 60 # Enhanced monitoring every 60 seconds
monitoring_role_arn = aws_iam_role.rds_enhanced_monitoring.arn
performance_insights_enabled = true
performance_insights_retention_period = 731 # 2 years

# Upgrade behavior
auto_minor_version_upgrade = false
apply_immediately = false # Apply changes during maintenance window

tags = {
  Environment = "production"
  Team = "platform-db"
  CostCenter = "engineering"
}

resource "aws_db_parameter_group" "postgres16_prod" {
  name = "postgres16-prod"
  family = "postgres16"

  parameter {
    name = "shared_buffers"
    value = "{DBInstanceClassMemory/4}" # 25% of instance memory
  }

  parameter {
    name = "effective_cache_size"
    value = "{DBInstanceClassMemory*3/4}"
  }

  parameter {
    name = "max_connections"
    value = "500"
  }

  parameter {
    name = "log_min_duration_statement"
    value = "1000" # Log queries over 1 second
  }

  parameter {
    name = "log_checkpoints"
    value = "1"
  }

  parameter {
    name = "log_lock_waits"

```

```
    value = "1"
  }

  parameter {
    name = "track_io_timing"
    value = "1"
  }

  parameter {
    name = "pg_stat_statements.track"
    value = "all"
  }

  parameter {
    name = "rds.log_retention_period"
    value = "10080" # 7 days in minutes
  }
}
```

IOPS math for production sizing: On gp3, the baseline 3,000 IOPS costs nothing extra, and each additional IOPS above that baseline is billed incrementally — check current AWS pricing for the exact rate, since it changes. Provisioned IOPS on io1/io2 are billed per-IOPS-month at a materially higher rate than gp3’s incremental IOPS pricing, which is why gp3 is the default recommendation for most workloads that don’t need io2’s higher ceiling or durability.

Monitoring storage saturation in CloudWatch:

```
# Check storage I/O metrics for an RDS instance
# ReadIOPS and WriteIOPS tell you current throughput
# ReadLatency and WriteLatency tell you if storage is saturating

aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name ReadLatency \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-primary \
  --start-time 2024-01-15T09:00:00Z \
  --end-time 2024-01-15T10:00:00Z \
  --period 60 \
  --statistics Average Maximum \
  --output table

# DiskQueueDepth > 1 consistently indicates storage saturation
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name DiskQueueDepth \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-primary \
  --start-time 2024-01-15T09:00:00Z \
  --end-time 2024-01-15T10:00:00Z \
  --period 60 \
  --statistics Average Maximum \
  --output table
```

If `DiskQueueDepth` averages above 1 and `WriteLatency` exceeds 10ms consistently, your provisioned IOPS are insufficient. Increase provisioned IOPS without downtime on gp3 — the change is applied immediately (though it takes up to 6 hours for the modification to complete depending on volume size).

The storage full emergency: RDS will put your database in read-only mode when storage reaches 100%. At that point, you cannot write data including WAL. The sequence of events is: free storage drops → autoscaling triggers (if enabled) → scaling takes 5–15 minutes → if autoscaling is not enabled or is at ceiling, the database goes read-only. Set a CloudWatch alarm at 20% free storage, not 10%, so you have time to respond before hitting autoscaling or the ceiling.

6.4 Parameter Groups: The Replacement for postgresql.conf

On a self-managed PostgreSQL instance, you edit `/etc/postgresql/16/main/postgresql.conf` directly and reload with `SELECT pg_reload_conf()` or restart the service. On RDS, that file is read-only. All configuration changes go through Parameter Groups — a named collection of configuration values that you associate with one or more DB instances.

There are two types: DB Parameter Groups (instance-level) and DB Cluster Parameter Groups (Aurora cluster-level). For non-Aurora RDS, you use DB Parameter Groups.

A parameter group has a family (e.g., `postgres16`, `mysql8.0`, `sqlserver-se-15.0`) and contains every configurable parameter for that engine version. AWS ships a `default.postgres16` parameter group that contains AWS-optimized defaults. You cannot modify the default group — you must create a custom one and attach it to your instance.

Parameters have two apply types: - **Static**: Requires a database restart to take effect. Examples: `shared_buffers`, `max_connections`, `wal_level`. - **Dynamic**: Takes effect immediately on `apply_immediately` or after the maintenance window. Examples: `log_min_duration_statement`, `effective_cache_size`, `work_mem`.

Common PostgreSQL parameter customizations for RDS:

```
# Create a custom parameter group and configure key parameters
aws rds create-db-parameter-group \
  --db-parameter-group-name postgres16-prod \
  --db-parameter-group-family postgres16 \
  --description "Production PostgreSQL 16 parameters" \
  --region us-east-1

# Apply multiple parameters in one call
aws rds modify-db-parameter-group \
  --db-parameter-group-name postgres16-prod \
  --parameters \
    "ParameterName=log_min_duration_statement,ParameterValue=1000,ApplyMethod=immediate" \
    "ParameterName=log_checkpoints,ParameterValue=1,ApplyMethod=immediate" \
    "ParameterName=log_lock_waits,ParameterValue=1,ApplyMethod=immediate" \
    "ParameterName=track_io_timing,ParameterValue=1,ApplyMethod=immediate" \
    "ParameterName=work_mem,ParameterValue=16384,ApplyMethod=immediate" \
    "ParameterName=maintenance_work_mem,ParameterValue=262144,ApplyMethod=immediate" \
    "ParameterName=checkpoint_completion_target,ParameterValue=0.9,ApplyMethod=immediate" \
    "ParameterName=random_page_cost,ParameterValue=1.1,ApplyMethod=immediate" \
  --region us-east-1

# Static parameters require pending reboot - apply these separately
aws rds modify-db-parameter-group \
  --db-parameter-group-name postgres16-prod \
  --parameters \
    "ParameterName=max_connections,ParameterValue=500,ApplyMethod=pending-reboot" \
    "ParameterName=wal_level,ParameterValue=logical,ApplyMethod=pending-reboot" \
    "ParameterName=max_replication_slots,ParameterValue=10,ApplyMethod=pending-reboot" \
    "ParameterName=max_wal_senders,ParameterValue=10,ApplyMethod=pending-reboot" \
  --region us-east-1

# Apply the parameter group to the instance (takes effect after maintenance window
# unless you use --apply-immediately, which causes a brief interruption)
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-primary \
  --db-parameter-group-name postgres16-prod \
  --region us-east-1
```

Verifying parameter group status: After applying a parameter group, check whether the instance requires a reboot for any parameters to take effect:

```
# Check for pending reboot parameters
aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-primary \
  --query 'DBInstances[0].DBParameterGroups' \
  --region us-east-1
```

The response includes a `ParameterApplyStatus` field. A value of `pending-reboot` means at least one static parameter has been changed and the instance must be restarted before it takes effect. A value of `in-sync` means all parameters are active.

You can also verify parameter values from inside PostgreSQL after applying:

```
-- PostgreSQL: Check current effective values of key parameters
SELECT name, setting, unit, context, source
FROM pg_settings
WHERE name IN (
  'shared_buffers',
  'work_mem',
  'maintenance_work_mem',
  'max_connections',
  'effective_cache_size',
  'checkpoint_completion_target',
  'random_page_cost',
  'wal_level',
  'track_io_timing',
  'log_min_duration_statement'
)
ORDER BY name;

-- Check which parameters are still using default values vs. customized
SELECT name, setting, boot_val, reset_val, source
FROM pg_settings
WHERE source != 'default'
  AND source != 'override'
ORDER BY name;
```

For SQL Server on RDS, parameter groups work somewhat differently. SQL Server exposes fewer runtime-adjustable parameters through the RDS parameter group interface than PostgreSQL does. Many SQL Server settings that you'd configure via `sp_configure` on a self-managed instance are either fixed by RDS, exposed as parameter group settings, or must be set at the database level.

SQL Server parameter group equivalent — key settings:

```
-- SQL Server: Check current sp_configure settings on RDS
-- (These reflect what RDS has applied from the parameter group)
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;

EXEC sp_configure;
GO

-- Check settings that affect query performance
SELECT
  name,
  value,
  value_in_use,
  description
FROM sys.configurations
WHERE name IN (
  'max degree of parallelism',
  'cost threshold for parallelism',
  'max server memory (MB)',
  'min server memory (MB)',
  'optimize for ad hoc workloads',
  'remote query timeout (s)'
)
ORDER BY name;
```

Parameter group gotchas:

1. **shared_buffers uses a formula syntax on RDS.** You cannot set it to a raw byte value like 8GB. Instead, RDS uses the expression `{DBInstanceClassMemory/4}` which resolves at runtime to 25% of instance memory. This is actually better than a hardcoded value because when you resize the instance, the buffer pool scales automatically.
2. **max_connections is hardcoded per instance class as a default.** AWS sets conservative defaults (roughly 1 connection per 9.6 MB of RAM for PostgreSQL). On a `db.r7g.4xlarge` with 128 GB RAM, the default is around 5,000 connections — far more than any application should open directly. If you're running PgBouncer or RDS Proxy, reduce this to 500–1000 to limit the overhead of idle connection memory.
3. **log_min_duration_statement is one of the most impactful parameters to set immediately.** The default is `-1` (disabled). Set it to 1000 (milliseconds) in production and 100 in QA. Slow query logging is how you identify query performance regressions during deployments.
4. **Parameter group changes do not propagate to read replicas automatically.** Each read replica has its own parameter group association. If you want replicas to use the same configuration, create a common parameter group and attach it to each instance individually.

6.5 Multi-AZ, Read Replicas, and High Availability Architecture

RDS offers two distinct HA constructs that serve different purposes: Multi-AZ deployments and Read Replicas. They are often conflated by engineers new to RDS, and using them incorrectly produces systems that are either more expensive than necessary or less resilient than assumed.

Multi-AZ: Synchronous standby for failover

In a Multi-AZ deployment, RDS maintains a synchronous standby replica in a different Availability Zone. Every write to the primary is synchronously replicated to the standby before the write is acknowledged to the application. This means:

- RPO is effectively zero (no committed transactions are lost on failover).
- RTO is 60–120 seconds for automatic failover (DNS propagation + database promotion).
- The standby cannot serve reads. It exists solely for failover.
- You pay for two instances but get one connection endpoint.

The Multi-AZ standby is completely transparent to applications — they connect to the instance endpoint (e.g., `prod-postgres-primary.abc123.us-east-1.rds.amazonaws.com`), and that DNS record is automatically updated to point to the new primary after failover.

Multi-AZ with two readable standbys (RDS Multi-AZ Cluster): AWS introduced a newer Multi-AZ option called Multi-AZ Cluster that provides two readable standby instances instead of one non-readable standby. This gives you read scalability built into the HA tier. It's available for MySQL 8.0 and PostgreSQL 13+. The cluster endpoint handles writes; reader endpoints handle reads. This is distinct from Aurora's multi-AZ model (covered in Chapter 7), and is worth considering for PostgreSQL workloads that need both HA and read scaling without the Aurora pricing model.

RDS Multi-AZ Cluster configuration:

```
# Create a Multi-AZ Cluster for PostgreSQL (not a standard Multi-AZ instance)
aws rds create-db-cluster \
  --db-cluster-identifier prod-postgres-cluster \
  --engine postgres \
  --engine-version 16.3 \
  --db-cluster-instance-class db.r7g.4xlarge \
  --master-username dbadmin \
  --master-user-password "${DB_PASSWORD}" \
  --storage-type io1 \
  --iops 12000 \
  --allocated-storage 500 \
  --backup-retention-period 14 \
  --vpc-security-group-ids sg-0abc123def456789 \
  --db-subnet-group-name private-db-subnets \
  --region us-east-1
```

Read Replicas: Asynchronous copies for read scaling and cross-region DR

Read Replicas use asynchronous replication. The primary does not wait for the replica to confirm a write. This means:

- Replicas have some replication lag (usually sub-second to a few seconds on healthy systems, potentially minutes under heavy write load).
- Replicas can serve reads, offloading analytical queries from the primary.
- Replicas can be in different AWS regions (cross-region read replicas), which provides geographic DR.
- A Read Replica can be promoted to a standalone primary — the promotion takes 5–15 minutes and creates a new independent instance.

Read Replicas are not a substitute for Multi-AZ on the primary. If your primary is Single-AZ and you have five Read Replicas, you still have a single point of failure on the primary. The correct production architecture is Multi-AZ primary with Read Replicas for read scaling.

Creating Read Replicas:

```
# Create a read replica in the same region
aws rds create-db-instance-read-replica \
  --db-instance-identifier prod-postgres-replica-01 \
  --source-db-instance-identifier prod-postgres-primary \
  --db-instance-class db.r7g.2xlarge \
  --availability-zone us-east-1b \
  --storage-type gp3 \
  --publicly-accessible false \
  --region us-east-1

# Create a cross-region read replica (for DR or regional read scaling)
# Run this in the target region
aws rds create-db-instance-read-replica \
  --db-instance-identifier prod-postgres-replica-usw2 \
  --source-db-instance-identifier arn:aws:rds:us-east-1:123456789012:db:prod-postgres-primary \
  --db-instance-class db.r7g.2xlarge \
  --storage-type gp3 \
  --region us-west-2
```

Monitoring replication lag:

```
# CloudWatch metric for replication lag (in seconds)
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name ReplicaLag \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-replica-01 \
  --start-time 2024-01-15T00:00:00Z \
  --end-time 2024-01-15T23:59:59Z \
```

```

--period 300 \
--statistics Average Maximum \
--output table

-- PostgreSQL: Check replication lag from within the primary
SELECT
    client_addr,
    state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag_bytes,
    write_lag,
    flush_lag,
    replay_lag
FROM pg_stat_replication
ORDER BY replay_lag DESC NULLS LAST;

-- From within the replica: check how far behind primary
SELECT
    now() - pg_last_xact_replay_timestamp() AS replication_lag,
    pg_is_in_recovery() AS is_replica,
    pg_last_wal_receive_lsn() AS received_lsn,
    pg_last_wal_replay_lsn() AS replayed_lsn,
    pg_wal_lsn_diff(
        pg_last_wal_receive_lsn(),
        pg_last_wal_replay_lsn()
    ) AS lag_bytes;

```

RDS Proxy: Connection pooling as a managed service

RDS Proxy sits between your application and the RDS instance and multiplexes database connections. It maintains a warm pool of connections to the database, allowing thousands of application connections to share a smaller number of actual database connections. This is most impactful for:

- Serverless workloads (Lambda functions that open and close connections on every invocation)
- Applications using a connection-per-request pattern
- Instances where `max_connections` is a bottleneck

RDS Proxy also provides a secondary benefit: during Multi-AZ failover, the proxy handles connection re-routing, reducing application-visible downtime from 60–120 seconds to as low as a few seconds.

```

# Create an RDS Proxy for the PostgreSQL instance
aws rds create-db-proxy \
    --db-proxy-name prod-postgres-proxy \
    --engine-family POSTGRES \
    --auth '[{
        "AuthScheme": "SECRETS",
        "SecretArn": "arn:aws:secretsmanager:us-east-1:123456789012:secret:prod/postgres/dbadmin",
        "IAMAuth": "DISABLED"
    }]' \
    --role-arn arn:aws:iam::123456789012:role/rds-proxy-role \
    --vpc-subnet-ids subnet-0abc123 subnet-0def456 subnet-0ghi789 \
    --vpc-security-group-ids sg-0proxy123 \
    --require-tls \
    --region us-east-1

# Register the target (point proxy at the RDS instance)
aws rds register-db-proxy-targets \
    --db-proxy-name prod-postgres-proxy \
    --db-instance-identifiers prod-postgres-primary \
    --region us-east-1

```

RDS Proxy has one significant limitation for PostgreSQL: it does not support all PostgreSQL

authentication methods and it does not support all connection parameters. Specifically, it does not support `SET SESSION AUTHORIZATION`, advisory locks that depend on connection identity, or `LISTEN/NOTIFY`. If your application relies on any of these, test thoroughly before putting Proxy in front of a production PostgreSQL instance.

6.6 Backup, Restore, and PITR

RDS automated backups use EBS volume snapshots plus WAL archiving to enable point-in-time recovery. Understanding the mechanics matters because the recovery behavior has important implications for your RTO.

How automated backups work:

1. During the backup window, RDS takes an EBS snapshot of the storage volume. For Multi-AZ, the snapshot is taken from the standby to avoid I/O impact on the primary.
2. Transaction logs (WAL for PostgreSQL, binary logs for MySQL) are continuously uploaded to S3 every 5 minutes.
3. PITR works by restoring the nearest snapshot before the target time, then replaying logs forward to the exact target timestamp.

Retention period: Backups are retained for 0–35 days. Setting retention to 0 disables automated backups (and PITR). Never set retention to 0 on a production instance. The standard retention for production is 14 days; compliance-heavy environments often require 35 days.

Manual snapshots are independent of the retention period. They persist until you explicitly delete them. Before any major change (schema migration, engine upgrade, instance resizing), take a manual snapshot:

```
# Take a manual snapshot before a risky operation
aws rds create-db-snapshot \
  --db-instance-identifier prod-postgres-primary \
  --db-snapshot-identifier prod-postgres-before-migration-20240115 \
  --region us-east-1

# Wait for the snapshot to complete
aws rds wait db-snapshot-completed \
  --db-snapshot-identifier prod-postgres-before-migration-20240115 \
  --region us-east-1

echo "Snapshot complete - safe to proceed with migration"
```

Point-in-time restore: PITR creates a new RDS instance restored to the specified timestamp. It does not restore in-place — you always get a new instance. This means you must update your application’s connection string after a PITR recovery.

```
# Restore to a specific point in time (creates a NEW instance)
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier prod-postgres-primary \
  --target-db-instance-identifier prod-postgres-recovered-20240115 \
  --restore-time 2024-01-15T08:30:00Z \
  --db-instance-class db.r7g.4xlarge \
  --storage-type gp3 \
  --multi-az \
  --region us-east-1

# Monitor restoration progress
aws rds describe-db-instances \
```

```
--db-instance-identifier prod-postgres-recovered-20240115 \  
--query 'DBInstances[0].[DBInstanceStatus,PercentProgress]' \  
--output table \  
--region us-east-1
```

The PITR recovery window: The earliest restorable time is limited by the oldest retained backup snapshot. You can check the exact recovery window:

```
# Check the earliest and latest restorable times  
aws rds describe-db-instances \  
--db-instance-identifier prod-postgres-primary \  
--query 'DBInstances[0].{  
    EarliestRestorableTime: EarliestRestorableTime,  
    LatestRestorableTime: LatestRestorableTime,  
    BackupRetentionPeriod: BackupRetentionPeriod  
}' \  
--output table \  
--region us-east-1
```

Restoring a snapshot to a different instance class or storage type: This is useful when you want to test a configuration change. Restore a production snapshot to a smaller instance in a non-production environment, validate, and then apply the same change to production.

```
# Restore from a manual snapshot to a different (cheaper) instance class  
# for testing or data recovery purposes  
aws rds restore-db-instance-from-db-snapshot \  
--db-instance-identifier staging-postgres-from-prod-snap \  
--db-snapshot-identifier prod-postgres-before-migration-20240115 \  
--db-instance-class db.r7g.xlarge \  
--storage-type gp3 \  
--no-multi-az \  
--no-publicly-accessible \  
--vpc-security-group-ids sg-0staging123 \  
--db-subnet-group-name staging-db-subnets \  
--region us-east-1
```

Cross-region backup copies: Automated backups do not automatically copy to other regions. For DR, you need to either configure cross-region automated backup copy or copy manual snapshots. AWS recently added automated cross-region backup to RDS, but manual snapshot copy remains the most flexible approach:

```
# Copy a manual snapshot to another region for DR  
aws rds copy-db-snapshot \  
--source-db-snapshot-identifier \  
    arn:aws:rds:us-east-1:123456789012:snapshot:prod-postgres-before-migration-20240115 \  
--target-db-snapshot-identifier prod-postgres-before-migration-20240115-copy \  
--kms-key-id arn:aws:kms:us-west-2:123456789012:key/mrk-abc123 \  
--region us-west-2
```

Note that cross-region snapshot copies transfer data across AWS backbone networks and incur data transfer costs. For large databases, automate periodic snapshot copies rather than copying every snapshot.

6.7 Upgrade Paths and Maintenance Windows

RDS engine upgrades come in two types: minor version upgrades (e.g., 16.2 → 16.3) and major version upgrades (e.g., 15 → 16). The operational complexity and risk differ significantly.

Minor version upgrades: - Applied automatically if `AutoMinorVersionUpgrade` is enabled, during the maintenance window. - Can be applied manually at any time. - For Multi-AZ: zero-

downtime upgrade (standby upgraded first, then failover, then original primary upgraded). - For Single-AZ: brief downtime (~5 minutes) during the upgrade. - Generally safe with no application changes required.

Major version upgrades: - Always require explicit action — no automatic major version upgrades. - Require a compatible parameter group for the new version (you must create one in advance). - In-place major version upgrades on RDS work for some engine combinations (PostgreSQL 14 → 15 → 16 is supported) but not all. - For PostgreSQL, in-place upgrade uses `pg_upgrade` underneath. This takes a snapshot first, runs the upgrade, and if it fails, rolls back to the snapshot. - Estimated downtime: 10–60 minutes depending on database size and version gap.

Pre-upgrade checklist for PostgreSQL major version upgrade:

```
-- Run these on the CURRENT version before upgrading
-- to identify potential incompatibilities

-- Check for deprecated settings that changed between versions
SELECT name, setting
FROM pg_settings
WHERE name IN (
    'operator_precedence_warning', -- removed in PG14
    'vacuum_cleanup_index_scale_factor', -- removed in PG14
    'stats_temp_directory' -- removed in PG15
);

-- Check for tables using deprecated data types or features
SELECT
    n.nspname AS schema,
    c.relname AS table_name,
    a.attname AS column_name,
    t.typname AS data_type
FROM pg_attribute a
JOIN pg_class c ON a.attrelid = c.oid
JOIN pg_type t ON a.atttypid = t.oid
JOIN pg_namespace n ON c.relnamespace = n.oid
WHERE t.typname IN ('abstime', 'reltime', 'tinterval') -- removed in PG12
    AND c.relkind = 'r'
    AND a.attnum > 0;

-- Check for extensions that may not be available in the target version
SELECT name, default_version, installed_version
FROM pg_available_extensions
WHERE installed_version IS NOT NULL
ORDER BY name;

-- Check for any invalid indexes (should be zero before upgrade)
SELECT schemaname, tablename, indexname
FROM pg_indexes
WHERE NOT EXISTS (
    SELECT 1 FROM pg_class c
    JOIN pg_index i ON i.indexrelid = c.oid
    WHERE c.relname = indexname
    AND pg_index_column_has_property(i.indexrelid, 1, 'returnable') IS NOT NULL
);
```

Performing a major version upgrade with pre-upgrade snapshot:

```
# Step 1: Take a manual snapshot before upgrading
aws rds create-db-snapshot \
    --db-instance-identifier prod-postgres-primary \
    --db-snapshot-identifier pre-upgrade-pg15-to-pg16-$(date +%Y%m%d) \
    --region us-east-1

aws rds wait db-snapshot-completed \
    --db-snapshot-identifier pre-upgrade-pg15-to-pg16-$(date +%Y%m%d) \
    --region us-east-1
```

```
# Step 2: Create the parameter group for the new major version FIRST
aws rds create-db-parameter-group \
  --db-parameter-group-name postgres16-prod \
  --db-parameter-group-family postgres16 \
  --description "Production PostgreSQL 16 parameters" \
  --region us-east-1

# Re-apply all your custom parameters to the new group (use the same
# modify-db-parameter-group commands from section 6.4)

# Step 3: Initiate the upgrade
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-primary \
  --engine-version 16.3 \
  --db-parameter-group-name postgres16-prod \
  --allow-major-version-upgrade \
  --apply-immediately \
  --region us-east-1

# Step 4: Monitor upgrade progress
watch -n 30 'aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-primary \
  --query "DBInstances[0].[DBInstanceStatus,EngineVersion,PendingModifiedValues]" \
  --output table \
  --region us-east-1'
```

Maintenance windows deserve careful thought in production. The default maintenance window is a 30-minute window during off-peak hours in the instance's region. AWS applies minor patches, security updates, and approved configuration changes during this window. For global applications with no true off-peak window, schedule maintenance during the lowest-traffic window you have, and ensure your application handles connection drops gracefully (retry logic, connection pool reconnect on error).

```
# Update the maintenance window to Sunday 2am-3am UTC
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-primary \
  --preferred-maintenance-window "sun:02:00-sun:03:00" \
  --region us-east-1

# Check pending maintenance actions (AWS may schedule security patches)
aws rds describe-pending-maintenance-actions \
  --region us-east-1 \
  --query 'PendingMaintenanceActions[*].{
    ResourceIdentifier: ResourceIdentifier,
    Action: PendingMaintenanceActionDetails[0].Action,
    AutoAppliedAfterDate: PendingMaintenanceActionDetails[0].AutoAppliedAfterDate,
    Description: PendingMaintenanceActionDetails[0].Description
  }' \
  --output table
```

If AWS schedules a forced patch with an `AutoAppliedAfterDate`, that patch will apply automatically after that date regardless of your maintenance window preference. Monitor this endpoint and apply patches proactively before AWS forces them, so you control the timing.

6.8 RDS Operational Monitoring and Alerting

The monitoring stack for RDS has three tiers: CloudWatch Basic Metrics (free, 1-minute granularity), Enhanced Monitoring (paid, 1–60 second granularity, OS-level metrics), and Performance Insights (paid, query-level attribution).

CloudWatch Basic Metrics include CPU utilization, database connections, free storage, read/write IOPS, read/write latency, replica lag, and network throughput. These are sufficient for capacity trend monitoring but insufficient for diagnosing query-level performance issues.

Enhanced Monitoring adds OS-level metrics that the basic metrics don't expose: per-process CPU, memory by component (RDS process, OS, file system cache), and disk I/O broken down by read/write and user/kernel. Enhanced Monitoring sends data to CloudWatch Logs rather than CloudWatch Metrics, which means you query it differently:

```
# Enhanced Monitoring is in the RDSOSMetrics log group
# Get the latest OS metrics for an instance
aws logs filter-log-events \
  --log-group-name "RDSOSMetrics" \
  --log-stream-name-prefix "prod-postgres-primary" \
  --start-time $(date -d '5 minutes ago' +%s000) \
  --limit 5 \
  --query 'events[*].message' \
  --output text \
  --region us-east-1
```

The output is JSON with detailed OS metrics. Parse it with `jq` for usable output:

```
aws logs filter-log-events \
  --log-group-name "RDSOSMetrics" \
  --log-stream-name-prefix "prod-postgres-primary" \
  --start-time $(date -d '5 minutes ago' +%s000) \
  --limit 1 \
  --query 'events[0].message' \
  --output text \
  --region us-east-1 | \
jq '{
  timestamp: .timestamp,
  cpu_idle: .cpuUtilization.idle,
  cpu_user: .cpuUtilization.user,
  memory_free_mb: (.memory.free / 1024 / 1024),
  disk_read_iops: .diskIO[0].readIOPS,
  disk_write_iops: .diskIO[0].writeIOPS,
  load_average_1m: .loadAverageMinute.one
}'
```

Performance Insights is the most powerful diagnostic tool for RDS. It collects query-level wait event data and lets you answer “what was the database doing at 2:34am when latency spiked?” It shows the database load broken down by wait event, SQL statement, user, host, or application. For PostgreSQL, it leverages `pg_stat_activity` and `pg_stat_statements` internally.

Set Performance Insights retention to at least 7 days (the free tier) and ideally to 731 days (2 years, paid) for production systems where you need to compare behavior across deployments and seasonal traffic patterns.

Essential CloudWatch alarms for any production RDS instance:

```
# Alarm: CPU utilization above 80% for 10 minutes
aws cloudwatch put-metric-alarm \
  --alarm-name rds-prod-postgres-cpu-high \
  --metric-name CPUUtilization \
  --namespace AWS/RDS \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-primary \
  --statistic Average \
  --period 300 \
  --evaluation-periods 2 \
  --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:db-alerts \
  --region us-east-1
```

```

# Alarm: Free storage below 20% (500 GB instance -> alarm at 100 GB free)
aws cloudwatch put-metric-alarm \
  --alarm-name rds-prod-postgres-low-storage \
  --metric-name FreeStorageSpace \
  --namespace AWS/RDS \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-primary \
  --statistic Average \
  --period 300 \
  --evaluation-periods 1 \
  --threshold 107374182400 \
  --comparison-operator LessThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:db-alerts \
  --region us-east-1

# Alarm: Replica lag above 30 seconds
aws cloudwatch put-metric-alarm \
  --alarm-name rds-prod-postgres-replica-lag \
  --metric-name ReplicaLag \
  --namespace AWS/RDS \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-replica-01 \
  --statistic Maximum \
  --period 60 \
  --evaluation-periods 3 \
  --threshold 30 \
  --comparison-operator GreaterThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:db-alerts \
  --region us-east-1

# Alarm: Database connections above 400 (of 500 max)
aws cloudwatch put-metric-alarm \
  --alarm-name rds-prod-postgres-connections-high \
  --metric-name DatabaseConnections \
  --namespace AWS/RDS \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-primary \
  --statistic Maximum \
  --period 60 \
  --evaluation-periods 2 \
  --threshold 400 \
  --comparison-operator GreaterThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:db-alerts \
  --region us-east-1

```

Querying Performance Insights via API for automated reporting:

```

# Get top SQL by average active sessions (last hour)
aws pi get-resource-metrics \
  --service-type RDS \
  --identifier db-ABCDEFGHIJ1234567 \
  --start-time $(date -d '1 hour ago' --iso-8601=seconds) \
  --end-time $(date --iso-8601=seconds) \
  --period-in-seconds 3600 \
  --metric-queries '[
    {
      "Metric": "db.load.avg",
      "GroupBy": {
        "Group": "db.sql",
        "Limit": 10
      }
    }
  ]' \
  --region us-east-1

```

The `identifier` in Performance Insights API calls is the DBI resource ID, not the instance name. Get it with:

```

aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-primary \
  --query 'DBInstances[0].DbiResourceId' \
  --region us-east-1

```

Key Takeaways

- **Engine version and licensing choices are harder to reverse than they appear.** Pin minor versions in production, stay within one major version of current, and understand the SQL Server and Oracle licensing models before committing to an instance size and edition — the hourly cost difference between editions on large instances is substantial.
 - **Instance family selection matters more than raw size.** Graviton (g-suffix) instances offer better price-performance for PostgreSQL and MySQL, while SQL Server and Oracle require Intel or AMD families. Never use T-series burstable instances for production OLTP workloads that sustain CPU pressure — CPU credit exhaustion causes silent, unpredictable throttling.
 - **gp3 storage is the correct default for nearly all new RDS deployments.** It decouples IOPS and throughput from storage size, eliminating the gp2 credit model at lower cost. Monitor `DiskQueueDepth` and `WriteLatency` to detect storage saturation; provision additional IOPS on gp3 before reaching the throughput ceiling rather than after observing latency spikes.
 - **Multi-AZ and Read Replicas solve different problems and are not interchangeable.** Multi-AZ provides synchronous failover with near-zero RPO but the standby cannot serve reads. Read Replicas provide asynchronous read scaling and cross-region DR but have replication lag and do not auto-failover. Production architectures typically need both: a Multi-AZ primary with Read Replicas for workload offloading.
 - **RDS backup and restore behavior requires architectural awareness.** PITR always produces a new instance — recovery requires updating application connection strings, not just waiting for an in-place restore. Automated backups do not copy cross-region by default. Take manual snapshots before any major change, verify the earliest restorable time matches your RPO requirements, and set CloudWatch alarms at 20% free storage rather than relying solely on autoscaling.
-

Chapter 7: Aurora PostgreSQL vs RDS PostgreSQL: When to Choose Which

Aurora PostgreSQL and RDS PostgreSQL both run PostgreSQL, both live inside your VPC, and both appear in the same AWS console. That surface-level similarity trips up engineers who assume they're interchangeable and then build the wrong architecture for their workload. They are not the same product. They have different storage engines, different replication topologies, different failover mechanics, different cost profiles, and different ceilings. This chapter gives you the framework to make the right call before you provision — not after you've already migrated 3 TB of data and discovered that Aurora's distributed storage doesn't behave the way you expected under write-heavy OLTP load.

The Architectural Divide: What's Actually Different Under the Hood

If you've run PostgreSQL on bare metal or on VMware, your mental model looks like this: a primary server writes to local disk using the PostgreSQL WAL mechanism, standby servers replay that WAL, and storage is something you manage at the OS layer — LVM, ZFS, EBS volumes, SAN LUNs. Both Aurora and RDS start by dismantling that model, but they do it in different ways.

RDS PostgreSQL is the more conservative option. AWS provisions an EC2 instance, attaches EBS storage to it, and runs stock PostgreSQL on top. The PostgreSQL version you get is a genuine community PostgreSQL build, with minor AWS patches for authentication and monitoring. Multi-AZ standby is implemented using synchronous physical replication, identical to what you'd configure manually with `synchronous_commit = on` and a hot standby. When the primary fails, AWS promotes the standby — AWS states standard RDS Multi-AZ failover typically completes within 60–120 seconds, depending on instance size and database load. The storage is EBS gp3 or io2 volumes. You manage storage capacity. You choose your IOPS tier. You pay for provisioned storage whether you use it or not.

Aurora PostgreSQL replaces this entire I/O path. Aurora separates compute from storage entirely. Your PostgreSQL process doesn't write to local EBS; it writes log records across Aurora's distributed storage layer — six copies across three Availability Zones, maintained by a purpose-built storage service that handles its own quorum writes. The PostgreSQL engine on the primary only

needs to confirm four of six storage nodes acknowledged the write. The standby instances don't replay WAL from the primary at all — they share the same storage layer and simply maintain their buffer caches. This is why Aurora can promote a read replica to primary in under 30 seconds without any data copy or log replay.

The practical consequences of this architecture:

- **Write amplification:** Aurora writes six copies of every log record at the storage layer. For write-heavy workloads with small transactions, this storage overhead is real. On large instances pushing tens of thousands of writes per second, you'll see higher per-write latency than equivalent RDS PostgreSQL on io2.
- **Read scaling:** Aurora read replicas share storage, so they're always in sync to within single-digit milliseconds. RDS read replicas replicate asynchronously over network WAL streaming and routinely lag 5–30 seconds under heavy write load.
- **Storage billing:** Aurora bills for storage actually consumed, not provisioned — check current AWS pricing for the per-GB-month rate. It grows automatically to 128 TB. RDS charges for provisioned storage whether you use it or not, and you must manage size manually or configure storage autoscaling.

Understanding which of these trade-offs matters for your workload is the entire decision.

When RDS PostgreSQL Is the Right Answer

RDS PostgreSQL wins in a specific class of workload: write-intensive, cost-sensitive, or requiring strict community PostgreSQL compatibility.

Write-heavy OLTP with high transaction rates. Aurora's distributed write path — writing to six storage copies across three AZs and waiting for a quorum of acknowledgments — introduces additional per-write latency that a local EBS write path does not have. For workloads that are heavily write-bound and sensitive to commit latency — logging pipelines, high-frequency inserts, heavy UPDATE/DELETE churn — it is worth benchmarking RDS PostgreSQL on io2 against Aurora with your actual workload rather than assuming one is faster; RDS PostgreSQL on io2 frequently comes out ahead on raw commit latency for this class of workload.

Strict PostgreSQL version parity requirements. Aurora PostgreSQL has historically lagged behind community PostgreSQL releases, sometimes by many months. At the time of writing, Aurora PostgreSQL 16.x is available, but specific minor versions and extensions you rely on may not be. RDS PostgreSQL tracks community releases faster. If your application requires a specific extension version, a specific PostgreSQL minor version, or a recently released PostgreSQL feature, check Aurora's compatibility matrix before assuming it's available. Extensions like `pg_partman`, `TimescaleDB`, `pgvector`, and `PostGIS` have different version constraints on Aurora versus RDS.

Single-tenant or small databases with predictable storage. For databases under 100 GB with well-understood growth, Aurora's storage pricing provides no advantage, and the additional per-request storage cost can exceed the RDS provisioned storage cost. A 50 GB database on `db.t4g.medium` running RDS PostgreSQL with `gp3` storage is meaningfully cheaper per month than the same database on Aurora.

Cost modeling matters here. As a rule of thumb, Aurora PostgreSQL on a given instance

class typically costs more per month than the equivalent RDS PostgreSQL configuration, because Aurora storage bills separately for I/O requests on top of storage consumed. Under high read/write workloads, those I/O request charges can add substantially to the bill — Aurora I/O-Optimized pricing exists specifically to eliminate per-request charges in exchange for a higher flat storage rate, and is worth evaluating once I/O charges become a significant fraction of your Aurora bill. Model both options against current AWS pricing for your actual instance class and storage footprint rather than relying on a fixed dollar comparison.

When Aurora PostgreSQL Is the Right Answer

Aurora earns its higher base cost in several specific scenarios where its architecture genuinely changes what's operationally possible.

Multi-reader workloads requiring near-zero replication lag. If you're running reporting, analytics, or read-heavy application tiers against read replicas, Aurora's shared storage architecture gives you replicas that are current to within 10–20 milliseconds. This isn't tunable in RDS — WAL-based replication lag is a function of network throughput and write rate, and under heavy write load it climbs. Applications that route reads to replicas and expect fresh data need Aurora. RDS replicas are appropriate only when staleness of seconds or tens of seconds is acceptable.

High-availability requirements with sub-30-second failover. RDS Multi-AZ failover involves a DNS cutover after the standby is promoted and confirmed healthy. AWS states this typically takes 60–120 seconds, and large buffer flush situations or high load can push it toward the higher end of that range or beyond. Aurora's failover to a replica — because no storage transfer occurs — completes in under 30 seconds, often under 15. For applications with aggressive availability SLAs (99.99% uptime targets translate to under 52 minutes of annual downtime), Aurora's failover behavior is often the deciding factor.

Aurora Serverless v2 for variable workloads. Aurora Serverless v2 scales compute capacity in increments of 0.5 Aurora Capacity Units (ACUs), where 1 ACU is approximately 2 GB of RAM and proportional CPU. It scales up within seconds and scales down during idle periods. For development environments, batch workloads with spiky demand, or SaaS applications with unpredictable per-tenant load, Serverless v2 eliminates the need to overprovision. RDS has no equivalent — you provision a fixed instance class and it runs at that size regardless of load.

Large storage with automatic growth. Aurora storage grows automatically in 10 GB increments, up to 128 TB, without downtime or manual intervention. If your database routinely grows unpredictably — large time-series tables, append-heavy audit logs, media metadata at scale — Aurora's storage management eliminates a class of operational work that RDS requires (monitoring storage thresholds, triggering resize events, managing replication lag during storage scaling).

Aurora Global Database for cross-region disaster recovery. Aurora Global Database replicates across up to five regions with typical replication lag under one second. You get a cross-region standby that can be promoted in under a minute. The equivalent RDS architecture requires a cross-region read replica, which has higher replication lag, longer promotion time, and more complex DNS management. If cross-region DR with near-zero RPO is a hard requirement, Aurora Global Database is the only RDS-family option that meets it.

Provisioning and Configuration: Hands-On Comparison

Here's how each looks at provisioning time using Terraform and the AWS CLI. Understanding the configuration surface tells you exactly where the operational differences live.

Provisioning RDS PostgreSQL with Multi-AZ:

```
# terraform/rds_postgresql.tf
resource "aws_db_instance" "prod_postgres" {
  identifier      = "prod-postgres-primary"
  engine          = "postgres"
  engine_version  = "16.3"
  instance_class  = "db.r6g.2xlarge"
  allocated_storage = 500
  storage_type    = "gp3"
  iops            = 12000
  storage_throughput = 500

  db_name = "appdb"
  username = "dbadmin"
  password = var.db_password

  multi_az      = true
  db_subnet_group_name = aws_db_subnet_group.private.name
  vpc_security_group_ids = [aws_security_group.rds_sg.id]

  # Parameter group with production tunings
  parameter_group_name = aws_db_parameter_group.postgres16_prod.name

  backup_retention_period = 14
  backup_window           = "03:00-04:00"
  maintenance_window      = "sun:04:00-sun:05:00"
  deletion_protection      = true
  performance_insights_enabled = true
  performance_insights_retention_period = 731 # 2 years

  enabled_cloudwatch_logs_exports = ["postgresql", "upgrade"]

  tags = {
    Environment = "production"
    Team        = "platform-db"
  }
}

resource "aws_db_parameter_group" "postgres16_prod" {
  name = "postgres16-prod"
  family = "postgres16"

  parameter {
    name = "shared_buffers"
    value = "{DBInstanceClassMemory/4}"
  }

  parameter {
    name = "max_connections"
    value = "500"
  }

  parameter {
    name = "work_mem"
    value = "65536" # 64 MB
  }

  parameter {
    name = "checkpoint_completion_target"
    value = "0.9"
  }

  parameter {
```

```
    name = "log_min_duration_statement"
    value = "1000" # Log queries over 1 second
  }

  parameter {
    name = "log_autovacuum_min_duration"
    value = "0" # Log all autovacuum activity
  }
}
```

Provisioning Aurora PostgreSQL with a read replica:

```
# terraform/aurora_postgresql.tf
resource "aws_rds_cluster" "aurora_prod" {
  cluster_identifier      = "aurora-prod-cluster"
  engine                 = "aurora-postgresql"
  engine_version         = "16.2"
  database_name          = "appdb"
  master_username        = "dbadmin"
  master_password        = var.db_password

  db_subnet_group_name   = aws_db_subnet_group.private.name
  vpc_security_group_ids = [aws_security_group.aurora_sg.id]
  db_cluster_parameter_group_name = aws_rds_cluster_parameter_group.aurora16_prod.name

  backup_retention_period = 14
  preferred_backup_window = "03:00-04:00"
  deletion_protection    = true

  # Aurora I/O Optimized eliminates per-request charges
  # Enable this when I/O costs exceed 25% of compute costs
  storage_type = "aurora-iopt1"

  enabled_cloudwatch_logs_exports = ["postgresql"]

  tags = {
    Environment = "production"
    Team        = "platform-db"
  }
}

resource "aws_rds_cluster_instance" "aurora_writer" {
  identifier      = "aurora-prod-writer"
  cluster_identifier = aws_rds_cluster.aurora_prod.id
  instance_class   = "db.r6g.2xlarge"
  engine           = aws_rds_cluster.aurora_prod.engine
  engine_version   = aws_rds_cluster.aurora_prod.engine_version

  performance_insights_enabled      = true
  performance_insights_retention_period = 731
  monitoring_interval               = 60
  monitoring_role_arn               = aws_iam_role.rds_enhanced_monitoring.arn

  db_parameter_group_name = aws_db_parameter_group.aurora16_instance_prod.name
}

resource "aws_rds_cluster_instance" "aurora_readers" {
  count      = 2
  identifier = "aurora-prod-reader-${count.index}"
  cluster_identifier = aws_rds_cluster.aurora_prod.id
  instance_class     = "db.r6g.xlarge" # Readers can be smaller
  engine             = aws_rds_cluster.aurora_prod.engine
  engine_version     = aws_rds_cluster.aurora_prod.engine_version

  # Readers in different AZs for HA
  availability_zone = data.aws_availability_zones.available.names[count.index]

  performance_insights_enabled = true
  db_parameter_group_name      = aws_db_parameter_group.aurora16_instance_prod.name
}
```

```
# Aurora has two-level parameter groups: cluster-level and instance-level
resource "aws_rds_cluster_parameter_group" "aurora16_prod" {
  name     = "aurora16-prod-cluster"
  family   = "aurora-postgresql16"

  parameter {
    name = "shared_preload_libraries"
    value = "pg_stat_statements,auto_explain"
  }

  parameter {
    name = "log_min_duration_statement"
    value = "1000"
  }
}

resource "aws_db_parameter_group" "aurora16_instance_prod" {
  name     = "aurora16-prod-instance"
  family   = "aurora-postgresql16"

  parameter {
    name = "max_connections"
    value = "1000"
  }

  parameter {
    name = "work_mem"
    value = "65536"
  }
}
```

Note the structural difference: Aurora has cluster-level parameter groups (applied to the cluster) and instance-level parameter groups (applied to each instance). RDS has only instance-level parameter groups. This matters because certain parameters — `shared_preload_libraries`, replication settings, and Aurora-specific parameters — must go into the cluster parameter group. Putting them in the instance parameter group has no effect, which is a common mistake when migrating RDS parameter group configurations to Aurora.

Failover, Replication, and Connection Management

Both services handle failover via DNS endpoint updates. Your application connects to the cluster endpoint (for Aurora) or the instance endpoint (for RDS), and AWS updates the DNS record to point at the new primary after failover. This means your application TTL settings matter — if your application caches DNS for 300 seconds, it will not discover the new primary for up to 5 minutes regardless of how fast the failover itself occurs.

For RDS PostgreSQL, the recommended mitigation is RDS Proxy. For Aurora, RDS Proxy is also available and adds connection pooling, but Aurora's own cluster endpoint DNS is designed with a 5-second TTL, which reduces connection failover time significantly without requiring Proxy. Both benefit from Proxy when you have large numbers of application connections (hundreds of Lambda functions, microservices with short-lived connections, or connection counts that approach `max_connections`).

Setting up RDS Proxy for Aurora PostgreSQL:

```
# Create IAM role for RDS Proxy to access Secrets Manager
aws iam create-role \
```



```

--role-name rds-proxy-role \
--assume-role-policy-document '{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Service": "rds.amazonaws.com"},
    "Action": "sts:AssumeRole"
  }]
}'

# Store credentials in Secrets Manager (don't use parameter groups for passwords)
aws secretsmanager create-secret \
  --name prod/aurora/dbadmin \
  --secret-string '{"username":"dbadmin","password":"your-password-here"}'

# Create the RDS Proxy
aws rds create-db-proxy \
  --db-proxy-name aurora-prod-proxy \
  --engine-family POSTGRES \
  --auth '[{
    "AuthScheme": "SECRETS",
    "SecretArn": "arn:aws:secretsmanager:us-east-1:123456789:secret:prod/aurora/dbadmin",
    "IAMAuth": "REQUIRED"
  }]' \
  --role-arn arn:aws:iam::123456789:role/rds-proxy-role \
  --vpc-subnet-ids subnet-aaa subnet-bbb subnet-ccc \
  --vpc-security-group-ids sg-proxy-sg \
  --require-tls

# Register the proxy target - point it at the Aurora cluster
aws rds register-db-proxy-targets \
  --db-proxy-name aurora-prod-proxy \
  --db-cluster-identifiers aurora-prod-cluster

```

With RDS Proxy in front of Aurora, you get connection multiplexing — thousands of application connections can map to a much smaller pool of actual database connections. Proxy is not free: it bills per vCPU-hour of the underlying database instance class — check current AWS pricing to size the added cost for your instance class.

Testing replication lag on Aurora read replicas:

```

-- Run on Aurora read replica to check replication currency
-- Aurora uses aurora_replica_status(), not standard pg_stat_replication
SELECT server_id,
       session_id,
       aws_query_id,
       durable_lsn,
       highest_lsn_rcvd,
       feedback_epoch,
       feedback_xmin,
       replica_lag_in_msec,  -- This is what you care about
       log_stream_speed_in_kib_per_second,
       log_buffer_sequence_number,
       log_buffer_count
FROM aurora_replica_status();

-- For RDS PostgreSQL read replicas, check from the primary:
SELECT client_addr,
       state,
       sent_lsn,
       write_lsn,
       flush_lsn,
       replay_lsn,
       write_lag,
       flush_lag,
       replay_lag
FROM pg_stat_replication;

```

```
-- Set up a replication slot health check for RDS read replicas
-- Check for dangerous slot lag that blocks WAL cleanup:
SELECT slot_name,
       active,
       pg_size_pretty(
         pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)
       ) AS replication_lag_size
FROM pg_replication_slots
WHERE slot_type = 'physical'
ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;
```

One important operational difference: RDS PostgreSQL read replicas have replication slots on the primary. If a replica becomes unreachable and you don't drop its slot, the primary will retain WAL indefinitely, eventually filling your storage. This is a well-known gotcha in self-managed PostgreSQL, and it's equally present in RDS. Aurora does not have this problem — because replicas share storage with the primary, there are no WAL slots to manage. This alone removes a significant class of production incidents.

Monitoring, Limits, and Production Gotchas

The monitoring surfaces are different between the two services, and knowing which metrics actually matter in production prevents misdiagnosis.

CloudWatch metrics that behave differently on Aurora vs RDS:

Metric	RDS PostgreSQL	Aurora PostgreSQL
FreeStorageSpace	Critical — approaches zero as disk fills	Not applicable — Aurora auto-extends
WriteIOPS	Direct EBS write IOPS	Aurora storage layer I/O (different billing unit)
ReplicaLag	Seconds of WAL lag	Milliseconds of replica lag
AuroraVolumeBytesLeftTotal	Not present	Remaining capacity before 128 TB limit
CommitLatency	P99 commit time	P99 commit time (higher baseline on Aurora)

Limits you will hit and need to plan for:

RDS PostgreSQL limits: - Maximum storage: up to 64 TiB, depending on storage type and instance class — check current AWS documentation for the exact ceiling on your configuration - Maximum IOPS: 256,000 (io2 Block Express on supported instance types) - Maximum read replicas: 5 per instance - Cross-region replicas: supported, with WAL streaming lag

Aurora PostgreSQL limits: - Maximum cluster storage: 128 TB - Maximum instances per cluster: 15 (1 writer + 14 readers) - Maximum Aurora Global Database regions: 5 secondary - Aurora Serverless v2 maximum ACUs per instance: AWS has raised this ceiling more than once — check current documentation for the exact value rather than assuming a fixed number - Not all PostgreSQL extensions are available — check the [Aurora PostgreSQL extensions list](#) before migrating

Production gotchas specific to Aurora PostgreSQL:

1. **Temp table storage uses local instance storage, not Aurora distributed storage.** Large sort operations, hash joins, and queries that spill to disk will use instance-local temp space. If your workload involves large analytical queries, the instance class must have sufficient local NVMe or you must tune `work_mem` aggressively enough to keep operations in memory. This catches teams who sized their Aurora instance purely on storage, not RAM.
2. **Vacuum behavior on Aurora is different.** Because Aurora uses a distributed storage model, the vacuum process interacts with storage nodes rather than local disk. In practice this means autovacuum runs normally, but manual `VACUUM FULL` is even more impactful on Aurora — it rewrites the entire table, which causes significant write amplification across all six storage copies. Avoid `VACUUM FULL` on Aurora. Use `pg_repack` if you need to reclaim space and reorganize tables.
3. **Aurora does not support all replication-related PostgreSQL parameters.** Parameters like `wal_level`, `archive_mode`, `archive_command`, and `restore_command` are managed by Aurora and cannot be modified. If you're using logical replication to feed a data warehouse or a CDC pipeline (Debezium, AWS DMS), Aurora supports logical replication, but the setup is different from what you'd do with `postgresql.conf` directly.

```
# Enable logical replication on Aurora via cluster parameter group
# (Cannot be done in postgresql.conf - Aurora manages the file)
aws rds modify-db-cluster-parameter-group \
  --db-cluster-parameter-group-name aurora16-prod-cluster \
  --parameters \
    ParameterName=rds.logical_replication,\
    ParameterValue=1,\
    ApplyMethod=pending-reboot

# Verify the change after reboot
aws rds describe-db-cluster-parameters \
  --db-cluster-parameter-group-name aurora16-prod-cluster \
  --query 'Parameters[?ParameterName==`rds.logical_replication`]'

# Then create a logical replication slot as you normally would
# (This is standard PostgreSQL from this point)
```

4. **Writer instance failure during a write storm.** During the few seconds of Aurora failover, the cluster endpoint DNS updates but existing connections receive connection errors. Applications must implement retry logic with exponential backoff. AWS recommends catching `OperationalError` with error code 08006 (`connection_failure`) or 57P01 (`admin_shutdown`) and retrying. Applications built for RDS that relied on longer Multi-AZ failover windows sometimes fail harder on Aurora because the faster failover catches them mid-transaction without proper error handling. Counterintuitively, faster failover reveals connection handling bugs that longer failover hid.
5. **Aurora Global Database promotion is not automatic.** If your primary region becomes unavailable, you must manually initiate the promoted-region failover in the Aurora console or via CLI. There is no automated global failover trigger. Plan this into your DR runbook and test it at least annually.

```
# Manual Aurora Global Database failover to secondary region
# Step 1: Verify the secondary cluster is healthy
aws rds describe-global-clusters \
  --global-cluster-identifier aurora-prod-global \
  --query 'GlobalClusters[0].GlobalClusterMembers'

# Step 2: Initiate failover to the secondary region
aws rds failover-global-cluster \
  --global-cluster-identifier aurora-prod-global \
```

```
--target-db-cluster-identifier \  
arn:aws:rds:eu-west-1:123456789:cluster:aurora-prod-secondary  
  
# Step 3: After promotion completes, update application endpoints  
# The former secondary is now a standalone writer  
# The former primary, when it recovers, must be re-added as a secondary  
# or you will have a split-brain topology  
  
# Monitor failover status  
aws rds describe-global-clusters \  
--global-cluster-identifier aurora-prod-global \  
--query 'GlobalClusters[0].Status'
```

Migration Path: RDS PostgreSQL to Aurora PostgreSQL

The most common real-world scenario is an existing RDS PostgreSQL instance that needs Aurora's replication or failover characteristics. AWS provides a snapshot-based migration path with minimal downtime.

Option 1: Restore from snapshot (offline migration)

Suitable for lower-traffic applications where you can tolerate a maintenance window of 30–90 minutes.

```
# Step 1: Take a final snapshot of the RDS instance  
aws rds create-db-snapshot \  
--db-instance-identifier prod-postgres-primary \  
--db-snapshot-identifier prod-postgres-pre-migration-$(date +%Y%m%d)  
  
# Step 2: Wait for snapshot completion  
aws rds wait db-snapshot-available \  
--db-snapshot-identifier prod-postgres-pre-migration-$(date +%Y%m%d)  
  
# Step 3: Restore the snapshot as an Aurora cluster  
aws rds restore-db-cluster-from-snapshot \  
--db-cluster-identifier aurora-prod-migrated \  
--snapshot-identifier prod-postgres-pre-migration-$(date +%Y%m%d) \  
--engine aurora-postgresql \  
--engine-version 16.2 \  
--db-cluster-parameter-group-name aurora16-prod-cluster \  
--vpc-security-group-ids sg-aurora-sg \  
--db-subnet-group-name private-subnet-group  
  
# Step 4: Add a writer instance to the new cluster  
aws rds create-db-instance \  
--db-instance-identifier aurora-prod-writer \  
--db-cluster-identifier aurora-prod-migrated \  
--db-instance-class db.r6g.2xlarge \  
--engine aurora-postgresql  
  
# Step 5: Wait for instance to be available  
aws rds wait db-instance-available \  
--db-instance-identifier aurora-prod-writer  
  
# Step 6: Verify data integrity before cutting over  
# Connect to Aurora and run count checks on critical tables  
psql -h aurora-prod-migrated.cluster-xxxx.us-east-1.rds.amazonaws.com \  
-U dbadmin -d appdb \  
-c "SELECT schemaname, tablename, n_live_tup FROM pg_stat_user_tables ORDER BY n_live_tup DESC LIMIT  
↵ 20;"
```

Option 2: Near-zero downtime migration using logical replication

For production systems where a maintenance window is not acceptable, logical replication between the RDS source and Aurora target lets you migrate data incrementally and cut over with seconds of downtime.

```
-- On RDS PostgreSQL source: create a publication
-- Requires wal_level = logical (set via parameter group and reboot first)
CREATE PUBLICATION migration_pub
    FOR ALL TABLES;

-- Verify publication is active
SELECT pubname, puballtables, pubinsert, pubupdate, pubdelete
FROM pg_publication;

# On RDS source: enable logical replication in parameter group if not already done
aws rds modify-db-parameter-group \
    --db-parameter-group-name postgres16-prod \
    --parameters \
        ParameterName=rds.logical_replication,\
        ParameterValue=1,\
        ApplyMethod=pending-reboot

# Reboot is required for this change
aws rds reboot-db-instance \
    --db-instance-identifier prod-postgres-primary

-- On Aurora PostgreSQL target: create the subscription
-- Run this after Aurora cluster is provisioned and schema is copied

-- First, copy the schema (logical replication replicates data, not DDL)
-- Use pg_dump schema-only from source to target before creating subscription
-- pg_dump -h <rds-endpoint> -U dbadmin -d appdb --schema-only | \
--     psql -h <aurora-endpoint> -U dbadmin -d appdb

-- Then create the subscription
CREATE SUBSCRIPTION migration_sub
    CONNECTION 'host=prod-postgres-primary.xxxx.us-east-1.rds.amazonaws.com
                port=5432
                dbname=appdb
                user=replication_user
                password=repl_password'
    PUBLICATION migration_pub;

-- Monitor replication progress
SELECT subname,
       received_lsn,
       latest_end_lsn,
       pg_size_pretty(
           pg_wal_lsn_diff(latest_end_lsn, received_lsn)
       ) AS remaining_lag
FROM pg_stat_subscription;

-- When lag reaches zero (or acceptably low), proceed with cutover:
-- 1. Stop writes to RDS (application maintenance mode or connection block)
-- 2. Confirm lag is zero on Aurora
-- 3. Drop the subscription on Aurora
-- 4. Update application connection strings to Aurora endpoint
-- 5. Restore writes

-- Drop subscription after cutover
DROP SUBSCRIPTION migration_sub;

-- Drop publication on old RDS instance after confirming Aurora is healthy
-- (Connect back to RDS for this)
DROP PUBLICATION migration_pub;
```

Schema migration gotcha: Sequences are not replicated by logical replication by default. After cutover, your sequences on Aurora may be behind the current state — subsequent inserts will conflict on primary key violations. Before enabling writes on Aurora, advance all sequences past

the highest observed value from the source:

```
-- Run on Aurora after cutover, before enabling application writes
-- Generate the ALTER SEQUENCE statements dynamically
SELECT 'SELECT setval(' || seq.sequence_schema || '.' || seq.sequence_name ||
      ''', COALESCE((SELECT MAX(' || col.column_name ||
      ') FROM ' || col.table_schema || '.' || col.table_name || '), 1));'
FROM information_schema.sequences seq
JOIN information_schema.columns col
  ON col.column_default LIKE '%' || seq.sequence_name || '%'
WHERE seq.sequence_schema NOT IN ('pg_catalog', 'information_schema')
ORDER BY seq.sequence_name;
-- Execute the output of this query to advance all sequences safely
```

The Decision Framework

After all of the above, the decision reduces to a set of concrete questions you can answer before you provision:

Question	Answer favoring RDS	Answer favoring Aurora
What is the primary bottleneck?	Write throughput, commit latency	Read scaling, high availability
How many read replicas do you need?	0–2, tolerate seconds of lag	3+, require millisecond freshness
What is your failover SLA?	60–120 seconds acceptable	Sub-30-second failover required
Database size and growth pattern?	Under 500 GB, predictable growth	Large or unpredictable growth
Is cross-region DR required?	Not required or RTO > 30 min	Required, RTO < 10 minutes
Does your workload vary significantly?	Stable, predictable load	Spiky, bursty, or variable load
Which PostgreSQL version do you need?	Latest community release	Willing to accept Aurora's version lag
Extension requirements?	Extensions not yet on Aurora	Extensions all available on Aurora

There is one override condition: if your application is already on Aurora and performing acceptably, the cost and risk of migrating to RDS for marginal write-latency improvement is almost never worth it. Aurora's operational advantages — storage auto-scaling, replication slot elimination, fast failover, Serverless v2 — compound over time, and the correct answer for most greenfield production systems today with moderate-to-high availability requirements is Aurora PostgreSQL. The reasons to choose RDS PostgreSQL are narrow but real: strict version requirements, extreme write sensitivity, or cost constraints at smaller scale.

Key Takeaways

- **Aurora and RDS PostgreSQL are architecturally distinct products.** Aurora replaces the local I/O path with a distributed six-copy storage layer, giving faster failover and millisecond replica lag at the cost of higher per-write latency and a more complex billing model. RDS runs stock PostgreSQL on EBS with standard WAL-based replication — simpler, faster for write-bound workloads, and closer to community PostgreSQL.
 - **Aurora wins on operational simplicity at scale.** Automatic storage growth to 128 TB, no replication slot management, sub-30-second failover, and Serverless v2 compute scaling make Aurora the lower operational overhead option for systems that need high availability and read scaling across multiple replicas.
 - **RDS PostgreSQL wins on write-heavy, cost-sensitive, or version-sensitive workloads.** For high-frequency insert or update workloads, commit latency on io2 RDS often outperforms Aurora’s quorum write path. RDS also tracks community PostgreSQL releases faster and carries a lower base cost at smaller scales.
 - **Connection handling and parameter group structure differ meaningfully between the two.** Aurora’s two-level parameter group system (cluster-level and instance-level), its custom monitoring functions (`aurora_replica_status()`), and its unique failover behavior require application and operational changes that are easy to underestimate when migrating from RDS.
 - **Migration from RDS to Aurora is well-supported but requires sequencing care.** Snapshot-based migration is straightforward for maintenance window scenarios; logical replication enables near-zero downtime migration but requires explicit sequence management at cutover to prevent primary key violations on the Aurora target.
-

Chapter 8: Aurora MySQL vs RDS MySQL

Aurora MySQL and RDS MySQL are both managed MySQL offerings on AWS, but they are architecturally different products that share a query interface. If you’ve spent years tuning InnoDB buffer pools and wrestling with MySQL replication, the differences matter enormously in production. One is a drop-in replacement for a self-managed MySQL installation with a managed control plane layered on top. The other is a purpose-built distributed storage engine disguised as MySQL. This chapter walks through how Aurora MySQL diverges from RDS MySQL at the storage layer, replication model, failover behavior, and cost structure — and how to choose between them for real production workloads.

8.1 Architectural Differences: Storage Layer and Replication Model

The single most important thing to understand about Aurora MySQL is that it is not running the InnoDB storage engine against local EBS volumes. Aurora separates compute from storage entirely. The Aurora storage subsystem is a distributed, fault-tolerant, self-healing storage layer that spans six storage nodes across three Availability Zones. When your writer instance commits a transaction, Aurora writes redo log records to quorum across those six nodes — four of six must acknowledge before the commit returns. The actual page materialization happens asynchronously in the storage layer. Aurora does not ship full page writes from compute to storage. It ships log records.

For a DBA who has tuned `innodb_flush_log_at_trx_commit` and worried about the fsync pipeline, this is a meaningful shift. The durability guarantee is enforced in the storage tier, not at the instance level. A single Aurora writer can sustain significantly higher write throughput than an equivalently sized RDS MySQL instance because it is not performing double-write buffer operations to local storage. The double-write buffer exists to protect against torn page writes — a problem Aurora’s storage architecture eliminates by design.

RDS MySQL, by contrast, runs MySQL — with its full InnoDB engine including double-write buffer, undo tablespaces, and local EBS storage — with AWS managing provisioning, patching, automated backups, and Multi-AZ failover. Multi-AZ RDS MySQL uses synchronous block-level replication to a standby in a second AZ. That standby is not readable. It is purely a failover target. Failover on Multi-AZ RDS MySQL typically takes 60 to 120 seconds, involves a DNS flip, and requires applications to reconnect.

Aurora MySQL failover works differently. The reader instances are not replicas in the traditional

sense — they share the same distributed storage volume as the writer. A reader can be promoted to writer in 30 seconds or less in typical conditions because it already has access to the same underlying storage. There is no replication lag between the writer and readers with respect to durable data; readers do see a brief lag in their buffer cache view of in-flight transactions, but they are reading from the same storage pages. Aurora advertises sub-30-second failover and typically achieves it.

Aurora also supports up to 15 read replicas. RDS MySQL supports up to 5 read replicas, and those replicas use standard MySQL binlog replication — which means they can lag, especially under heavy write load or large transactions. A single large `ALTER TABLE` on RDS MySQL can cause read replicas to fall minutes or hours behind. Aurora readers don't have that problem because they are not replaying binlog; they are reading pages that have already been materialized in the shared storage layer.

The trade-off: Aurora MySQL costs more per instance-hour than an equivalently sized RDS MySQL instance, and Aurora's storage pricing model is consumption-based (billed per GB-month for storage plus per-request I/O charges — check current AWS pricing for your region). For read-heavy, low-write workloads, RDS MySQL with read replicas may be cheaper. For write-intensive workloads where replication lag on reads is unacceptable, Aurora wins.

8.2 Provisioning Aurora MySQL vs RDS MySQL with Terraform and AWS CLI

Provisioning an Aurora MySQL cluster differs structurally from provisioning an RDS MySQL instance. Aurora has two objects: the cluster and the instances within it. You provision the cluster — which owns the storage volume, the endpoints, and the parameter groups — and then you provision one or more DB instances into that cluster.

```
# Create an Aurora MySQL 8.0 cluster
aws rds create-db-cluster \
  --db-cluster-identifier prod-aurora-mysql \
  --engine aurora-mysql \
  --engine-version 8.0.mysql_aurora.3.04.0 \
  --master-username admin \
  --master-user-password "YourSecurePassword123!" \
  --db-subnet-group-name prod-subnet-group \
  --vpc-security-group-ids sg-0abc123def456 \
  --backup-retention-period 7 \
  --preferred-backup-window "02:00-03:00" \
  --preferred-maintenance-window "sun:04:00-sun:05:00" \
  --storage-encrypted \
  --kms-key-id arn:aws:kms:us-east-1:123456789012:key/your-kms-key-id \
  --enable-cloudwatch-logs-exports '["audit","error","general","slowquery"]' \
  --deletion-protection

# Add the writer instance to the cluster
aws rds create-db-instance \
  --db-instance-identifier prod-aurora-mysql-writer \
  --db-cluster-identifier prod-aurora-mysql \
  --db-instance-class db.r6g.2xlarge \
  --engine aurora-mysql

# Add a reader instance
aws rds create-db-instance \
  --db-instance-identifier prod-aurora-mysql-reader-1 \
  --db-cluster-identifier prod-aurora-mysql \
  --db-instance-class db.r6g.xlarge \
  --engine aurora-mysql
```

The equivalent in Terraform makes the cluster-versus-instance separation explicit:

```
resource "aws_rds_cluster" "aurora_mysql" {
  cluster_identifier      = "prod-aurora-mysql"
  engine                 = "aurora-mysql"
  engine_version         = "8.0.mysql_aurora.3.04.0"
  database_name          = "appdb"
  master_username        = "admin"
  master_password        = var.db_master_password
  db_subnet_group_name   = aws_db_subnet_group.prod.name
  vpc_security_group_ids = [aws_security_group.aurora.id]

  backup_retention_period = 7
  preferred_backup_window = "02:00-03:00"
  storage_encrypted       = true
  kms_key_id              = aws_kms_key.rds.arn
  deletion_protection     = true
  skip_final_snapshot     = false
  final_snapshot_identifier = "prod-aurora-mysql-final"

  enabled_cloudwatch_logs_exports = ["audit", "error", "general", "slowquery"]

  db_cluster_parameter_group_name = aws_rds_cluster_parameter_group.aurora_mysql8.name
}

resource "aws_rds_cluster_instance" "writer" {
  identifier      = "prod-aurora-mysql-writer"
  cluster_identifier = aws_rds_cluster.aurora_mysql.id
  instance_class   = "db.r6g.2xlarge"
  engine           = aws_rds_cluster.aurora_mysql.engine
  engine_version   = aws_rds_cluster.aurora_mysql.engine_version

  db_parameter_group_name = aws_db_parameter_group.aurora_mysql8_instance.name

  performance_insights_enabled      = true
  performance_insights_retention_period = 7
  monitoring_interval               = 60
  monitoring_role_arn               = aws_iam_role.rds_enhanced_monitoring.arn
}

resource "aws_rds_cluster_instance" "readers" {
  count      = 2
  identifier = "prod-aurora-mysql-reader-${count.index + 1}"
  cluster_identifier = aws_rds_cluster.aurora_mysql.id
  instance_class     = "db.r6g.xlarge"
  engine              = aws_rds_cluster.aurora_mysql.engine
  engine_version      = aws_rds_cluster.aurora_mysql.engine_version

  db_parameter_group_name = aws_db_parameter_group.aurora_mysql8_instance.name
}

# Aurora has TWO levels of parameter groups: cluster-level and instance-level
resource "aws_rds_cluster_parameter_group" "aurora_mysql8" {
  name     = "prod-aurora-mysql8-cluster-params"
  family   = "aurora-mysql8.0"

  parameter {
    name = "character_set_server"
    value = "utf8mb4"
  }
  parameter {
    name = "time_zone"
    value = "UTC"
  }
  parameter {
    name = "slow_query_log"
    value = "1"
  }
  parameter {
    name = "long_query_time"
  }
}
```

```
        value = "1"
    }
}

resource "aws_db_parameter_group" "aurora_mysql8_instance" {
    name     = "prod-aurora-mysql8-instance-params"
    family   = "aurora-mysql8.0"

    parameter {
        name = "innodb_print_all_deadlocks"
        value = "1"
    }
}
```

Notice that Aurora uses two tiers of parameter groups: the cluster parameter group applies to all instances in the cluster and controls settings like character sets and binlog format. The instance parameter group applies to individual instances and controls settings like buffer pool behavior. In RDS MySQL, there is only one parameter group per instance. This is one of the first configuration gotchas that trips up DBAs coming from RDS MySQL.

For RDS MySQL, provisioning is simpler because there is no cluster abstraction:

```
# Create an RDS MySQL 8.0 Multi-AZ instance
aws rds create-db-instance \
  --db-instance-identifier prod-rds-mysql \
  --db-instance-class db.m6g.2xlarge \
  --engine mysql \
  --engine-version 8.0.35 \
  --master-username admin \
  --master-user-password "YourSecurePassword123!" \
  --allocated-storage 500 \
  --storage-type gp3 \
  --iops 12000 \
  --storage-encrypted \
  --multi-az \
  --db-subnet-group-name prod-subnet-group \
  --vpc-security-group-ids sg-0abc123def456 \
  --backup-retention-period 7 \
  --preferred-backup-window "02:00-03:00" \
  --preferred-maintenance-window "sun:04:00-sun:05:00" \
  --enable-performance-insights \
  --performance-insights-retention-period 7 \
  --monitoring-interval 60 \
  --monitoring-role-arn arn:aws:iam::123456789012:role/rds-monitoring-role \
  --deletion-protection \
  --no-publicly-accessible
```

With RDS MySQL you specify storage size explicitly — 500 GB in this case, provisioned with gp3 at 12,000 IOPS. With Aurora MySQL you never specify storage size; the storage grows automatically in 10 GB increments up to 128 TB. If your workload balloons unexpectedly, Aurora handles it. With RDS MySQL, you need to monitor free storage and either resize proactively or enable storage autoscaling.

8.3 Parameter Groups: What You Can and Cannot Tune

On-prem, you owned the MySQL configuration file entirely. On managed RDS or Aurora, AWS restricts a substantial number of parameters. Understanding which parameters are tunable, which are fixed, and which behave differently than on-prem is essential before production deployment.

The following query, run against a running Aurora MySQL instance, shows you which parameters

are customer-modifiable:

```
-- Connect to your Aurora instance and check parameter apply status
-- This is better done via AWS CLI against the parameter group

-- But once connected, verify current effective values
SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
SHOW VARIABLES LIKE 'innodb_io_capacity';
SHOW VARIABLES LIKE 'max_connections';
SHOW VARIABLES LIKE 'binlog_format';
SHOW VARIABLES LIKE 'transaction_isolation';

-- Aurora-specific variables worth checking
SHOW VARIABLES LIKE 'aurora%';
SHOW VARIABLES LIKE 'aurora_parallel_query';
SHOW VARIABLES LIKE 'aurora_read_replica_read_consistency';

# Check which parameters in your cluster parameter group are user-modifiable
aws rds describe-engine-default-cluster-parameters \
  --db-parameter-group-family aurora-mysql8.0 \
  --query 'EngineDefaults.Parameters[?IsModifiable==`true`].[ParameterName,ParameterValue,ApplyType]' \
  --output table | head -60

# Check a specific parameter's current value and constraints
aws rds describe-db-cluster-parameters \
  --db-cluster-parameter-group-name prod-aurora-mysql8-cluster-params \
  --query 'Parameters[?ParameterName==`innodb_io_capacity`]'
```

Key parameters and their cloud behavior:

innodb_buffer_pool_size: On RDS and Aurora, this is typically set automatically based on the instance class. On Aurora, the default is 75% of instance memory, which is generally correct. You can override it, but AWS recommends against it. On a `db.r6g.2xlarge` with 64 GB RAM, the buffer pool defaults to roughly 48 GB.

max_connections: Aurora sets this automatically based on a formula derived from instance memory — run `SHOW VARIABLES LIKE 'max_connections'` on your instance to see the actual default rather than assuming a fixed number. You can increase it, but doing so is usually a signal that you need RDS Proxy rather than a larger `max_connections` value. More on this in Section 8.4.

binlog_format: On Aurora MySQL, binlog is not required for replication within the cluster — Aurora uses its own log-based replication internally. However, if you're using Aurora as a source for DMS, or replicating to an external MySQL target, binlog must be enabled explicitly. Set `binlog_format=ROW` at the cluster parameter group level and enable `log_bin` if needed.

innodb_flush_log_at_trx_commit: This parameter exists on Aurora but has reduced significance. Aurora does not perform traditional InnoDB log flushing to local storage; the durability guarantee is enforced at the storage layer. The parameter is modifiable, but changing it from the default of 1 does not provide the traditional trade-off between durability and performance it would on self-managed MySQL.

Parameters you cannot change on managed services:

- `datadir` — fixed by AWS
- `innodb_data_file_path` — not applicable, Aurora manages tablespace layout
- `plugin_dir` — you cannot install arbitrary MySQL plugins
- `general_log` to a file location of your choosing — logs go to CloudWatch, not local filesystem
- `secure_file_priv` — `LOAD DATA INFILE` from arbitrary paths is restricted; use S3 integration instead

Aurora MySQL supports a native S3 integration for LOAD DATA operations:

```
-- Load data from S3 into Aurora MySQL (requires aws_default_s3_role IAM role attached to cluster)
LOAD DATA FROM S3 's3://your-bucket/data/customers.csv'
INTO TABLE customers
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

-- Export query results to S3
SELECT customer_id, email, created_at
INTO OUTFILE S3 's3://your-bucket/exports/customers_export'
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
FROM customers
WHERE created_at >= '2024-01-01';
```

This replaces the on-prem pattern of mounting a network share or copying files locally before LOAD DATA INFILE. Secure it properly: the IAM role attached to the Aurora cluster needs `s3:GetObject` and `s3:PutObject` on the target bucket, with `s3:ListBucket` on the bucket itself.

8.4 Connection Management: RDS Proxy and Aurora

One of the most persistent operational pain points when migrating from self-managed MySQL to RDS is connection management. Self-managed MySQL installations behind ProxySQL or HAProxy gave you control over connection pooling, load balancing, query routing, and failover handling. AWS provides RDS Proxy as its managed connection pooler, and it integrates with both RDS MySQL and Aurora MySQL.

RDS Proxy maintains a pool of persistent connections to the database and presents a proxy endpoint to applications. Applications connect to the proxy; the proxy multiplexes those connections against a smaller pool of real database connections. This is significant for Lambda functions, containerized applications, or any workload that opens and closes database connections frequently. Without a connection pooler, a spike in Lambda concurrency can open thousands of connections to Aurora simultaneously, exhausting `max_connections` and causing `Too many connections` errors.

```
# Create an RDS Proxy for Aurora MySQL
aws rds create-db-proxy \
  --db-proxy-name prod-aurora-proxy \
  --engine-family MYSQL \
  --auth '[{
    "AuthScheme": "SECRETS",
    "SecretArn": "arn:aws:secretsmanager:us-east-1:123456789012:secret:prod/aurora/admin-AbCdEf",
    "IAMAuth": "REQUIRED"
  }]' \
  --role-arn arn:aws:iam::123456789012:role/rds-proxy-role \
  --vpc-subnet-ids subnet-0abc123 subnet-0def456 subnet-0ghi789 \
  --vpc-security-group-ids sg-0abc123def456 \
  --require-tls

# Register the Aurora cluster with the proxy
aws rds register-db-proxy-targets \
  --db-proxy-name prod-aurora-proxy \
  --db-cluster-identifiers prod-aurora-mysql

# Create a read-only proxy endpoint pointing to readers
aws rds create-db-proxy-endpoint \
  --db-proxy-name prod-aurora-proxy \
  --db-proxy-endpoint-name prod-aurora-proxy-readonly \
```

```
--vpc-subnet-ids subnet-0abc123 subnet-0def456 subnet-0ghi789 \  
--vpc-security-group-ids sg-0abc123def456 \  
--target-role READ_ONLY
```

RDS Proxy uses AWS Secrets Manager to store and rotate credentials. If you set `IAMAuth:REQUIRED`, applications authenticate to the proxy using IAM authentication rather than username/password. The proxy then uses the stored secret to authenticate to the actual database. This is the cloud equivalent of what you used to do with `pg_hba.conf` certificate authentication or LDAP integration — centralize credential management, enforce consistent authentication policies, and enable rotation without application restarts.

```
import boto3  
import pymysql  
import os  
  
def get_db_connection_with_iam():  
    """  
    Connect to RDS Proxy using IAM authentication.  
    The proxy endpoint accepts an IAM auth token as the password.  
    """  
    region = 'us-east-1'  
    proxy_host = 'prod-aurora-proxy.proxy-abc123.us-east-1.rds.amazonaws.com'  
    db_user = 'app_user'  
    db_name = 'appdb'  
    port = 3306  
  
    # Generate an IAM authentication token  
    client = boto3.client('rds', region_name=region)  
    token = client.generate_db_auth_token(  
        DBHostname=proxy_host,  
        Port=port,  
        DBUsername=db_user,  
        Region=region  
    )  
  
    # The token is used as the password; it expires after 15 minutes  
    connection = pymysql.connect(  
        host=proxy_host,  
        user=db_user,  
        password=token,  
        database=db_name,  
        port=port,  
        ssl_ca='/path/to/us-east-1-bundle.pem', # Download from AWS  
        ssl_verify_cert=True,  
        connect_timeout=5,  
        read_timeout=30,  
        write_timeout=30  
    )  
  
    return connection  
  
def execute_with_retry(query, params=None, max_retries=3):  
    """  
    Production pattern: handle proxy reconnection on failover.  
    RDS Proxy maintains connections across Aurora failover,  
    but applications should still handle transient errors.  
    """  
    import time  
  
    for attempt in range(max_retries):  
        conn = None  
        try:  
            conn = get_db_connection_with_iam()  
            with conn.cursor() as cursor:  
                cursor.execute(query, params)  
                conn.commit()  
                return cursor.fetchall()  
        except:
```

```
except pymysql.err.OperationalError as e:
    error_code = e.args[0]
    # 2003: Can't connect, 2006: Server gone away
    if error_code in (2003, 2006) and attempt < max_retries - 1:
        wait = 2 ** attempt # exponential backoff
        time.sleep(wait)
        continue
    raise
finally:
    if conn:
        conn.close()
```

RDS Proxy has important gotchas to understand before production deployment:

Pinning: RDS Proxy multiplexes connections by reusing them between requests. When a client uses session-level state that the proxy cannot safely track — user variables (`SET @var = value`), temporary tables, or `LOCK TABLES` — the proxy “pins” that client to a specific backend connection and cannot share it. Excessive pinning defeats the purpose of connection pooling. Audit your application code for session-level state before enabling RDS Proxy. The CloudWatch metric `DatabaseConnectionsCurrentlySessionPinned` will tell you if this is a problem.

Transaction pinning: Any open transaction pins a connection until commit or rollback. Long-running transactions in a pooled environment are as harmful here as they are with any connection pooler.

Max connections to RDS Proxy: The proxy itself has a connection limit based on its size configuration. The default proxy scales automatically, but if you’re routing very high connection counts through it, verify the proxy’s `MaxConnectionsPercent` setting, which controls how much of the database’s `max_connections` the proxy is allowed to use.

8.5 Failover, Backups, and Aurora-Specific Features

Failover behavior

Aurora’s failover is tier-based. You assign a failover priority from 0 to 15 to each reader instance using the `PromotionTier` parameter. In the event the writer fails, Aurora promotes the reader with the lowest tier number. If two readers share the same tier, Aurora promotes the one with the largest instance class. Set your highest-capacity reader to tier 0 to make failover behavior predictable.

```
# Set failover priority on Aurora reader instances
aws rds modify-db-instance \
    --db-instance-identifier prod-aurora-mysql-reader-1 \
    --promotion-tier 0

aws rds modify-db-instance \
    --db-instance-identifier prod-aurora-mysql-reader-2 \
    --promotion-tier 1

# Trigger a manual failover to test promotion
aws rds failover-db-cluster \
    --db-cluster-identifier prod-aurora-mysql \
    --target-db-instance-identifier prod-aurora-mysql-reader-1
```

Applications connecting to the Aurora cluster endpoint automatically route to whichever instance is the current writer. After failover, the cluster endpoint resolves to the new writer within approximately 30 seconds. Applications must handle the brief reconnection window — typically by catching connection errors and retrying with exponential backoff, as shown in the Python example

above. Using RDS Proxy further reduces the impact because the proxy absorbs the failover and queues incoming requests while promotion completes.

Backups and point-in-time recovery

Aurora continuously backs up to S3. The backup window you configure is for the full snapshot, but redo logs are shipped to S3 continuously, enabling point-in-time recovery (PITR) to within 5 minutes of any point in the last 35 days (with maximum backup retention). RDS MySQL also supports PITR, but with a slightly different operational model: the automated snapshot establishes the restore baseline, and binary logs fill the gap between the snapshot and the target recovery time.

To initiate a point-in-time restore for Aurora:

```
# Restore Aurora cluster to a specific point in time
aws rds restore-db-cluster-to-point-in-time \
  --db-cluster-identifier prod-aurora-mysql-restored \
  --source-db-cluster-identifier prod-aurora-mysql \
  --restore-type full-copy \
  --restore-to-time 2024-03-15T14:30:00Z \
  --db-subnet-group-name prod-subnet-group \
  --vpc-security-group-ids sg-0abc123def456 \
  --enable-cloudwatch-logs-exports '["error","slowquery"]'

# After the cluster is restored, add an instance to it
aws rds create-db-instance \
  --db-instance-identifier prod-aurora-mysql-restored-writer \
  --db-cluster-identifier prod-aurora-mysql-restored \
  --db-instance-class db.r6g.2xlarge \
  --engine aurora-mysql
```

One operational advantage of Aurora's backup model is that restoring to a new cluster does not immediately read back all pages from S3. Aurora uses a lazy restore approach: the storage layer makes the cluster available immediately and fetches pages from S3 on demand as they are accessed. For large databases, this dramatically reduces the time to first query on a restored cluster compared to waiting for RDS MySQL to restore a full snapshot before the instance becomes available.

For RDS MySQL, point-in-time restore follows the same pattern but uses `restore-db-instance-to-point-in-time`:

```
# Restore RDS MySQL to a specific point in time
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier prod-rds-mysql \
  --target-db-instance-identifier prod-rds-mysql-restored \
  --restore-time 2024-03-15T14:30:00Z \
  --db-instance-class db.m6g.2xlarge \
  --db-subnet-group-name prod-subnet-group \
  --no-multi-az \
  --no-publicly-accessible
```

With RDS MySQL the restored instance must receive the entire snapshot to storage before it becomes accessible. For a 1 TB database this can take significantly longer than the equivalent Aurora restore.

Aurora-specific features worth knowing in production

Aurora Fast Clone: Aurora can create a clone of a cluster near-instantly using a copy-on-write mechanism at the storage layer. The clone shares storage pages with the source until writes diverge. This is extraordinarily useful for creating staging environments, schema migration tests, or load testing copies without paying for a full storage copy upfront.

```
# Clone Aurora cluster (near-instant, copy-on-write storage)
aws rds restore-db-cluster-to-point-in-time \
```

```
--db-cluster-identifier prod-aurora-mysql-clone \  
--source-db-cluster-identifier prod-aurora-mysql \  
--restore-type copy-on-write \  
--use-latest-restorable-time \  
--db-subnet-group-name staging-subnet-group \  
--vpc-security-group-ids sg-0staging123  
  
# Add an instance to the clone  
aws rds create-db-instance \  
--db-instance-identifier prod-aurora-mysql-clone-writer \  
--db-cluster-identifier prod-aurora-mysql-clone \  
--db-instance-class db.r6g.xlarge \  
--engine aurora-mysql
```

The clone is independent after creation — writes to production do not appear in the clone, and vice versa. Storage divergence increases over time as writes accumulate in both clusters. This is not a synchronization mechanism; it is a fast forking mechanism for ephemeral copies.

Aurora Global Database: For multi-region deployments, Aurora Global Database replicates from a primary region to up to five secondary regions using storage-level replication rather than MySQL binlog. Replication lag to secondary regions is typically under one second under normal load. Secondary regions are read-only unless you perform a planned or unplanned failover (detaching the secondary and promoting it to standalone primary). This is the pattern for disaster recovery across regions without the binlog lag and replication monitoring overhead you'd carry with RDS MySQL cross-region replicas.

Aurora Parallel Query: For analytical workloads on Aurora MySQL, Parallel Query pushes query processing down to the storage layer across multiple storage nodes. For certain full-scan, aggregation-heavy queries against large tables, this can substantially reduce query execution time by parallelizing work that would otherwise run single-threaded on the compute node. Enable it selectively:

```
-- Check if parallel query is available on your instance type  
SHOW VARIABLES LIKE 'aurora_parallel_query';  
  
-- Enable parallel query for the current session  
SET SESSION aurora_parallel_query = ON;  
  
-- Verify a specific query will use parallel query  
EXPLAIN  
SELECT region, COUNT(*), SUM(amount)  
FROM orders  
WHERE order_date >= '2023-01-01'  
GROUP BY region;  
-- Look for "Using parallel query" in the Extra column
```

Not all queries benefit from Parallel Query. It adds overhead for index-range scans that are already selective. It is most beneficial for large table scans with filtering and aggregation — the OLAP edge of a mixed OLTP/OLAP workload.

Aurora Serverless v2: Aurora Serverless v2 allows compute capacity to scale continuously in half-ACU (Aurora Capacity Unit) increments between a configured minimum and maximum. Unlike the original Aurora Serverless v1, v2 instances do not scale to zero (the minimum is 0.5 ACU) and they scale fast enough to handle production traffic spikes without cold-start latency. This is appropriate for workloads with highly variable load patterns where you'd otherwise overprovision a fixed instance class to handle peaks. Provisioned Aurora is still preferable for steady, predictable workloads where the per-ACU cost of Serverless v2 exceeds the equivalent provisioned instance cost.

8.6 Monitoring Aurora MySQL vs RDS MySQL

The monitoring surface between Aurora and RDS MySQL overlaps significantly — both surface metrics through CloudWatch and both support Performance Insights and Enhanced Monitoring. The differences lie in the Aurora-specific metrics that expose the distributed storage layer.

Key Aurora-specific CloudWatch metrics:

- **AuroraVolumeBytesLeftTotal**: Storage remaining in the Aurora volume. Alert when this drops below a comfortable threshold — Aurora automatically grows storage, but there is a maximum.
- **AuroraBinlogReplicaLag**: Lag for external replicas (non-Aurora) replicating from the Aurora cluster. This is different from **ReplicaLag**, which measures lag on Aurora reader instances.
- **CommitLatency** / **CommitThroughput**: These measure the latency and rate of transaction commits at the Aurora storage layer. A spike in **CommitLatency** with stable instance CPU suggests storage-tier contention, not compute saturation.
- **DDLlatency** / **DMLlatency** / **SelectLatency**: Fine-grained operation-type latency metrics available on Aurora, not on RDS MySQL.
- **BufferCacheHitRatio**: Percentage of page reads served from the in-memory buffer pool. Sustained values below 95% warrant investigation. A falling ratio suggests your working set is larger than your buffer pool.
- **EngineUptime**: Useful for detecting unexpected restarts during failover testing or incidents.

For RDS MySQL, the key operational metrics are the standard set:

- **FreeStorageSpace**: Must be monitored actively; RDS MySQL does not grow storage automatically unless you enable storage autoscaling.
- **FreeableMemory**: If this drops to near zero, MySQL is under memory pressure. The buffer pool may need shrinking or the instance class needs increasing.
- **ReadLatency** / **WriteLatency**: Storage I/O latency at the EBS layer.
- **ReplicaLag**: Binlog replication lag in seconds on read replicas. Alert at any value above your acceptable staleness threshold.
- **DiskQueueDepth**: Queue of pending I/O operations to EBS. Sustained high values indicate I/O saturation; increase provisioned IOPS or switch to gp3 with higher IOPS allocation.

Performance Insights for identifying slow queries

Both Aurora MySQL and RDS MySQL expose Performance Insights. Enable it at provisioning time (it is not retroactive for historical data). Performance Insights surfaces the top SQL statements by average active sessions (AAS), allowing you to identify which queries are consuming the most database time without requiring `slow_query_log` parsing.

```
# Query Performance Insights data via CLI (last hour of top SQL)
aws pi get-resource-metrics \
  --service-type RDS \
  --identifier db-ABCDEFGHIJKLMNQPQRSTUVWXYZ \
  --metric-queries '[{
    "Metric": "db.load.avg",
    "GroupBy": {
      "Group": "db.sql",
      "Limit": 10
    }
  }]
```

```

    }
  ]]' \
  --start-time $(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period-in-seconds 60

```

The `db.load.avg` metric represents average active sessions broken down by the grouping dimension — here, `db.sql`. The output ranks SQL statements by how much database load they generate. A query appearing at the top of this list with an AAS value greater than the number of vCPUs on your instance is a query that is saturating available concurrency.

CloudWatch alarms for production Aurora clusters

```

# Terraform: essential CloudWatch alarms for Aurora MySQL
resource "aws_cloudwatch_metric_alarm" "aurora_replica_lag" {
  alarm_name         = "aurora-mysql-replica-lag-high"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods = 3
  metric_name        = "AuroraReplicaLag"
  namespace           = "AWS/RDS"
  period              = 60
  statistic           = "Maximum"
  threshold           = 500 # milliseconds; adjust per your SLA
  alarm_description   = "Aurora reader lag exceeds 500ms"
  alarm_actions       = [aws_sns_topic.db_alerts.arn]

  dimensions = {
    DBClusterIdentifier = aws_rds_cluster.aurora_mysql.cluster_identifier
  }
}

resource "aws_cloudwatch_metric_alarm" "aurora_buffer_cache_hit" {
  alarm_name         = "aurora-mysql-buffer-cache-low"
  comparison_operator = "LessThanThreshold"
  evaluation_periods = 5
  metric_name        = "BufferCacheHitRatio"
  namespace           = "AWS/RDS"
  period              = 300
  statistic           = "Average"
  threshold           = 90
  alarm_description   = "Aurora buffer cache hit ratio below 90% for 25 minutes"
  alarm_actions       = [aws_sns_topic.db_alerts.arn]

  dimensions = {
    DBClusterIdentifier = aws_rds_cluster.aurora_mysql.cluster_identifier
  }
}

resource "aws_cloudwatch_metric_alarm" "aurora_commit_latency" {
  alarm_name         = "aurora-mysql-commit-latency-high"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods = 3
  metric_name        = "CommitLatency"
  namespace           = "AWS/RDS"
  period              = 60
  statistic           = "Average"
  threshold           = 20 # milliseconds
  alarm_description   = "Aurora commit latency exceeds 20ms - storage tier contention"
  alarm_actions       = [aws_sns_topic.db_alerts.arn]

  dimensions = {
    DBClusterIdentifier = aws_rds_cluster.aurora_mysql.cluster_identifier
  }
}

resource "aws_cloudwatch_metric_alarm" "rds_free_storage" {
  alarm_name         = "rds-mysql-free-storage-low"
  comparison_operator = "LessThanThreshold"

```

```
evaluation_periods = 2
metric_name        = "FreeStorageSpace"
namespace          = "AWS/RDS"
period             = 300
statistic          = "Average"
threshold          = 53687091200 # 50 GB in bytes
alarm_description  = "RDS MySQL free storage below 50GB"
alarm_actions      = [aws_sns_topic.db_alerts.arn]

dimensions = {
    DBInstanceIdentifier = "prod-rds-mysql"
}
}
```

8.7 Schema Changes and Online DDL in Aurora vs RDS MySQL

Schema changes are one of the most operationally fraught areas in MySQL production management. MySQL 8.0 improved online DDL substantially through the `ALGORITHM=INSTANT` path for many common operations, but not all DDL can run without locking. Understanding what runs instantly, what runs online but takes time, and what requires a full table rebuild — and how Aurora’s storage architecture affects each — is essential for zero-downtime deployments.

DDL algorithm tiers in MySQL 8.0:

- **ALGORITHM=INSTANT:** Modifies only metadata. Completes in milliseconds regardless of table size. Works for adding columns (with restrictions), changing default values, changing column comments, renaming indexes.
- **ALGORITHM=INPLACE:** Performs the rebuild in-place, allowing concurrent DML. Takes time proportional to table size, but does not copy the full table to a new location. Adding indexes falls here.
- **ALGORITHM=COPY:** Creates a full copy of the table. Blocks concurrent DML writes during the copy. Should be avoided on large tables in production.

```
-- Check which algorithm MySQL will use for a given ALTER
ALTER TABLE orders
  ADD COLUMN discount_code VARCHAR(50) DEFAULT NULL,
  ALGORITHM=INSTANT; -- Will succeed instantly on MySQL 8.0.29+

-- Adding a non-nullable column without default cannot use INSTANT
-- This falls back to INPLACE or COPY
ALTER TABLE orders
  ADD COLUMN region_id INT NOT NULL,
  ALGORITHM=INPLACE,
  LOCK=NONE; -- LOCK=NONE ensures you get an error rather than a silent lock

-- Adding an index: INPLACE, allows concurrent DML
ALTER TABLE orders
  ADD INDEX idx_customer_date (customer_id, order_date),
  ALGORITHM=INPLACE,
  LOCK=NONE;

-- Changing column data type: typically COPY, blocks writes
-- Use pt-online-schema-change or gh-ost instead
ALTER TABLE orders
  MODIFY COLUMN amount DECIMAL(12,4),
  ALGORITHM=COPY; -- Explicitly label it so you're not surprised
```

On Aurora MySQL, INPLACE DDL benefits from the storage architecture in one meaningful way: because Aurora readers share the same storage pages, there is no need to replay DDL changes on

each replica as a separate binlog event. However, large INPLACE operations still consume I/O and CPU on the writer, and readers will see a brief period of elevated lag in their buffer cache view while catching up to the new page versions.

Using `gh-ost` for large table changes on Aurora MySQL

For any table change that cannot use `ALGORITHM=INSTANT`, and where the table exceeds a size at which INPLACE operation would cause unacceptable production impact, `gh-ost` (GitHub's Online Schema Tool) is the standard tool. It works by replicating binlog events from the writer in real time, applying them to a shadow table alongside the bulk copy, and atomically swapping the tables when the copy is complete.

`gh-ost` requires binlog enabled on Aurora (`binlog_format=ROW`, `binlog_row_image=FULL`). To connect `gh-ost` to Aurora, point it at the cluster's reader endpoint as the replica source — this avoids adding binlog read load to the writer directly.

```
gh-ost \
  --user="ghuser" \
  --password="ghpassword" \
  --host="prod-aurora-mysql.cluster-ro-abc123.us-east-1.rds.amazonaws.com" \
  --port=3306 \
  --database="appdb" \
  --table="orders" \
  --alter="ADD COLUMN discount_code VARCHAR(50) DEFAULT NULL" \
  --master-user="ghuser" \
  --master-password="ghpassword" \
  --assume-master-host="prod-aurora-mysql.cluster-abc123.us-east-1.rds.amazonaws.com" \
  --chunk-size=1000 \
  --max-load="Threads_running=50" \
  --critical-load="Threads_running=100" \
  --switch-to-rbr \
  --allow-on-master \
  --execute
```

The `--host` flag points at the reader endpoint; `--assume-master-host` points at the writer endpoint. `gh-ost` reads binlog from the writer (via the explicit master host) but throttles itself based on load metrics read from the reader. Set `--max-load` and `--critical-load` conservatively for production: if the writer's `Threads_running` exceeds your threshold, `gh-ost` pauses the copy automatically.

`pt-online-schema-change` from Percona Toolkit is an alternative that uses triggers rather than binlog replication. On Aurora, `gh-ost` is generally preferred because trigger-based approaches can introduce write amplification and unexpected locking behavior at high concurrency.

8.8 Choosing Between Aurora MySQL and RDS MySQL

The choice between Aurora MySQL and RDS MySQL is not always obvious and depends heavily on workload characteristics, team operational preferences, and cost sensitivity. Here is a decision framework based on production scenarios.

Choose Aurora MySQL when:

- Your application requires sub-30-second failover and you cannot tolerate 60–120 seconds of downtime during a Multi-AZ RDS failover.

- You need more than 5 read replicas, or you cannot accept binlog replication lag on your read replicas.
- Your write throughput is high enough that the double-write buffer overhead on standard InnoDB would be a bottleneck. This threshold varies by instance size and I/O pattern, but Aurora’s log-based storage consistently outperforms equivalent RDS MySQL instances at high sustained write rates.
- You want fast database cloning for staging, migration testing, or branch development environments (Aurora Fast Clone).
- You need multi-region replication with sub-second lag and managed failover (Aurora Global Database).
- Your storage needs are unpredictable or growing rapidly — Aurora auto-grows without intervention.
- You are deploying a new application and have no migration cost from self-managed MySQL.

Choose RDS MySQL when:

- You need a specific MySQL version or plugin that Aurora MySQL does not support. Aurora MySQL lags behind community MySQL releases, and not every plugin or storage engine is available.
- Your workload is read-heavy with low write rates, and cost is a priority. RDS MySQL with read replicas can be meaningfully cheaper for this profile.
- You rely on MySQL features that Aurora does not expose identically — for example, certain full-text search behaviors, spatial data handling, or specific InnoDB configuration parameters that Aurora overrides.
- You need a staging or development environment where simplicity matters more than HA — a single-AZ RDS MySQL instance is the cheapest managed MySQL option on AWS.
- Your team has deep on-prem MySQL operational expertise and you are migrating with minimal changes. RDS MySQL behaves more like “MySQL you recognize” and fewer Aurora-specific behaviors will surprise you during the migration.
- You are running an engine version or configuration that you must validate against community MySQL behavior exactly, such as for a vendor-certified application stack.

Cost comparison guidance

Aurora MySQL is priced at a higher per-instance-hour rate than comparable RDS MySQL instance classes, plus per-I/O charges on storage. For write-intensive workloads, Aurora’s I/O costs can accumulate significantly. Aurora I/O-Optimized pricing (a newer pricing tier that eliminates per-I/O charges in exchange for a higher instance price) is worth evaluating if your I/O cost constitutes a large fraction of your Aurora bill. Run a cost comparison using your current storage bytes, I/O operations per second (visible in CloudWatch), and retention period against both pricing models before committing to either.

There is no single right answer. The scenarios above are starting points. Run both options through your actual workload profile, model the cost, and validate failover behavior in a staging environment before making the call in production.

Key Takeaways

- **Aurora MySQL and RDS MySQL are architecturally distinct:** Aurora separates compute from storage with a distributed, quorum-based storage layer that eliminates double-write buffer overhead and enables faster failover. RDS MySQL runs standard InnoDB against local EBS storage, behaving more like a managed version of self-managed MySQL.
 - **Failover and replication are fundamentally different:** Aurora achieves sub-30-second failover because readers share the same storage volume as the writer. RDS MySQL Multi-AZ failover relies on block-level replication to a non-readable standby and typically takes 60–120 seconds. RDS MySQL read replicas use binlog replication and can lag significantly under heavy write load; Aurora readers do not.
 - **Parameter groups in Aurora have two tiers:** Cluster-level parameter groups control settings shared across all instances (character sets, binlog format, slow query logging). Instance-level parameter groups control per-instance settings. RDS MySQL has only instance-level parameter groups. Misconfiguring this distinction is a common migration pitfall.
 - **RDS Proxy solves connection exhaustion but requires application awareness:** Proxy connection pooling is essential for serverless or highly concurrent application patterns, but session-level state (user variables, temporary tables, LOCK TABLES) causes connection pinning that defeats pooling efficiency. Audit your application's session state usage before enabling RDS Proxy.
 - **Schema changes require careful DDL strategy in both products:** Use `ALGORITHM=INSTANT` wherever MySQL 8.0 supports it. For large tables requiring `INPLACE` or `COPY` operations, use `gh-ost` with binlog-based replication rather than trigger-based tools. Aurora's shared storage architecture means DDL changes do not need to be replayed individually on each reader via binlog, but large `INPLACE` operations still carry meaningful write-side cost on the cluster.
-

Chapter 9: Multi-AZ vs Read Replicas: Architecture Decisions

Availability and read scalability are two distinct problems that cloud databases solve with two distinct mechanisms, and confusing them in production is one of the most expensive architectural mistakes a DBA can make. Multi-AZ deployments protect you from failures — infrastructure failures, AZ outages, host-level corruption — and do almost nothing for your read throughput. Read replicas scale your reads and do almost nothing for your write availability. Every experienced on-prem DBA has built some version of both concepts using streaming replication, log shipping, or Always On Availability Groups, but the managed cloud implementations have specific behaviors, limits, and gotchas that don't map cleanly to what you've done before. This chapter walks through how each mechanism works under the hood across AWS RDS, Aurora, Azure SQL, and Google Cloud SQL, how to configure them for production workloads, and how to make the architectural decisions that distinguish a well-designed cloud database tier from one that fails during the incident it was supposed to survive.

9.1 Multi-AZ: What It Is, What It Isn't, and How It Actually Works

When you enable Multi-AZ on an RDS instance, AWS provisions a synchronous standby replica in a different Availability Zone and manages the failover for you. The standby receives every write commit before the primary acknowledges it to your application — this is synchronous replication, not asynchronous. The standby is completely invisible to your application. It doesn't serve reads. It doesn't appear in your connection string. It exists solely to be promoted when the primary fails.

This is conceptually close to synchronous streaming replication in PostgreSQL or a synchronous commit Always On replica in SQL Server, except that AWS manages every operational aspect: the standby instance is provisioned, maintained, patched, and failed over without your involvement. The failover target is a DNS CNAME record — your RDS endpoint — which AWS flips to point to the promoted standby when a failure is detected. AWS states this typically takes 60 to 120 seconds in practice, though the exact time varies by failure type and workload.

The gotcha most DBAs hit immediately: your application must respect DNS TTL values, or failover will silently fail. If your JDBC connection pool, pg connection pool, or application ORM is caching the resolved IP of your RDS endpoint beyond the TTL (which RDS sets to 5 seconds), your connections will continue pointing at the old primary address after failover. The promoted

standby has a new IP. Your application sits there timing out while wondering why the database disappeared. The fix is to ensure your connection library honors TTL. For Java applications, set `networkaddress.cache.ttl=0` or use RDS Proxy, which maintains persistent connections through failover transparently. For Python applications using `psycopg2`, verify that your connection pool isn't holding resolved IPs indefinitely.

Aurora Multi-AZ works differently from RDS Multi-AZ, and this distinction matters. Aurora doesn't replicate the database to a standby instance — it writes to a distributed storage layer across six storage nodes spread across three AZs by default. The “replica” in Aurora Multi-AZ is a reader instance that shares this storage layer rather than receiving replicated writes over the network. Failover in Aurora is therefore faster, typically 30 seconds or less, because the promoted replica doesn't need to replay a redo log to catch up — it already has access to the same storage. The tradeoff is that Aurora's storage pricing model means you're paying for six copies of your storage whether you have one reader instance or five.

For Azure SQL Database, high availability is handled differently depending on the service tier. In the Business Critical tier, Azure SQL maintains an Always On Availability Group with four replicas internally — one primary and three secondaries, one of which is readable. You don't configure this. You don't see it in your portal. It happens automatically as part of what you're paying for in the Business Critical pricing tier. In the General Purpose tier, Azure uses remote storage with a stateless compute layer; failover involves restarting the compute layer against the same storage, which is slower (60 to 120 seconds) but dramatically cheaper. Knowing which tier you're in and what its failover behavior looks like is foundational to your SLA conversations.

Google Cloud SQL High Availability provisions a standby instance in a different zone within the same region. Writes are synchronously replicated to this standby. Failover is DNS-based, similar to RDS, with typical observed times of 60 to 90 seconds. Cloud SQL also has a quirk: during maintenance windows, the instance restarts on the standby node, and your application will see a brief disconnection. This is expected behavior, not a failure.

9.2 Read Replicas: Asynchronous Replication and the Lag Problem

Read replicas exist to take `SELECT` load off your primary. They do this through asynchronous replication, which means there is always some lag between what the primary has committed and what the replica has visible. The size of that lag depends on write volume, replica instance size, network conditions, and whether the replica is in the same region or a different one. In a well-tuned configuration with a modestly loaded primary, replica lag sits below one second. In a write-heavy system, or when the replica is under-provisioned, lag can grow to seconds, minutes, or more.

This asynchronous nature is not a bug — it's the fundamental tradeoff that makes read replicas useful. Synchronous replication (Multi-AZ) adds latency to every write commit because the primary must wait for the standby acknowledgment before returning to the application. If your standby is in a different AZ connected via a 1ms link, you're adding approximately 1ms to every write commit. Across a cross-region link of 50ms or more, synchronous replication would be operationally untenable. Asynchronous replication lets the replica fall behind so the primary can proceed.

The architectural decision this forces on you: you cannot route all reads to replicas unless you can tolerate reading stale data. Sessions that have just committed a write and immediately query that

data will read their own write from the primary, not the replica. Sessions that serve time-sensitive reporting — inventory counts, financial balances, order status — cannot use a replica with multiple seconds of lag without surfacing incorrect data to users. Sessions that serve analytics, historical reports, cached summaries, and background jobs are perfect replica candidates.

On RDS for PostgreSQL, replica lag is visible through the standard PostgreSQL view:

```
-- Run this on the replica to check lag
SELECT
  now() - pg_last_xact_replay_timestamp() AS replication_lag,
  pg_is_in_recovery() AS is_replica,
  pg_last_wal_receive_lsn() AS received_lsn,
  pg_last_wal_replay_lsn() AS replayed_lsn,
  pg_wal_lsn_diff(
    pg_last_wal_receive_lsn(),
    pg_last_wal_replay_lsn()
  ) AS lag_bytes;
```

In CloudWatch, RDS exposes `ReplicaLag` as a metric measured in seconds. Setting a CloudWatch alarm when `ReplicaLag` exceeds your application’s tolerance threshold — typically somewhere between 5 and 30 seconds depending on the workload — gives you early warning before replica lag becomes a user-visible problem.

Aurora handles read replicas differently from standard RDS in a way that directly affects lag. Because all Aurora instances in a cluster share the same distributed storage layer, Aurora read replicas don’t receive replicated WAL over the network. Instead, they apply log records generated by the storage layer. This results in typical replica lag of 10 to 20 milliseconds, which is an order of magnitude lower than what you’ll see with RDS read replicas under load. For applications where read-your-writes consistency matters and you’re considering routing certain reads to replicas, Aurora’s storage-level replication architecture makes that decision less risky.

Azure SQL Hyperscale introduces a different read scaling mechanism. Hyperscale read replicas are named “named replicas” or “high availability (HA) replicas” depending on their purpose. Named replicas in Hyperscale have their own endpoint, their own independent vCore allocation, and serve reads from the same page server layer. Lag in Hyperscale reads is measured in single-digit milliseconds for most workloads because replicas read from the shared page server tier rather than receiving replicated transaction logs. The practical effect is that Hyperscale read scale-out behaves more like Aurora than like traditional log-shipping replication.

Google AlloyDB provides a similar storage-sharing architecture. AlloyDB read pools — horizontally scalable groups of reader nodes — share the same distributed storage as the primary and advertise a single read pool endpoint. Adding a node to the read pool takes minutes rather than the hours required to provision and sync a traditional read replica. For workloads that need elastic read scaling during business hours, AlloyDB’s read pool architecture is significantly more operationally convenient than managing individual Cloud SQL read replicas.

9.3 Configuring Multi-AZ and Read Replicas Across Providers

The configuration mechanics differ enough across providers that it’s worth walking through each one with actual commands rather than summarizing abstractly.

AWS RDS: Enabling Multi-AZ and Creating a Read Replica

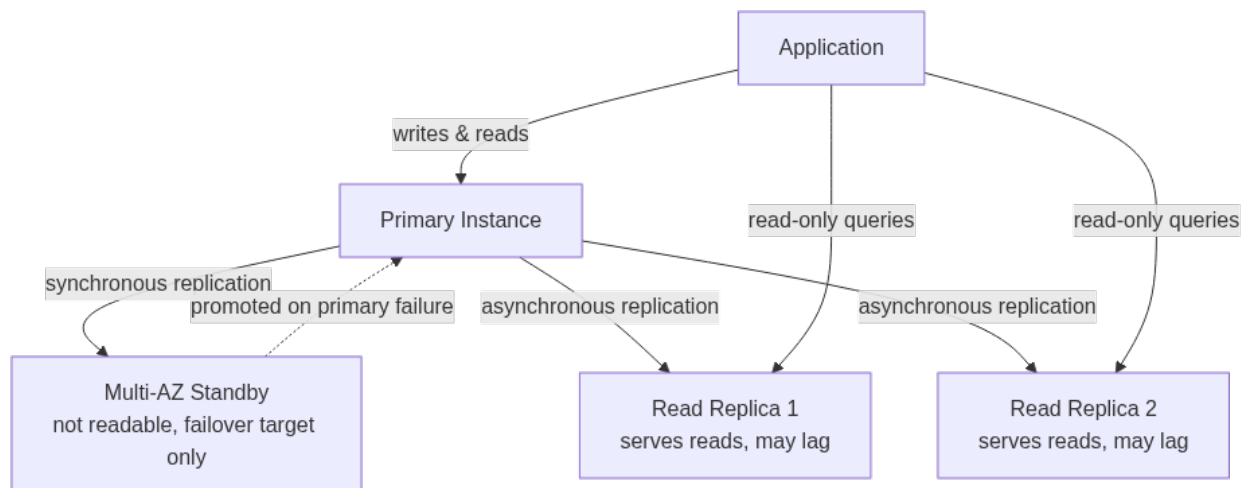


Figure 2: Multi-AZ protects availability through a synchronous, non-readable standby; read replicas scale read throughput through asynchronous, readable copies. The two mechanisms solve different problems and are typically combined, not chosen between.

You can enable Multi-AZ on an existing RDS instance with minimal downtime. AWS will provision the standby in the background, synchronize it, and then perform a brief failover to establish the synchronous replication pair. This background sync can take minutes to hours depending on database size.

```

# Enable Multi-AZ on an existing RDS instance
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-primary \
  --multi-az \
  --apply-immediately \
  --region us-east-1

# Create a read replica in the same region
aws rds create-db-instance-read-replica \
  --db-instance-identifier prod-postgres-replica-1 \
  --source-db-instance-identifier prod-postgres-primary \
  --db-instance-class db.r6g.2xlarge \
  --availability-zone us-east-1b \
  --publicly-accessible false \
  --region us-east-1

# Create a cross-region read replica (for DR read access or geo-distribution)
aws rds create-db-instance-read-replica \
  --db-instance-identifier prod-postgres-replica-us-west \
  --source-db-instance-identifier prod-postgres-primary \
  --db-instance-class db.r6g.2xlarge \
  --region us-west-2 \
  --source-region us-east-1

# Monitor replica lag via CloudWatch
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name ReplicaLag \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-replica-1 \
  --start-time "$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 60 \
  --statistics Average \
  --region us-east-1

```

Aurora: Cluster Configuration with Terraform

Aurora exposes Multi-AZ through the cluster layer rather than individual instances. The following Terraform block creates an Aurora PostgreSQL cluster with one writer and two readers, where one reader acts as the failover target and the other serves read traffic:

```
resource "aws_rds_cluster" "production" {
  cluster_identifier = "prod-aurora-cluster"
  engine             = "aurora-postgresql"
  engine_version     = "15.4"
  database_name      = "appdb"
  master_username    = "dbadmin"
  manage_master_user_password = true

  db_subnet_group_name = aws_db_subnet_group.production.name
  vpc_security_group_ids = [aws_security_group.rds.id]

  backup_retention_period = 7
  preferred_backup_window = "03:00-04:00"

  storage_encrypted = true
  kms_key_id        = aws_kms_key.rds.arn

  enabled_cloudwatch_logs_exports = ["postgresql"]

  deletion_protection = true
  skip_final_snapshot = false
  final_snapshot_identifier = "prod-aurora-final-snapshot"
}

resource "aws_rds_cluster_instance" "writer" {
  identifier = "prod-aurora-writer"
  cluster_identifier = aws_rds_cluster.production.id
  instance_class = "db.r6g.4xlarge"
  engine = aws_rds_cluster.production.engine
  engine_version = aws_rds_cluster.production.engine_version

  # Promotion tier 0 means this instance is preferred for failover promotion
  promotion_tier = 0
  availability_zone = "us-east-1a"
}

resource "aws_rds_cluster_instance" "reader_failover" {
  identifier = "prod-aurora-reader-failover"
  cluster_identifier = aws_rds_cluster.production.id
  instance_class = "db.r6g.4xlarge"
  engine = aws_rds_cluster.production.engine
  engine_version = aws_rds_cluster.production.engine_version

  # Same instance class as writer ensures promotion doesn't introduce a
  # performance cliff when this reader becomes the new writer
  promotion_tier = 1
  availability_zone = "us-east-1b"
}

resource "aws_rds_cluster_instance" "reader_read_traffic" {
  identifier = "prod-aurora-reader-reads"
  cluster_identifier = aws_rds_cluster.production.id
  instance_class = "db.r6g.2xlarge"
  engine = aws_rds_cluster.production.engine
  engine_version = aws_rds_cluster.production.engine_version

  # Lower promotion tier - this instance serves reads but won't be promoted
  # to primary unless all higher-priority readers are unavailable
  promotion_tier = 15
  availability_zone = "us-east-1c"
}
```

The `promotion_tier` setting on Aurora cluster instances is something many DBAs overlook. If you

have a large reader instance serving analytics workloads and a smaller reader instance designated for failover, you must set the analytics reader to a higher promotion tier number (lower priority). Otherwise Aurora may promote your undersized analytics reader to primary during a failover, and you've degraded your write path.

Azure SQL: Geo-Replication and Readable Secondaries

Azure SQL Database's equivalent of read replicas is Active Geo-Replication, which creates readable secondaries in other regions. Unlike RDS read replicas which are independent instances, Azure geo-secondary databases must be in an Elastic Pool or have their own service tier provisioned, and they're intended primarily for disaster recovery rather than everyday read offloading.

```
# Create an Azure SQL Database with Business Critical tier (includes HA replicas)
az sql db create \
  --resource-group prod-rg \
  --server prod-sql-server \
  --name appdb \
  --service-objective BC_Gen5_8 \
  --backup-storage-redundancy Zone \
  --zone-redundant true

# Add a geo-secondary for DR (readable secondary in another region)
az sql db replica create \
  --resource-group prod-rg \
  --server prod-sql-server \
  --name appdb \
  --partner-resource-group prod-dr-rg \
  --partner-server prod-sql-server-dr \
  --service-objective BC_Gen5_4 \
  --secondary-type Geo

# Check replication status and lag
az sql db replica list-links \
  --resource-group prod-rg \
  --server prod-sql-server \
  --name appdb \
  --output table

# Query replication lag from within Azure SQL (run on secondary)
# Azure SQL exposes this through sys.dm_database_replica_states

-- Check geo-replication lag on Azure SQL secondary
-- Run this against the secondary database
SELECT
  r.partner_server,
  r.role_desc,
  r.replication_state_desc,
  r.secondary_lag_seconds,
  r.last_commit_time,
  DATEDIFF(SECOND, r.last_commit_time, GETUTCDATE()) AS lag_seconds_computed
FROM sys.dm_geo_replication_link_status r;

-- On Business Critical tier, check internal HA replica state
SELECT
  replica_id,
  role_desc,
  operational_state_desc,
  recovery_health_desc,
  synchronization_health_desc,
  last_commit_time
FROM sys.dm_hadr_availability_replica_states
WHERE is_local = 0;
```

Cloud SQL: High Availability and Read Replica Configuration

```
# Create a Cloud SQL PostgreSQL instance with HA enabled
gcloud sql instances create prod-postgres \
```

```
--database-version=POSTGRES_15 \  
--tier=db-custom-8-32768 \  
--region=us-central1 \  
--availability-type=REGIONAL \  
--storage-type=SSD \  
--storage-size=500GB \  
--storage-auto-increase \  
--backup-start-time=03:00 \  
--retained-backups-count=7 \  
--retained-transaction-log-days=7 \  
--no-assign-ip \  
--network=projects/my-project/global/networks/prod-vpc \  
--enable-google-private-path  
  
# Add a read replica in the same region  
gcloud sql instances create prod-postgres-replica-1 \  
  --master-instance-name=prod-postgres \  
  --region=us-central1 \  
  --tier=db-custom-4-16384 \  
  --availability-type=ZONAL \  
  --no-assign-ip \  
  --network=projects/my-project/global/networks/prod-vpc  
  
# Add a cross-region read replica for geo-distributed reads  
gcloud sql instances create prod-postgres-replica-us-east \  
  --master-instance-name=prod-postgres \  
  --region=us-east1 \  
  --tier=db-custom-4-16384 \  
  --availability-type=ZONAL  
  
# Check replica lag using Cloud Monitoring  
gcloud monitoring metrics list \  
  --filter="metric.type=cloudsql.googleapis.com/database/replication/replica_lag"
```

9.4 Failover Behavior: What Actually Happens During the Event

Understanding what happens during a failover is more valuable than understanding how to configure one. Production incidents reveal the gaps in architectural assumptions, and the best time to close those gaps is before the incident, not during it.

When RDS Multi-AZ failover is triggered — by a host failure, storage failure, AZ outage, or a manual failover you initiate for testing — the sequence of events is:

1. RDS detects the primary instance is unreachable (detection typically takes 20 to 40 seconds for host failures, faster for storage failures which are detected at the volume level).
2. The standby is promoted to primary.
3. The RDS endpoint CNAME record is updated to point to the new primary's IP.
4. DNS propagation occurs — clients respecting the 5-second TTL will resolve the new IP within 5 to 10 seconds of the CNAME update.
5. New connections succeed; existing connections must be re-established.

The gap that trips production applications: existing database connections are dropped during failover. Connection pools that don't handle `OperationalError` exceptions and reconnect will sit with broken connections until their health check cycles. A well-configured connection pool — whether that's HikariCP in Java, pgbouncer, or RDS Proxy — will test connections before handing them to the application and replace broken connections automatically.

RDS Proxy addresses the failover connection problem more cleanly than any connection pool tuning.

RDS Proxy maintains a connection pool between itself and the RDS instance, and it presents a stable endpoint to applications. During failover, RDS Proxy handles the reconnection to the promoted standby internally. Applications see their connections paused briefly (the queries in flight will be retried automatically in some configurations, or will receive an error and need application-level retry) but don't need to re-resolve DNS. For serverless or bursty workloads that already benefit from connection pooling, RDS Proxy is the correct insertion point.

```
import boto3
import psycopg2
import time
from botocore.exceptions import ClientError

def initiate_and_monitor_failover(db_instance_id: str, region: str):
    """
    Initiate a manual Multi-AZ failover and monitor until completion.
    Use this in maintenance testing, not production incidents.
    """
    rds_client = boto3.client('rds', region_name=region)
    cw_client = boto3.client('cloudwatch', region_name=region)

    print(f"Initiating failover for {db_instance_id}")
    start_time = time.time()

    try:
        rds_client.reboot_db_instance(
            DBInstanceIdentifier=db_instance_id,
            ForceFailover=True
        )
    except ClientError as e:
        print(f"Failover initiation failed: {e}")
        raise

    # Poll until the instance is available again
    waiter = rds_client.get_waiter('db_instance_available')
    waiter.config.delay = 10
    waiter.config.max_attempts = 30

    print("Waiting for instance to become available after failover...")
    waiter.wait(DBInstanceIdentifiers=[db_instance_id])

    elapsed = time.time() - start_time
    print(f"Failover completed in {elapsed:.1f} seconds")

    # Verify we can connect to the new primary
    instance_info = rds_client.describe_db_instances(
        DBInstanceIdentifier=db_instance_id
    )['DBInstances'][0]

    endpoint = instance_info['Endpoint']['Address']
    new_az = instance_info['AvailabilityZone']
    print(f"New primary endpoint: {endpoint}")
    print(f"New primary AZ: {new_az}")

    return elapsed, endpoint

def test_connection_with_retry(host: str, port: int, dbname: str,
                               user: str, password: str,
                               max_retries: int = 10,
                               retry_interval: float = 5.0):
    """
    Test database connectivity with retry logic appropriate for post-failover
    reconnection. This pattern belongs in your application's connection layer.
    """
    for attempt in range(max_retries):
        try:
            conn = psycopg2.connect(
                host=host,
```



```
port=port,
dbname=dbname,
user=user,
password=password,
connect_timeout=5,
options="-c statement_timeout=10000"
)
# Confirm we're on the primary
with conn.cursor() as cur:
    cur.execute("SELECT pg_is_in_recovery(), inet_server_addr()")
    is_replica, server_addr = cur.fetchone()
    if is_replica:
        print(f"Warning: Connected to replica at {server_addr}")
    else:
        print(f"Connected to primary at {server_addr}")
conn.close()
return True
except psycopg2.OperationalError as e:
    print(f"Attempt {attempt + 1}/{max_retries} failed: {e}")
    if attempt < max_retries - 1:
        time.sleep(retry_interval)

return False
```

For Aurora, the failover process deserves a different mental model. Aurora maintains a cluster endpoint (the writer endpoint) and a reader endpoint that automatically load-balances across all healthy reader instances. When the writer fails, Aurora promotes one of the readers based on promotion tier order. Because readers share the storage layer, there is no redo log replay phase — promotion is essentially a metadata operation. The practical result is that Aurora writer failover typically completes in 30 seconds or less, while standard RDS Multi-AZ typically takes 60 to 120 seconds.

Azure SQL Business Critical failover is fast — typically under 30 seconds — because the Always On AG secondaries are already synchronized and one of them is always designated as the failover target. Azure SQL failover is transparent to applications using the server FQDN as the connection endpoint; the internal AG mechanics are not exposed. The main operational concern with Azure SQL failover is connection string validation: Azure SQL requires that your connection string targets the server name, not a specific IP, and that your firewall rules or private endpoint configuration is correct on both sides.

9.5 Architectural Decision Framework

With the mechanics of each mechanism understood across providers, the architectural decision reduces to a set of questions you answer about your workload characteristics, your SLA requirements, and your budget.

Decision 1: Do you need Multi-AZ at all?

The honest answer is yes for any production workload where unplanned downtime costs more than the Multi-AZ premium. Multi-AZ roughly doubles your instance cost (you're paying for the standby even though it serves no traffic in normal operation). The cases where Multi-AZ is genuinely optional are development and staging environments, short-lived analytical workloads, and internal tooling where an hour of downtime is annoying but not financially damaging. For anything customer-facing or revenue-impacting, the standby cost is straightforward insurance.

The secondary question is which tier of Multi-AZ you need. Standard RDS Multi-AZ gives you a single standby. Aurora Multi-AZ through multiple readers gives you faster failover and the ability to designate a same-class reader as the failover target while a smaller instance serves read traffic. If failover time matters enough that 60 seconds versus 30 seconds is part of your SLA, Aurora's storage architecture is the architectural argument for choosing it over standard RDS.

Decision 2: Do you have read scalability needs or just availability needs?

These are separate axes. A write-heavy OLTP system with modest read volume may need Multi-AZ for availability but zero read replicas — all the reads can comfortably run on the primary. A reporting-heavy system may run on a single instance with several read replicas. A high-volume consumer application may need both: Multi-AZ for availability and read replicas to horizontally scale read throughput.

The test for whether you need read replicas is whether your primary's CPU or I/O is constrained by read workload during peak hours. If `pg_stat_activity` on PostgreSQL or `sys.dm_exec_requests` on SQL Server shows your primary spending significant time on SELECT queries that could tolerate replica lag, you have a read replica case. If the primary is constrained by writes, adding replicas helps your reads but does nothing for your bottleneck.

```
-- PostgreSQL: Identify read vs. write load distribution on primary
-- Use this to determine whether read offloading would help

SELECT
    wait_event_type,
    wait_event,
    state,
    query_start,
    left(query, 80) AS query_snippet,
    COUNT(*) AS session_count
FROM pg_stat_activity
WHERE state != 'idle'
    AND pid != pg_backend_pid()
GROUP BY wait_event_type, wait_event, state, query_start, query_snippet
ORDER BY session_count DESC
LIMIT 20;

-- Classify active queries by type to estimate read/write ratio
SELECT
    CASE
        WHEN query ILIKE 'SELECT%' THEN 'read'
        WHEN query ILIKE 'INSERT%'
            OR query ILIKE 'UPDATE%'
            OR query ILIKE 'DELETE%' THEN 'write'
        WHEN query ILIKE 'WITH%' THEN 'read_cte'
        ELSE 'other'
    END AS query_type,
    COUNT(*) AS active_count
FROM pg_stat_activity
WHERE state = 'active'
    AND pid != pg_backend_pid()
GROUP BY query_type
ORDER BY active_count DESC;

-- SQL Server equivalent: classify active requests by read vs. write
-- Run on Azure SQL or SQL Server
SELECT
    CASE
        WHEN t.text LIKE 'SELECT%' THEN 'read'
        WHEN t.text LIKE 'INSERT%'
            OR t.text LIKE 'UPDATE%'
            OR t.text LIKE 'DELETE%' THEN 'write'
        WHEN t.text LIKE 'WITH%' THEN 'read_cte'
        ELSE 'other'
    END AS query_type,
```

```
END AS query_type,
COUNT(*) AS active_count,
AVG(r.cpu_time) AS avg_cpu_ms,
AVG(r.reads) AS avg_logical_reads
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.session_id != @@SPID
AND r.status = 'running'
GROUP BY
CASE
    WHEN t.text LIKE 'SELECT%' THEN 'read'
    WHEN t.text LIKE 'INSERT%'
        OR t.text LIKE 'UPDATE%'
        OR t.text LIKE 'DELETE%' THEN 'write'
    WHEN t.text LIKE 'WITH%' THEN 'read_cte'
    ELSE 'other'
END
ORDER BY active_count DESC;
```

Decision 3: Can your application tolerate replica lag, and how much?

This is an application architecture question, not a database question. The database will have lag. The question is whether your code paths can safely consume potentially stale reads.

The categories that are generally safe for replica routing:

- **Analytics and reporting queries:** Historical reports, aggregations over past periods, business intelligence dashboards that are explicitly described as “as of” a certain time. A sales report showing yesterday’s numbers doesn’t need to be reading from the primary.
- **Search and catalog reads:** Product catalog, content listings, search index support queries. These are typically updated through background processes, and a 100ms lag is invisible to users.
- **Session-independent reads:** Queries that don’t follow a user write operation within the same session. If a user submitted a form and you’re immediately redirecting them to see what they submitted, that read must come from the primary. If you’re loading their dashboard on next login, replica lag of seconds is invisible.
- **Background jobs and data exports:** Any process that doesn’t surface results directly to a waiting user is an ideal replica consumer.

The categories that require primary reads:

- **Read-your-own-writes:** Immediately after any write operation, reads of that data within the same user session must come from the primary or from a replica that has confirmed it has replayed through that LSN.
- **Financial and inventory operations:** Account balances, stock levels, seat availability. Stale reads here produce real business errors.
- **Distributed lock and sequence coordination:** Anything that reads state to make a conditional write decision must read current state from the primary.

Decision 4: How many replicas do you need, and where do you put them?

The scaling math for read replicas is straightforward in concept but subtle in practice. If your primary is receiving 50,000 read queries per second and each replica can handle 40,000 queries per second, you need more than one replica to provide headroom. But you also need to account for replica failover — if you have two replicas and one fails, the remaining replica receives 100% of the read traffic that was previously split. Size for the failure case, not the steady state.

For cross-region replicas, the motivation shifts from read scaling to geographic proximity and disaster recovery positioning. A replica in `us-west-2` serving a user population on the US West Coast reduces round-trip latency for those reads. That same replica, if promoted to standalone during a regional outage of `us-east-1`, becomes your DR database. Cross-region replicas are more complex to manage: replication lag is higher due to distance, promoting them to standalone (breaking the replication link) is an irreversible operation that requires application reconfiguration, and they cost more in data transfer fees.

```
# Aurora: promote a read replica to a standalone cluster
# (used for DR scenarios or regional expansion, not routine failover)
aws rds promote-read-replica-db-cluster \
  --db-cluster-identifier prod-aurora-replica-us-west \
  --region us-west-2

# RDS: promote a read replica to standalone instance
# This breaks the replication link permanently
aws rds promote-read-replica \
  --db-instance-identifier prod-postgres-replica-us-west \
  --region us-west-2

# Cloud SQL: promote a read replica to primary
gcloud sql instances promote-replica prod-postgres-replica-us-east \
  --region=us-east1
```

Decision 5: Where does RDS Proxy or PgBouncer fit?

Connection management becomes a separate architectural decision once you have Multi-AZ and read replicas in play. The naive configuration — application directly connecting to RDS endpoints — works until it doesn't. When it breaks, it breaks during failovers (when connection counts spike as every application thread tries to reconnect simultaneously), during traffic spikes (when your connection limit is exceeded because your application doesn't pool properly), and during Lambda-scale serverless deployments (where each function invocation may open a new connection).

RDS Proxy sits between your application and RDS/Aurora, maintains a persistent connection pool to the database, and provides a stable endpoint through failovers. It's the correct choice when:

- Your application uses serverless compute (Lambda functions opening raw database connections is an antipattern that RDS Proxy corrects)
- Your application cannot be configured to honor DNS TTL properly
- Your connection count frequently approaches the database instance limit
- You want IAM authentication at the proxy layer rather than managing database passwords in application code

PgBouncer is the correct choice when:

- You're running on PostgreSQL and want transaction-mode pooling to minimize held connections
- You're managing your own EC2-based infrastructure and want more operational control than RDS Proxy provides
- Your workload characteristics (many short-lived connections, transaction-mode acceptable) fit PgBouncer's strength
- Cost is a factor and you want to avoid the RDS Proxy per-connection pricing

For Azure SQL, the connection management story is different because Azure SQL itself implements connection throttling and pooling at the service level. Standard SQL Server connection pooling in ADO.NET, the JDBC driver, or pyodbc is generally sufficient. For very high connection count

scenarios, Azure SQL Elastic Pools provide connection sharing across multiple databases.

9.6 Testing Failover in Production-Like Environments

The architecture decisions above mean nothing if you haven't validated failover behavior under realistic conditions. Untested failover procedures fail in production in the same way untested backup procedures produce unusable backups.

The minimum failover test validates:

1. Your application detects the disconnection and retries appropriately
2. Failover completes within your SLA window
3. The promoted standby handles production traffic levels without degradation
4. Your monitoring fires the correct alerts during and after failover
5. The new standby (the old primary, restored and replication re-established) comes back online within your expected window

A more thorough test injects failure during active load:

```
import threading
import time
import psycopg2
import boto3
from datetime import datetime
from queue import Queue

def continuous_write_load(host, port, dbname, user, password,
                          stop_event, results_queue):
    """
    Generate continuous write load during failover testing.
    Track errors and recovery timing.
    """
    conn = None
    write_count = 0
    error_count = 0
    first_error_time = None
    recovery_time = None

    while not stop_event.is_set():
        try:
            if conn is None or conn.closed:
                conn = psycopg2.connect(
                    host=host,
                    port=port,
                    dbname=dbname,
                    user=user,
                    password=password,
                    connect_timeout=5
                )
                conn.autocommit = False
                if first_error_time and recovery_time is None:
                    recovery_time = datetime.now()
                    results_queue.put({
                        'event': 'recovered',
                        'time': recovery_time,
                        'gap_seconds': (recovery_time - first_error_time).total_seconds()
                    })

            with conn.cursor() as cur:
                cur.execute(
                    "INSERT INTO failover_test (ts, payload) VALUES (%s, %s)",
```

```

        (datetime.now(), f"test_write_{write_count}")
    )
    conn.commit()
    write_count += 1
    time.sleep(0.1)

except psycopg2.OperationalError as e:
    if first_error_time is None:
        first_error_time = datetime.now()
        results_queue.put({
            'event': 'first_error',
            'time': first_error_time,
            'error': str(e)
        })
    error_count += 1
    if conn and not conn.closed:
        try:
            conn.close()
        except Exception:
            pass
    conn = None
    time.sleep(1)

results_queue.put({
    'event': 'summary',
    'total_writes': write_count,
    'total_errors': error_count,
    'first_error': first_error_time,
    'recovered_at': recovery_time
})

def run_failover_test(db_instance_id, host, port, dbname, user, password, region):
    """
    Orchestrate a full failover test with concurrent write load.
    Run this against a staging or pre-production environment.
    """
    rds = boto3.client('rds', region_name=region)
    results = Queue()
    stop_event = threading.Event()

    # Ensure test table exists
    setup_conn = psycopg2.connect(
        host=host, port=port, dbname=dbname,
        user=user, password=password
    )
    with setup_conn.cursor() as cur:
        cur.execute("""
            CREATE TABLE IF NOT EXISTS failover_test (
                id SERIAL PRIMARY KEY,
                ts TIMESTAMPTZ NOT NULL,
                payload TEXT
            )
        """)
    setup_conn.commit()
    setup_conn.close()

    # Start background write load
    writer_thread = threading.Thread(
        target=continuous_write_load,
        args=(host, port, dbname, user, password, stop_event, results),
        daemon=True
    )
    writer_thread.start()

    # Let write load establish for 10 seconds
    print("Establishing write load for 10 seconds...")
    time.sleep(10)

    # Initiate failover

```

```
print(f"Initiating failover at {datetime.now()}")
failover_initiated = datetime.now()
rds.reboot_db_instance(
    DBInstanceIdentifier=db_instance_id,
    ForceFailover=True
)

# Wait for instance to come back
waiter = rds.get_waiter('db_instance_available')
waiter.wait(DBInstanceIdentifiers=[db_instance_id])
failover_complete = datetime.now()
print(f"Instance available at {failover_complete}")
print(f"Failover took {(failover_complete - failover_initiated).total_seconds():.1f}s")

# Continue running for 30 seconds post-failover to confirm stability
time.sleep(30)
stop_event.set()
writer_thread.join(timeout=15)

# Collect and display results
while not results.empty():
    result = results.get()
    print(f"Event: {result}")
```

Run this test regularly — quarterly at minimum, monthly if your SLA is aggressive — and treat the results as a living document of your actual failover behavior. The numbers you get from cloud provider documentation are typical values; your actual values depend on your instance size, your storage configuration, your connection pool behavior, and your application’s reconnection logic.

9.7 Cost Implications of Multi-AZ vs Read Replica Choices

Architecture decisions that ignore cost are incomplete. The pricing model for high availability and read scaling differs enough across providers to meaningfully influence design choices.

AWS RDS Multi-AZ pricing: The standby instance is billed at the same rate as the primary. Enabling Multi-AZ on a `db.r6g.4xlarge` instance effectively doubles the instance cost. Storage is provisioned on both instances. I/O is charged once (the standby doesn’t generate separate I/O charges for replicated writes, as those go through the primary’s volume). For a large production instance, Multi-AZ adds significant monthly cost, which makes the decision to skip it for non-production environments financially clear.

AWS Aurora pricing: Aurora charges for the storage layer once regardless of how many readers share it. The cost of adding a reader instance is the instance compute cost only — no additional storage per-reader. This means that for Aurora, adding a reader instance that functions as both a read scaler and a failover target costs only the compute for that reader instance. This is often cheaper than the RDS Multi-AZ model while providing faster failover.

RDS Read Replicas: Each read replica is a full instance with its own compute and storage charges. A replica in the same region has no data transfer cost between primary and replica. A cross-region replica incurs data transfer fees on all replicated bytes, which for write-heavy workloads can become significant. For terabyte-scale databases with multi-gigabyte-per-hour write rates, cross-region replica data transfer costs should be explicitly modeled before commitment.

Azure SQL Business Critical vs General Purpose: The Business Critical tier’s always-on HA replicas are not separately priced — you’re paying the Business Critical vCore price, which is

substantially higher than General Purpose, and HA plus one readable secondary are included. If you need the readable secondary for reporting workloads, this is potentially cost-effective compared to provisioning a separate reporting server. If you only need HA and not the readable secondary, General Purpose with zone-redundancy may be more cost-effective and has improved failover times compared to the earlier versions of that tier.

Google Cloud SQL HA: The Cloud SQL HA standby incurs compute charges for the standby instance (at a reduced rate compared to the primary) plus storage charges for the standby. AlloyDB's pricing model differs — the compute pool and storage are priced separately, and the distributed storage is shared across nodes, making the AlloyDB read pool more cost-efficient for workloads with many readers.

The cost optimization discipline is to match the tier of protection to the actual risk and business value: Multi-AZ on production, single-AZ on staging, read replicas sized to handle the failure case, and cross-region replicas only where the geographic or DR value justifies the transfer cost.

Key Takeaways

- **Multi-AZ protects availability; read replicas scale read throughput.** These solve different problems and should be chosen independently based on whether your constraint is unplanned downtime or read capacity. Enabling one does not replace the need for the other.
- **Failover time and connection handling are the operational realities that matter more than configuration.** Multi-AZ failover takes 30 to 120 seconds depending on the provider and tier, and applications that cache DNS resolutions or don't retry broken connections will fail even when the database successfully promotes a standby. DNS TTL compliance and connection pool retry logic are prerequisites for Multi-AZ to deliver on its promise.
- **Aurora's storage-sharing architecture produces materially faster failover and lower replica lag than standard RDS Multi-AZ and read replicas.** The tradeoff is Aurora's pricing model and minimum storage costs. For workloads where sub-30-second failover or single-digit-millisecond replica lag is a requirement, the Aurora architecture justifies the cost difference.
- **Replica routing decisions belong at the application layer, and the safe categories are narrower than most teams assume.** Analytics, historical reports, background jobs, and session-independent catalog reads are safe replica candidates. Read-your-own-writes, financial balances, and inventory counts must read from the primary. Systematic classification of queries by safety for replica routing prevents silent correctness bugs.
- **Test failover under load on a regular schedule.** The topology you've configured behaves differently from how you expect it to behave until you've observed it failing and recovering under production-representative load. Quarterly failover drills with concurrent write load and application monitoring active are the minimum for any Multi-AZ deployment backing a customer-facing service.

Chapter 10: RDS Proxy: Connection Pooling at Scale

Connection pooling is a problem every production DBA has solved before — PgBouncer on a dedicated VM, SQL Server’s built-in connection pooling through the driver, ProxySQL sitting between your application tier and MySQL. You know the problem well: thousands of application threads each wanting their own database connection, a database engine that pays real memory and CPU cost per connection, and an impedance mismatch between how application frameworks think about concurrency and how database engines actually work. In the cloud, that problem doesn’t disappear — it gets worse. Serverless functions spin up hundreds of ephemeral connections in milliseconds. Auto-scaling application tiers surge from 10 to 500 instances in minutes. Lambda functions have no persistent process to maintain a connection pool. RDS Proxy is AWS’s managed answer to this problem, and understanding it deeply — its architecture, its limits, its failure modes — is what separates a cloud DBA who simply enables it from one who operates it confidently in production.

10.1 Why Connection Pooling Is Different in the Cloud

On-premises, you controlled both ends of the connection. You knew exactly how many application servers existed, you sized PgBouncer or pgpool-II based on those numbers, and you tuned `max_connections` in `postgresql.conf` to match. The population was relatively static. In a cloud-native architecture, the application tier is ephemeral by design.

Consider what happens when you deploy an AWS Lambda function that queries RDS PostgreSQL. Lambda has a concurrency limit of up to 10,000 simultaneous executions per account per region by default. If each invocation opens a database connection — even briefly — and your RDS PostgreSQL instance is sized at `db.r6g.2xlarge` with 64 GB RAM, your practical `max_connections` default is capped at 5,000 (PostgreSQL’s default formula is `LEAST(DBInstanceClassMemory/9531392, 5000)` for RDS — at 64 GB of instance memory the formula’s uncapped result already exceeds 5,000, so the hard cap is what applies; you can verify the actual value by querying the parameter group). A traffic spike that triggers 2,000 concurrent Lambda invocations doesn’t just degrade performance — it throws **FATAL: sorry, too many clients already** errors and your application fails completely.

The same dynamic applies to containerized applications on ECS or EKS where horizontal pod autoscaling responds faster than your database can absorb connection growth. Each new pod starts

fresh, establishes its own connections, and the old pod's connections may linger in `TIME_WAIT` or hold idle transactions open while the process terminates.

What RDS Proxy does architecturally is sit as a fully managed, highly available connection pooling layer between your clients and your RDS or Aurora database. It maintains a pool of long-lived connections to the database and multiplexes thousands of client connections across that pool. From your database's perspective, it sees a small, stable number of connections from the proxy. From your application's perspective, it sees a normal database endpoint.

The fundamental multiplexing mechanism is called *connection pinning* in RDS Proxy's terminology — and understanding when pinning happens is the most important operational concept in this chapter.

Connection pinning occurs when RDS Proxy cannot safely multiplex a client connection across the shared pool and must dedicate a backend database connection exclusively to that client for the duration of the session. When a connection is pinned, you lose the pooling benefit entirely for that client. Pinning is triggered by:

- `SET` statements that change session-level configuration (e.g., `SET search_path TO myschema`)
- Temporary tables (`CREATE TEMP TABLE`)
- Table locks (`LOCK TABLE`)
- Cursors opened with `HOLD`
- Prepared statements (in certain configurations)
- `SET TRANSACTION ISOLATION LEVEL` before a transaction begins

For PostgreSQL users who relied on `SET search_path` in connection strings — a common pattern — this is an immediate compatibility issue. The fix is to set `search_path` at the user level in the database itself (`ALTER ROLE myapp SET search_path TO myschema`) rather than at session level in the connection string.

For SQL Server users, RDS Proxy does not support SQL Server. RDS Proxy supports MySQL-compatible engines (RDS MySQL, Aurora MySQL) and PostgreSQL-compatible engines (RDS PostgreSQL, Aurora PostgreSQL). SQL Server on RDS still uses driver-level connection pooling. Keep that limitation in mind throughout this chapter.

Maximum connections math: When you create an RDS Proxy, you set a `--max-connections-percent` parameter (1–100, default 100). This tells the proxy what percentage of the database's `max_connections` it may use for the backend pool. The proxy itself can accept far more client connections than that — check current AWS documentation for the exact client-connection ceiling, but it is designed to comfortably exceed the backend connection pool by a wide margin. The multiplexing ratio is what gives you scale. If your `db.r6g.2xlarge` has `max_connections = 1700` and you leave `max-connections-percent` at 100, the proxy will hold up to 1,700 backend connections but can serve tens of thousands of client connections across them — provided those client connections aren't all executing transactions simultaneously and aren't pinned.

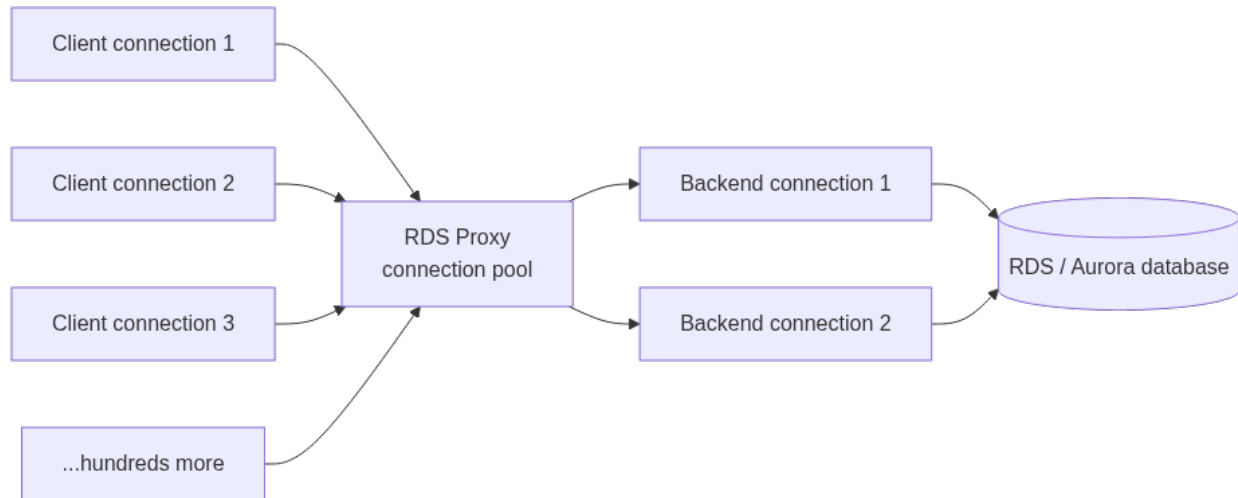


Figure 3: RDS Proxy multiplexes many short-lived or idle client connections onto a much smaller, warm pool of backend database connections — until a client is pinned, at which point it falls back to a dedicated one-to-one connection.

10.2 Creating and Configuring RDS Proxy

RDS Proxy runs inside your VPC. It creates network interfaces in your subnets and registers against your RDS endpoint. The proxy itself is managed by AWS — you have no EC2 instances to patch, no process to restart — but you do have configuration choices that matter.

IAM authentication is mandatory for RDS Proxy. The proxy authenticates to your database using credentials stored in AWS Secrets Manager. This is non-negotiable — you cannot use a proxy without Secrets Manager. The proxy fetches credentials from Secrets Manager on your behalf, which means the IAM role attached to the proxy must have `secretsmanager:GetSecretValue` on the specific secret. Your application, meanwhile, can authenticate to the proxy either using the same database credentials (passed through) or using IAM authentication (where the application gets an auth token from STS and uses it as a password).

Here is the sequence using AWS CLI to create a proxy for an Aurora PostgreSQL cluster:

```
# Step 1: Create the Secrets Manager secret for the database user
aws secretsmanager create-secret \
  --name "prod/aurora-pg/appuser" \
  --description "RDS Proxy credentials for appuser" \
  --secret-string '{"username":"appuser","password":"StrongPassword123!"}' \
  --region us-east-1

# Step 2: Get the secret ARN (you'll need this)
SECRET_ARN=$(aws secretsmanager describe-secret \
  --secret-id "prod/aurora-pg/appuser" \
  --query 'ARN' \
  --output text)

# Step 3: Create an IAM role for RDS Proxy
# First, create the trust policy document
cat > /tmp/proxy-trust-policy.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
```

```

    "Principal": {
      "Service": "rds.amazonaws.com"
    },
    "Action": "sts:AssumeRole"
  }
]
}
EOF

aws iam create-role \
  --role-name RDSProxyRole \
  --assume-role-policy-document file:///tmp/proxy-trust-policy.json

# Step 4: Create and attach the permissions policy
cat > /tmp/proxy-permissions.json << EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
      "Resource": "${SECRET_ARN}"
    },
    {
      "Effect": "Allow",
      "Action": "kms:Decrypt",
      "Resource": "arn:aws:kms:us-east-1:123456789012:key/your-kms-key-id",
      "Condition": {
        "StringEquals": {
          "kms:ViaService": "secretsmanager.us-east-1.amazonaws.com"
        }
      }
    }
  ]
}
}
EOF

aws iam put-role-policy \
  --role-name RDSProxyRole \
  --policy-name RDSProxySecretsAccess \
  --policy-document file:///tmp/proxy-permissions.json

ROLE_ARN=$(aws iam get-role --role-name RDSProxyRole --query 'Role.Arn' --output text)

# Step 5: Create the RDS Proxy
aws rds create-db-proxy \
  --db-proxy-name prod-aurora-pg-proxy \
  --engine-family POSTGRES \
  --auth '[
    {
      "Description": "App user credentials",
      "AuthScheme": "SECRETS",
      "SecretArn": "'${SECRET_ARN}'",
      "IAMAuth": "DISABLED"
    }
  ]' \
  --role-arn "${ROLE_ARN}" \
  --vpc-subnet-ids subnet-0a1b2c3d subnet-0e4f5a6b subnet-0c7d8e9f \
  --vpc-security-group-ids sg-0123456789abcdef0 \
  --require-tls \
  --idle-client-timeout 1800 \
  --debug-logging false \
  --region us-east-1

# Step 6: Register the Aurora cluster as the proxy target
aws rds register-db-proxy-targets \
  --db-proxy-name prod-aurora-pg-proxy \
  --db-cluster-identifiers prod-aurora-cluster \
  --region us-east-1

```

```
# Step 7: Verify proxy status (takes 3-5 minutes to become available)
aws rds describe-db-proxies \
  --db-proxy-name prod-aurora-pg-proxy \
  --query 'DBProxies[0].{Status:Status,Endpoint:Endpoint}' \
  --output table
```

The `--idle-client-timeout` parameter (in seconds, default 1800) determines how long the proxy waits before closing a client connection that has sent no traffic. This is distinct from the connection keep-alive at the backend. Tune this to be shorter than any NAT gateway or load balancer timeout in your network path — hitting a silent TCP timeout from a middlebox is harder to debug than an explicit proxy timeout.

Terraform configuration for the same setup, which is how you should manage this in production:

```
# terraform/modules/rds_proxy/main.tf

resource "aws_secretsmanager_secret" "db_credentials" {
  name           = "prod/aurora-pg/appuser"
  description    = "RDS Proxy credentials for appuser"
  recovery_window_in_days = 7

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
  }
}

resource "aws_secretsmanager_secret_version" "db_credentials" {
  secret_id = aws_secretsmanager_secret.db_credentials.id
  secret_string = jsonencode({
    username = "appuser"
    password = var.db_password # sourced from Terraform variable, never hardcoded
  })
}

resource "aws_iam_role" "rds_proxy" {
  name = "rds-proxy-role-${var.environment}"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { Service = "rds.amazonaws.com" }
      Action = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy" "rds_proxy_secrets" {
  name = "rds-proxy-secrets-access"
  role = aws_iam_role.rds_proxy.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = "secretsmanager:GetSecretValue"
      Resource = aws_secretsmanager_secret.db_credentials.arn
    }]
  })
}

resource "aws_db_proxy" "main" {
  name           = "prod-aurora-pg-proxy"
  debug_logging  = false
  engine_family  = "POSTGRESQL"
  idle_client_timeout = 1800
  require_tls    = true
}
```

```

role_arn          = aws_iam_role.rds_proxy.arn
vpc_security_group_ids = [aws_security_group.proxy.id]
vpc_subnet_ids    = var.private_subnet_ids

auth {
  auth_scheme = "SECRETS"
  description = "App user credentials"
  iam_auth    = "DISABLED"
  secret_arn  = aws_secretsmanager_secret.db_credentials.arn
}

tags = {
  Environment = "production"
  ManagedBy   = "terraform"
}

resource "aws_db_proxy_default_target_group" "main" {
  db_proxy_name = aws_db_proxy.main.name

  connection_pool_config {
    connection_borrow_timeout = 120 # seconds to wait for a connection from pool
    max_connections_percent   = 90  # leave 10% headroom for direct connections
    max_idle_connections_percent = 50 # proxy keeps this % of max as idle connections
    session_pinning_filters    = ["EXCLUDE_VARIABLE_SETS"] # reduces pinning for SET statements
  }
}

resource "aws_db_proxy_target" "aurora_cluster" {
  db_cluster_identifier = aws_rds_cluster.main.id
  db_proxy_name         = aws_db_proxy.main.name
  target_group_name     = aws_db_proxy_default_target_group.main.name
}

resource "aws_security_group" "proxy" {
  name        = "rds-proxy-sg"
  description = "Security group for RDS Proxy"
  vpc_id      = var.vpc_id

  ingress {
    from_port = 5432
    to_port   = 5432
    protocol  = "tcp"
    security_groups = [var.application_sg_id] # only app tier can reach proxy
  }

  egress {
    from_port = 5432
    to_port   = 5432
    protocol  = "tcp"
    security_groups = [var.rds_sg_id] # proxy can reach RDS
  }
}

output "proxy_endpoint" {
  value        = aws_db_proxy.main.endpoint
  description = "RDS Proxy endpoint for application connection strings"
}

```

The `session_pinning_filters = ["EXCLUDE_VARIABLE_SETS"]` setting in the connection pool configuration is worth highlighting. When enabled, the proxy does not pin connections when it encounters `SET` statements for a specific list of safe variables — those that don't affect transaction semantics. Check the current AWS documentation for which variables qualify, as the list expands with proxy engine updates. This reduces pinning significantly for applications that use `SET` for read-only configuration like `application_name` or `TimeZone`.

The `connection_borrow_timeout` (120 seconds by default) is the time a client waits for the proxy

to make a backend connection available before returning a timeout error. If you see applications timing out at exactly 120 seconds during traffic spikes, this is why — and the fix is usually to reduce `max_connections_percent` slightly so the proxy maintains more idle connections available for borrowing rather than running the pool at full saturation.

10.3 IAM Authentication Through the Proxy

IAM database authentication through RDS Proxy adds a second authentication layer and changes how you think about credential management. Instead of application code storing database passwords, applications generate short-lived authentication tokens using AWS STS. Those tokens expire after 15 minutes and are cryptographically tied to the IAM identity that generated them.

The flow is: 1. Your EC2 instance, Lambda function, or ECS task has an IAM role attached. 2. The application code calls `rds.generate_db_auth_token()` (boto3) to generate a token. 3. The application connects to the proxy endpoint using that token as the password. 4. The proxy validates the token against IAM, then presents stored credentials to the actual database.

This is the cloud equivalent of what you used to manage with `.pgpass` files, SSL certificates, and Kerberos — except IAM handles the identity federation and there are no long-lived passwords in your application configuration. What you did with `pg_hba.conf` entries and certificate authentication, you now do with IAM policies that specify which IAM principals can generate tokens for which database users.

```
# Python application code: IAM auth token generation for RDS Proxy
# This pattern works for Lambda, ECS tasks, and EC2 instances

import boto3
import psycopg2
import os
from botocore.exceptions import ClientError

def get_db_connection_iam():
    """
    Establish a database connection using IAM authentication through RDS Proxy.
    The IAM role attached to this execution environment must have
    rds-db:connect permission for the target database user.
    """

    rds_client = boto3.client('rds', region_name='us-east-1')

    proxy_endpoint = os.environ['DB_PROXY_ENDPOINT']
    # e.g., "prod-aurora-pg-proxy.proxy-abc123.us-east-1.rds.amazonaws.com"

    db_user = os.environ['DB_USER'] # "appuser" - must match IAM policy resource
    db_name = os.environ['DB_NAME'] # "production"
    db_port = int(os.environ.get('DB_PORT', '5432'))

    try:
        # Token is valid for 15 minutes; generate fresh on each connection
        # For connection pools in long-running processes, regenerate before expiry
        auth_token = rds_client.generate_db_auth_token(
            DBHostname=proxy_endpoint,
            Port=db_port,
            DBUsername=db_user,
            Region='us-east-1'
        )
    except ClientError as e:
        raise RuntimeError(f"Failed to generate IAM auth token: {e.response['Error']['Message']}")
```

```

# SSL is required for IAM authentication - the proxy enforces TLS
# Download the RDS certificate bundle from AWS:
# https://truststore.pki.rds.amazonaws.com/us-east-1/us-east-1-bundle.pem
ssl_cert_path = '/etc/ssl/certs/us-east-1-bundle.pem'

conn = psycopg2.connect(
    host=proxy_endpoint,
    port=db_port,
    database=db_name,
    user=db_user,
    password=auth_token,
    sslmode='verify-full',
    sslrootcert=ssl_cert_path,
    connect_timeout=10,
    application_name='my-production-service'
)

return conn

def get_db_connection_pool_iam():
    """
    For long-running services (ECS, EC2), use a connection pool with token refresh.
    psycopg2's connection pool doesn't handle token expiry natively -
    this wrapper handles the 15-minute token refresh requirement.
    """
    import psycopg2.pool
    import threading
    import time

    class IAMConnectionPool:
        def __init__(self, min_conn, max_conn, **conn_kwargs):
            self.conn_kwargs = conn_kwargs
            self.min_conn = min_conn
            self.max_conn = max_conn
            self._lock = threading.Lock()
            self._pool = None
            self._token_expiry = 0
            self._refresh_pool()

        def _generate_token(self):
            rds_client = boto3.client('rds', region_name='us-east-1')
            return rds_client.generate_db_auth_token(
                DBHostname=self.conn_kwargs['host'],
                Port=self.conn_kwargs['port'],
                DBUsername=self.conn_kwargs['user'],
                Region='us-east-1'
            )

        def _refresh_pool(self):
            """Recreate the pool with a fresh token. Called when token approaches expiry."""
            token = self._generate_token()
            self._token_expiry = time.time() + (14 * 60) # refresh at 14 min, expires at 15

            if self._pool:
                self._pool.closeall()

            self._pool = psycopg2.pool.ThreadedConnectionPool(
                self.min_conn,
                self.max_conn,
                password=token,
                **self.conn_kwargs
            )

        def getconn(self):
            with self._lock:
                if time.time() > self._token_expiry:
                    self._refresh_pool()
            return self._pool.getconn()

```



```

def putconn(self, conn):
    self._pool.putconn(conn)

return IAMConnectionPool(
    min_conn=2,
    max_conn=20,
    host=os.environ['DB_PROXY_ENDPOINT'],
    port=5432,
    database=os.environ['DB_NAME'],
    user=os.environ['DB_USER'],
    sslmode='verify-full',
    sslrootcert='/etc/ssl/certs/us-east-1-bundle.pem',
    connect_timeout=10
)

# Lambda handler example - Lambda has no persistent pool,
# but RDS Proxy handles the pooling on the server side
def lambda_handler(event, context):
    conn = None
    try:
        conn = get_db_connection_iam()
        with conn.cursor() as cur:
            cur.execute("SELECT COUNT(*) FROM orders WHERE status = %s", ('pending',))
            count = cur.fetchone()[0]
        conn.commit()
        return {'pending_orders': count}
    except Exception as e:
        if conn:
            conn.rollback()
        raise
    finally:
        if conn:
            conn.close() # Returns connection to proxy pool, not actual DB disconnection

```

The IAM policy that allows this token generation must specify the database user as part of the resource ARN:

```

# Create IAM policy permitting IAM DB auth for specific user through the proxy
aws iam create-policy \
  --policy-name RDSProxyIAMAuth \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": "rds-db:connect",
        "Resource": "arn:aws:rds-db:us-east-1:123456789012:dbuser:prx-0a1b2c3d4e5f/appuser"
      }
    ]
  }'

# Attach this policy to the Lambda execution role or ECS task role
aws iam attach-role-policy \
  --role-name MyLambdaExecutionRole \
  --policy-arn arn:aws:iam::123456789012:policy/RDSProxyIAMAuth

# The resource ARN format for proxy IAM auth is:
# arn:aws:rds-db:{region}:{account-id}:dbuser:{proxy-resource-id}/{db-username}
# Get the proxy resource ID:
aws rds describe-db-proxies \
  --db-proxy-name prod-aurora-pg-proxy \
  --query 'DBProxies[0].DBProxyArn' \
  --output text
# Extract "prx-XXXXXXXXXX" from the ARN - that's the resource ID portion

```

One gotcha that bites teams regularly: the IAM resource ARN uses the **proxy resource ID** (the `prx-` identifier), not the proxy name or the cluster ARN. When the policy is wrong, the application

gets a generic authentication failure with no indication that IAM authorization is the cause — the proxy rejects the token before the database is involved, and the error looks identical to a wrong password. Always test token generation separately from the connection attempt when debugging IAM auth failures.

10.4 Monitoring Proxy Performance and Connection Pinning

CloudWatch publishes RDSProxy metrics under the `AWS/RDS` namespace with the dimension `ProxyName`. The metrics you care about operationally break into three categories: connection counts, pooling efficiency, and latency.

Connection count metrics:

Metric	What it tells you
<code>ClientConnections</code>	Current number of client connections to the proxy
<code>DatabaseConnections</code>	Current number of backend connections the proxy holds to the database
<code>DatabaseConnectionsSetupSucceeded</code>	Rate of successful backend connection establishments
<code>DatabaseConnectionsSetupFailed</code>	Rate of failed backend connection establishments
<code>ClientConnectionsClosed</code>	Rate of client connection closures

The ratio of `ClientConnections` to `DatabaseConnections` is your effective multiplexing ratio. If you have 2,000 client connections and 300 database connections, the proxy is working as intended. If those numbers are equal — or worse, `DatabaseConnections` is approaching your `max_connections` limit while `ClientConnections` is only modestly higher — you have a pinning problem.

The pinning metric you must alarm on:

`DatabaseConnectionsCurrentlySessionPinned`

This metric reports how many backend connections are currently pinned to a specific client. A non-zero value is not necessarily a crisis — some applications legitimately use temporary tables or multi-statement transactions that require pinning. But a value that climbs toward your total `DatabaseConnections` count means you are getting no benefit from the proxy's multiplexing, and you should investigate which application code paths are causing the pins.

Setting up a CloudWatch dashboard and alarms:

```
# scripts/create_proxy_monitoring.py
# Creates CloudWatch alarms and dashboard for RDS Proxy health

import boto3
import json

PROXY_NAME = "prod-aurora-pg-proxy"
SNS_TOPIC_ARN = "arn:aws:sns:us-east-1:123456789012:rds-alerts"
REGION = "us-east-1"
```

```

cw = boto3.client('cloudwatch', region_name=REGION)

# Alarm: high pinning ratio - triggers when more than 25% of DB connections are pinned
cw.put_metric_alarm(
    AlarmName=f"{PROXY_NAME}-HighPinningRatio",
    AlarmDescription=(
        "More than 25% of proxy backend connections are session-pinned. "
        "Investigate application code for SET statements, temp tables, or cursors."
    ),
    Namespace="AWS/RDS",
    MetricName="DatabaseConnectionsCurrentlySessionPinned",
    Dimensions=[{"Name": "ProxyName", "Value": PROXY_NAME}],
    Statistic="Average",
    Period=60,
    EvaluationPeriods=3,
    Threshold=0, # absolute threshold isn't the right tool here; use metric math below
    ComparisonOperator="GreaterThanThreshold",
    TreatMissingData="notBreaching",
    AlarmActions=[SNS_TOPIC_ARN],
)

# Better approach: alarm on pinning RATIO using metric math
cw.put_metric_alarm(
    AlarmName=f"{PROXY_NAME}-PinningRatioExceeds25Pct",
    AlarmDescription=(
        "Session pinning ratio exceeds 25% of total backend database connections."
    ),
    Metrics=[
        {
            "Id": "pinned",
            "MetricStat": {
                "Metric": {
                    "Namespace": "AWS/RDS",
                    "MetricName": "DatabaseConnectionsCurrentlySessionPinned",
                    "Dimensions": [{"Name": "ProxyName", "Value": PROXY_NAME}]
                },
                "Period": 60,
                "Stat": "Average"
            },
            "ReturnData": False
        },
        {
            "Id": "total_db",
            "MetricStat": {
                "Metric": {
                    "Namespace": "AWS/RDS",
                    "MetricName": "DatabaseConnections",
                    "Dimensions": [{"Name": "ProxyName", "Value": PROXY_NAME}]
                },
                "Period": 60,
                "Stat": "Average"
            },
            "ReturnData": False
        },
        {
            "Id": "pinning_ratio",
            "Expression": "IF(total_db > 0, pinned / total_db * 100, 0)",
            "Label": "PinningRatioPct",
            "ReturnData": True,
            "Period": 60
        }
    ],
    ComparisonOperator: "GreaterThanThreshold",
    Threshold: 25,
    EvaluationPeriods: 3,
    TreatMissingData: "notBreaching",
    AlarmActions: [SNS_TOPIC_ARN],
)

# Alarm: connection borrow timeout - clients waiting for a backend connection

```

```

cw.put_metric_alarm(
    AlarmName=f"{PROXY_NAME}-BorrowTimeouts",
    AlarmDescription=(
        "Clients are timing out waiting for a backend connection from the proxy pool. "
        "Consider reducing max_connections_percent or scaling the database instance."
    ),
    Namespace="AWS/RDS",
    MetricName="ClientConnectionsWithLatency",
    # Use QueryDatabaseResponseLatency for query-level latency
    # Use DatabaseConnectionRequests for borrow attempt rate
    Dimensions=[{"Name": "ProxyName", "Value": PROXY_NAME}],
    Statistic="Sum",
    Period=60,
    EvaluationPeriods=2,
    Threshold=5,
    ComparisonOperator="GreaterThanOrEqualToThreshold",
    TreatMissingData="notBreaching",
    AlarmActions=[SNS_TOPIC_ARN],
)

# Dashboard definition
dashboard_body = {
    "widgets": [
        {
            "type": "metric",
            "x": 0, "y": 0, "width": 12, "height": 6,
            "properties": {
                "title": "Connection Multiplexing Ratio",
                "view": "timeSeries",
                "metrics": [
                    ["AWS/RDS", "ClientConnections", "ProxyName", PROXY_NAME,
                     {"label": "Client Connections", "color": "#2196F3"}],
                    ["AWS/RDS", "DatabaseConnections", "ProxyName", PROXY_NAME,
                     {"label": "Backend DB Connections", "color": "#FF9800"}],
                    ["AWS/RDS", "DatabaseConnectionsCurrentlySessionPinned",
                     "ProxyName", PROXY_NAME,
                     {"label": "Pinned Connections", "color": "#F44336"}]
                ],
                "period": 60,
                "stat": "Average",
                "yAxis": {"left": {"min": 0}}
            }
        },
        {
            "type": "metric",
            "x": 12, "y": 0, "width": 12, "height": 6,
            "properties": {
                "title": "Backend Connection Setup Rate",
                "view": "timeSeries",
                "metrics": [
                    ["AWS/RDS", "DatabaseConnectionsSetupSucceeded",
                     "ProxyName", PROXY_NAME,
                     {"label": "Setup Succeeded /min", "color": "#4CAF50", "stat": "Sum"}],
                    ["AWS/RDS", "DatabaseConnectionsSetupFailed",
                     "ProxyName", PROXY_NAME,
                     {"label": "Setup Failed /min", "color": "#F44336", "stat": "Sum"}]
                ],
                "period": 60
            }
        },
        {
            "type": "metric",
            "x": 0, "y": 6, "width": 24, "height": 6,
            "properties": {
                "title": "Query Response Latency (Proxy adds overhead here)",
                "view": "timeSeries",
                "metrics": [
                    ["AWS/RDS", "QueryDatabaseResponseLatency",
                     "ProxyName", PROXY_NAME,
                     {"label": "p50", "stat": "p50"}],

```

```

        ["AWS/RDS", "QueryDatabaseResponseLatency",
         "ProxyName", PROXY_NAME,
         {"label": "p95", "stat": "p95", "color": "#FF9800"}],
        ["AWS/RDS", "QueryDatabaseResponseLatency",
         "ProxyName", PROXY_NAME,
         {"label": "p99", "stat": "p99", "color": "#F44336"}]
    ],
    "period": 60
}
}
]
}

cw.put_dashboard(
    DashboardName=f"RDSPROxy-{PROXY_NAME}",
    DashboardBody=json.dumps(dashboard_body)
)

print(f"Dashboard created: RDSPROxy-{PROXY_NAME}")
print("Alarms created successfully")

```

Note the syntax errors in the alarm for `PinningRatioExceeds25Pct` in the snippet above — in real code you would not mix quoted and unquoted key styles. The intent is illustrated correctly; the actual boto3 call requires consistent Python dictionary syntax throughout. Test your monitoring code in a staging environment before wiring it to production alerting.

Enabling proxy debug logging is a tool you reach for when alarms fire and you need to understand which application queries are causing pins. Debug logging is disabled by default because it produces substantial volume and has a cost in both CloudWatch Logs ingestion and a small performance overhead on the proxy itself. Enable it temporarily:

```

aws rds modify-db-proxy \
    --db-proxy-name prod-aurora-pg-proxy \
    --debug-logging \
    --region us-east-1

# After enabling, tail the log group: /aws/rds/proxy/prod-aurora-pg-proxy
# Filter for pinning events:
aws logs filter-log-events \
    --log-group-name "/aws/rds/proxy/prod-aurora-pg-proxy" \
    --filter-pattern "pinned" \
    --start-time $(date -d '30 minutes ago' +%s000) \
    --region us-east-1 \
    --query 'events[*].message' \
    --output text

# When done diagnosing, disable debug logging
aws rds modify-db-proxy \
    --db-proxy-name prod-aurora-pg-proxy \
    --no-debug-logging \
    --region us-east-1

```

The debug log entries for pinning events include the reason for the pin and the SQL statement that triggered it. A log line might read:

```
[WARN] [clientId=c-0a1b2c3d] Pinning session to connection [connId=b-9f8e7d6c]
        because of statement: SET search_path TO analytics, public
```

That tells you exactly which application is issuing the `SET search_path` and gives you an actionable fix: move the `search_path` to a role-level default.

10.5 Failover Behavior: What RDS Proxy Changes (and What It Doesn't)

RDS Proxy's behavior during an Aurora failover is one of its strongest operational selling points, and it is also one of the most commonly misunderstood. Let's be precise.

Without the proxy, an Aurora failover looks like this from the application's perspective: 1. The writer instance fails or is manually failed over. 2. Aurora promotes a reader to writer — this typically completes within 30 to 60 seconds for Aurora PostgreSQL. 3. The cluster writer DNS endpoint (`cluster-id.cluster-xyz.region.rds.amazonaws.com`) updates its CNAME to point to the new writer — DNS TTL is typically 5 seconds, but client-side DNS caching in the JVM, Linux `nscd`, or application-level caching may hold the old IP for longer. 4. Existing connections are dropped (TCP RST or timeout depending on the failure mode). 5. Application must reconnect. If reconnect logic is not implemented, the application stays broken until a process restart.

With the proxy, the application's view changes substantially: 1. The writer instance fails. 2. Aurora promotes a reader to writer. 3. The proxy detects the topology change and transparently redirects new backend connections to the new writer — the proxy maintains its own connection to the cluster and watches for failover events through the Aurora internal notification mechanism, not just DNS. 4. The proxy endpoint for your application **does not change**. There is no DNS re-resolution required at the application level. 5. Client connections to the proxy that were idle (not mid-transaction) are preserved transparently — the proxy queues incoming requests briefly while re-establishing its backend connections. 6. Client connections that were mid-transaction when the failover hit will receive an error. This is unavoidable — the in-flight transaction was on the old writer and is gone. The proxy cannot make a transaction that was lost on a failed instance appear to succeed.

The practical result: the proxy reduces failover impact from “reconnect storm plus DNS propagation delay plus application-level reconnect logic” to “brief queuing plus mid-transaction errors only”. Applications with appropriate retry logic on the transaction level — not just the connection level — see dramatically reduced failover disruption. Applications that assume a single query can never fail still break, but they break for a shorter window and recover without needing to re-establish thousands of connections simultaneously.

A concrete test: you can observe this behavior by intentionally triggering a failover against a proxied Aurora cluster while running a connection monitor. The following script measures perceived downtime from the application side:

```
# scripts/failover_test.py
# Run this WHILE triggering a manual failover to measure client-side impact

import psycopg2
import time
import datetime
import os

PROXY_ENDPOINT = os.environ['DB_PROXY_ENDPOINT']
DB_NAME = os.environ['DB_NAME']
DB_USER = os.environ['DB_USER']
DB_PASSWORD = os.environ['DB_PASSWORD']

def make_connection():
    return psycopg2.connect(
        host=PROXY_ENDPOINT,
        port=5432,
        database=DB_NAME,
```

```

        user=DB_USER,
        password=DB_PASSWORD,
        sslmode='require',
        connect_timeout=5,
        application_name='failover-test'
    )

def probe_once(conn=None):
    """Returns (success, latency_ms, error_message)"""
    start = time.monotonic()
    try:
        if conn is None:
            conn = make_connection()
        with conn.cursor() as cur:
            cur.execute("SELECT 1")
            cur.fetchone()
        conn.commit()
        latency = (time.monotonic() - start) * 1000
        return True, latency, None, conn
    except psycopg2.OperationalError as e:
        latency = (time.monotonic() - start) * 1000
        # Connection lost - try fresh reconnect
        try:
            new_conn = make_connection()
            return False, latency, str(e), new_conn
        except Exception as reconnect_err:
            return False, latency, f"reconnect failed: {reconnect_err}", None

print(f"{'Timestamp':<26} {'Status':<10} {'Latency(ms)':<15} {'Notes'}")
print("-" * 70)

conn = make_connection()
errors_seen = 0
first_error_time = None
last_error_time = None

for _ in range(600): # 10 minutes of probing at 1-second intervals
    ts = datetime.datetime.now().isoformat(timespec='milliseconds')
    success, latency_ms, error_msg, conn = probe_once(conn)

    status = "OK" if success else "ERROR"
    note = ""

    if not success:
        errors_seen += 1
        if first_error_time is None:
            first_error_time = ts
        last_error_time = ts
        note = error_msg[:50] if error_msg else ""

    print(f"{'ts':<26} {'status':<10} {'latency_ms':<15.1f} {'note}'")
    time.sleep(1)

print(f"\nSummary: {errors_seen} errors observed")
if first_error_time:
    print(f"First error: {first_error_time}")
    print(f>Last error: {last_error_time}")

```

Run this script before triggering the failover (`aws rds failover-db-cluster --db-cluster-identifier prod-aurora-cluster`). The output will show you exactly how many seconds of errors your application tier experiences. Without the proxy, you typically see a sustained error period during DNS propagation. With the proxy, you typically see a brief burst of errors confined to the moment of the writer promotion, followed by immediate recovery — the proxy requeues requests and re-establishes its backend connections without your application needing to reconnect.

The multi-AZ RDS (non-Aurora) failover story is slightly different. Standard RDS Multi-AZ failover relies on DNS failover to the standby. RDS Proxy helps here too — the proxy tracks the database endpoint and handles the backend reconnection, still giving you the same client-connection stability benefit. However, the failover duration for standard Multi-AZ RDS is longer than Aurora failover because the promotion process involves more steps, and the proxy is waiting for the new primary to become available before re-establishing the backend pool. The proxy does not shorten the database failover itself — it only reduces the application-side reconnection overhead once the new primary is up.

10.6 Sizing and Scaling Considerations

RDS Proxy scales its own compute automatically — you do not choose an instance size for the proxy. AWS manages horizontal scaling of the proxy fleet. What you do control is the configuration that determines how efficiently the proxy uses backend connections, and that is where DBA judgment is applied.

Backend connection pool sizing decisions:

The `max_connections_percent` parameter is your primary lever. Setting it to 100 (the default) instructs the proxy to potentially use all of the database's `max_connections` for the proxy pool. In practice this is rarely correct for two reasons:

First, you almost always want to reserve some connections for emergency direct access. If a runaway proxy behavior or an application bug exhausts the pool, you need to be able to connect directly to the database as an administrator to diagnose the problem. A good rule of thumb is to reserve at least 5–10 connections for direct administrative access. With `max-connections-percent = 90` and your `max_connections` set to 1,000, the proxy can use up to 900 connections and you always have 100 available for direct connections.

Second, the proxy is most efficient when it is not running at full pool saturation. A fully saturated pool means every incoming client request must wait for a backend connection to become free. The queuing delay compounds under load. Setting `max-connections-percent` to 75–85% of what your database can actually sustain under realistic load gives the pool breathing room.

`max_idle_connections_percent` controls how many idle connections the proxy keeps warm in the backend pool. Idle connections consume memory on the database host — each PostgreSQL connection holds a backend process in memory even when doing nothing. Set this too high and you're paying connection memory cost constantly. Set it too low and the proxy must establish new backend connections under sudden traffic spikes, which adds latency during the connection establishment phase. A value of 50 (the default) means the proxy keeps up to half its maximum backend connections open and idle, ready to be assigned to incoming client requests immediately.

Instance sizing for the database itself does not change because of the proxy — the proxy reduces the number of backend connections the database manages, but the database still needs to be sized for the actual query workload. Where the proxy helps is that you can often run a smaller database instance than you would have needed without it, because you are no longer allocating 9 MB of RAM per potential connection when only a fraction of those connections are ever active simultaneously.

A sizing worked example (qualitative, not prescriptive):

Suppose you have an ECS service that autoscales from 10 to 200 tasks, each with a connection pool of 5 connections, giving a maximum potential connection demand of 1,000. Without the proxy, you size your database to handle 1,000 connections plus admin headroom. With the proxy, the database might actually only see 150–300 backend connections at peak because the proxy’s multiplexing collapses idle-connection demand. You can size the database for the actual query concurrency rather than the theoretical connection count.

Validate this by watching `DatabaseConnections` in CloudWatch during your peak traffic period after deploying the proxy. The actual number you observe is your actual database connection load. Use that to inform whether you’re running with more backend pool headroom than needed, or whether you’re running close to saturation and should consider scaling the database.

10.7 Limitations and When Not to Use RDS Proxy

Every managed service has a boundary at which it stops being the right tool. Knowing that boundary for RDS Proxy is as important as knowing how to configure it.

Long-running transactions: RDS Proxy multiplexes connections between requests, but while a transaction is open, the backend connection is held exclusively for that client — this is the same as pinning for the duration of the transaction. Applications that open transactions and hold them open for seconds or minutes while doing application-side work (processing data between SQL statements, waiting for user input in web applications, etc.) will see poor pool utilization. Design your application transactions to be short: open late, close early. If your application design fundamentally requires long-held transactions, the proxy’s multiplexing benefit is largely lost.

Stored procedures and session state: Applications that rely heavily on session-level state — temporary tables across multiple requests, `SET LOCAL` in transaction-scoped ways, advisory locks held across connections — require careful audit before proxying. None of these patterns become impossible with the proxy, but they may require session pinning, reducing the multiplexing benefit.

Binary protocols and non-standard drivers: RDS Proxy supports the standard PostgreSQL wire protocol and the MySQL wire protocol. Applications using non-standard connection mechanisms, administrative tools that use direct socket connections, or drivers with unusual protocol behavior may encounter compatibility issues. Test all connection paths — not just your main application — before enabling the proxy in production.

Cost considerations: RDS Proxy is not free. AWS charges per vCPU of the underlying database instance, per hour, while the proxy is attached. For small database instances with low connection counts, the cost may exceed the operational benefit. Evaluate the cost against the alternative of running a self-managed PgBouncer on an EC2 instance — for small workloads, PgBouncer on a `t3.small` can be significantly cheaper. For large, high-connection workloads, RDS Proxy’s managed nature and Aurora-native integration justify the cost. Review current AWS pricing for your region before committing.

SQL Server is not supported: Repeat this for your team: SQL Server on RDS does not support RDS Proxy. SQL Server’s connection pooling is handled by the ODBC or ADO.NET driver, which is session-based. If you’re running SQL Server on RDS and need connection pooling, you have three

options: driver-level pooling (built into most .NET and Java connection frameworks), a third-party proxy like ProxySQL (which has limited SQL Server support), or refactoring to reduce connection pressure at the application level.

Proxy-to-database network latency: The proxy sits in your VPC and adds one network hop between your application and the database. For latency-sensitive OLTP workloads executing many small queries, this hop is measurable. In practice it is a fraction of a millisecond within a region, but if you are optimizing for sub-millisecond query execution times, measure the added latency in your environment before committing to the proxy architecture.

10.8 Practical Runbook: Diagnosing a Connection Pool Exhaustion Event

When an on-call engineer gets paged at 2 AM with “database connections exhausted,” having a structured diagnostic path matters more than knowing every possible cause. Here is the runbook as a DBA would actually use it.

Step 1: Establish current state

```
# What does the proxy see right now?
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name ClientConnections \
  --dimensions Name=ProxyName,Value=prod-aurora-pg-proxy \
  --start-time $(date -u -d '10 minutes ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 60 \
  --statistics Average Maximum \
  --output table

aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name DatabaseConnections \
  --dimensions Name=ProxyName,Value=prod-aurora-pg-proxy \
  --start-time $(date -u -d '10 minutes ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 60 \
  --statistics Average Maximum \
  --output table

aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name DatabaseConnectionsCurrentlySessionPinned \
  --dimensions Name=ProxyName,Value=prod-aurora-pg-proxy \
  --start-time $(date -u -d '10 minutes ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 60 \
  --statistics Average Maximum \
  --output table
```

Step 2: Connect directly to the database (bypassing the proxy) and query pg_stat_activity

```
-- PostgreSQL: connect directly on port 5432 to the writer endpoint, not via proxy
-- You reserved direct connections for exactly this scenario

-- Who is connected and what are they doing?
SELECT
  pid,
  username,
```

```

    application_name,
    client_addr,
    state,
    wait_event_type,
    wait_event,
    now() - query_start      AS query_duration,
    now() - state_change    AS state_duration,
    LEFT(query, 100)        AS query_snippet
FROM pg_stat_activity
WHERE backend_type = 'client backend'
ORDER BY query_duration DESC NULLS LAST;

-- Count by state and application
SELECT
    application_name,
    state,
    COUNT(*) AS connection_count,
    MAX(now() - state_change) AS longest_in_state
FROM pg_stat_activity
WHERE backend_type = 'client backend'
GROUP BY application_name, state
ORDER BY connection_count DESC;

-- Are there long-running idle-in-transaction connections?
-- These hold backend connections pinned in the proxy
SELECT
    pid,
    username,
    application_name,
    state,
    now() - state_change AS idle_in_tx_duration,
    LEFT(query, 200) AS last_query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
    AND now() - state_change > interval '30 seconds'
ORDER BY idle_in_tx_duration DESC;

```

Step 3: If idle-in-transaction connections are the cause, terminate them

```

-- PostgreSQL: terminate long-idle-in-transaction connections
-- Use pg_terminate_backend, not pg_cancel_backend
-- (cancel only stops the running query; terminate closes the connection)

-- First preview what you'd terminate:
SELECT
    pid,
    username,
    application_name,
    now() - state_change AS idle_duration
FROM pg_stat_activity
WHERE state = 'idle in transaction'
    AND now() - state_change > interval '2 minutes';

-- Then terminate:
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE state = 'idle in transaction'
    AND now() - state_change > interval '2 minutes';

-- Verify the terminations took effect:
SELECT COUNT(*)
FROM pg_stat_activity
WHERE state = 'idle in transaction';

```

Step 4: Check if a specific application host is responsible

The proxy logs client IP addresses in debug mode. If debug logging is not already enabled:

```

# Enable debug logging temporarily
aws rds modify-db-proxy \

```

```
--db-proxy-name prod-aurora-pg-proxy \  
--debug-logging  
  
# Wait 60 seconds for logs to appear, then query for connection events by client IP  
aws logs filter-log-events \  
  --log-group-name "/aws/rds/proxy/prod-aurora-pg-proxy" \  
  --filter-pattern "[timestamp, requestId, level, clientId, message=\\\"*client*connected*\\\"]" \  
  --start-time "$(date -d '5 minutes ago' +%s000) \  
  --query 'events[*].message' \  
  --output text | sort | uniq -c | sort -rn | head -20
```

Step 5: Immediate mitigation options

If the pool is exhausted and the application is failing:

```
# Option A: Temporarily increase max_connections_percent to give more backend connections  
# This only works if the database instance has headroom  
aws rds modify-db-proxy-target-group \  
  --db-proxy-name prod-aurora-pg-proxy \  
  --target-group-name default \  
  --connection-pool-config MaxConnectionsPercent=95  
  
# Option B: If a runaway process has filled the pool, restart the ECS service  
# (replace with your actual cluster/service names)  
aws ecs update-service \  
  --cluster prod-cluster \  
  --service my-application-service \  
  --force-new-deployment  
  
# Option C: If an autoscaling event caused the spike and you need to shed connections,  
# reduce ECS desired count temporarily  
aws ecs update-service \  
  --cluster prod-cluster \  
  --service my-application-service \  
  --desired-count 10
```

Step 6: After mitigation, set a `idle_in_transaction_session_timeout` to prevent recurrence

```
-- PostgreSQL: set at the database level to auto-terminate stuck transactions  
-- This catches connections that the proxy cannot clean up itself  
  
ALTER DATABASE production  
  SET idle_in_transaction_session_timeout = '5min';  
  
-- Or at the role level for the application user specifically  
ALTER ROLE appuser  
  SET idle_in_transaction_session_timeout = '5min';  
  
-- Verify the setting:  
SHOW idle_in_transaction_session_timeout;
```

This setting causes PostgreSQL to automatically terminate any backend that has been idle within a transaction for longer than the specified duration. The proxy will see the backend connection dropped and will establish a fresh one. Pairing this database-level protection with the proxy's `idle-client-timeout` gives you defense in depth: the proxy closes idle client connections before they accumulate, and the database closes stuck transactions before they pin backend connections indefinitely.

Key Takeaways

- **RDS Proxy multiplexes thousands of client connections onto a much smaller backend connection pool**, reducing per-connection memory overhead on the database and eliminating reconnect storms during traffic spikes — but this benefit only materializes when client connections are not session-pinned. Monitor `DatabaseConnectionsCurrentlySessionPinned` and treat a rising pinning ratio as a signal to audit application code for `SET` statements, temporary tables, and long-held idle transactions.
 - **Connection pinning is the primary operational hazard of RDS Proxy**. Common causes include `SET search_path` in connection strings, `CREATE TEMP TABLE` in application code, and open cursors with `HOLD`. Fix pinning at the source: move role defaults to `ALTER ROLE ... SET`, avoid temporary tables in hot paths, and set `session_pinning_filters = ["EXCLUDE_VARIABLE_SETS"]` in the proxy target group configuration to exclude low-risk `SET` statements from triggering pins.
 - **IAM authentication through RDS Proxy eliminates long-lived passwords from application configuration**, replacing them with 15-minute cryptographic tokens tied to IAM roles. The IAM resource ARN in the `rds-db:connect` policy must reference the proxy's resource ID (the `prx-` identifier), not the proxy name or cluster ARN — a mistake that produces authentication errors indistinguishable from a wrong password at the application level.
 - **During Aurora failover, RDS Proxy preserves the application-facing endpoint and transparently re-establishes backend connections to the promoted writer**, reducing failover impact to mid-transaction errors only rather than a full reconnect storm across all application instances. The proxy does not shorten the Aurora failover duration itself — it reduces the application-side recovery work that follows.
 - **RDS Proxy is not universally the right choice**: SQL Server on RDS is not supported, long-held transactions negate the multiplexing benefit, and for low-connection workloads the per-vCPU cost may exceed the value of the managed service compared to running self-managed PgBouncer. Validate the business case with actual connection count metrics, test `max_connections_percent` and `max_idle_connections_percent` against your real traffic patterns, and always reserve a portion of `max_connections` for direct administrative access independent of the proxy pool.
-

Chapter 11: Aurora Serverless v2: Auto-Scaling in Practice

Aurora Serverless v2 represents one of the more significant architectural departures from traditional database capacity planning. Where you once sized an instance, benchmarked it, added 30% headroom, and hoped your estimates held through the next fiscal year, Aurora Serverless v2 lets the database scale compute capacity up and down in near real-time — in increments as small as 0.5 ACUs (Aurora Capacity Units) — without dropping connections or requiring a restart. For a DBA who has spent years managing instance fleets, over-provisioning for peak loads, and scheduling maintenance windows for vertical scaling, this requires a genuine shift in how you think about capacity. This chapter covers how ACUs work, how to configure scaling bounds, how the system behaves under real production load, what the billing model means for your architecture decisions, and where the gotchas live — particularly around connection behavior, writer/reader asymmetry, and the interactions between Serverless v2 and Aurora Global Database.

11.1 Aurora Capacity Units: What You’re Actually Scaling

Before configuring anything, you need a clear mental model of what an Aurora Capacity Unit is. AWS defines 1 ACU as approximately 2 GiB of memory, with a proportional allocation of CPU and networking. This is not an exact ratio — Aurora does not expose the raw vCPU count the way an EC2 instance does — but as a working approximation, a 64 ACU instance is roughly equivalent to a db.r6g.16xlarge in CPU and memory terms, and a 0.5 ACU instance is a tiny slice suitable only for near-idle workloads.

The scaling range you configure is expressed as a minimum and maximum ACU value. The minimum can be as low as 0.5 ACUs (though there are practical implications to setting it this low, covered in §11.3). The maximum can be as high as 256 ACUs as of the current service limits, though that ceiling has been raised several times and you should check the AWS documentation for the current value.

What makes Serverless v2 categorically different from Serverless v1 is that scaling is **continuous and fine-grained**. v1 scaled in discrete steps and could take 15–30 seconds to respond, during which queries would queue or time out. v2 scales in sub-second increments. In practice this means that a sudden spike — a batch job firing, a marketing email going out, an end-of-day report — gets absorbed without the application seeing a degraded response time curve typical of traditional vertical scaling events.

One thing experienced DBAs sometimes misunderstand early on: ACU scaling only applies to **compute**. Aurora storage is always decoupled from the compute tier. The Aurora distributed storage layer scales automatically regardless of your ACU configuration, handles up to 128 TiB per cluster, and you are billed for storage separately. This matches the Aurora architecture you learned in Chapter 9 — the storage layer is shared across all instances in the cluster.

The relationship between ACUs and what you can observe in CloudWatch is through the `ServerlessDatabaseCapacity` metric, which reports the current ACU level. Pair this with `CPUUtilization`, `DatabaseConnections`, and `FreeableMemory` to build a complete picture of scaling behavior.

```
-- Run this against Aurora PostgreSQL to correlate what you observe
-- in CloudWatch with what the database is actually doing
SELECT
  now() AS sample_time,
  pg_postmaster_start_time() AS db_start_time,
  (SELECT count(*) FROM pg_stat_activity) AS total_connections,
  (SELECT count(*) FROM pg_stat_activity
   WHERE state = 'active') AS active_connections,
  (SELECT count(*) FROM pg_stat_activity
   WHERE wait_event_type IS NOT NULL) AS waiting_connections,
  pg_size_pretty(pg_database_size(current_database())) AS db_size;
```

This query, run periodically from a monitoring process, establishes the connection load baseline you'll use to validate that your ACU minimum is sized appropriately for your idle-state connection overhead.

11.2 Provisioning Aurora Serverless v2 with the AWS CLI and Terraform

Creating a Serverless v2 cluster is a two-step process: you create the Aurora cluster, then you add instances to it with a serverless scaling configuration. This is different from Aurora Serverless v1, where the cluster itself was serverless. In v2, the cluster is a standard Aurora cluster and the serverless behavior is a property of the individual DB instances within it.

Creating the cluster via AWS CLI:

```
# Step 1: Create the Aurora cluster (MySQL-compatible shown first,
# then PostgreSQL pattern below)
aws rds create-db-cluster \
  --db-cluster-identifier prod-serverless-v2 \
  --engine aurora-postgresql \
  --engine-version 15.4 \
  --master-username dbadmin \
  --master-user-password "${DB_MASTER_PASSWORD}" \
  --vpc-security-group-ids sg-0abc123def456789a \
  --db-subnet-group-name prod-aurora-subnet-group \
  --backup-retention-period 7 \
  --preferred-backup-window "03:00-04:00" \
  --preferred-maintenance-window "sun:05:00-sun:06:00" \
  --storage-encrypted \
  --kms-key-id arn:aws:kms:us-east-1:123456789012:key/mrk-abc123 \
  --enable-cloudwatch-logs-exports '["postgresql"]' \
  --deletion-protection \
  --region us-east-1

# Step 2: Add the writer instance with Serverless v2 scaling configuration
aws rds create-db-instance \
```



```

--db-instance-identifier prod-serverless-v2-writer \
--db-cluster-identifier prod-serverless-v2 \
--db-instance-class db.serverless \
--engine aurora-postgresql \
--region us-east-1

# Step 3: Modify the cluster's scaling configuration
# MinCapacity: 2 ACUs (never go below this - rationale in §11.3)
# MaxCapacity: 64 ACUs (size to your known peak, not theoretical maximum)
aws rds modify-db-cluster \
  --db-cluster-identifier prod-serverless-v2 \
  --serverless-v2-scaling-configuration MinCapacity=2,MaxCapacity=64 \
  --apply-immediately \
  --region us-east-1

```

Equivalent Terraform configuration:

```

# terraform/aurora_serverless_v2/main.tf

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

locals {
  cluster_id = "prod-serverless-v2"
  engine_ver = "15.4"
  min_acu    = 2
  max_acu    = 64
}

resource "aws_rds_cluster" "serverless_v2" {
  cluster_identifier      = local.cluster_id
  engine                 = "aurora-postgresql"
  engine_version         = local.engine_ver
  master_username        = var.master_username
  master_password        = var.master_password
  vpc_security_group_ids = [aws_security_group.aurora.id]
  db_subnet_group_name   = aws_db_subnet_group.aurora.name
  backup_retention_period = 7
  preferred_backup_window = "03:00-04:00"
  storage_encrypted      = true
  kms_key_id             = var.kms_key_arn
  deletion_protection    = true
  skip_final_snapshot    = false
  final_snapshot_identifier = "${local.cluster_id}-final"

  enabled_cloudwatch_logs_exports = ["postgresql"]

  serverless_v2_scaling_configuration {
    min_capacity = local.min_acu
    max_capacity = local.max_acu
  }

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
    Owner       = "platform-dba"
  }
}

resource "aws_rds_cluster_instance" "writer" {
  identifier      = "${local.cluster_id}-writer"
  cluster_identifier = aws_rds_cluster.serverless_v2.id
  instance_class  = "db.serverless"
  engine          = aws_rds_cluster.serverless_v2.engine
}

```

```
engine_version      = aws_rds_cluster.serverless_v2.engine_version

performance_insights_enabled      = true
performance_insights_retention_period = 7
monitoring_interval              = 60
monitoring_role_arn              = var.enhanced_monitoring_role_arn

tags = {
  Role = "writer"
}
}

resource "aws_rds_cluster_instance" "reader" {
  count              = var.reader_count # typically 1 or 2 in production
  identifier         = "${local.cluster_id}-reader-${count.index}"
  cluster_identifier = aws_rds_cluster.serverless_v2.id
  instance_class     = "db.serverless"
  engine             = aws_rds_cluster.serverless_v2.engine
  engine_version     = aws_rds_cluster.serverless_v2.engine_version

  performance_insights_enabled      = true
  performance_insights_retention_period = 7
  monitoring_interval              = 60
  monitoring_role_arn              = var.enhanced_monitoring_role_arn

  tags = {
    Role = "reader"
  }
}
```

One point worth noting: `db.serverless` is the instance class you specify — it is not a size within a family like `db.r6g.xlarge`. The ACU range you set on the cluster governs the actual compute allocation. This means you can have a writer and multiple readers that each independently scale within the same ACU bounds. The writer and readers do not share an ACU budget; each instance scales individually.

11.3 Scaling Behavior: What Triggers It, How Fast It Responds

Understanding the scaling triggers prevents you from being surprised by billing spikes and helps you tune the min/max bounds correctly. Aurora Serverless v2 monitors several signals to decide when to scale:

- **CPU utilization** relative to current capacity
- **Memory pressure** (buffer pool fill rate, swap activity)
- **Connection count** relative to the `max_connections` value for the current ACU level
- **Database load** as reported by the Performance Insights average active sessions metric

When any of these signals indicates the current ACU level is insufficient, the service scales up. Scale-up is aggressive by design — it happens in small increments but very quickly, typically completing within 1–2 seconds. Scale-down is deliberate and slow. AWS does not want a brief lull between query bursts to trigger a scale-down that then requires a scale-up moments later. In practice, expect scale-down to lag behind reduced load by several minutes.

The minimum ACU floor matters more than people expect. At 0.5 ACUs, `max_connections` on Aurora PostgreSQL is approximately 90. That sounds fine until you realize that PgBouncer or RDS Proxy pools often open tens of connections just for health checks and pool maintenance, and your application likely has connection pools of its own. Setting

MinCapacity=0.5 on a production cluster will result in connection failures whenever the cluster is at its minimum — usually late at night when you don’t notice until your monitoring alerts on a failed 3 AM backup job.

A practical minimum for production PostgreSQL workloads is **2 ACUs** (approximately 4 GiB RAM, ~450 max connections). For MySQL-compatible Aurora, the connection scaling is more favorable at low ACU counts, but the same principle applies: never set your minimum to a value that makes the idle-state connection budget uncomfortably tight.

The `max_connections` formula for Aurora PostgreSQL Serverless v2 is:

```
LEAST({DBInstanceClassMemory/9531392}, 5000)
```

where `DBInstanceClassMemory` is in bytes. At 2 ACUs (4 GiB), this yields approximately 450 connections. At 8 ACUs (16 GiB), approximately 1800. At 32 ACUs (64 GiB) the uncapped formula would exceed 5000, so the hard cap of 5000 applies.

You can verify what your cluster is currently using:

```
-- Aurora PostgreSQL: check current connection limit and usage
SELECT
  current_setting('max_connections')::int           AS max_connections,
  count(*)                                           AS current_connections,
  count(*) FILTER (WHERE state = 'idle')            AS idle_connections,
  count(*) FILTER (WHERE state = 'active')          AS active_connections,
  count(*) FILTER (WHERE state = 'idle in transaction') AS idle_in_txn,
  round(
    count(*)::numeric /
    current_setting('max_connections')::numeric * 100,
    1)                                              AS pct_utilized
FROM pg_stat_activity
WHERE pid <> pg_backend_pid();
```

If `pct_utilized` regularly exceeds 70% at your minimum ACU level, your minimum is too low. A scale-up event that is triggered by memory or CPU pressure will also increase the connection limit headroom — but the scale-up takes 1–2 seconds, and in that window, new connection attempts against a fully saturated connection limit will fail immediately.

11.4 RDS Proxy Integration and the Connection Management Layer

For Serverless v2, RDS Proxy is not optional — it is the architectural pattern that makes the auto-scaling behavior compatible with application-level connection pooling. Here is why.

When Aurora Serverless v2 scales its ACU level, the database endpoint does not change, connections are not dropped, and there is no TCP-level interruption. This is the advertised behavior, and it holds true in steady-state scaling. However, at the extremes — when the cluster scales to its minimum and stays there for an extended idle period, or when you change the ACU bounds via a `modify-db-cluster` call — brief disruptions are possible. RDS Proxy absorbs these by maintaining its own connection pool to the backend and presenting a stable connection surface to applications.

Beyond that protective function, RDS Proxy serves the standard purpose of multiplexing many application connections onto fewer database connections — a pattern you would traditionally implement with PgBouncer or pgpool-II on-premises. RDS Proxy uses IAM authentication and Secrets

Manager to rotate credentials without connection disruption, which maps to what you previously handled manually in `pg_hba.conf` and password rotation scripts.

```
# Create RDS Proxy for the Serverless v2 cluster
aws rds create-db-proxy \
  --db-proxy-name prod-serverless-v2-proxy \
  --engine-family POSTGRESQL \
  --auth '[
    {
      "AuthScheme": "SECRETS",
      "SecretArn": "arn:aws:secretsmanager:us-east-1:123456789012:secret:prod/aurora/dbadmin",
      "IAMAuth": "REQUIRED"
    }
  ]' \
  --role-arn arn:aws:iam::123456789012:role/rds-proxy-role \
  --vpc-subnet-ids subnet-0abc123 subnet-0def456 subnet-0ghi789 \
  --vpc-security-group-ids sg-0abc123def456789a \
  --require-tls \
  --idle-client-timeout 1800 \
  --debug-logging false \
  --region us-east-1

# Register the Aurora cluster as the proxy target
aws rds register-db-proxy-targets \
  --db-proxy-name prod-serverless-v2-proxy \
  --db-cluster-identifiers prod-serverless-v2 \
  --region us-east-1

# Create a reader endpoint proxy target group for read scaling
aws rds create-db-proxy-endpoint \
  --db-proxy-name prod-serverless-v2-proxy \
  --db-proxy-endpoint-name prod-serverless-v2-proxy-reader \
  --vpc-subnet-ids subnet-0abc123 subnet-0def456 subnet-0ghi789 \
  --vpc-security-group-ids sg-0abc123def456789a \
  --target-role READ_ONLY \
  --region us-east-1
```

IAM authentication through RDS Proxy replaces the traditional `pg_hba.conf` + password authentication model. Your application no longer stores a database password in its configuration. Instead, it generates a short-lived authentication token using the AWS SDK:

```
# Python: Generate RDS IAM auth token and connect through RDS Proxy
# Requires boto3 and psycopg2

import boto3
import psycopg2
import os
from botocore.exceptions import ClientError

def get_rds_auth_token(
    hostname: str,
    port: int,
    username: str,
    region: str
) -> str:
    """
    Generate a 15-minute IAM authentication token for RDS/RDS Proxy.
    The EC2 instance or Lambda function must have an IAM role with
    rds-db:connect permission on the target resource.
    """
    client = boto3.client("rds", region_name=region)
    token = client.generate_db_auth_token(
        DBHostname=hostname,
        Port=port,
        DBUsername=username,
        Region=region
    )
    return token
```

```

def get_connection(read_only: bool = False) -> psycopg2.extensions.connection:
    """
    Return a psycopg2 connection through RDS Proxy.
    Uses IAM auth - no password stored in environment or config files.
    """
    region = os.environ["AWS_REGION"]

    if read_only:
        host = os.environ["RDS_PROXY_READER_ENDPOINT"]
    else:
        host = os.environ["RDS_PROXY_WRITER_ENDPOINT"]

    port = int(os.environ.get("DB_PORT", "5432"))
    username = os.environ["DB_USERNAME"]
    dbname = os.environ["DB_NAME"]

    token = get_rds_auth_token(host, port, username, region)

    conn = psycopg2.connect(
        host=host,
        port=port,
        user=username,
        password=token,          # IAM token used as password
        dbname=dbname,
        sslmode="verify-full",
        sslrootcert="/etc/ssl/certs/rds-combined-ca-bundle.pem",
        connect_timeout=10
    )

    # For read-only connections, enforce at the session level
    # as a defensive measure even though the proxy endpoint is read-only
    if read_only:
        conn.set_session(readonly=True)

    return conn


def execute_query(query: str, params: tuple = None, read_only: bool = False):
    """
    Execute a query and return results. Demonstrates connection reuse pattern
    appropriate for Lambda or short-lived application processes.
    For long-lived services, use a connection pool (e.g., psycopg2.pool or
    SQLAlchemy) with a max_overflow sized to your ACU minimum's connection budget.
    """
    conn = None
    try:
        conn = get_connection(read_only=read_only)
        with conn.cursor() as cur:
            cur.execute(query, params)
            if cur.description: # SELECT
                return cur.fetchall()
            conn.commit()
            return cur.rowcount
    except psycopg2.OperationalError as e:
        # Log and re-raise - let the caller decide on retry strategy
        # Common causes: proxy is rotating target, ACU scale event in progress
        raise RuntimeError(f"Database connection failed: {e}") from e
    finally:
        if conn and not conn.closed:
            conn.close()

```

One behavioral difference from on-premises PgBouncer that catches DBAs off guard: RDS Proxy operates in **transaction pooling mode** for MySQL and **session pooling mode** for PostgreSQL by default. Session pooling means that a proxy connection is pinned to a specific backend connection for the life of the session — it does not multiplex at the transaction level the way PgBouncer does in transaction mode. This limits the connection compression ratio for PostgreSQL workloads. The

practical implication is that you should not expect RDS Proxy to reduce your backend connection count by 10:1 for a PostgreSQL Serverless v2 cluster the way it might for MySQL. Budget for a proxy-to-backend ratio closer to 3:1 or 4:1 for PostgreSQL.

11.5 Monitoring, Alerting, and Cost Governance

Aurora Serverless v2 billing is straightforward to understand but easy to mismanage. You are billed per ACU-hour, plus storage, plus I/O (unless you use Aurora I/O-Optimized storage) — check the current AWS pricing page for your region for the ACU-hour rate. A cluster that runs at a fixed ACU level continuously will cost roughly the same as a provisioned instance sized to match that capacity, since Serverless v2's per-second billing offers no discount for steady-state usage — you're paying for the compute you use either way. The economic argument for Serverless v2 is workloads that are **spiky rather than flat**, where peak capacity would otherwise sit idle most of the time on a fixed provisioned instance.

CloudWatch provides the `ServerlessDatabaseCapacity` metric, which you should track alongside `CPUUtilization` and `FreeableMemory` to build a scaling profile. The following CloudWatch alarm setup uses the AWS CLI to alert when the cluster is approaching its maximum ACU ceiling:

```
# CloudWatch alarm: writer approaching MaxCapacity
# Fires when the cluster sustains >= 90% of MaxCapacity for 5 consecutive minutes
# Adjust threshold (57.6 = 90% of 64 ACU max) and dimensions for your cluster

aws cloudwatch put-metric-alarm \
  --alarm-name "aurora-serverless-approaching-max-capacity" \
  --alarm-description "Writer ACU >= 90% of MaxCapacity for 5 minutes - review scaling bounds" \
  --namespace "AWS/RDS" \
  --metric-name "ServerlessDatabaseCapacity" \
  --dimensions Name=DBClusterIdentifier,Value=prod-serverless-v2 \
  --statistic Average \
  --period 60 \
  --evaluation-periods 5 \
  --threshold 57.6 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --treat-missing-data notBreaching \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:dba-alerts \
  --ok-actions arn:aws:sns:us-east-1:123456789012:dba-alerts \
  --region us-east-1

# Complementary alarm: sustained high CPU suggesting compute is the bottleneck

aws cloudwatch put-metric-alarm \
  --alarm-name "aurora-serverless-high-cpu" \
  --alarm-description "CPU > 80% for 10 minutes while at or near MaxCapacity" \
  --namespace "AWS/RDS" \
  --metric-name "CPUUtilization" \
  --dimensions Name=DBClusterIdentifier,Value=prod-serverless-v2 \
  --statistic Average \
  --period 60 \
  --evaluation-periods 10 \
  --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --treat-missing-data notBreaching \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:dba-alerts \
  --region us-east-1

# Cost governance: set a monthly budget alert for the Aurora cluster
# Uses AWS Budgets, not CloudWatch - catches cost overruns before month-end

aws budgets create-budget \
  --account-id 123456789012 \
  --budget '{
```

```

    "BudgetName": "aurora-serverless-monthly",
    "BudgetLimit": {"Amount": "1500", "Unit": "USD"},
    "TimeUnit": "MONTHLY",
    "BudgetType": "COST",
    "CostFilters": {
      "Service": ["Amazon Relational Database Service"]
    }
  } \
--notifications-with-subscribers '[
  {
    "Notification": {
      "NotificationType": "ACTUAL",
      "ComparisonOperator": "GREATER_THAN",
      "Threshold": 80,
      "ThresholdType": "PERCENTAGE"
    },
    "Subscribers": [
      {
        "SubscriptionType": "SNS",
        "Address": "arn:aws:sns:us-east-1:123456789012:dba-alerts"
      }
    ]
  },
  {
    "Notification": {
      "NotificationType": "FORECASTED",
      "ComparisonOperator": "GREATER_THAN",
      "Threshold": 100,
      "ThresholdType": "PERCENTAGE"
    },
    "Subscribers": [
      {
        "SubscriptionType": "SNS",
        "Address": "arn:aws:sns:us-east-1:123456789012:dba-alerts"
      }
    ]
  }
] \
--region us-east-1

```

The two-alert pattern — one on `ServerlessDatabaseCapacity` approaching `MaxCapacity`, one on forecasted budget overage — covers both the operational signal (the database is compute-constrained) and the financial signal (your ACU consumption is trending above plan). These serve different audiences: the capacity alarm goes to the on-call DBA, the budget alert goes to engineering management.

Building a capacity utilization dashboard. Performance Insights is the most useful single tool for understanding whether your ACU bounds are correctly sized. The key metric to track is **Database Load** — the average number of active sessions divided by the number of vCPUs (or the ACU-normalized equivalent). When Database Load consistently exceeds your normalized ACU level, you are compute-bound and queries are queuing. When it is a small fraction of your `MinCapacity`, your minimum is over-provisioned.

A practical SQL query to track load patterns directly from the database, useful for historical analysis or during a troubleshooting session:

```

-- Aurora PostgreSQL: Identify top wait events contributing to load
-- Run this repeatedly during a high-load period to build a wait profile
SELECT
  wait_event_type,
  wait_event,
  state,
  count(*) AS session_count,
  round(count(*) * 100.0 /
    NULLIF(SUM(count(*)) OVER (), 0),

```

```

1)                                AS pct_of_total
FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
AND state <> 'idle'
GROUP BY wait_event_type, wait_event, state
ORDER BY session_count DESC
LIMIT 20;

```

When `wait_event_type = 'CPU'` dominates this output, the cluster is compute-bound and you should consider raising `MaxCapacity`. When lock waits (`wait_event_type = 'Lock'`) or `ClientRead` dominate, the issue is query design or application behavior, and raising ACU limits will not help. This distinction is the same analysis you would perform on any PostgreSQL instance — Serverless v2 does not change the diagnostic methodology, only the remediation path for compute-bound cases.

11.6 Writer/Reader Asymmetry and Failover Behavior

A subtlety in Serverless v2 that has tripped up production deployments: the writer and each reader instance scale **independently**. This is largely a benefit — a read-heavy reporting workload on a reader can scale to 32 ACUs without affecting the writer’s billing or performance — but it introduces asymmetry that you need to account for in failover design.

When a failover occurs (a writer instance failure, a blue/green deployment promotion, or an `failover-db-cluster` call), Aurora promotes one of the readers to writer. The promoted reader was scaling based on read traffic patterns. Immediately post-promotion, it must handle write traffic, which typically has a different CPU and memory profile. If your readers are sized with a lower `MaxCapacity` than the writer — which is tempting since reader workloads are often lighter — the newly promoted writer may scale up rapidly, potentially hitting the `MaxCapacity` ceiling before you have a chance to raise it.

The operationally conservative configuration is to **set the same ACU bounds on all instances in the cluster** during initial deployment. You can differentiate later once you have a clear empirical profile of reader versus writer load, but launching with asymmetric bounds introduces risk in the failover scenario.

You can test failover behavior explicitly in a maintenance window using the AWS CLI:

```

# Initiate a manual failover to observe post-failover scaling behavior
# The --target-db-instance-identifier specifies which reader to promote
aws rds failover-db-cluster \
  --db-cluster-identifier prod-serverless-v2 \
  --target-db-instance-identifier prod-serverless-v2-reader-0 \
  --region us-east-1

# Poll cluster status until failover completes
# Typical duration for Aurora PostgreSQL: 20-40 seconds
watch -n 5 'aws rds describe-db-clusters \
  --db-cluster-identifier prod-serverless-v2 \
  --query
  ↪ "DBClusters[0].{Status:Status,Writer:DBClusterMembers[?IsClusterWriter==\`true\`].DBInstanceIdentifier|[0]}"
  ↪ \
  --output table \
  --region us-east-1'

```

While the failover is in progress, monitor `ServerlessDatabaseCapacity` for both the newly promoted writer and the old writer (now a reader) to observe how quickly the scaling profile adjusts.

What you will typically see: the new writer’s ACU count climbs within the first 30–60 seconds as it absorbs write traffic, then stabilizes. The old writer, now a reader, may scale down if read traffic is lighter than write traffic was.

Aurora Global Database and Serverless v2. If you have deployed a Global Database for cross-region disaster recovery (covered in Chapter 12), each regional cluster has its own ACU configuration. The secondary region cluster handles only read traffic (Global Database secondary clusters are read-only), so you might configure a lower `MaxCapacity` there. However, the same failover asymmetry concern applies at the Global Database level: a regional failover promotes the secondary to a primary, and it must now handle write traffic with whatever ACU bounds you configured for a read-only workload. The recommendation is the same: match the ACU bounds between primary and secondary regions, or accept the operational burden of adjusting bounds immediately post-failover.

11.7 Practical Capacity Sizing: From Workload Profile to ACU Bounds

Given everything covered in this chapter, you need a concrete method for translating your workload characteristics into initial ACU min/max values. The following framework is not a formula — Aurora scaling behavior depends on too many workload-specific factors for a formula to be reliable — but it provides a structured starting point.

Step 1: Establish your peak compute requirement.

If you are migrating from a provisioned Aurora or RDS instance, your starting point is the instance class you are currently running. Map that to an approximate ACU equivalent:

Provisioned Instance	Approx ACU Equivalent
db.t3.medium	1–2 ACUs
db.r6g.large	2–4 ACUs
db.r6g.xlarge	4–8 ACUs
db.r6g.2xlarge	8–16 ACUs
db.r6g.4xlarge	16–32 ACUs
db.r6g.8xlarge	32–64 ACUs
db.r6g.16xlarge	64–128 ACUs

Your `MaxCapacity` should be set at or above the ACU equivalent of your current instance class. If your current instance runs at 60–70% CPU utilization at peak, you have headroom and your `MaxCapacity` could be set to the equivalent of your current instance. If your current instance regularly hits 80–90%, set `MaxCapacity` one step above.

Step 2: Profile your idle state.

The minimum ACU determines what you pay during off-peak hours and sets the connection budget floor. Run the connection utilization query from §11.3 during your quietest maintenance window:

```
-- Aurora PostgreSQL: Idle-state connection profile
-- Run during off-peak hours to establish minimum connection requirements
```

```

SELECT
    application_name,
    state,
    count(*) AS connection_count
FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
GROUP BY application_name, state
ORDER BY connection_count DESC;

```

Sum up the connections from monitoring agents, connection pools, and application idle pools. That total is your idle-state connection floor. Work backwards through the `max_connections` formula: if your idle state holds 200 connections, you need at least 2 ACUs (which yields ~450 `max_connections`). If your monitoring infrastructure alone holds 50 connections and your app pools another 100, 2 ACUs gives you adequate headroom.

Step 3: Review the ACU utilization history post-launch.

After running for two to four weeks, pull `ServerlessDatabaseCapacity` from CloudWatch and build a percentile distribution. The P99 value over that period is a strong candidate for your `MaxCapacity`. If the P99 is well below your configured maximum, consider lowering it — not because a higher maximum costs more at idle (it does not), but because a hard ceiling prevents runaway queries from consuming unbounded resources. A rogue cartesian join at 3 AM should not be allowed to scale your cluster to 256 ACUs.

```

# Pull ServerlessDatabaseCapacity statistics for the last 14 days
# This gives you the distribution of actual ACU consumption
aws cloudwatch get-metric-statistics \
    --namespace "AWS/RDS" \
    --metric-name "ServerlessDatabaseCapacity" \
    --dimensions Name=DBClusterIdentifier,Value=prod-serverless-v2 \
    --start-time "$(date -u -d '14 days ago' '+%Y-%m-%dT%H:%M:%SZ')" \
    --end-time "$(date -u '+%Y-%m-%dT%H:%M:%SZ')" \
    --period 3600 \
    --statistics Average Maximum \
    --query 'sort_by(Datapoints, &Timestamp)[*].{Time:Timestamp,Avg:Average,Max:Maximum}' \
    --output table \
    --region us-east-1

```

Use the `Maximum` statistic across all hourly periods to find your observed peak ACU level. Use the `Average` to understand your typical billing rate. The ratio of `Average` to `Maximum` is a good indicator of how spiky your workload is — a high ratio (close to 1.0) means your workload is relatively flat and Serverless v2 may not be the most cost-effective choice compared to Reserved Instance pricing on a provisioned cluster. A low ratio (0.2–0.4) means you are getting real economic benefit from the auto-scaling.

11.8 Limitations and When Serverless v2 Is Not the Right Choice

Serverless v2 is compelling for many workloads, but there are cases where you should stay with provisioned instances or at least mix serverless and provisioned within the same cluster.

I/O-intensive workloads and storage throughput. ACU scaling governs compute — it does not change the underlying storage I/O throughput in the same way a larger provisioned instance would. Very high I/O workloads (bulk data loads, intensive `VACUUM` operations, high-throughput analytics) may find that compute scales fine but storage I/O becomes the bottleneck. In these cases, the `VolumeReadIOPs` and `VolumeWriteIOPs` CloudWatch metrics will plateau while

CPU utilization remains moderate — a signature that compute auto-scaling is not addressing the actual constraint.

Latency-sensitive workloads during cold starts. If you set `MinCapacity` to 0.5 ACUs and your cluster sits truly idle overnight, the first queries of the morning will experience elevated latency as the cluster scales from its minimum. This is not the dramatic cold start of Serverless v1 (which could be 10–30 seconds), but there is a ramp period of a few seconds where response times are higher than steady-state. For applications where consistent sub-10ms query response is required across all hours, a minimum of 2 ACUs or higher effectively eliminates this behavior.

Aurora Parallel Query. As of the current service version, Aurora Parallel Query (which pushes query processing down to the storage layer for large table scans) is not available on Serverless v2 instances. If your workload depends on Parallel Query for analytics performance, you need to run those queries against a provisioned reader in the same cluster, rather than a serverless reader.

Cross-engine compatibility. Serverless v2 is available for both Aurora MySQL and Aurora PostgreSQL, but the specific minor version support changes with each engine release. Always verify that the engine version you need is available in the `db.serverless` instance class before committing to the architecture:

```
# Check which Aurora PostgreSQL versions support Serverless v2
aws rds describe-orderable-db-instance-options \
  --engine aurora-postgresql \
  --db-instance-class db.serverless \
  --query 'OrderableDBInstanceOptions[*].{Version:EngineVersion,MultiAZ:MultiAZCapable}' \
  --output table \
  --region us-east-1
```

RDS Proxy limitations with prepared statements. If your application uses named prepared statements that persist across transactions (common in some ORM frameworks and PostgreSQL-specific drivers), RDS Proxy session pinning behavior will hold a backend connection open for the life of the client session, effectively eliminating the multiplexing benefit. In these cases, the proxy adds latency overhead without reducing backend connection pressure. Profile your application's prepared statement behavior before deploying RDS Proxy in front of Serverless v2.

Key Takeaways

- **ACUs are compute abstractions, not instance sizes.** One ACU maps to approximately 2 GiB of memory with proportional CPU. The `db.serverless` instance class takes its actual compute allocation from the cluster's `serverless_v2_scaling_configuration` min/max bounds, and each instance in the cluster scales independently within those bounds.
- **Set your minimum ACU floor based on connection budget, not cost minimization.** The `max_connections` limit scales with ACU level. A 0.5 ACU minimum leaves fewer than 100 connections available — insufficient for most production workloads including monitoring agents and idle connection pools. A 2 ACU minimum is a more defensible floor for PostgreSQL clusters.
- **RDS Proxy is the recommended connection layer for Serverless v2, but understand its PostgreSQL session pooling behavior.** Unlike MySQL (which supports transaction pooling), RDS Proxy uses session pooling for PostgreSQL, meaning backend connec-

tions are pinned per client session. Plan for a 3:1 to 4:1 proxy-to-backend ratio rather than the higher compression ratios achievable with PgBouncer in transaction mode.

- **Writer and reader instances scale independently — account for failover asymmetry.** If readers are configured with lower `MaxCapacity` than the writer and a failover promotes a reader to writer, the new writer may rapidly approach its ACU ceiling under write traffic. Start with symmetric ACU bounds across all instances and differentiate only after empirical profiling.
 - **Use the ratio of average to peak `ServerlessDatabaseCapacity` to evaluate fit.** A flat workload (high average-to-peak ratio) derives little economic benefit from Serverless v2 compared to Reserved Instance pricing on a provisioned cluster. A spiky workload (low average-to-peak ratio) is where Serverless v2's billing model pays dividends. Build this analysis from two to four weeks of CloudWatch data before committing to the architecture long-term.
-

Chapter 12: Parameter Groups, Option Groups, and Tuning on RDS

If you’ve spent years editing `postgresql.conf` or SQL Server’s `sp_configure` to squeeze performance out of a physical server, RDS parameter groups will feel simultaneously familiar and frustrating. The concept is the same — a named collection of engine configuration values applied to one or more instances — but the execution lives entirely in AWS’s control plane, enforced through API calls rather than file edits, and constrained by what AWS chooses to expose. This chapter walks through how parameter groups and option groups work across RDS MySQL, PostgreSQL, and SQL Server, how to build a production tuning workflow around them, and where the managed service boundary forces you to rethink habits you’ve built over a decade.

12.1 How Parameter Groups Work: The RDS Configuration Model

On a self-managed PostgreSQL instance, you tune the database by editing `postgresql.conf`, reloading or restarting the server, and confirming changes with `SHOW work_mem` or `pg_file_settings`. You have full access to every knob the engine exposes. On RDS, AWS inserts an abstraction layer between you and those knobs.

A **parameter group** in RDS is a named container of engine parameters. Every RDS instance is associated with exactly one parameter group. When you create an instance without specifying one, AWS assigns the default parameter group for that engine version — and the default group is read-only. You cannot modify it. This is the first thing that trips up experienced DBAs: the moment you need to change any parameter, you must create a custom parameter group, modify it, and associate it with the instance.

Parameter groups come in two types:

- **DB Parameter Groups** — applied to a single RDS instance
- **DB Cluster Parameter Groups** — applied to an Aurora cluster, affecting all instances in that cluster

For Aurora specifically, you manage two separate parameter groups simultaneously: one at the cluster level (for shared settings like `binlog_format` in MySQL or `rds.force_ssl` in PostgreSQL) and one at the instance level (for memory settings like `work_mem` or `innodb_buffer_pool_size`). This split catches Aurora newcomers off guard when a parameter change doesn’t take effect because they edited the wrong group.

Parameters fall into two apply-types:

- **Static** — require a database restart to take effect
- **Dynamic** — can be applied immediately without a restart

In PostgreSQL terms, static parameters correspond to those that require a full server restart in `postgresql.conf` (like `max_connections` or `shared_buffers`). Dynamic parameters correspond to those that reload with `pg_reload_conf()`. The distinction matters enormously in production: changing `max_connections` on an RDS PostgreSQL instance requires a reboot, which means a brief outage on Single-AZ instances or a failover on Multi-AZ.

AWS exposes roughly 200–350 parameters per engine, depending on version. Many parameters that exist in the upstream engine are locked or absent. You cannot set `fsync = off` on RDS PostgreSQL regardless of what you put in a parameter group — RDS overrides it. You cannot configure `huge_pages` at the OS level. You get the knobs AWS decides you should have.

You can see which parameters are modifiable for a given engine version with the AWS CLI:

```
# List all modifiable parameters for a PostgreSQL 16 parameter group
aws rds describe-engine-default-parameters \
  --db-parameter-group-family postgres16 \
  --query "EngineDefaults.Parameters[?IsModifiable==`true`]" \
  --output table \
  --region us-east-1
```

This command returns the full parameter catalog for that engine family with current defaults, apply types, and allowed values. Run this before you start tuning — it tells you exactly what you’re working with.

The parameter group family name ties a group to a specific engine and major version. A group created for `postgres15` cannot be applied to a `postgres16` instance. When you upgrade a major version, you must either create a new parameter group for the target version or verify that your existing customizations are compatible with the new family. AWS does not migrate your tuning automatically.

12.2 Creating and Managing Parameter Groups

The production workflow for parameter groups follows a pattern: create, customize, test, associate. You should never modify a parameter group that is already associated with a running production instance without understanding the apply-type of every parameter you’re changing.

Here is the complete workflow for creating a custom PostgreSQL 16 parameter group using the AWS CLI:

```
# Create the custom parameter group
aws rds create-db-parameter-group \
  --db-parameter-group-name prod-postgres16-oltp \
  --db-parameter-group-family postgres16 \
  --description "Production OLTP tuning for PostgreSQL 16" \
  --region us-east-1

# Apply a batch of parameters
aws rds modify-db-parameter-group \
  --db-parameter-group-name prod-postgres16-oltp \
  --parameters \
```

```

"ParameterName=work_mem,ParameterValue=65536,ApplyMethod=immediate" \
"ParameterName=maintenance_work_mem,ParameterValue=2097152,ApplyMethod=immediate" \
"ParameterName=checkpoint_completion_target,ParameterValue=0.9,ApplyMethod=immediate" \
"ParameterName=effective_cache_size,ParameterValue=12582912,ApplyMethod=immediate" \
"ParameterName=random_page_cost,ParameterValue=1.1,ApplyMethod=immediate" \
"ParameterName=log_min_duration_statement,ParameterValue=1000,ApplyMethod=immediate" \

↪ "ParameterName=shared_preload_libraries,ParameterValue='pg_stat_statements,auto_explain',ApplyMethod=pending-reboot"
↪ \
--region us-east-1

# Associate the parameter group with an existing instance
aws rds modify-db-instance \
--db-instance-identifier prod-postgres-primary \
--db-parameter-group-name prod-postgres16-oltp \
--apply-immediately \
--region us-east-1

# Verify the parameter group is in sync (not pending-reboot)
aws rds describe-db-instances \
--db-instance-identifier prod-postgres-primary \
--query "DBInstances[0].DBParameterGroups" \
--region us-east-1

```

That last command returns either `in-sync` or `pending-reboot`. If you see `pending-reboot`, the static parameters you changed will not be active until the next restart. In production, this often means the instance is running with a mix of old static values and new dynamic values — a state that can last for weeks if nobody schedules the reboot.

The equivalent in Terraform, which is where production parameter group management belongs for teams running infrastructure as code:

```

resource "aws_db_parameter_group" "prod_postgres16_oltp" {
  name       = "prod-postgres16-oltp"
  family     = "postgres16"
  description = "Production OLTP tuning for PostgreSQL 16"

  parameter {
    name      = "work_mem"
    value     = "65536"
    apply_method = "immediate"
  }

  parameter {
    name      = "maintenance_work_mem"
    value     = "2097152"
    apply_method = "immediate"
  }

  parameter {
    name      = "checkpoint_completion_target"
    value     = "0.9"
    apply_method = "immediate"
  }

  parameter {
    name      = "effective_cache_size"
    value     = "12582912"
    apply_method = "immediate"
  }

  parameter {
    name      = "random_page_cost"
    value     = "1.1"
    apply_method = "immediate"
  }

  parameter {

```

```
    name      = "log_min_duration_statement"
    value      = "1000"
    apply_method = "immediate"
  }

  parameter {
    name      = "shared_preload_libraries"
    value      = "pg_stat_statements,auto_explain"
    apply_method = "pending-reboot"
  }

  tags = {
    Environment = "production"
    Engine      = "postgres16"
    Workload    = "oltp"
  }

  lifecycle {
    create_before_destroy = true
  }
}

resource "aws_db_instance" "prod_postgres_primary" {
  identifier      = "prod-postgres-primary"
  engine          = "postgres"
  engine_version  = "16.2"
  instance_class  = "db.r7g.4xlarge"
  parameter_group_name = aws_db_parameter_group.prod_postgres16_oltp.name

  # Additional configuration omitted for brevity
}
```

The `lifecycle { create_before_destroy = true }` block matters here. When Terraform needs to replace a parameter group (because you changed its family or name), it will create the new group before destroying the old one, avoiding a gap where the instance references a nonexistent group.

One operational discipline that saves significant pain: treat parameter groups as immutable artifacts. Instead of modifying `prod-postgres16-oltp` in place, create `prod-postgres16-oltp-v2`, validate it on a staging instance, then cut production over. In-place modification of a live parameter group makes rollback much harder, and AWS has no native parameter group version history. If you modify a group and the change causes problems, you need to know what the previous values were — which means you need your own version control.

12.3 Option Groups: SQL Server and MySQL Extensions

PostgreSQL on RDS handles engine extensions through parameter groups (specifically `shared_preload_libraries`) and the `CREATE EXTENSION SQL` command. MySQL and SQL Server on RDS use an additional construct: **option groups**.

An option group is a collection of engine-specific features that go beyond configuration parameters. For SQL Server, this includes features like Transparent Data Encryption (TDE), SQL Server Audit, and SQL Server Agent. For MySQL, it includes features like MariaDB Audit Plugin, MySQL Memcached, and S3 integration.

The distinction between parameters and options reflects the architectural difference in these engines. A MySQL parameter like `innodb_buffer_pool_size` is a configuration value passed to the engine at startup. The MySQL Memcached plugin, by contrast, is a daemon process that runs alongside

MySQL and requires its own configuration, network ports, and lifecycle management — it's an option, not a parameter.

For SQL Server DBAs, option groups are how you enable SQL Server Agent, which is critical because many SQL Server jobs, maintenance plans, and SSIS integrations depend on Agent. Without enabling it through the option group, your RDS SQL Server instance has no Agent — and you will not find a way around this constraint.

```
# Create an option group for SQL Server 2019 Enterprise
aws rds create-option-group \
  --option-group-name prod-sqlserver-ee-2019 \
  --engine-name sqlserver-ee \
  --major-engine-version "15.00" \
  --option-group-description "SQL Server 2019 EE production options" \
  --region us-east-1

# Add SQL Server Agent and SQL Server Audit
aws rds add-option-to-option-group \
  --option-group-name prod-sqlserver-ee-2019 \
  --options \

    ↪ '{"OptionName":"SQLSERVER_BACKUP_RESTORE","OptionSettings":[{"Name":"IAM_ROLE_ARN","Value":"arn:aws:iam::123456789012:role/SQLSERVER_BACKUP_RESTORE"}]}' \
    ↪ \

    ↪ '{"OptionName":"SQLSERVER_AUDIT","OptionSettings":[{"Name":"IAM_ROLE_ARN","Value":"arn:aws:iam::123456789012:role/SQLSERVER_AUDIT"}]}' \
    ↪ \
  --apply-immediately \
  --region us-east-1

# Verify options are active
aws rds describe-option-groups \
  --option-group-name prod-sqlserver-ee-2019 \
  --query "OptionGroupsList[0].Options[*].{Option:OptionName,Status:OptionSettings}" \
  --region us-east-1
```

The `SQLSERVER_BACKUP_RESTORE` option is what enables native backup/restore to and from S3 — the equivalent of your on-premises backup jobs writing to network shares. Without this option, you cannot restore a `.bak` file from S3 into RDS SQL Server. This is one of the most common blockers during SQL Server migrations to RDS, because DBAs attempt to restore their database and get The `RESTORE` option is not supported on this platform until they enable the option.

SQL Server TDE on RDS deserves special mention. You can enable TDE via option group, but the certificate management model is different from on-premises. RDS manages the TDE certificate internally, and you cannot export it. If you're migrating a TDE-encrypted database from on-premises to RDS, you must decrypt the database first, or use AWS DMS to migrate the data without the encryption artifact. This is a hard constraint, not a workaround-able limitation.

For MySQL, audit logging through the MariaDB Audit Plugin is configured as an option:

```
# Add MariaDB Audit Plugin to MySQL 8.0 option group
aws rds add-option-to-option-group \
  --option-group-name prod-mysql80-options \
  --options '[{
    "OptionName": "MARIADB_AUDIT_PLUGIN",
    "OptionSettings": [
      {"Name": "SERVER_AUDIT_EVENTS", "Value": "CONNECT,QUERY_DDL,QUERY_DML"},
      {"Name": "SERVER_AUDIT_EXCL_USERS", "Value": "rdsadmin"},
      {"Name": "SERVER_AUDIT_QUERY_LOG_LIMIT", "Value": "1024"}
    ]
  }]' \
  --apply-immediately \
  --region us-east-1
```

One operational constraint with option groups: certain options are persistent. A persistent option cannot be removed from an option group once added if there is a database in a state that depends on it. TDE is the most common example — once you enable TDE on a SQL Server instance, you cannot remove the option without first decrypting all affected databases. Plan your option group contents carefully before attaching them to production instances.

12.4 Production Tuning Patterns for RDS PostgreSQL

RDS PostgreSQL tuning starts from the same first principles as self-managed PostgreSQL but requires adjustments for the managed environment. You don't know the exact hardware specs of the underlying host — you know the instance class, and AWS publishes the vCPU count and RAM for each class, which is enough to derive reasonable starting values.

For a `db.r7g.4xlarge` (16 vCPU, 128 GB RAM), a production OLTP baseline looks like this:

Parameter	Value	Reasoning
<code>shared_buffers</code>	Not directly settable on RDS	RDS auto-sets based on instance RAM
<code>work_mem</code>	64 MB (65536 kB)	Conservative for OLTP with high concurrency
<code>maintenance_work_mem</code>	2 GB	Speeds up VACUUM and index builds
<code>effective_cache_size</code>	96 GB	75% of RAM, signals to planner what's cacheable
<code>checkpoint_completion_target</code>	0.9	Spreads checkpoint I/O
<code>random_page_cost</code>	1.1	For gp3/io1 SSD storage on RDS
<code>max_connections</code>	500	Static; requires reboot; use RDS Proxy if > 500
<code>log_min_duration_statement</code>	1000 ms	Log slow queries; adjust down for tighter SLA
<code>auto_vacuum_vacuum_cost_delay</code>	2 ms	Aggressive autovacuum for write-heavy workloads

A critical difference from self-managed PostgreSQL: `shared_buffers` is managed by RDS. AWS sets it to 25% of instance RAM by default and does not expose it as a directly settable parameter — the parameter exists in the group, but AWS overrides it based on instance class at startup. You will find `shared_buffers` listed in the parameter group with a formula like `{DBInstanceClassMemory/32768}` — that formula evaluates at instance startup, not when you modify the parameter group. This surprises DBAs who try to set `shared_buffers = 32GB` and then query `SHOW shared_buffers` and find a different value.

For OLAP workloads on the same instance class, you'd push `work_mem` to 256 MB or higher and set `max_parallel_workers_per_gather` to 4, trading concurrency headroom for single-query throughput. On RDS, parallel query is available and functions as it does on self-managed PostgreSQL —

but you cannot control OS scheduler settings or NUMA topology that affect parallel worker performance on large instances.

A Python script using boto3 to audit parameter group drift across your fleet — identifying instances where custom parameters have diverged from your standard baseline — is one of the most valuable operational tools you can build:

```
import boto3
import json

def audit_parameter_groups(region: str, baseline_group: str) -> dict:
    """
    Compare all custom parameter groups against a baseline group.
    Returns instances where parameter values differ from baseline.
    """
    rds = boto3.client('rds', region_name=region)

    # Load baseline parameters
    baseline_params = {}
    paginator = rds.get_paginator('describe_db_parameters')
    for page in paginator.paginate(DBParameterGroupName=baseline_group):
        for param in page['Parameters']:
            if param.get('ParameterValue') and param.get('IsModifiable'):
                baseline_params[param['ParameterName']] = param['ParameterValue']

    drift_report = {}

    # Get all DB instances
    instance_paginator = rds.get_paginator('describe_db_instances')
    for page in instance_paginator.paginate():
        for instance in page['DBInstances']:
            instance_id = instance['DBInstanceIdentifier']
            engine = instance['Engine']

            # Only check PostgreSQL instances
            if not engine.startswith('postgres'):
                continue

            param_groups = instance.get('DBParameterGroups', [])
            for pg in param_groups:
                pg_name = pg['DBParameterGroupName']
                pg_status = pg['ParameterApplyStatus']

                # Skip default groups and instances using baseline
                if pg_name.startswith('default.') or pg_name == baseline_group:
                    continue

                # Compare parameters
                instance_params = {}
                try:
                    for param_page in paginator.paginate(DBParameterGroupName=pg_name):
                        for param in param_page['Parameters']:
                            if param.get('ParameterValue') and param.get('IsModifiable'):
                                instance_params[param['ParameterName']] = param['ParameterValue']
                except rds.exceptions.DBParameterGroupNotFound:
                    continue

                diffs = {}
                for param_name, baseline_value in baseline_params.items():
                    instance_value = instance_params.get(param_name)
                    if instance_value and instance_value != baseline_value:
                        diffs[param_name] = {
                            'baseline': baseline_value,
                            'actual': instance_value
                        }

                if diffs or pg_status == 'pending-reboot':
                    drift_report[instance_id] = {
```

```
        'parameter_group': pg_name,
        'apply_status': pg_status,
        'parameter_diffs': diffs
    }

    return drift_report

if __name__ == '__main__':
    report = audit_parameter_groups(
        region='us-east-1',
        baseline_group='prod-postgres16-oltp'
    )
    print(json.dumps(report, indent=2))
```

Run this script as a daily CloudWatch Events job and publish the results to an SNS topic. Any instance with `pending-reboot` status means you have parameters that aren't actually in effect — a common source of mysterious performance differences between instances that should be identically tuned.

12.5 Tuning Limits: What RDS Won't Let You Do

Understanding the hard boundaries of RDS tuning is as important as knowing what you can configure. These limits are not bugs or oversights — they are deliberate constraints of the managed service model. Knowing them up front prevents the frustration of spending hours chasing a tuning approach that will never work on RDS.

OS-level tuning is unavailable. On self-managed PostgreSQL, you might set `vm.overcommit_memory = 2`, tune `vm.dirty_ratio`, configure `huge_pages`, or pin PostgreSQL to specific NUMA nodes. None of this is available on RDS. AWS handles the underlying OS configuration. Some instance classes benefit from huge pages, and AWS does enable them on instances with sufficient RAM (r-class instances above a certain size), but you cannot control this or verify it directly.

pg_hba.conf is replaced by security groups and IAM. The familiar host-based authentication file is not editable on RDS. Network access control is handled by VPC security groups, and authentication beyond password-based methods is handled through IAM database authentication. If your on-premises PostgreSQL security model relied on `pg_hba.conf` to restrict specific users to specific databases from specific IP ranges, you'll reconstruct that model using a combination of VPC security groups (for network-layer restriction), database-level `GRANT` statements (for authorization), and RDS IAM auth (for eliminating static passwords).

superuser access is restricted. RDS creates a master user that has broad privileges but is not a true PostgreSQL superuser. The `rds_superuser` role gives elevated access — you can create extensions, manage other users, and access most system functions — but you cannot access the PostgreSQL `superuser` privilege that allows operations like `pg_read_file()` on arbitrary paths, modifying `pg_authid` directly, or using `COPY ... TO/FROM PROGRAM`. Some extensions require true superuser installation and cannot be installed on RDS at all. Before planning an extension-heavy migration, verify each extension against the [list of supported extensions](#) for your target RDS engine version.

Write-ahead log access is limited. Logical replication slots work on RDS PostgreSQL, but you cannot inspect or manage physical WAL files directly. Tools like `pg_waldump`, custom WAL

archiving scripts that pull from the WAL stream, and certain CDC implementations that expect direct WAL file access won't function as they would on self-managed instances. Use RDS's native logical replication and the `pgoutput` or `wal2json` plugins instead.

Parameter group changes don't propagate to read replicas automatically. A read replica in RDS has its own parameter group association. When you tune the primary, you must also update the replica's parameter group separately. This is particularly easy to miss with `max_connections` — your primary handles 500 connections, your replica is still at the default 100, and connection attempts to the replica during a read-heavy event start failing. Set both explicitly, and consider whether replica parameter groups should be identical to or different from primary groups (replicas often benefit from higher `work_mem` values for analytical queries that run against them).

The following SQL gives you a quick view of which parameters are currently in effect on an RDS PostgreSQL instance, cross-referenced against what you'd expect from your parameter group — useful for validating that dynamic changes took hold:

```
-- Check effective parameter values and their sources
SELECT
    name,
    setting,
    unit,
    source,
    reset_val,
    pending_restart
FROM pg_settings
WHERE source NOT IN ('default', 'client')
    AND name IN (
        'work_mem',
        'maintenance_work_mem',
        'effective_cache_size',
        'max_connections',
        'shared_buffers',
        'checkpoint_completion_target',
        'random_page_cost',
        'log_min_duration_statement',
        'max_parallel_workers_per_gather',
        'autovacuum_vacuum_cost_delay'
    )
ORDER BY name;
```

The `source` column is particularly informative. A value showing `configuration file` came from your parameter group. A value showing `override` means RDS itself applied it, superseding your parameter group setting — this is how you discover that `shared_buffers` is being managed outside your group. A value showing `session` means a session-level `SET` command is overriding the group setting for that connection, which can be a source of confusing inconsistency when a developer has embedded `SET work_mem = '1GB'` in application startup code.

12.6 RDS MySQL and Aurora MySQL Parameter Tuning

MySQL on RDS follows the same structural model but with engine-specific parameters and a few MySQL-specific operational constraints that differ from PostgreSQL behavior.

The most impactful parameters for RDS MySQL 8.0 OLTP workloads:

Parameter	Recommended Value	Apply Type
<code>innodb_buffer_pool_size</code>	75% of RAM	Static
<code>innodb_log_file_size</code>	1–2 GB	Static
<code>innodb_flush_log_at_trx_commit</code>	1 (default, safest)	Dynamic
<code>innodb_io_capacity</code>	3000–6000 for gp3	Dynamic
<code>innodb_io_capacity_max</code>	2× <code>innodb_io_capacity</code>	Dynamic
<code>max_connections</code>	Instance-class dependent	Static
<code>query_cache_type</code>	0 (disabled in 8.0)	Static
<code>slow_query_log</code>	1	Dynamic
<code>long_query_time</code>	1	Dynamic
<code>log_queries_not_using_indexes</code>	1	Dynamic
<code>binlog_format</code>	ROW (required for RDS replicas)	Dynamic

The `innodb_buffer_pool_size` situation on RDS MySQL is different from `shared_buffers` on PostgreSQL. RDS does expose `innodb_buffer_pool_size` as a settable parameter, and the formula $\{\text{DBInstanceClassMemory} \times 3/4\}$ is the default expression — but you can override this with a concrete value in your parameter group. However, changing `innodb_buffer_pool_size` is a static parameter requiring a reboot on MySQL 5.7 and earlier. MySQL 8.0 supports online buffer pool resizing via `innodb_buffer_pool_chunk_size`, and RDS 8.0 exposes this as a dynamic operation through the parameter group.

For Aurora MySQL specifically, the cluster parameter group governs replication-related settings like `binlog_format` and `aurora_parallel_query`, while the instance parameter group governs memory and execution settings. A common misconfiguration in Aurora MySQL is enabling `aurora_parallel_query` at the cluster level but failing to set `aurora_disable_hash_join=0` at the instance level, resulting in parallel query being enabled but hash join being unavailable, which limits its effectiveness for analytical workloads.

```
# Aurora MySQL cluster parameter group with parallel query enabled
aws rds create-db-cluster-parameter-group \
  --db-cluster-parameter-group-name prod-aurora-mysql80-cluster \
  --db-parameter-group-family aurora-mysql8.0 \
  --description "Aurora MySQL 8.0 cluster parameters" \
  --region us-east-1

aws rds modify-db-cluster-parameter-group \
  --db-cluster-parameter-group-name prod-aurora-mysql80-cluster \
  --parameters \
    "ParameterName=aurora_parallel_query,ParameterValue=ON,ApplyMethod=pending-reboot" \
    "ParameterName=binlog_format,ParameterValue=ROW,ApplyMethod=immediate" \
    "ParameterName=character_set_server,ParameterValue=utf8mb4,ApplyMethod=immediate" \
    "ParameterName=collation_server,ParameterValue=utf8mb4_unicode_ci,ApplyMethod=immediate" \
  --region us-east-1

# Aurora MySQL instance parameter group
aws rds create-db-parameter-group \
  --db-parameter-group-name prod-aurora-mysql80-instance \
  --db-parameter-group-family aurora-mysql8.0 \
  --description "Aurora MySQL 8.0 instance parameters" \
  --region us-east-1

aws rds modify-db-parameter-group \
  --db-parameter-group-name prod-aurora-mysql80-instance \
  --parameters \
    "ParameterName=aurora_disable_hash_join,ParameterValue=0,ApplyMethod=immediate" \
    "ParameterName=innodb_io_capacity,ParameterValue=4000,ApplyMethod=immediate" \
```

```

"ParameterName=innodb_io_capacity_max,ParameterValue=8000,ApplyMethod=immediate" \
"ParameterName=slow_query_log,ParameterValue=1,ApplyMethod=immediate" \
"ParameterName=long_query_time,ParameterValue=1,ApplyMethod=immediate" \
"ParameterName=log_queries_not_using_indexes,ParameterValue=1,ApplyMethod=immediate" \
--region us-east-1

```

One MySQL-specific tuning concern on RDS: the `max_connections` default for each instance class is set by the formula `{DBInstanceClassMemory/12582880}`. For a `db.r7g.4xlarge` with 128 GB of RAM, this evaluates to roughly 10,000 connections — a value that seems generous until you realize that MySQL’s threading model means each connection allocates per-thread buffers, and 10,000 active connections at even minimal memory per thread can exhaust instance RAM. Set `max_connections` explicitly and pair it with RDS Proxy for applications with connection-heavy patterns.

12.7 RDS SQL Server Parameter Tuning

SQL Server on RDS uses parameter groups less extensively than MySQL or PostgreSQL, because SQL Server’s traditional configuration is managed through `sp_configure` rather than a flat configuration file. RDS exposes the most critical SQL Server configuration knobs through the parameter group system, but many settings that an on-premises DBA would manage through `sp_configure` are handled differently on RDS.

The key parameter group settings for RDS SQL Server:

Parameter	Notes
<code>rds.force_ssl</code>	Enforces TLS for all connections
<code>max degree of parallelism</code>	Maps directly to MAXDOP; apply type is dynamic
<code>cost threshold for parallelism</code>	Tune alongside MAXDOP; default 5 is usually too low
<code>max server memory (MB)</code>	Set to leave ~10% of RAM for OS and RDS agent
<code>min server memory (MB)</code>	Set to a floor that prevents excessive memory release
<code>optimize for ad hoc workloads</code>	Enable for OLTP workloads to reduce plan cache bloat
<code>remote query timeout (s)</code>	Set appropriately for linked server or distributed queries

On self-managed SQL Server, you’d run:

```

-- Self-managed SQL Server: configure MAXDOP and cost threshold
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;

EXEC sp_configure 'max degree of parallelism', 4;
EXEC sp_configure 'cost threshold for parallelism', 50;
RECONFIGURE;

```

On RDS SQL Server, the same configuration happens through the parameter group. But you can also still call `sp_configure` on RDS SQL Server for parameters that RDS exposes through it — the parameter group values and `sp_configure` values interact, with the parameter group acting as the persistent layer. If you change `MAXDOP` via `sp_configure` directly, that change may not survive an instance restart because the parameter group value takes precedence at startup. Use the parameter group as the source of truth for any setting it covers.

RDS SQL Server also supports a set of SQL-level configurations that you manage directly in T-SQL, without going through parameter groups, because they are database-scoped or session-scoped rather than instance-scoped:

```
-- Database-scoped configurations (SQL Server 2016+, available on RDS)
ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 4;
ALTER DATABASE SCOPED CONFIGURATION SET LEGACY_CARDINALITY_ESTIMATION = OFF;
ALTER DATABASE SCOPED CONFIGURATION SET PARAMETER_SNIFFING = ON;
ALTER DATABASE SCOPED CONFIGURATION SET QUERY_OPTIMIZER_HOTFIXES = ON;

-- Check current database-scoped configurations
SELECT
    name,
    value,
    value_for_secondary
FROM sys.database_scoped_configurations
ORDER BY name;
```

Database-scoped configurations survive restarts because they are stored in the database metadata, not in instance-level configuration. These give you a useful layer of per-database tuning that complements the instance-level parameter group — particularly valuable when you host multiple databases on a single RDS SQL Server instance with different workload characteristics.

12.8 Using RDS Performance Insights with Parameter Tuning

Parameter tuning in isolation is guesswork. The feedback loop that makes tuning effective on RDS is Performance Insights, AWS’s built-in query analysis tool that shows database load broken down by wait events, SQL statements, and sessions over time.

Performance Insights is enabled at the instance level and requires that `pg_stat_statements` (for PostgreSQL) or its MySQL/SQL Server equivalent is active. For PostgreSQL, you must include `pg_stat_statements` in `shared_preload_libraries` — a static parameter requiring a reboot — before Performance Insights can surface per-query statistics.

Once enabled, Performance Insights gives you:

- A timeline of **Average Active Sessions (AAS)** — the primary metric showing how loaded the database is relative to its vCPU count
- Breakdown by **wait event** — where sessions are spending time (I/O, lock waits, CPU, network)
- **Top SQL** by load contribution — which queries are consuming the most database time
- Historical data up to two years with the paid tier

The practical tuning workflow using Performance Insights looks like this:

1. Establish a Performance Insights baseline over a representative period (at minimum, a full business cycle)

2. Identify the dominant wait events during peak load
3. Cross-reference with top SQL statements to find the highest-impact queries
4. Apply parameter changes targeting the observed bottleneck
5. Compare Performance Insights before and after the change over equivalent load periods

For example: if your Performance Insights view shows `io/datafileread` as the dominant wait event and the top SQL statements are full-table scans on large tables, the parameter tuning response is to lower `random_page_cost` (already done in your baseline) and verify `effective_cache_size` is set high enough that the planner understands available memory, then focus on index coverage for those specific queries. Raising `work_mem` won't help `io/datafileread` — it addresses CPU-sort and hash-join pressure, which shows up as CPU wait or `BufferIO` events.

You can pull Performance Insights data programmatically for automated analysis:

```
import boto3
from datetime import datetime, timedelta, timezone

def get_top_sql_by_load(
    instance_id: str,
    region: str,
    lookback_hours: int = 24
) -> list:
    """
    Retrieve top SQL statements by average active sessions
    from Performance Insights.
    """
    pi = boto3.client('pi', region_name=region)
    rds = boto3.client('rds', region_name=region)

    # Get the DBI resource identifier (required by Performance Insights API)
    instance = rds.describe_db_instances(
        DBInstanceIdentifier=instance_id
    )['DBInstances'][0]
    dbi_resource_id = instance['DbiResourceId']

    end_time = datetime.now(timezone.utc)
    start_time = end_time - timedelta(hours=lookback_hours)

    response = pi.get_resource_metrics(
        ServiceType='RDS',
        Identifier=dbi_resource_id,
        MetricQueries=[
            {
                'Metric': 'db.load.avg',
                'GroupBy': {
                    'Group': 'db.sql',
                    'Dimensions': ['db.sql.statement'],
                    'Limit': 10
                }
            }
        ],
        StartTime=start_time,
        EndTime=end_time,
        PeriodInSeconds=3600
    )

    results = []
    for metric_list in response.get('MetricList', []):
        for key in metric_list.get('Keys', []):
            dimensions = key.get('Dimensions', {})
            sql_text = dimensions.get('db.sql.statement', 'N/A')
            # Sum the load values across the time window
            data_points = metric_list.get('DataPoints', [])
            total_load = sum(
                dp.get('Value', 0) for dp in data_points
                if dp.get('Key') == key
            )
```

```

    )
    results.append({
        'sql': sql_text[:200], # Truncate for display
        'total_aas': round(total_load, 3)
    })

    return sorted(results, key=lambda x: x['total_aas'], reverse=True)

if __name__ == '__main__':
    top_sql = get_top_sql_by_load(
        instance_id='prod-postgres-primary',
        region='us-east-1',
        lookback_hours=24
    )
    for rank, entry in enumerate(top_sql, 1):
        print(f"{rank}. AAS={entry['total_aas']:.3f} | {entry['sql'][:100]}")

```

This script gives you a programmatic version of the Performance Insights top-SQL view, which you can integrate into alerting pipelines or daily operational reports without requiring console access.

For SQL Server on RDS, Performance Insights surfaces SQL Server wait statistics — the same wait types you’d examine through `sys.dm_os_wait_stats` on a self-managed instance, but presented in a time-series interface without requiring DMV access. The common SQL Server wait types you’ll tune around:

Wait Type	Likely Cause	Parameter Response
PAGEIOLATCH_SH	Buffer pool pressure, reads hitting disk	Increase <code>max server memory</code> , review <code>effective_cache_size</code> equivalent
LCK_M_*	Lock contention	Application query changes, not parameter tuning
CXPACKET / CXCONSUMER	Parallel query coordination overhead	Reduce <code>MAXDOP</code> or increase <code>cost threshold for parallelism</code>
WRITELOG	Log write latency	Move to io1/io2 storage, increase IOPS provisioning
SOS_SCHEDULER_YIELD	CPU pressure, long-running queries competing	<code>MAXDOP</code> tuning, query-level optimization

The `CXPACKET` case illustrates the interplay between Performance Insights and parameter tuning directly. If you see high `CXPACKET` wait time in Performance Insights and the top SQL statements are short OLTP transactions (not the analytical queries that should benefit from parallelism), the response is to raise the `cost threshold for parallelism` parameter from the default value of 5 to something in the range of 40–75. This forces the optimizer to only use parallelism for queries with a high estimated cost, eliminating wasted parallel worker overhead on simple lookups.

12.9 Parameter Group Change Management in Production

The riskiest moment in RDS parameter group management is applying a static parameter change to a production instance, because it requires a restart. A Multi-AZ instance will failover, causing

a brief outage (typically under 60 seconds for DNS cutover). A Single-AZ instance will have a hard restart with a longer outage. Either way, you're making a conscious decision to introduce a disruption, and you need a disciplined process around it.

A production-grade parameter change workflow:

Step 1: Document the current state. Before changing anything, export the current parameter group values.

```
aws rds describe-db-parameters \
  --db-parameter-group-name prod-postgres16-oltp \
  --query "Parameters[?Source=='user']" \
  --output json > param-group-backup-$(date +%Y%m%d).json
```

The `Source=='user'` filter returns only parameters you've explicitly set, not the full catalog of engine defaults. This is your rollback artifact.

Step 2: Apply changes to a staging instance first. Create a copy of the parameter group, apply the changes, associate it with a staging instance that mirrors production's instance class and data volume, and run representative workloads. For static parameter changes, restart the staging instance and verify the application behaves correctly before touching production.

Step 3: Schedule the production change. Static parameter changes in production should happen during a maintenance window, with the application team aware that a brief failover will occur. For Multi-AZ deployments, the failover is automatic and transparent to the application if the application handles transient connection drops (using retry logic and connection pooling). For Single-AZ, the outage duration depends on instance size — larger instances take longer to restart.

Step 4: Validate the change took effect. After restart, confirm that the static parameters are active:

```
-- PostgreSQL: verify static parameters applied after restart
SELECT name, setting, source, pending_restart
FROM pg_settings
WHERE pending_restart = true;
-- Should return 0 rows after a successful restart

-- SQL Server: verify instance-level configuration
SELECT name, value_in_use, description
FROM sys.configurations
WHERE name IN (
  'max degree of parallelism',
  'cost threshold for parallelism',
  'max server memory (MB)',
  'optimize for ad hoc workloads'
)
ORDER BY name;
```

Step 5: Monitor Performance Insights for 24–48 hours. Compare AAS and wait event distribution against your pre-change baseline. A parameter change that looks reasonable in theory can increase load in practice — for example, raising `max_connections` above the instance's connection handling capacity can increase CPU wait if the additional connections generate more concurrent query load than the instance can absorb.

For teams managing many RDS instances, AWS Systems Manager Parameter Store (not to be confused with RDS parameter groups, despite the naming collision) is useful for storing the canonical parameter values for each tier in your fleet. Your Terraform modules read from SSM Parameter Store to populate parameter group values, creating a single source of truth that spans both the infrastructure code and operational documentation.

12.10 Monitoring Parameter Group Effectiveness

Tuning parameters without closing the feedback loop is not tuning — it's configuration editing. The monitoring layer that tells you whether your parameter changes produced the intended result includes Performance Insights (covered in 12.8), CloudWatch RDS metrics, and engine-level views.

The core CloudWatch metrics to monitor in relation to parameter changes:

Metric	What It Reflects	Related Parameters
ReadIOPS / WriteIOPS	Disk I/O activity	<code>random_page_cost</code> , <code>effective_cache_size</code>
ReadLatency / WriteLatency	Storage latency	Storage type and IOPS provisioning
CPUUtilization	Total CPU demand	<code>max_connections</code> , <code>MAXDOP</code> , <code>work_mem</code>
FreeableMemory	Available RAM	<code>shared_buffers</code> , <code>work_mem</code> , <code>max_connections</code>
DatabaseConnections	Active connection count	<code>max_connections</code> , RDS Proxy
DiskQueueDepth	Storage queue depth	<code>innodb_io_capacity</code> , checkpoint parameters
SwapUsage	Memory pressure overflow	<code>work_mem</code> set too high, <code>max_connections</code> too high

`SwapUsage` is the canary in the coal mine for over-aggressive memory parameter tuning. If `SwapUsage` climbs after you raise `work_mem` or `shared_buffers`, the instance is swapping to satisfy memory allocations, which will degrade performance far more than conservative memory settings would have. Set a CloudWatch alarm on `SwapUsage > 0` for production RDS instances — swap should never be in use on a properly sized and tuned instance.

A CloudWatch alarm for swap in Terraform:

```
resource "aws_cloudwatch_metric_alarm" "rds_swap_usage" {
  alarm_name      = "rds-${var.instance_id}-swap-usage"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods = 2
  metric_name     = "SwapUsage"
  namespace      = "AWS/RDS"
  period         = 300
  statistic      = "Average"
  threshold      = 0
  alarm_description = "RDS instance is using swap - possible memory pressure from parameter tuning"
  alarm_actions   = [var.sns_topic_arn]

  dimensions = {
    DBInstanceIdentifier = var.instance_id
  }
}
```

Beyond CloudWatch, query the engine directly for validation. On PostgreSQL, `pg_stat_bgwriter` tells you about checkpoint behavior relative to your `checkpoint_completion_target` setting:

```
-- PostgreSQL: checkpoint and buffer statistics
```

```
SELECT
    checkpoints_timed,
    checkpoints_req,
    checkpoint_write_time,
    checkpoint_sync_time,
    buffers_checkpoint,
    buffers_clean,
    buffers_backend,
    buffers_alloc,
    stats_reset
FROM pg_stat_bgwriter;
```

A high `checkpoints_req` (checkpoints forced by WAL fill rather than scheduled) relative to `checkpoints_timed` means WAL is filling faster than the checkpoint cycle. The response is to increase `max_wal_size` (available in the RDS parameter group for PostgreSQL 9.5+), which gives the checkpoint process more runway before forced flushes.

A high `buffers_backend` relative to `buffers_checkpoint` means backends are writing dirty buffers directly to disk because the background writer can't keep up. Tune `bgwriter_lru_maxpages` and `bgwriter_delay` to be more aggressive.

On MySQL, `SHOW ENGINE INNODB STATUS` and the information schema tables provide equivalent feedback:

```
-- MySQL: InnoDB buffer pool efficiency
SELECT
    pool_id,
    pool_size,
    free_buffers,
    database_pages,
    old_database_pages,
    modified_db_pages,
    pending_reads,
    pending_write_lrus,
    pending_write_flush_list,
    pages_read,
    pages_created,
    pages_written
FROM information_schema.innodb_buffer_pool_stats;
```

A `free_buffers` value consistently near zero means the buffer pool is full and pages are being evicted frequently — the signal that `innodb_buffer_pool_size` may be undersized for the working set. If `modified_db_pages` is consistently high relative to `pool_size`, the background flush threads aren't keeping up, and raising `innodb_io_capacity` may help.

Key Takeaways

- **Parameter groups are the configuration interface for RDS, but they are not omnipotent.** AWS enforces hard limits on what can be modified — OS-level tuning, superuser access, and certain engine internals are outside the parameter group system entirely. Effective RDS tuning starts by understanding these boundaries.
- **Static parameters require restarts; dynamic parameters do not.** This distinction governs every production change control decision. Always check the apply type before touching a production parameter group, and monitor `pending_restart` in `pg_settings` (PostgreSQL) or equivalent engine views to confirm that static changes have been applied after the scheduled restart.

- **Option groups extend RDS with engine-specific features** — SQL Server Agent, native backup/restore to S3, and audit plugins are option group features, not parameter group entries. For SQL Server migrations in particular, enabling `SQLSERVER_BACKUP_RESTORE` via an option group is a prerequisite that blocks database restoration if missed.
 - **Treat parameter groups as versioned artifacts under source control.** Modify parameter groups in Terraform or CloudFormation rather than through the console, export current user-set parameters before any change as a rollback artifact, and test all static parameter changes on staging before scheduling production restarts. In-place modification of production parameter groups without version control makes rollback guesswork.
 - **Performance Insights closes the tuning feedback loop.** Parameter changes without a monitoring baseline are guesses. Use Performance Insights AAS trends and wait event breakdowns to identify the actual bottleneck before choosing a parameter response, and compare the same metrics over equivalent load periods before and after the change to confirm the intended effect was achieved.
-

Chapter 13: Azure SQL Database vs Managed Instance vs SQL Server on VM

When you move SQL Server workloads to Azure, the first decision you face is not which region to use or how large an instance to provision — it is which deployment model to choose. Azure offers three fundamentally different ways to run SQL Server: Azure SQL Database (a fully managed PaaS service), Azure SQL Managed Instance (a near-complete SQL Server engine in a managed container), and SQL Server on Azure Virtual Machines (IaaS, essentially your on-premises server in the cloud). Each model trades administrative control for operational convenience at a different point on the spectrum. Getting this choice wrong costs money, causes compatibility headaches, and in some cases requires a full migration to correct. This chapter maps each deployment model to the scenarios where it genuinely fits, shows you how to configure them correctly from day one, and identifies the production gotchas that no marketing documentation warns you about.

13.1 The Three Deployment Models: Mapping the Landscape

Before writing a single line of Terraform or clicking through the Azure portal, you need to understand what you are actually buying with each model.

Azure SQL Database is a database-as-a-service offering. You provision a database (or a pool of databases), and Azure manages the underlying instance entirely. There is no SQL Server Agent in the traditional sense. Cross-database queries require elastic queries. Linked servers do not exist. Features like SQL Server Audit work, but you configure them through Azure-native tooling rather than T-SQL. The engine is constantly patched by Microsoft on a schedule you influence but do not fully control. For greenfield applications that fit the PaaS model, this is powerful. For lift-and-shift migrations, it is frequently a poor fit without application rework.

Azure SQL Managed Instance is the middle path. It runs a near-complete SQL Server engine — SQL Server Agent, cross-database queries, linked servers, CLR assemblies, Service Broker, Database Mail — inside a managed VNet-injected service. You get instance-level features that Azure SQL Database strips away. Patching still happens on Microsoft's schedule (with a configurable maintenance window), but you control far more of the engine. The compatibility surface area covers the large majority of on-premises SQL Server 2019 features, with a well-documented list of exceptions. Lift-and-shift migrations from on-premises SQL Server work with minimal code changes.

SQL Server on Azure VM is IaaS. You own the OS, you own the SQL Server installation, you patch everything yourself (though Azure provides automated patching assistance), and you have complete feature parity with on-premises SQL Server because it is on-premises SQL Server, running on Azure hardware. You pay for this control in operational overhead: OS patching, SQL Server patching, backup management, HA configuration, and storage optimization are all your responsibility.

The decision matrix looks like this in practice:

Requirement	Azure SQL DB	Managed Instance	SQL Server on VM
Lift-and-shift from on-prem	Poor	Excellent	Excellent
Greenfield cloud-native app	Excellent	Good	Overkill
SQL Server Agent jobs	No (elastic jobs)	Yes	Yes
Cross-database queries	Elastic query only	Yes	Yes
Linked servers	No	Yes	Yes
CLR assemblies	No	Yes (safe only)	Yes
Service Broker	No	Yes	Yes
Full OS access	No	No	Yes
Windows auth / AD	Azure AD only	Full AD integration	Full AD integration
HADR (Always On AG)	Built-in geo-replication	Built-in auto-failover groups	Manual config
Maximum single DB size	4 TB (General Purpose)	8 TB	64 TB (Premium SSD v2)

One mental shift experienced DBAs need to make: Managed Instance is not a hosted SQL Server. It is a service that exposes SQL Server semantics. You cannot RDP into it. You cannot modify startup parameters freely. You cannot install third-party tools at the OS layer. If you need to run a third-party backup agent or an encryption tool that hooks into the OS, you need SQL Server on VM.

13.2 Provisioning Each Model: Infrastructure as Code Patterns

Manual portal provisioning is fine for experimentation. Production deployments require repeatable infrastructure. The following examples use Terraform, which handles all three deployment models through the AzureRM provider.

Provisioning Azure SQL Database with an Elastic Pool:

```
# terraform/azure_sql_database.tf

terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
```



```

        version = "~> 3.75"
    }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "sql_rg" {
  name     = "rg-sql-production-eastus"
  location = "East US"
}

resource "azurerm_mssql_server" "logical_server" {
  name                               = "sql-prod-eastus-001"
  resource_group_name                = azurerm_resource_group.sql_rg.name
  location                          = azurerm_resource_group.sql_rg.location
  version                            = "12.0"
  administrator_login                = "sqladmin"
  administrator_login_password      = var.sql_admin_password

  # Enforce Azure AD authentication in addition to SQL auth
  azuread_administrator {
    login_username = "DBA-Admin-Group"
    object_id     = var.aad_admin_group_object_id
    tenant_id     = var.tenant_id
  }

  # Prevent public internet access - enforce private endpoint
  public_network_access_enabled = false

  tags = {
    environment = "production"
    owner       = "dba-team"
  }
}

resource "azurerm_mssql_elasticpool" "app_pool" {
  name                = "epool-prod-app-tier"
  resource_group_name = azurerm_resource_group.sql_rg.name
  location            = azurerm_resource_group.sql_rg.location
  server_name         = azurerm_mssql_server.logical_server.name

  sku {
    name     = "GP_Gen5"
    tier      = "GeneralPurpose"
    family   = "Gen5"
    capacity = 16 # 16 vCores shared across all databases in the pool
  }

  per_database_settings {
    min_capacity = 0.25 # Minimum vCores per DB - prevents starvation
    max_capacity = 8    # Maximum vCores a single DB can consume
  }

  max_size_gb = 512

  # Zone redundancy is non-negotiable for production - check current Azure pricing for your tier
  zone_redundant = true
}

resource "azurerm_mssql_database" "app_db" {
  name                = "AppDatabase"
  server_id           = azurerm_mssql_server.logical_server.id
  elastic_pool_id     = azurerm_mssql_elasticpool.app_pool.id

  # collation must match your application - changing this post-creation
  # requires a full database rebuild
  collation = "SQL_Latin1_General_CP1_CI_AS"
}

```

```
# geo-redundant backups add cost but are required for PITR across regions
geo_backup_enabled = true

short_term_retention_policy {
  retention_days      = 35
  backup_interval_in_hours = 12
}

long_term_retention_policy {
  weekly_retention    = "P4W"
  monthly_retention   = "P12M"
  yearly_retention    = "P5Y"
  week_of_year       = 1
}
}
```

Provisioning Azure SQL Managed Instance:

Managed Instance requires a dedicated subnet — this is non-negotiable and frequently misunderstood. The subnet must be delegated exclusively to `Microsoft.Sql/managedInstances` and cannot be shared with any other resource type.

```
# terraform/managed_instance.tf

resource "azurerm_virtual_network" "mi_vnet" {
  name                = "vnet-mi-production"
  resource_group_name = azurerm_resource_group.sql_rg.name
  location            = azurerm_resource_group.sql_rg.location
  address_space       = ["10.10.0.0/16"]
}

resource "azurerm_subnet" "mi_subnet" {
  name                = "snet-managed-instance"
  resource_group_name = azurerm_resource_group.sql_rg.name
  virtual_network_name = azurerm_virtual_network.mi_vnet.name

  # /27 gives you 27 IP addresses - enough for a 2-node MI cluster.
  # A General Purpose MI consumes ~5 IPs, Business Critical ~9 IPs.
  # Undersizing this subnet is a common and painful mistake.
  address_prefixes = ["10.10.1.0/27"]

  delegation {
    name = "managedInstanceDelegation"
    service_delegation {
      name = "Microsoft.Sql/managedInstances"
      actions = [
        "Microsoft.Network/virtualNetworks/subnets/join/action",
        "Microsoft.Network/virtualNetworks/subnets/prepareNetworkPolicies/action",
        "Microsoft.Network/virtualNetworks/subnets/unprepareNetworkPolicies/action"
      ]
    }
  }
}

resource "azurerm_mssql_managed_instance" "prod_mi" {
  name                = "mi-prod-eastus-001"
  resource_group_name = azurerm_resource_group.sql_rg.name
  location            = azurerm_resource_group.sql_rg.location

  administrator_login          = "sqladmin"
  administrator_login_password = var.sql_admin_password

  license_type = "LicenseIncluded" # Or "BasePrice" if you have SA
  sku_name     = "GP_Gen5"         # General Purpose, 5th generation
  vcores       = 8
  storage_size_in_gb = 256

  subnet_id = azurerm_subnet.mi_subnet.id
}
```

```
# Maintenance window - choose a low-traffic window.
# DayOfWeek 6 = Saturday, start at 02:00 UTC
maintenance_configuration_name = "SQL_EastUS_MI_2"

# Timezone cannot be changed post-creation
timezone_id = "Eastern Standard Time"

# Minimum TLS version enforcement
minimum_tls_version = "1.2"

tags = {
  environment = "production"
  tier         = "managed-instance"
}
}
```

Important subnet sizing note: A Business Critical Managed Instance with 2 nodes consumes approximately 9 IPs during normal operation and up to 17 during failover. A /27 subnet (27 usable IPs) is the absolute minimum for a single instance. For multiple instances or upgrade scenarios, use /24. Resizing a subnet after Managed Instance deployment requires taking the instance offline — plan this correctly the first time.

Azure CLI for quick deployment validation:

```
# Azure CLI - validate MI readiness before Terraform apply

# Check subnet delegation is correct
az network vnet subnet show \
  --resource-group rg-sql-production-eastus \
  --vnet-name vnet-mi-production \
  --name snet-managed-instance \
  --query "delegations[].serviceName" \
  --output tsv

# Verify no conflicting NSG rules blocking MI management ports
# MI requires inbound 9000, 9003, 1438, 1440, 1452 from AzureCloud service tag
az network nsg rule list \
  --resource-group rg-sql-production-eastus \
  --nsg-name nsg-managed-instance \
  --output table

# Check the MI provisioning state - MI can take 4-6 hours to provision
az sql mi show \
  --resource-group rg-sql-production-eastus \
  --name mi-prod-eastus-001 \
  --query "{State:state, FQDN:fullyQualifiedDomainName, vCores:vCores}" \
  --output table
```

13.3 Networking: Private Endpoints, VNet Integration, and the Public Internet

One of the largest mental shifts for on-premises DBAs is networking. On-premises, you controlled network access through firewall rules, `pg_hba.conf` equivalents, SQL Server login restrictions, and physical network segmentation. In Azure, the equivalent mechanisms are Private Endpoints, VNet Service Endpoints, firewall rules on the logical server, and network security groups.

Azure SQL Database uses a logical server model. By default, the logical server has a public endpoint. Every Azure best practice guide will tell you to disable it and use Private Endpoints

instead, and they are correct. A private endpoint creates a network interface in your VNet with a private IP address that maps to your logical server. Traffic never traverses the public internet.

```
# Azure CLI - create a private endpoint for Azure SQL Database

# Step 1: Disable public access on the logical server
az sql server update \
  --resource-group rg-sql-production-eastus \
  --name sql-prod-eastus-001 \
  --enable-public-network false

# Step 2: Create the private endpoint
az network private-endpoint create \
  --name pe-sql-prod-eastus \
  --resource-group rg-sql-production-eastus \
  --vnet-name vnet-application-eastus \
  --subnet snet-application-tier \
  --private-connection-resource-id $(az sql server show \
    --resource-group rg-sql-production-eastus \
    --name sql-prod-eastus-001 \
    --query id --output tsv) \
  --group-id sqlServer \
  --connection-name conn-sql-prod-eastus

# Step 3: Create a private DNS zone for name resolution
# Without this, your application cannot resolve the SQL server FQDN
az network private-dns zone create \
  --resource-group rg-sql-production-eastus \
  --name "privatelink.database.windows.net"

az network private-dns link vnet create \
  --resource-group rg-sql-production-eastus \
  --zone-name "privatelink.database.windows.net" \
  --name dns-link-vnet-app \
  --virtual-network vnet-application-eastus \
  --registration-enabled false

# Step 4: Create DNS zone group to auto-register the private endpoint IP
az network private-endpoint dns-zone-group create \
  --resource-group rg-sql-production-eastus \
  --endpoint-name pe-sql-prod-eastus \
  --name sql-dns-zone-group \
  --private-dns-zone "privatelink.database.windows.net" \
  --zone-name sql

# Verify resolution from within the VNet
# sql-prod-eastus-001.database.windows.net should now resolve to 10.x.x.x
nslookup sql-prod-eastus-001.database.windows.net
```

Managed Instance operates differently. Because it is VNet-injected, it only has a private endpoint by default — the instance lives inside your subnet. The challenge with MI networking is that it requires specific NSG rules for the management plane. Azure’s MI management infrastructure must be able to reach the instance on ports 9000, 9003, 1438, 1440, and 1452. If you lock down your NSG incorrectly, the MI enters a failed state and becomes unreachable. The required NSG rule uses the `SqlManagement` service tag as the source.

```
# Verify MI management traffic is not blocked
# Run this before deploying MI - a blocked management port causes provisioning failure

az network nsg rule create \
  --resource-group rg-sql-production-eastus \
  --nsg-name nsg-managed-instance \
  --name AllowAzureManagement \
  --priority 100 \
  --source-address-prefixes SqlManagement \
  --destination-address-prefixes "*" \
  --destination-port-ranges 9000 9003 1438 1440 1452 \
```

```
--access Allow \  
--protocol Tcp \  
--direction Inbound  
  
# Verify connectivity to MI from application subnet  
# MI uses port 1433 for TDS (same as SQL Server) and 3342 for public endpoint (if enabled)  
az network watcher test-connectivity \  
  --resource-group rg-sql-production-eastus \  
  --source-resource /subscriptions/${SUBSCRIPTION_ID} \  
  ↪ /resourceGroups/rg-application/providers/Microsoft.Compute/virtualMachines/vm-app-001 \  
  --dest-address mi-prod-eastus-001.a1b2c3d4e5f6.database.windows.net \  
  --dest-port 1433
```

SQL Server on VM uses standard Azure VM networking. You control NSGs at the NIC and subnet level, and you configure SQL Server’s firewall through SQL Server Configuration Manager — exactly as you would on-premises. The main gotcha is that Azure VM firewall rules and SQL Server’s built-in firewall are independent. Both must allow traffic.

13.4 High Availability and Business Continuity

This section is where the three deployment models diverge most dramatically from on-premises patterns. On-premises, you built Always On Availability Groups manually: you configured Windows Server Failover Clustering, set up quorum, joined replicas, created the listener, and spent a weekend validating failover behavior. In Azure, HA is either built into the service layer (Azure SQL DB and MI) or your responsibility (SQL Server on VM).

Azure SQL Database HA uses a multi-replica architecture internally. General Purpose tier uses remote storage (Azure Premium Storage) with a single compute node — there is no synchronous replica, so failover involves attaching storage to a new compute node, which takes 20–30 seconds. Business Critical tier maintains 3–4 synchronous replicas using Always On AG technology under the covers. You cannot see the replicas directly, but failover is near-instantaneous (typically under 5 seconds). The Business Critical tier also exposes one read-only replica that you can redirect read workloads to without extra cost.

Auto-failover groups provide geo-redundancy across regions for both Azure SQL DB and Managed Instance:

```
# Azure CLI - configure auto-failover group for Azure SQL Database  
  
# Create the secondary server in paired region  
az sql server create \  
  --name sql-prod-westus-001 \  
  --resource-group rg-sql-production-westus \  
  --location "West US" \  
  --admin-user sqladmin \  
  --admin-password "${SQL_ADMIN_PASSWORD}"  
  
# Create the failover group  
az sql failover-group create \  
  --name fog-prod-eastus-westus \  
  --server sql-prod-eastus-001 \  
  --resource-group rg-sql-production-eastus \  
  --partner-server sql-prod-westus-001 \  
  --partner-resource-group rg-sql-production-westus \  
  --failover-policy Automatic \  
  --grace-period 1 # Hours before automatic failover triggers - 1 is minimum  
  
# Add databases to the failover group
```

```

az sql failover-group update \
  --name fog-prod-eastus-westus \
  --server sql-prod-eastus-001 \
  --resource-group rg-sql-production-eastus \
  --add-db AppDatabase

# The failover group creates two listener endpoints:
# Read-write: fog-prod-eastus-westus.database.windows.net (points to primary)
# Read-only: fog-prod-eastus-westus.secondary.database.windows.net

# Test failover - this promotes the secondary to primary
az sql failover-group set-primary \
  --name fog-prod-eastus-westus \
  --server sql-prod-westus-001 \
  --resource-group rg-sql-production-westus

```

RPO and RTO characteristics in production:

Deployment	HA Tier	RTO (same region)	RTO (geo-failover)	RPO
Azure SQL DB - General Purpose	Built-in	20–30 sec	~30 min	Up to 1 hour
Azure SQL DB - Business Critical	Built-in	<5 sec	~30 min	~5 sec
Managed Instance - General Purpose	Built-in	~20 sec	~1 hour	~1 hour
Managed Instance - Business Critical	Built-in	<5 sec	~30 min	~5 sec
SQL Server on VM (manual AG)	DIY	~30 sec	DIY	Depends on sync mode

SQL Server on Azure VM requires you to build Always On AG using Windows Server Failover Clustering. Azure provides an Internal Load Balancer for the AG listener (equivalent to the on-premises listener IP). The key difference from on-premises is that the ILB requires a probe port configured in the AG listener, and the floating IP behavior differs slightly from on-premises due to Azure's networking model.

```

-- SQL Server on Azure VM: AG listener with Azure ILB probe
-- The probe port (59999 here) must match the ILB health probe configuration

ALTER AVAILABILITY GROUP [AG_Production]
ADD LISTENER 'AGListener' (
  WITH IP (('10.10.2.100', '255.255.255.0')),
  PORT = 1433
);
GO

-- Configure ILB probe response - run on each replica
-- Azure ILB sends TCP probes to 59999; only the primary should respond
EXEC sp_procoption N'sp_ag_probe', 'startup', 'true';
GO

-- Verify AG health
SELECT
  ag.name                                AS availability_group,
  ar.replica_server_name,

```

```
ars.role_desc,  
ars.synchronization_health_desc,  
ars.connected_state_desc,  
ars.last_redone_time,  
ars.log_send_queue_size      AS log_send_queue_kb,  
ars.redo_queue_size          AS redo_queue_kb  
FROM sys.availability_groups ag  
JOIN sys.availability_replicas ar  
    ON ag.group_id = ar.group_id  
JOIN sys.dm_hadr_availability_replica_states ars  
    ON ar.replica_id = ars.replica_id  
ORDER BY ag.name, ars.role_desc;
```

13.5 Security: Authentication, Encryption, and Compliance Patterns

On-premises SQL Server security centered on SQL Server logins, Windows Authentication through Active Directory, TDE for encryption at rest, and SSL/TLS for encryption in transit. Azure maps these concepts onto a cloud-native model that is simultaneously more automated and more complex to configure correctly.

Azure Active Directory Authentication replaces Windows Authentication in Azure SQL Database and Managed Instance. For applications, this means using Managed Identity — the application's Azure AD identity — rather than a SQL login with a stored password. This is the cloud equivalent of using a Kerberos service account, except the credential rotation is automatic.

```
# Python - connecting to Azure SQL Database using Managed Identity  
# No password stored anywhere - the token is obtained from Azure AD  
  
import struct  
import pyodbc  
from azure.identity import DefaultAzureCredential  
  
def get_connection_with_managed_identity(server: str, database: str) -> pyodbc.Connection:  
    """  
    Connect to Azure SQL Database using Managed Identity.  
    Works in Azure VMs, App Service, AKS, and Azure Functions  
    where Managed Identity is assigned.  
  
    On-premises equivalent: Kerberos/Windows auth with service account.  
    """  
    credential = DefaultAzureCredential()  
  
    # Obtain an access token for Azure SQL Database  
    # Resource ID for Azure SQL is always this URI  
    token = credential.get_token("https://database.windows.net/.default")  
  
    # SQL Server ODBC driver expects the token in a specific binary format  
    # This packing is required - it is not optional  
    token_bytes = token.token.encode("UTF-16-LE")  
    token_struct = struct.pack(  
        f"<I{len(token_bytes)}s",  
        len(token_bytes),  
        token_bytes  
    )  
  
    connection_string = (  
        f"DRIVER={{ODBC Driver 18 for SQL Server}};"  
        f"SERVER={server};"  
        f"DATABASE={database};"  
        f"Encrypt=yes;"
```

```

        f"TrustServerCertificate=no;"
    )

    SQL_COPT_SS_ACCESS_TOKEN = 1256 # ODBC driver attribute for token auth
    conn = pyodbc.connect(
        connection_string,
        attrs_before={SQL_COPT_SS_ACCESS_TOKEN: token_struct}
    )
    return conn

# Usage example
conn = get_connection_with_managed_identity(
    server="sql-prod-eastus-001.database.windows.net",
    database="AppDatabase"
)

# Grant the Managed Identity access inside the database
# Run this as an Azure AD admin:
"""
CREATE USER [my-app-service-name] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [my-app-service-name];
ALTER ROLE db_datawriter ADD MEMBER [my-app-service-name];
-- Grant execute on specific schemas, not broad roles, in production
GRANT EXECUTE ON SCHEMA::app TO [my-app-service-name];
"""

```

Transparent Data Encryption (TDE) is enabled by default on all three deployment models in Azure. The gotcha is key management. By default, TDE uses a Microsoft-managed key (service-managed TDE). For compliance scenarios requiring customer control over key material — PCI-DSS, HIPAA, some government frameworks — you need Customer-Managed Keys (CMK) stored in Azure Key Vault. Switching from service-managed to customer-managed TDE after the database is in production is possible but requires careful planning: if the Key Vault key becomes inaccessible (deleted, soft-deleted, or permissions revoked), the database becomes encrypted but unreadable within hours.

```

# Azure CLI - configure Customer-Managed Key TDE for Azure SQL Database

# Step 1: Create a Key Vault with soft delete and purge protection
# Purge protection is REQUIRED - without it, accidental key deletion destroys your data
az keyvault create \
    --name kv-sql-tde-prod \
    --resource-group rg-sql-production-eastus \
    --location "East US" \
    --enable-soft-delete true \
    --enable-purge-protection true \
    --sku premium # Premium tier required for HSM-backed keys

# Step 2: Create the TDE protector key
az keyvault key create \
    --vault-name kv-sql-tde-prod \
    --name tde-protector-sql-prod \
    --key RSA-HSM \
    --size 2048 \
    --ops wrapKey unwrapKey get

# Step 3: Grant the logical server's system-assigned identity access to the key
SERVER_IDENTITY=$(az sql server show \
    --resource-group rg-sql-production-eastus \
    --name sql-prod-eastus-001 \
    --query identity.principalId \
    --output tsv)

az keyvault set-policy \
    --name kv-sql-tde-prod \
    --object-id "${SERVER_IDENTITY}" \

```



```

--key-permissions get wrapKey unwrapKey

# Step 4: Set the TDE protector on the logical server
KEY_ID=$(az keyvault key show \
  --vault-name kv-sql-tde-prod \
  --name tde-protector-sql-prod \
  --query key.kid \
  --output tsv)

az sql server tde-key set \
  --resource-group rg-sql-production-eastus \
  --server sql-prod-eastus-001 \
  --server-key-type AzureKeyVault \
  --kid "${KEY_ID}"

# Step 5: Enable CMK TDE on the database
az sql db tde set \
  --resource-group rg-sql-production-eastus \
  --server sql-prod-eastus-001 \
  --database AppDatabase \
  --status Enabled

```

Auditing and compliance works differently across the three models. Azure SQL Database and Managed Instance both support Azure SQL Audit, which writes audit records to Azure Blob Storage, Log Analytics, or Azure Event Hubs. SQL Server on VM uses SQL Server Audit (T-SQL CREATE SERVER AUDIT) writing to Windows Event Log or a file path, which you then ship to whatever SIEM you run.

```

-- T-SQL: Configure server-level audit on Managed Instance
-- This is equivalent to on-premises SQL Server Audit
-- Destination: Blob Storage (most common for compliance retention)

CREATE SERVER AUDIT [ComplianceAudit]
TO EXTERNAL MONITOR -- On Managed Instance, use EXTERNAL MONITOR to route
                    -- to the Azure SQL Audit infrastructure

WITH (
  QUEUE_DELAY = 1000, -- Milliseconds - 0 is synchronous (performance impact)
  ON_FAILURE = CONTINUE -- FAIL_OPERATION stops workload if audit fails; evaluate carefully
);
GO

CREATE SERVER AUDIT SPECIFICATION [CriticalServerEvents]
FOR SERVER AUDIT [ComplianceAudit]
ADD (FAILED_LOGIN_GROUP),
ADD (SUCCESSFUL_LOGIN_GROUP),
ADD (SERVER_ROLE_MEMBER_CHANGE_GROUP),
ADD (DATABASE_PERMISSION_CHANGE_GROUP),
ADD (SCHEMA_OBJECT_PERMISSION_CHANGE_GROUP),
ADD (DBCC_GROUP),
ADD (BACKUP_RESTORE_GROUP)
WITH (STATE = ON);
GO

ALTER SERVER AUDIT [ComplianceAudit] WITH (STATE = ON);
GO

-- Verify audit state
SELECT
  a.name,
  a.audit_id,
  a.type_desc,
  a.on_failure_desc,
  a.queue_delay,
  a.is_state_enabled,
  asp.name AS specification_name,
  asp.is_state_enabled AS spec_enabled
FROM sys.server_audits a
LEFT JOIN sys.server_audit_specifications asp

```

```
ON a.audit_id = asp.audit_id;
```

13.6 Performance Tuning: Service Tiers, Resource Limits, and the Query Store

Performance tuning in Azure SQL differs from on-premises in one fundamental way: resources are finite and explicitly allocated. On-premises, if your SQL Server needed more CPU, the OS scheduler gave it whatever physical cores were available. In Azure, your vCore allocation is a hard ceiling. Understanding where that ceiling sits — and how to adjust it without downtime — is core DBA knowledge for this environment.

Azure SQL Database service tiers:

- **General Purpose (GP):** Remote Azure Premium Storage. Adequate IOPS and throughput for most OLTP workloads. Single compute node with no read replicas visible to you. Failover involves reattaching storage, so plan for brief disruptions.
- **Business Critical (BC):** Local SSD storage on the compute node, with 3–4 synchronous replicas. Significantly higher IOPS ceiling. One read-only replica is available for offloading reporting. Substantially higher cost.
- **Hyperscale:** Decouples compute from storage using a distributed storage layer. Storage scales automatically without size limits. Read replicas scale horizontally. Best for large OLAP workloads or databases growing unpredictably. The architecture is fundamentally different — not a drop-in replacement for GP or BC.

Scaling without downtime:

```
# Azure CLI - scale Azure SQL Database vCore count
# Scaling within the same tier (e.g., GP 8 vCores -> GP 16 vCores)
# causes a brief connection drop (~5 seconds for BC, 20-30 seconds for GP)

az sql db update \
  --resource-group rg-sql-production-eastus \
  --server sql-prod-eastus-001 \
  --name AppDatabase \
  --edition GeneralPurpose \
  --family Gen5 \
  --capacity 16 # vCores

# Monitor the scaling operation
az sql db show \
  --resource-group rg-sql-production-eastus \
  --server sql-prod-eastus-001 \
  --name AppDatabase \
  --query "{Status:status, ServiceObjective:currentServiceObjectiveName, MaxSizeGB:maxSizeBytes}" \
  --output table

# For Managed Instance scaling - this can take 4-6 hours
az sql mi update \
  --resource-group rg-sql-production-eastus \
  --name mi-prod-eastus-001 \
  --vcores 16 \
  --storage 512
```

Query Store is your primary performance diagnostic tool in Azure SQL and Managed Instance. Unlike on-premises where you might rely on Profiler, Extended Events, and third-party tools, the cloud-native approach centers on Query Store because it persists across connections and

failovers, captures execution plans with runtime statistics, and does not require elevated permissions to query.

```
-- Query Store: find top 10 regressed queries in the past 24 hours
-- Works identically on Azure SQL Database, Managed Instance, and SQL Server on VM

WITH RecentInterval AS (
    SELECT TOP 1 end_time AS cutoff
    FROM sys.query_store_runtime_stats_interval
    WHERE end_time >= DATEADD(HOUR, -24, GETUTCDATE())
    ORDER BY end_time ASC
)
SELECT TOP 10
    qsq.query_id,
    qsqt.query_sql_text,
    qsp.plan_id,
    qsrs.execution_type_desc,
    qsrs.count_executions,
    qsrs.avg_duration           / 1000.0 AS avg_duration_ms,
    qsrs.max_duration           / 1000.0 AS max_duration_ms,
    qsrs.avg_cpu_time           / 1000.0 AS avg_cpu_ms,
    qsrs.avg_logical_io_reads,
    qsrs.avg_rowcount,
    qsp.is_forced_plan,
    qsp.last_force_failure_reason_desc
FROM sys.query_store_query      qsq
JOIN sys.query_store_query_text qsqt ON qsqt.query_text_id = qsqt.query_text_id
JOIN sys.query_store_plan      qsp  ON qsq.query_id       = qsp.query_id
JOIN sys.query_store_runtime_stats qsrs ON qsp.plan_id     = qsrs.plan_id
JOIN sys.query_store_runtime_stats_interval qsrsi
    ON qsrs.runtime_stats_interval_id =
    qsrsi.runtime_stats_interval_id

CROSS JOIN RecentInterval ri
WHERE qsrsi.end_time >= ri.cutoff
    AND qsrs.execution_type = 0 -- Regular executions only
ORDER BY qsrs.avg_duration DESC;
GO

-- Force a specific plan when a regression is identified
-- This is the Azure-native equivalent of a plan guide
EXEC sys.sp_query_store_force_plan
    @query_id = 1234,
    @plan_id  = 5678;
GO

-- Verify forced plan status
SELECT
    qsq.query_id,
    qsp.plan_id,
    qsp.is_forced_plan,
    qsp.force_failure_count,
    qsp.last_force_failure_reason_desc,
    qsp.last_execution_time
FROM sys.query_store_query qsq
JOIN sys.query_store_plan qsp ON qsq.query_id = qsp.query_id
WHERE qsp.is_forced_plan = 1;
```

Resource pressure diagnostics — when your workload is hitting the vCore ceiling, the signal is `SOS_SCHEDULER_YIELD` waits combined with CPU utilization pinned at 100% of your allocated vCores. In Azure SQL Database, the `sys.dm_db_resource_stats` DMV provides a 15-second granularity view of resource consumption that on-premises DMVs do not offer.

```
-- Azure SQL Database only: resource consumption at 15-second granularity
-- Use this to identify when you are hitting CPU, I/O, or log rate limits

SELECT TOP 100
    end_time,
    avg_cpu_percent,
```

```
avg_data_io_percent,
avg_log_write_percent,
avg_memory_usage_percent,
xtp_storage_percent,
max_worker_percent,
max_session_percent
FROM sys.dm_db_resource_stats
ORDER BY end_time DESC;

-- Identify the top resource-consuming sessions right now
SELECT TOP 20
    s.session_id,
    s.login_name,
    s.host_name,
    s.program_name,
    r.status,
    r.wait_type,
    r.wait_time / 1000.0 AS wait_time_sec,
    r.cpu_time / 1000.0 AS cpu_time_sec,
    r.total_elapsed_time / 1000.0 AS elapsed_sec,
    r.logical_reads,
    r.writes,
    SUBSTRING(qt.text, (r.statement_start_offset / 2) + 1,
        ((CASE r.statement_end_offset
            WHEN -1 THEN DATALENGTH(qt.text)
            ELSE r.statement_end_offset
        END - r.statement_start_offset) / 2) + 1) AS current_statement
FROM sys.dm_exec_sessions s
JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) qt
WHERE s.is_user_process = 1
ORDER BY r.cpu_time DESC;
```

13.7 Migration Patterns: Moving to Each Deployment Model

Migration strategy depends on source and target. The three most common scenarios are on-premises SQL Server to Azure SQL Database, on-premises SQL Server to Managed Instance, and on-premises SQL Server to SQL Server on Azure VM.

Database Migration Service (DMS) is Microsoft's recommended path for online migrations with minimal downtime. It handles schema migration, data migration, and ongoing CDC-based replication from the source until you are ready to cut over. The important constraint: DMS requires the source SQL Server to have SQL Server Agent running, change tracking or CDC enabled, and adequate log retention to cover the migration window.

```
# Azure CLI - set up Database Migration Service for online migration to Managed Instance

# Step 1: Create the DMS instance (Managed Instance target requires Premium SKU)
az dms create \
    --resource-group rg-sql-production-eastus \
    --service-name dms-migration-prod \
    --location "East US" \
    --sku-name Premium_4vCores \
    --subnet $(az network vnet subnet show \
        --resource-group rg-sql-production-eastus \
        --vnet-name vnet-mi-production \
        --name snet-dms \
        --query id --output tsv)

# Step 2: Create a migration project
az dms project create \
    --resource-group rg-sql-production-eastus \
```

```

--service-name dms-migration-prod \
--name proj-onprem-to-mi \
--source-platform SQL \
--target-platform SQLMI \
--location "East US"

# Step 3: Create the migration task - connection details
# Source: on-premises SQL Server with SMB share for backup files
# Target: Managed Instance
az dms project task create \
  --resource-group rg-sql-production-eastus \
  --service-name dms-migration-prod \
  --project-name proj-onprem-to-mi \
  --task-type MigrateSqlServerSqlMISync \
  --name task-appdb-migration \
  --task-options '{
    "selectedDatabases": [{
      "name": "AppDatabase",
      "restoreDatabaseName": "AppDatabase",
      "backupFileShare": {
        "path": "\\onprem-sql-01\\SQLBackups\\AppDatabase",
        "userName": "svc-dms-backup",
        "password": "${BACKUP_SHARE_PASSWORD}"
      },
      "selectedTables": null
    }],
    "sourceConnectionInfo": {
      "dataSource": "onprem-sql-01",
      "authentication": "SqlAuthentication",
      "userName": "sa",
      "password": "${SOURCE_SA_PASSWORD}",
      "encryptConnection": true,
      "trustServerCertificate": true
    },
    "targetConnectionInfo": {
      "managedInstanceResourceId": "/subscriptions/${SUBSCRIPTION_ID}
↪ "'/resourceGroups/rg-sql-production-eastus/providers/Microsoft.Sql/managedInstances/mi-prod-eastus-001",
      "userName": "sqladmin",
      "password": "${MI_ADMIN_PASSWORD}"
    },
    "azureApp": {
      "applicationId": "${DMS_APP_ID}",
      "appKey": "${DMS_APP_KEY}",
      "tenantId": "${TENANT_ID}"
    },
    "storageResourceId": "/subscriptions/${SUBSCRIPTION_ID}
↪ "'/resourceGroups/rg-sql-production-eastus/providers/Microsoft.Storage/storageAccounts/stdmsmigrationprod"
  }'

# Monitor migration status
az dms project task show \
  --resource-group rg-sql-production-eastus \
  --service-name dms-migration-prod \
  --project-name proj-onprem-to-mi \
  --name task-appdb-migration \
  --expand output \
  --query "output[0].{Phase:migrationState, DB:databaseName, RestoredFiles:backupSets}" \
  --output table

```

For SQL Server on VM migrations, the backup/restore approach is the simplest and most reliable. Take a full backup on-premises, copy it to Azure Blob Storage, restore it on the VM. For larger databases, use backup striping and Azure Storage Transfer Service to accelerate the copy.

```

-- T-SQL: Backup directly to Azure Blob Storage from on-premises SQL Server
-- Requires SQL Server 2012 SP1 CU2 or later + credential for the storage account

-- Create credential for Azure Blob Storage access
CREATE CREDENTIAL [https://stdmsmigrationprod.blob.core.windows.net/sqlbackups]

```

```

WITH IDENTITY = 'SHARED ACCESS SIGNATURE',
SECRET = 'sv=2023-08-03&ss=b&srt=sco&sp=rwdlacupiytfx&se=2024-12-31T23:59:59Z&st=2024-01-01T00:00:00Z'
↪ &spr=https&sig=<your-sas-token>';
GO

-- Backup with striping to reduce transfer time
BACKUP DATABASE [AppDatabase]
TO URL = 'https://stdmsmigrationprod.blob.core.windows.net/sqlbackups/AppDB_001.bak',
URL = 'https://stdmsmigrationprod.blob.core.windows.net/sqlbackups/AppDB_002.bak',
URL = 'https://stdmsmigrationprod.blob.core.windows.net/sqlbackups/AppDB_003.bak',
URL = 'https://stdmsmigrationprod.blob.core.windows.net/sqlbackups/AppDB_004.bak'
WITH
    COMPRESSION,
    STATS = 5,
    CHECKSUM,
    INIT;
GO

-- On the Azure VM: restore from Blob Storage
-- First create the same credential on the target VM
RESTORE DATABASE [AppDatabase]
FROM URL = 'https://stdmsmigrationprod.blob.core.windows.net/sqlbackups/AppDB_001.bak',
URL = 'https://stdmsmigrationprod.blob.core.windows.net/sqlbackups/AppDB_002.bak',
URL = 'https://stdmsmigrationprod.blob.core.windows.net/sqlbackups/AppDB_003.bak',
URL = 'https://stdmsmigrationprod.blob.core.windows.net/sqlbackups/AppDB_004.bak'
WITH
    MOVE 'AppDatabase' TO 'D:\SQLData\AppDatabase.mdf',
    MOVE 'AppDatabase_log' TO 'L:\SQLLog\AppDatabase_ldf',
    STATS = 5,
    REPLACE;
GO

```

Compatibility assessment before migration — never migrate without running the Azure SQL Migration extension for Azure Data Studio (or the Azure Migrate appliance for large estates). It identifies feature gaps between your on-premises database and the target deployment model. Common blockers for Azure SQL Database include:

- Cross-database queries (requires elastic query or schema redesign)
- SQL Server Agent jobs (requires migration to Elastic Jobs)
- Linked servers (requires redesign or Managed Instance)
- xp_cmdshell, sp_OA* procedures (blocked in PaaS)
- Windows Authentication logins (requires Azure AD login migration)
- Database Mail (blocked in Azure SQL Database)

For Managed Instance, blockers are fewer but include: - Filestream/Filetable data (not supported) - Distributed transactions via MSDTC across instances (limited support) - Some CLR assemblies marked UNSAFE or EXTERNAL_ACCESS - xp_cmdshell (blocked)

```

# Azure Data Studio CLI: run compatibility assessment
# Install: az extension add --name datamigration

az datamigration get-assessment \
    --connection-string "Data Source=onprem-sql-01;Initial Catalog=AppDatabase;User
↪ Id=sa;Password=${SA_PASSWORD};TrustServerCertificate=True" \
    --target-platform AzureSqlManagedInstance \
    --output-folder ./assessment-results

# Review the JSON output for breaking changes
cat ./assessment-results/assessmentReport.json | \
    python3 -c "
import json, sys
report = json.load(sys.stdin)
for db in report.get('Databases', []):
    issues = db.get('AssessmentRecommendations', [])

```

```

breaking = [i for i in issues if i.get('IssueCategory') == 'Breaking']
if breaking:
    print(f"\nDatabase: {db['Name']} - {len(breaking)} breaking issues\n")
    for b in breaking:
        print(f"\n    [{b['RuleId']}] {b['Title']}\n")

```

13.8 Cost Optimization: Choosing the Right Tier and Right-Sizing

Azure SQL costs money in ways on-premises does not: you pay for compute, storage, backup storage, geo-redundancy, and licensing separately. Getting the sizing wrong — especially overprovisioning — is the most common cost problem in production Azure SQL deployments.

Reserved Instances are the single largest cost lever for predictable workloads. Committing to 1-year or 3-year reserved capacity typically yields substantial savings over pay-as-you-go pricing for the compute component. The tradeoff is that reserved capacity is committed — if you scale the instance up or down, the reservation may no longer match.

```

# Azure CLI - view current DTU/vCore consumption to right-size before reserving

# For Azure SQL Database: check average CPU over 14 days
az monitor metrics list \
  --resource $(az sql db show \
    --resource-group rg-sql-production-eastus \
    --server sql-prod-eastus-001 \
    --name AppDatabase \
    --query id --output tsv) \
  --metric "cpu_percent" \
  --interval PT1H \
  --start-time $(date -u -d '14 days ago' '+%Y-%m-%dT%H:%M:%SZ') \
  --end-time $(date -u '+%Y-%m-%dT%H:%M:%SZ') \
  --aggregation Average Maximum \
  --output table

# For Managed Instance: check vCore utilization
az monitor metrics list \
  --resource $(az sql mi show \
    --resource-group rg-sql-production-eastus \
    --name mi-prod-eastus-001 \
    --query id --output tsv) \
  --metric "avg_cpu_percent" \
  --interval PT1H \
  --aggregation Average Maximum \
  --start-time $(date -u -d '14 days ago' '+%Y-%m-%dT%H:%M:%SZ') \
  --end-time $(date -u '+%Y-%m-%dT%H:%M:%SZ') \
  --output table

```

Azure Hybrid Benefit allows you to apply existing on-premises SQL Server licenses with Software Assurance to Azure SQL deployments, reducing compute costs. If your organization has existing SA-covered SQL Server licenses, always evaluate whether **BasePrice** (license included) or **LicenseIncluded** pricing applies — the difference is significant for Managed Instance and SQL Server on VM.

```

# Apply Azure Hybrid Benefit to Managed Instance
az sql mi update \
  --resource-group rg-sql-production-eastus \
  --name mi-prod-eastus-001 \
  --license-type BasePrice # BasePrice = Hybrid Benefit; LicenseIncluded = no SA

# Apply to SQL Server on VM (registered with SQL IaaS Agent extension)
az sql vm update \

```

```
--resource-group rg-sql-production-eastus \  
--name vm-sqlserver-prod-001 \  
--license-type AHUB # AHUB = Azure Hybrid Use Benefit
```

Elastic Pools reduce cost when you have multiple databases with non-overlapping peak usage patterns. The economics work when individual databases have high peaks but low average utilization — the pool's shared vCores handle peaks from different databases at different times. The key metric to validate is the aggregate maximum concurrent vCore demand across all databases. If all databases peak simultaneously, an elastic pool provides no benefit over individually provisioned databases.

```
-- Query to identify databases suitable for elastic pool consolidation  
-- Run on Azure SQL Database logical server (requires master database connection)  
-- Identifies databases where max utilization is high but average is low
```

```
SELECT  
    d.name  
    rs.avg_cpu_percent_max  
    rs.avg_cpu_percent_mean  
    rs.avg_data_io_percent_max  
    rs.avg_log_write_percent_max  
    CASE  
        WHEN rs.avg_cpu_percent_max > 80  
            AND rs.avg_cpu_percent_mean < 20  
            THEN 'Good pool candidate'  
        WHEN rs.avg_cpu_percent_max > 80  
            AND rs.avg_cpu_percent_mean >= 20  
            THEN 'High sustained load - poor candidate'  
        ELSE 'Low utilization - good candidate'  
    END  
    AS pool_recommendation  
FROM sys.databases d  
CROSS APPLY (  
    SELECT  
        MAX(avg_cpu_percent) AS avg_cpu_percent_max,  
        AVG(avg_cpu_percent) AS avg_cpu_percent_mean,  
        MAX(avg_data_io_percent) AS avg_data_io_percent_max,  
        MAX(avg_log_write_percent) AS avg_log_write_percent_max  
    FROM sys.resource_stats  
    WHERE database_name = d.name  
        AND start_time >= DATEADD(DAY, -14, GETUTCDATE())  
    ) rs  
WHERE d.database_id > 4 -- exclude system databases  
ORDER BY rs.avg_cpu_percent_max DESC;
```

13.9 Production Gotchas: What the Documentation Does Not Tell You

This section distills the lessons that only emerge after running these services under real production load.

Gotcha 1: Managed Instance provisioning and scaling take hours, not minutes. When you provision a new Managed Instance or scale its vCore count, the operation can take between 4 and 6 hours for General Purpose, and up to 8 hours for Business Critical. This is not a bug — the service is reshaping underlying cluster resources. Plan MI scaling during maintenance windows. Do not attempt to scale MI two hours before a peak load event.

Gotcha 2: Azure SQL Database connection limits are lower than you expect. The maximum concurrent connections on General Purpose tier scale with vCores, but the ceiling is lower

than a comparable on-premises SQL Server. A 4-vCore GP database has a maximum worker thread count that will surprise teams migrating applications that use connection-per-thread architectures. The fix is connection pooling — always use a connection pool in front of Azure SQL Database. `max_worker_percent` in `sys.dm_db_resource_stats` is your early warning signal.

Gotcha 3: Maintenance windows do not guarantee zero-impact updates. Even with a maintenance window configured, Azure SQL Database and Managed Instance apply updates that cause brief connection resets. The duration is typically short — usually under 30 seconds for SQL Database and under 60 seconds for Managed Instance — but applications that do not implement retry logic on transient connection failures will surface errors to end users. Implement retry logic using exponential backoff before migration, not after.

```
# Python: retry logic for transient Azure SQL errors
# Transient error codes: 40613, 40197, 40501, 40549, 49918, 49919, 49920, 4221

import time
import pyodbc
from typing import Callable, TypeVar

T = TypeVar('T')

TRANSIENT_ERROR_CODES = {40613, 40197, 40501, 40549, 49918, 49919, 49920, 4221}

def execute_with_retry(
    operation: Callable[[], T],
    max_attempts: int = 5,
    base_delay_seconds: float = 1.0
) -> T:
    """
    Execute a database operation with exponential backoff retry.
    Handles transient Azure SQL errors (connection resets during maintenance,
    throttling, temporary unavailability).
    """
    for attempt in range(1, max_attempts + 1):
        try:
            return operation()
        except pyodbc.Error as e:
            error_code = e.args[0] if e.args else None

            # Parse SQL error code from the error message
            sql_error_code = None
            if len(e.args) > 1:
                try:
                    sql_error_code = int(str(e.args[1]).split()[0])
                except (ValueError, IndexError):
                    pass

            is_transient = (
                sql_error_code in TRANSIENT_ERROR_CODES
                or "connection" in str(e).lower()
                or "timeout" in str(e).lower()
            )

            if not is_transient or attempt == max_attempts:
                raise

            delay = base_delay_seconds * (2 ** (attempt - 1))
            # Add jitter to prevent thundering herd during mass reconnect
            jitter = delay * 0.1
            sleep_time = delay + (jitter * (2 * __import__('random').random() - 1))

            print(f"Transient error on attempt {attempt}/{max_attempts}. "
                  f"Retrying in {sleep_time:.1f}s. Error: {e}")
            time.sleep(sleep_time)

    # Should not reach here - max_attempts exhausted raises in loop
```

```
raise RuntimeError("Unreachable")
```

Gotcha 4: Backup storage costs accumulate silently. Azure SQL Database and Managed Instance take automated backups (full, differential, log) and store them in geo-redundant storage by default. You are billed for backup storage beyond the database size included in your tier. For a 1 TB database with 35-day retention, backup storage can exceed the database storage cost itself. Audit your backup retention policy. For non-production environments, reduce retention to 7 days and switch to locally redundant backup storage.

```
# Azure CLI - change backup redundancy to LRS for non-production (cost reduction)
az sql db update \
  --resource-group rg-sql-nonprod-eastus \
  --server sql-nonprod-eastus-001 \
  --name DevDatabase \
  --backup-storage-redundancy Local \
  --short-term-retention 7

# Check current backup storage consumption for a server
az sql db list-deleted \
  --resource-group rg-sql-production-eastus \
  --server sql-prod-eastus-001 \
  --output table

# For Managed Instance: set backup storage redundancy at creation time (cannot be changed after
↪ provisioning)
az sql mi create \
  --resource-group rg-sql-production-eastus \
  --name mi-prod-eastus-002 \
  --backup-storage-redundancy Geo \
  # ... other parameters
```

Gotcha 5: Geo-replication lag is invisible until a failover. Auto-failover groups use asynchronous replication for geo-redundancy. The secondary replica lags the primary — sometimes by seconds, sometimes by minutes under heavy write load. Most teams never check the replication lag until a failover reveals that the secondary is hours behind. Monitor replication lag continuously using the DMV `sys.dm_geo_replication_link_status` on Azure SQL Database or the `sys.dm_hadr_database_replica_states` on Managed Instance.

```
-- Azure SQL Database: check geo-replication lag
-- Run on the PRIMARY database

SELECT
  link_guid,
  partner_server,
  partner_database,
  replication_state_desc,
  role_desc,
  replication_lag_sec,
  last_replication,
  start_time
FROM sys.dm_geo_replication_link_status;

-- Alert threshold: if replication_lag_sec exceeds your RPO, investigate immediately
-- A sustained lag under normal write load indicates the secondary is undersized
-- or the network path between regions has degraded throughput

-- Managed Instance: equivalent check via AG DMVs
SELECT
  ag.name                                AS availability_group,
  ar.replica_server_name,
  drs.database_id,
  DB_NAME(drs.database_id)              AS database_name,
  drs.synchronization_state_desc,
  drs.last_commit_time,
  drs.log_send_queue_size                AS log_send_queue_kb,
```

```

    drs.redo_queue_size                AS redo_queue_kb,
    DATEDIFF(SECOND, drs.last_redone_time, GETUTCDATE()) AS seconds_behind_primary
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
JOIN sys.availability_groups ag
    ON ar.group_id = ag.group_id
WHERE drs.is_local = 0 -- remote replicas only
ORDER BY seconds_behind_primary DESC;

```

Gotcha 6: SQL Server on VM requires the IaaS Agent extension to be registered — and most teams skip this. Azure provides the SQL IaaS Agent extension for SQL Server on VMs. Without it, you lose automated backup integration with Azure Recovery Services Vault, automated patching scheduling, Key Vault integration for TDE, and the ability to apply Azure Hybrid Benefit through the portal. Register every SQL Server VM with this extension immediately after provisioning.

```

# Register SQL Server VM with IaaS Agent extension
az sql vm create \
  --resource-group rg-sql-production-eastus \
  --name vm-sqlserver-prod-001 \
  --location "East US" \
  --license-type PAYG \
  --sql-mgmt-type Full # Full management mode required for automated backup/patching

# If the VM already exists without the extension, register it:
az sql vm update \
  --resource-group rg-sql-production-eastus \
  --name vm-sqlserver-prod-001 \
  --sql-mgmt-type Full

# Configure automated backup through the extension
az sql vm update \
  --resource-group rg-sql-production-eastus \
  --name vm-sqlserver-prod-001 \
  --auto-backup-settings '{
    "enable": true,
    "retentionPeriod": 30,
    "storageAccountUrl": "https://stbackupprod.blob.core.windows.net/",
    "storageAccessToken": "'${STORAGE_SAS_TOKEN}'",
    "enableEncryption": true,
    "backupSystemDbs": true,
    "backupScheduleType": "Automated"
  }'

```

Gotcha 7: Tempdb configuration on Managed Instance is not user-controllable. On-premises and SQL Server on VM, you add multiple tempdb data files (one per logical CPU, up to 8) to reduce allocation page contention. On Managed Instance, tempdb is pre-configured by the service and you cannot add or resize tempdb files directly. This matters for workloads with heavy tempdb usage (sorts, spills, temp table creation at high concurrency). If your workload generates significant tempdb contention on Managed Instance, the primary lever is vCore scaling — more vCores means more pre-configured tempdb files.

```

-- Check tempdb file configuration on Managed Instance
-- You can observe but not alter the file layout

SELECT
    file_id,
    name,
    physical_name,
    type_desc,
    size * 8 / 1024 AS size_mb,
    max_size * 8 / 1024 AS max_size_mb,
    growth AS growth_pages,
    is_percent_growth

```

```
FROM tempdb.sys.database_files
ORDER BY file_id;

-- Diagnose tempdb allocation contention
SELECT
    wait_type,
    waiting_tasks_count,
    wait_time_ms,
    max_wait_time_ms,
    signal_wait_time_ms
FROM sys.dm_os_wait_stats
WHERE wait_type IN (
    'PAGELATCH_UP', 'PAGELATCH_EX', 'PAGELATCH_SH',
    'PAGEIOLATCH_UP', 'PAGEIOLATCH_EX'
)
ORDER BY wait_time_ms DESC;

-- Identify specific pages under contention (tempdb allocation pages are 1:1, 1:3)
SELECT
    session_id,
    wait_type,
    wait_duration_ms,
    resource_description -- Look for 2:1:1, 2:1:3 (tempdb GAM/SGAM pages)
FROM sys.dm_os_waiting_tasks
WHERE wait_type LIKE 'PAGELATCH%'
ORDER BY wait_duration_ms DESC;
```

13.10 Choosing Your Model: A Decision Framework

After reading through the technical details, the selection decision reduces to a structured evaluation across four dimensions: feature requirements, operational model preference, migration complexity, and cost profile.

Feature requirements drive the first cut. Ask these questions in order:

1. Does your workload require OS-level access (third-party agents, custom startup procedures, non-SQL Server processes on the same host)? → **SQL Server on VM**. This is the only answer. No other model provides OS access.
2. Does your workload require SQL Server Agent with complex multi-step jobs, cross-database queries, linked servers, Service Broker, or Database Mail? → **Managed Instance**. Azure SQL Database cannot satisfy these requirements without significant application rework.
3. Is your workload a greenfield application or a modernized application that does not depend on instance-level features? → **Azure SQL Database**. It provides the most managed experience, the lowest operational overhead, and the most direct path to cloud-native patterns like elastic scaling and serverless compute.

Operational model preference is the second cut. If your organization has a small DBA team that cannot sustain OS-level patching and HA configuration responsibilities, SQL Server on VM is a trap regardless of the feature match. Conversely, if your team includes DBAs with strong Windows Server and Always On skills, SQL Server on VM gives you the most control and the easiest path to validating the environment behaves exactly as expected.

Migration complexity determines timeline. A same-generation SQL Server database with no blocked features can be migrated to Managed Instance in a weekend using DMS online migration. The same database migrated to Azure SQL Database may require weeks of application rework to

eliminate cross-database queries, replace SQL Agent jobs, and refactor linked server dependencies. Be honest about the backlog this creates.

Cost profile determines long-term sustainability. Managed Instance is significantly more expensive than Azure SQL Database for equivalent compute because it runs a full SQL Server instance rather than individual databases. A single 8-vCore General Purpose Managed Instance costs more than multiple small Azure SQL Databases serving the same workload. For SaaS applications with many small tenant databases, Azure SQL Database with elastic pools is almost always the correct cost model. For enterprise line-of-business applications with 10–20 databases sharing an instance, Managed Instance matches the on-premises cost model more closely.

The following decision tree captures the practical logic:

One final consideration that experienced DBAs overlook: Managed Instance is not immune to the limitations of a managed service. You cannot run SQL Server Profiler against it. You cannot install extended events sessions that write to files on the OS. You cannot use third-party backup software. If your compliance team requires a third-party backup verification tool or a host-based encryption agent, only SQL Server on VM satisfies that requirement. Validate your full list of operational tooling against the managed service’s capability surface before committing to a deployment model.

Key Takeaways

- **Deployment model selection is irreversible in the short term.** Migrating from Azure SQL Database to Managed Instance, or from Managed Instance to SQL Server on VM, requires database migrations and application changes. Invest the time to assess feature dependencies accurately before choosing a model.
 - **Managed Instance requires dedicated subnet planning that cannot be corrected after deployment.** Size the subnet for multiple instances and upgrade headroom from day one; a /24 is preferable to a /27 for production environments expecting growth.
 - **Always On availability and geo-redundancy behave differently across tiers.** Business Critical tier provides near-instantaneous same-region failover with a local SSD replica; General Purpose tier has a longer recovery time due to remote storage reattachment. Match your RTO requirement to the correct tier before signing off on architecture.
 - **Retry logic for transient connection errors is mandatory, not optional, for any application connecting to Azure SQL Database or Managed Instance.** Maintenance windows and automated failovers produce brief connection resets that will surface as application errors without it.
 - **Cost optimization requires active management.** Reserved Instances, Azure Hybrid Benefit, elastic pool consolidation, and backup storage redundancy settings each independently reduce spend. None are configured automatically — they require deliberate action at provisioning time and ongoing review as workload patterns evolve.
-

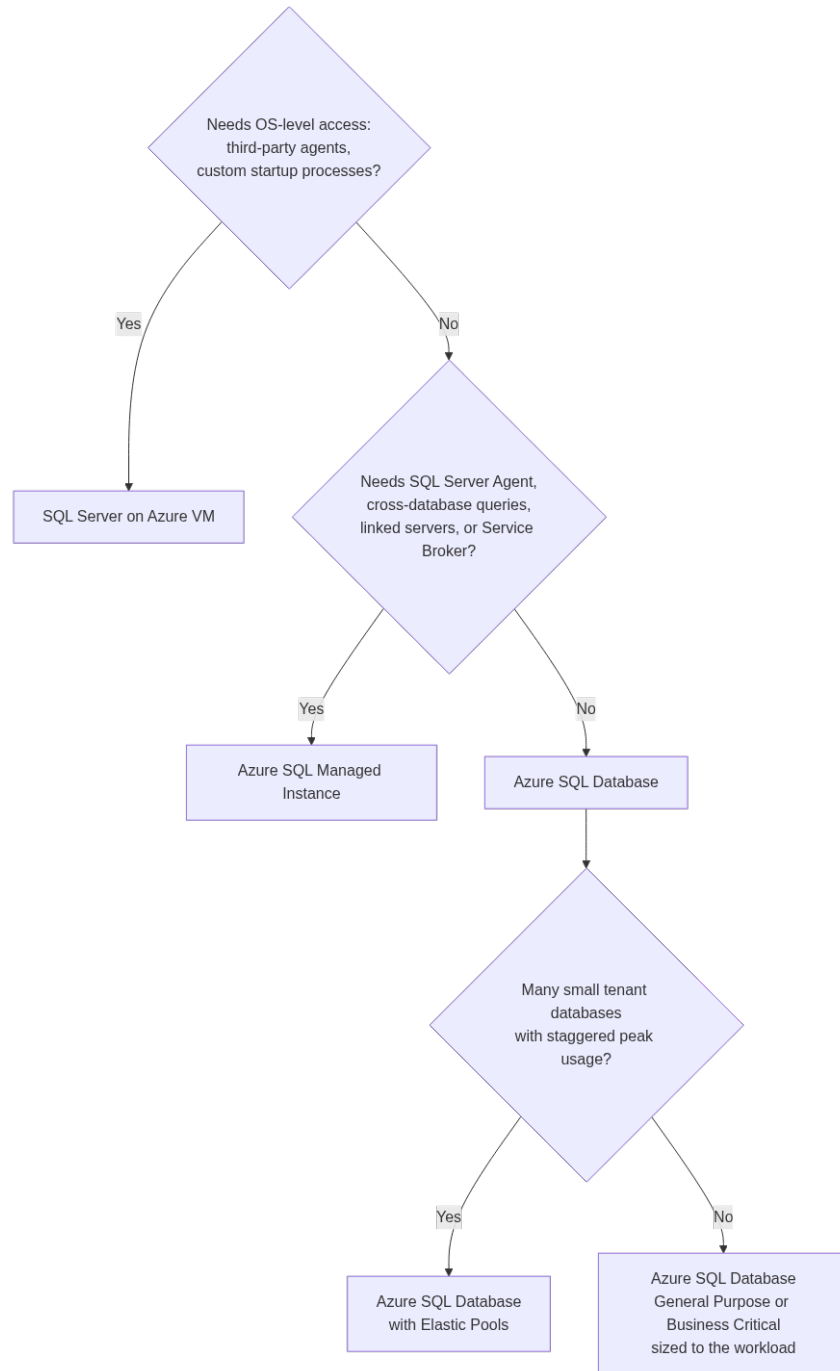


Figure 4: Feature requirements make the first cut; cost and consolidation patterns refine the choice within Azure SQL Database. Operational capacity and migration complexity, covered above, are the tie-breakers when more than one option technically fits.

Chapter 14: Elastic Pools: Multi-Tenant Cost Optimization

Running dozens of databases for separate tenants on dedicated instances is the fastest way to inflate a cloud bill. A SaaS platform with 200 customers, each assigned their own `db.t3.medium` RDS instance, burns through hundreds of instance-hours per hour—most of them idle at 3 AM. The economics that made dedicated instances sensible on-premises (you already paid for the hardware) break down completely when you're billed by the second in the cloud. This chapter covers the architectural patterns and platform-specific tools that let you consolidate multi-tenant workloads without sacrificing isolation: Azure SQL Elastic Pools, Aurora Serverless v2 shared clusters, RDS Proxy connection pooling, and equivalent consolidation strategies on Google Cloud SQL and AlloyDB. You'll learn how to right-size shared resource pools, enforce per-tenant resource governance, and build the monitoring scaffolding that tells you when a tenant is crowding out its neighbors.

14.1 The Multi-Tenant Database Problem: Why Dedicated Instances Fail at Scale

On-premises, the conversation around multi-tenancy usually came down to shared schemas, separate schemas within one instance, or fully separate databases on the same host. You had a server with 256 GB of RAM and 32 cores. You ran 40 databases on it. The marginal cost of database 41 was almost nothing—some buffer pool pressure, a few more background threads.

Cloud changes the unit economics entirely. Every instance is a billable entity, independently metered. A `db.t3.medium` on RDS runs roughly \$0.068/hour on-demand in us-east-1. Multiply that by 200 tenants and you're at \$13.60/hour just for compute—\$9,792/month before storage, I/O, or data transfer. If average CPU utilization across those instances is 5%, you're paying for 95% of compute you don't use.

The fundamental problem is variance in tenant load. Figure 14.1 (conceptually) shows a classic multi-tenant utilization curve: a few power users driving high load during business hours, a long tail of small customers almost always idle. The peak of one tenant rarely aligns with the peak of another. If you could stack those utilization profiles, the aggregate load fits comfortably in far fewer resources.

Three consolidation models exist in practice:

1. **Shared instance, separate databases** — All tenants share one RDS or Cloud SQL instance.

Cheapest, but noisy-neighbor risk is highest and blast radius of an instance failure covers all tenants simultaneously.

2. **Elastic pool / shared cluster** — Tenants have isolated database objects but draw from a shared resource pool with per-tenant caps. This is the Azure SQL Elastic Pool model and approximately what Aurora Serverless v2 achieves for a cluster.
3. **Connection multiplexing over dedicated instances** — Tenants keep dedicated instances but a proxy layer (RDS Proxy, PgBouncer) eliminates per-connection overhead so you can pack more application threads against fewer, larger instances.

The right model depends on tenant count, isolation requirements, and SLA commitments. Enterprise SaaS with contractual data isolation guarantees often must use model 3. High-volume SMB SaaS with hundreds or thousands of small tenants typically needs model 2. The rest of this chapter walks through concrete implementations of each.

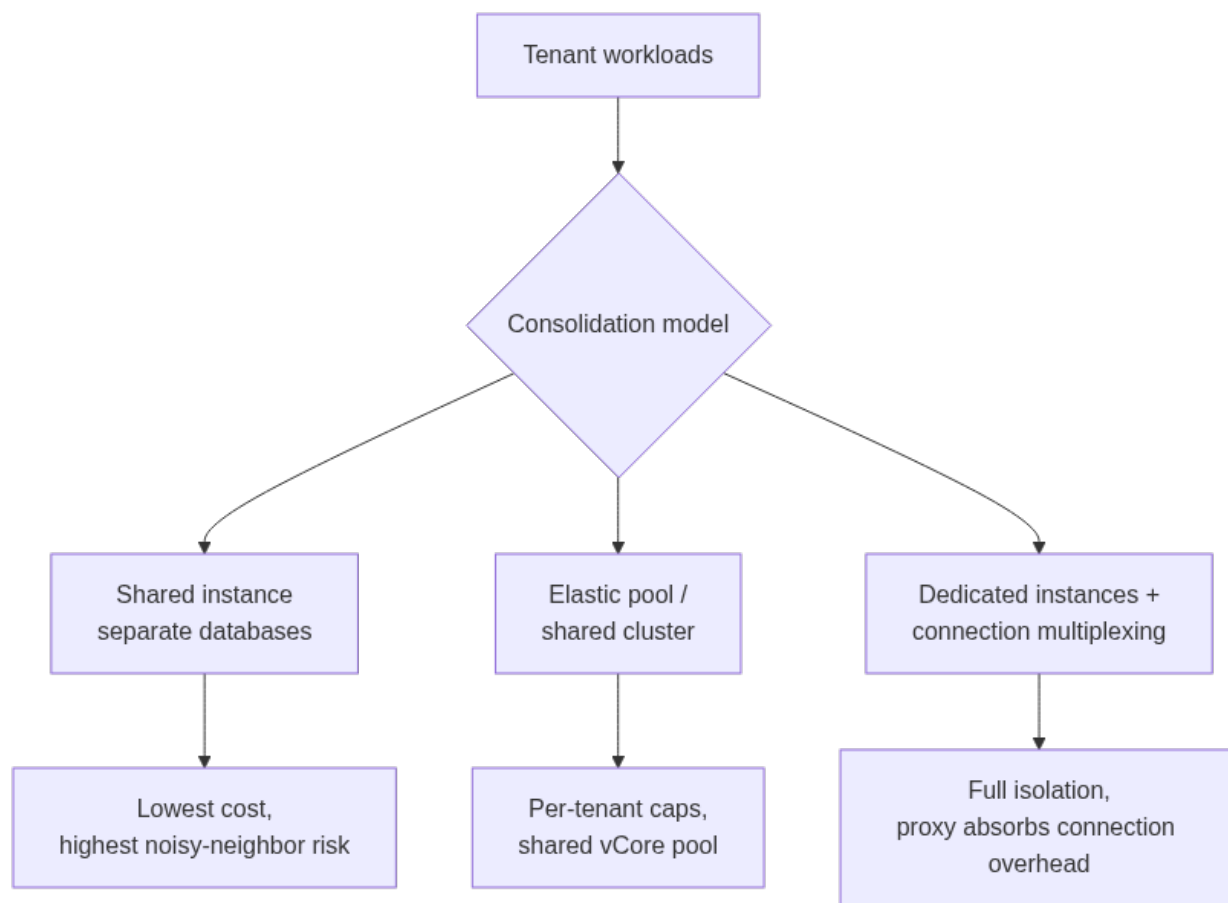


Figure 5: The three multi-tenant consolidation models, ordered roughly by increasing isolation and decreasing shared-resource risk.

14.2 Azure SQL Elastic Pools: The Reference Implementation

Azure SQL Elastic Pools are the most mature and complete implementation of the shared resource pool pattern in managed cloud databases. If you've worked with SQL Server, the mental model is a single SQL Server instance running multiple databases—except Azure enforces resource limits at the pool and per-database level using DTUs or vCores, and bills you for the pool rather than per database.

Pool configuration with Azure CLI:

```
# Create a resource group and logical server first
az group create \
  --name rg-saas-prod \
  --location eastus2

az sql server create \
  --name sql-saas-prod-eastus2 \
  --resource-group rg-saas-prod \
  --location eastus2 \
  --admin-user sqladmin \
  --admin-password "$(openssl rand -base64 24)"

# Create a General Purpose elastic pool - 8 vCores total shared across tenants
# min-capacity per database: 0 vCores (can scale to zero when idle)
# max-capacity per database: 4 vCores (no single tenant can consume more than 50%)
az sql elastic-pool create \
  --name ep-saas-tenants \
  --resource-group rg-saas-prod \
  --server sql-saas-prod-eastus2 \
  --edition GeneralPurpose \
  --family Gen5 \
  --capacity 8 \
  --db-min-capacity 0 \
  --db-max-capacity 4 \
  --storage 512000 \
  --zone-redundant true

# Add an existing tenant database to the pool
az sql db create \
  --name tenant-acme-corp \
  --resource-group rg-saas-prod \
  --server sql-saas-prod-eastus2 \
  --elastic-pool ep-saas-tenants \
  --max-size 50GB
```

The `--db-min-capacity 0` setting is important: databases configured this way pause their vCore reservation when connection count drops to zero, letting the pool's capacity shift entirely to active tenants. The `--db-max-capacity 4` cap is your noisy-neighbor protection. Without it, a runaway tenant query can consume all 8 vCores and starve everyone else.

Terraform equivalent for reproducible infrastructure:

```
resource "azurerm_mssql_elasticpool" "saas_pool" {
  name                        = "ep-saas-tenants"
  resource_group_name        = azurerm_resource_group.saas.name
  location                   = azurerm_resource_group.saas.location
  server_name                = azurerm_mssql_server.saas_prod.name
  max_size_gb                = 500

  sku {
    name       = "GP_Gen5"
    tier        = "GeneralPurpose"
    family     = "Gen5"
    capacity   = 8
  }
}
```

```

per_database_settings {
  min_capacity = 0
  max_capacity = 4
}

zone_redundant = true

tags = {
  Environment = "production"
  CostCenter  = "platform-engineering"
}
}

resource "azurerm_mssql_database" "tenant_db" {
  for_each      = toset(var.tenant_names)
  name          = "tenant-${each.key}"
  server_id     = azurerm_mssql_server.saas_prod.id
  elastic_pool_id = azurerm_mssql_elasticpool.saas_pool.id
  max_size_gb   = 50

  # Collation must match your application requirements
  # Changing this after creation requires a rebuild
  collation = "SQL_Latin1_General_CP1_CI_AS"
}

```

Right-sizing the pool: The most common mistake is sizing the pool based on the sum of per-database maximums rather than realistic concurrent peak load. If you have 50 tenant databases each with a `max_capacity` of 4 vCores, you might think you need 200 vCores. In practice, if peak concurrent load is never more than 10 databases simultaneously, an 8–16 vCore pool handles it. Azure publishes a workload simulator in their documentation, but the more reliable method is to run your tenants on dedicated instances for 2–4 weeks, capture the `sys.dm_db_resource_stats` DMV data, and use that to model peak concurrency.

```

-- Run this against each tenant database to capture utilization baseline
-- before migrating to a pool
SELECT
  start_time,
  end_time,
  avg_cpu_percent,
  avg_data_io_percent,
  avg_log_write_percent,
  avg_memory_usage_percent,
  max_worker_percent,
  max_session_percent
FROM sys.dm_db_resource_stats
ORDER BY start_time DESC;

-- Aggregate to find realistic peak across your tenant population
-- Run across all databases using a linked server or centralized monitoring schema
SELECT
  DB_NAME() AS tenant_name,
  MAX(avg_cpu_percent) AS peak_cpu_pct,
  AVG(avg_cpu_percent) AS avg_cpu_pct,
  PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY avg_cpu_percent)
    OVER () AS p95_cpu_pct,
  MAX(avg_data_io_percent) AS peak_io_pct
FROM sys.dm_db_resource_stats
WHERE start_time >= DATEADD(DAY, -14, GETUTCDATE());

```

Pool monitoring with Azure Monitor: Once in production, you need alerts on pool-level saturation—not just per-database. If 8 vCores are pegged at 100% for sustained periods, the pool is undersized regardless of how good individual tenant metrics look.

```

# Alert when pool CPU exceeds 85% for 5 consecutive minutes
az monitor metrics alert create \
  --name "ElasticPool-CPU-Saturation" \

```

```

--resource-group rg-saas-prod \
--scopes "/subscriptions/${SUBSCRIPTION_ID}"
↪ /resourceGroups/rg-saas-prod/providers/Microsoft.Sql/servers/sql-saas-prod-eastus2/elasticPools/ep-saas-tenants"
↪ \
--condition "avg Percentage CPU > 85" \
--window-size 5m \
--evaluation-frequency 1m \
--action-group ag-oncall-dba \
--description "Elastic pool approaching CPU saturation - evaluate adding vCores or shedding tenants"

# Alert on storage utilization
az monitor metrics alert create \
--name "ElasticPool-Storage-High" \
--resource-group rg-saas-prod \
--scopes "/subscriptions/${SUBSCRIPTION_ID}"
↪ /resourceGroups/rg-saas-prod/providers/Microsoft.Sql/servers/sql-saas-prod-eastus2/elasticPools/ep-saas-tenants"
↪ \
--condition "avg allocated_data_storage > 460000" \
--window-size 15m \
--evaluation-frequency 5m \
--action-group ag-oncall-dba \
--description "Pool storage above 90% of 500 GB allocation"

```

What Elastic Pools cannot do: Despite being the reference implementation, Elastic Pools have hard limits worth knowing before you commit. As of 2024, a single General Purpose pool supports a maximum of 500 databases. Business Critical tier caps at 100 databases. Hyperscale databases cannot participate in Elastic Pools at all—each Hyperscale database must have dedicated compute. Cross-region pools don’t exist; all databases in a pool must reside on the same logical server in the same region. If your architecture requires geo-distributed tenant databases, you’re back to per-database geo-replication, which is manageable but expensive.

14.3 Aurora Serverless v2: Fractional Scaling for AWS Multi-Tenancy

AWS doesn’t have a direct equivalent to Elastic Pools. What it has is Aurora Serverless v2, which achieves similar economics through a different mechanism: ACUs (Aurora Capacity Units) that scale in 0.5 ACU increments per database instance, down to a configured minimum. An Aurora cluster can mix Serverless v2 and provisioned instances, and multiple Serverless v2 instances in a cluster share the same underlying storage layer.

The key insight for multi-tenant architectures: you don’t get a shared resource pool across multiple Aurora clusters the way Azure lets multiple databases share one pool. Instead, the consolidation happens by running all tenants within a single Aurora cluster—either as separate databases within one instance or as separate schemas—and letting Serverless v2 scale the compute capacity down to near-zero when tenants are idle.

Creating a multi-tenant Aurora Serverless v2 cluster:

```

# Create parameter group customized for multi-tenant workloads
aws rds create-db-cluster-parameter-group \
--db-cluster-parameter-group-name aurora-pg-multitenant \
--db-parameter-group-family aurora-postgresql15 \
--description "Multi-tenant Aurora cluster parameters"

# Set connection and resource parameters
aws rds modify-db-cluster-parameter-group \
--db-cluster-parameter-group-name aurora-pg-multitenant \

```

```

--parameters \
"ParameterName=max_connections,ParameterValue=5000,ApplyMethod=pending-reboot" \
"ParameterName=shared_buffers,ParameterValue={DBInstanceClassMemory/4},ApplyMethod=pending-reboot" \
"ParameterName=work_mem,ParameterValue=4096,ApplyMethod=immediate" \
"ParameterName=idle_in_transaction_session_timeout,ParameterValue=30000,ApplyMethod=immediate" \
"ParameterName=statement_timeout,ParameterValue=300000,ApplyMethod=immediate"

# Create the Aurora Serverless v2 cluster
aws rds create-db-cluster \
--db-cluster-identifier aurora-saas-prod \
--engine aurora-postgresql \
--engine-version 15.4 \
--db-cluster-parameter-group-name aurora-pg-multitenant \
--master-username clusteradmin \
--master-user-password "$(aws secretsmanager get-random-password \
--password-length 32 \
--require-each-included-type \
--query RandomPassword \
--output text)" \
--vpc-security-group-ids sg-0abc123def456789 \
--db-subnet-group-name aurora-private-subnet-group \
--backup-retention-period 14 \
--preferred-backup-window "03:00-04:00" \
--preferred-maintenance-window "sun:05:00-sun:06:00" \
--storage-encrypted \
--kms-key-id arn:aws:kms:us-east-1:123456789012:key/mrk-abc123 \
--enable-cloudwatch-logs-exports postgresql \
--serverless-v2-scaling-configuration MinCapacity=0.5,MaxCapacity=16

# Create the writer instance as Serverless v2
aws rds create-db-instance \
--db-instance-identifier aurora-saas-prod-writer \
--db-cluster-identifier aurora-saas-prod \
--db-instance-class db.serverless \
--engine aurora-postgresql

# Create a reader instance - also Serverless v2, for read scaling
aws rds create-db-instance \
--db-instance-identifier aurora-saas-prod-reader-1 \
--db-cluster-identifier aurora-saas-prod \
--db-instance-class db.serverless \
--engine aurora-postgresql \
--promotion-tier 15

```

The `MinCapacity=0.5` is the economic lever. At 0.5 ACUs, each instance consumes approximately 1 GB of RAM and minimal CPU. If your tenants have well-separated business hours (e.g., a US-only SaaS product with all customers on Eastern time), overnight minimum capacity can be 0.5 ACUs for the writer while your total bill approaches zero. At 16 ACUs maximum, you have approximately 32 GB RAM and the equivalent of a db.r5.2xlarge available for peak load.

Gotcha: Serverless v2 does not scale to zero. The minimum is 0.5 ACUs. True scale-to-zero (Aurora Serverless v1 behavior) is deprecated for new workloads and came with cold start latencies of up to 30 seconds—unacceptable for production SaaS. Accept the 0.5 ACU floor as the cost of always-warm capacity.

Per-tenant resource governance in PostgreSQL: Unlike Azure SQL Elastic Pools, Aurora with PostgreSQL has no built-in per-database CPU or I/O caps. You enforce this at the application layer using connection pooling and at the database layer using PostgreSQL resource groups (`pg_query_settings` in Aurora) and per-role connection limits.

```

-- Create an isolated schema and role for each tenant
-- This runs once per tenant onboarding

-- Create tenant role with connection limit
CREATE ROLE tenant_acme_corp

```

```

WITH LOGIN
CONNECTION LIMIT 50
PASSWORD 'generated-at-runtime';

-- Create schema owned by tenant role
CREATE SCHEMA tenant_acme_corp AUTHORIZATION tenant_acme_corp;

-- Set default search path so tenant queries stay in their schema
ALTER ROLE tenant_acme_corp SET search_path = tenant_acme_corp, public;

-- Set per-role resource controls
ALTER ROLE tenant_acme_corp SET statement_timeout = '120s';
ALTER ROLE tenant_acme_corp SET idle_in_transaction_session_timeout = '30s';
ALTER ROLE tenant_acme_corp SET work_mem = '8MB';

-- Grant access to shared lookup tables in public schema
GRANT USAGE ON SCHEMA public TO tenant_acme_corp;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO tenant_acme_corp;

-- Verify connection limits across tenants
SELECT
    rolname,
    rolconlimit,
    (SELECT count(*) FROM pg_stat_activity WHERE username = r.rolname) AS active_connections
FROM pg_roles r
WHERE rolname LIKE 'tenant_%'
ORDER BY active_connections DESC;

```

14.4 RDS Proxy: Connection Pooling as Cost Optimization Infrastructure

If you're running provisioned RDS or Aurora instances—not Serverless v2—and your multi-tenant application creates many short-lived database connections (typical for Lambda-based or containerized SaaS backends), you're burning compute on connection overhead. Each PostgreSQL backend process consumes roughly 5–10 MB of memory. With 500 tenants each maintaining 10 application connections through NAT, your instance needs 25–50 GB just to hold idle connection state. That drives you to larger, more expensive instance classes.

RDS Proxy sits between your application tier and RDS/Aurora. It multiplexes thousands of application connections onto a much smaller set of database backend connections using transaction-level pinning. The application thinks it has 10,000 open connections. The database sees 200. This lets you run smaller instance classes while supporting larger application-layer concurrency—a direct cost reduction.

```

# Create RDS Proxy for a multi-tenant Aurora PostgreSQL cluster
# Proxy IAM role must have secretsmanager:GetSecretValue permission
aws iam create-role \
  --role-name rds-proxy-saas-role \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Principal": {"Service": "rds.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }]
  }'

aws iam put-role-policy \
  --role-name rds-proxy-saas-role \
  --policy-name rds-proxy-secrets-access \

```

```

--policy-document '{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["secretsmanager:GetSecretValue"],
    "Resource": "arn:aws:secretsmanager:us-east-1:123456789012:secret:rds/aurora-saas-prod/*"
  },{
    "Effect": "Allow",
    "Action": ["kms:Decrypt"],
    "Resource": "arn:aws:kms:us-east-1:123456789012:key/mrk-abc123",
    "Condition": {
      "StringEquals": {"kms:ViaService": "secretsmanager.us-east-1.amazonaws.com"}
    }
  }]
}'

# Create the proxy
aws rds create-db-proxy \
  --db-proxy-name rds-proxy-saas-prod \
  --engine-family POSTGRESQL \
  --auth '[
    {
      "Description": "Aurora cluster admin",
      "AuthScheme": "SECRETS",
      "SecretArn": "arn:aws:secretsmanager:us-east-1:123456789012:secret:rds/aurora-saas-prod/admin",
      "IAMAuth": "REQUIRED"
    }
  ]' \
  --role-arn arn:aws:iam::123456789012:role/rds-proxy-saas-role \
  --vpc-subnet-ids subnet-0aaa111 subnet-0bbb222 subnet-0ccc333 \
  --vpc-security-group-ids sg-0proxy123 \
  --require-tls \
  --idle-client-timeout 1800 \
  --connection-borrow-timeout 120 \
  --max-connections-percent 90 \
  --debug-logging false

# Register the Aurora cluster as a proxy target
aws rds register-db-proxy-targets \
  --db-proxy-name rds-proxy-saas-prod \
  --db-cluster-identifiers aurora-saas-prod

```

RDS Proxy limitations that catch people off guard:

- Proxy endpoints are regional. You cannot route through a proxy to a cross-region reader.
- Each proxy supports one target group (one cluster or one RDS instance). For multiple Aurora clusters, you need multiple proxies—and you pay per proxy plus per connection-hour.
- RDS Proxy does not support all PostgreSQL features. Specifically: SET LOCAL statements cause session pinning (the proxy pins the full application connection to one backend for the duration of the transaction, defeating multiplexing). If your application sets custom session variables per request—common in row-level security multi-tenant patterns—profile pinning rates carefully before declaring cost savings achieved.
- Pinning check: monitor the `DatabaseConnectionsCurrentlySessionPinned` CloudWatch metric. If pinning rate is above 40–50% of your connections, the proxy overhead may exceed its benefit for your workload.

```

import boto3
import pandas as pd
from datetime import datetime, timedelta

def analyze_proxy_pinning(proxy_name: str, hours: int = 24) -> dict:
    """
    Analyze RDS Proxy connection pinning to determine
    whether the proxy is actually saving database connections.
    """

```

```

cw = boto3.client('cloudwatch', region_name='us-east-1')
end_time = datetime.utcnow()
start_time = end_time - timedelta(hours=hours)

metrics_to_fetch = [
    'DatabaseConnections',
    'DatabaseConnectionsCurrentlySessionPinned',
    'ClientConnections',
    'QueryRequests',
    'MaxDatabaseConnectionsAllowed',
]

results = {}
for metric_name in metrics_to_fetch:
    response = cw.get_metric_statistics(
        Namespace='AWS/RDS',
        MetricName=metric_name,
        Dimensions=[
            {'Name': 'ProxyName', 'Value': proxy_name}
        ],
        StartTime=start_time,
        EndTime=end_time,
        Period=300, # 5-minute intervals
        Statistics=['Average', 'Maximum']
    )
    datapoints = sorted(response['Datapoints'], key=lambda x: x['Timestamp'])
    results[metric_name] = datapoints

# Calculate multiplexing efficiency
if results['DatabaseConnections'] and results['ClientConnections']:
    avg_db_conns = sum(d['Average'] for d in results['DatabaseConnections']) /
    len(results['DatabaseConnections'])
    avg_client_conns = sum(d['Average'] for d in results['ClientConnections']) /
    len(results['ClientConnections'])
    multiplexing_ratio = avg_client_conns / avg_db_conns if avg_db_conns > 0 else 0
else:
    multiplexing_ratio = 0

# Calculate pinning percentage
pinned_data = results.get('DatabaseConnectionsCurrentlySessionPinned', [])
db_conn_data = results.get('DatabaseConnections', [])

if pinned_data and db_conn_data:
    # Align by timestamp and calculate ratio
    pinned_by_time = {d['Timestamp']: d['Average'] for d in pinned_data}
    db_by_time = {d['Timestamp']: d['Average'] for d in db_conn_data}

    pinning_ratios = []
    for ts in pinned_by_time:
        if ts in db_by_time and db_by_time[ts] > 0:
            pinning_ratios.append(pinned_by_time[ts] / db_by_time[ts])

    avg_pinning_rate = sum(pinning_ratios) / len(pinning_ratios) if pinning_ratios else 0
else:
    avg_pinning_rate = 0

return {
    'proxy_name': proxy_name,
    'period_hours': hours,
    'avg_client_connections': round(avg_client_conns, 1) if 'avg_client_conns' in dir() else 0,
    'avg_db_connections': round(avg_db_conns, 1) if 'avg_db_conns' in dir() else 0,
    'multiplexing_ratio': round(multiplexing_ratio, 2),
    'avg_pinning_rate_pct': round(avg_pinning_rate * 100, 1),
    'recommendation': (
        'Proxy is effective - good multiplexing ratio'
        if multiplexing_ratio > 3 and avg_pinning_rate < 0.4
        else 'Evaluate pinning sources - proxy benefit may be limited'
    )
}

```

```
# Example output structure:
# {
#   'proxy_name': 'rds-proxy-saas-prod',
#   'period_hours': 24,
#   'avg_client_connections': 847.3,
#   'avg_db_connections': 142.6,
#   'multiplexing_ratio': 5.94,
#   'avg_pinning_rate_pct': 12.4,
#   'recommendation': 'Proxy is effective - good multiplexing ratio'
# }
```

RDS Proxy with per-tenant IAM authentication: For SaaS platforms with strict tenant isolation requirements, you can use IAM database authentication through RDS Proxy so each tenant’s application pod authenticates with a distinct IAM role, and the proxy maps that role to a specific database user. This avoids shared credentials while still benefiting from connection pooling.

```
# Create per-tenant IAM policy for database access
cat > /tmp/tenant-acme-db-policy.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "rds-db:connect",
      "Resource": "arn:aws:rds-db:us-east-1:123456789012:dbuser:prx-0abc123/tenant_acme_corp"
    }
  ]
}
EOF

aws iam create-policy \
  --policy-name rds-proxy-tenant-acme-corp \
  --policy-document file:///tmp/tenant-acme-db-policy.json
```

The `dbuser` component in the ARN (`tenant_acme_corp`) maps to the PostgreSQL role you created in section 14.3. The tenant application pods carry this IAM policy via their execution role, generate short-lived authentication tokens (15-minute expiry), and connect through the proxy endpoint. The proxy transparently pools those connections against the backend cluster.

14.5 Google Cloud SQL and AlloyDB: Consolidation Strategies

Google Cloud SQL does not have an equivalent to Azure SQL Elastic Pools. There is no shared capacity construct where multiple databases draw from a common resource allocation. Multi-tenant consolidation on Cloud SQL means one of two things: running multiple databases on the same Cloud SQL instance, or using AlloyDB with its own connection pooling capabilities.

Cloud SQL multi-database consolidation:

Cloud SQL instances support multiple databases, and the PostgreSQL edition gives you the full schema-per-tenant pattern without modification. A single Cloud SQL for PostgreSQL `db-n1-standard-8` instance (8 vCPUs, 30 GB RAM) can comfortably host 50–200 small tenant schemas depending on concurrency patterns. The challenge is that Cloud SQL instance classes are coarser-grained than Azure or Aurora—you pick a machine type, and scaling means changing it (with a brief restart).

```
# Create a Cloud SQL instance sized for multi-tenant consolidation
gcloud sql instances create cloudsql-saas-prod \
```



```

--database-version=POSTGRES_15 \
--tier=db-n1-standard-8 \
--region=us-central1 \
--availability-type=REGIONAL \
--enable-bin-log \
--backup-start-time=03:00 \
--retained-backups-count=14 \
--retained-transaction-log-days=7 \
--storage-type=SSD \
--storage-size=200GB \
--storage-auto-increase \
--storage-auto-increase-limit=2000 \
--database-flags=\
max_connections=2000,\
work_mem=8192,\
idle_in_transaction_session_timeout=30000,\
statement_timeout=120000,\
log_min_duration_statement=1000 \
--maintenance-window-day=SUN \
--maintenance-window-hour=5 \
--deletion-protection

# Create a database for a tenant
gcloud sql databases create tenant-acme-corp \
--instance=cloudsql-saas-prod \
--charset=UTF8 \
--collation=en_US.UTF8

# Create per-tenant user with connection limit
# Cloud SQL doesn't expose CONNECTION LIMIT via gcloud - use psql directly
gcloud sql connect cloudsql-saas-prod --user=postgres --quiet << 'PSQL'
CREATE ROLE tenant_acme_corp
WITH LOGIN
CONNECTION LIMIT 30
PASSWORD 'placeholder-use-secret-manager';

ALTER ROLE tenant_acme_corp SET statement_timeout = '120s';
ALTER ROLE tenant_acme_corp SET work_mem = '8MB';
PSQL

```

Cloud SQL Insights for per-tenant query monitoring:

Cloud SQL Query Insights gives you per-database and per-user query performance visibility, which is the closest analog to Azure's per-database DMV data in a managed context.

```

# Enable Query Insights on the instance
gcloud sql instances patch cloudsql-saas-prod \
--insights-config-query-insights-enabled \
--insights-config-record-client-address \
--insights-config-query-string-length=1024 \
--insights-config-query-plans-per-minute=20

# Query the Insights data via Cloud Monitoring
# The following filter returns top queries by total execution time
# for a specific database (tenant), useful for identifying noisy tenants
gcloud logging read \
'resource.type="cloudsql_database" AND
resource.labels.database_id="project-id:cloudsql-saas-prod" AND
jsonPayload.databaseName="tenant-acme-corp" AND
jsonPayload.duration>1000' \
--limit=50 \
--format=json

```

AlloyDB for high-concurrency multi-tenant workloads:

AlloyDB is Google's PostgreSQL-compatible managed database engineered for OLTP performance. Its architecture is meaningfully different from Cloud SQL: a decoupled storage layer (similar to Aurora), a columnar query engine for analytics, and built-in connection pooling via AlloyDB Auth

Proxy or the managed PgBouncer-based connection pooler available on AlloyDB Omni.

For multi-tenant SaaS, AlloyDB's most relevant feature is its connection pooler, which operates at the cluster level and reduces the per-connection overhead burden that drives Cloud SQL users toward oversized instances.

```
# Create AlloyDB cluster
gcloud alloydb clusters create alloydb-saas-prod \
  --region=us-central1 \
  --password="$(gcloud secrets versions access latest --secret=alloydb-admin-password)" \
  --network=projects/your-project/global/networks/saas-vpc

# Create primary instance
gcloud alloydb instances create alloydb-saas-prod-primary \
  --instance-type=PRIMARY \
  --cpu-count=8 \
  --cluster=alloydb-saas-prod \
  --region=us-central1 \
  --database-flags=\
max_connections=3000,\
work_mem=8192,\
idle_in_transaction_session_timeout=30000

# Create read pool instance for tenant read queries
gcloud alloydb instances create alloydb-saas-prod-readpool \
  --instance-type=READ_POOL \
  --cpu-count=4 \
  --read-pool-node-count=2 \
  --cluster=alloydb-saas-prod \
  --region=us-central1
```

Comparing the three platforms for multi-tenant consolidation:

Capability	Azure SQL Elastic Pools	Aurora Serverless v2	Cloud SQL / AlloyDB
Shared resource pool	Native — pool-level vCores shared across DBs	Cluster-level ACU scaling; all tenants share one cluster	No pool construct; instance-level resources
Per-tenant resource cap	db-max-capacity per database	Per-role connection limits only	Per-role connection limits only
Scale to near-zero	Yes (db-min-capacity=0)	Yes (0.5 ACU minimum per instance)	No — instance always running
Max databases per pool/cluster	500 (GP), 100 (BC)	Limited by max connections and storage	Effectively unlimited schemas/databases
Native connection pooling	Managed internally	RDS Proxy (separate service)	AlloyDB built-in pooler; Cloud SQL Auth Proxy
Cross-tenant query prevention	Database-level isolation	Schema/role isolation	Schema/role isolation

The table makes clear why Azure SQL Elastic Pools remain the reference implementation: they are the only platform with a native, enforced, shared resource pool where per-database resource limits are a first-class configuration attribute. AWS and GCP achieve similar economics through

instance-level scaling and application-layer pooling, but require more operational scaffolding to achieve equivalent tenant isolation guarantees.

14.6 Per-Tenant Resource Governance: Preventing the Noisy Neighbor

Every consolidation architecture has a noisy-neighbor failure mode. A tenant runs a full table scan, saturates I/O, and response times for every other tenant spike. The monitoring catches it 5 minutes later. Two tenants complain to support before you’ve remediated. Here is how to build preventive governance at each layer.

Layer 1: Connection-level limits (PostgreSQL / Aurora / Cloud SQL)

Connection limits per role are the first line of defense. A tenant whose application has a connection leak cannot drain the instance’s `max_connections` pool for everyone else.

```
-- PostgreSQL: enforce connection and memory limits at the role level
-- Apply during tenant onboarding, review during tier changes

ALTER ROLE tenant_acme_corp CONNECTION LIMIT 50;
ALTER ROLE tenant_acme_corp SET work_mem = '8MB';
ALTER ROLE tenant_acme_corp SET statement_timeout = '120000'; -- 2 minutes
ALTER ROLE tenant_acme_corp SET idle_in_transaction_session_timeout = '30000'; -- 30 seconds
ALTER ROLE tenant_acme_corp SET lock_timeout = '10000'; -- 10 seconds

-- For enterprise/premium tier tenants, allow more resources
ALTER ROLE tenant_enterprise_bigcorp CONNECTION LIMIT 200;
ALTER ROLE tenant_enterprise_bigcorp SET work_mem = '32MB';
ALTER ROLE tenant_enterprise_bigcorp SET statement_timeout = '300000'; -- 5 minutes

-- Monitor current connection consumption per tenant
SELECT
  username AS tenant_role,
  count(*) AS active_connections,
  (SELECT rolconnlimit FROM pg_roles WHERE rolname = sa.username) AS connection_limit,
  round(
    count(*)::numeric /
    NULLIF((SELECT rolconnlimit FROM pg_roles WHERE rolname = sa.username), -1) * 100,
    1
  ) AS pct_of_limit,
  sum(CASE WHEN state = 'idle in transaction' THEN 1 ELSE 0 END) AS idle_in_txn,
  sum(CASE WHEN state = 'active' THEN 1 ELSE 0 END) AS running_queries,
  max(extract(epoch from (now() - query_start)))::int AS longest_query_secs
FROM pg_stat_activity sa
WHERE username LIKE 'tenant_%'
GROUP BY username
ORDER BY active_connections DESC;
```

SQL Server equivalent (Azure SQL Elastic Pool): In SQL Server, you can’t set per-login connection limits at the engine level, but you can use Resource Governor (not available in Azure SQL—this is an on-premises feature) or rely on the pool’s `db-max-capacity` enforcement. The practical per-tenant protection in Azure SQL is the per-database vCore cap combined with monitoring-driven alerting when a specific tenant database shows high DTU/vCore consumption.

```
-- SQL Server / Azure SQL: identify the noisy tenant database
-- Run against the master database or via elastic query

-- From master: check per-database resource consumption in the pool
SELECT
  database_name,
```

```

    avg_cpu_percent,
    avg_data_io_percent,
    avg_log_write_percent,
    avg_memory_usage_percent,
    max_worker_percent,
    max_session_percent,
    avg_instance_cpu_percent,
    avg_instance_memory_percent
FROM sys.elastic_pool_resource_stats
WHERE start_time >= DATEADD(MINUTE, -30, GETUTCDATE())
    AND elastic_pool_name = 'ep-saas-tenants'
ORDER BY avg_cpu_percent DESC;

-- Identify long-running queries in a noisy tenant database
-- Connect to the specific tenant DB and run:
SELECT
    session_id,
    start_time,
    status,
    command,
    DATEDIFF(SECOND, start_time, GETDATE()) AS duration_secs,
    wait_type,
    wait_time / 1000.0 AS wait_secs,
    cpu_time / 1000.0 AS cpu_secs,
    reads,
    logical_reads,
    writes,
    LEFT(text, 200) AS query_text
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE status != 'background'
    AND session_id != @@SPID
ORDER BY duration_secs DESC;

```

Layer 2: Automated query termination

Monitoring catches noisy neighbors after the fact. Automated termination of runaway queries prevents prolonged impact. The PostgreSQL `statement_timeout` at the role level handles most cases. For cases where a tenant circumvents role-level settings (e.g., by resetting them in the session), you need an external watchdog.

```

import psycopg2
import logging
from datetime import datetime

logger = logging.getLogger(__name__)

def terminate_runaway_tenant_queries(
    conn_string: str,
    statement_timeout_secs: int = 180,
    exclude_roles: list = None
) -> list:
    """
    Watchdog: terminate tenant queries exceeding the statement timeout threshold.
    Returns list of terminated queries for audit logging.

    This complements (not replaces) role-level statement_timeout settings.
    Use for tenants who may reset session settings or for cases where
    pg_terminate_backend is needed rather than pg_cancel_backend.
    """
    if exclude_roles is None:
        exclude_roles = ['postgres', 'clusteradmin', 'rdsadmin']

    terminated = []

    with psycopg2.connect(conn_string) as conn:
        conn.autocommit = True
        with conn.cursor() as cur:

```

```

# Find runaway tenant queries
cur.execute("""
SELECT
    pid,
    username,
    application_name,
    state,
    EXTRACT(EPOCH FROM (now() - query_start))::int AS running_secs,
    LEFT(query, 300) AS query_preview
FROM pg_stat_activity
WHERE username LIKE 'tenant_%'
AND username != ALL(%)s)
AND state = 'active'
AND query_start IS NOT NULL
AND EXTRACT(EPOCH FROM (now() - query_start)) > %s
AND query NOT ILIKE '%pg_stat_activity%'
ORDER BY running_secs DESC
""", (exclude_roles, statement_timeout_secs))

runaway_queries = cur.fetchall()

for pid, username, app_name, state, running_secs, query_preview in runaway_queries:
    try:
        # Use pg_cancel_backend first (graceful)
        cur.execute("SELECT pg_cancel_backend(%s)", (pid,))
        result = cur.fetchone()[0]

        terminated.append({
            'pid': pid,
            'tenant': username,
            'running_secs': running_secs,
            'query_preview': query_preview,
            'action': 'cancelled',
            'success': result,
            'timestamp': datetime.utcnow().isoformat()
        })

        logger.warning(
            "Cancelled runaway query for tenant %s (pid=%d, running=%ds): %s",
            username, pid, running_secs, query_preview[:100]
        )
    except Exception as e:
        logger.error(
            "Failed to cancel query for tenant %s pid %d: %s",
            username, pid, e
        )

return terminated

```

Layer 3: Schema-level object size limits

A tenant who creates an accidental Cartesian join that generates a 200 GB temporary table will saturate disk I/O for every other tenant. PostgreSQL doesn't provide per-schema disk quotas natively, but you can implement soft enforcement through periodic monitoring and alerting.

```

-- Monitor per-tenant schema size growth
-- Run periodically (every 15 minutes) and alert on anomalous growth

WITH tenant_sizes AS (
    SELECT
        n.nspname AS schema_name,
        pg_size_pretty(sum(pg_total_relation_size(c.oid))) AS pretty_size,
        sum(pg_total_relation_size(c.oid)) AS size_bytes,
        count(c.oid) AS table_count
    FROM pg_class c
    JOIN pg_namespace n ON n.oid = c.relnamespace
    WHERE n.nspname LIKE 'tenant_%'
        AND c.relkind = 'r' -- tables only
    GROUP BY n.nspname
)

```

```

),
size_thresholds AS (
  SELECT
    schema_name,
    pretty_size,
    size_bytes,
    table_count,
    CASE
      WHEN size_bytes > 10737418240 THEN 'CRITICAL' -- 10 GB
      WHEN size_bytes > 5368709120 THEN 'WARNING' -- 5 GB
      ELSE 'OK'
    END AS size_status
  FROM tenant_sizes
)
SELECT *
FROM size_thresholds
WHERE size_status != 'OK'
ORDER BY size_bytes DESC;

-- Find the largest tables within a specific tenant schema
-- (replace tenant_acme_corp with the schema under investigation)
SELECT
  relname AS table_name,
  pg_size_pretty(pg_total_relation_size(oid)) AS total_size,
  pg_size_pretty(pg_relation_size(oid)) AS table_size,
  pg_size_pretty(pg_total_relation_size(oid) - pg_relation_size(oid)) AS index_size,
  n_live_tup AS live_rows,
  n_dead_tup AS dead_rows
FROM pg_class
WHERE relnamespace = 'tenant_acme_corp'::regnamespace
  AND relkind = 'r'
ORDER BY pg_total_relation_size(oid) DESC
LIMIT 20;

```

14.7 Tenant Tier Management: Mapping SaaS Plans to Resource Configurations

Real SaaS products have pricing tiers—Starter, Professional, Enterprise. The database resource configuration should reflect those tiers mechanically, not as a manual one-off each time a customer upgrades. Build a tier management system that maps plan changes to database resource changes without human intervention.

Tier configuration schema:

```

-- Central management database (separate from tenant databases)
-- Track tenant tiers and apply resource configurations

CREATE TABLE tenant_tiers (
  tier_name          VARCHAR(50) PRIMARY KEY,
  connection_limit  INT NOT NULL DEFAULT 20,
  statement_timeout_ms INT NOT NULL DEFAULT 30000,
  work_mem_mb       INT NOT NULL DEFAULT 4,
  idle_txn_timeout_ms INT NOT NULL DEFAULT 15000,
  max_schema_size_gb NUMERIC(6,1) NOT NULL DEFAULT 1.0,
  description       TEXT
);

INSERT INTO tenant_tiers VALUES
  ('starter',      20, 30000, 4, 15000, 1.0, 'Entry-level plan - shared pool, small quotas'),
  ('professional', 50, 120000, 8, 30000, 5.0, 'Mid-tier - higher concurrency and query budget'),
  ('enterprise',   200, 300000, 32, 60000, 25.0, 'Enterprise - priority resources, extended timeouts');

CREATE TABLE tenants (

```

```

tenant_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_slug    VARCHAR(100) UNIQUE NOT NULL,
db_role        VARCHAR(100) NOT NULL,
tier_name      VARCHAR(50) REFERENCES tenant_tiers(tier_name),
created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
tier_updated_at TIMESTAMPTZ
);

-- Function to generate the ALTER ROLE statements for a tier change
-- Application calls this and executes the result against the tenant cluster
CREATE OR REPLACE FUNCTION generate_tier_config_sql(
    p_tenant_slug VARCHAR,
    p_new_tier     VARCHAR
) RETURNS TEXT AS $$
DECLARE
    v_role VARCHAR;
    v_tier tenant_tiers%ROWTYPE;
BEGIN
    SELECT db_role INTO v_role
    FROM tenants
    WHERE tenant_slug = p_tenant_slug;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Tenant % not found', p_tenant_slug;
    END IF;

    SELECT * INTO v_tier
    FROM tenant_tiers
    WHERE tier_name = p_new_tier;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Tier % not found', p_new_tier;
    END IF;

    RETURN format(
        $$SQL$
        ALTER ROLE %I CONNECTION LIMIT %s;
        ALTER ROLE %I SET statement_timeout = '%s';
        ALTER ROLE %I SET work_mem = '%sMB';
        ALTER ROLE %I SET idle_in_transaction_session_timeout = '%s';
        $SQL$,
        v_role, v_tier.connection_limit,
        v_role, v_tier.statement_timeout_ms,
        v_role, v_tier.work_mem_mb,
        v_role, v_tier.idle_txn_timeout_ms
    );
END;
$$ LANGUAGE plpgsql;

```

Applying tier changes programmatically:

```

import psycpg2
import logging
from typing import Optional

logger = logging.getLogger(__name__)

class TenantTierManager:
    """
    Manages database resource configurations when tenants change pricing plans.
    Connects to both the management database and the tenant cluster.
    """

    def __init__(self, mgmt_conn_string: str, tenant_cluster_conn_string: str):
        self.mgmt_conn_string = mgmt_conn_string
        self.tenant_cluster_conn_string = tenant_cluster_conn_string

    def upgrade_tenant_tier(
        self,
        tenant_slug: str,

```

```

    new_tier: str,
    dry_run: bool = False
) -> dict:
    """
    Apply new tier resource limits to a tenant's database role.
    Returns dict with status and the SQL that was (or would be) applied.
    """
    # Get the configuration SQL from the management database
    with psycopg2.connect(self.mgmt_conn_string) as mgmt_conn:
        with mgmt_conn.cursor() as cur:
            cur.execute(
                "SELECT generate_tier_config_sql(%s, %s)",
                (tenant_slug, new_tier)
            )
            config_sql = cur.fetchone()[0]

    if dry_run:
        logger.info("DRY RUN - would apply to tenant %s:\n%s", tenant_slug, config_sql)
        return {'status': 'dry_run', 'sql': config_sql, 'tenant': tenant_slug}

    # Apply to the tenant cluster
    with psycopg2.connect(self.tenant_cluster_conn_string) as tenant_conn:
        tenant_conn.autocommit = True
        with tenant_conn.cursor() as cur:
            for statement in config_sql.strip().split(';'):
                statement = statement.strip()
                if statement:
                    cur.execute(statement)
                    logger.info("Applied: %s", statement[:80])

    # Record the tier change in the management database
    with psycopg2.connect(self.mgmt_conn_string) as mgmt_conn:
        with mgmt_conn.cursor() as cur:
            cur.execute("""
                UPDATE tenants
                SET tier_name = %s, tier_updated_at = now()
                WHERE tenant_slug = %s
            """, (new_tier, tenant_slug))
            mgmt_conn.commit()

    logger.info(
        "Tenant %s successfully upgraded to tier %s",
        tenant_slug, new_tier
    )
    return {
        'status': 'applied',
        'sql': config_sql,
        'tenant': tenant_slug,
        'new_tier': new_tier
    }
}

```

14.8 Cost Attribution: Knowing What Each Tenant Actually Costs

Consolidation creates an accounting problem. With dedicated instances, cost per tenant is `instance_cost / 1`. With shared infrastructure, you need a model to attribute shared costs to individual tenants—both for internal chargeback and to validate that your pricing tiers are actually profitable.

CloudWatch / Azure Monitor metrics for cost attribution:

On AWS Aurora, collect per-role query execution data using `pg_stat_statements` and correlate with ACU-hours consumed.


```

-- Enable pg_stat_statements on Aurora (requires parameter group change)
-- Then query per-tenant query costs

SELECT
    LEFT(username, 50) AS tenant,
    count(*) AS query_count,
    round(sum(total_exec_time)::numeric / 1000, 2) AS total_exec_secs,
    round(avg(mean_exec_time)::numeric, 2) AS avg_query_ms,
    sum(rows) AS total_rows_processed,
    round(sum(shared_blks_hit + shared_blks_read)::numeric / 128, 1) AS approx_data_mb_accessed,
    round(
        sum(total_exec_time)::numeric /
        NULLIF(sum(sum(total_exec_time)) OVER (), 0) * 100,
        2
    ) AS pct_of_total_db_cpu
FROM pg_stat_statements pss
JOIN pg_roles pr ON pr.oid = pss.userid
WHERE pr.rolname LIKE 'tenant_%'
GROUP BY username
ORDER BY total_exec_secs DESC;

-- Reset stats periodically (e.g., at the start of each billing period)
-- SELECT pg_stat_statements_reset();

```

SQL Server equivalent for Azure SQL Elastic Pool cost attribution:

```

-- Query Execution Statistics Aggregated by Database (Tenant)
-- Run against master database on the logical server

SELECT
    DB_NAME(database_id) AS tenant_database,
    count(*) AS query_count,
    sum(total_worker_time) / 1000000.0 AS total_cpu_secs,
    sum(total_elapsed_time) / 1000000.0 AS total_elapsed_secs,
    sum(total_logical_reads) AS total_logical_reads,
    sum(total_physical_reads) AS total_physical_reads,
    sum(total_logical_writes) AS total_logical_writes,
    round(
        cast(sum(total_worker_time) as float) /
        NULLIF(sum(sum(total_worker_time)) OVER (), 0) * 100,
        2
    ) AS pct_pool_cpu
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_plan_attributes(qs.plan_handle) pa
WHERE pa.attribute = 'dbid'
    AND DB_NAME(CAST(pa.value AS INT)) LIKE 'tenant-%'
GROUP BY database_id
ORDER BY total_cpu_secs DESC;

```

Combine these query-level attribution metrics with the pool-level billing data from Azure Cost Management or AWS Cost Explorer to build a per-tenant cost model. The query CPU percentage from `pg_stat_statements` becomes a weight to distribute the pool's compute bill across tenants proportionally. Add per-tenant storage (queried from `pg_database_size()` or Azure's `sys.database_files`) and you have a reasonable chargeback model even without per-database billing.

Key Takeaways

- **Azure SQL Elastic Pools are the only managed cloud database service with a native shared resource pool construct**, where multiple tenant databases draw from a common vCore/DTU allocation with enforced per-database caps. AWS and GCP achieve sim-

ilar economics through instance-level autoscaling (Aurora Serverless v2) and application-layer connection pooling (RDS Proxy, AlloyDB pooler), but require more operational scaffolding to enforce equivalent per-tenant isolation.

- **Right-size elastic pools using empirical utilization data, not peak-sum arithmetic.** Capture `sys.dm_db_resource_stats` (SQL Server) or `pg_stat_activity` / `pg_stat_statements` (PostgreSQL) for two to four weeks before migration, then model realistic peak concurrency rather than multiplying per-tenant maximums — in practice, realistic concurrent peak load is a small fraction of the sum of per-tenant maximums, so this exercise usually shrinks the required pool size substantially.
 - **RDS Proxy multiplexing effectiveness depends entirely on avoiding session pinning.** Applications that use `SET LOCAL`, per-request session variables for row-level security, or advisory locks will pin connections and may achieve little or no reduction in database-side connection count. Measure `DatabaseConnectionsCurrentlySessionPinned` before committing to RDS Proxy as a cost-reduction strategy.
 - **Per-tenant resource governance must operate at multiple layers simultaneously:** connection limits at the role level, statement and idle-in-transaction timeouts at the role level, query watchdog processes for circumvention, and storage growth monitoring for disk pressure — no single control is sufficient on its own.
 - **Tenant tier changes must propagate to database resource configurations automatically.** Maintaining a management schema that maps pricing plans to connection limits, `work_mem`, and statement timeouts, and applying those changes programmatically on plan upgrade or downgrade, eliminates the class of incidents where a customer on a starter plan exhausts pool resources because their tier was changed in the billing system but not in the database.
-

Chapter 15: Azure SQL Hyperscale: When You Need It

Azure SQL Hyperscale is Microsoft’s answer to a problem that on-prem DBAs know well: what happens when your database outgrows your storage architecture? Traditional SQL Server installations hit ceiling after ceiling — SAN I/O limits, filegroup management headaches, backup windows measured in hours, and the constant anxiety of a database that has grown beyond what your infrastructure was originally sized to handle. Azure SQL Hyperscale reimagines the storage layer entirely, decoupling compute from storage and introducing a distributed log service and page server architecture that lets you scale to 100 TB without a weekend maintenance window. This chapter covers when Hyperscale is the right choice, how it differs architecturally from standard Azure SQL Database tiers, how to provision and configure it correctly, what the production gotchas look like, and how to migrate into — and out of — it with your eyes open.

What Hyperscale Actually Is (And What It Isn’t)

Before you provision a Hyperscale database, you need to understand that it is not simply “a bigger Azure SQL Database.” It is a fundamentally different storage architecture that happens to present a familiar SQL Server surface. If you treat it like a standard Azure SQL tier with a higher storage cap, you will be surprised by behaviors that don’t match your mental model.

The Architecture

In a traditional SQL Server instance — on-prem or in an Azure SQL General Purpose or Business Critical tier — your compute node owns the data files. Pages live on local or attached storage that the database engine reads and writes directly. Backups are full snapshots of that storage. Replica setup involves copying or synchronizing those files. Everything is tightly coupled.

Hyperscale decouples these layers. The architecture has four distinct components:

1. **Compute replicas** — These are your SQL Server engine instances. They hold the buffer pool (in-memory pages) and execute queries, but they do not own the data files.
2. **Log service** — A distributed, append-only transaction log service that sits between compute and storage. When your primary compute node commits a transaction, it writes to the log service first. This is where the acknowledged commit happens. Page servers and secondary replicas consume from this log asynchronously.

3. **Page servers** — Multiple independent page server processes that maintain portions of the database’s page space and serve pages to compute nodes on demand. When a compute node needs a page not in its buffer pool, it fetches it from a page server over the network rather than from local disk. Each page server handles a shard of the database’s page space, and they scale out as the database grows.
4. **Azure Storage** — The durable backing store underneath the page servers. This is Azure Premium Storage, providing the actual durability guarantee.

The practical consequence: your compute node’s buffer pool is a cache in front of a distributed page store. A cold query that misses the buffer pool on a Hyperscale instance will be slower than the equivalent query on a Business Critical instance with local SSD, because it pays a network round-trip to a page server. On hot, well-cached workloads, Hyperscale performs comparably or better than other tiers for reads because you can add named replicas that each have their own independent buffer pool.

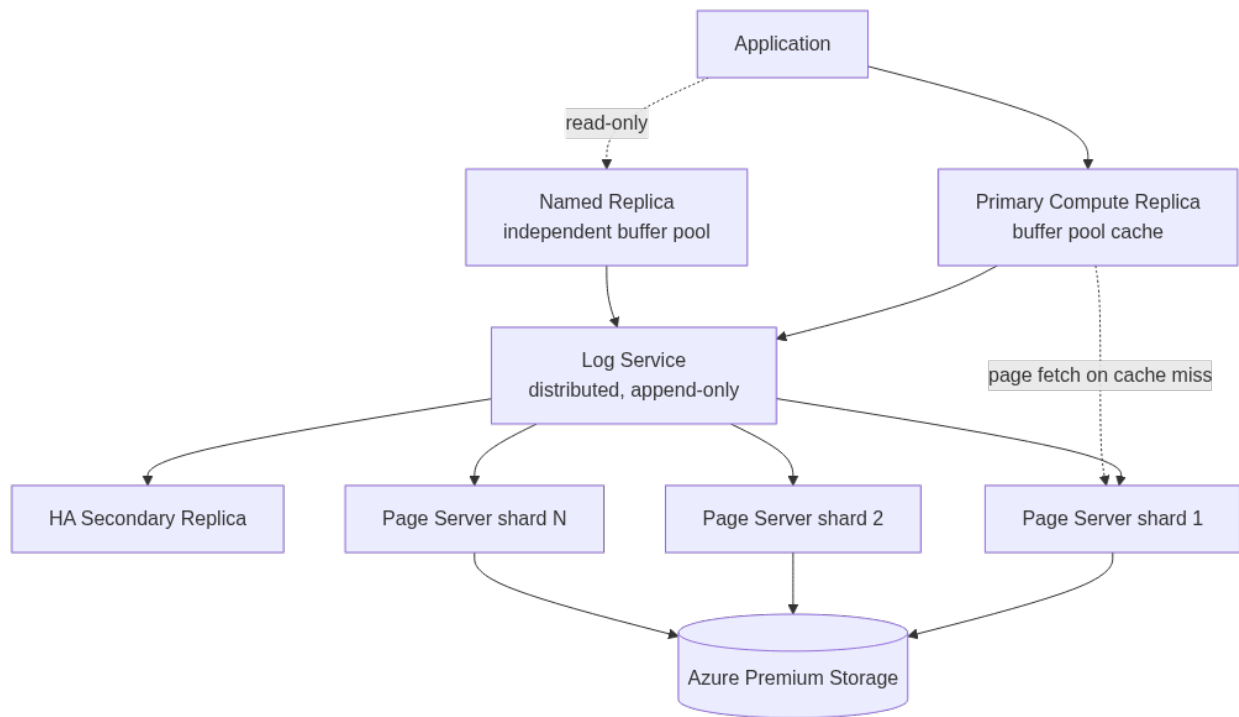


Figure 6: Hyperscale’s decoupled layers: compute replicas hold only a buffer-pool cache; durable page data lives in sharded page servers backed by Azure Storage, with the log service as the single write path all replicas consume from.

What It Is Designed For

Hyperscale makes sense when one or more of the following conditions apply:

- Your database exceeds 4 TB (the limit for General Purpose and Business Critical tiers)
- You need near-instant database creation and restore times regardless of database size
- Your backup and restore windows are a compliance or operational problem — a 50 TB Hyperscale database restores in roughly 5–10 minutes because Azure only needs to instantiate compute and point it at existing page server snapshots

- You have a mixed workload where you want to offload read-heavy reporting or analytics to a separate compute replica with its own memory and CPU, without the overhead of maintaining a full replica
- You have bursty storage growth patterns and don't want to over-provision

What It Is Not Designed For

Hyperscale is not a general upgrade from Business Critical. Business Critical uses local SSD with in-memory OLTP support and has lower read latency for cold pages because storage is local. If your workload is primarily OLTP with a well-tuned buffer pool hit ratio above 98%, Business Critical will often outperform Hyperscale for raw query latency. Hyperscale also has restrictions that on-prem DBAs find surprising: as of the current GA release, you cannot change a Hyperscale database back to another service tier. That decision is one-directional. You migrate in, and if you need to migrate out, you do it through export/import or backup/restore — not through a service tier change.

The no-downgrade restriction is not a technical limitation that might be lifted later. It reflects the fact that Hyperscale storage format is fundamentally different from the other tiers. Plan accordingly before you migrate production.

Provisioning Hyperscale: Configuration That Matters

When you create a Hyperscale database, the decisions that matter most are compute generation, vCore count, read replica topology, and backup redundancy. Storage is automatic and does not require sizing decisions the way you were accustomed to on-prem.

Creating a Hyperscale Database with Azure CLI

```
# Create a resource group and logical server first if not already present
az group create \
  --name rg-hyperscale-prod \
  --location eastus2

az sql server create \
  --name sql-hs-prod-eastus2 \
  --resource-group rg-hyperscale-prod \
  --location eastus2 \
  --admin-user sqladmin \
  --admin-password "$(az keyvault secret show \
    --vault-name kv-prod \
    --name sql-admin-password \
    --query value -o tsv)"

# Create the Hyperscale database
# Edition: Hyperscale
# Service objective: HS_Gen5_8 (Gen5, 8 vCores)
# High-availability replicas: 1 (adds a secondary compute replica for HA)
# Backup storage redundancy: Zone (zone-redundant backup storage)
az sql db create \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --name OrdersDB \
  --edition Hyperscale \
  --service-objective HS_Gen5_8 \
  --ha-replicas 1 \
  --backup-storage-redundancy Zone \
  --read-scale Enabled
```

```
# Verify the database properties
az sql db show \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --name OrdersDB \
  --query "{tier:sku.tier, size:sku.name, maxSize:maxSizeBytes,
  ↵  haReplicas:highAvailabilityReplicaCount}" \
  --output table
```

Hyperscale with Terraform

```
terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.80"
    }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "hs_rg" {
  name     = "rg-hyperscale-prod"
  location = "East US 2"
}

resource "azurerm_mssql_server" "hs_server" {
  name                        = "sql-hs-prod-eastus2"
  resource_group_name        = azurerm_resource_group.hs_rg.name
  location                   = azurerm_resource_group.hs_rg.location
  version                    = "12.0"
  administrator_login        = "sqladmin"
  administrator_login_password = var.sql_admin_password
  minimum_tls_version        = "1.2"

  azuread_administrator {
    login_username = "DBA-Admins"
    object_id      = var.aad_admin_group_object_id
  }
}

resource "azurerm_mssql_database" "orders_hs" {
  name       = "OrdersDB"
  server_id  = azurerm_mssql_server.hs_server.id

  sku_name = "HS_Gen5_8" # Hyperscale, Gen5, 8 vCores

  # Hyperscale-specific settings
  # high_availability_replica_count: 0 = no HA replica (cheaper, higher RTO)
  #                                  1 = one HA replica (recommended for production)
  #                                  2 = two HA replicas (for AZ failure tolerance)
  high_availability_replica_count = 1

  # read_replica_count: named replicas for read scale-out
  # These are separately provisioned and charged
  # Set to 0 here; add named replicas separately when needed

  # Backup storage redundancy: Geo, GeoZone, Local, Zone
  geo_backup_enabled = true

  tags = {
    Environment = "Production"
    ManagedBy   = "Terraform"
    CostCenter  = "Engineering"
  }
}
```

```
# Private endpoint for Hyperscale server
resource "azurerm_private_endpoint" "hs_pe" {
  name                = "pe-sql-hs-prod"
  location             = azurerm_resource_group.hs_rg.location
  resource_group_name = azurerm_resource_group.hs_rg.name
  subnet_id           = var.private_endpoint_subnet_id

  private_service_connection {
    name                = "psc-sql-hs-prod"
    private_connection_resource_id = azurerm_mssql_server.hs_server.id
    subresource_names     = ["sqlServer"]
    is_manual_connection   = false
  }
}
```

Compute Tiers and Sizing

Hyperscale compute uses the same vCore-based sizing as other Azure SQL tiers, but the service objective naming prefix is `HS_`. Common choices:

Service Objective	vCores	Memory	Notes
HS_Gen5_4	4	20.4 GB	Minimum recommended for production
HS_Gen5_8	8	40.8 GB	Good starting point for moderate OLTP
HS_Gen5_16	16	83 GB	High-concurrency workloads
HS_Gen5_32	32	166 GB	Large in-memory working sets
HS_Gen5_80	80	408 GB	Very large working sets, high throughput

There are also Fsv2 (compute-optimized) and M-series (memory-optimized) hardware generations available in select regions. The M-series configurations go up to 128 vCores and 3.7 TB of memory, which is territory that previously required dedicated hardware and significant capital expenditure.

You can scale compute up or down online. Scaling to a different vCore count within the same hardware generation takes approximately 2–3 minutes with minimal disruption. Changing hardware generation (Gen5 to Fsv2, for example) takes longer and involves a reconfiguration event.

Backup Storage Redundancy — Read This Before You Deploy

The `--backup-storage-redundancy` flag matters more than it appears. The options are:

- **Local** — Backups stored in a single datacenter. Lowest cost, lowest durability. Not appropriate for most production workloads.
- **Zone** — Zone-redundant backup storage. Appropriate for single-region deployments that need AZ tolerance.
- **Geo** — Geo-redundant backup storage. Replicates backups to a paired region. Default for most Azure SQL tiers historically.
- **GeoZone** — Zone-redundant in primary region plus geo-replication. Highest durability.

The critical gotcha: you can only set backup storage redundancy when you create the Hyperscale database. You cannot change it afterward without creating a new database. If you provision with **Local** and later realize you need geo-redundant backups for your DR policy, you are rebuilding the database.

Named Replicas: Read Scale-Out Without the Traditional Overhead

One of the most operationally useful Hyperscale features is named replicas. A named replica is a Hyperscale compute node pointed at the same page servers as the primary. It sees a consistent, near-real-time view of the data (typically within 1–5 seconds of the primary, sometimes less). Critically, it is a fully independent Azure SQL Database resource with its own service tier, its own connection string, its own buffer pool, and its own independent security configuration.

On-prem, you might have handled this with Always On readable secondaries. The fundamental difference is that a named replica in Hyperscale shares storage infrastructure with the primary — it is not maintaining a full copy of the data files. Adding a named replica takes minutes regardless of database size, because Azure is not copying 50 TB of data. It is instantiating a new compute node that can already read from the existing page servers.

When Named Replicas Make Sense

Named replicas are the right choice when:

- You have a large reporting or analytics workload that competes with OLTP on the primary
- You need to give a third-party team or application its own connection endpoint with no risk of their workload affecting production OLTP
- You need read scale-out during peak periods and want to spin replicas up and down dynamically

Named replicas are billed as separate Hyperscale databases with their own vCore cost. If you provision a named replica as `HS_Gen5_16`, you pay for those 16 vCores separately. For pure read offloading, you can often provision named replicas at a lower vCore count than the primary.

```
# Create a named replica of an existing Hyperscale database
# The named replica lives in the same logical server
# It uses a different service objective than the primary if needed

az sql db replica create \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --name OrdersDB \
  --partner-database OrdersDB_ReportingReplica \
  --partner-server sql-hs-prod-eastus2 \
  --service-objective HS_Gen5_4 \
  --partner-resource-group rg-hyperscale-prod

# Verify replication link
az sql db replica list-links \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --name OrdersDB \
  --output table

# Connect to the named replica using its own connection string
# Named replica endpoint: sql-hs-prod-eastus2.database.windows.net
# Database: OrdersDB_ReportingReplica
# Application read intent:
# Server=sql-hs-prod-eastus2.database.windows.net;
# Database=OrdersDB_ReportingReplica;
# ApplicationIntent=ReadOnly;
```

Replication Lag Monitoring

Named replicas and HA secondary replicas consume the log service asynchronously. You need to

monitor replication lag in production, particularly for workloads where reporting queries need data that is close to real-time.

```
-- Query replication lag on the primary (connect to primary, not replica)
-- sys.dm_database_replica_states is your tool here
SELECT
    r.replica_id,
    r.role_desc,
    r.secondary_lag_seconds,
    r.log_send_queue_size,
    r.log_send_rate,
    r.redo_queue_size,
    r.redo_rate,
    r.synchronization_health_desc,
    r.last_hardened_lsn,
    r.last_received_lsn
FROM sys.dm_hadr_database_replica_states r
WHERE r.is_local = 0; -- Non-local = secondary replicas

-- For Hyperscale-specific log service metrics
-- Check log generation rate and log service throughput
SELECT
    database_id,
    log_bytes_used,
    log_bytes_used_percent,
    log_committed_bytes,
    log_committed_percent,
    log_recovery_size_mb,
    log_since_last_log_backup_mb,
    log_since_last_checkpoint_mb
FROM sys.dm_db_log_stats(DB_ID());

-- Monitor page server health and latency from compute node perspective
-- This DMV shows remote page fetches (page server hits vs buffer pool hits)
SELECT
    page_server_reads,
    page_server_reads_in_progress,
    page_server_read_in_progress_ms,
    page_server_read_elapsed_ms,
    checkpoint_phase_desc,
    checkpoint_start_time
FROM sys.dm_exec_requests
WHERE page_server_reads > 0
ORDER BY page_server_reads DESC;
```

A replication lag above 30 seconds warrants investigation. Common causes are high log generation rate on the primary (large bulk operations, high-frequency small transactions), temporary page server pressure, or network congestion within the Azure fabric. You cannot directly tune the log service, but you can reduce peak log generation rate by batching large DML operations.

Migrating Into Hyperscale: Patterns That Work in Production

Migrating to Hyperscale from an on-prem SQL Server or from another Azure SQL tier requires more planning than a tier change, because Hyperscale is one-directional and the migration itself requires database-level operations rather than just a service tier slider.

Option 1: Backup and Restore via Azure Blob Storage

For on-prem databases below roughly 1 TB, this is the fastest path. You take a COPY_ONLY backup, upload to Azure Blob Storage, and restore into a Hyperscale database using RESTORE DATABASE ... FROM URL.

```

-- On-premises SQL Server: Create credential and backup to Azure Blob Storage
-- Step 1: Create SAS credential on source SQL Server
CREATE CREDENTIAL [https://storprodmigration.blob.core.windows.net/backups]
WITH IDENTITY = 'SHARED ACCESS SIGNATURE',
SECRET = 'sv=2023-01-03&ss=b&srt=sco&sp=rwdlacupiytfx&se=2024-12-31T00:00:00Z&st=2024-01-01T00:00:00Z'
↳ &spr=https&sig=<your-sas-token>;
GO

-- Step 2: Full backup with COPY_ONLY (non-disruptive to existing backup chain)
BACKUP DATABASE [OrdersDB]
TO URL = 'https://storprodmigration.blob.core.windows.net/backups/OrdersDB_full_20240115.bak'
WITH
    COPY_ONLY,
    COMPRESSION,
    STATS = 10,
    BLOCKSIZE = 65536,
    MAXTRANSFERSIZE = 4194304;
GO

-- For large databases, stripe the backup across multiple files
BACKUP DATABASE [OrdersDB]
TO
    URL = 'https://storprodmigration.blob.core.windows.net/backups/OrdersDB_stripe1.bak',
    URL = 'https://storprodmigration.blob.core.windows.net/backups/OrdersDB_stripe2.bak',
    URL = 'https://storprodmigration.blob.core.windows.net/backups/OrdersDB_stripe3.bak',
    URL = 'https://storprodmigration.blob.core.windows.net/backups/OrdersDB_stripe4.bak'
WITH
    COPY_ONLY,
    COMPRESSION,
    STATS = 5;
GO

-- Step 3: On the Hyperscale target (via SSMS or Azure Cloud Shell with sqlcmd)
-- Create credential on target server
CREATE CREDENTIAL [https://storprodmigration.blob.core.windows.net/backups]
WITH IDENTITY = 'SHARED ACCESS SIGNATURE',
SECRET = 'sv=2023-01-03&ss=b&srt=sco&sp=rwdlacupiytfx&se=2024-12-31T00:00:00Z&st=2024-01-01T00:00:00Z'
↳ &spr=https&sig=<your-sas-token>;
GO

-- Step 4: Restore into Hyperscale
-- Note: You must restore to a NEW database name, not overwrite an existing one
RESTORE DATABASE [OrdersDB]
FROM
    URL = 'https://storprodmigration.blob.core.windows.net/backups/OrdersDB_stripe1.bak',
    URL = 'https://storprodmigration.blob.core.windows.net/backups/OrdersDB_stripe2.bak',
    URL = 'https://storprodmigration.blob.core.windows.net/backups/OrdersDB_stripe3.bak',
    URL = 'https://storprodmigration.blob.core.windows.net/backups/OrdersDB_stripe4.bak'
WITH
    RECOVERY,
    STATS = 5;
GO

```

For minimal-downtime migrations, you restore the full backup, then apply differential and log backups during a short cutover window. This is functionally the same as the SQL Server DBA's traditional migration playbook — the cloud just adds the credential layer.

Option 2: Azure Database Migration Service

For larger databases or when you want a more managed migration process, Azure DMS handles the orchestration. You configure it to perform an initial load followed by continuous replication of changes until you cut over. It is comparable in concept to setting up log shipping or replication to a new instance, with the monitoring and retry logic handled for you.

```

# Create an Azure DMS instance (Premium tier for online migrations)
az dms create \

```

```
--resource-group rg-hyperscale-prod \  
--name dms-migration-prod \  
--location eastus2 \  
--sku-name Premium_4vCores \  
--subnet /subscriptions/<sub-id> \  
↪ /resourceGroups/rg-network/providers/Microsoft.Network/virtualNetworks/vnet-prod/subnets/snet-dms  
↪ \  
--storage-account /subscriptions/<sub-id> \  
↪ /resourceGroups/rg-hyperscale-prod/providers/Microsoft.Storage/storageAccounts/storprodmigration  
  
# Create a migration project  
az dms project create \  
--resource-group rg-hyperscale-prod \  
--service-name dms-migration-prod \  
--name proj-ordersdb-migration \  
--source-platform SQL \  
--target-platform AzureSqlDb \  
--location eastus2  
  
# Create a migration task (online migration with continuous sync)  
az dms project task create \  
--resource-group rg-hyperscale-prod \  
--service-name dms-migration-prod \  
--project-name proj-ordersdb-migration \  
--name task-ordersdb-migrate \  
--task-type OnlineMigration \  
--source-connection-json @source_connection.json \  
--target-connection-json @target_connection.json \  
--database-options-json @database_migration_options.json  
  
# Monitor migration status  
az dms project task show \  
--resource-group rg-hyperscale-prod \  
--service-name dms-migration-prod \  
--project-name proj-ordersdb-migration \  
--name task-ordersdb-migrate \  
--expand output \  
--query "output[0].migrationState"
```

Pre-Migration Compatibility Check

Hyperscale supports most SQL Server features, but there are exclusions. Run the Database Experimentation Assistant (DEA) or use the assessment feature in Azure Migrate before committing to a migration timeline. Common compatibility gaps that surface for on-prem databases:

- Linked servers — not supported in Azure SQL Database including Hyperscale. Replace with OPENDATASOURCE, external tables, or application-level joins.
- SQL Server Agent — not available. Use Azure Elastic Jobs or Azure Automation.
- Filestream / Filetable — not supported.
- Cross-database queries within the same instance — Azure SQL is database-scoped. If your application uses three-part names to query across databases, that needs rearchitecting.
- BULK INSERT from local file paths — requires Azure Blob Storage as the source.

These are not Hyperscale-specific limitations. They are Azure SQL Database limitations that apply across all tiers including Hyperscale. The point is that you discover them during migration planning, not after you have committed to the migration.

Production Gotchas and Operational Patterns

Hyperscale has behaviors that will surprise experienced SQL Server DBAs because they are not documented in the places you look first. This section covers the ones that cause the most friction in real production deployments.

Cold Query Latency and Buffer Pool Warming

After a compute replica failover — whether from a planned scaling event, an unplanned node failure, or the HA replica being promoted to primary — the new compute node starts with a cold buffer pool. Pages must be fetched from page servers on demand. For a large database, the period immediately following a failover will have noticeably higher read latency than steady state.

On Business Critical, failover also lands you on a cold buffer pool on the new node if it was an unplanned failover, but the page fetch is from local SSD. On Hyperscale, it is a network round-trip to a page server. The latency is typically measured in single-digit milliseconds per page fetch, but queries that scan large ranges and miss the buffer pool can feel sluggish for several minutes to an hour after failover, depending on your working set size and query patterns.

The mitigation strategies are:

1. Schedule planned failovers (compute scaling, maintenance) during low-traffic periods when possible.
2. Use connection retry logic in your application so that the brief (~30 second) failover window does not produce user-visible errors.
3. For workloads where cold start latency is particularly damaging, consider pre-warming by running a scheduled query after known maintenance windows that touches your most-accessed indexes.

```
-- Simple buffer pool pre-warming query pattern
-- Run this after a planned failover or compute scaling event
-- Adjust the table and index list to match your actual hot working set

-- Pre-warm a specific clustered index by scanning it
-- This forces pages into the buffer pool proactively
SELECT COUNT_BIG(*) FROM dbo.Orders WITH (NOLOCK);
SELECT COUNT_BIG(*) FROM dbo.OrderItems WITH (NOLOCK);
SELECT COUNT_BIG(*) FROM dbo.Customers WITH (NOLOCK);

-- You can also pre-warm specific index ranges if full scans are too expensive
-- Warm the last 90 days of orders (typical hot working set for OLTP)
SELECT COUNT_BIG(*)
FROM dbo.Orders WITH (NOLOCK)
WHERE OrderDate >= DATEADD(DAY, -90, GETDATE());

-- Monitor buffer pool state after warming
-- Check what percentage of a table is currently cached
SELECT
    t.name AS TableName,
    i.name AS IndexName,
    i.type_desc AS IndexType,
    COUNT(1) AS CachedPages,
    COUNT(1) * 8 / 1024 AS CachedMB,
    SUM(CASE WHEN b.is_modified = 1 THEN 1 ELSE 0 END) AS DirtyPages
FROM sys.dm_os_buffer_descriptors b
INNER JOIN sys.allocation_units au
    ON b.allocation_unit_id = au.allocation_unit_id
INNER JOIN sys.partitions p
    ON au.container_id = p.hobt_id
INNER JOIN sys.indexes i
    ON p.object_id = i.object_id AND p.index_id = i.index_id
```

```

INNER JOIN sys.tables t
  ON p.object_id = t.object_id
WHERE b.database_id = DB_ID()
GROUP BY t.name, i.name, i.type_desc
ORDER BY CachedMB DESC;

```

The Checkpoint Behavior Difference

On standard SQL Server, checkpoints flush dirty pages from the buffer pool to disk at the storage layer. On Hyperscale, checkpoint behavior is different because the primary compute node does not own the storage. Checkpoints on Hyperscale flush dirty pages from the compute buffer pool to the page servers. This means the write path goes: application DML → log service (commit acknowledged here) → page server receives log record and applies it → dirty page flushed from compute buffer pool on checkpoint.

The practical consequence is that I/O patterns around checkpoint look different in wait statistics. You will see `WRITELOG` waits that correspond to log service throughput, not local disk throughput. If you have historically used `WRITELOG` waits as a proxy for your storage I/O ceiling, you need to recalibrate — the constraint in Hyperscale is the log service throughput, which scales with your vCore tier.

```

-- Monitor log throughput and wait statistics relevant to Hyperscale
SELECT
  wait_type,
  waiting_tasks_count,
  wait_time_ms,
  max_wait_time_ms,
  signal_wait_time_ms,
  wait_time_ms - signal_wait_time_ms AS resource_wait_time_ms
FROM sys.dm_os_wait_stats
WHERE wait_type IN (
  'WRITELOG',
  'LOG_RATE_GOVERNOR',
  'HADR_LOGCAPTURE_WAIT',
  'HADR_WORK_QUEUE',
  'PAGEIOLATCH_SH',
  'PAGEIOLATCH_EX',
  'PAGEIOLATCH_UP',
  'REMOTEBUF_IO_WAIT' -- Hyperscale-specific: wait for remote page server I/O
)
ORDER BY wait_time_ms DESC;

-- LOG_RATE_GOVERNOR is particularly important for Hyperscale
-- It indicates your workload is hitting the log generation rate limit for your vCore tier
-- If you see high LOG_RATE_GOVERNOR waits, you need to scale up vCores or reduce log throughput

```

LOG_RATE_GOVERNOR: The Throttle You Will Hit

Each Hyperscale vCore tier has a maximum log generation rate measured in MB/s. When your transaction workload generates log records faster than this limit, SQL Server itself throttles new log-generating transactions, producing `LOG_RATE_GOVERNOR` waits. This is not a bug — it is an intentional throttle that protects the log service and downstream page servers.

The log rate limits increase with vCore count. If you are routinely hitting `LOG_RATE_GOVERNOR` waits, the options are:

1. Scale up vCores to a tier with a higher log rate limit.
2. Reduce log generation per unit of work — batch your DML operations, minimize unnecessary row-by-row processing, use minimally logged operations where appropriate.
3. Spread bulk operations over time rather than running them in large single transactions.

```

-- Identify the heaviest log-generating operations currently running
SELECT
    r.session_id,
    r.status,
    r.command,
    r.wait_type,
    r.wait_time,
    r.total_elapsed_time,
    r.reads,
    r.writes,
    r.logical_reads,
    t.text AS QueryText,
    qp.query_plan
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
CROSS APPLY sys.dm_exec_query_plan(r.plan_handle) qp
WHERE r.wait_type = 'LOG_RATE_GOVERNOR'
ORDER BY r.total_elapsed_time DESC;

-- Historical log generation rate from ring buffer
-- This gives you a picture of log throughput over recent minutes
SELECT
    end_time,
    delta_log_bytes_used / 1048576.0 AS log_MB_generated,
    avg_log_rate_value / 1048576.0 AS log_rate_MB_per_sec
FROM sys.dm_db_resource_stats
ORDER BY end_time DESC;

```

Snapshot Isolation and Read Committed Snapshot

Hyperscale databases have RCSI (Read Committed Snapshot Isolation) enabled by default and it cannot be disabled. For on-prem DBAs who have spent years managing version store pressure on tempdb, this is actually welcome news — Hyperscale stores version records in the page server layer, not in tempdb. The version store does not create the tempdb pressure that causes headaches on busy on-prem instances.

However, if your application was written against a database with `READ_COMMITTED_SNAPSHOT OFF` and relies on blocking behavior for correctness (for example, using locks as a coordination mechanism between application processes), you need to verify that RCSI does not change the behavior. Applications that take explicit `UPDLOCK` hints or use `SERIALIZABLE` transactions are not affected. Applications that relied on reader-writer blocking as an implicit synchronization mechanism will behave differently.

Tempdb Is Still Shared on Standard Compute

Unlike on-prem SQL Server where each instance has its own tempdb, Azure SQL Database shares tempdb resources across databases on the same logical server. On standard vCore tiers this is per the Azure SQL infrastructure. This is not a Hyperscale-specific behavior, but it catches DBAs who come from dedicated instances. Heavy tempdb consumers (large sorts, hash joins, spill-heavy plans, excessive temp table use) can see unexpected tempdb pressure from neighboring databases.

Monitor tempdb usage and address spills in your query plans:

```

-- Identify queries spilling to tempdb on Hyperscale
SELECT TOP 20
    qs.total_spills / qs.execution_count AS avg_spills_per_exec,
    qs.total_spills,
    qs.execution_count,
    qs.total_elapsed_time / qs.execution_count / 1000 AS avg_elapsed_ms,
    SUBSTRING(st.text, (qs.statement_start_offset / 2) + 1,
        ((CASE qs.statement_end_offset
            WHEN -1 THEN DATALength(st.text)
            ELSE qs.statement_end_offset

```

```

        END - qs.statement_start_offset) / 2) + 1) AS QueryText
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
WHERE qs.total_spills > 0
ORDER BY qs.total_spills DESC;

```

Monitoring Hyperscale in Production

Standard Azure SQL monitoring applies to Hyperscale, with some additional dimensions. You should be collecting metrics from Azure Monitor, DMV-based monitoring from within the database, and query store data for workload trends.

Azure Monitor Metrics for Hyperscale

The metrics that matter most for Hyperscale, beyond the standard CPU and connection count, are:

- **log_write_percent** — how close you are to the log rate limit. Watch for sustained values above 80%.
- **storage_percent** — Hyperscale does not have a fixed storage cap in the traditional sense, but this metric reflects page server utilization.
- **xtp_storage_percent** — In-Memory OLTP storage (if you use memory-optimized tables).
- **dwu_consumption_percent** — effective compute consumption.
- **sessions_percent** — connection count relative to the limit for your tier.

```

# Pull Hyperscale-specific metrics via Azure CLI
az monitor metrics list \
  --resource
  ↪ "/subscriptions/<sub-id>/resourceGroups/rg-hyperscale-prod/providers/Microsoft.Sql/servers/sql-hs-prod-eastus2/data
  ↪ \
  --metric "log_write_percent,cpu_percent,physical_data_read_percent,sessions_percent" \
  --interval PT1M \
  --start-time "$(date -u -d '1 hour ago' '+%Y-%m-%dT%H:%MZ')" \
  --end-time "$(date -u '+%Y-%m-%dT%H:%MZ')" \
  --output table

# Set up an alert for log rate throttling
az monitor metrics alert create \
  --name "hs-log-rate-high" \
  --resource-group rg-hyperscale-prod \
  --scopes
  ↪ "/subscriptions/<sub-id>/resourceGroups/rg-hyperscale-prod/providers/Microsoft.Sql/servers/sql-hs-prod-eastus2/data
  ↪ \
  --condition "avg log_write_percent > 80" \
  --window-size 5m \
  --evaluation-frequency 1m \
  --action-group /subscriptions/<sub-id>
  ↪ /resourceGroups/rg-hyperscale-prod/providers/Microsoft.Insights/actionGroups/ag-dba-oncall \
  --description "Hyperscale log rate sustained above 80% - risk of LOG_RATE_GVERNOR throttling"

```

Query Store for Workload Trend Analysis

Query Store is enabled by default on Hyperscale and is your primary tool for identifying plan regressions, tracking query performance over time, and understanding whether a performance problem is from a plan change or a data volume change.

```

-- Configure Query Store for production Hyperscale workloads
-- The defaults are reasonable but you may want to increase retention
ALTER DATABASE [OrdersDB]

```

```

SET QUERY_STORE = ON (
    OPERATION_MODE = READ_WRITE,
    CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS = 90),
    DATA_FLUSH_INTERVAL_SECONDS = 900,
    INTERVAL_LENGTH_MINUTES = 15,
    MAX_STORAGE_SIZE_MB = 4096,
    QUERY_CAPTURE_MODE = AUTO,
    MAX_PLANS_PER_QUERY = 200,
    SIZE_BASED_CLEANUP_MODE = AUTO,
    WAIT_STATS_CAPTURE_MODE = ON
);
GO

-- Find queries with the highest average elapsed time in the last 24 hours
SELECT TOP 20
    q.query_id,
    qt.query_sql_text,
    rs.avg_duration / 1000.0 AS avg_elapsed_ms,
    rs.avg_cpu_time / 1000.0 AS avg_cpu_ms,
    rs.avg_logical_io_reads,
    rs.avg_physical_io_reads, -- High value here = page server fetches
    rs.count_executions,
    p.plan_id,
    p.last_force_failure_reason_desc
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
JOIN sys.query_store_runtime_stats_interval i ON rs.runtime_stats_interval_id =
    ⇨ i.runtime_stats_interval_id
WHERE i.start_time >= DATEADD(HOUR, -24, GETUTCDATE())
ORDER BY rs.avg_duration DESC;

-- Find plan regressions: queries that have gotten slower recently
WITH recent_stats AS (
    SELECT
        q.query_id,
        AVG(rs.avg_duration) AS recent_avg_duration,
        MAX(rs.avg_duration) AS recent_max_duration,
        SUM(rs.count_executions) AS recent_executions
    FROM sys.query_store_query q
    JOIN sys.query_store_plan p ON q.query_id = p.query_id
    JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
    JOIN sys.query_store_runtime_stats_interval i ON rs.runtime_stats_interval_id =
        ⇨ i.runtime_stats_interval_id
    WHERE i.start_time >= DATEADD(HOUR, -4, GETUTCDATE())
    GROUP BY q.query_id
),
baseline_stats AS (
    SELECT
        q.query_id,
        AVG(rs.avg_duration) AS baseline_avg_duration
    FROM sys.query_store_query q
    JOIN sys.query_store_plan p ON q.query_id = p.query_id
    JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
    JOIN sys.query_store_runtime_stats_interval i ON rs.runtime_stats_interval_id =
        ⇨ i.runtime_stats_interval_id
    WHERE i.start_time >= DATEADD(HOUR, -28, GETUTCDATE())
        AND i.start_time < DATEADD(HOUR, -4, GETUTCDATE())
    GROUP BY q.query_id
)
SELECT
    r.query_id,
    qt.query_sql_text,
    b.baseline_avg_duration / 1000.0 AS baseline_avg_ms,
    r.recent_avg_duration / 1000.0 AS recent_avg_ms,
    r.recent_avg_duration / NULLIF(b.baseline_avg_duration, 0) AS regression_ratio
FROM recent_stats r
JOIN baseline_stats b ON r.query_id = b.query_id
JOIN sys.query_store_query q ON r.query_id = q.query_id

```



```
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
WHERE r.recent_avg_duration > b.baseline_avg_duration * 2 -- 2x regression threshold
      AND r.recent_executions >= 10 -- Only queries with meaningful execution count
ORDER BY regression_ratio DESC;
```

Page Server Latency DMV

Hyperscale exposes a DMV specifically for page server operations that has no equivalent in standard Azure SQL tiers:

```
-- sys.dm_user_db_resource_governance includes Hyperscale-specific limits
-- For page server I/O characteristics, use this combination:
SELECT
    io_stall,
    io_stall_read_ms,
    io_stall_write_ms,
    num_of_reads,
    num_of_writes,
    num_of_bytes_read,
    num_of_bytes_written,
    io_stall_queued_read_ms, -- time spent waiting to issue reads
    io_stall_queued_write_ms -- time spent waiting to issue writes
FROM sys.dm_io_virtual_file_stats(DB_ID(), NULL)
ORDER BY io_stall DESC;

-- Hyperscale-specific: check page server fetch activity
-- This is available in sys.dm_exec_requests during active queries
SELECT
    r.session_id,
    r.wait_type,
    r.wait_time,
    r.page_server_reads,
    r.page_server_reads_in_progress,
    r.page_server_read_elapsed_ms,
    CASE
        WHEN r.page_server_reads > 0
        THEN r.page_server_read_elapsed_ms / r.page_server_reads
        ELSE 0
    END AS avg_page_server_read_ms,
    SUBSTRING(t.text, r.statement_start_offset / 2 + 1,
        (CASE r.statement_end_offset
            WHEN -1 THEN DATALENGTH(t.text)
            ELSE r.statement_end_offset
        END - r.statement_start_offset) / 2 + 1) AS current_statement
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.page_server_reads > 0
ORDER BY r.page_server_reads DESC;
```

If your `avg_page_server_read_ms` values are consistently above 5–10 ms for a heavily accessed query, it is a signal that your working set does not fit in the buffer pool and you would benefit from more memory — either by scaling vCores up or by adding a named replica that caches a subset of pages independently.

Migrating Out of Hyperscale

Because the tier change from Hyperscale back to another Azure SQL tier is not supported through the portal or API, you need a data movement strategy. This is not a frequent operation, but the scenarios where it arises include consolidation, cost reduction after a data archiving project that brought the database below 4 TB, or a strategic decision to move to a different database service entirely.

Export to BACPAC

BACPAC export is the simplest path for databases that are not extremely large. It exports the schema and data to an Azure Blob Storage file that can be imported into any Azure SQL tier.

```
# Export Hyperscale database to BACPAC
# Note: For very large databases, this can take many hours
# Monitor progress via the export operation status

az sql db export \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --name OrdersDB \
  --admin-user sqladmin \
  --admin-password "$(az keyvault secret show --vault-name kv-prod --name sql-admin-password --query
  ↪ value -o tsv)" \
  --storage-key "$(az storage account keys list --resource-group rg-hyperscale-prod --account-name
  ↪ storprodmigration --query '[0].value' -o tsv)" \
  --storage-key-type StorageAccessKey \
  --storage-uri "https://storprodmigration.blob.core.windows.net/exports/OrdersDB_export.bacpac"

# Monitor the export operation
az sql db op list \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --database OrdersDB \
  --query "[?operation=='ExportDatabase']" \
  --output table

# Import BACPAC to a new General Purpose database
az sql db import \
  --resource-group rg-target \
  --server sql-gp-prod \
  --name OrdersDB \
  --admin-user sqladmin \
  --admin-password "$(az keyvault secret show --vault-name kv-prod --name sql-admin-password --query
  ↪ value -o tsv)" \
  --storage-key "$(az storage account keys list --resource-group rg-hyperscale-prod --account-name
  ↪ storprodmigration --query '[0].value' -o tsv)" \
  --storage-key-type StorageAccessKey \
  --storage-uri "https://storprodmigration.blob.core.windows.net/exports/OrdersDB_export.bacpac" \
  --edition GeneralPurpose \
  --service-objective GP_Gen5_8
```

BACPAC export is not point-in-time consistent for active databases. If the database is still receiving writes during export, the resulting BACPAC may have a logically inconsistent state for tables that were modified while the export was running. For a clean migration out of Hyperscale, the safest approach is to quiesce writes during export or use the backup-and-restore path with a log backup applied at cutover.

Transactional Replication for Live Cutover

For large databases or workloads that cannot tolerate extended downtime, transactional replication from the Hyperscale source to a target database in a different tier provides continuous synchronization until cutover. You configure Hyperscale as the publisher, the target as the subscriber, let replication catch up, then flip the connection string and remove the replication topology.

This approach requires careful attention to replication topology setup, but it is the same replication mechanism that on-prem DBAs have used for decades. Hyperscale supports being both a publisher and a subscriber for transactional replication, with the same T-SQL configuration commands that work on-prem.

Cost Modeling: When Hyperscale Is the Right Economic Choice

Hyperscale is not always the cheapest choice even for large databases, and it is worth doing the arithmetic before committing.

The cost components for a Hyperscale database are:

1. **Compute:** vCore-hours for the primary compute replica.
2. **HA replicas:** Each HA replica is billed at the same vCore rate as the primary. One HA replica effectively doubles your compute cost.
3. **Named replicas:** Each named replica is billed at its own vCore rate.
4. **Storage:** Hyperscale storage pricing is per-GB per-month beyond the first 100 GB. Storage consumption grows with your database and does not shrink automatically when you delete data (pages are not immediately reclaimed from page servers after DELETE operations — space reclamation happens through index rebuilds and DBCC SHRINKFILE, which should be used judiciously).
5. **Backup storage:** Billed per GB-month beyond 1x the database size.
6. **Log service egress:** Generally included in the tier, but cross-region configurations may have network transfer costs.

The inflection point where Hyperscale becomes economically competitive with Business Critical is roughly at the point where:

- The database exceeds 4 TB (forced migration anyway for General Purpose or Business Critical)
- Or the named replica topology you need would require multiple Always On nodes that would each need to be Business Critical size

For databases below 2 TB with no read scale-out requirement and a stable, OLTP-dominant workload, Business Critical often delivers better price-performance because of local SSD and the tighter latency profile.

Do the vCore math with your actual workload size, your replica topology needs, and the current Azure SQL pricing for your region. Prices change, reserved instance discounts alter the comparison significantly, and the Dev/Test pricing for non-production Hyperscale instances reduces the cost of running staging and QA environments on the same tier as production.

Key Takeaways

- **Hyperscale is architecturally distinct, not just a storage upgrade.** Its decoupled compute, log service, and page server model gives you near-instant backup and restore at any size, but introduces cold-page network latency that does not exist in Business Critical. Understand the tradeoff before provisioning.
- **The migration into Hyperscale is one-directional through the service tier API.** You can migrate out, but only through data movement operations like BACPAC export, backup/restore, or transactional replication — not through a tier change. Make the decision with full awareness of this constraint.

- **Named replicas are a genuinely different read scale-out model.** Because they share the page server layer rather than maintaining full storage copies, you can add or remove them in minutes regardless of database size. This makes dynamic read scale-out practical in ways that traditional readable secondaries are not.
 - **LOG_RATE_GOVERNOR is your primary throughput ceiling indicator.** High LOG_RATE_GOVERNOR wait times mean your log generation rate exceeds what your vCore tier allows. The response is either to scale vCores up or to reduce log generation through batching and minimally logged patterns.
 - **Backup storage redundancy and RCSI-on behavior are set at creation and cannot be changed.** Validate your DR requirements and your application's isolation level assumptions before the first provisioning command runs — these are not settings you can tune after the fact without rebuilding the database.
-

Chapter 16: Active Geo-Replication and Failover Groups

Geographic replication in cloud databases is the production answer to a question on-prem DBAs have wrestled with for decades: how do you keep a copy of your data synchronized across data centers without building your own log shipping infrastructure, managing your own replication slots, or manually promoting standby servers at 2 AM? This chapter covers how AWS Aurora Global Database, Azure SQL Active Geo-Replication and Failover Groups, and Google Cloud Spanner multi-region configurations solve this problem — and where each approach has sharp edges you need to understand before you commit to them in production. We cover configuration, latency expectations, failover mechanics, RPO/RTO characteristics, and the automation patterns that separate a well-architected geo-redundant deployment from one that fails its first real disaster recovery test.

16.1 The Geography Problem: What You’re Actually Solving

Before touching any configuration, get the requirements straight. Geo-replication serves three distinct purposes that are often conflated:

Disaster recovery — a full regional outage requires a replica in another region to take over. RPO of seconds to low minutes is achievable. RTO depends heavily on whether failover is automatic or manual.

Read scaling — routing read traffic to replicas closer to users in other regions reduces latency without touching the primary. A user in Tokyo reading from a US-East primary adds 150–200ms of round-trip latency that disappears when queries route to an Asia-Pacific replica.

Compliance and data residency — some regulated industries require data to exist within specific geographic boundaries, but still need disaster recovery within that boundary. These are not the same as global replication and sometimes conflict with it.

On-prem, you would have built this with SQL Server’s Always On availability groups across stretched clusters, or PostgreSQL streaming replication with custom failover scripts on top of Pacemaker/Corosync. The managed cloud equivalent is less flexible in some ways — you cannot, for instance, tune the replication protocol itself — but it removes an enormous amount of operational surface area.

Replication Lag Is Inherent and Physical

The speed of light is not a cloud vendor problem. At the physical limit, New York to London is roughly 70ms round-trip. Synchronous replication across that distance would add 70ms minimum to every committed write. Every cloud vendor therefore makes the same decision: cross-region replication is asynchronous. This means RPO is not zero. Measure it. Know your actual lag before a disaster occurs.

Typical lag numbers in production:

Scenario	Expected Replication Lag
Aurora Global, same continent	1–3 seconds typical, <5s P99
Aurora Global, US to EU	3–8 seconds typical, <10s P99
Azure SQL Active Geo-Replication, paired regions	2–5 seconds typical
Azure SQL Active Geo-Replication, unpaired regions	5–15 seconds depending on write volume
Cloud Spanner multi-region	Sub-second within a continent configuration
Spanner global multi-region	5–10 seconds external consistency latency

These numbers change dramatically under write load. A database doing 50,000 writes per second with 8KB average row size is generating roughly 400MB/s of WAL. Moving that across a WAN link takes real bandwidth, and replication lag expands if the link cannot sustain throughput. Size your workload against these numbers, not just the idle case.

The general shape of a managed failover is the same across Aurora Global Database, Azure SQL Failover Groups, and Cloud Spanner, even though the underlying replication mechanism differs: applications connect through a stable endpoint, the primary replicates asynchronously (or, for Spanner, via synchronous quorum) to one or more secondary regions, and a failover event promotes a secondary while the endpoint layer redirects traffic.

16.2 Aurora Global Database: Architecture and Configuration

Aurora Global Database uses a storage-level replication mechanism rather than logical replication. The primary region writes to Aurora’s distributed storage, and those writes are replicated asynchronously at the storage layer to secondary regions. This is architecturally different from PostgreSQL streaming replication, where the primary WAL sender streams WAL records to a standby’s WAL receiver. Aurora Global bypasses the Aurora writer instance in secondary regions entirely — secondary regions have their own Aurora cluster, but that cluster’s storage receives writes directly from the primary’s storage nodes.

The practical implications:

- Secondary clusters are read-only at the SQL layer
- You cannot use logical replication slots on a secondary cluster
- Secondary clusters can have their own read replicas within the secondary region
- Each secondary region introduces one hop of latency: primary writes to storage, storage replicates to secondary region storage, secondary cluster reads from its regional storage

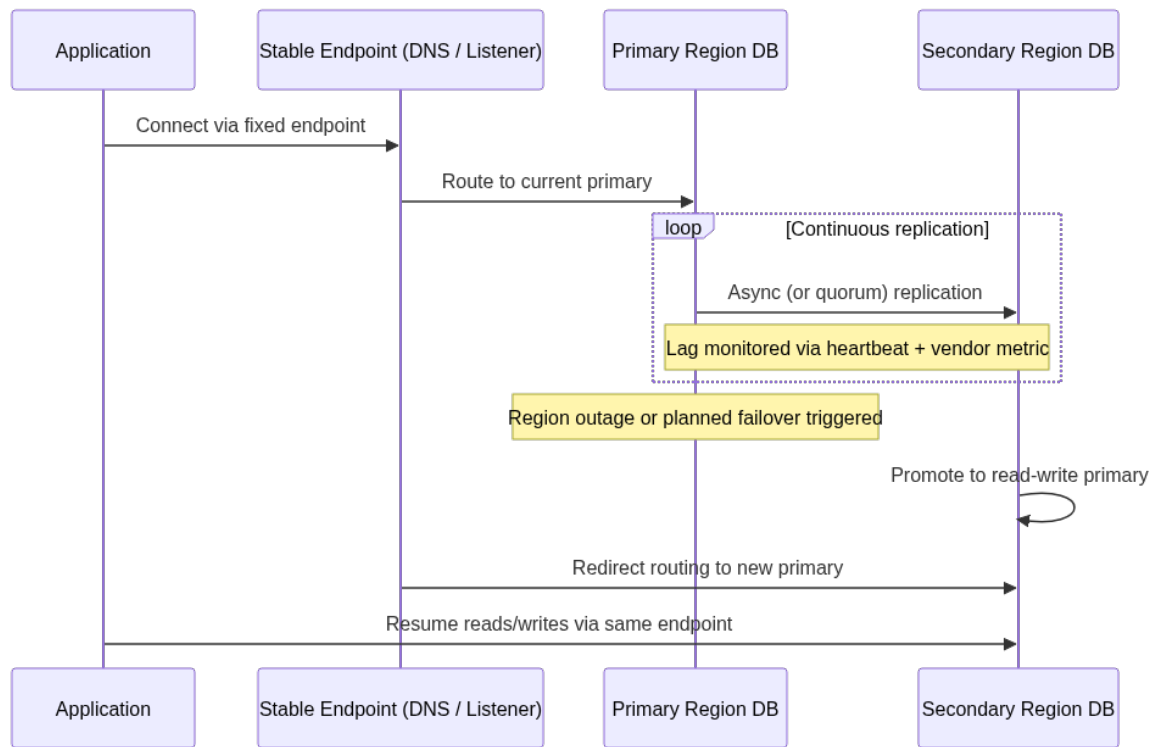


Figure 7: Generic failover topology shared across Aurora Global Database, Azure Failover Groups, and Spanner — the promotion and endpoint-redirection mechanics differ in detail per vendor, covered in the sections that follow.

Setting Up Aurora Global Database with AWS CLI

First, create the primary Aurora cluster if you do not already have one:

```
# Create the primary Aurora cluster in us-east-1
aws rds create-db-cluster \
  --db-cluster-identifier prod-global-primary \
  --engine aurora-postgresql \
  --engine-version 15.4 \
  --master-username pgadmin \
  --master-user-password "${DB_PASSWORD}" \
  --db-subnet-group-name prod-subnet-group \
  --vpc-security-group-ids sg-0abc123def456789 \
  --storage-encrypted \
  --kms-key-id arn:aws:kms:us-east-1:123456789012:key/mrk-abc123 \
  --region us-east-1

# Create the writer instance
aws rds create-db-instance \
  --db-instance-identifier prod-global-primary-writer \
  --db-cluster-identifier prod-global-primary \
  --db-instance-class db.r7g.4xlarge \
  --engine aurora-postgresql \
  --region us-east-1

# Create the Global Database wrapper (wraps the existing cluster)
aws rds create-global-cluster \
  --global-cluster-identifier prod-aurora-global \
  --source-db-cluster-identifier arn:aws:rds:us-east-1:123456789012:cluster:prod-global-primary \
  --region us-east-1
```

Adding a secondary region requires the secondary region to have its own VPC, subnet group, and KMS key. If you are using a customer-managed KMS key (which you should be in production), you need a multi-region key or separate regional keys. A common mistake is using a single-region KMS key and then being surprised when the secondary cluster creation fails.

```
# Add a secondary cluster in eu-west-1
aws rds create-db-cluster \
  --db-cluster-identifier prod-global-secondary-eu \
  --engine aurora-postgresql \
  --engine-version 15.4 \
  --global-cluster-identifier prod-aurora-global \
  --db-subnet-group-name prod-subnet-group-eu \
  --vpc-security-group-ids sg-0def456abc123789 \
  --storage-encrypted \
  --kms-key-id arn:aws:kms:eu-west-1:123456789012:key/mrk-def456 \
  --region eu-west-1

# Add a reader instance in the secondary cluster
aws rds create-db-instance \
  --db-instance-identifier prod-global-secondary-eu-reader \
  --db-cluster-identifier prod-global-secondary-eu \
  --db-instance-class db.r7g.2xlarge \
  --engine aurora-postgresql \
  --region eu-west-1
```

Monitoring Replication Lag with CloudWatch

Aurora Global exposes the `AuroraGlobalDBReplicationLag` metric in CloudWatch. Set an alarm on this. A reasonable alert threshold for most OLTP workloads is 5 seconds; a page-worthy threshold is 30 seconds.

```
# Create a CloudWatch alarm for Global DB replication lag
aws cloudwatch put-metric-alarm \
  --alarm-name aurora-global-replication-lag-warning \
  --alarm-description "Aurora Global replication lag exceeds 5 seconds" \
  --metric-name AuroraGlobalDBReplicationLag \
```



```
--namespace AWS/RDS \  
--statistic Maximum \  
--period 60 \  
--evaluation-periods 3 \  
--threshold 5000 \  
--comparison-operator GreaterThanThreshold \  
--dimensions Name=DBClusterIdentifier,Value=prod-global-secondary-eu \  
--alarm-actions arn:aws:sns:eu-west-1:123456789012:dba-alerts \  
--treat-missing-data notBreaching \  
--region eu-west-1
```

Failover Behavior and What to Expect

Aurora Global failover is not automatic by default. There is no built-in equivalent to an Always On availability group's automatic failover with a failure detection timeout. You must either trigger it manually or build automation around it.

When you initiate a managed failover (the preferred path that minimizes data loss), Aurora promotes the secondary cluster in the target region and demotes the primary. The process takes 1–2 minutes and involves:

1. Stopping writes on the primary
2. Waiting for the secondary to apply all pending replication entries
3. Promoting the secondary to read-write
4. Updating the Global Database configuration to reflect the new primary

If you use an unplanned failover (the primary region is completely unavailable), you detach the secondary cluster from the Global Database and promote it independently. In this scenario, the data in-flight that had not yet replicated is lost — that is your actual RPO.

Your application connection string must change after failover. Aurora does not automatically redirect the cluster endpoint. Applications pointing at the old primary region's cluster endpoint will fail until you update DNS or connection strings. This is a common operational gap. Solve it with Route53 health checks and a CNAME that points to whichever region is primary, or use a global accelerator pattern.

16.3 Azure SQL Active Geo-Replication and Failover Groups

Azure SQL offers two related but distinct features. Active Geo-Replication is the lower-level primitive: it creates readable secondary databases in up to four other regions, with asynchronous replication at the database level. Failover Groups are the higher-level abstraction: they manage one or more databases together, provide a stable listener endpoint that does not change after failover, and support automatic failover through a health-check based policy.

If you are building a production application, use Failover Groups. Active Geo-Replication alone requires you to update connection strings after failover. Failover Groups give you a fixed endpoint — `<group-name>.database.windows.net` for the primary and `<group-name>.secondary.database.windows.net` for the secondary — that routes to whichever server is currently primary.

Creating a Failover Group with Azure CLI

```
# Variables  
PRIMARY_SERVER="prod-sql-eastus"
```

```

SECONDARY_SERVER="prod-sql-westeu"
RESOURCE_GROUP="prod-rg"
FAILOVER_GROUP="prod-fog-01"
DATABASE_NAME="applicationdb"

# Create the secondary server in West Europe
az sql server create \
  --name $SECONDARY_SERVER \
  --resource-group $RESOURCE_GROUP \
  --location "westeurope" \
  --admin-user sqladmin \
  --admin-password "${SQL_ADMIN_PASSWORD}"

# Configure firewall rules and private endpoints on secondary
# (match whatever you have on primary)
az sql server firewall-rule create \
  --resource-group $RESOURCE_GROUP \
  --server $SECONDARY_SERVER \
  --name AllowAzureServices \
  --start-ip-address 0.0.0.0 \
  --end-ip-address 0.0.0.0

# Create the failover group
# This initiates geo-replication of all specified databases
az sql failover-group create \
  --name $FAILOVER_GROUP \
  --partner-server $SECONDARY_SERVER \
  --resource-group $RESOURCE_GROUP \
  --server $PRIMARY_SERVER \
  --add-db $DATABASE_NAME \
  --failover-policy Automatic \
  --grace-period 1

# The grace-period is in hours - 1 hour before automatic failover triggers
# Setting to 0 means immediate failover, which risks unnecessary failovers
# during transient connectivity issues between regions

```

The `--grace-period` parameter deserves attention. Azure uses a health-check mechanism that monitors connectivity between the primary and secondary. If the primary is unreachable for longer than the grace period, automatic failover triggers. Setting this to 0 — immediate failover — risks data loss and unnecessary failovers if there are transient network issues between Azure regions that do not represent actual primary failure. For most production workloads, 1 hour is the recommended minimum, and many organizations use 2–4 hours depending on their RPO tolerance.

Monitoring Geo-Replication Health

Azure SQL exposes geo-replication state through `sys.dm_geo_replication_link_states` in the primary database:

```

-- Run on the PRIMARY database
-- Check replication health for all geo-replicas
SELECT
  link_guid,
  partner_server,
  partner_database,
  last_replication,
  replication_lag_sec,
  replication_state_desc,
  role_desc,
  secondary_allow_connections_desc
FROM sys.dm_geo_replication_link_states
ORDER BY replication_lag_sec DESC;

-- Practical alert query: lag over 30 seconds is worth investigating
-- lag over 300 seconds (5 minutes) warrants a page
SELECT

```

```
partner_server,  
partner_database,  
replication_lag_sec,  
last_replication,  
CASE  
    WHEN replication_lag_sec > 300 THEN 'CRITICAL'  
    WHEN replication_lag_sec > 30 THEN 'WARNING'  
    ELSE 'OK'  
END AS lag_status  
FROM sys.dm_geo_replication_link_states  
WHERE role_desc = 'PRIMARY';
```

On the secondary, you can also query `sys.dm_database_replica_states` to see how far behind the secondary is in terms of log send queue and redo queue — the same DMVs you know from Always On, repurposed for geo-replication context.

Failover Group Listener and Application Connection Patterns

The Failover Group listener endpoint is where your application connection string points. After any failover — planned or unplanned — your application continues to connect to `prod-fog-01.database.windows.net` without any changes. Azure’s DNS resolves this to whichever server is currently primary.

```
# Initiate a planned failover (controlled, no data loss)  
# Run this for maintenance or DR drills  
az sql failover-group set-primary \  
--name $FAILOVER_GROUP \  
--resource-group $RESOURCE_GROUP \  
--server $SECONDARY_SERVER  
  
# Check failover group status after failover  
az sql failover-group show \  
--name $FAILOVER_GROUP \  
--resource-group $RESOURCE_GROUP \  
--server $SECONDARY_SERVER \  
--query "{primaryServer:replicationRole, location:location, databases:databases}"
```

One constraint that catches DBAs migrating from Always On: Failover Groups in Azure SQL Database (the fully managed PaaS tier) are limited to two servers — one primary and one secondary. You cannot have two readable secondaries in two different regions plus your primary in a single Failover Group. If you need multiple secondary regions, you use Active Geo-Replication directly and accept that you manage connection string routing yourself. SQL Managed Instance supports up to four regions in a Failover Group, which gives more flexibility.

Hyperscale Geo-Replication Differences

If you are using Azure SQL Hyperscale — the tier that decouples compute from storage and supports up to 100TB databases — geo-replication works differently. Hyperscale uses Named Replicas and Geo-Replicas as distinct concepts. A Geo-Replica in Hyperscale is seeded from the distributed page server layer rather than through log shipping, which means the initial seeding is extremely fast even for multi-terabyte databases — often measured in minutes rather than the hours traditional log shipping takes to initialize a large secondary.

The tradeoff: Hyperscale Geo-Replicas have higher replication lag than standard geo-replication at high write throughput. If your Hyperscale workload generates more than roughly 100MB/s of log, expect lag to grow during peak write periods.

16.4 Google Cloud Spanner and Cloud SQL: Multi-Region Strategies

Google Cloud offers two fundamentally different architectures for geo-redundancy, and choosing between them depends on whether you need strong consistency or are willing to tolerate eventual consistency for the benefit of scale.

Cloud Spanner Multi-Region Configurations

Spanner is the only globally distributed, strongly consistent, relational database available as a managed service. This is not marketing — it is an architectural statement with real implications. Spanner uses the TrueTime API, which relies on atomic clocks and GPS receivers in Google’s data centers to provide bounded clock uncertainty and enable external consistency across regions.

When you configure a Spanner instance with a multi-region configuration, writes are synchronously committed to a quorum of regions before returning success to the application. This means you get zero RPO under region failure — but you pay for it with higher write latency.

Spanner’s regional configurations use naming conventions that encode the topology:

- **nam4** — dual-region, US East and US Central, optimized for low-latency writes in the Eastern US
- **eur3** — dual-region, Netherlands and Belgium
- **nam-eur-asia1** — multi-region spanning North America, Europe, and Asia (five regions total)
- **nam6** — six North American regions

```
# Create a Spanner instance with multi-region configuration
gcloud spanner instances create prod-spanner-global \
  --config=nam-eur-asia1 \
  --description="Production global instance" \
  --nodes=3 \
  --project=prod-project-id

# Multi-region configs replicate compute across every region in the
# configuration, so a production-grade node count (3+ for quorum headroom)
# costs meaningfully more than the same node count in a single-region
# instance. Check current Spanner pricing and the specific replica
# topology for your chosen multi-region config before committing to it.

# Check instance replication details
gcloud spanner instances describe prod-spanner-global \
  --project=prod-project-id \
  --format="yaml(config,displayName,nodeCount,state)"

# Create a database in the multi-region instance
gcloud spanner databases create applicationdb \
  --instance=prod-spanner-global \
  --project=prod-project-id
```

The write latency penalty for global multi-region Spanner is real. For **nam-eur-asia1**, a write commit from a US East application needs to reach quorum across regions spanning multiple continents, and that commit latency is an order of magnitude higher than a single-region Spanner instance. Benchmark your own commit latency against your specific region pair before committing to a topology — published figures vary by configuration and change over time — and applications built for single-digit-millisecond write latency will need to reconsider whether global Spanner is the right tool.

Cloud SQL for PostgreSQL: Multi-Region Read Replicas

Cloud SQL does not offer Spanner’s synchronous multi-region consistency model. Instead, it offers cross-region read replicas backed by asynchronous replication, similar to Aurora Global. The replication mechanism is standard PostgreSQL streaming replication — the same process you know from on-prem setups, managed by Google’s control plane.

```
# Create a Cloud SQL primary instance
gcloud sql instances create prod-postgres-primary \
  --database-version=POSTGRES_15 \
  --tier=db-custom-16-65536 \
  --region=us-central1 \
  --storage-type=SSD \
  --storage-size=500GB \
  --storage-auto-increase \
  --backup-start-time=04:00 \
  --enable-bin-log \
  --availability-type=REGIONAL \
  --project=prod-project-id

# Create a cross-region read replica in europe-west1
gcloud sql instances create prod-postgres-replica-eu \
  --master-instance-name=prod-postgres-primary \
  --region=europe-west1 \
  --tier=db-custom-8-32768 \
  --database-version=POSTGRES_15 \
  --project=prod-project-id

# Check replication lag using Cloud SQL's built-in monitoring
gcloud sql instances describe prod-postgres-replica-eu \
  --project=prod-project-id \
  --format="yaml(replicaConfiguration,replicationLagSeconds)"
```

Cloud SQL cross-region replicas cannot be automatically promoted on regional failure. Promotion is a manual or scripted action. Unlike Aurora Global, there is no managed failover path that handles orchestration for you. You must:

1. Promote the replica to a standalone instance (which breaks replication)
2. Update your application’s connection string to point to the promoted instance
3. Once the primary region recovers, recreate the replica relationship from the newly promoted instance back toward the old primary, or rebuild from scratch

```
# Promote a cross-region replica after a regional outage
# This is a one-way operation - there is no "unpromote"
gcloud sql instances promote-replica prod-postgres-replica-eu \
  --project=prod-project-id

# After promotion, the instance becomes read-write standalone
# Verify it is now accepting writes
gcloud sql instances describe prod-postgres-replica-eu \
  --project=prod-project-id \
  --format="yaml(instanceType,region,state)"
# instanceType should now be "CLOUD_SQL_INSTANCE" not "READ_REPLICA_INSTANCE"
```

AlloyDB Cross-Region Considerations

AlloyDB, Google’s PostgreSQL-compatible high-performance managed database, as of 2024 supports cross-region replication through AlloyDB Omni (the self-managed distribution) and is rolling out managed cross-region secondary clusters. For production deployments requiring geo-redundancy on AlloyDB, verify the current regional availability of cross-region secondary clusters against the GCP documentation, as this feature has been in preview with limited region pairs. The architecture follows a similar storage-layer replication model to Aurora Global rather than WAL-stream replication.

16.5 Terraform Patterns for Multi-Region Database Infrastructure

Managing geo-replicated database infrastructure with manual CLI commands is viable for initial setup and DR drills. It is not viable for maintaining consistency across environments, tracking configuration drift, or onboarding new regions. Terraform brings the same discipline to database infrastructure that you would apply to application infrastructure.

The following Terraform module provisions an Aurora Global Database with a primary in us-east-1 and a secondary in eu-west-1, including CloudWatch alarms and the necessary KMS multi-region keys.

```
# aurora_global.tf
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# Provider aliases for multi-region
provider "aws" {
  alias = "primary"
  region = "us-east-1"
}

provider "aws" {
  alias = "secondary"
  region = "eu-west-1"
}

# Multi-region KMS key for Aurora storage encryption
resource "aws_kms_key" "aurora_global" {
  provider = aws.primary
  description = "Aurora Global Database encryption key"
  enable_key_rotation = true
  multi_region = true
  deletion_window_in_days = 30

  tags = {
    Environment = "production"
    ManagedBy = "terraform"
  }
}

# Replicate KMS key to secondary region
resource "aws_kms_replica_key" "aurora_global_eu" {
  provider = aws.secondary
  description = "Aurora Global Database encryption key replica (EU)"
  primary_key_arn = aws_kms_key.aurora_global.arn
}

# Primary Aurora cluster
resource "aws_rds_cluster" "primary" {
  provider = aws.primary
  cluster_identifier = "prod-global-primary"
  engine = "aurora-postgresql"
  engine_version = "15.4"
  master_username = "pgadmin"
  master_password = var.db_master_password
  db_subnet_group_name = var.primary_subnet_group
  vpc_security_group_ids = var.primary_security_groups
}
```

```

storage_encrypted      = true
kms_key_id             = aws_kms_key.aurora_global.arn
backup_retention_period = 14
preferred_backup_window = "03:00-04:00"
preferred_maintenance_window = "sun:05:00-sun:06:00"
deletion_protection    = true
skip_final_snapshot    = false
final_snapshot_identifier = "prod-global-primary-final"

tags = {
  Environment = "production"
  ManagedBy   = "terraform"
  Role        = "global-primary"
}
}

resource "aws_rds_cluster_instance" "primary_writer" {
  provider          = aws.primary
  identifier         = "prod-global-primary-writer"
  cluster_identifier = aws_rds_cluster.primary.id
  instance_class     = "db.r7g.4xlarge"
  engine             = aws_rds_cluster.primary.engine
  engine_version     = aws_rds_cluster.primary.engine_version
  db_subnet_group_name = var.primary_subnet_group
  monitoring_interval = 60
  monitoring_role_arn = var.rds_monitoring_role_arn

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
  }
}

# Global Database wrapper
resource "aws_rds_global_cluster" "this" {
  provider          = aws.primary
  global_cluster_identifier = "prod-aurora-global"
  source_db_cluster_identifier = aws_rds_cluster.primary.arn
  force_destroy      = false

  depends_on = [aws_rds_cluster_instance.primary_writer]
}

# Secondary Aurora cluster in eu-west-1
resource "aws_rds_cluster" "secondary" {
  provider          = aws.secondary
  cluster_identifier = "prod-global-secondary-eu"
  engine             = "aurora-postgresql"
  engine_version     = "15.4"
  global_cluster_identifier = aws_rds_global_cluster.this.id
  db_subnet_group_name = var.secondary_subnet_group
  vpc_security_group_ids = var.secondary_security_groups
  storage_encrypted    = true
  kms_key_id           = aws_kms_replica_key.aurora_global_eu.arn
  deletion_protection  = true
  skip_final_snapshot  = false
  final_snapshot_identifier = "prod-global-secondary-eu-final"

  # Secondary clusters must omit master credentials -
  # they are inherited from the primary through the Global Database
  lifecycle {
    ignore_changes = [master_password, master_username]
  }

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
    Role        = "global-secondary"
  }
}

```

```

resource "aws_rds_cluster_instance" "secondary_reader" {
  provider      = aws.secondary
  identifier     = "prod-global-secondary-eu-reader"
  cluster_identifier = aws_rds_cluster.secondary.id
  instance_class = "db.r7g.2xlarge"
  engine         = aws_rds_cluster.secondary.engine
  engine_version = aws_rds_cluster.secondary.engine_version
  db_subnet_group_name = var.secondary_subnet_group
  monitoring_interval = 60
  monitoring_role_arn = var.rds_secondary_monitoring_role_arn

  depends_on = [aws_rds_global_cluster.this]

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
  }
}

# CloudWatch alarm for replication lag in secondary region
resource "aws_cloudwatch_metric_alarm" "replication_lag_warning" {
  provider      = aws.secondary
  alarm_name     = "aurora-global-replication-lag-warning"
  alarm_description = "Aurora Global replication lag exceeds 5 seconds"
  metric_name    = "AuroraGlobalDBReplicationLag"
  namespace      = "AWS/RDS"
  statistic      = "Maximum"
  period         = 60
  evaluation_periods = 3
  threshold      = 5000
  comparison_operator = "GreaterThanThreshold"
  treat_missing_data = "notBreaching"
  alarm_actions    = [var.secondary_sns_topic_arn]

  dimensions = {
    DBClusterIdentifier = aws_rds_cluster.secondary.cluster_identifier
  }
}

resource "aws_cloudwatch_metric_alarm" "replication_lag_critical" {
  provider      = aws.secondary
  alarm_name     = "aurora-global-replication-lag-critical"
  alarm_description = "Aurora Global replication lag exceeds 30 seconds - page on-call"
  metric_name    = "AuroraGlobalDBReplicationLag"
  namespace      = "AWS/RDS"
  statistic      = "Maximum"
  period         = 60
  evaluation_periods = 2
  threshold      = 30000
  comparison_operator = "GreaterThanThreshold"
  treat_missing_data = "notBreaching"
  alarm_actions    = [var.secondaryPagerduty_sns_arn]

  dimensions = {
    DBClusterIdentifier = aws_rds_cluster.secondary.cluster_identifier
  }
}

```

Variables and separation of concerns

Keep the multi-region module clean by separating variables that differ per region from those that are shared. A common pattern is a `variables.tf` file at the module level with clear defaults and descriptions:

```

# variables.tf
variable "db_master_password" {
  description = "Master password for the Aurora cluster. Source from AWS Secrets Manager in CI/CD."
  type        = string
}

```



```
sensitive = true
}

variable "primary_subnet_group" {
  description = "DB subnet group name in the primary region (us-east-1)."
  type        = string
}

variable "secondary_subnet_group" {
  description = "DB subnet group name in the secondary region (eu-west-1)."
  type        = string
}

variable "primary_security_groups" {
  description = "Security group IDs attached to the primary Aurora cluster."
  type        = list(string)
}

variable "secondary_security_groups" {
  description = "Security group IDs attached to the secondary Aurora cluster."
  type        = list(string)
}

variable "rds_monitoring_role_arn" {
  description = "IAM role ARN for enhanced monitoring on the primary cluster."
  type        = string
}

variable "rds_secondary_monitoring_role_arn" {
  description = "IAM role ARN for enhanced monitoring on the secondary cluster."
  type        = string
}

variable "secondary_sns_topic_arn" {
  description = "SNS topic ARN in the secondary region for warning-level alarms."
  type        = string
}

variable "secondary_pagerduty_sns_arn" {
  description = "SNS topic ARN in the secondary region that routes to PagerDuty for critical alarms."
  type        = string
}
```

Equivalent Azure Terraform for Failover Groups

```
# azure_failover_group.tf
terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.80"
    }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "prod" {
  name     = "prod-rg"
  location = "East US"
}

resource "azurerm_mssql_server" "primary" {
  name                        = "prod-sql-eastus"
  resource_group_name        = azurerm_resource_group.prod.name
  location                   = "eastus"
  version                    = "12.0"
  administrator_login        = "sqladmin"
}
```

```

administrator_login_password = var.sql_admin_password

tags = {
  Environment = "production"
  ManagedBy   = "terraform"
  Role        = "primary"
}
}

resource "azurerm_mssql_server" "secondary" {
  name                        = "prod-sql-westeu"
  resource_group_name        = azurerm_resource_group.prod.name
  location                   = "westeurope"
  version                    = "12.0"
  administrator_login        = "sqladmin"
  administrator_login_password = var.sql_admin_password

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
    Role        = "secondary"
  }
}

resource "azurerm_mssql_database" "app_db" {
  name            = "applicationdb"
  server_id       = azurerm_mssql_server.primary.id
  sku_name        = "GP_Gen5_8"
  zone_redundant = true

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
  }
}

resource "azurerm_mssql_failover_group" "prod" {
  name      = "prod-fog-01"
  server_id = azurerm_mssql_server.primary.id
  databases = [azurerm_mssql_database.app_db.id]

  partner_server {
    id = azurerm_mssql_server.secondary.id
  }

  read_write_endpoint_failover_policy {
    mode          = "Automatic"
    grace_minutes = 60
  }

  readonly_endpoint_failover_policy_enabled = true

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
  }
}

# Azure Monitor alert for geo-replication lag
# Azure SQL exposes geo-replication metrics through Azure Monitor
resource "azurerm_monitor_metric_alert" "geo_replication_lag" {
  name            = "sql-geo-replication-lag-critical"
  resource_group_name = azurerm_resource_group.prod.name
  scopes          = [azurerm_mssql_database.app_db.id]
  description     = "Geo-replication lag exceeds threshold"
  severity        = 1
  frequency       = "PT1M"
  window_size     = "PT5M"

  criteria {

```

```

metric_namespace = "Microsoft.Sql/servers/databases"
metric_name       = "replication_lag"
aggregation       = "Maximum"
operator          = "GreaterThan"
threshold         = 30
}

action {
  action_group_id = var.dba_action_group_id
}
}

```

One detail worth emphasizing in the Azure module: the `readonly_endpoint_failover_policy_enabled = true` setting ensures that the secondary listener (`prod-fog-01.secondary.database.windows.net`) also follows the failover. Without this, after a failover the read-only endpoint continues pointing at the old secondary, which is now the primary — routing all read traffic to your write endpoint, eliminating any read scaling benefit.

16.6 Failover Automation and DR Drill Patterns

Geo-replication that has never been tested has unknown reliability. A configuration that looks correct in Terraform output and passes all health checks in CloudWatch can still fail a real DR event due to network ACL mismatches in the secondary VPC, missing IAM permissions for the promoted cluster, applications that cached the primary endpoint at startup, or secrets that exist only in the primary region's Secrets Manager.

The only way to discover these problems before a real disaster is to run regular, disruptive DR drills. Disruptive means you actually fail over, not that you simulate what would happen.

Aurora Global Failover Script

The following script performs a managed failover for Aurora Global and validates the secondary becomes the new primary. Run this quarterly at minimum, more frequently if your RTO or RPO requirements are strict.

```

#!/usr/bin/env bash
# aurora_global_failover_drill.sh
# Performs a planned managed failover of Aurora Global Database
# and validates the new primary is accepting writes.
set -euo pipefail

GLOBAL_CLUSTER_ID="prod-aurora-global"
SECONDARY_CLUSTER_ARN="arn:aws:rds:eu-west-1:123456789012:cluster:prod-global-secondary-eu"
PRIMARY_REGION="us-east-1"
SECONDARY_REGION="eu-west-1"
DRILL_LOG="failover_drill_$(date +%Y%m%d_%H%M%S).log"

log() {
  echo "[$(date -u '+%Y-%m-%dT%H:%M:%SZ')] $*" | tee -a "$DRILL_LOG"
}

log "=== Aurora Global Failover Drill Starting ==="
log "Global Cluster: $GLOBAL_CLUSTER_ID"
log "Target Region: $SECONDARY_REGION"

# Capture pre-failover replication lag
PRE_LAG=$(aws cloudwatch get-metric-statistics \
  --region "$SECONDARY_REGION" \
  --namespace AWS/RDS \

```

```

--metric-name AuroraGlobalDBReplicationLag \
--dimensions Name=DBClusterIdentifier,Value=prod-global-secondary-eu \
--start-time "$(date -u -d '5 minutes ago' '+%Y-%m-%dT%H:%M:%SZ')\" \
--end-time "$(date -u '+%Y-%m-%dT%H:%M:%SZ')\" \
--period 300 \
--statistics Maximum \
--query 'Datapoints[0].Maximum' \
--output text 2>/dev/null || echo "N/A")

log "Pre-failover replication lag (max 5min): ${PRE_LAG}ms"

if [[ "$PRE_LAG" != "N/A" ]] && (( $(echo "$PRE_LAG > 10000" | bc -l) )); then
    log "WARNING: Replication lag is ${PRE_LAG}ms - proceeding but RPO may be elevated"
fi

FAILOVER_START=$(date +%s)

log "Initiating managed planned failover..."
aws rds failover-global-cluster \
    --global-cluster-identifier "$GLOBAL_CLUSTER_ID" \
    --target-db-cluster-identifier "$SECONDARY_CLUSTER_ARN" \
    --region "$PRIMARY_REGION"

log "Failover initiated. Polling for completion..."

# Poll until the secondary cluster shows as writer
MAX_WAIT=300 # 5 minutes
ELAPSED=0
while [[ $ELAPSED -lt $MAX_WAIT ]]; do
    CLUSTER_STATUS=$(aws rds describe-db-clusters \
        --db-cluster-identifier prod-global-secondary-eu \
        --region "$SECONDARY_REGION" \
        --query 'DBClusters[0].{Status:Status,IAMAuth:IAMDatabaseAuthenticationEnabled}' \
        --output json 2>/dev/null || echo '{"Status":"unknown"}')

    GLOBAL_STATUS=$(aws rds describe-global-clusters \
        --global-cluster-identifier "$GLOBAL_CLUSTER_ID" \
        --region "$PRIMARY_REGION" \
        --query
        ↪ 'GlobalClusters[0].GlobalClusterMembers[?contains(DBClusterArn,`eu-west-1`)].{Writer:IsWriter}'
        ↪ \
        --output text 2>/dev/null || echo "False")

    if [[ "$GLOBAL_STATUS" == "True" ]]; then
        FAILOVER_END=$(date +%s)
        FAILOVER_DURATION=$((FAILOVER_END - FAILOVER_START))
        log "Failover completed in ${FAILOVER_DURATION} seconds"
        break
    fi

    sleep 10
    ELAPSED=$((ELAPSED + 10))
    log "Waiting... ${ELAPSED}s elapsed. EU writer status: $GLOBAL_STATUS"
done

if [[ $ELAPSED -ge $MAX_WAIT ]]; then
    log "ERROR: Failover did not complete within ${MAX_WAIT} seconds"
    exit 1
fi

# Validate write access on the new primary
log "Validating write access on new primary (eu-west-1)..."
# Replace with actual connection test appropriate to your environment
WRITE_TEST=$(psql \
    --host=prod-global-secondary-eu.cluster-xxxxxxxxxx.eu-west-1.rds.amazonaws.com \
    --username=pgadmin \
    --dbname=applicationdb \
    --no-password \
    --command="SELECT pg_is_in_recovery(), current_timestamp, inet_server_addr();" \
    --tuples-only 2>/dev/null || echo "CONNECTION_FAILED")

```

```
log "Write validation result: $WRITE_TEST"

if echo "$WRITE_TEST" | grep -q "f|"; then
  log "SUCCESS: New primary is accepting writes (pg_is_in_recovery = false)"
else
  log "WARN: Could not confirm write access - check connection string and security groups"
fi

log "=== DR Drill Complete. Review $DRILL_LOG for full record. ==="
log "ACTION REQUIRED: Update application connection string if not using Route53/CNAME abstraction."
log "ACTION REQUIRED: Fail back to us-east-1 after drill is complete."
```

After the drill, record the actual failover duration. Compare it against your RTO target. If your RTO is five minutes and the failover took 90 seconds, you have margin. If failover took four minutes and 15 seconds, you have a problem at any write volume spike, and you should investigate the replication lag at the time of the drill.

Failback After a Drill

Failback is the reverse operation: re-adding the old primary region as a secondary and then promoting it back to primary. After a managed failover, the original primary cluster becomes a secondary. It receives replication from the new primary in eu-west-1. Once replication lag drains and the old-primary cluster is caught up, you perform another managed failover to restore the original topology.

```
# After drill: check that original primary (now secondary) is caught up
aws cloudwatch get-metric-statistics \
  --region us-east-1 \
  --namespace AWS/RDS \
  --metric-name AuroraGlobalDBReplicationLag \
  --dimensions Name=DBClusterIdentifier,Value=prod-global-primary \
  --start-time "$(date -u -d '5 minutes ago' '+%Y-%m-%dT%H:%M:%SZ')" \
  --end-time "$(date -u '+%Y-%m-%dT%H:%M:%SZ')" \
  --period 60 \
  --statistics Maximum \
  --output text

# Once lag is acceptable (<5 seconds), fail back
aws rds failover-global-cluster \
  --global-cluster-identifier prod-aurora-global \
  --target-db-cluster-identifier arn:aws:rds:us-east-1:123456789012:cluster:prod-global-primary \
  --region eu-west-1
```

16.7 RPO Measurement and Continuous Validation

Publishing an RPO figure in a runbook without measuring it continuously is a compliance exercise, not an engineering practice. Replication lag changes under load, during maintenance windows, during storage rebalancing events, and during transient network events between regions. The only RPO figure that means something is the one you measure continuously from production.

PostgreSQL-Compatible RPO Tracking

For Aurora PostgreSQL or Cloud SQL for PostgreSQL, you can measure actual replication delay from the application perspective by writing a heartbeat record and measuring how long it takes to appear on the replica.

```
-- On the PRIMARY: Create a replication heartbeat table
-- Run once during setup
```

```

CREATE TABLE IF NOT EXISTS replication_heartbeat (
    id          SERIAL PRIMARY KEY,
    written_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    source      TEXT NOT NULL DEFAULT 'primary',
    sequence    BIGINT NOT NULL DEFAULT 0
);

CREATE INDEX IF NOT EXISTS idx_heartbeat_written_at
ON replication_heartbeat (written_at DESC);

-- Grant a monitoring user access
GRANT INSERT, SELECT ON replication_heartbeat TO monitoring_user;
GRANT USAGE, SELECT ON SEQUENCE replication_heartbeat_id_seq TO monitoring_user;

-- Heartbeat writer - run on primary every 10 seconds from a monitoring job
-- Use pg_cron, a Lambda, or a lightweight sidecar process
INSERT INTO replication_heartbeat (written_at, source, sequence)
SELECT
    NOW(),
    'primary',
    COALESCE(MAX(sequence), 0) + 1
FROM replication_heartbeat;

-- Keep the table small: retain only the last 24 hours
DELETE FROM replication_heartbeat
WHERE written_at < NOW() - INTERVAL '24 hours';

-- Heartbeat reader - run on REPLICA every 30 seconds from a monitoring job
-- Reports the actual observed RPO: how old is the newest heartbeat we can read?
SELECT
    MAX(written_at)                AS latest_heartbeat_on_replica,
    NOW()                         AS replica_clock,
    EXTRACT(EPOCH FROM (NOW() - MAX(written_at))) AS observed_rpo_seconds,
    CASE
        WHEN EXTRACT(EPOCH FROM (NOW() - MAX(written_at))) > 300 THEN 'CRITICAL'
        WHEN EXTRACT(EPOCH FROM (NOW() - MAX(written_at))) > 30  THEN 'WARNING'
        ELSE 'OK'
    END AS rpo_status
FROM replication_heartbeat;

```

This query runs on the replica. If the newest heartbeat row on the replica is 45 seconds old, your observed RPO at this moment is 45 seconds. That is more meaningful than CloudWatch's `AuroraGlobalDBReplicationLag` metric, which measures WAL application lag — not the time-to-visibility from an application query perspective.

SQL Server Equivalent: Azure SQL Geo-Replica RPO Tracking

```

-- On PRIMARY: Create heartbeat table
CREATE TABLE dbo.ReplicationHeartbeat (
    HeartbeatId INT IDENTITY(1,1) PRIMARY KEY,
    WrittenAt   DATETIMEOFFSET(3) NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    SourceServer NVARCHAR(128)    NOT NULL DEFAULT @@SERVERNAME,
    Sequence    BIGINT            NOT NULL DEFAULT 0
);

CREATE INDEX IX_ReplicationHeartbeat_WrittenAt
ON dbo.ReplicationHeartbeat (WrittenAt DESC);

-- Insert heartbeat every 10 seconds via SQL Agent job or Azure Automation
INSERT INTO dbo.ReplicationHeartbeat (WrittenAt, SourceServer, Sequence)
SELECT
    SYSDATETIMEOFFSET(),
    @@SERVERNAME,
    ISNULL((SELECT MAX(Sequence) FROM dbo.ReplicationHeartbeat), 0) + 1;

-- Purge old rows
DELETE FROM dbo.ReplicationHeartbeat
WHERE WrittenAt < DATEADD(HOUR, -24, SYSDATETIMEOFFSET());

```

```
-- On GEO-REPLICA: Measure observed RPO
SELECT
    MAX(WrittenAt)                                AS LatestHeartbeatOnReplica,
    SYSDATETIMEOFFSET()                          AS ReplicaClock,
    DATEDIFF(SECOND, MAX(WrittenAt), SYSDATETIMEOFFSET()) AS ObservedRPOSeconds,
    CASE
        WHEN DATEDIFF(SECOND, MAX(WrittenAt), SYSDATETIMEOFFSET()) > 300 THEN 'CRITICAL'
        WHEN DATEDIFF(SECOND, MAX(WrittenAt), SYSDATETIMEOFFSET()) > 30  THEN 'WARNING'
        ELSE 'OK'
    END AS RPOStatus
FROM dbo.ReplicationHeartbeat;
```

Push the `ObservedRPOSeconds` value to your observability platform as a custom metric alongside the vendor-supplied replication lag metrics. When these two numbers diverge significantly — vendor says 2 seconds, heartbeat says 45 seconds — it usually means the vendor metric measures log application lag while your heartbeat measures commit-to-visible lag including lock contention or log replay bottlenecks on the replica. The heartbeat is the more operationally relevant number.

16.8 Common Failure Modes and Operational Traps

Split-Brain After Unplanned Failover

In Aurora Global, split-brain cannot occur during a managed failover because the old primary is stopped before the new primary is promoted. But in an unplanned failover — where you detach the secondary and promote it because the primary region is unreachable — the primary region may actually be reachable from some clients. If applications are still writing to the old primary and the detached secondary is also accepting writes, you have split-brain: two diverged copies of the database, both accepting writes.

Prevent this by ensuring:

- Your application's primary connection string routes through a DNS layer (Route53 or Azure Traffic Manager) that you control
- You change DNS to point to the secondary before or immediately when you promote it
- You have a runbook step that explicitly verifies the old primary is no longer accepting application traffic before promoting the secondary

Replica Overload Under Read Scaling Patterns

A common antipattern is pointing all reporting workloads at the secondary cluster as a cost optimization. Heavy analytical queries on the secondary consume compute and I/O that compete with the replication apply process. Under high write volume from the primary, a secondary cluster running expensive queries can fall progressively further behind. The CloudWatch lag metric climbs, alert fires, on-call investigates and finds no infrastructure problem — the secondary is simply unable to keep up with both query load and replication apply simultaneously.

Monitor `DBLoad` on secondary instances alongside replication lag. If secondary `DBLoad` approaches the number of vCPUs consistently, resize the secondary upward or move the reporting workload to a dedicated replica that is not your only DR target.

KMS Key Region Mismatches

If you created an Aurora cluster encrypted with a single-region KMS key and then attempted to convert it to a Global Database, you will encounter failures when adding secondary regions. The secondary cluster's storage is encrypted with the key provided at creation time, and a single-region

key cannot encrypt storage in another region. Fix: use multi-region KMS keys from the start, or use AWS-managed keys (though customer-managed keys are preferable for most regulated workloads).

Azure SQL Failover Group and Long-Running Transactions

Azure SQL Failover Group planned failover will wait for all active transactions to complete before switching roles. A long-running transaction — a multi-hour bulk load or an uncommitted transaction from a runaway session — blocks planned failover indefinitely. Before initiating a planned failover, query `sys.dm_exec_requests` and kill any transactions expected to take longer than your RTO window.

```
-- Identify long-running transactions that will block planned failover
SELECT
    r.session_id,
    r.command,
    r.status,
    r.wait_type,
    r.wait_time / 1000.0                               AS wait_seconds,
    r.total_elapsed_time / 1000.0                       AS elapsed_seconds,
    SUBSTRING(st.text, (r.statement_start_offset/2)+1,
        ((CASE r.statement_end_offset
            WHEN -1 THEN DATALENGTH(st.text)
            ELSE r.statement_end_offset
        END - r.statement_start_offset)/2)+1)           AS current_statement,
    s.login_name,
    s.host_name,
    s.program_name
FROM sys.dm_exec_requests r
JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) st
WHERE r.session_id <> @@SPID
    AND r.total_elapsed_time > 60000 -- longer than 1 minute
ORDER BY r.total_elapsed_time DESC;
```

Cloud SQL Replica Promotion and the Replication Slot Problem

Cloud SQL for PostgreSQL uses logical replication slots internally for replication management. When you promote a read replica to a standalone instance, any replication slots that existed on the original primary are not transferred. If you used logical replication slots for CDC (Change Data Capture) or third-party integrations like Debezium pointing at the original primary, those slots are gone after promotion. You must re-create them on the newly promoted instance and restart your CDC consumers.

Document your replication slot inventory as part of your DR runbook. Include the slot names, plugin types (`pgoutput`, `wal2json`, etc.), publication names, and downstream consumers. Test slot re-creation in your DR drills.

16.9 Geo-Replication Checklist for Production Readiness

Before treating a geo-replicated deployment as production-ready, work through the following:

Configuration Validation - [] Replication lag alarm set and tested (warning and critical thresholds) - [] Heartbeat table deployed on primary; heartbeat reader running on secondary - [] Observed RPO measured under normal load and under peak write load - [] KMS keys are multi-region or region-appropriate (no single-region key in global setup) - [] Secondary cluster has correct security

groups, NACLs, and subnet groups - [] Secondary cluster's monitoring and alerting is configured independently of primary region

Failover Readiness - [] Failover runbook documented and reviewed by DBA team - [] Application connection string abstracted behind DNS or listener endpoint that survives failover without code change - [] DR drill performed (not simulated) within the last 90 days - [] Actual failover duration measured and compared against RTO target - [] Failback procedure documented and tested - [] Long-running transaction kill procedure included in planned failover runbook

Application-Layer Considerations - [] Application connection pools handle reconnection after failover gracefully - [] Read replicas in secondary region configured in application connection string (read/write split) - [] Session-level write operations not issued against the read endpoint - [] CDC / logical replication consumers tested against promoted secondary

Operational Runbooks - [] Unplanned failover procedure (primary region completely unavailable) - [] Split-brain prevention and detection steps documented - [] Replication slot inventory current and tested in DR drills - [] Post-failover validation steps with expected output documented

Key Takeaways

- **Cross-region replication is always asynchronous** in every managed cloud database service. The speed of light means synchronous writes across continents are impractical for OLTP workloads. Cloud Spanner is the single exception, using a quorum model with strong consistency, but at the cost of higher write latency and substantially higher per-region cost.
 - **RPO is not a configuration parameter — it is a measurement.** Replication lag fluctuates with write volume, secondary load, and network conditions. Deploy a heartbeat table on the primary and query it from the secondary continuously to measure observed RPO, and alert when it exceeds your contractual threshold.
 - **Abstraction layers at the connection string level are non-negotiable.** Aurora does not reroute your cluster endpoint after failover. Azure SQL Failover Groups solve this with a stable listener; Aurora Global requires you to solve it with Route53 or a similar DNS layer. Applications that hardcode regional cluster endpoints will fail during failover and require code changes under the worst possible conditions.
 - **DR drills must be disruptive to be valid.** A drill that does not actually perform a failover discovers nothing about real failure modes: missing IAM permissions in the secondary region, security group mismatches, secrets that exist only in the primary Secrets Manager, and CDC consumers that lose their replication slots. Run full failovers quarterly at minimum, measure the actual duration against your RTO target, and fix the gaps before the next drill.
 - **Secondary cluster resource sizing is a hidden operational trap.** Under-sized secondary clusters that serve both read traffic and replication apply will fall progressively further behind under combined load. Monitor DBLoad alongside replication lag on all secondary clusters, and size them based on combined replication and read query demand, not just one or the other.
-

Chapter 17: Azure SQL Security: TDE, Always Encrypted, Ledger

Azure SQL Database ships with encryption features that would have required significant infrastructure investment in an on-premises SQL Server environment. Transparent Data Encryption was always available on-prem, but managing certificates, key rotation, and backup encryption was a full operational concern. Always Encrypted required application-side key management that most teams deferred indefinitely. Azure SQL Ledger didn't exist at all. In the cloud, these three technologies form a layered security posture that satisfies everything from SOC 2 to PCI DSS to HIPAA — but only if you understand where each layer applies, what it actually protects, and what its operational costs are. This chapter walks through each feature from first principles to production deployment, with the gotchas that the official documentation skips.

17.1 Transparent Data Encryption: What It Does (and Doesn't) Protect

If you've managed SQL Server on-prem, you know TDE: it encrypts the data files, log files, and backup files at rest. Azure SQL Database has TDE enabled by default on every new database. That sentence creates a false sense of security that has burned more than a few DBAs during audit preparation.

What TDE protects: Data files on disk. An attacker who steals raw storage media or data files gets ciphertext. Backups are encrypted automatically. This satisfies the “data at rest” requirement in most compliance frameworks.

What TDE does not protect: Data in memory, data in transit, data from a compromised SQL login, or a DBA with elevated permissions. TDE encryption and decryption happens transparently at the storage layer. Once the database engine decrypts data for query execution, TDE offers no protection. A rogue DBA running `SELECT * FROM dbo.CreditCards` gets plaintext every time.

The default TDE configuration uses a service-managed key: Microsoft generates and manages the Database Encryption Key (DEK) and the certificate that protects it. For most applications this is acceptable. For regulated industries, you need Customer-Managed Keys (CMK) via Azure Key Vault — where you control key lifecycle, rotation, and revocation.

Enabling TDE with Customer-Managed Key via Azure CLI

Step 1: Create or identify the Azure Key Vault

```

az keyvault create \
  --name "prod-sqlsecurity-kv" \
  --resource-group "rg-prod-eastus" \
  --location "eastus" \
  --sku premium \
  --enable-purge-protection true \
  --retention-days 90

# Step 2: Create the key used as the TDE protector
az keyvault key create \
  --vault-name "prod-sqlsecurity-kv" \
  --name "sql-tde-protector" \
  --kts RSA-HSM \
  --size 2048

# Step 3: Assign the SQL Server managed identity access to the key vault
# First get the SQL Server's managed identity principal ID
SQL_IDENTITY=$(az sql server show \
  --name "prod-sql-server" \
  --resource-group "rg-prod-eastus" \
  --query identity.principalId \
  --output tsv)

az keyvault set-policy \
  --name "prod-sqlsecurity-kv" \
  --object-id "$SQL_IDENTITY" \
  --key-permissions get wrapKey unwrapKey list

# Step 4: Set the TDE protector to the customer-managed key
KEY_ID=$(az keyvault key show \
  --vault-name "prod-sqlsecurity-kv" \
  --name "sql-tde-protector" \
  --query key.kid \
  --output tsv)

az sql server tde-key set \
  --resource-group "rg-prod-eastus" \
  --server "prod-sql-server" \
  --server-key-type AzureKeyVault \
  --kid "$KEY_ID"

# Step 5: Verify TDE is active on the database
az sql db tde show \
  --resource-group "rg-prod-eastus" \
  --server "prod-sql-server" \
  --database "prod-app-db"

```

The output of that last command will show **"state": "Enabled"** when TDE is active. During the initial encryption scan — which runs as a background process — you'll see **"state": "Enabling"**. The scan duration scales with database size and can run for hours on multi-terabyte databases. Queries continue running during this window, but expect elevated I/O and some added query latency on write-heavy workloads until the scan completes — measure it against your own workload rather than assuming a fixed overhead.

Key Rotation

With CMK-based TDE, key rotation is a two-phase process: add the new key version, then set it as the active protector. The old key stays in Key Vault until you explicitly confirm rotation is complete — this is intentional, because any backup encrypted under the old key still needs the old key for restore.

```

# Rotate the TDE protector key
az keyvault key rotate \
  --vault-name "prod-sqlsecurity-kv" \
  --name "sql-tde-protector"

```

```
# Get the new key version ID
NEW_KEY_ID=$(az keyvault key show \
  --vault-name "prod-sqlsecurity-kv" \
  --name "sql-tde-protector" \
  --query key.kid \
  --output tsv)

# Update the server TDE protector to use the new version
az sql server tde-key set \
  --resource-group "rg-prod-eastus" \
  --server "prod-sql-server" \
  --server-key-type AzureKeyVault \
  --kid "$NEW_KEY_ID"
```

Production gotcha: If the SQL Server managed identity loses access to Key Vault — an expired policy, a deleted vault, a misconfigured network rule — the server goes into an inaccessible state within minutes. You cannot query the database, cannot restore from backup, cannot do anything until Key Vault access is restored. Set up Azure Monitor alerts on Key Vault access failures for the SQL Server managed identity before you go live with CMK.

17.2 Always Encrypted: Column-Level Encryption the Right Way

Always Encrypted addresses the threat model that TDE ignores: the privileged insider or compromised DBA account. With Always Encrypted, sensitive column data is encrypted client-side before it ever reaches the SQL Server engine. The database engine sees ciphertext. It stores ciphertext. It returns ciphertext. The plaintext never exists on the server.

Think of it this way: TDE protects your data from someone who steals the storage hardware. Always Encrypted protects your data from someone who steals the database credentials or has DBA access. These are different attackers in different positions.

The architecture has three components:

- **Column Encryption Key (CEK):** Encrypts the actual column data. One CEK per column group.
- **Column Master Key (CMK):** Encrypts the CEKs. Stored in Azure Key Vault (or Windows Certificate Store for dev). The database stores only metadata pointing to the CMK location — never the key itself.
- **Client application:** Holds the Azure Key Vault credentials. The ADO.NET or JDBC driver handles encryption/decryption transparently when the connection string includes `Column Encryption Setting=enabled`.

Setting Up Always Encrypted with Azure Key Vault

```
-- Step 1: Create the Column Master Key metadata in SQL Server
-- The actual key material lives in Azure Key Vault, not here
-- This T-SQL runs in SSMS or sqlcmd after configuring AKV access

CREATE COLUMN MASTER KEY [AKV_CMK_Production]
WITH (
  KEY_STORE_PROVIDER_NAME = 'AZURE_KEY_VAULT',
  KEY_PATH = 'https://prod-sqlsecurity-kv.vault.azure.net/keys/always-encrypted-cmk/abc123def456'
);
GO
```

```

-- Step 2: Create the Column Encryption Key
-- This operation contacts Key Vault to wrap the CEK with the CMK
-- Requires the executing account to have Key Vault Wrap permissions

CREATE COLUMN ENCRYPTION KEY [CEK_PII_Data]
WITH VALUES (
    COLUMN_MASTER_KEY = [AKV_CMK_Production],
    ALGORITHM = 'RSA_OAEP',
    ENCRYPTED_VALUE = 0x016E000001630075007200720065006E... -- wrapped CEK bytes
);
GO

-- Step 3: Create a table with encrypted columns
-- DETERMINISTIC encryption: same plaintext always produces same ciphertext
-- Supports equality comparisons (WHERE SSN = @ssn)
-- RANDOMIZED encryption: same plaintext produces different ciphertext each time
-- More secure but supports NO predicates - SELECT only by returning the column

CREATE TABLE dbo.CustomerPII (
    CustomerID          INT          NOT NULL PRIMARY KEY,
    FirstName           NVARCHAR(100) NOT NULL, -- plaintext, used in UI
    LastName            NVARCHAR(100) NOT NULL, -- plaintext
    SSN                 CHAR(11)     COLLATE Latin1_General_BIN2
                        ENCRYPTED WITH (
                            COLUMN_ENCRYPTION_KEY = CEK_PII_Data,
                            ENCRYPTION_TYPE = DETERMINISTIC,
                            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
                        ) NOT NULL,
    CreditCardNumber    NVARCHAR(19) COLLATE Latin1_General_BIN2
                        ENCRYPTED WITH (
                            COLUMN_ENCRYPTION_KEY = CEK_PII_Data,
                            ENCRYPTION_TYPE = RANDOMIZED,
                            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
                        ) NULL,
    DateOfBirth         DATE          ENCRYPTED WITH (
                            COLUMN_ENCRYPTION_KEY = CEK_PII_Data,
                            ENCRYPTION_TYPE = RANDOMIZED,
                            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
                        ) NULL
);
GO

```

Notice the `COLLATE Latin1_General_BIN2` requirement on the `NVARCHAR` columns. Always Encrypted requires BIN2 collation for deterministic encryption on string columns. If you're retrofitting Always Encrypted onto an existing table that uses `Latin1_General_CI_AS`, you'll need to rebuild those columns. That's a blocking operation on large tables.

Application Integration

```

# Python application connecting to Always Encrypted columns
# Requires: pip install pyodbc, azure-identity, azure-keyvault-keys

import pyodbc
from azure.identity import DefaultAzureCredential
from azure.keyvault.keys.crypto import CryptographyClient

# The connection string does the heavy lifting
# Column Encryption Setting=enabled tells the driver to handle crypto
connection_string = (
    "Driver={ODBC Driver 18 for SQL Server};"
    "Server=prod-sql-server.database.windows.net;"
    "Database=prod-app-db;"
    "Authentication=ActiveDirectoryManagedIdentity;"
    "Column Encryption Setting=enabled;"
    "KeyVaultClientId=<managed-identity-client-id>;"
    "KeyVaultClientSecret=notneeded;" # Not needed with managed identity
    "Encrypt=yes;"
)

```

```
"TrustServerCertificate=no;"
)

conn = pyodbc.connect(connection_string)
cursor = conn.cursor()

# INSERT: application sends ciphertext, SQL Server stores ciphertext
# The ODBC driver encrypts the SSN value before it leaves the client machine
cursor.execute(
    "INSERT INTO dbo.CustomerPII (CustomerID, FirstName, LastName, SSN) "
    "VALUES (?, ?, ?, ?)",
    (1001, 'Jane', 'Smith', '123-45-6789') # SSN encrypted by driver
)

# SELECT with parameterized query on deterministic column: works
# The driver encrypts the parameter value, SQL Server compares ciphertext
cursor.execute(
    "SELECT CustomerID, FirstName, LastName, SSN FROM dbo.CustomerPII "
    "WHERE SSN = ?",
    ('123-45-6789',)
)

row = cursor.fetchone()
if row:
    # SSN comes back as plaintext - decrypted by the driver on the client
    print(f"Found customer: {row.FirstName} {row.LastName}, SSN: {row.SSN}")

conn.commit()
conn.close()
```

The Operational Constraints You Must Know Before Deploying

Always Encrypted has constraints that are not obvious from the documentation until you run into them in production:

You cannot do arithmetic on encrypted columns. WHERE Salary > 50000 on an encrypted Salary column fails. RANDOMIZED encrypted columns support no predicates whatsoever. DETERMINISTIC columns support only equality predicates (=, IN).

Joins on encrypted columns work only for DETERMINISTIC encryption. JOIN ON t1.SSN = t2.SSN functions correctly with DETERMINISTIC. It fails with RANDOMIZED because the same plaintext produces different ciphertexts.

Stored procedures that accept encrypted column values must use parameterized execution. The stored procedure definition looks normal in T-SQL; the application must pass values as parameters, not as inline SQL literals.

GROUP BY, ORDER BY, and index ranges on encrypted columns: GROUP BY works with DETERMINISTIC. ORDER BY on encrypted data returns results ordered by ciphertext, which is meaningless for the application. There's no way around this. If you need sorted results on an encrypted column, sort client-side after decryption.

Column size expansion: Every encrypted value is larger than the plaintext. A CHAR(11) SSN value becomes a 65-byte binary blob. Size your columns and row sizes accordingly. Encryption adds approximately 54 bytes of overhead plus the encrypted value size, rounded up to the nearest AES block.

17.3 Always Encrypted with Secure Enclaves

The encryption predicate limitations in classic Always Encrypted (described above) are severe enough that many teams avoid the feature entirely. Azure SQL's Always Encrypted with Secure Enclaves removes most of them.

An enclave is a protected memory region in the SQL Server process — implemented via VBS (Virtualization Based Security) enclaves in Azure SQL Database. Sensitive cryptographic operations happen inside the enclave, where even the database engine process cannot inspect the memory. The application sends a CEK into the enclave via an attestation-verified channel. Query processing on encrypted columns happens inside the enclave, so the engine never exposes plaintext to the wider process.

With enclave-enabled Always Encrypted, you gain:

- Range queries on encrypted columns (`WHERE Age BETWEEN 30 AND 45`)
- Pattern matching (`LIKE '%smith%'` on RANDOMIZED encrypted columns with enclave-enabled CEKs)
- Index creation on encrypted columns
- In-place column encryption without data movement

The tradeoff is complexity: the application connection must include attestation configuration pointing to a Microsoft Azure Attestation endpoint, and the Key Vault CMK must be enclave-enabled.

```
# Enable enclave support on the Azure SQL logical server
az sql server update \
  --name "prod-sql-server" \
  --resource-group "rg-prod-eastus" \
  --set properties.enabledSecureEnclaveType="VBS"

# Verify the enclave type
az sql server show \
  --name "prod-sql-server" \
  --resource-group "rg-prod-eastus" \
  --query properties.enabledSecureEnclaveType
```

The attestation URL for VBS enclaves in Azure SQL follows the pattern `https://shardeduks.uks.attest.azure.net` (region-specific). The connection string needs:

```
Enclave Attestation Url=https://shardeduks.uks.attest.azure.net/attest/AzureIntelSGX;
Column Encryption Setting=enabled;
```

Enclave-enabled queries carry a performance cost: the attestation handshake adds ~5–10ms per new connection. For long-lived connection pools this is negligible. For high-frequency short-lived connections it's measurable. Test with your actual connection pool configuration before production rollout.

17.4 Azure SQL Ledger: Tamper-Evident Tables for Compliance

Azure SQL Ledger is the newest of the three technologies in this chapter, and it solves a different problem than encryption. Ledger doesn't hide your data — it makes your data tamper-evident. Any modification to ledger table data leaves a cryptographically verifiable trail that can prove, to an external auditor, that the data has not been altered since it was written.

The use cases are specific: financial transaction records, audit logs, configuration history, contract terms, any dataset where “we can prove this hasn’t been modified” is a legal or regulatory requirement.

Ledger works by maintaining a SHA-256 hash tree (similar to a blockchain Merkle tree) over table rows. Each database transaction produces a block containing a hash of all changed rows. Each block includes the hash of the previous block. The result is a chain: modifying any historical row invalidates all subsequent hashes. An auditor can verify the entire chain in seconds.

Azure SQL Ledger has two table types:

Updatable Ledger Tables: Support INSERT, UPDATE, DELETE. Maintain a separate `ledger_history` table containing the full history of every change, with who made it and when. The current table shows current state. The history table is append-only and protected by the hash chain.

Append-Only Ledger Tables: Support INSERT only. No UPDATE, no DELETE. Appropriate for immutable audit logs, transaction records, event streams.

```
-- Create an updatable ledger table for account balances
-- The LEDGER = ON clause activates all ledger machinery
CREATE TABLE dbo.AccountBalance (
    AccountID          INT          NOT NULL,
    CustomerName       NVARCHAR(100) NOT NULL,
    Balance            DECIMAL(18,2) NOT NULL,
    LastModified       DATETIME2    NOT NULL DEFAULT SYSUTCDATETIME()
)
WITH (
    SYSTEM_VERSIONING = ON,
    LEDGER = ON
);
GO

-- Azure SQL automatically creates dbo.AccountBalance_Ledger
-- and the ledger history table with these system columns:
-- ledger_start_transaction_id BIGINT
-- ledger_end_transaction_id   BIGINT
-- ledger_start_sequence_number BIGINT
-- ledger_end_sequence_number  BIGINT
-- ledger_operation_type       TINYINT (1=INSERT, 2=DELETE for update tracking)
-- ledger_operation_type_desc  NVARCHAR(128)

-- Create an append-only ledger table for payment events
CREATE TABLE dbo.PaymentAudit (
    PaymentID          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWSEQUENTIALID(),
    AccountID          INT              NOT NULL,
    Amount             DECIMAL(18,2)    NOT NULL,
    PaymentTimestamp    DATETIME2       NOT NULL DEFAULT SYSUTCDATETIME(),
    PaymentMethod       NVARCHAR(50)    NOT NULL,
    ProcessorRef        NVARCHAR(100)   NULL
)
WITH (
    LEDGER = ON (APPEND_ONLY = ON)
);
GO

-- Verify the ledger configuration
SELECT
    name,
    is_ledger,
    ledger_type_desc,
    is_dropped_ledger_table
FROM sys.tables
WHERE name IN ('AccountBalance', 'PaymentAudit');
GO
```

Verifying the Ledger

Verification is the feature that makes auditors comfortable. The `sp_verify_database_ledger` procedure recomputes the entire hash chain and reports any tampering.

```
-- Generate the database ledger digest (send to external immutable storage)
-- This digest is what you share with auditors as the "known good" state
EXECUTE sp_generate_database_ledger_digest;
GO
-- Returns JSON:
-- {
--   "database_name": "prod-app-db",
--   "block_id": 42,
--   "hash": "0x3A7F8B2C...",
--   "last_transaction_commit_time": "2024-11-15T14:22:11.123Z",
--   "digest_version": 1
-- }

-- Verify ledger integrity using a previously stored digest
-- Pass the digest JSON retrieved from your immutable storage (Azure Blob with WORM)
EXECUTE sp_verify_database_ledger
N|
↪ '["database_name":"prod-app-db","block_id":42,"hash":"0x3A7F8B2C...","last_transaction_commit_time":"2024-11-15T14:22:11.123Z"]'
↪ ;
GO
-- Returns: The database ledger has been successfully verified.

-- View the full audit history for a specific account
-- This queries the automatically maintained history table
SELECT
    h.AccountID,
    h.CustomerName,
    h.Balance,
    h.LastModified,
    h.ledger_operation_type_desc AS Operation,
    t.commit_time AS ChangeTime,
    t.principal_name AS ChangedBy
FROM dbo.AccountBalance_Ledger h
INNER JOIN sys.database_ledger_transactions t
    ON h.ledger_start_transaction_id = t.transaction_id
WHERE h.AccountID = 10042
ORDER BY h.ledger_start_sequence_number;
GO
```

Digest Storage Strategy

A ledger without an independently stored digest is less useful than it appears. If an attacker can modify both the ledger data and the digest stored in the same database, they can reconstruct a consistent but fraudulent chain. Digests must be stored in immutable external storage:

- **Azure Blob Storage with immutable storage policies (WORM):** The native Azure integration. Use `sp_generate_database_ledger_digest` on a schedule (hourly or after each significant batch) and write digests to a locked container.
- **Azure Confidential Ledger:** A purpose-built service using hardware-backed distributed ledger. Digests written here are independently verified by an infrastructure you don't control.

```
# Configure automatic digest storage to Azure Blob immutable storage
az sql db ledger-digest-uploads enable \
    --resource-group "rg-prod-eastus" \
    --server "prod-sql-server" \
    --database "prod-app-db" \
    --endpoint "https://prodauditdigests.blob.core.windows.net/ledger-digests"

# Verify digest upload status
az sql db ledger-digest-uploads show \
```

```
--resource-group "rg-prod-eastus" \
--server "prod-sql-server" \
--database "prod-app-db"
```

17.5 Layering the Security Stack: TDE + Always Encrypted + Ledger Together

These three features protect against different attackers and different threat vectors. In a production system handling regulated data, you often need all three:

Feature	Protects Against	Doesn't Protect Against
TDE	Stolen storage media, backup files	Privileged DB users, in-memory access
Always Encrypted	Privileged DBAs, compromised credentials	Tampered historical records
Ledger	Tampering with historical data	Data exposure to privileged users

The combination creates defense in depth: TDE ensures backups are useless without your Key Vault key. Always Encrypted ensures DBAs see ciphertext on sensitive columns. Ledger ensures that even if someone does gain write access, any modification is detectable.

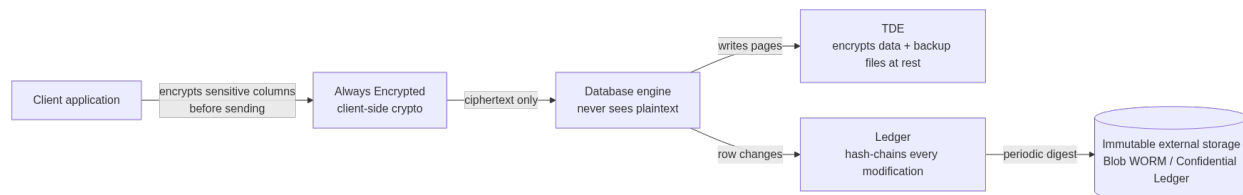


Figure 8: Each layer defends against a different attacker: TDE stops storage theft, Always Encrypted stops privileged-user data exposure, and Ledger makes any tampering with historical data provable — none of the three substitutes for the others.

Terraform: Provisioning a Secured Azure SQL Database

```
# terraform/azure_sql_secure.tf
# Provisions Azure SQL with TDE (CMK), auditing, and threat protection

terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.80"
    }
  }
}

resource "azurerm_resource_group" "prod" {
  name     = "rg-prod-eastus"
  location = "East US"
}
```

```

}

# Key Vault for TDE and Always Encrypted keys
resource "azurerm_key_vault" "sql_security" {
  name = "prod-sqlsecurity-kv"
  resource_group_name = azurerm_resource_group.prod.name
  location = azurerm_resource_group.prod.location
  tenant_id = data.azurerm_client_config.current.tenant_id
  sku_name = "premium"
  soft_delete_retention_days = 90
  purge_protection_enabled = true

  network_acls {
    default_action = "Deny"
    bypass = "AzureServices"
    ip_rules = var.allowed_ip_ranges
  }
}

# SQL Server with system-assigned managed identity
resource "azurerm_mssql_server" "prod" {
  name = "prod-sql-server"
  resource_group_name = azurerm_resource_group.prod.name
  location = azurerm_resource_group.prod.location
  version = "12.0"
  administrator_login = var.sql_admin_login
  administrator_login_password = var.sql_admin_password
  minimum_tls_version = "1.2"

  identity {
    type = "SystemAssigned"
  }

  azuread_administrator {
    login_username = var.aad_admin_username
    object_id = var.aad_admin_object_id
  }
}

# Grant SQL Server managed identity access to Key Vault
resource "azurerm_key_vault_access_policy" "sql_server" {
  key_vault_id = azurerm_key_vault.sql_security.id
  tenant_id = data.azurerm_client_config.current.tenant_id
  object_id = azurerm_mssql_server.prod.identity[0].principal_id

  key_permissions = [
    "Get", "WrapKey", "UnwrapKey", "List"
  ]
}

# TDE protector key in Key Vault
resource "azurerm_key_vault_key" "tde_protector" {
  name = "sql-tde-protector"
  key_vault_id = azurerm_key_vault.sql_security.id
  key_type = "RSA-HSM"
  key_size = 2048

  key_opts = ["wrapKey", "unwrapKey"]

  depends_on = [azurerm_key_vault_access_policy.sql_server]
}

# Set the TDE protector on the SQL Server
resource "azurerm_mssql_server_transparent_data_encryption" "prod" {
  server_id = azurerm_mssql_server.prod.id
  key_vault_key_id = azurerm_key_vault_key.tde_protector.id
}

# Production database
resource "azurerm_mssql_database" "prod_app" {

```

```
name          = "prod-app-db"
server_id     = azurerm_mssql_server.prod.id
collation     = "SQL_Latin1_General_CP1_CI_AS"
sku_name      = "GP_Gen5_4"
max_size_gb   = 256
zone_redundant = true

threat_detection_policy {
  state          = "Enabled"
  email_account_admins = "Enabled"
  retention_days  = 90
}

depends_on = [azurerm_mssql_server_transparent_data_encryption.prod]
}

# Storage account for audit logs with immutable policy
resource "azurerm_storage_account" "audit" {
  name                = "prodsqldataauditlogs"
  resource_group_name = azurerm_resource_group.prod.name
  location            = azurerm_resource_group.prod.location
  account_tier        = "Standard"
  account_replication_type = "GRS"
  min_tls_version     = "TLS1_2"

  blob_properties {
    versioning_enabled = true
  }
}

resource "azurerm_storage_container" "audit_logs" {
  name                = "sqlauditlogs"
  storage_account_name = azurerm_storage_account.audit.name
  container_access_type = "private"
}

# Server-level auditing to immutable storage
resource "azurerm_mssql_server_extended_auditing_policy" "prod" {
  server_id                = azurerm_mssql_server.prod.id
  storage_endpoint         = azurerm_storage_account.audit.primary_blob_endpoint
  storage_account_access_key = azurerm_storage_account.audit.primary_access_key
  storage_account_access_key_is_secondary = false
  retention_in_days        = 90
  log_monitoring_enabled   = true
}

data "azurerm_client_config" "current" {}
```

17.6 Operational Monitoring and Alerting for Security Features

Deploying TDE, Always Encrypted, and Ledger is only half the work. The features need ongoing monitoring because several failure modes are silent until they become critical.

Key Vault Access Failures

The most operationally dangerous failure mode with CMK-based TDE is losing Key Vault access. Azure SQL will enter an inaccessible state — all connections refused — if it cannot reach the Key Vault key. This can happen due to:

- Key Vault network rules blocking the SQL Server managed identity
- The managed identity's access policy being accidentally deleted

- The key being disabled or deleted in Key Vault
- Key Vault going into a soft-delete state

Set up Azure Monitor alerts on the Key Vault diagnostic logs before going live:

```
# Enable Key Vault diagnostic logging
az monitor diagnostic-settings create \
  --name "keyvault-to-loganalytics" \
  --resource
  ↪ "/subscriptions/<sub-id>/resourceGroups/rg-prod-eastus/providers/Microsoft.KeyVault/vaults/prod-sqlsecurity-kv"
  ↪ \
  --workspace
  ↪ "/subscriptions/<sub-id>/resourceGroups/rg-prod-eastus/providers/Microsoft.OperationalInsights/workspaces/prod-law"
  ↪ \
  --logs '[{"category": "AuditEvent", "enabled": true}]' \
  --metrics '[{"category": "AllMetrics", "enabled": true}]'

# Create an alert for failed Key Vault operation from the SQL Server identity
az monitor scheduled-query create \
  --name "SQL-KeyVault-Access-Failure" \
  --resource-group "rg-prod-eastus" \
  --scopes
  ↪ "/subscriptions/<sub-id>/resourceGroups/rg-prod-eastus/providers/Microsoft.OperationalInsights/workspaces/prod-law"
  ↪ \
  --condition "count 'Placeholder' > 0" \
  --condition-query "AzureDiagnostics
  | where ResourceType == 'VAULTS'
  | where ResourceId contains 'prod-sqlsecurity-kv'
  | where ResultType == 'Unauthorized' or ResultType == 'Forbidden'
  | where CallerIPAddress != ''
  | where TimeGenerated > ago(5m)" \
  --evaluation-frequency "5m" \
  --window-size "5m" \
  --severity 1 \
  --action-groups
  ↪ "/subscriptions/<sub-id>/resourceGroups/rg-prod-eastus/providers/Microsoft.Insights/actionGroups/prod-oncall"
```

Ledger Verification on a Schedule

Ledger verification should run on a schedule, not just during audits. A daily verification job catches tampering early. Wrap `sp_verify_database_ledger` in an Azure SQL elastic job or a simple Azure Function with a timer trigger:

```
-- Stored procedure to run verification and log results
-- Deploy this in your admin/DBA database or a monitoring schema

CREATE OR ALTER PROCEDURE dbo.usp_VerifyLedgerIntegrity
AS
BEGIN
  SET NOCOUNT ON;

  DECLARE @DigestJSON NVARCHAR(MAX);
  DECLARE @VerificationResult NVARCHAR(500);
  DECLARE @ErrorMessage NVARCHAR(1000);

  BEGIN TRY
    -- Retrieve the most recent stored digest from your audit table
    -- In production this would pull from Azure Blob or Confidential Ledger
    SELECT TOP 1 @DigestJSON = DigestJSON
    FROM dbo.LedgerDigestArchive
    ORDER BY CapturedAt DESC;

    IF @DigestJSON IS NULL
    BEGIN
      INSERT INTO dbo.LedgerVerificationLog (RunTime, Status, Message)
      VALUES (SYSUTCDATETIME(), 'WARNING', 'No stored digest found for verification.');
```

```
      RETURN;
```

```
END

-- Run verification
EXEC sp_verify_database_ledger @DigestJSON;

INSERT INTO dbo.LedgerVerificationLog (RunTime, Status, Message)
VALUES (SYSUTCDATETIME(), 'SUCCESS', 'Ledger verification passed.');
```

```
END TRY
BEGIN CATCH
    SET @ErrorMessage = ERROR_MESSAGE();

    INSERT INTO dbo.LedgerVerificationLog (RunTime, Status, Message)
    VALUES (SYSUTCDATETIME(), 'FAILURE', @ErrorMessage);

    -- Re-raise so the calling agent job marks the step as failed
    -- and triggers an alert
    THROW;
END CATCH
END;
GO

-- Supporting tables
CREATE TABLE dbo.LedgerDigestArchive (
    ArchiveID INT IDENTITY PRIMARY KEY,
    DigestJSON NVARCHAR(MAX) NOT NULL,
    CapturedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    BlockID INT NOT NULL,
    StoredBy NVARCHAR(128) NOT NULL DEFAULT SYSTEM_USER
);

CREATE TABLE dbo.LedgerVerificationLog (
    LogID INT IDENTITY PRIMARY KEY,
    RunTime DATETIME2 NOT NULL,
    Status NVARCHAR(20) NOT NULL, -- SUCCESS, FAILURE, WARNING
    Message NVARCHAR(1000) NULL
);
GO
```

Azure Defender for SQL

Azure Defender for SQL (now part of Microsoft Defender for Cloud) provides threat detection on top of the base security features — anomalous query patterns, SQL injection attempts, unusual data access volumes. Enable it at the server level and configure alert routing to your security team's channel, not just the DBA team:

```
# Enable Microsoft Defender for SQL on the server
az sql server microsoft-support-auditing-policy update \
    --resource-group "rg-prod-eastus" \
    --server "prod-sql-server" \
    --state Enabled

az security pricing create \
    --name "SqlServers" \
    --tier "Standard"

# Set security contact for Defender alerts
az security contact create \
    --name "security-team-contact" \
    --email "security-alerts@yourcompany.com" \
    --phone "+1-555-0100" \
    --alert-notifications "On" \
    --alerts-to-admins "On"
```

17.7 Migration Scenarios: Adding Security to Existing Databases

Enabling these features on a greenfield database is straightforward. Adding them to an existing production database — with live application traffic — requires careful sequencing.

Adding Always Encrypted to an Existing Table

The challenge: existing plaintext data must be encrypted in-place, or you must shadow-copy the data through an encryption pass. Azure SQL provides the `ALTER TABLE ... ALTER COLUMN` path with Always Encrypted, but it requires an offline migration window on older configurations, or can use the in-place encryption available with enclave-enabled Always Encrypted.

```
-- With enclave-enabled Always Encrypted, encrypt existing plaintext columns
-- WITHOUT taking the table offline (online encryption, available with VBS enclave)

-- First, verify the server has enclave enabled
SELECT
    SERVERPROPERTY('EnabledSecureEnclaveType') AS EnclaveType;
-- Should return: VBS

-- Verify the CEK is enclave-enabled
SELECT
    cek.name,
    cekv.enclave_computations_allowed
FROM sys.column_encryption_keys cek
JOIN sys.column_encryption_key_values cekv
    ON cek.column_encryption_key_id = cekv.column_encryption_key_id
WHERE cek.name = 'CEK_PII_Data';

-- Perform online encryption of an existing column
-- This requires an enclave-enabled connection (Enclave Attestation Url in connection string)
ALTER TABLE dbo.CustomerPII
ALTER COLUMN SSN CHAR(11) COLLATE Latin1_General_BIN2
    ENCRYPTED WITH (
        COLUMN_ENCRYPTION_KEY = CEK_PII_Data,
        ENCRYPTION_TYPE = DETERMINISTIC,
        ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
    ) NOT NULL
WITH (ONLINE = ON);
GO
```

The `WITH (ONLINE = ON)` clause for Always Encrypted column encryption is only available with enclave-enabled configurations. Without it, the `ALTER COLUMN` operation takes a schema modification lock that blocks reads and writes for the duration of the re-encryption scan. On a table with tens of millions of rows, that lock duration makes the classic path impractical during business hours.

Converting a Regular Table to a Ledger Table

Once created without ledger enabled, a regular SQL table cannot be converted to a ledger table in-place. There is no `ALTER TABLE ... SET LEDGER = ON`. The migration path is:

1. Create the new ledger table with the same schema
2. Insert the existing data from the old table (this seeds the ledger with the historical state)
3. Swap application traffic to the new table
4. Optionally drop or archive the old table

```
-- Migration: regular AccountBalance to ledger AccountBalance
-- Step 1: Create ledger version alongside the existing table
CREATE TABLE dbo.AccountBalance_Ledger_New (
    AccountID          INT          NOT NULL,
```



```
CustomerName    NVARCHAR(100)    NOT NULL,
Balance         DECIMAL(18,2)    NOT NULL,
LastModified    DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME()
)
WITH (
    SYSTEM_VERSIONING = ON,
    LEDGER = ON
);
GO

-- Step 2: Migrate existing data in batches to avoid log pressure
DECLARE @BatchSize INT = 10000;
DECLARE @Offset INT = 0;
DECLARE @Rows INT = 1;

WHILE @Rows > 0
BEGIN
    INSERT INTO dbo.AccountBalance_Ledger_New
        (AccountID, CustomerName, Balance, LastModified)
    SELECT
        AccountID, CustomerName, Balance, LastModified
    FROM dbo.AccountBalance
    ORDER BY AccountID
    OFFSET @Offset ROWS
    FETCH NEXT @BatchSize ROWS ONLY;

    SET @Rows = @@ROWCOUNT;
    SET @Offset += @BatchSize;

    -- Brief pause to reduce I/O pressure during business hours
    WAITFOR DELAY '00:00:00.100';
END;
GO

-- Step 3: Rename tables in a single transaction to minimize cutover window
BEGIN TRANSACTION;
EXEC sp_rename 'dbo.AccountBalance', 'dbo.AccountBalance_Old';
EXEC sp_rename 'dbo.AccountBalance_Ledger_New', 'dbo.AccountBalance';
COMMIT;
GO
```

Note that the data inserted during migration appears in the ledger as a batch of INSERT operations performed by the migration login. The ledger will correctly show that all initial rows were written at migration time by a specific principal — which is accurate and acceptable for audit purposes. Document the migration event in your compliance records so auditors understand the ledger history starts at the migration timestamp.

Key Takeaways

- **TDE encrypts data at rest and satisfies storage-layer compliance requirements, but it provides zero protection against a privileged database user or compromised credentials.** Use Customer-Managed Keys via Azure Key Vault when your compliance framework requires key ownership, and set up Key Vault access failure alerts before the first day of production traffic — losing access to the CMK makes the database completely inaccessible.
- **Always Encrypted moves the encryption boundary to the client application, ensuring the database engine never sees plaintext on sensitive columns.** The operational constraints — no range queries on classic Always Encrypted, mandatory BIN2 collation

on string columns, and size expansion of every encrypted value — must be evaluated against your query patterns before committing to the schema. Secure Enclaves lift the range query restriction at the cost of an attestation handshake on each new connection.

- **Azure SQL Ledger provides cryptographic proof of data integrity, not data confidentiality.** A tamper-evident audit trail is only meaningful if the digest is stored in immutable external storage (Azure Blob WORM policy or Azure Confidential Ledger) that the database owner cannot modify — otherwise a sophisticated attacker who compromises the database can also reconstruct the digest chain.
 - **The three features protect different threat vectors and are intended to be layered, not chosen between.** TDE covers storage-layer attackers, Always Encrypted covers insider threats and credential compromise, and Ledger covers retrospective tampering with historical records. A production system facing multiple compliance requirements will frequently need all three active simultaneously.
 - **Migration to these features on existing production databases requires careful sequencing.** Always Encrypted column encryption without Secure Enclaves requires a schema modification lock that can block reads and writes for hours on large tables; enclave-enabled `ALTER COLUMN ... WITH (ONLINE = ON)` is the production-safe path. Ledger tables cannot be altered in-place from regular tables — plan for a create-migrate-rename cutover with batched data movement to control transaction log growth.
-

Chapter 18: Azure Arc-Enabled SQL Server: Hybrid Cloud

Azure Arc-Enabled SQL Server extends Azure’s management plane to SQL Server instances running anywhere — on-premises data centers, colocation facilities, edge locations, or competing cloud providers. For DBAs managing hybrid environments, this changes the operational calculus: instead of maintaining separate toolchains for on-prem and cloud databases, you get unified monitoring, policy enforcement, security scanning, and licensing management through a single Azure portal and API surface. This chapter covers how Arc-Enabled SQL Server works architecturally, how to onboard existing SQL Server instances, how to configure Azure services like Microsoft Defender and Azure Monitor against non-Azure hosts, and how to use Arc as a bridge during phased cloud migrations rather than a permanent destination.

18.1 Understanding Azure Arc Architecture for SQL Server

Before Arc, hybrid database management meant running parallel monitoring stacks — perhaps SCOM or Datadog for on-premises SQL Server alongside Azure Monitor for Azure SQL. Security assessments ran separately. Licensing compliance was tracked in spreadsheets. Arc collapses this by projecting non-Azure resources into Azure Resource Manager (ARM) as first-class objects.

The architecture rests on two components: the **Connected Machine Agent** (the Arc agent installed on the host OS) and the **Arc SQL Server extension**. The Connected Machine agent establishes an outbound HTTPS connection to Azure’s Arc service endpoints on port 443. No inbound firewall rules are required. The agent authenticates using a service principal or managed identity, receives its configuration from ARM, and reports telemetry back to Azure. The SQL Server extension layer sits on top, communicating with the local SQL Server instance via a Windows service (`SqlServerExtension`) or `systemd` service on Linux.

On-Premises Host

- SQL Server 2019 (or 2022, 2017, 2016)
- Arc Connected Machine Agent
 - Heartbeat → Azure Arc Service Endpoints (TCP 443)
- Arc SQL Server Extension
 - Reads SQL metadata, license info, configuration
 - Sends to Azure Monitor Log Analytics
 - Applies policies from Azure Policy

From ARM’s perspective, your on-premises SQL Server appears under `Microsoft.AzureArcData/sqlServerInstance` as a resource type. You can tag it, apply RBAC, include it in Azure Policy assignments, and view it in the Azure portal alongside Azure SQL Managed Instances and Azure SQL Databases. This ARM projection is what makes the rest of Arc’s capabilities possible — every feature (Defender, Monitor, Best Practices Assessment) hooks into ARM resource identity.

What Arc does not do: It does not move your data. It does not replicate your databases to Azure. It does not change how SQL Server operates internally. The Arc agent has read-only access to SQL Server metadata; it cannot execute DML or DDL. DBAs coming from environments where any agent was treated as a security risk should understand this boundary clearly.

Network requirements differ from what DBAs typically manage. Arc agents must reach several Azure endpoints:

Endpoint	Purpose
<code>management.azure.com</code>	ARM operations
<code>login.microsoftonline.com</code>	Azure AD authentication
<code>*.his.arc.azure.com</code>	Hybrid Identity Service
<code>*.guestconfiguration.azure.com</code>	Policy guest configuration
<code>dc.services.visualstudio.com</code>	Telemetry

In air-gapped environments, Arc supports proxy server configuration — the agent can route all outbound traffic through an authenticated HTTP proxy. Some highly secure environments route Arc traffic through Azure Private Link by deploying a Private Link scope for Azure Monitor and Arc endpoints, though this requires significant network engineering.

Supported SQL Server versions span from SQL Server 2012 through 2022. Extended Security Updates (ESUs) through Arc are a significant licensing benefit for shops still running 2012 or 2014: Microsoft provides ESU patches free of charge for Arc-connected instances (subject to Software Assurance or Azure subscription conditions). For DBAs managing legacy infrastructure that cannot be migrated immediately, this alone often justifies the Arc onboarding effort.

18.2 Onboarding SQL Server Instances to Arc

Onboarding has three paths: single-machine (interactive), at-scale via script, and policy-driven. Interactive onboarding suits labs and initial testing. At-scale is what production environments use.

Single-machine onboarding using Azure CLI:

```
# Prerequisites: Azure CLI with arcdata extension
az extension add --name arcdata
az extension add --name connectedmachine

# Create a resource group for Arc resources
az group create \
  --name rg-arc-sqlserver-prod \
  --location eastus2

# Create a service principal for Arc onboarding
# Store credentials securely - this SP will be used by agents
az ad sp create-for-rbac \
```

```

--name "sp-arc-onboarding" \
--role "Azure Connected Machine Onboarding" \
--scopes "/subscriptions/<subscription-id>/resourceGroups/rg-arc-sqlserver-prod"

# Output:
# {
#   "appId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
#   "displayName": "sp-arc-onboarding",
#   "password": "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
#   "tenant": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
# }

# Download the onboarding script from Azure portal or generate via CLI
az connectedmachine connect \
  --resource-group rg-arc-sqlserver-prod \
  --location eastus2 \
  --subscription <subscription-id> \
  --tenant-id <tenant-id> \
  --service-principal-id <appId> \
  --service-principal-secret <password>
# Run this command ON the target SQL Server host

```

For at-scale onboarding across dozens or hundreds of servers, the recommended approach is a PowerShell script deployed via configuration management (Ansible, Chef, SCCM, or GPO):

```

# Deploy-ArcAgent.ps1
# Run this on each SQL Server host via your configuration management tool

param(
    [Parameter(Mandatory=$true)]
    [string]$TenantId,

    [Parameter(Mandatory=$true)]
    [string]$SubscriptionId,

    [Parameter(Mandatory=$true)]
    [string]$ResourceGroup,

    [Parameter(Mandatory=$true)]
    [string]$Location,

    [Parameter(Mandatory=$true)]
    [string]$ServicePrincipalId,

    [Parameter(Mandatory=$true)]
    [string]$ServicePrincipalSecret,

    [string]$Environment = "Production",
    [string]$CostCenter = "DBA-Ops"
)

# Download the Arc agent installer
$AgentInstallerUrl = "https://aka.ms/AzureConnectedMachineAgent"
$InstallerPath = "$env:TEMP\AzureConnectedMachineAgent.msi"

Write-Host "Downloading Arc Connected Machine Agent..."
Invoke-WebRequest -Uri $AgentInstallerUrl -OutFile $InstallerPath

# Install silently
Write-Host "Installing Arc Connected Machine Agent..."
msiexec.exe /i $InstallerPath /l*v "$env:TEMP\arc_install.log" /qn

# Wait for service to start
Start-Sleep -Seconds 30

# Connect to Azure Arc
$ArcConnectArgs = @(
    "connect",
    "--tenant-id", $TenantId,

```

```

    "--subscription-id", $SubscriptionId,
    "--resource-group", $ResourceGroup,
    "--location", $Location,
    "--service-principal-id", $ServicePrincipalId,
    "--service-principal-secret", $ServicePrincipalSecret,
    "--tags", "Environment=$Environment, CostCenter=$CostCenter, OnboardedBy=AutoScript"
)

& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" @ArcConnectArgs

if ($LASTEXITCODE -eq 0) {
    Write-Host "Arc agent connected successfully for $env:COMPUTERNAME"

    # Install SQL Server Arc extension
    # This happens automatically if the agent detects SQL Server,
    # but we can trigger it explicitly
    $ExtensionArgs = @(
        "extensions", "install",
        "--name", "WindowsAgent.SqlServer",
        "--publisher", "Microsoft.AzureData",
        "--version", "latest"
    )
    & "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" @ExtensionArgs
} else {
    Write-Error "Arc agent connection failed for $env:COMPUTERNAME with exit code $LASTEXITCODE"
    exit 1
}

```

After onboarding, verify the Arc SQL Server resource exists:

```

# Verify Arc-connected SQL Server instances
az sql server-arc list \
  --resource-group rg-arc-sqlserver-prod \
  --output table

# Get details of a specific instance
az sql server-arc show \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod

# List all databases discovered on the Arc SQL Server
az sql db-arc list \
  --server-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --output table

```

License type configuration is one of the most operationally important settings. Arc tracks whether each instance uses License Included, Azure Hybrid Benefit, or Pay-As-You-Go licensing. Setting this incorrectly leads to billing surprises or compliance gaps:

```

# Set Azure Hybrid Benefit license type for an Arc SQL instance
# Use this if you have existing SQL Server licenses with Software Assurance
az sql server-arc update \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --license-type LicenseIncluded

# License type options:
# LicenseIncluded - You're paying for the license through Azure (ESU scenario)
# PAYG - Pay-as-you-go (useful for dev/test)
# Paid - Azure Hybrid Benefit (bring your own SA license)

```

Gotcha: SQL Server instances with multiple named instances on the same host appear as separate Arc SQL Server resources, but they share the same Connected Machine Agent. If the agent goes offline, all instances become unreachable in Arc simultaneously. Monitor agent health separately from SQL Server health.

Gotcha: Linux-hosted SQL Server (SQL Server on RHEL, Ubuntu, SLES) is fully supported by Arc, but the extension service name differs (`mssql-server-arc` via `systemd`). Automation scripts that assume Windows service names will fail silently on Linux hosts.

18.3 Azure Monitor and Log Analytics Integration

Once Arc is connected, Azure Monitor becomes the unified observability layer for both your on-premises SQL Server and Azure SQL services. The key component is the **Log Analytics agent** (or its newer replacement, **Azure Monitor Agent / AMA**), which ships SQL Server performance counters and event log data to a Log Analytics workspace.

Configure Log Analytics data collection:

```
# Create a Log Analytics workspace
az monitor log-analytics workspace create \
  --resource-group rg-arc-sqlserver-prod \
  --workspace-name law-sqlserver-hybrid \
  --location eastus2 \
  --sku PerGB2018 \
  --retention-time 90

# Get workspace ID and key for agent configuration
WORKSPACE_ID=$(az monitor log-analytics workspace show \
  --resource-group rg-arc-sqlserver-prod \
  --workspace-name law-sqlserver-hybrid \
  --query customerId -o tsv)

WORKSPACE_KEY=$(az monitor log-analytics workspace get-shared-keys \
  --resource-group rg-arc-sqlserver-prod \
  --workspace-name law-sqlserver-hybrid \
  --query primarySharedKey -o tsv)

echo "Workspace ID: $WORKSPACE_ID"
echo "Workspace Key: $WORKSPACE_KEY"

# Enable monitoring extension on Arc SQL Server
az connectedmachine extension create \
  --machine-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --name MMA \
  --publisher Microsoft.EnterpriseCloud.Monitoring \
  --type MicrosoftMonitoringAgent \
  --settings "{\"workspaceId\": \"$WORKSPACE_ID\"}" \
  --protected-settings "{\"workspaceKey\": \"$WORKSPACE_KEY\"}"
```

Once data flows into Log Analytics, you write KQL queries against it exactly as you would for Azure SQL — the workspace abstraction normalizes the data regardless of origin:

```
// KQL: Find longest-running queries on Arc-connected SQL instances
// Run this in Log Analytics workspace
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.AZUREARCDATA"
| where Category == "SQLQueryStoreRuntime"
| where TimeGenerated > ago(1h)
| extend ExecutionDurationMs = todouble(max_elapsed_time_d) / 1000
| where ExecutionDurationMs > 5000
| project
  TimeGenerated,
  Resource,
  database_name_s,
  query_id_d,
  ExecutionDurationMs,
```

```

        total_worker_time_d,
        execution_count_d
    | order by ExecutionDurationMs desc
    | take 50

// KQL: Wait stats comparison across hybrid fleet
Perf
| where ObjectName == "SQLServer:Wait Statistics"
| where CounterName in ("Lock waits", "Log write waits", "Network IO waits")
| where TimeGenerated > ago(24h)
| summarize AvgWait = avg(CounterValue) by Computer, CounterName, bin(TimeGenerated, 1h)
| render timechart

// KQL: Identify Arc instances with high CPU over 80%
Perf
| where ObjectName == "Processor Information"
| where CounterName == "% Processor Time"
| where TimeGenerated > ago(1h)
| summarize AvgCPU = avg(CounterValue) by Computer, bin(TimeGenerated, 5m)
| where AvgCPU > 80
| join kind=inner (
    AzureDiagnostics
    | where ResourceProvider == "MICROSOFT.AZUREARCDATA"
    | distinct Computer = Resource
) on Computer
| project TimeGenerated, Computer, AvgCPU
| order by AvgCPU desc

```

Best Practices Assessment is an Arc-specific capability that runs scheduled SQL Server configuration assessments and reports findings to Log Analytics. It checks a broad set of rules covering security configuration, performance settings, availability configuration, and deprecated features — consult current Microsoft documentation for the exact rule count and categories, since these are added to over time. Unlike ad-hoc assessment tools, this runs on a schedule (weekly by default, configurable to daily) and trends results over time.

```

# Enable Best Practices Assessment on Arc SQL Server
az sql server-arc assessment create \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --workspace-id $WORKSPACE_ID \
  --schedule "0 2 * * 0" # Weekly at 2 AM Sunday (cron format)

# View assessment results via CLI
az sql server-arc assessment show \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod

# Assessment findings appear in Log Analytics under SqlAssessment_CL table

```

Query assessment findings programmatically:

```

// KQL: Critical and High severity assessment findings
SqlAssessment_CL
| where TimeGenerated > ago(7d)
| where Severity_s in ("Critical", "High")
| where IsRuleInWarningState_b == true
| project
    TimeGenerated,
    Computer,
    RuleName_s,
    Severity_s,
    Category_s,
    Description_s,
    Recommendation_s
| order by Severity_s asc, TimeGenerated desc

```

Alert rules for Arc SQL Server instances use the same Azure Monitor alert framework as Azure

SQL. The difference is that metric sources reference `microsoft.azurearcdata/sqlserverinstances` rather than `microsoft.sql/servers`. Some metrics available for Azure SQL (like DTU consumption) have no Arc equivalent since Arc instances aren't DTU-based.

```
# Create an alert rule for high CPU on Arc SQL instances
az monitor metrics alert create \
  --name "ArcSQL-HighCPU-Alert" \
  --resource-group rg-arc-sqlserver-prod \
  --scopes
  ↪ "/subscriptions/<sub-id>/resourceGroups/rg-arc-sqlserver-prod/providers/Microsoft.AzureArcData/sqlServerInstances/S
  ↪ \
  --condition "avg Processor/PercentProcessorTime > 85" \
  --window-size 5m \
  --evaluation-frequency 1m \
  --action-group "/subscriptions/<sub-id>/resourceGroups/rg-arc-sqlserver-prod/actionGroups/dba-oncall"
  ↪ \
  --description "CPU exceeded 85% on Arc-connected SQL Server SQLPROD01" \
  --severity 2
```

18.4 Microsoft Defender for SQL via Arc

Microsoft Defender for SQL provides vulnerability assessment and advanced threat protection for SQL Server instances. Without Arc, Defender for SQL was limited to Azure SQL services. Arc extends it to on-premises and multicloud SQL Server instances under the same policy and billing framework.

Enable Defender for SQL on Arc instances:

```
# Enable Defender for SQL at subscription level
# This applies to all Arc SQL instances in the subscription
az security pricing create \
  --name SqlServerVirtualMachines \
  --tier Standard

# Verify Defender is active on specific Arc instance
az security sql-vulnerability-assessment list \
  --server-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod

# Run an on-demand vulnerability assessment
az security sql-vulnerability-assessment scan \
  --server-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --database-name HRDB
```

Defender for SQL performs two distinct functions. **Vulnerability Assessment** scans SQL Server configuration against a database of known misconfigurations — things like `sa` account enabled, CLR enabled without justification, `xp_cmdshell` exposed, databases with `TRUSTWORTHY ON`, weak password policies, and over-privileged logins. It produces a baseline on first run and flags deviations from that baseline on subsequent scans.

Advanced Threat Protection analyzes SQL Server audit logs for behavioral anomalies: SQL injection patterns, unusual data export volumes, logins from unexpected locations, and brute-force login patterns. This requires SQL Server Audit to be configured and shipping logs to a storage account or Log Analytics workspace.

```
-- Configure SQL Server Audit for Defender ATP integration
-- Run on the on-premises Arc-connected SQL Server instance

-- Create server audit pointing to Log Analytics
```

```

-- For Arc instances, use FILE audit and let Arc extension ship logs
USE master;
GO

CREATE SERVER AUDIT [ArcDefenderAudit]
TO FILE (
    FILEPATH = 'D:\SQLAudit\',
    MAXSIZE = 100 MB,
    MAX_ROLLOVER_FILES = 50,
    RESERVE_DISK_SPACE = OFF
)
WITH (
    QUEUE_DELAY = 1000,
    ON_FAILURE = CONTINUE
);
GO

ALTER SERVER AUDIT [ArcDefenderAudit] WITH (STATE = ON);
GO

-- Create audit specification for Defender-relevant events
CREATE SERVER AUDIT SPECIFICATION [ArcDefenderAuditSpec]
FOR SERVER AUDIT [ArcDefenderAudit]
ADD (FAILED_LOGIN_GROUP),
ADD (SUCCESSFUL_LOGIN_GROUP),
ADD (LOGOUT_GROUP),
ADD (DATABASE_PERMISSION_CHANGE_GROUP),
ADD (SERVER_PERMISSION_CHANGE_GROUP),
ADD (DATABASE_ROLE_MEMBER_CHANGE_GROUP),
ADD (SERVER_ROLE_MEMBER_CHANGE_GROUP),
ADD (USER_CHANGE_PASSWORD_GROUP),
ADD (SCHEMA_OBJECT_ACCESS_GROUP)
WITH (STATE = ON);
GO

-- Verify audit is active
SELECT
    a.name AS audit_name,
    a.status_desc,
    a.type_desc,
    s.name AS spec_name,
    s.is_state_enabled
FROM sys.server_audits a
JOIN sys.server_audit_specifications s
    ON a.audit_guid = s.audit_guid
WHERE a.name = 'ArcDefenderAudit';

```

Defender for SQL alerts surface in Azure Security Center (now Microsoft Defender for Cloud) under the same dashboard as Azure SQL alerts. A DBA managing 200 on-premises instances previously had no centralized security alert surface for SQL injection attempts. With Arc, a brute-force attack against an on-premises SQL Server in a Chicago data center triggers the same Defender for Cloud alert workflow as an attack against an Azure SQL Database in East US.

Gotcha: Defender for SQL is billed per protected instance per month — check current Azure pricing for your region and tier before enabling it broadly. For shops with large SQL Server estates, enabling Defender at subscription scope without an instance count audit first produces unexpected billing. Run the count before enabling:

```

# Count Arc SQL instances before enabling Defender
az sql server-arc list \
  --subscription <subscription-id> \
  --query "length(@)" \
  --output tsv

# List instances with vCore count to estimate Defender cost
az sql server-arc list \

```

```
--subscription <subscription-id> \
--query "[Name:name, Location:location, vCores:properties.vCores, Edition:properties.edition]" \
--output table
```

18.5 Azure Policy for Hybrid SQL Server Governance

Azure Policy becomes your configuration enforcement mechanism across the hybrid fleet. Where you previously used SQL Server Policy-Based Management (PBM) for on-premises enforcement and separate tooling for Azure SQL, Arc unifies both under Azure Policy.

Built-in Arc SQL Server policies cover:

- SQL Server should have Azure Defender enabled
- SQL Server should have transparent data encryption enabled
- SQL Server should have the latest cumulative update installed
- SQL Server should not use the sa account
- Arc-enabled SQL Server instances should have Azure AD authentication configured

You assign these policies to management groups, subscriptions, or resource groups. Instances that drift from policy show as non-compliant in the Defender for Cloud compliance dashboard.

```
# Assign built-in policy: Defender for SQL should be enabled on Arc instances
POLICY_DEFINITION_ID=$(az policy definition list \
--query "[?displayName=='Microsoft Defender for SQL should be enabled for unprotected SQL
↵ Servers'].id" \
--output tsv)

az policy assignment create \
--name "require-defender-arcsql" \
--display-name "Require Defender for Arc SQL Servers" \
--policy $POLICY_DEFINITION_ID \
--scope "/subscriptions/<subscription-id>/resourceGroups/rg-arc-sqlserver-prod" \
--enforcement-mode Default

# Create custom policy: Ensure minimum SQL Server version
cat > custom-policy-sqlversion.json << 'EOF'
{
  "mode": "All",
  "policyRule": {
    "if": {
      "allOf": [
        {
          "field": "type",
          "equals": "Microsoft.AzureArcData/sqlServerInstances"
        },
        {
          "field": "Microsoft.AzureArcData/sqlServerInstances/version",
          "notIn": ["SQL Server 2019", "SQL Server 2022"]
        }
      ]
    },
    "then": {
      "effect": "audit"
    }
  },
  "parameters": {}
}
EOF

az policy definition create \
--name "require-sql-2019-or-later" \
```

```
--display-name "Arc SQL Server must be 2019 or later" \
--description "Audits Arc-connected SQL Servers running versions prior to SQL Server 2019" \
--rules custom-policy-sqlversion.json \
--mode All

# Assign the custom policy
az policy assignment create \
  --name "enforce-sql-version" \
  --policy "require-sql-2019-or-later" \
  --scope "/subscriptions/<subscription-id>"
```

Policy remediation tasks handle non-compliant resources automatically where possible. For example, a policy requiring TDE on all Arc SQLServer databases can include a `deployIfNotExists` effect that triggers a remediation task to enable TDE without manual DBA intervention:

```
# Create remediation task for non-compliant Arc SQL instances
az policy remediation create \
  --name "remediate-defender-arcsql" \
  --policy-assignment "require-defender-arcsql" \
  --resource-group rg-arc-sqlserver-prod \
  --resource-discovery-mode ReEvaluateCompliance

# Monitor remediation task progress
az policy remediation show \
  --name "remediate-defender-arcsql" \
  --resource-group rg-arc-sqlserver-prod \
  --query "{Status:properties.provisioningState, Succeeded:properties.resourceCounts.compliant,
  ↪ Failed:properties.resourceCounts.noncompliant}"

# List all non-compliant Arc SQL resources across subscription
az policy state list \
  --subscription <subscription-id> \
  --filter "complianceState eq 'NonCompliant' and resourceType eq
  ↪ 'Microsoft.AzureArcData/sqlServerInstances'" \
  --query "[].{Resource:resourceId, Policy:policyDefinitionName, State:complianceState}" \
  --output table
```

Guest Configuration policies extend enforcement into the OS and SQL Server configuration layer. Unlike standard Azure Policy which validates ARM resource properties, Guest Configuration runs inside the machine and can audit or enforce settings like SQL Server service account configuration, SQL Server error log retention, and SQL Agent job configurations. These policies deploy a Guest Configuration agent to the Arc-connected machine and run DSC-based configuration checks.

```
# List available Guest Configuration policy definitions for SQL Server
az policy definition list \
  --query "[?contains(displayName, 'SQL') && policyType=='BuiltIn' && contains(metadata.category, 'Guest
  ↪ Configuration')].{Name:displayName, Id:name}" \
  --output table

# Assign Guest Configuration policy for SQL Server service account validation
az policy assignment create \
  --name "gc-sql-service-account" \
  --display-name "SQL Server services should not use SA account" \
  --policy "/providers/Microsoft.Authorization/policyDefinitions/<definition-id>" \
  --scope "/subscriptions/<subscription-id>/resourceGroups/rg-arc-sqlserver-prod" \
  --mi-system-assigned \
  --location eastus2

# Guest Configuration policies require a managed identity on the assignment
# Grant the assignment's managed identity the necessary role
ASSIGNMENT_PRINCIPAL=$(az policy assignment show \
  --name "gc-sql-service-account" \
  --scope "/subscriptions/<subscription-id>/resourceGroups/rg-arc-sqlserver-prod" \
  --query identity.principalId -o tsv)

az role assignment create \
  --assignee $ASSIGNMENT_PRINCIPAL \
```

```
--role "Guest Configuration Resource Contributor" \
--scope "/subscriptions/<subscription-id>/resourceGroups/rg-arc-sqlserver-prod"
```

18.6 Arc as a Migration Bridge

Arc is frequently positioned as a permanent hybrid management solution, but its most underappreciated use case is as a **migration bridge** — a way to bring Azure governance and tooling to an on-premises instance before migrating it to Azure SQL Managed Instance, Azure SQL Database, or SQL Server on Azure VMs. This section addresses that phased migration pattern.

The migration bridge pattern works in three phases:

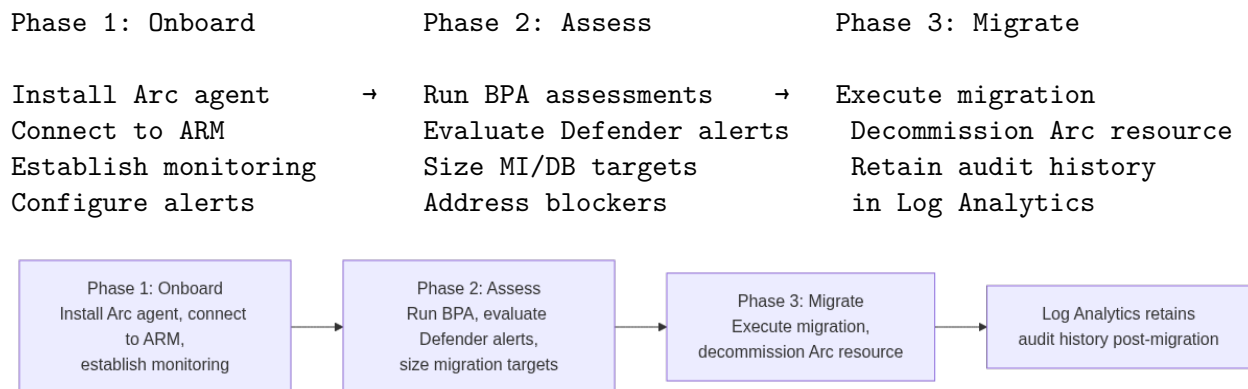


Figure 9: Arc used as a migration bridge: governance and assessment tooling attach to the on-premises instance before it is retired in favor of a native Azure SQL target.

During Phase 2, Arc provides two migration-critical capabilities: **Azure Migrate integration** and the **SQL Server assessment reports** that identify migration blockers.

```
# Create Azure Migrate project for hybrid SQL assessment
az migrate project create \
  --resource-group rg-arc-sqlserver-prod \
  --project-name migrate-sql-2024 \
  --location eastus2

# Arc-connected SQL Server instances appear automatically in Azure Migrate
# when the subscription and workspace are linked. View discovered instances:
az migrate machine list \
  --resource-group rg-arc-sqlserver-prod \
  --project-name migrate-sql-2024 \
  --output table

# The SQL Server migration extension provides target recommendation
az sql server-arc migration-assessment create \
  --name SQLPRD01 \
  --resource-group rg-arc-sqlserver-prod \
  --target-type ManagedInstance \
  --workspace-id $WORKSPACE_ID
```

The migration assessment identifies blockers such as linked servers (not supported in Azure SQL Database), cross-database queries (requires Managed Instance or Elastic Pools), CLR assemblies using unsafe or external access, distributed transactions, SQL Server Agent jobs using unsupported subsystems, and Windows Authentication without Azure AD federation configured.

Maintaining Arc connectivity during a rolling migration helps when you have multiple databases on a single instance and migrate them one at a time:

```
-- SQL Server: Track database migration status across hybrid fleet
-- Create a tracking table in a DBA management database (on-prem)
CREATE TABLE dba.MigrationTracker (
    InstanceName      NVARCHAR(128) NOT NULL,
    DatabaseName      NVARCHAR(128) NOT NULL,
    ArcResourceId      NVARCHAR(500) NULL,
    MigrationTarget    NVARCHAR(200) NULL, -- e.g., 'mi-prod.database.windows.net'
    MigrationStatus    NVARCHAR(50) NOT NULL DEFAULT 'Pending',
    -- Pending, Assessing, InProgress, Cutover, Complete
    AssessmentDate     DATETIME2 NULL,
    MigrationStartDate DATETIME2 NULL,
    CutoverDate        DATETIME2 NULL,
    BlockerCount       INT NULL,
    Notes              NVARCHAR(MAX) NULL,
    ModifiedBy         NVARCHAR(128) NOT NULL DEFAULT SYSUSER_SNAME(),
    ModifiedDate       DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_MigrationTracker PRIMARY KEY (InstanceName, DatabaseName)
);
GO

-- Populate with current databases
INSERT INTO dba.MigrationTracker (InstanceName, DatabaseName)
SELECT
    @@SERVERNAME,
    name
FROM sys.databases
WHERE name NOT IN ('master', 'model', 'msdb', 'tempdb')
AND state_desc = 'ONLINE';
GO

-- Query migration status
SELECT
    InstanceName,
    MigrationStatus,
    COUNT(*) AS DatabaseCount,
    MIN(AssessmentDate) AS EarliestAssessment,
    MAX(CutoverDate) AS LatestCutover
FROM dba.MigrationTracker
GROUP BY InstanceName, MigrationStatus
ORDER BY InstanceName, MigrationStatus;
```

After migration, clean up Arc resources for instances that have been fully migrated:

```
# Verify no remaining databases before disconnecting Arc
az sql db-arc list \
  --server-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --output table

# Disconnect the Arc SQL Server extension first
az connectedmachine extension delete \
  --machine-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --name WindowsAgent.SqlServer \
  --yes

# Disconnect the machine from Arc
# (Run this on the host itself OR remotely via ARM)
az connectedmachine delete \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --yes

# Or from the host, run the agent disconnect command:
# azcmagent disconnect --service-principal-id <id> --service-principal-secret <secret>
```

```
# Retain Log Analytics data per your retention policy
# The workspace data persists independently of Arc resource deletion
```

Gotcha: Deleting the Arc connected machine resource in ARM does not uninstall the Arc agent from the host OS. The agent continues running but in a disconnected state. On decommissioned hosts this is harmless; on hosts that will be repurposed, uninstall the agent explicitly using Add/Remove Programs on Windows or the package manager on Linux before reprovisioning.

18.7 Azure AD Authentication for Arc SQL Server

One of the more transformative capabilities Arc enables for on-premises SQL Server is **Azure Active Directory (Azure AD) authentication**, which Microsoft now calls Microsoft Entra ID authentication. Traditionally, on-premises SQL Server relied on Windows Authentication against local Active Directory, or SQL authentication. Arc-enabled SQL Server 2022 can authenticate users directly against Azure AD without requiring Azure AD Connect or AD Federation Services — the Arc agent handles the token validation.

This matters for hybrid environments where:

- Remote workers lack VPN access to on-premises AD but have Azure AD credentials
- Azure-native applications need to connect to on-premises SQL Server using managed identities
- You want unified audit trails across Azure SQL and on-premises SQL Server

Prerequisites for Azure AD auth on Arc SQL Server:

1. SQL Server 2022 (Azure AD auth is not available for earlier versions via Arc)
2. Arc Connected Machine Agent version 1.13 or later
3. Azure AD admin assigned to the SQL Server instance
4. Outbound connectivity to Azure AD endpoints

```
# Set Azure AD admin for Arc SQL Server instance
az sql server-arc ad-admin create \
  --name SQLPRD01 \
  --resource-group rg-arc-sqlserver-prod \
  --display-name "SQL Arc Admins" \
  --object-id <azure-ad-group-object-id> \
  --type Group

# Verify AD admin assignment
az sql server-arc ad-admin show \
  --name SQLPRD01 \
  --resource-group rg-arc-sqlserver-prod
```

Once configured, create Azure AD logins using T-SQL syntax:

```
-- SQL Server 2022: Create Azure AD logins on Arc-connected instance
-- Connect as sysadmin first

-- Create login for an Azure AD user
CREATE LOGIN [dbadmin@contoso.com]
FROM EXTERNAL PROVIDER;
GO

-- Create login for an Azure AD group
CREATE LOGIN [SQL-DBAs@contoso.com]
FROM EXTERNAL PROVIDER;
GO

-- Create login for an Azure managed identity
```

```

-- Useful when Azure Functions or App Service needs to connect to on-prem SQL via Arc
CREATE LOGIN [my-azure-function-app]
FROM EXTERNAL PROVIDER;
GO

-- Grant server-level permissions
ALTER SERVER ROLE [sysadmin] ADD MEMBER [SQL-DBAs@contoso.com];
GO

-- Create database user mapped to the Azure AD login
USE [HRDB];
GO

CREATE USER [dbadmin@contoso.com] FROM LOGIN [dbadmin@contoso.com];
GO

ALTER ROLE [db_datareader] ADD MEMBER [dbadmin@contoso.com];
ALTER ROLE [db_datawriter] ADD MEMBER [dbadmin@contoso.com];
GO

-- Verify Azure AD logins
SELECT
    name,
    type_desc,
    authentication_type_desc,
    create_date,
    is_disabled
FROM sys.server_principals
WHERE type IN ('E', 'X') -- E = External User, X = External Group
ORDER BY create_date DESC;

```

Connecting to Arc SQL Server using Azure AD authentication:

```

# Python: Connect to Arc-enabled SQL Server using Azure AD token
# Uses azure-identity and pyodbc

import pyodbc
from azure.identity import DefaultAzureCredential
import struct

def get_azure_ad_token():
    """Acquire Azure AD token for SQL Server authentication."""
    credential = DefaultAzureCredential()
    # Resource ID for SQL Server
    token = credential.get_token("https://database.windows.net/.default")
    return token.token

def connect_arc_sql_server(server_name: str, database: str) -> pyodbc.Connection:
    """
    Connect to Arc-enabled SQL Server using Azure AD authentication.
    server_name: FQDN or IP of on-premises SQL Server
    database: Target database name
    """
    token = get_azure_ad_token()

    # Convert token to bytes for ODBC access token attribute
    token_bytes = token.encode('utf-16-le')
    token_struct = struct.pack(f'<I{len(token_bytes)}s',
                               len(token_bytes),
                               token_bytes)

    connection_string = (
        f"DRIVER={{ODBC Driver 18 for SQL Server}};"
        f"SERVER={server_name};"
        f"DATABASE={database};"
        f"Encrypt=yes;"
        f"TrustServerCertificate=no;"
    )

    conn = pyodbc.connect(

```



```

        connection_string,
        attrs_before={1256: token_struct} # SQL_COPT_SS_ACCESS_TOKEN
    )
    return conn

# Usage
conn = connect_arc_sql_server("sqlprod01.corp.contoso.com", "HRDB")
cursor = conn.cursor()
cursor.execute("SELECT SYSTEM_USER, USER_NAME(), @@SERVERNAME")
row = cursor.fetchone()
print(f"Logged in as: {row[0]}, User: {row[1]}, Server: {row[2]}")
conn.close()

```

Gotcha: Azure AD authentication on Arc SQL Server 2022 requires the SQL Server certificate to be trusted by clients. Self-signed certificates generated during SQL Server installation will cause connection failures with `TrustServerCertificate=no`. Either issue a certificate from an enterprise CA or update clients to trust the self-signed certificate during a transition period — but do not leave `TrustServerCertificate=yes` in production connection strings permanently.

18.8 Operational Runbooks for Arc-Managed SQL Server

Managing an Arc-connected SQL Server fleet requires operational procedures that account for both the Azure control plane and the on-premises SQL Server layer. This section documents runbook patterns for the most common operational scenarios.

Runbook: Arc agent health check and remediation

Arc agent connectivity issues are the most common operational problem. The agent can lose connectivity due to proxy changes, certificate expiration, service account password rotation, or network policy changes. Detect and remediate with this pattern:

```

#!/bin/bash
# arc-agent-health-check.sh
# Run from Azure Cloud Shell or from a jumpbox with Azure CLI access

RESOURCE_GROUP="rg-arc-sqlserver-prod"
SUBSCRIPTION_ID="<subscription-id>"

echo "=== Arc Agent Health Check: $(date) ==="

# List all Arc machines and their connectivity status
az connectedmachine list \
  --resource-group $RESOURCE_GROUP \
  --query "[{.Name:name, Status:properties.status, LastHeartbeat:properties.lastStatusChange,
    ↪ AgentVersion:properties.agentVersion}]" \
  --output table

# Find machines that haven't reported in over 30 minutes
STALE_THRESHOLD=$(date -u -d "30 minutes ago" '+%Y-%m-%dT%H:%M:%SZ')
echo ""
echo "=== Machines with stale heartbeat (> 30 min) ==="
az connectedmachine list \
  --resource-group $RESOURCE_GROUP \
  --query "[?properties.lastStatusChange < '$STALE_THRESHOLD'}.{Name:name,
    ↪ LastHeartbeat:properties.lastStatusChange, Status:properties.status}]" \
  --output table

# Check agent version - identify outdated agents
echo ""
echo "=== Agent versions in fleet ==="
az connectedmachine list \

```

```
--resource-group $RESOURCE_GROUP \
--query "[Name:name, AgentVersion:properties.agentVersion]" \
--output table
```

On the host side, diagnose agent connectivity issues:

```
# Diagnose-ArcAgent.ps1
# Run on the Arc-connected host when connectivity issues are reported

# Check agent service status
Get-Service -Name "himds" | Select-Object Name, Status, StartType
Get-Service -Name "GCArcService" | Select-Object Name, Status, StartType

# Check agent connectivity
& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" check

# Common output codes:
# OK - All endpoints reachable
# FAILED - Cannot reach Azure endpoints (proxy/firewall issue)
# WARNING - Partial connectivity

# View agent logs
Get-Content "C:\ProgramData\AzureConnectedMachineAgent\Log\himds.log" -Tail 100

# Test specific endpoint connectivity
$endpoints = @(
    "management.azure.com",
    "login.microsoftonline.com",
    "guestnotificationservice.azure.com"
)

foreach ($endpoint in $endpoints) {
    $result = Test-NetConnection -ComputerName $endpoint -Port 443 -WarningAction SilentlyContinue
    Write-Host "$endpoint : $(if($result.TcpTestSucceeded){'REACHABLE'}else{'UNREACHABLE'})"
}

# Restart agent if connectivity checks pass but agent shows disconnected
if ((& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" check) -match "OK") {
    Restart-Service -Name "himds" -Force
    Start-Sleep -Seconds 15
    Get-Service -Name "himds" | Select-Object Name, Status
}
```

Runbook: Bulk license type audit and correction

In large estates, license types accumulate errors over time as servers are onboarded by different teams. This runbook identifies and corrects inconsistencies:

```
#!/bin/bash
# audit-arc-license-types.sh

RESOURCE_GROUP="rg-arc-sqlserver-prod"

echo "=== Arc SQL License Type Audit ==="

# Export all instances with their current license types
az sql server-arc list \
--resource-group $RESOURCE_GROUP \
--query "[Name:name, LicenseType:properties.licenseType, Edition:properties.edition,
  Version:properties.version, vCores:properties.vCores]" \
--output table

# Count by license type
echo ""
echo "=== License Type Summary ==="
az sql server-arc list \
--resource-group $RESOURCE_GROUP \
--query "[].properties.licenseType" \
--output tsv | sort | uniq -c | sort -rn
```

```
# Identify PAYG instances that should be Azure Hybrid Benefit
# (instances with 'Production' tag but PAYG license type - common misconfiguration)
echo ""
echo "=== Potential AHB Candidates (Production tagged, PAYG licensed) ==="
az sql server-arc list \
  --resource-group $RESOURCE_GROUP \
  --query "[?tags.Environment=='Production' && properties.licenseType=='PAYG'].{Name:name,
  ↪ LicenseType:properties.licenseType}" \
  --output table

# Correct license type for specific instances (requires approval workflow in production)
# Example: Update a single instance to LicenseIncluded (ESU scenario)
# az sql server-arc update \
#   --name SQLLEGACY01 \
#   --resource-group $RESOURCE_GROUP \
#   --license-type LicenseIncluded
```

Runbook: Forced Best Practices Assessment and report export

Before a major SQL Server upgrade or migration, trigger an on-demand BPA run and export results:

```
# Trigger immediate BPA assessment on all Arc SQL instances in resource group
for instance in $(az sql server-arc list \
  --resource-group rg-arc-sqlserver-prod \
  --query "[].name" --output tsv); do

  echo "Triggering assessment for $instance..."
  az sql server-arc assessment create \
    --name $instance \
    --resource-group rg-arc-sqlserver-prod \
    --workspace-id $WORKSPACE_ID

  echo "Assessment triggered for $instance"
done
```

Then extract results from Log Analytics:

```
// KQL: Export full BPA findings for pre-migration report
SqlAssessment_CL
| where TimeGenerated > ago(24h)
| where IsRuleInWarningState_b == true
| project
  AssessmentTimestamp = TimeGenerated,
  ServerName = Computer,
  Category = Category_s,
  Severity = Severity_s,
  RuleId = RuleId_s,
  RuleName = RuleName_s,
  Description = Description_s,
  Recommendation = Recommendation_s,
  AffectedObjects = AffectedObjects_s
| order by Severity asc, Category asc, ServerName asc
```

18.9 Limitations and When Not to Use Arc

Arc solves real problems, but it is not the right tool in every scenario. Understanding the limitations prevents over-engineering.

Arc does not provide real-time query performance insights equivalent to Query Store UI in SSMS. The data that flows to Log Analytics has ingestion latency — typically two to five minutes, sometimes longer under high telemetry load. For active performance troubleshooting of

an in-flight incident, DBAs still connect directly to the SQL Server instance using SSMS, Azure Data Studio, or direct T-SQL queries against DMVs. Arc is a fleet management and governance layer, not a low-latency diagnostic tool.

Arc agent overhead is minimal but not zero. The Connected Machine Agent and SQL Server extension consume approximately 200-400 MB of memory on the host and generate periodic CPU spikes during telemetry collection windows. On memory-constrained legacy servers (32 GB or less) running large SQL workloads, this overhead should be evaluated before mass rollout.

Connectivity dependency is real. If an Arc-connected instance loses outbound internet access, it continues running SQL Server normally — Arc has no operational dependency on Azure connectivity. However, policy evaluations stop updating, Defender alerts stop flowing, monitoring gaps appear in Log Analytics, and BPA assessments fail silently. Design monitoring for Arc agent connectivity as a separate health signal.

Arc is not a replication or high availability solution. It provides no data-layer capabilities. DBAs who encounter Arc for the first time sometimes conflate it with Azure SQL Managed Instance Link (which does provide replication). Arc manages metadata; Managed Instance Link synchronizes data. These are complementary, not competing.

Sovereign cloud and regulatory constraints affect Arc deployments. Arc endpoints route through Azure’s commercial cloud by default. Organizations subject to data sovereignty regulations in jurisdictions where Azure Government, Azure China, or Azure Germany regions apply must configure Arc to use the appropriate sovereign cloud endpoints — and some highly regulated industries prohibit any management-plane data leaving specific geographic boundaries, which Arc’s current architecture does not accommodate.

```
# Configure Arc for Azure Government cloud
# (Must be done before connecting the agent)
& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" config set cloud "AzureUSGovernment"

# Or for Azure China
& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" config set cloud "AzureChinaCloud"

# Verify cloud configuration
& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" config list
```

Key Takeaways

- **Arc projects on-premises SQL Server into Azure Resource Manager as a first-class resource**, enabling unified RBAC, tagging, policy enforcement, and monitoring through the same Azure portal and API surface used for Azure SQL services — without moving any data or changing SQL Server’s operational behavior.
- **Onboarding uses an outbound-only HTTPS connection** via the Connected Machine Agent; no inbound firewall rules are required. At-scale onboarding through configuration management tools (Ansible, SCCM, GPO) is the production approach, and license type configuration at onboarding time prevents billing discrepancies.
- **Azure Monitor, Log Analytics KQL queries, Defender for SQL, and Best Practices Assessment** all extend to Arc-connected instances under the same frameworks used for Azure

SQL, eliminating parallel monitoring and security toolchains for hybrid fleets — though Arc telemetry has ingestion latency unsuitable for real-time incident diagnostics.

- **Arc serves as a migration bridge**, not just a permanent hybrid solution: its integration with Azure Migrate and SQL Server assessment reporting surfaces migration blockers, tracks per-database migration status, and preserves historical audit data in Log Analytics that persists after the Arc resource is decommissioned.
 - **Azure AD authentication for Arc SQL Server 2022 enables token-based authentication** from Azure-native workloads and remote users without AD Federation Services or Azure AD Connect, but requires properly trusted certificates on the SQL Server host — self-signed certificates must be addressed before enabling this capability in production.
-

Chapter 19: Cloud SQL: PostgreSQL and SQL Server on GCP

Google Cloud SQL is the workhorse managed database service on GCP — not the most glamorous product in Google’s portfolio, but the one that most production teams actually run day-to-day. This chapter covers two of its three supported engines: PostgreSQL and SQL Server. If you’ve spent years tuning `postgresql.conf`, managing WAL archiving, and nursing SQL Server AlwaysOn through network partitions, Cloud SQL will feel both familiar and occasionally frustrating. Google handles the infrastructure, but the tradeoffs are real: you gain automated patching, HA failover, and integrated backups while giving up superuser access, direct OS-level tuning, and certain PostgreSQL extensions you may have relied on. This chapter maps your existing DBA knowledge onto Cloud SQL’s architecture, walks through production configuration for both engines, covers HA and replication patterns, and shows you how IAM authentication, Private Service Connect, and Cloud SQL Auth Proxy replace the access-control mechanisms you’ve used on-premises.

19.1 Cloud SQL Architecture: What GCP Manages and What You Still Own

Cloud SQL instances run on managed VMs inside Google’s infrastructure. Unlike AWS RDS, which abstracts the underlying VM almost completely from day one, Cloud SQL has historically been more transparent — you can see machine types (shared-core, standard, high-memory), attached SSD storage, and even the regional zone placement directly in the console and API.

What Google manages:

- OS patching and kernel updates (maintenance windows apply)
- Storage encryption at rest (Google-managed or customer-managed CMEK)
- Automated backups and point-in-time recovery (PITR)
- High availability via synchronous regional replication
- Minor and major version upgrades (in-place for PostgreSQL, with caveats)
- The physical failover between primary and standby

What you still own:

- Database schema and query performance
- Flag configuration (the Cloud SQL equivalent of `postgresql.conf` and SQL Server `sp_configure`)

- User management and privilege grants
- Connection pooling (Cloud SQL does not include a built-in proxy equivalent to PgBouncer — the Auth Proxy handles TLS tunneling, not pooling)
- Logical replication setup for outbound replication to external consumers
- Extension management within the allowed extension list

The superuser question: On Cloud SQL for PostgreSQL, you never get a true PostgreSQL superuser. The default administrative user (`postgres`) is a highly privileged user, but certain system-level operations are blocked. You cannot run `pg_read_file()`, cannot directly manipulate `pg_hba.conf`, and cannot install arbitrary extensions from source. The allowed extension list is maintained by Google and includes common extensions such as `pgvector`, `pg_stat_statements`, `postgis`, `pg_partman`, and `pglogical` — but not `plsh`, `multicorn`, or any extension requiring loading of an external `.so` not included in the base image. Check the current Cloud SQL documentation for the exact list per PostgreSQL version, since it changes over time.

For SQL Server, Cloud SQL runs SQL Server 2017, 2019, and 2022 across Enterprise, Standard, and Express editions. You get `sysadmin`-equivalent access via the `sqlserver` user, but you cannot execute `xp_cmdshell` (disabled and blocked at the API level), cannot directly manage SQL Server Agent jobs through T-SQL in the traditional sense (the agent is present but surfaced through GCP APIs), and cannot access the underlying file system.

Instance sizing follows Google Compute Engine machine families:

Tier	vCPUs	RAM	Max Storage	Max Connections (PG)
db-f1-micro	1 (shared)	0.6 GB	3 TB	~25
db-n1-standard-4	4	15 GB	64 TB	~500
db-n1-highmem-8	8	52 GB	64 TB	~1000
db-custom-16-65536	16	64 GB	64 TB	~2000
db-custom-96-393216	96	384 GB	64 TB	~12000

The connection limits shown are practical guideline values — Cloud SQL does not hard-cap connections at these numbers by default, but PostgreSQL’s `max_connections` defaults to 100 for `db-f1-micro` and scales with memory. For anything above 500 connections, you should plan your connection pooling architecture before deployment, not after.

19.2 Provisioning Cloud SQL Instances: PostgreSQL and SQL Server

Provisioning with gcloud

The fastest path to a production-ready Cloud SQL instance uses `gcloud sql instances create`. Every flag here maps to something you configured manually on-premises.

```
# Create a Cloud SQL for PostgreSQL 15 instance
# -- equivalent to provisioning a new PostgreSQL server with WAL archiving,
#   streaming replication standby, and daily base backups preconfigured

gcloud sql instances create prod-pg-01 \
  --database-version=POSTGRES_15 \
```



```

--tier=db-custom-8-32768 \
--region=us-central1 \
--availability-type=REGIONAL \
--storage-type=SSD \
--storage-size=500GB \
--storage-auto-increase \
--backup-start-time=02:00 \
--enable-bin-log \
--retained-backups-count=14 \
--retained-transaction-log-days=7 \
--maintenance-window-day=SUN \
--maintenance-window-hour=4 \
--deletion-protection \
--no-assign-ip \
--network=projects/my-project/global/networks/prod-vpc

# Create the default database and application user
gcloud sql databases create appdb --instance=prod-pg-01

gcloud sql users create appuser \
--instance=prod-pg-01 \
--password=CHANGE_ME_USE_SECRET_MANAGER

```

A few things worth noting here:

- `--availability-type=REGIONAL` enables synchronous HA replication to a standby in a second zone within the same region. This is not cross-region. The standby is not readable — it exists only for failover. Cross-region reads require read replicas configured separately.
- `--enable-bin-log` on PostgreSQL enables WAL archiving for PITR. Without it, you only get daily backup restoration. For production, this flag is non-negotiable.
- `--no-assign-ip` with `--network` places the instance on a private IP in your VPC. Always do this. Public IP Cloud SQL instances exist, but deploying them in production means relying entirely on authorized networks IP allowlisting, which is a fragile access control model.

Cloud SQL for SQL Server:

```

# Create Cloud SQL for SQL Server 2022 Standard Edition
gcloud sql instances create prod-mssql-01 \
--database-version=SQLSERVER_2022_STANDARD \
--tier=db-custom-8-32768 \
--region=us-east1 \
--availability-type=REGIONAL \
--storage-type=SSD \
--storage-size=1TB \
--storage-auto-increase \
--backup-start-time=03:00 \
--retained-backups-count=14 \
--maintenance-window-day=SAT \
--maintenance-window-hour=2 \
--deletion-protection \
--no-assign-ip \
--network=projects/my-project/global/networks/prod-vpc \
--root-password=CHANGE_ME_USE_SECRET_MANAGER

# Create a database
gcloud sql databases create SalesDB --instance=prod-mssql-01

# Create a SQL login
gcloud sql users create sqladmin \
--instance=prod-mssql-01 \
--password=CHANGE_ME_USE_SECRET_MANAGER

```

Provisioning with Terraform is the right pattern for production environments where you need reproducible, version-controlled infrastructure:

```
# terraform/modules/cloud_sql_postgres/main.tf
```

```
resource "google_sql_database_instance" "prod_pg" {
  name           = "prod-pg-01"
  database_version = "POSTGRES_15"
  region         = "us-central1"
  deletion_protection = true

  settings {
    tier                = "db-custom-8-32768"
    availability_type   = "REGIONAL"
    disk_type           = "PD_SSD"
    disk_size           = 500
    disk_autoresize     = true

    backup_configuration {
      enabled                = true
      start_time              = "02:00"
      point_in_time_recovery_enabled = true
      transaction_log_retention_days = 7
      backup_retention_settings {
        retained_backups = 14
        retention_unit    = "COUNT"
      }
    }
  }

  maintenance_window {
    day      = 7 # Sunday
    hour     = 4
    update_track = "stable"
  }

  ip_configuration {
    ipv4_enabled = false
    private_network = "projects/${var.project_id}/global/networks/prod-vpc"
  }

  database_flags {
    name = "max_connections"
    value = "500"
  }

  database_flags {
    name = "log_min_duration_statement"
    value = "1000" # Log queries slower than 1 second
  }

  database_flags {
    name = "pg_stat_statements.track"
    value = "all"
  }

  insights_config {
    query_insights_enabled = true
    query_string_length    = 4096
    record_application_tags = true
    record_client_address   = true
  }
}

resource "google_sql_database" "appdb" {
  name     = "appdb"
  instance = google_sql_database_instance.prod_pg.name
}

resource "google_sql_user" "appuser" {
  name     = "appuser"
  instance = google_sql_database_instance.prod_pg.name
  password = var.db_password # Sourced from Secret Manager via data source
}
```

The `insights_config` block enables **Query Insights**, which is Cloud SQL’s integrated slow query analysis tool — think of it as a managed `pg_stat_statements` dashboard built into the GCP console. It captures query plans, wait events, and client attribution. For PostgreSQL shops accustomed to `pg_stat_statements` + `pgBadger`, this is a useful first-line tool, though it doesn’t replace dedicated APM for complex workloads.

19.3 Database Flags: Configuring PostgreSQL and SQL Server Parameters

What you previously controlled through `postgresql.conf` and `pg_hba.conf` (PostgreSQL) or `sp_configure` and SQL Server Configuration Manager (SQL Server) is surfaced in Cloud SQL as **database flags**. Most parameters are settable; some are fixed by Google and cannot be changed regardless of your flag configuration.

Fixed parameters in Cloud SQL for PostgreSQL (you cannot override these):

- `wal_level` — fixed at `logical` (actually beneficial for logical replication)
- `archive_mode` — managed by Google for PITR
- `ssl` — always on
- `log_destination` — managed by Cloud Logging
- `shared_preload_libraries` — you can add to this list from approved extensions, but certain libraries are always present

Configurable flags (selected important ones):

```
# Apply multiple performance flags to a PostgreSQL instance  
# Note: Some flags require instance restart; Cloud SQL will warn you
```

```
gcloud sql instances patch prod-pg-01 \  
  --database-flags=\  
max_connections=500,\  
shared_buffers=8192MB,\  
work_mem=64MB,\  
maintenance_work_mem=2GB,\  
effective_cache_size=24576MB,\  
checkpoint_completion_target=0.9,\  
wal_buffers=64MB,\  
random_page_cost=1.1,\  
log_min_duration_statement=1000,\  
log_checkpoints=on,\  
log_connections=on,\  
log_lock_waits=on,\  
autovacuum_vacuum_scale_factor=0.02,\  
autovacuum_analyze_scale_factor=0.01,\  
pg_stat_statements.track=all,\  
track_io_timing=on
```

An important gotcha: Cloud SQL does not let you set `shared_buffers` directly by name on some tiers — the flag name is `shared_buffers` and the value must be expressed in kB or MB, not the shorthand notation PostgreSQL accepts directly. Always check the [Cloud SQL flags documentation](#) for your specific database version because accepted value formats differ from what you’d type in `postgresql.conf`.

For `work_mem`: be conservative. `work_mem=64MB` sounds modest but with 200 concurrent connections each potentially running a sort or hash join, your effective RAM usage for working memory alone is $200 \times 64 \text{ MB} \times (\text{number of sort operations per query}) = \text{easily } 25+ \text{ GB}$ on a busy OLTP instance. The number in `postgresql.conf` is per-sort-node, not per-connection — this is a footgun that causes OOM kills even on generously sized instances.

SQL Server flags are surfaced similarly but map to `sp_configure` options:

```
# Configure SQL Server instance flags
gcloud sql instances patch prod-mssql-01 \
  --database-flags=\
max\ degree\ of\ parallelism=4,\
cost\ threshold\ for\ parallelism=50,\
optimize\ for\ ad\ hoc\ workloads=on,\
remote\ query\ timeout\ \((s)=600,\
fill\ factor\ \((\%)=80
```

What you cannot do with SQL Server on Cloud SQL: - `xp_cmdshell` is permanently disabled. There is no workaround. - Linked servers are supported in limited form (to other Cloud SQL instances and Cloud Spanner via a preview connector), but not to arbitrary external servers. - SQL Server Integration Services (SSIS) is not available. For ETL, you're expected to use Dataflow or Cloud Data Fusion. - SQL Server Reporting Services (SSRS) is not available. - SQL Server Analysis Services (SSAS) is not available. - Database mirroring is replaced by the HA standby (which is automatic and not configurable as a traditional mirroring endpoint).

If your current SQL Server environment depends heavily on SSIS packages, linked servers to Oracle, or SSRS-based reporting, Cloud SQL is not the right target without significant architectural changes.

19.4 High Availability, Replication, and Failover

Regional HA in Cloud SQL works through synchronous replication to a standby instance in a different zone within the same region. This is conceptually similar to PostgreSQL synchronous streaming replication (`synchronous_commit = remote_apply`) but implemented entirely by Google's infrastructure layer. You don't configure replication slots, `recovery.conf`, or standby `postgresql.conf`. When you set `--availability-type=REGIONAL`, the standby materializes automatically.

Failover characteristics: - **Automatic failover** triggers when the primary becomes unavailable, typically completing in 60–120 seconds. This is not near-instantaneous like some marketing materials imply. - The standby is in a **different zone** but the **same region**. A full regional outage takes down both primary and standby. - The standby is **not readable**. There is no hot-standby query capability on the HA standby. For read scaling, you need read replicas. - After failover, the old primary becomes the standby. You can initiate a manual failover (called a “switchover” in Cloud SQL) to return the primary to the original zone.

```
# Initiate a manual failover (useful for DR testing)
gcloud sql instances failover prod-pg-01

# Check replication status
gcloud sql instances describe prod-pg-01 \
  --format="value(settings.availabilityType,failoverReplica)"
```

Read replicas are separate Cloud SQL instances that receive asynchronous replication from the primary:

```
# Create a read replica in a different region (cross-region replica)
gcloud sql instances create prod-pg-01-replica-us-east \
  --master-instance-name=prod-pg-01 \
  --region=us-east1 \
  --tier=db-custom-4-16384 \
  --availability-type=ZONAL \
```

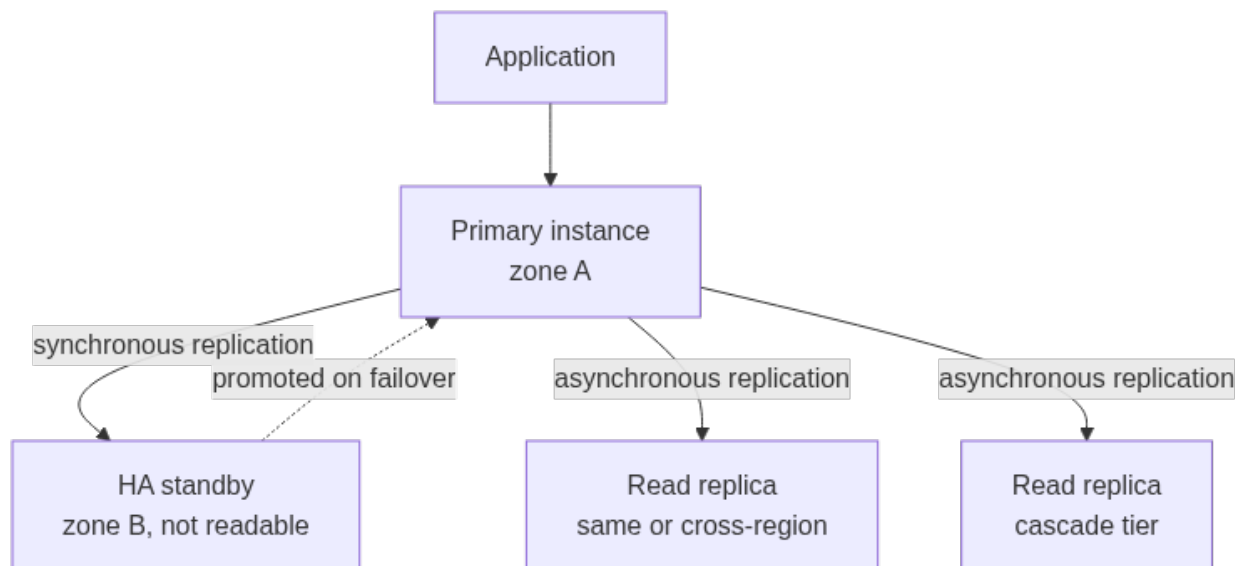


Figure 10: Regional HA gives synchronous, non-readable failover protection within one region; read replicas are separate, asynchronously replicated instances used for read scaling and cross-region access.

```

--no-assign-ip \
--network=projects/my-project/global/networks/prod-vpc

# Promote a replica to a standalone instance (for migrations or DR activation)
gcloud sql instances promote-replica prod-pg-01-replica-us-east

```

Cascade replicas (replicas of replicas) are supported in Cloud SQL for PostgreSQL — useful when you want to offload replication traffic from the primary by having multiple tiers of read replicas without every replica pulling directly from the primary.

Replication lag monitoring should be part of your operational baseline. Cloud SQL surfaces replication lag as a Cloud Monitoring metric:

```

# Query replication lag for all replicas using gcloud monitoring
# (More practical to set up an alerting policy via Terraform)
gcloud monitoring metrics list \
  --filter="metric.type=cloudsql.googleapis.com/database/replication/replica_lag"

```

In practice, you want an alerting policy that fires when `replica_lag` exceeds 30 seconds for OLTP replicas used for read traffic, and perhaps 300 seconds for reporting replicas. Lag above 30 seconds on a read replica that's serving application queries means users are reading stale data that may materially affect application behavior.

External replication — Cloud SQL supports acting as a replica of an external PostgreSQL primary (for live migrations from on-premises) and also supports serving as a primary to external PostgreSQL replicas via logical replication (using `pglogical` or native logical replication in PostgreSQL 10+). This architecture is commonly used for zero-downtime migrations:

```

-- On your on-premises PostgreSQL primary:
-- Enable logical replication (wal_level must be logical)
-- Create a replication publication
CREATE PUBLICATION cloud_migration_pub
  FOR TABLE orders, customers, products, order_items;

```

```
-- On Cloud SQL (after setting up as external replica):
-- Cloud SQL handles the subscription setup via the import/replica mechanism,
-- but for pglogical you would configure it as follows after enabling the extension:

CREATE EXTENSION pglogical;

SELECT pglogical.create_node(
  node_name := 'cloud_sql_subscriber',
  dsn := 'host=127.0.0.1 port=5432 dbname=appdb'
);

SELECT pglogical.create_subscription(
  subscription_name := 'onprem_to_cloudsql',
  provider_dsn := 'host=10.0.1.50 port=5432 dbname=appdb
                  user=replication_user password=REPL_PASS',
  replication_sets := ARRAY['default']
);
```

19.5 IAM Authentication and Network Security

On-premises PostgreSQL access control lives in two places: `pg_hba.conf` controls who can connect from where and via which authentication method, and `pg_ident.conf` handles OS user mapping. In Cloud SQL, this model is replaced — and improved in some ways — by IAM integration and Cloud SQL Auth Proxy.

IAM database authentication lets you authenticate to PostgreSQL or SQL Server using a Google Cloud service account or user identity token instead of a database password. This is meaningful for production because it eliminates the credential rotation problem: IAM tokens are short-lived (1 hour), and service accounts can be centrally managed through IAM policies without touching the database.

```
# Grant IAM authentication to a service account for PostgreSQL
# The user format is the service account email address

gcloud sql users create "myapp-sa@my-project.iam.gserviceaccount.com" \
  --instance=prod-pg-01 \
  --type=CLOUD_IAM_SERVICE_ACCOUNT

# Grant IAM authentication to a user account (for human access)
gcloud sql users create "dba@mycompany.com" \
  --instance=prod-pg-01 \
  --type=CLOUD_IAM_USER

# Then grant database-level permissions using standard PostgreSQL GRANT
# Connect as the postgres user first, then:

-- Grant the IAM service account access to the application database
-- The IAM user appears in PostgreSQL as the email address (service account)
-- or as the user portion before @ for human accounts
GRANT CONNECT ON DATABASE appdb TO "myapp-sa@my-project.iam.gserviceaccount.com";
GRANT USAGE ON SCHEMA public TO "myapp-sa@my-project.iam.gserviceaccount.com";
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public
  TO "myapp-sa@my-project.iam.gserviceaccount.com";

-- For new tables created in the future:
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES
  TO "myapp-sa@my-project.iam.gserviceaccount.com";
```

Cloud SQL Auth Proxy handles TLS tunnel establishment and IAM-based authentication. Think of it as the operational layer between your application and the database — it intercepts

connections, authenticates to Google's APIs, and forwards traffic over an encrypted tunnel. It is not a connection pooler. Applications that open many short-lived connections still need PgBouncer or `pgpool-II` in front of the Auth Proxy.

```
# Download and run Cloud SQL Auth Proxy (Linux/amd64)
curl -o cloud-sql-proxy \
  https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy/v2.7.0/cloud-sql-proxy.linux.amd64
chmod +x cloud-sql-proxy

# Start proxy for a single PostgreSQL instance
# Applications connect to localhost:5432
./cloud-sql-proxy \
  --address 127.0.0.1 \
  --port 5432 \
  my-project:us-central1:prod-pg-01

# For Kubernetes deployments, run Auth Proxy as a sidecar container
# The GKE workload identity binds the pod's service account to the GCP service account
# No credential files needed - identity is automatic

# For multiple instances (useful in multi-tenant architectures)
./cloud-sql-proxy \
  my-project:us-central1:prod-pg-01?port=5432 \
  my-project:us-central1:prod-pg-02?port=5433 \
  my-project:us-east1:prod-pg-01-replica-us-east?port=5434
```

Private Service Connect is Google's newer private connectivity mechanism, replacing the older Private IP approach for certain architectures. Where Private IP places the Cloud SQL instance directly on your VPC subnet via a VPC peering arrangement, Private Service Connect creates a private endpoint in your VPC that routes to a Cloud SQL instance without requiring VPC peering or consuming IP ranges from your subnet. This matters at enterprise scale where VPC peering IP range exhaustion becomes a real operational constraint.

```
# Create a Private Service Connect endpoint for Cloud SQL
# First, ensure the Cloud SQL instance has PSC enabled (set at creation time
# or via patch if the feature is available for the tier)

gcloud sql instances patch prod-pg-01 \
  --enable-private-service-connect \
  --allowed-psc-projects=my-project,consumer-project-1,consumer-project-2

# Create the Private Service Connect forwarding rule in the consumer VPC
gcloud compute forwarding-rules create cloud-sql-psc-endpoint \
  --region=us-central1 \
  --network=prod-vpc \
  --address=10.100.0.50 \
  --target-service-attachment=\
  projects/my-project/regions/us-central1/serviceAttachments/prod-pg-01-psc
```

Authorized networks (IP allowlisting) applies only to public IP instances. If you've correctly configured your instance with private IP only, authorized networks are irrelevant. For development instances with public IP, the mechanism is straightforward but operationally brittle for anything beyond a small static set of IPs.

19.6 Backup, Recovery, and PITR

Cloud SQL's backup model has two components that work together for full recovery capability: **automated backups** (full daily snapshots) and **transaction log archiving** (continuous WAL archiving for PostgreSQL, transaction log backups for SQL Server). Understanding the distinction

matters when you're designing your RTO/RPO targets.

Automated backups are taken once daily during the backup window you configure. They are stored in Google-managed Cloud Storage buckets across two regions automatically — you do not manage this storage directly, and it does not appear in your own GCS bucket unless you explicitly export. Retention is configurable up to 365 backups.

Point-in-time recovery requires transaction log retention to be enabled (`--enable-bin-log` on the `gcloud create` command, or `point_in_time_recovery_enabled = true` in Terraform). With PITR enabled, you can recover to any second within the retained log window, which you configure from 1 to 7 days. Logs are stored in addition to daily backups; both contribute to your storage bill.

Recovery operations:

```
# List available backups for an instance
gcloud sql backups list --instance=prod-pg-01

# Restore from a specific backup to the same instance
# WARNING: This overwrites the existing instance data
gcloud sql backups restore BACKUP_ID \
  --restore-instance=prod-pg-01

# Restore from a backup to a NEW instance (preferred for production DR testing)
gcloud sql backups restore BACKUP_ID \
  --restore-instance=prod-pg-01-restored \
  --backup-instance=prod-pg-01

# Point-in-time recovery: restore to a specific timestamp
# The target time must fall within the transaction log retention window
gcloud sql instances clone prod-pg-01 prod-pg-01-pitr \
  --point-in-time=2024-10-15T14:30:00.000Z

# For SQL Server: same mechanism, different timestamp format is accepted
gcloud sql instances clone prod-mssql-01 prod-mssql-01-pitr \
  --point-in-time=2024-10-15T14:30:00.000Z
```

The `clone` command with `--point-in-time` is the practical approach for PITR in Cloud SQL. It creates a new instance restored to the target time without touching the original. This is valuable: you can validate the recovery, run queries against the cloned instance to confirm data integrity, and then either promote the clone or discard it. This is operationally cleaner than the on-premises pattern of restoring to the same server and hoping you got the recovery target right.

On-demand backups are useful before risky schema changes or data migrations:

```
# Trigger an on-demand backup
gcloud sql backups create --instance=prod-pg-01 \
  --description="Pre-migration backup before schema change 2024-10-15"

# Export to Cloud Storage (for long-term retention beyond the 365-backup limit,
# or for cross-project/cross-region portability)
gcloud sql export sql prod-pg-01 \
  gs://my-backups-bucket/exports/prod-pg-01-$(date +%Y%m%d).sql \
  --database=appdb \
  --offload # Runs export from a serverless offload process, not on the instance

# For SQL Server: export as BAK file to GCS
gcloud sql export bak prod-mssql-01 \
  gs://my-backups-bucket/exports/SalesDB-$(date +%Y%m%d).bak \
  --database=SalesDB \
  --bak-type=FULL
```

The `--offload` flag for PostgreSQL SQL exports is worth emphasizing. Without it, the export process consumes CPU and I/O on the primary instance directly. With `--offload`, Google runs

the export using a serverless process that reads from a snapshot, relieving production load. Always use `--offload` for large databases.

Import operations follow the reverse path:

```
# Import a SQL dump into a Cloud SQL PostgreSQL instance
# The GCS bucket must be accessible by the Cloud SQL service account
gcloud sql import sql prod-pg-01 \
  gs://my-backups-bucket/imports/dump.sql \
  --database=appdb

# Import a SQL Server BAK file
gcloud sql import bak prod-mssql-01 \
  gs://my-backups-bucket/imports/SalesDB.bak \
  --database=SalesDB \
  --bak-type=FULL
```

Recovery testing cadence: Backups that are never tested are a liability. Automate a monthly or quarterly recovery test using Cloud Scheduler + Cloud Functions or a simple Cloud Run job that triggers a PITR clone, runs a validation query set (row counts, critical business logic checks), and then deletes the clone. This pattern takes the human out of the loop and gives you documented evidence that your recovery capability actually works.

```
# Example: automated PITR test script (called from Cloud Scheduler)
#!/bin/bash
set -euo pipefail

INSTANCE="prod-pg-01"
TEST_INSTANCE="prod-pg-01-pitr-test"
PROJECT="my-project"
# Restore to 4 hours ago
RECOVERY_TIME=$(date -u -d '4 hours ago' +%Y-%m-%dT%H:%M:%S.000Z)

echo "Starting PITR test: restoring ${INSTANCE} to ${RECOVERY_TIME}"

gcloud sql instances clone "${INSTANCE}" "${TEST_INSTANCE}" \
  --point-in-time="${RECOVERY_TIME}" \
  --project="${PROJECT}"

# Wait for the clone to become RUNNABLE
for i in {1..30}; do
  STATUS=$(gcloud sql instances describe "${TEST_INSTANCE}" \
    --format="value(state)" --project="${PROJECT}")
  if [[ "${STATUS}" == "RUNNABLE" ]]; then
    echo "Clone is RUNNABLE"
    break
  fi
  echo "Waiting for clone... status=${STATUS}"
  sleep 30
done

# Run validation queries via Cloud SQL Auth Proxy
./cloud-sql-proxy "${PROJECT}:us-central1:${TEST_INSTANCE}" \
  --port=5433 &
PROXY_PID=$!
sleep 5

PGPASSWORD="${DB_PASSWORD}" psql \
  -h 127.0.0.1 -p 5433 -U postgres -d appdb \
  -c "SELECT COUNT(*) FROM orders; SELECT MAX(created_at) FROM orders;"

VALIDATION_RESULT=$?
kill "${PROXY_PID}"

# Cleanup regardless of validation result
gcloud sql instances delete "${TEST_INSTANCE}" \
  --quiet --project="${PROJECT}"
```

```

if [[ "${VALIDATION_RESULT}" -ne 0 ]]; then
    echo "PITR validation FAILED"
    exit 1
fi

echo "PITR test PASSED"

```

19.7 Performance Tuning: PostgreSQL on Cloud SQL

Performance tuning Cloud SQL for PostgreSQL follows the same fundamentals as tuning any PostgreSQL instance — the query planner, index structures, VACUUM behavior, and statistics collection are identical to community PostgreSQL. What changes is where you intervene and what visibility you have.

Query Insights is your first stop. Accessible in the GCP console under Cloud SQL → Instance → Query Insights, it surfaces:

- Top queries by total execution time, mean latency, and call count
- Query plans with execution time breakdowns
- Wait event analysis (lock waits, I/O waits, client wait)
- Application tag attribution (requires `pg_stat_statements` and application-level tagging via the `application_name` connection parameter)

For deeper investigation, connect directly and query `pg_stat_statements`:

```

-- Top 20 queries by total time in PostgreSQL on Cloud SQL
-- (pg_stat_statements must be enabled as a flag)
SELECT
    LEFT(query, 120)                                AS query_snippet,
    calls,
    ROUND(total_exec_time::numeric, 2)              AS total_ms,
    ROUND(mean_exec_time::numeric, 2)               AS mean_ms,
    ROUND(stddev_exec_time::numeric, 2)             AS stddev_ms,
    ROUND((100 * total_exec_time /
        SUM(total_exec_time) OVER ()):numeric, 2)   AS pct_total,
    rows
FROM pg_stat_statements
WHERE query NOT ILIKE '%pg_stat_statements%'
ORDER BY total_exec_time DESC
LIMIT 20;

-- Identify queries with high variance (unstable execution plans)
SELECT
    LEFT(query, 120)                                AS query_snippet,
    calls,
    ROUND(mean_exec_time::numeric, 2)              AS mean_ms,
    ROUND(stddev_exec_time::numeric, 2)             AS stddev_ms,
    ROUND((stddev_exec_time / NULLIF(mean_exec_time,0))::numeric, 4) AS cv
FROM pg_stat_statements
WHERE calls > 100
    AND mean_exec_time > 100
ORDER BY cv DESC
LIMIT 20;

```

Index bloat and **table bloat** are tuning concerns that exist in Cloud SQL exactly as they do in self-managed PostgreSQL. AUTOVACUUM runs by default and can be tuned via flags, but you have the same visibility into its behavior:

```

-- Check autovacuum activity

```

```

SELECT
  schemaname,
  tablename,
  last_vacuum,
  last_autovacuum,
  last_analyze,
  last_autoanalyze,
  n_live_tup,
  n_dead_tup,
  ROUND(n_dead_tup::numeric / NULLIF(n_live_tup + n_dead_tup, 0) * 100, 2) AS dead_pct
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC;

-- Estimate table bloat (approximate)
SELECT
  schemaname,
  tablename,
  pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS total_size,
  pg_size_pretty(pg_relation_size(schemaname||'.'||tablename))      AS table_size,
  pg_size_pretty(
    pg_total_relation_size(schemaname||'.'||tablename)
    - pg_relation_size(schemaname||'.'||tablename)
  )                                                                    AS index_size
FROM pg_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC
LIMIT 20;

```

Connection pooling architecture is an operational prerequisite for PostgreSQL on Cloud SQL at scale. Cloud SQL does not include PgBouncer. Your options:

1. **PgBouncer on GKE or a GCE VM** in the same VPC, fronting Cloud SQL via private IP. This is the most common pattern and gives you full PgBouncer configuration control.
2. **Cloud SQL Auth Proxy + PgBouncer**: Deploy Auth Proxy on the same host as PgBouncer. Auth Proxy handles TLS and IAM auth; PgBouncer handles connection multiplexing. Applications connect to PgBouncer; PgBouncer connects to Auth Proxy.
3. **AlloyDB** instead of Cloud SQL: Google's AlloyDB for PostgreSQL includes a built-in connection pooler. If connection pooling complexity is a significant operational burden, AlloyDB is worth evaluating for large PostgreSQL workloads (though it costs more than Cloud SQL).

A minimal PgBouncer configuration for Cloud SQL:

```

# pgbouncer.ini for Cloud SQL PostgreSQL
[databases]
appdb = host=10.100.0.10 port=5432 dbname=appdb

[pgbouncer]
listen_addr = 0.0.0.0
listen_port = 6432
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
pool_mode = transaction
max_client_conn = 2000
default_pool_size = 20
min_pool_size = 5
reserve_pool_size = 5
reserve_pool_timeout = 3
server_idle_timeout = 600
client_idle_timeout = 0
log_connections = 0
log_disconnections = 0
log_pooler_errors = 1
stats_period = 60

# TLS to Cloud SQL

```

```
server_tls_sslmode = require
```

`pool_mode = transaction` is the correct mode for most OLTP applications on Cloud SQL — it multiplexes connections efficiently because it releases the server connection back to the pool after each transaction, not after each session. The tradeoff is that session-level state (`SET`, temporary tables, advisory locks) does not persist across transactions. Applications that depend on session-level PostgreSQL state must use `pool_mode = session` instead.

Parallel query in PostgreSQL on Cloud SQL works the same way as in self-managed PostgreSQL, governed by `max_parallel_workers_per_gather` and `max_parallel_workers`. For OLTP workloads, you often want to reduce or disable parallel query to prevent a single analytical query from consuming all available worker processes:

```
-- Disable parallelism for a specific query (OLTP sessions)
SET max_parallel_workers_per_gather = 0;

-- Check current parallel query settings
SELECT name, setting, unit, category
FROM pg_settings
WHERE name LIKE '%parallel%'
ORDER BY name;
```

19.8 Performance Tuning: SQL Server on Cloud SQL

SQL Server tuning on Cloud SQL follows standard SQL Server DBA practice for the most part. The query optimizer, statistics management, index maintenance, and wait statistics analysis are identical to on-premises SQL Server. The constraints are around what you can configure and what diagnostic tools you can reach.

Wait statistics are your primary diagnostic entry point:

```
-- Top wait types on SQL Server Cloud SQL instance
-- Identical to on-premises SQL Server DMV usage
SELECT TOP 20
    wait_type,
    waiting_tasks_count,
    wait_time_ms,
    max_wait_time_ms,
    signal_wait_time_ms,
    wait_time_ms - signal_wait_time_ms AS resource_wait_time_ms,
    CAST(100.0 * wait_time_ms
        / SUM(wait_time_ms) OVER ()
        AS DECIMAL(5,2)) AS pct_total
FROM sys.dm_os_wait_stats
WHERE wait_type NOT IN (
    -- Filter out benign background waits
    'SLEEP_TASK', 'BROKER_TO_FLUSH', 'BROKER_EVENTHANDLER',
    'CHECKPOINT_QUEUE', 'DBMIRROR_EVENTS_QUEUE', 'SQLTRACE_BUFFER_FLUSH',
    'CLR_AUTO_EVENT', 'DISPATCHER_QUEUE_SEMAPHORE', 'XE_TIMER_EVENT',
    'WAIT_XTP_OFFLINE_CKPT_NEW_LOG', 'LAZYWRITER_SLEEP', 'LOGMGR_QUEUE',
    'ONDEMAND_TASK_QUEUE', 'REQUEST_FOR_DEADLOCK_SEARCH', 'RESOURCE_QUEUE',
    'SERVER_IDLE_CHECK', 'SLEEP_DBSTARTUP', 'SLEEP_DBRECOVER',
    'SLEEP_MASTERDBREADY', 'SLEEP_MASTERMDREADY', 'SLEEP_MASTERUPGRADED',
    'SLEEP_MSDBSTARTUP', 'SLEEP_SYSTEMTASK', 'SLEEP_TEMPDBSTARTUP',
    'SNI_HTTP_ACCEPT', 'SP_SERVER_DIAGNOSTICS_SLEEP', 'SQLTRACE_INCREMENTAL_FLUSH_SLEEP',
    'WAITFOR', 'XE_DISPATCHER_WAIT', 'XE_DISPATCHER_JOIN'
)
ORDER BY wait_time_ms DESC;

-- Query store: top queries by total CPU (SQL Server 2017+)
```

```
-- Query Store is enabled by default on Cloud SQL for SQL Server
SELECT TOP 20
  qs.query_id,
  LEFT(qt.query_sql_text, 150)          AS query_snippet,
  SUM(rs.count_executions)              AS total_executions,
  SUM(rs.avg_cpu_time * rs.count_executions) / 1000.0 AS total_cpu_ms,
  AVG(rs.avg_cpu_time) / 1000.0        AS avg_cpu_ms,
  AVG(rs.avg_duration) / 1000.0        AS avg_duration_ms,
  AVG(rs.avg_logical_io_reads)         AS avg_logical_reads
FROM sys.query_store_query             qs
JOIN sys.query_store_query_text        qt ON qt.query_text_id = qs.query_text_id
JOIN sys.query_store_plan              qp ON qp.query_id      = qs.query_id
JOIN sys.query_store_runtime_stats     rs ON rs.plan_id       = qp.plan_id
GROUP BY qs.query_id, qt.query_sql_text
ORDER BY total_cpu_ms DESC;
```

Index maintenance follows the same rebuild/reorganize pattern but you must run it through T-SQL rather than SQL Server Agent GUI (since the Agent is surfaced through GCP APIs rather than directly). A standard index maintenance script works without modification:

```
-- Index maintenance script for Cloud SQL SQL Server
-- Reorganize fragmented indexes, rebuild heavily fragmented ones
DECLARE @SchemaName NVARCHAR(128),
        @TableName NVARCHAR(128),
        @IndexName NVARCHAR(128),
        @Fragmentation FLOAT,
        @SQL NVARCHAR(MAX);

DECLARE index_cursor CURSOR FOR
SELECT
  s.name,
  t.name,
  i.name,
  ps.avg_fragmentation_in_percent
FROM sys.dm_db_index_physical_stats(
  DB_ID(), NULL, NULL, NULL, 'LIMITED') ps
JOIN sys.indexes i ON i.object_id = ps.object_id
  AND i.index_id = ps.index_id
JOIN sys.tables t ON t.object_id = i.object_id
JOIN sys.schemas s ON s.schema_id = t.schema_id
WHERE ps.avg_fragmentation_in_percent > 10
  AND ps.page_count > 1000
  AND i.name IS NOT NULL
ORDER BY ps.avg_fragmentation_in_percent DESC;

OPEN index_cursor;
FETCH NEXT FROM index_cursor
  INTO @SchemaName, @TableName, @IndexName, @Fragmentation;

WHILE @@FETCH_STATUS = 0
BEGIN
  IF @Fragmentation > 30
    SET @SQL = N'ALTER INDEX ' + QUOTENAME(@IndexName)
      + N' ON ' + QUOTENAME(@SchemaName) + N'.' + QUOTENAME(@TableName)
      + N' REBUILD WITH (ONLINE = ON, FILLFACTOR = 80);';
  ELSE
    SET @SQL = N'ALTER INDEX ' + QUOTENAME(@IndexName)
      + N' ON ' + QUOTENAME(@SchemaName) + N'.' + QUOTENAME(@TableName)
      + N' REORGANIZE;';

  EXEC sp_executesql @SQL;

  FETCH NEXT FROM index_cursor
    INTO @SchemaName, @TableName, @IndexName, @Fragmentation;
END;

CLOSE index_cursor;
DEALLOCATE index_cursor;
```

Statistics update should follow index maintenance:

```
-- Update all statistics in the current database after index maintenance
EXEC sp_updatestats;

-- Or update specific table statistics with full scan
UPDATE STATISTICS dbo.Orders WITH FULLSCAN;
```

TempDB configuration on Cloud SQL for SQL Server: unlike on-premises SQL Server where DBA best practice is to create multiple TempDB data files equal to the number of logical processor cores (up to 8), Cloud SQL manages TempDB file configuration internally. You cannot directly alter the number of TempDB files. If you observe heavy PAGELATCH contention on TempDB allocation pages (indicated by PAGELATCH_EX or PAGELATCH_SH waits on page types 2:1:1, 2:1:2, 2:1:3), this is a constraint of the managed service rather than something you can resolve through normal TempDB tuning.

Memory-optimized tables (In-Memory OLTP) are not available in Cloud SQL for SQL Server. If your workload depends on in-memory OLTP objects (SCHEMA_AND_DATA memory-optimized tables, natively compiled stored procedures), Cloud SQL is not a compatible migration target without significant application rework.

19.9 Monitoring, Alerting, and Operational Visibility

Cloud SQL integrates with Cloud Monitoring (formerly Stackdriver) for metrics, Cloud Logging for database logs, and Query Insights for query-level analysis. As a DBA migrating from on-premises, you're replacing your existing monitoring stack (Nagios, Zabbix, DataDog, custom Grafana dashboards against `pg_stat_*` views) with a combination of GCP-native tools and whatever you choose to layer on top.

Key Cloud Monitoring metrics for Cloud SQL:

Metric	Type	Alert Threshold (example)
<code>cloudsql.googleapis.com/database/cpu/utilization</code>	Usage	> 0.80 for 5 min
<code>cloudsql.googleapis.com/database/memory/utilization</code>	Usage	> 0.90 for 5 min
<code>cloudsql.googleapis.com/database/disk/utilization</code>	Usage	> 0.85
<code>cloudsql.googleapis.com/database/disk/write_ops_count</code>	Data	Baseline + 3
<code>cloudsql.googleapis.com/database/replication/replica_lag</code>	Usage	> 30s for read replicas
<code>cloudsql.googleapis.com/database/postgresql/num_backends</code>	Usage	> 80% of max_connections
<code>cloudsql.googleapis.com/database/postgresql/deadlock_count</code>	Data	> 0 (any deadlock)
<code>cloudsql.googleapis.com/database/postgresql/transaction_Baseline</code>	Data	Baseline deviation
<code>cloudsql.googleapis.com/database/sqlserver/connections/count</code>	Usage	> 80% of max

Terraform-managed alerting policy for PostgreSQL connection saturation:

```
# terraform/monitoring/cloud_sql_alerts.tf

resource "google_monitoring_alert_policy" "pg_connection_saturation" {
  display_name = "Cloud SQL PostgreSQL - Connection Saturation"
  combiner     = "OR"

  conditions {
```

```

display_name = "PostgreSQL connections > 80% of max_connections"

condition_threshold {
  filter = <<-EOT
    resource.type = "cloudsql_database"
    AND resource.labels.database_id = "my-project:prod-pg-01"
    AND metric.type = "cloudsql.googleapis.com/database/postgresql/num_backends"
  EOT

  comparison      = "COMPARISON_GT"
  threshold_value = 400 # 80% of max_connections=500
  duration        = "300s"

  aggregations {
    alignment_period = "60s"
    per_series_aligner = "ALIGN_MEAN"
  }
}

notification_channels = [var.pagerduty_channel_id]

alert_strategy {
  auto_close = "1800s"
}
}

resource "google_monitoring_alert_policy" "pg_replication_lag" {
  display_name = "Cloud SQL PostgreSQL - Replica Lag Excessive"
  combiner     = "OR"

  conditions {
    display_name = "Replication lag > 30 seconds"

    condition_threshold {
      filter = <<-EOT
        resource.type = "cloudsql_database"
        AND metric.type = "cloudsql.googleapis.com/database/replication/replica_lag"
      EOT

      comparison      = "COMPARISON_GT"
      threshold_value = 30
      duration        = "120s"

      aggregations {
        alignment_period = "60s"
        per_series_aligner = "ALIGN_MAX"
      }
    }
  }

  notification_channels = [var.pagerduty_channel_id]
}

```

Cloud Logging captures PostgreSQL log output routed through Cloud Logging's structured logging pipeline. The database flags `log_min_duration_statement`, `log_connections`, `log_disconnections`, `log_lock_waits`, and `log_checkpoints` all write to Cloud Logging when enabled. Query these logs with Log Explorer using structured filter syntax:

```

# Log Explorer filter: slow queries from Cloud SQL PostgreSQL
resource.type="cloudsql_database"
resource.labels.database_id="my-project:prod-pg-01"
textPayload=~"duration: [0-9]+ ms"
severity>=DEFAULT

```

```
# Filter for lock wait events
resource.type="cloudsql_database"
textPayload=~"process [0-9]+ acquired"
```

For production observability, consider exporting Cloud Logging to BigQuery for long-term retention and ad-hoc analysis of historical query patterns:

```
# Log sink to BigQuery for Cloud SQL logs
resource "google_logging_project_sink" "cloud_sql_to_bq" {
  name      = "cloud-sql-logs-to-bigquery"
  destination = "bigquery.googleapis.com/projects/my-project/datasets/cloud_sql_logs"

  filter = <<-EOT
    resource.type="cloudsql_database"
    AND (
      textPayload=~"duration:"
      OR textPayload=~"deadlock"
      OR textPayload=~"checkpoint"
      OR textPayload=~"lock wait"
    )
  EOT

  unique_writer_identity = true
}
```

Custom dashboards via Grafana are a common complement to the GCP console for teams that want unified visibility across Cloud SQL and other infrastructure. The Cloud Monitoring data source plugin for Grafana connects directly to the Cloud Monitoring API using a service account, pulling the same metrics surfaced in the GCP console into your existing Grafana environment. No Cloud SQL-specific agent or exporter is needed.

19.10 Migrations to Cloud SQL: Practical Patterns

Most production migrations to Cloud SQL come from one of three sources: on-premises PostgreSQL or SQL Server, another cloud provider's managed database service, or a self-managed database on GCE. The migration approach depends on your acceptable downtime window and database size.

Database Migration Service (DMS) is Google's managed migration tool for Cloud SQL. It supports continuous replication migrations (minimal downtime) and one-time migrations. For PostgreSQL, DMS uses logical replication under the hood. For SQL Server, it uses backup/restore from GCS followed by ongoing transaction log shipping.

```
# Create a DMS migration job for PostgreSQL (minimal downtime migration)
# Prerequisites:
# - Source PostgreSQL must have wal_level=logical
# - Cloud SQL target must exist
# - Connectivity between source and Cloud SQL (VPN, Interconnect, or public IP)

# Step 1: Create a connection profile for the source
gcloud database-migration connection-profiles create pg-source \
  --region=us-central1 \
  --type=POSTGRES \
  --host=10.0.1.50 \
  --port=5432 \
  --username=migration_user \
  --password=MIGRATION_PASS \
  --display-name="On-Premises PostgreSQL Source"

# Step 2: Create a connection profile for the Cloud SQL target
```



```

gcloud database-migration connection-profiles create pg-target \
  --region=us-central1 \
  --type=CLOUDSQL \
  --cloudsql-instance-id=prod-pg-01 \
  --display-name="Cloud SQL Target"

# Step 3: Create the migration job
gcloud database-migration migration-jobs create pg-migration-01 \
  --region=us-central1 \
  --type=CONTINUOUS \
  --source=pg-source \
  --destination=pg-target \
  --display-name="Production PostgreSQL Migration"

# Step 4: Start the migration (initial full load + continuous CDC)
gcloud database-migration migration-jobs start pg-migration-01 \
  --region=us-central1

# Monitor migration status
gcloud database-migration migration-jobs describe pg-migration-01 \
  --region=us-central1

# Step 5: Promote (cutover) when ready
# This stops replication and promotes Cloud SQL as the primary
gcloud database-migration migration-jobs promote pg-migration-01 \
  --region=us-central1

```

Manual migration for PostgreSQL using `pg_dump` and `pg_restore` is appropriate for databases under a few hundred gigabytes where a maintenance window is acceptable:

```

# Dump from source (run on a host with access to both source and Cloud SQL)
pg_dump \
  -h source-pg-host \
  -U postgres \
  -d appdb \
  -Fc \
  --no-owner \
  --no-acl \
  -f appdb_dump.pgc

# Restore to Cloud SQL via Auth Proxy
./cloud-sql-proxy my-project:us-central1:prod-pg-01 --port=5432 &

pg_restore \
  -h 127.0.0.1 \
  -p 5432 \
  -U postgres \
  -d appdb \
  --no-owner \
  --no-acl \
  -j 4 \ # Parallel restore with 4 workers
  appdb_dump.pgc

```

The `--no-owner` and `--no-acl` flags are typically needed because Cloud SQL's `postgres` user cannot set ownership to roles that don't exist on the target, and ACL re-application may fail if the source had superuser-only grants. Review ownership and permissions manually after the restore.

For SQL Server migrations, the typical pattern is:

1. Take a full backup on source as a `.bak` file
2. Upload the `.bak` file to a GCS bucket accessible by Cloud SQL
3. Import using `gcloud sql import bak`
4. Configure transaction log shipping from source to Cloud SQL for the cutover window (via DMS continuous migration)

```
# Upload SQL Server backup to GCS
```

```
gsutil cp SalesDB_full.bak gs://my-migration-bucket/SalesDB_full.bak

# Import to Cloud SQL SQL Server
gcloud sql import bak prod-mssql-01 \
  gs://my-migration-bucket/SalesDB_full.bak \
  --database=SalesDB \
  --bak-type=FULL \
  --no-recovery # Leave in restoring state for subsequent log restores

# Apply transaction log backups during the migration window
gcloud sql import bak prod-mssql-01 \
  gs://my-migration-bucket/SalesDB_log_001.bak \
  --database=SalesDB \
  --bak-type=TLOG \
  --no-recovery

# Final log backup with recovery (brings database online)
gcloud sql import bak prod-mssql-01 \
  gs://my-migration-bucket/SalesDB_log_final.bak \
  --database=SalesDB \
  --bak-type=TLOG
# No --no-recovery on the final restore
```

Post-migration validation should be scripted and run before decommissioning the source:

```
-- PostgreSQL: row count comparison across all tables
-- Run on both source and target, compare output
SELECT
  schemaname,
  tablename,
  n_live_tup AS estimated_rows
FROM pg_stat_user_tables
ORDER BY schemaname, tablename;

-- More precise (slower on large tables):
SELECT
  table_schema,
  table_name,
  (xpath('/row/cnt/text()',
    query_to_xml(
      format('SELECT COUNT(*) AS cnt FROM %I.%I',
        table_schema, table_name),
      false, true, ''))
  )[1]::text::bigint AS exact_row_count
FROM information_schema.tables
WHERE table_type = 'BASE TABLE'
  AND table_schema NOT IN ('pg_catalog', 'information_schema')
ORDER BY table_schema, table_name;

-- SQL Server: row count comparison across all tables
SELECT
  s.name          AS schema_name,
  t.name          AS table_name,
  p.rows          AS row_count
FROM sys.tables   t
JOIN sys.schemas s ON s.schema_id = t.schema_id
JOIN sys.partitions p ON p.object_id = t.object_id AND p.index_id IN (0, 1)
ORDER BY s.name, t.name;
```

Key Takeaways

- Cloud SQL gives you managed PostgreSQL and SQL Server without OS-level access, replacing `postgresql.conf`, `pg_hba.conf`, and `sp_configure` with database flags surfaced through the `gcloud` CLI, Console, and Terraform. The allowed configuration space is

large enough for production tuning but excludes superuser-only operations, arbitrary extension installation, and OS-level interventions like TempDB file count changes on SQL Server.

- **HA in Cloud SQL is synchronous and regional, not global:** the standby lives in a different zone within the same region and is not readable. Cross-region resilience and read scaling require separately configured read replicas, and replica lag monitoring must be part of your operational baseline rather than an afterthought.
 - **IAM database authentication and Cloud SQL Auth Proxy replace the `pg_hba.conf` + password credential model** for production access control. Short-lived IAM tokens eliminate static credential rotation, workload identity on GKE removes credential files entirely, and Private Service Connect addresses IP range exhaustion at enterprise scale — but Auth Proxy is a TLS tunnel, not a connection pooler, so PgBouncer or an equivalent remains necessary for high-connection OLTP workloads.
 - **PITR via instance clone is the operationally correct recovery pattern** on Cloud SQL: clone to a new instance at the target timestamp, validate data integrity with row count and business-logic checks, then promote or discard. Automating this monthly with Cloud Scheduler removes the human dependency and produces documented recovery evidence. For SQL Server migrations, the `--no-recovery` flag on intermediate log restores preserves the restoring state for sequential log application before final cutover.
 - **Performance tuning fundamentals are unchanged** — `pg_stat_statements`, wait statistics, index fragmentation analysis, and autovacuum tuning apply identically to Cloud SQL as to self-managed instances. What changes is the tooling layer: Query Insights replaces pgBadger as the first-line slow query tool, Cloud Monitoring replaces Nagios/Zabbix for infrastructure alerting, and Cloud Logging with BigQuery export replaces local log retention for historical analysis. Gaps remain (no In-Memory OLTP, no SSIS, no arbitrary PostgreSQL extensions) and must be identified before committing to Cloud SQL as a migration target.
-

Chapter 20: AlloyDB: PostgreSQL-Compatible with ML Built In

Google’s AlloyDB for PostgreSQL is not simply a managed PostgreSQL service with a different logo. It is a ground-up reimagining of the PostgreSQL storage engine — wire-compatible with PostgreSQL 14 and 15, but architecturally closer to Aurora in its separation of compute and storage, and unlike any other managed PostgreSQL offering in its native integration of machine learning inference directly at the query layer. For production DBAs who have spent years tuning autovacuum, managing WAL pressure, and hand-crafting connection pools, AlloyDB represents a shift in what the database engine itself takes responsibility for. This chapter covers how AlloyDB’s columnar engine, memory-optimized storage, and ML inference extensions work in practice, how to provision and connect clusters, how to migrate existing PostgreSQL workloads, and where AlloyDB genuinely outperforms Cloud SQL for PostgreSQL and where it falls short in ways that will surprise you.

20.1 Architecture: What AlloyDB Actually Is Under the Hood

If you are coming from self-managed PostgreSQL, you understand the storage hierarchy: the buffer pool in shared memory, heap files on disk, WAL on a separate spindle if you are careful, and the constant negotiation between autovacuum and your OLTP workload. AlloyDB dismantles this hierarchy and reassembles it differently.

Disaggregated Storage

AlloyDB separates the primary instance compute from a distributed storage layer, similar to Aurora’s approach. The storage layer — Google calls it the *Log Store* — receives and durably replicates WAL records across multiple zones before acknowledging writes; consult current AlloyDB documentation for the exact replica topology in your region. The primary instance never writes heap files directly to local disk. Instead, background storage nodes apply WAL to materialized heap pages asynchronously. This means your primary is doing dramatically less I/O work than a traditional PostgreSQL primary, and crash recovery does not require replaying hours of WAL — the storage layer maintains consistent page state independently.

What this means for you operationally:

- **Checkpoint pressure disappears.** In self-managed PostgreSQL, tuning `checkpoint_completion_target`, `max_wal_size`, and `checkpoint_timeout` is a ritual. In AlloyDB, checkpoints still exist in the PostgreSQL code path, but their I/O impact on the primary is negligible because no dirty pages are being flushed to local disk in the traditional sense.
- **Read replicas are fast to provision.** A read replica in AlloyDB attaches to the same storage layer — it does not stream WAL and rebuild a base backup. A new replica is queryable in seconds to minutes, not hours.
- **Replication lag behaves differently.** Because replicas read from the shared storage layer rather than applying streamed WAL, replication lag is reported differently than `pg_stat_replication.replay_lag`. AlloyDB exposes this through its own metrics, and you need to retrain your intuition about what “lag” means.

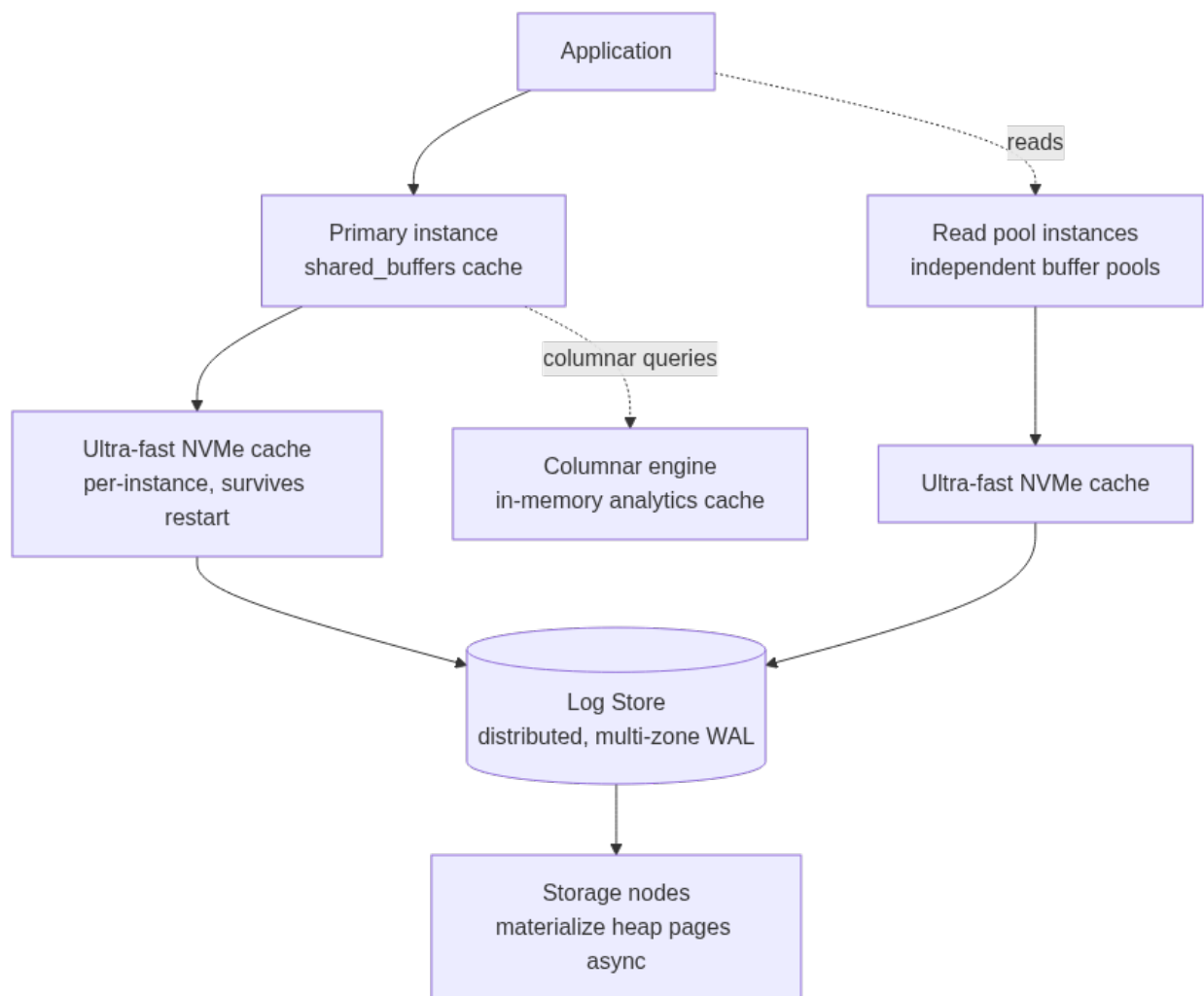


Figure 11: AlloyDB’s disaggregated architecture: compute instances hold only cache layers (buffer pool + NVMe); durable page state lives in the shared Log Store, which is why new read pool instances attach in seconds rather than rebuilding from a base backup.

The Ultra-Fast Shared Memory Cache

AlloyDB introduces a tiered memory architecture: beyond the standard PostgreSQL `shared_buffers`, it maintains a second layer called the *ultra-fast cache* built on local NVMe SSD attached to each instance. This cache survives instance restarts because it is backed by NVMe rather than DRAM. When a page is evicted from `shared_buffers`, it goes to this NVMe cache before it goes back to the distributed storage layer. The practical implication: your read-heavy workloads that overflow `shared_buffers` do not pay remote storage latency on every cache miss — NVMe latency is meaningfully lower than a round trip to the distributed storage layer, though exact figures depend on instance type and should be measured against your own workload rather than assumed.

In sizing terms, `shared_buffers` in AlloyDB is set automatically based on instance size and is not a parameter you directly tune in the same way, with the NVMe cache providing another layer below it.

The Columnar Engine

AlloyDB includes an optional columnar engine — a PostgreSQL foreign data wrapper-style extension that maintains a compressed, columnar representation of tables or table partitions in memory. When a query contains analytical predicates (range scans, aggregations, GROUP BY) that the query planner identifies as benefiting from columnar access, it routes those operations through the columnar engine transparently. You write standard SQL; the planner decides.

This is meaningful because it means you can run mixed OLTP and analytical workloads on the same AlloyDB cluster without physically separating your data into a separate OLAP store. The columnar engine is not a replacement for BigQuery or a data warehouse at scale — but for the DBA who has been maintaining a separate reporting replica just to run month-end aggregates without killing the OLTP primary, it is a serious operational simplification.

Enabling the columnar engine on a table:

```
-- Enable the AlloyDB columnar engine extension
-- This is typically already installed in AlloyDB clusters
SHOW alloydb_enable_columnar_engine;

-- Enable columnar engine for a specific table
ALTER TABLE orders SET (columnar_engine = 'on');

-- You can also enable it for specific columns or let AlloyDB auto-select
-- Check what the auto-columnarization has selected
SELECT *
FROM google_columnar_engine_recommended_tables()
ORDER BY estimated_memory_mb DESC;

-- View current columnar engine population status
SELECT table_name,
       status,
       rows_count,
       size_in_bytes
FROM google_columnar_engine_tables;
```

The `google_columnar_engine_recommended_tables()` function is one of AlloyDB's genuinely useful operational functions — it analyzes your query workload and recommends which tables will benefit most from columnarization, with estimated memory requirements. Treat this output as advisory; validate with `EXPLAIN (ANALYZE, BUFFERS)` before and after.

20.2 Provisioning AlloyDB Clusters and Instances

AlloyDB uses a two-tier model: a *cluster* and *instances within that cluster*. The cluster defines the regional boundary, the VPC network, and the storage configuration. Instances are the PostgreSQL compute endpoints — a primary instance and optionally one or more read pool instances.

Provisioning with gcloud

```
# Set your project and region
gcloud config set project your-gcp-project-id
gcloud config set compute/region us-central1

# Create an AlloyDB cluster
# This provisions the storage layer - no compute yet
gcloud alloydb clusters create prod-alloydb-cluster \
  --region=us-central1 \
  --network=projects/your-gcp-project-id/global/networks/prod-vpc \
  --password=your-initial-password \
  --database-version=POSTGRES_15

# Create the primary instance
# machine-type options: db-n1-standard-2, db-n1-highmem-8, db-n1-highmem-32, etc.
gcloud alloydb instances create prod-primary \
  --cluster=prod-alloydb-cluster \
  --region=us-central1 \
  --instance-type=PRIMARY \
  --cpu-count=8 \
  --no-assign-inbound-public-ip

# Create a read pool instance (for read scaling and analytics)
# Read pools in AlloyDB are groups of nodes - min 2 for HA
gcloud alloydb instances create prod-read-pool \
  --cluster=prod-alloydb-cluster \
  --region=us-central1 \
  --instance-type=READ_POOL \
  --cpu-count=8 \
  --read-pool-node-count=2 \
  --no-assign-inbound-public-ip

# Verify both instances are READY
gcloud alloydb instances list \
  --cluster=prod-alloydb-cluster \
  --region=us-central1
```

Provisioning with Terraform

Terraform is the better choice for production clusters where you need reproducible infrastructure. The AlloyDB Terraform provider reached GA in 2024 and covers the full feature set:

```
# main.tf - AlloyDB cluster and instances

terraform {
  required_providers {
    google = {
      source = "hashicorp/google"
      version = ">= 5.0"
    }
  }
}

provider "google" {
  project = var.project_id
  region = var.region
}

# The AlloyDB cluster - regional, tied to your VPC
resource "google_alloydb_cluster" "prod" {
```



```
cluster_id = "prod-alloydb-cluster"
location   = var.region

network_config {
  network = "projects/${var.project_id}/global/networks/${var.vpc_name}"
}

initial_user {
  password = var.db_password
}

automated_backup_policy {
  enabled      = true
  location     = var.region
  backup_window = "02:00-06:00"

  weekly_schedule {
    days_of_week = ["SUNDAY"]
    start_times {
      hours   = 2
      minutes = 0
    }
  }
}

quantity_based_retention {
  count = 14
}

continuous_backup_config {
  enabled           = true
  recovery_window_days = 14
}

# Primary instance - this is your read/write endpoint
resource "google_alloydb_instance" "primary" {
  cluster      = google_alloydb_cluster.prod.name
  instance_id  = "prod-primary"
  instance_type = "PRIMARY"

  machine_config {
    cpu_count = 8
  }

  query_insights_config {
    query_string_length      = 1024
    record_application_tags  = true
    record_client_address    = false
    query_plans_per_minute  = 20
  }

  depends_on = [google_alloydb_cluster.prod]
}

# Read pool instance - horizontally scaled read replicas
resource "google_alloydb_instance" "read_pool" {
  cluster      = google_alloydb_cluster.prod.name
  instance_id  = "prod-read-pool"
  instance_type = "READ_POOL"

  machine_config {
    cpu_count = 8
  }

  read_pool_config {
    node_count = 2
  }

  depends_on = [google_alloydb_instance.primary]
```

```
}

output "primary_ip" {
  value      = google_alloydb_instance.primary.ip_address
  description = "Primary instance private IP - use this for read/write connections"
}

output "read_pool_ip" {
  value      = google_alloydb_instance.read_pool.ip_address
  description = "Read pool private IP - use this for analytics and read-heavy workloads"
}
```

Connection Topology

AlloyDB does not expose a CNAME-based endpoint the way Aurora does. Connection is always by private IP within your VPC. There is no AlloyDB equivalent of RDS Proxy built into the service — you bring your own connection pooler, and PgBouncer running on a GCE instance or Cloud Run is the standard pattern. If you are coming from Aurora, this is a gap you will feel immediately. The AlloyDB Auth Proxy handles IAM authentication and SSL termination and is the recommended client-side component:

```
# Download and run the AlloyDB Auth Proxy
# This is the equivalent of Cloud SQL Auth Proxy - same pattern, different binary

wget https://storage.googleapis.com/alloydb-auth-proxy/v1.x.x/alloydb-auth-proxy.linux.amd64 \
  -O alloydb-auth-proxy
chmod +x alloydb-auth-proxy

# Run the proxy - it listens on localhost:5432 and handles IAM auth
./alloydb-auth-proxy \
  "projects/your-project/locations/us-central1/clusters/prod-alloydb-cluster/instances/prod-primary" \
  --port=5432 \
  --credentials-file=/path/to/service-account.json

# Application connection string then becomes:
# host=127.0.0.1 port=5432 dbname=mydb user=iam_service_account sslmode=disable
# SSL is handled by the proxy - disable in the client connection string
```

The AlloyDB Auth Proxy terminates SSL and handles token refresh automatically, which means your application connection strings do not embed database passwords. Instead, the service account identity determines access. This is the same pattern you used with Cloud SQL Auth Proxy if you have worked with Cloud SQL before.

20.3 IAM Authentication and Access Control

In self-managed PostgreSQL, `pg_hba.conf` is your access control boundary. You define which hosts can connect, which users, and which authentication method — `md5`, `scram-sha-256`, `cert`, `ldap`. In AlloyDB, the network boundary is your VPC (enforced at the infrastructure level), and authentication has two modes: native PostgreSQL password authentication and IAM-based authentication.

IAM authentication in AlloyDB works similarly to Cloud SQL's IAM auth. A Google identity — a service account or a user account — is mapped to a PostgreSQL role. The IAM-authenticated connection uses a short-lived OAuth2 token as the password, which the AlloyDB Auth Proxy or the `alloydb_pg_iam_authn` extension handles.

Setting Up IAM Authentication

```
# Grant IAM user or service account the AlloyDB Client role
```

```

# This is the network-level permission - required before database-level grants
gcloud projects add-iam-policy-binding your-gcp-project-id \
  --member="serviceAccount:app-service-account@your-project.iam.gserviceaccount.com" \
  --role="roles/alloydb.client"

# For IAM database authentication specifically:
gcloud projects add-iam-policy-binding your-gcp-project-id \
  --member="serviceAccount:app-service-account@your-project.iam.gserviceaccount.com" \
  --role="roles/alloydb.databaseUser"

-- Inside the AlloyDB instance, create the IAM-mapped PostgreSQL role
-- The username format for service accounts: strip @project.iam.gserviceaccount.com
-- For user accounts: the full email address

-- Create the PostgreSQL role corresponding to the IAM service account
CREATE USER "app-service-account@your-project.iam" WITH LOGIN;

-- Grant appropriate database permissions
GRANT CONNECT ON DATABASE production TO "app-service-account@your-project.iam";
GRANT USAGE ON SCHEMA public TO "app-service-account@your-project.iam";
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public
  TO "app-service-account@your-project.iam";

-- For future tables:
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES
  TO "app-service-account@your-project.iam";

-- Verify the role exists and has the right attributes
SELECT rolname, rolcanlogin, rolsuper, rolcreatedb
FROM pg_roles
WHERE rolname LIKE '%iam%';

```

What You Cannot Do That You Could in Self-Managed PostgreSQL

The `pg_hba.conf` comparison is instructive here. In self-managed PostgreSQL, you can define rules like:

```

# Allow cert auth from specific CIDR, reject everything else
hostssl all all 10.0.0.0/8 cert
host    all all all          reject

```

In AlloyDB, your network-level controls are VPC firewall rules and Private Service Connect configuration — you cannot write arbitrary `pg_hba.conf` entries. AlloyDB manages its own `pg_hba.conf` internally. You can enable or disable IAM authentication at the cluster level, and you can require SSL (which is on by default), but fine-grained auth method selection per CIDR range or per database is not exposed. This is a real constraint for organizations that use client certificate authentication today.

Connection from Python Using IAM Token

```

import google.auth
import google.auth.transport.requests
import psycopg2
from google.oauth2 import service_account

def get_alloydb_connection_iam(
    host: str,
    database: str,
    iam_user: str,
    service_account_file: str
) -> psycopg2.extensions.connection:
    """
    Connect to AlloyDB using IAM authentication.
    The AlloyDB Auth Proxy must be running, or use direct IP with SSL.
    """

```

```
For proxy connections, host is 127.0.0.1 and no SSL needed at client level.
"""
credentials = service_account.Credentials.from_service_account_file(
    service_account_file,
    scopes=["https://www.googleapis.com/auth/cloud-platform"]
)

# Refresh to get a current token
auth_request = google.auth.transport.requests.Request()
credentials.refresh(auth_request)

# The OAuth2 token is used as the password
token = credentials.token

conn = psycopg2.connect(
    host=host,
    port=5432,
    database=database,
    user=iam_user, # e.g., "app-service-account@your-project.iam"
    password=token,
    sslmode="disable" # SSL handled by Auth Proxy
)

return conn

def run_query_with_retry(host: str, database: str, query: str) -> list:
    """
    Production pattern: IAM tokens expire in 1 hour.
    Use a connection factory that refreshes tokens on reconnect.
    """
    import time

    max_retries = 3
    for attempt in range(max_retries):
        try:
            conn = get_alloydb_connection_iam(
                host=host,
                database=database,
                iam_user="app-service-account@your-project.iam",
                service_account_file="/etc/secrets/sa-key.json"
            )
            with conn.cursor() as cur:
                cur.execute(query)
                results = cur.fetchall()
            conn.close()
            return results

        except psycopg2.OperationalError as e:
            if attempt < max_retries - 1:
                time.sleep(2 ** attempt) # Exponential backoff
                continue
            raise

    return []
```

The token expiry behavior is something that catches people coming from password-based PostgreSQL connections. A long-lived connection established with an IAM token does not get disconnected when the token expires — but any new connection attempt after the token expires will fail. PgBouncer in transaction pooling mode mitigates this because connections are returned to the pool and reestablished regularly, but you need to wire the token refresh into your connection factory, not just your startup code.

20.4 ML Inference with google_ml_integration

This is where AlloyDB genuinely has no equivalent in Aurora, Azure SQL, or Cloud SQL for PostgreSQL. AlloyDB includes a built-in extension called `google_ml_integration` that allows you to call Vertex AI hosted models directly from SQL, with the model call executing as a PostgreSQL function. The ML inference runs server-side, in the database tier, without your application code needing to orchestrate a round trip to Vertex AI.

The practical use cases for DBAs are:

- **Text embedding generation** at insert time, stored as vector columns (AlloyDB natively supports `pgvector`)
- **Classification and scoring** within a query, avoiding application-side ML overhead
- **Semantic search** using vector similarity within standard SQL queries

Enabling the ML Integration Extension

```
-- Enable the extension (requires superuser or alloydbsuperuser role)
CREATE EXTENSION IF NOT EXISTS google_ml_integration CASCADE;

-- Verify installation
SELECT extname, extversion
FROM pg_extension
WHERE extname IN ('google_ml_integration', 'vector');

-- Grant usage to application roles
GRANT EXECUTE ON FUNCTION embedding(text, text) TO app_role;
GRANT EXECUTE ON FUNCTION ml_predict_row(text, jsonb) TO app_role;

# Before calling Vertex AI from AlloyDB, the AlloyDB service account
# needs permission to invoke the Vertex AI API
# Get your AlloyDB service account
gcloud alloydb clusters describe prod-alloydb-cluster \
  --region=us-central1 \
  --format="value(serviceAccountEmailAddress)"

# Grant the Vertex AI User role to the AlloyDB service account
gcloud projects add-iam-policy-binding your-gcp-project-id \
  --member="serviceAccount:alloydb-sa@your-project.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"
```

Generating Embeddings in SQL

```
-- Create a table with a vector column for semantic search
CREATE TABLE product_catalog (
  product_id SERIAL PRIMARY KEY,
  product_name TEXT NOT NULL,
  description TEXT,
  category TEXT,
  price NUMERIC(10,2),
  -- 768-dimension vector for text-embedding-004 model
  description_embedding VECTOR(768)
);

-- Create an IVFFlat index for approximate nearest neighbor search
-- lists = sqrt(row_count) is a common starting point
CREATE INDEX idx_product_embedding
ON product_catalog
USING ivfflat (description_embedding vector_cosine_ops)
WITH (lists = 100);

-- Generate embeddings using Vertex AI text-embedding-004
-- The embedding() function calls Vertex AI and returns a VECTOR
UPDATE product_catalog
SET description_embedding = embedding(
  'textembedding-gecko@003', -- Vertex AI model ID
```

```

        description          -- The text to embed
    )
WHERE description_embedding IS NULL
    AND description IS NOT NULL;

-- Semantic search: find products similar to a natural language query
-- This entire operation happens inside PostgreSQL - no app-side ML call
SELECT
    product_id,
    product_name,
    description,
    price,
    1 - (description_embedding <=> embedding(
        'textembedding-gecko@003',
        'lightweight running shoes for trail conditions'
    )) AS cosine_similarity
FROM product_catalog
WHERE category = 'footwear'
ORDER BY description_embedding <=> embedding(
    'textembedding-gecko@003',
    'lightweight running shoes for trail conditions'
)
LIMIT 10;

-- Verify the index is being used (set_limit controls IVFFlat probe count)
SET ivfflat.probes = 10;
EXPLAIN (ANALYZE, BUFFERS)
SELECT product_id, product_name,
    description_embedding <=> embedding(
        'textembedding-gecko@003',
        'trail running footwear'
    ) AS distance
FROM product_catalog
ORDER BY description_embedding <=> embedding(
    'textembedding-gecko@003',
    'trail running footwear'
)
LIMIT 5;

```

Calling Custom Vertex AI Models

Beyond the built-in embedding function, you can call any Vertex AI endpoint using `ml_predict_row`:

```

-- Register a custom model endpoint
-- This maps a logical name to a Vertex AI endpoint URL
INSERT INTO google_ml.model_info (
    model_id,
    model_type,
    model_endpoint,
    model_qualified_name
)
VALUES (
    'churn-predictor-v2',
    'classification',

    ↪ 'https://us-central1-aiplatform.googleapis.com/v1/projects/your-project/locations/us-central1/endpoints/123456789'
    ↪ ,
    'churn-predictor-v2'
);

-- Use the model in a query to score customer churn risk
-- ml_predict_row sends each row as a JSON payload to the model endpoint
-- and returns the prediction as JSONB
SELECT
    c.customer_id,
    c.customer_name,
    c.months_active,
    c.total_spend,

```

```

(ml_predict_row(
  'churn-predictor-v2',
  jsonb_build_object(
    'months_active',      c.months_active,
    'total_spend',        c.total_spend,
    'support_tickets',    c.support_tickets_last_90d,
    'login_frequency',    c.avg_logins_per_month,
    'last_purchase_days', EXTRACT(EPOCH FROM (NOW() - c.last_purchase_date) / 86400)
  )
) ->> 'churn_probability')::NUMERIC AS churn_probability
FROM customers c
WHERE c.account_status = 'active'
ORDER BY churn_probability DESC NULLS LAST
LIMIT 100;

```

The `ml_predict_row` function sends the JSONB payload to the Vertex AI endpoint over an authenticated HTTPS call and returns the response as JSONB. The function blocks — it is a synchronous call from inside your SQL execution. This has performance implications you need to understand before putting it in a hot query path.

Performance Considerations for In-Database ML

Calling a remote Vertex AI endpoint from within a SQL query introduces network round-trip latency on every row where the function is evaluated. A query that calls `ml_predict_row` on 10,000 rows is making 10,000 sequential HTTP calls unless you batch. AlloyDB provides a batching mechanism through `ml_predict` (the batch variant of the function):

```

-- Batch variant: operates on a set of rows rather than row by row
-- More efficient for bulk scoring operations
WITH customer_features AS (
  SELECT
    customer_id,
    jsonb_build_object(
      'months_active',      months_active,
      'total_spend',        total_spend,
      'support_tickets',    support_tickets_last_90d,
      'login_frequency',    avg_logins_per_month,
      'last_purchase_days', EXTRACT(EPOCH FROM (NOW() - last_purchase_date) / 86400)
    ) AS features
  FROM customers
  WHERE account_status = 'active'
  AND churn_score_updated_at < NOW() - INTERVAL '24 hours'
)
UPDATE customers
SET
  churn_score = (prediction ->> 'churn_probability')::NUMERIC,
  churn_score_updated_at = NOW()
FROM (
  SELECT
    customer_id,
    ml_predict_row('churn-predictor-v2', features) AS prediction
  FROM customer_features
) scored
WHERE customers.customer_id = scored.customer_id;

```

In practice, treat in-database ML inference the way you treat in-database foreign data wrapper queries: appropriate for scheduled batch jobs and low-volume per-request lookups, not appropriate for your critical path OLTP queries. A customer-facing API endpoint that triggers a `ml_predict_row` call in its SQL will add the Vertex AI inference latency — typically 50–300ms depending on model and payload — to every request. The semantic search pattern using pre-computed embeddings stored as `VECTOR` columns is significantly more practical for production because the costly Vertex AI call happens at write time, and the query-time operation is pure vector arithmetic inside the database.

20.5 Migrating PostgreSQL Workloads to AlloyDB

AlloyDB is wire-compatible with PostgreSQL 14 and 15. Most PostgreSQL applications require no SQL or driver changes to run on AlloyDB. The migration work is concentrated in three areas: extension compatibility, parameter behavior differences, and operational tooling changes.

Extension Compatibility

AlloyDB supports a large subset of PostgreSQL extensions, but not all. The most common issues encountered in migrations:

Extension	AlloyDB Status	Notes
<code>pg_stat_statements</code>	Supported	Enabled by default
<code>pgvector</code>	Supported	Native, used with <code>google_ml_integration</code>
<code>postgis</code>	Supported	Standard PostGIS versions
<code>pg_cron</code>	Supported	Runs on primary only
<code>pg_partman</code>	Supported	Works normally
<code>timescaledb</code>	Not supported	TimescaleDB requires its own storage hooks
<code>pg_repack</code>	Not available	Not needed — AlloyDB handles bloat differently
<code>pg_logical</code>	Not available	AlloyDB has its own replication architecture
<code>pg_prewarm</code>	Not useful	Replaced by the NVMe cache layer
Custom extensions	Not supported	AlloyDB does not load arbitrary <code>.so</code> files

The `pg_repack` absence is worth explaining: in self-managed PostgreSQL, `pg_repack` is your tool for rebuilding bloated tables without taking a full lock the way `VACUUM FULL` does. In AlloyDB, table bloat is managed differently — autovacuum runs more aggressively and the storage layer reclaims space at a lower level. In practice, AlloyDB clusters rarely accumulate the bloat levels that make `pg_repack` necessary in self-managed environments. If you are currently running regular `pg_repack` jobs as part of your maintenance routine, plan to remove them and monitor bloat in AlloyDB using `pg_stat_user_tables` and `pgstattuple` instead.

Parameter Differences

AlloyDB manages several PostgreSQL parameters on your behalf and does not allow you to override them:

```
-- Parameters you can still tune in AlloyDB:
-- (These work the same as self-managed PostgreSQL)
ALTER SYSTEM SET work_mem = '64MB';
ALTER SYSTEM SET maintenance_work_mem = '512MB';
ALTER SYSTEM SET effective_cache_size = '48GB'; -- set based on your RAM
ALTER SYSTEM SET random_page_cost = 1.1; -- NVMe cache makes this lower than HDD default
ALTER SYSTEM SET effective_io_concurrency = 200;
```



```

ALTER SYSTEM SET max_parallel_workers_per_gather = 4;
ALTER SYSTEM SET max_worker_processes = 16;
ALTER SYSTEM SET lock_timeout = '30s';
ALTER SYSTEM SET idle_in_transaction_session_timeout = '5min';

-- Parameters you cannot meaningfully tune (AlloyDB manages them):
-- shared_buffers - set automatically by AlloyDB
-- wal_level      - managed internally
-- archive_mode   - managed internally
-- max_wal_senders - managed internally
-- wal_keep_size  - not applicable to AlloyDB's WAL architecture
-- checkpoint_completion_target - less relevant, but adjustable
-- full_page_writes - managed internally

-- Parameters with different behavior in AlloyDB:
-- synchronous_commit: 'on' means WAL is durable in the distributed Log Store
-- (3-zone durable), not just on the primary's local disk
SHOW synchronous_commit;

-- autovacuum is more aggressive in AlloyDB by default
-- You can still tune per-table:
ALTER TABLE high_churn_table SET (
    autovacuum_vacuum_scale_factor = 0.01,
    autovacuum_vacuum_cost_delay = 2,
    autovacuum_vacuum_threshold = 100
);

```

Migration Using Database Migration Service

Google's Database Migration Service (DMS) is the recommended path for migrating to AlloyDB. DMS supports homogeneous migrations (PostgreSQL to AlloyDB) with continuous replication, meaning you can keep your source database live during migration and cut over with minimal downtime.

```

# Step 1: Create a connection profile for the source PostgreSQL database
gcloud database-migration connection-profiles create source-pg \
    --region=us-central1 \
    --type=POSTGRES \
    --host=10.0.1.50 \
    --port=5432 \
    --username=migration_user \
    --password=migration_password

# Step 2: Create a connection profile for the AlloyDB destination
gcloud database-migration connection-profiles create dest-alloydb \
    --region=us-central1 \
    --type=ALLOYDB \
    --cluster=prod-alloydb-cluster \
    --instance=prod-primary

# Step 3: Create the migration job
# --dump-type=LOGICAL uses pg_dump under the hood for initial load
# --migration-job-type=CONTINUOUS keeps changes streaming via logical replication
gcloud database-migration migration-jobs create pg-to-alloydb-migration \
    --region=us-central1 \
    --source=source-pg \
    --destination=dest-alloydb \
    --migration-job-type=CONTINUOUS \
    --dump-type=LOGICAL

# Step 4: Start the migration
gcloud database-migration migration-jobs start pg-to-alloydb-migration \
    --region=us-central1

# Step 5: Monitor replication lag during continuous phase
# Wait until lag is consistently near zero before cutting over
gcloud database-migration migration-jobs describe pg-to-alloydb-migration \
    --region=us-central1 \

```

```
--format="value(state,duration,error)"
```

On the source PostgreSQL side, DMS requires logical replication enabled and a replication user with appropriate permissions:

```
-- On the source PostgreSQL database (self-managed or Cloud SQL)
-- Ensure logical replication is enabled
SHOW wal_level; -- must be 'logical'

-- If not logical, update postgresql.conf and restart:
-- wal_level = logical

-- Create migration user with replication permissions
CREATE USER migration_user WITH REPLICATION LOGIN PASSWORD 'secure_password';

-- Grant select on all tables that will be migrated
GRANT SELECT ON ALL TABLES IN SCHEMA public TO migration_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT ON TABLES TO migration_user;

-- Grant replication permissions for the publication
GRANT pg_read_all_data TO migration_user;

-- DMS creates its own publication - verify after DMS initializes
SELECT pubname, puballtables, pubinsert, pubupdate, pubdelete
FROM pg_publication;
```

Schema Compatibility Validation

Before starting the migration, validate schema compatibility. The most common issues are:

```
-- Check for unsupported data types
-- JSONB, arrays, custom types all work in AlloyDB, but verify
SELECT
    table_schema,
    table_name,
    column_name,
    data_type,
    udt_name
FROM information_schema.columns
WHERE table_schema NOT IN ('pg_catalog', 'information_schema')
    AND data_type IN ('USER-DEFINED')
ORDER BY table_schema, table_name;

-- Check for extensions used by your schema
SELECT extname, extversion
FROM pg_extension
WHERE extname NOT IN ('plpgsql', 'pg_stat_statements');
-- Cross-reference this with the AlloyDB supported extension list

-- Check for custom functions that reference system internals
SELECT routine_schema, routine_name, routine_definition
FROM information_schema.routines
WHERE routine_type = 'FUNCTION'
    AND routine_schema = 'public'
    AND routine_definition ILIKE '%pg_catalog%';

-- Check for triggers on system tables (not portable)
SELECT trigger_schema, trigger_name, event_object_table
FROM information_schema.triggers
WHERE trigger_schema NOT IN ('pg_catalog', 'information_schema');

-- Sequences: verify all serial/identity columns will migrate correctly
SELECT
    sequence_schema,
    sequence_name,
    start_value,
    increment,
    maximum_value
```

```
FROM information_schema.sequences  
WHERE sequence_schema = 'public';
```

20.6 AlloyDB vs Cloud SQL for PostgreSQL: When to Use Which

This is the question every GCP-based PostgreSQL DBA faces when a new project starts or when an existing Cloud SQL cluster reaches its limits. AlloyDB is substantially more expensive than Cloud SQL for the same vCPU count, but it offers meaningfully different capabilities. The decision is not simply “AlloyDB is Cloud SQL Pro” — there are workloads where Cloud SQL is the correct answer and AlloyDB is overengineered overhead.

Use AlloyDB when:

Your workload is mixed OLTP + analytics. If you are running complex reporting queries alongside OLTP, and those queries are causing resource contention or requiring a separate reporting replica, AlloyDB’s columnar engine provides a genuine architectural solution. The columnar engine is not a toy feature — for queries that scan millions of rows with aggregations, the performance difference can be dramatic.

You need sub-second read replica provisioning. AlloyDB read pool nodes attach to the shared storage layer and can be added or removed quickly. If your application has traffic spikes that require rapid read scaling, this matters operationally.

Your data exceeds what fits in Cloud SQL’s max instance size. Cloud SQL for PostgreSQL has a maximum instance size that is substantially smaller than AlloyDB’s maximum. For databases that have grown to require more RAM and CPU than Cloud SQL can provide, AlloyDB is the path forward without leaving managed PostgreSQL.

You are building semantic search or vector similarity features. AlloyDB’s native integration of `pgvector` and `google_ml_integration` provides a development and operational experience that is better than bolting `pgvector` onto Cloud SQL.

Use Cloud SQL for PostgreSQL when:

Your database is small to medium and purely OLTP. If your dataset fits comfortably in RAM, your query mix is point lookups and simple joins, and you have no analytical workload, you are paying for AlloyDB’s columnar engine and distributed storage without using them. Cloud SQL delivers fine performance for standard OLTP workloads and at lower cost.

You need specific extensions that AlloyDB does not support. If your schema depends on TimescaleDB, a custom extension loaded from `.so`, or `pg_logical` for external replication, Cloud SQL is more likely to support your requirements.

You need cross-region replicas with native replication. Cloud SQL supports cross-region read replicas using standard PostgreSQL streaming replication. AlloyDB’s cross-region capabilities are more limited — as of mid-2024, AlloyDB cross-region disaster recovery was available as a preview feature but had more operational constraints than Cloud SQL’s cross-region replicas.

You are running a development or staging environment. AlloyDB’s minimum instance sizes and cluster overhead make it a poor fit for non-production environments. Cloud SQL’s ability to stop instances outside business hours and its lower floor cost make it better for dev/test.

Side-by-Side Performance Comparison Approach

Do not rely on benchmarks from Google’s documentation or third-party publications when making this decision for your specific workload. Run your own benchmark using your actual schema and query mix. The methodology:

```
# Step 1: Provision equivalent-sized instances
# AlloyDB: 8 vCPU primary + 2-node read pool
# Cloud SQL: 8 vCPU PostgreSQL instance with a read replica

# Step 2: Load representative data
# Use pgbench with a scale factor that produces your typical data volume
# or restore a production backup (anonymized if needed)

pgbench -i -s 500 -h ALLOYDB_IP -U postgres benchdb
pgbench -i -s 500 -h CLOUDSQL_IP -U postgres benchdb

# Step 3: Run a mixed workload test
# Default pgbench is pure OLTP - add custom scripts for your analytical queries
pgbench -c 50 -j 4 -T 300 \
  -f oltp_script.sql \
  -f analytics_script.sql \
  -h ALLOYDB_IP -U postgres benchdb

# Step 4: Measure at the right layer
# Don't just look at pgbench TPS - measure:
# - p50, p95, p99 query latency (use pg_stat_statements)
# - CPU and memory utilization (Cloud Monitoring)
# - Replication lag under load
# - Buffer hit ratio (indicates whether columnar engine is helping)
```

20.7 Monitoring AlloyDB in Production

AlloyDB exposes metrics through Cloud Monitoring, and the query-level visibility comes through Query Insights — the same tool available in Cloud SQL but with AlloyDB-specific additions for the columnar engine.

Essential Cloud Monitoring Metrics

```
# Useful gcloud monitoring queries for AlloyDB
# (In practice, you build dashboards in the Cloud Monitoring UI or Terraform)

# List available AlloyDB metrics
gcloud monitoring metrics list \
  --filter="metric.type=starts_with('alloydb.googleapis.com')" \
  --format="value(metric.type)"
```

The key metrics you should alarm on for a production AlloyDB cluster:

Metric	Description	Alert Threshold
alloydb.googleapis.com/instance/ <i>instance</i> /cpu_utilization	CPU utilization	> 80% sustained
alloydb.googleapis.com/instance/ <i>instance</i> /memory_utilization	Memory utilization	> 90%
alloydb.googleapis.com/instance/ <i>instance</i> /postgres/replication_lag	Replication lag	> 30s
alloydb.googleapis.com/instance/ <i>instance</i> /postgres/connections	Active PostgreSQL connections	> 80% of max_connections
alloydb.googleapis.com/instance/ <i>instance</i> /storage/used_bytes	Storage used in bytes	> 80% of provisioned
alloydb.googleapis.com/instance/ <i>instance</i> /columnar_engine/memory_used	Columnar engine memory used	> 80% of allocated

Query-Level Monitoring

AlloyDB inherits `pg_stat_statements` and adds its own query observability layer:

```
-- Top queries by total execution time (standard pg_stat_statements)
SELECT
  LEFT(query, 80) AS query_snippet,
  calls,
  ROUND(total_exec_time::NUMERIC, 2) AS total_ms,
  ROUND(mean_exec_time::NUMERIC, 2) AS mean_ms,
  ROUND(stddev_exec_time::NUMERIC, 2) AS stddev_ms,
  rows,
  ROUND(100.0 * shared_blks_hit /
    NULLIF(shared_blks_hit + shared_blks_read, 0), 2) AS buffer_hit_pct
FROM pg_stat_statements
WHERE calls > 100
ORDER BY total_exec_time DESC
LIMIT 20;

-- Identify queries that are NOT hitting the columnar engine
-- but look like they should (table scans on large tables)
SELECT
  schemaname,
  relname,
  seq_scan,
  seq_tup_read,
  idx_scan,
  n_live_tup,
  ROUND(seq_tup_read::NUMERIC / NULLIF(seq_scan, 0), 0) AS avg_rows_per_seq_scan
FROM pg_stat_user_tables
WHERE seq_scan > 10
  AND n_live_tup > 100000
ORDER BY seq_tup_read DESC
LIMIT 20;

-- Check columnar engine status and memory usage
SELECT *
FROM google_columnar_engine_tables
ORDER BY size_in_bytes DESC;

-- Check if specific queries used the columnar engine
-- Look for 'Columnar Scan' nodes in EXPLAIN output
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
  category,
  DATE_TRUNC('month', order_date) AS month,
  COUNT(*) AS order_count,
  SUM(order_total) AS revenue
FROM orders
WHERE order_date >= '2023-01-01'
GROUP BY category, DATE_TRUNC('month', order_date)
ORDER BY month, revenue DESC;

-- Monitor autovacuum activity (still relevant in AlloyDB)
SELECT
  schemaname,
  relname,
  n_dead_tup,
  n_live_tup,
  ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_pct,
  last_autovacuum,
  last_autoanalyze
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY dead_tup DESC
LIMIT 15;
```

AlloyDB-Specific Operational Queries

```
-- Check storage layer health (AlloyDB-specific view)
```

```
SELECT *
FROM pg_stat_replication;
-- Note: In AlloyDB, this shows internal replication to the storage nodes,
-- not traditional streaming replication to replicas.
-- Replica lag is monitored via Cloud Monitoring, not this view.

-- Connection count by state
SELECT
    state,
    wait_event_type,
    wait_event,
    COUNT(*) AS connection_count
FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
GROUP BY state, wait_event_type, wait_event
ORDER BY connection_count DESC;

-- Long-running transactions (a leading indicator of lock contention)
SELECT
    pid,
    username,
    application_name,
    client_addr,
    state,
    wait_event_type,
    wait_event,
    EXTRACT(EPOCH FROM (NOW() - xact_start)) AS xact_age_seconds,
    LEFT(query, 100) AS current_query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
    AND state <> 'idle'
    AND EXTRACT(EPOCH FROM (NOW() - xact_start)) > 30
ORDER BY xact_age_seconds DESC;

-- Lock contention view
SELECT
    blocked.pid AS blocked_pid,
    blocked.username AS blocked_user,
    blocking.pid AS blocking_pid,
    blocking.username AS blocking_user,
    blocked.query AS blocked_query,
    blocking.query AS blocking_query
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocking
    ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE pg_blocking_pids(blocked.pid) <> '{}';
```

20.8 High Availability and Backup in AlloyDB

AlloyDB's HA model differs from Cloud SQL's in a way that matters for failover behavior. Cloud SQL HA uses a warm standby instance in a different zone that takes over on primary failure. AlloyDB's HA is embedded in the storage layer: because the Log Store replicates across three zones, the primary instance can fail and the storage state is immediately available for the replacement instance. Failover in AlloyDB typically completes in under 60 seconds, and because no WAL replay is needed, the new primary instance resumes from exactly the storage state at failure time.

Failover Testing

```
# Trigger a manual failover to test HA behavior
# This simulates the primary instance failing
gcloud alloydb instances failover prod-primary \
    --cluster=prod-alloydb-cluster \
    --region=us-central1
```

```
# Monitor the cluster state during failover
watch -n 5 gcloud alloydb instances list \
  --cluster=prod-alloydb-cluster \
  --region=us-central1

# After failover completes, verify the primary is healthy
gcloud alloydb instances describe prod-primary \
  --cluster=prod-alloydb-cluster \
  --region=us-central1 \
  --format="value(state,ipAddress)"
```

Continuous Backup and Point-in-Time Recovery

AlloyDB’s backup model combines automated periodic backups with continuous WAL archiving to achieve PITR with recovery windows of up to 14 days. The continuous backup is not a feature you enable in addition to snapshots — it is the primary backup mechanism, and the periodic backups are baseline checkpoints within the continuous stream.

```
# View available backups
gcloud alloydb backups list \
  --region=us-central1 \
  --cluster=prod-alloydb-cluster

# Restore to a point in time - this creates a NEW cluster
# AlloyDB cannot restore in-place to the same cluster
# The --restore-time is in RFC3339 format
gcloud alloydb clusters restore prod-alloydb-cluster-restored \
  --region=us-central1 \
  --network=projects/your-project/global/networks/prod-vpc \
  --source-cluster=prod-alloydb-cluster \
  --restore-time=2024-08-15T14:30:00Z

# After restore, create a primary instance in the restored cluster
gcloud alloydb instances create prod-primary \
  --cluster=prod-alloydb-cluster-restored \
  --region=us-central1 \
  --instance-type=PRIMARY \
  --cpu-count=8
```

The restore-to-a-new-cluster behavior is an operational consideration: you cannot restore a backup over your existing cluster. If you need to recover from data corruption or an accidental mass delete, you restore to a new cluster, validate the data, and then either rename/reroute connections or copy the data back to the original cluster. This is the same operational pattern as Aurora’s PITR restore, and it means you need a runbook for “what happens when we need to recover data” that accounts for this constraint.

Manual Backups

```
# Create an on-demand backup before a risky schema change
gcloud alloydb backups create pre-migration-backup-20240815 \
  --cluster=prod-alloydb-cluster \
  --region=us-central1 \
  --description="Backup before v2.4 schema migration"

# List backups with their retention expiration
gcloud alloydb backups list \
  --region=us-central1 \
  --format="table(name,state,createTime,expiryTime,description)"
```

Key Takeaways

- **AlloyDB’s disaggregated storage architecture** eliminates checkpoint I/O pressure on the primary and enables read replicas to provision in seconds rather than hours, but the shared Log Store model means replication lag metrics are fundamentally different from `pg_stat_replication.replay_lag` and require Cloud Monitoring for accurate measurement.
 - **The columnar engine enables mixed OLTP and analytical workloads** on a single AlloyDB cluster by transparently routing analytical queries through a compressed, in-memory columnar representation of selected tables — use `google_columnar_engine_recommended_tables()` to identify candidates and validate with `EXPLAIN (ANALYZE, BUFFERS)` before committing to columnarization.
 - **In-database ML inference via google_ml_integration** allows embedding generation and model scoring directly from SQL using Vertex AI endpoints, but synchronous per-row invocation adds Vertex AI round-trip latency to query execution time; the practical production pattern is generating embeddings at write time and using `pgvector` similarity search at query time.
 - **AlloyDB is not a drop-in replacement for all Cloud SQL workloads** — small OLTP databases, workloads that depend on unsupported extensions like TimescaleDB, development environments, and applications that require cross-region read replicas today are better served by Cloud SQL for PostgreSQL at lower operational cost.
 - **AlloyDB’s HA failover and PITR backup model** provides sub-60-second primary failover without WAL replay, and continuous backup with up to 14-day PITR windows, but point-in-time recovery always creates a new cluster rather than restoring in-place, which requires explicit runbook design for data recovery scenarios.
-

Chapter 21: Spanner: When You Need Global Consistency

Google Cloud Spanner sits in a category by itself. It is the only production-grade relational database that offers external consistency across geographically distributed nodes — not eventual consistency, not tunable consistency, but true serializable transactions that span continents without the application needing to know where the data physically lives. For experienced DBAs, this sounds like marketing copy. It is not. Spanner achieves this through TrueTime, Google’s GPS and atomic clock-based timekeeping infrastructure, which bounds clock uncertainty to single-digit milliseconds and uses that bound as a synchronization primitive. If you have ever spent nights managing multi-region PostgreSQL with Patroni, or fought with SQL Server Always On availability groups across data centers, understanding what Spanner removes from your operational surface area is genuinely striking. This chapter covers when Spanner is the right choice, how its data model differs from what you know, how to design schemas that perform at scale, how to operate Spanner in production, and where the limits and costs will surprise you.

21.1 TrueTime and External Consistency: What It Actually Means

Before touching configuration or schema design, you need to internalize the consistency model, because it drives every subsequent architectural decision.

What you’re used to. In PostgreSQL with streaming replication, you have a primary that accepts writes and standbys that replay WAL. If you read from a standby, you accept replication lag — typically milliseconds in a healthy system but unbounded in theory. If you need your read to reflect a write that just happened, you read from the primary. With Citus or logical replication across regions, the problem compounds. You either accept stale reads or you route everything through one node and sacrifice the performance benefit of having multiple regions. With SQL Server Distributed Availability Groups, the story is similar: synchronous commit to one replica, asynchronous everywhere else, and a failover that requires manual intervention or complex automation.

What Spanner provides. Spanner guarantees external consistency, which means: if transaction T1 commits before transaction T2 begins (in real-world time), then T2 will always see T1’s effects. Always. Regardless of which Spanner node processes T2. Regardless of which geographic region that node is in. There is no replication lag observable to the application. A write that commits in Iowa is immediately visible to a read in Singapore.

This works because TrueTime assigns each transaction a commit timestamp from a globally synchronized clock. Spanner will actually pause a transaction before committing — for up to the clock uncertainty bound, roughly 7ms — to ensure that the committed timestamp is genuinely in the past relative to real time. No subsequent transaction can receive an earlier timestamp. The consequence is a globally consistent external view of all committed data.

What this means operationally. The consistency model that you used to enforce through careful read/write routing, synchronous replication configuration, and application-level retry logic is handled by the database engine itself. You do not configure synchronous commit. You do not choose which replica to read from. You do not manage replication lag monitoring as a first-class operational concern. Those problems simply do not exist in Spanner in the way they exist in traditional distributed systems.

The tradeoff is write latency. A write to Spanner must be replicated across a quorum of nodes before it commits. In a single-region configuration, that means three nodes within one GCP region — latency comparable to synchronous standby commits in PostgreSQL. In a multi-region configuration, quorum involves nodes across multiple regions, and your commit latency will reflect the round-trip time between those regions — the wider the geographic spread of the configuration, the higher the floor on write latency. Measure actual commit latency against your chosen instance configuration rather than assuming a figure; it is not tunable, and you do not choose eventual consistency to reduce latency. If low write latency matters more than global consistency, Spanner is the wrong tool.

The external consistency guarantee in SQL terms. Spanner’s consistency level maps directly to `SERIALIZABLE` isolation in PostgreSQL terms — the strictest isolation level. Spanner does not offer `READ COMMITTED` or `REPEATABLE READ` as selectable alternatives. Every read-write transaction is serializable. Read-only transactions use a snapshot that is consistent with the external consistency model. This means your application cannot accidentally be misconfigured to use weaker isolation. It also means some transaction patterns that worked under weaker isolation levels will behave differently, and you need to be aware of that during migration.

21.2 Spanner’s Data Model: Interleaving, Keys, and Schema Design

Spanner is a relational database. It supports ANSI SQL (via GoogleSQL dialect), foreign keys, secondary indexes, check constraints, and views. But its physical data organization is fundamentally different from heap-based storage in PostgreSQL or the B-tree clustered index model in SQL Server, and designing schemas without understanding this will result in severe performance problems at scale.

Tables and the primary key requirement. Every Spanner table must have a primary key. There are no heap tables. There are no sequences in the PostgreSQL or SQL Server sense. This matters immediately: you cannot use `SERIAL` or `IDENTITY` columns that generate monotonically increasing integers. If you do, you will create a write hotspot — all inserts go to the same tablet (Spanner’s unit of data sharding), serialized behind each other, and you will never achieve horizontal write scalability. This is the single most common mistake DBAs make when migrating to Spanner.

The correct patterns for generating primary keys in Spanner: - **UUID v4:** Randomly distributed across the key space. Spanner distributes tablets based on key ranges, so random UUIDs spread

writes across many splits immediately. - **Bit-reversed sequences:** Spanner natively supports `GET_NEXT_SEQUENCE_VALUE` with bit-reversal, which takes an incrementing integer and reverses its bits, producing a value that distributes evenly across the key space without the monotonicity problem. - **Hash prefix:** If you need a meaningful natural key, prepend a hash of the key to distribute writes.

```
-- CREATE TABLE with UUID primary key (GoogleSQL dialect)
CREATE TABLE Orders (
  OrderId      STRING(36)  NOT NULL,
  CustomerId   STRING(36)  NOT NULL,
  OrderDate    TIMESTAMP   NOT NULL,
  Status       STRING(32)  NOT NULL,
  TotalAmount  NUMERIC     NOT NULL,
  CreatedAt    TIMESTAMP   NOT NULL OPTIONS (allow_commit_timestamp = true),
) PRIMARY KEY (OrderId);

-- Bit-reversed sequence (avoids hotspotting on sequential inserts)
CREATE SEQUENCE OrderSequence OPTIONS (
  sequence_kind = "bit_reversed_positive"
);

CREATE TABLE OrdersSeq (
  OrderId      INT64       NOT NULL DEFAULT (GET_NEXT_SEQUENCE_VALUE(SEQUENCE OrderSequence)),
  CustomerId   STRING(36)  NOT NULL,
  OrderDate    TIMESTAMP   NOT NULL,
  Status       STRING(32)  NOT NULL,
) PRIMARY KEY (OrderId);
```

Interleaving: physically co-locating parent and child rows. In PostgreSQL, if you join `Orders` and `OrderItems` frequently, you rely on indexes, buffer cache, and query planning to make that join fast. The rows for a given order's items are scattered across the heap based on insertion order. In Spanner, you can declare that `OrderItems` rows are physically interleaved within `Orders` rows — the storage engine places each `OrderItems` row immediately after its parent `Orders` row on disk. The join between `Orders` and `OrderItems` for a specific `OrderId` becomes a sequential scan of a contiguous storage region rather than a random access join.

```
-- Interleaved table: OrderItems rows stored inside Orders rows
CREATE TABLE OrderItems (
  OrderId      STRING(36)  NOT NULL,
  ItemId       STRING(36)  NOT NULL,
  ProductId    STRING(36)  NOT NULL,
  Quantity     INT64       NOT NULL,
  UnitPrice    NUMERIC     NOT NULL,
  LineTotal    NUMERIC     NOT NULL,
) PRIMARY KEY (OrderId, ItemId),
  INTERLEAVE IN PARENT Orders ON DELETE CASCADE;
```

The constraint is that the child table's primary key must begin with the parent's primary key. `OrderItems(OrderId, ItemId)` begins with `OrderId`, which is `Orders`' primary key. Deletion from the parent with `ON DELETE CASCADE` removes interleaved children atomically. Without `ON DELETE CASCADE`, Spanner enforces referential integrity at the storage level — you cannot delete a parent row that has children.

Interleaving depth is limited (check current Spanner documentation for the exact maximum), but in practice, two or three levels covers the vast majority of real schemas. Over-interleaving creates wide rows and affects tablet splitting behavior.

Secondary indexes. Spanner supports secondary indexes with a behavior that DBA experience on traditional RDBMS does not fully prepare you for. When you create a secondary index on a column, Spanner creates a separate interleaved table to store the index. Reads against a secondary

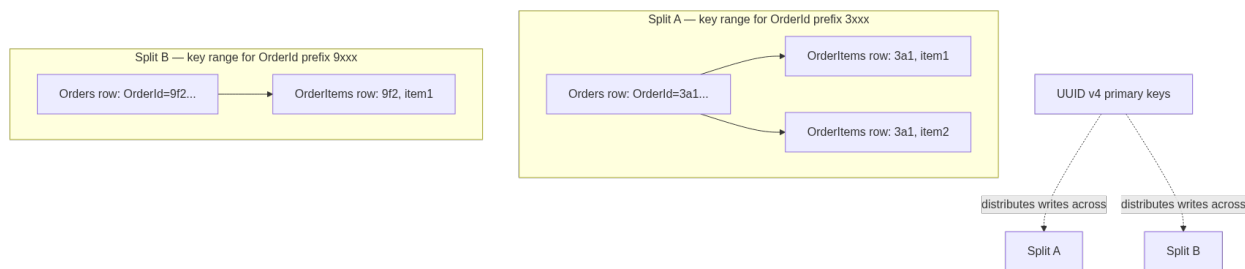


Figure 12: Interleaving places each **OrderItems** row physically next to its parent **Orders** row, turning the join into a sequential scan; UUID primary keys spread rows (and writes) across many splits instead of hotspotting one.

index may need to join back to the base table to retrieve columns not stored in the index — called a back-join or index back-lookup. For large tables, this back-join crosses tablet boundaries and is expensive.

The solution is to store additional columns in the index itself using **STORING**:

```
-- Secondary index without STORING: back-join required for Status and TotalAmount
CREATE INDEX IX_Orders_CustomerId ON Orders(CustomerId);

-- Secondary index with STORING: no back-join needed for common query
CREATE INDEX IX_Orders_CustomerId_Storing ON Orders(CustomerId)
STORING (Status, TotalAmount, OrderDate);
```

When you define a query in your application that filters by **CustomerId** and reads **Status** and **TotalAmount**, use the second index form. The query reads entirely from the index without touching the base table rows. At 100 million rows, the difference between a back-join and a covering index in Spanner is the difference between a 50ms query and a 5000ms query.

Array and STRUCT columns. Spanner supports **ARRAY<T>** columns natively, allowing you to store repeated values within a single row. This is useful for tags, labels, and other set-valued attributes that would require a junction table in a normalized relational schema. This is not a replacement for interleaving — use arrays for small, bounded sets of scalar values, and use interleaved tables for child entities with their own attribute columns.

21.3 Provisioning and Configuration: Instances, Databases, and Processing Units

Spanner's resource model does not map to anything in traditional DBA experience. There are no instance types to select, no vCPU counts to choose, no memory configurations to evaluate. Spanner's compute capacity is expressed in **Processing Units (PUs)**.

Processing Units and nodes. 1 node = 1000 PUs. Google's documentation expresses smaller configurations in PUs (100 PU, 500 PU) and larger ones in nodes (1 node, 2 nodes, 10 nodes). For production workloads, the minimum meaningful configuration is 1 node (1000 PU) per region. Sub-node configurations (100–500 PU) are suitable for development and testing. Per-node throughput is highly dependent on row size, transaction complexity, and read/write mix — check current Google documentation for the reference QPS figures per node rather than relying on a fixed number, and

benchmark your own schema before sizing a production instance. The one hard, well-documented ceiling worth planning around is storage: Spanner recommends staying under roughly 2 TB per node before cross-split transaction overhead starts to bite (Spanner auto-splits data, but very large datasets relative to node count increase that overhead).

Provisioning with Terraform:

```
# Spanner instance and database via Terraform
resource "google_spanner_instance" "production" {
  name       = "prod-spanner"
  config     = "nam6"          # North America multi-region config
  display_name = "Production Spanner"
  num_nodes  = 3               # 3 nodes across nam6 multi-region footprint

  labels = {
    environment = "production"
    team        = "platform-data"
  }
}

resource "google_spanner_database" "orders_db" {
  instance = google_spanner_instance.production.name
  name     = "orders"

  version_retention_period = "7d" # Point-in-time recovery window: 7 days
  deletion_protection      = true

  ddl = [
    <<-EOT
    CREATE TABLE Orders (
      OrderId      STRING(36) NOT NULL,
      CustomerId   STRING(36) NOT NULL,
      OrderDate    TIMESTAMP  NOT NULL,
      Status       STRING(32)  NOT NULL,
      TotalAmount  NUMERIC    NOT NULL,
      CreatedAt    TIMESTAMP  NOT NULL OPTIONS (allow_commit_timestamp = true)
    ) PRIMARY KEY (OrderId)
    EOT
    ,
    <<-EOT
    CREATE INDEX IX_Orders_CustomerId ON Orders(CustomerId)
    STORING (Status, TotalAmount, OrderDate)
    EOT
  ]
}
```

Instance configurations: regional vs. multi-region. The `config` parameter in the instance definition determines the geographic footprint. This is not a setting you change after the fact without recreating the instance. Choose carefully.

Configuration	Regions	Write Latency Trend	Use Case
us-central1	Single region, 3 zones	Lowest — single-region quorum	Regional apps, dev/test
nam4	Iowa + Virginia (2 regions)	Higher — cross-region quorum	US multi-region
nam6	4 North American regions	Higher still — wider quorum footprint	High-availability US
eur5	5 European regions	Higher — cross-region quorum	EU data residency

Configuration	Regions	Write Latency Trend	Use Case
global2	Multiple continents	Highest — intercontinental quorum	Truly global workloads

Benchmark actual write commit latency for your chosen configuration rather than relying on a published figure — it depends on the specific region pairing, transaction size, and current network conditions between Google’s data centers.

Multi-region configurations also affect cost significantly — a multi-region instance costs meaningfully more than an equivalent node count in a single region, because Spanner replicates across more physical nodes under the hood to maintain quorum across all participating regions. Check current Spanner pricing for your specific configuration before committing to a topology.

Autoscaler. Spanner does not automatically scale compute capacity. You must either use the open-source [Spanner Autoscaler](#) project (a Cloud Function-based solution that monitors CPU utilization and adjusts node count) or manage scaling manually. Google recommends keeping Spanner CPU utilization below 65% for single-region and below 45% for high-priority traffic in multi-region configurations. Above these thresholds, tail latency increases significantly.

```
# Check current instance configuration and node count
gcloud spanner instances describe prod-spanner \
  --format="table(name, config, nodeCount, processingUnits, state)"

# Scale up to 5 nodes for peak traffic period
gcloud spanner instances update prod-spanner \
  --nodes=5 \
  --description="Scaled for Q4 peak"

# Verify the update applied (scaling is online, no downtime)
gcloud spanner instances describe prod-spanner \
  --format="value(nodeCount)"
```

Scaling is online. There is no restart, no maintenance window, no connection interruption. Spanner redistributes tablet ownership across the new capacity within minutes.

21.4 Transactions, Mutations, and the Application Layer

If your application was built against PostgreSQL or SQL Server, the way it issues reads, writes, and transactions will need adjustment for Spanner. Not because Spanner is semantically weaker — it is stronger — but because some patterns that are idiomatic in traditional databases are inefficient or unsupported in Spanner.

Transaction types. Spanner offers three transaction modes that the application explicitly chooses between:

1. **Read-write transactions:** Full ACID, serializable, can include both reads and writes. All reads within the transaction see a consistent snapshot. Writes are buffered and committed atomically. Any conflict causes a retry-able abort error (`ABORTED` status). Your application must handle retries.

2. **Read-only transactions:** No locks, no aborts, no retries needed. Reads are served from a consistent snapshot at a timestamp you specify (or the current time). Extremely efficient and scalable. If you are reading data and not writing, always use a read-only transaction.
3. **Partitioned DML:** For large-scale data modification (backfills, bulk deletes, archiving). Runs as a series of smaller transactions internally. Does not guarantee atomicity across the full affected row set. Suitable for operations where eventual completion matters more than all-or-nothing atomicity (clearing expired sessions, archiving old orders). The SQL analog is running a batch delete in chunks.

The most common operational mistake from PostgreSQL-experienced DBAs is using read-write transactions for read-only operations. In PostgreSQL, `BEGIN; SELECT ...; COMMIT;` is harmless. In Spanner, it acquires shared locks unnecessarily and increases contention on hot rows.

Mutations vs. DML. Spanner supports two ways to write data: DML (`INSERT`, `UPDATE`, `DELETE` statements) and Mutations (a batch API that specifies key-value changes directly). Mutations are more efficient for bulk operations because they bypass the SQL parsing and execution layer. For single-row writes in OLTP context, the difference is negligible. For batch loading thousands of rows, mutations are substantially faster.

```
# Python client: read-write transaction with proper retry handling
from google.cloud import spanner
from google.cloud.spanner_v1 import param_types
import uuid

client = spanner.Client(project="your-gcp-project")
instance = client.instance("prod-spanner")
database = instance.database("orders")

def insert_order_with_items(order_data, items_data):
    """
    Insert an order and its items in a single read-write transaction.
    Spanner client handles ABORTED retries automatically via run_in_transaction.
    """
    def _transaction_body(transaction):
        order_id = str(uuid.uuid4())

        # DML insert for the parent row
        transaction.execute_update(
            """
            INSERT INTO Orders (OrderId, CustomerId, OrderDate, Status, TotalAmount, CreatedAt)
            VALUES (@order_id, @customer_id, @order_date, @status, @total, PENDING_COMMIT_TIMESTAMP())
            """,
            params={
                "order_id": order_id,
                "customer_id": order_data["customer_id"],
                "order_date": order_data["order_date"],
                "status": "PENDING",
                "total": order_data["total_amount"],
            },
            param_types={
                "order_id": param_types.STRING,
                "customer_id": param_types.STRING,
                "order_date": param_types.TIMESTAMP,
                "status": param_types.STRING,
                "total": param_types.NUMERIC,
            }
        )

    # Batch mutations for child rows (more efficient than per-row DML)
    item_mutations = []
    for item in items_data:
        item_mutations.append(
            spanner.Mutation(
                # placeholder - use spanner.Mutation directly
```

```

    )

    # Using mutations for bulk child inserts
    transaction.insert(
        table="OrderItems",
        columns=("OrderId", "ItemId", "ProductId", "Quantity", "UnitPrice", "LineTotal"),
        values=[
            (order_id, str(uuid.uuid4()), item["product_id"],
             item["quantity"], item["unit_price"], item["line_total"])
            for item in items_data
        ]
    )

    return order_id

# run_in_transaction retries automatically on ABORTED
return database.run_in_transaction(_transaction_body)

def read_customer_orders(customer_id, limit=100):
    """
    Read-only transaction: no locks, no retry needed, served from snapshot.
    Always use with_snapshot for read-only workloads.
    """
    with database.snapshot() as snapshot:
        results = snapshot.execute_sql(
            """
            SELECT o.OrderId, o.OrderDate, o.Status, o.TotalAmount
            FROM   Orders@{FORCE_INDEX=IX_Orders_CustomerId_Storing} o
            WHERE  o.CustomerId = @customer_id
            ORDER BY o.OrderDate DESC
            LIMIT  @limit
            """,
            params={"customer_id": customer_id, "limit": limit},
            param_types={
                "customer_id": param_types.STRING,
                "limit":       param_types.INT64,
            }
        )
    return [dict(row) for row in results]

```

Note the `@{FORCE_INDEX=...}` hint in the SQL. Spanner's query optimizer will use an index if it determines the index is beneficial, but for queries on large tables where you know the index is correct, forcing index usage prevents surprising full-table-scan query plans after statistics updates change the optimizer's estimate.

Stale reads. If your application can tolerate reading data that is up to N seconds old, stale reads dramatically improve throughput and reduce latency. A stale read can be served by any Spanner replica locally, without coordination across regions. For analytics queries, dashboard data, and reference data lookups, 15-second stale reads are typically acceptable and can cut read latency by 50% in multi-region configurations.

```

import datetime
from google.cloud.spanner_v1 import DirectedReadOptions

# Bounded staleness read: data no older than 15 seconds
with database.snapshot(max_staleness=datetime.timedelta(seconds=15)) as snapshot:
    results = snapshot.execute_sql(
        "SELECT COUNT(*) as order_count FROM Orders WHERE Status = 'PENDING'"
    )
    for row in results:
        print(f"Pending orders: {row[0]}")

# Exact timestamp read: useful for repeatable analytics
# Read the database as it existed exactly 5 minutes ago
five_minutes_ago = datetime.datetime.utcnow() - datetime.timedelta(minutes=5)

```



```

with database.snapshot(read_timestamp=five_minutes_ago) as snapshot:
    results = snapshot.execute_sql(
        "SELECT SUM(TotalAmount) as revenue FROM Orders WHERE Status = 'COMPLETED'"
    )

```

21.5 Observability, Operational Patterns, and Migration Realities

Running Spanner in production requires a different observability mindset than traditional RDBMS operations. The metrics and signals that matter are different, and the absence of some traditional DBA tools is compensated by managed infrastructure but requires adjustment.

Key metrics to monitor. In PostgreSQL, you track `pg_stat_activity`, lock wait events, replication lag, vacuum activity, and bloat. In Spanner, your primary operational signals are:

Metric	Alert Threshold	Consequence of Breach
CPU Utilization (single-region)	> 65%	Latency tail increase
CPU Utilization (multi-region, high-priority)	> 45%	Latency tail increase
Request Latency P99 (reads)	> 100ms	Application SLO risk
Request Latency P99 (writes)	> 200ms (multi-region)	Application SLO risk
Transaction Abort Rate	> 5%	Inefficient transaction design
Locked Resource Wait Time	> 50ms	Hot key contention

```

# Export Spanner metrics to Cloud Monitoring and set alerting policy
gcloud alpha monitoring policies create \
  --policy-from-file=spanner-cpu-alert.yaml

# spanner-cpu-alert.yaml content:
cat <<'EOF' > spanner-cpu-alert.yaml
displayName: "Spanner High CPU Utilization"
conditions:
  - displayName: "CPU > 65% for 5 minutes"
    conditionThreshold:
      filter: >
        resource.type = "spanner_instance"
        AND metric.type = "spanner.googleapis.com/instance/cpu/utilization"
        AND resource.labels.instance_id = "prod-spanner"
      comparison: COMPARISON_GT
      thresholdValue: 0.65
      duration: 300s
      aggregations:
        - alignmentPeriod: 60s
          perSeriesAligner: ALIGN_MEAN
alertStrategy:
  autoClose: 604800s
notificationChannels:
  - projects/your-project/notificationChannels/your-channel-id
EOF

```

Schema changes in production. Spanner handles DDL differently from PostgreSQL or SQL Server. In PostgreSQL, `ALTER TABLE ADD COLUMN` is typically fast (for nullable columns) but `ALTER TABLE ADD COLUMN NOT NULL DEFAULT value` requires a table rewrite. In SQL Server, online index operations require Enterprise edition. In Spanner, all DDL changes are non-blocking and online — you issue the DDL statement and Spanner applies it in the background while reads and writes

continue. Adding a column, adding an index, modifying a column's nullable status — all proceed without locking the table or interrupting traffic.

The operational consequence is that DDL changes in Spanner do not require maintenance windows. However, they are not instantaneous. Adding an index on a 10-billion-row table takes minutes to hours. During that period, the index is being backfilled. You can monitor DDL operation progress:

```
# List ongoing schema change operations
gcloud spanner databases operations list \
  --instance=prod-spanner \
  --database=orders \
  --filter="metadata.@type:UpdateDatabaseDdlMetadata"

# Describe a specific operation to check progress
gcloud spanner databases operations describe \
  projects/your-project/instances/prod-spanner/databases/orders/operations/OPERATION_ID \
  --instance=prod-spanner \
  --database=orders
```

You can also check DDL operation status via the Information Schema:

```
-- Check pending or running schema change operations in GoogleSQL
SELECT
  OPERATION_ID,
  STATEMENTS,
  START_TIME,
  COMMIT_TIMESTAMPS,
  DONE
FROM INFORMATION_SCHEMA.SPANNER_SYS.SCHEMA_CHANGES
ORDER BY START_TIME DESC
LIMIT 20;
```

Point-in-time recovery (PITR). The `version_retention_period` on the database (set earlier in the Terraform example to 7d) enables PITR. Spanner retains all data versions for the specified window. To recover from an accidental bulk delete, you issue a DML query reading from a past timestamp and reinserting those rows:

```
-- Recover accidentally deleted orders from 2 hours ago
-- GoogleSQL: FOR SYSTEM_TIME AS OF takes an ISO 8601 timestamp
INSERT INTO Orders
SELECT *
FROM Orders FOR SYSTEM_TIME AS OF '2024-11-15T10:00:00Z'
WHERE CustomerId = 'cust-abc-123'
AND OrderDate >= '2024-11-14'
AND OrderDate < '2024-11-15';
```

This is online, transactional, and does not require spinning up a separate recovery instance. The same temporal query syntax lets you audit past states, compare snapshots, or answer “what did this row look like yesterday?”

Hot spots and the key design problem in operations. The most common production incident pattern in Spanner is a hot tablet. If a burst of writes all target the same key prefix — because a new feature started generating sequential IDs, or an importer is bulk-loading data with a fixed `CustomerId` prefix — all those writes land on one Spanner split. You will see CPU spike on one node while others are idle, and write latency increases nonlinearly.

Detection:

```
# Key Visualizer: Google's built-in hot-spot detection tool
# Open in Cloud Console: Navigation > Spanner > Instance > Database > Key Visualizer
# Shows a heatmap of reads and writes by key range over time
# A vertical bright stripe in one key range = hot tablet

# Programmatic: check for high lock contention via Cloud Monitoring
```

```
gcloud monitoring metrics list \
  --filter="metric.type = spanner.googleapis.com/lock_stat/total/lock_wait_count"
```

Remediation depends on root cause: - **Monotonic key generation**: Replace with UUID v4 or bit-reversed sequences. - **Imbalanced secondary index**: If the secondary index itself is hotspotting (e.g., a low-cardinality indexed column receiving all new writes), add a hash prefix to the index key or restructure the index. - **Single large customer hotspot**: Consider a shard column — adding a small integer (0–N) prefix to the key for high-volume entities and distributing writes across shards in the application layer. This complicates application logic but solves physical hotspots.

Migrating to Spanner from PostgreSQL. Migrations are not schema-port-and-done. The differences that require active decisions:

1. **Sequences and identity columns**: As discussed, replace all `SERIAL`/`IDENTITY`/`BIGSERIAL` columns with UUIDs or bit-reversed sequences. This is a data model change, not just a syntax change — foreign keys referencing those columns must also change.
2. **Stored procedures and triggers**: Spanner does not support stored procedures or triggers. Any business logic housed in PL/pgSQL or T-SQL procedures must move to the application layer. This is a significant refactor for schema-heavy systems.
3. **Joins across uninterleaved tables**: Cross-table joins in Spanner work correctly but can be expensive for large tables without careful index design. Audit your most critical queries before migration and verify that either interleaving or covering indexes will serve them efficiently.
4. **Application retry logic**: Your application must handle `ABORTED` errors from read-write transactions. PostgreSQL serializable transactions also produce serialization failures, but many applications avoid `SERIALIZABLE` precisely to avoid retries. With Spanner, you have no choice. The client libraries provide automatic retry wrappers (`run_in_transaction` in Python, `ReadWriteTransaction` with `retry` in Go and Java), and you should use them rather than implementing retry logic manually.
5. **Data type mapping**: Several PostgreSQL types have no direct Spanner equivalent. `JSONB` maps to Spanner's `JSON` type but without the rich indexing operators that make `JSONB` performant in PostgreSQL. `ENUM` types must become `STRING` columns with check constraints. `NUMERIC` in Spanner is exact decimal with 29 digits of precision and 9 decimal places — verify that your financial calculations fit within this precision. `ARRAY` of complex types is not supported; flatten to scalar arrays or use interleaved tables.

A practical migration sequence:

Phase 1: Schema translation

- Map PostgreSQL types to Spanner types
- Replace sequential PKs with UUIDs or bit-reversed sequences
- Identify interleaving candidates (parent-child tables with co-located access patterns)
- Translate stored procedures to application-layer service classes
- Create Spanner schema DDL, test in staging

Phase 2: Data load

- Use Dataflow + Spanner connector for large tables (parallelized mutations, handles rate limit)
- Validate row counts and spot-check data fidelity
- Verify referential integrity in Spanner post-load

Phase 3: Dual-write validation

- Write to both PostgreSQL (old) and Spanner (new) simultaneously
- Read from PostgreSQL, compare against Spanner reads for consistency
- Run this phase until confidence threshold is met (typically 2-4 weeks)

Phase 4: Read cutover

- Shift reads to Spanner, keep PostgreSQL writes as rollback option
- Monitor latency, error rates, and abort rates closely

Phase 5: Write cutover and PostgreSQL decommission

- Stop writes to PostgreSQL
- Verify Spanner as sole source of truth
- Archive PostgreSQL instance before termination

Costs at scale. Spanner pricing has three components: compute (per node-hour or per 1000 PU-hour), storage (per GB-month), and network egress. For large workloads, the compute cost dominates. A 3-node multi-region `nam6` instance runs continuously at a cost that can reach tens of thousands of dollars per month. This is not a replacement for a \$500/month RDS PostgreSQL instance. Spanner is justified when the alternative cost — engineering time for distributed systems infrastructure, operational incidents, data consistency bugs, multi-region replication management — exceeds its compute cost.

Before committing to Spanner for a new workload, calculate: - What node count will sustain your peak QPS with < 65% CPU? - What is your storage growth rate over 3 years? - Are you in a single region or multi-region configuration (the price difference is substantial)? - Is the consistency guarantee actually load-bearing for your application, or could Cloud SQL with a read replica serve equally well?

The last question is the most important. Spanner is not automatically better than a well-tuned Cloud SQL PostgreSQL instance for workloads that are regional, write-moderate, and can tolerate a small replication lag window. Spanner earns its cost when global external consistency and near-unlimited horizontal scalability are genuine requirements, not aspirational architecture.

21.6 Query Execution and the Spanner Query Optimizer

Spanner's query optimizer behaves similarly enough to PostgreSQL's planner that experienced DBAs will feel oriented quickly — but there are meaningful differences in how to interpret and influence query plans.

Reading EXPLAIN output. In PostgreSQL, you use `EXPLAIN (ANALYZE, BUFFERS)`. In Spanner, query plan information is surfaced through the Cloud Console's query execution plan viewer, or programmatically via the `EXPLAIN` statement in the GoogleSQL dialect:

```
-- View query execution plan in Spanner
-- Run from Cloud Console query editor or via spanner-cli

-- Force an index and verify it in the plan
EXPLAIN
SELECT o.OrderId, o.OrderDate, o.TotalAmount
```

```
FROM Orders@{FORCE_INDEX=IX_Orders_CustomerId_Storing} o
WHERE o.CustomerId = 'cust-abc-123'
ORDER BY o.OrderDate DESC;
```

The plan output shows distributed union nodes (how the query is parallelized across splits), index scans vs. table scans, and cross-apply operations (Spanner's term for nested loop joins against interleaved data). A plan that shows a **Table Scan** on a large table without an index filter is your signal to create a covering index.

Statistics freshness. Spanner automatically updates table and column statistics. Unlike PostgreSQL where `AUTOVACUUM` and `ANALYZE` frequency are configurable (and sometimes fail to run on high-churn tables), Spanner's statistics are managed by the engine. You cannot manually trigger `ANALYZE`. If you observe a sudden query plan change after a significant data load, the statistics update cycle is the likely cause. In practice, Spanner's optimizer is less sensitive to statistics staleness than PostgreSQL's planner, because its cost model is anchored to distributed execution rather than local I/O.

Cross-region query routing. In multi-region Spanner, read-write transactions are always routed to the leader region (the region where the Paxos leader for the relevant key ranges resides). Read-only transactions and stale reads can be served by any replica. If your application runs in a region that is not the Spanner leader region, read-write transactions will incur additional round-trip latency to reach the leader. You can configure the leader region explicitly and should co-locate your primary application servers with the Spanner leader region for latency-sensitive write paths.

```
# View the current leader region for the instance
gcloud spanner instances describe prod-spanner \
  --format="value(config)"

# For multi-region configs, leader region is determined by the config type
# nam6 config defaults to Iowa (us-central1) as leader
# To influence leader placement, use the directed reads API for reads
# or accept that writes will always route to the leader
```

Directed reads. For read-only workloads where you want reads served from a specific replica (to minimize latency for users in a specific geographic region), Spanner supports directed reads:

```
from google.cloud.spanner_v1.types import DirectedReadOptions

# Route reads to the europe-west1 replica for European users
directed_read_options = DirectedReadOptions(
    include_replicas=DirectedReadOptions.IncludeReplicas(
        replica_selections=[
            DirectedReadOptions.ReplicaSelection(
                location="europe-west1",
                type=DirectedReadOptions.ReplicaSelection.Type.READ_ONLY
            )
        ],
        auto_failover_disabled=False
    )
)

with database.snapshot(directed_read_options=directed_read_options) as snapshot:
    results = snapshot.execute_sql(
        "SELECT OrderId, Status FROM Orders WHERE CustomerId = @cid",
        params={"cid": customer_id},
        param_types={"cid": param_types.STRING}
    )
```

This pattern allows global applications to serve reads locally while writes still go through the leader — the standard pattern for globally distributed OLTP systems built on Spanner.

Key Takeaways

- **TrueTime makes external consistency a database primitive, not an application concern.** Spanner’s serializable, globally consistent transactions eliminate the need for read/write routing logic, replication lag monitoring, and manual synchronous commit configuration — at the cost of write latency that reflects the physics of cross-region round trips.
- **Primary key design determines whether Spanner scales horizontally or collapses onto a single hot tablet.** Never use monotonically increasing integers as primary keys. Use UUIDs, bit-reversed sequences, or hash-prefixed natural keys to distribute writes evenly across Spanner’s tablet sharding.
- **Interleaving and covering indexes with STORING are the two most impactful schema design choices for query performance.** Interleaving physically co-locates parent and child rows, eliminating distributed join overhead. STORING columns in secondary indexes eliminates back-joins to the base table. Both decisions must be made at schema design time, not retrofitted under load.
- **Always use read-only transactions and stale reads for non-writing workloads.** Read-only transactions acquire no locks, require no retries, and can be served by any replica. Stale reads reduce latency further by eliminating cross-region coordination. Using read-write transactions for reads is the most common performance anti-pattern when migrating from traditional RDBMS.
- **Spanner’s operational cost is justified when global external consistency and horizontal write scalability are genuine requirements, not aspirational ones.** For regional workloads, write-moderate applications, or cases where small replication lag is acceptable, Cloud SQL PostgreSQL is simpler and substantially less expensive. Spanner earns its cost when the engineering overhead of building equivalent consistency guarantees on top of traditional distributed replication exceeds its node-hour pricing.

Chapter 22: BigQuery for Analytics: When to Push Workloads Out

Google BigQuery sits in a different category from every other database discussed in this book. It is not a replacement for Cloud SQL or AlloyDB. It is not a competitor to Aurora or Azure SQL. BigQuery is a serverless, columnar analytical engine designed to scan terabytes in seconds and return aggregated results to dashboards, data science notebooks, and reporting pipelines — workloads that would bring any transactional database to its knees if you left them there long enough. This chapter explains the architectural reasons why certain query patterns belong in BigQuery rather than your OLTP engine, how to move data from RDS, Cloud SQL, and AlloyDB into BigQuery efficiently, how to configure BigQuery for production use including IAM, partitioning, and cost control, and how to build the operational habits that keep your analytics layer running without surprise bills or query timeouts at 2 AM.

22.1 Understanding Why OLTP Engines Break Under Analytics Load

Before you can decide what to push to BigQuery, you need to understand precisely why transactional databases fail at analytical workloads — not in a vague hand-waving sense, but mechanically.

PostgreSQL, SQL Server, and their managed equivalents on AWS and Azure store data in row-oriented heap files. Every row's columns are stored contiguously on disk. This design is optimal when you fetch a complete record: a customer order, a user profile, a payment transaction. The buffer pool fills with pages containing complete rows, and repeated transactional reads and writes work efficiently.

Analytics queries do the opposite. A query like `SELECT region, SUM(revenue) FROM orders WHERE order_date >= '2023-01-01' GROUP BY region` needs only two columns from a table that might have forty. On a row store, the database must read all forty columns from every qualifying row page into memory just to extract two of them. If your `orders` table is 800 GB, you are reading 800 GB to return a handful of rows. Buffer pool pressure spikes. I/O throughput maxes out. Other sessions wait for locks on shared buffer pages. Your OLTP application starts throwing connection timeouts.

RDS PostgreSQL on an `db.r6g.4xlarge` instance gives you 128 GB of RAM and a finite, instance-class-capped EBS throughput ceiling — check the current instance specs for the exact number. A full table scan of an 800 GB table takes real, measurable minutes in pure I/O time even on a

well-provisioned volume, ignoring CPU for hashing and aggregation. Now put that query in a BI tool that twenty analysts hit simultaneously. You have created a resource contention crisis on the same database your application depends on for writes.

Read replicas help but do not solve this. A read replica on RDS or Aurora still uses a row-oriented storage engine. You have moved the scan off the primary, but you still saturate the replica's I/O and memory. Aurora's parallel query feature can push some aggregation down to the storage layer, but it targets specific single-table scan patterns, not complex multi-join analytical workloads.

BigQuery's columnar storage changes the math entirely. For that same two-column query on 800 GB of row data, BigQuery reads only the bytes belonging to **region** and **revenue** columns — a small fraction of the total data volume, with the exact percentage depending on column widths and compression ratios. It distributes that read across thousands of storage shards in parallel. Slot-based compute (more on this shortly) applies CPU in parallel across the result shards. The practical result is that a query taking minutes on a saturated OLTP replica typically completes in single-digit seconds in BigQuery — benchmark your own query mix rather than assuming a fixed multiplier.

The workloads that belong in BigQuery share a recognizable profile:

- Aggregations over hundreds of millions to billions of rows
- Historical trend analysis spanning months or years of data
- Ad hoc exploration queries from analysts who don't know in advance which columns they'll filter on
- JOIN operations between a large fact table and multiple dimension tables
- Machine learning feature engineering pipelines
- Any query where full or near-full table scans are unavoidable

The workloads that do *not* belong in BigQuery are equally important to recognize:

- Point lookups by primary key: `SELECT * FROM users WHERE user_id = 12345`
- Low-latency transactional writes
- Queries requiring sub-100ms response for user-facing applications
- Workloads requiring row-level locking or transactional consistency

BigQuery has a minimum query latency of roughly 400–800ms even for tiny result sets, due to job scheduling overhead. It is architecturally the wrong tool for anything a user is waiting on in real time.

22.2 BigQuery Architecture: What a DBA Needs to Know

You arrive at BigQuery with mental models built around instances, storage engines, buffer pools, and query plans. BigQuery discards most of those concepts. Building accurate new mental models before you write production SQL saves you from expensive mistakes.

Storage and compute separation

BigQuery stores data in Google's Capacitor columnar format across a distributed file system called Colossus. Colossus is independent of compute. When you run a query, BigQuery allocates compute units called *slots* — each slot is a combination of CPU and memory — dynamically from a shared

pool. When the query completes, those slots return to the pool. You pay for storage separately from compute, and compute is measured in slot-hours, not instance uptime.

This means there are no instances to size. There are no storage volumes to provision. There are no connection pools to manage. For a DBA coming from RDS or Cloud SQL, this feels disorienting. The operational handles you're used to pulling simply do not exist.

Tables are not heap files

BigQuery tables have no visible physical layout you can control the way you would with `FILLFACTOR` in PostgreSQL or filegroup placement in SQL Server. What you *can* control — and must, for cost and performance — are partitioning and clustering.

Partitioning in BigQuery means dividing a table into segments by a date/timestamp column or an integer range, or using ingestion time. BigQuery physically separates partitions into distinct storage segments. When a query includes a filter on the partition column, BigQuery reads only qualifying partitions — called partition pruning. A table with three years of daily data has roughly 1,095 partitions. A query filtering to one month reads 30/1,095 of the data. Bytes billed drops by a factor of 36.

Clustering goes one level deeper. Within each partition, BigQuery sorts data by up to four clustering columns and stores it in sorted blocks with block-level metadata. Queries filtering on clustering columns skip non-qualifying blocks. Clustering is the BigQuery equivalent of a clustered index scan range, but it is approximate, not exact — the storage layer reorganizes data asynchronously as you insert, so clustering benefits improve over time rather than being guaranteed immediately.

Datasets, projects, and IAM

The organizational hierarchy is: Google Cloud Project → Dataset → Table. A dataset is roughly analogous to a schema in PostgreSQL. IAM permissions apply at the project, dataset, or table level. There is no `pg_hba.conf` equivalent and no database-level user management. Access control is entirely IAM-based — `roles/bigquery.dataViewer` grants `SELECT` on a dataset, `roles/bigquery.dataEditor` adds `INSERT/UPDATE/DELETE`, `roles/bigquery.admin` grants full control.

Pricing model — the number that surprises DBAs most

BigQuery has two pricing models. On-demand pricing charges \$6.25 per TB of bytes processed by queries (as of current GCP pricing — verify at time of implementation). Flat-rate pricing (now called BigQuery editions with capacity commitments) reserves a fixed number of slots for a flat monthly cost. For organizations running many concurrent heavy queries, flat-rate is significantly cheaper. For organizations running occasional analytical workloads, on-demand is fine.

The operational implication: in on-demand pricing, a runaway query that scans a 10 TB unpartitioned table costs \$62.50 per execution. Run by twenty analysts simultaneously, that is \$1,250 in query fees in a few minutes. Partition pruning, clustering, and query governance are not optional. They are cost controls.

22.3 Loading Data from RDS, Cloud SQL, and AlloyDB into BigQuery

There are four primary patterns for getting data from your OLTP engines into BigQuery. The right choice depends on your latency requirements, data volume, and operational complexity tolerance.

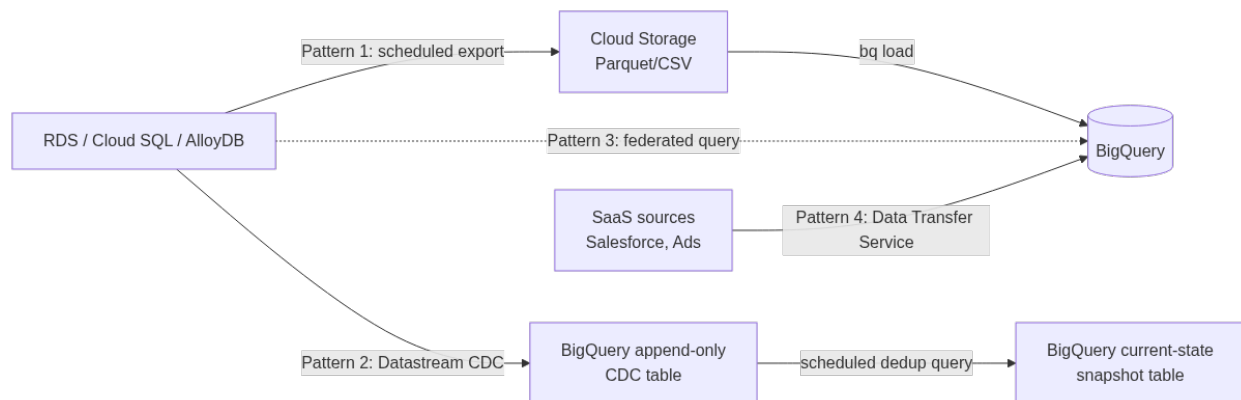


Figure 13: Four paths from OLTP engines into BigQuery, ordered by increasing freshness and operational complexity — batch export is simplest, CDC via Datastream is closest to real-time but requires a snapshot layer, and federated queries avoid data movement at the cost of query latency.

Pattern 1: Batch export via Cloud Storage

The simplest and most reliable pattern for many shops: export data from your OLTP database on a schedule (hourly, every 15 minutes, or nightly), land it in Cloud Storage as Parquet or CSV, and load it into BigQuery via a load job or external table.

From Cloud SQL for PostgreSQL, you can export directly to GCS:

```
# Export a table from Cloud SQL to GCS as CSV
gcloud sql export csv my-production-instance \
  gs://my-analytics-bucket/exports/orders/orders_$(date +%Y%m%d_%H%M).csv \
  --database=production \
  --query="SELECT * FROM orders WHERE updated_at >= NOW() - INTERVAL '1 hour'" \
  --offload

# Load the exported file into BigQuery
bq load \
  --source_format=CSV \
  --skip_leading_rows=1 \
  --autodetect \
  --time_partitioning_field=order_date \
  --clustering_fields=region,customer_segment \
  analytics_dataset.orders \
  "gs://my-analytics-bucket/exports/orders/orders_$(date +%Y%m%d_%H%M).csv"
```

The `--offload` flag on the Cloud SQL export is important — it routes the export through a read replica offload path rather than consuming CPU on the primary instance.

For RDS PostgreSQL, there is no direct export-to-GCS command. You use `pg_dump` or `COPY` to a local file, upload to GCS, then load into BigQuery:

```
# Export from RDS PostgreSQL using COPY to S3 first, then transfer to GCS
# Step 1: Export from RDS to S3 using aws_s3 extension
psql -h myinstance.us-east-1.rds.amazonaws.com -U admin -d production << 'EOF'
SELECT * FROM aws_s3.query_export_to_s3(
```

```

'SELECT order_id, customer_id, region, revenue, order_date
FROM orders
WHERE order_date >= CURRENT_DATE - INTERVAL '1 day'',
aws_commons.create_s3_uri(
  'my-rds-export-bucket',
  'exports/orders/daily_export.csv',
  'us-east-1'
),
options := 'format csv, header true'
);
EOF

# Step 2: Transfer from S3 to GCS using Storage Transfer Service
gcloud transfer jobs create \
  --source-agent-pool=s3-source \
  --source-creds-file=aws-creds.json \
  s3://my-rds-export-bucket/exports/orders/ \
  gs://my-analytics-bucket/exports/orders/

# Step 3: Load into BigQuery
bq load \
  --source_format=CSV \
  --skip_leading_rows=1 \
  --time_partitioning_field=order_date \
  analytics_dataset.orders \
  "gs://my-analytics-bucket/exports/orders/daily_export.csv"

```

Pattern 2: Datastream for CDC-based replication

Google Datastream is a managed change data capture service that reads from MySQL or PostgreSQL binlog/WAL and streams changes into BigQuery with typical latency of 30–120 seconds. For RDS PostgreSQL or Cloud SQL, this is the standard near-real-time path.

```

# Create a Datastream connection profile for RDS PostgreSQL
gcloud datastream connection-profiles create rds-postgres-source \
  --location=us-central1 \
  --type=postgresql \
  --postgresql-hostname=myinstance.us-east-1.rds.amazonaws.com \
  --postgresql-port=5432 \
  --postgresql-username=datastream_user \
  --postgresql-password=REDACTED \
  --postgresql-database=production \
  --display-name="RDS Production PostgreSQL"

# Create the BigQuery destination profile
gcloud datastream connection-profiles create bigquery-dest \
  --location=us-central1 \
  --type=bigquery \
  --display-name="Analytics BigQuery"

# Create the stream
gcloud datastream streams create orders-cdc-stream \
  --location=us-central1 \
  --display-name="Orders CDC Stream" \
  --source=rds-postgres-source \
  --postgresql-source-config-include-objects=public.orders,public.customers \
  --destination=bigquery-dest \
  --bigquery-destination-config-dataset-id=analytics_dataset \
  --backfill-all

```

Before creating the stream, your RDS instance needs logical replication enabled. Set the `rds.logical_replication` parameter to 1 in your RDS parameter group, which sets `wal_level=logical`. The Datastream user needs `REPLICATION` privilege and `SELECT` on the tables being replicated.

One production gotcha with Datastream and BigQuery: Datastream writes CDC events into Big-

Query as append-only rows with a `_metadata_row_is_deleted` column. The destination table accumulates all versions of each row. To get the current state of your data, you query with a deduplication pattern:

```
-- Query current state of orders from CDC-replicated table
-- This pattern is essential when using Datastream into BigQuery
WITH ranked_orders AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY order_id
      ORDER BY _metadata_timestamp DESC
    ) AS row_num
  FROM `my-project.analytics_dataset.orders`
  WHERE _metadata_row_is_deleted IS FALSE
     OR _metadata_row_is_deleted IS NULL
)
SELECT
  order_id,
  customer_id,
  region,
  revenue,
  order_date,
  status
FROM ranked_orders
WHERE row_num = 1;
```

In practice, you materialize this into a separate `orders_current` table using a scheduled query that runs every 15–30 minutes, rather than running the deduplication logic on every analytical query.

Pattern 3: AlloyDB Omni and federated queries

AlloyDB supports BigQuery federated queries via the AlloyDB connector. This allows BigQuery to query AlloyDB tables directly without a separate ETL step. The latency is higher than native BigQuery storage (AlloyDB still does row-oriented reads), but it removes the data movement entirely for low-frequency analytical queries.

Pattern 4: BigQuery Data Transfer Service for SaaS sources

If your data originates in SaaS platforms — Salesforce, Google Ads, YouTube — BigQuery Data Transfer Service ingests it natively. This is outside the DBA’s typical concern but worth noting when the analytics team asks why their CRM data isn’t in the same place as the orders data.

22.4 Configuring BigQuery for Production Use

Partitioning and clustering strategy

Apply partitioning and clustering at table creation time. Retrofitting them requires recreating the table.

```
-- Create a production-ready partitioned and clustered orders table
CREATE TABLE `my-project.analytics_dataset.orders`
(
  order_id      INT64 NOT NULL,
  customer_id   INT64 NOT NULL,
  region        STRING NOT NULL,
  customer_segment STRING,
  revenue       NUMERIC(18, 2),
  order_date    DATE NOT NULL,
  status        STRING,
```

```

    created_at      TIMESTAMP,
    updated_at      TIMESTAMP
)
PARTITION BY order_date
CLUSTER BY region, customer_segment
OPTIONS (
  partition_expiration_days = 1825, -- 5 years
  require_partition_filter = TRUE,   -- Reject queries that don't filter on order_date
  description = "Orders fact table - partitioned by order_date, clustered by region and segment"
);

```

The `require_partition_filter = TRUE` option is one of the most important production controls available. It forces every query against this table to include a filter on `order_date`, preventing full table scans. An analyst who forgets a `WHERE` clause gets an error rather than a \$62 surprise on the billing report. You will get pushback from analysts who want to run aggregate queries across all time. The answer is to create a separate summary table for those use cases, not to remove the partition filter from the fact table.

IAM and access control

```

# Grant dataset-level read access to the analytics team service account
bq add-iam-policy-binding \
  --member=serviceAccount:analytics-readers@my-project.iam.gserviceaccount.com \
  --role=roles/bigquery.dataViewer \
  my-project:analytics_dataset

# Grant job execution permission at project level (required to run queries)
gcloud projects add-iam-policy-binding my-project \
  --member=serviceAccount:analytics-readers@my-project.iam.gserviceaccount.com \
  --role=roles/bigquery.jobUser

# Grant Looker Studio service account access to specific tables only
bq add-iam-policy-binding \
  --member=serviceAccount:looker-studio@my-project.iam.gserviceaccount.com \
  --role=roles/bigquery.dataViewer \
  --table \
  my-project:analytics_dataset.orders_summary

```

Note the separation between `roles/bigquery.dataViewer` (read data) and `roles/bigquery.jobUser` (execute queries). Both are required. A common mistake is granting `dataViewer` but forgetting `jobUser`, then spending an hour debugging why a service account with read access can't run queries.

Cost controls with per-user query quotas

```

# Set a per-user daily query quota of 1 TB via the BigQuery API
# This prevents any single analyst from burning through budget
cat > quota-policy.json << 'EOF'
{
  "name": "projects/my-project/locations/us/quotas/QUERY_USAGE_PER_USER_PER_DAY",
  "preferredValue": "1099511627776"
}
EOF

# Apply via gcloud (requires quota admin permissions)
gcloud alpha bigquery quota update \
  --project=my-project \
  --location=us \
  --quota-id=QUERY_USAGE_PER_USER_PER_DAY \
  --preferred-value=1099511627776

```

The preferred value is in bytes. 1,099,511,627,776 bytes = 1 TB. Analysts hitting this limit receive an error rather than a bill. Set this on day one before handing BigQuery access to anyone.

Reservations and slot commitments for predictable costs

If your organization runs more than roughly 50 heavy analytical queries per day, on-demand pricing becomes expensive relative to BigQuery editions (slot reservations):

```
# Purchase a 100-slot baseline commitment (1-year)
gcloud bigquery reservations capacity-commitments create \
  --location=us-central1 \
  --slot-count=100 \
  --plan=ANNUAL \
  --project=my-project

# Create a reservation
gcloud bigquery reservations create analytics-baseline \
  --location=us-central1 \
  --slot-count=100 \
  --project=my-project

# Assign the analytics project to this reservation
gcloud bigquery reservations assignments create \
  --reservation=analytics-baseline \
  --location=us-central1 \
  --assignee-type=PROJECT \
  --assignee-id=my-project \
  --job-type=QUERY \
  --project=my-project
```

With 100 committed slots, all queries in the assigned project draw from this reservation rather than the on-demand pool. Costs are predictable. The downside: 100 slots can become a bottleneck if five analysts simultaneously run multi-TB scans. Monitor slot utilization in Cloud Monitoring and adjust commitments quarterly.

22.5 Query Patterns, Optimization, and What Surprises Production DBAs

Reading INFORMATION_SCHEMA for query performance

BigQuery's equivalent of `pg_stat_statements` or SQL Server DMVs is the `INFORMATION_SCHEMA.JOBS` view. You query it with SQL like any other table:

```
-- Find the top 20 most expensive queries in the past 7 days
-- This is your first stop when investigating cost overruns
SELECT
  user_email,
  job_id,
  creation_time,
  ROUND(total_bytes_processed / POW(1024, 4), 3) AS tb_processed,
  ROUND(total_bytes_processed / POW(1024, 4) * 6.25, 2) AS estimated_cost_usd,
  ROUND(total_slot_ms / 1000.0, 1) AS total_slot_seconds,
  SUBSTR(query, 1, 200) AS query_preview,
  statement_type,
  state,
  error_result
FROM `region-us`.INFORMATION_SCHEMA.JOBS
WHERE
  creation_time >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 7 DAY)
  AND job_type = 'QUERY'
  AND state = 'DONE'
ORDER BY total_bytes_processed DESC
LIMIT 20;
```

Run this query weekly. You will find analysts running full table scans immediately. The pattern is always the same: someone copied a SQL Server query from a reporting tool that works fine on a

10 GB SQL Server table, pointed it at a 2 TB BigQuery table, and the partition filter was missing.

Understanding query execution plans

BigQuery's EXPLAIN equivalent is the query execution graph in the Cloud Console, and the INFORMATION_SCHEMA.JOBS view's query_plan column (available as JSON). For automated analysis, the Python client is more practical:

```
from google.cloud import bigquery
import json

client = bigquery.Client(project="my-project")

# Retrieve the execution plan for a completed job
job = client.get_job("my-project.us.bq_job_id_here", location="us")

if hasattr(job, 'query_plan'):
    for step in job.query_plan:
        print(f"Stage: {step.name}")
        print(f"  Slot ms: {step.slot_ms}")
        print(f"  Records read: {step.records_read}")
        print(f"  Records written: {step.records_written}")
        print(f"  Shuffle output bytes: {step.shuffle_output_bytes}")
        for substep in step.steps:
            print(f"    {substep.kind}: {' '.join(substep.substeps)}")
        print()

# Also check bytes billed vs bytes processed
# Bytes billed = max(bytes_processed, 10 MB) - BigQuery charges a 10 MB minimum
print(f"Bytes processed: {job.total_bytes_processed:,}")
print(f"Bytes billed: {job.total_bytes_billed:,}")
print(f"Estimated cost: ${job.total_bytes_billed / 1e12 * 6.25:.4f}")
```

The shuffle output bytes metric reveals data skew. If one stage writes dramatically more bytes than others, your aggregation key has a skew problem — a small number of values (NULL, 'UNKNOWN', a single country code) dominate a GROUP BY, causing one slot to process far more data than others. The fix is the same as in Spark or Hive: filter skewed values separately, or use APPROX_COUNT_DISTINCT instead of COUNT(DISTINCT ...) for non-exact counts.

Three query patterns that surprise DBAs from relational backgrounds

First, COUNT(DISTINCT column) on a billion-row table is extremely expensive in BigQuery. It requires a global sort and deduplication. Use APPROX_COUNT_DISTINCT instead if the business requirement allows for a ~1% error margin. Cardinality estimates for dashboard widgets virtually always permit this.

Second, SELECT * is not merely a bad practice in BigQuery — it is a direct cost multiplier. Because bytes processed equals dollars spent, selecting forty columns when you need two costs twenty times more. Enforce column selection in your analytics team's coding standards and review it during query approval for scheduled reports.

Third, JOIN order matters differently than it does in PostgreSQL. BigQuery's query optimizer handles many cases automatically, but when joining a large fact table to multiple dimension tables, always put the largest table first (leftmost) in the FROM clause and join smaller dimension tables to it. BigQuery broadcasts small tables to all slots for hash joins; if BigQuery misidentifies the small table, the broadcast goes the wrong direction and performance collapses.

Materialized views for recurring aggregations

BigQuery supports materialized views that auto-refresh incrementally. For recurring dashboard

queries that aggregate the same data repeatedly, materialized views eliminate redundant full scans:

```
-- Materialized view for daily regional revenue summary
-- Refreshes incrementally as new data arrives in the base table
CREATE MATERIALIZED VIEW `my-project.analytics_dataset.orders_daily_regional_summary`
PARTITION BY report_date
CLUSTER BY region
OPTIONS (
  enable_refresh = TRUE,
  refresh_interval_minutes = 60
)
AS
SELECT
  order_date                AS report_date,
  region,
  customer_segment,
  COUNT(*)                  AS order_count,
  SUM(revenue)              AS total_revenue,
  AVG(revenue)              AS avg_order_value,
  COUNT(DISTINCT customer_id) AS unique_customers
FROM `my-project.analytics_dataset.orders`
WHERE order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 90 DAY)
GROUP BY
  order_date,
  region,
  customer_segment;
```

BigQuery's query optimizer will automatically rewrite queries against the base `orders` table to read from this materialized view when it determines the view can satisfy the query — a feature called *smart tuning*. Analysts do not need to change their queries. The rewrite happens transparently. You can verify whether a query used a materialized view by examining the `referenced_tables` field in `INFORMATION_SCHEMA.JOBS`.

One limitation to understand: BigQuery materialized views support only a restricted SQL dialect. Subqueries, window functions, and UNION operations in the view definition are not supported. The view must be a simple SELECT with aggregations, GROUP BY, and filters on the base table. If your summarization logic requires window functions, materialize through a scheduled query instead.

Scheduled queries for snapshot tables

For the deduplication pattern from Datastream CDC, and for complex aggregations that exceed materialized view SQL restrictions, scheduled queries are the production tool:

```
-- Scheduled query: run every 30 minutes
-- Overwrites the current-state snapshot table from CDC feed
DECLARE cutoff_ts TIMESTAMP DEFAULT TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 24 HOUR);

CREATE OR REPLACE TABLE `my-project.analytics_dataset.orders_current`
PARTITION BY DATE(updated_at)
CLUSTER BY region, status
AS
WITH ranked AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY order_id
      ORDER BY _metadata_timestamp DESC
    ) AS rn
  FROM `my-project.analytics_dataset.orders`
  WHERE _metadata_row_is_deleted IS DISTINCT FROM TRUE
)
SELECT
  order_id,
  customer_id,
  region,
  customer_segment,
```



```

revenue,
order_date,
status,
created_at,
updated_at
FROM ranked
WHERE rn = 1;

```

Schedule this in the BigQuery console under Scheduled Queries, or via the API. Set an alert on the scheduled query's failure notification — a missed snapshot at midnight means your morning dashboard is running against yesterday's data with no visible error to the consumer.

22.6 Monitoring, Alerting, and Operational Runbooks

Cloud Monitoring metrics for BigQuery

BigQuery emits metrics to Cloud Monitoring that you should baseline and alert on:

```

# Create an alerting policy for slot utilization exceeding 90%
# of reserved capacity - indicates queries queuing behind slot limit
gcloud alpha monitoring policies create \
  --policy-from-file-- << 'EOF'
{
  "displayName": "BigQuery Slot Utilization High",
  "conditions": [
    {
      "displayName": "Slot utilization above 90%",
      "conditionThreshold": {
        "filter": "resource.type=\"bigquery_project\" AND
↪ metric.type=\"bigquery.googleapis.com/slot/allocated_for_project\"",
        "comparison": "COMPARISON_GT",
        "thresholdValue": 90,
        "duration": "300s",
        "aggregations": [
          {
            "alignmentPeriod": "60s",
            "perSeriesAligner": "ALIGN_MEAN"
          }
        ]
      }
    ]
  ],
  "notificationChannels": [
    "projects/my-project/notificationChannels/YOUR_CHANNEL_ID"
  ],
  "alertStrategy": {
    "autoClose": "604800s"
  }
}
EOF

```

The key metrics to watch in production are:

- `bigquery.googleapis.com/slot/allocated_for_project` — how many slots are actively in use. Sustained near-capacity means queries are queuing.
- `bigquery.googleapis.com/storage/stored_bytes` — total bytes stored, broken down by table. Tracks growth and validates partition expiration is working.
- `bigquery.googleapis.com/job/num_in_flight` — number of jobs currently executing. Spikes here with high slot utilization indicate a query storm.

INFORMATION_SCHEMA queries for operational visibility

Run these as daily monitoring queries, either manually or via a scheduled query that writes results to a monitoring table:

```
-- Daily operational summary: jobs by user, errors, and cost
SELECT
  DATE(creation_time)                                AS query_date,
  user_email,
  COUNT(*)                                            AS total_jobs,
  COUNTIF(state = 'DONE' AND error_result IS NULL)  AS successful_jobs,
  COUNTIF(error_result IS NOT NULL)                 AS failed_jobs,
  ROUND(SUM(total_bytes_processed) / POW(1024, 4), 2) AS total_tb_processed,
  ROUND(SUM(total_bytes_processed) / POW(1024, 4) * 6.25, 2) AS est_cost_usd,
  ROUND(AVG(
    TIMESTAMP_DIFF(end_time, start_time, MILLISECOND)
  ) / 1000.0, 1)                                     AS avg_job_duration_seconds
FROM `region-us`.INFORMATION_SCHEMA.JOBS
WHERE
  creation_time >= TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 30 DAY)
  AND job_type = 'QUERY'
GROUP BY
  query_date,
  user_email
ORDER BY
  query_date DESC,
  est_cost_usd DESC;

-- Find tables with no partition filter enforcement
-- that are larger than 100 GB - these are cost exposure risks
SELECT
  t.table_catalog,
  t.table_schema,
  t.table_name,
  ROUND(ps.total_logical_bytes / POW(1024, 3), 1) AS size_gb,
  t.time_partitioning_field,
  t.require_partition_filter
FROM `my-project.analytics_dataset.INFORMATION_SCHEMA.TABLES` t
LEFT JOIN `my-project.analytics_dataset.INFORMATION_SCHEMA.TABLE_STORAGE` ps
  ON t.table_name = ps.table_name
WHERE
  t.table_type = 'BASE TABLE'
  AND (t.require_partition_filter IS FALSE OR t.require_partition_filter IS NULL)
  AND ps.total_logical_bytes > 100 * POW(1024, 3)
ORDER BY
  ps.total_logical_bytes DESC;
```

Review the second query monthly. Tables that grew past the 100 GB threshold without partition filter enforcement are candidates for a migration to a partitioned schema — create a new partitioned table, load historical data, swap it in, and enable the filter.

Runbook: responding to unexpected cost spikes

When your billing alert fires because BigQuery spend is $3\times$ normal for the day, this is the investigation sequence:

1. Query `INFORMATION_SCHEMA.JOBS` ordered by `total_bytes_processed` DESC for the billing period. Identify the job IDs and user emails responsible for the top 10% of bytes.
2. Check `referenced_tables` for those jobs. If they reference unpartitioned tables, that is the root cause — the query bypassed partition pruning.
3. Check query text for `SELECT *` patterns, missing `WHERE` clauses on partition columns, or cross joins.
4. If the expensive queries are from a service account rather than a human, trace the service account back to the scheduled job or application that triggered it. Check whether a Datastream

backfill was initiated (backfills read all historical data) or a scheduled query was manually rerun against a full time range.

5. Engage the quota controls from Section 22.4 if a human user is the source. Per-user daily limits prevent recurrence while you address the root query.
6. If the spike is from a legitimate new workload (a new analyst team onboarded, a new dashboard deployed), evaluate whether to move to a slot reservation to cap costs at a flat rate.

22.7 When Not to Use BigQuery: Recognizing the Antipatterns

BigQuery solves a real problem elegantly, and that creates a temptation to use it for problems it is not designed for. Recognizing when a team is misapplying BigQuery saves significant operational pain.

The high-frequency small query problem

Some teams, having discovered that BigQuery is “easy,” begin routing application queries to it — things like “fetch this user’s last 10 orders” or “look up whether this session ID exists.” BigQuery’s 400–800ms minimum job scheduling latency makes it unsuitable for any user-facing request. The billing impact is also punishing: the minimum 10 MB billed per query means 10,000 point lookups per day costs the same as scanning 100 GB, with no useful analytical benefit.

If you observe an application service account in `INFORMATION_SCHEMA.JOBS` running thousands of tiny queries daily (each scanning less than 1 MB), the team has routed transactional lookups to BigQuery. Redirect them to Firestore, Cloud Spanner, or back to Cloud SQL depending on their consistency and scale requirements.

The raw event store with no transformation layer

Another antipattern: streaming every raw application event into BigQuery and having analysts query the raw events table directly. The table accumulates petabytes, has no meaningful partitioning because events don’t have natural date-based query patterns, and every analyst query becomes a full table scan because the schema doesn’t match any analyst’s actual question.

The production pattern is: raw events land in BigQuery in a raw dataset (or in Cloud Storage), a transformation pipeline (dbt, Dataform, or Cloud Composer/Airflow) produces cleaned, partitioned, clustered derived tables in a `analytics` dataset, and analysts query only the derived layer. Analysts never have direct access to the raw events tables.

The CDC table without a snapshot layer

Described in Section 22.3: using Datastream without materializing a current-state snapshot forces every analytical query to run the `ROW_NUMBER()` deduplication logic. On a table with a billion CDC events, this deduplication itself becomes a multi-TB scan. Build the snapshot layer on day one, not after the first performance complaint.

22.8 Integrating BigQuery with Your Broader Data Architecture

BigQuery does not operate in isolation. In a mature cloud data architecture, it connects to several other systems that a production DBA needs to understand at the integration level.

dbt (data build tool)

dbt is the dominant transformation framework for BigQuery. It expresses data transformations as SQL `SELECT` statements organized into a DAG, compiles them to BigQuery DDL and DML, and handles dependency ordering, testing, and documentation. As a DBA, you will interact with dbt in three ways: reviewing the SQL it generates for correctness and cost efficiency, managing the service account permissions it uses, and understanding that dbt models map to BigQuery tables or views in your analytics dataset.

A dbt model that runs a full table scan on every refresh is just as expensive as a manual query doing the same thing. Review dbt model configurations for `partition_by` and `cluster_by` directives on large tables:

```
-- dbt model configuration block (placed at top of .sql file)
-- Ensures the dbt-generated table has proper partitioning and clustering
{{
  config(
    materialized='table',
    partition_by={
      "field": "order_date",
      "data_type": "date",
      "granularity": "day"
    },
    cluster_by=["region", "customer_segment"],
    require_partition_filter=true
  )
}}

SELECT
  order_id,
  customer_id,
  region,
  customer_segment,
  revenue,
  order_date,
  status
FROM {{ ref('stg_orders') }}
WHERE order_date IS NOT NULL
```

Looker and Looker Studio

Looker compiles LookML definitions into SQL and runs them against BigQuery. Looker Studio (formerly Data Studio) runs queries at dashboard load time. Both can generate expensive full table scans if the underlying LookML or report definition does not filter on partition columns. Review Looker explores and Looker Studio data source settings for the analysts' BI tools the same way you review application query plans — they are SQL generators, and they can generate bad SQL.

Vertex AI and BigQuery ML

BigQuery ML allows training machine learning models using SQL syntax directly inside BigQuery, without exporting data to a separate ML platform. For a DBA team supporting data scientists, this matters because training a BigQuery ML model is a BigQuery job — it consumes slots and processes bytes. A data scientist running `CREATE MODEL` on a 50 TB feature table without understanding the cost implications can generate a bill that rivals your entire monthly BigQuery spend in a single

job. Apply the same per-user quota controls to data scientists as to analysts, and document the expected cost of model training jobs before they run.

```
-- Example BigQuery ML model creation
-- Ensure data scientists run this in a sandbox project first
-- to validate byte consumption before running in production
CREATE OR REPLACE MODEL `my-project.ml_dataset.churn_classifier`
OPTIONS (
  model_type = 'logistic_reg',
  input_label_cols = ['churned'],
  l1_reg = 0.1,
  max_iterations = 20
) AS
SELECT
  customer_id,
  tenure_days,
  avg_monthly_spend,
  support_tickets_90d,
  last_login_days_ago,
  churned
FROM `my-project.analytics_dataset.customer_features`
WHERE feature_date = DATE_SUB(CURRENT_DATE(), INTERVAL 1 DAY);
```

Key Takeaways

- **BigQuery’s columnar architecture solves a structural problem that read replicas cannot:** row-oriented engines must read all columns from qualifying pages even for two-column aggregations, while BigQuery reads only the columns referenced, reducing bytes scanned by an order of magnitude for typical analytical queries and eliminating I/O contention on your OLTP engine.
 - **Partitioning with `require_partition_filter = TRUE` and per-user query quotas are non-negotiable production controls:** without them, a single analyst running a forgotten full table scan on a multi-TB table can generate a billing event that exceeds your daily compute budget; apply both before granting any human access to production datasets.
 - **The CDC pattern via Datastream requires a snapshot materialization layer:** append-only CDC tables accumulate all row versions, and running deduplication logic on every query compounds into multi-TB scans at scale; build the scheduled snapshot job on day one or the analytics layer becomes unusable within weeks of onboarding.
 - **INFORMATION_SCHEMA.JOBS is your primary operational instrument:** it replaces `pg_stat_statements`, DMVs, and slow query logs in the BigQuery world — regular queries against it reveal cost outliers, missing partition filters, and service accounts routing transactional lookups incorrectly before they escalate into billing incidents.
 - **BigQuery is one layer in an architecture, not a general-purpose database:** point lookups, low-latency application queries, and transactional writes belong in Cloud SQL, AlloyDB, RDS, or Cloud Spanner; when teams route those patterns to BigQuery, the combination of minimum billing thresholds and job scheduling latency makes it both more expensive and slower than the OLTP engine it was meant to offload.
-

Chapter 23: AlloyDB Omni: Running Google’s Database On-Premises

AlloyDB Omni brings Google’s columnar engine, adaptive query optimization, and PostgreSQL-compatible runtime to hardware you control — your on-premises servers, your colocation cage, or your edge deployment. For DBAs who have spent years tuning PostgreSQL instances but need the performance headroom of AlloyDB without migrating workloads into Google Cloud, Omni represents a genuinely different proposition: a managed-engine experience running in a container on your own infrastructure. This chapter covers deploying AlloyDB Omni from scratch, understanding what the columnar engine actually does under the hood, connecting Omni to Google Cloud services for observability and management, and building production patterns that keep the deployment maintainable over time. If you have worked with AlloyDB in Google Cloud (covered in Chapter 22), Omni will feel familiar in its SQL layer but distinctly different in its operational model.

23.1 What AlloyDB Omni Actually Is — and What It Is Not

Before deploying anything, you need a clear mental model of what Omni provides versus what you get from a self-compiled PostgreSQL binary or a full AlloyDB instance in Google Cloud.

The core architecture

AlloyDB Omni ships as a container image (`gcr.io/alloydb-omni/alloydb-omni`) that wraps a modified PostgreSQL 15 engine. The modifications include three components that differentiate it from vanilla Postgres:

1. **The columnar engine** — an in-memory columnar store that automatically populates based on query patterns. Google’s documentation calls this the “columnar cache.” It stores frequently accessed column data in a compressed, vectorized format alongside the standard row-based heap. You do not manage this explicitly; the engine decides what to populate based on workload.
2. **The adaptive query optimizer** — an overlay on top of the standard PostgreSQL planner that uses statistics and runtime feedback to choose between row and columnar execution paths. Unlike a traditional hint-based approach, this happens automatically per query.
3. **The AlloyDB AI extensions** — `google_ml_integration` and `pgvector`, pre-installed and configured for embedding generation calls that can reach out to Vertex AI endpoints if you configure a service account.

What Omni is not

Omni is not a fully managed service. You own the container runtime, the host OS, the storage subsystem, backup scheduling, and HA failover. Google does not patch it for you. You pull new container images when updates release, test them, and roll them out on your own schedule. This is the fundamental trade-off: you get the engine, not the platform.

Omni also does not replicate the AlloyDB storage layer from Google Cloud. In GCP, AlloyDB uses a distributed, disaggregated storage system that durably replicates across multiple zones within a region — consult current AlloyDB documentation for the exact replica topology. In Omni, you get standard PostgreSQL WAL-based storage on whatever block device you mount into the container. The columnar engine still works, but the durability model is your problem to design.

Comparing Omni to PostgreSQL

Feature	Vanilla PostgreSQL 15	AlloyDB Omni
Row storage		
Columnar engine		(automatic)
<code>pgvector</code> extension	Manual install	Pre-installed
<code>google_ml_integration</code>		
GCP Observability integration	Manual	Built-in agent
License	PostgreSQL License	Google ToS
Patching	Your binary	Pull new container
Max column store memory	N/A	Configurable, default 1 GB

The PostgreSQL ToS difference matters for production legal review. Omni is licensed under Google's terms, not the PostgreSQL License. Run that past your legal team before deploying in regulated environments.

23.2 Deploying AlloyDB Omni on Bare Metal and Virtual Machines

Omni runs on any container runtime that supports Docker or Podman. In practice, most production deployments use either Docker Engine on Linux or Kubernetes with the AlloyDB Omni Operator. This section covers both paths.

Prerequisites

Minimum host specifications for a production deployment:

- 8 vCPUs (16 recommended for OLAP workloads using the columnar engine)
- 32 GB RAM (the columnar engine uses a configurable slice of this)
- Local NVMe or SAN storage with at least 3,000 IOPS sustained
- Linux kernel 5.4 or newer (tested on RHEL 8/9, Ubuntu 20.04/22.04)
- Docker Engine 20.10+ or Podman 4.0+

Standalone container deployment

```
# Pull the AlloyDB Omni image
# You need to authenticate with Google Container Registry first
```



```

gcloud auth configure-docker gcr.io

docker pull gcr.io/alloydb-omni/alloydb-omni:latest

# Create persistent storage directories on the host
# Use a dedicated mount point backed by your fastest storage
mkdir -p /alloydb/data /alloydb/logs /alloydb/tmp

# Deploy the container with production settings
# POSTGRES_PASSWORD sets the initial superuser password
# ALLOYDB_ENABLE_COLUMNAR_ENGINE enables the in-memory columnar cache
docker run -d \
  --name alloydb-omni \
  --restart unless-stopped \
  -e POSTGRES_PASSWORD='YourSecureP@ssword123' \
  -e ALLOYDB_ENABLE_COLUMNAR_ENGINE=true \
  -e ALLOYDB_COLUMNAR_MEMORY_SIZE_MB=8192 \
  -p 5432:5432 \
  -v /alloydb/data:/var/lib/postgresql/data \
  -v /alloydb/logs:/var/log/postgresql \
  --shm-size=2g \
  --cpus=16 \
  --memory=64g \
  gcr.io/alloydb-omni/alloydb-omni:latest

# Verify the container started cleanly
docker logs alloydb-omni | tail -30

# Connect and confirm AlloyDB extensions are present
docker exec -it alloydb-omni psql -U postgres -c "\dx"

```

The `--shm-size` parameter is easy to forget and causes immediate performance problems. PostgreSQL uses shared memory for the buffer pool and inter-process communication. If you leave it at the Docker default of 64 MB, you will see errors in the PostgreSQL log about shared memory allocation failing when you try to set `shared_buffers` to a reasonable value.

Configuring the columnar engine

Once the container is running, you control the columnar engine through GUC parameters that AlloyDB Omni adds to the standard PostgreSQL parameter namespace:

```

-- Connect as postgres superuser and check columnar engine status
SELECT name, setting, unit, short_desc
FROM pg_settings
WHERE name LIKE 'alloydb%'
ORDER BY name;

-- Key parameters to understand:
-- alloydb.enable_columnar_engine           (on/off)
-- alloydb.columnar_memory_size_mb          (default: 1024)
-- alloydb.auto_columnar_scan_threshold     (rows, default: 10000)
-- alloydb.columnar_engine_mode             (auto, manual)

-- Check what the columnar engine has populated
-- This view shows which relations have columnar data
SELECT
  schemaname,
  tablename,
  pg_size_pretty(columnar_size_bytes) AS columnar_size,
  columnar_row_count,
  last_refresh_time
FROM alloydb_columnar_info
ORDER BY columnar_size_bytes DESC;

-- Force a specific table into the columnar cache explicitly
-- Useful during initial load before query patterns establish themselves
ALTER TABLE order_line_items SET (columnar = on);

```

```

-- Check actual query execution to see if columnar path was chosen
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
    product_id,
    SUM(quantity) AS total_qty,
    SUM(unit_price * quantity) AS revenue
FROM order_line_items
WHERE order_date >= '2024-01-01'
GROUP BY product_id;
-- Look for "Columnar Scan" nodes in the plan output
-- If you see "Seq Scan" instead, the table may not yet be in the columnar cache
-- or the row count is below the threshold

```

The `auto_columnar_scan_threshold` parameter is the one most DBAs misconfigure. If a table has fewer rows than this threshold, the planner will not consider the columnar path even if the table is marked for columnar storage. Set it too high and small-but-wide analytical tables miss out. Set it too low and you waste columnar memory on tables too small to benefit. A starting value of 50,000 rows works well for most mixed OLTP/OLAP environments.

Kubernetes deployment with the AlloyDB Omni Operator

For environments running Kubernetes on-premises (OpenShift, Rancher, vanilla kubeadm clusters), Google provides the AlloyDB Omni Operator, which adds CRDs for `DBCluster` resources:

```

# Install the AlloyDB Omni Operator using Helm
helm repo add alloydb-omni-operator \
    https://storage.googleapis.com/alloydb-omni-operator

helm repo update

helm install alloydb-omni-operator \
    alloydb-omni-operator/alloydb-omni-operator \
    --namespace alloydb-omni-system \
    --create-namespace \
    --set image.tag=1.2.0

# Verify the operator deployed successfully
kubectl get pods -n alloydb-omni-system
kubectl get crds | grep alloydb

# production-dbcluster.yaml
# AlloyDB Omni DBCluster manifest for Kubernetes
apiVersion: alloydbomni.dbadmin.goog/v1
kind: DBCluster
metadata:
  name: prod-analytics-cluster
  namespace: database
spec:
  databaseVersion: "15.5"
  primarySpec:
    adminUser:
      passwordRef:
        name: alloydb-admin-secret
        key: password
    resources:
      cpu: "16"
      memory: 64Gi
      disks:
        - name: DataDisk
          size: 2Ti
          storageClass: fast-nvme
  features:
    enableColumnarEngine: true
    columnarEngineSizeGB: 16
  parameters:
    work_mem: "64MB"

```

```

max_connections: "500"
checkpoint_completion_target: "0.9"
wal_level: "logical"
allowExternalIncomingTraffic: false

# Apply the manifest and watch the cluster come up
kubectl apply -f production-dbcluster.yaml
kubectl get dbcluster prod-analytics-cluster -n database -w

# Get the service endpoint once Ready
kubectl get svc -n database | grep prod-analytics

```

23.3 Connecting AlloyDB Omni to Google Cloud Services

Running Omni on-premises does not mean operating in isolation. Google designed Omni to optionally connect back to Google Cloud for observability, AI feature access, and fleet management through the AlloyDB Omni management API. This section covers those integrations and when to use them.

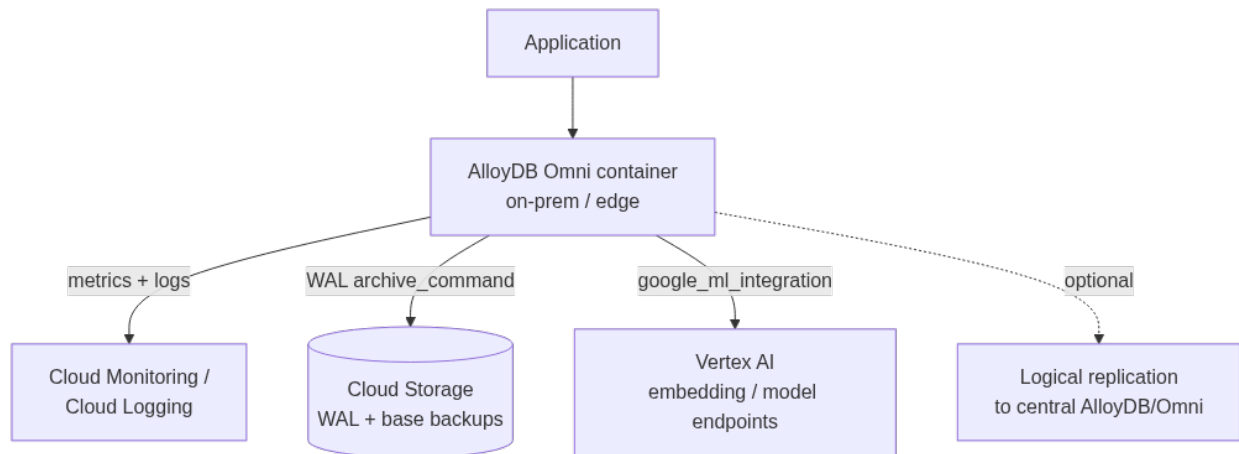


Figure 14: Omni owns its own storage and HA on-premises; the connections back to Google Cloud are opt-in integrations for observability, backup, and ML inference — each crosses the network boundary and should be evaluated against data-residency requirements before enabling.

Google Cloud Observability integration

Omni can ship logs and metrics to Cloud Logging and Cloud Monitoring through the built-in observability agent. This is the fastest way to get production-grade monitoring without building a Prometheus stack from scratch.

```

# Create a Google Cloud service account for the Omni observability agent
# Run this from a machine with gcloud access to your GCP project
gcloud iam service-accounts create alloydb-omni-agent \
  --project=your-project-id \
  --description="AlloyDB Omni observability and ML integration agent" \
  --display-name="AlloyDB Omni Agent"

# Grant the minimum required roles
gcloud projects add-iam-policy-binding your-project-id \
  --member="serviceAccount:alloydb-omni-agent@your-project-id.iam.gserviceaccount.com" \
  --role="roles/monitoring.metricWriter"

```

```

gcloud projects add-iam-policy-binding your-project-id \
  --member="serviceAccount:alloydb-omni-agent@your-project-id.iam.gserviceaccount.com" \
  --role="roles/logging.logWriter"

gcloud projects add-iam-policy-binding your-project-id \
  --member="serviceAccount:alloydb-omni-agent@your-project-id.iam.gserviceaccount.com" \
  --role="roles/aiplatform.user"

# Create and download a JSON key for the service account
# Store this securely - treat it like a root password
gcloud iam service-accounts keys create /etc/alloydb/gcp-credentials.json \
  --iam-account=alloydb-omni-agent@your-project-id.iam.gserviceaccount.com

chmod 600 /etc/alloydb/gcp-credentials.json

# Redeploy the container with GCP credentials mounted
# The GOOGLE_APPLICATION_CREDENTIALS env var activates the observability agent
docker stop alloydb-omni && docker rm alloydb-omni

docker run -d \
  --name alloydb-omni \
  --restart unless-stopped \
  -e POSTGRES_PASSWORD='YourSecureP@ssword123' \
  -e ALLOYDB_ENABLE_COLUMNAR_ENGINE=true \
  -e ALLOYDB_COLUMNAR_MEMORY_SIZE_MB=8192 \
  -e GOOGLE_APPLICATION_CREDENTIALS=/etc/alloydb/gcp-credentials.json \
  -e GOOGLE_CLOUD_PROJECT=your-project-id \
  -e ALLOYDB_OBSERVABILITY_ENABLED=true \
  -p 5432:5432 \
  -v /alloydb/data:/var/lib/postgresql/data \
  -v /alloydb/logs:/var/log/postgresql \
  -v /etc/alloydb:/etc/alloydb:ro \
  --shm-size=2g \
  --cpus=16 \
  --memory=64g \
  gcr.io/alloydb-omni/alloydb-omni:latest

# After 2-3 minutes, verify metrics appear in Cloud Monitoring
gcloud monitoring metrics list \
  --filter="metric.type=starts_with(\"alloydb.googleapis.com\")" \
  --project=your-project-id | head -30

```

Once connected, you can build Cloud Monitoring dashboards that show AlloyDB Omni metrics — active connections, cache hit ratios, columnar engine memory usage, transaction throughput — alongside metrics from your Cloud SQL and AlloyDB instances in GCP. This unified view is one of the practical reasons to wire Omni back to Google Cloud even when you prefer keeping data on-premises.

Vertex AI integration for embedding generation

The `google_ml_integration` extension in Omni can call Vertex AI to generate text embeddings from inside SQL queries. This is the same capability available in AlloyDB in GCP, now runnable from your on-premises database. The service account you created above needs the `roles/aiplatform.user` role for this to work.

```

-- Enable the ML integration extension
CREATE EXTENSION IF NOT EXISTS google_ml_integration;

-- Configure the Vertex AI endpoint
-- Replace the region and model with your Vertex AI configuration
SELECT google_ml.create_model(
  model_id => 'text-embedding-005',
  model_provider => 'google',
  model_type => 'text_embedding',
  model_endpoint =>

```

```
↪ 'https://us-central1-aiplatform.googleapis.com/v1/projects/your-project-id/locations/us-central1/publishers/g
↪ ,
model_qualified_name => 'publishers/google/models/text-embedding-005'
);

-- Test embedding generation
-- This call goes from your on-premises Omni instance to Vertex AI over HTTPS
SELECT google_ml.embedding(
  'text-embedding-005',
  'AlloyDB Omni columnar engine performance benchmark'
)::vector(768);

-- Practical use: generate embeddings for a product catalog
-- and store them for similarity search
ALTER TABLE product_catalog
ADD COLUMN IF NOT EXISTS description_embedding vector(768);

UPDATE product_catalog
SET description_embedding = google_ml.embedding(
  'text-embedding-005',
  product_description
)::vector(768)
WHERE description_embedding IS NULL
  AND product_description IS NOT NULL;

-- Create an IVFFlat index for approximate nearest neighbor search
CREATE INDEX idx_product_embeddings
ON product_catalog
USING ivfflat (description_embedding vector_cosine_ops)
WITH (lists = 100);
```

The network path here matters for latency and for compliance. Every embedding call in a SQL query exits your on-premises network and reaches Google's API endpoints. If your data classification policy prohibits row data leaving the facility, passing product descriptions to Vertex AI may trigger a compliance review. Know what data you are embedding before you build this into production query patterns.

23.4 Backup, HA, and the Operational Gap

This is where the honest conversation about Omni needs to happen. In AlloyDB in GCP, Google handles backup, point-in-time recovery, HA failover, and storage durability automatically. In Omni, none of that is included. You are running a PostgreSQL-compatible engine in a container, and backup and HA are entirely your design responsibility.

Backup strategy

For Omni deployments, you have three practical backup approaches:

1. **pg_basebackup** — same as vanilla PostgreSQL, works reliably
2. **WAL archiving to Cloud Storage** — gives you PITR with Google Cloud as the archive destination
3. **Volume snapshots** — snapshot the underlying block device at the host level

WAL archiving to Cloud Storage is the approach that best preserves the Google Cloud integration story:

```
# Create a Cloud Storage bucket for WAL archiving
gsutil mb -l us-central1 gs://your-company-alloydb-omni-wal
```

```

# Set lifecycle policy to expire old WAL segments after 30 days
cat > /tmp/wal-lifecycle.json << 'EOF'
{
  "lifecycle": {
    "rule": [
      {
        "action": {"type": "Delete"},
        "condition": {"age": 30}
      }
    ]
  }
}
EOF

gsutil lifecycle set /tmp/wal-lifecycle.json \
  gs://your-company-alloydb-omni-wal

# Inside the container, configure postgresql.conf for WAL archiving
# AlloyDB Omni respects all standard PostgreSQL archive parameters
docker exec -it alloydb-omni psql -U postgres << 'EOF'
ALTER SYSTEM SET wal_level = 'replica';
ALTER SYSTEM SET archive_mode = 'on';
ALTER SYSTEM SET archive_command =
  'gcloud storage cp %p gs://your-company-alloydb-omni-wal/%f';
ALTER SYSTEM SET archive_timeout = '300';
SELECT pg_reload_conf();
EOF

# Restart required for wal_level and archive_mode changes
docker restart alloydb-omni

# Take a base backup to Cloud Storage to anchor your PITR chain
docker exec alloydb-omni pg_basebackup \
  -U postgres \
  -D /tmp/basebackup \
  -Ft \
  -z \
  --checkpoint=fast \
  --wal-method=stream

# Upload the base backup
tar czf - /tmp/basebackup | \
  gcloud storage cp - \
  gs://your-company-alloydb-omni-wal/basebackups/$(date +%Y%m%d-%H%M%S).tar.gz

```

High availability with Patroni

The most production-proven HA approach for Omni is running Patroni to orchestrate multiple container instances. You already know how Patroni works with vanilla PostgreSQL; the configuration for Omni is identical at the Patroni level because Omni exposes the same replication API as PostgreSQL.

```

# /etc/patroni/patroni.yml for AlloyDB Omni primary node
scope: alloydb-omni-cluster
namespace: /db/
name: omni-node-01

restapi:
  listen: 0.0.0.0:8008
  connect_address: 10.0.1.10:8008

etcd3:
  hosts:
    - 10.0.1.20:2379
    - 10.0.1.21:2379
    - 10.0.1.22:2379

```

```
bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 1048576 # 1 MB - adjust for your RPO tolerance
  initdb:
    - encoding: UTF8
    - data-checksums

postgresql:
  listen: 0.0.0.0:5432
  connect_address: 10.0.1.10:5432
  data_dir: /alloydb/data
  bin_dir: /usr/lib/postgresql/15/bin
  parameters:
    max_connections: 500
    shared_buffers: 16GB
    work_mem: 64MB
    wal_level: replica
    max_wal_senders: 10
    max_replication_slots: 10
    hot_standby: on
    # AlloyDB Omni specific
    alloydb.enable_columnar_engine: on
    alloydb.columnar_memory_size_mb: 8192

tags:
  nofailover: false
  noloadbalance: false
  clonefrom: false
  nosync: false
```

One thing to verify before committing to this architecture: the columnar engine state is **not replicated**. The columnar cache on the primary gets rebuilt from scratch on a standby that promotes to primary. The first 15–30 minutes after a failover will have degraded analytical query performance until the new primary repopulates the columnar cache from query traffic. This is acceptable in most cases but needs to be communicated to application teams as expected post-failover behavior.

23.5 Performance Tuning the Columnar Engine

The columnar engine in AlloyDB Omni is the feature that most DBAs want to exploit first, and the place where the most time gets wasted chasing configuration parameters that do not move the needle as much as query and schema design.

What the columnar engine actually accelerates

The columnar engine provides meaningful speedup for queries that:

- Scan a large number of rows but access only a subset of columns
- Perform aggregations (SUM, AVG, COUNT) over wide tables
- Filter on low-cardinality columns where column statistics enable fast skipping

It provides minimal or no benefit for:

- Point lookups by primary key (these use the row store via index)
- Heavy write workloads (columnar cache is read-optimized)
- Queries on tables with fewer than 50,000 rows (below the default threshold)
- Joins where both sides are small and fit in `work_mem`

Diagnosing columnar engine utilization

```

-- Check if the columnar engine is being used for your analytical queries
-- Run this after the workload has been running for at least 30 minutes
-- to allow the columnar cache to warm up

EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
    c.customer_region,
    p.product_category,
    DATE_TRUNC('month', o.order_date) AS order_month,
    COUNT(*) AS order_count,
    SUM(ol.quantity * ol.unit_price) AS total_revenue,
    AVG(ol.unit_price) AS avg_unit_price
FROM orders o
JOIN order_line_items ol ON o.order_id = ol.order_id
JOIN customers c ON o.customer_id = c.customer_id
JOIN products p ON ol.product_id = p.product_id
WHERE o.order_date >= '2023-01-01'
      AND o.order_date < '2024-01-01'
GROUP BY 1, 2, 3
ORDER BY 1, 2, 3;

-- In the plan output, look for nodes like:
-- "Columnar Scan on order_line_items"
-- "Columnar Scan on orders"
-- If you see "Seq Scan" on large tables, the columnar cache has not
-- yet populated those tables or the threshold is preventing columnar access

-- Check current columnar cache contents
SELECT
    schemaname,
    tablename,
    pg_size_pretty(columnar_size_bytes) AS columnar_size,
    columnar_row_count,
    to_char(last_refresh_time, 'YYYY-MM-DD HH24:MI:SS') AS last_refresh
FROM alloydb_columnar_info
WHERE columnar_row_count > 0
ORDER BY columnar_size_bytes DESC;

-- Check columnar engine memory utilization
SELECT
    name,
    setting,
    unit
FROM pg_settings
WHERE name IN (
    'alloydb.columnar_memory_size_mb',
    'alloydb.enable_columnar_engine',
    'alloydb.auto_columnar_scan_threshold',
    'alloydb.columnar_engine_mode'
);

-- Force specific columns' tables into the columnar cache
-- when automatic population is too slow for your testing cadence
SELECT alloydb_columnar_refresh('public.order_line_items');
SELECT alloydb_columnar_refresh('public.orders');

```

Tuning memory allocation

The columnar engine competes with `shared_buffers` for RAM. The right split depends heavily on your workload ratio. A useful starting heuristic:

- **Primarily OLTP (< 20% analytical queries):** `shared_buffers` = 25% of RAM, columnar memory = 5–10% of RAM
- **Mixed OLTP/OLAP (20–60% analytical queries):** `shared_buffers` = 20% of RAM, columnar memory = 15–20% of RAM
- **Primarily OLAP (> 60% analytical queries):** `shared_buffers` = 15% of RAM, columnar memory = 10–15% of RAM

nar memory = 25–30% of RAM

These are starting points, not rules. Use the query timing before and after adjustments to validate whether moving memory between the two caches improves your actual workload.

```
-- On a 64 GB host with a mixed OLTP/OLAP workload
-- Adjust shared_buffers and columnar memory simultaneously
-- Note: shared_buffers requires a restart; columnar memory does not

ALTER SYSTEM SET shared_buffers = '12GB';
-- Restart container after this change

-- Columnar memory can be adjusted without restart
ALTER SYSTEM SET alloydb.columnar_memory_size_mb = 12288; -- 12 GB
SELECT pg_reload_conf();

-- After workload runs for 30+ minutes, measure hit rates
-- Columnar reads vs total analytical scans
SELECT
  SUM(columnar_rows_read) AS columnar_rows,
  SUM(heap_rows_read) AS heap_rows,
  ROUND(
    SUM(columnar_rows_read)::numeric /
    NULLIF(SUM(columnar_rows_read) + SUM(heap_rows_read), 0) * 100,
    2
  ) AS columnar_pct
FROM alloydb_columnar_query_stats;
```

Schema design patterns that maximize columnar benefit

The columnar engine rewards the same schema design choices that help any columnar store — wide, flat tables with fewer joins. If you have a heavily normalized schema designed for OLTP efficiency, you will get less columnar benefit than someone running a star schema. This does not mean you should denormalize your OLTP tables; it means you should consider whether Omni is the right home for your most analytical workloads, or whether you should push aggregated data into dedicated wide tables that the columnar engine can work through efficiently.

```
-- PATTERN: Create a materialized summary table optimized for columnar access
-- Refresh this periodically from the normalized OLTP tables

CREATE TABLE sales_summary_columnar (
  summary_date      DATE NOT NULL,
  customer_region   TEXT NOT NULL,
  product_category  TEXT NOT NULL,
  product_subcategory TEXT NOT NULL,
  sales_channel     TEXT NOT NULL,
  order_count       INTEGER NOT NULL,
  unit_count        INTEGER NOT NULL,
  gross_revenue     NUMERIC(18, 4) NOT NULL,
  discount_amount   NUMERIC(18, 4) NOT NULL,
  net_revenue       NUMERIC(18, 4) NOT NULL,
  return_count      INTEGER NOT NULL DEFAULT 0,
  return_amount     NUMERIC(18, 4) NOT NULL DEFAULT 0
)
-- Partition by month to improve columnar segment pruning
PARTITION BY RANGE (summary_date);

-- Create monthly partitions
CREATE TABLE sales_summary_columnar_2024_01
  PARTITION OF sales_summary_columnar
  FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

CREATE TABLE sales_summary_columnar_2024_02
  PARTITION OF sales_summary_columnar
  FOR VALUES FROM ('2024-02-01') TO ('2024-03-01');

-- Force the summary table into the columnar cache immediately
```

```
-- rather than waiting for query pattern detection
ALTER TABLE sales_summary_columnar SET (columnar = on);
```

Partitioning interacts with the columnar engine in an important way: the columnar cache tracks data at the partition level. A query that filters on the partition key can skip entire partitions before the columnar engine even evaluates individual rows. Combine partition pruning with columnar scanning and you get compounding performance improvements on date-range analytical queries — the most common pattern in reporting workloads.

23.6 Upgrading AlloyDB Omni Without Downtime

Omni updates release as new container image tags. You control when to apply them, but letting upgrades accumulate creates larger, riskier jumps. A rolling upgrade process for standalone container deployments and Kubernetes deployments looks different but shares the same validation structure.

Standalone container upgrade procedure

```
#!/bin/bash
# alloydb_omni_upgrade.sh
# Safe upgrade procedure for a standalone AlloyDB Omni deployment
# Prerequisites: Patroni or application-level connection retry in place

set -euo pipefail

NEW_IMAGE="gcr.io/alloydb-omni/alloydb-omni:15.6.0"
CONTAINER_NAME="alloydb-omni"
DATA_DIR="/alloydb/data"
LOG_DIR="/alloydb/logs"
CREDS_DIR="/etc/alloydb"
POSTGRES_PASSWORD="YourSecureP@ssword123"
GCP_PROJECT="your-project-id"
COLUMNAR_MB=8192

echo "=== AlloyDB Omni Upgrade ==="
echo "Target image: ${NEW_IMAGE}"
echo "Start time: $(date)"

# Step 1: Pull the new image while the current container is still running
echo "[1/7] Pulling new image..."
docker pull "${NEW_IMAGE}"

# Step 2: Verify current cluster health before touching anything
echo "[2/7] Checking cluster health..."
docker exec "${CONTAINER_NAME}" psql -U postgres -c "SELECT version();" || {
    echo "ERROR: Current instance not healthy. Aborting upgrade."
    exit 1
}

# Step 3: Capture current configuration for comparison after upgrade
echo "[3/7] Capturing current configuration snapshot..."
docker exec "${CONTAINER_NAME}" psql -U postgres -t -c \
    "SELECT name || '=' || setting FROM pg_settings
    WHERE source = 'configuration file'
    ORDER BY name;" > /tmp/pre_upgrade_config.txt

# Step 4: Checkpoint and graceful shutdown
echo "[4/7] Initiating checkpoint and graceful shutdown..."
docker exec "${CONTAINER_NAME}" psql -U postgres -c "CHECKPOINT;"
docker stop "${CONTAINER_NAME}"

# Step 5: Rename the old container (preserve it for rollback)
echo "[5/7] Archiving old container..."
```

```

docker rename "${CONTAINER_NAME}" "${CONTAINER_NAME}-prev-$(date +%Y%m%d%H%M%S)"

# Step 6: Start the new container on the same data directory
echo "[6/7] Starting new container version..."
docker run -d \
  --name "${CONTAINER_NAME}" \
  --restart unless-stopped \
  -e POSTGRES_PASSWORD="${POSTGRES_PASSWORD}" \
  -e ALLOYDB_ENABLE_COLUMNAR_ENGINE=true \
  -e ALLOYDB_COLUMNAR_MEMORY_SIZE_MB="${COLUMNAR_MB}" \
  -e GOOGLE_APPLICATION_CREDENTIALS=/etc/alloydb/gcp-credentials.json \
  -e GOOGLE_CLOUD_PROJECT="${GCP_PROJECT}" \
  -e ALLOYDB_OBSERVABILITY_ENABLED=true \
  -p 5432:5432 \
  -v "${DATA_DIR}:/var/lib/postgresql/data" \
  -v "${LOG_DIR}:/var/log/postgresql" \
  -v "${CREDS_DIR}:/etc/alloydb:ro" \
  --shm-size=2g \
  --cpus=16 \
  --memory=64g \
  "${NEW_IMAGE}"

# Step 7: Validate the new instance
echo "[7/7] Validating new instance..."
sleep 10 # Allow PostgreSQL to finish startup

docker exec "${CONTAINER_NAME}" psql -U postgres -c "SELECT version();" && \
docker exec "${CONTAINER_NAME}" psql -U postgres -c \
  "SELECT count(*) FROM pg_settings WHERE name LIKE 'alloydb%';" && \
echo "Upgrade complete. New version running." || {
  echo "ERROR: Post-upgrade validation failed."
  echo "To rollback, stop the new container and start the -prev container."
  exit 1
}

echo "Upgrade complete at: $(date)"
echo "Previous container preserved as: ${CONTAINER_NAME}-prev-*"
echo "Once satisfied with stability, remove old container with:"
echo "  docker rm $(docker ps -a --filter name=${CONTAINER_NAME}-prev -q)"

```

Kubernetes rolling upgrade

With the AlloyDB Omni Operator, upgrades are handled by patching the `DBCluster` resource with the new version. The Operator coordinates a rolling restart that respects the primary/standby topology:

```

# Check available versions
kubectl describe dbcluster prod-analytics-cluster -n database | grep -i version

# Patch the databaseVersion field to trigger a rolling upgrade
kubectl patch dbcluster prod-analytics-cluster \
  -n database \
  --type merge \
  -p '{"spec":{"databaseVersion":"15.6.0"}}'

# Watch the Operator coordinate the upgrade
kubectl get dbcluster prod-analytics-cluster -n database -w

# The Operator upgrades standbys first, then performs a controlled
# switchover before upgrading the original primary.
# This produces a brief connection interruption at switchover time -
# typically a few seconds if your application handles connection retries.

# Verify post-upgrade
kubectl exec -n database \
  $(kubectl get pods -n database -l dbcluster=prod-analytics-cluster \
    -o jsonpath='{.items[0].metadata.name}') \
  -- psql -U postgres -c "SELECT version();"

```

23.7 Edge Deployment Patterns

One use case that AlloyDB Omni enables and cloud-only AlloyDB cannot is edge deployment: running a full AlloyDB-compatible engine at a location with intermittent or constrained connectivity to Google Cloud. Factory floors, retail point-of-sale systems, ships, remote mining sites — anywhere that needs analytical query capability locally but cannot tolerate cloud round-trip latency or potential connectivity loss.

Design principles for edge deployments

Edge deployments require a different set of trade-offs than datacenter deployments:

- **Resource constraints are real.** An edge device may have 8 GB of RAM and a 4-core industrial CPU. The columnar engine's default memory allocation of 1 GB may be more than you can spare. Be aggressive about setting `alloydb.columnar_memory_size_mb` to a value that leaves adequate headroom for OS and application processes.
- **Connectivity is intermittent by definition.** Design your backup and observability pipeline to tolerate periods without Cloud Storage or Cloud Monitoring access. The WAL archiving `archive_command` should fail gracefully and queue locally rather than crashing the database when the WAN link is down.
- **Data sync is a separate problem.** Omni does not solve bidirectional sync between edge and central. You need to design replication flows explicitly — whether that is logical replication from edge to a central AlloyDB instance, a change-data-capture pipeline, or periodic bulk export.

```
# Edge deployment: resource-constrained container configuration
# Assumes 16 GB RAM host, 8 vCPUs, local SSD storage

docker run -d \
  --name alloydb-omni-edge \
  --restart unless-stopped \
  -e POSTGRES_PASSWORD='EdgeSecurePass456' \
  -e ALLOYDB_ENABLE_COLUMNAR_ENGINE=true \
  -e ALLOYDB_COLUMNAR_MEMORY_SIZE_MB=1024 \
  -e ALLOYDB_OBSERVABILITY_ENABLED=false \
  -p 5432:5432 \
  -v /alloydb/data:/var/lib/postgresql/data \
  -v /alloydb/wal-archive:/var/lib/postgresql/wal-archive \
  --shm-size=512m \
  --cpus=6 \
  --memory=12g \
  gcr.io/alloydb-omni/alloydb-omni:latest

# Configure archiving to local directory when cloud is unavailable
# A separate sync process uploads to Cloud Storage when connectivity resumes
docker exec alloydb-omni-edge psql -U postgres << 'EOF'
ALTER SYSTEM SET archive_mode = 'on';
ALTER SYSTEM SET archive_command =
  'test ! -f /var/lib/postgresql/wal-archive/%f &&
  cp %p /var/lib/postgresql/wal-archive/%f';
ALTER SYSTEM SET archive_timeout = '600';
SELECT pg_reload_conf();
EOF

# Sync script to upload locally archived WAL to Cloud Storage
# Run this as a cron job - it handles intermittent connectivity gracefully
#!/bin/bash
```

```
# edge_wal_sync.sh

WAL_ARCHIVE_DIR="/alloydb/wal-archive"
GCS_BUCKET="gs://your-company-edge-wal-archive"
SITE_ID="factory-floor-01"

# Attempt upload - gsutil exits 0 on success, non-zero on network failure
# The loop ensures already-uploaded files are skipped (gsutil -n flag)
gsutil -m cp -n \
  "${WAL_ARCHIVE_DIR}"/*.gz \
  "${GCS_BUCKET}/${SITE_ID}/" 2>/var/log/edge_wal_sync.log

if [ $? -eq 0 ]; then
  # Clean up local WAL files older than 7 days after successful upload
  find "${WAL_ARCHIVE_DIR}" -name "*.gz" -mtime +7 -delete
  echo "$(date): WAL sync completed successfully" >> /var/log/edge_wal_sync.log
else
  echo "$(date): WAL sync failed - will retry next cycle" >> /var/log/edge_wal_sync.log
fi
```

Logical replication from edge to central AlloyDB

If you need near-real-time data flow from edge Omni instances to a central AlloyDB cluster in GCP (or to another Omni instance in your datacenter), logical replication is the right mechanism. AlloyDB Omni supports PostgreSQL logical replication as both publisher and subscriber.

```
-- On the edge AlloyDB Omni instance (publisher side)
-- Requires wal_level = logical, which we set in the earlier configuration

-- Create a publication for the tables you want to replicate centrally
CREATE PUBLICATION edge_to_central_pub
FOR TABLE
  sensor_readings,
  production_events,
  quality_inspection_results
WITH (publish = 'insert, update, delete');

-- Create a replication user with minimal privileges
CREATE ROLE replication_user
  LOGIN REPLICATION PASSWORD 'ReplPass789!';

GRANT SELECT ON
  sensor_readings,
  production_events,
  quality_inspection_results
TO replication_user;

-- Verify publication
SELECT pubname, pubtables FROM pg_publication_tables
WHERE pubname = 'edge_to_central_pub';

-- On the central AlloyDB instance (subscriber side)
-- This runs on either AlloyDB in GCP or another AlloyDB Omni instance

-- The target tables must already exist with compatible schemas
-- Create the subscription pointing to the edge Omni instance
CREATE SUBSCRIPTION central_from_edge_sub
CONNECTION 'host=edge-site-01.internal
  port=5432
  dbname=production
  user=replication_user
  password=ReplPass789!
  sslmode=require'
PUBLICATION edge_to_central_pub
WITH (
  connect = true,
  enabled = true,
  create_slot = true,
  copy_data = true,
```

```
synchronous_commit = 'off' -- async is appropriate for edge replication
);

-- Monitor replication lag
SELECT
    subname,
    pid,
    received_lsn,
    latest_end_lsn,
    latest_end_time
FROM pg_stat_subscription;
```

One operational concern with logical replication from edge sites: if the edge site loses connectivity for an extended period, the replication slot on the publisher holds WAL on disk until the subscriber reconnects. On a space-constrained edge device, this can fill the data volume. Set a `max_slot_wal_keep_size` parameter to bound WAL retention, accepting that a subscriber that falls too far behind will need a full re-sync rather than catching up through replication:

```
-- On the edge Omni instance: limit WAL held for replication slots
ALTER SYSTEM SET max_slot_wal_keep_size = '10GB';
SELECT pg_reload_conf();

-- Monitor slot WAL retention
SELECT
    slot_name,
    pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS wal_retained,
    active
FROM pg_replication_slots;
```

23.8 Security Hardening for Production Omni Deployments

Running a database in a container introduces security considerations beyond what you face with a binary installed directly on a host. Container security, network isolation, credential management, and PostgreSQL-level access controls all need attention.

Container security baseline

```
# Run the container as a non-root user where the image supports it
# AlloyDB Omni's image runs the PostgreSQL process as uid 999 internally
# but the container itself should not be started as root on the host

# Verify the process user inside the container
docker exec alloydb-omni ps aux | grep postgres

# Restrict container capabilities - PostgreSQL does not need most Linux capabilities
docker stop alloydb-omni && docker rm alloydb-omni

docker run -d \
    --name alloydb-omni \
    --restart unless-stopped \
    --security-opt no-new-privileges:true \
    --cap-drop ALL \
    --cap-add CHOWN \
    --cap-add SETUID \
    --cap-add SETGID \
    --cap-add DAC_OVERRIDE \
    --read-only \
    --tmpfs /tmp:rw,noexec,nosuid,size=512m \
    --tmpfs /run:rw,noexec,nosuid,size=64m \
    -e POSTGRES_PASSWORD='YourSecureP@ssword123' \
    -e ALLOYDB_ENABLE_COLUMNAR_ENGINE=true \
    -e ALLOYDB_COLUMNAR_MEMORY_SIZE_MB=8192 \
```

```

-e GOOGLE_APPLICATION_CREDENTIALS=/etc/alloydb/gcp-credentials.json \
-e GOOGLE_CLOUD_PROJECT=your-project-id \
-p 127.0.0.1:5432:5432 \
-v /alloydb/data:/var/lib/postgresql/data \
-v /alloydb/logs:/var/log/postgresql \
-v /etc/alloydb:/etc/alloydb:ro \
--shm-size=2g \
--cpus=16 \
--memory=64g \
gcr.io/alloydb-omni/alloydb-omni:latest

```

Note the `-p 127.0.0.1:5432:5432` binding. Binding the port to loopback rather than `0.0.0.0` means PostgreSQL is only reachable from the host itself. Applications connect through a reverse proxy or pgBouncer that handles TLS termination and connection pooling. This is preferable to exposing the PostgreSQL port directly to the network.

PostgreSQL-level access controls

```

-- Harden the pg_hba.conf defaults
-- AlloyDB Omni ships with permissive defaults for developer convenience
-- Replace them with production-appropriate rules

-- View current HBA configuration
SELECT line_number, type, database, user_name, address, auth_method
FROM pg_hba_file_rules
ORDER BY line_number;

-- Remove the 'trust' auth method for local connections
-- Edit pg_hba.conf inside the data volume and reload
-- (Do this through a volume-mounted config file in production,
-- not by editing inside the container)

-- Create role hierarchy for application access
-- Never allow applications to connect as postgres superuser

CREATE ROLE app_readonly NOLOGIN;
CREATE ROLE app_readwrite NOLOGIN;
CREATE ROLE app_user LOGIN PASSWORD 'AppUserPass321!'
  IN ROLE app_readwrite;
CREATE ROLE reporting_user LOGIN PASSWORD 'ReportingPass654!'
  IN ROLE app_readonly;

-- Grant schema-level privileges rather than table-by-table
GRANT USAGE ON SCHEMA public TO app_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT ON TABLES TO app_readonly;

GRANT USAGE ON SCHEMA public TO app_readwrite;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public
  TO app_readwrite;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_readwrite;

-- Audit who has superuser access
SELECT username, usesuper, usecreatedb, usecreatorole
FROM pg_user
WHERE usesuper = true;

```

TLS configuration

AlloyDB Omni will use the certificates you place in the data directory. Generate a proper certificate for production rather than relying on the self-signed certificate the engine creates on first startup:

```

# Generate a certificate signing request on the host
openssl req -new -newkey rsa:4096 \
  -keyout /alloydb/tls/server.key \

```

```
-out /alloydb/tls/server.csr \  
-nodes \  
-subj "/CN=alloydb-omni.internal/O=YourCompany/C=US"  
  
# Have your CA sign it (or use a self-signed cert for internal networks)  
# Copy the signed cert to the data directory where PostgreSQL expects it  
cp /alloydb/tls/server.crt /alloydb/data/server.crt  
cp /alloydb/tls/server.key /alloydb/data/server.key  
chmod 600 /alloydb/data/server.key  
  
# Configure PostgreSQL to require TLS  
docker exec alloydb-omni psql -U postgres << 'EOF'  
ALTER SYSTEM SET ssl = 'on';  
ALTER SYSTEM SET ssl_cert_file = 'server.crt';  
ALTER SYSTEM SET ssl_key_file = 'server.key';  
ALTER SYSTEM SET ssl_min_protocol_version = 'TLSv1.2';  
SELECT pg_reload_conf();  
EOF
```

Key Takeaways

- **AlloyDB Omni delivers the columnar engine and AI extensions on your hardware**, but it does not replicate the managed-service durability model from GCP. Backup, HA, and patching are your complete responsibility; plan the operational model before deploying in production.
- **The columnar cache is automatic but not instantaneous.** New deployments and post-failover primaries start with a cold columnar cache. Use `ALTER TABLE ... SET (columnar = on)` and `alloydb_columnar_refresh()` to seed the cache proactively for your most important analytical tables rather than waiting for query-pattern detection to populate it.
- **Connecting Omni to Google Cloud is optional but valuable.** WAL archiving to Cloud Storage, metrics shipping to Cloud Monitoring, and Vertex AI embedding calls are all available through the service account integration. Each requires evaluating the data-egress and compliance implications before enabling in regulated environments.
- **Edge deployments are a genuine use case**, but they require explicit design for intermittent connectivity. Use `max_slot_wal_keep_size` to prevent replication slots from filling edge storage, and design your WAL archive sync to tolerate WAN outages gracefully rather than failing hard.
- **The columnar engine rewards query and schema design choices, not just memory tuning.** Wide, flat tables with date-range partitioning extract more benefit from the columnar path than heavily normalized schemas. Materialized summary tables populated from OLTP data are often the fastest path to realizing columnar performance gains without restructuring your transactional schema.

Chapter 24: Multi-Cloud Database Strategy: Benefits and Traps

Running databases across AWS, Azure, and Google Cloud simultaneously sounds like an architect’s dream until the first cross-cloud replication lag spikes at 3 AM and you realize your failover runbook assumes a single-provider world. Multi-cloud database strategy is not a technology decision — it is an operational commitment with real costs in complexity, latency, and engineering time. This chapter examines why organizations pursue multi-cloud deployments, what the architecture patterns actually look like in production, where the traps are hiding, and how to build the operational discipline that makes the difference between a multi-cloud strategy and a multi-cloud disaster. For experienced DBAs moving from on-premises to cloud, this chapter bridges the gap between what vendors promise in whitepapers and what your on-call rotation will actually face.

24.1 Why Organizations Go Multi-Cloud (And Why Some Shouldn’t)

The business motivations for multi-cloud databases fall into three honest categories: avoiding vendor lock-in, meeting regulatory requirements, and optimizing cost by workload. A fourth category — “our CFO negotiated deals with two cloud providers” — is more common than any conference talk will admit.

Avoiding vendor lock-in has real merit when you are running a workload that could plausibly move. If your application uses ANSI-standard SQL against PostgreSQL on RDS today, migrating to Cloud SQL or Azure Database for PostgreSQL is mechanically possible. The same is not true if you have deeply integrated Aurora-specific features like Aurora Global Database’s native replication API, or if you depend on AlloyDB’s columnar engine for analytical acceleration. Every proprietary feature you consume increases migration cost, so the vendor lock-in calculation is really a features-versus-flexibility trade-off you make at the schema design level.

Regulatory requirements often mandate data residency in specific geographies. An organization operating in the EU, US, and Australia may find that one provider’s regional coverage meets two of three requirements and forces a second provider for the third. This is a legitimate multi-cloud driver with a clear scope — you are not running the same workload on two clouds, you are running different regional instances on the best-available provider per region.

Cost optimization by workload works when your workloads have genuinely different characteristics. Analytical batch workloads that run eight hours and then stop are cheaper on spot/preemptible

instances. Transactional OLTP with strict SLAs belongs on reserved capacity. If one provider offers a materially better price-performance ratio for a specific engine or instance class, the savings can justify the operational overhead — but only if the operational overhead is actually accounted for.

The organizations that should not pursue multi-cloud database strategy are those doing it defensively without a concrete trigger. If you do not have a regulatory requirement, a demonstrated cost delta, or a proven vendor-reliability concern, you are adding complexity in exchange for optionality you may never exercise. Two clouds means two monitoring stacks, two IAM models, two network topologies, two DBA skill sets, and twice the surface area for misconfiguration.

A pragmatic test: can your team pass a realistic failover drill to the secondary cloud within your RTO today? If the answer is no, you have a multi-cloud architecture without a multi-cloud capability.

24.2 Architecture Patterns That Actually Work in Production

Multi-cloud database architectures in production cluster around four patterns. Each has a different cost profile, consistency model, and operational complexity.

Active-Passive with Periodic Replication places the primary workload on one cloud and maintains a warm standby on a second cloud, synchronized by scheduled or continuous logical replication. Recovery is manual or semi-automated. This is the lowest-cost pattern and the most honest — you are buying disaster recovery, not high availability. RTO is typically 15–60 minutes depending on how current your replication is and how rehearsed your runbook is.

Active-Passive with Continuous Logical Replication uses tools like AWS DMS, the PostgreSQL logical replication protocol, or a commercial CDC layer to maintain a near-real-time replica on the secondary cloud. RTO drops to 5–15 minutes if the failover process is automated. The trap here is replication lag accumulation under write-heavy workloads and the DDL limitation — PostgreSQL logical replication does not replicate DDL, so schema changes require coordinated application across both clouds.

Active-Active with Application-Level Routing runs write-capable instances on both clouds and uses application logic or a proxy layer to route writes. This works for workloads that can be partitioned by tenant, geography, or data domain. It does not work for workloads that require globally consistent transactions. If you have ever debugged a split-brain scenario in a self-managed Galera cluster, active-active multi-cloud will feel familiar — the same problems at larger scale with less control over the networking layer.

Read Scaling Across Clouds is the most underused and most appropriate pattern for many organizations. The primary write workload lives on one cloud. Read replicas or read-optimized copies live on a second cloud, serving analytics, reporting, or API reads that tolerate eventual consistency. This pattern eliminates the write-coordination problem entirely and gives you genuine leverage on the cost-versus-read-performance axis.

The following Terraform configuration demonstrates a realistic starting point: an Aurora PostgreSQL primary in AWS with logical replication feeding a Cloud SQL PostgreSQL read workload on GCP.

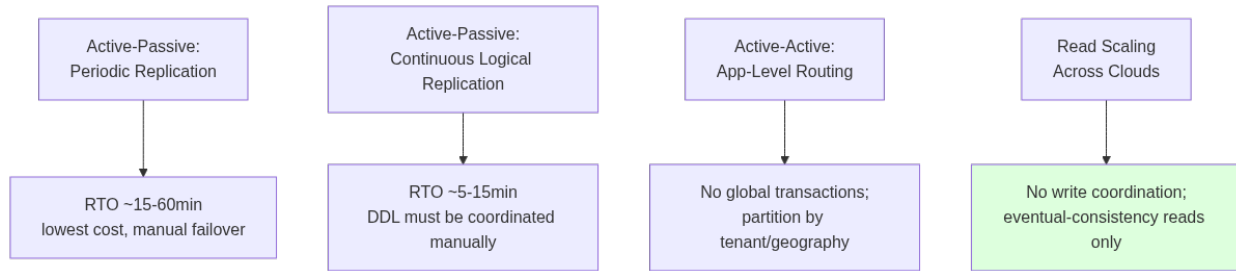


Figure 15: Four multi-cloud database patterns, ordered roughly by operational complexity. Read scaling avoids the write-coordination problem entirely and is the pattern most teams should start with.

```

# terraform/multi_cloud_db/main.tf
# AWS Aurora PostgreSQL primary cluster

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
    google = {
      source = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

provider "google" {
  project = var.gcp_project_id
  region = "us-central1"
}

# -----
# AWS: Aurora PostgreSQL cluster (primary write workload)
# -----
resource "aws_rds_cluster" "primary" {
  cluster_identifier      = "multicloud-primary"
  engine                 = "aurora-postgresql"
  engine_version         = "15.4"
  database_name          = "appdb"
  master_username        = "dbadmin"
  manage_master_user_password = true # stores in Secrets Manager

  db_subnet_group_name = aws_db_subnet_group.primary.name
  vpc_security_group_ids = [aws_security_group.aurora.id]

  backup_retention_period = 7
  deletion_protection     = true

  # Enable logical replication for cross-cloud CDC
  db_cluster_parameter_group_name = aws_rds_cluster_parameter_group.pg15_logical.name

  tags = {
    Environment = "production"
    Role        = "multi-cloud-primary"
  }
}

```

```

resource "aws_rds_cluster_parameter_group" "pg15_logical" {
  family = "aurora-postgresql15"
  name    = "pg15-logical-replication"

  parameter {
    name      = "rds.logical_replication"
    value     = "1"
    apply_method = "pending-reboot"
  }

  parameter {
    name      = "max_replication_slots"
    value     = "10"
    apply_method = "pending-reboot"
  }

  parameter {
    name      = "max_wal_senders"
    value     = "10"
    apply_method = "pending-reboot"
  }

  parameter {
    name      = "wal_sender_timeout"
    value     = "60000"
    apply_method = "immediate"
  }
}

resource "aws_rds_cluster_instance" "primary_writer" {
  cluster_identifier = aws_rds_cluster.primary.id
  instance_class     = "db.r7g.2xlarge"
  engine             = aws_rds_cluster.primary.engine
  engine_version     = aws_rds_cluster.primary.engine_version

  performance_insights_enabled      = true
  performance_insights_retention_period = 731 # 2 years
  monitoring_interval               = 60
  monitoring_role_arn               = aws_iam_role.rds_monitoring.arn
}

# -----
# GCP: Cloud SQL PostgreSQL (cross-cloud read / analytics workload)
# -----
resource "google_sql_database_instance" "analytics_replica" {
  name            = "multicloud-analytics"
  database_version = "POSTGRES_15"
  region          = "us-central1"
  deletion_protection = true

  settings {
    tier                = "db-custom-8-32768" # 8 vCPU, 32 GB RAM
    availability_type   = "REGIONAL"          # HA within GCP region

    backup_configuration {
      enabled = true
      start_time = "03:00"
    }

    database_flags {
      name = "max_connections"
      value = "500"
    }

    database_flags {
      name = "shared_buffers"
      value = "8192" # MB - Cloud SQL sets this as MB, not percentage
    }

    ip_configuration {

```

```

    ipv4_enabled    = false
    private_network = google_compute_network.db_vpc.id
  }
}
}

```

The replication slot on the Aurora primary feeds a logical subscriber on Cloud SQL. Schema changes must be coordinated manually — this is the DDL gap every PostgreSQL DBA hits in production.

24.3 Cross-Cloud Networking: The Layer That Breaks Everything

On-premises DBAs are accustomed to controlling network topology with routing tables, firewall rules, and VLAN configuration. In a multi-cloud deployment, you are working with VPCs that were designed in isolation by three different organizations, connected across the public internet or through dedicated interconnect circuits, with latency characteristics that are fundamentally different from a datacenter backplane.

Cross-cloud latency is a real planning input, not an afterthought. Round-trip time between two cloud regions over the public internet is a function of physical distance and routing path, typically in the low tens of milliseconds for same-continent region pairs — measure the actual RTT between your specific region pair before designing around an assumed number, since it varies by provider backbone and route. These numbers sound small until you are running synchronous replication and every write transaction pays that round-trip cost as a floor on commit latency. At sustained high write rates, that added latency per transaction becomes your binding throughput constraint.

The three viable connectivity options are public internet with TLS, cloud provider interconnect products (AWS Direct Connect, Azure ExpressRoute, GCP Cloud Interconnect), and third-party private backbone services like Megaport or Equinix Fabric. For production database replication, public internet is acceptable only for asynchronous patterns where replication lag is tolerable. Synchronous cross-cloud replication requires dedicated circuits to achieve predictable latency; check current pricing from your interconnect provider and factor the recurring circuit cost into your multi-cloud ROI calculation before committing to synchronous replication.

The following AWS CLI and gcloud commands establish the baseline connectivity verification you should run before configuring any cross-cloud replication:

```

# -----
# Step 1: Verify cross-cloud reachability from AWS side
# Run this from an EC2 instance in the same VPC as your Aurora cluster
# -----

# Check MTU path - database replication is sensitive to fragmentation
tracertpath <cloud-sql-private-ip>

# Sustained latency test - 1000 packets, 100ms interval
# PostgreSQL replication keepalive fires every wal_receiver_timeout (default 60s)
# You want p99 latency, not just average
ping -c 1000 -i 0.1 <cloud-sql-private-ip> | tail -2

# Verify TCP throughput - replication streams are long-lived, high-throughput
iperf3 -c <cloud-sql-private-ip> -p 5432 -t 30 -P 4

# -----
# Step 2: GCP side - verify routing and firewall rules
# -----

```

```

# Confirm the Cloud SQL instance is reachable on its private IP
gcloud sql instances describe multicloud-analytics \
  --format="value(ipAddresses)"

# List VPC firewall rules - ensure PostgreSQL port is open from AWS CIDR
gcloud compute firewall-rules list \
  --filter="network=db-vpc" \
  --format="table(name,direction,sourceRanges,allowed)"

# Create firewall rule allowing Aurora subnet to reach Cloud SQL
gcloud compute firewall-rules create allow-aurora-replication \
  --network=db-vpc \
  --direction=INGRESS \
  --priority=1000 \
  --source-ranges=10.0.0.0/16 \
  --allow=tcp:5432 \
  --target-tags=cloudsql-replica \
  --description="Allow Aurora logical replication from AWS us-east-1"

# -----
# Step 3: AWS side - update security group to allow outbound to GCP
# -----

# Get the Aurora cluster security group ID
AURORA_SG=$(aws rds describe-db-clusters \
  --db-cluster-identifier multicloud-primary \
  --query 'DBClusters[0].VpcSecurityGroups[0].VpcSecurityGroupId' \
  --output text)

# Allow outbound PostgreSQL to Cloud SQL private IP range
aws ec2 authorize-security-group-egress \
  --group-id $AURORA_SG \
  --protocol tcp \
  --port 5432 \
  --cidr 10.128.0.0/20 \
  --tag-specifications
  ↪ 'ResourceType=security-group-rule,Tags=[{Key=Purpose,Value=cross-cloud-replication}]'

# -----
# Step 4: Validate replication slot creation on Aurora primary
# Connect via psql from a bastion in the Aurora VPC
# -----

psql -h <aurora-cluster-endpoint> -U dbadmin -d appdb << 'EOF'
-- Create a named replication slot for the GCP subscriber
SELECT pg_create_logical_replication_slot(
  'gcp_analytics_slot',
  'pgoutput'
);

-- Verify the slot exists and check its state
SELECT slot_name, plugin, slot_type, active, restart_lsn, confirmed_flush_lsn
FROM pg_replication_slots
WHERE slot_name = 'gcp_analytics_slot';
EOF

```

The replication slot is your most dangerous cross-cloud dependency. If the GCP subscriber loses connectivity for an extended period and you do not monitor `pg_replication_slots.restart_lsn`, Aurora will hold WAL indefinitely to service that slot. In production environments with 50–200 GB/hour of WAL generation, a disconnected replication slot fills your storage within hours and triggers an emergency. Set a CloudWatch alarm on `TransactionLogsDiskUsage` with a threshold at 70% of your allocated storage, and configure automatic slot invalidation at your WAL retention limit.

24.4 Data Consistency and the CAP Theorem in Practice

The CAP theorem is not an academic concern in multi-cloud architecture — it is the framework that explains why your active-active design that looked correct on a whiteboard fails in production. When a cross-cloud network partition occurs (and it will occur), you must choose between consistency and availability. The choice you make in the design phase determines whether your system fails safely or fails corruptly.

PostgreSQL logical replication is asynchronous by design. There is no equivalent of Aurora Global Database’s managed RPO enforcement in a cross-cloud logical replication setup. Your GCP subscriber can fall arbitrarily behind the Aurora primary, and no built-in mechanism will stop writes on the primary when lag exceeds your RPO threshold. You have to build this check yourself.

The following Python script implements the cross-cloud replication lag monitor that should run as a Lambda function or Cloud Function every 60 seconds:

```
# cross_cloud_lag_monitor.py
# Monitors replication lag between Aurora primary (AWS) and Cloud SQL subscriber (GCP)
# Designed to run as AWS Lambda or as a container in ECS/GKE

import boto3
import psycopg2
import json
import logging
import os
from datetime import datetime, timezone
from typing import Optional

logger = logging.getLogger()
logger.setLevel(logging.INFO)

# Configuration from environment variables - never hardcode credentials
AURORA_SECRET_ARN = os.environ['AURORA_SECRET_ARN']
CLOUDSQL_SECRET_ARN = os.environ['CLOUDSQL_SECRET_ARN']
MAX_LAG_BYTES = int(os.environ.get('MAX_LAG_BYTES', 104857600)) # 100 MB default
MAX_LAG_SECONDS = int(os.environ.get('MAX_LAG_SECONDS', 300)) # 5 minutes default
SNS_ALERT_TOPIC_ARN = os.environ['SNS_ALERT_TOPIC_ARN']
CLOUDWATCH_NAMESPACE = 'MultiCloud/ReplicationLag'

def get_secret(secret_arn: str) -> dict:
    """Retrieve database credentials from AWS Secrets Manager."""
    client = boto3.client('secretsmanager')
    response = client.get_secret_value(SecretId=secret_arn)
    return json.loads(response['SecretString'])

def get_aurora_slot_lag(secret: dict, slot_name: str) -> dict:
    """
    Query Aurora primary for replication slot lag.
    Returns lag in bytes and an approximate seconds estimate.
    """
    conn = psycopg2.connect(
        host=secret['host'],
        port=secret.get('port', 5432),
        dbname=secret['dbname'],
        user=secret['username'],
        password=secret['password'],
        sslmode='require',
        connect_timeout=10
    )
    conn.autocommit = True

    try:
        with conn.cursor() as cur:
```

```

cur.execute("""
    SELECT
        slot_name,
        active,
        pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn) AS lag_bytes,
        extract(epoch FROM (now() - pg_last_xact_replay_timestamp())) AS lag_seconds_approx,
        pg_size_pretty(
            pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)
        ) AS retained_wal
    FROM pg_replication_slots
    WHERE slot_name = %s
""", (slot_name,))

row = cur.fetchone()
if row is None:
    raise ValueError(f"Replication slot '{slot_name}' not found on primary")

return {
    'slot_name': row[0],
    'active': row[1],
    'lag_bytes': row[2] or 0,
    'lag_seconds': row[3] or 0,
    'retained_wal': row[4]
}
finally:
    conn.close()

def get_cloudsql_subscriber_lag(secret: dict) -> dict:
    """
    Query Cloud SQL subscriber for subscription status and lag.
    pg_stat_subscription gives per-subscription lag on the subscriber side.
    """
    conn = psycopg2.connect(
        host=secret['host'],
        port=secret.get('port', 5432),
        dbname=secret['dbname'],
        user=secret['username'],
        password=secret['password'],
        sslmode='require',
        connect_timeout=10
    )
    conn.autocommit = True

    try:
        with conn.cursor() as cur:
            cur.execute("""
                SELECT
                    subname,
                    pid,
                    received_lsn,
                    last_msg_send_time,
                    last_msg_receipt_time,
                    extract(epoch FROM (now() - last_msg_receipt_time)) AS seconds_since_last_message,
                    latest_end_lsn,
                    extract(epoch FROM (now() - latest_end_time)) AS seconds_since_last_flush
                FROM pg_stat_subscription
                WHERE subname = 'aurora_to_gcp_sub'
            """)

            row = cur.fetchone()
            if row is None:
                return {'connected': False, 'error': 'Subscription worker not running'}

            return {
                'connected': True,
                'subname': row[0],
                'worker_pid': row[1],
                'received_lsn': str(row[2]),
                'last_message_seconds_ago': float(row[5] or 0),
            }
    
```



```

        'seconds_since_last_flush': float(row[7] or 0)
    }
finally:
    conn.close()

def publish_metrics(lag_data: dict) -> None:
    """Push lag metrics to CloudWatch for unified alerting."""
    cw = boto3.client('cloudwatch')

    metric_data = [
        {
            'MetricName': 'ReplicationLagBytes',
            'Dimensions': [
                {'Name': 'Source', 'Value': 'aurora-us-east-1'},
                {'Name': 'Target', 'Value': 'cloudsql-us-central1'}
            ],
            'Value': lag_data.get('lag_bytes', 0),
            'Unit': 'Bytes',
            'Timestamp': datetime.now(timezone.utc)
        },
        {
            'MetricName': 'SecondsSinceLastMessage',
            'Dimensions': [
                {'Name': 'Source', 'Value': 'aurora-us-east-1'},
                {'Name': 'Target', 'Value': 'cloudsql-us-central1'}
            ],
            'Value': lag_data.get('last_message_seconds_ago', 0),
            'Unit': 'Seconds',
            'Timestamp': datetime.now(timezone.utc)
        }
    ]

    cw.put_metric_data(
        Namespace=CLLOUDWATCH_NAMESPACE,
        MetricData=metric_data
    )
    logger.info("Published %d metrics to CloudWatch", len(metric_data))

def send_alert(subject: str, message: str) -> None:
    """Publish an SNS alert for lag threshold violations."""
    sns = boto3.client('sns')
    sns.publish(
        TopicArn=SNS_ALERT_TOPIC_ARN,
        Subject=subject,
        Message=message
    )
    logger.warning("ALERT SENT: %s", subject)

def lambda_handler(event, context):
    """Lambda entry point for cross-cloud replication lag monitoring."""
    aurora_secret = get_secret(AURORA_SECRET_ARN)
    cloudsql_secret = get_secret(CLOUDSQL_SECRET_ARN)

    slot_lag = get_aurora_slot_lag(aurora_secret, 'gcp_analytics_slot')
    subscriber_lag = get_cloudsql_subscriber_lag(cloudsql_secret)

    combined = {**slot_lag, **subscriber_lag, 'check_time': datetime.now(timezone.utc).isoformat()}
    logger.info("Lag report: %s", json.dumps(combined, default=str))

    publish_metrics(combined)

    # Alert conditions
    if not subscriber_lag.get('connected', False):
        send_alert(
            subject="CRITICAL: Cross-cloud replication subscriber disconnected",
            message=f"The Cloud SQL subscription worker is not running.\n\nDetails:
↪ {json.dumps(combined, default=str)}"

```

```

    )
    elif slot_lag['lag_bytes'] > MAX_LAG_BYTES:
        send_alert(
            subject=f"WARNING: Cross-cloud replication lag {slot_lag['lag_bytes']} // 1048576} MB",
            message=f"Replication lag exceeds {MAX_LAG_BYTES // 1048576} MB threshold.\n"
                    f"Retained WAL on primary: {slot_lag['retained_wal']}\n\n"
                    f"Details: {json.dumps(combined, default=str)}"
        )
    elif subscriber_lag.get('last_message_seconds_ago', 0) > MAX_LAG_SECONDS:
        send_alert(
            subject=f"WARNING: No replication message for
↪ {int(subscriber_lag['last_message_seconds_ago'])} seconds",
            message=f"Cloud SQL subscriber has not received a replication message in "
                    f"{int(subscriber_lag['last_message_seconds_ago'])} seconds.\n\n"
                    f"Details: {json.dumps(combined, default=str)}"
        )
    )

    return {'statusCode': 200, 'body': json.dumps(combined, default=str)}

```

Run this monitor on a schedule independent of your application tier. If the Lambda itself fails, you want your CloudWatch alarm on `ReplicationLagBytes` to go stale and trigger a missing-data alert — configure the alarm’s `TreatMissingData` property to `breaching` so silence is treated as a failure.

24.5 Identity, Access, and the Security Boundary Problem

On a single cloud, IAM is complicated. Across two or three clouds, IAM is a cross-organizational governance problem with no shared standard and significant risk of privilege misconfiguration. Each provider’s identity model differs in ways that matter for database access: AWS uses resource-based policies and IAM roles with short-lived STS tokens; GCP uses service accounts with keys or Workload Identity Federation; Azure uses Managed Identities and Entra ID (formerly Azure AD) groups.

The trap that catches multi-cloud teams is the long-lived credential that gets created to “just make the connection work” during initial setup and never rotated. A service account key exported from GCP and stored in an EC2 parameter — not a secret, a parameter — is a real pattern found in real production environments. It represents a static credential with no expiry, no rotation, and read access to your analytics database, stored in plaintext in infrastructure that your entire engineering organization can query.

The correct architecture uses short-lived credential exchange at the boundary. AWS supports Workload Identity Federation with GCP, which means an EC2 instance or Lambda can obtain a GCP service account token by presenting an AWS STS token to GCP’s STS endpoint — no static keys required. The following configuration establishes this trust:

```

# -----
# Configure AWS-to-GCP Workload Identity Federation
# This allows AWS Lambda to authenticate to GCP without static keys
# -----

# Step 1: Create a GCP Workload Identity Pool
gcloud iam workload-identity-pools create aws-replication-pool \
    --location="global" \
    --display-name="AWS Replication Monitor Pool" \
    --description="Allows AWS Lambda to authenticate to GCP for lag monitoring"

# Step 2: Add an AWS provider to the pool

```

```

# Replace ACCOUNT_ID with your AWS account number
gcloud iam workload-identity-pools providers create-aws aws-provider \
  --location="global" \
  --workload-identity-pool="aws-replication-pool" \
  --account-id="$AWS_ACCOUNT_ID" \
  --attribute-mapping=|
  ↪ "google.subject=assertion.arn,attribute.aws_role=assertion.arn.extract('/assumed-role/{role}/')"

# Step 3: Create a GCP service account for the monitoring workload
gcloud iam service-accounts create cross-cloud-monitor \
  --display-name="Cross-Cloud Replication Monitor" \
  --description="Used by AWS Lambda to query Cloud SQL replication status"

# Step 4: Grant the service account Cloud SQL viewer access
gcloud projects add-iam-policy-binding $GCP_PROJECT_ID \
  --member="serviceAccount:cross-cloud-monitor@$GCP_PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/cloudsql.viewer"

# Also need ability to connect and run SELECT on pg_stat_subscription
# This is a database-level grant, not an IAM grant
# Connect to Cloud SQL and run:
# GRANT pg_monitor TO cross_cloud_monitor_user;

# Step 5: Bind the AWS Lambda execution role to the GCP service account
# Replace LAMBDA_ROLE_ARN with your Lambda's execution role ARN
gcloud iam service-accounts add-iam-policy-binding \
  cross-cloud-monitor@$GCP_PROJECT_ID.iam.gserviceaccount.com \
  --role="roles/iam.workloadIdentityUser" \
  --member="principalSet://iam.googleapis.com/projects/$GCP_PROJECT_NUMBER|
  ↪ /locations/global/workloadIdentityPools/aws-replication-pool/attribute.aws_role/cross-cloud-replication-monitor"

```

With this configuration, your Lambda function uses the `google-auth` library to exchange its AWS credentials for a short-lived GCP access token. No exported service account key exists anywhere in the system. When the Lambda execution role is rotated or decommissioned, GCP access is automatically revoked.

The analogous pattern in the other direction — GCP Cloud Functions authenticating to AWS RDS — uses GCP's OIDC token issued to a service account, presented to AWS STS via `AssumeRoleWithWebIdentity`. Both directions eliminate the long-lived credential problem, but both require upfront configuration that teams skip under schedule pressure.

Database-level access control must be treated as a separate layer from cloud IAM. Cloud IAM controls who can manage infrastructure (create instances, modify parameter groups, take snapshots). Database users control who can read and write data. A service account with `roles/cloudsql.admin` cannot log into your PostgreSQL database without a corresponding database user — but `roles/cloudsql.admin` can reset the postgres superuser password and create that user. The two layers must be audited independently.

24.6 Schema Management Across Cloud Boundaries

Every multi-cloud database deployment eventually faces the schema synchronization problem. Your migration tooling — Flyway, Liquibase, or raw SQL scripts applied in CI/CD — was almost certainly designed to target one database. In a multi-cloud topology, you have at minimum two independently managed database instances that must stay schema-compatible.

The options are: apply migrations to both targets in sequence from a single pipeline, apply to the primary and let replication carry the changes, or maintain separate migration tracks per envi-

ronment. The second option — relying on replication to propagate DDL — does not work with PostgreSQL logical replication, which replicates DML only. DDL changes on the primary are not forwarded to subscribers. If you run `ALTER TABLE orders ADD COLUMN discount_cents INTEGER DEFAULT 0` on Aurora and forget to run the same statement on Cloud SQL, inserts that include `discount_cents` will begin failing on the subscriber because the column does not exist.

The following Flyway configuration and wrapper script implements a two-target migration with coordinated execution and rollback detection:

```
# migrate_multi_cloud.py
# Applies a Flyway migration to Aurora primary first, then Cloud SQL subscriber
# Validates schema hash agreement after migration
# Designed to run in CI/CD pipeline (GitHub Actions, GitLab CI, etc.)

import subprocess
import sys
import json
import logging
import os
import psycopg2
import hashlib

logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s %(message)s')
logger = logging.getLogger(__name__)

def run_flyway(target_url: str, user: str, password: str, locations: str) -> bool:
    """Execute Flyway migrate against a target URL. Returns True on success."""
    cmd = [
        'flyway',
        f'-url={target_url}',
        f'-user={user}',
        f'-password={password}',
        f'-locations={locations}',
        '-outOfOrder=false',
        '-validateOnMigrate=true',
        'migrate'
    ]

    logger.info("Running Flyway against %s", target_url.split('@')[-1]) # log host only, not
    ↪ credentials

    result = subprocess.run(cmd, capture_output=True, text=True, timeout=300)

    if result.returncode != 0:
        logger.error("Flyway migration FAILED:\n%s\n%s", result.stdout, result.stderr)
        return False

    logger.info("Flyway migration succeeded:\n%s", result.stdout)
    return True

def get_schema_fingerprint(host: str, port: int, dbname: str, user: str, password: str) -> str:
    """
    Compute a stable fingerprint of the public schema's table and column structure.
    Uses pg_catalog to enumerate tables, columns, data types, and constraints
    in a deterministic order, then hashes the result.

    This is not a substitute for full schema comparison - it detects gross divergence.
    """
    conn = psycopg2.connect(
        host=host, port=port, dbname=dbname,
        user=user, password=password, sslmode='require'
    )
    try:
        with conn.cursor() as cur:
            cur.execute("""
```

```

        SELECT
            t.table_name,
            c.column_name,
            c.data_type,
            c.character_maximum_length,
            c.numeric_precision,
            c.is_nullable,
            c.column_default
        FROM information_schema.tables t
        JOIN information_schema.columns c
            ON c.table_schema = t.table_schema
            AND c.table_name = t.table_name
        WHERE t.table_schema = 'public'
            AND t.table_type = 'BASE TABLE'
        ORDER BY t.table_name, c.ordinal_position
    """
    rows = cur.fetchall()

finally:
    conn.close()

fingerprint_input = json.dumps(rows, default=str, sort_keys=True)
return hashlib.sha256(fingerprint_input.encode()).hexdigest()

def main():
    # Target configurations - in CI/CD these come from vault or secret store
    aurora_config = {
        'url': os.environ['AURORA_JDBC_URL'],
        'user': os.environ['AURORA_DB_USER'],
        'password': os.environ['AURORA_DB_PASSWORD'],
        'host': os.environ['AURORA_HOST'],
        'port': int(os.environ.get('AURORA_PORT', 5432)),
        'dbname': os.environ['AURORA_DBNAME'],
    }

    cloudsql_config = {
        'url': os.environ['CLOUDSQL_JDBC_URL'],
        'user': os.environ['CLOUDSQL_DB_USER'],
        'password': os.environ['CLOUDSQL_DB_PASSWORD'],
        'host': os.environ['CLOUDSQL_HOST'],
        'port': int(os.environ.get('CLOUDSQL_PORT', 5432)),
        'dbname': os.environ['CLOUDSQL_DBNAME'],
    }

    migration_locations = os.environ.get('FLYWAY_LOCATIONS', 'filesystem:./migrations')

    # Phase 1: Apply to Aurora primary
    logger.info("=== Phase 1: Migrating Aurora primary ===")
    aurora_ok = run_flyway(aurora_config['url'], aurora_config['user'],
                          aurora_config['password'], migration_locations)

    if not aurora_ok:
        logger.error("Aurora migration failed - aborting. Cloud SQL not touched.")
        sys.exit(1)

    # Phase 2: Apply same migration set to Cloud SQL subscriber
    # NOTE: Logical replication does NOT carry DDL. This step is mandatory.
    logger.info("=== Phase 2: Migrating Cloud SQL subscriber ===")
    cloudsql_ok = run_flyway(cloudsql_config['url'], cloudsql_config['user'],
                           cloudsql_config['password'], migration_locations)

    if not cloudsql_ok:
        logger.error(
            "Cloud SQL migration FAILED after Aurora succeeded. "
            "MANUAL INTERVENTION REQUIRED: schemas are now diverged. "
            "Apply the migration manually to Cloud SQL before the next deployment."
        )
        sys.exit(2) # distinct exit code so CI/CD can page on this specifically

    # Phase 3: Fingerprint comparison - catch silent divergence

```

```
logger.info("=== Phase 3: Schema fingerprint verification ===")

aurora_fp = get_schema_fingerprint(
    aurora_config['host'], aurora_config['port'],
    aurora_config['dbname'], aurora_config['user'], aurora_config['password']
)

cloudsql_fp = get_schema_fingerprint(
    cloudsql_config['host'], cloudsql_config['port'],
    cloudsql_config['dbname'], cloudsql_config['user'], cloudsql_config['password']
)

if aurora_fp == cloudsql_fp:
    logger.info("Schema fingerprints match: %s", aurora_fp)
    logger.info("Multi-cloud migration completed successfully.")
else:
    logger.error(
        "SCHEMA FINGERPRINT MISMATCH after migration.\n"
        "Aurora: %s\n"
        "Cloud SQL: %s\n"
        "Investigate pg_dump --schema-only output on both targets.",
        aurora_fp, cloudsql_fp
    )
    sys.exit(3)

if __name__ == '__main__':
    main()
```

The fingerprint comparison at the end catches accumulated divergence — cases where a hotfix was applied directly to one target during an incident and never reconciled. Run this check as a separate scheduled job independent of deployments so you discover schema drift before it causes a replication failure.

24.7 Observability Across Cloud Boundaries

Single-cloud observability is solved: pick CloudWatch, Azure Monitor, or Cloud Monitoring, configure your dashboards and alerts, and operate. Multi-cloud observability is not solved — it is a decision about where your source of truth lives and how you aggregate signals from providers that have incompatible metric naming conventions, different label cardinalities, and different default retention periods.

The practical answer for most teams is a neutral aggregation layer: Prometheus with `remote_write`, Datadog, Grafana Cloud, or a self-managed Victoria Metrics cluster that ingests from all three clouds. The PostgreSQL-level metrics you need — replication lag, connection count, query duration percentiles, lock wait times, cache hit ratio, WAL generation rate — are the same regardless of which cloud hosts the instance. The collection mechanism differs.

For Aurora, the PostgreSQL metrics come from two places: CloudWatch publishes instance-level metrics (CPU, I/O, connections, FreeableMemory) and Performance Insights publishes query-level metrics accessible via the AWS PI API. For Cloud SQL, the equivalent instance-level metrics are in Cloud Monitoring under the `cloudsql.googleapis.com` metric prefix, and query-level insight is in Cloud SQL Insights. Neither provider's native metrics are queryable from the other provider's console.

The following Prometheus configuration demonstrates scraping PostgreSQL exporter instances de-

ployed as sidecars on both clouds:

```
# prometheus/multi_cloud_scrape.yml
# Prometheus scrape configuration for cross-cloud PostgreSQL monitoring
# postgres_exporter runs as a sidecar on an EC2 bastion (AWS) and
# a Cloud Run job (GCP), each connected to their respective database

global:
  scrape_interval:    30s
  evaluation_interval: 30s
  external_labels:
    environment: production
    monitoring_tier: multi-cloud

scrape_configs:

  # Aurora primary - via postgres_exporter on EC2 bastion in AWS
  - job_name: aurora_primary
    scheme: https
    tls_config:
      ca_file: /etc/prometheus/certs/internal-ca.crt
    static_configs:
      - targets: ['aurora-exporter.internal.example.com:9187']
        labels:
          cloud:    aws
          region:   us-east-1
          db_role:  primary
          engine:   aurora-postgresql

  # Relabel to normalize metric names across clouds
  metric_relabel_configs:
    - source_labels: [__name__]
      regex: 'pg_(.*)'
      target_label: __name__
      replacement: 'postgres_${1}'

  # Cloud SQL analytics subscriber - via postgres_exporter on GCP Cloud Run
  - job_name: cloudsql_subscriber
    scheme: https
    tls_config:
      ca_file: /etc/prometheus/certs/internal-ca.crt
    static_configs:
      - targets: ['cloudsql-exporter.internal.example.com:9187']
        labels:
          cloud:    gcp
          region:   us-central1
          db_role:  subscriber
          engine:   cloud-sql-postgresql

  metric_relabel_configs:
    - source_labels: [__name__]
      regex: 'pg_(.*)'
      target_label: __name__
      replacement: 'postgres_${1}'

  # Alertmanager routing - single alert destination regardless of source cloud
  alerting:
    alertmanagers:
      - static_configs:
          - targets: ['alertmanager:9093']

rule_files:
  - '/etc/prometheus/rules/cross_cloud_replication.yml'
  - '/etc/prometheus/rules/postgres_health.yml'

# prometheus/rules/cross_cloud_replication.yml
groups:
  - name: cross_cloud_replication
    interval: 60s
    rules:
```

```
- alert: CrossCloudReplicationSlotInactive
  expr: |
    postgres_replication_slot_active{cloud="aws"} == 0
  for: 2m
  labels:
    severity: critical
    team: database
  annotations:
    summary: "Replication slot {{ $labels.slot_name }} is inactive on Aurora"
    description: |
      The logical replication slot feeding the Cloud SQL subscriber is not active.
      WAL retention is accumulating. Check Cloud SQL subscriber worker and
      cross-cloud network connectivity.

- alert: CrossCloudLagExceedsRPO
  expr: |
    postgres_replication_slot_pg_wal_lsn_diff_bytes{cloud="aws", slot_name="gcp_analytics_slot"} >
↪ 524288000
  for: 5m
  labels:
    severity: warning
    team: database
  annotations:
    summary: "Cross-cloud replication lag exceeds 500 MB"
    description: |
      Replication lag to Cloud SQL has exceeded 500 MB for 5 minutes.
      Current lag: {{ $value | humanize1024 }} bytes.
      Check subscriber write throughput and network throughput between regions.

- alert: CrossCloudSubscriberSilent
  expr: |
    (time() - postgres_stat_subscription_last_msg_receipt_time_seconds{cloud="gcp"}) > 300
  for: 0m
  labels:
    severity: critical
    team: database
  annotations:
    summary: "Cloud SQL subscriber has not received messages for {{ $value | humanizeDuration }}"
    description: |
      The Cloud SQL subscription worker last received a message
      {{ $value | humanizeDuration }} ago. This may indicate a network partition
      or a crashed subscription worker. Check Cloud SQL logs immediately.
```

The `for: 0m` on `CrossCloudSubscriberSilent` is intentional — silence on the replication channel is an immediate concern, not something you wait five minutes to confirm. The lag threshold alert uses a five-minute window to avoid false positives from transient network hiccups.

24.8 Failover Runbooks and the Discipline of Practice

A failover runbook that has never been executed is a fiction document. The most significant difference between organizations that successfully execute multi-cloud failover and those that do not is rehearsal frequency. Organizations that drill quarterly discover the gaps — the IAM role that expired, the DNS TTL that takes four minutes to propagate, the application config that has the read replica endpoint hardcoded — before a real incident.

The following is a production-caliber partial runbook for promoting the Cloud SQL subscriber to an independent writable instance after Aurora becomes unavailable. This is not a full recovery runbook — it covers the database layer only. Application reconfiguration, DNS cutover, and external service dependencies require separate runbook sections maintained by the teams who own those layers.


```

# Runbook: Promote Cloud SQL PostgreSQL Subscriber to Standalone Primary
# Trigger: Aurora primary unavailable AND decision made to activate GCP secondary
# Owner: Database Engineering
# Last tested: (fill in date during each drill)
# RTO target: 15 minutes from trigger to Cloud SQL accepting application writes

## Pre-conditions (verify before starting)
- [ ] Aurora primary confirmed unreachable (not a monitoring false positive)
- [ ] Incident commander has approved cross-cloud failover
- [ ] Application team has been notified - they must update connection strings
- [ ] Confirm replication lag at time of outage from last Prometheus scrape:
    `postgres_replication_slot_pg_wal_lsn_diff_bytes{slot_name="gcp_analytics_slot"}`
    Record this value: _____ bytes (this is your data loss exposure)

## Step 1 - Stop the subscription worker (5 min)
# On Cloud SQL subscriber, via psql:

-- Disable the subscription cleanly to prevent reconnection attempts
ALTER SUBSCRIPTION aurora_to_gcp_sub DISABLE;

-- Verify worker has stopped
SELECT subname, pid FROM pg_stat_subscription WHERE subname = 'aurora_to_gcp_sub';
-- Expected: pid IS NULL after disabling

## Step 2 - Remove the subscription object (2 min)
-- Required before Cloud SQL can accept writes to subscribed tables
-- Dropping the subscription does NOT drop the replicated tables or data

DROP SUBSCRIPTION aurora_to_gcp_sub;

-- Verify tables and data are intact
SELECT count(*) FROM orders;          -- spot-check a high-write table
SELECT max(created_at) FROM orders;    -- verify recency of replicated data

## Step 3 - Reset sequences (CRITICAL - 3 min)
-- PostgreSQL logical replication does NOT replicate sequence state.
-- If the Aurora primary had orders.id at 9,847,231, Cloud SQL may have
-- the sequence at a much lower value depending on when it was last synced.
-- Inserting with a stale sequence causes primary key violations.

-- Find the maximum value for each identity/serial column:
SELECT 'SELECT setval(' || seq.relname || ', max(' || col.column_name || ') + 10000)'
    || ' FROM ' || col.table_name || ';'
FROM information_schema.columns col
JOIN pg_class seq ON seq.relname = col.table_name || '_' || col.column_name || '_seq'
WHERE col.column_default LIKE 'nextval%'
    AND col.table_schema = 'public'
ORDER BY col.table_name, col.column_name;
-- Execute each generated statement

-- Alternatively, for tables using GENERATED ALWAYS AS IDENTITY:
SELECT 'SELECT setval(pg_get_serial_sequence(' || table_name || ', ' || column_name || '), '
    || '(SELECT max(' || column_name || ') FROM ' || table_name || ') + 10000);'
FROM information_schema.columns
WHERE column_default IS NULL
    AND is_identity = 'YES'
    AND table_schema = 'public'
ORDER BY table_name, column_name;

## Step 4 - Verify write capability (2 min)
BEGIN;
INSERT INTO _failover_test (initiated_at, note)
VALUES (now(), 'cross-cloud failover test write');
-- Verify row inserted
SELECT * FROM _failover_test ORDER BY initiated_at DESC LIMIT 1;
COMMIT;

## Step 5 - Notify application team
-- Provide them:
-- Write endpoint: <cloud-sql-private-ip>:5432

```

```
-- Database name: appdb
-- Application connection strings must be updated before traffic is routed

## Step 6 - Monitor first 15 minutes of write traffic
-- Watch for:
SELECT pid, username, application_name, state, wait_event_type, wait_event, query
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY backend_start;

-- Watch for lock contention from concurrent inserts (sequence conflicts surface here)
SELECT relation::regclass, mode, granted, pid, age(now(), query_start) AS wait_time
FROM pg_locks l JOIN pg_stat_activity a USING (pid)
WHERE NOT granted
ORDER BY wait_time DESC;
```

The sequence reset step (Step 3) is the one that fails silently in underprepared failovers. PostgreSQL logical replication replicates sequence increments as part of DML operations — when a row is inserted on the primary, the sequence advances on the primary, and the row (with its assigned ID) arrives on the subscriber. But the subscriber’s sequence object itself does not track the primary’s current sequence value. After promotion, the subscriber’s sequence may attempt to assign IDs that already exist in the replicated data, producing primary key violations on the first batch of inserts.

Adding 10,000 to the observed maximum is a conservative buffer. In high-write environments where the Aurora primary was generating thousands of inserts per second before failure, increase this buffer to 100,000 or more, or query the primary’s last-known sequence state from your monitoring data before it became unavailable.

24.9 When to Walk Away from Multi-Cloud

This chapter would be incomplete without an honest accounting of the situations where the correct answer is to consolidate back to a single cloud rather than continue investing in multi-cloud complexity.

Your team does not have the operational depth. Multi-cloud database operations require DBAs fluent in at least two providers’ tooling, networking models, and failure modes. If your database team is one or two people whose primary expertise is on-premises SQL Server or Oracle, adding a second cloud provider is not a force multiplier — it is a liability. The work expands beyond what the team can maintain with quality.

Your application cannot tolerate the latency. If your transaction commit time SLA is under 50 ms and you are considering synchronous cross-cloud replication with 20 ms one-way latency, the math does not work. Asynchronous patterns will not meet your RPO requirements. This is not a solvable engineering problem — it is a physics constraint.

The cost savings are theoretical. Run the actual numbers: dedicated interconnect circuit costs, additional DBA hours for operational overhead, additional licensing if you are using commercial replication tools, and the engineering time to build and maintain the custom monitoring and failover automation. If the savings on compute from using a cheaper cloud for a workload are smaller than the sum of these overhead costs, multi-cloud has a negative ROI.

Your data governance team cannot manage it. Multi-cloud database environments complicate data lineage, access auditing, and regulatory compliance reporting. If you are subject to SOC

2, PCI-DSS, or HIPAA, demonstrating compliance across two clouds with two separate audit trails requires tooling and process that most compliance teams are not resourced to operate.

In each of these cases, the right engineering judgment is to consolidate. A single well-operated cloud deployment is always preferable to a multi-cloud deployment that nobody fully understands. The goal is not architectural sophistication — it is reliable data storage with a team that can operate and recover it under pressure.

Key Takeaways

- Multi-cloud database strategy is an operational commitment, not just an architectural decision — the benefits only materialize if your team can actually execute failover, manage schema changes across boundaries, and monitor replication health continuously across both providers.
 - Cross-cloud logical replication works reliably for read-scaling and analytics patterns but requires explicit DDL coordination, sequence reset procedures at failover time, and replication slot monitoring to prevent WAL accumulation on the primary.
 - Network latency between cloud regions is a hard constraint that determines which consistency models are viable — asynchronous replication tolerates latency but accepts data loss exposure, while synchronous cross-cloud replication adds the full round-trip cost to every write transaction commit.
 - Identity and access management across clouds must be built on short-lived credential exchange (Workload Identity Federation, OIDC token exchange) rather than static service account keys, which are the most common security vulnerability in cross-cloud database deployments.
 - The highest-value investment in any multi-cloud database program is rehearsed failover drills on a regular cadence — undiscovered gaps in runbooks, expired credentials, stale DNS TTLs, and sequence state problems only surface under the conditions that a drill replicates, not during normal operations.
-

Chapter 25: Cross-Cloud Replication Architectures

Replication across cloud providers is one of the most architecturally demanding problems a cloud DBA will face. The need arises more often than you might expect: regulatory requirements that mandate data residency in specific regions served by different providers, acquisitions that land you with a PostgreSQL workload on GCP when your primary platform is AWS, disaster recovery strategies that treat provider failure as a first-class concern, or hybrid architectures where one cloud handles analytical workloads while another handles OLTP. Whatever the driver, cross-cloud replication forces you to reason about things that same-provider replication abstracts away—network transit costs, TLS certificate management across trust boundaries, WAL format incompatibilities between managed service versions, and the operational reality that you cannot place both ends of the replication stream inside the same VPC. This chapter covers the architectural patterns, tooling options, and production configuration required to build reliable cross-cloud replication between AWS RDS/Aurora, Azure SQL Database, and Google Cloud SQL/AlloyDB.

25.1 Understanding the Cross-Cloud Replication Landscape

Before writing a single line of configuration, you need to accept a fundamental constraint: no managed database service natively supports another cloud’s replication protocol. AWS Aurora’s Global Database uses Aurora-specific storage replication. Azure SQL’s active geo-replication uses proprietary log shipping. Google’s Spanner uses its own Paxos-based replication internally. None of these speak to each other natively.

This means every cross-cloud replication architecture uses one of three approaches:

Logical replication over the public internet or dedicated interconnects. PostgreSQL’s logical replication (via `pg_logical` or the native logical replication built into PostgreSQL 10+) works between any two PostgreSQL-compatible endpoints that can reach each other over TCP. AWS RDS for PostgreSQL, Google Cloud SQL for PostgreSQL, and Azure Database for PostgreSQL all support logical replication. The catch is that they may run slightly different PostgreSQL minor versions, and WAL-level logical decoding requires `wal_level = logical` to be set—something you control through Parameter Groups on RDS, flags on Cloud SQL, and server parameters on Azure Database for PostgreSQL.

Change Data Capture (CDC) through an intermediary. Tools like AWS DMS, Debezium, and Striim sit between the source and target, read the source’s transaction log, transform the

events, and write them to the target. This is the most flexible approach because the source and target do not need to run the same database engine. You can replicate from SQL Server on AWS RDS to PostgreSQL on Google Cloud SQL using DMS with SCT-generated schema mappings, for example.

Application-level dual-write. The application writes to both cloud databases simultaneously, with one treated as authoritative for conflict resolution. This is the most operationally fragile pattern and only makes sense when you cannot get replication lag below acceptable thresholds for your workload’s consistency requirements.

The practical choice for most production architectures is logical replication for same-engine scenarios and CDC via DMS or Debezium for cross-engine scenarios. The remainder of this chapter covers both.

Network connectivity is the prerequisite you cannot skip. Cross-cloud replication requires stable, low-latency, encrypted connectivity between the two clouds. Your options, roughly ordered by reliability and cost:

- **AWS Direct Connect + Azure ExpressRoute or Google Cloud Interconnect:** dedicated private circuits, lowest jitter, highest recurring cost — check current per-port and cross-connect pricing from each provider and your colocation facility before budgeting.
- **VPN over the public internet:** IPsec tunnels between VGW (AWS) and VPN Gateway (Azure/GCP), adequate for lower-throughput replication, with cross-region latency typically in the tens of milliseconds and a much lower recurring cost than dedicated interconnects.
- **Public internet with TLS:** no dedicated circuit, lowest cost, highest variability. Acceptable for replication that can tolerate seconds of lag; not appropriate for anything approaching synchronous replication.

For production replication carrying more than ~100 MB/s of WAL data, you want dedicated interconnects. For secondary replicas used for disaster recovery where RPO of a few seconds is acceptable, a well-tuned VPN tunnel is usually sufficient.

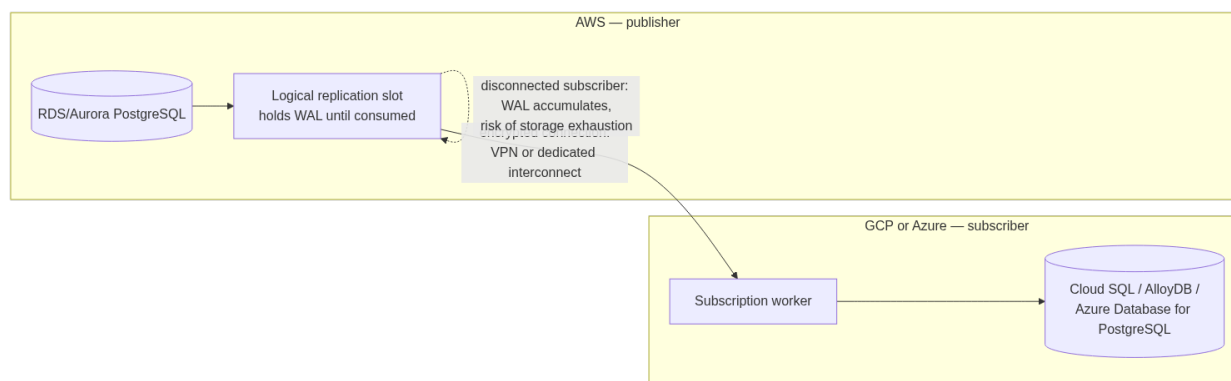


Figure 16: Cross-cloud logical replication in its simplest form: the publisher’s slot holds WAL until the remote subscriber confirms receipt, which is why slot monitoring — not just replication lag — is the critical operational signal.

25.2 Logical Replication: PostgreSQL Across AWS, Azure, and GCP

PostgreSQL logical replication uses a publisher/subscriber model. The publisher exposes a publication (a named set of tables), and subscribers connect to it using a replication slot, decode the WAL stream, and apply changes. This is conceptually the same as setting up `pg_hba.conf` to allow a replication user and granting `REPLICATION` privilege—except that in the managed service world, you configure connectivity through security groups, firewall rules, and IAM policies rather than filesystem-level configuration files.

Enabling logical replication on AWS RDS for PostgreSQL:

```
# Create or modify a parameter group to enable logical replication
aws rds create-db-parameter-group \
  --db-parameter-group-name prod-pg15-logical \
  --db-parameter-group-family postgres15 \
  --description "PostgreSQL 15 with logical replication enabled" \
  --region us-east-1

# Set wal_level to logical (requires reboot)
aws rds modify-db-parameter-group \
  --db-parameter-group-name prod-pg15-logical \
  --parameters \
    "ParameterName=rds.logical_replication,ParameterValue=1,ApplyMethod=pending-reboot" \
  --region us-east-1

# Apply the parameter group to the instance
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-primary \
  --db-parameter-group-name prod-pg15-logical \
  --apply-immediately \
  --region us-east-1

# Reboot to apply (schedule this during maintenance window)
aws rds reboot-db-instance \
  --db-instance-identifier prod-postgres-primary \
  --region us-east-1
```

On RDS, setting `rds.logical_replication = 1` automatically promotes `wal_level` to `logical` and sets `max_replication_slots` to at least 10 and `max_wal_senders` to at least 10. You do not set `wal_level` directly—RDS controls that parameter. This is one of the managed service constraints you adapt to rather than fight.

Enabling logical replication on Google Cloud SQL for PostgreSQL:

```
# Enable logical replication via Cloud SQL flags
gcloud sql instances patch prod-postgres-replica \
  --database-flags cloudsql.logical_decoding=on \
  --project my-gcp-project \
  --region us-central1

# Verify the flag was applied
gcloud sql instances describe prod-postgres-replica \
  --project my-gcp-project \
  --format="json(settings.databaseFlags)"
```

Setting up the publication on AWS RDS (publisher side):

```
-- Connect to the AWS RDS PostgreSQL instance
-- Create a dedicated replication user
CREATE ROLE replication_user WITH REPLICATION LOGIN PASSWORD 'use-a-vault-managed-secret';

-- Grant SELECT on all tables that will be replicated
GRANT SELECT ON ALL TABLES IN SCHEMA public TO replication_user;
```

```

ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO replication_user;

-- Create the publication
-- Publishing all tables is convenient but adds overhead for large schemas;
-- publishing specific tables is better for production
CREATE PUBLICATION cross_cloud_pub
  FOR TABLE orders, order_items, customers, inventory
  WITH (publish = 'insert, update, delete');

-- Verify the publication
SELECT pubname, puballtables, pubinsert, pubupdate, pubdelete
FROM pg_publication;

-- Create a replication slot for the GCP subscriber
-- Named slots track LSN position and ensure no WAL is discarded before the subscriber consumes it
SELECT pg_create_logical_replication_slot('gcp_subscriber_slot', 'pgoutput');

-- Monitor slot lag - this is critical; unconsumed slots cause WAL accumulation
SELECT slot_name,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)) AS
↪ replication_lag_size,
       active
FROM pg_replication_slots;

```

Setting up the subscription on Google Cloud SQL (subscriber side):

```

-- Connect to the Google Cloud SQL PostgreSQL instance
-- The schema must exist on the subscriber before the subscription starts
-- Use pg_dump --schema-only on the publisher or SCT to generate it

-- Create the subscription pointing to the AWS RDS publisher
-- The host is the RDS endpoint; connectivity must be established first (VPN or public)
CREATE SUBSCRIPTION cross_cloud_sub
  CONNECTION 'host=prod-postgres-primary.c3bxyz123.us-east-1.rds.amazonaws.com
             port=5432
             dbname=productiondb
             user=replication_user
             password=use-a-vault-managed-secret
             sslmode=require'
  PUBLICATION cross_cloud_pub
  WITH (
    slot_name = 'gcp_subscriber_slot',
    create_slot = false,           -- we already created it on the publisher
    enabled = true,
    copy_data = true              -- performs initial table sync before streaming
  );

-- Monitor subscription status
SELECT subname,
       pid,
       received_lsn,
       latest_end_lsn,
       pg_size_pretty(pg_wal_lsn_diff(latest_end_lsn, received_lsn)) AS lag
FROM pg_stat_subscription;

```

Critical production gotcha: replication slot WAL accumulation. If the GCP subscriber falls behind or disconnects, the replication slot on AWS RDS will prevent WAL segments from being recycled. A subscriber that is down for 6 hours on a write-heavy database can accumulate tens of gigabytes of WAL. RDS will run out of storage and become unavailable before it drops the slot. Set an alarm on the TransactionLogsDiskUsage CloudWatch metric and configure a slot monitoring job:

```

import boto3
import psycpg2

def check_replication_slot_lag(rds_host, port, dbname, user, password, slot_name, lag_threshold_gb=10):
    """

```

```

Monitor replication slot lag and publish to CloudWatch.
Run this every 5 minutes from a Lambda or EC2 instance.
"""
conn = psycopg2.connect(
    host=rds_host,
    port=port,
    dbname=dbname,
    user=user,
    password=password,
    sslmode='require'
)

cur = conn.cursor()
cur.execute("""
    SELECT slot_name,
           active,
           pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn) AS lag_bytes
    FROM pg_replication_slots
    WHERE slot_name = %s
""", (slot_name,))

row = cur.fetchone()
if not row:
    print(f"WARNING: Slot {slot_name} does not exist")
    return

slot_name_db, active, lag_bytes = row
lag_gb = lag_bytes / (1024 ** 3)

cw = boto3.client('cloudwatch', region_name='us-east-1')
cw.put_metric_data(
    Namespace='CrossCloudReplication',
    MetricData=[{
        'MetricName': 'ReplicationSlotLagGB',
        'Dimensions': [
            {'Name': 'SlotName', 'Value': slot_name},
            {'Name': 'Instance', 'Value': rds_host.split('.')[0]}
        ],
        'Value': lag_gb,
        'Unit': 'None'
    }]
)

if lag_gb > lag_threshold_gb:
    print(f"ALERT: Slot {slot_name} lag is {lag_gb:.2f} GB - exceeds {lag_threshold_gb} GB
    ↪ threshold")
    # Drop the slot if subscriber has been inactive for extended period
    # Only do this if you have a clear runbook for resynchronization
    if not active:
        print(f"Slot {slot_name} is INACTIVE. Consider dropping and resynchronizing.")

cur.close()
conn.close()

```

25.3 AWS DMS for Cross-Cloud CDC Replication

AWS Database Migration Service supports continuous CDC replication to and from non-AWS endpoints. Despite the name, DMS is not just for one-time migrations—its ongoing replication mode makes it viable for keeping a GCP or Azure database synchronized with an AWS source. The key limitation is that DMS applies changes with a latency of roughly 1–10 seconds under normal load, making it appropriate for near-real-time analytics replicas and DR targets but not for synchronous replication or read scaling.

Terraform configuration for a DMS cross-cloud replication task (AWS RDS PostgreSQL to Cloud SQL PostgreSQL):

```
# terraform/cross_cloud_dms.tf

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# DMS Replication Instance - runs in your VPC, needs NAT gateway for outbound to GCP
resource "aws_dms_replication_instance" "cross_cloud" {
  replication_instance_id      = "cross-cloud-dms-prod"
  replication_instance_class    = "dms.r6i.2xlarge" # 8 vCPU, 64 GB RAM - appropriate for ~5,000 TPS
  allocated_storage             = 500               # GB, for LOB column buffering
  engine_version                = "3.5.2"
  multi_az                     = true                # HA for the replication instance itself
  publicly_accessible           = false
  vpc_security_group_ids       = [aws_security_group.dms_sg.id]
  replication_subnet_group_id = aws_dms_replication_subnet_group.main.id

  tags = {
    Environment = "production"
    Owner       = "database-team"
  }
}

# Source endpoint: AWS RDS PostgreSQL
resource "aws_dms_endpoint" "source_rds" {
  endpoint_id      = "source-rds-postgres-prod"
  endpoint_type    = "source"
  engine_name      = "postgres"

  server_name = "prod-postgres-primary.c3bxyz123.us-east-1.rds.amazonaws.com"
  port        = 5432
  database_name = "productiondb"
  username     = "dms_user"
  password     = var.rds_dms_password # Use Secrets Manager in production
  ssl_mode     = "require"

  postgres_settings {
    capture_ddls      = false # DDL capture requires additional setup
    heartbeat_enable  = true
    heartbeat_frequency = 5    # seconds between heartbeats
    slot_name         = "dms_replication_slot"
    plugin_name       = "pglogical" # or "test_decoding" - pglogical is more reliable for DMS
  }
}

# Target endpoint: Google Cloud SQL PostgreSQL
# GCP Cloud SQL must have a public IP or Cloud SQL Proxy configured
# DMS connects over the internet or through VPN
resource "aws_dms_endpoint" "target_cloud_sql" {
  endpoint_id      = "target-cloud-sql-prod"
  endpoint_type    = "target"
  engine_name      = "postgres"

  server_name = "34.xxx.xxx.xxx" # Cloud SQL public IP with authorized network for DMS replication
  # instance's EIP
  port        = 5432
  database_name = "productiondb_replica"
  username     = "dms_target_user"
  password     = var.cloud_sql_dms_password
  ssl_mode     = "require"
}
```

```

# Replication task: full load + ongoing CDC
resource "aws_dms_replication_task" "cross_cloud_sync" {
  replication_task_id      = "prod-aws-to-gcp-sync"
  migration_type           = "full-load-and-cdc"
  replication_instance_arn = aws_dms_replication_instance.cross_cloud.replication_instance_arn
  source_endpoint_arn      = aws_dms_endpoint.source_rds.endpoint_arn
  target_endpoint_arn      = aws_dms_endpoint.target_cloud_sql.endpoint_arn

  table_mappings = jsonencode({
    rules = [
      {
        rule-type = "selection"
        rule-id   = "1"
        rule-name = "include-production-tables"
        object-locator = {
          schema-name = "public"
          table-name  = "%" # all tables - narrow this for large schemas
        }
        rule-action = "include"
      },
      {
        rule-type = "selection"
        rule-id   = "2"
        rule-name = "exclude-audit-logs"
        object-locator = {
          schema-name = "public"
          table-name  = "audit_%"
        }
        rule-action = "exclude"
      }
    ]
  })

  replication_task_settings = jsonencode({
    TargetMetadata = {
      TargetSchema      = "public"
      SupportLobs        = true
      FullLobMode       = false
      LobChunkSize      = 64 # KB - increase for large BYTEA columns
      LimitedSizeLobMode = true
      LobMaxSize        = 32768 # KB = 32 MB max LOB size
    }
    FullLoadSettings = {
      TargetTablePrepMode      = "DO_NOTHING" # TRUNCATE_BEFORE_LOAD or DO_NOTHING
      CreatePkAfterFullLoad   = false
      StopTaskCachedChangesApplied = false
      StopTaskCachedChangesNotApplied = false
      MaxFullLoadSubTasks     = 8
      TransactionConsistencyTimeout = 600
      CommitRate              = 50000 # rows per commit on target
    }
    Logging = {
      EnableLogging = true
      LogComponents = [
        { Id = "SOURCE_UNLOAD", Severity = "LOGGER_SEVERITY_DEFAULT" },
        { Id = "TARGET_LOAD",   Severity = "LOGGER_SEVERITY_DEFAULT" },
        { Id = "TASK_MANAGER",  Severity = "LOGGER_SEVERITY_DEFAULT" }
      ]
    }
    ControlTablesSettings = {
      historyTimeslotInMinutes = 5
      StatusTableEnabled      = true
      SuspendedTablesTableEnabled = true
    }
  })
}

```

DMS gotchas for cross-cloud replication:

First, DMS cannot replicate DDL changes in CDC mode for PostgreSQL by default. If a developer adds a column to a replicated table, the CDC stream will start failing for that table once rows with the new column arrive. You need a DDL change management process: apply the schema change to the target first, then apply it to the source, with a brief replication pause if the DDL is destructive.

Second, the DMS replication instance needs connectivity to both endpoints. For a private RDS source, you route through the VPC normally. For the Cloud SQL target, you have two options: enable Cloud SQL's public IP and whitelist the DMS replication instance's Elastic IP address in the Cloud SQL authorized networks list, or use Cloud SQL Auth Proxy running on an EC2 instance in the same VPC as DMS. The proxy approach is more secure and preferred for production.

Third, DMS requires that the PostgreSQL source has `wal_level = logical` and that the `pglogical` extension is installed. On RDS, `pglogical` is available as a supported extension—you create it with `CREATE EXTENSION pglogical`; once `rds.logical_replication = 1` is set. DMS creates and manages its own replication slot; you do not create it manually.

25.4 Azure SQL as a Cross-Cloud Replication Target

Azure SQL Database (the PaaS offering on SQL Server's engine) does not natively participate in PostgreSQL or MySQL logical replication. Getting data into or out of Azure SQL from another cloud's database engine requires one of two approaches: DMS (which supports Azure SQL as a target), or SQL Server's own replication mechanisms—transactional replication or the Azure Data Factory copy pipelines—when Azure SQL is involved on both ends.

For a common production scenario—AWS RDS for SQL Server replicating to Azure SQL Database for analytics or DR—you have a genuine option in SQL Server's native transactional replication, but only if the AWS side is RDS for SQL Server (not Aurora). The Azure SQL end acts as the subscriber. The challenge is that Azure SQL is always behind a fully qualified domain name in a Microsoft-managed network, and the publisher (RDS for SQL Server) must be able to reach the Azure SQL subscriber. You accomplish this through an Azure Private Link endpoint published to AWS via Transit Gateway or VPN.

For the more common scenario today—PostgreSQL on AWS to Azure Database for PostgreSQL for cross-cloud DR—logical replication works as described in Section 25.2. Azure Database for PostgreSQL Flexible Server supports logical replication from PostgreSQL 11+ and sets `wal_level = logical` through the `azure.replication_support` server parameter.

Enabling logical replication on Azure Database for PostgreSQL Flexible Server:

```
# Set replication support level using Azure CLI
az postgres flexible-server parameter set \
  --resource-group rg-database-prod \
  --server-name prod-postgres-azure \
  --name azure.replication_support \
  --value LOGICAL

# Restart the server to apply the parameter
az postgres flexible-server restart \
  --resource-group rg-database-prod \
  --name prod-postgres-azure

# Verify the parameter
az postgres flexible-server parameter show \
  --resource-group rg-database-prod \
```

```

--server-name prod-postgres-azure \
--name azure.replication_support \
--query value

# Add the AWS NAT Gateway IP to the Azure firewall rules
# (if using public internet connectivity rather than VPN)
az postgres flexible-server firewall-rule create \
  --resource-group rg-database-prod \
  --name prod-postgres-azure \
  --rule-name allow-aws-nat-gateway \
  --start-ip-address 54.xxx.xxx.xxx \
  --end-ip-address 54.xxx.xxx.xxx

```

Monitoring cross-cloud replication lag with Azure Monitor and CloudWatch together:

A mature cross-cloud replication operation monitors lag from both ends. The publisher (AWS) measures how much WAL is in the slot waiting to be consumed. The subscriber (Azure) measures how far behind its applied LSN is from the publisher's current LSN. Building a unified view requires pulling metrics from both clouds into a single observability system—Datadog, Grafana with multi-cloud datasources, or a custom pipeline.

```

import boto3
import psycopg2
from azure.monitor.query import MetricsQueryClient, MetricAggregationType
from azure.identity import DefaultAzureCredential
from datetime import datetime, timedelta, timezone

def get_cross_cloud_replication_health(
    rds_host: str,
    rds_creds: dict,
    azure_subscription_id: str,
    azure_resource_group: str,
    azure_server_name: str,
    slot_name: str
) -> dict:
    """
    Collect replication health metrics from both AWS (publisher) and Azure (subscriber).
    Returns a unified status dict suitable for alerting or dashboarding.
    """
    result = {
        'timestamp': datetime.now(timezone.utc).isoformat(),
        'publisher': {},
        'subscriber': {}
    }

    # --- AWS RDS: publisher-side metrics ---
    try:
        conn = psycopg2.connect(
            host=rds_host,
            dbname=rds_creds['dbname'],
            user=rds_creds['user'],
            password=rds_creds['password'],
            sslmode='require',
            connect_timeout=10
        )
        cur = conn.cursor()

        cur.execute("""
            SELECT
                slot_name,
                active,
                pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn) AS lag_bytes,
                confirmed_flush_lsn::text,
                pg_current_wal_lsn()::text AS current_lsn
            FROM pg_replication_slots
            WHERE slot_name = %s
        """, (slot_name,))

```

```

row = cur.fetchone()
if row:
    result['publisher'] = {
        'slot_name': row[0],
        'slot_active': row[1],
        'lag_bytes': row[2],
        'lag_gb': round(row[2] / (1024 ** 3), 3),
        'confirmed_flush_lsn': row[3],
        'current_lsn': row[4]
    }
else:
    result['publisher']['error'] = f"Slot {slot_name} not found"

cur.close()
conn.close()
except Exception as e:
    result['publisher']['error'] = str(e)

# --- Azure Database for PostgreSQL: subscriber-side metrics ---
# Azure Monitor publishes server-level metrics; replication-specific lag
# is best queried from the subscriber's pg_stat_subscription directly
try:
    azure_conn = psycopg2.connect(
        host=f"{azure_server_name}.postgres.database.azure.com",
        dbname=rds_creds['dbname'],
        user=rds_creds['user'],
        password=rds_creds['password'],
        sslmode='require',
        connect_timeout=10
    )
    azure_cur = azure_conn.cursor()

    azure_cur.execute("""
        SELECT
            subname,
            received_lsn::text,
            last_msg_send_time,
            last_msg_receipt_time,
            latest_end_lsn::text,
            EXTRACT(EPOCH FROM (now() - last_msg_receipt_time)) AS seconds_since_last_message
        FROM pg_stat_subscription
    """)

    rows = azure_cur.fetchall()
    result['subscriber']['subscriptions'] = [
        {
            'name': r[0],
            'received_lsn': r[1],
            'last_msg_send_time': r[2].isoformat() if r[2] else None,
            'last_msg_receipt_time': r[3].isoformat() if r[3] else None,
            'latest_end_lsn': r[4],
            'seconds_since_last_message': float(r[5]) if r[5] else None
        }
        for r in rows
    ]

    azure_cur.close()
    azure_conn.close()
except Exception as e:
    result['subscriber']['error'] = str(e)

# Compute composite health status
lag_gb = result['publisher'].get('lag_gb', 0)
slot_active = result['publisher'].get('slot_active', False)
subs = result['subscriber'].get('subscriptions', [])
seconds_stale = max(
    (s['seconds_since_last_message'] or 0) for s in subs
) if subs else None

if lag_gb > 10 or not slot_active or (seconds_stale and seconds_stale > 300):

```

```

    result['health'] = 'CRITICAL'
elif lag_gb > 2 or (seconds_stale and seconds_stale > 60):
    result['health'] = 'WARNING'
else:
    result['health'] = 'OK'

return result

```

This function is designed to be called from a Lambda (in the AWS account) that has network access to both the RDS instance and the Azure Database for PostgreSQL endpoint—typically via a VPN tunnel through which the Lambda’s VPC can reach the Azure virtual network.

25.5 Handling Schema Changes and DDL Propagation

Logical replication in PostgreSQL does not replicate DDL. This is not a limitation of managed services—it is a fundamental design choice in the PostgreSQL logical replication protocol. The publication/subscription mechanism streams row-level changes (INSERTs, UPDATEs, DELETEs) and sequence changes (from PostgreSQL 16+), but ALTER TABLE, CREATE INDEX, DROP COLUMN, and every other DDL statement are invisible to the subscriber. You manage schema evolution manually, and the process matters greatly for cross-cloud architectures because the communication overhead of coordinating changes across teams in different cloud environments is higher than coordinating within a single provider.

The safe DDL change sequence for cross-cloud logical replication:

The sequence depends on the type of DDL operation. Additive changes (adding a nullable column, creating an index) are safer than destructive changes (dropping a column, changing a column type).

For adding a nullable column:

```

-- Step 1: Apply on the SUBSCRIBER (Azure or GCP) first
-- The subscriber must be able to handle rows with the new column
-- before the publisher starts emitting them

-- On Azure Database for PostgreSQL (subscriber):
ALTER TABLE orders ADD COLUMN fulfillment_partner_id INTEGER;

-- Step 2: Apply on the PUBLISHER (AWS RDS)
-- Once the publisher starts sending rows with the new column,
-- the subscriber is already prepared to receive them

-- On AWS RDS PostgreSQL (publisher):
ALTER TABLE orders ADD COLUMN fulfillment_partner_id INTEGER;

-- No replication disruption occurs because PostgreSQL logical replication
-- maps columns by name. A column present in the publisher but missing in
-- the subscriber causes an error. A column present in the subscriber but
-- missing in the publisher is simply ignored (the subscriber column gets NULL).

```

For dropping a column, reverse the order:

```

-- Step 1: Drop on the PUBLISHER first
-- Once the publisher stops sending data for the column,
-- removing it from the subscriber is safe

-- On AWS RDS PostgreSQL (publisher):
ALTER TABLE orders DROP COLUMN legacy_carrier_code;

-- Step 2: Drop on the SUBSCRIBER
-- On Azure Database for PostgreSQL (subscriber):

```

```
ALTER TABLE orders DROP COLUMN legacy_carrier_code;
```

For type changes that affect wire representation (e.g., `VARCHAR(50)` to `TEXT`, or `INTEGER` to `BIGINT`), you must pause replication, apply the change on both ends, and resume. There is no safe online path for these changes in native logical replication because the encoded binary representation in the WAL changes. The practical approach:

```
-- This procedure applies to column type changes that alter binary encoding.
-- For changes that are wire-compatible (VARCHAR(50) to VARCHAR(100)), replication continues.

-- Step 1: Disable the subscription on the subscriber to pause replication
-- On Azure Database for PostgreSQL (subscriber):
ALTER SUBSCRIPTION cross_cloud_sub DISABLE;

-- Step 2: Record the current LSN on the publisher before applying DDL
-- On AWS RDS PostgreSQL (publisher):
SELECT pg_current_wal_lsn();
-- Save this value: e.g., '0/1A3F28C0'

-- Step 3: Apply the type change on the publisher
-- On AWS RDS PostgreSQL (publisher):
-- PostgreSQL does not support direct ALTER COLUMN TYPE with changed encoding in-place for replicated
-- tables.
-- Use the expand-contract pattern:
ALTER TABLE orders ADD COLUMN order_ref_v2 BIGINT;
UPDATE orders SET order_ref_v2 = order_ref::BIGINT; -- batch this for large tables
-- At this point order_ref (INTEGER) is still the replicated column

-- Step 4: Apply the same schema change on the subscriber
-- On Azure Database for PostgreSQL (subscriber):
ALTER TABLE orders ADD COLUMN order_ref_v2 BIGINT;

-- Step 5: Add the new column to the publication
-- On AWS RDS PostgreSQL (publisher):
-- If you used FOR TABLE with explicit columns, update the publication.
-- If you used FOR ALL TABLES or FOR TABLE orders (all columns), it's included automatically.

-- Step 6: Re-enable the subscription
-- On Azure Database for PostgreSQL (subscriber):
ALTER SUBSCRIPTION cross_cloud_sub ENABLE;

-- Step 7: After validating replication of order_ref_v2,
-- drop the old column following the safe drop sequence above.
```

Automating DDL propagation with an event trigger:

For teams that prefer automated DDL capture, PostgreSQL event triggers can log DDL statements to a side table, and a separate process can replay them on the subscriber after a validation step. This is not plug-and-play—it requires careful handling of object dependencies and ordering—but it is the approach used by tools like pglogical 2.x and Bucardo.

```
-- On the AWS RDS PostgreSQL publisher:
-- Create a DDL audit table
CREATE TABLE ddl_replication_log (
    id BIGSERIAL PRIMARY KEY,
    event_time TIMESTAMPTZ NOT NULL DEFAULT now(),
    command_tag TEXT NOT NULL,
    object_type TEXT,
    schema_name TEXT,
    object_name TEXT,
    ddl_text TEXT,
    applied_to_subscriber BOOLEAN NOT NULL DEFAULT false
);

-- Create an event trigger function
CREATE OR REPLACE FUNCTION log_ddl_for_replication()
RETURNS event_trigger
```



```
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
    r RECORD;
BEGIN
    FOR r IN SELECT * FROM pg_event_trigger_ddl_commands()
    LOOP
        INSERT INTO ddl_replication_log (command_tag, object_type, schema_name, object_name, ddl_text)
        VALUES (
            r.command_tag,
            r.object_type,
            r.schema_name,
            r.objid::regclass::text,
            current_query()
        );
    END LOOP;
END;
$$;

-- Attach the event trigger to DDL events
CREATE EVENT TRIGGER capture_ddl_for_cross_cloud
ON ddl_command_end
EXECUTE FUNCTION log_ddl_for_replication();
```

A scheduled job on the subscriber side polls `ddl_replication_log` (via a separate read connection to the publisher, not through the replication stream), reviews pending DDL entries marked `applied_to_subscriber = false`, applies them in order, and marks them applied. The job should include a human approval step for destructive DDL in production. This is the foundation of a DDL replication workflow—teams add validation, Slack notifications, and rollback procedures on top.

25.6 Failover and Switchover in Cross-Cloud Architectures

Replication is infrastructure; failover is the operational procedure that uses it. Cross-cloud replication introduces complexity to failover that same-cloud DR does not: DNS changes propagate through a different provider's infrastructure, the application tier on one cloud must redirect to a database on another cloud, and the cost of cross-cloud failover in terms of latency must be validated before you rely on it.

The two failure modes you must plan for:

1. **Publisher failure** (primary cloud becomes unavailable). The subscriber has data up to the last consumed LSN. There is no LSN on the subscriber you can promote from because PostgreSQL logical replication does not establish the subscriber as a standby in the streaming replication sense—it is a separate database cluster. Promotion here means pointing your application connection strings at the subscriber and accepting that any transactions that were committed on the publisher after the last consumed LSN are lost (RPO = replication lag at time of failure).
2. **Subscriber failure** (replica cloud becomes unavailable). The publisher continues operating normally. The replication slot accumulates WAL. When the subscriber recovers, replication resumes from the slot's last confirmed LSN. The risk here is WAL accumulation causing publisher storage exhaustion, as discussed in Section 25.2.

Automated failover runbook for publisher failure:

```
#!/bin/bash
# cross_cloud_failover.sh
# Promotes the Azure/GCP logical replication subscriber to primary role.
# Prerequisites:
#   - PGPASSWORD or .pgpass configured for subscriber connection
#   - Application connection string managed through Route 53 or Azure Traffic Manager
#   - Replication lag was below RPO threshold at time of publisher failure

set -euo pipefail

SUBSCRIBER_HOST="${SUBSCRIBER_HOST:-prod-postgres-azure.postgres.database.azure.com}"
SUBSCRIBER_PORT="${SUBSCRIBER_PORT:-5432}"
SUBSCRIBER_DB="${SUBSCRIBER_DB:-productiondb}"
SUBSCRIBER_USER="${SUBSCRIBER_USER:-dbadmin}"
SUB_NAME="cross_cloud_sub"

echo "$(date -u): Starting cross-cloud failover promotion"

# Step 1: Verify the publisher is unreachable (don't fail over if it's a false alarm)
PUBLISHER_HOST="prod-postgres-primary.c3bxyz123.us-east-1.rds.amazonaws.com"
if pg_isready -h "${PUBLISHER_HOST}" -p 5432 -t 10 2>/dev/null; then
    echo "ERROR: Publisher appears to be reachable. Aborting failover to prevent split-brain."
    echo "Verify the outage independently before proceeding."
    exit 1
fi

echo "$(date -u): Publisher is confirmed unreachable. Proceeding with failover."

# Step 2: Drop the subscription on the subscriber
# This removes the replication connection and allows the subscriber to accept writes.
# The subscriber's data represents the last LSN consumed from the publisher.
psql \
    -h "${SUBSCRIBER_HOST}" \
    -p "${SUBSCRIBER_PORT}" \
    -U "${SUBSCRIBER_USER}" \
    -d "${SUBSCRIBER_DB}" \
    -c "DROP SUBSCRIPTION ${SUB_NAME};" \
    --set ON_ERROR_STOP=on

echo "$(date -u): Subscription dropped. Subscriber is now a standalone primary."

# Step 3: Record the last received LSN for post-failover reconciliation
psql \
    -h "${SUBSCRIBER_HOST}" \
    -p "${SUBSCRIBER_PORT}" \
    -U "${SUBSCRIBER_USER}" \
    -d "${SUBSCRIBER_DB}" \
    -c "SELECT pg_current_wal_lsn() AS promoted_at_lsn, now() AS promoted_at;" \
    --set ON_ERROR_STOP=on

# Step 4: Update the application connection strings
# This is environment-specific. Common approaches:
#   - Update AWS Route 53 CNAME record for db.internal.mycompany.com
#   - Update Azure Traffic Manager profile
#   - Update Kubernetes ConfigMap and trigger rolling pod restart
#   - Update AWS Systems Manager Parameter Store and trigger Lambda to notify app

echo "$(date -u): Update application connection strings to: ${SUBSCRIBER_HOST}"
echo "$(date -u): Example Route 53 update:"
echo "  aws route53 change-resource-record-sets \"\"
echo "    --hosted-zone-id Z1234567890ABC \"\"
echo "    --change-batch file://failover-dns-change.json"

# Step 5: Validate write capability on the promoted subscriber
psql \
    -h "${SUBSCRIBER_HOST}" \
    -p "${SUBSCRIBER_PORT}" \
    -U "${SUBSCRIBER_USER}" \
    -d "${SUBSCRIBER_DB}" \
    -c "INSERT INTO failover_events (event_type, event_time, notes)"
```

```
VALUES ('cross_cloud_failover', now(), 'Promoted from logical replication subscriber');" \
--set ON_ERROR_STOP=on

echo "$(date -u): Failover complete. Validate application connectivity and data integrity."
echo "$(date -u): Begin post-failover runbook: reconcile lost transactions, notify stakeholders."
```

Re-establishing replication after failover (switchback):

After the original publisher recovers (or after you build a new primary in the original cloud), you must rebuild replication from scratch. There is no automatic resynchronization in PostgreSQL logical replication—you cannot simply re-enable a subscription to a recovered publisher and have it seamlessly pick up from where it left off, because the former subscriber (now the active primary) has diverged. The steps are:

1. Take a logical dump of the current primary (former subscriber) using `pg_dump`.
2. Restore it to the new replica instance (former publisher or a fresh instance).
3. Create a new publication on the current primary and a new subscription on the new replica.
4. Let the initial table sync (`copy_data = true`) bring the replica to current state before enabling streaming replication.

This process can take hours for large databases, which is why cross-cloud RPO and RTO targets must account for resynchronization time, not just immediate failover time.

25.7 Conflict Resolution in Bidirectional Cross-Cloud Replication

Most cross-cloud architectures are unidirectional: one cloud is the writer, one cloud is the reader/DR target. Bidirectional replication—where both ends accept writes—introduces conflict resolution requirements that significantly increase complexity. This is worth discussing because teams ask for it frequently (“can we write to AWS in us-east-1 and also write to GCP in europe-west1 with bidirectional sync?”) and the honest answer is: yes, but the operational burden is substantial.

PostgreSQL’s native logical replication has no built-in conflict resolution. If row ID 42 is updated on AWS at T1 and also updated on GCP at T2 where T1 and T2 are close together and the replication lag exceeds T2-T1, both updates succeed locally and the replication stream will attempt to apply each conflicting update to the other side. The subscriber will throw an error like:

```
ERROR:  duplicate key value violates unique constraint "orders_pkey"
```

or, for UPDATE conflicts:

```
ERROR:  could not find row to update in table "orders"
```

At this point, the subscription worker stops. You are now in manual conflict resolution territory.

Strategies for handling bidirectional conflicts:

Last-writer-wins with application timestamps. Include a `last_modified_at` `TIMESTAMPTZ` column in every replicated table. Write application-level triggers or a custom logical replication apply function (available in PostgreSQL 15+ with custom apply handlers) that compares timestamps and keeps the later modification.

```
-- PostgreSQL 15+ custom conflict resolution function
-- Applied on the subscriber side to handle UPDATE conflicts
```

```
CREATE OR REPLACE FUNCTION resolve_update_conflict()
RETURNS void
LANGUAGE plpgsql
AS $$
BEGIN
    -- This function is called by the logical replication worker when
    -- configured as the conflict handler. The actual API for custom
    -- apply functions is implemented via pg_logical or third-party
    -- extensions in PostgreSQL 15; native support is experimental.
    -- Illustrative pattern only - production implementations use
    -- pglogical 2.x or Citus for managed bidirectional conflict resolution.
    NULL;
END;
$$;
```

Partition writes by cloud. A cleaner approach for most real-world bidirectional needs: partition your write traffic so that certain entities are always owned by a specific cloud. Customers in Europe are written only to GCP europe-west1; customers in North America are written only to AWS us-east-1. Replication carries the data to both sides for read access, but writes are never truly concurrent on the same row across clouds. Conflict resolution becomes unnecessary because you have eliminated the condition that creates conflicts.

Use a tool with built-in conflict resolution. pglogical 2.x (the commercial extension from EDB, or the open-source version) supports configurable conflict resolution policies: `error` (default), `apply_remote` (remote always wins), `keep_local` (local always wins), `last_update_wins` (timestamp comparison). Bucardo supports per-table conflict resolution functions. For teams that need genuine bidirectional replication without building a custom conflict handler, these tools are worth the operational overhead of deploying them alongside the managed service endpoints.

25.8 Cost Modeling for Cross-Cloud Replication

Replication traffic has a cost that is easy to underestimate in architecture reviews. Data egress from AWS costs per-GB (rates vary by region and destination; inter-region and internet egress are charged, while intra-region transfer within the same AZ is free). GCP and Azure have similar egress pricing models. When your WAL stream is moving gigabytes per hour across providers, the egress charges accumulate quickly.

Estimating cross-cloud replication egress cost:

```
def estimate_cross_cloud_replication_cost(
    write_throughput_mbps: float,
    wal_amplification_factor: float = 2.5,
    egress_cost_per_gb: float = 0.09, # AWS to internet, per GB - verify current pricing
    hours_per_month: int = 730
) -> dict:
    """
    Estimate monthly egress cost for cross-cloud replication.

    write_throughput_mbps: Application write rate in megabits per second.
    wal_amplification_factor: WAL generated per byte of actual write data.
        Typically 2x-4x for row-level changes; higher for small, frequent updates.
    egress_cost_per_gb: Cloud provider egress rate per GB.
    """
    wal_throughput_mbps = write_throughput_mbps * wal_amplification_factor
    wal_throughput_mb_per_hour = wal_throughput_mbps * 3600 / 8 # convert mbps to MB/hour
    wal_throughput_gb_per_month = (wal_throughput_mb_per_hour * hours_per_month) / 1024
```

```

monthly_egress_cost = wal_throughput_gb_per_month * egress_cost_per_gb

return {
    'write_throughput_mbps': write_throughput_mbps,
    'wal_throughput_mbps': wal_throughput_mbps,
    'estimated_wal_gb_per_month': round(wal_throughput_gb_per_month, 1),
    'estimated_monthly_egress_cost_usd': round(monthly_egress_cost, 2),
    'notes': (
        "WAL amplification varies by workload. "
        "TOAST-heavy workloads (large text/JSONB) amplify less; "
        "high-update workloads on narrow rows amplify more. "
        "Verify egress pricing against current cloud provider rate cards."
    )
}

# Example: 10 Mbps application write rate
estimate = estimate_cross_cloud_replication_cost(
    write_throughput_mbps=10,
    wal_amplification_factor=3.0
)
# Result: ~30 Mbps of WAL traffic, translating to several thousand GB/month -
# plug in current egress pricing for your provider and region to get a dollar figure
print(estimate)

```

Beyond egress, factor in:

- **DMS replication instance cost:** an instance class like `dms.r6i.2xlarge` has a fixed hourly cost regardless of throughput — check current AWS DMS pricing for your region before budgeting.
- **Direct Connect or VPN:** As noted in Section 25.1, dedicated circuits add a recurring cost but eliminate per-GB egress charges between endpoints on the private circuit. For high-throughput replication, there is a crossover point where a dedicated circuit becomes cheaper than public egress — do the arithmetic for your specific throughput and current circuit pricing rather than assuming a threshold.
- **Monitoring and tooling:** Slot monitoring Lambda executions, CloudWatch custom metrics, Datadog per-host fees for the DMS instance—these are minor individually but add up.

The honest architectural advice: model the total cost of cross-cloud replication before committing to it. For many teams, the DR or compliance value justifies the cost. For others, it prompts a redesign toward a single-cloud multi-region architecture where egress stays intra-provider and costs are lower.

Key Takeaways

- **No managed database service natively replicates to another cloud’s engine.** Every cross-cloud replication architecture uses logical replication (for same-engine, same-version scenarios), CDC via a tool like AWS DMS or Debezium (for cross-engine or cross-version scenarios), or application-level dual-write. Choose the approach that matches your RPO tolerance and schema complexity before writing any configuration.
- **Replication slot WAL accumulation is the most common production failure mode for cross-cloud logical replication.** When the subscriber disconnects or falls behind, the publisher’s replication slot prevents WAL recycling. Monitor slot lag continuously with a CloudWatch custom metric or equivalent, set a storage alarm well below disk capacity, and

maintain a documented runbook for dropping and resynchronizing a lagged slot before it causes publisher unavailability.

- **DDL changes are invisible to the logical replication stream and require a coordinated manual process.** Additive DDL (adding nullable columns) must be applied to the subscriber first; destructive DDL (dropping columns) must be applied to the publisher first. Type changes that alter wire encoding require pausing the subscription, applying changes on both ends, and resuming. An event trigger DDL audit log is the practical foundation for teams needing automation in this area.
 - **Cross-cloud failover is a distinct operation from same-cloud promotion.** Promoting a logical replication subscriber to primary means dropping the subscription (which removes the replication connection and allows writes), recording the last applied LSN, and redirecting application traffic. There is no automatic resynchronization when the original publisher recovers—you must rebuild replication from a logical dump, which is the dominant factor in RTO for this architecture.
 - **Egress cost is a first-class architectural constraint, not an afterthought.** WAL data volumes for write-heavy workloads at cross-cloud egress rates can reach significant monthly costs before accounting for DMS instance fees, dedicated circuit costs, and monitoring overhead. Model total replication cost against the value of the cross-cloud architecture before committing to it, and revisit the model when write throughput grows substantially.
-

Chapter 26: Data Sovereignty and Residency Compliance

Data sovereignty — the principle that data is subject to the laws and governance structures of the nation where it physically resides — has transformed from a legal abstraction into an operational constraint that every cloud DBA must architect around. GDPR in the European Union, the Personal Data Protection Bill in India, LGPD in Brazil, PIPL in China, and sector-specific mandates like HIPAA and FedRAMP in the United States all impose requirements that translate directly into database placement decisions, replication topology restrictions, encryption key custody, and audit logging configurations. If you’ve spent your career managing on-premises databases, you controlled data residency by controlling which rack the server sat in. In the cloud, that control is exercised through region selection, availability zone pinning, replication constraints, and — critically — understanding what the managed service does automatically on your behalf that you may not have realized. This chapter maps the compliance landscape to the actual configuration knobs available in AWS RDS, Azure SQL, and Google Cloud SQL, and shows you how to enforce residency guarantees that will survive a regulator’s audit.

26.1 Understanding the Regulatory Landscape for Cloud DBAs

Before configuring anything, you need to understand what regulators actually care about, because the mandates are more specific — and more operationally demanding — than most DBAs expect until they’re sitting across from an auditor.

What regulators want to know:

- Where does the data physically reside at rest?
- Where does the data travel during replication, backup, and recovery operations?
- Who holds the encryption keys, and where are those keys stored?
- Who can access the data — including cloud vendor employees?
- Can you prove all of the above with durable audit logs?

GDPR Article 44 prohibits transfers of personal data to third countries unless specific adequacy decisions or contractual protections are in place. This applies to your database backups. AWS automated backups for RDS are stored in S3 within the same region by default, but cross-region automated backup copies, Read Replicas in other regions, and cross-region snapshot copies are all “data transfers” under a strict GDPR reading. The fact that the transfer is automated and invisible doesn’t make it compliant by default — it makes it a trap.

The sovereignty matrix for cloud DBAs:

Requirement	On-Prem Equivalent	Cloud Mechanism
Data at rest in country X	Server physically in country X	Region selection + no cross-region replication
Encryption key custody	HSM in your data center	Customer-managed keys (CMK) with regional KMS
Access logging	Audit plugin, pgaudit	CloudTrail, Azure Audit Logs, Cloud Audit Logs
Vendor employee access restriction	Physical security, OS ACLs	Nitro enclaves, Confidential Computing, access controls
Backup residency	Tape library location	Backup region configuration, cross-region copy policies

The managed service assumption problem:

When you self-managed PostgreSQL, you decided explicitly where every file went. RDS, Azure SQL, and Cloud SQL all make decisions on your behalf — automated backups, transaction log backups, minor version patching, and parameter group replication all involve data movement and access that happens without a specific operator action. Your job as a cloud DBA is to audit every one of those automatic behaviors against your compliance requirements before you're in production.

A common shock: AWS RDS Multi-AZ with synchronous replication stays within a single region, but the two AZs may be in different cities. For most regulations, this is fine — they're within the same legal jurisdiction. But some financial regulators in specific countries have interpreted “data residency” down to the metropolitan area. Know your specific regulatory language before assuming Multi-AZ is compliant.

26.2 AWS RDS and Aurora: Region, Replication, and Key Custody

AWS gives you fine-grained control over data residency, but you must exercise that control explicitly. The defaults are permissive, not restrictive.

Region selection and subnet architecture:

Every RDS or Aurora instance is deployed into a VPC subnet, which lives in a specific Availability Zone within a specific region. The region is the top-level residency boundary. For EU data subject to GDPR, `eu-west-1` (Ireland), `eu-central-1` (Frankfurt), `eu-south-1` (Milan), and `eu-north-1` (Stockholm) are all within the EU, but they are different legal jurisdictions for some regulatory interpretations. German financial regulators and French data authorities have historically preferred data to remain within their national borders — not just the EU broadly.

```
# Create an RDS PostgreSQL instance explicitly pinned to eu-central-1 (Frankfurt)
# and document the region choice in tags for audit purposes
```

```
aws rds create-db-instance \
  --db-instance-identifier prod-gdpr-postgres \
  --db-instance-class db.r6g.2xlarge \
  --engine postgres \
  --engine-version 15.4 \
```



```

--master-username dbadmin \
--master-user-password "$(aws secretsmanager get-secret-value \
  --secret-id prod/postgres/masterpassword \
  --region eu-central-1 \
  --query SecretString \
  --output text)" \
--db-subnet-group-name prod-eu-central-private-subnet-group \
--vpc-security-group-ids sg-0a1b2c3d4e5f67890 \
--storage-encrypted \
--kms-key-id arn:aws:kms:eu-central-1:123456789012:key/mrk-abc123 \
--backup-retention-period 7 \
--preferred-backup-window "02:00-03:00" \
--no-publicly-accessible \
--region eu-central-1 \
--tags \
  Key=DataResidency,Value=EU-Germany \
  Key=ComplianceFramework,Value=GDPR \
  Key=DataClassification,Value=PersonalData \
  Key=RegionLocked,Value=true

```

Customer-managed keys and key residency:

This is where most DBAs underestimate the complexity. AWS KMS regional keys are region-specific — a key created in `eu-central-1` cannot be used to encrypt data in `us-east-1`. This is good for residency, because it forces you to use a region-local key for your data. But AWS KMS Multi-Region Keys (MRKs) can replicate key material across regions, which is a compliance problem if your regulation requires key material to remain in-country.

For the strictest sovereignty requirements, use AWS CloudHSM deployed in your target region, then configure KMS to use CloudHSM as its custom key store. Key material never leaves the HSM, and the HSM is physically in a specific AWS data center within your region.

```

# Verify that an existing RDS instance is using a CMK (not AWS-managed key)
# and confirm the key is regional, not a multi-region key

aws rds describe-db-instances \
  --db-instance-identifier prod-gdpr-postgres \
  --region eu-central-1 \
  --query 'DBInstances[0].{
    KmsKeyId: KmsKeyId,
    StorageEncrypted: StorageEncrypted,
    DBInstanceArn: DBInstanceArn
  }'

# Check if the KMS key is a multi-region key (MultiRegion: true would be a compliance flag)
aws kms describe-key \
  --key-id arn:aws:kms:eu-central-1:123456789012:key/mrk-abc123 \
  --region eu-central-1 \
  --query 'KeyMetadata.{
    KeyId: KeyId,
    MultiRegion: MultiRegion,
    KeyState: KeyState,
    KeyManager: KeyManager,
    Origin: Origin
  }'

# Origin: AWS_KMS = key material in KMS
# Origin: EXTERNAL = you imported key material (highest control)
# Origin: AWS_CLOUDHSM = key material in CloudHSM (strongest for compliance)

```

Preventing cross-region replication and backup copies:

The most dangerous residency violation is an automated process that no one remembers setting up. Use AWS Service Control Policies (SCPs) at the AWS Organizations level to deny cross-region RDS actions for accounts handling sovereign data.

```
# SCP to prevent cross-region RDS snapshot copies OUT of eu-central-1
# Apply this to the OU containing GDPR-scoped accounts
# Save as gdpr-rds-residency-scp.json and attach via Organizations
```

```
cat > gdpr-rds-residency-scp.json << 'EOF'
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyRDSCrossRegionSnapshotCopy",
      "Effect": "Deny",
      "Action": [
        "rds:CopyDBSnapshot",
        "rds:CopyDBClusterSnapshot"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": "eu-central-1"
        }
      }
    },
    {
      "Sid": "DenyRDSCrossRegionReadReplica",
      "Effect": "Deny",
      "Action": [
        "rds:CreateDBInstanceReadReplica"
      ],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": "eu-central-1"
        }
      }
    },
    {
      "Sid": "DenyRDSGlobalCluster",
      "Effect": "Deny",
      "Action": [
        "rds:CreateGlobalCluster",
        "rds:AddMemberToGlobalCluster"
      ],
      "Resource": "*"
    }
  ]
}
EOF
```

```
# Attach to the target organizational unit
aws organizations create-policy \
  --name "GDPR-RDS-Residency-Enforcement" \
  --description "Prevents RDS data from leaving eu-central-1" \
  --content file://gdpr-rds-residency-scp.json \
  --type SERVICE_CONTROL_POLICY

aws organizations attach-policy \
  --policy-id p-examplepolicyid \
  --target-id ou-exampleouid
```

Aurora Global Database and sovereignty:

Aurora Global Database provides sub-second replication across regions for disaster recovery — which is architecturally attractive but potentially legally problematic. If you operate a Global Database with a primary in `eu-central-1` and a secondary in `us-east-1`, you have EU personal data replicating to US servers. This may violate GDPR without appropriate data transfer mechanisms (Standard Contractual Clauses, Binding Corporate Rules) in place. The SCP above prevents this at the organizational level. If you need cross-region DR within compliant boundaries

(e.g., `eu-central-1` to `eu-west-1`), that’s permissible for most EU regulations — but document it explicitly with your Data Protection Officer.

26.3 Azure SQL: Geo-Replication, Paired Regions, and Data Boundary

Microsoft has invested heavily in its Azure Data Boundary commitment — a contractual and technical guarantee that customer data stored and processed in the EU stays within the EU. For cloud DBAs, this is useful, but it doesn’t eliminate the need for explicit configuration.

Azure SQL geo-redundant backup storage:

When you create an Azure SQL Database, the default backup storage redundancy is **Geo** — meaning your backups are replicated to the Azure paired region automatically. For **West Europe** (Netherlands), the paired region is **North Europe** (Ireland). Both are in the EU, so GDPR compliance is maintained for EU data subjects. But if you’re in **Brazil South**, the paired region is **Brazil Southeast**. Both are in Brazil, satisfying LGPD. Know your paired regions before accepting the default.

For regulations that prohibit any cross-location replication (some government cloud mandates), set backup storage redundancy to **Zone** or **Local**:

```
# Create Azure SQL Database with zone-redundant backup storage
# (no cross-region backup replication)
```

```
az sql db create \
  --resource-group rg-prod-eu \
  --server sql-prod-west europe \
  --name gdpr-production-db \
  --edition GeneralPurpose \
  --compute-model Provisioned \
  --family Gen5 \
  --capacity 8 \
  --zone-redundant true \
  --backup-storage-redundancy Zone \
  --location west europe

# Verify the backup storage redundancy setting
az sql db show \
  --resource-group rg-prod-eu \
  --server sql-prod-west europe \
  --name gdpr-production-db \
  --query '{
    name: name,
    location: location,
    backupStorageRedundancy: currentBackupStorageRedundancy,
    zoneRedundant: zoneRedundant
  }'
```

Azure SQL Always Encrypted for column-level sovereignty:

Always Encrypted is the Azure SQL equivalent of a technical guarantee that the cloud provider — including Microsoft employees — cannot read sensitive column values. Encryption and decryption happen exclusively in the client application. The database engine stores and retrieves ciphertext and never sees plaintext values. For regulated industries where vendor access is the residency concern (not geographic placement), this is a powerful tool.

Always Encrypted uses a two-tier key hierarchy: Column Master Keys (CMK) stored in Azure Key Vault, and Column Encryption Keys (CEK) encrypted with the CMK. Your Key Vault can be configured with HSM-backed keys for the strongest custody guarantee.

```
-- Configure Always Encrypted on Azure SQL for a PII table
-- This runs in SSMS or Azure Data Studio with the Always Encrypted wizard
-- or can be scripted via PowerShell/az CLI

-- Step 1: Create the Column Master Key reference in the database
-- (actual key material is in Azure Key Vault, not the database)
CREATE COLUMN MASTER KEY CMK_AzureKeyVault
WITH (
    KEY_STORE_PROVIDER_NAME = 'AZURE_KEY_VAULT',
    KEY_PATH = 'https://prod-keyvault-eu.vault.azure.net/keys/AlwaysEncryptedCMK/abc123def456'
);

-- Step 2: Create the Column Encryption Key, encrypted with the CMK
CREATE COLUMN ENCRYPTION KEY CEK_PersonalData
WITH VALUES (
    COLUMN_MASTER_KEY = CMK_AzureKeyVault,
    ALGORITHM = 'RSA_OAEP',
    ENCRYPTED_VALUE = 0x016E... -- Generated by client tooling with access to Key Vault
);

-- Step 3: Create the table with encrypted columns
-- Deterministic encryption: allows equality comparisons (for WHERE clauses)
-- Randomized encryption: stronger, but no WHERE/ORDER BY/GROUP BY on the column
CREATE TABLE dbo.CustomerPersonalData (
    CustomerID INT IDENTITY(1,1) PRIMARY KEY,
    FullName NVARCHAR(200) COLLATE Latin1_General_BIN2
        ENCRYPTED WITH (
            COLUMN_ENCRYPTION_KEY = CEK_PersonalData,
            ENCRYPTION_TYPE = Deterministic,
            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
        ) NOT NULL,
    NationalID NVARCHAR(20) COLLATE Latin1_General_BIN2
        ENCRYPTED WITH (
            COLUMN_ENCRYPTION_KEY = CEK_PersonalData,
            ENCRYPTION_TYPE = Randomized,
            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
        ) NULL,
    DateOfBirth DATE
        ENCRYPTED WITH (
            COLUMN_ENCRYPTION_KEY = CEK_PersonalData,
            ENCRYPTION_TYPE = Randomized,
            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
        ) NULL,
    CreatedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);

-- The database engine sees only ciphertext in FullName, NationalID, DateOfBirth
-- Azure support engineers with direct server access cannot read these values
-- SELECT * FROM dbo.CustomerPersonalData will return encrypted bytes to non-AE clients
```

Azure Policy for residency enforcement:

Like AWS SCPs, Azure Policy can enforce residency at the management group or subscription level, preventing resources from being created in non-compliant regions.

```
# Create and assign an Azure Policy that restricts SQL resources to West Europe only
# Built-in policy: "Allowed locations" - policy definition ID varies by tenant

# First, get the built-in policy definition for "Allowed locations"
POLICY_DEF_ID=$(az policy definition list \
    --query "[?displayName=='Allowed locations'].id" \
    --output tsv)

# Assign the policy to a resource group, restricting to westeurope only
```

```
az policy assignment create \  
  --name "sql-residency-vesteurope" \  
  --display-name "SQL Resources Must Stay in West Europe" \  
  --scope "/subscriptions/sub-id-here/resourceGroups/rg-prod-eu" \  
  --policy "$POLICY_DEF_ID" \  
  --params '{  
    "listOfAllowedLocations": {  
      "value": ["vesteurope"]  
    }  
  }' \  
  --enforcement-mode Default  
  
# Verify the assignment  
az policy assignment show \  
  --name "sql-residency-vesteurope" \  
  --scope "/subscriptions/sub-id-here/resourceGroups/rg-prod-eu" \  
  --query '{  
    name: name,  
    enforcementMode: enforcementMode,  
    parameters: parameters  
  }'
```

Active Geo-Replication and legal review:

Azure SQL Active Geo-Replication lets you create up to four readable secondary databases in any region. Unlike AWS Global Clusters, this is configured per-database rather than per-cluster, giving you finer control. But the sovereignty risk is identical: verify that all secondary regions are within legally compliant jurisdictions before enabling it. For financial institutions in the EU operating under DORA (Digital Operational Resilience Act), cross-region replication may be required for resilience — confirm the current DORA text and your regulator’s guidance on permitted DR regions before finalizing a topology, as ICT resilience regulation in this area continues to evolve.

26.4 Google Cloud SQL, AlloyDB, and Spanner: Residency Controls

Google Cloud’s approach to data residency spans three products with significantly different residency capabilities.

Cloud SQL: regional by design, with explicit controls:

Cloud SQL instances are regional. Unlike Cloud Spanner, they don’t span regions by default. Cross-region read replicas are opt-in, and automated backups stay in the same region as the instance. For most sovereignty requirements, Cloud SQL’s defaults are residency-safe, but verify them explicitly.

```
# Create a Cloud SQL PostgreSQL instance with explicit region pinning  
# and verify residency configuration
```

```
gcloud sql instances create prod-gdpr-postgres \  
  --database-version=POSTGRES_15 \  
  --tier=db-custom-8-32768 \  
  --region=europe-west3 \  
  --availability-type=REGIONAL \  
  --storage-type=SSD \  
  --storage-size=500GB \  
  --storage-auto-increase \  
  --backup-start-time=02:00 \  
  --enable-bin-log \  
  --retained-backups-count=7 \  
  --deletion-protection
```

```

--no-assign-ip \
--network=projects/my-project/global/networks/prod-vpc \
--labels=data-residency=eu-germany,compliance=gdpr,data-classification=personal

# Verify the instance location and backup configuration
gcloud sql instances describe prod-gdpr-postgres \
  --format="yaml(name,region,settings.backupConfiguration,settings.locationPreference)"

# List all replicas for this instance to verify none are cross-region
gcloud sql instances list \
  --filter="masterInstanceName:prod-gdpr-postgres" \
  --format="table(name,region,databaseVersion,state)"

```

Assured Workloads for regulatory compliance:

Google Cloud Assured Workloads is the platform-level mechanism for enforcing data residency and access controls for regulated workloads. It restricts which Google Cloud services and regions can be used within a folder, enforces organization policies, and — for sovereign cloud configurations — restricts Google personnel access to a subset of employees in specific jurisdictions.

For EU sovereign requirements, the “EU Regions and Support” configuration restricts your folder to EU regions and ensures that Google support staff who can access your data are located in the EU. This addresses the vendor employee access concern that regulators increasingly scrutinize.

```

# Enable Assured Workloads folder for EU sovereign configuration
# This requires Organization Admin permissions

gcloud assured workloads create \
  --organization=123456789 \
  --location=europe-west3 \
  --display-name="GDPR-Production-Workload" \
  --compliance-regime=EU_REGIONS_AND_SUPPORT \
  --billing-account=billingAccounts/ABCDEF-123456-GHIJKL \
  --resource-settings='[
    {
      "resourceType": "CONSUMER_PROJECT",
      "displayName": "gdpr-prod-project"
    }
  ]'

# After creation, verify the organization policies applied to the folder
gcloud resource-manager org-policies list \
  --folder=FOLDER_ID_FROM_ABOVE \
  --format="table(constraint,spec.rules[0].enforce)"

# Look for:
# constraints/gcp.resourceLocations - restricts to EU regions
# constraints/iam.allowedPolicyMemberDomains - restricts IAM membership
# constraints/gcp.restrictServiceUsage - restricts to approved services

```

Cloud Spanner and the multi-region trap:

Spanner’s multi-region configurations are architecturally elegant for global applications but are a sovereignty minefield. A `nam6` (North America multi-region) configuration spans multiple US states. A `eur3` configuration spans Netherlands and Belgium — both EU member states, so GDPR-compliant. But `global` configurations span multiple continents, and `asia1` spans Tokyo and Osaka.

The key residency risk with Spanner is the witness replica. In a multi-region configuration, Spanner places leader replicas in some regions and witness replicas in others as defined by the named configuration. The witness replica location is fixed per configuration and may not be in your desired jurisdiction — consult the current Spanner instance-configs documentation (or `gcloud spanner instance-configs describe`) for the exact replica placement of any multi-region config you’re evaluating, including `eur3`. Before choosing any multi-region Spanner configuration, verify

every replica placement against your regulatory requirements, not just the primary region.

If you need single-region Spanner for strict sovereignty, use a regional instance — you sacrifice global reads and multi-region writes but gain unambiguous data residency.

```
# Create a single-region Spanner instance for strict residency
# (regional, not multi-region - no data leaves europe-west3)

gcloud spanner instances create prod-sovereign-spanner \
  --config=regional-europe-west3 \
  --description="GDPR-compliant single-region Spanner instance" \
  --nodes=3 \
  --project=gdpr-production-project

# Verify the instance configuration - inspect which regions are involved
gcloud spanner instance-configs describe regional-europe-west3 \
  --format="yaml(displayName,replicas)"

# List all databases on this instance
gcloud spanner databases list \
  --instance=prod-sovereign-spanner \
  --format="table(name,state,encryptionInfo.encryptionType)"

# Create a database with customer-managed encryption
gcloud spanner databases create customer-data \
  --instance=prod-sovereign-spanner \
  --kms-key=
↳ projects/gdpr-production-project/locations/europe-west3/keyRings/spanner-keyring/cryptoKeys/spanner-key
```

26.5 Audit Logging for Residency Compliance

Regulatory frameworks don't just require that you implement residency controls — they require that you prove those controls are in effect and working. Audit logging for sovereignty compliance is different from performance monitoring or application debugging. It must capture specific events: data access, schema changes, privilege escalations, failed access attempts, and — for the highest-sensitivity requirements — every query that touches regulated columns.

AWS RDS audit logging with CloudWatch Logs:

RDS PostgreSQL ships logs to CloudWatch Logs within the same region as the instance — ensuring the audit trail itself satisfies residency requirements. For PostgreSQL, `pgaudit` is the standard audit extension, providing object-level and statement-level logging.

```
-- Enable pgaudit on RDS PostgreSQL
-- First, add pgaudit to shared_preload_libraries via parameter group
-- Then configure audit scope per session or globally

-- Verify pgaudit is loaded
SHOW shared_preload_libraries;

-- Set global audit logging (in parameter group: pgaudit.log = 'ddl, write, role')
-- Or set per-session for fine-grained control:
SET pgaudit.log = 'all';
SET pgaudit.log_catalog = on;
SET pgaudit.log_relation = on;
SET pgaudit.log_parameter = on; -- logs bind parameters - important for PII audits

-- Audit-specific objects only (table-level granularity)
-- Useful when you want to audit only tables containing personal data
CREATE EXTENSION IF NOT EXISTS pgaudit;

-- Enable object auditing for a specific table
SELECT pgaudit.set_log('SELECT,INSERT,UPDATE,DELETE', 'public', 'CustomerPersonalData');
```

```

-- SQL Server equivalent: enable server audit for Azure SQL and RDS SQL Server
-- Azure SQL uses built-in auditing; RDS SQL Server requires SQL Server Audit

-- Create a server audit (SQL Server on-premises equivalent approach for RDS)
CREATE SERVER AUDIT GDPR_Audit
TO FILE (
    FILEPATH = 'D:\SQLAudit\',
    MAXSIZE = 100 MB,
    MAX_ROLLOVER_FILES = 20,
    RESERVE_DISK_SPACE = OFF
)
WITH (
    QUEUE_DELAY = 1000,
    ON_FAILURE = CONTINUE
);

-- Create database audit specification targeting PII tables
CREATE DATABASE AUDIT SPECIFICATION GDPR_DB_Audit
FOR SERVER AUDIT GDPR_Audit
ADD (SELECT, INSERT, UPDATE, DELETE
    ON dbo.CustomerPersonalData BY public),
ADD (SCHEMA_OBJECT_ACCESS_GROUP),
ADD (DATABASE_ROLE_MEMBER_CHANGE_GROUP),
ADD (DATABASE_PERMISSION_CHANGE_GROUP)
WITH (STATE = ON);

ALTER SERVER AUDIT GDPR_Audit WITH (STATE = ON);

```

For Azure SQL, the built-in auditing feature writes to Azure Blob Storage, Log Analytics, or Event Hub. Configure it to write to a storage account in the same region as the database — do not allow cross-region audit log storage, which would create a secondary residency concern.

```

# Enable Azure SQL auditing to a storage account in the same region
az sql db audit-policy update \
  --resource-group rg-prod-eu \
  --server sql-prod-west europe \
  --name gdpr-production-db \
  --state Enabled \
  --storage-account "prodauditwest europe" \
  --retention-days 90 \
  --storage-endpoint "https://prodauditwest europe.blob.core.windows.net" \
  --storage-key "$(az storage account keys list \
    --account-name prodauditwest europe \
    --resource-group rg-prod-eu \
    --query '[0].value' \
    --output tsv)"

# Verify auditing configuration
az sql db audit-policy show \
  --resource-group rg-prod-eu \
  --server sql-prod-west europe \
  --name gdpr-production-db \
  --query '{
    state: state,
    storageEndpoint: storageEndpoint,
    retentionDays: retentionDays,
    auditActionsAndGroups: auditActionsAndGroups
  }'

```

Google Cloud SQL audit logging with Cloud Audit Logs:

Cloud SQL integrates with Cloud Audit Logs for data access events. By default, Admin Activity logs are always enabled. Data Access logs (which capture SELECT, INSERT, UPDATE, DELETE) must be explicitly enabled at the project or organization level.

```

# Enable Data Access audit logs for Cloud SQL at the project level
# This captures all DML and DDL operations for compliance evidence

```



```

gcloud projects get-iam-policy gdpr-production-project \
  --format=json > current-policy.json

# Add Data Access log configuration to the policy
cat > audit-log-patch.json << 'EOF'
{
  "auditConfigs": [
    {
      "service": "cloudsql.googleapis.com",
      "auditLogConfigs": [
        { "logType": "ADMIN_READ" },
        { "logType": "DATA_READ" },
        { "logType": "DATA_WRITE" }
      ]
    }
  ]
}
EOF

# Apply via org-policy (preferred for organization-wide enforcement)
gcloud organizations add-iam-policy-binding 123456789 \
  --member="allUsers" \
  --role="roles/logging.viewer" \
  --condition=None

# More precisely: set audit log config at org level using gcloud logging
gcloud logging settings update \
  --organization=123456789 \
  --log-filter='resource.type="cloudsql_database"' \
  --storage-location=europe-west3

```

Building a cross-platform residency audit dashboard:

For organizations running regulated workloads across multiple clouds — common for multinationals hedging vendor dependency — you need a unified view of residency compliance. The following shows how to structure a compliance query that runs as a scheduled check, pulling evidence that all databases remain in their declared regions with the expected encryption and replication constraints.

```

#!/bin/bash
# residency-compliance-check.sh
# Run this as a scheduled job (daily minimum, hourly for high-assurance environments)
# Outputs a JSON compliance report for ingestion into SIEM or compliance tooling

OUTPUT_FILE="residency-check-$(date +%Y%m%d-%H%M%S).json"
EXPECTED_REGION_AWS="eu-central-1"
EXPECTED_REGION_GCP="europe-west3"

echo "{\"timestamp\": \"$(date -u +%Y-%m-%dT%H:%M:%SZ)\", \"findings\": [" > "$OUTPUT_FILE"

# ---- AWS RDS Check ----
echo "Checking AWS RDS instances..."

aws rds describe-db-instances \
  --region "$EXPECTED_REGION_AWS" \
  --query 'DBInstances[?Tags[?Key==`ComplianceFramework` && Value==`GDPR`]].{' \
    id: DBInstanceIdentifier, \
    region: AvailabilityZone, \
    encrypted: StorageEncrypted, \
    kmsKey: KmsKeyId, \
    multiAZ: MultiAZ, \
    publiclyAccessible: PubliclyAccessible, \
    backupRetention: BackupRetentionPeriod, \
    readReplicas: ReadReplicaDBInstanceIdentifiers \
  }' \
  --output json | python3 -c "
import sys, json
instances = json.load(sys.stdin)
for inst in instances:

```

```

        finding = {
            'platform': 'AWS_RDS',
            'resource': inst['id'],
            'checks': {
                'encrypted': inst['encrypted'],
                'not_public': not inst['publiclyAccessible'],
                'backup_retention_ok': inst['backupRetention'] >= 7,
                'no_cross_region_replicas': len(inst.get('readReplicas', [])) == 0
            }
        }
        finding['compliant'] = all(finding['checks'].values())
        print(json.dumps(finding) + ',')
" >> "$OUTPUT_FILE"

# ---- GCP Cloud SQL Check ----
echo "Checking GCP Cloud SQL instances..."

gcloud sql instances list \
  --project=gdpr-production-project \
  --filter="labels.compliance=gdpr" \
  --format=json | python3 -c "
import sys, json
instances = json.load(sys.stdin)
for inst in instances:
    region = inst.get('region', '')
    replicas = inst.get('replicaNames', [])
    finding = {
        'platform': 'GCP_CloudSQL',
        'resource': inst['name'],
        'checks': {
            'correct_region': region == 'europe-west3',
            'backup_enabled': inst.get('settings', {}).get('backupConfiguration', {}).get('enabled',
↪ False),
            'ssl_required': inst.get('settings', {}).get('ipConfiguration', {}).get('requireSsl',
↪ False),
            'no_cross_region_replicas': len(replicas) == 0
        }
    }
    finding['compliant'] = all(finding['checks'].values())
    print(json.dumps(finding) + ',')
" >> "$OUTPUT_FILE"

# Close JSON array (remove trailing comma from last entry in production)
echo "]" }" >> "$OUTPUT_FILE"

echo "Compliance check written to $OUTPUT_FILE"

# Alert on any non-compliant findings
NON_COMPLIANT=$(python3 -c "
import json
with open('$OUTPUT_FILE') as f:
    content = f.read().rstrip()
    # Normalize trailing comma issue for demo purposes
    data = json.loads(content)
non_compliant = [f for f in data['findings'] if not f.get('compliant', True)]
print(len(non_compliant))
")

if [ "$NON_COMPLIANT" -gt 0 ]; then
    echo "ALERT: $NON_COMPLIANT non-compliant resources detected. Review $OUTPUT_FILE immediately."
    # Integrate with PagerDuty, SNS, or your alerting platform here
    exit 1
fi

```

26.6 Data Classification and Residency-Aware Schema Design

Residency compliance is not binary across your entire database — it applies to specific data elements. A customer's shipping address is personal data subject to GDPR. The timestamp of a page view might not be. Designing schemas with residency awareness means knowing which columns carry sovereignty obligations and ensuring those obligations are enforceable at the database layer, not just in application logic.

Tagging columns with classification metadata (PostgreSQL):

PostgreSQL doesn't have a native column classification system, but you can implement one using `pg_description` (the comment table) and a naming convention enforced by DDL triggers or linting tools in your CI/CD pipeline.

```
-- PostgreSQL: document data classification in column comments
-- This creates a queryable audit trail of which columns hold regulated data

CREATE TABLE customers (
  customer_id    BIGSERIAL PRIMARY KEY,
  email          TEXT NOT NULL,
  full_name      TEXT,
  date_of_birth  DATE,
  national_id    TEXT,
  country_code   CHAR(2) NOT NULL,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Tag columns with classification metadata via COMMENT
COMMENT ON COLUMN customers.email IS

↵ '{"classification":"PERSONAL_DATA","gdpr_category":"contact","lawful_basis":"contract","retention_days":2555}'
↵ ;

COMMENT ON COLUMN customers.full_name IS

↵ '{"classification":"PERSONAL_DATA","gdpr_category":"identity","lawful_basis":"contract","retention_days":2555}'
↵ ;

COMMENT ON COLUMN customers.date_of_birth IS

↵ '{"classification":"SENSITIVE_PERSONAL_DATA","gdpr_category":"identity","lawful_basis":"legal_obligation","retention_days":2555}'
↵ ;

COMMENT ON COLUMN customers.national_id IS

↵ '{"classification":"SENSITIVE_PERSONAL_DATA","gdpr_category":"government_id","lawful_basis":"legal_obligation","retention_days":2555}'
↵ ;

COMMENT ON COLUMN customers.country_code IS
  '{"classification":"OPERATIONAL","gdpr_category":"none","lawful_basis":"none","retention_days":-1}';

-- Query to get all regulated columns across all tables
-- Use this to drive access control reviews and encryption audits
SELECT
  n.nspname          AS schema_name,
  c.relname          AS table_name,
  a.attname          AS column_name,
  pg_catalog.col_description(c.oid, a.attnum)::jsonb AS classification
FROM
  pg_catalog.pg_attribute a
  JOIN pg_catalog.pg_class c ON c.oid = a.attrelid
  JOIN pg_catalog.pg_namespace n ON n.oid = c.relnamespace
WHERE
  a.attnum > 0
```

```

AND NOT a.attisdropped
AND pg_catalog.col_description(c.oid, a.attnum) IS NOT NULL
AND pg_catalog.col_description(c.oid, a.attnum) LIKE '%"classification"'
AND (pg_catalog.col_description(c.oid, a.attnum)::jsonb->>'classification')
    IN ('PERSONAL_DATA', 'SENSITIVE_PERSONAL_DATA')
ORDER BY
    schema_name, table_name, column_name;

-- SQL Server equivalent: use extended properties for column classification
-- This integrates with SQL Server's built-in classification infrastructure

-- Add data classification extended properties
EXEC sys.sp_addextendedproperty
    @name = N'sys_information_type_name',
    @value = N'National Identification Number',
    @level0type = N'SHEMA', @level0name = N'dbo',
    @level1type = N'TABLE', @level1name = N'Customers',
    @level2type = N'COLUMN', @level2name = N'NationalID';

EXEC sys.sp_addextendedproperty
    @name = N'sys_sensitivity_label_name',
    @value = N'Highly Confidential',
    @level0type = N'SHEMA', @level0name = N'dbo',
    @level1type = N'TABLE', @level1name = N'Customers',
    @level2type = N'COLUMN', @level2name = N'NationalID';

-- Azure SQL / SQL Server 2019+: use the built-in classification feature
-- This integrates with Microsoft Purview and Azure Defender for SQL
ADD SENSITIVITY CLASSIFICATION TO
    dbo.Customers.NationalID
WITH (
    LABEL = 'Highly Confidential',
    LABEL_ID = 'bf91e08c-f4f0-478a-b016-25164b2a65ff',
    INFORMATION_TYPE = 'National Identification Number',
    INFORMATION_TYPE_ID = '3dc733e8-e8a8-4c7c-b285-c6bdc9acacdf',
    RANK = CRITICAL
);

-- Query all classified columns for compliance reporting
SELECT
    schema_name(t.schema_id)          AS schema_name,
    t.name                           AS table_name,
    c.name                           AS column_name,
    sc.information_type               AS information_type,
    sc.label                         AS sensitivity_label,
    sc.rank_desc                     AS rank
FROM
    sys.sensitivity_classifications sc
    JOIN sys.columns c ON sc.major_id = c.object_id AND sc.minor_id = c.column_id
    JOIN sys.tables t ON c.object_id = t.object_id
ORDER BY
    sc.rank DESC, schema_name, table_name, column_name;

```

Row-level residency for multinational databases:

Some architectures store data for multiple jurisdictions in a single database cluster — a common pattern for SaaS providers serving customers in both the EU and the US. If you cannot shard by region at the database level, row-level security becomes a compensating control that limits access to data based on its jurisdictional tag.

```

-- PostgreSQL: Row-Level Security for jurisdiction-partitioned data
-- Ensures application sessions tagged for one jurisdiction cannot
-- access another jurisdiction's personal data

ALTER TABLE customers ENABLE ROW LEVEL SECURITY;

-- Policy: sessions must declare their jurisdiction via a session variable
-- Application sets: SET app.jurisdiction = 'EU' before executing queries

```

```

CREATE POLICY jurisdiction_isolation ON customers
USING (
    country_code = ANY(
        CASE current_setting('app.jurisdiction', TRUE)
        WHEN 'EU' THEN ARRAY['DE','FR','ES','IT','NL','BE','PL','SE','AT','DK','FI','PT','IE',
            ↪ 'GR','CZ','RO','HU','SK','BG','HR','LT','LV','EE','SI','CY','LU','MT']
        WHEN 'US' THEN ARRAY['US']
        WHEN 'BR' THEN ARRAY['BR']
        ELSE ARRAY[]::CHAR(2)[]
        END
    )
);

-- Allow superusers to bypass RLS for administrative operations
-- (administrative access should itself be audited and jurisdictionally constrained)
ALTER TABLE customers FORCE ROW LEVEL SECURITY;
-- FORCE means even table owner is subject to RLS; remove for DBA bypass if needed

-- Test the policy:
SET app.jurisdiction = 'EU';
SELECT COUNT(*) FROM customers; -- Returns only EU country_code rows

SET app.jurisdiction = 'US';
SELECT COUNT(*) FROM customers; -- Returns only US rows

```

26.7 Right to Erasure and the Database Engineering Challenge

GDPR Article 17 and equivalent provisions in LGPD, PIPL, and CCPA establish the right to erasure — the obligation to delete a data subject’s personal data upon request within defined timeframes. For a cloud DBA, this is one of the most architecturally demanding compliance requirements because deletion in a relational database is not truly deletion: it persists in transaction logs, MVCC dead tuples, automated backups, read replica lag windows, and any audit logs that recorded the original insert.

The erasure surface area in managed databases:

When you execute `DELETE FROM customers WHERE customer_id = 12345`, the following still contain the data:

1. **PostgreSQL MVCC dead tuples** — until `VACUUM` reclaims them. With autovacuum running, this may be minutes to hours.
2. **WAL / transaction logs** — the WAL record of the delete contains the old row values (before-image). WAL segments are retained for replication lag tolerance; WAL archiving for PITR means they may persist for your full backup retention period.
3. **Read replicas** — the replica applies the WAL and creates its own dead tuples, which must also be vacuumed.
4. **Automated backups** — backups taken before the delete permanently contain the deleted record until the backup expires.
5. **Audit logs** — if `pgaudit` or SQL Server Audit logged the original `INSERT` with parameter logging enabled, the personal data is in the audit log.
6. **Application caches** — outside the database, but part of the erasure obligation.

The pseudonymization strategy:

For production databases where immediate physical deletion is operationally risky, pseudonymiza-

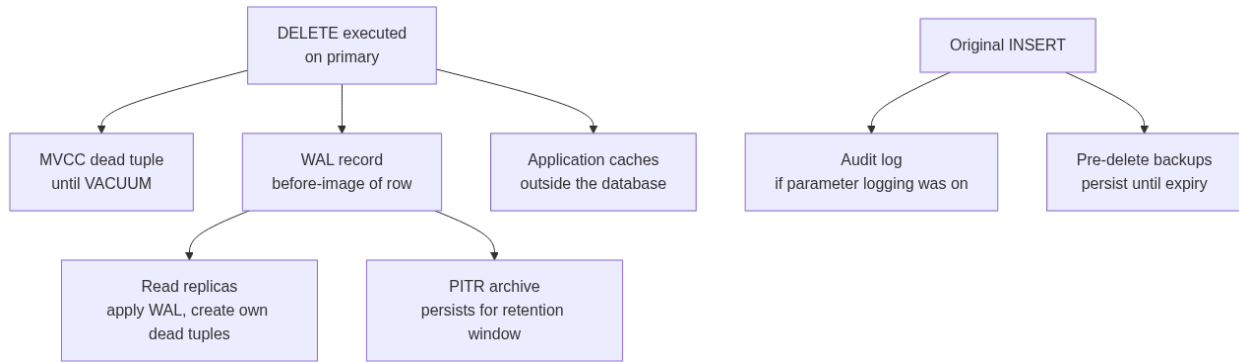


Figure 17: A single row deletion leaves the data recoverable from at least six other locations — the DELETE statement is the start of the erasure obligation, not the end of it.

tion of personal data upon erasure request is an accepted GDPR technique — replacing identifying values with irreversible tokens while retaining the record structure for referential integrity and analytics.

```

-- PostgreSQL: Pseudonymization procedure for right-to-erasure requests
-- Replaces personal data with deterministic tokens that cannot be reversed
-- Maintains referential integrity and audit trail of the erasure event

CREATE TABLE erasure_requests (
    request_id      BIGSERIAL PRIMARY KEY,
    customer_id     BIGINT NOT NULL,
    requested_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at    TIMESTAMPTZ,
    requested_by    TEXT NOT NULL,    -- Legal team identifier, not personal data
    status          TEXT NOT NULL DEFAULT 'PENDING'
    CHECK (status IN ('PENDING', 'PROCESSING', 'COMPLETED', 'FAILED'))
);

CREATE OR REPLACE PROCEDURE process_erasure_request(
    p_customer_id  BIGINT,
    p_request_id   BIGINT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_erasure_token TEXT;
BEGIN
    -- Generate a non-reversible erasure token for audit evidence
    v_erasure_token := 'ERASED-' || encode(
        digest(p_customer_id::TEXT || NOW()::TEXT, 'sha256'),
        'hex'
    );

    -- Pseudonymize all personal data columns
    -- Replace with the erasure token - clearly marked as erased, not null
    UPDATE customers
    SET
        email          = v_erasure_token || '@erased.invalid',
        full_name       = '[ERASED]',
        date_of_birth   = NULL,
        national_id     = NULL,
        updated_at      = NOW()
    WHERE customer_id = p_customer_id;

    -- Pseudonymize related tables containing personal data
    UPDATE customer_addresses
    SET
        street_line_1 = '[ERASED]',

```

```

        street_line_2 = NULL,
        city          = '[ERASED]',
        postal_code   = '[ERASED]'
WHERE customer_id = p_customer_id;

UPDATE customer_contact_preferences
SET
    phone_number = NULL,
    mobile_number = NULL
WHERE customer_id = p_customer_id;

-- Record completion in the erasure audit table
UPDATE erasure_requests
SET
    completed_at = NOW(),
    status       = 'COMPLETED'
WHERE request_id = p_request_id;

-- Log the erasure event (the token, not the original data)
INSERT INTO erasure_audit_log (
    customer_id, erasure_token, erased_at, request_id
) VALUES (
    p_customer_id, v_erasure_token, NOW(), p_request_id
);

COMMIT;

EXCEPTION WHEN OTHERS THEN
    UPDATE erasure_requests
    SET status = 'FAILED'
    WHERE request_id = p_request_id;
    RAISE;
END;
$$;

-- SQL Server equivalent: Erasure procedure using UPDATE with masked values
-- Compatible with Azure SQL and RDS SQL Server

CREATE PROCEDURE dbo.ProcessErasureRequest
    @CustomerID      INT,
    @RequestID       BIGINT,
    @RequestedBy     NVARCHAR(200)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;

        DECLARE @ErasureToken NVARCHAR(100) =
            'ERASED-' + CONVERT(NVARCHAR(64),
                HASHBYTES('SHA2_256',
                    CAST(@CustomerID AS NVARCHAR(20)) +
                    CONVERT(NVARCHAR(30), SYSUTCDATETIME(), 126)
                ),
                2);

        -- Pseudonymize main customer record
        UPDATE dbo.Customers
        SET
            Email          = @ErasureToken + N'@erased.invalid',
            FullName       = N'[ERASED]',
            DateOfBirth    = NULL,
            NationalID     = NULL,
            UpdatedAt      = SYSUTCDATETIME()
        WHERE CustomerID = @CustomerID;

        -- Pseudonymize address data
        UPDATE dbo.CustomerAddresses
        SET
            StreetLine1 = N'[ERASED]',

```

```
        StreetLine2 = NULL,
        City        = N'[ERASED]',
        PostalCode   = N'[ERASED]'
WHERE CustomerID = @CustomerID;

-- Mark request complete
UPDATE dbo.ErasureRequests
SET
    CompletedAt = SYSUTCDATETIME(),
    Status      = N'COMPLETED'
WHERE RequestID = @RequestID;

-- Record in audit log (token only, no original PII)
INSERT INTO dbo.ErasureAuditLog
    (CustomerID, ErasureToken, ErasedAt, RequestID)
VALUES
    (@CustomerID, @ErasureToken, SYSUTCDATETIME(), @RequestID);

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

    UPDATE dbo.ErasureRequests
    SET Status = N'FAILED'
    WHERE RequestID = @RequestID;

    THROW;
END CATCH;
END;
```

Handling backups after erasure:

After pseudonymizing the live data, backups taken before the erasure event still contain original personal data. Your backup retention policy determines the maximum window of exposure. For many organizations, the practical approach is:

1. Document the erasure completion date in the erasure audit log.
2. Note the oldest backup that pre-dates the erasure (and therefore contains the original data).
3. Set a calendar reminder to confirm that pre-erasure backups have expired based on your retention period.
4. For high-assurance environments, encrypt each customer's data with a unique DEK (Data Encryption Key) stored in KMS. To perform cryptographic erasure, delete the customer's DEK — all backups containing that customer's data become computationally unreadable even though the ciphertext persists.

The DEK-per-customer approach is the most elegant technical solution for backup erasure, but it requires application-layer envelope encryption rather than relying on the database engine's built-in encryption — a significant architectural commitment that should be designed in from the beginning, not retrofitted.

26.8 Operationalizing Sovereignty: Runbooks and Governance

Technical controls are necessary but insufficient. Residency compliance fails in operations when an engineer under incident pressure creates a cross-region read replica to diagnose a performance problem, or when a backup copy is created in a non-compliant region by a script that predates the sovereignty policy. Governance structures prevent these operational violations.

The residency violation response runbook:

```
# Runbook: Responding to Data Residency Violation Alert
# Version: 1.0 | Owner: Cloud DBA Team | Review: Quarterly

## Trigger Conditions
- Compliance check script reports non-compliant resource
- AWS Config Rule "rds-no-cross-region-replica" fires
- Azure Policy compliance dashboard shows violation
- Manual discovery of misconfigured resource

## Immediate Response (within 1 hour)

### Step 1: Identify scope
- Determine which resource is out of compliance
- Identify what data is affected (classification level, jurisdictions)
- Determine when the violation began (CloudTrail / Activity Log timestamp)
- Identify who made the change (IAM identity from audit log)

### Step 2: Contain the violation
- Do NOT simply delete the out-of-region resource without preserving evidence
- Disable any replication sending data to the non-compliant location
- Revoke network access to the violating resource if it contains live data
- For RDS cross-region replica: promote replica (breaks replication),
  then destroy the promoted instance in the non-compliant region

### Step 3: Notify
- Alert Data Protection Officer (DPO) within 1 hour of discovery
- Engage Legal if data has been accessible to non-compliant region > 24 hours
- GDPR: regulator notification may be required within 72 hours if personal
  data was accessed or if exposure constitutes a "high risk" breach
- Document notification timestamps in the incident record

### Step 4: Remediate
- Restore compliant configuration
- Re-run compliance check script to confirm remediation
- Review SCPs / Azure Policy for gaps that allowed the violation

### Step 5: Post-incident
- Complete incident report with timeline and root cause
- Update SCP / Policy to prevent recurrence
- Schedule training review if human error was root cause
```

CI/CD integration for schema and migration residency checks:

New database migrations can introduce sovereignty violations — adding a foreign key to a non-compliant table, adding a replication publication that includes new personal data tables, or changing a column type that bypasses Always Encrypted. Integrate residency checks into your migration pipeline.

```
#!/usr/bin/env python3
"""
migration_residency_check.py
Pre-deployment check for database migrations that affect classified columns.
Run as a CI/CD gate before applying any migration in production.
"""

import re
import sys
from pathlib import Path

# Patterns that indicate a migration may affect personal data handling
SENSITIVE_PATTERNS = [
    (r'\bALTER\s+TABLE\s+\S+\s+ADD\s+COLUMN\b', 'new_column_added'),
    (r'\bALTER\s+TABLE\s+\S+\s+ALTER\s+COLUMN\b', 'column_type_changed'),
    (r'\bCREATE\s+TABLE\b', 'new_table_created'),
    (r'\bCREATE\s+PUBLICATION\b', 'logical_replication_publication'),
    (r'\bALTER\s+PUBLICATION\s+\S+\s+ADD\s+TABLE\b', 'table_added_to_publication'),
```

```
(r'\bGRANT\s+SELECT\b', 'select_privilege_granted'),
(r'\bGRANT\s+ALL\b', 'all_privileges_granted'),
]

# Columns and table names that are known to contain personal data
# In production, derive this from the classification metadata query shown earlier
KNOWN_PERSONAL_DATA_TABLES = {
    'customers', 'customer_addresses', 'customer_contact_preferences',
    'patient_records', 'employee_details', 'payment_methods',
}

def check_migration_file(filepath: str) -> list[dict]:
    issues = []
    content = Path(filepath).read_text(encoding='utf-8')
    content_lower = content.lower()

    for pattern, issue_type in SENSITIVE_PATTERNS:
        matches = re.findall(pattern, content, re.IGNORECASE)
        if matches:
            # Check if any known personal data table is referenced nearby
            for table in KNOWN_PERSONAL_DATA_TABLES:
                if table in content_lower:
                    issues.append({
                        'file': filepath,
                        'issue_type': issue_type,
                        'affected_table': table,
                        'requires_review': True,
                        'message': (
                            f"Migration affects '{table}' which contains personal data. "
                            f"Requires DPO sign-off and residency impact assessment."
                        )
                    })

    return issues

if __name__ == '__main__':
    if len(sys.argv) < 2:
        print("Usage: migration_residency_check.py <migration_file> [...]")
        sys.exit(1)

    all_issues = []
    for migration_file in sys.argv[1:]:
        all_issues.extend(check_migration_file(migration_file))

    if all_issues:
        print("RESIDENCY REVIEW REQUIRED:")
        for issue in all_issues:
            print(f" [{issue['issue_type']}] {issue['message']}")
        print("\nThis migration cannot be auto-deployed. Submit for DPO review.")
        sys.exit(1)
    else:
        print("No residency concerns detected. Proceeding with deployment.")
        sys.exit(0)
```

Key Takeaways

- **Managed services make decisions on your behalf** — automated backups, replication, and minor version patches all involve data movement. Audit every automatic behavior against your regulatory requirements before going to production, and use Service Control Policies (AWS), Azure Policy, and Assured Workloads (GCP) to enforce boundaries that survive operational mistakes.

- **Key custody is as important as data location** — a database encrypted with AWS-managed keys stored in the same region may still fail a strict sovereignty audit if your regulator requires that key material never be accessible to cloud vendor employees. Use customer-managed keys backed by CloudHSM, Azure Key Vault HSM, or Cloud KMS with Cloud HSM to maintain demonstrable key custody.
 - **Audit logs are first-class residency artifacts** — if your audit logs are stored in a different region than your database, you have created a secondary residency problem. Configure CloudWatch Logs, Azure Blob Storage, and Cloud Logging to retain audit data in the same region as the database it describes, and include audit log residency in your compliance check scripts.
 - **Right to erasure requires architectural planning, not just a DELETE statement** — personal data persists in MVCC dead tuples, WAL archives, backup files, and audit logs long after the row is deleted. Design erasure into your schema from the start using pseudonymization procedures and, for the highest assurance, per-customer envelope encryption with DEK deletion as cryptographic erasure.
 - **Operational governance closes the gap that technical controls leave open** — SCPs and Azure Policies prevent many violations, but incident pressure, inherited scripts, and human error will occasionally produce residency gaps. A tested residency violation runbook, CI/CD migration checks, and regular compliance scans ensure that violations are detected quickly, contained, and documented in a way that satisfies regulatory incident response timelines.
-

Chapter 27: Hybrid Cloud: On-Premises and Cloud Together

Most production environments do not migrate to the cloud in a single weekend. They live in the space between — some databases running on-premises in your data center, some running in RDS or Azure SQL or Cloud SQL, and the two worlds needing to communicate as if the network boundary does not exist. This chapter is about that in-between state: how to architect it correctly, how to keep data synchronized across boundaries, how to handle failover and latency, and how to run hybrid configurations in production without turning them into a maintenance nightmare. If you are three months into a cloud migration and still have seventeen on-premises SQL Server instances feeding data to Azure SQL, this chapter is written for your situation.

27.1 Network Architecture: Connecting Your Data Center to the Cloud

The foundation of any hybrid database architecture is the network. Before a single row of data moves between your on-premises PostgreSQL instance and RDS, before DMS can reach your source database, and before an application server in AWS can query your on-premises Oracle, the network path must be stable, low-latency, and secured. This is not optional plumbing — it is the structural prerequisite that determines whether your hybrid architecture works or falls apart under production load.

The Two Options: VPN vs. Direct Connect (and Their Azure/GCP Equivalents)

AWS gives you two primary options for connecting on-premises infrastructure to your VPC: AWS Site-to-Site VPN and AWS Direct Connect.

Site-to-Site VPN runs over the public internet, uses IPSec encryption, and is inexpensive to set up. Bandwidth is limited to approximately 1.25 Gbps per tunnel, latency is variable (typically 20–100ms depending on geography and ISP), and it is prone to jitter. For database replication where your logs are small and your RPO tolerance is measured in minutes, VPN is acceptable. For bulk migration of a 20 TB database or for real-time streaming replication where lag matters in seconds, VPN will create problems.

AWS Direct Connect provides a dedicated physical connection from your data center to an AWS Direct Connect location. It delivers 1 Gbps, 10 Gbps, or 100 Gbps connections with consistent latency (typically 2–10ms from most data centers to a nearby AWS region). You pay for port hours

and data transfer, but you eliminate the jitter and bandwidth contention of internet-based VPN. For production hybrid database architectures where you are replicating continuously or running applications that span on-premises and cloud, Direct Connect is the correct choice.

Azure's equivalents are **Azure VPN Gateway** (VPN) and **Azure ExpressRoute** (dedicated circuit). GCP offers **Cloud VPN** and **Cloud Interconnect** (Dedicated or Partner). The architectural decisions are identical regardless of cloud provider — VPN for non-critical or low-bandwidth scenarios, dedicated connectivity for production.

Setting Up an AWS Site-to-Site VPN to Your Data Center

```
# Step 1: Create a Customer Gateway representing your on-premises VPN device
aws ec2 create-customer-gateway \
  --type ipsec.1 \
  --public-ip 203.0.113.12 \
  --bgp-asn 65000 \
  --region us-east-1 \
  --tag-specifications 'ResourceType=customer-gateway,Tags=[{Key=Name,Value=datacenter-primary}]'

# Step 2: Create a Virtual Private Gateway and attach it to your VPC
aws ec2 create-vpn-gateway \
  --type ipsec.1 \
  --amazon-side-asn 64512 \
  --region us-east-1

# Capture the VpnGatewayId from the output, then attach
aws ec2 attach-vpn-gateway \
  --vpn-gateway-id vgw-0abc123def456789a \
  --vpc-id vpc-0123456789abcdef0

# Step 3: Create the Site-to-Site VPN connection
aws ec2 create-vpn-connection \
  --type ipsec.1 \
  --customer-gateway-id cgw-0abc123def456789a \
  --vpn-gateway-id vgw-0abc123def456789a \
  --options '{"StaticRoutesOnly":false}' \
  --region us-east-1

# Step 4: Enable route propagation so VPC route tables learn on-premises CIDRs
aws ec2 enable-vgw-route-propagation \
  --route-table-id rtb-0123456789abcdef0 \
  --gateway-id vgw-0abc123def456789a
```

Once the VPN is established, your RDS instances inside the VPC can communicate with on-premises servers using private IP addresses. Your security groups control what traffic is allowed — treat your on-premises CIDR block (say, 10.0.0.0/8) as a trusted network source, but be precise about which ports you open. Port 5432 for PostgreSQL, 1433 for SQL Server, 3306 for MySQL — no broader rules.

Private Link for Specific Service Exposure

If you do not want to route your entire data center network into a VPC, AWS PrivateLink lets you expose a single service endpoint across a network boundary without full VPC peering or VPN. This is useful when an on-premises application needs to reach a specific RDS endpoint and nothing else.

Azure Private Link functions identically for Azure SQL: you create a private endpoint inside a VNet, and the on-premises network (connected via ExpressRoute or VPN Gateway) reaches the database through that private IP rather than the public Azure SQL endpoint.

```
# Create a PrivateLink endpoint for an RDS instance
# First, the VPC endpoint service must exist - RDS uses Interface Endpoints

aws ec2 create-vpc-endpoint \
  --vpc-endpoint-type Interface \
  --vpc-id vpc-0123456789abcdef0 \
  --service-name com.amazonaws.us-east-1.rds \
  --subnet-ids subnet-0abc123def456789a subnet-0abc123def456790b \
  --security-group-ids sg-0abc123def456789a \
  --private-dns-enabled \
  --region us-east-1
```

What Latency Actually Means for Your Databases

When your application is in AWS and your database is on-premises (or vice versa), every synchronous query crosses the network. A 20ms round-trip added to every SELECT statement compounds quickly when an ORM generates 40 queries per page load. The math is brutal: 40 queries $\times$ 20ms = 800ms of pure network time before any query execution.

Measure your actual latency before you commit to any hybrid query pattern. Run a simple ping and then a **pgbench** or **sysbench** test across the VPN or Direct Connect circuit before you design your application topology. If your latency is above 5ms, co-locate the application with its primary database. Use async replication and caching to bridge the gap rather than synchronous cross-boundary queries.

27.2 Data Synchronization Patterns Across Boundaries

With the network established, the next problem is data: how do you keep data synchronized between an on-premises database and its cloud counterpart? The answer depends entirely on whether you need bidirectional synchronization, unidirectional replication, or bulk periodic loads — and on how much lag you can tolerate.

AWS Database Migration Service for Ongoing Replication

AWS DMS is covered in detail in Chapter 25 in the context of migration. In hybrid architectures, DMS is also a production replication tool. You can configure a replication instance, a source endpoint pointing at your on-premises PostgreSQL or SQL Server, and a target endpoint pointing at RDS, and run DMS in continuous replication (CDC) mode indefinitely.

The on-premises PostgreSQL source requires logical replication enabled and a replication slot. For SQL Server, the source must have MS-CDC enabled on the database and each table you want to replicate.

```
# Create DMS replication instance in the VPC that has connectivity to on-premises
aws dms create-replication-instance \
  --replication-instance-identifier hybrid-repl-01 \
  --replication-instance-class dms.r6i.xlarge \
  --allocated-storage 200 \
  --vpc-security-group-ids sg-0abc123def456789a \
  --replication-subnet-group-identifier my-dms-subnet-group \
  --multi-az \
  --engine-version 3.5.2 \
  --publicly-accessible false

# Source endpoint: on-premises PostgreSQL at 10.10.5.20
```

```

aws dms create-endpoint \
  --endpoint-identifier onprem-pg-source \
  --endpoint-type source \
  --engine-name postgres \
  --username dms_replication_user \
  --password 'SecureP@ssword123' \
  --server-name 10.10.5.20 \
  --port 5432 \
  --database-name production_db \
  --extra-connection-attributes "captureDdls=true;pluginName=pglogical"

# Target endpoint: RDS PostgreSQL
aws dms create-endpoint \
  --endpoint-identifier rds-pg-target \
  --endpoint-type target \
  --engine-name postgres \
  --username dms_target_user \
  --password 'SecureP@ssword123' \
  --server-name mydb.c9akciq32.us-east-1.rds.amazonaws.com \
  --port 5432 \
  --database-name production_db

# Create and start the replication task in CDC mode
aws dms create-replication-task \
  --replication-task-identifier hybrid-cdc-task \
  --source-endpoint-arn arn:aws:dms:us-east-1:123456789012:endpoint:ABCDEF... \
  --target-endpoint-arn arn:aws:dms:us-east-1:123456789012:endpoint:GHIJKL... \
  --replication-instance-arn arn:aws:dms:us-east-1:123456789012:rep:MNOPQR... \
  --migration-type cdc \
  --table-mappings file://table-mappings.json \
  --replication-task-settings file://task-settings.json

```

Monitor your DMS task latency continuously. In hybrid configurations, replication lag above 60 seconds is a warning threshold — above 300 seconds, investigate immediately. DMS exposes `CDCLatencySource` and `CDCLatencyTarget` metrics in CloudWatch.

```

import boto3

cloudwatch = boto3.client('cloudwatch', region_name='us-east-1')

def get_dms_replication_lag(task_arn_suffix: str) -> dict:
    """
    Fetch CDC latency metrics for a DMS replication task.
    task_arn_suffix is the last segment of the replication task ARN.
    """
    response = cloudwatch.get_metric_statistics(
        Namespace='AWS/DMS',
        MetricName='CDCLatencyTarget',
        Dimensions=[
            {
                'Name': 'ReplicationInstanceIdentifier',
                'Value': 'hybrid-repl-01'
            },
            {
                'Name': 'ReplicationTaskIdentifier',
                'Value': 'hybrid-cdc-task'
            }
        ],
        StartTime='2024-03-01T00:00:00Z',
        EndTime='2024-03-01T01:00:00Z',
        Period=60,
        Statistics=['Average', 'Maximum']
    )

    datapoints = response['Datapoints']
    if not datapoints:
        return {'status': 'no_data', 'lag_seconds': None}

    latest = sorted(datapoints, key=lambda x: x['Timestamp'])[-1]

```



```

lag = latest['Maximum']

status = 'healthy'
if lag > 60:
    status = 'warning'
if lag > 300:
    status = 'critical'

return {
    'status': status,
    'lag_seconds': lag,
    'timestamp': latest['Timestamp'].isoformat()
}

result = get_dms_replication_lag('hybrid-cdc-task')
print(f"DMS replication lag: {result['lag_seconds']}s - Status: {result['status']}")

```

Native Replication as an Alternative to DMS

DMS is convenient but it adds a layer of abstraction with its own failure modes. For PostgreSQL-to-PostgreSQL hybrid replication, you can use native logical replication directly — your on-premises primary publishes, your RDS instance subscribes.

```

-- On the on-premises PostgreSQL primary (source):
-- Ensure wal_level = logical in postgresql.conf
-- Then create a publication for the tables you want to replicate

CREATE PUBLICATION hybrid_pub
FOR TABLE
    orders,
    order_items,
    customers,
    products
WITH (publish = 'insert, update, delete');

-- Grant replication privilege to the user RDS will connect as
CREATE USER rds_subscriber WITH REPLICATION LOGIN PASSWORD 'ReplicaP@ss456';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO rds_subscriber;

-- On RDS PostgreSQL (target) - the subscriber side:
-- Tables must already exist with matching schema

CREATE SUBSCRIPTION hybrid_sub
CONNECTION 'host=10.10.5.20 port=5432 dbname=production_db user=rds_subscriber password=ReplicaP@ss456'
PUBLICATION hybrid_pub
WITH (
    copy_data = true,
    create_slot = true,
    enabled = true,
    connect = true
);

-- Verify subscription status
SELECT subname, subenabled, subslotname, subpublications
FROM pg_subscription;

-- Check replication slot on the source to monitor lag
-- Run this on the on-premises primary:
SELECT slot_name,
    confirmed_flush_lsn,
    pg_current_wal_lsn() - confirmed_flush_lsn AS lag_bytes,
    pg_size_pretty(pg_current_wal_lsn() - confirmed_flush_lsn) AS lag_pretty
FROM pg_replication_slots
WHERE slot_type = 'logical';

```

The important difference between native logical replication and DMS: native replication maintains a replication slot on the source. If the connection drops or the subscriber falls behind, the source

accumulates WAL files to serve those changes when the slot reconnects. On-premises servers often have less disk than cloud instances — an unconsumed replication slot can fill your `pg_wal` directory and crash your primary. Set `max_slot_wal_keep_size` in PostgreSQL 13+ to cap this growth.

Azure Data Sync for SQL Server Hybrid

Azure SQL Data Sync is Microsoft’s native solution for bidirectional synchronization between on-premises SQL Server and Azure SQL Database. Unlike DMS (which is unidirectional from source to target), Data Sync can handle bidirectional sync — useful when applications write to both the on-premises database and Azure SQL simultaneously.

The configuration involves a Hub database in Azure SQL, a Data Sync Agent installed on your on-premises server, and member databases at both locations. Sync happens on a configurable schedule (minimum 5 minutes) rather than in real time. This is not a replication solution for low-lag requirements — it is a synchronization solution for scenarios where eventual consistency over minutes is acceptable.

Know the limits: Azure Data Sync does not support `TRUNCATE`, does not replicate DDL changes, has a 500-column limit per table, and cannot handle tables without primary keys. These are hard limits, not configuration options.

27.3 Hybrid Failover and Disaster Recovery

The most valuable use case for hybrid architecture is disaster recovery: your on-premises data center serves as DR for your cloud environment, or cloud serves as DR for your on-premises production. Either direction requires careful design around failover triggering, DNS cutover, and data divergence.

Cloud as DR for On-Premises Production

In this pattern, your on-premises PostgreSQL or SQL Server is the primary. The cloud database (RDS, Azure SQL) is a continuously-updated replica. If your data center fails, you promote the cloud replica, update your DNS or application connection strings, and resume operations.

The technical requirements are straightforward. The architectural challenges are operational: who decides to trigger failover? What is the SLA that justifies activating it? How do you handle applications that are split between on-premises compute and the cloud replica?

For PostgreSQL, native streaming replication or logical replication keeps the RDS replica current. Monitor `pg_stat_replication` on the primary and alert when lag exceeds your RPO threshold.

```
-- Run on the on-premises primary to monitor replica lag toward RDS
SELECT
    client_addr,
    state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    sent_lsn - replay_lsn AS total_lag_bytes,
    pg_size_pretty(sent_lsn - replay_lsn) AS lag_pretty,
    write_lag,
    flush_lag,
```

```

replay_lag
FROM pg_stat_replication
WHERE application_name = 'rds_replica';

-- If lag_pretty exceeds your RPO threshold (e.g., 100MB ~ 60s at typical write rates),
-- alert your on-call DBA before it becomes a recovery gap problem.

```

When you need to promote the RDS replica after an on-premises failure, the process for RDS PostgreSQL involves breaking the replication and promoting the read replica:

```

# Promote the RDS Read Replica to a standalone instance
aws rds promote-read-replica \
  --db-instance-identifier rds-dr-replica \
  --backup-retention-period 7 \
  --preferred-backup-window "03:00-04:00" \
  --region us-east-1

# Monitor until the instance status becomes "available"
aws rds wait db-instance-available \
  --db-instance-identifier rds-dr-replica

# Update the CNAME record for your application database endpoint
# (Using Route 53 here - adapt to your DNS provider)
aws route53 change-resource-record-sets \
  --hosted-zone-id Z1D633PJN98FT9 \
  --change-batch '{
    "Changes": [{
      "Action": "UPSERT",
      "ResourceRecordSet": {
        "Name": "db.internal.example.com",
        "Type": "CNAME",
        "TTL": 60,
        "ResourceRecords": [{"Value": "rds-dr-replica.c9akciq32.us-east-1.rds.amazonaws.com"}]
      }
    }]
  }'

```

Set your DNS TTL to 60 seconds — not 300 or 3600. When you need to cut over, you want the change to propagate in one minute, not five.

On-Premises as DR for Cloud Production

This is less common but occurs when regulatory requirements mandate that a copy of data remain in a specific physical location, or when cloud costs make on-premises a more economical secondary tier.

For RDS PostgreSQL, you cannot configure standard streaming replication *from* RDS to an external server the way you would between two self-managed instances. RDS does not expose the primary's WAL stream for external replication subscription without specific configuration. You have two options:

1. Use RDS logical replication (`rds.logical_replication = 1` in the parameter group) to publish from RDS to an on-premises PostgreSQL subscriber.
2. Use AWS DMS in reverse — RDS as the source, on-premises as the target.

```

# Enable logical replication on RDS PostgreSQL via Parameter Group
aws rds modify-db-parameter-group \
  --db-parameter-group-name production-pg15-params \
  --parameters 'ParameterName=rds.logical_replication,ParameterValue=1,ApplyMethod=pending-reboot'

# Reboot is required for this parameter
aws rds reboot-db-instance \
  --db-instance-identifier prod-pg-primary

```

```
# After reboot, on the RDS instance, create the publication:
# (Connect via psql first)
# CREATE PUBLICATION onprem_dr_pub FOR ALL TABLES;

# On the on-premises PostgreSQL DR server, create the subscription:
# CREATE SUBSCRIPTION onprem_dr_sub
# CONNECTION 'host=prod-pg-primary.c9akciq32.us-east-1.rds.amazonaws.com
#             port=5432 dbname=production_db
#             user=replication_user password=SecurePassword'
# PUBLICATION onprem_dr_pub;
```

27.4 Hybrid Identity and Access Management

Network connectivity handles the infrastructure layer. Identity handles the security layer. In your on-premises data center, you used `pg_hba.conf`, Active Directory service accounts, or LDAP for database authentication. In the cloud, IAM controls who and what can connect. Hybrid environments need a coherent identity strategy that spans both worlds.

AWS IAM Database Authentication Across Boundaries

RDS supports IAM authentication for PostgreSQL and MySQL. Instead of a username/password, the application generates a short-lived authentication token using its AWS IAM credentials. The token is valid for 15 minutes. This works perfectly for applications running inside AWS. For on-premises applications, it requires that the on-premises server has access to IAM credentials — either through an IAM user’s access key (acceptable for service accounts, not ideal) or through AWS STS and cross-account role assumption.

The practical pattern: create an IAM user specifically for on-premises database access, assign it a policy that allows `rds-db:connect` for the specific RDS resource and database user, and rotate the access keys using AWS Secrets Manager rotation.

```
import boto3
import psycopg2

def connect_to_rds_from_onpremises(
    db_host: str,
    db_port: int,
    db_name: str,
    db_user: str,
    region: str,
    aws_access_key_id: str,
    aws_secret_access_key: str
) -> psycopg2.extensions.connection:
    """
    Connect to RDS PostgreSQL from an on-premises server using IAM authentication.
    The IAM user must have rds-db:connect permission on the db resource.
    """
    # Generate the auth token using STS credentials
    session = boto3.Session(
        aws_access_key_id=aws_access_key_id,
        aws_secret_access_key=aws_secret_access_key,
        region_name=region
    )

    rds_client = session.client('rds')

    # This generates a token valid for 15 minutes
    auth_token = rds_client.generate_db_auth_token(
        DBHostname=db_host,
```

```

        Port=db_port,
        DBUsername=db_user,
        Region=region
    )

    # Connect using the token as the password
    # SSL is required when using IAM auth
    conn = psycopg2.connect(
        host=db_host,
        port=db_port,
        database=db_name,
        user=db_user,
        password=auth_token,
        sslmode='verify-full',
        sslrootcert='/path/to/us-east-1-bundle.pem'
    )

    return conn

# Usage from on-premises server
connection = connect_to_rds_from_onpremises(
    db_host='prod-rds.c9akciq32.us-east-1.rds.amazonaws.com',
    db_port=5432,
    db_name='production',
    db_user='app_iam_user',
    region='us-east-1',
    aws_access_key_id='AKIAIOSFODNN7EXAMPLE',
    aws_secret_access_key='wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY'
)

```

The IAM user's policy for this access:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["rds-db:connect"],
      "Resource": [
        "arn:aws:rds-db:us-east-1:123456789012:dbuser:db-ABC123/app_iam_user"
      ]
    }
  ]
}

```

Note the resource ARN format: `rds-db` (not `rds`), followed by the `db-resource-id` (available from `aws rds describe-db-instances`, look for `DbiResourceId`), and the specific database user name. The database user must exist in PostgreSQL and must have the `rds_iam` role granted:

```

-- On the RDS PostgreSQL instance, create the IAM-authenticated user
CREATE USER app_iam_user WITH LOGIN;
GRANT rds_iam TO app_iam_user;

-- Grant this user the permissions it needs
GRANT CONNECT ON DATABASE production TO app_iam_user;
GRANT USAGE ON SCHEMA public TO app_iam_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_iam_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_iam_user;

```

Active Directory Integration for SQL Server Hybrid

For SQL Server shops, Active Directory integration is typically the correct identity bridge. On-premises SQL Server already authenticates Windows accounts via Kerberos. Azure SQL Managed Instance supports Azure AD-joined authentication, and Azure AD Connect can synchronize your on-premises AD with Azure AD, giving you a unified identity surface.

For plain Azure SQL Database (not Managed Instance), you use Azure AD authentication with on-premises AD synchronized through Azure AD Connect. The SQL Server client on-premises connects using an Azure AD token rather than a SQL login.

```
-- On Azure SQL Database: create a contained database user from Azure AD
-- (Run as Azure AD admin)
CREATE USER [corp\svc_appaccount] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [corp\svc_appaccount];
ALTER ROLE db_datawriter ADD MEMBER [corp\svc_appaccount];

-- On-premises SQL Server: check linked server connectivity using Windows auth passthrough
-- Create a linked server from on-premises SQL Server to Azure SQL
EXEC sp_addlinkedserver
    @server = N'AzureSQLLinked',
    @srvproduct = N'',
    @provider = N'MSOLEDBSQL',
    @datasrc = N'yourserver.database.windows.net',
    @catalog = N'production_db';

EXEC sp_addlinkedsrvlogin
    @rmtsrvname = N'AzureSQLLinked',
    @useself = N'FALSE',
    @locallogin = NULL,
    @rmtuser = N'svc_appaccount@corp.example.com',
    @rmtpassword = N'ServiceAccountPassword';

-- Test the linked server
SELECT TOP 10 * FROM AzureSQLLinked.production_db.dbo.orders;
```

Linked servers across cloud boundaries work — but they carry the same latency penalty described in section 27.1. A distributed query that joins a local on-premises table to an Azure SQL table via linked server will ship the entire remote result set across the network unless the optimizer pushes the predicate remotely. Test your execution plans with `SET STATISTICS IO ON` and verify that remote row counts are bounded.

27.5 Cross-Boundary Query Patterns and Their Performance Implications

Section 27.1 established that latency is the enemy of synchronous cross-boundary queries. This section goes deeper into the specific patterns that appear in hybrid architectures and how to engineer around their limitations.

The Linked Server Anti-Pattern

Developers who discover linked servers (SQL Server) or `dblink/postgres_fdw` (PostgreSQL) sometimes treat them as a transparent way to query remote tables as if they are local. This works adequately in same-datacenter scenarios. Across a hybrid cloud boundary, it becomes a performance problem at any meaningful scale.

```
-- SQL Server: this query looks innocent but can be expensive across a WAN link
SELECT
    o.order_id,
    o.customer_id,
    c.customer_name,      -- c comes from on-premises
    p.product_name       -- p comes from Azure SQL via linked server
FROM dbo.orders o        -- local (Azure SQL)
INNER JOIN AzureSQLLinked.production.dbo.customers c -- remote (on-premises)
ON o.customer_id = c.customer_id
```

```

INNER JOIN dbo.products p                                -- local (Azure SQL)
  ON o.product_id = p.product_id
WHERE o.order_date >= '2024-01-01';

-- Check how SQL Server plans this
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
-- If the Remote Query operator shows millions of rows, you have a problem.
-- The optimizer may fetch the entire customers table and filter locally.

```

The fix is rarely at the query level. The fix is to replicate the lookup tables (customers, products) to both sides of the boundary so cross-boundary joins become local joins with a small synchronization lag.

```

-- PostgreSQL equivalent using postgres_fdw
-- Demonstrate the same anti-pattern and the monitoring approach

-- First, set up the foreign server and user mapping
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

CREATE SERVER onprem_pg
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host '10.10.5.20', port '5432', dbname 'production_db');

CREATE USER MAPPING FOR current_user
  SERVER onprem_pg
  OPTIONS (user 'fdw_user', password 'FdwP@ssword789');

-- Import the remote schema
IMPORT FOREIGN SCHEMA public
  LIMIT TO (customers, products)
  FROM SERVER onprem_pg
  INTO remote_tables;

-- Check estimated vs. actual row counts for foreign scans
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
  o.order_id,
  c.customer_name
FROM orders o
JOIN remote_tables.customers c ON o.customer_id = c.customer_id
WHERE o.order_date >= '2024-01-01';

-- Look for "Foreign Scan" nodes in the plan.
-- If rows= estimate is far from actual, update statistics:
ANALYZE remote_tables.customers;

```

For `postgres_fdw`, set `use_remote_estimate = true` on the foreign server to let PostgreSQL query the remote server for row count estimates. This dramatically improves plan quality for foreign tables:

```

ALTER SERVER onprem_pg OPTIONS (ADD use_remote_estimate 'true');

```

Materialized Caching Pattern

The correct pattern for frequently-queried reference data that lives on the other side of a cloud boundary is to replicate it locally with a controlled refresh cycle. Call it a materialized cache, a shadow table, or a local replica — the concept is the same.

```

-- PostgreSQL: create a local shadow of the on-premises customers table
-- This runs on the RDS side and refreshes on a schedule

CREATE TABLE customers_shadow (
  customer_id    BIGINT PRIMARY KEY,
  customer_name  TEXT NOT NULL,
  email          TEXT,

```

```

    region          TEXT,
    created_at      TIMESTAMPTZ,
    synced_at       TIMESTAMPTZ DEFAULT now() -- tracks when we last pulled this row
);

CREATE INDEX idx_customers_shadow_region ON customers_shadow (region);

-- Refresh function: pull changes from the foreign table since last sync
CREATE OR REPLACE FUNCTION refresh_customers_shadow()
RETURNS void
LANGUAGE plpgsql
AS $$
DECLARE
    v_last_sync TIMESTAMPTZ;
    v_rows_affected INT;
BEGIN
    SELECT COALESCE(MAX(synced_at), '1970-01-01'::TIMESTAMPTZ)
    INTO v_last_sync
    FROM customers_shadow;

    -- Upsert changes from the foreign table
    INSERT INTO customers_shadow (
        customer_id, customer_name, email, region, created_at, synced_at
    )
    SELECT
        customer_id,
        customer_name,
        email,
        region,
        created_at,
        now()
    FROM remote_tables.customers
    WHERE created_at > v_last_sync
        OR updated_at > v_last_sync -- assumes updated_at column exists on source

    ON CONFLICT (customer_id) DO UPDATE SET
        customer_name = EXCLUDED.customer_name,
        email         = EXCLUDED.email,
        region        = EXCLUDED.region,
        synced_at     = EXCLUDED.synced_at;

    GET DIAGNOSTICS v_rows_affected = ROW_COUNT;

    RAISE NOTICE 'customers_shadow refreshed: % rows upserted', v_rows_affected;
END;
$$;

-- Schedule via pg_cron (if available) or an external scheduler
-- SELECT cron.schedule('*/* * * * *', 'SELECT refresh_customers_shadow()');

-- SQL Server equivalent: shadow table and refresh stored procedure
CREATE TABLE dbo.customers_shadow (
    customer_id    BIGINT        NOT NULL PRIMARY KEY,
    customer_name  NVARCHAR(255) NOT NULL,
    email          NVARCHAR(320) NULL,
    region         NVARCHAR(50)  NULL,
    created_at     DATETIME2      NULL,
    synced_at      DATETIME2      NOT NULL DEFAULT SYSUTCDATETIME()
);

CREATE NONCLUSTERED INDEX ix_customers_shadow_region
ON dbo.customers_shadow (region);
GO

CREATE OR ALTER PROCEDURE dbo.usp_RefreshCustomersShadow
AS
BEGIN
    SET NOCOUNT ON;

```



```
DECLARE @last_sync DATETIME2;
SELECT @last_sync = ISNULL(MAX(synced_at), '19700101')
FROM dbo.customers_shadow;

MERGE dbo.customers_shadow AS tgt
USING (
    SELECT customer_id, customer_name, email, region, created_at
    FROM AzureSQLLinked.production_db.dbo.customers
    WHERE created_at > @last_sync
    OR updated_at > @last_sync
) AS src
ON tgt.customer_id = src.customer_id

WHEN MATCHED THEN UPDATE SET
    tgt.customer_name = src.customer_name,
    tgt.email         = src.email,
    tgt.region        = src.region,
    tgt.synced_at     = SYSUTCDATETIME()

WHEN NOT MATCHED BY TARGET THEN INSERT (
    customer_id, customer_name, email, region, created_at, synced_at
) VALUES (
    src.customer_id, src.customer_name, src.email,
    src.region, src.created_at, SYSUTCDATETIME()
);

PRINT CAST(@@ROWCOUNT AS VARCHAR) + ' rows merged into customers_shadow.';
END;
GO
```

The shadow table approach decouples your application queries from the remote source. When the VPN connection degrades or the on-premises server is under maintenance, your cloud-side application continues to work against the last-synced data. The lag is controlled and predictable — you know exactly how stale the shadow can be because you control the refresh interval.

27.6 Monitoring a Hybrid Database Environment

Monitoring is harder in hybrid environments because your observability tools exist on both sides of the boundary, and combining their signals into a coherent view requires deliberate design.

Centralized Metrics Collection

The pattern that works at scale is to centralize all database metrics into your cloud monitoring layer and push on-premises metrics to it. CloudWatch agent, Azure Monitor agent, or the Prometheus `node_exporter` with `remote_write` can all collect on-premises metrics and ship them to the cloud.

For PostgreSQL, the `postgres_exporter` (Prometheus) running on your on-premises server can scrape database metrics and push them to a centralized Prometheus/Thanos or Victoria Metrics deployment.

```
# prometheus.yml: scrape config for both on-premises and RDS PostgreSQL
# On-premises instances are scraped directly.
# RDS instances are scraped via postgres_exporter running in the VPC.

global:
  scrape_interval: 30s
  evaluation_interval: 30s

scrape_configs:
  - job_name: 'postgres_onprem'
```

```
static_configs:
  - targets:
    - '10.10.5.20:9187' # postgres_exporter on on-premises server
    - '10.10.5.21:9187'
  labels:
    environment: 'onpremises'
    datacenter: 'dc-east'

- job_name: 'postgres_rds'
  static_configs:
    - targets:
      - '10.100.1.50:9187' # postgres_exporter in VPC, scraping RDS
  labels:
    environment: 'aws'
    region: 'us-east-1'
```

The `postgres_exporter` configuration for RDS must use a connection string pointing at the RDS endpoint — the exporter runs inside your VPC and connects to RDS over the private endpoint, then exposes its HTTP metrics port for Prometheus to scrape.

Alerting Across Boundaries

Define alerts that specifically watch for hybrid-specific failure modes. These are the signals that pure cloud or pure on-premises setups never need to alert on:

1. **Replication slot lag exceeding disk safety threshold** — On-premises primary running out of WAL disk because the cloud subscriber is behind.
2. **DMS task latency spike** — Network congestion or VPN instability causing CDC lag to build.
3. **Cross-boundary query response time degradation** — P99 latency on federated queries rising above acceptable threshold.
4. **VPN tunnel state change** — The network layer itself going down is a database event from your perspective.
5. **Shadow table staleness** — The refresh procedure last ran more than N minutes ago, meaning your local cache is drifting.

Prometheus alerting rules for hybrid-specific conditions

```
groups:
  - name: hybrid_database_alerts
    rules:

      - alert: ReplicationSlotLagCritical
        expr: |
          pg_replication_slots_pg_wal_lsn_diff{slot_type="logical"} > 10737418240
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "Replication slot {{ $labels.slot_name }} lag exceeds 10GB"
          description: >
            The logical replication slot {{ $labels.slot_name }} on
            {{ $labels.instance }} has {{ $value | humanize1024 }}B of unconsumed WAL.
            If the subscriber does not catch up, disk will fill and the primary will stop.

      - alert: ShadowTableStalenessWarning
        expr: |
          (time() - pg_shadow_table_last_refresh_epoch{table="customers_shadow"}) > 600
        for: 2m
        labels:
          severity: warning
        annotations:
```

```
summary: "customers_shadow has not refreshed in over 10 minutes"
description: >
  The shadow table customers_shadow on {{ $labels.instance }}
  last refreshed {{ $value | humanizeDuration }} ago.
  Check the refresh job and VPN connectivity.

- alert: DMSReplicationLagCritical
  expr: |
    aws_dms_cdc_latency_target_maximum > 300
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "DMS task replication lag exceeds 300 seconds"
    description: >
      AWS DMS task on replication instance {{ $labels.replication_instance }}
      has a target latency of {{ $value }}s.
      Investigate network connectivity and DMS task errors.
```

Tracing Queries Across Boundaries in SQL Server

For SQL Server, Extended Events and Query Store give you the on-premises view. Azure SQL has its own Query Store, `sys.dm_exec_requests`, and Azure Monitor. Correlating a slow distributed query that hits both sides requires matching timestamps and session identifiers manually — there is no native end-to-end tracing for linked-server queries.

The pragmatic approach: log the session ID and the start timestamp of every distributed query in your application layer, then correlate with SQL Server's DMVs using that information after the fact.

```
-- SQL Server: capture distributed query performance with Extended Events
-- Run this on the on-premises instance to capture remote query costs

CREATE EVENT SESSION [hybrid_query_trace] ON SERVER
ADD EVENT sqlserver.sql_statement_completed (
  WHERE (
    sqlserver.database_name = N'production_db'
    AND package0.greater_than_equal_uint64(duration, 1000000) -- >= 1 second
  )
  ACTION (
    sqlserver.sql_text,
    sqlserver.session_id,
    sqlserver.client_hostname,
    sqlserver.username
  )
),
ADD EVENT sqlserver.rpc_completed (
  WHERE (
    package0.greater_than_equal_uint64(duration, 1000000)
  )
  ACTION (
    sqlserver.sql_text,
    sqlserver.session_id
  )
)
ADD TARGET package0.ring_buffer (SET max_memory = 51200)
WITH (
  MAX_DISPATCH_LATENCY = 5 SECONDS,
  TRACK_CAUSALITY = ON -- enables cross-event correlation via activity_id
);

ALTER EVENT SESSION [hybrid_query_trace] ON SERVER STATE = START;
```

27.7 Managing Schema Changes Across Hybrid Environments

Schema changes are the most operationally treacherous part of hybrid database management. When your production schema exists in two places — on-premises and cloud — every migration must be coordinated. Applied out of order, a schema change will break replication, corrupt data, or crash applications.

The Synchronization Problem

Logical replication (both native PostgreSQL and DMS) replicates data but not DDL in most configurations. If you add a column to the on-premises source table, the RDS target does not automatically receive that column. Your replication task will fail when it encounters an INSERT with a column count the target does not recognize.

DMS's default behavior on DDL changes is to stop the task with an error. Native PostgreSQL logical replication will fail with a type mismatch or column count error. Azure Data Sync ignores DDL changes entirely and lets data become inconsistent.

The correct migration sequence for adding a column in a hybrid environment:

1. Apply the schema change to the TARGET (cloud) FIRST
2. Verify the change is present and replication is healthy
3. Apply the schema change to the SOURCE (on-premises)
4. Verify replication resumes without errors

This order matters because the target can accept rows that do not fill optional columns (NULL-able additions), but the source cannot replicate data into a column that does not exist on the target. Adding the column to the target first gives you a window of grace.

```
-- Step 1: Apply on the RDS target (PostgreSQL) FIRST
-- Connect to RDS and run:
ALTER TABLE orders ADD COLUMN discount_code TEXT;
ALTER TABLE orders ADD COLUMN discount_amount NUMERIC(10,2) DEFAULT 0.00;

-- Verify logical replication slot is still active and not lagging
SELECT slot_name, active, confirmed_flush_lsn,
       pg_size_pretty(pg_current_wal_lsn() - confirmed_flush_lsn) AS slot_lag
FROM pg_replication_slots;

-- Step 2: Apply on the on-premises source SECOND
-- Connect to on-premises PostgreSQL and run:
ALTER TABLE orders ADD COLUMN discount_code TEXT;
ALTER TABLE orders ADD COLUMN discount_amount NUMERIC(10,2) DEFAULT 0.00;

-- Verify replication is flowing normally after the DDL on the source
SELECT pid, username, application_name, client_addr, state, write_lag, flush_lag, replay_lag
FROM pg_stat_replication;

-- SQL Server equivalent for DMS replication:
-- Step 1: Apply on RDS SQL Server target FIRST
ALTER TABLE dbo.orders ADD discount_code NVARCHAR(50) NULL;
ALTER TABLE dbo.orders ADD discount_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00;

-- Step 2: Verify DMS task is healthy in CloudWatch before proceeding
-- (Check CDCLatencyTarget in CloudWatch - should remain stable)

-- Step 3: Apply on on-premises SQL Server source SECOND
ALTER TABLE dbo.orders ADD discount_code NVARCHAR(50) NULL;
ALTER TABLE dbo.orders ADD discount_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00;

-- MS-CDC captures the DDL event; DMS will handle the new columns on the next batch
```

```
-- Monitor DMS task for errors for 10 minutes after source DDL application
```

Schema Migration Tools in Hybrid Context

Tools like Flyway and Liquibase can manage schema versions across multiple environments, but they need explicit configuration to understand that your “on-premises” and “cloud” targets are logically the same schema lineage.

The pattern that works: treat the on-premises and cloud databases as two members of the same migration graph, but always execute the cloud target migration first (via your CI/CD pipeline connecting to the RDS endpoint), then execute the on-premises migration immediately after (via your CI/CD pipeline connecting to the on-premises endpoint through the VPN).

```
<!-- Liquibase: hybrid migration pipeline configuration -->
<!-- liquibase-cloud.properties - targets RDS first -->
changeLogFile=db/changelog/db.changelog-master.xml
url=jdbc:postgresql://rds-primary.c9akciq32.us-east-1.rds.amazonaws.com:5432/production_db
username=liquibase_runner
password=${LIQUIBASE_PASSWORD}
driver=org.postgresql.Driver

<!-- liquibase-onprem.properties - targets on-premises second -->
changeLogFile=db/changelog/db.changelog-master.xml
url=jdbc:postgresql://10.10.5.20:5432/production_db
username=liquibase_runner
password=${LIQUIBASE_PASSWORD_ONPREM}
driver=org.postgresql.Driver

#!/bin/bash
# hybrid-migration.sh - always apply cloud target before on-premises source

set -e # Exit on any error

echo "=== Applying migration to cloud target (RDS) ==="
liquibase --defaultsFile=liquibase-cloud.properties update

echo "=== Verifying replication health before applying to source ==="
python3 check_replication_lag.py --max-lag-seconds 60 || {
    echo "ERROR: Replication lag too high. Do not apply migration to source yet."
    exit 1
}

echo "=== Applying migration to on-premises source ==="
liquibase --defaultsFile=liquibase-onprem.properties update

echo "=== Migration complete. Monitoring replication for 5 minutes ==="
for i in $(seq 1 10); do
    sleep 30
    python3 check_replication_lag.py --max-lag-seconds 120
done

echo "=== Hybrid migration pipeline completed successfully ==="
```

The `check_replication_lag.py` script queries `pg_stat_replication` on the on-premises primary and exits with a non-zero code if lag exceeds the threshold. This gate prevents applying source-side DDL when the target is already behind, which would compound the risk.

27.8 Cost Engineering in Hybrid Architectures

Hybrid architectures incur costs on both sides of the boundary simultaneously. On-premises hardware, power, and administration costs do not disappear when you add cloud costs. Network egress pricing adds a third dimension. Understanding where the money goes lets you make trade-offs rather than just absorb them.

Data Transfer Costs Across the Hybrid Boundary

AWS charges for data transfer OUT of AWS to the internet (and in some configurations to on-premises over VPN). Direct Connect has its own data transfer rates. Azure charges for egress. GCP charges for egress. The pattern is consistent: inbound data to the cloud is typically free, outbound is not.

For hybrid database replication, your primary concern is the volume of change data flowing from source to target — or from cloud primary to on-premises DR. If your database generates 50 GB of WAL changes per day and you are shipping all of it across Direct Connect at the outbound rate, that is a meaningful monthly line item.

Minimize cross-boundary data movement through: 1. **Filtering replication scope**: replicate only the tables that need to exist on both sides. DMS table mappings and PostgreSQL publication table lists let you be selective. 2. **Compressing replication streams**: PostgreSQL logical replication does not inherently compress WAL. DMS compresses its transfer stream. For Direct Connect, work with your network team on link-level compression. 3. **Co-locating the application with the data it reads most**: if 90% of your application's reads are against the cloud database, do not run the application on-premises and pay for every result set to traverse the boundary.

Right-Sizing Your Hybrid Footprint

A common mistake is running both the on-premises and cloud databases at full production sizing simultaneously. If your cloud database is a DR replica that is rarely queried, size it as a DR replica — smaller instance class, minimal IOPS provisioning, no Multi-AZ until you are within a maintenance window of promoting it. Promote it to full production sizing as part of your DR activation runbook.

```
# Example: DR replica running at reduced capacity in normal operation
aws rds create-db-instance-read-replica \
  --db-instance-identifier prod-dr-replica \
  --source-db-instance-identifier prod-primary \
  --db-instance-class db.r6g.large \           # Smaller than production's db.r6g.4xlarge
  --publicly-accessible false \
  --region us-west-2

# The DR activation runbook scales it up BEFORE promotion:
aws rds modify-db-instance \
  --db-instance-identifier prod-dr-replica \
  --db-instance-class db.r6g.4xlarge \
  --apply-immediately

aws rds wait db-instance-available \
  --db-instance-identifier prod-dr-replica

# Then promote
aws rds promote-read-replica \
  --db-instance-identifier prod-dr-replica
```

One caveat: apply the scale-up before promoting, not after. After promotion, the instance is no longer a replica and carries full production traffic. Scaling a primary under load is riskier than scaling a cold replica.

Key Takeaways

- **Network is the foundation:** every hybrid database pattern — replication, cross-boundary queries, failover — depends on stable, low-latency connectivity. Use Direct Connect or ExpressRoute for production workloads; accept Site-to-Site VPN only for low-bandwidth or non-critical scenarios. Measure actual latency before designing your query topology.
 - **Synchronous cross-boundary queries do not scale:** latency compounds across every query in a page load or transaction. Use native replication, DMS CDC, or the materialized shadow table pattern to bring frequently-needed data to the same side as its consumers. Reserve linked servers and `postgres_fdw` for administrative queries and low-volume operations.
 - **DDL changes must be sequenced target-first:** when logical replication is in place, always apply schema changes to the replication target before the source. Applying them to the source first will break replication. Build this ordering into your migration automation with a replication lag check as a gate between the two steps.
 - **Replication slots and WAL accumulation are on-premises-specific risks:** cloud databases manage WAL retention with built-in safeguards; on-premises servers can fill their `pg_wal` directory and crash if a logical replication subscriber falls behind. Set `max_slot_wal_keep_size`, monitor slot lag aggressively, and build alerts that fire well before disk is exhausted.
 - **Right-size your hybrid footprint continuously:** DR replicas do not need production-class instance sizing until the moment they are activated. Run them smaller, scale up as part of your DR runbook before promotion, and reduce cloud spend during normal operation without sacrificing recovery capability.
-

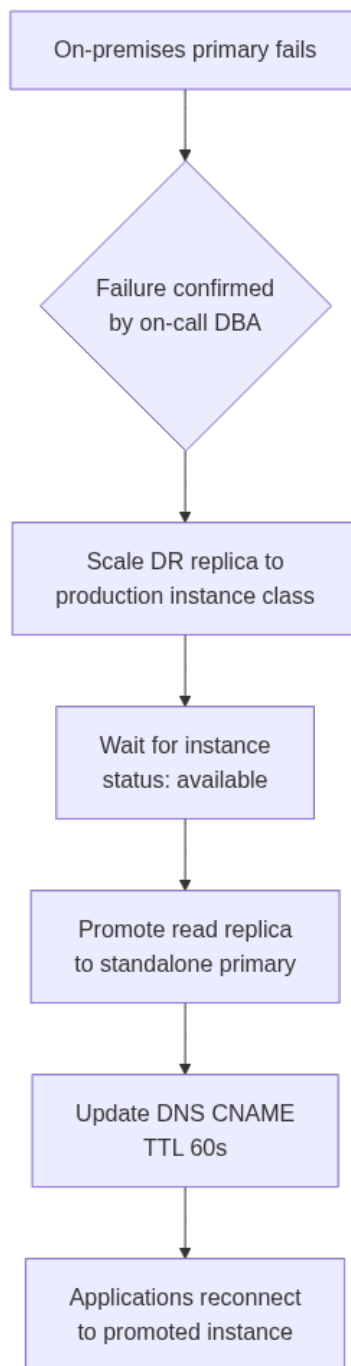


Figure 18: Cloud-as-DR failover sequence: scale before promotion, promotion before DNS cutover.

Chapter 28: Avoiding Vendor Lock-In Without Over-Engineering

Vendor lock-in is the anxiety that keeps cloud architects up at night and the excuse that keeps organizations running aging on-premises hardware long past its useful life. The fear is legitimate: AWS Aurora uses a proprietary storage layer, Azure SQL has T-SQL extensions that don't exist in PostgreSQL, and Cloud Spanner's distributed SQL dialect diverges from ANSI SQL in ways that matter. But the response to that fear — building elaborate abstraction layers, refusing managed features, or maintaining parallel deployments on three clouds simultaneously “just in case” — creates its own category of engineering debt. This chapter is about finding the line between prudent portability and counterproductive over-engineering. You will learn which cloud-specific features are worth adopting despite the lock-in they create, which abstractions are worth building, where portable infrastructure-as-code genuinely pays dividends, and how to migrate out if you ever need to — without having spent five years pretending you would never use the cloud's native capabilities.

28.1 Understanding What Lock-In Actually Means in Practice

Before building defenses against vendor lock-in, you need to be precise about what you are actually locked into. The cloud database ecosystem has at least four distinct layers where lock-in operates differently, and conflating them leads to bad architectural decisions.

Data lock-in is the most serious. If your data is in a format that only one vendor can read — proprietary binary logs, Spanner's internal encoding, Aurora's custom storage pages — you cannot move without a full export-import cycle. In practice, managed PostgreSQL on RDS, Azure Database for PostgreSQL, and Cloud SQL all store data in standard PostgreSQL format. A `pg_dump` from RDS will restore into a self-managed PostgreSQL instance. This is a meaningful and often underappreciated fact: the data itself is portable because the engines are standard.

Feature lock-in is where most architectural decisions actually happen. Aurora's fast failover (typically under 30 seconds) depends on its shared storage architecture — you cannot replicate that behavior on standard PostgreSQL without significant infrastructure investment. Azure SQL's Always Encrypted with secure enclaves is a feature with no direct equivalent elsewhere. If you build your application to depend on Aurora global database's 1-second RPO cross-region replication, you are genuinely locked into Aurora's architecture, not just AWS's billing.

Operational lock-in is often invisible until you try to leave. Your team knows CloudWatch dashboards, your alerting is wired into SNS, your runbooks reference AWS Systems Manager Pa-

parameter Store for connection strings. When you evaluate migration cost, this is frequently the largest component — not the data movement, but the operational muscle memory and tooling.

Pricing lock-in is what vendors want you to confuse with architectural lock-in. Egress costs, reserved instance commitments, and licensing bundles create financial friction around leaving. These are real but they are financial constraints, not technical constraints, and they should be analyzed separately.

The productive framing is this: accept feature lock-in deliberately for features that provide genuine operational value, maintain data portability as a non-negotiable baseline, and invest in operational portability proportionally to the probability and cost of migration.

A useful exercise before any new feature adoption is to complete this sentence: “If we need to migrate off this platform in 18 months, the cost of having adopted this feature is ____.” For Aurora’s storage layer, that cost is a `pg_dump/restore` cycle plus replication lag during cutover — measurable and manageable. For a deep integration with Spanner’s interleaved tables and `THEN` expressions in change streams, the cost is a significant application rewrite.

28.2 The Abstraction Layer Trap — and Where Abstractions Actually Help

The classic over-engineering response to vendor lock-in is a database abstraction layer: an ORM or a custom SQL wrapper that translates generic queries into vendor-specific SQL, theoretically making your application portable across PostgreSQL, MySQL, SQL Server, and anything else. In practice, these layers tend to optimize for the lowest common denominator, preventing you from using window functions efficiently, forcing suboptimal query patterns to maintain compatibility, and adding a debugging layer between your application and the actual query plan.

The distinction that matters is between **application-level abstractions** (often harmful) and **infrastructure-level abstractions** (often valuable).

Application-level abstractions that try to hide database vendor differences from your query logic almost always create more problems than they solve. The databases you are running in production — Aurora PostgreSQL, Azure SQL, Cloud SQL — do not have identical SQL dialects, query planners, or performance characteristics. Code that treats them as interchangeable will perform well on none of them.

Infrastructure-level abstractions — specifically Terraform modules that can provision an equivalent logical database tier across providers — provide genuine value without compromising your ability to use native features. The key is that the abstraction operates at the provisioning level, not the query level.

Here is a concrete example. Rather than a monolithic Terraform configuration locked to one provider, you can structure your database infrastructure modules to accept a provider as a variable while still allowing provider-specific feature blocks:

```
# modules/managed_postgres/variables.tf
variable "provider_type" {
  type        = string
  description = "Cloud provider: aws, azure, or gcp"
  validation {
```

```
        condition      = contains(["aws", "azure", "gcp"], var.provider_type)
        error_message = "Provider must be aws, azure, or gcp."
    }
}

variable "instance_class" {
    type = map(string)
    default = {
        aws   = "db.r6g.2xlarge"
        azure = "GP_Gen5_8"
        gcp   = "db-custom-8-32768"
    }
}

variable "high_availability" {
    type    = bool
    default = true
}

variable "storage_gb" {
    type    = number
    default = 500
}

variable "engine_version" {
    type    = string
    default = "15"
}

# modules/managed_postgres/main.tf
# AWS path
resource "aws_db_instance" "postgres" {
    count = var.provider_type == "aws" ? 1 : 0

    identifier      = var.db_identifier
    engine           = "postgres"
    engine_version   = var.engine_version
    instance_class   = var.instance_class["aws"]
    allocated_storage = var.storage_gb
    storage_type      = "gp3"
    storage_encrypted = true
    multi_az         = var.high_availability
    deletion_protection = true

    db_subnet_group_name = var.subnet_group_name
    vpc_security_group_ids = var.security_group_ids

    backup_retention_period = 14
    backup_window           = "03:00-04:00"
    maintenance_window      = "sun:04:30-sun:05:30"

    performance_insights_enabled = true
    monitoring_interval          = 60

    tags = var.common_tags
}

# GCP path
resource "google_sql_database_instance" "postgres" {
    count = var.provider_type == "gcp" ? 1 : 0

    name            = var.db_identifier
    database_version = "POSTGRES_${var.engine_version}"
    region          = var.region

    settings {
        tier                = var.instance_class["gcp"]
        availability_type   = var.high_availability ? "REGIONAL" : "ZONAL"
        disk_size           = var.storage_gb
        disk_type           = "PD_SSD"
    }
}
```

```
backup_configuration {
  enabled = true
  point_in_time_recovery_enabled = true
  backup_retention_settings {
    retained_backups = 14
  }
}

insights_config {
  query_insights_enabled = true
  query_string_length    = 1024
  record_application_tags = true
}

deletion_protection = true
}
```

This structure gives you a consistent interface at the Terraform module level while still allowing each provider's resource to use its own native features. The abstraction is in the module interface, not in the SQL or in some runtime compatibility shim.

The places where abstraction genuinely pays:

- **Connection string management:** Store connection details in a secrets manager (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager) behind a consistent naming convention. Your application retrieves `db/prod/primary` and the secret's content changes during migration, not the application code.
- **Migration tooling:** Tools like Flyway and Liquibase handle DDL migrations in a provider-agnostic way and are worth using for that reason — not to abstract your queries, but to manage schema evolution consistently.
- **Monitoring aggregation:** A single observability layer (Datadog, Grafana with multiple datasources, or even a well-structured CloudWatch-to-BigQuery pipeline) that normalizes metrics across providers gives you genuine operational flexibility.

The places where abstraction creates problems:

- Query-level ORM configurations that disable provider-specific index types, JSON operators, or parallel query hints
- Connection pooling layers that don't understand RDS Proxy's IAM authentication or Azure SQL's connection string requirements
- Schema management tooling that caps you at ANSI SQL DDL and prevents you from using partition pruning, BRIN indexes, or columnstore indexes

28.3 Choosing Which Proprietary Features Are Worth the Lock-In

Not all proprietary features carry equal lock-in cost. Some Aurora-specific behaviors can be replicated with reasonable effort on standard PostgreSQL. Others would require rewriting your entire replication topology. The decision framework is straightforward: measure the operational value delivered, measure the realistic migration cost, and decide deliberately.

Features worth adopting despite lock-in:

Aurora's shared storage architecture affects failover time, storage autoscaling, and read replica behavior. The operational value — 20-30 second failovers versus 60-120 seconds for standard RDS, automatic storage growth up to 128 TiB without provisioning, up to 15 read replicas sharing the same underlying storage — is substantial for production workloads. The migration cost is a standard `pg_dump/restore` or DMS migration. Adopt this.

RDS Proxy terminates connections at the proxy layer and multiplexes them against your database, reducing connection overhead for serverless workloads that might otherwise exhaust `max_connections` — a limit that scales with instance memory and is reached surprisingly fast under Lambda-style burst connection patterns. The migration cost if you leave AWS is that you provision PgBouncer yourself. Adopt this.

Azure SQL Hyperscale's rapid compute scaling and its distributed storage architecture — which decouples storage growth from provisioning and avoids the multi-hour resizing operations that standard Azure SQL tiers require as databases grow large — are meaningfully different from standard SQL Server behavior. If your workload needs those characteristics, the value is real.

AlloyDB's columnar engine, which maintains a column store alongside the row store and automatically routes analytical queries to it, eliminates a class of architecture decisions about HTAP workloads. The migration cost is that on standard PostgreSQL you would provision a separate analytical replica or implement `pg_partman` more aggressively.

Features where you should think harder before adopting:

Cloud Spanner deserves special attention because it is the cloud database most unlike anything in the traditional DBA toolkit. Spanner provides horizontal write scaling across regions with strong consistency and true ACID transactions — capabilities that do not exist in any PostgreSQL or SQL Server deployment. But Spanner's SQL dialect has meaningful differences: no stored procedures as of 2024, limited subquery support in some contexts, a batch mutation API that does not map cleanly to standard INSERT/UPDATE patterns, and pricing that scales dramatically with cross-region writes. If you are adopting Spanner, you are adopting it for its unique capabilities, and the portability question is largely moot — you cannot replicate what Spanner does on anything else.

Aurora global database's cross-region replication (typical replication lag under 1 second, RPO near zero for DR scenarios) is a compelling capability, but it binds your DR architecture to Aurora specifically. If your DR requirement could be met by PostgreSQL streaming replication with 5-10 second lag, you have more flexibility.

Azure SQL's Always Encrypted with secure enclaves provides encryption that the database engine itself cannot decrypt — the decryption happens inside a trusted execution environment. This is a genuine security capability for regulated workloads, but it requires the Always Encrypted driver on the application side and creates a hard dependency on the Azure SQL engine.

Here is a practical checklist for evaluating any proprietary feature:

```
-- Document your proprietary feature dependencies explicitly
-- Run this against your Azure SQL instance to identify
-- features that have no direct PostgreSQL equivalent

SELECT
  o.name AS object_name,
  o.type_desc,
  m.definition,
  CASE
    WHEN m.definition LIKE '%COLUMNSTORE%'
    THEN 'Columnstore index - Aurora/AlloyDB equivalent exists'
```

```

    WHEN m.definition LIKE '%TRY_CONVERT%' OR m.definition LIKE '%TRY_CAST%'
    THEN 'T-SQL safe conversion - rewrite required for PostgreSQL'
    WHEN m.definition LIKE '%@@ROWCOUNT%'
    THEN 'T-SQL rowcount - use GET DIAGNOSTICS in PostgreSQL'
    WHEN m.definition LIKE '%NOLOCK%' OR m.definition LIKE '%WITH (NOLOCK)%'
    THEN 'Table hints - no equivalent in PostgreSQL MVCC model'
    WHEN m.definition LIKE '%MERGE%'
    THEN 'MERGE statement - INSERT ON CONFLICT in PostgreSQL 9.5+'
    WHEN m.definition LIKE '%OPENROWSET%' OR m.definition LIKE '%OPENQUERY%'
    THEN 'Linked server / external query - no direct equivalent'
    ELSE 'Review manually'
  END AS portability_note
FROM sys.sql_modules m
JOIN sys.objects o ON m.object_id = o.object_id
WHERE o.type IN ('P', 'FN', 'IF', 'TF', 'V', 'TR')
  AND (
    m.definition LIKE '%COLUMNSTORE%'
    OR m.definition LIKE '%TRY_CONVERT%'
    OR m.definition LIKE '%@@ROWCOUNT%'
    OR m.definition LIKE '%NOLOCK%'
    OR m.definition LIKE '%MERGE%'
    OR m.definition LIKE '%OPENROWSET%'
    OR m.definition LIKE '%OPENQUERY%'
  )
ORDER BY o.type_desc, o.name;

```

Run an equivalent analysis against your PostgreSQL workloads for Aurora-specific extensions:

```

-- Identify Aurora/RDS-specific extension dependencies
-- Run on your RDS PostgreSQL instance
SELECT
  e.extname AS extension_name,
  e.extversion,
  CASE e.extname
    WHEN 'aws_s3' THEN 'AWS-specific: S3 import/export. Use COPY with foreign wrapper'
    ↪ elsewhere.'
    WHEN 'aws_commons' THEN 'AWS-specific: Helper types for aws_s3.'
    WHEN 'aws_ml' THEN 'AWS-specific: SageMaker integration. No direct equivalent.'
    WHEN 'pg_hint_plan' THEN 'Available on self-managed PostgreSQL. Lower lock-in risk.'
    WHEN 'pgaudit' THEN 'Available everywhere. No lock-in.'
    WHEN 'pg_stat_statements' THEN 'Available everywhere. No lock-in.'
    WHEN 'pg_partman' THEN 'Available everywhere. No lock-in.'
    WHEN 'postgis' THEN 'Available everywhere. No lock-in.'
    WHEN 'vector' THEN 'pgvector - available on RDS, Aurora, AlloyDB, Cloud SQL,'
    ↪ self-managed.'
    ELSE 'Review for portability'
  END AS portability_assessment
FROM pg_extension e
ORDER BY
  CASE WHEN e.extname LIKE 'aws_%' THEN 0 ELSE 1 END,
  e.extname;

```

28.4 Portable Infrastructure Patterns That Earn Their Complexity

Some cross-provider infrastructure patterns are genuinely worth building, not because you expect to migrate, but because they encode good practices that benefit you regardless of which cloud you are running on.

Pattern 1: Provider-Agnostic Secret Management

Connection credentials should never be hardcoded or stored in application configuration files. The exact implementation differs across providers, but the interface your application uses should be consistent:

```

# db_connection.py
# Abstracts secret retrieval across AWS, Azure, and GCP
# without abstracting the database connection itself

import os
import json
import boto3
from azure.keyvault.secrets import SecretClient
from azure.identity import DefaultAzureCredential
from google.cloud import secretmanager
import psycpg2
from typing import Dict

def get_db_credentials(secret_name: str) -> Dict[str, str]:
    """
    Retrieve database credentials from the appropriate secrets manager
    based on the CLOUD_PROVIDER environment variable.

    Returns a dict with keys: host, port, dbname, user, password
    """
    provider = os.environ.get("CLOUD_PROVIDER", "aws").lower()

    if provider == "aws":
        client = boto3.client(
            "secretsmanager",
            region_name=os.environ.get("AWS_REGION", "us-east-1")
        )
        response = client.get_secret_value(SecretId=secret_name)
        return json.loads(response["SecretString"])

    elif provider == "azure":
        vault_url = os.environ["AZURE_KEY_VAULT_URL"]
        credential = DefaultAzureCredential()
        client = SecretClient(vault_url=vault_url, credential=credential)
        # Azure Key Vault stores individual values, not JSON blobs
        # Convention: secret_name-host, secret_name-password, etc.
        return {
            "host": client.get_secret(f"{secret_name}-host").value,
            "port": client.get_secret(f"{secret_name}-port").value,
            "dbname": client.get_secret(f"{secret_name}-dbname").value,
            "user": client.get_secret(f"{secret_name}-user").value,
            "password": client.get_secret(f"{secret_name}-password").value,
        }

    elif provider == "gcp":
        project_id = os.environ["GCP_PROJECT_ID"]
        client = secretmanager.SecretManagerServiceClient()
        name = f"projects/{project_id}/secrets/{secret_name}/versions/latest"
        response = client.access_secret_version(request={"name": name})
        return json.loads(response.payload.data.decode("UTF-8"))

    else:
        raise ValueError(f"Unsupported cloud provider: {provider}")

def get_connection(secret_name: str) -> psycpg2.extensions.connection:
    """
    Return a psycpg2 connection using credentials from the appropriate
    secrets manager. SSL is enforced regardless of provider.
    """
    creds = get_db_credentials(secret_name)

    conn = psycpg2.connect(
        host=creds["host"],
        port=int(creds.get("port", 5432)),
        dbname=creds["dbname"],
        user=creds["user"],
        password=creds["password"],
        sslmode="require",
        connect_timeout=10,
    )

```

```

        options="-c statement_timeout=30000"
    )
    return conn

```

The important thing here is what this abstraction does *not* do: it does not abstract your SQL, your query patterns, or your connection pooling strategy. The application still uses psycopg2 directly. Only the credential retrieval is abstracted, which is the piece that legitimately differs across providers.

Pattern 2: Database Migration with Flyway Across Providers

Schema versioning through Flyway or Liquibase is one of the few application-layer tools that earns its place in a cloud database stack. The tooling is the same; only the JDBC URL changes:

```

# Flyway configuration for AWS RDS PostgreSQL
flyway \
  -url="jdbc:postgresql://${RDS_ENDPOINT}:5432/${DB_NAME}?ssl=true&sslmode=require" \
  -user="${DB_USER}" \
  -password="${DB_PASSWORD}" \
  -locations="filesystem:./migrations" \
  -baselineOnMigrate=true \
  migrate

# Same Flyway configuration for Azure Database for PostgreSQL
flyway \
  -url="jdbc:postgresql://${AZURE_PG_HOST}:5432/${DB_NAME} \
  ↪ ?ssl=true&sslmode=require&sslfactory=org.postgresql.ssl.NonValidatingFactory" \
  -user="${DB_USER}@${AZURE_SERVER_NAME}" \
  -password="${DB_PASSWORD}" \
  -locations="filesystem:./migrations" \
  migrate

# Same Flyway configuration for Cloud SQL PostgreSQL
# Using Cloud SQL Auth Proxy on localhost:5432
flyway \
  -url="jdbc:postgresql://localhost:5432/${DB_NAME}" \
  -user="${DB_USER}" \
  -password="${DB_PASSWORD}" \
  -locations="filesystem:./migrations" \
  migrate

# The migration files themselves are identical across all three
# V1_initial_schema.sql
# V2_add_customer_index.sql
# V3_partition_orders_table.sql

```

The migration files remain standard PostgreSQL DDL. The URL format changes; the SQL does not. This is the right level of abstraction.

Pattern 3: Monitoring with a Portable Metrics Layer

Rather than building dashboards directly in CloudWatch, Azure Monitor, and Cloud Monitoring in parallel, instrument your databases to emit metrics in a consistent format that a single observability platform (Grafana, Datadog, or a self-hosted Victoria Metrics stack) can consume:

```

# Export CloudWatch RDS metrics to a common format
# using the CloudWatch Metric Streams → Kinesis Firehose → S3 pipeline
# or more directly via the aws cli for spot checks

aws cloudwatch get-metric-statistics \
  --namespace "AWS/RDS" \
  --metric-name "DatabaseConnections" \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-01 \
  --start-time "$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ)" \
  --end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \

```



```

--period 60 \
--statistics Average Maximum \
--output table

# Equivalent for Azure SQL
az monitor metrics list \
--resource "/subscriptions/${SUBSCRIPTION_ID}/resourceGroups/${RG}/providers/Microsoft.Sql/servers/
↳ ${SERVER}/databases/${DB}" \
--metric "connection_successful" "connection_failed" "deadlock" \
--interval PT1M \
--start-time "$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ)" \
--end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
--output table

# Equivalent for Cloud SQL
gcloud monitoring metrics list \
--filter="metric.type=starts_with(\"cloudsql.googleapis.com\")" \
--format="table(metric.type)"

# For actual metric data from Cloud SQL:
gcloud logging read \
'resource.type="cloudsql_database" AND
logName="projects/${PROJECT_ID}/logs/cloudsql.googleapis.com%2Fpostgres.log"' \
--limit=100 \
--format=json \
--project="${PROJECT_ID}"

```

The principle: normalize the *output* of your monitoring into a common format rather than trying to normalize the *input* by pretending all three monitoring systems work the same way.

28.5 Realistic Migration Paths — Planning the Exit You May Never Take

The paradox of vendor lock-in planning is that the teams who plan their exit most carefully are usually the ones who never need to use that plan, because the planning discipline forces them to make architectural decisions that are inherently more maintainable.

A realistic cloud database migration involves three phases: data movement, application cutover, and operational transition. Understanding the constraints of each phase tells you where to invest in portability.

Data movement constraints by platform:

Migrating from Aurora PostgreSQL to any other PostgreSQL-compatible system is, from a data perspective, a `pg_dump` and restore problem. Aurora's storage format is standard PostgreSQL on-disk format. The complication is size and downtime:

```

# Phase 1: Create a logical backup from Aurora
# For databases under 100GB: direct pg_dump is fine
pg_dump \
--host="${AURORA_ENDPOINT}" \
--port=5432 \
--username="${DB_USER}" \
--dbname="${DB_NAME}" \
--format=custom \
--compress=9 \
--verbose \
--file="/backup/${DB_NAME}_$(date +%Y%m%d_%H%M%S).dump"

# For larger databases, use parallel dump with multiple jobs

```

```

pg_dump \
--host="${AURORA_ENDPOINT}" \
--port=5432 \
--username="${DB_USER}" \
--dbname="${DB_NAME}" \
--format=directory \
--jobs=8 \
--verbose \
--file="/backup/${DB_NAME}_parallel/"

# Phase 2: Establish logical replication to maintain sync
# during cutover window
# On the Aurora source, verify wal_level and replication slots
psql --host="${AURORA_ENDPOINT}" --username="${DB_USER}" --dbname="${DB_NAME}" <<'EOF'
-- Check current WAL configuration
SHOW wal_level;           -- should be logical
SHOW max_replication_slots;
SHOW max_wal_senders;

-- Create a replication slot for the migration
SELECT pg_create_logical_replication_slot(
    'migration_slot',
    'pgoutput'
);

-- Create a publication covering all tables
CREATE PUBLICATION migration_pub FOR ALL TABLES;

-- Check publication
SELECT pubname, puballtables, pubinsert, pubupdate, pubdelete
FROM pg_publication;
EOF

# Phase 3: On the target (Cloud SQL, Azure Database for PostgreSQL,
# or self-managed), set up subscription after restoring the base dump
psql --host="${TARGET_ENDPOINT}" --username="${DB_USER}" --dbname="${DB_NAME}" <<'EOF'
-- After restoring the pg_dump, create subscription
-- to catch changes that occurred during restore
CREATE SUBSCRIPTION migration_sub
    CONNECTION 'host=AURORA_ENDPOINT port=5432 dbname=DB_NAME user=replication_user password=SECRET
    ↪ sslmode=require'
    PUBLICATION migration_pub
    WITH (
        copy_data = false,           -- data already restored from dump
        create_slot = false,         -- use existing slot
        slot_name = 'migration_slot'
    );

-- Monitor replication lag
SELECT
    subname,
    received_lsn,
    latest_end_lsn,
    pg_size_pretty(pg_wal_lsn_diff(latest_end_lsn, received_lsn)) AS replication_lag_bytes,
    latest_end_time
FROM pg_stat_subscription;
EOF

```

This pattern — dump to establish a baseline, logical replication to close the gap, then a short cutover window to flip the connection string — minimizes downtime to the seconds it takes to flush replication lag and update DNS or your load balancer target.

Migrating from Azure SQL to PostgreSQL is meaningfully harder because you are crossing engine boundaries. The SQL Server to PostgreSQL translation surface is the most significant source of migration work:

```

-- Azure SQL: Inventory objects that require translation
-- Run this before estimating migration effort

```

```

WITH object_inventory AS (
    SELECT
        o.type_desc
        AS object_type,
        COUNT(*)
        AS object_count,
        COUNT(CASE WHEN m.definition LIKE '%EXEC%'
            OR m.definition LIKE '%EXECUTE%'
            THEN 1 END)
        AS uses_dynamic_exec,
        COUNT(CASE WHEN m.definition LIKE '%CURSOR%'
            THEN 1 END)
        AS uses_cursors,
        COUNT(CASE WHEN m.definition LIKE '###'
            THEN 1 END)
        AS uses_global_temp,
        COUNT(CASE WHEN m.definition LIKE '%RAISERROR%'
            OR m.definition LIKE '%THROW%'
            THEN 1 END)
        AS uses_error_handling,
        COUNT(CASE WHEN m.definition LIKE '%NOLOCK%'
            THEN 1 END)
        AS uses_nolock_hints
    FROM sys.objects o
    LEFT JOIN sys.sql_modules m ON o.object_id = m.object_id
    WHERE o.type IN ('P', 'FN', 'IF', 'TF', 'V', 'TR')
    GROUP BY o.type_desc
),
table_features AS (
    SELECT
        COUNT(*) AS total_tables,
        COUNT(CASE WHEN t.temporal_type = 2 THEN 1 END) AS temporal_tables,
        COUNT(CASE WHEN i.type = 5 THEN 1 END) AS clustered_columnstore,
        COUNT(CASE WHEN i.type = 6 THEN 1 END) AS nonclustered_columnstore,
        COUNT(CASE WHEN c.is_computed = 1 THEN 1 END) AS computed_columns,
        COUNT(CASE WHEN c.is_identity = 1 THEN 1 END) AS identity_columns
    FROM sys.tables t
    LEFT JOIN sys.indexes i ON t.object_id = i.object_id
    LEFT JOIN sys.columns c ON t.object_id = c.object_id
)
SELECT 'Object inventory' AS section, * FROM object_inventory
UNION ALL
SELECT 'Table features' AS section,
    total_tables, temporal_tables, clustered_columnstore,
    nonclustered_columnstore, computed_columns, identity_columns,
    0, 0
FROM table_features;

```

The objects that consistently require the most effort in a SQL Server to PostgreSQL migration are:

- **Temporal tables** (SQL Server’s `SYSTEM_TIME` versioning): PostgreSQL has no built-in equivalent; you implement history tracking with triggers or the `temporal_tables` extension.
- **Clustered columnstore indexes**: PostgreSQL’s approach to analytical queries differs fundamentally — you use table partitioning, BRIN indexes, and optionally AlloyDB’s columnar engine, but there is no direct drop-in replacement.
- **Stored procedures that use EXEC with dynamic SQL**: The translation from `EXEC sp_executesql` to PostgreSQL’s `EXECUTE` in PL/pgSQL is mechanical but requires touching every procedure.
- **NOLOCK hints**: SQL Server’s `WITH (NOLOCK)` is used pervasively to avoid blocking reads. PostgreSQL’s MVCC model means reads never block writes and writes never block reads, so `NOLOCK` has no equivalent — but code that relied on the dirty-read semantics of `NOLOCK` may behave differently under PostgreSQL’s snapshot isolation.

Application cutover constraints:

The application cutover is where the abstraction decisions from earlier in this chapter pay off. If your application retrieves its connection string from a secrets manager using a consistent naming convention, you change the secret value and restart the application pool. If it is hardcoded in a

configuration file deployed with the application, you are coordinating a deployment with a database cutover simultaneously.

A clean cutover procedure looks like this:

```
#!/bin/bash
# cutover.sh - Database migration cutover script
# Run after logical replication has caught up to < 1 second lag

set -euo pipefail

DB_NAME="${1:?DB_NAME required}"
SECRET_NAME="db/${DB_NAME}/primary"
OLD_ENDPOINT="${2:?OLD_ENDPOINT required}"
NEW_ENDPOINT="${3:?NEW_ENDPOINT required}"

echo "=== Cutover starting: $(date) ==="

# Step 1: Verify replication lag on target is acceptable
LAG_BYTES=$(psql --host="${NEW_ENDPOINT}" \
  --username="${DB_USER}" \
  --dbname="${DB_NAME}" \
  --tuples-only \
  --command="SELECT COALESCE(pg_wal_lsn_diff(latest_end_lsn, received_lsn), 0)
    FROM pg_stat_subscription WHERE subname = 'migration_sub';")

if [ "${LAG_BYTES}" -gt 1048576 ]; then # 1MB threshold
  echo "ERROR: Replication lag is ${LAG_BYTES} bytes. Aborting cutover."
  exit 1
fi

echo "Replication lag: ${LAG_BYTES} bytes. Proceeding."

# Step 2: Set source database to read-only to freeze writes
psql --host="${OLD_ENDPOINT}" \
  --username="${DB_USER}" \
  --dbname="${DB_NAME}" \
  --command="ALTER DATABASE ${DB_NAME} SET default_transaction_read_only = on;"

echo "Source database set to read-only: $(date)"

# Step 3: Wait for replication to fully catch up
for i in {1..30}; do
  LAG=$(psql --host="${NEW_ENDPOINT}" \
    --username="${DB_USER}" \
    --dbname="${DB_NAME}" \
    --tuples-only \
    --command="SELECT COALESCE(pg_wal_lsn_diff(latest_end_lsn, received_lsn), 0)
      FROM pg_stat_subscription WHERE subname = 'migration_sub';")

  echo "Lag check ${i}/30: ${LAG} bytes remaining"

  if [ "${LAG}" -eq 0 ]; then
    echo "Replication fully caught up: $(date)"
    break
  fi
  sleep 2
done

# Step 4: Update the secret in secrets manager
aws secretsmanager update-secret \
  --secret-id "${SECRET_NAME}" \
  --secret-string "{
    \"host\": \"${NEW_ENDPOINT}\",
    \"port\": \"5432\",
    \"dbname\": \"${DB_NAME}\",
    \"user\": \"${DB_USER}\",
    \"password\": \"${DB_PASSWORD}\"
  }"
```

```
echo "Secret updated to point to new endpoint: $(date)"

# Step 5: Signal application tier to reload connections
# This depends on your deployment - could be a rolling restart,
# a Lambda invocation, or a cache invalidation signal
echo "Signal your application pool to reload connections now."
echo "If using RDS Proxy or PgBouncer, issue: RELOAD on the proxy."

# Step 6: Remove subscription on target and slot on source
# Only after verifying the application is healthy on the new endpoint
echo ""
echo "After verifying application health, run:"
echo "  psql target: DROP SUBSCRIPTION migration_sub;"
echo "  psql source: SELECT pg_drop_replication_slot('migration_slot');"
echo "  psql source: ALTER DATABASE ${DB_NAME} SET default_transaction_read_only = off;"
echo ""
echo "=== Cutover script complete: $(date) ==="
```

The total downtime window in this procedure is the time between setting the source database read-only and the application pool successfully reconnecting to the new endpoint — typically under two minutes for well-prepared migrations with pre-warmed connection pools.

Operational transition:

The third phase — transitioning your runbooks, alerting, and operational muscle memory — is the part that rarely gets budgeted. If your on-call engineers respond to `HIGH_DB_CONNECTIONS` alerts by navigating CloudWatch and running specific RDS API calls, those runbooks need to be rewritten for any new environment. Plan for this explicitly. Write runbooks that are testable in the new environment before the migration, not after.

28.6 When Multi-Cloud Database Architectures Are Actually Justified

Most multi-cloud database arguments reduce to one of three legitimate cases and one illegitimate case.

Legitimate case 1: Regulatory data residency requirements. Some regulated industries require that customer data for specific geographies remain within those geographies, and a single cloud provider may not have coverage in all required regions. A European financial institution may run Azure SQL in EU-West for European customer data and AWS RDS in us-east-1 for US customer data not because of cloud strategy but because of compliance requirements. In this case, the multi-cloud architecture is not a choice; it is a constraint.

Legitimate case 2: Acquired applications with existing cloud commitments. Post-acquisition integration frequently involves databases on different cloud providers, and the cost of immediate migration often exceeds the cost of operating a transitional multi-cloud state for 12-24 months while the integration work proceeds. This is a pragmatic acceptance of temporary complexity, not a design philosophy.

Legitimate case 3: Specific workload characteristics that favor different providers. AlloyDB's columnar engine for mixed OLTP/analytical workloads, Azure SQL Hyperscale for databases that grow unpredictably beyond the size limits of standard tiers, and Aurora's fast failover for latency-sensitive applications are genuine capability differences. If different workloads

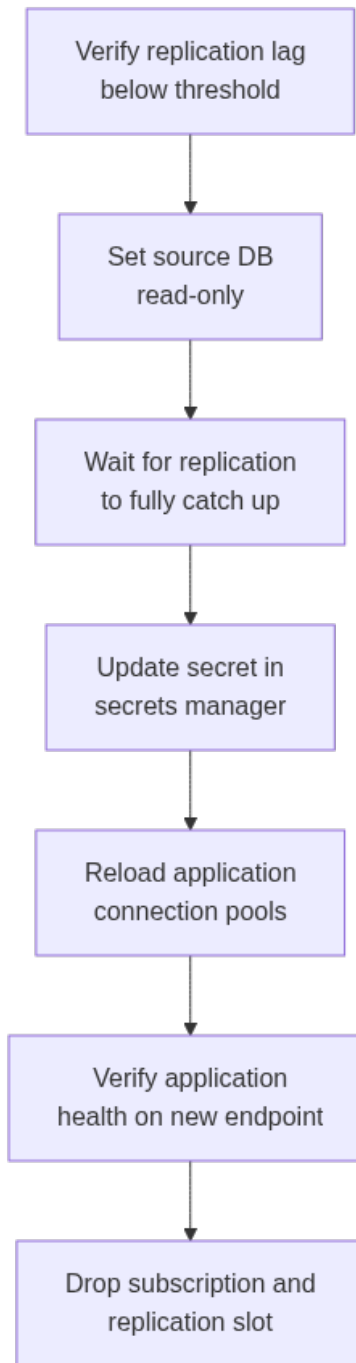


Figure 19: Cutover sequence: each step gates the next, minimizing the read-only window to the time replication needs to drain.

in your portfolio genuinely benefit from different platforms, running both is defensible — provided the operational overhead is understood and budgeted.

The illegitimate case: Theoretical redundancy. “We’ll spread our databases across AWS and Azure so that if one cloud goes down, we keep running.” This argument ignores several realities. First, your application tier, networking, and identity systems also live on one of those clouds. Second, replicating a write workload synchronously across two distinct cloud providers introduces latency that will degrade write performance significantly. Third, the operational complexity of maintaining equivalent databases across two providers with different APIs, different monitoring systems, different access control models, and different network primitives is substantial — and that complexity itself becomes a reliability risk. Genuine cloud failure events are rare enough that the added complexity of active-active multi-cloud typically creates more incidents than it prevents.

If you are building multi-cloud databases for redundancy purposes, you almost always get better risk reduction by investing in a more resilient single-cloud architecture: Aurora global database across multiple AWS regions, Azure SQL Hyperscale with geo-replication, or Cloud SQL cross-region replicas. The failure domain you are protecting against (an entire cloud provider region or service becoming unavailable) is different from the failure domain you are usually experiencing (misconfigurations, application bugs, single-region outages).

28.7 Practical Portability Audit: What to Document Right Now

The most actionable thing this chapter can leave you with is a portability audit you can run against your current database deployments. This is not about planning to migrate — it is about understanding your current state so that migration decisions, if they ever arise, are based on facts rather than anxiety or overconfidence.

Step 1: Extension and feature inventory

Run the extension inventory query from Section 28.3 against all PostgreSQL-compatible instances. For Azure SQL and SQL Server, run the T-SQL object inventory query. Record the results in your infrastructure documentation alongside each database.

Step 2: Data volume and growth rate

```
-- PostgreSQL: Database size and growth trend
-- Run on each database

SELECT
    datname                                AS database_name,
    pg_size_pretty(pg_database_size(datname)) AS current_size,
    pg_database_size(datname)              AS size_bytes
FROM pg_database
WHERE datistemplate = false
ORDER BY pg_database_size(datname) DESC;

-- Per-table breakdown for the largest contributors
SELECT
    schemaname,
    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname || '.' || tablename)) AS total_size,
    pg_size_pretty(pg_relation_size(schemaname || '.' || tablename))        AS table_size,
    pg_size_pretty(pg_indexes_size(schemaname || '.' || tablename))         AS indexes_size,
    pg_total_relation_size(schemaname || '.' || tablename)                  AS total_bytes
FROM pg_tables
```

```

WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname || '.' || tablename) DESC
LIMIT 20;

-- SQL Server / Azure SQL: Database size and per-table breakdown

SELECT
    DB_NAME()                                AS database_name,
    SUM(size * 8.0 / 1024)                   AS total_size_mb,
    SUM(CASE WHEN type = 0
        THEN size * 8.0 / 1024 END)          AS data_size_mb,
    SUM(CASE WHEN type = 1
        THEN size * 8.0 / 1024 END)          AS log_size_mb
FROM sys.database_files;

-- Per-table size breakdown
SELECT TOP 20
    SCHEMA_NAME(t.schema_id)                AS schema_name,
    t.name                                  AS table_name,
    p.rows                                  AS row_count,
    SUM(a.total_pages) * 8 / 1024            AS total_size_mb,
    SUM(a.used_pages) * 8 / 1024            AS used_size_mb,
    SUM(a.data_pages) * 8 / 1024            AS data_size_mb
FROM sys.tables t
JOIN sys.indexes i    ON t.object_id = i.object_id
JOIN sys.partitions p  ON i.object_id = p.object_id
                     AND i.index_id = p.index_id
JOIN sys.allocation_units a ON p.partition_id = a.container_id
WHERE t.is_ms_shipped = 0
GROUP BY t.schema_id, t.name, p.rows
ORDER BY SUM(a.total_pages) DESC;

```

Step 3: Dependency map for AWS-specific integrations

Document which applications use `aws_s3` for direct S3 imports, which Lambda functions connect directly to RDS without a proxy, which IAM roles are used for RDS IAM authentication, and which Parameter Store paths hold connection strings. This map is the operational topology you would need to recreate in any migration.

Step 4: Estimate migration effort by category

Use a simple three-column table in your documentation:

Component	Estimated Migration Effort	Blocking?
Data (pg_dump/restore)	Hours to days depending on size	No — logical replication closes gap
Application connection strings	Minutes if using secrets manager; days if hardcoded	Potentially
AWS-specific SQL extensions (<code>aws_s3</code> , <code>aws_ml</code>)	Days to rewrite affected procedures	Yes for those features
CloudWatch alerting and dashboards	Days to weeks to rebuild in new monitoring	No — parallel operation possible
IAM-based authentication	Hours — replace with password auth or new auth model	No
Runbooks and operational docs	Weeks — but can be done in parallel	No

This table, kept up to date, is more valuable than any elaborate abstraction layer. It converts the

abstract fear of lock-in into a concrete understanding of what migration would actually cost, which lets you make rational adoption decisions about proprietary features.

Key Takeaways

- **Lock-in has four distinct layers** — data, feature, operational, and pricing — and they require different responses. Data portability is a non-negotiable baseline (standard PostgreSQL format means `pg_dump` portability); feature lock-in should be accepted deliberately when the operational value justifies the migration cost.
 - **Application-level SQL abstractions almost always cost more than they save.** The right level for portability investment is infrastructure provisioning (Terraform modules with consistent interfaces), secrets management (a consistent naming convention across providers), and schema migration tooling (Flyway/Liquibase with standard DDL). These abstractions do not compromise your ability to use native database features.
 - **Proprietary features worth adopting despite lock-in** are those where the migration cost is a mechanical data movement exercise: Aurora storage architecture, RDS Proxy, AlloyDB's columnar engine. Features that require application rewrites to migrate away from (Spanner's batch mutations, Azure SQL Always Encrypted, SQL Server temporal tables) deserve explicit adoption decisions with documented rationale.
 - **Realistic migration planning focuses on three phases** — data movement, application cutover, and operational transition — and the largest effort is almost always the third. Logical replication can reduce the data movement cutover window to under two minutes; the operational transition (runbooks, alerting, team familiarity) takes weeks. Build operational portability by standardizing on provider-agnostic observability platforms and consistent secret naming conventions.
 - **Multi-cloud database architectures are justified** by regulatory data residency requirements, acquired-application integration timelines, and genuine workload-capability differences between providers. They are not justified by theoretical redundancy against full cloud provider failures — a more resilient single-cloud architecture with global replication almost always delivers better reliability at lower operational complexity than an active-active multi-cloud database topology.
-

Chapter 29: Backup and Restore in Cloud: Automated Snapshots vs Manual

Cloud-managed databases don't eliminate the need for backup strategy — they redistribute the responsibility. The platforms handle the mechanics of snapshot creation and storage, but the decisions about retention windows, cross-region durability, point-in-time recovery granularity, and restore validation remain squarely in your hands. This chapter walks through how AWS RDS, Aurora, Azure SQL, and Google Cloud SQL implement backup systems, where automated snapshots fall short of what on-premises DBAs are accustomed to controlling, and how to build a production-grade backup posture that accounts for the gaps. You will also see the restore side in detail, because backup systems that have never been tested are guesses, not strategies.

29.1 How Automated Snapshots Work Under the Hood

When you ran backups on-premises, you controlled everything: backup agent, target storage, compression, parallelism, verification jobs. In cloud-managed RDS, the platform takes daily automated snapshots during a configurable maintenance window and retains transaction logs for point-in-time recovery (PITR). Understanding what actually happens during these operations — not just what the console tells you — changes how you design around them.

RDS Automated Snapshots

RDS snapshots are storage-volume snapshots backed by EBS. For single-AZ instances, the snapshot process briefly suspends I/O — typically for a few seconds on modern instances, but measurable. For Multi-AZ deployments, the snapshot is taken from the standby replica, which eliminates the I/O pause on the primary. This is the same reason you should always run production workloads as Multi-AZ, not just for failover, but for backup behavior.

The retention window accepts values from 0 to 35 days. Setting it to 0 disables automated snapshots entirely — and also disables PITR, because transaction log uploads to S3 depend on automated snapshots being enabled. A common misconfiguration is setting retention to 1 day to save costs on a “non-critical” system, then discovering that PITR only reaches back 24 hours when a logic error surfaces three days later.

Transaction logs are uploaded to S3 every five minutes. This gives you a recovery point objective

(RPO) of approximately five minutes for PITR restores. You cannot restore to an arbitrary second — you restore to a specific UTC timestamp and RDS selects the nearest log position.

Aurora Automated Backups

Aurora’s backup model differs architecturally. Because Aurora stores data in a distributed storage layer (six copies across three AZs), snapshots are not EBS volume captures — they are metadata pointers into the shared storage. This means Aurora snapshots are nearly instantaneous and impose no I/O impact regardless of whether you’re using a Multi-AZ writer or not. Snapshot creation time is measured in seconds even for multi-terabyte clusters.

Aurora’s PITR window covers the same 1–35 day range, and because Aurora streams changes continuously into its storage layer rather than uploading transaction logs in five-minute batches, its restorable window trails the current time more tightly than standard RDS in practice.

Azure SQL Automated Backups

Azure SQL Database takes full backups weekly, differential backups every 12 hours, and transaction log backups every 5–12 minutes. These are stored in geo-redundant storage (GRS) by default, giving you cross-region durability without additional configuration. The retention window for Point-in-Time Restore (PITR) ranges from 1 to 35 days on General Purpose and Business Critical tiers; the Hyperscale tier handles backup differently and warrants its own treatment below.

One difference from RDS that catches on-premises DBAs: in Azure SQL, you cannot directly access backup files. You restore to a new database — you cannot restore over an existing database in-place. This forces a rename-and-swap procedure that has implications for connection strings and dependent jobs.

Cloud SQL Automated Backups

Cloud SQL stores automated backups in the same region as the instance by default. Cross-region backup storage requires explicit configuration via backup export or cross-region replicas. Cloud SQL’s PITR uses binary log retention for MySQL-compatible instances and WAL archiving for PostgreSQL-compatible instances, with a configurable retention window from 1 to 7 days — shorter than AWS or Azure. If you need more than 7 days of PITR coverage on Cloud SQL PostgreSQL, you are responsible for shipping WAL files to Cloud Storage manually using `pg_basebackup` exports or the `pg_receivewal` utility.

29.2 Configuring Automated Backup Windows and Retention

The defaults are rarely appropriate for production. Every platform gives you the ability to adjust backup windows, retention periods, and storage locations, but the configuration paths differ.

RDS Backup Configuration via AWS CLI

```
# Create an RDS PostgreSQL instance with backup configured for production
aws rds create-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --db-instance-class db.r6g.2xlarge \
  --engine postgres \
  --engine-version 15.4 \
  --master-username dbadmin \
  --master-user-password "${DB_MASTER_PASS}" \
  --allocated-storage 500 \
```

```

--storage-type gp3 \
--multi-az \
--backup-retention-period 14 \
--preferred-backup-window "02:00-03:00" \
--preferred-maintenance-window "sun:04:00-sun:05:00" \
--deletion-protection \
--no-publicly-accessible \
--db-subnet-group-name prod-db-subnet-group \
--vpc-security-group-ids sg-0abc123def456789 \
--tags Key=Environment,Value=Production Key=Owner,Value=DBA-Team

# Modify an existing instance's backup retention (takes effect immediately)
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --backup-retention-period 21 \
  --apply-immediately

```

The `--preferred-backup-window` uses UTC. Set it outside your peak transaction period. If you don't specify it, AWS assigns a random window — and randomly assigned maintenance windows have a habit of landing during business hours in some time zones.

RDS Cross-Region Backup Replication

Automated snapshots do not replicate cross-region by default. You must enable cross-region automated backup replication separately:

```

# Enable cross-region automated backup replication
# Source: us-east-1, Destination: us-west-2
aws rds start-export-task \
  --export-task-identifier backup-export-$(date +%Y%m%d) \
  --source-arn arn:aws:rds:us-east-1:123456789012:snapshot:rds:prod-postgres-01-2024-01-15 \
  --s3-bucket-name prod-db-backups-dr \
  --s3-prefix postgres-exports/ \
  --iam-role-arn arn:aws:iam::123456789012:role/RDSEXPORTRole \
  --kms-key-id arn:aws:kms:us-west-2:123456789012:key/mrk-abcdef1234567890

# Enable cross-region backup copy for all future automated snapshots
aws rds enable-cross-region-automated-backups \
  --source-db-instance-identifier prod-postgres-01 \
  --backup-retention-period 7 \
  --s3-export-path s3://prod-db-backups-dr/cross-region/ \
  --destination-region us-west-2

```

Note: cross-region backup replication for RDS is a separate feature from cross-region read replicas. The replicated backups are usable independently of the source instance — if your primary region fails, you can restore from the replicated backups in the DR region without access to the source.

Terraform: Aurora Cluster with Backup Configuration

```

resource "aws_rds_cluster" "aurora_prod" {
  cluster_identifier      = "aurora-prod-cluster"
  engine                 = "aurora-postgresql"
  engine_version         = "15.4"
  database_name          = "appdb"
  master_username        = "dbadmin"
  master_password        = var.db_master_password

  # Backup configuration
  backup_retention_period = 14
  preferred_backup_window = "03:00-04:00"
  preferred_maintenance_window = "sun:05:00-sun:06:00"

  # Cross-region backup replication
  enable_global_write_forwarding = false

  # Storage and encryption
  storage_encrypted = true

```

```

kms_key_id          = aws_kms_key.rds_key.arn

# Network
db_subnet_group_name = aws_db_subnet_group.prod.name
vpc_security_group_ids = [aws_security_group.rds.id]

# Protection
deletion_protection = true
skip_final_snapshot = false
final_snapshot_identifier = "aurora-prod-final-snapshot-${formatdate("YYYYMMDD", timestamp())}"

tags = {
    Environment = "Production"
    BackupPolicy = "14-day-retention"
}
}

resource "aws_rds_cluster_instance" "aurora_instances" {
    count                = 2
    identifier           = "aurora-prod-instance-${count.index}"
    cluster_identifier   = aws_rds_cluster.aurora_prod.id
    instance_class       = "db.r6g.2xlarge"
    engine               = aws_rds_cluster.aurora_prod.engine
    engine_version       = aws_rds_cluster.aurora_prod.engine_version

    performance_insights_enabled = true
    performance_insights_retention_period = 731 # 2 years
    monitoring_interval          = 60
    monitoring_role_arn          = aws_iam_role.rds_enhanced_monitoring.arn
}

```

Azure SQL Backup Configuration

```

# Check current backup policy on an Azure SQL Database
az sql db show \
    --resource-group prod-rg \
    --server prod-sql-server \
    --name proddb \
    --query "{name:name, shortTermRetention:requestedServiceObjectiveName}" \
    --output table

# Configure short-term backup retention (PITR window)
az sql db str-policy set \
    --resource-group prod-rg \
    --server prod-sql-server \
    --name proddb \
    --retention-days 21

# Configure long-term retention (LTR) policy
# Weekly: keep 12 weeks, Monthly: keep 12 months, Yearly: keep 5 years
az sql db ltr-policy set \
    --resource-group prod-rg \
    --server prod-sql-server \
    --name proddb \
    --weekly-retention P12W \
    --monthly-retention P12M \
    --yearly-retention P5Y \
    --week-of-year 1

# List available LTR backups
az sql db ltr-backup list \
    --location eastus \
    --server prod-sql-server \
    --database proddb \
    --output table

```

The LTR (Long-Term Retention) policy in Azure SQL maps to what on-premises DBAs implemented with backup-to-tape or offsite archive policies. Azure stores LTR backups in geo-redundant storage and they survive even if you delete the database or server — a meaningful difference from

RDS where manual snapshots are deleted if you delete the instance unless you explicitly retain them.

Cloud SQL Backup Configuration

```
# Create a Cloud SQL PostgreSQL instance with backup configured
gcloud sql instances create prod-postgres-01 \
  --database-version=POSTGRES_15 \
  --tier=db-custom-8-32768 \
  --region=us-central1 \
  --availability-type=REGIONAL \
  --backup-start-time=03:00 \
  --enable-bin-log \
  --retained-backups-count=14 \
  --retained-transaction-log-days=7 \
  --storage-type=SSD \
  --storage-size=500GB \
  --storage-auto-increase \
  --deletion-protection \
  --no-assign-ip \
  --network=projects/myproject/global/networks/prod-vpc

# Verify backup configuration
gcloud sql instances describe prod-postgres-01 \
  --format="json(settings.backupConfiguration)"
```

The `--retained-transaction-log-days` cap of 7 on Cloud SQL is the most significant constraint compared to RDS or Azure SQL. If your organization requires 14 or 30 days of PITR, Cloud SQL PostgreSQL cannot satisfy it natively, and you must supplement with exported logical backups.

29.3 Manual Snapshots and On-Demand Backups

Automated snapshots cover the daily cadence. Manual snapshots address the scenarios automated backups were not designed for: pre-migration checkpoints, pre-schema-change captures, compliance archiving, and cross-account copies for DR testing.

When Manual Snapshots Matter

On-premises DBAs took pre-change backups before every major schema migration. In cloud managed databases, that discipline translates directly to manual snapshots. Unlike automated snapshots — which are deleted when their retention window expires or when the instance is deleted (unless explicitly protected) — manual snapshots persist until you explicitly delete them. This makes them appropriate for compliance retention.

```
# Create a manual RDS snapshot before a schema migration
aws rds create-db-snapshot \
  --db-instance-identifier prod-postgres-01 \
  --db-snapshot-identifier prod-postgres-01-pre-migration-v2-$(date +%Y%m%d%H%M) \
  --tags Key=Purpose,Value=pre-migration Key=Schema,Value=v2 Key=Retention,Value=90days

# Check snapshot status
aws rds describe-db-snapshots \
  --db-snapshot-identifier prod-postgres-01-pre-migration-v2-20240115-1430 \
  --query "DBSnapshots[0].{Status:Status,Progress:PercentProgress,Size:AllocatedStorage}" \
  --output table

# Copy a manual snapshot to another region for DR validation
aws rds copy-db-snapshot \
  --source-db-snapshot-identifier \
    arn:aws:rds:us-east-1:123456789012:snapshot:prod-postgres-01-pre-migration-v2-20240115-1430 \
  --target-db-snapshot-identifier prod-postgres-01-pre-migration-v2-dr-copy \
```

```
--kms-key-id arn:aws:kms:us-west-2:123456789012:key/mrk-abcdef1234567890 \
--source-region us-east-1 \
--region us-west-2

# Share a snapshot with another AWS account (for DR account access)
aws rds modify-db-snapshot-attribute \
--db-snapshot-identifier prod-postgres-01-pre-migration-v2-20240115-1430 \
--attribute-name restore \
--values-to-add 987654321098 # DR account ID
```

Cloud SQL On-Demand Backups

```
# Create an on-demand backup before a maintenance operation
gcloud sql backups create \
--instance=prod-postgres-01 \
--description="Pre-major-version-upgrade backup $(date +%Y%m%d)"

# List all backups (automated and on-demand)
gcloud sql backups list \
--instance=prod-postgres-01 \
--filter="type=ON_DEMAND" \
--format="table(id, windowStartTime, status, diskEncryptionConfiguration.kmsKeyName)"

# Export a logical backup to Cloud Storage (supplements snapshot-based backup)
gcloud sql export sql prod-postgres-01 \
gs://prod-db-exports/postgres/prod-postgres-01-$(date +%Y%m%d).sql.gz \
--database=appdb \
--offload
```

The `--offload` flag on Cloud SQL exports runs the export from a serverless export process rather than the instance itself, reducing impact on production. Without it, the export operation uses instance CPU and creates transient I/O load.

Aurora Manual Snapshots and Export to S3

Aurora adds a capability that standard RDS does not have in the same form: you can export a snapshot to S3 as Parquet files organized by table, making the data queryable by Athena without restoring the full database. This is useful for compliance audits and forensic queries on point-in-time data.

```
# Export Aurora snapshot to S3 as Parquet (for Athena querying)
aws rds start-export-task \
--export-task-identifier audit-export-20240115 \
--source-arn arn:aws:rds:us-east-1:123456789012:cluster-snapshot:aurora-prod-cluster-2024-01-15 \
--s3-bucket-name prod-audit-exports \
--s3-prefix aurora-exports/2024-01-15/ \
--iam-role-arn arn:aws:iam::123456789012:role/RDSEExportRole \
--kms-key-id arn:aws:kms:us-east-1:123456789012:key/mrk-export-key \
--export-only "appdb.orders" "appdb.customers" # Export only specific tables

# Monitor export progress
aws rds describe-export-tasks \
--export-task-identifier audit-export-20240115 \
--query
  ↪ "ExportTasks[0].{Status:Status,PercentProgress:PercentProgress,TotalExtractedDataInGB:TotalExtractedDataInGB}"
```

29.4 Point-in-Time Recovery: Configuration and Execution

PITR is where backup strategy meets recovery strategy. The mechanics differ substantially across platforms, and the gotchas matter more here than anywhere else.

RDS PITR Restore

PITR on RDS always creates a new DB instance — it does not restore in place. This is a fundamental difference from on-premises PostgreSQL where you recover into the existing data directory. The new instance inherits the parameter group, security group, and subnet group from the original, but you must manually update application connection strings or your DNS CNAME to point to the new instance.

```
# Restore RDS to a specific point in time
# Note: this creates a NEW instance, not an in-place restore
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier prod-postgres-01 \
  --target-db-instance-identifier prod-postgres-01-restored-20240115T0230 \
  --restore-time 2024-01-15T02:30:00Z \
  --db-instance-class db.r6g.2xlarge \
  --multi-az \
  --no-publicly-accessible \
  --db-subnet-group-name prod-db-subnet-group \
  --vpc-security-group-ids sg-0abc123def456789 \
  --tags Key=Purpose,Value=PITR-restore Key=RestoreTime,Value=2024-01-15T02:30:00Z

# Check restore status
aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-01-restored-20240115T0230 \
  --query "DBInstances[0].{Status:DBInstanceStatus,Endpoint:Endpoint.Address,Port:Endpoint.Port}"

# After validation, modify DNS to point to restored instance
# (or promote to replace the original - must be done outside AWS)
```

Determining the Correct Restore Timestamp

This is where the DBA skill matters. Identifying the exact timestamp before a problematic event requires querying application logs, CloudWatch Logs Insights, or PostgreSQL log files. A script to correlate an error event with the appropriate restore target:

```
import boto3
from datetime import datetime, timedelta, timezone

def find_pitr_restore_window(instance_id: str, event_timestamp: datetime) -> dict:
    """
    Given an incident timestamp, identify the earliest and latest
    usable PITR restore points around that event.
    """
    rds = boto3.client('rds', region_name='us-east-1')

    # Get instance details to check backup retention and earliest restore time
    response = rds.describe_db_instances(DBInstanceIdentifier=instance_id)
    instance = response['DBInstances'][0]

    earliest_restore = instance.get('LatestRestorableTime')
    # Note: AWS SDK returns LatestRestorableTime but you need both ends

    # Check automated snapshots to find earliest available point
    snapshots = rds.describe_db_snapshots(
        DBInstanceIdentifier=instance_id,
        SnapshotType='automated',
        Filters=[
            {'Name': 'snapshot-create-time',
             'Values': ['2024-01-01T00:00:00Z']} # Adjust range
        ]
    )

    snapshot_times = sorted([
        s['SnapshotCreateTime']
        for s in snapshots['DBSnapshots']
        if s['Status'] == 'available'
    ])

    # Recommended restore point: 5 minutes before the event
```

```

recommended_restore = event_timestamp - timedelta(minutes=5)

result = {
    'incident_time': event_timestamp.isoformat(),
    'recommended_restore_target': recommended_restore.isoformat(),
    'earliest_available_snapshot': snapshot_times[0].isoformat() if snapshot_times else None,
    'latest_restorable_time': earliest_restore.isoformat() if earliest_restore else None,
    'pitr_feasible': (
        snapshot_times[0] <= recommended_restore <= earliest_restore
        if snapshot_times and earliest_restore else False
    )
}

return result

# Example usage
incident_time = datetime(2024, 1, 15, 2, 47, 0, tzinfo=timezone.utc)
window = find_pitr_restore_window('prod-postgres-01', incident_time)

print(f"Incident occurred at: {window['incident_time']}")
print(f"Recommended restore to: {window['recommended_restore_target']}")
print(f"PITR feasible: {window['pitr_feasible']}")

```

Azure SQL PITR

```

# Restore Azure SQL Database to a point in time
# Creates a new database - cannot restore in place
az sql db restore \
    --resource-group prod-rg \
    --server prod-sql-server \
    --name proddb \
    --dest-name proddb-restored-20240115 \
    --time "2024-01-15T02:30:00" \
    --edition GeneralPurpose \
    --service-objective GP_Gen5_8 \
    --backup-storage-redundancy Geo

# Monitor restore status
az sql db show \
    --resource-group prod-rg \
    --server prod-sql-server \
    --name proddb-restored-20240115 \
    --query "{status:status, createTime:createTime}" \
    --output table

# After validation: rename original (if replacing), then rename restored
# Azure SQL supports database rename via ALTER DATABASE
# Connect to master and run:
# ALTER DATABASE [proddb] MODIFY NAME = [proddb-old-20240115];
# ALTER DATABASE [proddb-restored-20240115] MODIFY NAME = [proddb];

```

The rename approach in Azure SQL is your equivalent of the on-premises “restore over the original” workflow. It requires brief downtime during the rename operation, and active connections must be terminated. Azure SQL does not allow renaming a database with active connections — you must set it to single-user mode first via the portal or SSMS, or terminate connections via `sys.dm_exec_sessions`.

Cloud SQL PITR

```

# Restore Cloud SQL instance to a point in time
# PITR on Cloud SQL creates a clone of the instance
gcloud sql instances clone prod-postgres-01 prod-postgres-01-pitr-restore \
    --point-in-time="2024-01-15T02:30:00.000Z"

# Check clone status
gcloud sql instances describe prod-postgres-01-pitr-restore \
    --format="value(state)"

```

```
# After validation, if replacing the original:
# 1. Rename original (Cloud SQL doesn't support rename directly)
# 2. Update application connection strings or Cloud SQL proxy config
# 3. Delete original once validated

# List available transaction logs to verify PITR window
gcloud sql instances describe prod-postgres-01 \
  --format=json(settings.backupConfiguration.transactionLogRetentionDays,\
settings.backupConfiguration.backupRetentionSettings)"
```

One detail that trips up Cloud SQL users: PITR clone operations require that binary logging (for MySQL) or WAL archiving (for PostgreSQL) was enabled **before** the target timestamp. If you enable PITR on an existing instance today, you cannot restore to a timestamp from yesterday. The continuous log chain only begins from when the feature was activated.

29.5 Cross-Region and Cross-Account Backup Strategies

The backup sitting in the same region as your database is not a disaster recovery backup — it is a convenience restore mechanism. A region-level outage takes down both your database and your same-region backups simultaneously. Production posture requires cross-region backup copies, and for organizations with strict data segregation requirements, cross-account copies.

RDS Cross-Region Snapshot Automation

Automating cross-region snapshot copies requires either AWS Backup, a Lambda function triggered by snapshot events, or an EventBridge rule. The native `enable-cross-region-automated-backups` CLI command shown earlier is the simplest path, but AWS Backup gives you a unified policy view across services.

```
# Terraform: AWS Backup plan with cross-region copy for RDS
resource "aws_backup_plan" "rds_backup_plan" {
  name = "prod-rds-backup-plan"

  rule {
    rule_name      = "daily-backup-14day-retention"
    target_vault_name = aws_backup_vault.primary.name
    schedule       = "cron(0 3 * * ? *)" # Daily at 3 AM UTC

    start_window      = 60 # minutes
    completion_window = 180 # minutes

    lifecycle {
      delete_after = 14
    }

    copy_action {
      destination_vault_arn = aws_backup_vault.dr_region.arn

      lifecycle {
        delete_after = 7 # Keep 7 days in DR region
      }
    }
  }

  rule {
    rule_name      = "weekly-backup-90day-retention"
    target_vault_name = aws_backup_vault.primary.name
    schedule       = "cron(0 4 ? * SUN *)" # Weekly on Sunday at 4 AM UTC

    lifecycle {
```

```
        delete_after = 90
    }

    copy_action {
        destination_vault_arn = aws_backup_vault.dr_region.arn

        lifecycle {
            delete_after = 90
        }
    }
}

advanced_backup_setting {
    backup_options = {
        WindowsVSS = "disabled"
    }
    resource_type = "RDS"
}

resource "aws_backup_vault" "primary" {
    name           = "prod-rds-backup-vault"
    kms_key_arn    = aws_kms_key.backup_key.arn

    tags = {
        Environment = "Production"
    }
}

resource "aws_backup_vault" "dr_region" {
    provider      = aws.us_west_2
    name          = "prod-rds-backup-vault-dr"
    kms_key_arn   = aws_kms_key.backup_key_dr.arn
}

resource "aws_backup_selection" "rds_selection" {
    name           = "prod-rds-instances"
    plan_id        = aws_backup_plan.rds_backup_plan.id
    iam_role_arn   = aws_iam_role.backup_role.arn

    resources = [
        "arn:aws:rds:*:db:prod-*",
        "arn:aws:rds:*:cluster:aurora-prod-*"
    ]

    # Tag-based selection as alternative to ARN list
    selection_tag {
        type = "STRINGEQUALS"
        key  = "BackupPolicy"
        value = "14-day-retention"
    }
}

resource "aws_backup_vault_policy" "primary_vault_policy" {
    backup_vault_name = aws_backup_vault.primary.name

    policy = jsonencode({
        Version = "2012-10-17"
        Statement = [
            {
                Effect = "Deny"
                Principal = { AWS = "*" }
                Action = [
                    "backup:DeleteBackupVault",
                    "backup:DeleteRecoveryPoint",
                    "backup:UpdateRecoveryPointLifecycle"
                ]
                Resource = "*"
                Condition = {
                    StringNotLike = {
```

The vault policy above implements a critical control: it prevents deletion of recovery points by anyone except the designated BackupAdminRole. This protects against ransomware scenarios where an attacker gains broad IAM access and attempts to destroy backups before encrypting data.

For organizations that run separate AWS accounts for production and DR, copying snapshots across account boundaries requires both accounts to cooperate:

A note on KMS keys and cross-account copies: you cannot copy a snapshot encrypted with a source-account KMS key into a destination account without re-encrypting it with a key that the destination account controls. The `--kms-key-id` in the destination-account copy command must reference a key in the destination account. This is a deliberate design — it prevents a compromised source account from planting inaccessible encrypted snapshots in the DR account.

Azure SQL's default GRS backup storage gives you geo-restore capability without explicit cross-region configuration. However, the restore target must be in a paired region:

521

```
# If you need PITR in the DR region, you need active geo-replication,  
# not just geo-restore from backup.
```

This distinction between geo-restore and geo-replication is important for RPO calculations. Geo-restore gives you the most recent backup, which may be hours old. Active geo-replication gives you near-zero RPO with the ability to failover with seconds of lag. For workloads requiring low RPO in a DR scenario, backup alone is insufficient.

29.6 Restore Testing and Validation Frameworks

A backup system that has never been tested is an assumption. The volume of data people have lost because they assumed backups were working — and discovered otherwise during an actual incident — makes restore testing one of the most important recurring operational practices a DBA team can establish.

What to Test, Not Just Whether It Restores

Many teams verify that a restore completes without error and stop there. That confirms the backup file is intact. It does not confirm:

- Schema consistency (all objects present, all constraints valid)
- Data completeness (row counts match expected ranges)
- Application connectivity (connection pools, service accounts, certificate trust)
- Performance characteristics (indexes are present, statistics are current)
- Dependent service configuration (secrets rotation, application config pointing to restored instance)

A restore validation script needs to cover all of these:

```
import psycpg2
import boto3
import json
import time
from datetime import datetime, timezone
from typing import Optional

class RestoreValidator:
    """
    Validates a restored RDS PostgreSQL instance against a production
    baseline. Run this after every scheduled restore test.
    """

    def __init__(
        self,
        restored_endpoint: str,
        prod_endpoint: str,
        port: int,
        database: str,
        username: str,
        password: str
    ):
        self.restored_endpoint = restored_endpoint
        self.prod_endpoint = prod_endpoint
        self.port = port
        self.database = database
        self.credentials = {
            'user': username,
            'password': password,
            'database': database,
```

```

        'port': port,
        'connect_timeout': 10
    }
    self.results = []

def _connect(self, host: str):
    return psycopg2.connect(host=host, **self.credentials)

def _record(self, check: str, passed: bool, detail: str):
    self.results.append({
        'check': check,
        'passed': passed,
        'detail': detail,
        'timestamp': datetime.now(timezone.utc).isoformat()
    })
    status = "PASS" if passed else "FAIL"
    print(f"[{status}] {check}: {detail}")

def check_connectivity(self) -> bool:
    """Verify the restored instance accepts connections."""
    try:
        conn = self._connect(self.restored_endpoint)
        conn.close()
        self._record("connectivity", True, f"Connected to {self.restored_endpoint}")
        return True
    except Exception as e:
        self._record("connectivity", False, str(e))
        return False

def check_schema_completeness(self) -> bool:
    """
    Compare table count and object counts between production
    and restored instance.
    """
    schema_query = """
    SELECT
        schemaname,
        COUNT(*) FILTER (WHERE relkind = 'r') AS tables,
        COUNT(*) FILTER (WHERE relkind = 'i') AS indexes,
        COUNT(*) FILTER (WHERE relkind = 'S') AS sequences,
        COUNT(*) FILTER (WHERE relkind = 'v') AS views
    FROM pg_catalog.pg_class c
    JOIN pg_catalog.pg_namespace n ON n.oid = c.relnamespace
    WHERE schemaname NOT IN ('pg_catalog', 'information_schema', 'pg_toast')
    GROUP BY schemaname
    ORDER BY schemaname;
    """

    try:
        prod_conn = self._connect(self.prod_endpoint)
        rest_conn = self._connect(self.restored_endpoint)

        with prod_conn.cursor() as cur:
            cur.execute(schema_query)
            prod_schema = {row[0]: row[1:] for row in cur.fetchall()}

        with rest_conn.cursor() as cur:
            cur.execute(schema_query)
            rest_schema = {row[0]: row[1:] for row in cur.fetchall()}

        prod_conn.close()
        rest_conn.close()

        mismatches = []
        for schema, prod_counts in prod_schema.items():
            rest_counts = rest_schema.get(schema)
            if rest_counts is None:
                mismatches.append(f"Schema '{schema}' missing from restore")
            elif prod_counts != rest_counts:
                mismatches.append(

```

```

        f"Schema '{schema}': prod={prod_counts}, "
        f"restored={rest_counts}"
    )

    if mismatches:
        self._record("schema_completeness", False, "; ".join(mismatches))
        return False
    else:
        self._record(
            "schema_completeness", True,
            f"Schema matches across {len(prod_schema)} schemas"
        )
        return True

except Exception as e:
    self._record("schema_completeness", False, str(e))
    return False

def check_row_count_variance(
    self,
    tables: list[tuple[str, str]],
    max_variance_pct: float = 5.0
) -> bool:
    """
    Check that row counts on key tables in the restore are within
    an acceptable variance of production counts.

    tables: list of (schema, table) tuples to check
    max_variance_pct: maximum acceptable difference in row counts
    """
    all_passed = True

    try:
        prod_conn = self._connect(self.prod_endpoint)
        rest_conn = self._connect(self.restored_endpoint)

        for schema, table in tables:
            count_query = f"SELECT COUNT(*) FROM {schema}.{table}"

            with prod_conn.cursor() as cur:
                cur.execute(count_query)
                prod_count = cur.fetchone()[0]

            with rest_conn.cursor() as cur:
                cur.execute(count_query)
                rest_count = cur.fetchone()[0]

            if prod_count == 0:
                variance_pct = 0.0 if rest_count == 0 else 100.0
            else:
                variance_pct = abs(prod_count - rest_count) / prod_count * 100

            passed = variance_pct <= max_variance_pct
            all_passed = all_passed and passed
            self._record(
                f"row_count_{schema}_{table}",
                passed,
                f"prod={prod_count:},, restored={rest_count:},, "
                f"variance={variance_pct:.2f}%"
            )

        prod_conn.close()
        rest_conn.close()

    except Exception as e:
        self._record("row_count_variance", False, str(e))
        all_passed = False

    return all_passed

```



```

def check_constraint_validity(self) -> bool:
    """
    Verify no invalid constraints exist on the restored instance.
    Invalid constraints indicate partial restores or corruption.
    """
    constraint_query = """
        SELECT
            conrelid::regclass AS table_name,
            conname AS constraint_name,
            contype AS constraint_type,
            convalidated AS is_valid
        FROM pg_constraint
        WHERE NOT convalidated
            AND contype IN ('f', 'c')
        ORDER BY table_name, constraint_name;
    """

    try:
        conn = self._connect(self.restored_endpoint)
        with conn.cursor() as cur:
            cur.execute(constraint_query)
            invalid = cur.fetchall()
            conn.close()

        if invalid:
            detail = ", ".join(
                f"{row[0]}.{row[1]}({row[2]})" for row in invalid
            )
            self._record(
                "constraint_validity", False,
                f"{len(invalid)} invalid constraints: {detail}"
            )
            return False
        else:
            self._record(
                "constraint_validity", True,
                "All constraints valid"
            )
            return True

    except Exception as e:
        self._record("constraint_validity", False, str(e))
        return False

def check_statistics_freshness(self) -> bool:
    """
    Verify autovacuum/analyze has run on key tables, or explicitly
    run ANALYZE on the restored instance.
    """
    stats_query = """
        SELECT
            schemaname,
            tablename,
            last_analyze,
            last_autoanalyze,
            n_live_tup
        FROM pg_stat_user_tables
        WHERE n_live_tup > 10000
            AND last_analyze IS NULL
            AND last_autoanalyze IS NULL
        ORDER BY n_live_tup DESC
        LIMIT 10;
    """

    try:
        conn = self._connect(self.restored_endpoint)
        with conn.cursor() as cur:
            cur.execute(stats_query)
            stale_tables = cur.fetchall()

```

```

    if stale_tables:
        # Run ANALYZE on tables with stale stats
        for row in stale_tables:
            with conn.cursor() as cur:
                cur.execute(
                    f"ANALYZE {row[0]}.{row[1]}"
                )
            conn.commit()
        self._record(
            "statistics_freshness", True,
            f"Ran ANALYZE on {len(stale_tables)} tables with stale stats"
        )
    else:
        self._record(
            "statistics_freshness", True,
            "Table statistics are current"
        )

    conn.close()
    return True

except Exception as e:
    self._record("statistics_freshness", False, str(e))
    return False

def generate_report(self, output_path: Optional[str] = None) -> dict:
    """Generate a validation report for audit and alerting."""
    total = len(self.results)
    passed = sum(1 for r in self.results if r['passed'])
    failed = total - passed

    report = {
        'validation_run': datetime.now(timezone.utc).isoformat(),
        'restored_endpoint': self.restored_endpoint,
        'summary': {
            'total_checks': total,
            'passed': passed,
            'failed': failed,
            'overall_status': 'PASS' if failed == 0 else 'FAIL'
        },
        'checks': self.results
    }

    if output_path:
        with open(output_path, 'w') as f:
            json.dump(report, f, indent=2)

    return report

# Example usage in a scheduled restore test pipeline
def run_restore_validation(restored_endpoint: str):
    import os

    validator = RestoreValidator(
        restored_endpoint=restored_endpoint,
        prod_endpoint=os.environ['PROD_DB_ENDPOINT'],
        port=5432,
        database='appdb',
        username=os.environ['DB_VALIDATION_USER'],
        password=os.environ['DB_VALIDATION_PASS']
    )

    # Run all checks
    validator.check_connectivity()
    validator.check_schema_completeness()
    validator.check_row_count_variance([
        ('public', 'orders'),
        ('public', 'customers'),
        ('public', 'products'),
    ])

```

```

        ('audit', 'change_log')
    ])
    validator.check_constraint_validity()
    validator.check_statistics_freshness()

    report = validator.generate_report('/tmp/restore-validation-report.json')

    if report['summary']['overall_status'] == 'FAIL':
        # Alert via SNS or PagerDuty
        sns = boto3.client('sns')
        sns.publish(
            TopicArn=os.environ['OPS_ALERT_TOPIC'],
            Subject='RESTORE VALIDATION FAILED',
            Message=json.dumps(report['summary'], indent=2)
        )
        raise SystemExit(1)

    return report

```

Scheduling Regular Restore Tests

An automated restore test should run at least monthly for production databases — weekly for high-criticality systems. The test pipeline should:

1. Trigger a PITR or snapshot restore to an isolated VPC subnet
2. Run the validation framework above
3. Compare results against the previous test baseline
4. Terminate the restored instance to avoid ongoing compute cost
5. Log results to a compliance-accessible audit store

EventBridge-triggered Lambda or Step Function for monthly restore test
Sample shell pipeline for CloudWatch Events / Step Functions:

```

#!/bin/bash
set -euo pipefail

INSTANCE_ID="prod-postgres-01"
RESTORE_ID="restore-test-$(date +%Y%m%d%H%M)"
RESTORE_TIME=$(date -u -d '2 hours ago' +%Y-%m-%dT%H:%M:%SZ)
SUBNET_GROUP="restore-test-isolated-subnet"
SECURITY_GROUP="sg-restore-test-isolated"

echo "Starting restore test: ${RESTORE_ID}"

# Step 1: Trigger PITR restore
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier "${INSTANCE_ID}" \
  --target-db-instance-identifier "${RESTORE_ID}" \
  --restore-time "${RESTORE_TIME}" \
  --db-instance-class db.r6g.xlarge \
  --no-multi-az \
  --no-publicly-accessible \
  --db-subnet-group-name "${SUBNET_GROUP}" \
  --vpc-security-group-ids "${SECURITY_GROUP}"

# Step 2: Wait for available status (poll with backoff)
echo "Waiting for restored instance to become available..."
for i in $(seq 1 60); do
    STATUS=$(aws rds describe-db-instances \
      --db-instance-identifier "${RESTORE_ID}" \
      --query "DBInstances[0].DBInstanceStatus" \
      --output text)
    echo "  Status: ${STATUS} (attempt ${i}/60)"
    if [ "${STATUS}" = "available" ]; then
        break
    fi
    sleep 60
done

```

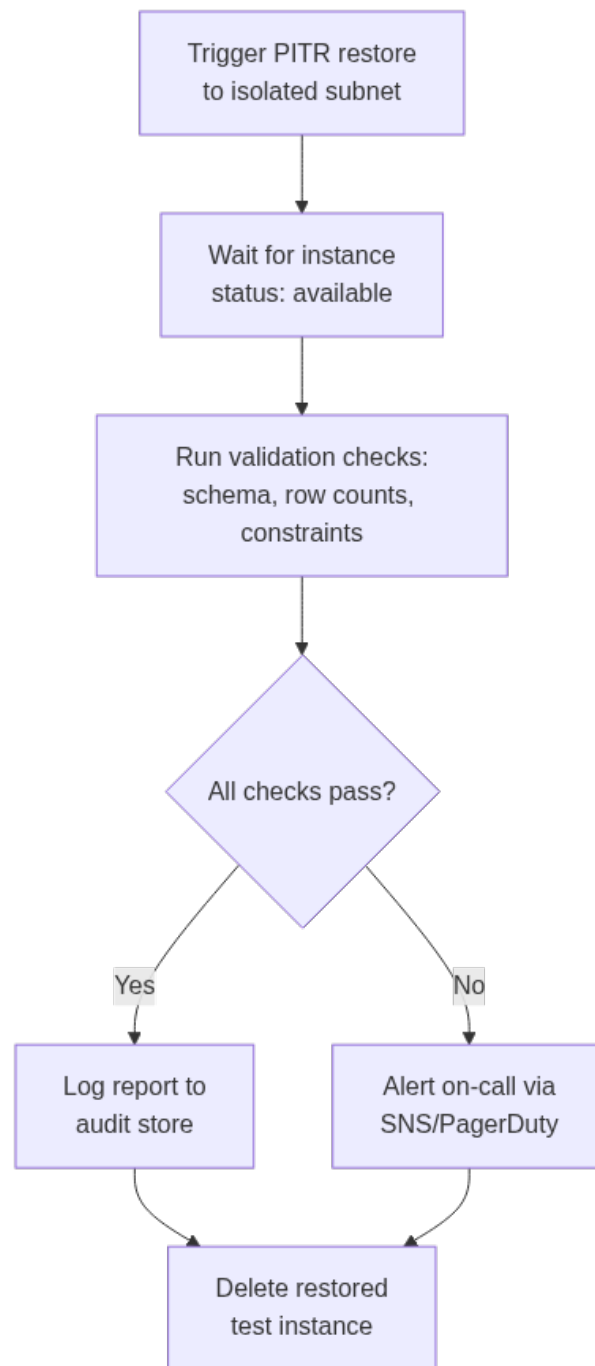


Figure 20: Scheduled restore test pipeline: validation runs before cleanup, and failures alert regardless of whether cleanup proceeds.

```
done

if [ "${STATUS}" != "available" ]; then
    echo "ERROR: Instance did not become available within 60 minutes"
    exit 1
fi

# Step 3: Get endpoint
ENDPOINT=$(aws rds describe-db-instances \
    --db-instance-identifier "${RESTORE_ID}" \
    --query "DBInstances[0].Endpoint.Address" \
    --output text)

# Step 4: Run validation
python3 /opt/restore-validator/validate.py \
    --endpoint "${ENDPOINT}" \
    --report-path "/tmp/restore-test-${RESTORE_ID}.json"

# Step 5: Tag result and upload report
aws s3 cp "/tmp/restore-test-${RESTORE_ID}.json" \
    "s3://prod-audit-logs/restore-tests/$(date +%Y/%m)/restore-test-${RESTORE_ID}.json"

# Step 6: Cleanup - delete the test instance
echo "Deleting restore test instance: ${RESTORE_ID}"
aws rds delete-db-instance \
    --db-instance-identifier "${RESTORE_ID}" \
    --skip-final-snapshot \
    --delete-automated-backups

echo "Restore test complete."
```

29.7 Logical Backups as a Supplement to Snapshots

Snapshot-based backups restore the entire instance. Sometimes you need to restore a single table, a single schema, or a specific subset of rows — and snapshot restores are not designed for that. Logical backups using `pg_dump`, `mysqldump`, or BACPAC files fill this gap.

PostgreSQL Logical Backup via `pg_dump` from RDS/Cloud SQL

```
# Logical backup of a specific schema from RDS PostgreSQL
# Use --no-password with .pgpass or PGPASSWORD environment variable
export PGPASSWORD="${DB_PASSWORD}"

pg_dump \
    --host=prod-postgres-01.cluster-xyz.us-east-1.rds.amazonaws.com \
    --port=5432 \
    --username=dbadmin \
    --dbname=appdb \
    --schema=public \
    --format=custom \
    --compress=9 \
    --file=/tmp/appdb-public-$(date +%Y%m%d).dump \
    --verbose \
    --no-password \
    --lock-wait-timeout=30s

# Upload to S3
aws s3 cp /tmp/appdb-public-$(date +%Y%m%d).dump \
    s3://prod-logical-backups/postgres/appdb/$(date +%Y/%m)/ \
    --sse aws:kms \
    --sse-kms-key-id alias/prod-backup-key

# For single-table restore from the dump:
pg_restore \
```

```
--host=restore-target.cluster-xyz.us-east-1.rds.amazonaws.com \
--port=5432 \
--username=dbadmin \
--dbname=appdb \
--table=orders \
--format=custom \
--no-owner \
--no-privileges \
/tmp/appdb-public-20240115.dump
```

SQL Server BACPAC Export from Azure SQL

The Azure SQL equivalent of a logical backup is a BACPAC export — a zip archive containing the schema and data:

```
# Export Azure SQL Database as BACPAC to Azure Blob Storage
az sql db export \
  --resource-group prod-rg \
  --server prod-sql-server \
  --name proddb \
  --admin-user dbadmin \
  --admin-password "${DB_PASSWORD}" \
  --storage-key "${STORAGE_ACCOUNT_KEY}" \
  --storage-key-type StorageAccessKey \
  --storage-uri "https://prodbackupstorage.blob.core.windows.net/bacpacs/proddb-$(date +%Y%m%d).bacpac"

# Import a BACPAC to restore a database (or specific data)
az sql db import \
  --resource-group prod-rg \
  --server restore-sql-server \
  --name proddb-logical-restore \
  --admin-user dbadmin \
  --admin-password "${DB_PASSWORD}" \
  --storage-key "${STORAGE_ACCOUNT_KEY}" \
  --storage-key-type StorageAccessKey \
  --storage-uri "https://prodbackupstorage.blob.core.windows.net/bacpacs/proddb-20240115.bacpac" \
  --edition GeneralPurpose \
  --service-objective GP_Gen5_4 \
  --max-size-bytes 10737418240
```

For SQL Server, the T-SQL equivalent when you need selective table restores is to import the BACPAC into a temporary database, then use `INSERT INTO ... SELECT` to move only the needed rows back into production:

```
-- SQL Server: selective row restoration from a BACPAC-restored database
-- Run against the temporary restored database to extract rows

-- Step 1: Identify the rows to recover
SELECT
  o.order_id,
  o.customer_id,
  o.order_date,
  o.order_total,
  o.status
FROM proddb_logical_restore.dbo.orders o
WHERE o.order_date >= '2024-01-15 02:00:00'
  AND o.order_date < '2024-01-15 03:00:00'
  AND o.status = 'deleted' -- rows incorrectly deleted
ORDER BY o.order_date;

-- Step 2: Insert recovered rows into production
-- (assumes production is on same server for this example)
INSERT INTO proddb.dbo.orders
  (order_id, customer_id, order_date, order_total, status, created_at)
SELECT
  order_id, customer_id, order_date, order_total,
  'recovered' AS status, -- flag as recovered for audit trail
  GETUTCDATE() AS created_at
```

```
FROM proddb_logical_restore.dbo.orders
WHERE order_date >= '2024-01-15 02:00:00'
      AND order_date < '2024-01-15 03:00:00'
      AND status = 'deleted'
      AND order_id NOT IN (SELECT order_id FROM proddb.dbo.orders);
-- ~~~ avoid duplicating rows that weren't deleted

-- PostgreSQL equivalent on a restored RDS instance:
/*
INSERT INTO appdb.public.orders
  (order_id, customer_id, order_date, order_total, status, created_at)
SELECT
  order_id, customer_id, order_date, order_total,
  'recovered'::text AS status,
  NOW() AS created_at
FROM restored_appdb.public.orders
WHERE order_date >= '2024-01-15 02:00:00'
      AND order_date < '2024-01-15 03:00:00'
      AND status = 'deleted'
      AND order_id NOT IN (SELECT order_id FROM appdb.public.orders);
*/
```

29.8 Backup Encryption, Access Control, and Compliance

Backups represent the same data as your production database, which means they require the same classification and access controls. A common gap in cloud database security reviews is discovering that backups are encrypted at rest but accessible to a broad set of IAM principals, or that the encryption key is not under BYOK (bring your own key) control.

RDS Backup Encryption

RDS snapshots inherit the encryption status of the source instance. An unencrypted RDS instance produces unencrypted snapshots. You cannot encrypt a snapshot in place — you must copy it and specify a KMS key during the copy. The production enforcement practice is to ensure all RDS instances are created with encryption enabled using a customer-managed KMS key (CMK), not the AWS-managed `aws/rds` key.

```
# Encrypt an unencrypted snapshot by copying with a CMK
# (migrate a legacy unencrypted instance's snapshot to encrypted)
aws rds copy-db-snapshot \
  --source-db-snapshot-identifier prod-postgres-legacy-snapshot-20240115 \
  --target-db-snapshot-identifier prod-postgres-legacy-encrypted-20240115 \
  --kms-key-id arn:aws:kms:us-east-1:123456789012:key/mrk-abcdef1234567890 \
  --copy-tags \
  --region us-east-1

# Verify encryption status
aws rds describe-db-snapshots \
  --db-snapshot-identifier prod-postgres-legacy-encrypted-20240115 \
  --query "DBSnapshots[0].{Encrypted:Encrypted,KmsKeyId:KmsKeyId}" \
  --output table
```

IAM Policy for Backup Access Control

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowSnapshotRestore",
      "Effect": "Allow",
      "Action": [
        "rds:RestoreDBInstanceFromDBSnapshot",
```

```

    "rds:RestoreDBInstanceToPointInTime",
    "rds:DescribeDBSnapshots",
    "rds:DescribeDBInstances"
  ],
  "Resource": "*",
  "Condition": {
    "StringEquals": {
      "aws:RequestedRegion": ["us-east-1", "us-west-2"]
    },
    "StringLike": {
      "rds:DatabaseName": "prod-*"
    }
  }
},
{
  "Sid": "DenySnapshotDeletion",
  "Effect": "Deny",
  "Action": [
    "rds:DeleteDBSnapshot",
    "rds:ModifyDBSnapshotAttribute"
  ],
  "Resource": "arn:aws:rds:*:*:snapshot:prod-*",
  "Condition": {
    "StringNotLike": {
      "aws:PrincipalArn": [
        "arn:aws:iam::123456789012:role/BackupAdminRole",
        "arn:aws:iam::123456789012:role/BreakGlassRole"
      ]
    }
  }
},
{
  "Sid": "AllowSnapshotCreate",
  "Effect": "Allow",
  "Action": [
    "rds:CreateDBSnapshot",
    "rds:CopyDBSnapshot",
    "rds:AddTagsToResource"
  ],
  "Resource": "*",
  "Condition": {
    "StringEquals": {
      "aws:RequestedRegion": ["us-east-1", "us-west-2"]
    }
  }
}
]
}

```

Azure SQL Backup Access Control

Azure SQL backup access is governed by RBAC roles at the resource group or server level. The minimum set of roles for a DBA who needs to perform restores without the ability to delete databases or modify server configuration:

```

# Assign SQL DB Contributor role for restore capability (not server admin)
az role assignment create \
  --assignee dba-restore-team@company.com \
  --role "SQL DB Contributor" \
  --scope "/subscriptions/sub-id/resourceGroups/prod-rg/providers/Microsoft.Sql/servers/prod-sql-server"

# Deny destructive actions via custom deny assignment (requires Azure Blueprint or Policy)
# Azure Policy to audit and deny deletion of SQL databases with Production tag:
cat <<'EOF' > /tmp/deny-sql-delete-policy.json
{
  "mode": "All",
  "policyRule": {
    "if": {
      "allOf": [

```



```

    {
      "field": "type",
      "equals": "Microsoft.Sql/servers/databases"
    },
    {
      "field": "tags.Environment",
      "equals": "Production"
    }
  ]
},
"then": {
  "effect": "Deny"
}
}
}
EOF

az policy definition create \
  --name "deny-prod-sql-db-deletion" \
  --display-name "Deny deletion of Production SQL Databases" \
  --description "Prevents deletion of Azure SQL databases tagged as Production" \
  --rules /tmp/deny-sql-delete-policy.json \
  --mode All

az policy assignment create \
  --name "deny-prod-sql-db-deletion-assignment" \
  --policy "deny-prod-sql-db-deletion" \
  --scope "/subscriptions/sub-id/resourceGroups/prod-rg"

```

Compliance Reporting: Verifying Backup Coverage

For audits and compliance reviews (SOC 2, PCI-DSS, HIPAA), you need to demonstrate that all production databases have backup retention meeting the policy requirement. A cross-platform compliance check:

```

import boto3
import json
from datetime import datetime, timezone

def audit_rds_backup_compliance(
    region: str,
    min_retention_days: int = 14
) -> list[dict]:
    """
    Returns a list of RDS and Aurora instances that do not meet
    minimum backup retention requirements.
    """
    rds = boto3.client('rds', region_name=region)
    non_compliant = []

    # Check RDS instances
    paginator = rds.get_paginator('describe_db_instances')
    for page in paginator.paginate():
        for instance in page['DBInstances']:
            retention = instance.get('BackupRetentionPeriod', 0)
            tags = {t['Key']: t['Value'] for t in instance.get('TagList', [])}
            env = tags.get('Environment', 'unknown')

            if env == 'Production' and retention < min_retention_days:
                non_compliant.append({
                    'resource_type': 'RDS Instance',
                    'identifier': instance['DBInstanceIdentifier'],
                    'environment': env,
                    'retention_days': retention,
                    'required_days': min_retention_days,
                    'encrypted': instance.get('StorageEncrypted', False),
                    'deletion_protection': instance.get('DeletionProtection', False),
                    'multi_az': instance.get('MultiAZ', False)
                })

```

```
# Check Aurora clusters
paginator = rds.get_paginator('describe_db_clusters')
for page in paginator.paginate():
    for cluster in page['DBClusters']:
        retention = cluster.get('BackupRetentionPeriod', 0)
        tags = {t['Key']: t['Value'] for t in cluster.get('TagList', [])}
        env = tags.get('Environment', 'unknown')

        if env == 'Production' and retention < min_retention_days:
            non_compliant.append({
                'resource_type': 'Aurora Cluster',
                'identifier': cluster['DBClusterIdentifier'],
                'environment': env,
                'retention_days': retention,
                'required_days': min_retention_days,
                'encrypted': cluster.get('StorageEncrypted', False),
                'deletion_protection': cluster.get('DeletionProtection', False),
                'multi_az': len(cluster.get('AvailabilityZones', [])) > 1
            })

    return non_compliant

def generate_compliance_report(regions: list[str], min_retention: int = 14) -> dict:
    report = {
        'generated_at': datetime.now(timezone.utc).isoformat(),
        'policy_min_retention_days': min_retention,
        'regions_audited': regions,
        'non_compliant_resources': [],
        'summary': {}
    }

    for region in regions:
        findings = audit_rds_backup_compliance(region, min_retention)
        report['non_compliant_resources'].extend(findings)

    total = len(report['non_compliant_resources'])
    report['summary'] = {
        'total_non_compliant': total,
        'compliance_status': 'COMPLIANT' if total == 0 else 'NON-COMPLIANT',
        'unencrypted_count': sum(
            1 for r in report['non_compliant_resources']
            if not r['encrypted']
        ),
        'no_deletion_protection_count': sum(
            1 for r in report['non_compliant_resources']
            if not r['deletion_protection']
        )
    }

    return report

# Run and emit report
report = generate_compliance_report(
    regions=['us-east-1', 'us-west-2', 'eu-west-1'],
    min_retention=14
)
print(json.dumps(report, indent=2))
```

29.9 Backup Cost Management

Backup storage costs accumulate quietly. Snapshots on a 10 TB database at 30-day retention across two regions with daily snapshots represent a storage footprint that warrants active management,

not passive acceptance.

Understanding Incremental Snapshot Storage

RDS and Aurora snapshots are incremental after the first full snapshot — only changed blocks are stored in subsequent snapshots. This means the storage cost of daily snapshots is proportional to your daily change rate, not your total database size. A mostly-read database with low write volume will have a dramatically smaller snapshot storage footprint than a write-heavy OLTP system of the same size.

Aurora's snapshot storage model is even more efficient: because the shared storage layer tracks changes at the block level across all snapshots, the total storage for 14 daily snapshots may be only marginally larger than the database itself if the change rate is low.

```
# Check snapshot storage consumption across all RDS snapshots
aws rds describe-db-snapshots \
  --snapshot-type manual \
  --query "DBSnapshots[?Status=='available'].{ID:DBSnapshotIdentifier,
    Date:SnapshotCreateTime,
    SizeGB:AllocatedStorage,
    Encrypted:Encrypted}" \
  --output table \
  --region us-east-1

# Get CloudWatch metric for automated snapshot storage billed
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name FreeStorageSpace \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-01 \
  --start-time $(date -u -d '30 days ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 86400 \
  --statistics Average \
  --output table

# List manual snapshots older than 90 days for cleanup review
aws rds describe-db-snapshots \
  --snapshot-type manual \
  --query "DBSnapshots[?SnapshotCreateTime<='2023-10-15' && Status=='available'].
    {ID:DBSnapshotIdentifier,Date:SnapshotCreateTime,Size:AllocatedStorage}" \
  --output table
```

Cost Optimization: Tiered Retention Strategy

A tiered approach balances recovery capability against storage cost:

Backup Type	Frequency	Retention	Storage Location	Use Case
Automated snapshot	Daily	14 days	Same region	Standard PITR and daily restore
Manual snapshot (weekly)	Weekly	90 days	Same region	Extended window restores
Cross-region copy (weekly)	Weekly	30 days	DR region	Regional disaster recovery
Logical export (monthly)	Monthly	1 year	S3 Glacier	Compliance archiving, table-level restores

Logical exports to S3 Glacier Instant Retrieval cost a fraction of RDS snapshot storage for long-term archiving. For compliance requirements mandating 7-year retention, Glacier Deep Archive is the appropriate tier — with the understanding that retrieval takes hours and is appropriate only for compliance audit scenarios, not operational recovery.

```
# Create a lifecycle rule on the S3 bucket holding logical exports
aws s3api put-bucket-lifecycle-configuration \
  --bucket prod-logical-backups \
  --lifecycle-configuration '{
    "Rules": [
      {
        "ID": "logical-backup-tiering",
        "Status": "Enabled",
        "Filter": {"Prefix": "postgres/"},
        "Transitions": [
          {
            "Days": 30,
            "StorageClass": "STANDARD_IA"
          },
          {
            "Days": 90,
            "StorageClass": "GLACIER_IR"
          },
          {
            "Days": 365,
            "StorageClass": "DEEP_ARCHIVE"
          }
        ],
        "Expiration": {
          "Days": 2555
        }
      }
    ]
  }'
```

Key Takeaways

- **Automated snapshots are a starting point, not a complete backup strategy.** Platform defaults — short retention windows, single-region storage, no cross-account copies — require deliberate reconfiguration before production use. The most common production backup failures involve systems where someone accepted the defaults and discovered the limitations during an actual incident.
- **PITR mechanics differ significantly across platforms.** RDS uploads transaction logs every five minutes (approx. five-minute RPO); Aurora's shared storage model provides near-second RPO; Cloud SQL PostgreSQL caps PITR window at seven days. Knowing these limits determines whether your platform choice is compatible with your recovery objectives before a problem occurs.
- **Restore always creates a new instance — plan your cutover procedure before you need it.** All three platforms (RDS, Azure SQL, Cloud SQL) restore to a new database endpoint rather than restoring in place. The rename-and-swap workflow, DNS update, or connection string change must be documented and rehearsed as part of your incident response runbooks, not improvised under pressure.
- **Backup validation is a recurring operational discipline, not a one-time setup task.** A restore test that checks connectivity and stops there tells you the backup file is readable,

not that the database is usable. Schema completeness, constraint validity, row count variance, and statistics freshness checks are the difference between a tested recovery capability and a false sense of preparedness.

- **Cross-region and cross-account backup copies require explicit configuration and have encryption key implications.** Snapshots in the source region are not automatically replicated; cross-account copies require re-encryption with a destination-account KMS key; and backup vault policies should explicitly deny deletion by all principals except a designated admin role to protect against both accidental and malicious removal of recovery points.
-

Chapter 30: Monitoring Cloud Databases: CloudWatch, Azure Monitor, Cloud Logging

Cloud-managed databases remove your access to the operating system, which means the monitoring stack you built around `top`, `iostat`, `pg_stat_activity`, and custom Nagios checks no longer has a foundation to stand on. The platform now owns the infrastructure layer, and you must rebuild your observability posture using the native tooling each cloud provider offers. This chapter walks through CloudWatch for AWS RDS and Aurora, Azure Monitor for Azure SQL and Elastic Pools, and Cloud Logging and Cloud Monitoring for Google Cloud SQL and AlloyDB. For each platform, you will learn how to configure metrics collection, set meaningful alert thresholds grounded in production experience, build dashboards that surface what matters, and avoid the most common instrumentation gaps that cause DBAs to miss incidents until users are already calling.

30.1 What You Lost and What You Gained: Monitoring Philosophy for Managed Databases

On-premises PostgreSQL and SQL Server gave you unrestricted access. You could run `sar -u 1 10` to sample CPU across ten seconds, attach `perf` to the Postgres process, read `/proc/meminfo` for exact memory pressure, and correlate all of it with `pg_stat_bgwriter` in the same terminal session. The trade-off for managed databases is real: you cannot SSH into an RDS instance, you cannot inspect `/var/log/postgresql/` directly, and you cannot run `top` on the underlying host.

What you gain in exchange is a pre-wired telemetry pipeline. CloudWatch, Azure Monitor, and Google Cloud Monitoring collect dozens of metrics automatically from the moment the instance starts, with no agent installation, no scrape configuration, and no storage management for the metric data itself. The platform also exposes metrics that would have been difficult or impossible to capture yourself — RDS Enhanced Monitoring samples OS-level statistics at one-second intervals using an agent that runs inside the managed environment, which is more granular than most self-managed deployments bother to configure.

The mental model shift is this: you are moving from **pull-based, CLI-driven observation** to **push-based, subscription-driven alerting**. Instead of checking the server, you define what matters and let the platform tell you when thresholds are crossed. This demands that you know your thresholds before an incident, not during one.

The four signal categories you must cover on any cloud database platform:

Signal	Managed equivalent	What it replaces
CPU and OS resource	CloudWatch, Azure Metrics, Cloud Monitoring	<code>top</code> , <code>vmstat</code> , <code>sar</code>
Storage and I/O	Platform metrics + Performance Insights	<code>iostat</code> , <code>df</code> , storage vendor tools
Query-level performance	Performance Insights, Query Store, Query Insights	<code>pg_stat_statements</code> , Profiler traces
Replication and HA state	Platform events, replica lag metrics	<code>pg_stat_replication</code> , AlwaysOn DMVs

A note on metric resolution and cost. All three platforms charge for high-resolution metrics and extended retention beyond a free tier — CloudWatch bills per metric beyond an initial free allotment, Azure Monitor bills on ingested log volume, and Cloud Monitoring bills per custom metric beyond a free byte threshold. Check current pricing for your account before enabling maximum-resolution monitoring across a large fleet; the bill scales directly with instance count and metric cardinality.

30.2 CloudWatch: Monitoring AWS RDS and Aurora in Production

CloudWatch is the primary observability surface for RDS and Aurora. It collects standard metrics at one-minute intervals automatically, and at ten-second intervals when Enhanced Monitoring is enabled on the instance. Performance Insights sits alongside CloudWatch and provides query-level attribution. Understanding which tool answers which question saves considerable time during incidents.

Standard RDS CloudWatch metrics and their production thresholds:

Metric	Namespace	Healthy range (general guidance)
<code>CPUUtilization</code>	<code>AWS/RDS</code>	Sustained < 75%; alert at 85% for 5 min
<code>FreeStorageSpace</code>	<code>AWS/RDS</code>	Alert when < 10% of allocated storage
<code>DatabaseConnections</code>	<code>AWS/RDS</code>	Alert at 80% of <code>max_connections</code>
<code>ReadLatency</code> / <code>WriteLatency</code>	<code>AWS/RDS</code>	Alert > 20 ms for OLTP workloads
<code>ReplicaLag</code>	<code>AWS/RDS</code>	Alert > 30 seconds for read replicas
<code>AuroraReplicaLag</code>	<code>AWS/RDS</code>	Alert > 100 ms for Aurora replicas
<code>Deadlocks</code>	<code>AWS/RDS</code>	Alert on any sustained increase
<code>FreeableMemory</code>	<code>AWS/RDS</code>	Alert when < 256 MB on <code>db.r6g.large</code>

`ReplicaLag` for standard RDS read replicas is a physical replication metric measured in seconds. Aurora's `AuroraReplicaLag` is fundamentally different — it measures how far behind an Aurora

replica is in applying redo log pages to its local buffer cache from the shared storage layer, and it is usually measured in single-digit milliseconds. Do not apply the same alert threshold to both.

Enabling Enhanced Monitoring and Performance Insights via AWS CLI:

```
# Enable Enhanced Monitoring (1-second granularity) and Performance Insights
# on an existing RDS instance. The monitoring role must exist first.

# Step 1: Create the Enhanced Monitoring IAM role if it doesn't exist
aws iam create-role \
  --role-name rds-monitoring-role \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Principal": { "Service": "monitoring.rds.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }]
  }'
```

```
aws iam attach-role-policy \
  --role-name rds-monitoring-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonRDSEnhancedMonitoringRole

# Step 2: Modify the instance to enable both features
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --monitoring-interval 1 \
  --monitoring-role-arn arn:aws:iam::123456789012:role/rds-monitoring-role \
  --enable-performance-insights \
  --performance-insights-retention-period 731 \
  --performance-insights-kms-key-id alias/aws/rds \
  --apply-immediately

# Step 3: Verify the configuration
aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-01 \
  --query 'DBInstances[0].{
    MonitoringInterval: MonitoringInterval,
    PIEnabled: PerformanceInsightsEnabled,
    PIRetention: PerformanceInsightsRetentionPeriod
  }'
```

Performance Insights retention of 731 days (two years) is the maximum. The default is 7 days free; anything beyond that is billed per vCPU-hour of retained history, and the cost scales with both instance size and how far back you extend the window. For a large multi-vCPU instance, two-year retention is a meaningful recurring line item — check current Performance Insights pricing and budget for it explicitly before enabling extended retention fleet-wide.

Creating a CloudWatch alarm for connection saturation:

```
# Alert when DatabaseConnections exceeds 80% of max_connections.
# The default max_connections value is derived from instance memory and
# varies by instance class - always verify the actual value from your
# Parameter Group before calculating this threshold; do not assume a
# stock number.

aws cloudwatch put-metric-alarm \
  --alarm-name "rds-prod-postgres-01-connection-saturation" \
  --alarm-description "RDS connections above 80% of max_connections" \
  --metric-name DatabaseConnections \
  --namespace AWS/RDS \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-01 \
  --statistic Average \
  --period 60 \
  --evaluation-periods 3 \
  --threshold "${CONNECTION_ALARM_THRESHOLD}" \
```

```

--comparison-operator GreaterThanOrEqualToThreshold \
--alarm-actions arn:aws:sns:us-east-1:123456789012:prod-db-alerts \
--ok-actions arn:aws:sns:us-east-1:123456789012:prod-db-alerts \
--treat-missing-data notBreaching

# Retrieve current max_connections from the parameter group
aws rds describe-db-parameters \
  --db-parameter-group-name prod-postgres15-params \
  --query 'Parameters[?ParameterName==`max_connections`].{Name:ParameterName,Value:ParameterValue}'

```

One gotcha with DatabaseConnections: RDS Parameter Groups compute `max_connections` dynamically based on instance memory by default (`LEAST({DBInstanceClassMemory/9531392}, 5000)`). When you resize an instance, `max_connections` changes silently. If you hardcode alarm thresholds, they become wrong after a resize. Build automation to recalculate thresholds whenever an instance class changes, or use a CloudWatch metric math expression that accounts for the dynamic value.

Querying Performance Insights for top wait events using Python:

```

import boto3
from datetime import datetime, timedelta, timezone

def get_top_waits(db_instance_id: str, region: str = "us-east-1", minutes: int = 60):
    """
    Retrieve top wait events from Performance Insights for the last N minutes.
    Equivalent to querying pg_stat_activity for wait_event_type in PostgreSQL.
    """
    pi = boto3.client("pi", region_name=region)
    rds = boto3.client("rds", region_name=region)

    # PI requires the DbiResourceId, not the instance identifier
    response = rds.describe_db_instances(DBInstanceIdentifier=db_instance_id)
    resource_id = response["DBInstances"][0]["DbiResourceId"]

    end_time = datetime.now(timezone.utc)
    start_time = end_time - timedelta(minutes=minutes)

    response = pi.get_resource_metrics(
        ServiceType="RDS",
        Identifier=resource_id,
        StartTime=start_time,
        EndTime=end_time,
        PeriodInSeconds=60,
        MetricQueries=[
            {
                "Metric": "db.load.avg",
                "GroupBy": {
                    "Group": "db.wait_event",
                    "Dimensions": ["db.wait_event.name", "db.wait_event.type"],
                    "Limit": 10,
                },
            },
        ],
    )

    print(f"\nTop wait events for {db_instance_id} (last {minutes} minutes):\n")
    print(f'{"Wait Event":<45} {"Type":<20} {"Avg DB Load":>12}')
    print("-" * 80)

    for result in response.get("MetricList", []):
        for key_dim in result.get("DataPoints", []):
            pass # We want the grouped keys

    # The grouped response structure differs - use Keys endpoint
    keys_response = pi.get_resource_metadata(
        ServiceType="RDS",
        Identifier=resource_id,

```

```
)

# Simpler approach: get dimension keys directly
dim_response = pi.describe_dimension_keys(
    ServiceType="RDS",
    Identifier=resource_id,
    StartTime=start_time,
    EndTime=end_time,
    PeriodInSeconds=3600,
    Metric="db.load.avg",
    GroupBy={
        "Group": "db.wait_event",
        "Dimensions": ["db.wait_event.name", "db.wait_event.type"],
        "Limit": 10,
    },
)

for key in dim_response.get("Keys", []):
    wait_name = key["Dimensions"].get("db.wait_event.name", "unknown")
    wait_type = key["Dimensions"].get("db.wait_event.type", "unknown")
    avg_load = key.get("Total", 0)
    print(f"{wait_name:<45} {wait_type:<20} {avg_load:>12.4f}")

return dim_response

if __name__ == "__main__":
    results = get_top_waits("prod-postgres-01", region="us-east-1", minutes=60)
```

The output maps directly to what you would see in `pg_stat_activity.wait_event` and `wait_event_type`. If `LWLock:buffer_mapping` appears at the top, your buffer pool is under pressure. If `IO:DataFileRead` dominates, your working set exceeds buffer pool capacity. The interpretation is identical to on-premises; only the collection mechanism differs.

Aurora-specific metrics worth explicit attention:

Aurora exposes metrics standard RDS does not. `AuroraVolumeBytesLeftTotal` tracks remaining capacity in the Aurora cluster volume. The cluster volume grows in 10 GB increments and supports up to 128 TB, but it does not shrink automatically. `BufferCacheHitRatio` for Aurora should stay above 99.9% for OLTP; if it drops below 99%, you are seeing genuine I/O latency against the distributed storage layer. `CommitLatency` and `DDLlatency` are Aurora-specific metrics that have no direct CloudWatch equivalent in standard RDS.

30.3 Azure Monitor: Metrics, Diagnostics, and Log Analytics for Azure SQL

Azure Monitor is a unified observability platform that covers Azure SQL Database, Managed Instances, and Elastic Pools through two distinct systems: **Metrics** (pre-aggregated, 93-day retention, low cost) and **Diagnostic Logs** (raw records streamed to a Log Analytics Workspace, Storage Account, or Event Hub, with cost proportional to data volume).

The equivalent of CloudWatch Enhanced Monitoring is the **Intelligent Performance** blade in the Azure portal, which surfaces Query Store data through the Query Performance Insight interface. The equivalent of Performance Insights is the `sys.dm_exec_query_stats` and `sys.query_store_*` views in Azure SQL, which remain accessible because Azure SQL gives you T-SQL access to DMVs — a significant advantage over RDS if you come from a SQL Server background.

Core Azure SQL metrics and alert thresholds:

Metric name	Alert threshold (OLTP)	Notes
cpu_percent	> 80% for 5 minutes	Based on DTU or vCore allocation
storage_percent	> 85%	Alert well before 100% — autogrow is not automatic
connection_failed	> 10 per minute	Indicates authentication or firewall issues
deadlock	Any increase > baseline	Query Store records the deadlock graph
dtu_consumption_percent	> 85%	DTU model only — consider vCore if consistently high
workers_percent	> 80%	Maximum worker threads; hitting 100% causes errors
log_write_percent	> 90%	Indicates log I/O becoming a bottleneck

Configuring Diagnostic Settings to stream metrics to Log Analytics:

```

# Prerequisites: Log Analytics Workspace already created
# Retrieve the workspace resource ID
WORKSPACE_ID=$(az monitor log-analytics workspace show \
  --resource-group prod-rg \
  --workspace-name prod-db-logs \
  --query id \
  --output tsv)

# Get the Azure SQL Database resource ID
DB_RESOURCE_ID=$(az sql db show \
  --resource-group prod-rg \
  --server prod-sql-server-01 \
  --name prod-appdb \
  --query id \
  --output tsv)

# Enable all diagnostic categories to Log Analytics
# SQLInsights includes query-level data from Query Store
# AutomaticTuning includes index recommendation events

```

```

# QueryStoreRuntimeStatistics is the most valuable for query troubleshooting
az monitor diagnostic-settings create \
  --name "prod-appdb-diagnostics" \
  --resource "$DB_RESOURCE_ID" \
  --workspace "$WORKSPACE_ID" \
  --logs '[
    {"category": "SQLInsights", "enabled": true, "retentionPolicy": {"enabled": false}},
    {"category": "AutomaticTuning", "enabled": true, "retentionPolicy": {"enabled": false}},
    {"category": "QueryStoreRuntimeStatistics", "enabled": true, "retentionPolicy": {"enabled": false}},
    {"category": "QueryStoreWaitStatistics", "enabled": true, "retentionPolicy": {"enabled": false}},
    {"category": "Errors", "enabled": true, "retentionPolicy": {"enabled": false}},
    {"category": "DatabaseWaitStatistics", "enabled": true, "retentionPolicy": {"enabled": false}},
    {"category": "Deadlocks", "enabled": true, "retentionPolicy": {"enabled": false}},
    {"category": "Blocks", "enabled": true, "retentionPolicy": {"enabled": false}}
  ]' \
  --metrics '[
    {"category": "Basic", "enabled": true},
    {"category": "InstanceAndAppAdvanced", "enabled": true}
  ]'

# Verify the diagnostic setting
az monitor diagnostic-settings list \
  --resource "$DB_RESOURCE_ID" \
  --query '[].{Name:name, Workspace:workspaceId}' \
  --output table

```

There is a meaningful delay between enabling Diagnostic Settings and data appearing in Log Analytics — typically 5 to 15 minutes. Do not assume the feature is broken if the workspace shows no data immediately after configuration. Also note that enabling diagnostic settings on an Elastic Pool is separate from enabling them on each database within the pool. The pool-level settings cover pool resource metrics (eDTUs, storage); per-database settings cover query-level telemetry.

Querying Log Analytics with KQL for wait statistics:

```

# Run a KQL query against the Log Analytics Workspace to find top wait types
# over the last 24 hours - equivalent to querying sys.dm_os_wait_stats
# on an on-premises SQL Server instance

az monitor log-analytics query \
  --workspace "$WORKSPACE_ID" \
  --analytics-query '
AzureDiagnostics
| where ResourceType == "DATABASES"
| where Category == "DatabaseWaitStatistics"
| where TimeGenerated > ago(24h)
| where Resource == "PROD-APPDB"
| summarize
  TotalWaitTime_ms = sum(delta_wait_time_ms_d),
  AvgWaitTime_ms = avg(delta_wait_time_ms_d),
  SampleCount = count()
  by wait_type_s
| sort by TotalWaitTime_ms desc
| take 15
| project
  WaitType = wait_type_s,
  TotalWait_Seconds = round(TotalWaitTime_ms / 1000.0, 1),
  AvgWait_ms = round(AvgWaitTime_ms, 2),
  Samples = SampleCount
' \
  --timespan P1D \
  --output table

# Find the top 10 CPU-consuming queries from Query Store via Log Analytics
az monitor log-analytics query \
  --workspace "$WORKSPACE_ID" \
  --analytics-query '
AzureDiagnostics
| where Category == "QueryStoreRuntimeStatistics"

```

```
| where TimeGenerated > ago(6h)
| where Resource == "PROD-APPDB"
| summarize
    TotalCPU_ms = sum(cpu_time_d),
    TotalDuration_ms = sum(duration_d),
    ExecutionCount = sum(count_executions_d),
    AvgLogicalReads = avg(avg_logical_io_reads_d)
    by query_hash_s, query_id_d
| sort by TotalCPU_ms desc
| take 10
| project
    QueryID = query_id_d,
    QueryHash = query_hash_s,
    TotalCPU_seconds = round(TotalCPU_ms / 1000000.0, 2),
    Executions = ExecutionCount,
    AvgLogicalReads = round(AvgLogicalReads, 0)
\
--output table
```

The KQL query against `QueryStoreRuntimeStatistics` maps directly to `sys.query_store_runtime_stats` joined with `sys.query_store_plan` and `sys.query_store_query` in on-premises SQL Server. If you have built Query Store dashboards on-premises, the transition to KQL is largely a syntax change rather than a conceptual one. The `query_hash_s` field lets you correlate across databases or time windows even when query IDs differ.

Creating a metric alert for Elastic Pool DTU consumption:

```
# Alert when Elastic Pool eDTU consumption exceeds 85% for 10 minutes
POOL_RESOURCE_ID=$(az sql elastic-pool show \
    --resource-group prod-rg \
    --server prod-sql-server-01 \
    --name prod-elastic-pool-01 \
    --query id \
    --output tsv)

ACTION_GROUP_ID=$(az monitor action-group show \
    --resource-group prod-rg \
    --name prod-db-oncall \
    --query id \
    --output tsv)

az monitor metrics alert create \
    --name "elastic-pool-dtu-saturation" \
    --resource-group prod-rg \
    --scopes "$POOL_RESOURCE_ID" \
    --condition "avg dtu_consumption_percent > 85" \
    --window-size 10m \
    --evaluation-frequency 5m \
    --severity 2 \
    --description "Elastic Pool eDTU consumption above 85% - consider scaling or workload redistribution"
↵ \
    --action "$ACTION_GROUP_ID"
```

One alert configuration mistake that appears frequently: setting the evaluation frequency equal to the window size. If your window is 10 minutes and you evaluate every 10 minutes, a spike that occurs in the last two minutes of a window will not fire an alert until the next evaluation cycle — potentially 18 minutes after the spike began. Use an evaluation frequency of 5 minutes or less when the window size is 10 minutes.

30.4 Google Cloud Monitoring and Cloud Logging for Cloud SQL and AlloyDB

Google Cloud Monitoring (formerly Stackdriver) collects Cloud SQL and AlloyDB metrics automatically with no configuration required. The metrics are available in the `cloudsql.googleapis.com` and `alloydb.googleapis.com` namespaces. Cloud Logging captures database flags, slow query logs, and general logs as structured log entries that can be queried with the Logs Explorer or exported to BigQuery for long-term analysis.

The model is similar to Azure Monitor: there is a metrics layer (low cost, pre-aggregated, 6-week default retention in Cloud Monitoring) and a logging layer (structured logs in Cloud Logging, charged per GB ingested beyond the free 50 GB per project per month, with 30-day default retention).

Critical Cloud SQL metrics:

Metric	Full path	Alert threshold
CPU utilization	<code>cloudsql.googleapis.com/database/cpu/utilization</code>	75%
Disk utilization	<code>cloudsql.googleapis.com/database/disk/utilization</code>	85%
Memory utilization	<code>cloudsql.googleapis.com/database/memory/utilization</code>	90%
Active connections	<code>cloudsql.googleapis.com/database/postgresql/active_connections</code>	80%
Replica lag (seconds)	<code>cloudsql.googleapis.com/database/replication/replica_lag</code>	30s
Deadlock count	<code>cloudsql.googleapis.com/database/postgresql/deadlock_count</code>	1
Transaction ID utilization	<code>cloudsql.googleapis.com/database/postgresql/transaction_id_utilization</code>	70%

`transaction_id_utilization` is specific to PostgreSQL and has no equivalent in SQL Server monitoring. It measures the fraction of the 2^{31} transaction ID space that has been consumed since the last `VACUUM FREEZE` cycle. On-premises DBAs who ran `autovacuum` aggressively often never encountered transaction ID wraparound; in Cloud SQL, `autovacuum` still runs, but understanding this metric is important because a Cloud SQL instance under heavy write load can approach the wraparound limit if `autovacuum` is tuned too conservatively. Alert at 70% utilization and treat 80% as an emergency requiring manual `VACUUM FREEZE` intervention.

Enabling the PostgreSQL slow query log and Cloud Logging export using gcloud:

```
# Enable the slow query log for a Cloud SQL PostgreSQL instance.
# log_min_duration_statement is set in milliseconds.
# Setting it to 1000 logs any query taking longer than 1 second.

gcloud sql instances patch prod-cloudsql-pg01 \
  --database-flags \
    log_min_duration_statement=1000,\
    log_checkpoints=on,\
    log_connections=off,\
    log_disconnections=off,\
    log_lock_waits=on,\
    log_temp_files=10240,\
    log_autovacuum_min_duration=500

# Verify the flags were applied
gcloud sql instances describe prod-cloudsql-pg01 \
  --format="value(settings.databaseFlags)"

# Cloud SQL logs flow automatically to Cloud Logging.
# Create a log sink to export slow query logs to BigQuery for long-term analysis.
```

```
# The filter captures log entries from the postgres database log.
gcloud logging sinks create prod-cloudsql-slowquery-bq \
  bigquery.googleapis.com/projects/prod-project/datasets/db_monitoring \
  --log-filter='
    resource.type="cloudsql_database"
    resource.labels.database_id="prod-project:prod-cloudsql-pg01"
    log_id("cloudsql.googleapis.com/postgres.log")
    textPayload=~"duration:"
  ' \
  --use-partitioned-tables

# Grant the sink service account write access to the BigQuery dataset
SINK_SA=$(gcloud logging sinks describe prod-cloudsql-slowquery-bq \
  --format="value(writerIdentity)")

bq add-iam-policy-binding \
  --member="$SINK_SA" \
  --role="roles/bigquery.dataEditor" \
  prod-project:db_monitoring
```

After the sink is in place, slow query logs accumulate in the `db_monitoring` dataset partitioned by date. This allows long-term trend analysis — comparing this week’s p99 query latency against the same period last month — that would be impractical inside Cloud Logging’s 30-day default retention window.

Querying Cloud SQL slow query logs in BigQuery:

```
-- BigQuery SQL: Find the top 20 slowest queries from the Cloud SQL PostgreSQL
-- slow query log over the last 7 days, grouped by normalized query pattern.
-- The textPayload field contains the raw log line from PostgreSQL.

SELECT
  REGEXP_EXTRACT(textPayload, r'duration: ([0-9.]+) ms') AS duration_ms_str,
  CAST(REGEXP_EXTRACT(textPayload, r'duration: ([0-9.]+) ms') AS FLOAT64) AS duration_ms,
  -- Normalize the statement by removing literal values (basic normalization)
  REGEXP_REPLACE(
    REGEXP_EXTRACT(textPayload, r'statement: (.+)\$'),
    r"'[^']*'|[0-9]+",
    '?'
  ) AS normalized_query,
  textPayload AS raw_log,
  timestamp
FROM
  `prod-project.db_monitoring.prod-project_prod-cloudsql-pg01-*`
WHERE
  _TABLE_SUFFIX >= FORMAT_DATE('%Y%m%d', DATE_SUB(CURRENT_DATE(), INTERVAL 7 DAY))
  AND textPayload LIKE '%duration:%'
ORDER BY
  duration_ms DESC
LIMIT 20;

-- Aggregate by normalized pattern to find the highest cumulative load
SELECT
  REGEXP_REPLACE(
    REGEXP_EXTRACT(textPayload, r'statement: (.+)\$'),
    r"'[^']*'|[0-9]+",
    '?'
  ) AS normalized_query,
  COUNT(*) AS execution_count,
  ROUND(AVG(CAST(REGEXP_EXTRACT(textPayload, r'duration: ([0-9.]+) ms') AS FLOAT64)), 2) AS
    ↪ avg_duration_ms,
  ROUND(MAX(CAST(REGEXP_EXTRACT(textPayload, r'duration: ([0-9.]+) ms') AS FLOAT64)), 2) AS
    ↪ max_duration_ms,
  ROUND(SUM(CAST(REGEXP_EXTRACT(textPayload, r'duration: ([0-9.]+) ms') AS FLOAT64)) / 1000.0, 1) AS
    ↪ total_cpu_seconds
FROM
  `prod-project.db_monitoring.prod-project_prod-cloudsql-pg01-*`
WHERE
```



```

_TABLE_SUFFIX >= FORMAT_DATE('%Y%m%d', DATE_SUB(CURRENT_DATE(), INTERVAL 7 DAY))
AND textPayload LIKE '%duration:%'
GROUP BY
  normalized_query
HAVING
  execution_count > 10
ORDER BY
  total_cpu_seconds DESC
LIMIT 25;

```

The regex-based normalization above is a reasonable first pass, but it does not handle all cases — dollar-quoted strings, array literals, and multi-line queries will not normalize cleanly. For production use, consider using the `pg_stat_statements` extension in Cloud SQL, which PostgreSQL normalizes internally, and exporting its contents periodically to BigQuery using a scheduled job or a Dataflow pipeline.

Creating a Cloud Monitoring alerting policy for transaction ID utilization:

```

# Create an alerting policy that fires when transaction ID utilization
# exceeds 70% on any Cloud SQL PostgreSQL instance in the project.
# This uses a JSON policy file for precision.

cat > txid-alert-policy.json << 'EOF'
{
  "displayName": "Cloud SQL PostgreSQL - Transaction ID Wraparound Warning",
  "conditions": [
    {
      "displayName": "TXID utilization above 70%",
      "conditionThreshold": {
        "filter": "resource.type = \"cloudsql_database\" AND metric.type =",
        ↪ \"cloudsql.googleapis.com/database/postgresql/transaction_id_utilization\",",
        "aggregations": [
          {
            "alignmentPeriod": "300s",
            "perSeriesAligner": "ALIGN_MEAN",
            "crossSeriesReducer": "REDUCE_MAX"
          }
        ],
        "comparison": "COMPARISON_GT",
        "thresholdValue": 0.70,
        "duration": "300s",
        "trigger": {
          "count": 1
        }
      }
    ]
  },
  "alertStrategy": {
    "autoClose": "3600s"
  },
  "combiner": "OR",
  "enabled": true,
  "notificationChannels": [],
  "severity": "WARNING"
}
EOF

# Create the policy (notification channels must be added separately or via the console)
gcloud alpha monitoring policies create \
  --policy-from-file=txid-alert-policy.json \
  --project=prod-project

# List existing notification channels to attach to the policy
gcloud alpha monitoring channels list \
  --project=prod-project \
  --format="table(name, displayName, type)"

```

AlloyDB-specific monitoring considerations:

AlloyDB exposes metrics in the `alloydb.googleapis.com` namespace and shares the same Cloud Monitoring and Cloud Logging infrastructure as Cloud SQL. The key metrics that are specific to AlloyDB’s columnar engine and distributed storage architecture include:

- `alloydb.googleapis.com/database/memory/columnar_engine_fill_ratio` — the fraction of the columnar engine cache that is occupied. If this consistently approaches 1.0, the columnar engine is evicting data before it can be used effectively.
- `alloydb.googleapis.com/database/replication/log_archive_lag_seconds` — the lag in archiving WAL to AlloyDB’s distributed storage. Unlike standard PostgreSQL replication lag, this measures the pipeline between the primary’s WAL writer and the durable storage layer, not the replica application lag.
- `alloydb.googleapis.com/database/postgresql/num_backends` — shared with the Cloud SQL metric name for consistency, but the threshold context differs. AlloyDB’s connection pooling architecture through AlloyDB Auth Proxy or PgBouncer affects how this number relates to application concurrency.

For AlloyDB, enabling the columnar engine requires setting the `google_columnar_engine.enabled` database flag and then explicitly marking tables for columnar storage or letting the auto-columnarization feature select candidates. Monitoring `columnar_engine_fill_ratio` without first verifying that the engine is actually enabled for your workload is a common instrumentation mistake.

30.5 Building Effective Dashboards Across All Three Platforms

A monitoring system that generates alerts but has no dashboard is a fire alarm with no sprinkler system. Alerts tell you something is wrong; dashboards tell you why, in what context, and how it compares to recent history. Every production cloud database environment should have at minimum two dashboard tiers: a **fleet overview** dashboard visible to anyone on-call, and a **per-instance deep-dive** dashboard used during active troubleshooting.

Fleet overview dashboard — what to include:

The fleet overview should answer the question “which of my databases is sick right now?” in under 30 seconds. Design for this use case explicitly:

- One row per database instance, with CPU, memory, connections, storage, and replication lag displayed as colored status indicators (green/yellow/red) rather than raw numbers.
- An alert count widget showing open alarms grouped by severity.
- A timeline of alert state changes over the last 24 hours — this lets on-call engineers see whether a current alert is a new event or something that has been flapping.
- Storage growth trend over 7 days, not current utilization — current utilization at 72% is a different situation depending on whether it was 60% last week or 71% yesterday.

Per-instance deep-dive dashboard — structure by signal type:

Instance: `prod-postgres-01` Region: `us-east-1` Class: `r6g.4x1`

CPU % (1h line)	FreeableMem (1h line)	DatabaseConnections (1h line, max_conn reference line)	StorageFree (7d line)
Read Latency (ms) (1h, p50/p95/p99)	Write Latency (ms) (1h, p50/p95/p99)	Network Throughput (1h line)	
Top 10 Wait Events (Performance Insights / Query Insights) [stacked bar chart by wait type over time]			
ReplicaLag (seconds) (1h line)	Active Transactions (1h line)	Deadlocks/min (1h bar)	

The key design principle: place the **top wait events chart** in the center of the deep-dive dashboard. When CPU is high, the wait event breakdown immediately reveals whether it is CPU-bound work (CPU wait types), lock contention (lock wait types), or I/O pressure (I/O wait types). This is the single widget that most accelerates root cause identification.

CloudWatch dashboard defined as code using AWS CLI:

```
# Define a CloudWatch dashboard for a single RDS PostgreSQL instance
# as a JSON body, then create it via the CLI.
# This approach allows dashboards to be version-controlled and deployed via CI/CD.

cat > rds-instance-dashboard.json << 'DASHBOARD_EOF'
{
  "widgets": [
    {
      "type": "metric",
      "x": 0, "y": 0, "width": 6, "height": 4,
      "properties": {
        "title": "CPU Utilization %",
        "metrics": [
          ["AWS/RDS", "CPUtilization", "DBInstanceIdentifier", "prod-postgres-01",
            {"stat": "Average", "period": 60, "label": "CPU %"}],
          [{"expression": "85", "label": "Alert Threshold", "color": "#d62728"}]
        ],
        "view": "timeSeries",
        "period": 60,
        "yAxis": {"left": {"min": 0, "max": 100}}
      }
    },
    {
      "type": "metric",
      "x": 6, "y": 0, "width": 6, "height": 4,
      "properties": {
        "title": "Database Connections",
        "metrics": [
          ["AWS/RDS", "DatabaseConnections", "DBInstanceIdentifier", "prod-postgres-01",
            {"stat": "Maximum", "period": 60, "label": "Active Connections"}],
          [{"expressions": "69", "label": "80% of max_connections", "color": "#ff7f0e"}]
        ],
        "view": "timeSeries",
        "period": 60
      }
    },
    {
      "type": "metric",
      "x": 12, "y": 0, "width": 6, "height": 4,
      "properties": {
        "title": "Read / Write Latency (ms)",
```

```

    "metrics": [
      ["AWS/RDS", "ReadLatency", "DBInstanceIdentifier", "prod-postgres-01",
        {"stat": "p99", "period": 60, "label": "Read p99", "color": "#1f77b4"}],
      ["AWS/RDS", "WriteLatency", "DBInstanceIdentifier", "prod-postgres-01",
        {"stat": "p99", "period": 60, "label": "Write p99", "color": "#e377c2"}]
    ],
    "view": "timeSeries",
    "period": 60
  }
},
{
  "type": "metric",
  "x": 18, "y": 0, "width": 6, "height": 4,
  "properties": {
    "title": "Free Storage Space (GB)",
    "metrics": [
      [{"expression": "m1/1073741824", "label": "Free Storage GB", "id": "e1"}],
      ["AWS/RDS", "FreeStorageSpace", "DBInstanceIdentifier", "prod-postgres-01",
        {"id": "m1", "visible": false, "stat": "Minimum", "period": 300}]
    ],
    "view": "timeSeries",
    "period": 300
  }
},
{
  "type": "metric",
  "x": 0, "y": 4, "width": 12, "height": 4,
  "properties": {
    "title": "Replica Lag (seconds)",
    "metrics": [
      ["AWS/RDS", "ReplicaLag", "DBInstanceIdentifier", "prod-postgres-01-replica-1",
        {"stat": "Maximum", "period": 60, "label": "Replica 1 Lag"}],
      [{"expression": "30", "label": "Alert Threshold (30s)", "color": "#d62728"}]
    ],
    "view": "timeSeries",
    "period": 60
  }
},
{
  "type": "metric",
  "x": 12, "y": 4, "width": 12, "height": 4,
  "properties": {
    "title": "Freeable Memory (GB)",
    "metrics": [
      [{"expression": "m1/1073741824", "label": "Freeable Memory GB", "id": "e1"}],
      ["AWS/RDS", "FreeableMemory", "DBInstanceIdentifier", "prod-postgres-01",
        {"id": "m1", "visible": false, "stat": "Minimum", "period": 60}]
    ],
    "view": "timeSeries",
    "period": 60
  }
}
]
}
DASHBOARD_EOF

aws cloudwatch put-dashboard \
  --dashboard-name "RDS-ProdPostgres01-DeepDive" \
  --dashboard-body file://rds-instance-dashboard.json

echo "Dashboard URL:"
echo "https://console.aws.amazon.com/cloudwatch/home#dashboards:name=RDS-ProdPostgres01-DeepDive"

```

The “80% of max_connections” threshold line in the Database Connections widget above uses a literal expression value — substitute your own instance’s actual `max_connections` (queried from the parameter group, per the gotcha above) times 0.8 before deploying this dashboard. Do not copy the placeholder number as-is.

Azure Monitor workbook for Azure SQL — defined as an ARM template snippet:

Azure Monitor Workbooks are the dashboard mechanism for Azure SQL. They support KQL queries, metric charts, and parameter-driven filtering, and they can be exported as ARM templates for version control.

```
{
  "version": "Notebook/1.0",
  "items": [
    {
      "type": 9,
      "content": {
        "version": "KqlParameterItem/1.0",
        "parameters": [
          {
            "id": "db-param",
            "version": "KqlParameterItem/1.0",
            "name": "DatabaseName",
            "type": 2,
            "query": "AzureDiagnostics | where ResourceType == 'DATABASES' | summarize by Resource |
              ↪ project value = Resource, label = Resource",
            "queryType": 0,
            "resourceType": "microsoft.operationalinsights/workspaces"
          },
          {
            "id": "time-param",
            "version": "KqlParameterItem/1.0",
            "name": "TimeRange",
            "type": 4,
            "value": { "durationMs": 86400000 }
          }
        ]
      },
      "name": "parameters"
    },
    {
      "type": 3,
      "content": {
        "version": "KqlItem/1.0",
        "query": "AzureDiagnostics\n| where Category == 'QueryStoreWaitStatistics'\n| where
          ↪ TimeGenerated {TimeRange}\n| where Resource == '{DatabaseName}'\n| summarize TotalWait_ms =
          ↪ sum(delta_wait_time_ms_d) by wait_type_s, bin(TimeGenerated, 5m)\n| render areachart",
        "queryType": 0,
        "title": "Wait Statistics Over Time (stacked area)",
        "timeContext": { "durationMs": 86400000 }
      },
      "name": "wait-stats-chart"
    }
  ]
}
```

Store workbook definitions in source control and deploy them using `az portal` CLI extensions or ARM deployments. This prevents dashboards from drifting between environments and ensures new team members start with the same view as experienced engineers.

30.6 Alerting Strategy: Avoiding Alert Fatigue Without Missing Real Incidents

Every DBA who has operated a cloud database for more than a few months has experienced alert fatigue — a state where the volume of alerts is high enough that engineers begin to ignore them, which is worse than having no alerts at all. The root cause is almost always over-alerting on

symptoms rather than on actionable signals.

The three tests for a well-designed alert:

1. **Actionability test:** When this alert fires at 3 a.m., is there a specific action an on-call engineer can take within 15 minutes that will resolve or mitigate the condition? If the answer is “they need to wait for business hours to investigate,” the alert severity should be at most a warning, not a page.
2. **Specificity test:** Does the alert condition uniquely identify a real problem, or does it fire frequently for transient conditions that resolve themselves? CPU at 85% for 60 seconds might be a routine batch job; CPU at 85% for 5 continuous minutes is more likely to be an actual problem.
3. **Freshness test:** Has this alert been reviewed in the last 90 days? Thresholds that were correct when set often become wrong as workloads evolve, instances are resized, or application behavior changes.

Alert tiering for production databases:

Tier	Criteria	Response expectation
P1 — Page immediately	Database unavailable, replica lag > 5 minutes, storage > 95%, workers at 100%	Wake on-call engineer immediately
P2 — Page within 30 min	CPU > 85% for 10 min, connections > 90% of max, storage > 90%, deadlock rate spike	On-call responds before next check cycle
P3 — Ticket during business hours	CPU > 75% for 30 min, storage growth rate anomaly, slow query count increase	Engineer reviews within next business day
P4 — Dashboard annotation	Configuration drift, index recommendation events, autovacuum duration spikes	Visible but does not generate a notification

Multi-step alert suppression during maintenance windows:

All three platforms support alert suppression or maintenance window configuration, but the mechanisms differ:

- **CloudWatch:** Use CloudWatch Alarms in **ALARM** state with an SNS topic that has a subscription filter. During maintenance, disable the SNS subscription rather than the alarm. This preserves alarm state history.
- **Azure Monitor:** Use Action Groups with webhooks and Logic Apps to implement time-based suppression, or use the built-in **Alert suppression rules** (now called **Action rules**) which can suppress by resource, resource group, or tag during a defined time window.

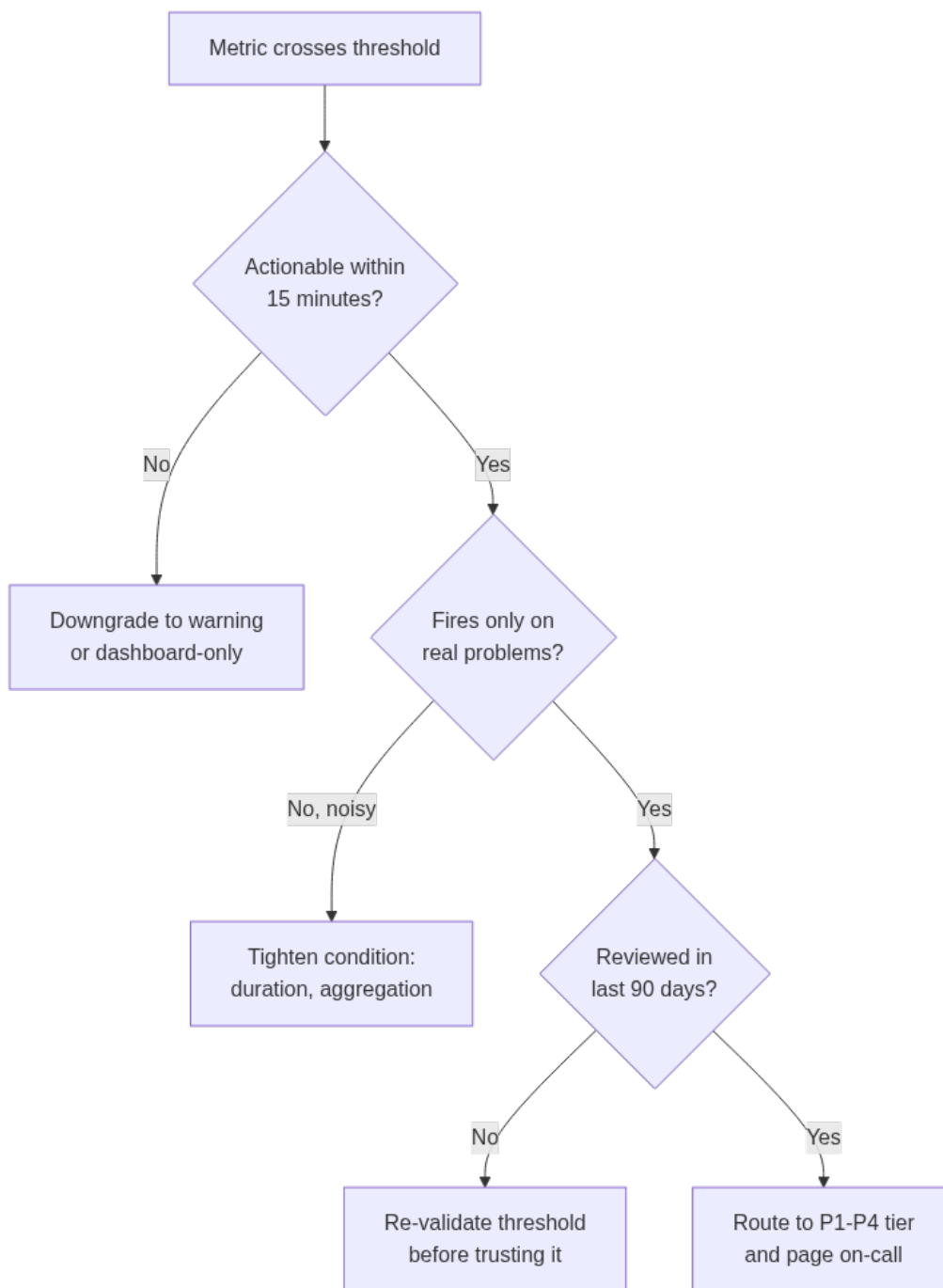


Figure 21: The three tests applied as a gate before an alert is allowed to page anyone.

- **Cloud Monitoring:** Use **Alerting Policy** snoozes, which can be created programmatically via the Cloud Monitoring API and scoped to a specific policy or resource.

```
# Google Cloud Monitoring: Create a snooze for maintenance window
# This suppresses the TXID wraparound alert during a planned VACUUM FREEZE operation

POLICY_NAME=$(gcloud alpha monitoring policies list \
  --project=prod-project \
  --filter="displayName:'Cloud SQL PostgreSQL - Transaction ID Wraparound Warning'" \
  --format="value(name)")

# Calculate maintenance window times (UTC)
START_TIME=$(date -u -d "+30 minutes" +"%Y-%m-%dT%H:%M:%SZ")
END_TIME=$(date -u -d "+3 hours" +"%Y-%m-%dT%H:%M:%SZ")

cat > maintenance-snooze.json << EOF
{
  "displayName": "Planned VACUUM FREEZE maintenance snooze",
  "interval": {
    "startTime": "$START_TIME",
    "endTime": "$END_TIME"
  },
  "criteria": {
    "policies": ["$POLICY_NAME"]
  }
}
EOF

gcloud alpha monitoring snoozes create \
  --snooze-from-file=maintenance-snooze.json \
  --project=prod-project
```

The storage runaway alert pattern: One alert that every cloud DBA should implement but frequently omits is not a threshold alert but a **rate-of-change alert**. Disk at 80% is concerning; disk growing at 5 GB per hour when it was growing at 200 MB per hour yesterday is an emergency — a runaway batch job, a missing partition pruning condition, or an accidental cross join is generating data at an anomalous rate. All three platforms support rate-of-change alerting through metric math (CloudWatch), dynamic thresholds (Azure Monitor Metric Alerts), and absent data plus threshold combination (Cloud Monitoring).

```
# CloudWatch metric math alert for storage growth rate
# RATE() computes the per-second rate of change of the metric
# Multiply by 3600 to convert to GB per hour

aws cloudwatch put-metric-alarm \
  --alarm-name "rds-prod-postgres-01-storage-growth-anomaly" \
  --alarm-description "Storage shrinking faster than 1 GB/hour (FreeStorageSpace decreasing rapidly)" \
  --metrics '[
    {
      "Id": "m1",
      "MetricStat": {
        "Metric": {
          "Namespace": "AWS/RDS",
          "MetricName": "FreeStorageSpace",
          "Dimensions": [{"Name": "DBInstanceIdentifier", "Value": "prod-postgres-01"}]
        },
        "Period": 300,
        "Stat": "Minimum"
      },
      "ReturnData": false
    },
    {
      "Id": "growth_rate",
      "Expression": "RATE(m1) * 3600 / -1073741824",
      "Label": "Storage consumption GB/hour",
      "ReturnData": true
    }
  ]'
```



```

] ' \
--comparison-operator GreaterThanThreshold \
--threshold 1.0 \
--evaluation-periods 2 \
--alarm-actions arn:aws:sns:us-east-1:123456789012:prod-db-alerts \
--treat-missing-data notBreaching

```

The `RATE()` function returns the per-second change; the sign is negative when free space is shrinking (consumption is growing), so we multiply by `-1` to make the threshold comparison intuitive. If free storage is decreasing by more than 1 GB per hour for two consecutive 5-minute periods, the alarm fires.

30.7 Common Instrumentation Gaps and How to Close Them

After reviewing how monitoring is configured across a range of production cloud database environments, a set of instrumentation gaps appears consistently. These are conditions that the platform does not alert on by default, that engineers assume are covered when they are not.

Gap 1: Missing connection pool exhaustion detection

CloudWatch's `DatabaseConnections` metric counts connections at the RDS level. If you use PgBouncer, RDS Proxy, or a connection pool in front of RDS, the application-side pool can be exhausted while RDS shows a healthy connection count. You need application-side metrics (PgBouncer's `SHOW POOLS` output, RDS Proxy's `DatabaseConnectionRequests` and `QueryRequests` metrics in CloudWatch) to detect pool starvation. RDS Proxy specifically surfaces `ClientConnections`, `DatabaseConnections`, `QueryRequests`, and `ClientConnectionsClosed` as distinct metrics.

```

# CloudWatch alarm for RDS Proxy client connection saturation
# RDS Proxy max client connections depend on instance class and max_connections
aws cloudwatch put-metric-alarm \
  --alarm-name "rds-proxy-prod-client-saturation" \
  --metric-name ClientConnections \
  --namespace AWS/RDS \
  --dimensions Name=ProxyName,Value=prod-rds-proxy \
  --statistic Maximum \
  --period 60 \
  --evaluation-periods 3 \
  --threshold 900 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:prod-db-alerts

```

Gap 2: Autovacuum and dead tuple accumulation (PostgreSQL on all platforms)

Neither CloudWatch, Azure Monitor, nor Cloud Monitoring alerts on dead tuple accumulation by default. A table with millions of dead tuples causes index bloat, query plan degradation, and eventually transaction ID wraparound risk. You must push this data from `pg_stat_user_tables` to your monitoring system manually.

```

# Python script to push pg_stat_user_tables dead tuple metrics to CloudWatch.
# Run this as a scheduled Lambda function every 5 minutes on RDS PostgreSQL,
# or as a Cloud Run job for Cloud SQL.

import boto3
import psycopg2
from datetime import datetime, timezone
import os

```

```

def push_dead_tuple_metrics():
    conn = psycopg2.connect(
        host=os.environ["DB_HOST"],
        dbname=os.environ["DB_NAME"],
        user=os.environ["DB_USER"],
        password=os.environ["DB_PASSWORD"],
        sslmode="require"
    )
    cw = boto3.client("cloudwatch", region_name="us-east-1")
    db_instance = os.environ["DB_INSTANCE_ID"]

    with conn.cursor() as cur:
        cur.execute("""
            SELECT
                schemaname,
                relname,
                n_dead_tup,
                n_live_tup,
                CASE WHEN (n_live_tup + n_dead_tup) > 0
                    THEN ROUND(n_dead_tup::numeric / (n_live_tup + n_dead_tup) * 100, 2)
                    ELSE 0
                END AS dead_ratio_pct,
                last_autovacuum,
                last_autoanalyze
            FROM pg_stat_user_tables
            WHERE n_dead_tup > 10000
            ORDER BY n_dead_tup DESC
            LIMIT 20
        """)
        rows = cur.fetchall()

    metric_data = []
    now = datetime.now(timezone.utc)

    for schema, table, dead, live, dead_pct, last_vac, last_analyze in rows:
        table_name = f"{schema}.{table}"
        metric_data.extend([
            {
                "MetricName": "DeadTuples",
                "Dimensions": [
                    {"Name": "DBInstanceIdentifier", "Value": db_instance},
                    {"Name": "TableName", "Value": table_name}
                ],
                "Timestamp": now,
                "Value": float(dead),
                "Unit": "Count"
            },
            {
                "MetricName": "DeadTupleRatioPct",
                "Dimensions": [
                    {"Name": "DBInstanceIdentifier", "Value": db_instance},
                    {"Name": "TableName", "Value": table_name}
                ],
                "Timestamp": now,
                "Value": float(dead_pct),
                "Unit": "Percent"
            }
        ])

    # CloudWatch PutMetricData accepts max 20 metrics per call
    for i in range(0, len(metric_data), 20):
        cw.put_metric_data(
            Namespace="CustomRDS/PostgreSQL",
            MetricData=metric_data[i:i+20]
        )

    conn.close()
    print(f"Pushed {len(metric_data)} dead tuple metrics to CloudWatch.")

if __name__ == "__main__":

```

```
push_dead_tuple_metrics()
```

After this runs, create CloudWatch alarms on `CustomRDS/PostgreSQL/DeadTupleRatioPct > 20` filtered by table name to catch tables where autovacuum is not keeping up. The same pattern applies on Azure SQL (using `sys.dm_db_index_physical_stats` for fragmentation) and on Cloud SQL (using the same `pg_stat_user_tables` query with metrics pushed to Cloud Monitoring custom metrics).

Gap 3: Long-running transaction detection

Long-running transactions block autovacuum, hold locks, and inflate `FreeableMemory` consumption through WAL retention. None of the three platforms alert on long-running transactions by default.

```
-- Query to push to a monitoring system.
-- Works on both RDS PostgreSQL and Cloud SQL PostgreSQL.
-- Returns sessions with open transactions older than 5 minutes.

-- PostgreSQL version
SELECT
    pid,
    username AS username,
    application_name,
    client_addr,
    EXTRACT(EPOCH FROM (now() - xact_start))::INT AS xact_age_seconds,
    state,
    LEFT(query, 200) AS current_query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
    AND state != 'idle'
    AND now() - xact_start > INTERVAL '5 minutes'
ORDER BY xact_age_seconds DESC;

-- SQL Server equivalent (Azure SQL)
SELECT
    s.session_id,
    s.login_name,
    s.program_name,
    s.client_interface_name,
    DATEDIFF(SECOND, t.transaction_begin_time, GETUTCDATE()) AS xact_age_seconds,
    r.status AS session_status,
    LEFT(st.text, 200) AS current_query
FROM sys.dm_tran_active_transactions t
JOIN sys.dm_exec_sessions s ON t.transaction_id = s.transaction_id
LEFT JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) st
WHERE DATEDIFF(SECOND, t.transaction_begin_time, GETUTCDATE()) > 300
    AND s.session_id != @@SPID
ORDER BY xact_age_seconds DESC;
```

Push the row count from the PostgreSQL version as a custom metric. Alert when any transaction has been open for more than 30 minutes — at that point, it is almost never intentional and is very likely to be causing collateral damage to other queries waiting on locks.

Gap 4: Logical replication slot lag

Logical replication slots in PostgreSQL retain WAL indefinitely if a consumer stops consuming. This causes storage to grow unboundedly and is not reflected in the standard `FreeStorageSpace` metric until the situation is already severe. On RDS PostgreSQL, logical replication slots are used by AWS DMS, Debezium, and pglogical consumers. Monitor `pg_replication_slots` directly.

```
-- PostgreSQL: Check logical replication slot lag
-- Works on RDS PostgreSQL, Cloud SQL PostgreSQL, and AlloyDB
SELECT
    slot_name,
```

```
plugin,
slot_type,
active,
active_pid,
pg_size_pretty(
    pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)
) AS retained_wal_size,
pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) AS retained_wal_bytes,
confirmed_flush_lsn
FROM pg_replication_slots
WHERE slot_type = 'logical'
ORDER BY retained_wal_bytes DESC;
```

If `retained_wal_bytes` exceeds your available storage headroom, the instance will eventually run out of space. Alert when any logical slot retains more than 5 GB of WAL. On RDS, you can set the `max_slot_wal_keep_size` parameter (available in PostgreSQL 13+) to cap WAL retention per slot, but doing so means a lagging consumer will have its slot invalidated — a trade-off you must manage at the application level.

Gap 5: Backup window overlap with production load

Automated backups on RDS and Cloud SQL cause brief I/O elevation during the backup window. On heavily loaded instances, this can push I/O latency above alert thresholds, generating pages that resolve themselves once the backup completes. The fix is not to raise the alert threshold — it is to annotate the dashboard with the backup window and ensure the backup window is scheduled during the lowest-traffic period. Both RDS and Cloud SQL allow the backup window to be configured.

```
# RDS: Move the backup window to 02:00-03:00 UTC (lowest traffic window)
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --preferred-backup-window "02:00-03:00" \
  --preferred-maintenance-window "sun:03:30-sun:04:30" \
  --apply-immediately

# Cloud SQL: Move the backup window start time
gcloud sql instances patch prod-cloudsql-pg01 \
  --backup-start-time 02:00
```

Separate the backup window from the maintenance window by at least 30 minutes to prevent overlap between backup I/O and any parameter or minor version updates.

30.8 Integrating Cloud Database Monitoring with Existing Observability Stacks

Most engineering organizations already have a Prometheus/Grafana stack, a Datadog deployment, or a Splunk environment before they migrate databases to the cloud. Bridging cloud-native metrics into these existing systems is preferable to maintaining parallel alert configurations in two places.

Exporting CloudWatch metrics to Prometheus:

The `yet-another-cloudwatch-exporter` (YACE) project is the most widely deployed solution for pulling CloudWatch metrics into Prometheus. Deploy it as a Kubernetes Deployment or an ECS task in the same VPC as your RDS instances.

```
# yace-config.yaml - configuration for pulling RDS metrics into Prometheus
apiVersion: v1alpha1
discovery:
```

```

jobs:
- type: AWS/RDS
  regions:
  - us-east-1
  metrics:
  - name: CPUUtilization
    statistics: [Average]
    period: 60
    length: 300
  - name: DatabaseConnections
    statistics: [Maximum]
    period: 60
    length: 300
  - name: FreeStorageSpace
    statistics: [Minimum]
    period: 300
    length: 900
  - name: ReadLatency
    statistics: [p99]
    period: 60
    length: 300
  - name: WriteLatency
    statistics: [p99]
    period: 60
    length: 300
  - name: ReplicaLag
    statistics: [Maximum]
    period: 60
    length: 300
  - name: FreeableMemory
    statistics: [Minimum]
    period: 60
    length: 300
  dimensionNameRequirements:
  - DBInstanceIdentifier

```

Once YACE is running, the RDS metrics appear in Prometheus as `aws_rds_cpu_utilization_average`, `aws_rds_database_connections_maximum`, and so on. You can then build Grafana dashboards and Prometheus alerting rules using these metrics exactly as you would for self-managed databases.

Azure Monitor to Prometheus using the Azure Monitor Metrics Exporter:

Azure provides a managed Prometheus offering through Azure Monitor managed service for Prometheus, available when using Azure Kubernetes Service. For non-AKS environments, the `azure-metrics-exporter` project provides similar functionality to YACE.

Google Cloud Monitoring to Prometheus:

The `stackdriver_exporter` (now maintained as part of the `prometheus-community` Helm charts) scrapes Cloud Monitoring metrics and exposes them in Prometheus format. The configuration maps `cloudsql.googleapis.com/database/cpu/utilization` to `stackdriver_cloudsql_database_cpu_utilization` in Prometheus.

Sending cloud database alerts to PagerDuty via SNS (AWS example):

```

# Configure SNS to forward RDS CloudWatch alerts to PagerDuty
# PagerDuty provides an SNS integration endpoint per service

PAGERDUTY_ENDPOINT="https://events.pagerduty.com/integration/YOUR_INTEGRATION_KEY/enqueue"

# Create a subscription on the existing prod-db-alerts SNS topic
aws sns subscribe \
  --topic-arn arn:aws:sns:us-east-1:123456789012:prod-db-alerts \
  --protocol https \
  --notification-endpoint "$PAGERDUTY_ENDPOINT"

```

```
# After subscribing, PagerDuty sends a confirmation URL.
# CloudWatch SNS messages include the alarm name, state, and metric values,
# which PagerDuty maps to incident fields automatically.

# Verify the subscription is confirmed
aws sns list-subscriptions-by-topic \
  --topic-arn arn:aws:sns:us-east-1:123456789012:prod-db-alerts \
  --query 'Subscriptions[].{Protocol:Protocol, Endpoint:Endpoint, Status:SubscriptionArn}'
```

When integrating with PagerDuty, Opsgenie, or VictorOps, pay attention to the deduplication key. CloudWatch sends both ALARM and OK state change notifications. Your incident management tool should close incidents automatically when it receives the OK notification. Configure this deduplication explicitly — the default behavior differs by integration and by the incident tool’s version.

Key Takeaways

- **The monitoring mental model shift is from pull to push.** Cloud-managed databases remove your direct access to the OS and log files, but replace it with pre-wired telemetry pipelines. Your job is to define thresholds and subscriptions before incidents occur, not to react to them with ad-hoc CLI commands during an outage.
- **Platform-specific layers serve different purposes.** CloudWatch Metrics, Azure Monitor Metrics, and Cloud Monitoring handle pre-aggregated, low-cost, high-availability signal collection. CloudWatch Logs Insights, Log Analytics (KQL), and Cloud Logging with BigQuery export handle raw query-level and event-level forensics. Use each layer for what it is designed for — routing everything into logs is expensive; keeping everything in metrics loses detail.
- **Several critical signals require custom instrumentation.** Dead tuple accumulation, long-running transactions, logical replication slot lag, and connection pool saturation at the proxy layer are not covered by any platform’s default alerting. These gaps are responsible for a disproportionate share of production incidents that seem surprising because engineers assumed the platform would have flagged the condition.
- **Alert design determines whether your monitoring system is an asset or a liability.** Apply the three tests — actionability, specificity, and freshness — to every alert rule. Tier alerts into P1 through P4 based on what response they require. Rate-of-change alerts for storage catch runaway data growth events far earlier than simple threshold alerts on current utilization.
- **Cloud-native monitoring integrates with existing observability stacks through well-supported exporters.** YACE for CloudWatch-to-Prometheus, the azure-metrics-exporter for Azure Monitor, and the stackdriver-exporter for Cloud Monitoring all allow you to surface cloud database metrics in Grafana dashboards and Prometheus alerting rules alongside your application and infrastructure metrics, preserving a single pane of glass without duplicating alert logic across two systems.

Chapter 31: IAM-Based Database Authentication

Cloud-managed databases fundamentally change how you think about authentication. The `pg_hba.conf` entries you’ve maintained for years, the SQL Server logins tied to Windows accounts, the service accounts rotated manually every quarter — all of that shifts toward a unified identity model built on the cloud provider’s IAM plane. IAM-based database authentication means the database validates identity through short-lived tokens issued by the cloud provider rather than long-lived passwords stored in the database itself. This chapter walks through how AWS RDS, Azure SQL, and Google Cloud SQL implement IAM authentication, how to configure it for production workloads, and where the sharp edges are. The goal is not to replace your understanding of database security — it is to map that knowledge onto the identity primitives each cloud provider offers.

31.1 Why IAM Authentication Changes the Security Model

Traditional database authentication operates on a shared-secret model. A password is created, stored in the database, distributed to the application, and periodically rotated. Each step introduces risk: passwords end up in configuration files, rotation breaks connections, and auditing who authenticated when requires database-level logging that many teams never configure properly.

IAM authentication flips this. Instead of a password, a client requests a short-lived token from the IAM service — typically valid for 15 minutes — and presents that token as the password to the database. The database validates the token against the IAM service in real time. No static secret leaves the IAM boundary. When the token expires, it cannot be reused. Rotation happens automatically because tokens are never stored.

The analogy to your existing knowledge: if `pg_hba.conf` is the gatekeeper that says “this IP range may connect with md5 authentication,” then IAM roles are the gatekeeper that says “this EC2 instance profile or this Kubernetes service account may connect with a token it obtained from the IAM service.” The database still has user accounts — a PostgreSQL role or a SQL Server login — but the credential used to prove identity is ephemeral and cloud-managed.

The operational implications are significant. You no longer rotate passwords; you manage which IAM principals have permission to generate tokens. Audit trails move from database logs into CloudTrail, Azure Activity Log, or Cloud Audit Logs — platforms with retention, alerting, and SIEM integration that most teams already operate. The attack surface of stolen credentials shrinks

because a token captured in transit is useless after 15 minutes.

The tradeoffs are real. Applications that open connections slowly or use connection pools with long-lived connections need to handle token renewal. Not every database engine on every cloud platform supports IAM authentication. Some configurations require specific parameter groups or engine versions. You need to understand these constraints before committing to IAM auth in production.

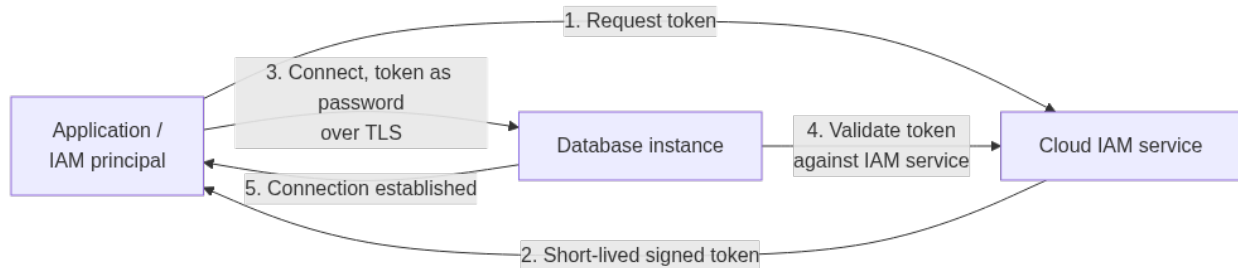


Figure 22: No static secret ever leaves the IAM boundary — the token substitutes for a password and expires on its own.

31.2 IAM Authentication on AWS RDS and Aurora

AWS supports IAM database authentication for RDS MySQL, RDS PostgreSQL, Aurora MySQL, and Aurora PostgreSQL. It is not available for RDS SQL Server, RDS Oracle, or RDS MariaDB. The mechanism is the same across supported engines: the application calls `generate-db-auth-token` to get a signed token, then passes that token as the password when connecting. The token is valid for 15 minutes.

Enabling IAM Authentication

IAM authentication is an instance-level setting, not a database-level setting. You enable it when creating or modifying an instance.

```

# Enable IAM authentication on an existing RDS PostgreSQL instance
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --enable-iam-database-authentication \
  --apply-immediately

# Verify the setting
aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-01 \
  --query 'DBInstances[0].IAMDatabaseAuthenticationEnabled'
  
```

Enabling IAM auth does not disable password authentication. Existing users can still connect with passwords. You enable IAM auth at the instance level, then selectively grant the `rds_iam` role to specific database users. This lets you migrate gradually.

Creating the Database User

On PostgreSQL, you create a regular role and grant it the `rds_iam` group. The role must not have a password — or more precisely, the password is irrelevant because it will never be used for IAM-authenticated connections.


```

-- On RDS PostgreSQL or Aurora PostgreSQL
-- Create an application user that authenticates via IAM
CREATE USER app_service WITH LOGIN;
GRANT rds_iam TO app_service;

-- Grant necessary schema privileges
GRANT CONNECT ON DATABASE production TO app_service;
GRANT USAGE ON SCHEMA public TO app_service;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_service;

-- For future tables created in the schema
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_service;

```

On RDS MySQL and Aurora MySQL, the syntax is slightly different:

```

-- On RDS MySQL or Aurora MySQL
CREATE USER 'app_service'@'%' IDENTIFIED WITH AWSAuthenticationPlugin AS 'RDS';
GRANT SELECT, INSERT, UPDATE, DELETE ON production.* TO 'app_service'@'%';

```

IAM Policy for Token Generation

The IAM principal that needs database access — an EC2 instance role, an ECS task role, a Lambda execution role, a Kubernetes service account with IRSA — needs an IAM policy that allows the `rds-db:connect` action. The resource ARN encodes the database instance, region, account, and database username.

```

# Get the resource ID needed for the IAM policy (it's NOT the instance identifier)
aws rds describe-db-instances \
    --db-instance-identifier prod-postgres-01 \
    --query 'DBInstances[0].DbiResourceId'
# Returns something like: db-ABCDEFGHIJKLMNQRSTUWXYZ

```

The IAM policy:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "rds-db:connect",
      "Resource":
        ↪ "arn:aws:rds-db:us-east-1:123456789012:dbuser:db-ABCDEFGHIJKLMNQRSTUWXYZ/app_service"
    }
  ]
}

```

The resource format is: `arn:aws:rds-db:<region>:<account-id>:dbuser:<db-resource-id>/<db-username>`. Using the `DbiResourceId` rather than the instance identifier is a common gotcha — the IAM policy silently allows the action but the token will not work if you use the wrong resource identifier.

Generating and Using the Auth Token

```

import boto3
import psycpg2
import ssl

def get_connection():
    rds_client = boto3.client('rds', region_name='us-east-1')

    # Token is valid for 15 minutes - generate it close to connection time
    token = rds_client.generate_db_auth_token(
        DBHostname='prod-postgres-01.abc123.us-east-1.rds.amazonaws.com',
        Port=5432,
        DBUsername='app_service',
        Region='us-east-1'
    )

```

```

# SSL is required for IAM authentication - the token is useless without it
# Download the RDS CA bundle: https://truststore.pki.rds.amazonaws.com/
ssl_context = ssl.create_default_context()
ssl_context.load_verify_locations('us-east-1-bundle.pem')

conn = psycopg2.connect(
    host='prod-postgres-01.abc123.us-east-1.rds.amazonaws.com',
    port=5432,
    database='production',
    user='app_service',
    password=token,
    sslmode='verify-full',
    sslrootcert='us-east-1-bundle.pem'
)

return conn

```

SSL is mandatory, not optional. AWS validates the token only over encrypted connections. If your application currently connects with `sslmode=disable` or `sslmode=allow`, IAM authentication will not work regardless of how correctly you’ve configured everything else.

IAM Auth with RDS Proxy

RDS Proxy adds an important capability: it handles token renewal on behalf of the application. When using RDS Proxy with IAM authentication, the application authenticates to the Proxy using a token, but the Proxy manages the connection pool to the underlying RDS instance using whatever credential model you’ve configured. This is the recommended pattern for Lambda and ECS applications that open many short-lived connections.

```

# Create an RDS Proxy with IAM authentication enabled
aws rds create-db-proxy \
  --db-proxy-name prod-postgres-proxy \
  --engine-family POSTGRES \
  --auth '[{
    "AuthScheme": "SECRETS",
    "IAMAuth": "REQUIRED",
    "SecretArn": "arn:aws:secretsmanager:us-east-1:123456789012:secret:rds-proxy-secret"
  }]' \
  --role-arn arn:aws:iam::123456789012:role/rds-proxy-role \
  --vpc-subnet-ids subnet-abc123 subnet-def456 \
  --vpc-security-group-ids sg-789xyz \
  --require-tls

```

With `IAMAuth: REQUIRED` on the proxy, every connection to the proxy endpoint must use a valid IAM token. The proxy handles the actual database connection internally.

Terraform Configuration

```

resource "aws_db_instance" "prod_postgres" {
  identifier      = "prod-postgres-01"
  engine         = "postgres"
  engine_version = "15.4"
  instance_class = "db.r6g.2xlarge"
  allocated_storage = 500
  storage_type     = "gp3"
  db_name         = "production"
  username        = "dbadmin"
  password        = var.master_password

  # Enable IAM authentication
  iam_database_authentication_enabled = true

  # SSL enforcement via parameter group
  parameter_group_name = aws_db_parameter_group.postgres_ssl.name
}

```

```

multi_az          = true
deletion_protection = true

tags = {
  Environment = "production"
  Team        = "platform-data"
}
}

resource "aws_db_parameter_group" "postgres_ssl" {
  name     = "prod-postgres-ssl-params"
  family   = "postgres15"

  parameter {
    name = "rds.force_ssl"
    value = "1"
  }
}

# IAM policy allowing a specific role to connect as app_service
resource "aws_iam_policy" "rds_connect" {
  name = "rds-iam-connect-app-service"

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = "rds-db:connect"
      Resource =
↪ "arn:aws:rds-db:us-east-1:${var.account_id}:dbuser:${aws_db_instance.prod_postgres.resource_id}/app_service"
    }]
  })
}

```

31.3 Azure SQL and Entra ID (Azure Active Directory) Authentication

Azure SQL calls its IAM authentication “Microsoft Entra ID authentication” (formerly Azure Active Directory authentication). The architecture differs from AWS: rather than generating a token manually, the application obtains an OAuth 2.0 access token from the Entra ID endpoint and uses it as the password for SQL connections. The token is bearer-based, valid for approximately one hour.

Azure SQL supports Entra ID authentication for Azure SQL Database, Azure SQL Managed Instance, and Azure Synapse Analytics. Unlike AWS, where password and IAM auth coexist on the same user, Azure SQL can enforce Entra-only authentication at the server level, blocking all SQL authentication logins.

Configuring Entra ID Authentication

```

# Set an Entra ID admin for the Azure SQL server
# This is a prerequisite - Entra ID auth requires an Entra admin be set
az sql server ad-admin create \
  --resource-group prod-rg \
  --server prod-sql-server \
  --display-name "DBA-Admin-Group" \
  --object-id "a1b2c3d4-e5f6-7890-abcd-ef1234567890"

# Optionally enforce Entra-only authentication (disables SQL logins entirely)
az sql server ad-only-auth enable \
  --resource-group prod-rg \

```

```
--name prod-sql-server

# Verify the setting
az sql server ad-only-auth get \
  --resource-group prod-rg \
  --name prod-sql-server
```

The `ad-only-auth enable` command is a one-way operation in terms of impact — it immediately breaks any application using SQL authentication. Test this in staging with realistic connection patterns before applying to production. The master admin password continues to work even with Entra-only auth enabled because Azure maintains an emergency bypass for server administration, but you cannot create new SQL logins or reset passwords for existing ones.

Creating Entra ID Users in the Database

```
-- Connect to the Azure SQL database as the Entra ID admin
-- Create a contained database user mapped to an Entra ID service principal
-- (No password is specified - authentication happens at the Entra level)

-- For an Entra ID group
CREATE USER [platform-data-app-group] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [platform-data-app-group];
ALTER ROLE db_datawriter ADD MEMBER [platform-data-app-group];

-- For a specific managed identity (application service principal)
CREATE USER [prod-app-service-identity] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [prod-app-service-identity];
ALTER ROLE db_datawriter ADD MEMBER [prod-app-service-identity];

-- Check what external users exist in the database
SELECT name, type_desc, authentication_type_desc
FROM sys.database_principals
WHERE authentication_type_desc = 'EXTERNAL';
```

The `FROM EXTERNAL PROVIDER` clause is the Azure SQL equivalent of creating a Windows-authenticated login in SQL Server on-premises. If you've managed SQL Server in a domain environment, this will feel familiar — except you're mapping to Entra ID identities instead of Active Directory accounts.

Acquiring Tokens and Connecting from Applications

```
from azure.identity import DefaultAzureCredential, ManagedIdentityCredential
import pyodbc
import struct

def get_azure_sql_connection():
    # DefaultAzureCredential tries: environment variables, managed identity,
    # Azure CLI credentials, Visual Studio Code - in that order.
    # In production on Azure (App Service, AKS, VM), this resolves to managed identity.
    credential = DefaultAzureCredential()

    # Request a token for Azure SQL Database
    token = credential.get_token("https://database.windows.net/.default")

    # pyodbc requires the token in a specific binary format
    # This is a known gotcha - the raw token string does not work directly
    token_bytes = token.token.encode("UTF-16-LE")
    token_struct = struct.pack(f'<I{len(token_bytes)}s', len(token_bytes), token_bytes)

    SQL_COPT_SS_ACCESS_TOKEN = 1256 # defined in msodbcsql.h

    conn = pyodbc.connect(
        "Driver={ODBC Driver 18 for SQL Server};"
        "Server=prod-sql-server.database.windows.net,1433;"
        "Database=production;"
        "Encrypt=yes;"
```

```
    "TrustServerCertificate=no;",
    attrs_before={SQL_COPT_SS_ACCESS_TOKEN: token_struct}
)

return conn
```

The `struct.pack` step is the most common source of errors when first implementing this. The ODBC driver expects the token encoded as UTF-16-LE with a 4-byte little-endian length prefix. Passing the token as a plain string in the connection password field does not work with pyodbc; it works only with certain JDBC drivers that handle token injection differently.

System-Assigned vs. User-Assigned Managed Identity

For Azure resources connecting to Azure SQL, managed identities are the standard pattern. A system-assigned managed identity is created with the resource and deleted when the resource is deleted. A user-assigned managed identity is a standalone Azure resource you can assign to multiple services — useful when you want multiple App Service instances to share a single database identity without maintaining separate database users for each.

```
# Create a user-assigned managed identity
az identity create \
  --name prod-app-db-identity \
  --resource-group prod-rg \
  --location eastus

# Assign it to an App Service
az webapp identity assign \
  --resource-group prod-rg \
  --name prod-web-app \
  --identities
  ↪ /subscriptions/sub-id/resourceGroups/prod-rg/providers/Microsoft.ManagedIdentity/userAssignedIdentities/prod-app-db-identity

# Get the client ID - needed when explicitly requesting a token for a specific identity
az identity show \
  --name prod-app-db-identity \
  --resource-group prod-rg \
  --query clientId
```

When a resource has multiple managed identities assigned, `DefaultAzureCredential` may not select the right one. In that case, use `ManagedIdentityCredential(client_id="<client-id>")` explicitly to target the correct identity.

31.4 Google Cloud SQL and AlloyDB IAM Authentication

Google Cloud SQL supports IAM database authentication for PostgreSQL instances and MySQL instances. AlloyDB for PostgreSQL supports IAM authentication as well. The implementation on Cloud SQL uses the Cloud SQL Auth Proxy or the direct IAM authentication feature added in later versions; both ultimately validate against the Google Cloud IAM service.

Enabling IAM Authentication on Cloud SQL

```
# Create a Cloud SQL PostgreSQL instance with IAM authentication enabled
gcloud sql instances create prod-postgres-01 \
  --database-version=POSTGRES_15 \
  --tier=db-custom-8-32768 \
  --region=us-central1 \
  --availability-type=REGIONAL \
  --storage-type=SSD \
  --storage-size=500GB \
```

```

--storage-auto-increase \
--database-flags=cloudsql.iam_authentication=on

# Enable IAM authentication on an existing instance
gcloud sql instances patch prod-postgres-01 \
--database-flags=cloudsql.iam_authentication=on

# Verify
gcloud sql instances describe prod-postgres-01 \
--format=json(settings.databaseFlags)"

```

Adding IAM Users

On Cloud SQL, IAM users are added at the instance level through the Cloud SQL API, which automatically creates the corresponding PostgreSQL role. You do not create the role manually in psql.

```

# Add a service account as an IAM database user
# The username in PostgreSQL will be the service account email truncated at the @ symbol
gcloud sql users create prod-app-sa@project-id.iam.gserviceaccount.com \
--instance=prod-postgres-01 \
--type=CLOUD_IAM_SERVICE_ACCOUNT

# Add a human user (Google account)
gcloud sql users create rajesh@company.com \
--instance=prod-postgres-01 \
--type=CLOUD_IAM_USER

# List IAM users on the instance
gcloud sql users list --instance=prod-postgres-01

```

After adding the IAM user at the instance level, you still need to grant database-level privileges inside PostgreSQL. The service account email becomes a PostgreSQL role, but the PostgreSQL role name drops the domain suffix:

```

-- Connect to the database as the superuser
-- Grant privileges to the IAM service account role
-- Note: the role name is truncated - just the part before the @
GRANT CONNECT ON DATABASE production TO "prod-app-sa";
GRANT USAGE ON SCHEMA public TO "prod-app-sa";
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO "prod-app-sa";

-- The full email format is used for CLOUD_IAM_USER types
GRANT CONNECT ON DATABASE production TO "rajesh@company.com";

-- Verify the roles exist
SELECT rolname, rolcanlogin FROM pg_roles
WHERE rolname IN ('prod-app-sa', 'rajesh@company.com');

```

Using the Cloud SQL Auth Proxy

The Cloud SQL Auth Proxy handles IAM authentication and SSL termination transparently. It runs as a sidecar process alongside your application and listens on a local port. Your application connects to 127.0.0.1:5432 as if connecting to a local PostgreSQL instance; the proxy handles authentication to Cloud SQL.

```

# Download and run the Cloud SQL Auth Proxy
# In Kubernetes, this runs as a sidecar container
curl -o cloud-sql-proxy \
https://storage.googleapis.com/cloud-sql-connectors/cloud-sql-proxy/v2.7.0/cloud-sql-proxy.linux.amd64
chmod +x cloud-sql-proxy

# Run the proxy with IAM authentication
./cloud-sql-proxy \
--auto-iam-authn \
--port=5432 \

```

```
project-id:us-central1:prod-postgres-01

# The application then connects to localhost:5432 with the IAM username
# No password needed - the proxy provides the credential
psql "host=127.0.0.1 port=5432 dbname=production user=prod-app-sa sslmode=disable"
```

The `--auto-iam-authn` flag is what enables IAM-based authentication through the proxy. Without it, the proxy provides SSL termination and authorized network access but still requires password authentication. When connecting through the proxy with IAM auth, you do not provide a password — the proxy inserts the token automatically.

Kubernetes Deployment Pattern

In GKE, the Cloud SQL Auth Proxy runs as a sidecar container within the application pod. The pod uses a Kubernetes service account annotated with Workload Identity to assume a Google service account.

```
# Create a Google service account for the application
gcloud iam service-accounts create prod-app-db-sa \
  --display-name="Production App Database SA"

# Grant it the Cloud SQL Client role
gcloud projects add-iam-policy-binding project-id \
  --member="serviceAccount:prod-app-db-sa@project-id.iam.gserviceaccount.com" \
  --role="roles/cloudsql.client"

# Allow the Kubernetes service account to impersonate the Google service account
gcloud iam service-accounts add-iam-policy-binding prod-app-db-sa@project-id.iam.gserviceaccount.com \
  --role="roles/iam.workloadIdentityUser" \
  --member="serviceAccount:project-id.svc.id.goog[prod-namespace/prod-app-ksa]"

# Annotate the Kubernetes service account
kubectl annotate serviceaccount prod-app-ksa \
  --namespace prod-namespace \
  iam.gke.io/gcp-service-account=prod-app-db-sa@project-id.iam.gserviceaccount.com
```

31.5 Connection Pooling and Token Renewal in Production

The 15-minute token lifetime on AWS and the approximately one-hour token lifetime on Azure and GCP create a challenge for connection pools. A connection pool that opens connections at startup and reuses them indefinitely will present expired tokens when it tries to open new connections after the token window has passed. The existing connections continue to work — IAM authentication validates the token at connection establishment, not on every query — but new connections will fail.

This distinction matters: IAM auth does not re-validate the token on every query. Once a connection is established, it stays open until closed or until the database server terminates it. The problem occurs when the pool needs to create a new connection, typically because the application load increases or an idle connection times out.

AWS with PgBouncer

PgBouncer in transaction or session pooling mode works with IAM authentication if you implement token rotation in the configuration. PgBouncer reads the password from its `userlist.txt` file; you need an external process to refresh that file with a new token before expiration.

A practical pattern: run a sidecar process that regenerates the token every 10 minutes and rewrites `userlist.txt`, then sends `RELOAD` to PgBouncer:

```
#!/bin/bash
# token-refresher.sh - runs as a sidecar alongside PgBouncer
# Refresh interval is 10 minutes, well within the 15-minute token window

PGBOUNCER_USERLIST="/etc/pgbouncer/userlist.txt"
DB_HOST="prod-postgres-01.abc123.us-east-1.rds.amazonaws.com"
DB_USER="app_service"
DB_REGION="us-east-1"

while true; do
    TOKEN=$(aws rds generate-db-auth-token \
        --hostname "$DB_HOST" \
        --port 5432 \
        --username "$DB_USER" \
        --region "$DB_REGION")

    if [ $? -eq 0 ]; then
        # Write the new token - escape any special characters
        echo "\"${DB_USER}\" \"${TOKEN}\"" > "${PGBOUNCER_USERLIST}.tmp"
        mv "${PGBOUNCER_USERLIST}.tmp" "${PGBOUNCER_USERLIST}"

        # Signal PgBouncer to reload its configuration
        psql -h 127.0.0.1 -p 6432 -U pgbouncer pgbouncer -c "RELOAD"
        echo "$(date): Token refreshed successfully"
    else
        echo "$(date): Token generation failed - keeping existing token" >&2
    fi

    sleep 600 # 10-minute interval
done
```

The `mv` (atomic rename) prevents PgBouncer from reading a partially written file. The `RELOAD` command causes PgBouncer to re-read its configuration without dropping existing connections — the active connection pool is preserved; only the credential used for new connections is updated.

AWS with RDS Proxy

As mentioned in Section 31.2, RDS Proxy is the cleanest solution to the token renewal problem on AWS. The proxy maintains a persistent connection pool to RDS using credentials from Secrets Manager; it handles renewal internally. The application presents a token to the proxy endpoint and never needs to worry about pool-level token expiration.

Azure SQL with Connection Pools

Azure SQL tokens have a longer validity window, but the pattern is the same. The `azure-identity` SDK caches tokens and handles renewal automatically when you use `DefaultAzureCredential` or `ManagedIdentityCredential`. If you call `credential.get_token(...)` on every new connection from the pool, the SDK returns the cached token when it still has sufficient lifetime and transparently requests a new one when needed.

```
from azure.identity import ManagedIdentityCredential
from azure.core.credentials import AccessToken
import pyodbc
import struct
import threading
import time

class AzureSQLConnectionFactory:
    """
    Token-aware connection factory for use with any connection pool
    that accepts a factory callable for creating new connections.
```


The azure-identity SDK handles token caching internally - we simply call get_token() on every new connection request.

```

def __init__(self, server: str, database: str, client_id: str):
    self.server = server
    self.database = database
    self.credential = ManagedIdentityCredential(client_id=client_id)
    self._token_resource = "https://database.windows.net/.default"

def _token_to_struct(self, token_str: str) -> bytes:
    token_bytes = token_str.encode("UTF-16-LE")
    return struct.pack(f'<I{len(token_bytes)}s', len(token_bytes), token_bytes)

def create_connection(self) -> pyodbc.Connection:
    token = self.credential.get_token(self._token_resource)
    token_struct = self._token_to_struct(token.token)

    conn = pyodbc.connect(
        f"Driver={{ODBC Driver 18 for SQL Server}};"
        f"Server={self.server},1433;"
        f"Database={self.database};"
        f"Encrypt=yes;"
        f"TrustServerCertificate=no;"
        attrs_before={1256: token_struct}
    )
    return conn

```

With SQLAlchemy, you pass a creator callable to create_engine:

```

from sqlalchemy import create_engine

factory = AzureSQLConnectionFactory(
    server="prod-sql-server.database.windows.net",
    database="production",
    client_id="<user-assigned-managed-identity-client-id>"
)

engine = create_engine(
    "mssql+pyodbc://",
    creator=factory.create_connection,
    pool_size=10,
    max_overflow=20,
    pool_pre_ping=True,      # validates connections before checkout
    pool_recycle=1800        # recycle connections every 30 minutes to prevent stale tokens
)

```

The pool_recycle=1800 setting ensures that even connections that are never explicitly closed get recycled every 30 minutes, which is well within the token validity window for Azure. pool_pre_ping=True sends a lightweight query before handing a connection to the application, catching broken connections before the application sees an error.

GCP with the Cloud SQL Python Connector

Google provides an official Cloud SQL Python Connector library that integrates directly with the IAM authentication mechanism, handles connection refreshes, and manages the certificate rotation that the Cloud SQL service uses internally.

```

from google.cloud.sql.connector import Connector, IPTypes
import sqlalchemy
import pg8000

def create_cloud_sql_engine(
    instance_connection_name: str, # "project:region:instance"
    database: str,
    db_user: str,
    ip_type: IPTypes = IPTypes.PRIVATE

```

```

) -> sqlalchemy.engine.Engine:

connector = Connector(ip_type=ip_type)

def getconn() -> pg8000.dbapi.Connection:
    conn = connector.connect(
        instance_connection_name,
        "pg8000",
        user=db_user,
        db=database,
        enable_iam_auth=True,    # use IAM authentication instead of password
        ip_type=ip_type,
    )
    return conn

engine = sqlalchemy.create_engine(
    "postgresql+pg8000://",
    creator=getconn,
    pool_size=5,
    max_overflow=10,
    pool_pre_ping=True,
)

return engine

# Usage
engine = create_cloud_sql_engine(
    instance_connection_name="my-project:us-central1:prod-postgres-01",
    database="production",
    db_user="prod-app-sa",    # truncated service account name (before the @)
)

```

The `Connector` object maintains background threads that refresh connection certificates and tokens before they expire. A single `Connector` instance is meant to be long-lived — create it once at application startup, not per request. Creating a new `Connector` on every request defeats the refresh logic and adds latency.

31.6 Auditing and Monitoring IAM-Authenticated Connections

One of the core arguments for IAM authentication is that it shifts your audit trail to the cloud provider's logging infrastructure. That shift is only an improvement if you actually configure logging to capture what you need.

AWS CloudTrail for RDS IAM Events

Every `generate-db-auth-token` API call is a data plane event that CloudTrail captures if you enable data events for RDS. Management events (creating instances, modifying parameter groups) are captured by default; data events require explicit configuration.

```

# Create a CloudTrail trail with RDS data events enabled
aws cloudtrail create-trail \
    --name prod-database-audit \
    --s3-bucket-name prod-audit-logs-bucket \
    --is-multi-region-trail \
    --enable-log-file-validation

# Start logging
aws cloudtrail start-logging --name prod-database-audit

# Add data event logging for RDS
aws cloudtrail put-event-selectors \

```

```
--trail-name prod-database-audit \
--event-selectors '[{
  "ReadWriteType": "All",
  "IncludeManagementEvents": true,
  "DataResources": [{
    "Type": "AWS::RDS::DBInstance",
    "Values": ["arn:aws:rds:us-east-1:123456789012:db:prod-postgres-01"]
  }]
}]'
```

In the CloudTrail logs, a successful `generate-db-auth-token` call shows the `userIdentity` of the IAM principal (the role ARN of the EC2 instance, ECS task, or Lambda function), the timestamp, the source IP, and the target database username. This gives you a precise record of which workload generated credentials to connect as which database user at what time.

At the database level, RDS PostgreSQL's `log_connections` and `log_disconnections` parameters capture the connection events alongside the PostgreSQL role name. Correlating CloudTrail token generation events with PostgreSQL connection logs gives you end-to-end traceability.

```
-- Enable connection logging on RDS PostgreSQL via parameter group
-- (Set in the RDS parameter group, not directly in postgresql.conf)
-- log_connections = 1
-- log_disconnections = 1
-- log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '

-- Query pg_stat_activity to see current IAM-authenticated connections
-- IAM-authenticated connections look identical to password connections at the pg_stat level
-- The distinction is visible only in the authentication log messages
SELECT
  pid,
  username,
  application_name,
  client_addr,
  backend_start,
  state,
  wait_event_type,
  wait_event
FROM pg_stat_activity
WHERE username = 'app_service'
ORDER BY backend_start DESC;
```

Azure SQL Auditing

Azure SQL has a built-in auditing feature that writes events to Azure Blob Storage, Log Analytics, or an Event Hub. For Entra ID authentication specifically, audit logs capture the authentication type alongside the principal name.

```
# Enable auditing to a Log Analytics workspace
az sql server audit-policy update \
--resource-group prod-rg \
--name prod-sql-server \
--state Enabled \
--log-analytics-workspace-resource-id
↪ /subscriptions/sub-id/resourceGroups/prod-rg/providers/Microsoft.OperationalInsights/workspaces/prod-log-analytics
↪ \
--log-analytics-target-state Enabled

# Configure database-level diagnostic settings
az monitor diagnostic-settings create \
--name sql-audit-diagnostics \
--resource
↪ /subscriptions/sub-id/resourceGroups/prod-rg/providers/Microsoft.Sql/servers/prod-sql-server/databases/production
↪ \
--workspace
↪ /subscriptions/sub-id/resourceGroups/prod-rg/providers/Microsoft.OperationalInsights/workspaces/prod-log-analytics
↪ \
```

```
--logs '[{"category": "SQLSecurityAuditEvents", "enabled": true}]'
```

Once events are in Log Analytics, you can query them with Kusto Query Language:

```
// Find all Entra ID authentication events in the last 24 hours
AzureDiagnostics
| where Category == "SQLSecurityAuditEvents"
| where TimeGenerated > ago(24h)
| where action_name_s == "DATABASE AUTHENTICATION SUCCEEDED"
| where authentication_type_s == "EXTERNAL PROVIDER"
| project TimeGenerated,
           server_instance_name_s,
           database_name_s,
           session_server_principal_name_s,
           client_ip_s,
           application_name_s
| order by TimeGenerated desc

// Alert on authentication failures for Entra ID users
AzureDiagnostics
| where Category == "SQLSecurityAuditEvents"
| where TimeGenerated > ago(1h)
| where action_name_s == "DATABASE AUTHENTICATION FAILED"
| where authentication_type_s == "EXTERNAL PROVIDER"
| summarize FailureCount = count() by session_server_principal_name_s, client_ip_s
| where FailureCount > 5
```

GCP Cloud Audit Logs

Cloud SQL IAM authentication events appear in the Cloud Audit Logs under Data Access logs. You need to explicitly enable data access logging for the Cloud SQL API, as it is not on by default due to volume considerations.

```
# Enable data access audit logging for Cloud SQL
# This is done at the project level in IAM policy
gcloud projects get-iam-policy project-id --format=json > policy.json

# Edit policy.json to add the auditLogConfig, then update:
gcloud projects set-iam-policy project-id policy.json
```

The audit config JSON block to add:

```
{
  "auditConfigs": [
    {
      "service": "cloudsql.googleapis.com",
      "auditLogConfigs": [
        {"logType": "DATA_READ"},
        {"logType": "DATA_WRITE"}
      ]
    }
  ]
}
```

Querying the logs in Cloud Logging:

```
resource.type="cloudsql_database"
protoPayload.serviceName="cloudsql.googleapis.com"
protoPayload.methodName="cloudsql.instances.connect"
protoPayload.authenticationInfo.principalEmail="prod-app-sa@project-id.iam.gserviceaccount.com"
```

31.7 Troubleshooting IAM Authentication Failures

IAM authentication failures produce error messages that are often less informative than standard password authentication failures. The database typically reports a generic authentication error; the real cause is in the IAM policy, the token generation process, or the SSL configuration. A systematic approach to diagnosis saves significant time.

Common AWS RDS IAM Auth Failures

FATAL: PAM authentication failed for user "app_service"

Despite mentioning PAM, this error appears when RDS IAM authentication fails for PostgreSQL. The actual causes in order of frequency:

1. **Wrong DbResourceId in the IAM policy.** Verify by running `aws rds describe-db-instances --query 'DBInstances[0].DbResourceId'` and comparing to what's in the IAM policy resource ARN.
2. **The `rds_iam` grant is missing.** Confirm inside the database: `SELECT * FROM pg_roles WHERE rolname = 'app_service';` and check that `rds_iam` appears in `pg_auth_members`.
3. **SSL not enforced on the client side.** The token is a signed URL-style string; if the connection is not encrypted, it will not validate. Confirm `sslmode=verify-full` or at minimum `sslmode=require`.
4. **Token generated for the wrong hostname.** The token is cryptographically bound to the hostname passed to `generate-db-auth-token`. If you connect via an alias, CNAME, or Route 53 record, generate the token with that same hostname.
5. **Clock skew.** The token includes a timestamp; if the client's clock is more than a few minutes ahead of or behind the AWS service clock, validation fails. On EC2, this is rare; in containers, it can occur if the host NTP configuration is broken.

```
# Quick diagnostic: test token generation and connection from the environment
# where the failure is occurring, not from your workstation

TOKEN=$(aws rds generate-db-auth-token \
  --hostname prod-postgres-01.abc123.us-east-1.rds.amazonaws.com \
  --port 5432 \
  --username app_service \
  --region us-east-1)

echo "Token generated: ${TOKEN:0:50}..." # Print first 50 chars to confirm non-empty

PGPASSWORD="$TOKEN" psql \
  "host=prod-postgres-01.abc123.us-east-1.rds.amazonaws.com \
  port=5432 \
  dbname=production \
  user=app_service \
  sslmode=require \
  sslrootcert=us-east-1-bundle.pem"
```

Common Azure SQL Entra ID Failures

Login failed for user '<token-identified principal>'.
Error: 18456, Severity: 14, State: 1

This error from Azure SQL with Entra ID authentication usually points to one of:

1. **The managed identity has not been added as a database user.** The `CREATE USER ... FROM EXTERNAL PROVIDER` step in Section 31.3 is easy to miss. The identity can have the Entra ID admin role on the server and still fail to connect to individual databases if the contained database user is not created.
2. **The managed identity client ID is wrong.** When a VM or App Service has multiple managed identities, specifying the wrong `client_id` in `ManagedIdentityCredential` results in a token for a different identity that has no database access.
3. **The Entra ID admin is not set on the server.** Entra ID authentication on Azure SQL requires an Entra admin to be designated, even if that admin is not the one connecting.

```
# Verify the Entra admin is set
az sql server ad-admin list \
  --resource-group prod-rg \
  --server-name prod-sql-server

# Verify the database user exists for the managed identity
# Connect using the Entra ID admin and check
# (Use Azure Data Studio or sqlcmd with an interactive Entra ID login)
sqlcmd -S prod-sql-server.database.windows.net \
  -d production \
  -G \
  -Q "SELECT name, type_desc, authentication_type_desc FROM sys.database_principals WHERE type IN
    ↳ ('E','X');"
# -G enables Azure Active Directory authentication
```

Common GCP Cloud SQL IAM Failures

FATAL: Cloud SQL IAM user authentication failed for user "prod-app-sa"

GCP-specific issues to check:

1. **The username format mismatch.** For service accounts, the PostgreSQL role name is the service account email address with the `@project-id.iam.gserviceaccount.com` portion truncated — only the part before the `@` is used. If the service account is `prod-app-sa@myproject.iam.gserviceaccount.com`, the PostgreSQL user is `prod-app-sa`. For accounts with project IDs in the name, this truncation can be unintuitive.
2. **The Cloud SQL IAM service account was added at the instance level but not granted PostgreSQL privileges.** Adding the user via `gcloud sql users create` creates the PostgreSQL role, but that role has no database privileges until you explicitly grant them inside the database.
3. **The `cloudsql.iam_authentication` flag is not set.** The instance-level flag must be enabled. Verify with `gcloud sql instances describe --format="json(settings.databaseFlags)"`.
4. **Workload Identity binding is incomplete.** In GKE, if the binding between the Kubernetes service account and the Google service account is missing or the annotation is incorrect, the pod runs without the expected identity and token generation fails silently or falls back to the compute metadata service default identity.

```
# Verify the Workload Identity binding from inside the GKE pod
kubectl exec -it <pod-name> -n prod-namespace -- \
  curl -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/email"
# Should return the expected service account email

# Test token acquisition directly
```

```
kubectl exec -it <pod-name> -n prod-namespace -- \
  curl -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token"
```

31.8 Security Hardening Beyond Basic IAM Authentication

Enabling IAM authentication is a starting point. Production security posture requires additional controls layered on top.

Enforce SSL at the Parameter Group Level

Never rely on client-side SSL configuration alone. Enforce SSL at the database engine level so that connections attempting to bypass SSL are rejected even if IAM token validation somehow succeeded.

```
# AWS: enforce SSL via parameter group (PostgreSQL)
aws rds modify-db-parameter-group \
  --db-parameter-group-name prod-postgres-params \
  --parameters "ParameterName=rds.force_ssl,ParameterValue=1,ApplyMethod=immediate"

# Azure: enforced by default for Entra-only auth
# Verify SSL enforcement on Azure SQL
az sql server show \
  --resource-group prod-rg \
  --name prod-sql-server \
  --query "minimalTlsVersion"

# GCP: enforce SSL
gcloud sql instances patch prod-postgres-01 \
  --require-ssl
```

Principle of Least Privilege on IAM Policies

The `rds-db:connect` resource ARN scopes permission to a specific combination of instance and database username. Avoid wildcards:

```
{
  "Effect": "Allow",
  "Action": "rds-db:connect",
  "Resource": "arn:aws:rds-db:us-east-1:123456789012:dbuser:db-ABCDEFGF/app_service"
}
```

This is preferable to:

```
{
  "Effect": "Allow",
  "Action": "rds-db:connect",
  "Resource": "arn:aws:rds-db:us-east-1:123456789012:dbuser:*"
}
```

The wildcard form grants the IAM principal permission to authenticate as any database user on any RDS instance in the account — a significant overreach that is easy to miss because the ARN format is unfamiliar territory for most developers.

Separate IAM Roles by Workload Function

Rather than a single database IAM role shared across all application components, create separate roles for distinct workloads: one for the web tier (read-write on specific tables), one for the reporting service (read-only, different schema), one for the migration tool (DDL permissions, used only during deployment windows). This matches the principle of least privilege and makes the audit trail

actionable — you can distinguish between a spike in read traffic and unexpected write activity by identity alone.

Regular Review of IAM-to-Database User Mappings

IAM principals proliferate. Service accounts get created for short-lived projects, EC2 instances are retired but their roles persist, merge activity adds roles from acquired companies. Schedule a periodic review:

```
-- PostgreSQL: list all roles with rds_iam or IAM authentication access
SELECT
    r.rolname AS database_user,
    r.rolcanlogin,
    r.rolcreatorole,
    r.rolcreatedb,
    array_agg(g.rolname) AS granted_roles
FROM pg_roles r
LEFT JOIN pg_auth_members m ON r.oid = m.member
LEFT JOIN pg_roles g ON m.roleid = g.oid
WHERE g.rolname = 'rds_iam' OR r.rolname IN (
    SELECT username FROM pg_user WHERE usesuper = false
)
GROUP BY r.rolname, r.rolcanlogin, r.rolcreatorole, r.rolcreatedb
HAVING array_agg(g.rolname) @> ARRAY['rds_iam']
ORDER BY r.rolname;

-- Azure SQL: list all external (Entra ID) users and their role memberships
SELECT
    dp.name AS database_user,
    dp.type_desc,
    dp.authentication_type_desc,
    STRING_AGG(rp.name, ', ') AS database_roles
FROM sys.database_principals dp
LEFT JOIN sys.database_role_members drm ON dp.principal_id = drm.member_principal_id
LEFT JOIN sys.database_principals rp ON drm.role_principal_id = rp.principal_id
WHERE dp.authentication_type_desc = 'EXTERNAL'
GROUP BY dp.name, dp.type_desc, dp.authentication_type_desc
ORDER BY dp.name;
```

Cross-reference the results against your active IAM principals. Any database user whose corresponding IAM principal no longer exists in the cloud account should be dropped from the database. The cloud IAM principal being deleted does not automatically remove the database user — that cleanup must be done manually.

Key Takeaways

- IAM database authentication replaces long-lived passwords with short-lived tokens issued by the cloud provider's identity service, eliminating the credential storage and rotation challenges of traditional shared-secret authentication, but requiring applications to handle token renewal when establishing new connections after the token window expires.
- Each cloud provider implements IAM auth differently at the protocol level: AWS generates a signed token passed as the password field; Azure SQL uses an OAuth 2.0 bearer token injected via a driver-level connection attribute in a specific binary format; Google Cloud SQL uses either the Auth Proxy sidecar or the Python Connector library, which handle token management transparently alongside SSL certificate rotation.
- SSL encryption is not optional with IAM authentication on any of the three platforms — the

token is meaningful only over an encrypted channel, and all three providers enforce this at the connection level regardless of client-side configuration.

- Connection pooling requires deliberate design around token expiry: AWS RDS Proxy handles renewal transparently at the proxy layer; Azure’s **azure-identity** SDK caches and renews tokens automatically when **get_token()** is called per connection; GCP’s Cloud SQL Connector manages refresh internally via background threads in a long-lived **Connector** instance.
 - Enabling IAM authentication shifts the primary audit trail from database-level logs to the cloud provider’s centralized logging infrastructure (CloudTrail, Azure Monitor/Log Analytics, Cloud Audit Logs), but realizing that shift requires explicitly enabling data access logging — which is off by default on all three platforms due to volume — and performing cross-database user cleanup when IAM principals are retired, since deleting a cloud identity does not automatically remove the corresponding database user.
-

Chapter 32: VPC, Private Endpoints, and Network Security

Network security for cloud databases is where experienced on-prem DBAs often feel the most disoriented. On-premises, you controlled the physical network, configured firewall rules on the host, and trusted that your database server sat inside a known perimeter. In the cloud, that perimeter dissolves and reforms as a collection of software-defined constructs — VPCs, subnets, security groups, private endpoints, and DNS resolution chains — that you must understand completely before putting production data anywhere near them. This chapter covers how AWS, Azure, and Google Cloud each implement database network isolation, how to connect applications to managed databases without exposing them to the public internet, and how to build defense-in-depth network security that will survive a security audit. The patterns here apply equally to RDS, Aurora, Azure SQL, Cloud SQL, and AlloyDB.

32.1 Virtual Networks: The Foundation of Cloud Database Isolation

Before touching a single database configuration, you need a clear mental model of how virtual networks work across the three clouds. The terminology differs but the concepts are nearly identical.

Mapping Your On-Prem Concepts

On-premises, you had VLANs, subnets, ACLs, and firewall rules implemented in hardware. In the cloud:

On-Premises Concept	AWS	Azure	GCP
Data center network	VPC	Virtual Network (VNet)	VPC
VLAN / network segment	Subnet	Subnet	Subnet
Firewall rules (host-based)	Security Group	NSG (Network Security Group)	Firewall Rules
Network-level ACL	Network ACL	NSG on subnet	VPC Firewall Rules
Private RFC 1918 space	Private subnets	Private subnets	Private subnets

On-Premises Concept	AWS	Azure	GCP
DNS server	Route 53 Private Hosted Zone	Azure Private DNS Zone	Cloud DNS private zone

The one thing that consistently surprises on-prem DBAs: **security groups in AWS are stateful** (return traffic is automatically allowed), while Network ACLs are stateless. For database connectivity, you typically work with security groups, not NACLs directly.

Subnet Architecture for Database Workloads

The standard production layout places your database instances in private subnets — subnets with no route to an internet gateway. Your application tier sits in separate subnets, and all traffic flows through security group rules, not public IP addresses.

For RDS and Aurora, AWS requires subnets in at least two Availability Zones to create a DB Subnet Group. This is not optional — it is a hard requirement even for single-AZ deployments:

```
# Create a DB subnet group spanning two AZs - required before any RDS deployment
aws rds create-db-subnet-group \
  --db-subnet-group-name prod-db-subnet-group \
  --db-subnet-group-description "Production database subnets" \
  --subnet-ids subnet-0a1b2c3d4e5f67890 subnet-0f9e8d7c6b5a43210 \
  --tags Key=Environment,Value=production Key=Team,Value=database

# Verify the subnet group is in the correct VPC and AZs
aws rds describe-db-subnet-groups \
  --db-subnet-group-name prod-db-subnet-group \
  --query
  ↪ 'DBSubnetGroups[0].{VpcId:VpcId,Subnets:Subnets[*].{AZ:SubnetAvailabilityZone.Name,Id:SubnetIdentifier}}'
```

The equivalent in Terraform, which is how most production environments should be managing this:

```
# Terraform: VPC layout for production RDS deployment
resource "aws_vpc" "prod" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support   = true

  tags = {
    Name        = "prod-vpc"
    Environment = "production"
  }
}

resource "aws_subnet" "database_az1" {
  vpc_id            = aws_vpc.prod.id
  cidr_block        = "10.0.32.0/24"
  availability_zone = "us-east-1a"

  tags = {
    Name = "prod-db-subnet-az1"
    Tier = "database"
  }
}

resource "aws_subnet" "database_az2" {
  vpc_id            = aws_vpc.prod.id
  cidr_block        = "10.0.33.0/24"
  availability_zone = "us-east-1b"

  tags = {
    Name = "prod-db-subnet-az2"
    Tier = "database"
  }
}
```

```
    }  
  }  
  
  resource "aws_db_subnet_group" "prod" {  
    name           = "prod-db-subnet-group"  
    subnet_ids     = [aws_subnet.database_az1.id, aws_subnet.database_az2.id]  
  
    tags = {  
      Name = "Production DB Subnet Group"  
    }  
  }  
}  
  
# Security group: database instances  
# Only allows inbound from the application tier security group  
resource "aws_security_group" "database" {  
  name           = "prod-database-sg"  
  description     = "Security group for RDS/Aurora instances"  
  vpc_id         = aws_vpc.prod.id  
  
  ingress {  
    description     = "PostgreSQL from application tier"  
    from_port       = 5432  
    to_port         = 5432  
    protocol        = "tcp"  
    security_groups = [aws_security_group.application.id]  
  }  
  
  egress {  
    description = "All outbound - needed for IAM auth token validation"  
    from_port   = 0  
    to_port     = 0  
    protocol    = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  
  tags = {  
    Name = "prod-database-sg"  
  }  
}
```

What You Cannot Control in Managed Services

On self-managed PostgreSQL, you had `pg_hba.conf` for host-based authentication, could bind to specific interfaces, and could install host-level firewalls. With RDS:

- You cannot access the operating system or configure `pg_hba.conf` directly
- You cannot install host-based firewall rules
- You cannot bind to a specific IP address — RDS assigns its own private IP within your subnet
- The RDS endpoint DNS name resolves to different IPs during failover — never hardcode IPs in connection strings

This last point is where on-prem DBAs get burned. Always use the CNAME endpoint, never the underlying IP address.

32.2 Private Endpoints and Private Link: Eliminating Public Exposure

The default configuration for many cloud services — including Azure SQL — allows public connectivity. Turning it off and routing all traffic through private endpoints is non-negotiable for production database workloads.

AWS PrivateLink and RDS

For RDS and Aurora, the database instance itself sits in your private subnet and never has a public IP if you configure it correctly (`PubliclyAccessible: false`). But connectivity from other AWS services — Lambda, ECS tasks, other VPCs — requires either VPC peering, AWS Transit Gateway, or VPC endpoints for the AWS APIs your database tooling uses.

For Aurora Serverless v2 and RDS Proxy, you also need VPC endpoints for Secrets Manager if you are rotating credentials:

```
# Verify RDS instance is NOT publicly accessible - this should be false in production
aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-01 \
  --query
  ↪ 'DBInstances[0].{PubliclyAccessible:PubliclyAccessible,Endpoint:Endpoint,VpcId:DBSubnetGroup.VpcId}'

# Create VPC endpoint for Secrets Manager - required for RDS Proxy credential rotation
# without routing through the public internet
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-0a1b2c3d4e5f67890 \
  --service-name com.amazonaws.us-east-1.secretsmanager \
  --vpc-endpoint-type Interface \
  --subnet-ids subnet-0a1b2c3d4e5f67890 subnet-0f9e8d7c6b5a43210 \
  --security-group-ids sg-0a1b2c3d4e5f67890 \
  --private-dns-enabled \
  --tag-specifications 'ResourceType=vpc-endpoint,Tags=[{Key=Name,Value=secretsmanager-endpoint}]'
```

Azure SQL Private Endpoint Configuration

Azure SQL has both a public endpoint (disabled by requirement in secure environments) and supports Private Link, which creates a private IP in your VNet that maps to the Azure SQL managed instance or logical server. The Azure CLI workflow:

```
# Step 1: Disable public access on Azure SQL Server
az sql server update \
  --name prod-sql-server \
  --resource-group prod-rg \
  --enable-public-network false

# Step 2: Create private endpoint in your VNet
az network private-endpoint create \
  --name prod-sql-private-endpoint \
  --resource-group prod-rg \
  --vnet-name prod-vnet \
  --subnet prod-db-subnet \
  --private-connection-resource-id \
  ↪ /subscriptions/{subscription-id}/resourceGroups/prod-rg/providers/Microsoft.Sql/servers/prod-sql-server
  ↪ \
  --group-id sqlServer \
  --connection-name prod-sql-private-link

# Step 3: Create private DNS zone for Azure SQL
az network private-dns zone create \
  --resource-group prod-rg \
  --name "privatelink.database.windows.net"

# Step 4: Link DNS zone to your VNet
az network private-dns link vnet create \
  --resource-group prod-rg \
  --zone-name "privatelink.database.windows.net" \
  --name prod-vnet-dns-link \
  --virtual-network prod-vnet \
  --registration-enabled false

# Step 5: Create DNS record for the private endpoint
```

```
az network private-endpoint dns-zone-group create \  
  --resource-group prod-rg \  
  --endpoint-name prod-sql-private-endpoint \  
  --name sql-dns-zone-group \  
  --private-dns-zone "privatelink.database.windows.net" \  
  --zone-name sql
```

After completing this sequence, your application connects to `prod-sql-server.database.windows.net` and the private DNS zone resolves it to the private IP address in your VNet — not the public Azure SQL endpoint. Verify this from within the VNet:

```
# From a VM in the VNet - should resolve to a 10.x.x.x private IP  
nslookup prod-sql-server.database.windows.net  
# Expected: Non-authoritative answer showing a 10.x.x.x address  
# If it shows a public Azure IP, your DNS zone or link is misconfigured
```

Google Cloud Private Service Connect for Cloud SQL

Google Cloud SQL uses Private Service Connect (PSC) or older VPC peering. PSC is the current recommended approach and creates a forwarding rule with a private IP in your VPC:

```
# Enable Private Service Connect on Cloud SQL instance  
gcloud sql instances patch prod-postgres-instance \  
  --project=my-project \  
  --enable-private-service-connect \  
  --allowed-psc-projects=consumer-project-id  
  
# Get the PSC service attachment URI from the instance  
gcloud sql instances describe prod-postgres-instance \  
  --project=my-project \  
  --format='value(settings.ipConfiguration.pscConfig.serviceAttachmentLink)'  
  
# Create a forwarding rule pointing to the PSC attachment  
gcloud compute forwarding-rules create prod-cloudsql-psc \  
  --region=us-central1 \  
  --network=prod-vpc \  
  --address=prod-cloudsql-private-ip \  
  --target-service-attachment=|  
  ↪ projects/PRODUCER_PROJECT/regions/us-central1/serviceAttachments/ATTACHMENT_NAME \  
  --project=my-project  
  
# Create DNS entry so applications use a consistent hostname  
gcloud dns record-sets create prod-postgres.internal.example.com. \  
  --rrdatas=10.0.32.15 \  
  --type=A \  
  --ttl=300 \  
  --zone=internal-zone \  
  --project=my-project
```

The Common Gotcha: DNS Resolution

Private endpoints are useless if your application's DNS resolver bypasses the private zone. This happens most frequently when:

1. Applications run in a different VPC without VPC peering and shared DNS resolution
2. Containers use their own DNS servers instead of the VPC resolver
3. On-premises systems connecting over VPN or Direct Connect use corporate DNS that does not forward to the cloud private zones

Test DNS resolution from every network segment that needs database access, not just from one representative host.

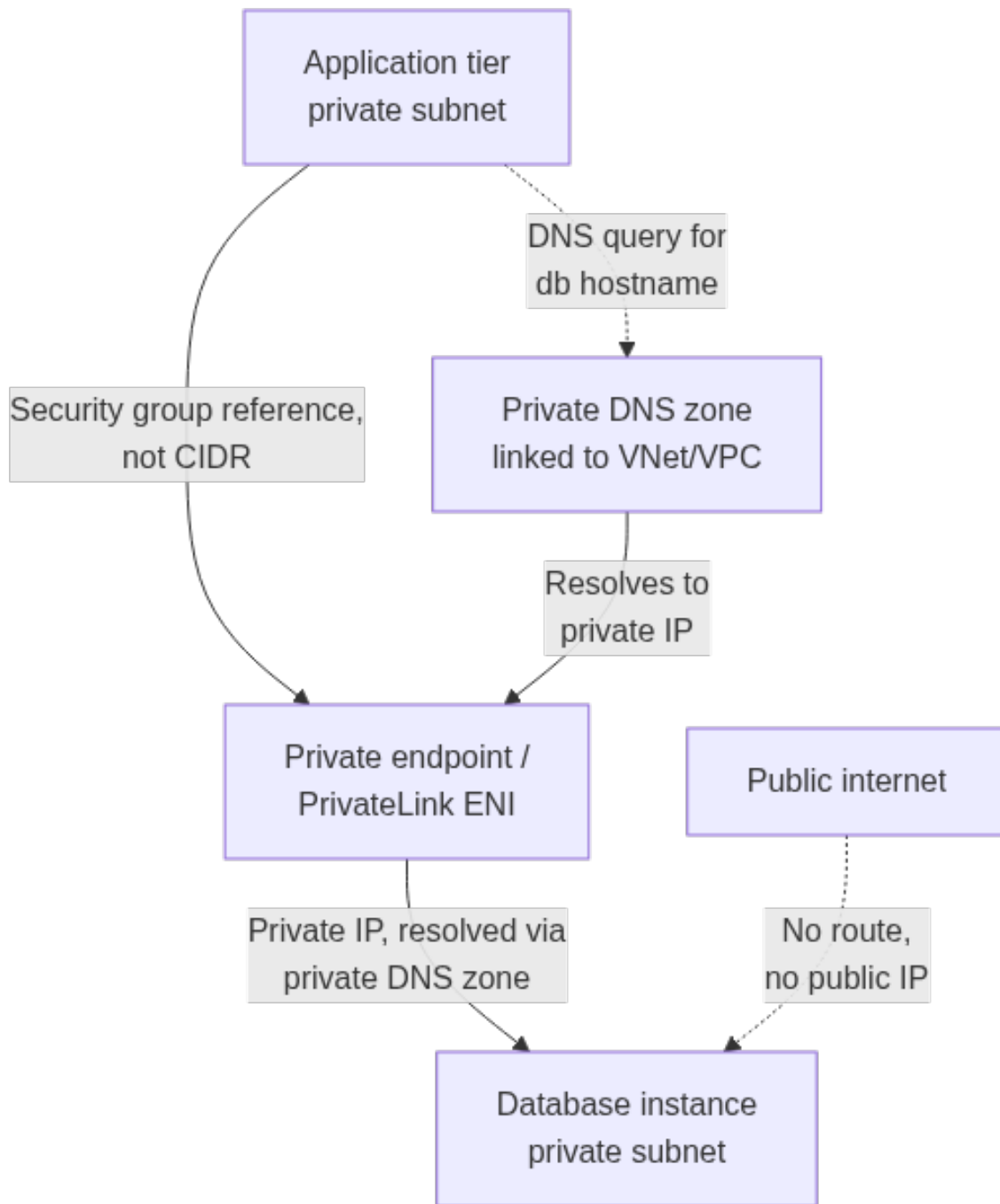


Figure 23: Traffic never leaves the VPC/VNet: the private DNS zone is what keeps applications from silently falling back to the public endpoint.

32.3 Security Groups, Firewall Rules, and the Principle of Least Privilege

Think of security groups as the evolved equivalent of iptables rules — but managed, stateful, and attached to resources rather than hosts. For databases, the configuration philosophy is the same as on-prem: explicit allow, implicit deny, minimum necessary access.

Security Group Design for Multi-Tier Applications

The correct pattern chains security groups by reference, not by CIDR range. This means the database security group allows traffic from the application security group, not from 10.0.0.0/8. When auto-scaling adds application instances, they inherit the application security group automatically and immediately get database access — no rule changes required.

```
# Audit existing RDS security group rules - look for 0.0.0.0/0 which should never appear
aws ec2 describe-security-groups \
  --group-ids sg-0a1b2c3d4e5f67890 \
  --query 'SecurityGroups[0].IpPermissions[*].{
    Port:FromPort,
    Protocol:IpProtocol,
    Sources:IpRanges[*].CidrIp,
    SGSources:UserIdGroupPairs[*].GroupId
  }' \
  --output table

# Add RDS Proxy security group as an allowed source to the database SG
# This is required when you introduce RDS Proxy in front of your instances
aws ec2 authorize-security-group-ingress \
  --group-id sg-database-sg-id \
  --protocol tcp \
  --port 5432 \
  --source-group sg-rdsproxy-sg-id

# Verify: RDS Proxy needs outbound to the database, database needs inbound from proxy
# Application needs outbound to proxy, proxy needs inbound from application
# These are four distinct security group rules, not two
```

Azure SQL Firewall Rules and Service Endpoints

Azure SQL has two distinct layers: server-level firewall rules (IP-based, older model) and VNet service endpoints or Private Endpoints (the correct production model). Service endpoints are a middle-ground — they restrict access to specific VNet subnets but traffic still traverses the Azure backbone routing to the public endpoint. Private Endpoints are stronger: traffic stays entirely within your VNet.

```
# Remove overly permissive firewall rules from Azure SQL
# First, list all existing rules
az sql server firewall-rule list \
  --server prod-sql-server \
  --resource-group prod-rg \
  --output table

# Delete any rule that allows 0.0.0.0 to 255.255.255.255 ("Allow Azure Services")
# This rule allows ALL Azure services, including other tenants' services
az sql server firewall-rule delete \
  --server prod-sql-server \
  --resource-group prod-rg \
  --name "AllowAllWindowsAzureIps"

# Add VNet rule for your application subnet (service endpoint approach)
az sql server vnet-rule create \
  --server prod-sql-server \
  --resource-group prod-rg \
  --name allow-app-subnet \
```

```
--vnet-name prod-vnet \  
--subnet app-subnet
```

Note that “Allow Azure Services” firewall rule in Azure SQL is frequently misunderstood. It does not mean “allow only my services” — it means allow any traffic originating from any Azure datacenter IP, including other customers’ Azure services. Remove it.

GCP Firewall Rules for Cloud SQL

Cloud SQL with Private Service Connect or VPC peering does not use GCP firewall rules directly for database port access — the peering or PSC handles routing. However, you still need firewall rules to control which compute resources can reach the database:

```
# GCP Firewall rule: allow PostgreSQL from application instances  
# Tagged approach - apply 'db-client' tag to instances that need database access  
gcloud compute firewall-rules create allow-postgres-from-app \  
  --project=my-project \  
  --network=prod-vpc \  
  --direction=INGRESS \  
  --priority=1000 \  
  --action=ALLOW \  
  --rules=tcp:5432 \  
  --source-tags=db-client \  
  --target-tags=db-server \  
  --description="Allow PostgreSQL from tagged application instances"  
  
# For Cloud SQL specifically, also allow the Cloud SQL proxy if used  
gcloud compute firewall-rules create allow-cloudsql-proxy \  
  --project=my-project \  
  --network=prod-vpc \  
  --direction=EGRESS \  
  --priority=1000 \  
  --action=ALLOW \  
  --rules=tcp:3307 \  
  --destination-ranges=35.191.0.0/16,130.211.0.0/22 \  
  --description="Allow outbound to Cloud SQL proxy API"
```

32.4 Cross-VPC Connectivity: Peering, Transit Gateway, and PrivateLink

Production environments rarely have a single VPC. You will have dedicated VPCs for different applications, different environments (dev, staging, prod), or different business units. Getting databases to talk across these boundaries without exposing them to the internet requires deliberate architecture.

AWS VPC Peering vs. Transit Gateway

VPC peering connects two VPCs directly. It does not support transitive routing — if VPC A peers with VPC B, and VPC B peers with VPC C, VPC A cannot reach VPC C through VPC B. For database access patterns where multiple application VPCs need to reach a central database VPC, this forces a hub-and-spoke design with Transit Gateway:

```
# Terraform: Transit Gateway setup for central database VPC access  
resource "aws_ec2_transit_gateway" "main" {  
  description           = "Central TGW for database connectivity"  
  default_route_table_association = "disable"  
  default_route_table_propagation = "disable"  
  dns_support           = "enable"  
  vpn_ecmp_support      = "enable"  
}
```

```
tags = {
  Name = "prod-tgw"
}
}

# Attach the database VPC to the Transit Gateway
resource "aws_ec2_transit_gateway_vpc_attachment" "database_vpc" {
  transit_gateway_id = aws_ec2_transit_gateway.main.id
  vpc_id             = aws_vpc.database.id
  subnet_ids        = [aws_subnet.database_az1.id, aws_subnet.database_az2.id]

  dns_support      = "enable"
  ipv6_support     = "disable"

  tags = {
    Name = "db-vpc-attachment"
  }
}

# Attach an application VPC
resource "aws_ec2_transit_gateway_vpc_attachment" "app_vpc" {
  transit_gateway_id = aws_ec2_transit_gateway.main.id
  vpc_id             = aws_vpc.application.id
  subnet_ids        = [aws_subnet.app_az1.id, aws_subnet.app_az2.id]

  tags = {
    Name = "app-vpc-attachment"
  }
}

# Route table: application VPC -> database VPC only
# Database traffic is 10.0.0.0/16, restrict to only that CIDR
resource "aws_ec2_transit_gateway_route_table" "app_to_db" {
  transit_gateway_id = aws_ec2_transit_gateway.main.id

  tags = {
    Name = "app-to-db-only"
  }
}

resource "aws_ec2_transit_gateway_route" "app_to_db" {
  destination_cidr_block = "10.0.0.0/16" # Database VPC CIDR
  transit_gateway_attachment_id = aws_ec2_transit_gateway_vpc_attachment.database_vpc.id
  transit_gateway_route_table_id = aws_ec2_transit_gateway_route_table.app_to_db.id
}
```

Connecting On-Premises to Cloud Databases

This is the most common architecture during migration phases: on-premises applications need to reach cloud databases, or cloud databases need to reach on-premises systems for DMS replication. The options in order of preference:

1. **AWS Direct Connect / Azure ExpressRoute / GCP Dedicated Interconnect** — dedicated physical connection, predictable latency (typically 1–5ms for co-located facilities), no internet exposure. Required for any data volume above a few hundred MB/day and for compliance environments
2. **Site-to-Site VPN** — encrypted tunnel over internet, variable latency (20–100ms typical), acceptable for lower-volume migration workloads
3. **Public internet with SSL/TLS** — last resort, acceptable only for development

For Direct Connect with RDS, you still need a Virtual Private Gateway attached to your VPC and route propagation configured. The database endpoint resolves to a private IP that routes over the Direct Connect circuit:

```
# Verify that RDS endpoint resolves to a private IP from on-premises
# (run this from an on-premises host after Direct Connect + BGP route propagation)
dig +short prod-postgres-01.cluster-xyz.us-east-1.rds.amazonaws.com
# Should return: 10.x.x.x (private IP in your VPC subnet)
# If returns: 52.x.x.x or 34.x.x.x (public AWS IP), your routing is misconfigured

# Check route propagation from Virtual Private Gateway
aws ec2 describe-route-tables \
  --filters Name=vpc-id,Values=vpc-0a1b2c3d4e5f67890 \
  --query
  ↪ 'RouteTables[*].Routes[?GatewayId!=`local`].{Destination:DestinationCidrBlock,Target:GatewayId,State:State}'
```

Azure VNet Peering for SQL Managed Instance

Azure SQL Managed Instance (not the logical server) requires its own dedicated subnet within a VNet and has strict requirements: the subnet must be delegated to `Microsoft.Sql/managedInstances`, must have a route table attached, and must have specific NSG rules. Peering from application VNets to the Managed Instance VNet requires the peering to allow gateway transit:

```
# Create VNet peering from application VNet to SQL MI VNet
az network vnet peering create \
  --name app-to-sqlmi-peering \
  --resource-group prod-rg \
  --vnet-name app-vnet \
  --remote-vnet /subscriptions/{sub-id}/resourceGroups/prod-rg/virtualNetworks/sqlmi-vnet \
  --allow-vnet-access true \
  --allow-forwarded-traffic true

# Create the reverse peering (required - Azure peering is not automatically bidirectional)
az network vnet peering create \
  --name sqlmi-to-app-peering \
  --resource-group prod-rg \
  --vnet-name sqlmi-vnet \
  --remote-vnet /subscriptions/{sub-id}/resourceGroups/prod-rg/virtualNetworks/app-vnet \
  --allow-vnet-access true \
  --allow-forwarded-traffic true

# Verify peering state - both sides must show "Connected" not "Initiated"
az network vnet peering show \
  --name app-to-sqlmi-peering \
  --resource-group prod-rg \
  --vnet-name app-vnet \
  --query peeringState
```

32.5 IAM Authentication vs. Network Authentication: Combining Database and Network Security

Network controls limit which machines can attempt a connection. Database authentication controls which identities succeed when they do. Production security requires both layers, and the cloud adds a third layer: cloud IAM integration with the database authentication system itself.

AWS IAM Database Authentication

RDS and Aurora support IAM authentication as an alternative to password-based logins. Instead of storing a password, the application generates a short-lived authentication token using its AWS IAM identity (an IAM role). The token is valid for 15 minutes and is generated locally using AWS credentials — it never requires a call to an external auth service at connection time, only for token generation.

The workflow:

1. Create a database user mapped to IAM
2. Grant the `rds_iam` role (Aurora PostgreSQL) or equivalent
3. Application uses its EC2 instance profile or ECS task role to generate the token
4. Application connects with the token as the password

```
-- PostgreSQL on RDS/Aurora: create IAM-authenticated user
-- Run this as the master user
CREATE USER app_service WITH LOGIN;
GRANT rds_iam TO app_service;

-- Verify the IAM role association
SELECT rolname, rolsuper, rolinherit
FROM pg_roles
WHERE rolname IN ('rds_iam', 'app_service');

-- SQL Server on RDS: IAM auth uses Windows authentication via Active Directory
-- or certificate-based auth - IAM token auth is PostgreSQL/MySQL-specific
-- For SQL Server, use AD integration instead:
CREATE LOGIN [DOMAIN\app-service-account] FROM WINDOWS;
CREATE USER [DOMAIN\app-service-account] FOR LOGIN [DOMAIN\app-service-account];
ALTER ROLE db_datareader ADD MEMBER [DOMAIN\app-service-account];
```

Generating the token from application code (Python example pattern):

```
import boto3
import psycpg2

def get_rds_auth_token(host, port, db_user, region):
    """Generate a short-lived IAM auth token for RDS."""
    client = boto3.client('rds', region_name=region)
    token = client.generate_db_auth_token(
        DBHostname=host,
        Port=port,
        DBUsername=db_user,
        Region=region
    )
    return token

def connect_with_iam_auth():
    host = "prod-postgres-01.cluster-xyz.us-east-1.rds.amazonaws.com"
    port = 5432
    db_user = "app_service"
    region = "us-east-1"

    token = get_rds_auth_token(host, port, db_user, region)

    conn = psycpg2.connect(
        host=host,
        port=port,
        database="appdb",
        user=db_user,
        password=token,
        sslmode="verify-full",
        sslrootcert="/etc/ssl/certs/rds-ca-bundle.pem"
    )
    return conn
```

Note the `sslmode="verify-full"` — IAM authentication requires SSL. Without it, the token cannot be validated. This is a hard requirement, not a suggestion, and it means you must distribute the RDS CA certificate bundle to your application hosts.

AWS IAM Auth: Performance Considerations

Token generation makes an API call to `rds.amazonaws.com`. If your application opens many short-lived connections (poor connection pooling, Lambda without RDS Proxy), this adds latency. The

solution: use RDS Proxy with IAM auth. The proxy handles token generation and maintains a persistent connection pool. Applications authenticate to the proxy using IAM, and the proxy handles the database connection:

```
# Create RDS Proxy with IAM authentication
aws rds create-db-proxy \
  --db-proxy-name prod-postgres-proxy \
  --engine-family POSTGRESQL \
  --auth '[{
    "AuthScheme": "SECRETS",
    "SecretArn": "arn:aws:secretsmanager:us-east-1:123456789012:secret:prod/postgres/credentials",
    "IAMAuth": "REQUIRED"
  }]' \
  --role-arn arn:aws:iam::123456789012:role/RDSProxyRole \
  --vpc-subnet-ids subnet-0a1b2c3d4e5f67890 subnet-0f9e8d7c6b5a43210 \
  --vpc-security-group-ids sg-rdsproxy-sg-id \
  --require-tls

# Register the RDS cluster with the proxy
aws rds register-db-proxy-targets \
  --db-proxy-name prod-postgres-proxy \
  --db-cluster-identifiers prod-aurora-cluster
```

Azure SQL and Azure Active Directory Authentication

Azure SQL supports three authentication modes: SQL authentication (username/password stored in the database), Azure AD authentication (identities managed by Azure AD), and mixed mode. In production, enforce Azure AD-only:

```
# Set Azure AD admin on SQL Server - this enables AAD auth
az sql server ad-admin create \
  --server prod-sql-server \
  --resource-group prod-rg \
  --display-name "DBA-Team" \
  --object-id $(az ad group show --group "DBA-Team" --query id -o tsv)

# Disable SQL (password-based) authentication - Azure AD only
az sql server update \
  --name prod-sql-server \
  --resource-group prod-rg \
  --set administratorLogin="" \
  --minimal-tls-version 1.2
```

Creating contained database users mapped to Azure AD identities (managed identities for applications):

```
-- Azure SQL: create user for a managed identity (application service principal)
-- Run as Azure AD admin
CREATE USER [my-app-managed-identity] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [my-app-managed-identity];
ALTER ROLE db_datawriter ADD MEMBER [my-app-managed-identity];

-- For a service principal (app registration):
CREATE USER [my-app-registration] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [my-app-registration];

-- Verify contained users
SELECT name, type_desc, authentication_type_desc
FROM sys.database_principals
WHERE type IN ('E', 'X') -- E=external user, X=external group
ORDER BY name;
```

GCP Cloud SQL and IAM Database Authentication

Cloud SQL PostgreSQL supports IAM database authentication where the Google Cloud IAM identity (service account) maps to a PostgreSQL role:

```
# Enable IAM database authentication on Cloud SQL instance
gcloud sql instances patch prod-postgres-instance \
  --project=my-project \
  --database-flags=cloudsql.iam_authentication=on

# Add IAM service account as a Cloud SQL user
gcloud sql users create app-service@my-project.iam.gserviceaccount.com \
  --instance=prod-postgres-instance \
  --project=my-project \
  --type=CLOUD_IAM_SERVICE_ACCOUNT

-- Cloud SQL PostgreSQL: the IAM user is created automatically when added via gcloud
-- Grant appropriate privileges
GRANT CONNECT ON DATABASE appdb TO "app-service@my-project.iam";
GRANT USAGE ON SCHEMA public TO "app-service@my-project.iam";
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public
  TO "app-service@my-project.iam";

-- For future tables:
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES
  TO "app-service@my-project.iam";

-- Verify the IAM user in pg_roles
SELECT rolname, rolcanlogin
FROM pg_roles
WHERE rolname LIKE '%iam%' OR rolname LIKE '%gserviceaccount%';
```

32.6 TLS Enforcement and Certificate Management

Network isolation prevents unwanted access. TLS encryption protects the data in transit from eavesdropping on the connections that are allowed. For managed cloud databases, you should be enforcing TLS everywhere and verifying certificates, not just encrypting.

Enforcing TLS on AWS RDS

RDS PostgreSQL and MySQL use parameter group settings to enforce SSL. Without enforcement, clients can connect unencrypted even when SSL is available:

```
# Create a parameter group that enforces SSL for PostgreSQL
aws rds create-db-cluster-parameter-group \
  --db-cluster-parameter-group-name prod-postgres-enforced-ssl \
  --db-parameter-group-family aurora-postgresql15 \
  --description "Forces all connections to use SSL"

# Set rds.force_ssl = 1 (1 = reject non-SSL connections)
aws rds modify-db-cluster-parameter-group \
  --db-cluster-parameter-group-name prod-postgres-enforced-ssl \
  --parameters ParameterName=rds.force_ssl,ParameterValue=1,ApplyMethod=immediate

# Apply to your cluster
aws rds modify-db-cluster \
  --db-cluster-identifier prod-aurora-cluster \
  --db-cluster-parameter-group-name prod-postgres-enforced-ssl \
  --apply-immediately

-- PostgreSQL: verify SSL is required - check current connections
SELECT pid, usename, application_name, client_addr, ssl, ssl_version, ssl_cipher
FROM pg_stat_ssl
JOIN pg_stat_activity USING (pid)
WHERE backend_type = 'client backend'
ORDER BY ssl, client_addr;

-- SQL Server on RDS: check SSL/TLS for existing connections
```

```
SELECT session_id, encrypt_option, net_transport, auth_scheme
FROM sys.dm_exec_connections
ORDER BY encrypt_option;
```

Certificate Verification: The Difference Between Encryption and Security

Using SSL without certificate verification protects against passive eavesdropping but not against man-in-the-middle attacks. Applications should verify the server certificate using the RDS Certificate Authority bundle. AWS periodically rotates CA certificates (the `rds-ca-rsa2048-g1` to `rds-ca-rsa4096-g1` rotation is an example), and failing to update the CA bundle in applications causes connection failures:

```
# Download current RDS CA bundle
curl -o /etc/ssl/certs/global-bundle.pem \
  https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem

# Verify what CA your RDS instance is currently using
aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-01 \
  --query 'DBInstances[0].CACertificateIdentifier'

# Check when the certificate expires
aws rds describe-certificates \
  --certificate-identifier rds-ca-rsa2048-g1 \
  --query 'Certificates[0].{ValidTill:ValidTill,CustomOverride:CustomOverride}'

# List all available CA certificates
aws rds describe-certificates \
  --query 'Certificates[*].{ID:CertificateIdentifier,ValidFrom:ValidFrom,ValidTill:ValidTill}' \
  --output table
```

For Azure SQL, the DigiCert root CA is used. The connection string must include `Encrypt=True;TrustServerCertificate=False;` — if `TrustServerCertificate=True`, certificate verification is bypassed, which is acceptable only in development:

```
# Correct production Azure SQL connection string (ADO.NET style)
Server=tcp:prod-sql-server.database.windows.net,1433;
Initial Catalog=appdb;
Authentication=Active Directory Managed Identity;
Encrypt=True;
TrustServerCertificate=False;
Connection Timeout=30;

# Incorrect - TrustServerCertificate=True means no cert verification
Server=tcp:prod-sql-server.database.windows.net,1433;
Initial Catalog=appdb;
User ID=sqladmin;Password=...;
Encrypt=True;
TrustServerCertificate=True; # DO NOT USE IN PRODUCTION
```

Minimum TLS Version Enforcement

TLS 1.0 and 1.1 have known vulnerabilities. Enforce TLS 1.2 as the minimum everywhere:

```
# AWS RDS: enforce minimum TLS via parameter group
# For PostgreSQL, this is controlled by ssl_min_protocol_version
aws rds modify-db-cluster-parameter-group \
  --db-cluster-parameter-group-name prod-postgres-enforced-ssl \
  --parameters ParameterName=ssl_min_protocol_version,ParameterValue=TLSv1.2,ApplyMethod=immediate
```



```
# Azure SQL: set minimum TLS version at server level
az sql server update \
  --name prod-sql-server \
  --resource-group prod-rg \
  --minimal-tls-version 1.2

# GCP Cloud SQL: set minimum TLS version
gcloud sql instances patch prod-postgres-instance \
  --project=my-project \
  --database-flags=ssl_min_protocol_version=TLSv1.2
```

32.7 Auditing Network Access and Detecting Anomalies

Configuration is not complete without visibility. You need logs showing who connected from where, and alerts when connection patterns deviate from baseline.

AWS VPC Flow Logs for Database Traffic

VPC Flow Logs capture connection-level metadata (source IP, destination IP, port, protocol, accept/reject) for all traffic in your VPC. Enabling them on the database subnets gives you a full record of connection attempts:

```
# Enable VPC Flow Logs on database subnets, sending to CloudWatch Logs
aws ec2 create-flow-logs \
  --resource-type Subnet \
  --resource-ids subnet-0a1b2c3d4e5f67890 subnet-0f9e8d7c6b5a43210 \
  --traffic-type ALL \
  --log-destination-type cloud-watch-logs \
  --log-group-name /vpc/database-subnet-flow-logs \
  --deliver-logs-permission-arn arn:aws:iam::123456789012:role/VPCFlowLogsRole \
  --log-format '${version} ${account-id} ${interface-id} ${srcaddr} ${dstaddr} ${srcport} ${dstport} \
  ↳ ${protocol} ${packets} ${bytes} ${start} ${end} ${action} ${log-status}'

# Query flow logs in CloudWatch Insights for rejected connections to PostgreSQL port
# This reveals port-scanning or misconfigured application connectivity
# (CloudWatch Insights query - run in the console or via CLI)
```

```
# CloudWatch Logs Insights query: rejected connections to port 5432
fields @timestamp, srcaddr, dstaddr, srcport, dstport, action
| filter dstport = 5432 and action = "REJECT"
| stats count(*) as attempts by srcaddr
| sort attempts desc
| limit 20
```

RDS Enhanced Monitoring and Connection Tracking

RDS Enhanced Monitoring, distinct from standard CloudWatch metrics, runs a monitoring agent inside the RDS instance and provides per-process metrics at 1-second granularity. For connection tracking, the CloudWatch metric `DatabaseConnections` combined with `RDS.DBConnections` provides aggregate counts. For per-client connection tracking, use the PostgreSQL `pg_stat_activity` view:

```
-- PostgreSQL: connection audit - active connections by client IP and user
SELECT
  client_addr,
  username,
  application_name,
  state,
  COUNT(*) AS connection_count,
```

```

    MAX(EXTRACT(EPOCH FROM (NOW() - state_change))) AS max_idle_seconds
FROM pg_stat_activity
WHERE backend_type = 'client backend'
GROUP BY client_addr, username, application_name, state
ORDER BY connection_count DESC;

-- Identify connections from unexpected source addresses
-- Baseline: connections should come only from known application server IPs
SELECT DISTINCT client_addr, username
FROM pg_stat_activity
WHERE client_addr NOT IN (
    '10.0.1.10', -- app server 1
    '10.0.1.11', -- app server 2
    '10.0.2.5'  -- batch processing host
)
AND backend_type = 'client backend';

-- SQL Server: audit active connections by login and application
SELECT
    login_name,
    host_name,
    program_name,
    client_net_address,
    COUNT(*) AS session_count,
    SUM(CASE WHEN status = 'sleeping' THEN 1 ELSE 0 END) AS idle_sessions
FROM sys.dm_exec_sessions s
JOIN sys.dm_exec_connections c ON s.session_id = c.session_id
WHERE s.is_user_process = 1
GROUP BY login_name, host_name, program_name, client_net_address
ORDER BY session_count DESC;

```

Azure SQL Auditing

Azure SQL has a built-in auditing feature that logs connection events, query executions, and schema changes to Azure Blob Storage, Log Analytics, or Event Hub. Enable it at the server level to cover all databases:

```

# Enable Azure SQL auditing to Log Analytics workspace
az sql server audit-policy update \
  --server prod-sql-server \
  --resource-group prod-rg \
  --state Enabled \
  --log-analytics-target-state Enabled \
  --log-analytics-workspace-resource-id \

↪ /subscriptions/{sub-id}/resourceGroups/prod-rg/providers/Microsoft.OperationalInsights/workspaces/prod-law

# Enable Advanced Threat Protection - detects anomalous connection patterns,
# SQL injection attempts, and access from unusual locations
az sql server threat-policy update \
  --server prod-sql-server \
  --resource-group prod-rg \
  --state Enabled \
  --storage-account prod-audit-storage \
  --email-addresses dba-team@company.com security@company.com \
  --email-account-admins Enabled

```

Querying Azure SQL audit logs from Log Analytics:

```

// KQL: Failed login attempts to Azure SQL in the last 24 hours
AzureDiagnostics
| where ResourceType == "SERVERS/DATABASES"
| where Category == "SQLSecurityAuditEvents"
| where action_name_s == "LOGIN FAILED"
| where TimeGenerated > ago(24h)
| summarize FailedAttempts = count() by client_ip_s, server_instance_name_s, succeeded_s
| order by FailedAttempts desc

// KQL: Connections from unexpected IP ranges

```

```
AzureDiagnostics
| where Category == "SQLSecurityAuditEvents"
| where action_name_s == "LOGIN SUCCEEDED"
| where client_ip_s !startswith "10.0." // adjust to your private CIDR
| project TimeGenerated, client_ip_s, server_principal_name_s, database_principal_name_s
| order by TimeGenerated desc
```

32.8 Network Security Automation and Infrastructure as Code

Manual security group changes are how security incidents happen. Every network configuration change affecting database access should go through code review, be tracked in version control, and be deployed through CI/CD. This section covers the patterns for automating network security configuration.

Detecting Security Group Drift

Configuration drift — when the live environment diverges from what Terraform (or CloudFormation) defines — is the silent enemy of security posture. Terraform plan run against an existing environment will show drift:

```
# Run terraform plan to detect any manual changes made outside Terraform
# Output showing changes = drift that needs investigation
cd infrastructure/network
terraform plan -out=plan.out

# For a more targeted audit, use AWS Config rules
# This rule checks that RDS instances are not publicly accessible
aws configservice get-compliance-details-by-config-rule \
  --config-rule-name rds-instance-public-access-check \
  --compliance-types NON_COMPLIANT \
  --query
↪ 'EvaluationResults[*].{ResourceId:EvaluationResultIdentifier.EvaluationResultQualifier.ResourceId,Annotation:Annotation}'
```

AWS Config Rules for Database Network Security

AWS Config provides managed rules that continuously evaluate your RDS configuration against defined security policies:

```
# Enable key Config rules for database network security
# These rules run continuously and flag non-compliant resources

# Rule 1: RDS must not be publicly accessible
aws configservice put-config-rule \
  --config-rule '{
    "ConfigRuleName": "rds-instance-public-access-check",
    "Source": {
      "Owner": "AWS",
      "SourceIdentifier": "RDS_INSTANCE_PUBLIC_ACCESS_CHECK"
    },
    "Scope": {
      "ComplianceResourceTypes": ["AWS::RDS::DBInstance"]
    }
  }'

# Rule 2: RDS instances must be in a VPC
aws configservice put-config-rule \
  --config-rule '{
    "ConfigRuleName": "rds-instance-deployment-in-vpc",
    "Source": {
      "Owner": "AWS",
      "SourceIdentifier": "RDS_IN_BACKUP_PLAN"
    }
  }'
```

```

}'

# Rule 3: No security group should allow unrestricted inbound access
aws configservice put-config-rule \
--config-rule '{
  "ConfigRuleName": "restricted-common-ports",
  "Source": {
    "Owner": "AWS",
    "SourceIdentifier": "RESTRICTED_INCOMING_TRAFFIC"
  },
  "InputParameters": "{\"blockedPort1\": \"5432\", \"blockedPort2\": \"1433\", \"blockedPort3\": \"3306\"}"
}'

```

Automated Remediation for Security Violations

Detecting a violation is valuable. Automatically remediating it is better for low-risk, high-confidence rules. For an RDS instance found to have `PubliclyAccessible: true`, you can configure AWS Config to invoke a Lambda that immediately corrects it:

```

# Lambda function: auto-remediate RDS public accessibility violations
import boto3
import json

def lambda_handler(event, context):
    """
    Triggered by AWS Config when an RDS instance is found publicly accessible.
    Immediately sets PubliclyAccessible=False and logs the action.
    """
    config_item = event.get('configurationItem', {})
    resource_id = config_item.get('resourceId')
    resource_type = config_item.get('resourceType')

    if resource_type != 'AWS::RDS::DBInstance':
        return {'statusCode': 200, 'body': 'Not an RDS instance, skipping'}

    rds = boto3.client('rds')

    # Check current state before modifying
    response = rds.describe_db_instances(DBInstanceIdentifier=resource_id)
    instance = response['DBInstances'][0]

    if not instance['PubliclyAccessible']:
        return {'statusCode': 200, 'body': f'{resource_id} already not publicly accessible'}

    # Remediate: disable public accessibility
    rds.modify_db_instance(
        DBInstanceIdentifier=resource_id,
        PubliclyAccessible=False,
        ApplyImmediately=True
    )

    # Log to CloudWatch for audit trail
    print(json.dumps({
        'action': 'remediated_public_rds',
        'instance_id': resource_id,
        'previous_state': 'PubliclyAccessible=True',
        'new_state': 'PubliclyAccessible=False',
        'triggered_by': 'AWS Config auto-remediation'
    })))

    return {
        'statusCode': 200,
        'body': f'Successfully remediated {resource_id}: PubliclyAccessible set to False'
    }

```

Terraform Security Group Policy as Code

For teams using Terraform across multiple environments, a shared module that enforces security group structure prevents permissive rules from being introduced:

```
# modules/database-security-group/main.tf
# Reusable module that enforces correct database security group structure

variable "vpc_id" {
  description = "VPC ID where the database security group will be created"
  type        = string
}

variable "allowed_security_group_ids" {
  description = "List of security group IDs allowed to connect to the database"
  type        = list(string)
}

variable "database_port" {
  description = "Database port (5432 for PostgreSQL, 1433 for SQL Server)"
  type        = number
  validation {
    condition   = contains([5432, 1433, 3306, 5439], var.database_port)
    error_message = "Database port must be a known database port (5432, 1433, 3306, 5439)."
  }
}

variable "environment" {
  type = string
}

resource "aws_security_group" "database" {
  name        = "${var.environment}-database-sg"
  description = "Database security group - managed by Terraform, do not modify manually"
  vpc_id      = var.vpc_id

  # No ingress from CIDR ranges - only from specific security groups
  dynamic "ingress" {
    for_each = var.allowed_security_group_ids
    content {
      description      = "Database access from approved security group"
      from_port        = var.database_port
      to_port          = var.database_port
      protocol         = "tcp"
      security_groups  = [ingress.value]
    }
  }

  # Egress: minimal - only what the database actually needs outbound
  # RDS needs HTTPS outbound for IAM auth token validation and enhanced monitoring
  egress {
    description = "HTTPS for IAM auth and AWS APIs"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  lifecycle {
    # Prevent accidental deletion of the security group
    prevent_destroy = true
  }

  tags = {
    Name        = "${var.environment}-database-sg"
    Environment = var.environment
    ManagedBy   = "terraform"
    Security    = "database-tier"
  }
}
```

```
output "security_group_id" {
  value      = aws_security_group.database.id
  description = "Security group ID for the database tier"
}
```

32.9 Common Misconfigurations and How to Find Them

The security incidents that affect cloud databases almost never involve a sophisticated zero-day exploit. They result from misconfigurations that are entirely preventable. Here is a systematic checklist for auditing your database network security.

The Audit Checklist

```
#!/bin/bash
# Database network security audit script - AWS
# Run this periodically or as part of a CI/CD security gate

echo "=== RDS Network Security Audit ==="
echo ""

echo "--- Checking for publicly accessible RDS instances ---"
aws rds describe-db-instances \
  --query 'DBInstances[?PubliclyAccessible==`true`].{ID:DBInstanceIdentifier,Engine:Engine,PubliclyAccessible:PubliclyAccessible}' \
  --output table

echo ""
echo "--- Checking for RDS instances NOT in a VPC ---"
aws rds describe-db-instances \
  --query 'DBInstances[?DBSubnetGroup==null].{ID:DBInstanceIdentifier,Engine:Engine}' \
  --output table

echo ""
echo "--- Checking for security groups with 0.0.0.0/0 on database ports ---"
for port in 5432 1433 3306 5439 27017; do
  echo "Port $port:"
  aws ec2 describe-security-groups \
    --filters "Name=ip-permission.from-port,Values=$port" \
    "Name=ip-permission.to-port,Values=$port" \
    "Name=ip-permission.cidr,Values=0.0.0.0/0" \
    --query 'SecurityGroups[*].{ID:GroupId,Name:GroupName,VPC:VpcId}' \
    --output table
done

echo ""
echo "--- Checking RDS parameter groups for SSL enforcement ---"
aws rds describe-db-cluster-parameter-groups \
  --query 'DBClusterParameterGroups[*].DBClusterParameterGroupName' \
  --output text | tr '\t' '\n' | while read pgname; do
    ssl_setting=$(aws rds describe-db-cluster-parameters \
      --db-cluster-parameter-group-name "$pgname" \
      --query "Parameters[?ParameterName=='rds.force_ssl'].ParameterValue" \
      --output text)
    echo "Parameter group: $pgname | rds.force_ssl = ${ssl_setting:-not_set}"
done

echo ""
echo "--- Checking for RDS snapshots that are publicly accessible ---"
aws rds describe-db-snapshots \
  --query 'DBSnapshots[?Attribute=='restore' && contains(AttributeValues,`all`)].{ID:DBSnapshotIdentifier,Instance:DBInstanceIdentifier}' \
  --output table 2>/dev/null || \
aws rds describe-db-snapshots \
```

```

--include-shared \
--query
↪ 'DBSnapshots[?SnapshotType=='public'].{ID:DBSnapshotIdentifier,Instance:DBInstanceIdentifier}' \
--output table

echo ""
echo "=== Audit complete ==="

```

Publicly Accessible Snapshots

One of the highest-severity misconfigurations that is easy to miss: RDS snapshots can be made public, allowing anyone with an AWS account to copy and restore them. This has been the source of several well-documented data exposures. The audit script above checks for this, but the explicit remediation:

```

# Find and remediate public RDS snapshots
aws rds describe-db-snapshots \
--snapshot-type public \
--query 'DBSnapshots[*].DBSnapshotIdentifier' \
--output text | tr '\t' '\n' | while read snapshot_id; do
echo "Revoking public access from snapshot: $snapshot_id"
aws rds modify-db-snapshot-attribute \
--db-snapshot-identifier "$snapshot_id" \
--attribute-name restore \
--values-to-remove all
done

```

The “Allow Azure Services” Trap

Already mentioned in section 32.3, but worth repeating because it appears in so many production Azure SQL deployments: the “Allow access to Azure services” toggle in the Azure SQL firewall adds a rule from 0.0.0.0 to 0.0.0.0 that permits any traffic from Azure datacenters. This is often added during initial setup to make connectivity easy, then forgotten. Audit for it:

```

# Audit Azure SQL firewall rules - look for the overly permissive rule
az sql server firewall-rule list \
--server prod-sql-server \
--resource-group prod-rg \
--query "[?startIpAddress=='0.0.0.0' ||
↪ endIpAddress=='255.255.255.255'].{Name:name,Start:startIpAddress,End:endIpAddress}" \
--output table

# If you see "AllowAllWindowsAzureIps" or 0.0.0.0/255.255.255.255 entries, remove them
az sql server firewall-rule list \
--server prod-sql-server \
--resource-group prod-rg \
--output table

```

RDS Subnet Group in Public Subnets

A subnet group can be created with public subnets (subnets with a route to an internet gateway). If `PubliclyAccessible` is also set to true, the database gets a public IP. Audit for subnet groups containing public subnets:

```

# Check if subnets in DB subnet groups have routes to internet gateway
# First, get all subnet IDs used by DB subnet groups
aws rds describe-db-subnet-groups \
--query 'DBSubnetGroups[*].Subnets[*].SubnetIdentifier' \
--output text | tr '\t' '\n' | sort -u | while read subnet_id; do
# Check if subnet has a route to an internet gateway (0.0.0.0/0 -> igw-*)
igw_route=$(aws ec2 describe-route-tables \
--filters "Name=association.subnet-id,Values=$subnet_id" \
--query "RouteTables[*].Routes[?GatewayId!=null && starts_with(GatewayId,'igw-')].GatewayId" \
--output text)

```

```
if [ -n "$igw_route" ]; then
  echo "WARNING: Subnet $subnet_id has route to Internet Gateway: $igw_route"
  echo "  This subnet should NOT be used for database instances"
fi
done
```

Key Takeaways

- **Network isolation is the first line of defense:** Place all managed database instances in private subnets with no route to an internet gateway, use security groups chained by reference rather than CIDR range, and explicitly audit for `PubliclyAccessible: true` and public snapshot exposure on a scheduled basis — these misconfigurations are the most common source of cloud database exposure.
 - **Private endpoints require correct DNS to be effective:** Configuring an Azure Private Endpoint, AWS VPC endpoint, or GCP Private Service Connect attachment does nothing for hosts whose DNS resolver does not consult the private DNS zone. Verify name resolution from every network segment — application servers, Lambda functions, containers, and on-premises hosts connected via VPN or Direct Connect — not just from a single representative host.
 - **Cloud IAM authentication combines network and identity security:** AWS IAM auth tokens, Azure AD managed identity logins, and GCP Cloud SQL IAM authentication eliminate long-lived passwords from application configurations. Pairing IAM auth with RDS Proxy or Azure AD-only mode removes the credential rotation problem entirely and makes the cloud IAM audit trail your authoritative record of who connected and when.
 - **TLS enforcement means both encryption and certificate verification:** Setting `rds.force_ssl=1` or `Encrypt=True` encrypts the wire but does not prevent a man-in-the-middle attack if the application uses `TrustServerCertificate=True` or skips `sslmode=verify-full`. Distribute CA bundles to application hosts, pin to a minimum TLS version of 1.2 at the server level across all three clouds, and track CA certificate expiration dates in your change calendar.
 - **Automate detection and remediation of network drift:** Manual security group changes and portal-level firewall modifications that bypass Terraform are how production security posture silently degrades. Use AWS Config managed rules, Azure Policy, or GCP Organization Policies to continuously evaluate compliance, pipe violations into your alerting system, and — for high-confidence rules like RDS public accessibility — trigger automated remediation so the window between misconfiguration and correction is measured in minutes rather than audit cycles.
-

Chapter 33: Database Migration to Cloud: Practical Runbook

Migration is where cloud theory meets operational reality. You have spent months planning your cloud architecture, selecting instance types, designing VPC topology, and writing Terraform modules. Now someone hands you a production Oracle 19c database, a legacy SQL Server 2012 instance, or a PostgreSQL 14 cluster running on bare metal, and says: get it to cloud without losing data and without a multi-hour outage. This chapter is the runbook for that moment. It covers the full migration lifecycle from assessment through cutover — using AWS DMS, the Schema Conversion Tool, Azure Database Migration Service, and GCP Database Migration Service — with the specific commands, gotchas, and production patterns that experienced DBAs need when the pressure is on.

33.1 Migration Assessment: Knowing What You’re Moving Before You Move It

The single most common migration failure mode is discovering surprises during execution rather than during planning. A migration that looks like a simple database lift-and-shift often hides Oracle DBMS packages, SQL Server Agent jobs, linked server dependencies, or stored procedures that reference system tables unavailable in managed services. Your assessment phase exists to eliminate those surprises.

Inventory What Managed Services Cannot Replicate

Before touching a migration tool, build an explicit incompatibility list. Managed RDS, Azure SQL, and Cloud SQL impose hard restrictions that self-managed databases do not:

- RDS PostgreSQL: no `pg_hba.conf` direct edit, no superuser, no OS-level extensions not on the approved list, no `wal_level = logical` on read replicas by default
- Azure SQL (PaaS): no SQL Agent with full capability (use Elastic Jobs instead), no linked servers to on-prem without Azure Arc, no `xp_cmdshell`, no CLR assemblies without explicit opt-in
- Cloud SQL PostgreSQL: no superuser, no `pg_basebackup` direct access, limited extension list per engine version
- Aurora PostgreSQL: `rds_superuser` role replaces `postgres` superuser; `aurora_replica_status()` replaces direct `pg_stat_replication` in some contexts

AWS provides the Schema Conversion Tool (SCT) for automated incompatibility scanning. Run

it even for homogeneous migrations — PostgreSQL to Aurora PostgreSQL — because it catches version-specific function differences.

```
# Install AWS SCT CLI wrapper and run an assessment report
# SCT itself is a GUI tool, but you can script it via its project files
# First, install SCT from: https://docs.aws.amazon.com/SchemaConversionTool/

# Generate an assessment report using SCT command line
java -jar aws-schema-conversion-tool.jar \
  --source-engine postgresql \
  --source-host onprem-pg.internal \
  --source-port 5432 \
  --source-user migration_user \
  --source-password "${SOURCE_PW}" \
  --source-database prod_db \
  --target-engine aurora-postgresql \
  --report-file /reports/assessment-$(date +%Y%m%d).pdf \
  --action GenerateAssessmentReport

# For SQL Server to Aurora PostgreSQL migrations, run conversion
java -jar aws-schema-conversion-tool.jar \
  --source-engine sqlserver \
  --source-host onprem-sql.internal \
  --source-port 1433 \
  --source-user sct_user \
  --source-password "${SOURCE_PW}" \
  --source-database AdventureWorks \
  --target-engine aurora-postgresql \
  --report-file /reports/sqlserver-pg-assessment.pdf \
  --action GenerateAssessmentReport
```

Gather Baseline Metrics Before Migration

You cannot validate a successful migration without knowing what normal looks like on the source. Capture these metrics at least two weeks before cutover:

Metric	Why It Matters
Peak TPS and QPS	Determines target instance class
95th percentile query latency	Post-migration baseline for regression detection
Replication lag (if HA)	Sets your DMS replication lag acceptance threshold
Database size + growth rate per week	Determines migration window length and storage provisioning
Active connections at peak	Informs RDS Proxy or Azure connection pool sizing
Long-running transactions (> 60 seconds)	DMS and logical replication will struggle with these

```
-- Run on source PostgreSQL to gather migration sizing data
-- Capture database sizes
SELECT
  datname AS database_name,
  pg_size_pretty(pg_database_size(datname)) AS size,
  pg_database_size(datname) AS size_bytes
FROM pg_database
WHERE datistemplate = FALSE
ORDER BY size_bytes DESC;

-- Identify tables that will cause DMS problems: no primary key
SELECT
  schemaname,
```

```

    tablename,
    pg_size_pretty(pg_total_relation_size(schemaname || '.' || tablename)) AS total_size
FROM pg_tables t
WHERE NOT EXISTS (
    SELECT 1 FROM pg_constraint c
    JOIN pg_class cl ON c.conrelid = cl.oid
    JOIN pg_namespace n ON cl.relnamespace = n.oid
    WHERE c.contype = 'p'
        AND n.nspname = t.schemaname
        AND cl.relname = t.tablename
)
AND schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_total_relation_size(schemaname || '.' || tablename) DESC;

-- Find objects with dependencies that may not migrate cleanly
SELECT
    dependent_ns.nspname AS dependent_schema,
    dependent_view.relname AS dependent_view,
    source_ns.nspname AS source_schema,
    source_table.relname AS source_table
FROM pg_depend
JOIN pg_rewrite ON pg_depend.objid = pg_rewrite.oid
JOIN pg_class AS dependent_view ON pg_rewrite.ev_class = dependent_view.oid
JOIN pg_class AS source_table ON pg_depend.refobjid = source_table.oid
JOIN pg_namespace dependent_ns ON dependent_ns.oid = dependent_view.relnamespace
JOIN pg_namespace source_ns ON source_ns.oid = source_table.relnamespace
WHERE source_table.relkind = 'r'
AND dependent_view.relname != source_table.relname
ORDER BY 1, 2;

```

Migration Strategy Selection

Match your strategy to your downtime budget:

Strategy	Downtime	Complexity	Use Case
Dump and restore	Hours to days	Low	Small databases < 50 GB, scheduled maintenance
AWS DMS full load + CDC	Minutes	Medium	Databases up to several TB with minimal downtime requirement
Logical replication (pg_logical / pglogical)	Minutes	Medium-High	PostgreSQL to PostgreSQL with full control
Physical snapshot + DMS CDC	Minutes	High	Large databases > 500 GB where full DMS load is too slow
AWS DataSync + DMS	Minutes	High	Very large databases with NFS/SFTP accessible backups

For most production migrations, the physical snapshot seeding approach is optimal at scale: restore a backup to the target, then use DMS or logical replication for CDC to catch up the delta, then cut over.

33.2 AWS DMS: Full Load and Change Data Capture

AWS Database Migration Service handles the replication engine so you do not have to manage Debezium, pglogical, or custom trigger-based CDC infrastructure. The trade-off is that DMS runs on managed replication instances that you cannot SSH into, and its behavior with edge cases — LOB columns, tables without primary keys, DDL changes mid-migration — requires explicit configuration.

Provision the DMS Infrastructure with Terraform

```
# terraform/dms-migration.tf
# Replication instance - this is the compute that runs DMS tasks
resource "aws_dms_replication_instance" "migration" {
  replication_instance_id   = "prod-migration-ri"
  replication_instance_class = "dms.r6i.2xlarge" # 8 vCPU, 64 GB - suitable for most production loads
  allocated_storage         = 500                # GB - DMS uses this for LOB caching and logs
  engine_version            = "3.5.2"
  multi_az                  = false               # Single AZ is fine during migration; this is not HA
  ↪ production
  publicly_accessible       = false
  vpc_security_group_ids    = [aws_security_group.dms_sg.id]
  replication_subnet_group_id = aws_dms_replication_subnet_group.migration.id

  tags = {
    Environment = "migration"
    Project     = "db-to-cloud"
  }
}

# Source endpoint - on-premises PostgreSQL
resource "aws_dms_endpoint" "source_pg" {
  endpoint_id   = "onprem-postgres-source"
  endpoint_type = "source"
  engine_name   = "postgres"

  server_name = "10.0.1.50" # Internal IP of source DB, reachable via Direct Connect or VPN
  port        = 5432
  database_name = "prod_db"
  username     = "dms_replication_user"
  password     = var.source_db_password # Store in AWS Secrets Manager, reference here

  extra_connection_attributes = join(";", [
    "pluginName=pglogical",          # Use pglogical plugin for CDC
    "heartbeatEnable=true",
    "heartbeatFrequency=5",
    "heartbeatSchema=dms_heartbeat", # Schema must exist on source
    "captureDDLs=false",            # Disable DDL capture - apply schema changes manually
    "executeTimeout=120"
  ])

  ssl_mode = "require"
}

# Target endpoint - Aurora PostgreSQL cluster
resource "aws_dms_endpoint" "target_aurora" {
  endpoint_id   = "aurora-pg-target"
  endpoint_type = "target"
  engine_name   = "aurora-postgresql"

  server_name = aws_rds_cluster.aurora_prod.endpoint
  port        = 5432
  database_name = "prod_db"
  username     = "dms_target_user"
  password     = var.target_db_password

  extra_connection_attributes = join(";", [
    "executeTimeout=300",
  ])
```

```

    "maxFileSize=512000", # 512 MB file size for bulk load
    "parallelLoadThreads=4"
  })
}

# Replication task - full load then CDC
resource "aws_dms_replication_task" "full_load_cdc" {
  replication_task_id      = "prod-full-load-cdc"
  migration_type           = "full-load-and-cdc"
  replication_instance_arn = aws_dms_replication_instance.migration.replication_instance_arn
  source_endpoint_arn      = aws_dms_endpoint.source_pg.endpoint_arn
  target_endpoint_arn      = aws_dms_endpoint.target_aurora.endpoint_arn

  table_mappings = jsonencode({
    rules = [
      {
        rule-type = "selection"
        rule-id   = "1"
        rule-name = "include-all-tables"
        object-locator = {
          schema-name = "public"
          table-name  = "%"
        }
        rule-action = "include"
      },
      {
        # Exclude audit/log tables that are too large and not needed on target
        rule-type = "selection"
        rule-id   = "2"
        rule-name = "exclude-audit-tables"
        object-locator = {
          schema-name = "public"
          table-name  = "audit_%"
        }
        rule-action = "exclude"
      }
    ]
  })

  replication_task_settings = jsonencode({
    TargetMetadata = {
      TargetSchema      = ""
      SupportLobs        = true
      FullLobMode        = false # Limited LOB mode is faster; full LOB mode for large BLOBs
      LobChunkSize       = 64    # KB
      LimitedSizeLobMode = true
      LobMaxSize         = 32768 # 32 MB max LOB size in limited mode
    }
    FullLoadSettings = {
      TargetTablePrepMode = "DROP_AND_CREATE" # Safe for initial load; change to DO_NOTHING for
↪ reseeding
      CreatePkAfterFullLoad = true             # Apply PKs after load for performance
      StopTaskCachedChangesApplied = false
      StopTaskCachedChangesNotApplied = false
      MaxFullLoadSubTasks   = 8                # Parallel table loads
      TransactionConsistencyTimeout = 600
      CommitRate            = 50000            # Rows per commit in full load
    }
    Logging = {
      EnableLogging = true
      LogComponents = [
        { Id = "SOURCE_UNLOAD", Severity = "LOGGER_SEVERITY_DEFAULT" },
        { Id = "TARGET_LOAD",   Severity = "LOGGER_SEVERITY_DEFAULT" },
        { Id = "TASK_MANAGER",  Severity = "LOGGER_SEVERITY_DEFAULT" }
      ]
    }
    ControlTablesSettings = {
      historyTimeslotInMinutes = 5
      StatusTableEnabled      = true
      SuspendedTablesTableEnabled = true
    }
  })
}

```

```
}  
})  
}
```

Prepare the Source PostgreSQL for DMS Logical Replication

DMS requires logical replication to be enabled on the source and a replication user with appropriate privileges. On self-managed PostgreSQL, you control this directly. On RDS PostgreSQL source (if migrating between RDS instances), the parameter group controls it.

```
-- On source PostgreSQL (self-managed): enable logical replication  
-- Edit postgresql.conf or run as superuser:  
ALTER SYSTEM SET wal_level = 'logical';  
ALTER SYSTEM SET max_replication_slots = 10;  
ALTER SYSTEM SET max_wal_senders = 10;  
-- Requires a restart:  
-- SELECT pg_reload_conf(); -- Only for parameters that don't require restart  
  
-- Create the DMS replication user with minimum required privileges  
CREATE USER dms_replication_user WITH  
    REPLICATION  
    LOGIN  
    PASSWORD 'strong_password_here';  
  
-- Grant access to all tables DMS needs to replicate  
GRANT CONNECT ON DATABASE prod_db TO dms_replication_user;  
GRANT USAGE ON SCHEMA public TO dms_replication_user;  
GRANT SELECT ON ALL TABLES IN SCHEMA public TO dms_replication_user;  
ALTER DEFAULT PRIVILEGES IN SCHEMA public  
    GRANT SELECT ON TABLES TO dms_replication_user;  
  
-- Create heartbeat schema for DMS health checks  
CREATE SCHEMA IF NOT EXISTS dms_heartbeat;  
GRANT ALL ON SCHEMA dms_heartbeat TO dms_replication_user;  
  
-- If using pglogical plugin (recommended over test_decoding for production):  
CREATE EXTENSION IF NOT EXISTS pglogical;  
  
-- Verify logical replication is active  
SELECT slot_name, plugin, slot_type, active, restart_lsn  
FROM pg_replication_slots;  
  
-- Monitor DMS replication lag during migration  
SELECT  
    slot_name,  
    pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS replication_lag_size,  
    active,  
    active_pid  
FROM pg_replication_slots  
WHERE slot_type = 'logical';
```

Critical DMS Gotchas in Production

DMS has a set of well-documented limitations that will surprise you the first time they surface at 2 AM:

Tables without primary keys fall back to full-table scans for CDC, which is catastrophically slow on large tables. Run the assessment query from section 33.1 to identify these before starting. Either add surrogate primary keys to the source before migration or exclude these tables from CDC and migrate them separately at cutover.

LOB column handling in limited LOB mode will silently truncate data exceeding your `LobMaxSize` setting. Set it conservatively high and verify with a row count and checksum comparison after full load.

DDL changes during CDC — DMS does not replicate DDL by default for PostgreSQL sources. If your source application runs `ALTER TABLE` during the migration window, you must apply those DDL changes manually to the target and then resume replication. This is one strong argument for freeze windows on schema changes during migration.

DMS does not migrate sequences. After full load completes, manually synchronize sequence values:

```
-- Run on source after full load completes, before cutover
-- Generate sequence synchronization statements
SELECT
  'SELECT setval(' || schemaname || '.' || sequencename ||
    ', COALESCE((SELECT MAX(' || quote_ident(a.attname) ||
    ') FROM ' || schemaname || '.' || tablename || '), 1), true);' AS sync_command
FROM pg_sequences s
JOIN pg_class c ON c.relname = s.sequencename
JOIN pg_depend d ON d.objid = c.oid
JOIN pg_class ct ON d.refobjid = ct.oid
JOIN pg_attribute a ON a.attrelid = ct.oid AND a.attnum = d.refobjsubid
JOIN pg_tables t ON t.tablename = ct.relname AND t.schemaname = s.schemaname
WHERE s.schemaname = 'public';

-- Execute each of those statements on the TARGET Aurora instance
-- This is typically a 5-10 minute manual step during cutover
```

33.3 Azure Database Migration Service and SQL Server Migration

When moving SQL Server workloads to Azure SQL — either Azure SQL Database, Azure SQL Managed Instance, or SQL Server on Azure VM — the Azure Database Migration Service (DMS) handles the heavy lifting. The key architectural decision you make before starting is whether your target is Azure SQL Database (PaaS, fully managed, most restrictive), Azure SQL Managed Instance (PaaS, near-full SQL Server compatibility, fewer restrictions), or SQL Server on Azure VM (IaaS, full control, your responsibility for patching and HA).

For most enterprise SQL Server migrations, Azure SQL Managed Instance is the right target: it supports SQL Agent, cross-database queries, linked servers, CLR, Service Broker, and the full SQL Server surface area — things Azure SQL Database does not. The trade-off is cost; Managed Instance is more expensive.

Assess Compatibility Before Running DMS

The Data Migration Assistant (DMA) is your pre-migration assessment tool. Run it against every source SQL Server database:

```
# Azure DMA is a Windows GUI tool, but you can automate assessments via its command line
# Install: https://www.microsoft.com/en-us/download/details.aspx?id=53595

# Run DMA assessment (from Windows command prompt or PowerShell)
"C:\Program Files\Microsoft Data Migration Assistant\DmaCmd.exe" `
  /AssessmentName="ProdDBAssessment" `
  /AssessmentSourcePlatform=SqlServer `
  /AssessmentTargetPlatform=AzureSqlDatabaseManagedInstance `
  /AssessmentDatabases="Server=onprem-sql.internal;Initial Catalog=ProdDB;User
    ⇐ Id=dma_user;Password=$env:DMA_PASSWORD" `
  /AssessmentEvaluateCompatibilityIssues=True `
  /AssessmentEvaluateFeatureParity=True `
  /AssessmentOverwriteResult=True `
  /AssessmentResultJson="C:\Reports\assessment-result.json"

# Parse results to get action items
```

```
python parse_dma_results.py --input C:\Reports\assessment-result.json

# parse_dma_results.py - Extract blocking issues from DMA output
import json
import sys
from pathlib import Path

def parse_dma_report(report_path: str) -> None:
    with open(report_path) as f:
        report = json.load(f)

    blocking_issues = []
    warning_issues = []

    for server_report in report.get("serverReports", []):
        for db_report in server_report.get("databaseReports", []):
            db_name = db_report.get("databaseName")

            for category in db_report.get("items", []):
                for rule_result in category.get("ruleResults", []):
                    severity = rule_result.get("severity", "")
                    title = rule_result.get("title", "")
                    impact = rule_result.get("impact", "")
                    affected_objects = [
                        obj.get("name") for obj in rule_result.get("affectedObjects", [])
                    ]

                    entry = {
                        "database": db_name,
                        "title": title,
                        "impact": impact,
                        "affected_objects": affected_objects[:5], # Top 5
                        "total_affected": len(affected_objects)
                    }

                    if severity == "Breaking":
                        blocking_issues.append(entry)
                    elif severity == "Warning":
                        warning_issues.append(entry)

    print(f"\n{'='*60}")
    print(f"BLOCKING ISSUES ({len(blocking_issues)}) - Must fix before migration:")
    print(f"{'='*60}")
    for issue in blocking_issues:
        print(f"\n[{issue['database']}] {issue['title']}")
        print(f"  Impact: {issue['impact']}")
        print(f"  Affected objects ({issue['total_affected']}): {' '.join(issue['affected_objects'])}")

    print(f"\n{'='*60}")
    print(f"WARNINGS ({len(warning_issues)}) - Review post-migration:")
    print(f"{'='*60}")
    for issue in warning_issues[:10]: # Show top 10 warnings
        print(f"\n[{issue['database']}] {issue['title']}")

if __name__ == "__main__":
    parse_dma_report(sys.argv[1])
```

Configure Azure DMS for Online Migration to Managed Instance

Azure DMS for SQL Managed Instance uses SQL Server transaction log shipping combined with log replay service to achieve minimal downtime:

```
# Create Azure DMS instance
az dms create \
  --name prod-migration-dms \
  --resource-group rg-migration \
  --location eastus \
  --sku-name Premium_4vCores \
```



```

--subnet /subscriptions/${SUBSCRIPTION_ID}]
↪ /resourceGroups/rg-networking/providers/Microsoft.Network/virtualNetworks/migration-vnet/subnets/dms-subnet

# Create the source connection info
SOURCE_CONNECTION='{
  "serverName": "onprem-sql.internal",
  "port": 1433,
  "authentication": "SqlAuthentication",
  "userName": "migration_user",
  "password": "${SOURCE_PASSWORD}"
}'

# Create migration project
az dms project create \
  --name sql-to-mi-project \
  --resource-group rg-migration \
  --service-name prod-migration-dms \
  --source-platform SQL \
  --target-platform SQLMI \
  --location eastus

# Create the migration task
# For SQL MI online migration, DMS uses log shipping via Azure Blob Storage
az dms task create \
  --database-options-json ' [{
    "name": "ProdDB",
    "targetDatabaseName": "ProdDB",
    "backupFileShare": {
      "path": "\\onprem-fileserver\\sqlbackups",
      "userName": "backup_user",
      "password": "${BACKUP_SHARE_PASSWORD}"
    },
    "selectedTables": null,
    "makeSourceDbReadOnly": false
  } ]' \
  --name online-migration-task \
  --project-name sql-to-mi-project \
  --resource-group rg-migration \
  --service-name prod-migration-dms \
  --source-connection-json "${SOURCE_CONNECTION}" \
  --target-connection-json '{
    "serverName": "${MI_FQDN}",
    "port": 1433,
    "authentication": "SqlAuthentication",
    "userName": "mi_admin",
    "password": "${TARGET_PASSWORD}"
  }' \
  --task-type OnlineMigration

```

Prepare the Source SQL Server for Online Migration

Online migration to SQL MI requires full recovery model and a recent full backup. These are baseline requirements, but there are additional source-side preparations that prevent mid-migration failures:

```

-- Run on source SQL Server: pre-migration preparation

-- 1. Verify databases are in FULL recovery model
SELECT name, recovery_model_desc
FROM sys.databases
WHERE name NOT IN ('master', 'model', 'msdb', 'tempdb')
ORDER BY name;

-- Fix any databases not in FULL recovery
-- ALTER DATABASE [ProdDB] SET RECOVERY FULL;

-- 2. Check for deprecated features that DMA flagged
-- Text/NText/Image columns -- must be aware of migration behavior

```

```

SELECT
    t.name AS table_name,
    c.name AS column_name,
    ty.name AS data_type
FROM sys.columns c
JOIN sys.tables t ON c.object_id = t.object_id
JOIN sys.types ty ON c.user_type_id = ty.user_type_id
WHERE ty.name IN ('text', 'ntext', 'image')
    AND t.is_ms_shipped = 0
ORDER BY t.name, c.name;

-- 3. Find linked servers that will break after migration
SELECT
    name AS linked_server_name,
    product,
    provider,
    data_source,
    is_remote_login_enabled,
    modify_date
FROM sys.servers
WHERE is_linked = 1
ORDER BY name;

-- 4. Identify SQL Agent jobs and their schedules
SELECT
    j.name AS job_name,
    j.enabled,
    j.description,
    c.name AS category,
    l.name AS owner,
    j.date_modified
FROM msdb.dbo.sysjobs j
JOIN msdb.dbo.syscategories c ON j.category_id = c.category_id
JOIN sys.syslogins l ON j.owner_sid = l.sid
ORDER BY j.name;

-- 5. Check current log backup chain - DMS needs an unbroken chain
-- If log backups have gaps, you need a new full backup
EXEC sp_helpdb 'ProdDB';

-- 6. Identify certificates and asymmetric keys (TDE, encrypted objects)
SELECT
    name,
    certificate_id,
    pvt_key_encryption_type_desc,
    is_active_for_begin_dialog
FROM sys.certificates
WHERE database_id = DB_ID('ProdDB');

-- If TDE is enabled, export the certificate BEFORE migration
-- Failure to do this means the database cannot be restored on MI
-- BACKUP CERTIFICATE TDECert
-- TO FILE = '\\backup-server\certs\TDECert.cer'
-- WITH PRIVATE KEY (
--     FILE = '\\backup-server\certs\TDECert_key.pvk',
--     ENCRYPTION BY PASSWORD = 'cert_export_password'
-- );

```

Monitor Azure DMS Migration Progress

```

# Poll migration task status
watch_migration() {
    local RESOURCE_GROUP="rg-migration"
    local SERVICE="prod-migration-dms"
    local PROJECT="sql-to-mi-project"
    local TASK="online-migration-task"

    while true; do
        STATUS=$(az dms task show \
            --resource-group "${RESOURCE_GROUP}" \

```

```
--service-name "${SERVICE}" \
--project-name "${PROJECT}" \
--name "${TASK}" \
--expand output \
--query "output[0].migrationState" \
--output tsv 2>/dev/null)

LAG=$(az dms task show \
  --resource-group "${RESOURCE_GROUP}" \
  --service-name "${SERVICE}" \
  --project-name "${PROJECT}" \
  --name "${TASK}" \
  --expand output \
  --query "output[0].databaseSummary.ProdDB.cdcLatency" \
  --output tsv 2>/dev/null)

echo "[$(date '+%H:%M:%S')] State: ${STATUS} | CDC Lag: ${LAG}s"

if [[ "${STATUS}" == "ReadyForCutover" ]]; then
  echo ">>> MIGRATION IS READY FOR CUTOVER <<<"
  break
fi

sleep 30
done
}

watch_migration

# Initiate cutover when ready
az dms task cutover \
  --resource-group rg-migration \
  --service-name prod-migration-dms \
  --project-name sql-to-mi-project \
  --name online-migration-task \
  --object-name ProdDB
```

33.4 GCP Database Migration Service

Google Cloud's Database Migration Service (DMS) is the youngest of the three major cloud DMS offerings. It supports PostgreSQL-to-Cloud SQL PostgreSQL, MySQL-to-Cloud SQL MySQL, and PostgreSQL/Oracle-to-AlloyDB migrations. Its setup model differs from AWS and Azure DMS in one important way: connectivity to the source is managed through a *connection profile*, and you can route traffic through a reverse-SSH tunnel, VPN, or VPC peering — which is particularly useful when your source environment does not permit inbound connections from the cloud.

Create a GCP DMS Migration Job via gcloud

```
# Set project and region
gcloud config set project your-project-id
REGION="us-central1"
SOURCE_IP="10.0.1.50" # On-prem PostgreSQL IP, reachable via Cloud VPN or Interconnect

# 1. Create the source connection profile
# --password with no value will prompt interactively - use Secret Manager in CI/CD
gcloud database-migration connection-profiles create postgresql \
  --region="${REGION}" \
  --connection-profile-id="onprem-pg-source" \
  --host="${SOURCE_IP}" \
  --port=5432 \
  --username="migration_user" \
  --password \
  --database-service="postgres" \
```

```

--display-name="On-Prem PostgreSQL Source"

# 2. Create the destination connection profile (Cloud SQL instance)
gcloud database-migration connection-profiles create cloudsql \
  --region="${REGION}" \
  --connection-profile-id="cloudsql-pg-target" \
  --cloudsql-instance-id="prod-cloudsql-pg" \
  --display-name="Cloud SQL PostgreSQL Target"

# 3. Create the migration job (continuous / minimal downtime)
gcloud database-migration migration-jobs create \
  --region="${REGION}" \
  --migration-job-id="prod-pg-migration" \
  --source="onprem-pg-source" \
  --destination="cloudsql-pg-target" \
  --type=CONTINUOUS \
  --display-name="Production PostgreSQL Migration" \
  --dump-path="gs://your-migration-bucket/pg-dump/"
# --dump-path is for initial seeding from a Cloud Storage dump;
# omit to let DMS perform the initial dump automatically

# 4. Verify connectivity before starting
gcloud database-migration migration-jobs verify \
  --region="${REGION}" \
  --migration-job-id="prod-pg-migration"

# Expected output includes per-check pass/fail - fix any FAIL before proceeding
# Common failures: firewall rules blocking DMS egress IPs, wal_level not set to logical,
# replication user lacks REPLICATION privilege

# 5. Start the migration
gcloud database-migration migration-jobs start \
  --region="${REGION}" \
  --migration-job-id="prod-pg-migration"

# 6. Monitor status
gcloud database-migration migration-jobs describe \
  --region="${REGION}" \
  --migration-job-id="prod-pg-migration" \
  --format="yaml(state, phase, error, duration)"

```

GCP DMS Source PostgreSQL Requirements

GCP DMS uses `pglogical` extension internally. The requirements mirror AWS DMS but with slightly different parameter names:

```

-- On source PostgreSQL for GCP DMS:

-- Required parameter settings (edit postgresql.conf or use ALTER SYSTEM)
ALTER SYSTEM SET wal_level = 'logical';
ALTER SYSTEM SET max_replication_slots = 10;
ALTER SYSTEM SET max_wal_senders = 10;
ALTER SYSTEM SET wal_sender_timeout = 0; -- Prevent timeout during large transactions

-- GCP DMS requires the pglogical extension
CREATE EXTENSION IF NOT EXISTS pglogical;

-- Create migration user with required privileges
CREATE USER gcp_dms_user WITH REPLICATION LOGIN PASSWORD 'strong_password';
GRANT CONNECT ON DATABASE prod_db TO gcp_dms_user;

-- pglogical requires the user to have superuser OR specific privileges:
-- On managed sources (like RDS), use rds_superuser grant if applicable
-- On self-managed: grant superuser, OR grant these specific privileges:
GRANT SELECT ON ALL TABLES IN SCHEMA public TO gcp_dms_user;
GRANT SELECT ON ALL SEQUENCES IN SCHEMA public TO gcp_dms_user;
GRANT USAGE ON SCHEMA public TO gcp_dms_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO gcp_dms_user;

```

```
-- pglogical also needs SELECT on pg_catalog system tables - superuser handles this
-- For non-superuser, grant replication role privilege:
GRANT pg_read_all_data TO gcp_dms_user; -- PostgreSQL 14+

-- Verify pg_hba.conf allows replication connections from DMS egress IPs
-- Add to pg_hba.conf:
-- host replication gcp_dms_user 34.0.0.0/8 md5 -- GCP DMS IP range (verify current range)
-- host all gcp_dms_user 34.0.0.0/8 md5
```

GCP DMS: Promote to Standalone (Cutover)

When the migration job reaches CDC_RUNNING phase and replication lag is near zero, you promote the Cloud SQL instance from replica to standalone:

```
# Monitor lag before promoting
watch_gcp_migration() {
    while true; do
        gcloud database-migration migration-jobs describe \
            --region=us-central1 \
            --migration-job-id="prod-pg-migration" \
            --format="table(state,phase,duration,error.message)" 2>/dev/null

        # Also check CDC lag via Cloud SQL metrics
        gcloud monitoring metrics list \
            --filter="metric.type=cloudsql.googleapis.com/database/replication/replica_lag" \
            2>/dev/null | head -20

        sleep 30
    done
}

# Promote when lag is consistently < 5 seconds and during your cutover window
gcloud database-migration migration-jobs promote \
    --region=us-central1 \
    --migration-job-id="prod-pg-migration"

# After promotion, Cloud SQL is no longer a replica - it accepts writes
# This is the point of no return for the replication stream
```

33.5 Cutover Runbook: The Final 60 Minutes

The migration tools handle data movement. Cutover is where you transition application traffic. A poorly executed cutover causes data loss or extended downtime even when the underlying migration was perfect. This section is a scripted runbook — execute each step in order, with named owners and time checkpoints.

Pre-Cutover Checklist (T-48 hours)

- [] All blocking DMA/SCT issues resolved and tested on target
- [] Application connection strings tested against target database (read-only smoke test)
- [] DNS TTLs reduced to 60 seconds (they were at 300+ before - change 48h ahead)
- [] Backup taken of source database within 24h of cutover
- [] Rollback plan documented and tested (see section 33.6)
- [] Monitoring alerts configured on target instance (CPU, connections, replication lag)
- [] On-call rotation confirmed with names and phone numbers
- [] Schema change freeze in effect on source (no DDL until post-cutover)
- [] Sequence values audited - sync script ready to execute
- [] All DMS/migration job metrics reviewed: no errors, lag < 10 seconds

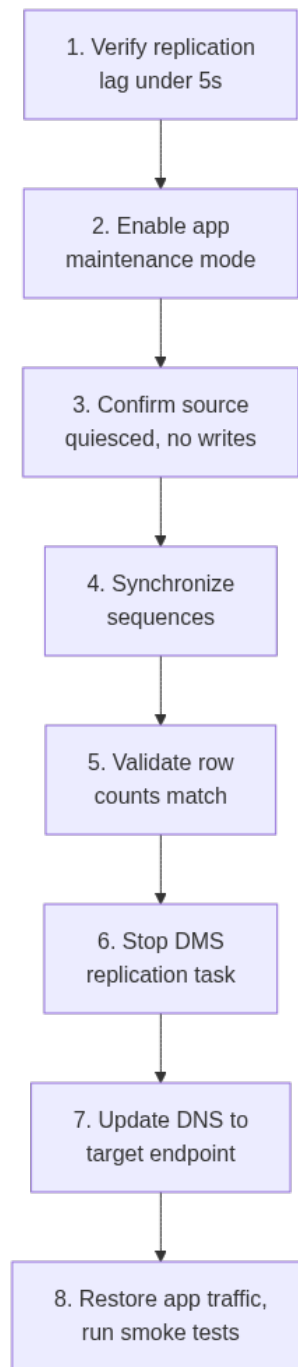


Figure 24: Each step gates the next — this is also the sequence a rollback must unwind if smoke tests fail within the first hour.

Cutover Execution (T-0)

```
#!/bin/bash
# cutover.sh - Execute with two DBAs present: one driving, one verifying
# Time each step and log to a shared document in real time

set -euo pipefail
LOGFILE="/tmp/cutover-$(date +%Y%m%d-%H%M).log"
exec >>(tee -a "${LOGFILE}") 2>&1

log_step() {
    echo ""
    echo "=====
    echo "[$(date '+%H:%M:%S')] STEP: $1"
    echo "=====
}

#--- STEP 1: Final validation of replication lag ---#
log_step "1. Verify replication lag < 5 seconds"

# For AWS DMS:
aws dms describe-replication-tasks \
    --filters Name=replication-task-id,Values=prod-full-load-cdc \
    --query
    ↪ 'ReplicationTasks[0].{Status:Status,CDCLatencyTarget:CDCLatencyTarget,CDCLatencySource:CDCLatencySource}'
    ↪ \
    --output table

# For GCP DMS:
# gcloud database-migration migration-jobs describe --region=us-central1
# ↪ --migration-job-id="prod-pg-migration"

echo "Confirm lag is acceptable before proceeding. Press ENTER to continue or Ctrl+C to abort."
read -r

#--- STEP 2: Put application in maintenance mode ---#
log_step "2. Enable maintenance mode on application"
# This is application-specific - examples:
# - Set load balancer target group to maintenance page
# - Toggle feature flag in config service
# - Scale down application pods to 0
# kubectl scale deployment app-deployment --replicas=0 -n production

echo "Application maintenance mode enabled. Confirm no new writes to source. Press ENTER."
read -r

#--- STEP 3: Verify zero active write transactions on source ---#
log_step "3. Confirm source is quiesced (no active writes)"

# PostgreSQL source
psql -h "${SOURCE_HOST}" -U dba_user -d prod_db -c "
SELECT count(*) AS active_write_transactions
FROM pg_stat_activity
WHERE state = 'active'
    AND query !~ '^(SELECT|SHOW|EXPLAIN|--)'
    AND backend_type = 'client backend'
    AND pid != pg_backend_pid();
"

# SQL Server source
# sqlcmd -S "${SOURCE_SQL_HOST}" -U dba_user -Q "
# SELECT COUNT(*) AS active_write_spids
# FROM sys.dm_exec_sessions s
# JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
# WHERE r.command NOT IN ('SELECT', 'BACKUP DATABASE', 'BACKUP LOG')
#     AND s.database_id = DB_ID('ProdDB')
#     AND s.is_user_process = 1;
# "

echo "Source quiesced. Press ENTER to proceed with sequence sync."
```

```

read -r

#--- STEP 4: Synchronize sequences ---#
log_step "4. Synchronize sequences on target"

# Generate and execute sequence sync (PostgreSQL targets)
psql -h "${SOURCE_HOST}" -U dba_user -d prod_db -t -A -c "
SELECT 'SELECT setval('' || schemaname || '.' || sequencename ||
      ''', COALESCE((SELECT MAX(' || quote_ident(a.attname) ||
      ') FROM ' || schemaname || '.' || tablename || '), 1), true);'
FROM pg_sequences s
JOIN pg_class c ON c.relname = s.sequencename
JOIN pg_depend d ON d.objid = c.oid
JOIN pg_class ct ON d.refobjid = ct.oid
JOIN pg_attribute a ON a.attrelid = ct.oid AND a.attnum = d.refobjsubid
JOIN pg_tables t ON t.tablename = ct.relname AND t.schemaname = s.schemaname
WHERE s.schemaname NOT IN ('pg_catalog', 'information_schema');
" > /tmp/sequence_sync.sql

echo "Generated sequence sync SQL. Review /tmp/sequence_sync.sql then press ENTER."
cat /tmp/sequence_sync.sql
read -r

psql -h "${TARGET_HOST}" -U target_dba_user -d prod_db -f /tmp/sequence_sync.sql
echo "Sequences synchronized."

#--- STEP 5: Row count validation ---#
log_step "5. Validate row counts source vs target"

validate_counts() {
    local HOST=$1
    local LABEL=$2
    psql -h "${HOST}" -U dba_user -d prod_db -t -A -c "
SELECT tablename || '|' || n_live_tup
FROM pg_stat_user_tables
ORDER BY tablename;
" > "/tmp/counts_${LABEL}.txt"
}

validate_counts "${SOURCE_HOST}" "source"
validate_counts "${TARGET_HOST}" "target"

DIFF_COUNT=$(diff /tmp/counts_source.txt /tmp/counts_target.txt | grep -c "^[<>]" || true)
if [[ "${DIFF_COUNT}" -gt 0 ]]; then
    echo "WARNING: Row count differences detected:"
    diff /tmp/counts_source.txt /tmp/counts_target.txt
    echo "Investigate before proceeding. Press ENTER to continue anyway or Ctrl+C to abort."
    read -r
else
    echo "Row counts match. Proceeding."
fi

#--- STEP 6: Stop DMS task ---#
log_step "6. Stop DMS replication task"

# AWS DMS:
aws dms stop-replication-task \
    --replication-task-arn "${DMS_TASK_ARN}"

echo "Waiting for DMS task to stop..."
aws dms wait replication-task-stopped \
    --filters "Name=replication-task-arn,Values=${DMS_TASK_ARN}"

echo "DMS task stopped."

#--- STEP 7: Update DNS / connection strings ---#
log_step "7. Update application DNS to point to target"

# Update Route 53 (AWS example)
aws route53 change-resource-record-sets \

```



```

--hosted-zone-id "${HOSTED_ZONE_ID}" \
--change-batch '{
  "Changes": [{
    "Action": "UPSERT",
    "ResourceRecordSet": {
      "Name": "db.internal.example.com",
      "Type": "CNAME",
      "TTL": 60,
      "ResourceRecords": [{"Value": "'${AURORA_ENDPOINT}'"}]
    }
  ]
}'

echo "DNS updated. TTL is 60s - wait 90 seconds for propagation."
sleep 90

#--- STEP 8: Restore application traffic ---#
log_step "8. Restore application traffic"

# Scale application back up / disable maintenance mode
# kubectl scale deployment app-deployment --replicas=4 -n production

echo "Application traffic restored. Begin smoke testing."

```

Post-Cutover Smoke Tests

```

-- Run on target immediately after traffic restoration
-- These queries confirm application behavior is correct

-- PostgreSQL target: verify active connections coming in
SELECT
  client_addr,
  username,
  application_name,
  state,
  count(*) AS connection_count
FROM pg_stat_activity
WHERE backend_type = 'client backend'
  AND datname = current_database()
GROUP BY client_addr, username, application_name, state
ORDER BY connection_count DESC
LIMIT 20;

-- Verify no long-running idle transactions (application connection pool issue)
SELECT
  pid,
  username,
  application_name,
  state,
  now() - xact_start AS transaction_age,
  left(query, 100) AS query_snippet
FROM pg_stat_activity
WHERE state IN ('idle in transaction', 'idle in transaction (aborted)')
  AND xact_start < now() - interval '60 seconds';

-- SQL Server target: equivalent connection validation
-- SELECT
--   s.session_id,
--   s.login_name,
--   s.program_name,
--   s.status,
--   r.command,
--   r.wait_type,
--   s.last_request_start_time
-- FROM sys.dm_exec_sessions s
-- LEFT JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
-- WHERE s.is_user_process = 1
--   AND s.database_id = DB_ID('ProdDB')
-- ORDER BY s.last_request_start_time DESC;

```

33.6 Rollback Plan

Every cutover runbook needs a tested rollback path. The time to design the rollback is not during an incident at 3 AM — it is before you start the migration. The rollback strategy depends on your migration type.

Rollback for DMS-Based Migrations (PostgreSQL)

If you detect data corruption or application failures within the first hour after cutover, you can roll back by directing traffic back at the source — but only if the source was not taken offline or made read-only.

```
# ROLLBACK PROCEDURE - Execute only if smoke tests fail within 1 hour of cutover
# This window closes when you decommission the source or disable writes to it

log_step "ROLLBACK: Redirecting traffic to source database"

# 1. Put application in maintenance mode again
# kubectl scale deployment app-deployment --replicas=0 -n production

# 2. Revert DNS to source
aws route53 change-resource-record-sets \
  --hosted-zone-id "${HOSTED_ZONE_ID}" \
  --change-batch '{
    "Changes": [{
      "Action": "UPSERT",
      "ResourceRecordSet": {
        "Name": "db.internal.example.com",
        "Type": "CNAME",
        "TTL": 60,
        "ResourceRecords": [{"Value": "'${SOURCE_HOST}'"}]}
    ]
  }'

echo "DNS reverted to source. Waiting 90s for propagation."
sleep 90

# 3. Restore application traffic
# kubectl scale deployment app-deployment --replicas=4 -n production

echo "ROLLBACK COMPLETE. Applications are writing to source again."
echo "Assess target failures before rescheduling migration."
```

Important rollback constraint: once you stop the DMS task and allow writes to the target, the replication stream is broken. If you discover a problem after writes have occurred on the target, you cannot roll back cleanly without accepting data loss. This is why the cutover window (Step 2 through Step 8 in the cutover script) must be executed quickly — typically under 15 minutes for most workloads — and smoke tests must be decisive.

For SQL Server migrations to Azure SQL Managed Instance, Azure DMS keeps the migration task in `ReadyForCutover` state, and you have not yet invoked the cutover command. In this state, you can abandon the migration task without any impact to the source. Once you issue the cutover command, the commitment is final.

33.7 Post-Migration Optimization

The database has landed in cloud. The application is running. The work is not done. Cloud databases expose different performance characteristics than on-premises ones: network-attached storage introduces latency patterns you did not see on direct-attached SAN, the managed service's shared infrastructure imposes IOPS limits that vary by instance class, and the optimizer's statistics may not yet reflect production query patterns.

Rebuild Statistics and Indexes on Target

```
-- PostgreSQL (Aurora / Cloud SQL): rebuild statistics after full load
-- DMS full load does not run ANALYZE; statistics are stale

-- Run ANALYZE on all user tables immediately after migration
DO $$
DECLARE
    tbl RECORD;
BEGIN
    FOR tbl IN
        SELECT schemaname, tablename
        FROM pg_tables
        WHERE schemaname NOT IN ('pg_catalog', 'information_schema', 'pg_toast')
    LOOP
        EXECUTE format('ANALYZE %I.%I', tbl.schemaname, tbl.tablename);
        RAISE NOTICE 'Analyzed: %.', tbl.schemaname, tbl.tablename;
    END LOOP;
END $$;

-- For large tables, run ANALYZE with a progress check
-- Aurora/RDS exposes pg_stat_progress_analyze (PostgreSQL 13+)
SELECT
    phase,
    sample_blks_total,
    sample_blks_scanned,
    ext_stats_total,
    ext_stats_computed,
    child_tables_total,
    child_tables_done
FROM pg_stat_progress_analyze;

-- Check for bloated indexes from the full load process
-- DMS loads data in bulk; indexes may have high dead-tuple ratios
SELECT
    schemaname,
    tablename,
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch
FROM pg_stat_user_indexes
JOIN pg_index USING (indexrelid)
WHERE idx_scan = 0 -- Indexes never used since migration
    AND NOT indisprimary
    AND NOT indisunique
ORDER BY pg_relation_size(indexrelid) DESC
LIMIT 20;

-- SQL Server (Azure SQL MI / Azure SQL Database): post-migration statistics
-- Rebuild statistics after full load (equivalent operation)

-- Update all statistics with full scan
EXEC sp_updatestats;

-- For a more thorough rebuild on a specific database:
USE Proddb;
```

```

DECLARE @sql NVARCHAR(MAX) = N'';
SELECT @sql += N'UPDATE STATISTICS ' + QUOTENAME(SCHEMA_NAME(t.schema_id))
      + N'.' + QUOTENAME(t.name) + N' WITH FULLSCAN;' + CHAR(10)
FROM sys.tables t
WHERE t.is_ms_shipped = 0;

-- Review before executing
PRINT @sql;
-- EXEC sp_executesql @sql; -- Uncomment to execute

-- Identify indexes with high fragmentation post-migration
SELECT
    DB_NAME() AS database_name,
    OBJECT_NAME(ips.object_id) AS table_name,
    i.name AS index_name,
    ips.index_type_desc,
    ROUND(ips.avg_fragmentation_in_percent, 2) AS fragmentation_pct,
    ips.page_count,
    ips.record_count
FROM sys.dm_db_index_physical_stats(
    DB_ID(), NULL, NULL, NULL, 'LIMITED'
) ips
JOIN sys.indexes i
    ON ips.object_id = i.object_id
   AND ips.index_id = i.index_id
WHERE ips.avg_fragmentation_in_percent > 30
      AND ips.page_count > 1000
ORDER BY ips.avg_fragmentation_in_percent DESC;

```

Verify Query Plan Stability

The first week after migration is when query plan regressions appear. Queries that ran well on the source may receive poor plans on the target due to different optimizer defaults, missing statistics, or engine version differences.

```

-- PostgreSQL: enable pg_stat_statements to catch slow queries post-migration
-- Confirm extension is installed (it should be on RDS/Aurora/Cloud SQL)
SELECT * FROM pg_available_extensions WHERE name = 'pg_stat_statements';

CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Top 20 slowest queries by mean execution time since migration
SELECT
    round(mean_exec_time::numeric, 2) AS mean_ms,
    round(total_exec_time::numeric, 2) AS total_ms,
    calls,
    round((stddev_exec_time / NULLIF(mean_exec_time, 0) * 100)::numeric, 1) AS variability_pct,
    rows / NULLIF(calls, 0) AS avg_rows,
    left(query, 120) AS query_snippet
FROM pg_stat_statements
WHERE userid = (SELECT oid FROM pg_roles WHERE rolname = current_user)
      OR userid IN (SELECT oid FROM pg_roles WHERE rolname != 'rdsadmin')
ORDER BY mean_exec_time DESC
LIMIT 20;

-- Reset stats baseline after initial stabilization (24-48h post-migration)
-- SELECT pg_stat_statements_reset();

-- SQL Server: use Query Store to compare plans before and after migration
-- Query Store should be enabled on Azure SQL by default

-- Check Query Store is active
SELECT name, is_query_store_on
FROM sys.databases
WHERE name = 'ProdDB';

-- Find queries with high variance in execution time (plan instability)
SELECT TOP 20
    q.query_id,

```

```
qt.query_sql_text,
rs.count_executions,
ROUND(rs.avg_duration / 1000.0, 2) AS avg_duration_ms,
ROUND(rs.max_duration / 1000.0, 2) AS max_duration_ms,
ROUND(rs.stdev_duration / 1000.0, 2) AS stdev_duration_ms,
rs.avg_logical_io_reads
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
JOIN sys.query_store_runtime_stats_interval rsi
ON rs.runtime_stats_interval_id = rsi.runtime_stats_interval_id
WHERE rsi.start_time > DATEADD(HOUR, -24, GETUTCDATE())
ORDER BY rs.stdev_duration DESC;
```

Right-Size the Instance After One Week

Cloud instances provisioned for migration are often over-provisioned. After one week of production traffic, review actual utilization and downsize if appropriate:

```
# AWS: check RDS instance utilization via CloudWatch
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name CPUUtilization \
  --dimensions Name=DBInstanceIdentifier,Value=prod-aurora-instance \
  --start-time $(date -d '7 days ago' --iso-8601=seconds) \
  --end-time $(date --iso-8601=seconds) \
  --period 3600 \
  --statistics Average Maximum \
  --query 'sort_by(Datapoints, &Timestamp)[*].{Time:Timestamp,Avg:Average,Max:Maximum}' \
  --output table

# Azure: check SQL MI CPU over 7 days
az monitor metrics list \
  --resource "/subscriptions/${SUBSCRIPTION_ID} \
  ↪ /resourceGroups/rg-prod/providers/Microsoft.Sql/managedInstances/prod-sql-mi" \
  --metric "avg_cpu_percent" \
  --interval PT1H \
  --start-time $(date -d '7 days ago' --iso-8601=seconds) \
  --end-time $(date --iso-8601=seconds) \
  --output table

# GCP: check Cloud SQL CPU utilization
gcloud monitoring metrics list \
  --filter="metric.type=cloudsql.googleapis.com/database/cpu/utilization" \
  2>/dev/null

# Or use the Metrics Explorer in GCP Console for visual review
```

Key Takeaways

- **Assessment before tooling:** run SCT and DMA against every source database before touching a migration tool — blocking incompatibilities discovered during execution cost ten times more to fix than the same issues discovered during assessment.
- **DMS does not migrate everything:** sequences, SQL Agent jobs, linked servers, certificates for TDE, and DDL changes during CDC are outside the replication stream; each requires a separate manual migration step baked into the cutover runbook.
- **Cutover speed determines rollback viability:** the longer your cutover window stays open, the more writes accumulate on the target and the harder it becomes to roll back cleanly; practice the cutover steps in a staging environment until they execute consistently in

under 15 minutes.

- **Statistics are stale immediately after full load:** DMS bulk load bypasses autovacuum and statistics collection; run `ANALYZE` (PostgreSQL) or `UPDATE STATISTICS` (SQL Server) as the first post-migration step before allowing application traffic, or query plans will be based on empty histograms.
 - **Right-size after one week, not before:** provisioning decisions made before migration are guesses; only after one week of real production traffic on the target instance do CloudWatch, Azure Monitor, and Cloud Monitoring metrics provide the utilization data needed to make a defensible downsize decision without risking performance regression.
-

Chapter 34: Zero-Downtime Cloud Migrations

Moving a production database to the cloud without downtime is one of the most operationally demanding challenges a DBA will face. The raw mechanics—logical replication, change data capture, cutover scripting—are not new. What changes in the cloud is the tooling surface, the failure modes, and the organizational expectations. Cloud migrations often carry a political weight that on-prem upgrades did not: leadership expects zero impact, and the migration team is responsible for delivering it against managed services that hide their internals behind API abstractions. This chapter covers the full arc of a zero-downtime migration: pre-migration assessment, continuous replication setup using AWS DMS, Azure Database Migration Service, and GCP Database Migration Service, schema conversion for heterogeneous workloads, cutover orchestration, and post-migration validation. Every pattern here is drawn from production environments where downtime was measured in seconds, not minutes.

34.1 Pre-Migration Assessment: What You Must Know Before You Touch Anything

Zero-downtime migrations fail during planning, not during execution. The actual data movement phase is deterministic. The failures come from unexamined assumptions made weeks earlier.

Schema Complexity Inventory

Start by cataloging every object type in your source database. For PostgreSQL sources, run:

```
-- Inventory of all non-trivial schema objects in PostgreSQL source
SELECT
    n.nspname                AS schema_name,
    c.relname                AS object_name,
    CASE c.relkind
        WHEN 'r' THEN 'TABLE'
        WHEN 'v' THEN 'VIEW'
        WHEN 'm' THEN 'MATERIALIZED_VIEW'
        WHEN 'f' THEN 'FOREIGN_TABLE'
        WHEN 'p' THEN 'PARTITIONED_TABLE'
    END                      AS object_type,
    pg_size_pretty(pg_total_relation_size(c.oid)) AS total_size,
    s.n_live_tup             AS live_rows,
    s.n_dead_tup             AS dead_rows,
    CASE WHEN s.n_live_tup > 0
        THEN ROUND(100.0 * s.n_dead_tup / s.n_live_tup, 2)
        ELSE 0
    END                      AS bloat_pct
```

```

FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
LEFT JOIN pg_stat_user_tables s
  ON s.schemaname = n.nspname AND s.relname = c.relname
WHERE n.nspname NOT IN ('pg_catalog', 'information_schema', 'pg_toast')
  AND c.relkind IN ('r', 'v', 'm', 'f', 'p')
ORDER BY pg_total_relation_size(c.oid) DESC NULLS LAST;

```

For SQL Server sources heading to Azure SQL or RDS for SQL Server, run the Database Migration Assistant (DMA) locally before involving any cloud tooling. DMA will surface blockers—cross-database queries, linked server dependencies, CLR assemblies, Service Broker endpoints—that no cloud migration service will handle automatically. A 50-table OLTP database with two linked servers and a CLR aggregate function is not a zero-downtime migration candidate until those dependencies are resolved. Treat DMA output as a go/no-go gate, not a to-do list.

Replication Eligibility

AWS DMS requires that source tables have a primary key or a unique non-nullable index for CDC (Change Data Capture) mode. Tables without either will be replicated using full-load only, meaning they receive no ongoing changes during the migration window. Query this on your source before you start:

```

-- Find tables with no primary key or unique constraint (PostgreSQL)
SELECT
  t.table_schema,
  t.table_name,
  pg_size_pretty(pg_total_relation_size(
    (t.table_schema || '.' || t.table_name)::regclass
  )) AS table_size
FROM information_schema.tables t
WHERE t.table_type = 'BASE TABLE'
  AND t.table_schema NOT IN ('pg_catalog', 'information_schema')
  AND NOT EXISTS (
    SELECT 1
    FROM information_schema.table_constraints tc
    WHERE tc.table_schema = t.table_schema
      AND tc.table_name = t.table_name
      AND tc.constraint_type IN ('PRIMARY KEY', 'UNIQUE')
  )
ORDER BY pg_total_relation_size(
  (t.table_schema || '.' || t.table_name)::regclass
) DESC;

```

Any table returned here needs a decision: add a surrogate primary key before migration, accept full-load-only semantics for it, or exclude it from replication and handle it with a separate bulk transfer at cutover.

Network Bandwidth Estimation

The migration network path matters more than the instance size. A db.r6g.8xlarge target instance is irrelevant if your source datacenter has 1 Gbps uplink shared with production traffic. Measure actual source database write throughput during peak hours using `pg_stat_bgwriter` and your existing monitoring:

- At 100 MB/s sustained write rate, you generate 360 GB/hour of WAL. DMS CDC must consume this continuously.
- AWS DMS replication instance throughput scales with instance class (vCPU, memory, and network bandwidth); larger `dms.r5`/`dms.r6i` classes handle higher sustained change rates, but the ceiling for your specific workload depends on row size, LOB usage, and table count

— benchmark it against your own source before committing to an instance class, rather than relying on a generic throughput figure.

- A 10 TB database at 50 MB/s average write rate will produce roughly 1.5 TB of change data over an 8-hour migration window. Your DMS replication instance log storage must accommodate this.

Budget at least 30% of your uplink capacity exclusively for migration traffic, or schedule the full-load phase during off-peak hours when application writes drop below 20% of peak.

34.2 Continuous Replication with AWS DMS

AWS Database Migration Service is the most commonly used tool for migrations into RDS and Aurora. It operates in two phases: full load (bulk copy) and CDC (ongoing change capture). Understanding both phases at the configuration level is what separates a successful migration from one that requires an emergency rollback.

Infrastructure Setup

Provision the DMS replication instance, endpoints, and task via Terraform to keep the configuration reproducible:

```
# Terraform: DMS replication instance and task configuration
resource "aws_dms_replication_instance" "migration" {
  replication_instance_id   = "prod-migration-ri"
  replication_instance_class = "dms.r5.4xlarge"
  allocated_storage         = 500                # GB - covers CDC buffer
  multi_az                  = false               # Single-AZ is fine; this is transient
  publicly_accessible       = false
  vpc_security_group_ids    = [var.migration_sg_id]
  replication_subnet_group_id = aws_dms_replication_subnet_group.migration.id
  engine_version             = "3.5.2"
  auto_minor_version_upgrade = false

  tags = {
    Environment = "migration"
    Owner       = "dba-team"
  }
}

resource "aws_dms_endpoint" "source_pg" {
  endpoint_id   = "source-postgres-prod"
  endpoint_type = "source"
  engine_name   = "postgres"

  username      = var.source_pg_user
  password      = var.source_pg_password
  server_name    = var.source_pg_host
  port          = 5432
  database_name = var.source_db_name
  ssl_mode      = "require"

  postgres_settings {
    capture_ddls          = false # DDL capture is experimental; manage DDL separately
    max_file_size         = 32768
    heartbeat_enable      = true
    heartbeat_frequency   = 5
    heartbeat_schema      = "dms_heartbeat"
    slot_name             = "dms_migration_slot"
    plugin_name           = "pglogical"
    execute_timeout       = 60000
    after_connect_script  = "SET search_path TO '$user', public"
  }
}
```

```

    }
  }

  resource "aws_dms_endpoint" "target_aurora" {
    endpoint_id      = "target-aurora-postgres"
    endpoint_type    = "target"
    engine_name      = "aurora-postgresql"

    username         = var.aurora_admin_user
    password         = var.aurora_admin_password
    server_name      = var.aurora_cluster_endpoint
    port             = 5432
    database_name    = var.target_db_name
    ssl_mode         = "require"
  }

  resource "aws_dms_replication_task" "full_load_cdc" {
    replication_task_id      = "prod-migration-full-cdc"
    migration_type           = "full-load-and-cdc"
    replication_instance_arn = aws_dms_replication_instance.migration.replication_instance_arn
    source_endpoint_arn      = aws_dms_endpoint.source_pg.endpoint_arn
    target_endpoint_arn      = aws_dms_endpoint.target_aurora.endpoint_arn

    table_mappings = jsonencode({
      rules = [
        {
          rule-type = "selection"
          rule-id   = "1"
          rule-name = "include-all"
          object-locator = {
            schema-name = "%"
            table-name  = "%"
          }
          rule-action = "include"
        },
        {
          rule-type = "selection"
          rule-id   = "2"
          rule-name = "exclude-audit-log"
          object-locator = {
            schema-name = "public"
            table-name  = "audit_log_archive"
          }
          rule-action = "exclude"
        }
      ]
    })

    replication_task_settings = jsonencode({
      TargetMetadata = {
        TargetSchema      = ""
        SupportLobs        = true
        FullLobMode        = false
        LobChunkSize       = 64
        LimitedSizeLobMode = true
        LobMaxSize         = 32768
      }
      FullLoadSettings = {
        TargetTablePrepMode      = "DO_NOTHING" # Schema pre-created via SCT
        CreatePkAfterFullLoad    = false
        StopTaskCachedChangesApplied = false
        StopTaskCachedChangesNotApplied = false
        MaxFullLoadSubTasks      = 8
        TransactionConsistencyTimeout = 600
        CommitRate                = 50000
      }
      Logging = {
        EnableLogging = true
        LogComponents = [
          { Id = "SOURCE_UNLOAD", Severity = "LOGGER_SEVERITY_DEFAULT" },

```

```

    { Id = "TARGET_LOAD",    Severity = "LOGGER_SEVERITY_DEFAULT" },
    { Id = "TASK_MANAGER",   Severity = "LOGGER_SEVERITY_DEFAULT" }
  ]
}
ControlTablesSettings = {
  historyTimeslotInMinutes = 5
  StatusTableEnabled       = true
  SuspendedTablesTableEnabled = true
  replicationStatusTableEnabled = true
}
})
}

```

Critical DMS Configuration Details

`TargetTablePrepMode = "DO_NOTHING"` means DMS assumes you have pre-created the schema using the AWS Schema Conversion Tool or a manual DDL script. Never use `DROP_AND_CREATE` in a zero-downtime scenario—it will truncate target tables if the task is restarted.

`CreatePkAfterFullLoad = false` means primary keys must exist before full load begins. This is the correct setting when you pre-create schema. If you set it to `true`, DMS drops constraints during load and recreates them afterward, which significantly speeds up large table loads but creates a window where the target has no constraints.

`MaxFullLoadSubTasks = 8` controls parallel table loading. On a `dms.r5.4xlarge` replication instance, 8 parallel subtasks is a reasonable default. Increasing this beyond 16 rarely improves throughput and increases memory pressure on the replication instance.

Monitoring DMS Lag

Once full load completes and CDC begins, lag monitoring is your primary operational responsibility. DMS exposes `CDCLatencySource` and `CDCLatencyTarget` via CloudWatch. `CDCLatencySource` is the lag between a change occurring in the source WAL and DMS reading it. `CDCLatencyTarget` is the lag between DMS reading the change and applying it to the target. A healthy migration keeps both metrics below 5 seconds during peak and below 30 seconds during business hours for non-time-sensitive migrations.

```

# Monitor DMS CDC latency via AWS CLI
aws cloudwatch get-metric-statistics \
  --namespace AWS/DMS \
  --metric-name CDCLatencySource \
  --dimensions Name=ReplicationInstanceIdentifier,Value=prod-migration-ri \
  --start-time "$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ)" \
  --end-time "$(date -u +%Y-%m-%dT%H:%M:%SZ)" \
  --period 60 \
  --statistics Average,Maximum \
  --output table

# Check replication task status
aws dms describe-replication-tasks \
  --filters Name=replication-task-arn,Values=<task-arn> \
  --query
  ↪ 'ReplicationTasks[0].{Status:Status,FullLoadProgress:ReplicationTaskStats.FullLoadProgressPercent,TablesLoaded:Repl
  ↪ \
  --output table

```

PostgreSQL Source: WAL and Replication Slot Management

DMS creates a PostgreSQL logical replication slot on the source when using `pglogical` or `pgoutput` plugins. This slot is one of the most operationally dangerous objects in your source database during migration. If DMS falls behind or stops, PostgreSQL will retain WAL segments to satisfy the slot,

and your source disk will fill.

Set `max_slot_wal_keep_size` in PostgreSQL 13+ to cap this:

```
-- PostgreSQL 13+: Limit WAL retained for replication slots
ALTER SYSTEM SET max_slot_wal_keep_size = '50GB';
SELECT pg_reload_conf();

-- Monitor slot lag continuously during migration
SELECT
    slot_name,
    plugin,
    slot_type,
    active,
    active_pid,
    pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal,
    pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn)) AS pending_wal,
    EXTRACT(EPOCH FROM (now() - pg_stat_replication.write_lag))::int AS write_lag_seconds
FROM pg_replication_slots
LEFT JOIN pg_stat_replication USING (pid)
WHERE slot_name LIKE 'dms%';
```

If `retained_wal` exceeds 20 GB on a source with limited disk, investigate immediately. If DMS has stopped and you cannot restart it within minutes, you will need to either increase disk on the source or drop the slot and restart the task from the beginning.

34.3 Schema Conversion and Heterogeneous Migrations

When the source and target engines differ—Oracle or SQL Server migrating to PostgreSQL or Aurora—schema conversion is a migration project in itself. AWS Schema Conversion Tool (SCT) handles this partially. The “partially” is what you need to manage.

SCT Assessment Report

SCT classifies every schema object into three tiers: automatically convertible, with issues (requiring manual intervention), and unsupported. In Oracle-to-Aurora-PostgreSQL migrations, the general pattern is:

- Simple tables and views convert automatically in the large majority of cases
- Stored procedures and functions convert automatically far less consistently — the proportion depends heavily on how much the source PL/SQL relies on Oracle-specific packages, cursors, and exception handling; run the SCT assessment report against your own schema rather than assuming a conversion rate
- Oracle-specific types (XMLTYPE, spatial types, nested tables): manual conversion required
- Database links and synonyms: not converted, must be refactored at the application layer

For SQL Server to Azure SQL migrations, the conversion rate is higher because Microsoft designed Azure SQL to be a near-superset of SQL Server on-prem. The blockers are typically CLR user-defined types, cross-database transactions, and FILESTREAM columns.

Running SCT from CLI for Automation

SCT has a GUI that most DBAs use interactively, but it also has a command-line interface for automation in migration pipelines:

```
#!/bin/bash
# Run AWS SCT assessment in headless mode
```

```

# Prerequisites: SCT installed, Java 11+

SCT_HOME="/opt/aws-schema-conversion-tool"
PROJECT_FILE="migration_project.sct"
REPORT_DIR="/var/migration/reports/$(date +%Y%m%d)"

mkdir -p "$REPORT_DIR"

# Generate assessment report without applying conversion
java -jar "${SCT_HOME}/AWSSchemaConversionTool.jar" \
  --project-file "${PROJECT_FILE}" \
  --assess \
  --output-folder "${REPORT_DIR}" \
  --report-name "migration_assessment" \
  --log-file "${REPORT_DIR}/sct.log" \
  --override-settings \
    sourceType=ORACLE \
    targetType=AURORA_POSTGRESQL \
    assessmentReportFormats=PDF,CSV

# Parse blockers from CSV report
echo "=== HIGH-ACTION ITEMS ==="
awk -F',' ' $5 == "Action Required" ' \
  "${REPORT_DIR}/migration_assessment.csv" | \
  cut -d',' -f1,2,3,6 | head -50

echo "=== CONVERSION SUMMARY ==="
grep -E "(Total|Automatically|Manually|Could not)" \
  "${REPORT_DIR}/migration_assessment.csv"

```

Handling Oracle Sequences in Aurora PostgreSQL

Oracle sequences and Aurora PostgreSQL sequences behave differently under concurrent load mainly because of their default cache size, not because of transactional semantics — Oracle's default `CACHE` is 20, while PostgreSQL's default `CACHE` is 1, meaning every `NEXTVAL` call updates the sequence's on-disk state unless you raise the cache explicitly. When converting Oracle applications that rely on high-frequency sequence calls, replace them with PostgreSQL sequences that use a comparably large cache value:

```

-- Aurora PostgreSQL: Replace Oracle-style sequence with cached equivalent
-- Oracle default: INCREMENT BY 1, CACHE 20
-- Under high concurrency, this causes sequence contention on Aurora

-- Correct approach for high-throughput OLTP on Aurora
CREATE SEQUENCE order_id_seq
  START WITH 1
  INCREMENT BY 1
  CACHE 1000          -- Oracle's 20 is too low for Aurora's parallel connections
  NO CYCLE;

-- Verify sequence cache in pg_sequences
SELECT
  schemaname,
  sequencename,
  start_value,
  increment_by,
  cache_size,
  last_value
FROM pg_sequences
WHERE sequencename = 'order_id_seq';

-- For UUID-based primary keys (preferred on Aurora for distributed workloads)
-- Replace integer sequences entirely
ALTER TABLE orders
  ALTER COLUMN order_id SET DEFAULT gen_random_uuid();

```

The cache value of 1,000 is not arbitrary. On Aurora with 200+ concurrent connections each calling

NEXTVAL on every INSERT, a cache of 20 causes the sequence to flush to storage every 20 calls. At 500 INSERTs/second across 200 connections, that is 25 storage flushes per second for sequences alone. A cache of 1,000 reduces this to one flush per connection per 1,000 inserts.

34.4 Azure and GCP Migration Paths

AWS DMS gets the most industry attention, but Azure Database Migration Service and GCP Database Migration Service each have distinct characteristics that affect cutover strategy.

Azure Database Migration Service

Azure DMS supports online (continuous sync) migration for PostgreSQL to Azure Database for PostgreSQL - Flexible Server and SQL Server to Azure SQL Managed Instance. For PostgreSQL sources, it uses the logical replication protocol—the same mechanism as DMS—but its configuration surface is smaller and its limitations are more restrictive.

Azure DMS online migration requires that the source PostgreSQL instance have `wal_level = logical`, `max_replication_slots >= 5`, `max_wal_senders >= 10`, and that `pg_hba.conf` allows the Azure DMS IP range with replication privilege. In a managed source environment where you cannot freely edit `postgresql.conf`, this becomes an operational constraint.

Set up an Azure DMS migration project via CLI:

```
#!/bin/bash
# Azure CLI: Create and start an online PostgreSQL migration to Flexible Server

RESOURCE_GROUP="rg-migration-prod"
LOCATION="eastus2"
DMS_SERVICE="dms-prod-migration"
PROJECT_NAME="pg-to-flexible-migration"
TASK_NAME="pg-migration-task-01"

# Create DMS service instance (Premium tier required for online migration)
az dms create \
  --resource-group "$RESOURCE_GROUP" \
  --name "$DMS_SERVICE" \
  --location "$LOCATION" \
  --sku-name "Premium_4vCores" \
  --subnet "/subscriptions/${SUBSCRIPTION_ID}/resourceGroups/${RESOURCE_GROUP}/\
    ↪ /providers/Microsoft.Network/virtualNetworks/vnet-migration/subnets/snet-dms"

# Create migration project
az dms project create \
  --resource-group "$RESOURCE_GROUP" \
  --service-name "$DMS_SERVICE" \
  --name "$PROJECT_NAME" \
  --source-platform "PostgreSQL" \
  --target-platform "AzureDbForPostgreSql" \
  --location "$LOCATION"

# Create task with source and target connection info
az dms project task create \
  --resource-group "$RESOURCE_GROUP" \
  --service-name "$DMS_SERVICE" \
  --project-name "$PROJECT_NAME" \
  --name "$TASK_NAME" \
  --task-type "OnlineMigration" \
  --source-connection-json '{
    "serverName": "'"$SOURCE_PG_HOST"'",
    "serverPort": 5432,
```

```

    "databaseName": "'"$SOURCE_DB"'",
    "userName": "migration_user",
    "password": "'"$SOURCE_PG_PASS"'",
    "encryptConnection": true,
    "trustServerCertificate": false
  }' \
--target-connection-json '{
  "serverName": "'"$AZURE_FLEX_HOST"'",
  "serverPort": 5432,
  "databaseName": "'"$TARGET_DB"'",
  "userName": "adminuser",
  "password": "'"$AZURE_FLEX_PASS"'",
  "encryptConnection": true
}' \
--selected-databases '[{
  "name": "'"$SOURCE_DB"'",
  "targetDatabaseName": "'"$TARGET_DB"'",
  "migrationSetting": {
    "desiredRPOs": 0
  },
  "sourceSetting": {},
  "targetSetting": {}
}]'

# Monitor migration status
az dms project task show \
--resource-group "$RESOURCE_GROUP" \
--service-name "$DMS_SERVICE" \
--project-name "$PROJECT_NAME" \
--name "$TASK_NAME" \
--expand output \
--query 'output[0].migrationState'

```

GCP Database Migration Service

GCP DMS uses connection profiles and migration jobs as its primary abstractions. For Cloud SQL for PostgreSQL targets, GCP DMS is competitive with AWS DMS in capability. For AlloyDB targets, GCP DMS is the only supported path from external PostgreSQL sources.

```

#!/bin/bash
# GCP CLI: Database Migration Service for PostgreSQL to AlloyDB

PROJECT_ID="prod-migration-project"
REGION="us-central1"
SOURCE_PROFILE="source-postgres-prod"
DEST_PROFILE="target-alloydb-prod"
JOB_ID="pg-to-alloydb-job-01"

# Create source connection profile
gcloud database-migration connection-profiles create postgresql \
--project="$PROJECT_ID" \
--region="$REGION" \
--connection-profile-id="$SOURCE_PROFILE" \
--host="$SOURCE_PG_HOST" \
--port=5432 \
--username="migration_user" \
--password="$SOURCE_PG_PASS" \
--database-service="$SOURCE_DB" \
--display-name="Production PostgreSQL Source"

# Create destination connection profile (AlloyDB)
gcloud database-migration connection-profiles create alloydb \
--project="$PROJECT_ID" \
--region="$REGION" \
--connection-profile-id="$DEST_PROFILE" \
--cluster="projects/${PROJECT_ID}/locations/${REGION}/clusters/alloydb-prod" \
--display-name="AlloyDB Production Target"

# Create migration job (continuous replication)

```

```
gcloud database-migration migration-jobs create \  
  --project="$PROJECT_ID" \  
  --region="$REGION" \  
  --migration-job-id="$JOB_ID" \  
  --type=CONTINUOUS \  
  --source="$SOURCE_PROFILE" \  
  --destination="$DEST_PROFILE" \  
  --display-name="PostgreSQL to AlloyDB Production Migration"  
  
# Verify migration job configuration before starting  
gcloud database-migration migration-jobs verify \  
  --project="$PROJECT_ID" \  
  --region="$REGION" \  
  --migration-job-id="$JOB_ID"  
  
# Start migration (begins full dump followed by CDC)  
gcloud database-migration migration-jobs start \  
  --project="$PROJECT_ID" \  
  --region="$REGION" \  
  --migration-job-id="$JOB_ID"  
  
# Monitor status  
gcloud database-migration migration-jobs describe \  
  --project="$PROJECT_ID" \  
  --region="$REGION" \  
  --migration-job-id="$JOB_ID" \  
  --format="value(state,phase,error.message)"
```

GCP DMS handles the replication slot lifecycle internally for AlloyDB targets. Unlike AWS DMS, you do not need to pre-create the slot or manage its retention manually. However, GCP DMS requires that the source PostgreSQL server is reachable from a GCP-managed IP range, which means configuring `pg_hba.conf` for a CIDR block that GCP publishes and updates periodically. Check the current published ranges before migration and add them to source firewall rules.

GCP DMS Lag Monitoring via Cloud Monitoring

```
# Query CDC replication lag for GCP DMS job using gcloud metrics  
gcloud monitoring metrics list \  
  --filter="metric.type = starts_with('datamigration.googleapis.com')" \  
  --format="value(metric.type)"  
  
# Via Cloud Monitoring API (suitable for integration into alerting pipelines)  
gcloud monitoring dashboards create --config-from-file=dms-dashboard.json  
  
# Direct lag check via DMS describe (includes lag in seconds when available)  
gcloud database-migration migration-jobs describe \  
  --project="$PROJECT_ID" \  
  --region="$REGION" \  
  --migration-job-id="$JOB_ID" \  
  --format="json" | jq '.cdc.maximumLag, .phase, .state'
```

34.5 Cutover Orchestration

Cutover is the moment the migration earns its “zero-downtime” qualifier—or loses it. The mechanics must be scripted, rehearsed, and timed. Improvisation during cutover is the primary cause of unexpected downtime.

The Cutover Decision Gate

Cutover should only proceed when three conditions are simultaneously true:

1. CDC lag on both source and target legs is below your agreed threshold (typically 5 seconds or less).
2. Row counts on the target match the source for all tables (verified by the pre-cutover validation script below).
3. The application team has a validated rollback plan: connection strings pointing back to the source can be restored within your rollback time objective.

Pre-Cutover Row Count Validation

Run this on both source and target immediately before cutover. Any discrepancy halts the cutover.

```
-- PostgreSQL: Generate row counts for all user tables (run on source AND target)
-- Compare output side by side; differences indicate replication gaps

SELECT
    schemaname,
    tablename,
    n_live_tup                               AS estimated_rows,
    pg_size_pretty(pg_total_relation_size(
        schemaname || '.' || tablename
    ))                                       AS table_size,
    last_analyze,
    last_autoanalyze
FROM pg_stat_user_tables
WHERE schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY schemaname, tablename;

-- For exact counts on critical tables (slower but authoritative)
-- Run this for tables where estimated counts diverge between source and target
DO $$
DECLARE
    r RECORD;
    cnt BIGINT;
    sql TEXT;
BEGIN
    FOR r IN
        SELECT schemaname, tablename
        FROM pg_stat_user_tables
        WHERE schemaname = 'public'
        ORDER BY tablename
    LOOP
        sql := format('SELECT COUNT(*) FROM %I.%I', r.schemaname, r.tablename);
        EXECUTE sql INTO cnt;
        RAISE NOTICE 'Table: %.% | Count: %', r.schemaname, r.tablename, cnt;
    END LOOP;
END $$;
```

For SQL Server sources and Azure SQL targets:

```
-- SQL Server / Azure SQL: Row count comparison across all tables
SELECT
    s.name                               AS schema_name,
    t.name                               AS table_name,
    p.rows                               AS row_count,
    SUM(a.total_pages) * 8 / 1024        AS total_size_mb
FROM sys.tables t
JOIN sys.schemas s ON s.schema_id = t.schema_id
JOIN sys.indexes i ON i.object_id = t.object_id
    AND i.index_id IN (0, 1)             -- heap or clustered index
JOIN sys.partitions p ON p.object_id = t.object_id
    AND p.index_id = i.index_id
JOIN sys.allocation_units a ON a.container_id = p.partition_id
WHERE t.is_ms_shipped = 0
GROUP BY s.name, t.name, p.rows
ORDER BY s.name, t.name;
```

The Cutover Runbook

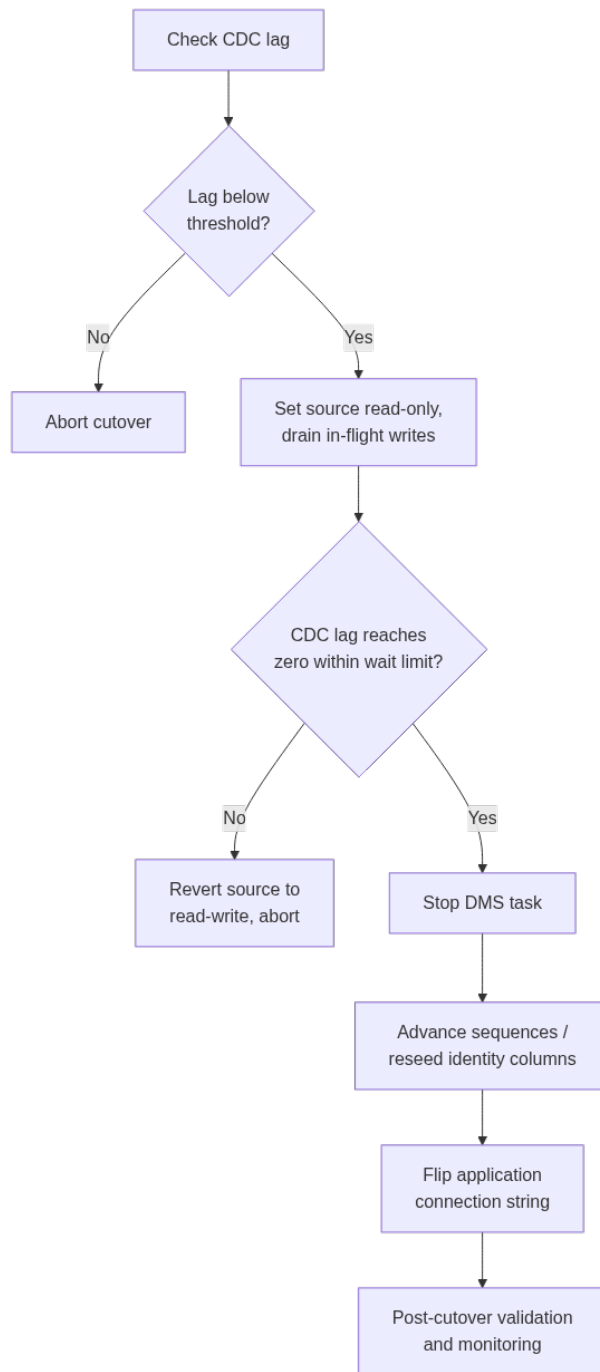


Figure 25: The cutover decision gate: every branch that fails routes back to a safe, reversible state rather than pressing forward.

A production cutover runbook is a sequence of time-boxed steps with a “proceed/abort” decision at each gate. The following template covers a PostgreSQL-to-Aurora cutover via AWS DMS:

```
#!/bin/bash
# cutover_runbook.sh - PostgreSQL to Aurora zero-downtime cutover
# Execute this interactively. Each step requires explicit confirmation.
# Total expected duration: 8-12 minutes from start to traffic on target.

set -euo pipefail

SOURCE_HOST="prod-pg-primary.internal"
SOURCE_DB="appdb"
SOURCE_USER="migration_user"
TARGET_HOST="aurora-cluster.cluster-xxxx.us-east-1.rds.amazonaws.com"
TARGET_DB="appdb"
TARGET_USER="admin"
DMS_TASK_ARN="arn:aws:dms:us-east-1:123456789:task:XXXXXXX"
APP_CONFIG_SSM="/prod/app/db_connection_string"
ROLLBACK_SSM="/prod/app/db_connection_string_rollback"

log() { echo "[$(date -u +%H:%M:%SZ)] $*"; }
confirm() {
    read -r -p "==" $1 [y/N]: " resp
    [[ "${resp,,}" == "y" ]] || { log "ABORTED by operator."; exit 1; }
}

# STEP 1: Verify CDC lag is below threshold
log "STEP 1: Checking DMS CDC lag..."
LAG=$(aws dms describe-replication-tasks \
    --filters Name=replication-task-arn,Values="$DMS_TASK_ARN" \
    --query 'ReplicationTasks[0].ReplicationTaskStats.CDCLatencySource' \
    --output text)

log "Current CDCLatencySource: ${LAG} seconds"
if (( $(echo "$LAG > 10" | bc -l) )); then
    log "ERROR: CDC lag ${LAG}s exceeds 10s threshold. ABORTING."
    exit 1
fi
confirm "CDC lag is acceptable. Proceed to STEP 2?"

# STEP 2: Set source database to read-only to drain in-flight writes
log "STEP 2: Setting source database read_only..."
PGPASSWORD="$SOURCE_PG_PASS" psql -h "$SOURCE_HOST" -U "$SOURCE_USER" -d "$SOURCE_DB" <<'SQL'
-- Terminate active write connections (allow reads to continue)
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = current_database()
    AND state = 'active'
    AND query NOT ILIKE '%select%'
    AND pid <> pg_backend_pid();

-- Set default transaction to read-only for new connections
ALTER DATABASE appdb SET default_transaction_read_only = on;
SQL
log "Source database is now read-only."
confirm "Source is read-only. Proceed to wait for CDC to drain?"

# STEP 3: Wait for CDC lag to reach zero
log "STEP 3: Waiting for CDC lag to reach zero..."
WAIT_LIMIT=120 # seconds
ELAPSED=0
while true; do
    LAG=$(aws dms describe-replication-tasks \
        --filters Name=replication-task-arn,Values="$DMS_TASK_ARN" \
        --query 'ReplicationTasks[0].ReplicationTaskStats.CDCLatencySource' \
        --output text)
    log "CDCLatencySource: ${LAG}s (elapsed: ${ELAPSED}s)"
    if (( $(echo "$LAG <= 1" | bc -l) )); then
        log "CDC lag drained to ${LAG}s. Proceeding."
    fi
done
```

```

        break
    fi
    if (( ELAPSED >= WAIT_LIMIT )); then
        log "ERROR: CDC did not drain within ${WAIT_LIMIT}s. ABORTING."
        # Roll back read-only immediately
        PGPASSWORD="$SOURCE_PG_PASS" psql -h "$SOURCE_HOST" \
            -U "$SOURCE_USER" -d "$SOURCE_DB" \
            -c "ALTER DATABASE appdb SET default_transaction_read_only = off;"
        exit 1
    fi
    sleep 5
    ELAPSED=$(( ELAPSED + 5 ))
done
confirm "CDC drained. Proceed to stop DMS task and flip traffic?"

# STEP 4: Stop DMS task
log "STEP 4: Stopping DMS replication task..."
aws dms stop-replication-task --replication-task-arn "$DMS_TASK_ARN"
log "DMS task stop initiated. Waiting for STOPPED state..."
aws dms wait replication-task-stopped \
    --filters Name=replication-task-arn,Values="$DMS_TASK_ARN"
log "DMS task stopped."

# STEP 5: Flip application connection string in SSM Parameter Store
log "STEP 5: Updating application connection string in SSM..."
NEW_CONN="postgres://${TARGET_USER}@${TARGET_HOST}:5432/${TARGET_DB}?sslmode=require"
aws ssm put-parameter \
    --name "$APP_CONFIG_SSM" \
    --value "$NEW_CONN" \
    --type SecureString \
    --overwrite
log "SSM parameter updated. Application will pick up new connection on next config reload."

# STEP 6: Trigger application config reload (deployment-specific)
log "STEP 6: Triggering application config reload..."
# Example: ECS service update to force new task launch with updated SSM param
aws ecs update-service \
    --cluster prod-cluster \
    --service app-service \
    --force-new-deployment \
    --query 'service.deploymentConfiguration'

confirm "Deployment triggered. Monitor application metrics for 5 minutes before declaring success."

# STEP 7: Post-cutover validation
log "STEP 7: Running post-cutover row count spot check on target..."
PGPASSWORD="$TARGET_PG_PASS" psql -h "$TARGET_HOST" -U "$TARGET_USER" \
    -d "$TARGET_DB" -c "
SELECT schemaname, tablename, n_live_tup
FROM pg_stat_user_tables
WHERE schemaname = 'public'
ORDER BY n_live_tup DESC
LIMIT 20;
"

log "Cutover complete. Monitor dashboards for 30 minutes."
log "Rollback: revert SSM parameter to value stored in ${ROLLBACK_SSM} and re-enable writes on source."

```

Sequence Numbers and Auto-Increment Gaps

After cutover, the target database's sequences will not automatically advance to match the source's current value. If DMS replicated an INSERT with `id = 450000` but the Aurora sequence still starts at 1, the first application-generated INSERT after cutover will collide. Fix this before opening writes on the target:

```

-- Aurora PostgreSQL: Advance all sequences to match source maximum values
-- Run this AFTER CDC drains but BEFORE allowing write traffic

```

```

DO $$
DECLARE
    r RECORD;
    max_val BIGINT;
    seq_name TEXT;
    sql TEXT;
BEGIN
    FOR r IN
        SELECT
            t.table_schema,
            t.table_name,
            c.column_name,
            pg_get_serial_sequence(t.table_schema || '.' || t.table_name, c.column_name) AS seq
        FROM information_schema.tables t
        JOIN information_schema.columns c
            ON c.table_schema = t.table_schema
            AND c.table_name = t.table_name
        WHERE t.table_type = 'BASE TABLE'
            AND t.table_schema = 'public'
            AND c.column_default LIKE 'nextval%'
    LOOP
        IF r.seq IS NOT NULL THEN
            sql := format('SELECT MAX(%I) FROM %I.%I',
                r.column_name, r.table_schema, r.table_name);
            EXECUTE sql INTO max_val;
            IF max_val IS NOT NULL THEN
                PERFORM setval(r.seq, max_val, true);
                RAISE NOTICE 'Advanced sequence % to %', r.seq, max_val;
            END IF;
        END IF;
    END LOOP;
END $$;

```

For SQL Server to Azure SQL migrations, identity columns require a different approach:

```

-- Azure SQL: Reseed identity columns after migration
-- Run against each table where identity column exists on target

DECLARE @table_name NVARCHAR(256);
DECLARE @col_name NVARCHAR(256);
DECLARE @max_val BIGINT;
DECLARE @sql NVARCHAR(MAX);

DECLARE identity_cursor CURSOR FOR
    SELECT
        OBJECT_NAME(ic.object_id) AS table_name,
        COL_NAME(ic.object_id, ic.column_id) AS column_name
    FROM sys.identity_columns ic
    JOIN sys.tables t ON t.object_id = ic.object_id
    WHERE t.is_ms_shipped = 0;

OPEN identity_cursor;
FETCH NEXT FROM identity_cursor INTO @table_name, @col_name;

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @sql = N'SELECT @max = MAX(' + QUOTENAME(@col_name) + N') FROM ' + QUOTENAME(@table_name);
    EXEC sp_executesql @sql, N'@max BIGINT OUTPUT', @max = @max_val OUTPUT;

    IF @max_val IS NOT NULL
    BEGIN
        SET @sql = N'DBCC CHECKIDENT ('' + @table_name + N'', RESEED, ' + CAST(@max_val AS NVARCHAR)
            + N')';
        EXEC sp_executesql @sql;
        PRINT 'Reseeded ' + @table_name + '.' + @col_name + ' to ' + CAST(@max_val AS NVARCHAR);
    END

    FETCH NEXT FROM identity_cursor INTO @table_name, @col_name;
END

```

```
CLOSE identity_cursor;
DEALLOCATE identity_cursor;
```

34.6 Post-Migration Validation

Declaring the migration complete before validation is complete is an operational mistake. The application may be writing to the new database, but that does not mean the data is correct, the performance is acceptable, or the ancillary objects (triggers, functions, views) are functioning correctly.

Data Integrity Validation

Use a hash-based approach for tables where row counts alone are insufficient. Hash comparison on sorted row data detects column-level data corruption that row counts miss:

```
-- PostgreSQL: Hash-based table validation (run on source, compare to target output)
-- Use MD5 for speed; use SHA256 if regulatory requirements demand it

CREATE OR REPLACE FUNCTION validate_table_hash(
    p_schema TEXT,
    p_table TEXT
)
RETURNS TABLE(
    schema_name TEXT,
    table_name TEXT,
    row_count BIGINT,
    content_hash TEXT,
    sampled_at TIMESTAMPTZ
)
LANGUAGE plpgsql AS $$
DECLARE
    v_sql TEXT;
    v_cols TEXT;
BEGIN
    -- Build column list, casting all columns to TEXT for consistent hashing
    SELECT string_agg(
        'COALESCE(' || quote_ident(column_name) || '::TEXT, ''')',
        ' || '' || ' || '
        ORDER BY ordinal_position
    )
    INTO v_cols
    FROM information_schema.columns
    WHERE table_schema = p_schema
        AND table_name = p_table;

    v_sql := format(
        'SELECT %L, %L, COUNT(*), MD5(String_agg(row_hash, '' ' ORDER BY row_hash)), NOW()
        FROM (
            SELECT MD5(%s) AS row_hash
            FROM %I.%I
            ) subq',
        p_schema, p_table, v_cols, p_schema, p_table
    );

    RETURN QUERY EXECUTE v_sql;
END $$;

-- Run for all tables in public schema
SELECT * FROM validate_table_hash('public', 'orders');
SELECT * FROM validate_table_hash('public', 'customers');
SELECT * FROM validate_table_hash('public', 'order_items');
```

This function works for tables with up to tens of millions of rows within a reasonable time window. For tables with hundreds of millions of rows, sample-based validation is more practical:

```
-- PostgreSQL: Sample-based validation for very large tables
-- Compare random 1% sample hash between source and target using same random seed

SELECT
    MD5(String_Agg(row_hash, '' ORDER BY id)) AS sample_hash,
    COUNT(*) AS sample_count
FROM (
    SELECT
        id,
        MD5(COALESCE(id::TEXT, '')
            || COALESCE(customer_id::TEXT, '')
            || COALESCE(total_amount::TEXT, '')
            || COALESCE(status, '')
            || COALESCE(created_at::TEXT, '')) AS row_hash
    FROM orders
    TABLESAMPLE BERNOULLI (1) REPEATABLE (42) -- same seed on source and target
) t;
```

Performance Baseline Validation

After cutover, the top-20 queries by total execution time on the source should perform within an acceptable range on the target. Capture baseline query performance on the source before migration using `pg_stat_statements`:

```
-- PostgreSQL: Capture query performance baseline on SOURCE before migration
-- Export this to a file and compare against target after cutover

SELECT
    LEFT(query, 120) AS query_preview,
    calls,
    ROUND(total_exec_time::numeric, 2) AS total_exec_ms,
    ROUND(mean_exec_time::numeric, 2) AS mean_exec_ms,
    ROUND(stddev_exec_time::numeric, 2) AS stddev_exec_ms,
    rows,
    ROUND(100.0 * total_exec_time /
        SUM(total_exec_time) OVER (), 2) AS pct_total_time,
    shared_blks_hit,
    shared_blks_read
FROM pg_stat_statements
WHERE calls > 100
ORDER BY total_exec_time DESC
LIMIT 30;
```

Run the same query on the target 24 hours post-migration and compare. Queries that are significantly slower on the target typically point to one of three causes:

1. **Missing indexes** — SCT or manual schema migration missed an index, or an index was not yet rebuilt after full load.
2. **Stale statistics** — Aurora runs `ANALYZE` automatically, but freshly loaded tables may not have had time for autovacuum to gather representative statistics. Run `ANALYZE VERBOSE` on affected tables immediately after cutover.
3. **Parameter differences** — the source PostgreSQL instance had tuned `work_mem`, `effective_cache_size`, or `random_page_cost` values that the cloud managed instance does not match. Review and apply equivalent parameter group settings on the target.

Functional Validation: Stored Procedures and Triggers

For heterogeneous migrations (Oracle or SQL Server to PostgreSQL), converted stored procedures require functional testing against the target. Automate this with a suite of stored procedure call tests that compare output between source and target for identical inputs:

```
-- PostgreSQL: Validate converted function returns expected results
-- Run on target after migration; compare output against source baseline captured pre-migration

CREATE OR REPLACE PROCEDURE validate_function_outputs()
LANGUAGE plpgsql AS $$
DECLARE
    test_results JSONB := '[]'::JSONB;
    result_row    RECORD;
BEGIN
    -- Test 1: calculate_order_total function
    FOR result_row IN
        SELECT
            order_id,
            calculate_order_total(order_id)          AS computed_total,
            expected_total,
            ABS(calculate_order_total(order_id)
                - expected_total) < 0.01             AS passed
        FROM migration_validation.expected_function_outputs
        WHERE function_name = 'calculate_order_total'
        LIMIT 1000
    LOOP
        IF NOT result_row.passed THEN
            RAISE WARNING 'FUNCTION MISMATCH: order_id=% computed=% expected=%',
                result_row.order_id,
                result_row.computed_total,
                result_row.expected_total;
        END IF;
    END LOOP;

    RAISE NOTICE 'Function validation complete.';
END $$;

CALL validate_function_outputs();
```

The `migration_validation.expected_function_outputs` table is populated pre-migration by running the same function calls against the source and capturing their outputs. This pattern—capture expected outputs on source, validate actual outputs on target—is the most reliable functional validation approach for converted procedures.

34.7 Rollback Strategy and Post-Migration Cleanup

Zero-downtime migration is not complete without a tested rollback path. The rollback window closes when you cannot reconnect the source to production traffic—either because the source has been decommissioned, the replication slot has been dropped, or the source instance has been scaled down. Keep the source operational and writable for at minimum 72 hours post-cutover, and keep the DMS task in STOPPED (not deleted) state so it can be restarted if needed.

Activating Rollback

If a critical issue is detected post-cutover, the rollback sequence is the reverse of the cutover sequence:

```
#!/bin/bash
# rollback.sh - Emergency rollback from target back to source
# Execute only when declared by the migration lead after post-cutover incident detection.

set -euo pipefail

SOURCE_HOST="prod-pg-primary.internal"
SOURCE_DB="appdb"
SOURCE_USER="migration_user"
```



```

APP_CONFIG_SSM="/prod/app/db_connection_string"
ROLLBACK_CONN_SSM="/prod/app/db_connection_string_rollback"

log() { echo "[$(date -u +%H:%M:%SZ)] ROLLBACK: $*"; }

# STEP R1: Restore source database to read-write
log "R1: Restoring source database to read-write mode..."
PGPASSWORD="$SOURCE_PG_PASS" psql -h "$SOURCE_HOST" -U "$SOURCE_USER" \
  -d "$SOURCE_DB" \
  -c "ALTER DATABASE appdb SET default_transaction_read_only = off;"
log "Source is read-write."

# STEP R2: Revert application connection string
log "R2: Reverting SSM connection string to source..."
ROLLBACK_CONN=$(aws ssm get-parameter \
  --name "$ROLLBACK_CONN_SSM" \
  --with-decryption \
  --query 'Parameter.Value' \
  --output text)
aws ssm put-parameter \
  --name "$APP_CONFIG_SSM" \
  --value "$ROLLBACK_CONN" \
  --type SecureString \
  --overwrite
log "SSM parameter reverted."

# STEP R3: Force application redeployment to pick up old connection
log "R3: Forcing ECS service redeployment..."
aws ecs update-service \
  --cluster prod-cluster \
  --service app-service \
  --force-new-deployment \
  --query 'service.deployments[0].status'

log "Rollback initiated. Verify application health on source database."
log "Do NOT restart DMS task until post-incident review is complete."

```

Post-Migration Cleanup Checklist

After a 72-hour observation window with no issues, decommission migration infrastructure:

1. **Drop the DMS replication slot on the source:** `SELECT pg_drop_replication_slot('dms_migration_...)` — failure to do this will cause WAL accumulation on the source indefinitely.
2. **Delete the DMS task, endpoints, and replication instance** via Terraform destroy or AWS console to stop instance billing.
3. **Remove migration-specific database users** from the source and target.
4. **Drop the `dms_heartbeat` schema** from the source if created during DMS configuration.
5. **Reset `default_transaction_read_only`** on the source to off (if not already done during rollback path).
6. **Archive the SCT project file, migration runbook, and DMS task configuration** to the team's migration artifact repository for audit purposes.
7. **Verify that `max_slot_wal_keep_size`** is reset or removed from the source's `postgresql.conf` if it was added specifically for this migration.

```

-- PostgreSQL: Post-migration cleanup on source database
-- Run after 72-hour observation window confirms migration success

-- 1. Verify no active replication slots remain from migration
SELECT slot_name, active, plugin, restart_lsn
FROM pg_replication_slots
WHERE slot_name LIKE 'dms%';

-- 2. Drop migration slot if inactive
SELECT pg_drop_replication_slot(slot_name)

```

```
FROM pg_replication_slots
WHERE slot_name LIKE 'dms%' AND active = false;

-- 3. Remove migration user
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM migration_user;
REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM migration_user;
ALTER ROLE migration_user WITH NOREPLICATION;    -- REPLICATION is a role attribute, not a grantable
--> privilege
REVOKE rds_replication FROM migration_user;      -- if on RDS source
DROP USER migration_user;

-- 4. Drop heartbeat schema
DROP SCHEMA IF EXISTS dms_heartbeat CASCADE;

-- 5. Confirm WAL retention is back to normal
SELECT name, setting, unit
FROM pg_settings
WHERE name IN ('max_slot_wal_keep_size', 'wal_keep_size', 'max_replication_slots');
```

Key Takeaways

- **Pre-migration assessment is the highest-leverage activity in a zero-downtime migration.** Tables without primary keys, linked server dependencies, and unsupported object types identified before replication begins are solvable problems. Identified during cutover, they become outages.
 - **AWS DMS, Azure DMS, and GCP DMS all use logical replication under the hood, but they expose different operational surfaces.** DMS on AWS requires explicit replication slot management and WAL retention monitoring on the source; GCP DMS manages the slot lifecycle internally; Azure DMS requires `pg_hba.conf` configuration for managed IP ranges. Know which operational responsibility belongs to you versus the service.
 - **Cutover is a scripted, time-boxed procedure, not an ad hoc event.** Setting the source to read-only, draining CDC lag to near-zero, advancing sequences on the target, and flipping the application connection string must each be rehearsed in a staging environment before the production window.
 - **Post-migration validation closes the loop that replication cannot.** Row count checks confirm volume, hash-based comparisons confirm content integrity, and query performance baselines confirm that the target is behaviorally equivalent to the source under production load.
 - **The rollback window is a contractual obligation, not an afterthought.** Keep the source database operational and writable, the DMS task stopped but not deleted, and the rollback connection string pre-staged in your secrets store for a minimum of 72 hours post-cutover. The cost of maintaining this rollback posture is far lower than the cost of an unrecoverable production incident on a freshly migrated database.
-

Chapter 35: Query Performance Tuning on Managed Services

Query performance tuning on managed cloud databases is fundamentally the same discipline you’ve practiced for years — identify the slow query, understand the execution plan, fix the root cause — but the environment imposes constraints that change how you investigate, what levers you can pull, and where the bottlenecks actually live. You can’t attach a profiler directly to the OS, you can’t touch the kernel scheduler, and you can’t install arbitrary extensions. What you get instead is a set of cloud-native observability tools, managed parameter systems, and platform-specific optimizations that, once mastered, give you most of what you need. This chapter covers the full tuning workflow on AWS RDS/Aurora, Azure SQL, and Google Cloud SQL/AlloyDB — from capturing slow queries to fixing execution plans to understanding where managed service limits actually bite you in production.

35.1 Capturing and Identifying Slow Queries

The first step in any tuning engagement is visibility: find the queries causing problems before you guess at solutions. On self-managed PostgreSQL, you enabled `pg_stat_statements`, tailed the slow query log, or ran `EXPLAIN ANALYZE` in a session. On managed services, the same data exists but reaches you through different channels.

AWS RDS and Aurora (PostgreSQL)

On RDS PostgreSQL, slow query logging is controlled through the parameter group. The critical parameters are `log_min_duration_statement` (in milliseconds), `log_statement`, and `pg_stat_statements.track`. You cannot edit the parameter group in the console and expect it to take effect instantly on production without a reboot for static parameters — but `log_min_duration_statement` is dynamic, so you can apply it without restart.

```
# Modify the RDS parameter group to enable slow query logging
aws rds modify-db-parameter-group \
  --db-parameter-group-name prod-postgres15-params \
  --parameters \
    "ParameterName=log_min_duration_statement,ParameterValue=1000,ApplyMethod=immediate" \
    "ParameterName=log_statement,ParameterValue=none,ApplyMethod=immediate" \
    "ParameterName=log_lock_waits,ParameterValue=1,ApplyMethod=immediate" \
    "ParameterName=log_temp_files,ParameterValue=10240,ApplyMethod=immediate"

# Verify the change took effect without a reboot
aws rds describe-db-parameters \
  --db-parameter-group-name prod-postgres15-params \
```

```
--query "Parameters[?ParameterName=='log_min_duration_statement']"
```

Logs are streamed to CloudWatch Logs automatically if you have `EnableCloudwatchLogsExports` set. From there, CloudWatch Logs Insights lets you run structured queries against the raw log text:

```
# Query CloudWatch Logs Insights for the top slow queries in the last hour
aws logs start-query \
  --log-group-name /aws/rds/instance/prod-db-01/postgresql \
  --start-time $(date -d '1 hour ago' +%s) \
  --end-time $(date +%s) \
  --query-string '
    fields @timestamp, @message
    | filter @message like /duration:/
    | parse @message "duration: * ms statement: *" as duration_ms, statement
    | stats avg(duration_ms) as avg_ms, max(duration_ms) as max_ms, count() as executions
      by statement
    | sort max_ms desc
    | limit 25
  '
```

This gives you query text, average duration, and execution count directly from CloudWatch — no SSH session required.

For RDS SQL Server, Performance Insights is the primary tool. It surfaces the Average Active Sessions (AAS) metric, broken down by wait type, SQL statement, host, and user. This is the managed equivalent of querying `sys.dm_exec_query_stats` combined with `sys.dm_os_wait_stats`. Enable it at instance creation or modify the instance to turn it on:

```
aws rds modify-db-instance \
  --db-instance-identifier prod-sqlserver-01 \
  --enable-performance-insights \
  --performance-insights-retention-period 731 \
  --apply-immediately
```

Performance Insights retains 7 days free; extending retention up to the 731-day maximum incurs an additional charge billed per vCPU-hour, which scales with both instance size and how far back you extend the window. Check current AWS pricing for your region before enabling extended retention fleet-wide — on a large multi-vCPU instance it becomes a meaningful recurring line item.

Azure SQL Database

Azure SQL surfaces query performance data through Query Store, which is enabled by default on Azure SQL Database (unlike on-premises SQL Server where you had to enable it manually). The Query Store captures execution plans, runtime statistics, and wait statistics automatically. Combine this with the Query Performance Insight blade in the Azure portal or query it directly:

```
-- Top 20 queries by total CPU time in the last 24 hours from Query Store
SELECT TOP 20
  q.query_id,
  qt.query_sql_text,
  rs.count_executions,
  rs.avg_duration / 1000.0 AS avg_duration_ms,
  rs.avg_cpu_time / 1000.0 AS avg_cpu_ms,
  rs.avg_logical_io_reads,
  rs.avg_rowcount,
  CAST(rs.avg_cpu_time * rs.count_executions / 1e6 AS DECIMAL(10,2)) AS total_cpu_seconds
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
JOIN sys.query_store_runtime_stats_interval rsi ON rs.runtime_stats_interval_id =
  ↳ rsi.runtime_stats_interval_id
WHERE rsi.start_time >= DATEADD(HOUR, -24, GETUTCDATE())
ORDER BY total_cpu_seconds DESC;
```

Google Cloud SQL and AlloyDB

Cloud SQL for PostgreSQL streams logs to Cloud Logging. Enable `log_min_duration_statement` through a database flag on the instance:

```
# Set slow query threshold to 500ms on Cloud SQL PostgreSQL
gcloud sql instances patch prod-cloudsql-pg \
  --database-flags \
    log_min_duration_statement=500,\
    log_lock_waits=on,\
    log_temp_files=10240,\
    track_io_timing=on

# Query slow queries from Cloud Logging using the logging API
gcloud logging read \
  'resource.type="cloudsql_database" AND
  textPayload=~"duration:" AND
  resource.labels.database_id="my-project:prod-cloudsql-pg"' \
  --limit=50 \
  --format="value(textPayload)" \
  --freshness=1h
```

AlloyDB adds the `google_columnar_engine` extension for OLAP acceleration and exposes query insights through the AlloyDB console UI and Cloud Monitoring. The `pg_stat_statements` view is available on both, and AlloyDB's Query Insights surfaces plan regressions automatically — a capability you would have built yourself with a custom baseline table in a self-managed environment.

35.2 Reading Execution Plans on Managed Services

Getting an execution plan on a managed service is mechanically simple, but interpreting the plan in the context of managed infrastructure requires calibrating your assumptions. The optimizer is the same PostgreSQL or SQL Server optimizer you know — but the cost model is tuned differently, statistics maintenance behaves differently, and some plan choices are shaped by the storage architecture.

PostgreSQL Plans on RDS and Aurora

`EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)` works exactly as it does on self-managed PostgreSQL. The difference on Aurora is in what the buffer counts mean: Aurora uses a distributed storage layer, so buffer hits that look like shared cache hits may actually be served from the Aurora buffer pool, while misses trigger Aurora storage I/O rather than OS page cache misses. The I/O cost model in the planner doesn't fully account for Aurora's storage characteristics, which sometimes causes the planner to prefer sequential scans when an index seek would be faster on spinning disk, because Aurora's storage is effectively SSD-backed with 6-way replication — the sequential scan penalty is lower than what the `seq_page_cost` parameter assumes.

```
-- Capture a plan with full detail and serialize it for storage
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, FORMAT JSON)
SELECT
  o.order_id,
  o.customer_id,
  o.order_date,
  SUM(oi.quantity * oi.unit_price) AS order_total
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.order_date >= '2024-01-01'
  AND o.status = 'COMPLETED'
GROUP BY o.order_id, o.customer_id, o.order_date
```

```

HAVING SUM(oi.quantity * oi.unit_price) > 500
ORDER BY order_total DESC
LIMIT 100;

```

When you see a sequential scan on a large table and suspect the planner is making the wrong choice, check the statistics before blaming the plan:

```

-- Check when statistics were last collected and how accurate they are
SELECT
    schemaname,
    tablename,
    n_live_tup,
    n_dead_tup,
    last_autoanalyze,
    last_analyze,
    last_autoanalyze - now() AS time_since_autoanalyze,
    n_mod_since_analyze
FROM pg_stat_user_tables
WHERE tablename IN ('orders', 'order_items')
ORDER BY n_mod_since_analyze DESC;

-- Check column statistics for a specific column
SELECT
    attname,
    n_distinct,
    correlation,
    most_common_vals,
    most_common_freqs
FROM pg_stats
WHERE tablename = 'orders'
    AND attname IN ('status', 'order_date');

```

On RDS, `autovacuum` runs under the managed service’s control. You can tune its parameters through the parameter group, but you cannot run `VACUUM` manually with OS-level priority settings or kill the `autovacuum` workers through system commands. If statistics are stale on a large partitioned table during a bulk load window, you can run `ANALYZE orders;` manually as the database superuser — RDS grants you `rds_superuser` which has sufficient privileges for this.

Aurora-Specific Plan Considerations

Aurora PostgreSQL’s parallel query feature (available on Aurora v3, which is PostgreSQL 15-compatible) can push query processing down to the distributed storage layer. When enabled, plans show `Parallel Seq Scan` nodes that indicate storage-level parallelism. This is structurally different from PostgreSQL’s parallel workers, which run on the compute node. You can control whether a query uses Aurora parallel query by setting `aurora_parallel_query`:

```

-- Enable Aurora parallel query for a specific session and compare plans
SET aurora_parallel_query = ON;
EXPLAIN SELECT COUNT(*), AVG(total_amount) FROM orders WHERE order_date >= '2023-01-01';

SET aurora_parallel_query = OFF;
EXPLAIN SELECT COUNT(*), AVG(total_amount) FROM orders WHERE order_date >= '2023-01-01';

```

Parallel query shows the largest gains on full table scans of tables larger than 1 million rows where the result set fits in memory after aggregation. For OLTP queries hitting indexes, it adds overhead.

SQL Server Plans on Azure SQL

Azure SQL Database uses the same execution plan XML format as SQL Server on-premises. The key difference is that `sys.dm_exec_query_plan` and `sys.dm_exec_query_stats` are available, but `sys.dm_os_ring_buffers` and some internal DMVs are restricted. For plan capture and analysis, use Query Store rather than ad hoc DMV queries — Query Store persists plans across failovers,

which DMV caches do not.

To force a specific plan (the equivalent of using a plan guide on-premises), Azure SQL supports Query Store plan forcing:

```
-- Force a specific plan for a regressed query using Query Store
-- First, identify the query_id and the plan_id of the good historical plan
SELECT
    q.query_id,
    p.plan_id,
    rs.avg_duration,
    rs.count_executions,
    p.query_plan
FROM sys.query_store_query q
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
WHERE q.query_id = 1547
ORDER BY rs.last_execution_time DESC;

-- Force the known-good plan
EXEC sys.sp_query_store_force_plan
    @query_id = 1547,
    @plan_id = 892;

-- Verify the forcing is active
SELECT
    q.query_id,
    p.plan_id,
    p.is_forced_plan,
    p.force_failure_count,
    p.last_force_failure_reason_desc
FROM sys.query_store_query q
JOIN sys.query_store_plan p ON q.query_id = p.query_id
WHERE q.query_id = 1547;
```

Plan forcing in Query Store survives database restarts and failovers on Azure SQL — this is a meaningful improvement over on-premises plan guides, which you had to manage separately.

35.3 Index Tuning in the Cloud: What Works and What Doesn't

Index tuning on managed services is conceptually identical to on-premises: add missing indexes, remove unused ones, choose the right index type for the access pattern. The cloud layer changes the operational mechanics — how you identify candidates, how you apply changes safely, and what the platform does automatically.

AWS RDS PostgreSQL: `pg_stat_user_indexes` and Missing Index Detection

PostgreSQL has no built-in missing index advisor. On self-managed systems, you'd combine `pg_stat_statements` with EXPLAIN output and `pg_stat_user_indexes` to find candidates. On RDS, you have the same data plus the option to use the RDS Performance Insights API to correlate wait events with specific queries:

```
-- Find indexes that are never used (candidates for removal)
-- Run this after a representative workload period (at least 7 days post-restart)
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch,
```

```

    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE idx_scan = 0
    AND schemaname NOT IN ('pg_catalog', 'information_schema')
ORDER BY pg_relation_size(indexrelid) DESC;

-- Find sequential scans on large tables that might benefit from an index
SELECT
    schemaname,
    relname AS tablename,
    seq_scan,
    seq_tup_read,
    idx_scan,
    n_live_tup,
    pg_size_pretty(pg_total_relation_size(relid)) AS total_size,
    ROUND(seq_scan::NUMERIC / NULLIF(seq_scan + idx_scan, 0) * 100, 2) AS seq_scan_pct
FROM pg_stat_user_tables
WHERE n_live_tup > 100000
    AND seq_scan > 100
ORDER BY seq_tup_read DESC
LIMIT 20;

```

A gotcha on RDS: `pg_stat_user_indexes.idx_scan` resets when the instance restarts. RDS Multi-AZ failovers cause a restart on the new primary. If you collected baseline data before a recent failover, your unused index candidates may be incorrect. Always check `pg_stat_bgwriter.stats_reset` to see when statistics were last reset:

```
SELECT stats_reset FROM pg_stat_bgwriter;
```

Creating Indexes Concurrently

`CREATE INDEX CONCURRENTLY` works on RDS and Cloud SQL exactly as it does on-premises — it builds the index without holding an `AccessExclusiveLock` on the table, so production reads and writes continue. The gotcha is that if the concurrent build fails (connection drop, out-of-disk-space), it leaves an invalid index behind that you must drop manually. On Aurora, an invalid index in `pg_index.indisvalid = false` will not be used by the planner but still slows down DML because the executor still attempts to update it.

```

-- Build a partial index concurrently for the active order pattern
CREATE INDEX CONCURRENTLY idx_orders_active_date
ON orders (customer_id, order_date DESC)
WHERE status IN ('PENDING', 'PROCESSING');

-- Verify the index built successfully
SELECT
    indexname,
    indisvalid,
    indisready
FROM pg_indexes
JOIN pg_index ON indexrelid = (
    SELECT oid FROM pg_class WHERE relname = 'idx_orders_active_date'
)
WHERE tablename = 'orders'
    AND indexname = 'idx_orders_active_date';

```

Azure SQL: Automatic Index Recommendations

Azure SQL Database includes an automatic tuning engine that recommends and — if you enable it — automatically creates and drops indexes based on Query Store data. This is the cloud equivalent of the Database Engine Tuning Advisor, but it runs continuously and learns from production workload patterns.

```

# Check current automatic tuning state via Azure CLI
az sql db show \

```



```
--resource-group prod-rg \  
--server prod-sql-server \  
--name prod-database \  
--query "autoPauseDelay, automaticTuning"  
  
# Enable automatic index creation recommendations (monitoring only, not auto-apply)  
az sql db update \  
--resource-group prod-rg \  
--server prod-sql-server \  
--name prod-database \  
--auto-tuning-mode Auto
```

The recommendation: enable automatic tuning in Auto mode for index recommendations on read-heavy OLAP databases. For transactional OLTP databases where index creation causes brief write stalls and where any schema change needs a change control approval, leave it in recommendation-only mode and apply changes manually during maintenance windows.

AlloyDB: Columnar Engine for HTAP Workloads

AlloyDB's columnar engine maintains a column store in memory alongside the row store. When a query would benefit from columnar access, the planner routes it to the columnar engine automatically. You can check which tables have columnar data populated and force or hint columnar scans:

```
-- Check columnar engine population status  
SELECT  
    database_name,  
    schema_name,  
    table_name,  
    column_count,  
    row_count,  
    memory_size_in_bytes / 1024 / 1024 AS memory_mb  
FROM g_columnar_relations;  
  
-- Verify a query is using the columnar engine  
EXPLAIN SELECT  
    product_category,  
    SUM(revenue) AS total_revenue,  
    AVG(margin_pct) AS avg_margin  
FROM sales_facts  
WHERE sale_date BETWEEN '2024-01-01' AND '2024-12-31'  
GROUP BY product_category;  
-- Look for "Custom Scan (columnar scan)" in the plan output
```

35.4 Wait Event Analysis and Resource Bottleneck Identification

Experienced DBAs know that execution time alone is incomplete — a query that runs in 10ms but waits 2 seconds for a lock isn't a query problem, it's a concurrency problem. Cloud managed services surface wait events through different mechanisms than on-premises but with equivalent depth.

RDS Performance Insights Wait Event Analysis

Performance Insights breaks down Average Active Sessions by wait event type. For PostgreSQL, the wait event categories map directly to `pg_stat_activity.wait_event_type` values: LWLock, Lock, BufferPin, IO, Client. For SQL Server, they map to SQL Server wait types: LCK_M_X, PAGEIOLATCH_SH, ASYNC_NETWORK_IO.

Use the Performance Insights API to pull wait data programmatically for trend analysis:

```

import boto3
from datetime import datetime, timedelta, timezone

pi_client = boto3.client('pi', region_name='us-east-1')

# Pull top wait events for the last 3 hours at 1-minute granularity
response = pi_client.get_resource_metrics(
    ServiceType='RDS',
    Identifier='db:prod-db-01', # Performance Insights resource ID
    StartTime=datetime.now(timezone.utc) - timedelta(hours=3),
    EndTime=datetime.now(timezone.utc),
    PeriodInSeconds=60,
    MetricQueries=[
        {
            'Metric': 'db.load.avg',
            'GroupBy': {
                'Group': 'db.wait_event',
                'Dimensions': ['db.wait_event.type', 'db.wait_event.name'],
                'Limit': 10
            }
        }
    ]
)

for metric_list in response['MetricList']:
    key = metric_list['Key']
    wait_type = key.get('Dimensions', {}).get('db.wait_event.type', 'unknown')
    wait_name = key.get('Dimensions', {}).get('db.wait_event.name', 'unknown')

    # Calculate average load contribution for this wait event
    data_points = metric_list['DataPoints']
    if data_points:
        avg_load = sum(dp.get('Value', 0) for dp in data_points) / len(data_points)
        print(f"{wait_type}/{wait_name}: avg AAS contribution = {avg_load:.3f}")

```

On Aurora PostgreSQL, the most common problematic wait events in production:

- IO:DataFileRead — Buffer pool misses, often from missing indexes or OLAP queries on OLTP instances
- Lock:relation — DDL blocking DML or long-running transactions holding table locks
- LWLock:buffer_content — High-concurrency UPDATE hot spots (counter tables, queue tables)
- Client:ClientRead — Network latency between application and database; often a VPC placement issue or RDS Proxy connection overhead

Azure SQL: sys.dm_os_wait_stats and Extended Events

On Azure SQL, `sys.dm_os_wait_stats` is available with some restrictions — the `SQLTRACE_*` and `DBMIRROR_*` wait types are filtered out as they're internal to the platform. For real-time wait analysis, Extended Events replace SQL Trace (Profiler is unsupported on Azure SQL):

```

-- Create a lightweight Extended Events session to capture query waits
CREATE EVENT SESSION [prod_wait_analysis] ON DATABASE
ADD EVENT sqlserver.sql_statement_completed (
    ACTION (
        sqlserver.sql_text,
        sqlserver.query_hash,
        sqlserver.database_name,
        sqlserver.username
    )
    WHERE (
        duration > 500000 -- 500ms in microseconds
        AND sqlserver.database_name = N'ProdDB'
    )
),

```

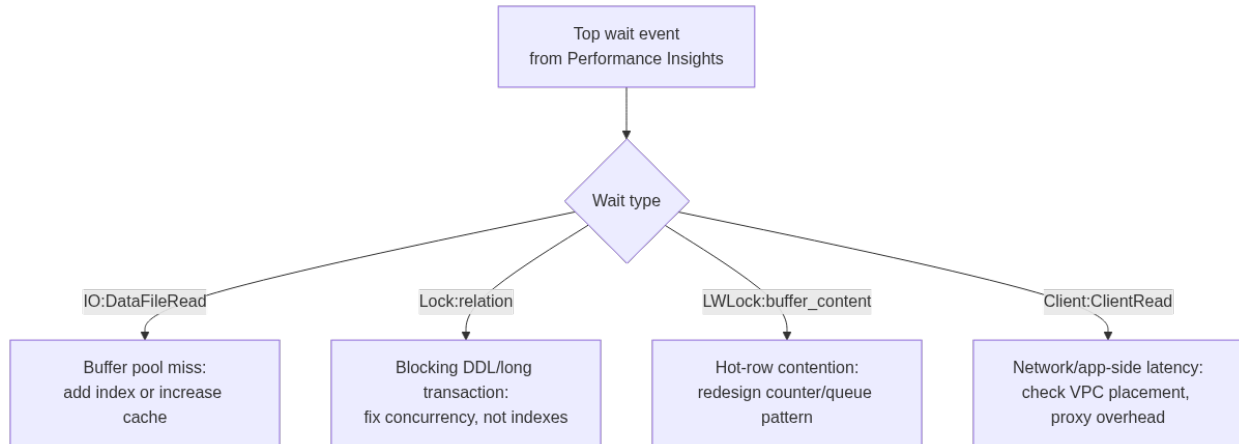


Figure 26: Let the dominant wait event categorize the problem before choosing a remedy — an index fix does nothing for a lock-contention problem.

```

ADD EVENT sqlserver.wait_info (
  ACTION (sqlserver.sql_text, sqlserver.query_hash)
  WHERE (
    duration > 100000 -- 100ms wait
    AND opcode = 1    -- end of wait only
  )
)
ADD TARGET package0.ring_buffer (SET max_memory = 51200)
WITH (
  MAX_DISPATCH_LATENCY = 5 SECONDS,
  TRACK_CAUSALITY = ON
);

ALTER EVENT SESSION [prod_wait_analysis] ON DATABASE STATE = START;

-- Read the ring buffer results
SELECT
  event_data.value('@name'), 'nvarchar(128)' AS event_name,
  event_data.value('(action[@name="sql_text"]/value)[1]', 'nvarchar(max)') AS sql_text,
  event_data.value('(data[@name="wait_type"]/text)[1]', 'nvarchar(128)') AS wait_type,
  event_data.value('(data[@name="duration"]/value)[1]', 'bigint') / 1000 AS duration_ms
FROM (
  SELECT CAST(target_data AS XML) AS target_data_xml
  FROM sys.dm_xe_database_sessions s
  JOIN sys.dm_xe_database_session_targets t ON s.address = t.event_session_address
  WHERE s.name = 'prod_wait_analysis'
  AND t.target_name = 'ring_buffer'
) AS ring_buffer_data
CROSS APPLY target_data_xml.nodes('//RingBufferTarget/event') AS XEventData(event_data)
ORDER BY duration_ms DESC;

```

Cloud SQL PostgreSQL: pg_stat_activity for Live Session Analysis

On Cloud SQL, `pg_stat_activity` is available with the same columns as on-premises PostgreSQL. Because you cannot attach a debugger or strace to a backend process, this view — combined with `pg_locks` — is your primary tool for live session and blocking investigation:

```

-- Find blocking chains on Cloud SQL (works identically on RDS Aurora)
WITH RECURSIVE lock_tree AS (
  -- Base: sessions that are waiting for a lock
  SELECT
    blocked.pid AS blocked_pid,
    blocked.query AS blocked_query,
    blocked.wait_event_type,

```

```

        blocked.wait_event,
        blocked.state,
        blocking.pid AS blocking_pid,
        blocking.query AS blocking_query,
        blocking.query_start AS blocking_started,
        now() - blocking.query_start AS blocking_duration,
        1 AS depth
FROM pg_stat_activity blocked
JOIN pg_locks blocked_locks
    ON blocked.pid = blocked_locks.pid
    AND NOT blocked_locks.granted
JOIN pg_locks blocking_locks
    ON blocking_locks.locktype = blocked_locks.locktype
    AND blocking_locks.relation IS NOT DISTINCT FROM blocked_locks.relation
    AND blocking_locks.page IS NOT DISTINCT FROM blocked_locks.page
    AND blocking_locks.tuple IS NOT DISTINCT FROM blocked_locks.tuple
    AND blocking_locks.transactionid IS NOT DISTINCT FROM blocked_locks.transactionid
    AND blocking_locks.classid IS NOT DISTINCT FROM blocked_locks.classid
    AND blocking_locks.objid IS NOT DISTINCT FROM blocked_locks.objid
    AND blocking_locks.objsubid IS NOT DISTINCT FROM blocked_locks.objsubid
    AND blocking_locks.granted
    AND blocking_locks.pid != blocked_locks.pid
JOIN pg_stat_activity blocking
    ON blocking.pid = blocking_locks.pid

UNION ALL

-- Recursive: find sessions blocking the already-identified blockers
SELECT
    lt.blocked_pid,
    lt.blocked_query,
    lt.wait_event_type,
    lt.wait_event,
    lt.state,
    blocking.pid AS blocking_pid,
    blocking.query AS blocking_query,
    blocking.query_start AS blocking_started,
    now() - blocking.query_start AS blocking_duration,
    lt.depth + 1
FROM lock_tree lt
JOIN pg_locks blocked_locks ON lt.blocking_pid = blocked_locks.pid AND NOT blocked_locks.granted
JOIN pg_locks blocking_locks
    ON blocking_locks.locktype = blocked_locks.locktype
    AND blocking_locks.relation IS NOT DISTINCT FROM blocked_locks.relation
    AND blocking_locks.granted
    AND blocking_locks.pid != blocked_locks.pid
JOIN pg_stat_activity blocking ON blocking.pid = blocking_locks.pid
WHERE lt.depth < 5
)
SELECT
    depth,
    blocked_pid,
    LEFT(blocked_query, 80) AS blocked_query_snippet,
    wait_event_type || ':' || wait_event AS wait,
    blocked_pid,
    LEFT(blocking_query, 80) AS blocking_query_snippet,
    blocking_duration
FROM lock_tree
ORDER BY depth, blocking_duration DESC;

```

35.5 Parameter Tuning on Managed Services

On self-managed PostgreSQL or SQL Server, you tuned memory allocations, parallelism settings, and I/O behavior through `postgresql.conf` or `sp_configure` and had full control over the restart

cycle. On managed services, parameters are set through the platform’s parameter system, some changes require a restart (making them high-impact in production), and some parameters are immutable — the platform controls them.

RDS PostgreSQL: Tunable vs. Platform-Controlled Parameters

The most performance-relevant parameters you can tune on RDS PostgreSQL:

Parameter	Default	Tuning Direction	Restart Required
<code>shared_buffers</code>	25% of RAM	Increase to 25-40% RAM	Yes
<code>work_mem</code>	4MB	Increase carefully per session	No
<code>maintenance_work_mem</code>	64MB	Increase for faster VACUUM/index builds	No
<code>effective_cache_size</code>	16GB	Set to ~75% of total RAM	No
<code>random_page_cost</code>	4.0	Lower to 1.1-1.5 for SSD/Aurora	No
<code>max_parallel_workers_per_gather</code>	2	Tune based on vCPU count	No
<code>checkpoint_completion_target</code>	0.9	Keep at 0.9	No
<code>wal_buffers</code>	auto	Usually auto is fine	Yes
<code>autovacuum_vacuum_scale_factor</code>	0.2	Lower to 0.01-0.05 for large tables	No

Parameters RDS does not expose (they are controlled by the platform):

- `wal_level` — Fixed at `logical` on Aurora, `replica` on RDS standard
- `max_wal_size` and `min_wal_size` — Platform-managed
- `archive_mode` / `archive_command` — Managed by AWS for automated backups
- `fsync`, `full_page_writes` — Always enabled; platform guarantees these for durability

A common misconfiguration on RDS is leaving `work_mem` at its default 4MB on an instance with 64GB RAM running analytics queries. The result is excessive temp file spills. The right approach is not to set `work_mem` globally high — that multiplies by `max_connections` and exhausts RAM — but to set it per session or per role for analytics workloads:

```
-- Set work_mem for an analytics role without changing the global default
ALTER ROLE analytics_user SET work_mem = '256MB';

-- Or set it for the current session only when running a known expensive query
SET work_mem = '128MB';
EXPLAIN (ANALYZE, BUFFERS) SELECT ... FROM large_aggregation_query;
RESET work_mem;
```

The session-level `SET` approach is particularly useful when you’re tuning a specific batch job that runs under a dedicated database user — it doesn’t affect OLTP connections using the same instance.

Aurora-Specific: cluster-level vs. instance-level parameter groups

Aurora uses two layers of parameter groups: the DB cluster parameter group (applies to the whole cluster, controls parameters like `rds.force_ssl`, `shared_preload_libraries`) and the DB instance parameter group (applies to a specific instance, controls `work_mem`, `max_connections`, etc.). Mistakes happen when engineers apply changes to the wrong level. Adding `pg_stat_statements` to `shared_preload_libraries` at the instance level has no effect on Aurora — it must be in the cluster parameter group because the extension is loaded at cluster startup, not per instance.

```
# Correct: add pg_stat_statements to the cluster parameter group
aws rds modify-db-cluster-parameter-group \
  --db-cluster-parameter-group-name prod-aurora-cluster-params \
  --parameters \

  ↪ "ParameterName=shared_preload_libraries,ParameterValue=pg_stat_statements,ApplyMethod=pending-reboot"

# Check which parameter group is attached to the cluster vs instance
aws rds describe-db-clusters \
  --db-cluster-identifier prod-aurora-cluster \
  --query "DBClusters[0].DBClusterParameterGroup"

aws rds describe-db-instances \
  --db-instance-identifier prod-aurora-cluster-instance-1 \
  --query "DBInstances[0].DBParameterGroups"
```

Azure SQL: Resource Governance and DTU/vCore Limits

Azure SQL Database manages memory, CPU, and I/O through its resource governance layer. In the DTU model, all resources are bundled. In the vCore model, you have more visibility: `sys.dm_user_db_resource_governance` shows the effective limits applied to your database:

```
-- View effective resource limits for the current Azure SQL database
SELECT
  database_id,
  logical_database_guid,
  max_cpu,
  max_db_memory_mb,
  max_log_rate_mbps,
  max_workers,
  max_sessions,
  max_instance_max_db_memory_mb
FROM sys.dm_user_db_resource_governance;

-- Check current resource consumption against limits
SELECT
  avg_cpu_percent,
  avg_data_io_percent,
  avg_log_write_percent,
  avg_memory_usage_percent,
  max_worker_percent,
  max_session_percent
FROM sys.dm_db_resource_stats
ORDER BY end_time DESC
OFFSET 0 ROWS FETCH NEXT 30 ROWS ONLY;
```

If `avg_log_write_percent` is consistently near 100%, log write throughput is the bottleneck. The fix is not to tune a parameter — you cannot increase the log rate limit without changing service tier. This is the key difference from on-premises: some bottlenecks on Azure SQL are resolved only by scaling up or out, not by configuration.

Google Cloud SQL: Flags and Connection Limits

Cloud SQL exposes PostgreSQL parameters as database flags. The flag naming matches PostgreSQL parameter names exactly. The most impactful flags for tuning on Cloud SQL:

```
# Apply a tuning set for a high-concurrency OLTP workload on Cloud SQL
gcloud sql instances patch prod-cloudsql-pg \
  --database-flags \
    shared_buffers=8GB,\
    effective_cache_size=24GB,\
    work_mem=16MB,\
    maintenance_work_mem=2GB,\
    random_page_cost=1.1,\
    effective_io_concurrency=200,\
    max_parallel_workers_per_gather=4,\
```

```

max_parallel_workers=8,\
autovacuum_max_workers=5,\
autovacuum_vacuum_scale_factor=0.02,\
autovacuum_analyze_scale_factor=0.01,\
log_autovacuum_min_duration=250,\
checkpoint_completion_target=0.9,\
wal_compression=on

```

Note: shared_buffers requires an instance restart on Cloud SQL
Cloud SQL will restart the instance automatically when static flags change

Cloud SQL does not allow `max_connections` above a service-tier-specific ceiling. If your application requires more than the allowed maximum, you must use Cloud SQL Auth Proxy with connection pooling (PgBouncer or the built-in `pgconnection_pool`) rather than setting a higher `max_connections` on the instance.

35.6 Query Rewrites and Application-Level Optimizations

After index tuning, statistics refresh, and parameter adjustments, some queries still perform poorly because the query structure itself is inefficient for the access pattern. These rewrites are platform-agnostic in logic but have platform-specific considerations in execution.

N+1 Patterns Detected Through `pg_stat_statements`

The N+1 problem — issuing one query per row from a parent result set — produces a distinctive signature in `pg_stat_statements`: a parameterized query with very high `calls` count relative to `total_exec_time`, where the per-call time is low but the total time is large. This pattern is invisible in slow query logs because each individual call is fast.

```

-- Identify N+1 candidates in pg_stat_statements
SELECT
  LEFT(query, 100) AS query_snippet,
  calls,
  mean_exec_time,
  total_exec_time,
  rows / calls AS avg_rows_per_call
FROM pg_stat_statements
WHERE calls > 10000
  AND mean_exec_time < 5    -- Fast individually (< 5ms)
  AND total_exec_time > 60000 -- But expensive in aggregate (> 60 seconds total)
ORDER BY total_exec_time DESC
LIMIT 20;

```

The fix is a query rewrite at the application layer — replace the loop with a single query using `WHERE id = ANY($1)` with an array parameter, or with a `JOIN`. The PostgreSQL rewrite is straightforward:

```

-- Before: N+1 pattern (application issues one query per order)
SELECT * FROM order_items WHERE order_id = $1;
-- (called 50,000 times in a reporting window)

-- After: single query with array unnesting
SELECT
  oi.*,
  o.customer_id,
  o.order_date
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
WHERE oi.order_id = ANY($1::bigint[]);
-- $1 is passed as an array of all order_ids needed

```

The equivalent fix in SQL Server uses a table-valued parameter or a JSON array to pass multiple IDs in a single round trip:

```
-- SQL Server: replace N+1 with a single query using JSON input
-- Before: called once per order_id
SELECT * FROM OrderItems WHERE OrderId = @order_id;

-- After: pass the full set as JSON
CREATE OR ALTER PROCEDURE dbo.GetOrderItemsBatch
    @order_ids NVARCHAR(MAX) -- JSON array: [1001, 1002, 1003]
AS
BEGIN
    SELECT
        oi.*,
        o.CustomerId,
        o.OrderDate
    FROM OrderItems oi
    JOIN Orders o ON oi.OrderId = o.OrderId
    WHERE oi.OrderId IN (
        SELECT value FROM OPENJSON(@order_ids) WITH (value BIGINT '$')
    );
END;
```

Pagination Anti-Patterns

OFFSET/LIMIT (PostgreSQL) and OFFSET/FETCH (SQL Server) performance degrades as the offset grows because the database must scan and discard all rows before the offset position. On large tables with deep pagination, this becomes a full table scan disguised as a pagination query. Use keyset pagination instead:

```
-- PostgreSQL: OFFSET-based pagination (slow for large offsets)
SELECT order_id, customer_id, order_date, total
FROM orders
ORDER BY order_date DESC, order_id DESC
OFFSET 50000 LIMIT 20;

-- PostgreSQL: Keyset pagination (constant time regardless of page depth)
-- Client passes the last seen (order_date, order_id) from the previous page
SELECT order_id, customer_id, order_date, total
FROM orders
WHERE (order_date, order_id) < ($last_order_date, $last_order_id)
ORDER BY order_date DESC, order_id DESC
LIMIT 20;

-- Requires a composite index to be efficient:
CREATE INDEX idx_orders_date_id ON orders (order_date DESC, order_id DESC);

-- SQL Server: OFFSET/FETCH (slow for large page numbers)
SELECT OrderId, CustomerId, OrderDate, Total
FROM Orders
ORDER BY OrderDate DESC, OrderId DESC
OFFSET 50000 ROWS FETCH NEXT 20 ROWS ONLY;

-- SQL Server: Keyset pagination
SELECT OrderId, CustomerId, OrderDate, Total
FROM Orders
WHERE OrderDate < @last_order_date
    OR (OrderDate = @last_order_date AND OrderId < @last_order_id)
ORDER BY OrderDate DESC, OrderId DESC
OFFSET 0 ROWS FETCH NEXT 20 ROWS ONLY;

-- Supporting index:
CREATE INDEX IX_Orders_Date_Id ON Orders (OrderDate DESC, OrderId DESC);
```

Covering Indexes for High-Frequency Lookup Patterns

A covering index — one that includes all columns referenced in a query — eliminates the heap/table

fetch entirely. On PostgreSQL, the `INCLUDE` clause adds non-key columns to the index leaf pages. On SQL Server, `INCLUDE` on a nonclustered index does the same:

```
-- PostgreSQL: covering index with INCLUDE for a common customer lookup
CREATE INDEX CONCURRENTLY idx_orders_customer_covering
ON orders (customer_id, order_date DESC)
INCLUDE (status, total_amount, order_id);

-- Query that can be satisfied entirely from the index:
SELECT order_id, order_date, status, total_amount
FROM orders
WHERE customer_id = $1
  AND order_date >= NOW() - INTERVAL '90 days'
ORDER BY order_date DESC
LIMIT 50;
-- EXPLAIN will show "Index Only Scan" - no heap access

-- SQL Server: covering nonclustered index with INCLUDE
CREATE NONCLUSTERED INDEX IX_Orders_Customer_Date_Covering
ON Orders (CustomerId, OrderDate DESC)
INCLUDE (Status, TotalAmount, OrderId);

-- The same query pattern uses the index without a Key Lookup:
SELECT OrderId, OrderDate, Status, TotalAmount
FROM Orders
WHERE CustomerId = @customer_id
  AND OrderDate >= DATEADD(DAY, -90, GETDATE())
ORDER BY OrderDate DESC;
-- SSMS plan should show Index Seek without Key Lookup
```

35.7 Monitoring Query Performance Over Time

One-time tuning sessions fix the immediate problem. Production systems need continuous monitoring to catch regressions before users report them. Cloud managed services provide the infrastructure for this; you provide the alerting thresholds and response playbooks.

Automated Plan Regression Detection on Azure SQL

Query Store's Automatic Plan Correction detects when a plan regresses (execution time degrades beyond a threshold compared to the previous plan) and forces the last known good plan automatically. You can configure the aggressiveness:

```
-- Enable Automatic Plan Correction in Azure SQL
ALTER DATABASE [ProdDB]
SET AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON);

-- Check the history of automatic corrections
SELECT
  qs.query_id,
  qs.query_sql_text,
  dra.reason,
  dra.score,
  dra.state_transition_reason,
  JSON_VALUE(dra.details, '$.implementationDetails.script') AS correction_script
FROM sys.dm_db_tuning_recommendations dra
JOIN sys.query_store_query qs ON CAST(JSON_VALUE(dra.details, '$.queryId') AS BIGINT) = qs.query_id
JOIN sys.query_store_query_text qsqt ON qs.query_text_id = qsqt.query_text_id
WHERE dra.type = 'FORCE_LAST_GOOD_PLAN'
ORDER BY dra.score DESC;
```

RDS: CloudWatch Alarms for Query Performance Metrics

Set CloudWatch alarms on the database-level metrics that correlate with query performance degradation. The most reliable leading indicators are `ReadLatency`, `WriteLatency`, `DiskQueueDepth`, and `DBLoad` (via Performance Insights):

```
# Create a CloudWatch alarm for high database load (>= instance vCPU count indicates saturation)
VCPU_COUNT=16 # for db.r6g.4xlarge

aws cloudwatch put-metric-alarm \
  --alarm-name "prod-db-high-load" \
  --alarm-description "DB load exceeds vCPU count - query saturation likely" \
  --metric-name DBLoad \
  --namespace AWS/RDS \
  --statistic Average \
  --period 300 \
  --evaluation-periods 3 \
  --threshold $VCPU_COUNT \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --dimensions Name=DBInstanceIdentifier,Value=prod-db-01 \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:db-oncall-alerts \
  --ok-actions arn:aws:sns:us-east-1:123456789012:db-oncall-alerts \
  --treat-missing-data notBreaching

# Alarm on read latency > 20ms average over 5 minutes
aws cloudwatch put-metric-alarm \
  --alarm-name "prod-db-high-read-latency" \
  --alarm-description "Read latency elevated - check for missing indexes or table scans" \
  --metric-name ReadLatency \
  --namespace AWS/RDS \
  --statistic Average \
  --period 300 \
  --evaluation-periods 2 \
  --threshold 0.020 \
  --comparison-operator GreaterThanThreshold \
  --dimensions Name=DBInstanceIdentifier,Value=prod-db-01 \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:db-oncall-alerts
```

Building a Query Performance Baseline Table

The most durable monitoring approach is a scheduled job that snapshots `pg_stat_statements` (PostgreSQL) or Query Store aggregates (SQL Server) into a history table. This lets you compare current performance against any historical baseline, detect gradual regressions over days or weeks, and tie performance changes to deployment events.

```
-- PostgreSQL: create a baseline snapshot table
CREATE TABLE query_performance_snapshots (
  snapshot_time    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  queryid          BIGINT NOT NULL,
  query_text       TEXT,
  calls            BIGINT,
  total_exec_time_ms  DOUBLE PRECISION,
  mean_exec_time_ms  DOUBLE PRECISION,
  stddev_exec_time_ms DOUBLE PRECISION,
  rows            BIGINT,
  shared_blks_hit    BIGINT,
  shared_blks_read   BIGINT,
  temp_blks_written  BIGINT,
  PRIMARY KEY (snapshot_time, queryid)
) PARTITION BY RANGE (snapshot_time);

CREATE TABLE query_performance_snapshots_2024_q4
PARTITION OF query_performance_snapshots
FOR VALUES FROM ('2024-10-01') TO ('2025-01-01');

-- Snapshot procedure (run via pg_cron or an external scheduler)
INSERT INTO query_performance_snapshots (
  queryid, query_text, calls, total_exec_time_ms,
  mean_exec_time_ms, stddev_exec_time_ms, rows,
  shared_blks_hit, shared_blks_read, temp_blks_written
)
```

```

        shared_blks_hit, shared_blks_read, temp_blks_written
    )
    SELECT
        queryid,
        LEFT(query, 1000),
        calls,
        total_exec_time,
        mean_exec_time,
        stddev_exec_time,
        rows,
        shared_blks_hit,
        shared_blks_read,
        temp_blks_written
    FROM pg_stat_statements
    WHERE calls > 10
    ON CONFLICT (snapshot_time, queryid) DO UPDATE
        SET calls = EXCLUDED.calls,
            total_exec_time_ms = EXCLUDED.total_exec_time_ms,
            mean_exec_time_ms = EXCLUDED.mean_exec_time_ms;

-- Compare current performance vs. 7-day baseline
WITH baseline AS (
    SELECT
        queryid,
        AVG(mean_exec_time_ms) AS baseline_mean_ms,
        AVG(calls) AS baseline_calls_per_snapshot
    FROM query_performance_snapshots
    WHERE snapshot_time BETWEEN NOW() - INTERVAL '8 days'
        AND NOW() - INTERVAL '1 day'

    GROUP BY queryid
),
current AS (
    SELECT
        queryid,
        mean_exec_time AS current_mean_ms,
        calls AS current_calls
    FROM pg_stat_statements
)
SELECT
    c.queryid,
    ROUND(c.current_mean_ms::NUMERIC, 2) AS current_mean_ms,
    ROUND(b.baseline_mean_ms::NUMERIC, 2) AS baseline_mean_ms,
    ROUND((c.current_mean_ms / NULLIF(b.baseline_mean_ms, 0) - 1) * 100, 1) AS pct_change,
    LEFT(ps.query, 120) AS query_snippet
FROM current c
JOIN baseline b ON c.queryid = b.queryid
JOIN pg_stat_statements ps ON c.queryid = ps.queryid
WHERE c.current_mean_ms > b.baseline_mean_ms * 1.5 -- 50% regression threshold
    AND c.current_calls > 100
ORDER BY pct_change DESC
LIMIT 20;

-- SQL Server equivalent: snapshot Query Store runtime stats for regression detection
CREATE TABLE dbo.QueryPerfSnapshots (
    SnapshotTime DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    QueryId BIGINT NOT NULL,
    PlanId BIGINT NOT NULL,
    QuerySqlText NVARCHAR(MAX),
    CountExecutions BIGINT,
    AvgDurationMs FLOAT,
    AvgCpuMs FLOAT,
    AvgLogicalIoReads FLOAT,
    AvgRowcount FLOAT,
    CONSTRAINT PK_QueryPerfSnapshots PRIMARY KEY (SnapshotTime, QueryId, PlanId)
);

-- Populate the snapshot (schedule via SQL Agent job)
INSERT INTO dbo.QueryPerfSnapshots (
    QueryId, PlanId, QuerySqlText, CountExecutions,
    AvgDurationMs, AvgCpuMs, AvgLogicalIoReads, AvgRowcount

```

```
)
SELECT
    q.query_id,
    p.plan_id,
    qt.query_sql_text,
    rs.count_executions,
    rs.avg_duration / 1000.0,
    rs.avg_cpu_time / 1000.0,
    rs.avg_logical_io_reads,
    rs.avg_rowcount
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
JOIN sys.query_store_runtime_stats_interval rsi ON rs.runtime_stats_interval_id =
    ↳ rsi.runtime_stats_interval_id
WHERE rsi.start_time >= DATEADD(MINUTE, -15, SYSUTCDATETIME());

-- Regression detection query
WITH Baseline AS (
    SELECT
        QueryId,
        AVG(AvgDurationMs) AS BaselineDurationMs
    FROM dbo.QueryPerfSnapshots
    WHERE SnapshotTime BETWEEN DATEADD(DAY, -8, SYSUTCDATETIME())
        AND DATEADD(DAY, -1, SYSUTCDATETIME())
    GROUP BY QueryId
),
Current AS (
    SELECT
        QueryId,
        AVG(AvgDurationMs) AS CurrentDurationMs,
        MAX(QuerySqlText) AS QuerySqlText
    FROM dbo.QueryPerfSnapshots
    WHERE SnapshotTime >= DATEADD(HOUR, -1, SYSUTCDATETIME())
    GROUP BY QueryId
)
SELECT TOP 20
    c.QueryId,
    ROUND(c.CurrentDurationMs, 2) AS CurrentDurationMs,
    ROUND(b.BaselineDurationMs, 2) AS BaselineDurationMs,
    ROUND((c.CurrentDurationMs / NULLIF(b.BaselineDurationMs, 0) - 1) * 100, 1) AS PctChange,
    LEFT(c.QuerySqlText, 150) AS QuerySnippet
FROM Current c
JOIN Baseline b ON c.QueryId = b.QueryId
WHERE c.CurrentDurationMs > b.BaselineDurationMs * 1.5
ORDER BY PctChange DESC;
```

Key Takeaways

- **Slow query identification on managed services routes through cloud-native log sinks** — CloudWatch Logs Insights on AWS, Query Store on Azure SQL, and Cloud Logging on GCP — rather than direct filesystem access. Master the log query syntax for your platform before a production incident forces you to learn it under pressure.
- **Execution plans on managed PostgreSQL require Aurora-aware interpretation:** Aurora’s distributed storage layer changes the effective cost of sequential scans versus index scans compared to what the planner’s default cost model assumes. Set `random_page_cost` to reflect SSD-backed storage (1.1–1.5) and validate that plan choices align with observed I/O behavior through Performance Insights wait events.

- **Parameter tuning scope matters as much as the parameter value itself:** on Aurora, `shared_preload_libraries` must be set in the cluster parameter group or the change has no effect; on Azure SQL, hitting a resource governor ceiling (log write rate, worker threads) requires a service tier change rather than a configuration adjustment. Know which constraints are tunable and which require scaling.
 - **Wait event analysis is the fastest path from symptom to root cause:** a query running slowly due to `Lock:relation` waits needs concurrency fixes, not index changes. A query slow due to `IO:DataFileRead` needs better cache usage or a covering index. Let the wait event categorize the problem before choosing the remedy.
 - **Continuous regression detection requires owning the baseline data yourself:** cloud platforms surface current performance well but make week-over-week comparison difficult without a history table. Scheduling periodic snapshots of `pg_stat_statements` or Query Store aggregates into a dedicated history table gives you a durable record that survives instance restarts, failovers, and statistics resets — and lets you correlate deployment events with the performance changes they cause.
-

Chapter 36: Storage Tiers: gp2 vs gp3 vs io1 vs io2

Storage decisions that took minutes on-premises — “provision a 2TB LUN from the SAN, set stripe size to 64KB, done” — become architecture decisions in AWS RDS with lasting cost and performance consequences. The volume type you select at instance creation determines not just your baseline throughput and IOPS ceiling, but also how you scale, what you pay, and whether you can migrate away without downtime. This chapter breaks down the four RDS storage types — gp2, gp3, io1, and io2 — from the perspective of a DBA who already understands disk I/O mechanics and just needs to know how AWS maps those mechanics onto managed storage. You will learn when to use each tier, how to configure them correctly via CLI and Terraform, how to avoid the gp2 burst credit trap that degrades production performance silently, and how to make the right call for OLTP, analytics, and mixed workloads.

Understanding the Storage Landscape: What Replaced Your SAN

On-premises, you controlled every layer: disk spindle count, RAID level, controller cache, queue depth, stripe alignment. RDS abstracts all of that, but the underlying physics still apply. What AWS gives you is a set of pre-tuned abstractions backed by EBS (Elastic Block Store), and your job is to pick the right abstraction for your workload pattern.

The four volume types map loosely to familiar concepts:

Volume Type	On-Premises Analog	Best For
gp2	Mid-range SAN with burst tier	Dev, test, small OLTP
gp3	Performance SAN with flat pricing	Most production workloads
io1	All-flash array, dedicated IOPS	I/O-intensive OLTP
io2	All-flash array, enterprise durability	Mission-critical, financial systems

Key distinction before anything else: gp2 and gp3 are “general purpose” SSDs. io1 and io2 are “provisioned IOPS” SSDs. The naming matters because it controls what guarantees AWS makes about latency and IOPS consistency. General purpose volumes can throttle. Provisioned IOPS volumes guarantee sub-millisecond latency at the configured IOPS level.

The one fact that changes everything: gp2 ties IOPS to storage size. gp3 decouples them. This single architectural difference makes gp3 almost always the better default for new provisioning in 2024.

gp2: The Legacy Default You Should Stop Using

gp2 allocates 3 IOPS per GB of storage, with a minimum of 100 IOPS and a maximum of 16,000 IOPS. That ceiling hits at 5,334 GB (approximately 5.3 TB). Beyond that, you get no additional IOPS regardless of how much storage you add.

The burst mechanism works exactly like EC2 T-series CPU burst credits. Volumes under 1 TB accumulate burst credits allowing 3,000 IOPS for short periods. When credits are exhausted, performance drops to the baseline 3 IOPS/GB. A 100 GB gp2 volume has a baseline of 300 IOPS. During credit burst it can hit 3,000 IOPS. When the credits run out — which happens after roughly 30 minutes of sustained 3,000 IOPS on a 100 GB volume — performance collapses to 300 IOPS. Applications do not receive warnings. CloudWatch metrics will show **BurstBalance** declining, but unless you have alarms configured, you will discover the problem when your query times triple at 2 AM.

The burst credit math:

Credit balance: starts at 5,400,000 (enough for 30 minutes at 3,000 IOPS)

Credit earn rate: $\text{baseline_IOPS} * 1 \text{ credit/second}$

Credit consumption: $(\text{actual_IOPS} - \text{baseline_IOPS}) * 1 \text{ credit/second}$

For a 200 GB gp2 volume (600 IOPS baseline):

- Burst IOPS available: 3,000
- Consumption rate at full burst: 2,400 credits/second
- Depletion time: $5,400,000 / 2,400 = 37.5 \text{ minutes}$

For volumes over 1 TB (baseline > 3,000 IOPS), the burst mechanism no longer applies because the baseline already meets or exceeds 3,000 IOPS. This is why many teams accidentally run fine on gp2 for years — their storage grew large enough to push past the burst ceiling before they noticed.

gp3: The Right Default for Most Production Workloads

gp3 decouples IOPS from storage size entirely. Every gp3 volume starts with 3,000 IOPS and 125 MB/s throughput included in the base price, regardless of volume size. You can provision up to 16,000 IOPS and 1,000 MB/s independently, paying only for what you configure above the baseline.

This matters practically: a 100 GB gp3 volume gets the same guaranteed 3,000 IOPS as a 10 TB gp3 volume at base pricing. No burst credits, no performance collapse, no minimum storage floor to hit an IOPS target.

Pricing structure (verify exact current rates for your region in the AWS Pricing Calculator before budgeting — EBS pricing has changed over time and varies by region):

Volume Type	Storage Cost	IOPS Cost	Throughput Cost
gp2	Per GB-month, IOPS bundled into the rate	Included (tied to size)	Included

Volume Type	Storage Cost	IOPS Cost	Throughput Cost
gp3	Per GB-month, lower than gp2's rate	3,000 IOPS included free, then billed per additional provisioned IOPS	125 MB/s included free, then billed per additional MB/s
io1	Per GB-month, higher than gp2/gp3	Billed per provisioned IOPS	Included
io2	Same per-GB rate as io1	Billed per provisioned IOPS (tiered — lower per-unit rate at very high IOPS)	Included

The storage cost alone makes gp3 cheaper per GB than gp2. The bigger driver of savings, though, is structural: on a volume that needs IOPS well above the 3-IOPS-per-GB gp2 ratio, gp2 forces you to over-provision storage just to reach the IOPS target, while gp3 lets you buy the IOPS and the storage independently. Run the actual numbers for your target IOPS and storage size against current pricing before committing — the relative savings depend on how far your IOPS requirement outpaces what gp2's ratio would force you to provision, not on a fixed percentage.

io1 vs io2: When You Need Guaranteed IOPS

Provisioned IOPS volumes exist for workloads where gp3's 16,000 IOPS ceiling is insufficient, or where you need single-digit millisecond latency guarantees that you are willing to pay for explicitly.

io1: The Workhorse Provisioned Tier

io1 allows up to 64,000 IOPS per volume (on Nitro-based instances) and up to 1,000 MB/s throughput. The IOPS-to-storage ratio constraint is critical: you can provision a maximum of 50 IOPS per GB of storage. A volume needing 64,000 IOPS requires at minimum 1,280 GB of storage, even if your actual data fits in 200 GB. This is the io1 “storage tax” — you pay for storage you do not need just to satisfy the ratio constraint.

io1 durability is 99.8–99.9% (an annual failure rate of between 0.1% and 0.2% per volume). This sounds low, but io2 improves it dramatically.

io2: io1's Successor with Better Economics

io2 offers 99.999% durability (100x more durable than io1), the same IOPS ceiling of 64,000 per volume, and a more favorable IOPS-to-storage ratio of 500 IOPS per GB (10x better than io1). The 500:1 ratio means that same 64,000 IOPS volume only requires 128 GB of storage rather than 1,280 GB, eliminating most of the storage tax.

Per-IOPS pricing between io1 and io2 is the same, making io2 strictly better than io1 for new provisioning — you get the same IOPS cost with a far better IOPS-to-storage ratio and materially higher durability. AWS has not officially deprecated io1, but the industry direction is clear: io2 is the recommended provisioned tier.

io2 Block Express (available on R5b, X2idn, X2iedn, and newer instance families) extends the ceiling further: 256,000 IOPS, 4,000 MB/s throughput, and sub-millisecond P99 latency. For extreme OLTP workloads — financial transaction systems, high-frequency trading audit logs, real-time fraud detection — this is the tier to evaluate.

When provisioned IOPS actually makes sense:

- You need more than 16,000 IOPS (gp3 ceiling)
- Your workload has consistent I/O patterns and latency sensitivity that gp3's shared infrastructure cannot guarantee
- You have a mixed read/write ratio with concurrent I/O from dozens of connections
- Regulatory requirements mandate documented performance guarantees (some compliance frameworks require it)

When provisioned IOPS is overkill:

- Workloads with high average IOPS but acceptable burst latency
- Development and staging environments
- Read-heavy workloads with caching layers (ElastiCache, RDS read replicas) absorbing the majority of reads
- Any workload under 16,000 sustained IOPS

Configuring Storage: CLI, Terraform, and Modification Patterns

Creating Instances with Specific Storage Types

```
# Create a production PostgreSQL RDS instance with gp3 storage
# Note: --iops and --storage-throughput only apply to gp3/io1/io2
aws rds create-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --db-instance-class db.r6g.2xlarge \
  --engine postgres \
  --engine-version 15.4 \
  --master-username dbadmin \
  --master-user-password "$(aws secretsmanager get-secret-value \
    --secret-id prod/rds/master-password \
    --query SecretString \
    --output text)" \
  --allocated-storage 500 \
  --storage-type gp3 \
  --iops 6000 \
  --storage-throughput 500 \
  --storage-encrypted \
  --kms-key-id arn:aws:kms:us-east-1:123456789012:key/mrk-abc123 \
  --vpc-security-group-ids sg-0abc123def456789 \
  --db-subnet-group-name prod-db-subnet-group \
  --backup-retention-period 7 \
  --preferred-backup-window "03:00-04:00" \
  --preferred-maintenance-window "sun:05:00-sun:06:00" \
  --deletion-protection \
  --region us-east-1

# Create an io2 instance for high-IOPS OLTP workload
aws rds create-db-instance \
  --db-instance-identifier prod-sqlserver-oltp \
  --db-instance-class db.r6i.4xlarge \
  --engine sqlserver-ee \
  --engine-version 15.00.4312.2.v1 \
  --master-username sa \
```

```

--master-user-password "${SA_PASSWORD}" \
--allocated-storage 2000 \
--storage-type io2 \
--iops 64000 \
--storage-encrypted \
--license-model license-included \
--region us-east-1

# Modify an existing gp2 instance to gp3
# This is the most common migration task - no downtime required
aws rds modify-db-instance \
  --db-instance-identifier legacy-postgres-gp2 \
  --storage-type gp3 \
  --iops 6000 \
  --storage-throughput 500 \
  --apply-immediately

# Monitor the modification status
aws rds describe-db-instances \
  --db-instance-identifier legacy-postgres-gp2 \
  --query 'DBInstances[0].{
    Status:DBInstanceStatus,
    StorageType:StorageType,
    IOPS:Iops,
    Throughput:StorageThroughput,
    PendingModified:PendingModifiedValues
  }' \
  --output table

# Check the pending modified values before apply-immediately
aws rds describe-db-instances \
  --db-instance-identifier legacy-postgres-gp2 \
  --query 'DBInstances[0].PendingModifiedValues'

# Monitor gp2 burst balance - set this alarm before any production gp2 usage
aws cloudwatch put-metric-alarm \
  --alarm-name "RDS-BurstBalance-Low-legacy-postgres" \
  --alarm-description "gp2 burst balance below 30% - performance degradation imminent" \
  --metric-name BurstBalance \
  --namespace AWS/RDS \
  --statistic Average \
  --dimensions Name=DBInstanceIdentifier,Value=legacy-postgres-gp2 \
  --period 300 \
  --evaluation-periods 2 \
  --threshold 30 \
  --comparison-operator LessThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:rds-alerts \
  --ok-actions arn:aws:sns:us-east-1:123456789012:rds-alerts \
  --treat-missing-data missing

# Also alarm on IOPS hitting the provisioned limit
aws cloudwatch put-metric-alarm \
  --alarm-name "RDS-IOPSNearLimit-prod-postgres" \
  --alarm-description "IOPS exceeding 90% of provisioned capacity" \
  --metric-name ReadIOPS \
  --namespace AWS/RDS \
  --statistic Average \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-01 \
  --period 60 \
  --evaluation-periods 5 \
  --threshold 5400 \
  --comparison-operator GreaterThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:rds-alerts

```

Terraform Configuration for Storage Tiers

```

# terraform/modules/rds/main.tf
# Production-grade RDS module with parameterized storage configuration

```

```

variable "storage_config" {
  description = "Storage configuration for the RDS instance"
  type = object({
    type                = string
    allocated_gb        = number
    iops                = optional(number)
    throughput_mbps     = optional(number)
    max_allocated_gb    = optional(number) # enables autoscaling
  })

  validation {
    condition = contains(["gp2", "gp3", "io1", "io2"], var.storage_config.type)
    error_message = "storage_config.type must be one of: gp2, gp3, io1, io2"
  }

  validation {
    condition = !(var.storage_config.type == "gp2" && var.storage_config.iops != null)
    error_message = "gp2 does not support explicit IOPS configuration"
  }

  validation {
    condition = !(
      contains(["io1", "io2"], var.storage_config.type) &&
      var.storage_config.iops == null
    )
    error_message = "io1 and io2 require explicit IOPS configuration"
  }
}

locals {
  # io1 max ratio: 50 IOPS/GB; io2 max ratio: 500 IOPS/GB
  max_iops_ratio = {
    io1 = 50
    io2 = 500
    gp3 = 500 # gp3 allows up to 500 IOPS/GB above 3000 baseline
    gp2 = 3    # implicit, not configurable
  }

  # Validate io1/io2 IOPS-to-storage ratio
  iops_to_storage_ratio = (
    var.storage_config.iops != null ?
    var.storage_config.iops / var.storage_config.allocated_gb :
    0
  )
}

resource "aws_db_instance" "this" {
  identifier      = var.db_identifier
  engine          = var.engine
  engine_version  = var.engine_version
  instance_class  = var.instance_class

  # Storage configuration
  allocated_storage      = var.storage_config.allocated_gb
  max_allocated_storage  = var.storage_config.max_allocated_gb # null disables autoscaling
  storage_type           = var.storage_config.type
  iops                   = var.storage_config.iops
  storage_throughput     = var.storage_config.throughput_mbps # gp3 only

  storage_encrypted = true
  kms_key_id        = var.kms_key_arn

  # Prevent accidental deletion
  deletion_protection = var.environment == "prod"
  skip_final_snapshot = var.environment != "prod"
  final_snapshot_identifier = var.environment == "prod" ? "${var.db_identifier}-final" : null

  tags = merge(var.common_tags, {
    StorageTier      = var.storage_config.type
    ProvisionedIOPS = toString(coalesce(var.storage_config.iops, 0))
  })
}

```

```
    })
  }

# Outputs for downstream monitoring configuration
output "storage_details" {
  value = {
    type           = aws_db_instance.this.storage_type
    allocated_gb   = aws_db_instance.this.allocated_storage
    iops           = aws_db_instance.this.iops
    throughput     = aws_db_instance.this.storage_throughput
    arn            = aws_db_instance.this.arn
  }
}

# terraform/environments/prod/rds_instances.tf
# Example invocations for different workload profiles

# Standard OLTP - gp3 with boosted IOPS
module "app_database" {
  source      = "../../modules/rds"
  db_identifier = "app-prod-01"
  engine      = "postgres"
  engine_version = "15.4"
  instance_class = "db.r6g.2xlarge"
  environment   = "prod"

  storage_config = {
    type           = "gp3"
    allocated_gb   = 500
    iops           = 9000
    throughput_mbps = 500
    max_allocated_gb = 2000 # autoscale up to 2TB
  }
}

# High-frequency transactional - io2
module "payments_database" {
  source      = "../../modules/rds"
  db_identifier = "payments-prod-01"
  engine      = "postgres"
  engine_version = "15.4"
  instance_class = "db.r6i.8xlarge"
  environment   = "prod"

  storage_config = {
    type           = "io2"
    allocated_gb   = 2000
    iops           = 64000
    # throughput not configurable for io2, included
  }
}

# Development - gp3 minimum configuration
module "dev_database" {
  source      = "../../modules/rds"
  db_identifier = "app-dev-01"
  engine      = "postgres"
  engine_version = "15.4"
  instance_class = "db.t3.medium"
  environment   = "dev"

  storage_config = {
    type           = "gp3"
    allocated_gb   = 100
    # No IOPS override - baseline 3,000 IOPS is sufficient
  }
}
```

Storage Autoscaling, Modification Limits, and the Gotchas Nobody Tells You

Storage Can Only Grow

This is the most important operational constraint in RDS storage management: **you cannot shrink allocated storage**. Once you provision 2 TB, you are paying for 2 TB forever on that instance. The only escape is a snapshot restore to a new instance with smaller storage, which requires downtime or a blue-green deployment.

The practical consequence: start conservatively and use autoscaling.

```
# Enable storage autoscaling on an existing instance
# max-allocated-storage must be at least 10% larger than allocated-storage
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --max-allocated-storage 2000 \
  --apply-immediately

# Disable autoscaling by setting max equal to current allocated (not below)
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --max-allocated-storage 500 \
  --apply-immediately
```

RDS autoscaling triggers when: free storage falls below 10% of allocated storage AND remains there for 5 minutes, AND at least 6 hours have passed since the last autoscale event. It increases storage in increments of either 10 GB or 10% of current allocation, whichever is larger. It will not scale above `max-allocated-storage`.

The 6-hour cooldown is a real operational risk. If your database has a runaway load that consumes storage at 1 GB/minute, autoscaling at the 5-minute threshold fires once, adds 50 GB, and then is locked for 6 hours while storage continues filling. The right pattern is autoscaling plus a CloudWatch alarm on free storage at the 15% threshold so you have warning before autoscaling even activates.

The Modification Window Problem

Storage type changes (e.g., gp2 to gp3) use `--apply-immediately`, but storage size increases do not always complete immediately. A storage modification on a running instance triggers an optimization phase that AWS calls “storage optimization” — this is essentially an online resize. Depending on volume size, this can take minutes to hours. During optimization, the instance remains available, but:

- You cannot trigger another storage modification
- The instance shows status `storage-optimization` in `describe-db-instances`
- Failover times for Multi-AZ may increase slightly during optimization

```
# Check if a storage modification is still in progress before attempting another
aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-01 \
  --query 'DBInstances[0].{
    Status:DBInstanceStatus,
    PendingValues:PendingModifiedValues,
    StorageType:StorageType,
    AllocatedStorage:AllocatedStorage,
    IOPS:Iops
  }'
```

```
# Example output when optimization is in progress:
# {
#   "Status": "storage-optimization",
#   "PendingValues": {},
#   "StorageType": "gp3",
#   "AllocatedStorage": 1000,
#   "IOPS": 6000
# }
```

Multi-AZ and Storage Modifications

For Multi-AZ instances, storage modifications are applied to the standby first, then the primary. This means:

1. AWS modifies the standby EBS volume
2. A failover occurs (brief interruption, typically under 60 seconds)
3. The former standby (now primary) runs on the new storage
4. The former primary (now standby) gets its storage updated

Applications connected with proper retry logic will reconnect automatically. Connection poolers like RDS Proxy absorb the failover without the application seeing it. If you are connecting directly to the RDS endpoint without retry logic, that 60-second window causes errors.

This also means `--apply-immediately` does not mean “instantly” for Multi-AZ instances — it means “during this maintenance event” rather than “at next maintenance window,” but the actual modification still requires the failover cycle.

Benchmarking Your Storage Tier Before Migrating Production

Before migrating a workload from on-premises or from one storage tier to another, establish a baseline. This requires understanding what I/O profile your database actually has — sequential vs. random, read vs. write, queue depth.

```
#!/usr/bin/env python3
"""
rds_storage_profiler.py
Profiles I/O patterns on an RDS instance using CloudWatch metrics to recommend
the appropriate storage tier.

Requires: boto3, configured AWS credentials with cloudwatch:GetMetricStatistics
and rds:DescribeDBInstances permissions.
"""

import boto3
import json
import sys
from datetime import datetime, timedelta, timezone
from dataclasses import dataclass, asdict
from typing import Optional

@dataclass
class StorageProfile:
    db_identifier: str
    storage_type: str
    allocated_gb: int
    provisioned_iops: Optional[int]
    avg_read_iops: float
    avg_write_iops: float
    peak_read_iops: float
```

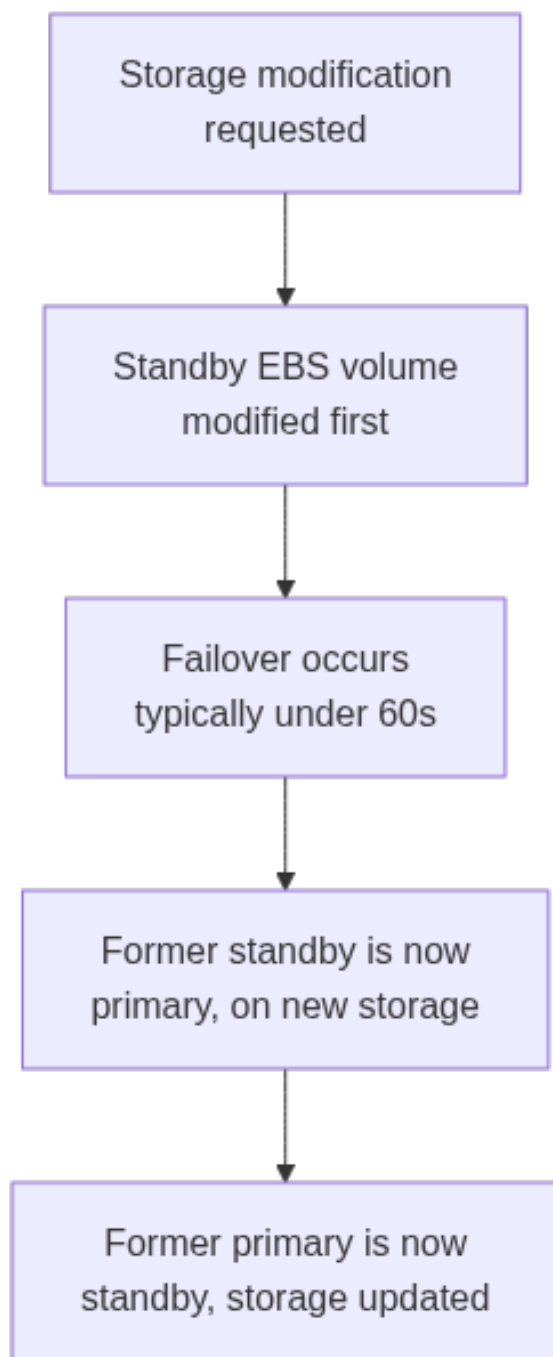


Figure 27: Multi-AZ storage changes go through a failover — apps without retry logic will see a brief connection error during step 3.


```

peak_write_iops: float
avg_read_latency_ms: float
avg_write_latency_ms: float
avg_queue_depth: float
avg_throughput_mbps: float
burst_balance_min: Optional[float]
recommended_tier: str
recommendation_reason: str

def get_metric_stats(cw_client, db_id: str, metric: str, stat: str,
                    start: datetime, end: datetime, period: int = 300) -> list:
    response = cw_client.get_metric_statistics(
        Namespace='AWS/RDS',
        MetricName=metric,
        Dimensions=[{'Name': 'DBInstanceIdentifier', 'Value': db_id}],
        StartTime=start,
        EndTime=end,
        Period=period,
        Statistics=[stat]
    )
    return sorted(response['Datapoints'], key=lambda x: x['Timestamp'])

def profile_instance(db_identifier: str, lookback_days: int = 14) -> StorageProfile:
    rds = boto3.client('rds')
    cw = boto3.client('cloudwatch')

    end = datetime.now(timezone.utc)
    start = end - timedelta(days=lookback_days)

    # Get instance metadata
    desc = rds.describe_db_instances(DBInstanceIdentifier=db_identifier)
    inst = desc['DBInstances'][0]
    storage_type = inst['StorageType']
    allocated_gb = inst['AllocatedStorage']
    provisioned_iops = inst.get('Iops')

    def avg_metric(metric, stat='Average'):
        points = get_metric_stats(cw, db_identifier, metric, stat, start, end)
        if not points:
            return 0.0
        return sum(p[stat] for p in points) / len(points)

    def max_metric(metric):
        points = get_metric_stats(cw, db_identifier, metric, 'Maximum', start, end)
        if not points:
            return 0.0
        return max(p['Maximum'] for p in points)

    avg_read_iops = avg_metric('ReadIOPS')
    avg_write_iops = avg_metric('WriteIOPS')
    peak_read_iops = max_metric('ReadIOPS')
    peak_write_iops = max_metric('WriteIOPS')

    # Latency in seconds from CloudWatch, convert to ms
    avg_read_latency_ms = avg_metric('ReadLatency') * 1000
    avg_write_latency_ms = avg_metric('WriteLatency') * 1000

    avg_queue_depth = avg_metric('DiskQueueDepth')

    # Throughput: ReadThroughput + WriteThroughput (bytes/sec -> MB/s)
    avg_throughput_mbps = (
        avg_metric('ReadThroughput') + avg_metric('WriteThroughput')
    ) / (1024 * 1024)

    # BurstBalance only relevant for gp2
    burst_balance_min = None
    if storage_type == 'gp2':
        points = get_metric_stats(cw, db_identifier, 'BurstBalance', 'Minimum', start, end)

```

```

    if points:
        burst_balance_min = min(p['Minimum'] for p in points)

    peak_total_iops = peak_read_iops + peak_write_iops
    avg_total_iops = avg_read_iops + avg_write_iops

    # Recommendation logic
    recommended_tier = 'gp3'
    recommendation_reason = ''

    if peak_total_iops > 64000:
        recommended_tier = 'io2'
        recommendation_reason = (
            f'Peak IOPS ({peak_total_iops:.0f}) exceeds gp3 ceiling of 16,000 '
            f'and io2/io1 ceiling of 64,000 - evaluate io2 Block Express on '
            f'supported instance families.'
        )
    elif peak_total_iops > 16000:
        recommended_tier = 'io2'
        recommendation_reason = (
            f'Peak IOPS ({peak_total_iops:.0f}) exceeds gp3 ceiling of 16,000. '
            f'io2 recommended over io1 for identical cost and higher durability.'
        )
    elif avg_write_latency_ms > 5.0 or avg_read_latency_ms > 5.0:
        recommended_tier = 'io2'
        recommendation_reason = (
            f'Latency exceeds 5ms average (read: {avg_read_latency_ms:.2f}ms, '
            f'write: {avg_write_latency_ms:.2f}ms). Provisioned IOPS provides '
            f'more consistent sub-millisecond latency guarantees.'
        )
    elif avg_queue_depth > 10:
        recommended_tier = 'io2'
        recommendation_reason = (
            f'High average queue depth ({avg_queue_depth:.1f}) indicates I/O '
            f'saturation. Provisioned IOPS will reduce queuing latency.'
        )
    elif storage_type == 'gp2':
        if burst_balance_min is not None and burst_balance_min < 50:
            recommended_tier = 'gp3'
            recommendation_reason = (
                f'Currently on gp2 with minimum burst balance of '
                f'{burst_balance_min:.1f}%. Performance is credit-constrained. '
                f'Migrating to gp3 eliminates burst risk at lower cost per GB.'
            )
        else:
            recommended_tier = 'gp3'
            recommendation_reason = (
                'Currently on gp2. gp3 is cheaper per GB and provides '
                'guaranteed 3,000 IOPS baseline without burst credits. '
                'Migrate to gp3 for equivalent or better performance at '
                'reduced cost.'
            )
    elif peak_total_iops < 8000 and avg_total_iops < 3000:
        recommended_tier = 'gp3'
        recommendation_reason = (
            f'Workload is within gp3 baseline (avg: {avg_total_iops:.0f} IOPS, '
            f'peak: {peak_total_iops:.0f} IOPS). gp3 at base configuration is '
            f'sufficient and most cost-effective.'
        )
    else:
        recommended_tier = 'gp3'
        recommendation_reason = (
            f'Peak IOPS ({peak_total_iops:.0f}) within gp3 range. Configure '
            f'gp3 with {int(peak_total_iops * 1.2):.0f} IOPS to provide '
            f'20% headroom above observed peak.'
        )

    return StorageProfile(
        db_identifier=db_identifier,
        storage_type=storage_type,

```

```

        allocated_gb=allocated_gb,
        provisioned_iops=provisioned_iops,
        avg_read_iops=round(avg_read_iops, 1),
        avg_write_iops=round(avg_write_iops, 1),
        peak_read_iops=round(peak_read_iops, 1),
        peak_write_iops=round(peak_write_iops, 1),
        avg_read_latency_ms=round(avg_read_latency_ms, 3),
        avg_write_latency_ms=round(avg_write_latency_ms, 3),
        avg_queue_depth=round(avg_queue_depth, 2),
        avg_throughput_mbps=round(avg_throughput_mbps, 2),
        burst_balance_min=round(burst_balance_min, 1) if burst_balance_min is not None else None,
        recommended_tier=recommended_tier,
        recommendation_reason=recommendation_reason,
    )

if __name__ == '__main__':
    if len(sys.argv) < 2:
        print("Usage: rds_storage_profiler.py <db-instance-identifier> [lookback-days]")
        sys.exit(1)

    db_id = sys.argv[1]
    lookback = int(sys.argv[2]) if len(sys.argv) > 2 else 14

    print(f"Profiling {db_id} over the last {lookback} days...")
    profile = profile_instance(db_id, lookback)
    print(json.dumps(asdict(profile), indent=2))

```

Run this profiler against your existing RDS instances before making storage tier decisions. The 14-day lookback captures weekly patterns including business-day peaks and any scheduled batch jobs. For databases with monthly cycles (end-of-month reporting, billing runs), extend to 35 days.

Workload-Specific Recommendations

OLTP (Online Transaction Processing)

OLTP workloads are characterized by many small, random I/Os with low latency requirements and a high write percentage. Typical patterns:

- Queue depth: 1–20
- Average I/O size: 4–16 KB
- Read/write ratio: 60/40 to 40/60
- Latency sensitivity: high (users feel > 10ms query times)

For OLTP under 16,000 IOPS: gp3 with IOPS configured to 120% of your measured peak. Set throughput to 500 MB/s unless you can measure a specific throughput requirement. Use autoscaling with a max 2–3x your starting allocation.

For OLTP between 16,000 and 64,000 IOPS: io2. The IOPS guarantee matters here — a gp3 volume sharing underlying infrastructure with other tenants can experience latency variability during neighbor-noisy periods. io2's dedicated provisioning removes that variable.

PostgreSQL-specific: Monitor `pg_stat_bgwriter` to understand checkpoint behavior. Excessive checkpoint writes can appear as write IOPS spikes that push you toward provisioned storage when query tuning (`checkpoint_completion_target`, `max_wal_size`) would solve it more cheaply.

```

-- PostgreSQL: check checkpoint write pressure and buffer hit rate
-- Run this on your RDS PostgreSQL instance to evaluate whether

```

```

-- storage pressure is workload-driven or configuration-driven

SELECT
    checkpoints_timed,
    checkpoints_req,
    -- High checkpoints_req relative to checkpoints_timed
    -- indicates checkpoint pressure - increase max_wal_size before
    -- upgrading storage tier
    ROUND(checkpoints_req::numeric /
        NULLIF(checkpoints_timed + checkpoints_req, 0) * 100, 2)
        AS pct_forced_checkpoints,
    buffers_checkpoint,
    buffers_clean,
    buffers_backend,
    -- High buffers_backend indicates backends writing directly to disk
    -- bypassing bgwriter - tune bgwriter_lru_maxpages
    ROUND(buffers_backend::numeric /
        NULLIF(buffers_checkpoint + buffers_clean + buffers_backend, 0) * 100, 2)
        AS pct_backend_writes,
    pg_size_pretty(buffers_checkpoint * 8192) AS checkpoint_data_written,
    stats_reset
FROM pg_stat_bgwriter;

-- Also check buffer cache hit rate
-- Below 95% for OLTP suggests either insufficient shared_buffers
-- or a genuinely I/O-bound workload requiring more storage IOPS
SELECT
    ROUND(
        100.0 * sum(heap_blks_hit) /
        NULLIF(sum(heap_blks_hit) + sum(heap_blks_read), 0),
        2
    ) AS buffer_hit_rate_pct,
    ROUND(
        100.0 * sum(idx_blks_hit) /
        NULLIF(sum(idx_blks_hit) + sum(idx_blks_read), 0),
        2
    ) AS index_hit_rate_pct
FROM pg_statio_user_tables;

-- SQL Server: check I/O stall statistics to identify storage pressure
-- vs. locking or other wait types
-- Run on RDS SQL Server via SSMS or sqlcmd

SELECT
    DB_NAME(vfs.database_id)          AS database_name,
    mf.physical_name,
    mf.type_desc,
    vfs.io_stall_read_ms,
    vfs.num_of_reads,
    CASE WHEN vfs.num_of_reads = 0 THEN 0
        ELSE vfs.io_stall_read_ms / vfs.num_of_reads
    END                               AS avg_read_stall_ms,
    vfs.io_stall_write_ms,
    vfs.num_of_writes,
    CASE WHEN vfs.num_of_writes = 0 THEN 0
        ELSE vfs.io_stall_write_ms / vfs.num_of_writes
    END                               AS avg_write_stall_ms,
    vfs.io_stall,
    vfs.num_of_bytes_read / 1048576.0 AS mb_read,
    vfs.num_of_bytes_written / 1048576.0 AS mb_written
FROM sys.dm_io_virtual_file_stats(NULL, NULL) vfs
JOIN sys.master_files mf
    ON vfs.database_id = mf.database_id
    AND vfs.file_id = mf.file_id
WHERE DB_NAME(vfs.database_id) NOT IN ('master', 'model', 'msdb', 'tempdb')
ORDER BY vfs.io_stall DESC;

-- Decision rule:
-- avg_read_stall_ms or avg_write_stall_ms consistently > 20ms:
-- Storage tier upgrade likely needed

```

```
-- avg stall < 5ms but high IOPS: you're at gp3 ceiling, evaluate io2
-- avg stall > 20ms but low IOPS: check for lock contention,
-- inefficient queries before blaming storage
```

Analytics and Data Warehousing

Analytics workloads are the inverse of OLTP: large sequential scans, high throughput requirements, lower IOPS sensitivity per query (but many concurrent queries compound this), and predominantly read-heavy with periodic bulk loads.

For Redshift, BigQuery, or Athena you are not picking EBS storage types directly — those services manage their own storage. But for RDS PostgreSQL used as an analytics database, or SQL Server used for mixed OLTP/reporting:

- **Throughput matters more than IOPS.** A 500 MB/s sequential read is one IOPS at 500 MB transfer size, or 125,000 IOPS at 4 KB. CloudWatch `ReadThroughput` is more meaningful than `ReadIOPS` for these workloads.
- **gp3 at maximum throughput (1,000 MB/s) is often sufficient.** Analytics queries scanning hundreds of GBs benefit from high sequential throughput that gp3 can provide without the cost of provisioned IOPS.
- **Read replicas change the calculus.** If reporting queries run against a read replica rather than the primary, the primary's storage only needs to handle write I/O and WAL shipping. The replica can be on a different storage tier than the primary.

```
# Create a read replica for reporting with different storage configuration
# The replica can have its own storage tier independent of the primary
aws rds create-db-instance-read-replica \
  --db-instance-identifier prod-postgres-replica-reporting \
  --source-db-instance-identifier prod-postgres-01 \
  --db-instance-class db.r6g.4xlarge \
  --storage-type gp3 \
  --iops 3000 \
  --storage-throughput 1000 \
  --region us-east-1

# Note: read replicas inherit the primary's allocated storage size.
# You cannot specify a smaller allocated storage for a replica.
# The storage type and IOPS/throughput are independently configurable.
```

Mixed Workloads: OLTP + Reporting on the Same Instance

The most common real-world RDS deployment is not pure OLTP or pure analytics — it is a production OLTP database that also serves some reporting queries, either during business hours or in scheduled overnight batch windows. This creates competing I/O patterns:

- Daytime: random 4–16 KB reads/writes, low queue depth, latency-sensitive
- Overnight: large sequential scans for reporting, high throughput demand

Storage tier selection for mixed workloads should optimize for the latency-sensitive daytime OLTP pattern, because:

1. Report queries can be made to wait; user-facing OLTP cannot
2. Query plans for reports can often be rewritten to use indexes, reducing scan I/O
3. Read replicas or scheduled reports against S3 exports are architectural alternatives

For a mixed workload, configure gp3 at the IOPS level needed for daytime OLTP peaks, with throughput at 500–1,000 MB/s. If overnight reports are degrading the primary, the right fix is a

read replica or a scheduled export to S3 followed by Athena queries — not a storage tier upgrade.

Decision Framework: Picking the Right Tier

Use this decision tree before provisioning any RDS storage:

START

Is this dev/test/staging?

YES → gp3, base configuration (no IOPS override needed)

Rationale: 3,000 IOPS baseline sufficient for non-production

Do you need > 64,000 sustained IOPS?

YES → io2 Block Express on supported instance types

If not available, consider sharding or read replicas

Do you need > 16,000 sustained IOPS?

YES → io2 (identical price to io1, better durability, better ratio)

Size storage to satisfy 500 IOPS/GB constraint

Is your workload latency-sensitive AND did benchmarking show

>5ms average latency on gp3?

YES → Evaluate io2. But first: check whether the latency comes

from storage vs. lock contention (see SQL Server/PG queries above)

Is your workload currently on gp2?

YES → Migrate to gp3. Always. No exceptions.

Even if gp2 is "working," gp3 is cheaper and more predictable.

Everything else → gp3

Configure IOPS to 120% of measured peak.

Configure throughput to 500 MB/s unless measurement shows you need more.

Enable autoscaling with 2x headroom on max_allocated_storage.

The “migrate gp2 to gp3” branch deserves emphasis. There is no scenario where gp2 is preferable to gp3 for a new provisioning decision. The only reason gp2 instances exist in 2024 is organizational inertia — they were created before gp3 was available (gp3 launched in late 2020) or before teams updated their provisioning playbooks.

Monitoring Storage Health in Production

Beyond the burst balance alarm covered earlier, a complete storage monitoring setup for RDS should include:

```
#!/usr/bin/env python3
"""
```

```

rds_storage_monitor.py
Creates a comprehensive CloudWatch dashboard and alarm set for RDS storage health.
Covers gp2 burst, IOPS saturation, latency thresholds, and free storage.
"""

import boto3
import json
from typing import Any

def create_storage_alarms(
    db_identifier: str,
    storage_type: str,
    provisioned_iops: int | None,
    sns_topic_arn: str,
    region: str = 'us-east-1'
) -> list[str]:
    """
    Creates appropriate CloudWatch alarms based on storage tier.
    Returns list of created alarm names.
    """
    cw = boto3.client('cloudwatch', region_name=region)
    created_alarms = []

    common_dims = [{'Name': 'DBInstanceIdentifier', 'Value': db_identifier}]

    def put_alarm(name: str, metric: str, stat: str, threshold: float,
                  operator: str, period: int, eval_periods: int,
                  description: str, unit: str | None = None) -> None:
        kwargs: dict[str, Any] = dict(
            AlarmName=f'RDS-{db_identifier}-{name}',
            AlarmDescription=description,
            MetricName=metric,
            Namespace='AWS/RDS',
            Statistic=stat,
            Dimensions=common_dims,
            Period=period,
            EvaluationPeriods=eval_periods,
            Threshold=threshold,
            ComparisonOperator=operator,
            AlarmActions=[sns_topic_arn],
            OKActions=[sns_topic_arn],
            TreatMissingData='missing',
        )
        if unit:
            kwargs['Unit'] = unit
        cw.put_metric_alarm(**kwargs)
        created_alarms.append(kwargs['AlarmName'])

    # Free storage alarm - 15% threshold gives time to act before autoscaling
    # Uses absolute bytes: 15% of typical starting sizes
    # For production, calculate 15% of allocated storage in bytes
    put_alarm(
        name='FreeStorageLow',
        metric='FreeStorageSpace',
        stat='Average',
        threshold=10 * 1024 ** 3, # 10 GB floor as a starting point
        operator='LessThanThreshold',
        period=300,
        eval_periods=3,
        description=(
            f'Free storage on {db_identifier} below 10 GB. '
            'Review growth rate and autoscaling configuration.'
        ),
        unit='Bytes',
    )

    # Write latency - threshold tuned per tier
    write_latency_threshold = 0.005 # 5ms for gp3
    if storage_type in ('io1', 'io2'):
        write_latency_threshold = 0.002 # 2ms for provisioned

```

```

put_alarm(
    name='WriteLatencyHigh',
    metric='WriteLatency',
    stat='Average',
    threshold=write_latency_threshold,
    operator='GreaterThanThreshold',
    period=60,
    eval_periods=5,
    description=(
        f'Write latency on {db_identifier} exceeded '
        f'{write_latency_threshold * 1000:.0f}ms average for 5 minutes.'
    ),
    unit='Seconds',
)

put_alarm(
    name='ReadLatencyHigh',
    metric='ReadLatency',
    stat='Average',
    threshold=write_latency_threshold,
    operator='GreaterThanThreshold',
    period=60,
    eval_periods=5,
    description=(
        f'Read latency on {db_identifier} exceeded '
        f'{write_latency_threshold * 1000:.0f}ms average for 5 minutes.'
    ),
    unit='Seconds',
)

# Queue depth - sustained > 64 indicates serious I/O saturation
put_alarm(
    name='DiskQueueDepthHigh',
    metric='DiskQueueDepth',
    stat='Average',
    threshold=64,
    operator='GreaterThanThreshold',
    period=300,
    eval_periods=3,
    description=(
        f'Disk queue depth on {db_identifier} sustained above 64. '
        'I/O is saturated - consider storage tier upgrade or query optimization.'
    ),
)

# IOPS saturation - 90% of provisioned or gp3 maximum
if provisioned_iops:
    iops_ceiling = provisioned_iops
elif storage_type == 'gp3':
    iops_ceiling = 16000
else: # gp2
    iops_ceiling = 16000 # theoretical max; burst balance alarm is more useful

put_alarm(
    name='IOPSNearCeiling',
    metric='ReadIOPS',
    stat='Average',
    threshold=iops_ceiling * 0.9,
    operator='GreaterThanThreshold',
    period=300,
    eval_periods=3,
    description=(
        f'Read IOPS on {db_identifier} at 90% of '
        f'provisioned ceiling ({iops_ceiling}). '
        'Review workload or increase IOPS provisioning.'
    ),
)

# gp2-specific: burst balance

```



```

if storage_type == 'gp2':
    put_alarm(
        name='BurstBalanceLow',
        metric='BurstBalance',
        stat='Minimum',
        threshold=30,
        operator='LessThanThreshold',
        period=300,
        eval_periods=2,
        description=(
            f'gp2 burst balance on {db_identifier} below 30%. '
            'Performance degradation likely. Migrate to gp3 immediately.'
        ),
    )

return created_alarms

if __name__ == '__main__':
    import sys

    if len(sys.argv) < 4:
        print("Usage: rds_storage_monitor.py <db-id> <storage-type> "
              "<sns-topic-arn> [provisioned-iops]")
        sys.exit(1)

    db_id      = sys.argv[1]
    stype      = sys.argv[2]
    sns_arn    = sys.argv[3]
    prov_iops  = int(sys.argv[4]) if len(sys.argv) > 4 else None

    alarms = create_storage_alarms(db_id, stype, prov_iops, sns_arn)
    print(f"Created {len(alarms)} alarms:")
    for a in alarms:
        print(f"  {a}")

```

Migration Playbook: gp2 to gp3 at Scale

For environments with many RDS instances, a systematic gp2-to-gp3 migration requires inventory, planning, and staged execution. Here is the operational procedure:

```

#!/bin/bash
# gp2_to_gp3_migration.sh
# Identifies all gp2 RDS instances in an account/region and
# generates a migration plan with cost comparison.
# Execute the actual migrations separately after review.

set -euo pipefail

REGION="${AWS_DEFAULT_REGION:-us-east-1}"
OUTPUT_DIR="./gp2_migration_$(date +%Y%m%d_%H%M%S)"
mkdir -p "$OUTPUT_DIR"

echo "Scanning for gp2 RDS instances in $REGION..."

# Get all gp2 instances
aws rds describe-db-instances \
  --region "$REGION" \
  --query 'DBInstances[?StorageType==`gp2`].{
    ID:DBInstanceIdentifier,
    Class:DBInstanceClass,
    Engine:Engine,
    Status:DBInstanceStatus,
    StorageGB:AllocatedStorage,
    MultiAZ:MultiAZ,

```

```

    Tags:TagList
  }' \
  --output json > "$OUTPUT_DIR/gp2_instances.json"

INSTANCE_COUNT=$(jq length "$OUTPUT_DIR/gp2_instances.json")
echo "Found $INSTANCE_COUNT gp2 instances."

# Generate migration commands with cost comparison
jq -r '.[] | .ID + " " + (.StorageGB | tostring)' \
  "$OUTPUT_DIR/gp2_instances.json" | \
while IFS=' ' read -r DB_ID STORAGE_GB; do
  # gp2 IOPS (3 per GB, min 100, max 16000)
  GP2_IOPS=$(( STORAGE_GB * 3 ))
  if (( GP2_IOPS < 100 )); then GP2_IOPS=100; fi
  if (( GP2_IOPS > 16000 )); then GP2_IOPS=16000; fi

  # Target gp3 IOPS: max(3000, gp2_equivalent)
  # Add 20% headroom
  TARGET_IOPS=$(( GP2_IOPS > 3000 ? GP2_IOPS * 12 / 10 : 3000 ))
  if (( TARGET_IOPS > 16000 )); then TARGET_IOPS=16000; fi

  cat >> "$OUTPUT_DIR/migration_commands.sh" <<EOF
# Migrate $DB_ID: ${STORAGE_GB}GB gp2 → gp3 (${TARGET_IOPS} IOPS)
aws rds modify-db-instance \
  --db-instance-identifier "$DB_ID" \
  --storage-type gp3 \
  --iops $TARGET_IOPS \
  --storage-throughput 500 \
  --region "$REGION"
# Remove --apply-immediately for maintenance window scheduling
# Add --apply-immediately to execute within minutes

EOF
done

echo "Migration plan written to $OUTPUT_DIR/"
echo "Review $OUTPUT_DIR/migration_commands.sh before executing."
echo ""
echo "IMPORTANT CHECKLIST before running migrations:"
echo "  1. Verify BurstBalance CloudWatch alarms exist for each gp2 instance"
echo "  2. Confirm Multi-AZ instances have retry logic or RDS Proxy in place"
echo "  3. Schedule migrations during low-traffic windows for gp2 volumes > 1TB"
echo "  4. Monitor storage-optimization status after each modification"
echo "  5. Validate performance metrics 24-48 hours post-migration"

```

The migration from gp2 to gp3 is online — no maintenance window required, no downtime for the database engine. The underlying EBS optimization process runs while the instance serves traffic. For Multi-AZ, the failover cycle described earlier applies, but modern applications with connection retry should handle it transparently.

Key Takeaways

- **gp3 is the right default for virtually every new RDS provisioning decision.** It decouples IOPS from storage size, costs less per GB than gp2, and provides a guaranteed 3,000 IOPS baseline with no burst credit mechanics. The only reasons to choose otherwise are specific IOPS or latency requirements that gp3 cannot satisfy.
- **gp2's burst credit system is a silent performance trap.** Volumes under 1 TB will eventually deplete burst credits under sustained load, dropping to as few as 300 IOPS with no application-level warning. If you have gp2 instances in production, instrument **BurstBalance** in CloudWatch and migrate to gp3 before the next burst event.

- **io2 is strictly superior to io1 for provisioned IOPS workloads.** Same price, 100x higher durability rating, and a 500 IOPS/GB ratio versus io1's 50 IOPS/GB — meaning less “storage tax” to meet ratio constraints. Use io2 whenever your workload genuinely exceeds gp3's 16,000 IOPS ceiling or requires single-digit millisecond latency guarantees under sustained concurrent load.
 - **RDS storage can only grow, never shrink.** Plan initial allocations conservatively and use `max_allocated_storage` autoscaling for organic growth. Reducing storage requires a snapshot restore to a new instance, which demands downtime or a blue-green deployment. The 6-hour autoscaling cooldown is a real operational risk for rapidly-growing databases — pair autoscaling with a 15% free-storage CloudWatch alarm.
 - **Storage tier decisions should follow measured workload data, not estimates.** Use CloudWatch metrics — `ReadIOPS`, `WriteIOPS`, `ReadLatency`, `WriteLatency`, `DiskQueueDepth` — over a 14-day minimum lookback to understand actual peak and average I/O patterns before provisioning. Upgrading from gp3 to io2 without measurement often wastes money; staying on gp2 or under-provisioning gp3 without measurement silently degrades production performance. The `rds_storage_profiler.py` script in this chapter automates that analysis and produces a tier recommendation you can act on directly.
-

Chapter 37: Read Replica Economics: When They Pay Off

Read replicas are one of the most frequently misunderstood tools in cloud database architecture. On paper, the pitch is simple: pay for a second instance and double your read throughput. In practice, the economics are rarely that clean. Replicas introduce replication lag, connection management complexity, application-level routing logic, and real dollar costs that can easily exceed the value they deliver—unless you understand exactly when and where they make sense. This chapter gives you the framework to evaluate replica deployments across AWS RDS, Aurora, Azure SQL, and Cloud SQL, with attention to the cost models, performance characteristics, and operational patterns that separate a well-designed read scaling strategy from an expensive misfire.

37.1 How Cloud Read Replicas Differ from On-Premises Streaming Replication

If you’ve run PostgreSQL streaming replication or SQL Server Always On Availability Groups on-premises, you already understand the fundamentals: WAL records or log blocks ship to a secondary, the secondary applies them, and reads can be offloaded to the standby. What changes in the cloud is everything surrounding that core mechanism—storage architecture, failover behavior, lag semantics, and, critically, how you pay for it.

RDS PostgreSQL and MySQL replicas use the same logical streaming replication you know from self-managed Postgres. The primary generates WAL, the replica applies it asynchronously by default. This means the replication lag you measured with `pg_stat_replication` on-premises is the same metric you’ll watch in CloudWatch under `ReplicaLag`. The difference is that RDS manages the replication slot, the `wal_keep_size` configuration, and the standby’s `recovery.conf` (now `postgresql.auto.conf` in Postgres 12+) for you. What you lose is the ability to tune `recovery_min_apply_delay` to create intentional lag standbys—that’s not exposed in RDS Parameter Groups. You also cannot promote a replica without going through the RDS API, which matters for your runbooks.

Aurora replicas operate on an entirely different model. Because Aurora uses a shared distributed storage layer across the writer and all readers, Aurora replicas don’t ship WAL at all. Instead, the writer writes to the shared storage volume, and replicas read from the same pages directly. The replication “lag” in Aurora is not WAL application lag—it’s the time for cache invalidation metadata to propagate. Aurora calls this `AuroraReplicaLag`, and it’s typically measured in single-

digit milliseconds rather than the seconds or minutes you might see in RDS streaming replication under write pressure. This is a fundamental architectural difference with real implications: Aurora can support up to 15 read replicas per cluster with sub-10ms lag even during heavy write workloads, while RDS PostgreSQL becomes increasingly difficult to manage at more than 3–4 replicas before lag compounds.

Azure SQL doesn't use the term "read replica" in the same way. On General Purpose tier, readable secondaries are not available for geo-replicas except at the Business Critical tier. At Business Critical, you get an Always On AG cluster under the hood, and one of the secondary replicas is readable at no additional cost—this is a meaningful economic advantage that's often overlooked. Azure SQL Hyperscale changes the model again: Hyperscale supports up to 30 named compute replicas that read from a shared log service and page server architecture similar in concept to Aurora's shared storage. Lag in Hyperscale is governed by log apply rate, not page transfer.

Cloud SQL uses PostgreSQL logical or streaming replication under the hood depending on version, and Spanner uses an entirely different synchronous Paxos-based replication. AlloyDB, Google's Aurora-equivalent, uses a disaggregated storage model similar to Aurora where read pool instances share the log stream without applying WAL independently.

The on-premises DBA's first instinct when reaching for a replica is to solve a read throughput problem. That's valid. But before you provision, you need to understand three cost dimensions that don't exist on-premises: compute cost of the replica instance, cross-AZ data transfer cost, and the cost of the engineering effort to route reads correctly.

37.2 The Real Cost Model: What You're Actually Paying For

Cloud providers make replica pricing look simple in their calculators. It is not simple in production.

RDS Replica Cost Components

Adding a read replica at the same instance class as the primary roughly doubles your compute spend on that instance class — check current on-demand pricing for your specific instance class and region rather than relying on a generic per-hour figure, since RDS pricing varies by engine, region, and changes over time. But compute is not the whole picture:

- **Storage:** Replicas in RDS have their own EBS volumes. Storage is billed separately per replica at the same GP3 or io2 rates as the primary. A 2TB primary means you're paying for 2TB of storage on every replica.
- **Data transfer:** RDS charges for data transferred between Availability Zones (a per-GB rate, billed each direction). Replication traffic between a primary in one AZ and a replica in another crosses that boundary continuously. For a busy OLTP database generating tens of GB per hour of write throughput, this becomes a real recurring line item that is frequently omitted from cost analysis — model it against your own measured write throughput and current AWS cross-AZ transfer pricing rather than assuming it is negligible.
- **Cross-region replicas:** If you create a cross-region read replica for disaster recovery or latency reduction, data transfer fees are billed at standard inter-region rates, which vary meaningfully by region pairing. Check the current rate for your specific source and destination regions before estimating cost — the difference between nearby and distant region pairs is substantial enough to change the economics.

```

# Estimate your current replication data volume before provisioning replicas
# Check bytes sent to replicas from the primary

aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name NetworkTransmitThroughput \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-primary \
  --start-time 2024-01-15T00:00:00Z \
  --end-time 2024-01-16T00:00:00Z \
  --period 3600 \
  --statistics Average \
  --region us-east-1

# ReplicaLag monitoring - keep this under 30 seconds for most OLAP workloads
# Alert if it exceeds your application's staleness tolerance
aws cloudwatch put-metric-alarm \
  --alarm-name "prod-replica-lag-warning" \
  --metric-name ReplicaLag \
  --namespace AWS/RDS \
  --statistic Maximum \
  --period 60 \
  --threshold 30 \
  --comparison-operator GreaterThanThreshold \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-replica-1 \
  --evaluation-periods 3 \
  --alarm-actions arn:aws:sns:us-east-1:123456789:db-alerts \
  --region us-east-1

```

Aurora Replica Cost Components

Aurora pricing separates storage from compute. The cluster storage is billed once, shared across writer and all readers—this is the key economic difference from RDS. You pay for:

- Compute: Each reader instance at its instance class rate (same as primary compute)
- Storage: Once, shared, billed at Aurora's per-GB-month rate
- I/O: billed per request, unless using Aurora I/O-Optimized pricing, which trades a higher compute rate for eliminating per-I/O charges entirely — whether that trade is worthwhile depends on your I/O volume relative to compute size, so model it against your own workload's request volume and current pricing for both modes rather than a rule of thumb
- No cross-AZ replication data transfer charges—Aurora's shared storage handles this internally

The practical result: at scale, Aurora replicas cost less per replica than RDS replicas because you eliminate the per-replica storage cost. A cluster with 4TB of data and five read replicas pays for 4TB of storage once rather than $6\times$ (one primary plus five replicas) at RDS rates.

Azure SQL Business Critical Readable Secondary

This is where Azure's model creates a genuine economic advantage for mixed read-write workloads. At Business Critical tier, you're already paying for an Always On cluster. One secondary is exposed as readable at no additional compute charge. If your read traffic can tolerate potential AG lag during failover scenarios, this effectively gives you a read replica for free as part of your HA investment.

```

# Check if your Azure SQL instance is Business Critical tier
az sql db show \
  --resource-group prod-rg \
  --server prod-sqlserver \
  --name prod-database \
  --query "{sku: sku.tier, readScale: readScale}" \
  --output table

# Enable read scale-out on an existing Business Critical database
az sql db update \
  --resource-group prod-rg \

```

```
--server prod-sqlserver \
--name prod-database \
--read-scale Enabled

# Connection string to target the readable secondary
# Add ApplicationIntent=ReadOnly to the connection string
# Server=prod-sqlserver.database.windows.net;Database=prod-database;
# Authentication=ActiveDirectoryMsi;ApplicationIntent=ReadOnly
```

The Hidden Cost: Application Complexity

Every replica introduces routing logic. Read/write splitting requires your application to maintain two connection strings, or you deploy a proxy layer (RDS Proxy, ProxySQL, PgBouncer on EC2). RDS Proxy is billed per vCPU-hour of the database instance it fronts, so the proxy cost scales with the size of the instance it sits in front of — check current RDS Proxy pricing against your instance class before assuming it's negligible. Self-hosting PgBouncer on a small EC2 instance is typically cheaper in raw compute terms, but you own the HA and patching for it. Factor either cost into your model before assuming replicas are a net win.

37.3 When Replicas Actually Pay Off: Workload Patterns That Justify the Cost

The honest answer is that read replicas pay off in a narrower set of scenarios than most architecture diagrams suggest. Here are the patterns where the economics are genuinely favorable.

Pattern 1: Long-Running Analytical Queries Against an OLTP Database

This is the canonical case. You have a production OLTP database handling transactional traffic, and the business runs nightly or near-real-time reports against the same database. The reports lock nothing (they're read-only) but consume enormous I/O and CPU, creating contention with OLTP queries for buffer pool space and CPU cycles. A read replica isolates this workload completely. The replica doesn't need to match the primary's instance size—if reports run in off-peak hours, you might run the replica at `db.r6g.xlarge` and the primary at `db.r6g.4xlarge`, reducing the per-replica cost.

```
-- Query to identify read-heavy sessions that are candidates for replica offload
-- Run on RDS PostgreSQL primary before deciding on replica strategy
SELECT
    pid,
    username,
    application_name,
    state,
    wait_event_type,
    wait_event,
    EXTRACT(EPOCH FROM (now() - query_start)) AS query_duration_seconds,
    LEFT(query, 120) AS query_preview
FROM pg_stat_activity
WHERE state = 'active'
    AND query NOT LIKE 'autovacuum%'
    AND query NOT LIKE 'VACUUM%'
ORDER BY query_duration_seconds DESC
LIMIT 20;

-- Check read/write ratio on the primary - if reads are 70%+ of total queries,
-- replicas are worth modeling. Below 50%, evaluate query optimization first.
SELECT
    sum(seq_scan + idx_scan) AS total_reads,
```



```
sum(n_tup_ins + n_tup_upd + n_tup_del) AS total_writes,
round(
    sum(seq_scan + idx_scan)::numeric /
    NULLIF(sum(seq_scan + idx_scan + n_tup_ins + n_tup_upd + n_tup_del), 0) * 100,
    2
) AS read_percentage
FROM pg_stat_user_tables;
```

Pattern 2: Geographically Distributed Read Traffic

If your users span continents and the majority of their operations are reads—product catalogs, content delivery, configuration data—cross-region replicas can reduce latency from 150–200ms (transatlantic round trip) to under 20ms by serving reads from a regional replica. The business case here is user experience, not throughput. Do the math with current inter-region transfer pricing for your specific region pair and your measured replication data volume — the monthly transfer cost for a database generating tens of GB per day of replication traffic to a distant region is a real, non-trivial line item, but the exact figure depends heavily on which regions you’re pairing. That cost is justifiable if the alternative is building a separate regional database with its own write path and synchronization logic.

Pattern 3: Read-Heavy Microservices with Read-Mostly Data

Reference data services—product catalogs, geography tables, lookup tables—that are updated rarely but read thousands of times per second are strong replica candidates. The replication lag is irrelevant when data changes at most a few times per day. Here you can often use a smaller replica instance and cache aggressively from it.

Pattern 4: Reporting and BI Tool Isolation

Business intelligence tools like Tableau, Power BI, and Looker are notorious for generating unoptimized queries—full table scans, cross joins, high-memory sort operations. Pointing these tools at a replica protects the primary from analyst-generated chaos. This is a support and reliability argument as much as a performance argument. The cost is straightforward to justify: the alternative is hiring a DBA to babysit BI query impact on production.

When Replicas Don’t Pay Off

- **Write-heavy workloads:** If your write throughput is high, replicas will perpetually lag. A replica that’s 30 seconds behind is worthless for any query that needs current data, and most OLTP application reads need current data.
- **Low read-to-write ratios:** If you’re doing more writes than reads, you have a sharding or caching problem, not a read scaling problem.
- **Small datasets that fit in buffer pool:** If your working set fits in shared_buffers, a replica adds cost without adding performance. A `db.r6g.4xlarge` with 128 GB RAM can cache a 50 GB working set entirely. Adding a replica doesn’t help a workload that never hits storage.
- **Short connection pooling exhaustion:** If you’re running out of connections, you need PgBouncer or RDS Proxy, not a replica. Connection exhaustion is a connection management problem.

37.4 Provisioning and Routing: Getting the Architecture Right

Understanding the economics is half the work. Building the routing layer correctly is where replicas either function cleanly or become an operational liability.

Creating Replicas: RDS and Aurora with Terraform

```
# Terraform: RDS PostgreSQL read replica configuration
# Note: replica inherits VPC, subnet group, and parameter group from primary
# unless explicitly overridden

resource "aws_db_instance" "primary" {
  identifier      = "prod-postgres-primary"
  engine          = "postgres"
  engine_version  = "15.4"
  instance_class  = "db.r6g.2xlarge"
  allocated_storage = 500
  storage_type     = "gp3"
  storage_throughput = 250 # MiB/s, gp3 allows tuning independently
  iops            = 12000

  multi_az      = true
  db_subnet_group_name = aws_db_subnet_group.prod.name
  vpc_security_group_ids = [aws_security_group.rds.id]

  parameter_group_name = aws_db_parameter_group.postgres15.name

  backup_retention_period = 14
  backup_window           = "03:00-04:00"
  maintenance_window      = "sun:04:00-sun:05:00"

  deletion_protection = true
  skip_final_snapshot = false
  final_snapshot_identifier = "prod-postgres-final-snapshot"

  tags = {
    Environment = "production"
    Role        = "primary"
  }
}

resource "aws_db_instance" "read_replica" {
  identifier      = "prod-postgres-replica-1"
  replicate_source_db = aws_db_instance.primary.identifier

  # Replica can use a different (smaller) instance class for cost optimization
  # Match primary for HA-style replicas, downsize for analytics-only replicas
  instance_class = "db.r6g.xlarge"

  # Storage is provisioned independently for RDS replicas
  # Must be >= primary storage
  allocated_storage = 500
  storage_type       = "gp3"

  # Place replica in a different AZ from primary for resilience
  availability_zone = "us-east-1b"

  # Replicas should NOT be Multi-AZ themselves unless used as failover targets
  multi_az = false

  # Replicas can have independent parameter groups
  # Useful for tuning work_mem and max_parallel_workers for analytics workloads
  parameter_group_name = aws_db_parameter_group.postgres15_replica.name

  # Auto minor version upgrade should match primary policy
  auto_minor_version_upgrade = true

  # Performance Insights at replica level - separate from primary
  performance_insights_enabled = true
}
```

```

performance_insights_retention_period = 7

vpc_security_group_ids = [aws_security_group.rds.id]

tags = {
    Environment = "production"
    Role        = "read-replica"
    Workload    = "analytics"
}
}

# Aurora read replica cluster configuration
resource "aws_rds_cluster" "aurora_primary" {
    cluster_identifier      = "prod-aurora-cluster"
    engine                 = "aurora-postgresql"
    engine_version         = "15.4"
    database_name          = "proddb"
    master_username        = "dbadmin"
    manage_master_user_password = true # Secrets Manager integration

    db_subnet_group_name    = aws_db_subnet_group.prod.name
    vpc_security_group_ids  = [aws_security_group.aurora.id]

    backup_retention_period = 14
    preferred_backup_window = "03:00-04:00"

    # Aurora I/O-Optimized: trades per-I/O charges for a higher compute rate
    # Model your actual I/O request volume against current pricing for both
    # modes before switching - the break-even point depends on your workload
    # storage_type = "aurora-iopt1"

    deletion_protection = true

    tags = {
        Environment = "production"
    }
}

resource "aws_rds_cluster_instance" "writer" {
    identifier      = "prod-aurora-writer"
    cluster_identifier = aws_rds_cluster.aurora_primary.id
    instance_class  = "db.r6g.2xlarge"
    engine          = aws_rds_cluster.aurora_primary.engine
    engine_version  = aws_rds_cluster.aurora_primary.engine_version

    performance_insights_enabled = true

    tags = {
        Role = "writer"
    }
}

resource "aws_rds_cluster_instance" "readers" {
    count          = 2
    identifier     = "prod-aurora-reader-${count.index + 1}"
    cluster_identifier = aws_rds_cluster.aurora_primary.id
    instance_class  = "db.r6g.xlarge" # Readers can be smaller for cost
    engine          = aws_rds_cluster.aurora_primary.engine
    engine_version  = aws_rds_cluster.aurora_primary.engine_version

    # Distribute readers across AZs
    availability_zone = element(["us-east-1b", "us-east-1c"], count.index)

    performance_insights_enabled = true

    tags = {
        Role = "reader"
    }
}

```

Connection Routing with RDS Proxy

The Aurora cluster endpoint handles read/write routing natively—the writer endpoint always points to the current writer, and the reader endpoint load-balances across all reader instances. For RDS PostgreSQL, you need to build this routing yourself or use RDS Proxy.

```
# Python application-level read/write splitting with SQLAlchemy
# This pattern works for both RDS PostgreSQL and Aurora PostgreSQL

import boto3
from sqlalchemy import create_engine, event
from sqlalchemy.orm import sessionmaker
from sqlalchemy.pool import QueuePool
import os

# For Aurora, use cluster endpoints directly
# For RDS, use instance endpoints or RDS Proxy endpoints

WRITER_ENDPOINT = os.environ.get('DB_WRITER_HOST') # Primary or Aurora writer
READER_ENDPOINT = os.environ.get('DB_READER_HOST') # Replica or Aurora reader endpoint
DB_PORT = 5432
DB_NAME = os.environ.get('DB_NAME')

def get_db_password():
    """Retrieve password from Secrets Manager - standard pattern in production"""
    client = boto3.client('secretsmanager', region_name='us-east-1')
    secret = client.get_secret_value(SecretId=os.environ.get('DB_SECRET_ARN'))
    import json
    return json.loads(secret['SecretString'])['password']

def get_iam_token(host, port, user, region='us-east-1'):
    """Generate IAM auth token for RDS - replaces password auth"""
    rds_client = boto3.client('rds', region_name=region)
    token = rds_client.generate_db_auth_token(
        DBHostname=host,
        Port=port,
        DBUsername=user,
        Region=region
    )
    return token

# Writer engine - all INSERT, UPDATE, DELETE, DDL
writer_engine = create_engine(
    f"postgresql+psycopg2://dbadmin:{get_db_password()}@{WRITER_ENDPOINT}:{DB_PORT}/{DB_NAME}",
    poolclass=QueuePool,
    pool_size=5,
    max_overflow=10,
    pool_pre_ping=True, # Validates connections before use - critical for replicas
    pool_recycle=1800, # Recycle connections every 30 minutes
    connect_args={
        "connect_timeout": 5,
        "application_name": "prod-app-writer",
        "sslmode": "require",
    }
)

# Reader engine - SELECT queries that can tolerate some staleness
reader_engine = create_engine(
    f"postgresql+psycopg2://dbadmin:{get_db_password()}@{READER_ENDPOINT}:{DB_PORT}/{DB_NAME}",
    poolclass=QueuePool,
    pool_size=10, # Larger pool for readers - typically more read concurrency
    max_overflow=20,
    pool_pre_ping=True,
    pool_recycle=1800,
    connect_args={
        "connect_timeout": 5,
        "application_name": "prod-app-reader",
        "sslmode": "require",
        "options": "-c default_transaction_read_only=on" # Enforce read-only at session level
    }
)
```

```

    }
)

WriterSession = sessionmaker(bind=writer_engine)
ReaderSession = sessionmaker(bind=reader_engine)

class DatabaseRouter:
    """
    Explicit read/write routing.
    Anti-pattern: automatic detection of SELECT vs. DML.
    Correct pattern: caller declares intent explicitly.
    """

    def writer_session(self):
        return WriterSession()

    def reader_session(self, require_fresh=False):
        """
        require_fresh=True: fall back to writer when replica lag is unacceptable.
        Use for queries that read data immediately after writing.
        """
        if require_fresh:
            return WriterSession()
        return ReaderSession()

db = DatabaseRouter()

# Usage pattern in application code
def get_product_catalog():
    """Catalog reads tolerate replication lag - use reader"""
    with db.reader_session() as session:
        return session.execute(
            "SELECT id, name, price FROM products WHERE active = true ORDER BY name"
        ).fetchall()

def create_order_and_get_details(order_data):
    """Write then immediately read - must use writer for the read"""
    with db.writer_session() as session:
        order_id = session.execute(
            "INSERT INTO orders (customer_id, total) VALUES (:cid, :total) RETURNING id",
            {"cid": order_data["customer_id"], "total": order_data["total"]}
        ).scalar()
        session.commit()

    # Read must go to writer - replica may not have this row yet
    with db.writer_session() as session:
        return session.execute(
            "SELECT * FROM orders WHERE id = :id",
            {"id": order_id}
        ).fetchone()

```

The `require_fresh` flag pattern is worth internalizing. The most common replica-related application bug is a write followed immediately by a read routed to the replica. The user creates an order, gets redirected to the order confirmation page, and the replica hasn't applied the INSERT yet—so the confirmation page 404s. This class of bug is invisible in testing (your test environment probably uses a single instance) and intermittent in production (it only triggers when replication lag exceeds the time between the write and the subsequent read).

Monitoring Replication Lag to Drive Routing Decisions

A more sophisticated approach is to check replica lag before routing and fall back to the writer automatically.

```

import boto3
from datetime import datetime, timedelta, timezone

```

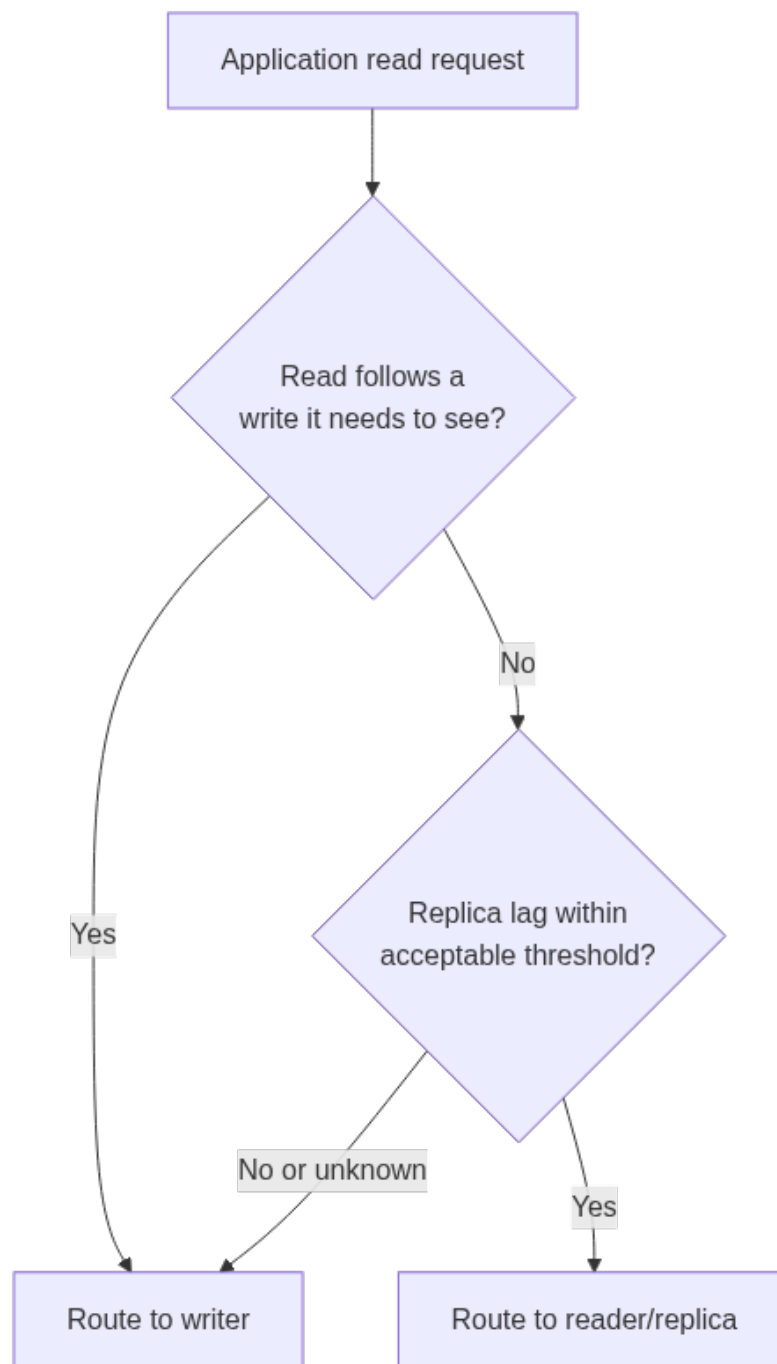


Figure 28: Explicit routing beats automatic SELECT/DML detection — the caller declares intent, and lag is checked only for the reads that could tolerate a replica.

```
class LagAwareRouter:
    """
    Checks CloudWatch ReplicaLag before routing to replica.
    Falls back to writer if lag exceeds threshold.
    Cache the lag check result for 30 seconds to avoid CloudWatch API overhead.
    """

    def __init__(self, replica_identifier, lag_threshold_seconds=10):
        self.replica_id = replica_identifier
        self.lag_threshold = lag_threshold_seconds
        self._lag_cache = None
        self._lag_cache_time = None
        self._cache_ttl = 30 # seconds
        self.cw = boto3.client('cloudwatch', region_name='us-east-1')

    def _get_replica_lag(self):
        now = datetime.now(timezone.utc)

        # Return cached value if fresh enough
        if (self._lag_cache is not None and
            self._lag_cache_time is not None and
            (now - self._lag_cache_time).seconds < self._cache_ttl):
            return self._lag_cache

        try:
            response = self.cw.get_metric_statistics(
                Namespace='AWS/RDS',
                MetricName='ReplicaLag',
                Dimensions=[{
                    'Name': 'DBInstanceIdentifier',
                    'Value': self.replica_id
                }],
                StartTime=now - timedelta(minutes=2),
                EndTime=now,
                Period=60,
                Statistics=['Maximum']
            )

            datapoints = response.get('Datapoints', [])
            if datapoints:
                lag = max(d['Maximum'] for d in datapoints)
                self._lag_cache = lag
                self._lag_cache_time = now
                return lag

            # No datapoints available - assume lag is acceptable
            return 0.0

        except Exception:
            # If we can't check lag, route to writer to be safe
            return float('inf')

    def should_use_replica(self, max_acceptable_lag_seconds=None):
        threshold = max_acceptable_lag_seconds or self.lag_threshold
        current_lag = self._get_replica_lag()
        return current_lag <= threshold

    def get_session(self, read_only=True, max_acceptable_lag=None):
        if read_only and self.should_use_replica(max_acceptable_lag):
            return ReaderSession()
        return WriterSession()
```

For Aurora, the equivalent metric is `AuroraReplicaLag` in the `AWS/RDS` namespace. The same pattern applies, but your threshold can be far more aggressive—Aurora replica lag measured in hundreds of milliseconds rather than seconds represents a degraded state, not normal operation.

37.5 Replica Lag: Measuring, Managing, and Designing Around It

Replication lag is not a static number. It's a dynamic function of write throughput on the primary, I/O capacity on the replica, and network conditions between them. Understanding its behavior under load is essential before you send application traffic to a replica.

What Causes Lag to Spike

In RDS PostgreSQL streaming replication, lag spikes typically come from three sources:

1. **Large transactions on the primary:** A single UPDATE that touches 10 million rows generates a burst of WAL that the replica must apply sequentially. The replica's single apply process (the WAL receiver → startup process pipeline) becomes the bottleneck.
2. **Vacuum activity generating WAL:** Aggressive autovacuum on large tables generates significant WAL volume. In PostgreSQL, vacuum writes visibility map updates and page rewrites to WAL.
3. **Index creation or DDL on the primary:** CREATE INDEX CONCURRENTLY on a primary generates substantial WAL as it builds the index in multiple passes.

In Aurora, lag spikes come from storage I/O saturation on the shared storage tier, or from the invalidation message queue becoming backlogged during write bursts. Both manifest as elevated AuroraReplicaLag in CloudWatch.

Querying Lag from Within PostgreSQL

```
-- On the PRIMARY: check replication status for all connected standbys
SELECT
    application_name,
    client_addr,
    state,
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    sent_lsn - replay_lsn AS replication_lag_bytes,
    write_lag,
    flush_lag,
    replay_lag,
    sync_state
FROM pg_stat_replication
ORDER BY replay_lag DESC NULLS LAST;

-- On the REPLICA: check how far behind we are from the primary
-- pg_last_wal_replay_lsn() is the last WAL position applied
-- pg_last_wal_receive_lsn() is the last WAL position received from primary
SELECT
    pg_is_in_recovery() AS is_replica,
    pg_last_wal_receive_lsn() AS received_through,
    pg_last_wal_replay_lsn() AS applied_through,
    pg_last_wal_receive_lsn() - pg_last_wal_replay_lsn() AS receive_to_apply_lag_bytes,
    now() - pg_last_xact_replay_timestamp() AS lag_from_primary;

-- On the REPLICA: identify if a long-running query on the replica is blocking
-- WAL application (query conflict with recovery)
SELECT
    pid,
    wait_event_type,
    wait_event,
    state,
    query_start,
    EXTRACT(EPOCH FROM (now() - query_start)) AS running_seconds,
    LEFT(query, 100) AS query
FROM pg_stat_activity
```



```
WHERE wait_event = 'RecoveryConflictSnapshot'
OR wait_event_type = 'Lock'
ORDER BY running_seconds DESC;
```

The last query identifies a subtle problem unique to read replicas: recovery conflicts. In PostgreSQL, a query running on the replica can conflict with WAL application when the primary vacuums a row that the replica query is still reading. PostgreSQL resolves this by either canceling the replica query (if `hot_standby_feedback` = off and `max_standby_streaming_delay` is reached) or delaying WAL application until the query finishes (if `max_standby_streaming_delay` = -1). This tradeoff—replica query cancellations versus lag accumulation—is a tuning decision you need to make explicitly based on your workload.

Tuning RDS Parameter Groups for Replica Workloads

```
-- Parameter group settings optimized for analytics replica workloads
-- These differ from OLTP-optimized settings on the primary

-- Allow replica queries more time before being canceled due to recovery conflict
-- Default is 30 seconds; set higher for long-running analytics queries
-- Max value: -1 (no cancellation - replica can lag indefinitely waiting for query)
-- Recommended: 300 seconds for analytics replicas
-- ALTER SYSTEM is not available on RDS; use Parameter Group

-- In your RDS Parameter Group (analytics replica):
-- max_standby_streaming_delay = 300000 (milliseconds)
-- max_standby_archive_delay = 300000

-- Prevent primary from vacuuming rows that replica is reading
-- Reduces recovery conflicts but increases primary table bloat risk
-- hot_standby_feedback = 1 (enable)

-- Allow parallel query on replica for analytics workloads
-- max_parallel_workers_per_gather = 4
-- max_parallel_workers = 8
-- parallel_leader_participation = on

-- Increase work_mem for analytics on replica - larger sorts, hash joins
-- Primary might use 64MB; replica can use 256MB if dedicated to analytics
-- work_mem = 262144 (KB = 256MB)

-- Verify these settings are applied on the replica
SELECT name, setting, unit, context
FROM pg_settings
WHERE name IN (
    'max_standby_streaming_delay',
    'max_standby_archive_delay',
    'hot_standby_feedback',
    'max_parallel_workers',
    'work_mem'
)
ORDER BY name;
```

Azure SQL: Monitoring Readable Secondary Lag

Azure SQL Business Critical does not expose replica lag as a direct metric in the same way as RDS. The readable secondary uses Always On AG synchronous commit within the same region, which means lag under normal conditions is negligible—commit doesn't return to the application until the log record is hardened on the secondary. However, during failover tests or under extreme load, you can observe query result differences.

```
-- SQL Server: Check AG synchronization state on Business Critical
-- Connect to the primary and inspect the AG health
-- (Available in Azure SQL via sys.dm_hadr_availability_replica_states)

SELECT
```

```

    ar.replica_server_name,
    ars.role_desc,
    ars.operational_state_desc,
    ars.connected_state_desc,
    ars.synchronization_health_desc,
    adc.synchronization_state_desc,
    adc.synchronization_health_desc AS db_sync_health,
    ars.last_connect_error_number,
    ars.last_connect_error_description
FROM sys.dm_hadr_availability_replica_states ars
JOIN sys.availability_replicas ar
    ON ars.replica_id = ar.replica_id
JOIN sys.dm_hadr_database_replica_states adc
    ON ars.replica_id = adc.replica_id
ORDER BY ars.role_desc;

-- On the readable secondary (ApplicationIntent=ReadOnly connection):
-- Check how far behind we are in terms of log send queue
SELECT
    database_name,
    log_send_queue_size,      -- KB of log not yet sent to secondary
    log_send_rate,           -- KB/s send rate
    redo_queue_size,         -- KB of log received but not yet applied
    redo_rate,               -- KB/s redo rate
    secondary_lag_seconds    -- Estimated lag in seconds
FROM sys.dm_hadr_database_replica_states
WHERE is_local = 1;

```

37.6 Operational Patterns: Replica Promotion, Failover, and Maintenance

Replicas don't exist in a steady state forever. You'll promote them, maintenance-window them, and deal with cascading failures. These scenarios require advance planning.

Promoting an RDS Read Replica

Replica promotion converts a read replica into a standalone primary. This is a one-way operation—the replica stops receiving replication from the source and becomes an independent writable instance. Use cases include:

- Disaster recovery when the primary is unrecoverable
- Environment cloning (promote a replica to create a staging database)
- Blue-green database migrations where the new version runs on the promoted replica

```

# Promote an RDS read replica to standalone instance
# This is irreversible - the replica becomes independent
aws rds promote-read-replica \
    --db-instance-identifier prod-postgres-replica-1 \
    --backup-retention-period 7 \
    --preferred-backup-window "05:00-06:00" \
    --region us-east-1

# Monitor promotion progress - instance goes into 'modifying' state
aws rds describe-db-instances \
    --db-instance-identifier prod-postgres-replica-1 \
    --query
    ↪ 'DBInstances[0].{Status,DBInstanceStatus,ReadReplicaSourceDBInstanceIdentifier:ReadReplicaSourceDBInstanceIdentifier}'
    ↪ \
    --output table

# After promotion, ReadReplicaSourceDBInstanceIdentifier will be null

```

```
# The instance will accept writes once status returns to 'available'
```

Aurora Replica Failover

Aurora failover to a reader is different from RDS replica promotion. Because Aurora readers share the storage tier with the writer, failover is a metadata operation—the reader is designated as the new writer without storage reconfiguration. Aurora typically completes this in under 30 seconds for same-AZ failover, compared to several minutes for RDS Multi-AZ failover.

```
# Initiate a controlled Aurora failover to a specific reader
# Use this for testing and planned maintenance, not just emergency failover
aws rds failover-db-cluster \
  --db-cluster-identifier prod-aurora-cluster \
  --target-db-instance-identifier prod-aurora-reader-1 \
  --region us-east-1

# Check cluster status during failover
aws rds describe-db-clusters \
  --db-cluster-identifier prod-aurora-cluster \
  --query
  ↪ 'DBClusters[0].{Status:Status,Writer:DBClusterMembers[?IsClusterWriter==`true`].DBInstanceIdentifier|[0]}'
  ↪ \
  --output table

# Aurora cluster endpoint automatically updates DNS to point to new writer
# Applications using the cluster endpoint reconnect transparently
# Applications with hardcoded instance endpoints will fail - audit for this
```

Replica Maintenance Windows and Patching

RDS minor version upgrades on replicas happen during the maintenance window. If you have multiple replicas, they can be upgraded sequentially, meaning one replica may be down or unavailable while another is being patched. During this window, your read routing layer must handle the unavailable replica gracefully.

```
# Health-check aware connection pool using psycopg2
# Verifies replica is accepting connections before routing reads

import psycopg2
from psycopg2 import OperationalError
import logging

logger = logging.getLogger(__name__)

class ReplicaHealthChecker:
    def __init__(self, replica_host, port, dbname, user, password):
        self.connection_params = {
            "host": replica_host,
            "port": port,
            "dbname": dbname,
            "user": user,
            "password": password,
            "connect_timeout": 3,
            "sslmode": "require",
        }
        self._is_healthy = False

    def check_health(self):
        """
        Verify replica is:
        1. Accepting connections
        2. In recovery (is actually a replica, not accidentally promoted)
        3. Not excessively lagged
        """
        try:
            conn = psycopg2.connect(**self.connection_params)
```

```
conn.autocommit = True
cur = conn.cursor()

# Verify this is still a replica
cur.execute("SELECT pg_is_in_recovery();")
is_replica = cur.fetchone()[0]

if not is_replica:
    logger.warning("Host %s is no longer in recovery - it was promoted",
                  self.connection_params['host'])
    self._is_healthy = False
    return False

# Check lag
cur.execute("""
    SELECT EXTRACT(EPOCH FROM (now() - pg_last_xact_replay_timestamp()))
    AS lag_seconds
""")
lag = cur.fetchone()[0]

cur.close()
conn.close()

if lag is not None and lag > 60:
    logger.warning("Replica lag is %.1f seconds - marking unhealthy", lag)
    self._is_healthy = False
    return False

self._is_healthy = True
return True

except OperationalError as e:
    logger.error("Replica health check failed: %s", str(e))
    self._is_healthy = False
    return False

@property
def is_healthy(self):
    return self._is_healthy
```

37.7 Cost Optimization Strategies for Replica Fleets

If you've committed to replicas and the workload justifies them, there are several strategies to reduce their operating cost without sacrificing availability.

Right-Size Replicas for Their Actual Workload

Replicas don't need to match the primary's instance size unless they serve as hot standbys for failover. An analytics replica that runs nightly batch reports needs large memory but doesn't need the primary's I/O-optimized storage class. A reporting replica running aggregations benefits more from additional CPU than from matching the primary's memory:

```
# Modify a replica to a smaller instance class during off-peak hours
# Use this for replicas that only handle batch analytics
aws rds modify-db-instance \
    --db-instance-identifier prod-postgres-replica-analytics \
    --db-instance-class db.r6g.xlarge \
    --apply-immediately \
    --region us-east-1

# For Aurora: modify individual reader instances independently
aws rds modify-db-instance \
    --db-instance-identifier prod-aurora-reader-2 \
```

```
--db-instance-class db.r6g.large \
--apply-immediately \
--region us-east-1
```

Aurora Serverless v2 Readers for Variable Read Workloads

Aurora Serverless v2 readers can scale from 0.5 ACUs to 128 ACUs in fine-grained increments, billed per ACU-hour. If your read workload is spiky—heavy during business hours, negligible overnight—a Serverless v2 reader can dramatically reduce costs compared to a fixed provisioned reader running at minimum overnight capacity.

```
# Terraform: Aurora cluster with one provisioned writer and one Serverless v2 reader
resource "aws_rds_cluster_instance" "serverless_reader" {
  identifier      = "prod-aurora-serverless-reader"
  cluster_identifier = aws_rds_cluster.aurora_primary.id
  instance_class  = "db.serverless"
  engine          = aws_rds_cluster.aurora_primary.engine
  engine_version  = aws_rds_cluster.aurora_primary.engine_version

  tags = {
    Role = "serverless-reader"
  }
}

resource "aws_rds_cluster" "aurora_primary" {
  # ... other config ...

  serverlessv2_scaling_configuration {
    min_capacity = 0.5 # Minimum ACUs - scales down during idle periods
    max_capacity = 16  # Maximum ACUs - caps cost during peak reads
  }
}
```

Scheduled Replica Scaling

For workloads with predictable patterns—business hours reads, nightly batch analytics—you can schedule replica creation and deletion using AWS EventBridge and Lambda rather than running replicas 24/7.

```
# Lambda function to create/delete replicas on schedule
# Triggered by EventBridge rules on a cron schedule

import boto3
import os
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

rds = boto3.client('rds', region_name=os.environ.get('AWS_REGION', 'us-east-1'))

REPLICA_IDENTIFIER = os.environ['REPLICA_IDENTIFIER']
SOURCE_DB_IDENTIFIER = os.environ['SOURCE_DB_IDENTIFIER']
INSTANCE_CLASS = os.environ.get('REPLICA_INSTANCE_CLASS', 'db.r6g.xlarge')

def create_replica(event, context):
    """Create replica at start of business hours"""
    try:
        response = rds.create_db_instance_read_replica(
            DBInstanceIdentifier=REPLICA_IDENTIFIER,
            SourceDBInstanceIdentifier=SOURCE_DB_IDENTIFIER,
            DBInstanceClass=INSTANCE_CLASS,
            AvailabilityZone='us-east-1b',
            PubliclyAccessible=False,
            AutoMinorVersionUpgrade=True,
            Tags=[
                {'Key': 'Environment', 'Value': 'production'},
```

```
        {'Key': 'ManagedBy', 'Value': 'scheduled-lambda'},
    ]
)
logger.info("Replica creation initiated: %s",
            response['DBInstance']['DBInstanceStatus'])
return {"status": "creating", "identifier": REPLICA_IDENTIFIER}
except rds.exceptions.DBInstanceAlreadyExistsFault:
    logger.info("Replica already exists - skipping creation")
    return {"status": "already_exists"}

def delete_replica(event, context):
    """Delete replica at end of business hours to save cost"""
    try:
        response = rds.delete_db_instance(
            DBInstanceIdentifier=REPLICA_IDENTIFIER,
            SkipFinalSnapshot=True, # Replicas don't hold unique data
            DeleteAutomatedBackups=True
        )
        logger.info("Replica deletion initiated: %s",
                    response['DBInstance']['DBInstanceStatus'])
        return {"status": "deleting", "identifier": REPLICA_IDENTIFIER}
    except rds.exceptions.DBInstanceNotFoundFault:
        logger.info("Replica not found - nothing to delete")
        return {"status": "not_found"}
```

Note that replica creation is not instantaneous—RDS PostgreSQL replicas take several minutes to become available after creation, and the time scales with database size because the replica must perform an initial snapshot sync. Plan your schedule to create replicas at least 15–20 minutes before the read workload begins.

Use Reserved Instances for Long-Running Replicas

If a replica runs continuously—a dedicated reporting replica, a cross-region DR replica—purchasing a 1-year or 3-year Reserved Instance for that instance class can meaningfully reduce compute cost versus on-demand. The exact discount and break-even point depend on the payment option (No/Partial/All Upfront), term length, instance class, and current pricing — run the numbers through the AWS pricing tools for your specific configuration rather than assuming a fixed percentage or payback period. Replicas that run on a schedule (created and deleted daily) should stay on-demand, since a reservation only pays off with continuous utilization.

```
# List available reserved instance offerings for your replica instance class
aws rds describe-reserved-db-instances-offerings \
  --db-instance-class db.r6g.xlarge \
  --product-description "postgresql" \
  --offering-type "Partial Upfront" \
  --region us-east-1 \
  --query
  ↪ 'ReservedDBInstancesOfferings[*].{Class:DBInstanceClass,Duration:Duration,FixedPrice:FixedPrice,UsagePrice:UsagePri
  ↪ \
  --output table
```

37.8 Cross-Cloud Comparisons: Choosing the Right Read Scaling Tool

When you have a choice of cloud platform or are evaluating a migration, the read scaling architecture can be a deciding factor.

Feature	RDS PostgreSQL	Aurora PostgreSQL	Azure SQL BC	Cloud SQL PostgreSQL	AlloyDB
Max read replicas	5	15	1 readable secondary (included)	10	20
Typical replica lag	Seconds to minutes	Single-digit ms	Near-zero (synchronous)	Seconds to minutes	Sub-second
Per-replica storage cost	Yes (full copy)	No (shared)	No (shared AG storage)	Yes (full copy)	No (shared)
Cross-AZ transfer cost	Yes	No	No	Yes	No
Automatic failover to replica	Yes (with RDS API)	Yes (sub-30s)	Yes (AG-managed)	Yes	Yes
Serverless reader option	No	Yes (v2)	No	No	No

The shared storage architectures (Aurora, Azure SQL Business Critical, AlloyDB) are superior from a replica cost and lag perspective at scale. If you’re running more than two read replicas, the storage savings alone often justify Aurora over RDS PostgreSQL despite Aurora’s higher compute pricing.

For workloads where reads genuinely dominate writes by a factor of 10 or more, and the data is amenable to eventual consistency, consider whether Cloud Spanner or DynamoDB with global tables is a more appropriate fit than replication on a relational engine—though that’s a broader architectural shift outside the scope of this chapter.

Key Takeaways

- **Read replicas pay off in specific scenarios**—long-running analytics against OLTP primaries, geographically distributed reads, and BI tool isolation—but not as a general-purpose solution for any read pressure. Query optimization and caching should be exhausted first.
- **The true cost of a replica includes compute, per-replica storage (RDS/Cloud SQL), cross-AZ data transfer charges, and the engineering cost of routing logic**—omitting any of these from your cost model will produce an underestimate.
- **Shared-storage architectures (Aurora, Azure SQL Business Critical, AlloyDB) eliminate per-replica storage costs and cross-AZ transfer fees**, making them economically superior when running three or more replicas at scale.
- **Replication lag is a dynamic, workload-driven quantity**—design your application routing to account for it explicitly, using the `require_fresh` pattern for reads that follow writes, and configure `max_standby_streaming_delay` and `hot_standby_feedback` deliberately on PostgreSQL replicas based on your analytics query duration tolerance.
- **Cost optimization for replica fleets involves right-sizing instances to actual workload, using Aurora Serverless v2 for spiky read patterns, scheduling ephemeral replicas for batch workloads, and purchasing Reserved Instances only for replicas**

with guaranteed continuous utilization.

Chapter 38: Reserved Instances vs On-Demand: The Math

Cloud compute pricing is one of the most consequential decisions a cloud DBA makes, yet it's also one of the most frequently delegated to finance teams who lack the technical context to optimize it properly. The result is organizations either overpaying for on-demand flexibility they don't need, or committing to Reserved Instances at the wrong tier and locking in costs that hurt when the architecture changes. This chapter walks through the actual pricing mechanics across AWS RDS, Azure SQL Database, and Google Cloud SQL—with the arithmetic laid out plainly—so you can make purchasing recommendations backed by data rather than guesswork. The patterns here apply regardless of which cloud you're standardizing on, and the tooling examples show how to audit your existing spend before signing anything.

38.1 How Cloud Database Pricing Works Under the Hood

Before running any numbers, you need a mental model of what you're actually buying. On-premises, you paid for hardware, licenses, and labor separately. In the cloud, those costs are bundled into an hourly or per-second rate that changes based on your commitment level.

AWS RDS Pricing Layers

RDS charges have four components that stack on top of each other:

1. **Instance hours** — the compute cost, billed per second with a one-minute minimum
2. **Storage** — billed per GB-month, with the rate depending on volume type (GP3, io1, io2); provisioned-IOPS tiers add a separate per-IOPS charge
3. **I/O requests** — GP2 and GP3 include free I/O up to a baseline; io1/io2 charges per million requests
4. **Data transfer** — inbound is generally free; outbound to the internet and cross-AZ replication traffic are both billed per GB, at different rates

The instance hour rate is where Reserved Instances make their impact, and this is the number you must pull fresh for your own analysis — do not use a remembered or published figure from any point-in-time source, including this book, because RDS on-demand and reserved pricing varies by instance class, region, engine, and changes over time. The mechanics that matter regardless of the specific rate: a 1-year commitment (No Upfront, Partial Upfront, or All Upfront) buys a materially lower effective hourly rate than on-demand, and a 3-year commitment buys a lower rate still. Get the current numbers for your instance class and region from the AWS Pricing Calculator or the

Price List API (shown below) before running any break-even analysis — never plug in a percentage discount you recall from elsewhere.

Azure SQL Database Pricing Layers

Azure SQL Database pricing depends on the service tier you're running. The General Purpose tier bills per vCore-hour. The Business Critical tier carries a substantially higher multiplier for the same vCore count due to the in-memory OLTP and local SSD backing — pull the current multiplier for your region rather than assuming a fixed ratio, since Microsoft has adjusted relative pricing between tiers over time.

Reserved Capacity for Azure SQL follows the same shape as AWS RI mechanics: a 1-year reservation buys a lower effective hourly rate than pay-as-you-go, and a 3-year reservation buys a lower rate still. Pull current rates for your vCore count, tier, and region from the Azure Pricing Calculator before modeling savings.

Azure's Reserved Instance discounts apply to the compute portion only. Storage is charged separately, per GB-month, and is not discountable via reservations.

Google Cloud SQL and AlloyDB Pricing Layers

Cloud SQL charges per vCPU-hour and per GB RAM-hour separately, plus a separate storage charge. This is different from AWS and Azure, which bundle CPU and RAM into instance family pricing. Google calls its commitment discounts Committed Use Discounts (CUDs) rather than Reserved Instances, but the economic model is the same: commit to a resource quantity for 1 or 3 years in exchange for a lower effective hourly rate than on-demand. Pull current per-vCPU-hour and per-GB-hour on-demand and CUD rates from the GCP Pricing Calculator for your region and instance shape — the discount percentage for 1-year versus 3-year commitments has been published by Google at different points, so verify it rather than citing a remembered figure.

38.2 Running the Break-Even Analysis

The decision to commit always comes down to one question: how long will this workload run at this scale? If the answer is uncertain, on-demand is the defensible choice regardless of the percentage discount. If the answer is “at least 12 months at this capacity,” the math almost always favors reserving.

The break-even formula is straightforward:

Break-Even Hours = Upfront Cost / (On-Demand Hourly Rate - Reserved Hourly Rate)

For example, for an AWS RDS instance on a 1-year Partial Upfront term, plug your own current on-demand hourly rate, reserved upfront payment, and reserved hourly rate (all pulled fresh from the AWS Pricing Calculator or Price List API for your specific instance class and region) into the formula:

Break-Even Hours = Upfront Payment / (On-Demand Hourly - Reserved Hourly)

Break-Even Days = Break-Even Hours / 24

If the instance runs for longer than the break-even point, the Partial Upfront option has already paid for itself compared to pure on-demand. To project the total annual savings once you have

current numbers:

$$\begin{aligned} \text{Annual Savings} &= (\text{On-Demand Hourly} \times 8760) - (\text{Upfront Payment} + \text{Reserved Hourly} \times 8760) \\ \text{Savings \%} &= \text{Annual Savings} / (\text{On-Demand Hourly} \times 8760) \times 100 \end{aligned}$$

Run this with your own instance's real pricing rather than a figure from any published example — the break-even point and savings percentage both shift with instance class, region, term length, and payment option, and change whenever the provider updates pricing. Multiply the per-instance savings across every instance you're evaluating to see the aggregate impact of a commitment decision on a multi-instance production cluster.

The following Python script automates this analysis across multiple instance types by pulling current pricing from the AWS Price List API:

```
import boto3
import json
from decimal import Decimal

def get_rds_ondemand_price(region_code, instance_class, engine, deployment_option):
    """
    Retrieves on-demand hourly price for an RDS instance.
    Note: AWS Price List API is only available in us-east-1.
    """
    client = boto3.client('pricing', region_name='us-east-1')

    filters = [
        {'Type': 'TERM_MATCH', 'Field': 'instanceType', 'Value': instance_class},
        {'Type': 'TERM_MATCH', 'Field': 'databaseEngine', 'Value': engine},
        {'Type': 'TERM_MATCH', 'Field': 'deploymentOption', 'Value': deployment_option},
        {'Type': 'TERM_MATCH', 'Field': 'location', 'Value': region_code},
        {'Type': 'TERM_MATCH', 'Field': 'termType', 'Value': 'OnDemand'},
    ]

    response = client.get_products(
        ServiceCode='AmazonRDS',
        Filters=filters,
        MaxResults=1
    )

    if not response['PriceList']:
        return None

    price_item = json.loads(response['PriceList'][0])
    terms = price_item['terms']['OnDemand']

    for term_key in terms:
        for price_dimension_key in terms[term_key]['priceDimensions']:
            pd = terms[term_key]['priceDimensions'][price_dimension_key]
            if pd['unit'] == 'Hrs':
                return Decimal(pd['pricePerUnit']['USD'])

    return None

def calculate_breakeven(ondemand_hourly, reserved_hourly, upfront_cost):
    """
    Returns break-even in hours and days.
    """
    hourly_savings = ondemand_hourly - reserved_hourly
    if hourly_savings <= 0:
        return None, None

    breakeven_hours = float(upfront_cost) / float(hourly_savings)
    breakeven_days = breakeven_hours / 24
    return breakeven_hours, breakeven_days

def project_annual_savings(ondemand_hourly, reserved_hourly, upfront_cost,
```

```

        utilization_hours=8760):
    """
    Projects total cost and savings over one year at a given utilization.
    Assumes utilization_hours = 8760 for 24x7 workloads.
    """
    ondemand_annual = float(ondemand_hourly) * utilization_hours
    reserved_annual = float(upfront_cost) + (float(reserved_hourly) * utilization_hours)
    savings = ondemand_annual - reserved_annual
    savings_pct = (savings / ondemand_annual) * 100

    return {
        'on_demand_annual': round(ondemand_annual, 2),
        'reserved_annual': round(reserved_annual, 2),
        'savings': round(savings, 2),
        'savings_pct': round(savings_pct, 1)
    }

# Example: db.r6g.2xlarge, PostgreSQL, Multi-AZ, us-east-1
# These rates are illustrative; pull live rates from AWS Price List or Savings Plans API
instances_to_evaluate = [
    {
        'instance_class': 'db.r6g.2xlarge',
        'engine': 'PostgreSQL',
        'deployment': 'Multi-AZ',
        'ondemand_hourly': 0.960, # Multi-AZ doubles compute cost
        'reserved_upfront': 2200,
        'reserved_hourly': 0.352,
        'term': '1yr Partial Upfront'
    },
    {
        'instance_class': 'db.r6g.4xlarge',
        'engine': 'PostgreSQL',
        'deployment': 'Multi-AZ',
        'ondemand_hourly': 1.920,
        'reserved_upfront': 4400,
        'reserved_hourly': 0.704,
        'term': '1yr Partial Upfront'
    }
]

for inst in instances_to_evaluate:
    beh, bed = calculate_breakeven(
        inst['ondemand_hourly'],
        inst['reserved_hourly'],
        inst['reserved_upfront']
    )
    projection = project_annual_savings(
        inst['ondemand_hourly'],
        inst['reserved_hourly'],
        inst['reserved_upfront']
    )

    print(f"\n{'='*60}")
    print(f"Instance:      {inst['instance_class']} ({inst['deployment']})")
    print(f"Term:          {inst['term']}")
    print(f"Break-even:     {beh:.0f} hours ({bed:.0f} days)")
    print(f"On-Demand/yr:   ${projection['on_demand_annual']:,.2f}")
    print(f"Reserved/yr:    ${projection['reserved_annual']:,.2f}")
    print(f"Annual savings: ${projection['savings']:,.2f} ({projection['savings_pct']}%)")

```

Running this against your actual instance inventory gives you a defensible business case for RI purchases without relying on vendor-provided calculators that tend to assume 100% utilization.

38.3 Reserved Instance Mechanics: What the Contracts Actually Say

Understanding the contract structure prevents the most common RI mistakes—buying the wrong scope, overcommitting on instance family, or missing the difference between Convertible and Standard RIs.

AWS RI Types and Flexibility Rules

AWS RDS offers two RI types:

- **Standard Reserved Instances:** Maximum discount, but instance family is fixed. You cannot modify a `db.r6g` Standard RI to cover a `db.r7g` instance.
- **Convertible Reserved Instances:** A somewhat smaller discount than Standard RIs at the same term and payment option — check the current gap for your instance class, since AWS has adjusted relative pricing between the two types over time — but you can exchange them for different instance types, regions, or database engines at any time during the term.

The flexibility argument for Convertible RIs is real when you're within 18–24 months of a likely architecture change—say, a PostgreSQL 13 cluster that will migrate to Aurora PostgreSQL 16 within the RI term. Standard RIs would strand that commitment; Convertible RIs let you exchange them.

AWS also separates RI scope into **Regional** and **Zonal**: - Regional RIs apply to any instance of the matching type in any AZ within that region and get Availability Zone flexibility - Zonal RIs apply to a specific AZ only, but guarantee capacity reservation in that AZ—useful if you're running in capacity-constrained regions

For production databases, Regional scope is almost always correct unless you have a specific capacity reservation requirement.

Checking your current RDS RI inventory and utilization:

```
# List all current RDS Reserved Instances
aws rds describe-reserved-db-instances \
  --region us-east-1 \
  --output table \
  --query 'ReservedDBInstances[*].{
    ID:ReservedDBInstanceId,
    Class:DBInstanceClass,
    Engine:ProductDescription,
    MultiAZ:MultiAZ,
    State:State,
    StartTime:StartTime,
    Duration:Duration,
    FixedPrice:FixedPrice,
    UsagePrice:UsagePrice,
    Count:DBInstanceCount
  }'

# List RI offerings available for purchase (useful for comparing options)
aws rds describe-reserved-db-instances-offerings \
  --region us-east-1 \
  --db-instance-class db.r6g.2xlarge \
  --product-description "postgresql" \
  --multi-az \
  --offering-type "Partial Upfront" \
  --output table \
  --query 'ReservedDBInstancesOfferings[*].{
    OfferingID:ReservedDBInstancesOfferingId,
```

```
Class:DBInstanceClass,
Duration:Duration,
FixedPrice:FixedPrice,
UsagePrice:UsagePrice,
MultiAZ:MultiAZ
}',

# Purchase a 1-year Partial Upfront RI (replace offering-id with actual ID from above)
aws rds purchase-reserved-db-instances-offering \
  --reserved-db-instances-offering-id "offering-id-from-previous-command" \
  --reserved-db-instance-id "prod-postgres-primary-ri-2024" \
  --db-instance-count 1 \
  --region us-east-1
```

Azure SQL Reserved Capacity

Azure sells reserved capacity for SQL Database and SQL Managed Instance at the vCore level. A reservation for 8 vCores of General Purpose covers any General Purpose database in the selected region that together consume 8 vCores—whether that’s one 8-vCore database or two 4-vCore databases. This pooling behavior is more flexible than AWS’s per-instance model.

```
# View Azure SQL reserved capacity offerings
az reservations catalog show \
  --service-name "sql" \
  --location eastus \
  --output table

# Purchase a 1-year reservation for 8 vCores General Purpose SQL Database
# Note: az reservations requires the reservations extension
az extension add --name reservations

az reservations reservation-order purchase \
  --reservation-order-id "${uuidgen}" \
  --sku "SQL_Database_General_Purpose_Compute_8_vCores" \
  --location eastus \
  --reserved-resource-type "SqlDatabases" \
  --billing-scope "/subscriptions/your-subscription-id" \
  --term P1Y \
  --billing-plan Upfront \
  --quantity 1 \
  --display-name "prod-sqlldb-gp-8vcores-1yr"
```

Google Cloud Committed Use Discounts

GCP’s CUD model commits to a resource quantity (vCPUs and memory) rather than a specific instance type. This means a 3-year CUD for 8 vCPUs and 32 GB of memory in `us-central1` can cover any Cloud SQL instance in that region as long as the aggregate resource consumption matches the committed amount.

```
# Create a 1-year Committed Use Discount for Cloud SQL compute
# CUDs are created at the project level via the Compute Engine API
gcloud compute commitments create prod-cloudsql-commitment \
  --plan 12-month \
  --region us-central1 \
  --resources vcpu=8,memory=32GB \
  --type GENERAL_PURPOSE \
  --project your-project-id

# View existing commitments and their utilization
gcloud compute commitments list \
  --region us-central1 \
  --project your-project-id \
  --format="table(name, plan, status, startTimestamp, endTimestamp)"
```

38.4 Sizing Commitments Against Real Workload Data

The most expensive RI mistake is buying capacity based on current provisioning rather than actual utilization. If you're running a `db.r6g.4xlarge` at 15% average CPU utilization because the instance was provisioned for peak load three years ago, committing to that instance class locks in overspend.

Before any RI purchase, pull at least 90 days of utilization data. For AWS RDS, that data lives in CloudWatch.

```
import boto3
from datetime import datetime, timedelta
import statistics

def get_rds_utilization_stats(db_instance_id, region, days=90):
    """
    Retrieves CPU, FreeableMemory, DatabaseConnections, and ReadIOPS/WriteIOPS
    for the last N days to inform RI sizing decisions.
    """
    cloudwatch = boto3.client('cloudwatch', region_name=region)

    end_time = datetime.utcnow()
    start_time = end_time - timedelta(days=days)

    metrics = {
        'CPUUtilization': {'stat': 'Average', 'unit': 'Percent'},
        'FreeableMemory': {'stat': 'Average', 'unit': 'Bytes'},
        'DatabaseConnections': {'stat': 'Maximum', 'unit': 'Count'},
        'ReadIOPS': {'stat': 'Average', 'unit': 'Count/Second'},
        'WriteIOPS': {'stat': 'Average', 'unit': 'Count/Second'},
    }

    results = {}

    for metric_name, config in metrics.items():
        response = cloudwatch.get_metric_statistics(
            Namespace='AWS/RDS',
            MetricName=metric_name,
            Dimensions=[{'Name': 'DBInstanceIdentifier', 'Value': db_instance_id}],
            StartTime=start_time,
            EndTime=end_time,
            Period=3600, # 1-hour granularity
            Statistics=[config['stat']]
        )

        datapoints = sorted(response['Datapoints'], key=lambda x: x['Timestamp'])
        values = [dp[config['stat']] for dp in datapoints]

        if values:
            results[metric_name] = {
                'mean': round(statistics.mean(values), 2),
                'median': round(statistics.median(values), 2),
                'p95': round(sorted(values)[int(len(values) * 0.95)], 2),
                'p99': round(sorted(values)[int(len(values) * 0.99)], 2),
                'max': round(max(values), 2),
                'samples': len(values)
            }

    return results

def recommend_instance_class(stats, current_instance_class):
    """
    Provides a rough right-sizing recommendation based on CPU utilization.
    In practice, combine with memory pressure and connection count analysis.
    """
    cpu_p95 = stats.get('CPUUtilization', {}).get('p95', 100)
    cpu_mean = stats.get('CPUUtilization', {}).get('mean', 100)
```

```

recommendation = current_instance_class
notes = []

if cpu_p95 < 20:
    notes.append(f"P95 CPU {cpu_p95}% - strong candidate for downsizing one instance class")
elif cpu_p95 < 40:
    notes.append(f"P95 CPU {cpu_p95}% - current sizing is adequate; commit at this level")
elif cpu_p95 < 70:
    notes.append(f"P95 CPU {cpu_p95}% - healthy utilization; commit at this level")
else:
    notes.append(f"P95 CPU {cpu_p95}% - consider upsizing before committing")

return recommendation, notes

# Example usage
db_instance_id = 'prod-postgres-primary'
region = 'us-east-1'
current_class = 'db.r6g.4xlarge'

print(f"Analyzing {db_instance_id} over 90 days...")
stats = get_rds_utilization_stats(db_instance_id, region, days=90)

for metric, values in stats.items():
    print(f"\n{metric}:")
    print(f"    Mean: {values['mean']} | Median: {values['median']}")
    print(f"    P95: {values['p95']} | P99: {values['p99']} | Max: {values['max']}")

recommended_class, notes = recommend_instance_class(stats, current_class)
print(f"\nRight-sizing analysis for {current_class}:")
for note in notes:
    print(f"    → {note}")

```

For Azure SQL, the equivalent utilization data comes from Azure Monitor. The `sys.dm_db_resource_stats` DMV provides per-database resource consumption at 15-second intervals for the last hour; `sys.resource_stats` in the master database covers the last 14 days at 1-minute granularity.

```

-- Azure SQL Database: 14-day resource utilization summary
-- Run this against the master database of your Azure SQL logical server
SELECT
    database_name,
    CAST(start_time AS DATE) AS sample_date,
    ROUND(AVG(avg_cpu_percent), 2) AS avg_cpu_pct,
    ROUND(MAX(avg_cpu_percent), 2) AS max_cpu_pct,
    ROUND(AVG(avg_memory_usage_percent), 2) AS avg_mem_pct,
    ROUND(MAX(avg_memory_usage_percent), 2) AS max_mem_pct,
    ROUND(AVG(avg_data_io_percent), 2) AS avg_data_io_pct,
    ROUND(MAX(avg_data_io_percent), 2) AS max_data_io_pct,
    MAX(max_session_count) AS peak_sessions,
    COUNT(*) AS sample_count
FROM
    sys.resource_stats
WHERE
    start_time >= DATEADD(DAY, -14, GETUTCDATE()) -- sys.resource_stats retains 14 days
GROUP BY
    database_name,
    CAST(start_time AS DATE)
ORDER BY
    database_name,
    sample_date DESC;

-- Identify databases consistently below 20% CPU - right-sizing candidates
-- PERCENTILE_CONT is a per-row windowed value, so compute it in a CTE
-- before aggregating - combining it directly with GROUP BY would collapse
-- each database to a single row before the percentile has anything to work over
WITH per_row_percentiles AS (
    SELECT

```



```

        database_name,
        avg_cpu_percent,
        PERCENTILE_CONT(0.95) WITHIN GROUP
            (ORDER BY avg_cpu_percent) OVER
            (PARTITION BY database_name) AS p95_cpu_pct
    FROM sys.resource_stats
    WHERE start_time >= DATEADD(DAY, -14, GETUTCDATE()) -- sys.resource_stats retains 14 days
)
SELECT
    database_name,
    ROUND(AVG(avg_cpu_percent), 2) AS avg_cpu_pct,
    ROUND(MAX(p95_cpu_pct), 2) AS p95_cpu_pct,
    ROUND(MAX(avg_cpu_percent), 2) AS max_cpu_pct
FROM per_row_percentiles
GROUP BY database_name
HAVING
    AVG(avg_cpu_percent) < 20
    AND MAX(avg_cpu_percent) < 40
ORDER BY
    avg_cpu_pct ASC;

```

For Google Cloud SQL, utilization data is available through Cloud Monitoring (formerly Stackdriver). The `gcloud` CLI can pull metric time series, but for bulk analysis the Python client is more practical:

```

from google.cloud import monitoring_v3
from datetime import datetime, timedelta
import time

def get_cloudsql_utilization(project_id, instance_id, region, days=90):
    """
    Retrieves CPU utilization and memory utilization for a Cloud SQL instance
    over the specified window.
    """
    client = monitoring_v3.MetricServiceClient()
    project_name = f"projects/{project_id}"

    end_time = datetime.utcnow()
    start_time = end_time - timedelta(days=days)

    interval = monitoring_v3.TimeInterval(
        {
            "end_time": {"seconds": int(end_time.timestamp())},
            "start_time": {"seconds": int(start_time.timestamp())},
        }
    )

    metrics_to_pull = {
        "cloudsql.googleapis.com/database/cpu/utilization": "CPU",
        "cloudsql.googleapis.com/database/memory/utilization": "Memory",
    }

    results = {}

    for metric_type, label in metrics_to_pull.items():
        aggregation = monitoring_v3.Aggregation(
            {
                "alignment_period": {"seconds": 3600},
                "per_series_aligner": monitoring_v3.Aggregation.Aligner.ALIGN_MEAN,
            }
        )

        filter_str = (
            f'metric.type="{metric_type}" '
            f'AND resource.labels.database_id="{project_id}:{instance_id}" '
        )

        results_iter = client.list_time_series(
            request={

```

```

        "name":      project_name,
        "filter":    filter_str,
        "interval":  interval,
        "view":      monitoring_v3.ListTimeSeriesRequest.TimeSeriesView.FULL,
        "aggregation": aggregation,
    }
)

values = []
for series in results_iter:
    for point in series.points:
        values.append(point.value.double_value * 100) # convert to percentage

if values:
    sorted_vals = sorted(values)
    results[label] = {
        "mean":    round(sum(values) / len(values), 2),
        "p95":     round(sorted_vals[int(len(sorted_vals) * 0.95)], 2),
        "p99":     round(sorted_vals[int(len(sorted_vals) * 0.99)], 2),
        "max":     round(max(values), 2),
        "samples": len(values),
    }

return results

# Example usage
project_id = "your-gcp-project"
instance_id = "prod-postgres-primary"
region = "us-central1"

stats = get_cloudsql_utilization(project_id, instance_id, region, days=90)
for resource, values in stats.items():
    print(f"\n{resource} Utilization (90-day):")
    print(f"  Mean: {values['mean']}% | P95: {values['p95']}% | P99: {values['p99']}% | Max: {values['max']}%")
    ↪ {values['max']}%"

```

The utilization analysis should drive the commitment quantity, not the provisioned instance size. A general rule used in practice: commit at the P75 utilization level for stable workloads, leaving headroom for P95 spikes to absorb against on-demand pricing. For databases with highly seasonal traffic—a retail OLTP database with holiday peaks, for example—consider committing at base load and letting the seasonal burst run on-demand.

38.5 Cross-Cloud Comparison: Where Each Provider’s Model Has Edges

No single cloud’s commitment model is strictly superior. The right choice depends on workload characteristics, architectural stability, and financial flexibility requirements.

AWS RDS Advantages - Convertible RIs give genuine architectural flexibility without sacrificing the full discount in one step - Per-instance granularity means you can commit selectively: reserve your three stable production primaries, leave dev/test and read replicas on-demand - Multi-AZ RIs are purchased as a single SKU; you don’t separately reserve primary and standby - Savings Plans (compute-level, not database-level) can cover EC2-based database workloads if you’re running databases on EC2 rather than RDS

Azure SQL Advantages - vCore pooling means a single reservation can float across multiple databases as they scale up and down, reducing the risk of stranded reservation capacity - The Azure Hybrid Benefit can stack on top of reservations: if you have existing SQL Server licenses

with Software Assurance, you can apply them to reduce the effective rate further by eliminating the license portion of the rate — check current Azure Hybrid Benefit terms and your license entitlement to quantify the exact reduction for your configuration - Reservation exchanges (swapping to a different SKU) are supported in Azure without the Convertible RI equivalent overhead

Google Cloud SQL Advantages - CUDs apply at the resource level (vCPUs + memory) rather than the instance level, giving the most flexibility for workloads that fluctuate in instance shape - Unlike Compute Engine VMs, Cloud SQL does not receive automatic Sustained Use Discounts — the CUD is the only discount lever for Cloud SQL compute, so there is no passive safety net for teams that fail to commit; verify current GCP documentation for your specific service before assuming any automatic discount applies - AlloyDB CUDs follow the same model as Cloud SQL, so a commitment made today survives an AlloyDB migration without re-purchasing

When On-Demand Is the Correct Answer

The math strongly favors reservations for stable production workloads, but on-demand is genuinely correct in several scenarios:

1. **Workloads under 6 months:** Even with a 1-year commitment, if there's meaningful probability the workload ends or changes significantly within 6–9 months, on-demand is lower total risk.
2. **Active migration projects:** A database in the middle of a cloud-to-cloud or on-prem-to-cloud migration is not a stable commitment candidate. Buy the RI 30–60 days after the migration stabilizes.
3. **Development and testing environments:** Unless your dev environment is truly persistent and large enough to make the math work, on-demand keeps options open for scaling down or pausing instances.
4. **Pilot workloads on a new engine:** If you're evaluating Aurora DSQL, AlloyDB, or any new database product, you don't yet know the steady-state instance class. Running on-demand through the evaluation period is the correct approach.

38.6 Auditing Existing Commitments for Waste

Just as important as making the right purchase decision is monitoring whether existing commitments are being utilized. Stranded Reserved Instances—reservations covering instances that were resized, migrated, or decommissioned—are a common source of invisible waste.

AWS: Checking RI Utilization via Cost Explorer

AWS Cost Explorer provides RI utilization reports through both the console and the API. The API is useful for scheduled reporting.

```
import boto3
from datetime import datetime, timedelta

def get_ri_utilization_report(region, lookback_days=30):
    """
    Pulls RI utilization from AWS Cost Explorer.
    Flags any RI with utilization below 80% as potentially stranded.
    """
    ce_client = boto3.client('ce', region_name='us-east-1') # CE API is global, endpoint is us-east-1
```

```

end_date = datetime.utcnow().strftime('%Y-%m-%d')
start_date = (datetime.utcnow() - timedelta(days=lookback_days)).strftime('%Y-%m-%d')

response = ce_client.get_reservation_utilization(
    TimePeriod={
        'Start': start_date,
        'End': end_date,
    },
    GroupBy=[
        {'Type': 'DIMENSION', 'Key': 'SUBSCRIPTION_ID'},
    ],
    Filter={
        'Dimensions': {
            'Key': 'SERVICE',
            'Values': ['Amazon Relational Database Service'],
        }
    }
)

print(f"\nRDS RI Utilization Report ({start_date} to {end_date})")
print("=" * 70)

total_purchased_hours = 0
total_utilized_hours = 0
stranded_reservations = []

for group in response.get('UtilizationsByTime', []):
    for item in group.get('Groups', []):
        sub_id = item['Keys'][0]
        util = item['Utilization']

        purchased_hours = float(util.get('PurchasedHours', 0))
        utilized_hours = float(util.get('TotalActualHours', 0))
        util_pct = float(util.get('UtilizationPercentage', 0))
        unused_cost = float(util.get('UnusedAmortizedUpfrontFeeForBillingPeriod', 0)) \
            + float(util.get('UnusedRecurringFee', 0))

        total_purchased_hours += purchased_hours
        total_utilized_hours += utilized_hours

        status = " OK" if util_pct >= 80 else " REVIEW"
        print(f"{status} Subscription: {sub_id[:20]:<22} "
              f"Utilization: {util_pct:5.1f}% "
              f"Unused cost: ${unused_cost:,.2f}")

        if util_pct < 80:
            stranded_reservations.append({
                'subscription_id': sub_id,
                'utilization_pct': util_pct,
                'unused_cost': unused_cost
            })

overall_util = (total_utilized_hours / total_purchased_hours * 100) \
    if total_purchased_hours > 0 else 0
print(f"\nOverall RDS RI Utilization: {overall_util:.1f}%")

if stranded_reservations:
    total_stranded_cost = sum(r['unused_cost'] for r in stranded_reservations)
    print(f"\n {len(stranded_reservations)} reservation(s) below 80% utilization")
    print(f" Estimated wasted spend this period: ${total_stranded_cost:,.2f}")
    print(" Consider modifying, exchanging (Convertible only), or ")
    print(" listing on the AWS Reserved Instance Marketplace.")

return stranded_reservations

stranded = get_ri_utilization_report('us-east-1', lookback_days=30)

```

Azure: Reservation Utilization via Azure Monitor

Azure surfaces reservation utilization through the Cost Management API. A reservation utilization below 100% means you're paying for vCore-hours you're not consuming.

```
# Get reservation utilization summary for the past 30 days
az consumption reservation summary list \
  --reservation-order-id "your-reservation-order-id" \
  --reservation-id "your-reservation-id" \
  --grain monthly \
  --start-date "$(date -d '30 days ago' +%Y-%m-%d)" \
  --end-date "$(date +%Y-%m-%d)" \
  --output table \
  --query '[].{
    Period:usageDate,
    PurchasedQty:minUtilizationPercentage,
    AvgUtil:avgUtilizationPercentage,
    MaxUtil:maxUtilizationPercentage
  }'
```

For Azure SQL specifically, if a reservation is consistently below utilization, the correct remediation is to exchange it for a smaller vCore count reservation—Azure allows this without penalty during the reservation term.

Google Cloud: Commitment Utilization

GCP commitment utilization is visible in the Cloud Billing reports and via the gcloud CLI:

```
# View commitment utilization breakdown
gcloud compute commitments describe prod-cloudsql-commitment \
  --region us-central1 \
  --project your-project-id \
  --format="yaml(name, plan, status, resources, startTimestamp, endTimestamp)"

# For programmatic access to billing data, use the Cloud Billing Budget API
# or export billing data to BigQuery and query it directly
bq query --use_legacy_sql=false \
'SELECT
  service.description           AS service,
  sku.description               AS sku,
  SUM(usage.amount)             AS total_usage,
  SUM(cost)                     AS total_cost,
  SUM(credits.amount)           AS total_credits
FROM
  `your-billing-project.billing_export.gcp_billing_export_v1_XXXXXX`
WHERE
  DATE(usage_start_time) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
  AND service.description LIKE "%SQL%"
GROUP BY
  1, 2
ORDER BY
  total_cost DESC
LIMIT 20'
```

38.7 The RI Purchase Workflow for Production Teams

The mechanics of the analysis matter less than having a repeatable process that prevents both panic-buying and perpetual deferral. The following workflow can be run quarterly and produces a prioritized recommendation list ready for finance approval.

Step 1: Inventory all running database instances

Pull a complete list of running database instances across all accounts and regions, including their instance class, engine, Multi-AZ status, and whether they're currently covered by an RI.

```
# AWS: inventory all RDS instances across regions
for region in $(aws ec2 describe-regions --query 'Regions[].RegionName' --output text); do
  echo "=== $region ==="
  aws rds describe-db-instances \
    --region "$region" \
    --query 'DBInstances[*].{
      ID:DBInstanceIdentifier,
      Class:DBInstanceClass,
      Engine:Engine,
      EngineVersion:EngineVersion,
      MultiAZ:MultiAZ,
      Status:DBInstanceStatus,
      Storage:AllocatedStorage
    }' \
    --output table 2>/dev/null
done

# Cross-reference against existing RIs to find uncovered instances
aws rds describe-reserved-db-instances \
  --region us-east-1 \
  --query 'ReservedDBInstances[?State==`active`].{
    Class:DBInstanceClass,
    Engine:ProductDescription,
    MultiAZ:MultiAZ,
    Count:DBInstanceCount,
    Expiry:StartTime
  }' \
  --output table
```

Step 2: Score each instance on commitment suitability

Not all instances deserve the same commitment strategy. Score each candidate on three axes:

Axis	Low Suitability (0)	High Suitability (2)
Workload stability	Active migration in progress	Running unchanged 12+ months
Utilization efficiency	P95 CPU < 20% (likely to be right-sized)	P95 CPU 40–70%
Time horizon	Architectural change expected < 9 months	No changes planned for 18+ months

Instances with a total score of 4 or higher are strong RI candidates. Instances with scores of 2 or below should run on-demand until conditions improve.

Step 3: Calculate the purchase recommendation

For each high-scoring instance, run the break-even and annual savings calculation from Section 38.2. Produce a ranked list sorted by total annual savings, not by savings percentage—a 38% discount on a large instance produces more absolute dollars saved than a 40% discount on a small one.

Step 4: Stage the purchase

Don't commit the entire inventory in one transaction. A staged approach reduces risk:

- **Week 1:** Purchase RIs for instances with scores of 6 (maximum) that have been running stable for 18+ months
- **Month 1:** Add instances with scores of 4–5 after confirming no architectural changes are imminent
- **Quarter 2:** Reassess remaining on-demand instances for newly earned stability

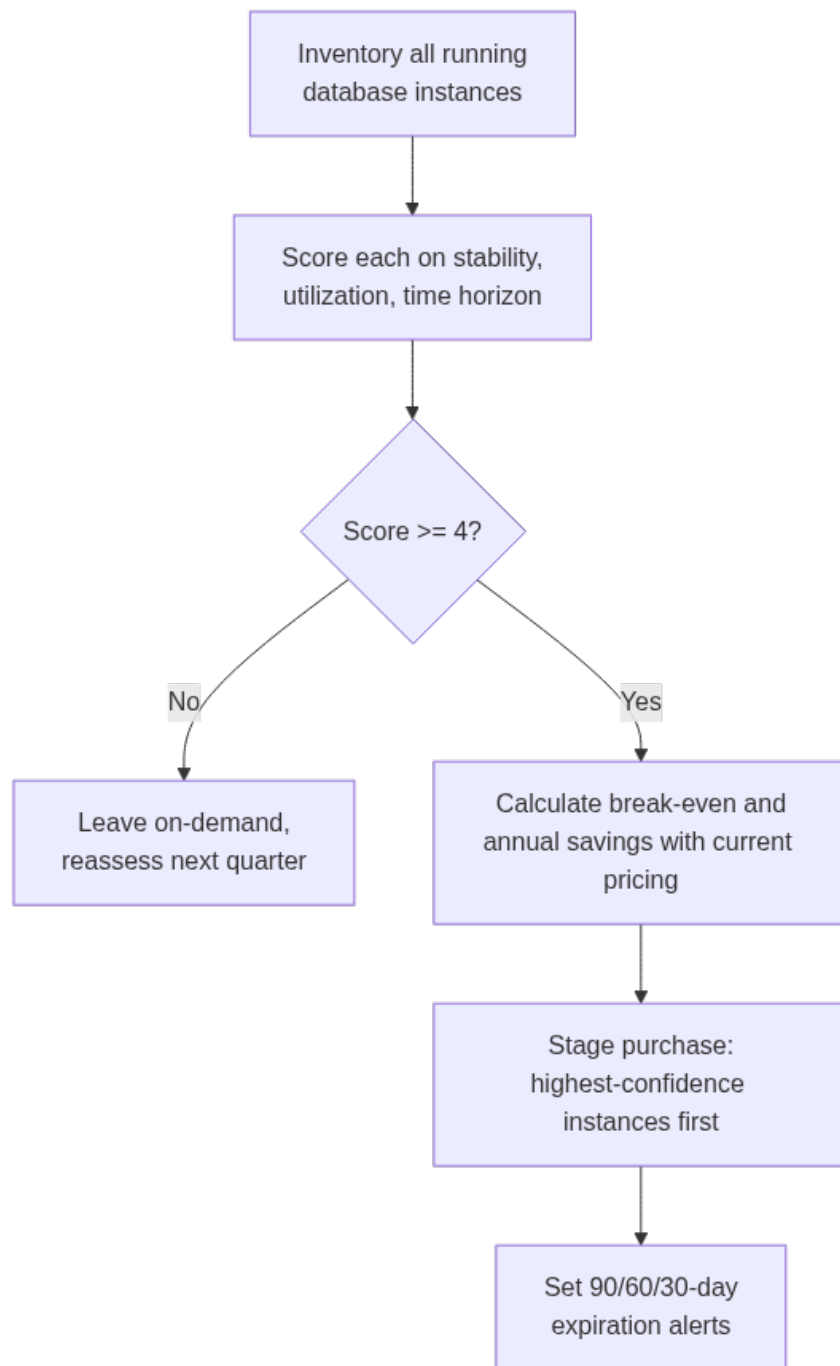


Figure 29: A repeatable quarterly process beats a one-time bulk purchase decision — staging limits exposure if workload assumptions turn out wrong.

For AWS, if you're uncertain between Standard and Convertible RIs, run the first purchase as Convertible. The discount gap is modest, and the exchange flexibility is genuinely valuable during initial portfolio optimization.

Step 5: Set expiration alerts

RI expirations are a known risk: a reservation that expires unnoticed flips the instance back to on-demand rates silently. Set calendar alerts 90, 60, and 30 days before each RI expiration date. For AWS, you can automate this with EventBridge:

```
# Create a CloudWatch alarm that fires when an RI is expiring within 60 days
# This uses the AWS Health API to surface RI expiration events
aws events put-rule \
  --name "RDSReservedInstanceExpirationAlert" \
  --event-pattern '{
    "source": ["aws.health"],
    "detail-type": ["AWS Health Event"],
    "detail": {
      "service": ["RDS"],
      "eventTypeCategory": ["scheduledChange"],
      "eventTypeCode": ["AWS_RDS_RESERVED_INSTANCE_EXPIRATION_SCHEDULED"]
    }
  }' \
  --state ENABLED \
  --region us-east-1

# Attach an SNS topic to the rule for notification
aws events put-targets \
  --rule "RDSReservedInstanceExpirationAlert" \
  --targets '[{
    "Id": "1",
    "Arn": "arn:aws:sns:us-east-1:123456789012:db-ops-alerts"
  }]' \
  --region us-east-1
```

Key Takeaways

- **The discount is real and substantial, but so is the commitment risk:** across all three major clouds, 1-year commitments deliver a meaningfully lower effective hourly rate than on-demand, and 3-year commitments deliver a lower rate still — but the exact percentage varies by provider, instance class, region, and payment option, and changes as providers update pricing. Pull current numbers before committing, and only after the workload runs as expected. Analyze workload stability before size, and size before committing.
- **Utilization data must precede the purchase:** Committing to an instance class based on provisioned size rather than 90-day utilization history locks in overspend for the entire term. Pull P95 CPU and memory data, right-size first, then commit.
- **Each cloud's commitment model has distinct flexibility characteristics:** AWS Convertible RIs allow mid-term exchanges at the cost of a modest discount reduction; Azure vCore pooling floats coverage across multiple databases automatically; GCP CUDs apply at the resource level and survive instance type changes. Match the commitment type to your architectural confidence level.
- **Stranded reservations are as costly as missing reservations:** A quarterly audit of RI utilization using Cost Explorer (AWS), Azure Cost Management, or GCP Billing export will

surface underutilized commitments early enough to exchange or sell them before the waste compounds.

- **Build a repeatable purchase workflow, not a one-time event:** Inventory all instances, score them on stability and utilization efficiency, calculate ranked savings, stage purchases over weeks rather than days, and set automated expiration alerts. Organizations that treat RI management as an ongoing operational process consistently outperform those that approach it as an annual finance exercise.
-

Chapter 39: Right-Sizing Cloud Database Instances

Cloud billing has a way of focusing the mind. On-premises, a server purchased three years ago costs the same whether it runs at 8% CPU or 95%. In the cloud, every vCPU-hour, every gigabyte of provisioned IOPS, and every GB of storage is metered continuously. Right-sizing — selecting the correct instance class, storage configuration, and compute tier for each workload — is therefore not an optimization exercise you do once during migration. It is an ongoing engineering discipline that directly affects both application performance and monthly spend. This chapter covers the mechanics of right-sizing across AWS RDS and Aurora, Azure SQL Database, and Google Cloud SQL and AlloyDB: how to read the signals that indicate a mismatch between workload and instance, how to resize with minimal disruption, and how to build the automation that keeps sizing decisions current as workloads evolve.

39.1 Understanding the Cloud Instance Taxonomy

Before you can right-size anything, you need a working mental model of how cloud providers organize their compute offerings. Every managed database service presents a hierarchy: a general-purpose tier, a memory-optimized tier, and — on some platforms — a burstable tier that has no on-premises equivalent at all.

AWS RDS instance families follow the EC2 naming convention. `db.t3.medium` and `db.t4g.medium` are burstable instances that accumulate CPU credits when idle and burn them during bursts. `db.m7g` and `db.m6i` are the general-purpose workhorses. `db.r7g`, `db.r6i`, and `db.x2idn` are memory-optimized, relevant when your buffer pool needs to hold more than 32 GB in RAM. Aurora uses the same families but the underlying hardware is shared; Aurora Serverless v2 adds a fractional ACU model that sits orthogonally to the instance class hierarchy.

Azure SQL Database abstracts this differently. In the General Purpose service tier, you choose vCores (2 through 128) and the memory scales with them at a fixed ratio (approximately 5.1 GB per vCore in Gen5). The Business Critical tier uses local SSD and adds a readable secondary replica at no extra compute cost. Hyperscale decouples storage from compute entirely and adds independent read replicas called named replicas. The DTU model — which you will encounter on older provisioned databases — bundles compute, memory, and I/O into a single opaque unit; avoid it for new workloads because it gives you no visibility into which resource is actually the constraint.

Google Cloud SQL uses machine types derived from Compute Engine: shared-core for develop-

ment (db-f1-micro, db-g1-small), standard (db-n1-standard-), and high-memory (db-n1-highmem-). AlloyDB uses a separate tier system anchored to vCPUs, with columnar memory and a distributed cache that changes the RAM sizing calculus significantly compared to vanilla Cloud SQL.

The mapping that helps on-premises DBAs most: think of memory-to-vCPU ratio, not raw core count. A `db.r6i.2xlarge` gives you 8 vCPUs and 64 GB RAM (8:1). A `db.m6i.2xlarge` gives you 8 vCPUs and 32 GB RAM (4:1). If your workload is read-heavy and buffer-pool-bound, the r6i is the right family even if it costs more per vCore. If your workload is CPU-bound — complex reporting queries, high connection rates, stored procedure execution — the m6i is more efficient.

Burstable instances deserve special attention. `db.t3.medium` (2 vCPU, 4 GB RAM) looks attractive for small workloads, and it often is appropriate for development and small OLTP databases. The risk is CPU credit exhaustion. When credits run out, performance drops to the CPU baseline, which for `db.t3.medium` is 20% of a single vCPU. On-premises you never had a server that suddenly ran at one-fifth its rated speed for sustained periods because the accounting department decided the bill was too high. In the cloud, this happens silently if you do not monitor credit balance.

```
-- Query to estimate buffer pool hit ratio on PostgreSQL (RDS or Cloud SQL)
-- A ratio below 0.98 on a read-heavy OLTP workload suggests the instance
-- needs more RAM, not necessarily more CPU.
SELECT
    sum(heap_blks_hit) AS heap_hit,
    sum(heap_blks_read) AS heap_read,
    round(
        sum(heap_blks_hit)::numeric /
        nullif(sum(heap_blks_hit) + sum(heap_blks_read), 0),
        4
    ) AS hit_ratio
FROM pg_statio_user_tables;
```

On a well-sized instance running a warm OLTP workload, this ratio should be 0.985 or higher. Values below 0.95 indicate the working set is larger than the buffer pool, and the correct response is a memory upgrade before anything else.

39.2 Identifying Sizing Mismatches from Cloud Metrics

Right-sizing begins with observation, and cloud monitoring platforms expose far more granular data than most on-premises monitoring setups. The challenge is knowing which metrics to read and what thresholds actually indicate a problem versus normal variation.

On AWS, CloudWatch provides the primary signal set for RDS. The metrics that matter most for sizing decisions are:

- **CPUUtilization** — obvious, but interpret it in the context of the instance class. 70% CPU on a `db.r6i.8xlarge` is different from 70% on a `db.t3.medium`.
- **FreeableMemory** — the absolute free RAM plus cache/buffers that can be reclaimed. If this drops below 10–15% of total instance RAM consistently during business hours, you are working the buffer pool hard.
- **ReadIOPS / WriteIOPS** — compare against your provisioned IOPS. If you are consistently at 90%+ of provisioned IOPS, you either need more IOPS or a faster storage type.
- **DatabaseConnections** — watch for this approaching the `max_connections` limit. On RDS PostgreSQL, `max_connections` defaults to a formula based on RAM (`LEAST({DBInstanceClassMemory/9531392}, 5000)`), which works out to roughly 3,600

connections on a `db.r6i.xlarge` (32 GiB of `DBInstanceClassMemory` divided by 9,531,392 bytes). Once you approach 80% of that number with real connections, connection pressure becomes a latency multiplier.

- **CPUCreditBalance** (burstable only) — if this trends toward zero during business hours, you have chronic CPU credit exhaustion, and the correct fix is moving to a non-burstable instance.

*# Retrieve RDS CPUUtilization average over the last 7 days using AWS CLI
Useful for periodic right-sizing reviews in a weekly cron or CI pipeline*

```
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name CPUUtilization \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-01 \
  --start-time $(date -u -d '7 days ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 3600 \
  --statistics Average Maximum \
  --output table
```

Also pull FreeableMemory to pair with CPU

```
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name FreeableMemory \
  --dimensions Name=DBInstanceIdentifier,Value=prod-postgres-01 \
  --start-time $(date -u -d '7 days ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 3600 \
  --statistics Minimum \
  --output table
```

On Azure SQL, the equivalent layer is Azure Monitor with Query Performance Insight built on top of Query Store. The metrics that drive sizing decisions are:

- **cpu_percent** — percentage of DTUs or vCores consumed. Sustained values above 80% warrant investigation.
- **physical_data_read_percent** — I/O consumption relative to the tier limit.
- **log_write_percent** — transaction log throughput as a percentage of tier limit; this is the metric most commonly missed by DBAs moving from on-premises, where log throughput limits were rarely a concern.
- **workers_percent** — percentage of the maximum worker thread count consumed. If this approaches 100%, connections will queue even if CPU is idle.

On Google Cloud SQL, Cloud Monitoring exposes metrics under the `cloudsql.googleapis.com` namespace. Particularly useful for right-sizing:

- **database/cpu/utilization** — normalized 0–1.
- **database/memory/utilization** — includes buffer cache in the denominator.
- **database/disk/read_ops_count** and **database/disk/write_ops_count** — raw I/O operations, not normalized to a tier limit.

For AlloyDB, the columnar memory pool (the HTAP layer) has its own memory allocation separate from PostgreSQL’s `shared_buffers`. You need to monitor both `database/memory/components` to understand whether columnar memory pressure is evicting PostgreSQL buffer pool pages.

AWS Compute Optimizer is worth enabling even if you are skeptical of automated recommendations. It ingests 14 days of CloudWatch metrics and produces a recommendation per instance that categorizes the current state as Under-provisioned, Over-provisioned, or Optimized, along with the projected cost and performance impact of moving to the recommended instance. These

recommendations are not always correct — they cannot know about once-monthly batch jobs that do not appear in the 14-day window — but they are a useful starting point that on-premises DBAs would have had to build manually.

39.3 Vertical Scaling: Resizing the Instance

When you have identified a mismatch — too little CPU, too little RAM, or wrong storage — the corrective action on a managed database is usually a vertical resize. This is simpler than on-premises (no hardware procurement, no OS reinstall) but it is not zero-impact. Understanding the downtime windows and how to minimize them is the operational skill.

RDS instance class changes require a brief outage. AWS performs the change by stopping the instance, modifying it, and restarting it. The typical duration is 3–10 minutes, though the official documentation declines to commit to specific numbers because workload state (dirty buffer pool pages to flush, active transactions to drain) affects the actual time. If you apply the change immediately during a maintenance window, that window is the impact boundary. If you apply it via `modify-db-instance` with `--apply-immediately`, it happens at the next low-activity moment that AWS determines — which in practice means during your next call to `modify-db-instance` for any reason, which is easy to forget.

```
# Resize an RDS PostgreSQL instance to db.r6i.2xlarge
# --no-apply-immediately means the change waits for the next maintenance window
# Remove that flag (or use --apply-immediately) to resize at the next opportunity

aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --db-instance-class db.r6i.2xlarge \
  --no-apply-immediately \
  --region us-east-1

# Verify the pending modification
aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-01 \
  --query 'DBInstances[0].PendingModifiedValues' \
  --output json
```

For Aurora, the vertical resize story is better. In Aurora Serverless v2, scaling is continuous and can happen without a restart: the instance scales from the minimum ACU to the maximum ACU as load increases, in increments as small as 0.5 ACUs. One Aurora Capacity Unit is approximately 2 GB of RAM and a proportional share of CPU. A minimum of 0.5 ACUs with a maximum of 128 ACUs covers the range from a tiny dev database to a 256 GB RAM instance. The catch: Aurora Serverless v2 does not scale to zero (unlike v1), and the minimum of 0.5 ACUs means you are paying for that perpetually — check the current per-ACU-hour rate for your region before budgeting, since it is a meaningful recurring floor cost even for an idle cluster.

For provisioned Aurora instances, the resize still requires a brief restart, but if you have Aurora read replicas, you can resize the replicas first, promote one to primary via a failover, resize the old primary (now a replica), and keep application downtime to the 15–30 seconds of the failover operation rather than the full instance restart time.

Azure SQL vertical scaling is the smoothest across all three platforms because it is a service operation rather than a VM resize. Changing vCores in the General Purpose tier takes roughly 5–10 seconds for in-memory state transfer; the connection is briefly interrupted. Changing service

tiers (e.g., General Purpose to Business Critical) takes longer — 10–30 minutes — because the underlying storage layer changes.

```
# Scale an Azure SQL Database from GP_Gen5_4 to GP_Gen5_8 (vCores)
az sql db update \
  --resource-group prod-rg \
  --server prod-sqlserver \
  --name prod-database \
  --service-objective GP_Gen5_8 \
  --no-wait

# Monitor the scaling operation
az sql db show \
  --resource-group prod-rg \
  --server prod-sqlserver \
  --name prod-database \
  --query '{status: status, serviceLevelObjective: currentServiceObjectiveName}' \
  --output json
```

Google Cloud SQL instance tier changes require the instance to restart. For standard instances (non-HA), this is a brief outage. For HA instances (primary + standby), Cloud SQL performs a failover to the standby, resizes the old primary, then fails back — similar to the Aurora replica promotion pattern but managed transparently. The total time is typically 2–5 minutes.

Terraform management of instance sizing deserves attention because many production environments manage infrastructure as code, and a size change that happens outside Terraform will create drift. The pattern is to encode the instance class in a variable and use a `terraform plan + terraform apply` workflow for all resizes:

```
# terraform/rds/variables.tf
variable "db_instance_class" {
  description = "RDS instance class - change this variable to trigger a resize"
  type        = string
  default     = "db.r6i.xlarge"
}

variable "apply_immediately" {
  description = "Apply instance changes immediately vs next maintenance window"
  type        = bool
  default     = false
}

# terraform/rds/main.tf
resource "aws_db_instance" "prod_postgres" {
  identifier      = "prod-postgres-01"
  engine          = "postgres"
  engine_version  = "15.4"
  instance_class  = var.db_instance_class
  allocated_storage = 500
  max_allocated_storage = 2000
  storage_type    = "gp3"
  iops            = 6000
  apply_immediately = var.apply_immediately
  maintenance_window = "Sun:03:00-Sun:04:00"
  backup_window    = "01:00-02:00"
  backup_retention_period = 7

  # Prevent accidental deletion in production
  deletion_protection = true

  tags = {
    Environment = "production"
    CostCenter   = "engineering"
    LastResizeDate = "2024-11"
  }
}
```

Tagging instances with the last resize date is a low-tech but effective practice. When you pull cost reports and see a large instance that has not been reviewed in 18 months, the tag tells you whether the sizing decision was intentional or inherited.

39.4 Storage Right-Sizing: IOPS, Throughput, and Autoscaling

Storage is frequently the second-largest cost item on a cloud database bill, and it is the area where DBAs most commonly over-provision because the on-premises intuition (provision for peak, never run out) collides with cloud storage pricing that charges for provisioned capacity and IOPS regardless of usage.

AWS RDS storage types:

- **gp2** — general-purpose SSD. IOPS scale with volume size: 3 IOPS per GB, minimum 100, maximum 16,000. If you provision 500 GB of gp2 storage, you get exactly 1,500 IOPS. This coupling of size to IOPS was the dominant reason teams over-provisioned storage.
- **gp3** — the 2022 successor to gp2. IOPS and throughput are independently configurable. The baseline is 3,000 IOPS and 125 MB/s included in the storage price. You can provision up to 16,000 IOPS and 1,000 MB/s separately. For most workloads, gp3 is cheaper than gp2 and gives you more control. Migrate existing gp2 instances to gp3 unless you have a specific reason not to.
- **io1 / io2** — provisioned IOPS SSD. Use these when you need more than 16,000 IOPS or when sub-millisecond storage latency consistency is a hard requirement (financial systems, high-frequency OLTP). io2 Block Express supports up to 256,000 IOPS per volume at multi-attach.

The specific gotcha: **RDS storage can only be increased, never decreased**. If you provision 2 TB and decide 500 GB was sufficient, you are paying for 2 TB until you rebuild the instance. This is fundamentally different from on-premises SANs where you could reclaim LUNs. Provision conservatively and rely on Storage Autoscaling:

```
# Enable storage autoscaling on an existing RDS instance
# max-allocated-storage sets the ceiling; autoscaling triggers when free
# space drops below 10% of allocated storage (sustained for 5+ minutes),
# then grows storage by whichever is larger: 10 GB or 10% of current allocation

aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --max-allocated-storage 2000 \
  --apply-immediately

# Storage autoscaling does NOT require an instance restart -
# it is one of the few RDS modifications that is truly online
```

Azure SQL storage in the General Purpose tier is provisioned in 1 GB increments from 1 GB to 4 TB, and can be increased online. Unlike RDS, Azure SQL does allow storage reduction on the General Purpose tier (with some restrictions). The Business Critical tier uses local SSD attached to the compute node, so storage limits are tied to the instance size rather than being independently configurable — a BC_Gen5_4 instance gets 1 TB of local SSD.

Google Cloud SQL storage always starts at HDD or SSD. Use SSD for any production workload; HDD pricing is lower but the latency difference is significant. Cloud SQL supports storage autoresize by default (enabled via `--storage-auto-increase` or the console), and it grows storage

automatically when free space drops below the configured threshold — check current GCP documentation for the exact increment calculation, since it is not a fixed flat amount. The equivalent caution applies: Cloud SQL storage cannot be decreased after it has been increased.

IOPS sizing math for RDS gp3: Start with your observed peak IOPS from the source system. Add 20% headroom. Compare that to the 3,000 IOPS baseline included in gp3. If your peak is below 3,000 IOPS, gp3 at baseline is sufficient. If your peak is 8,000 IOPS, provision 10,000 IOPS on gp3. Pull the current per-provisioned-IOPS rate from AWS pricing before estimating the monthly cost — gp3's marginal IOPS rate is priced well below io1's per-IOPS rate, which combined with gp3's free 3,000-IOPS baseline and lower per-GB storage rate is what makes it the cheaper choice at equivalent IOPS; do not assume the two volume types charge the same rate per additional IOPS.

```
# Python script to calculate gp3 IOPS cost vs io1 for a given workload
# Useful for building a right-sizing cost model during migration planning

import boto3
from datetime import datetime, timedelta

def estimate_storage_cost(
    storage_gb: int,
    peak_iops: int,
    peak_throughput_mbps: int,
    region: str = "us-east-1"
) -> dict:
    """
    Compare gp3 vs io1 costs for a given storage/IOPS requirement.
    IMPORTANT: the pricing constants below are placeholders only.
    Pull current rates from the AWS Price List API (see the RDS pricing
    function earlier in this chapter) or the AWS Pricing Calculator before
    using this for a real decision - do not ship these numbers to production
    budgeting. gp3's marginal per-IOPS rate is priced well below io1's;
    they are NOT the same rate, even though both are shown here as
    placeholders pending your own lookup.
    """

    # gp3 pricing - PLACEHOLDERS, replace with current rates for your region
    GP3_STORAGE_PER_GB = None # $/GB-month - fetch current rate
    GP3_IOPS_INCLUDED = 3000
    GP3_IOPS_EXTRA = None # $/provisioned IOPS-month above 3000 - fetch current rate
    GP3_THROUGHPUT_INCL = 125 # MB/s included
    GP3_THROUGHPUT_EXTRA = None # $/MB/s-month above 125 - fetch current rate

    # io1 pricing - PLACEHOLDERS, replace with current rates for your region
    IO1_STORAGE_PER_GB = None # $/GB-month - fetch current rate
    IO1_IOPS_PER_MONTH = None # $/provisioned IOPS-month - fetch current rate

    if None in (GP3_STORAGE_PER_GB, GP3_IOPS_EXTRA, GP3_THROUGHPUT_EXTRA,
                IO1_STORAGE_PER_GB, IO1_IOPS_PER_MONTH):
        raise ValueError(
            "Populate the pricing constants above with current AWS rates "
            "before running this comparison - see get_rds_ondemand_price() "
            "earlier in this chapter for the Price List API pattern."
        )

    # gp3 cost
    gp3_storage_cost = storage_gb * GP3_STORAGE_PER_GB
    gp3_extra_iops = max(0, peak_iops - GP3_IOPS_INCLUDED)
    gp3_iops_cost = gp3_extra_iops * GP3_IOPS_EXTRA
    gp3_extra_tput = max(0, peak_throughput_mbps - GP3_THROUGHPUT_INCL)
    gp3_tput_cost = gp3_extra_tput * GP3_THROUGHPUT_EXTRA
    gp3_total = gp3_storage_cost + gp3_iops_cost + gp3_tput_cost

    # io1 cost
    io1_storage_cost = storage_gb * IO1_STORAGE_PER_GB
```

```

io1_iops_cost    = peak_iops * IO1_IOPS_PER_MONTH
io1_total        = io1_storage_cost + io1_iops_cost

return {
    "gp3": {
        "storage": round(gp3_storage_cost, 2),
        "iops":    round(gp3_iops_cost, 2),
        "throughput": round(gp3_tput_cost, 2),
        "total":   round(gp3_total, 2),
    },
    "io1": {
        "storage": round(io1_storage_cost, 2),
        "iops":    round(io1_iops_cost, 2),
        "throughput": 0.0,
        "total":   round(io1_total, 2),
    },
    "recommendation": "gp3" if gp3_total < io1_total else "io1",
    "monthly_savings_with_gp3": round(io1_total - gp3_total, 2),
}

if __name__ == "__main__":
    result = estimate_storage_cost(
        storage_gb=1000,
        peak_iops=8000,
        peak_throughput_mbps=300,
    )
    print(f"gp3 monthly total: ${result['gp3']['total']}")
    print(f"io1 monthly total: ${result['io1']['total']}")
    print(f"Recommendation: {result['recommendation']}")
    print(f"Monthly savings with gp3: ${result['monthly_savings_with_gp3']}")

```

39.5 Horizontal Scaling Patterns and Read Replica Sizing

Vertical scaling has a ceiling. Every cloud platform has a largest available instance class, and even before you reach it, the cost curve steepens sharply. When a single large instance cannot absorb the workload, horizontal scaling through read replicas is the standard answer for read-heavy workloads.

The key principle that on-premises DBAs sometimes miss: read replicas in cloud managed databases are not just disaster recovery standbys. They are production traffic receivers. Aurora allows up to 15 read replicas per cluster. RDS PostgreSQL allows up to 5 read replicas per instance. Azure SQL Hyperscale allows up to 30 named replicas. Distributing read traffic across replicas reduces the primary's CPU and I/O burden in ways that vertical scaling cannot, because a single instance's CPU throughput is bounded even at 64 vCPUs, whereas four read replicas each running at 16 vCPUs give you a higher aggregate query throughput for read workloads.

Sizing read replicas independently is one of the most under-utilized cloud features. On-premises, a standby server was typically identical to the primary because it might need to assume primary duties at any time. In the cloud, a read replica that will never be promoted can be a smaller, cheaper instance. An `db.r6i.8xlarge` primary (32 vCPUs, 256 GB RAM) does not necessarily need `db.r6i.8xlarge` replicas for reporting queries. If your reporting queries scan large tables but involve no writes, an `db.r6i.4xlarge` replica (16 vCPUs, 128 GB RAM) is often sufficient, at roughly half the compute cost.

The constraint: the replica must be able to keep up with the primary's replication stream. If the primary is generating write-heavy traffic (large bulk loads, aggressive UPDATE workloads), a smaller replica may fall behind, and replication lag becomes an application concern. Monitor `ReplicaLag`

in CloudWatch (for RDS) or `replication_delay` in `pg_stat_replication` to determine whether your replica sizing is adequate.

```
-- On the primary: check replication lag for all connected standbys
-- Works on RDS PostgreSQL, Cloud SQL PostgreSQL, AlloyDB

SELECT
  application_name,
  client_addr,
  state,
  sent_lsn,
  write_lsn,
  flush_lsn,
  replay_lsn,
  (sent_lsn - replay_lsn)      AS replication_lag_bytes,
  write_lag,
  flush_lag,
  replay_lag,
  sync_state
FROM pg_stat_replication
ORDER BY replication_lag_bytes DESC NULLS LAST;

-- On SQL Server (Azure SQL): check secondary replica lag
-- Applies to Business Critical tier (readable secondaries) and
-- Always On Availability Groups in SQL Managed Instance

SELECT
  ar.replica_server_name,
  drs.database_name,
  drs.synchronization_state_desc,
  drs.synchronization_health_desc,
  drs.last_commit_lsn,
  drs.last_hardened_lsn,
  -- Approximate lag in log blocks; convert to bytes or time via log_send_queue_size
  drs.log_send_queue_size      AS unsent_log_kb,
  drs.redo_queue_size          AS unredone_log_kb,
  drs.redo_rate                 AS redo_rate_kb_per_sec
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
  ON drs.replica_id = ar.replica_id
WHERE drs.is_local = 0
ORDER BY drs.redo_queue_size DESC;
```

Connection routing for read replicas is an application-layer concern that directly affects whether your correctly-sized replicas actually receive traffic. AWS provides a reader endpoint for Aurora clusters that load-balances across all available reader instances. RDS does not provide a managed reader endpoint for multi-AZ or read replica configurations — you need to route to the replica’s endpoint explicitly, which means your connection pool (PgBouncer, RDS Proxy, application-level logic) must distinguish read-only connections from read-write connections.

RDS Proxy can be an effective layer here. It maintains a pool of connections to both the primary and read replicas, and for Aurora it can direct read-only transactions to the reader endpoint automatically:

```
# Terraform: RDS Proxy with separate read/write and read-only endpoints
resource "aws_db_proxy" "prod_proxy" {
  name           = "prod-postgres-proxy"
  debug_logging  = false
  engine_family  = "POSTGRESQL"
  idle_client_timeout = 1800
  require_tls    = true
  role_arn       = aws_iam_role.rds_proxy_role.arn
  vpc_security_group_ids = [aws_security_group.rds_proxy_sg.id]
  vpc_subnet_ids = var.private_subnet_ids

  auth {
```

```
    auth_scheme = "SECRETS"
    description = "prod postgres credentials"
    iam_auth     = "REQUIRED"
    secret_arn   = aws_secretsmanager_secret.db_credentials.arn
  }

  tags = {
    Environment = "production"
  }
}

# Default endpoint routes to the writer
resource "aws_db_proxy_default_target_group" "prod_proxy_tg" {
  db_proxy_name = aws_db_proxy.prod_proxy.name

  connection_pool_config {
    connection_borrow_timeout      = 120
    max_connections_percent        = 100
    max_idle_connections_percent   = 50
  }
}

resource "aws_db_proxy_target" "prod_proxy_target" {
  db_proxy_name      = aws_db_proxy.prod_proxy.name
  target_group_name  = aws_db_proxy_default_target_group.prod_proxy_tg.name
  db_cluster_identifier = aws_rds_cluster.prod_aurora.id
}

# Separate read-only endpoint for reporting traffic
resource "aws_db_proxy_endpoint" "prod_proxy_readonly" {
  db_proxy_name      = aws_db_proxy.prod_proxy.name
  db_proxy_endpoint_name = "prod-postgres-proxy-ro"
  vpc_subnet_ids     = var.private_subnet_ids
  vpc_security_group_ids = [aws_security_group.rds_proxy_sg.id]
  target_role         = "READ_ONLY"
}
```

AlloyDB read pool instances work differently from standard PostgreSQL replicas. AlloyDB separates the storage layer (a distributed log-structured store) from the compute nodes. Read pool nodes read from the shared storage directly rather than replaying WAL from the primary, which eliminates most of the replication lag that affects standard PostgreSQL read replicas under heavy write workloads. This makes it safer to route time-sensitive read traffic to AlloyDB read pool instances; the data is close to current by design rather than by sufficient replica capacity. You can also scale the read pool node count without restarting the primary, making AlloyDB’s horizontal scaling particularly well-suited for workloads with variable read peaks.

39.6 Automated Right-Sizing with Scheduled Scaling

The limiting factor in most right-sizing programs is that the correct size changes over time. A database sized correctly in January may be over-provisioned in March when a batch job moves off-database, and under-provisioned in November during seasonal peak. Building automation that adjusts sizing on a schedule — or in response to metrics — addresses this without requiring manual intervention every quarter.

Aurora Serverless v2 scheduled scaling is the most elegant solution when your workload has predictable daily patterns. You can define a schedule that increases the maximum ACU capacity ahead of a known peak and reduces it after:

```
# Create an Aurora Serverless v2 scaling schedule using AWS CLI
```

```
# This raises the max capacity from 4 to 32 ACUs before business hours
# and reduces it again overnight
```

```
# Scale up before 8 AM UTC on weekdays
aws application-autoscaling put-scheduled-action \
  --service-namespace rds \
  --resource-id cluster:prod-aurora-serverless \
  --scalable-dimension rds:cluster:ReadReplicaCount \
  --scheduled-action-name weekday-scale-up \
  --schedule "cron(0 7 ? * MON-FRI *)" \
  --scalable-target-action MinCapacity=4,MaxCapacity=32
```

```
# Scale down after 8 PM UTC on weekdays
aws application-autoscaling put-scheduled-action \
  --service-namespace rds \
  --resource-id cluster:prod-aurora-serverless \
  --scalable-dimension rds:cluster:ReadReplicaCount \
  --scheduled-action-name weekday-scale-down \
  --schedule "cron(0 20 ? * MON-FRI *)" \
  --scalable-target-action MinCapacity=1,MaxCapacity=4
```

Note that Aurora Serverless v2 scales compute ACUs automatically within the min/max bounds; the scheduled action adjusts the bounds themselves. During the day, the instance can scale from 4 to 32 ACUs as needed. At night, it can scale from 1 to 4. This approach limits the maximum billing exposure while keeping headroom available during peak.

Azure SQL Elastic Jobs and Automation can be used to schedule tier changes, though tier changes in Azure SQL are more disruptive than Aurora ACU scaling. A more practical pattern on Azure is to use Elastic Pools when you have multiple databases on the same server with complementary load patterns. An Elastic Pool with a shared eDTU or vCore allocation distributes compute across databases without requiring you to right-size each database independently. A database that needs 16 vCores at 9 AM can borrow capacity from a database that only needs 4 vCores at that hour, provided the pool's total allocation covers the combined peak:

```
# Create an Azure SQL Elastic Pool and add databases to it
az sql elastic-pool create \
  --resource-group prod-rg \
  --server prod-sqlserver \
  --name prod-elastic-pool \
  --edition GeneralPurpose \
  --capacity 16 \
  --family Gen5 \
  --min-capacity 0 \
  --max-capacity 16

# Add an existing database to the pool
az sql db update \
  --resource-group prod-rg \
  --server prod-sqlserver \
  --name reporting-db \
  --elastic-pool prod-elastic-pool

# Add a second database with different peak hours
az sql db update \
  --resource-group prod-rg \
  --server prod-sqlserver \
  --name operational-db \
  --elastic-pool prod-elastic-pool
```

The prerequisite for Elastic Pools is that the databases share the same logical server. If you have databases on separate servers for isolation reasons, the pool option is not available without consolidating the servers first — a separate architectural decision.

Metric-driven auto-scaling with Lambda is the most general solution, applicable across plat-

forms and useful when workload patterns are not predictable enough to schedule in advance. The pattern is: a Lambda function runs on a CloudWatch Events schedule (every 15 minutes), reads the last 30 minutes of metrics, applies a sizing decision algorithm, and calls the modify API if the recommendation differs from the current instance class.

```
# Lambda function: auto-right-size an RDS instance based on metric signals
# Deploy with an IAM role that allows cloudwatch:GetMetricStatistics
# and rds:ModifyDBInstance, rds:DescribeDBInstances

import boto3
import json
import os
from datetime import datetime, timedelta

rds = boto3.client('rds')
cw = boto3.client('cloudwatch')
sns = boto3.client('sns')

INSTANCE_ID = os.environ['DB_INSTANCE_ID']
SNS_TOPIC_ARN = os.environ['SNS_TOPIC_ARN']

# Ordered list of instance classes from smallest to largest
# Modify this list to match the families and sizes appropriate for your workload
INSTANCE_LADDER = [
    "db.r6i.xlarge",    # 4 vCPU, 32 GB
    "db.r6i.2xlarge",   # 8 vCPU, 64 GB
    "db.r6i.4xlarge",   # 16 vCPU, 128 GB
    "db.r6i.8xlarge",   # 32 vCPU, 256 GB
    "db.r6i.16xlarge",  # 64 vCPU, 512 GB
]

CPU_SCALE_UP_THRESHOLD = 75.0 # % - scale up if avg CPU exceeds this
CPU_SCALE_DOWN_THRESHOLD = 25.0 # % - scale down candidate if avg CPU below this
MEM_SCALE_UP_THRESHOLD = 0.10 # fraction free - scale up if below 10% free RAM

def get_metric_average(metric_name: str, stat: str, period_minutes: int = 30) -> float:
    end = datetime.utcnow()
    start = end - timedelta(minutes=period_minutes)

    response = cw.get_metric_statistics(
        Namespace = 'AWS/RDS',
        MetricName = metric_name,
        Dimensions = [{'Name': 'DBInstanceIdentifier', 'Value': INSTANCE_ID}],
        StartTime = start,
        EndTime = end,
        Period = period_minutes * 60,
        Statistics = [stat],
    )
    datapoints = response.get('Datapoints', [])
    if not datapoints:
        return 0.0
    return datapoints[0].get(stat, 0.0)

def get_instance_class() -> str:
    resp = rds.describe_db_instances(DBInstanceIdentifier=INSTANCE_ID)
    inst = resp['DBInstances'][0]
    # If a modification is pending, return the pending class to avoid churn
    pending = inst.get('PendingModifiedValues', {}).get('DBInstanceClass')
    return pending if pending else inst['DBInstanceClass']

def get_instance_memory_gb(instance_class: str) -> float:
    """Approximate memory map for r6i family. Extend for other families."""
    memory_map = {
        "db.r6i.xlarge": 32,
        "db.r6i.2xlarge": 64,
```

```

        "db.r6i.4xlarge": 128,
        "db.r6i.8xlarge": 256,
        "db.r6i.16xlarge": 512,
    }
    return memory_map.get(instance_class, 64)

def lambda_handler(event, context):
    current_class = get_instance_class()
    current_index = INSTANCE_LADDER.index(current_class) \
        if current_class in INSTANCE_LADDER else None

    if current_index is None:
        print(f"Instance class {current_class} not in ladder; skipping.")
        return

    avg_cpu = get_metric_average('CPUUtilization', 'Average')
    min_free_mem_b = get_metric_average('FreeableMemory', 'Minimum')
    total_mem_gb = get_instance_memory_gb(current_class)
    free_mem_ratio = (min_free_mem_b / (1024**3)) / total_mem_gb if total_mem_gb else 1.0

    print(f"Current class: {current_class}")
    print(f"Avg CPU (30m): {avg_cpu:.1f}% Free mem ratio: {free_mem_ratio:.3f}")

    target_class = current_class

    # Scale-up conditions
    if avg_cpu > CPU_SCALE_UP_THRESHOLD or free_mem_ratio < MEM_SCALE_UP_THRESHOLD:
        if current_index < len(INSTANCE_LADDER) - 1:
            target_class = INSTANCE_LADDER[current_index + 1]
            reason = (
                f"CPU {avg_cpu:.1f}% > {CPU_SCALE_UP_THRESHOLD}%"
                if avg_cpu > CPU_SCALE_UP_THRESHOLD
                else f"FreeMemRatio {free_mem_ratio:.3f} < {MEM_SCALE_UP_THRESHOLD}"
            )
        else:
            print("Already at maximum instance class in ladder.")
            return

    # Scale-down condition - only if CPU is low AND memory is comfortable
    elif (avg_cpu < CPU_SCALE_DOWN_THRESHOLD
          and free_mem_ratio > 0.40
          and current_index > 0):
        target_class = INSTANCE_LADDER[current_index - 1]
        reason = f"CPU {avg_cpu:.1f}% < {CPU_SCALE_DOWN_THRESHOLD}% and memory ample"

    if target_class == current_class:
        print("No resize needed.")
        return

    print(f"Recommending resize: {current_class} -> {target_class} ({reason})")

    # Notify before modifying - allows human review if SNS is wired to a Slack/PagerDuty
    sns.publish(
        TopicArn = SNS_TOPIC_ARN,
        Subject = f"RDS Auto-Resize: {INSTANCE_ID}",
        Message = json.dumps({
            "instance": INSTANCE_ID,
            "from_class": current_class,
            "to_class": target_class,
            "reason": reason,
            "avg_cpu_pct": round(avg_cpu, 1),
            "free_mem_ratio": round(free_mem_ratio, 3),
            "timestamp": datetime.utcnow().isoformat(),
        }, indent=2),
    )

    rds.modify_db_instance(
        DBInstanceIdentifier = INSTANCE_ID,
        DBInstanceClass = target_class,

```

```

        ApplyImmediately    = False,    # Apply at next maintenance window
    )
    print(f"Modify request submitted. Change will apply at next maintenance window.")

```

Two production safeguards this implementation intentionally includes: the resize is scheduled for the next maintenance window (`ApplyImmediately=False`), and an SNS notification fires before the API call. This gives your on-call engineer a chance to abort if the recommendation fires during an incident window where a restart would be catastrophic. If you want fully automatic immediate resizing, remove `ApplyImmediately=False` and add a maintenance window check — do not apply immediately outside the approved window.

39.7 Cost Allocation and Showback Reports

Right-sizing decisions need to be made by people who can see the cost impact clearly. If a development team has no visibility into what their oversized RDS instance costs, they have no incentive to right-size it. Cost allocation tags solve this structurally.

Tagging strategy for databases should include at minimum: - **Environment** (production, staging, development) - **Team** or **CostCenter** - **Application** or **Service** - **LastResizedBy** and **LastResizeDate** (operational provenance) - **SizingStatus** (right-sized, under-review, known-overprovisioned)

The **SizingStatus** tag is a convention you create, not something AWS, Azure, or GCP provides. Use it to mark instances that Compute Optimizer flags as over-provisioned but where you have a known reason for the current size (an upcoming load test, a quarterly batch job not visible in the 14-day metrics window). This prevents automated alerts from repeatedly nagging about a conscious decision.

AWS Cost Explorer allows you to filter and group by tag. A monthly query that groups by **Team** and **Application** and filters to the RDS service gives each team their database bill. The API can drive automated reports:

```

import boto3
from datetime import date
from dateutil.relativedelta import relativedelta

ce = boto3.client('ce', region_name='us-east-1')

def get_rds_cost_by_team(months_back: int = 1) -> list:
    end = date.today().replace(day=1)
    start = end - relativedelta(months=months_back)

    response = ce.get_cost_and_usage(
        TimePeriod = {
            'Start': start.strftime('%Y-%m-%d'),
            'End': end.strftime('%Y-%m-%d'),
        },
        Granularity = 'MONTHLY',
        Filter = {
            'And': [
                {'Dimensions': {'Key': 'SERVICE', 'Values': ['Amazon Relational Database Service']}},
                {'Dimensions': {'Key': 'USAGE_TYPE_GROUP', 'Values': ['RDS: Running Hours']}},
            ]
        },
        GroupBy = [
            {'Type': 'TAG', 'Key': 'Team'},
            {'Type': 'TAG', 'Key': 'Environment'},

```



```

    ],
    Metrics = ['UnblendedCost'],
)

results = []
for period in response['ResultsByTime']:
    for group in period['Groups']:
        keys = group['Keys']
        cost = float(group['Metrics']['UnblendedCost']['Amount'])
        unit = group['Metrics']['UnblendedCost']['Unit']
        team = keys[0].replace('Team$', '') if keys else 'untagged'
        env = keys[1].replace('Environment$', '') if len(keys) > 1 else 'unknown'
        results.append({
            'period': period['TimePeriod']['Start'],
            'team': team,
            'env': env,
            'cost': round(cost, 2),
            'unit': unit,
        })

return sorted(results, key=lambda x: x['cost'], reverse=True)

if __name__ == '__main__':
    rows = get_rds_cost_by_team(months_back=3)
    print(f"{'Period':<12} {'Team':<20} {'Env':<12} {'Cost (USD)':>12}")
    print('-' * 60)
    for r in rows:
        print(f"{r['period']:<12} {r['team']:<20} {r['env']:<12} {r['cost']:>12.2f}")

```

Running this monthly and distributing the output to engineering leads creates the accountability loop that drives right-sizing decisions without requiring a central FinOps team to police every instance.

39.8 Right-Sizing During Migration Waves

Migration projects are the natural moment to establish the right size for a database, but they are also the moment when over-provisioning is most likely to occur. The instinct is: if we are migrating a database from a 48-core on-premises server, we should land on something equivalent. This instinct is usually wrong for two reasons.

First, on-premises servers ran at low average utilization. A 48-core server purchased to handle a projected peak that never arrived runs at 12% CPU on average. The equivalent cloud instance is not a 48-vCPU database; it is a 4–8 vCPU instance that handles 12% of a 48-core machine's capacity, which is comfortably within range for most OLTP workloads.

Second, cloud storage I/O is different from on-premises SAN I/O. A workload that needed a 40,000 IOPS SAN because it ran on spinning disk (or on HDD with heavy caching) may need only 8,000 IOPS on cloud SSD because the lower latency reduces queue depths and the access pattern changes.

The right-sizing process during a migration wave:

1. **Collect two weeks of on-premises metrics** before migration. Peak CPU (not average), peak IOPS, peak throughput, buffer cache hit ratio, and active connection count at peak.
2. **Translate to cloud instance using the memory-first heuristic:** choose a family based on the RAM needed to hold the working set (buffer pool + connection overhead + OS overhead), then verify the vCPU count is sufficient for the CPU workload.

3. **Start one tier smaller than the translation suggests.** You can scale up in minutes; you cannot get the week of over-spending back.
4. **Load test on the cloud instance before cutover.** Use pgbench, HammerDB, or a replay of captured query workload against the target instance.
5. **Validate for 72 hours post-cutover before declaring the size final.** Real workloads differ from load tests; the first week's actual peak may reveal a bottleneck the load test did not surface.

The following query helps you estimate buffer pool requirements on an existing PostgreSQL source instance before migrating:

```
-- Estimate effective working set size on a PostgreSQL source instance
-- pg_buffercache requires the pg_buffercache extension (usually available)
-- Run on the source BEFORE migration to size the target buffer pool

CREATE EXTENSION IF NOT EXISTS pg_buffercache;

SELECT
    c.relname                                AS table_name,
    pg_size_pretty(
        count(*) * current_setting('block_size')::bigint
    )                                         AS cached_size,
    round(100.0 * count(*) /
        (SELECT count(*) FROM pg_buffercache WHERE relfilenode IS NOT NULL),
        2)                                   AS pct_of_buffer_pool
FROM pg_buffercache bc
JOIN pg_class c ON bc.relfilenode = pg_relation_filenode(c.oid)
WHERE bc.relfilenode IS NOT NULL
GROUP BY c.relname
ORDER BY count(*) DESC
LIMIT 20;
```

The total `cached_size` across all tables gives you the minimum RAM the buffer pool needs to hold the hot working set. Add 20–30% headroom for new pages, temporary sort memory, and OS file system cache, and you have a defensible RAM requirement for the target instance.

For SQL Server workloads migrating to Azure SQL or SQL Managed Instance:

```
-- Estimate SQL Server buffer pool working set before migration
-- Identifies the top consumers of buffer pool memory by database object

SELECT TOP 20
    OBJECT_NAME(p.object_id)                AS table_name,
    i.name                                   AS index_name,
    COUNT_BIG(*)                             AS buffer_pages,
    COUNT_BIG(*) * 8 / 1024                  AS cached_mb,
    ROUND(
        100.0 * COUNT_BIG(*) /
        (SELECT COUNT_BIG(*) FROM sys.dm_os_buffer_descriptors
         WHERE database_id = DB_ID()),
        2)                                   AS pct_of_buffer_pool
FROM sys.dm_os_buffer_descriptors bd
JOIN sys.allocation_units au ON bd.allocation_unit_id = au.allocation_unit_id
JOIN sys.partitions p
    ON au.container_id = CASE
        WHEN au.type IN (1, 3) THEN p.hobt_id
        ELSE p.partition_id
    END
JOIN sys.indexes i
    ON p.object_id = i.object_id AND p.index_id = i.index_id
WHERE bd.database_id = DB_ID()
    AND bd.page_type IN ('DATA_PAGE', 'INDEX_PAGE')
GROUP BY p.object_id, i.name
ORDER BY buffer_pages DESC;
```

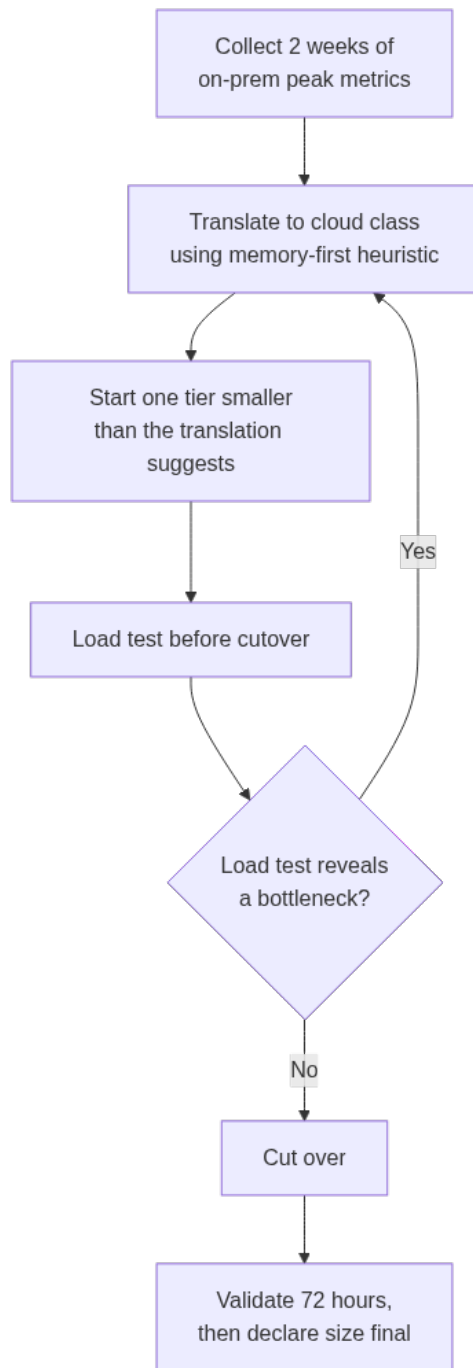


Figure 30: The loop back to translation on a failed load test is deliberate — it’s cheaper to catch under-sizing before cutover than after.

These two queries — one for PostgreSQL, one for SQL Server — are among the most useful pre-migration tools you can run. The output from each quantifies the RAM requirement with direct evidence from production, which makes a sizing recommendation to management far more defensible than an estimate based on the source server’s spec sheet.

Key Takeaways

- **Memory drives family selection, CPU drives size within the family.** Use buffer pool hit ratio and FreeableMemory metrics to determine whether the primary bottleneck is RAM; reserve CPU-focused resizing for workloads that are demonstrably compute-bound rather than I/O or memory-bound.
 - **Cloud storage cannot be shrunk after provisioning on RDS or Cloud SQL — provision conservatively and enable autoscaling.** Migrate existing RDS gp2 volumes to gp3 to decouple IOPS from storage size and reduce costs for most workloads that operate below the 16,000 IOPS gp3 maximum.
 - **Read replicas are production traffic receivers, not just HA standbys.** Right-size replicas independently of the primary based on the query patterns they serve; monitor replication lag to confirm that a smaller replica can keep pace with the primary’s write stream.
 - **Automate the right-sizing feedback loop.** A Lambda function or scheduled job that compares CloudWatch metrics against thresholds, proposes a resize via SNS, and submits the modify API call for the next maintenance window costs almost nothing to build and prevents the drift that occurs when sizing decisions are made once and never revisited.
 - **Migration is the best opportunity to correct on-premises over-provisioning.** Collect two weeks of source metrics, apply the memory-first sizing heuristic, start one tier smaller than the direct translation suggests, and validate under real traffic before declaring the target instance size final.
-

Chapter 40: The Cloud DBA’s Cost and Performance Dashboard

Every cloud DBA eventually faces the same uncomfortable conversation with finance: “Why did our database bill jump 40% last month?” The honest answer is almost always the same — visibility was incomplete, and decisions were made without a unified view of cost alongside performance. On-premises, you knew exactly what the hardware cost because you signed the purchase order. In the cloud, costs are fluid, metered by the second, and shaped by dozens of variables — instance class, storage IOPS, data transfer, backup retention, cross-AZ replication, and licensing uplift. A performance fix that doubles your IOPS can simultaneously double your bill without anyone noticing until the invoice arrives.

This final chapter brings together everything covered in this book into a practical, deployable dashboard framework that spans AWS RDS, Aurora, Azure SQL, and Google Cloud SQL. You will build monitoring views that correlate query latency with cost drivers, set up alerting thresholds grounded in real production numbers, and establish the governance habits that prevent runaway spend. This is the operational layer that ties cloud database engineering together — not a collection of theoretical advice, but the actual queries, CLI commands, and Terraform configurations used in production environments.

40.1 The Cost-Performance Relationship in Cloud Databases

On-premises DBAs think about performance in isolation from cost because the hardware is already paid for. A slow query costs you nothing extra in dollars — it only costs time and user experience. In the cloud, that mental model breaks immediately. A query that triggers a full table scan on an Aurora cluster with Serverless v2 scaling will literally increase your AWS bill as the ACU count climbs to handle the load. A missing index on Azure SQL Hyperscale is not just a performance problem — it is a spending problem.

The fundamental shift is this: **in cloud databases, compute and I/O are metered continuously**. This creates a tight coupling between query efficiency, configuration choices, and monthly cost that has no analog in on-premises work.

The three cost drivers every cloud DBA must track:

Compute is typically the largest line item. An `db.r6g.4xlarge` on RDS costs approximately \$0.96/hour in `us-east-1` (On-Demand). Running it 24/7 for 30 days is roughly \$691. A Reserved Instance (1-year, no upfront) drops that to around \$0.57/hour — \$410/month — a 40% reduction.

DBAs who migrate from on-prem and leave everything On-Demand for six months are leaving tens of thousands of dollars on the table.

Storage on RDS gp3 is \$0.115/GB/month with a baseline of 3,000 IOPS and 125 MB/s throughput included. If your application was provisioned with io1 at 10,000 IOPS, you are paying approximately \$0.065/IOPS/month on top of storage — that is \$650/month in IOPS alone for a moderately sized database. Migrating from io1 to gp3 with manually provisioned IOPS is one of the highest-ROI changes a cloud DBA can make.

Data transfer is the stealth cost. Reads stay within a region cheaply, but every byte leaving a VPC to the internet costs \$0.09/GB. Cross-AZ traffic between your application tier and your RDS Multi-AZ instance costs \$0.02/GB each direction. For a high-throughput reporting database that ships hundreds of gigabytes to an analytics layer, this becomes significant.

The dashboard you build must surface all three drivers alongside the performance metrics that cause them to spike.

40.2 Building the Unified Monitoring Foundation

Before building dashboards, you need the telemetry infrastructure in place. CloudWatch, Azure Monitor, and Cloud Monitoring each require specific configuration to expose database-level metrics at useful granularity.

AWS: Enabling Enhanced Monitoring and Performance Insights

Standard CloudWatch RDS metrics report at 1-minute intervals and cover CPU, storage, and basic connection counts. Enhanced Monitoring adds 50+ OS-level metrics at intervals down to 1 second — this is what you need to catch connection storms, memory pressure, and disk saturation that standard metrics miss.

```
# Enable Enhanced Monitoring and Performance Insights on an existing RDS instance
# Enhanced Monitoring requires an IAM role with the rds-monitoring policy

# Step 1: Create the monitoring role if it doesn't exist
aws iam create-role \
  --role-name rds-enhanced-monitoring-role \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Principal": {"Service": "monitoring.rds.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }]
  }'
```

```
aws iam attach-role-policy \
  --role-name rds-enhanced-monitoring-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonRDSEnhancedMonitoringRole

# Step 2: Enable Enhanced Monitoring (granularity: 5 seconds)
# and Performance Insights (retention: 7 days free, 731 days = paid)
aws rds modify-db-instance \
  --db-instance-identifier prod-postgres-01 \
  --monitoring-interval 5 \
  --monitoring-role-arn arn:aws:iam::123456789012:role/rds-enhanced-monitoring-role \
  --enable-performance-insights \
  --performance-insights-retention-period 7 \
  --apply-immediately
```

```
# Step 3: Verify the changes applied
aws rds describe-db-instances \
  --db-instance-identifier prod-postgres-01 \
  --query 'DBInstances[0].{
    MonitoringInterval:MonitoringInterval,
    PIEnabled:PerformanceInsightsEnabled,
    PIRetention:PerformanceInsightsRetentionPeriod
  }'
```

Azure SQL: Enabling Diagnostic Settings

Azure SQL does not ship query-level metrics to Log Analytics by default. You must explicitly configure Diagnostic Settings for each database or elastic pool. Without this, you are flying blind on query performance data.

```
# Enable Diagnostic Settings for Azure SQL to send to Log Analytics
# Assumes you have a Log Analytics workspace already created

RESOURCE_GROUP="prod-rg"
SQL_SERVER="prod-sql-server-eastus"
DATABASE="prod-db-01"
WORKSPACE_ID=
↪ "/subscriptions/xxxx/resourceGroups/prod-rg/providers/Microsoft.OperationalInsights/workspaces/prod-law"

# Get the full resource ID of the database
DB_RESOURCE_ID=$(az sql db show \
  --resource-group $RESOURCE_GROUP \
  --server $SQL_SERVER \
  --name $DATABASE \
  --query id -o tsv)

# Create diagnostic setting - capture QueryStoreRuntimeStatistics,
# SQLInsights, Errors, and basic metrics
az monitor diagnostic-settings create \
  --name "prod-db-diag-settings" \
  --resource $DB_RESOURCE_ID \
  --workspace $WORKSPACE_ID \
  --logs '[
    {"category": "SQLInsights", "enabled": true, "retentionPolicy": {"days": 30, "enabled": true}},
    {"category": "QueryStoreRuntimeStatistics", "enabled": true, "retentionPolicy": {"days": 30,
↪ "enabled": true}},
    {"category": "QueryStoreWaitStatistics", "enabled": true, "retentionPolicy": {"days": 30, "enabled":
↪ true}},
    {"category": "Errors", "enabled": true, "retentionPolicy": {"days": 30, "enabled": true}},
    {"category": "DatabaseWaitStatistics", "enabled": true, "retentionPolicy": {"days": 30, "enabled":
↪ true}},
    {"category": "Timeouts", "enabled": true, "retentionPolicy": {"days": 30, "enabled": true}},
    {"category": "Blocks", "enabled": true, "retentionPolicy": {"days": 30, "enabled": true}},
    {"category": "Deadlocks", "enabled": true, "retentionPolicy": {"days": 30, "enabled": true}}
  ]' \
  --metrics '[
    {"category": "Basic", "enabled": true, "retentionPolicy": {"days": 30, "enabled": true}},
    {"category": "InstanceAndAppAdvanced", "enabled": true, "retentionPolicy": {"days": 30, "enabled":
↪ true}}
  ]'

echo "Diagnostic settings applied to: $DATABASE"
```

Google Cloud SQL: Enabling Query Insights

Cloud SQL's Query Insights is the equivalent of Performance Insights — it captures per-query latency, wait events, and execution plans. It is not enabled by default.

```
# Enable Query Insights on Cloud SQL for PostgreSQL
# Note: Requires restart for some flags on Cloud SQL

gcloud sql instances patch prod-postgres-instance \
```

```

--database-flags="cloudsql.enable_query_insights=on" \
--project=my-gcp-project

# Set sampling rate (1000 = 100% of queries sampled, 0 = disabled)
# For high-traffic instances, use a lower rate to control overhead
gcloud sql instances patch prod-postgres-instance \
--database-flags="\
cloudsql.enable_query_insights=on,\
cloudsql.query_insights_sample_rate=200,\
cloudsql.query_string_length=1024,\
cloudsql.record_application_tags=on,\
cloudsql.record_client_address=on" \
--project=my-gcp-project

# Verify current flags
gcloud sql instances describe prod-postgres-instance \
--project=my-gcp-project \
--format="json" | jq '.settings.databaseFlags'

```

The monitoring coverage matrix — before building dashboards, verify you have the following telemetry active on every production instance:

Signal	AWS RDS	Azure SQL	Cloud SQL
CPU/Memory	CloudWatch (1m)	Azure Monitor (1m)	Cloud Monitoring (1m)
Per-query stats	Performance Insights	Query Store + Log Analytics	Query Insights
Wait events	Performance Insights	sys.dm_os_wait_stats	pg_stat_activity
Replication lag	CloudWatch ReplicaLag	sys.dm_hadr_database_replica_monitoring	Cloud Monitoring replica-tion/replica_lag
Storage I/O	Enhanced Monitoring	Azure Monitor IOPS metrics	Cloud Monitoring disk/write_ops_count
Cost allocation	Cost Explorer tags	Azure Cost Management tags	Cloud Billing labels

40.3 Query-Level Cost Attribution

The most actionable insight a cloud DBA can provide is answering: “Which queries are costing us the most money?” This is not a theoretical exercise. On Aurora Serverless v2, a single unoptimized reporting query that runs at 2 AM and scales the cluster from 2 ACUs to 32 ACUs for 20 minutes is generating a measurable cost spike. Identifying and fixing it is both a performance win and a direct cost reduction.

AWS RDS/Aurora: Extracting Top Queries from Performance Insights

```

import boto3
from datetime import datetime, timezone, timedelta
import json

def get_top_cost_queries(db_instance_id: str, region: str = "us-east-1", top_n: int = 20):
    """
    Retrieve the top SQL queries by db.load (average active sessions)
    from Performance Insights. High db.load on cloud = high cost.
    """

```



```

db.load is the primary metric - it represents the number of active
sessions normalized against vCPU count. A db.load > vCPU count means
your instance is under pressure. On Serverless, it triggers scaling.
"""
client = boto3.client("pi", region_name=region)

# Get the DI (Data Source) identifier for this instance
rds_client = boto3.client("rds", region_name=region)
instance_info = rds_client.describe_db_instances(
    DBInstanceIdentifier=db_instance_id
)
dbi_resource_id = instance_info["DBInstances"][0]["DbiResourceId"]

end_time = datetime.now(timezone.utc)
start_time = end_time - timedelta(hours=24)

response = client.get_resource_metrics(
    ServiceType="RDS",
    Identifier=dbi_resource_id,
    MetricQueries=[
        {
            "Metric": "db.load.avg",
            "GroupBy": {
                "Group": "db.sql",
                "Dimensions": ["db.sql.id", "db.sql.tokenized_id"],
                "Limit": top_n
            }
        }
    ],
    StartTime=start_time,
    EndTime=end_time,
    PeriodInSeconds=3600 # 1-hour buckets
)

# Extract and sort by average load contribution
query_loads = []
for key_metric in response.get("MetricList", []):
    sql_id = key_metric["Key"].get("Dimensions", {}).get("db.sql.tokenized_id", "unknown")

    # Calculate average load across the 24h window
    data_points = key_metric.get("DataPoints", [])
    if data_points:
        avg_load = sum(
            dp["Value"] for dp in data_points if "Value" in dp
        ) / len(data_points)

        query_loads.append({
            "sql_tokenized_id": sql_id,
            "avg_db_load": round(avg_load, 4),
            "data_points": len(data_points)
        })

# Sort by average load - highest first
query_loads.sort(key=lambda x: x["avg_db_load"], reverse=True)

print(f"\nTop {top_n} queries by DB Load (24h window) - {db_instance_id}")
print(f"{'Rank':<6} {'Avg DB Load':<15} {'SQL Tokenized ID'}")
print("-" * 60)
for rank, q in enumerate(query_loads[:top_n], 1):
    print(f"{'rank':<6} {'avg_db_load':<15} {'sql_tokenized_id'}")

return query_loads

def estimate_aurora_serverless_cost_impact(
    avg_load_units: float,
    vcpu_per_acu: float = 0.5,
    acu_price_per_hour: float = 0.12,
    hours_window: int = 24

```

```

) -> dict:
    """
    Estimate the cost contribution of a workload component on Aurora Serverless v2.

    Aurora Serverless v2 pricing:
    - Each ACU = 2 GB RAM + ~0.5 vCPU equivalent
    - Price: $0.12/ACU-hour (us-east-1)
    - Min ACUs: 0.5, Max ACUs: 128

    If a query contributes 4.0 units of db.load and baseline is 2.0,
    it's doubling the required ACU footprint.
    """
    estimated_acus_consumed = avg_load_units / vcpu_per_acu
    estimated_cost = estimated_acus_consumed * acu_price_per_hour * hours_window

    return {
        "avg_load_units": avg_load_units,
        "estimated_acus": round(estimated_acus_consumed, 2),
        "estimated_cost_24h": round(estimated_cost, 4),
        "acu_price_per_hour": acu_price_per_hour
    }

# Example usage
if __name__ == "__main__":
    queries = get_top_cost_queries(
        db_instance_id="prod-aurora-postgres-cluster-instance-1",
        region="us-east-1",
        top_n=10
    )

    if queries:
        top_query = queries[0]
        cost_impact = estimate_aurora_serverless_cost_impact(
            avg_load_units=top_query["avg_db_load"]
        )
        print(f"\nCost impact estimate for top query:")
        print(json.dumps(cost_impact, indent=2))

```

Azure SQL: Query Store Cost Attribution

On Azure SQL, Query Store is the equivalent telemetry system. Unlike Performance Insights, it lives inside the database itself and persists across connections. Because you already know Query Store from SQL Server on-premises, the transition here is familiar — but the DTU/vCore billing layer makes query efficiency directly financial.

```

-- Azure SQL: Top 20 queries by total CPU time over the last 24 hours
-- This directly maps to DTU consumption and therefore billing cost.
-- On Azure SQL Hyperscale, CPU cost = vCore seconds consumed.
-- Cross-reference with Query Store wait statistics to find root cause.

```

```

WITH recent_interval AS (
    SELECT
        rsi.start_time,
        rsi.end_time,
        rsi.runtime_stats_interval_id
    FROM sys.query_store_runtime_stats_intervals rsi
    WHERE rsi.start_time >= DATEADD(HOUR, -24, GETUTCDATE())
),
query_stats AS (
    SELECT
        qs.query_id,
        qs.query_sql_text,
        qs.object_id,
        OBJECT_NAME(qs.object_id) AS proc_name,
        COUNT(*) AS execution_count,
        SUM(qsrs.count_executions) AS total_executions,
        AVG(qsrs.avg_cpu_time) AS avg_cpu_us,

```

```

MAX(qsrs.max_cpu_time)                AS max_cpu_us,
SUM(qsrs.avg_cpu_time * qsrs.count_executions) AS total_cpu_us,
AVG(qsrs.avg_duration)                AS avg_duration_us,
AVG(qsrs.avg_logical_io_reads)        AS avg_logical_reads,
SUM(qsrs.avg_logical_io_reads * qsrs.count_executions) AS total_logical_reads,
AVG(qsrs.avg_rowcount)                AS avg_rows_returned,
-- Regressed queries: max > 3x average is a flag for plan instability
CASE
  WHEN MAX(qsrs.max_cpu_time) > AVG(qsrs.avg_cpu_time) * 3
  THEN 'PLAN_REGRESSION_RISK'
  ELSE 'STABLE'
END                                   AS plan_stability
FROM sys.query_store_query qsq
JOIN sys.query_store_query_text qsqt
  ON qsq.query_text_id = qsqt.query_text_id
JOIN sys.query_store_plan qsp
  ON qsq.query_id = qsp.query_id
JOIN sys.query_store_runtime_stats qsrs
  ON qsp.plan_id = qsrs.plan_id
JOIN recent_interval ri
  ON qsrs.runtime_stats_interval_id = ri.runtime_stats_interval_id
WHERE qsq.is_internal_query = 0 -- Exclude system queries
GROUP BY
  qsq.query_id,
  qsqt.query_sql_text,
  qsq.object_id
),
ranked AS (
  SELECT
    *,
    RANK() OVER (ORDER BY total_cpu_us DESC) AS cpu_rank,
    RANK() OVER (ORDER BY total_logical_reads DESC) AS io_rank,
    -- Composite cost score: CPU contributes 60%, I/O contributes 40%
    -- Adjust weights based on your instance's bottleneck profile
    (RANK() OVER (ORDER BY total_cpu_us DESC) * 0.6 +
     RANK() OVER (ORDER BY total_logical_reads DESC) * 0.4) AS composite_cost_rank
  FROM query_stats
)
SELECT TOP 20
  cpu_rank,
  io_rank,
  CAST(composite_cost_rank AS INT) AS composite_rank,
  query_id,
  LEFT(query_sql_text, 120) AS query_preview,
  proc_name,
  total_executions,
  CAST(avg_cpu_us / 1000.0 AS DECIMAL(10,2)) AS avg_cpu_ms,
  CAST(total_cpu_us / 1000000.0 AS DECIMAL(12,2)) AS total_cpu_seconds,
  CAST(avg_duration_us / 1000.0 AS DECIMAL(10,2)) AS avg_duration_ms,
  avg_logical_reads,
  total_logical_reads,
  plan_stability
FROM ranked
ORDER BY composite_cost_rank ASC;

```

40.4 Cost Governance with Terraform and Tagging

Cost dashboards without governance are just reports. The operational change that prevents budget overruns is **enforcing tagging standards at provisioning time** and building Terraform modules that emit cost-compatible configurations by default.

Every RDS instance, Azure SQL server, and Cloud SQL instance in production should carry tags/labels that answer: Which team owns it? Which application uses it? What environment

is it in? Without this, your cost reports show a single line item — “RDS” — and no one can determine what is wasting money.

Terraform: Production-Grade RDS Module with Cost Controls

```
# terraform/modules/rds_postgres/main.tf
# Production RDS PostgreSQL module with mandatory tagging,
# cost-optimized storage defaults, and alerting configuration.
#
# Key cost decisions enforced in this module:
# - gp3 storage by default (not io1, not gp2)
# - Performance Insights enabled (no cost at 7-day retention)
# - Enhanced Monitoring enabled (minimal CloudWatch cost vs. visibility gained)
# - deletion_protection = true (prevents accidental teardown and data recovery costs)
# - Reserved Instance: NOT managed in Terraform - handled separately via
#   AWS console or aws_db_reserved_instances_offering resource

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

variable "identifier" {
  type        = string
  description = "RDS instance identifier - must be unique within the AWS account/region"
}

variable "instance_class" {
  type        = string
  description = "RDS instance class. Cost reference: db.r6g.large ~$0.24/hr, db.r6g.xlarge ~$0.48/hr,
  ↪ db.r6g.2xlarge ~$0.96/hr (us-east-1 On-Demand)"
  default     = "db.r6g.large"

  validation {
    # Prevent accidentally provisioning oversized instances without explicit intent
    condition     = !contains(["db.r6g.8xlarge", "db.r6g.16xlarge", "db.r5.8xlarge", "db.r5.16xlarge"],
    ↪ var.instance_class)
    error_message = "Instances larger than 4xlarge require explicit approval. Contact the platform
    ↪ team."
  }
}

variable "allocated_storage_gb" {
  type        = number
  description = "Initial storage in GB. gp3 baseline: 3,000 IOPS and 125 MB/s throughput included."
  default     = 100

  validation {
    condition     = var.allocated_storage_gb >= 20 && var.allocated_storage_gb <= 65536
    error_message = "Storage must be between 20 GB and 65,536 GB."
  }
}

variable "max_allocated_storage_gb" {
  type        = number
  description = "Max storage for autoscaling. Set to 0 to disable autoscaling."
  default     = 500
}

variable "engine_version" {
  type        = string
  default     = "16.3"
}

variable "database_name" {
```

```

    type = string
}

variable "master_username" {
    type = string
}

variable "subnet_group_name" {
    type      = string
    description = "DB subnet group - must be in private subnets only"
}

variable "vpc_security_group_ids" {
    type = list(string)
}

variable "parameter_group_name" {
    type      = string
    default = null
}

variable "monitoring_role_arn" {
    type      = string
    description = "IAM role ARN for Enhanced Monitoring (requires AmazonRDSEnhancedMonitoringRole policy)"
}

variable "performance_insights_kms_key_id" {
    type      = string
    default = null
}

variable "backup_retention_days" {
    type      = number
    default    = 7
    description = "Backup retention. Cost: ~$0.095/GB-month for automated backups beyond 1x storage. Keep
↪ at 7 unless compliance requires more."
}

variable "tags" {
    type      = map(string)
    description = "Required tags for cost attribution. Must include: team, application, environment,
↪ cost-center"

    validation {
        condition = alltrue([
            contains(keys(var.tags), "team"),
            contains(keys(var.tags), "application"),
            contains(keys(var.tags), "environment"),
            contains(keys(var.tags), "cost-center")
        ])
        error_message = "Tags must include: team, application, environment, cost-center. Finance uses these
↪ for chargebacks."
    }
}

# Local computed values
locals {
    # Merge mandatory module-level tags with caller-provided tags
    merged_tags = merge(var.tags, {
        "managed-by"    = "terraform"
        "module"        = "rds_postgres"
        "engine"        = "postgres-${var.engine_version}"
        "instance-class" = var.instance_class
    })
}

resource "aws_db_instance" "this" {
    identifier      = var.identifier
    engine         = "postgres"
    engine_version = var.engine_version

```

```

instance_class    = var.instance_class

# Storage: gp3 by default - cost-optimized vs. io1/gp2
# gp3 includes 3,000 IOPS and 125 MB/s at no extra charge
# Provision additional IOPS only if pg_stat_io shows consistent saturation
storage_type      = "gp3"
allocated_storage = var.allocated_storage_gb
max_allocated_storage = var.max_allocated_storage_gb > 0 ? var.max_allocated_storage_gb : null
storage_encrypted  = true

db_name = var.database_name
username = var.master_username
# Password managed via Secrets Manager - not in Terraform state
manage_master_user_password = true

db_subnet_group_name = var.subnet_group_name
vpc_security_group_ids = var.vpc_security_group_ids
publicly_accessible    = false # Never expose RDS directly to internet

parameter_group_name = var.parameter_group_name

# Multi-AZ: required for production
# Cost: ~2x single-AZ price. Non-negotiable for production write workloads.
multi_az = var.tags["environment"] == "prod" ? true : false

# Backup configuration
backup_retention_period = var.backup_retention_days
backup_window           = "03:00-04:00" # UTC - adjust for your region
maintenance_window      = "Sun:04:00-Sun:05:00"

# Performance Insights: 7 days is free, 731 days costs ~$0.02/vCPU/hour
performance_insights_enabled      = true
performance_insights_retention_period = 7
performance_insights_kms_key_id    = var.performance_insights_kms_key_id

# Enhanced Monitoring: 5-second granularity
monitoring_interval = 5
monitoring_role_arn = var.monitoring_role_arn

# Prevent accidental deletion
deletion_protection = true
skip_final_snapshot = false
final_snapshot_identifier = "${var.identifier}-final-snapshot"

# Auto minor version upgrades: enabled during maintenance window
auto_minor_version_upgrade = true

tags = local.merged_tags
}

# CloudWatch Alarms - cost-correlated thresholds
resource "aws_cloudwatch_metric_alarm" "cpu_high" {
  alarm_name        = "${var.identifier}-cpu-high"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods = 3
  metric_name       = "CPUUtilization"
  namespace         = "AWS/RDS"
  period            = 300
  statistic         = "Average"
  threshold         = 80
  alarm_description = "CPU > 80% for 15 min. On Serverless: this is scaling. On provisioned: consider
↪ vertical scaling or query optimization."
  alarm_actions     = [] # Populate with SNS ARN

  dimensions = {
    DBInstanceIdentifier = aws_db_instance.this.identifier
  }

  tags = local.merged_tags
}

```

```
resource "aws_cloudwatch_metric_alarm" "freeable_memory_low" {
  alarm_name         = "${var.identifier}-low-memory"
  comparison_operator = "LessThanThreshold"
  evaluation_periods = 2
  metric_name        = "FreeableMemory"
  namespace          = "AWS/RDS"
  period             = 300
  statistic          = "Average"
  # 256 MB threshold - below this, PostgreSQL shared_buffers are likely being evicted
  threshold          = 268435456 # 256 MB in bytes
  alarm_description  = "Freeable memory below 256 MB. shared_buffers eviction likely. Consider upsizing
↪ or tuning work_mem."
  alarm_actions      = []

  dimensions = {
    DBInstanceIdentifier = aws_db_instance.this.identifier
  }

  tags = local.merged_tags
}

resource "aws_cloudwatch_metric_alarm" "read_iops_high" {
  alarm_name         = "${var.identifier}-read-iops-high"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods = 3
  metric_name        = "ReadIOPS"
  namespace          = "AWS/RDS"
  period             = 300
  statistic          = "Average"
  # gp3 baseline is 3,000 IOPS. Sustained above 2,700 indicates
  # either a missing index or a workload that needs io1/io2 provisioning.
  threshold          = 2700
  alarm_description  = "Read IOPS near gp3 baseline limit (3,000). Investigate top queries in
↪ Performance Insights."
  alarm_actions      = []

  dimensions = {
    DBInstanceIdentifier = aws_db_instance.this.identifier
  }

  tags = local.merged_tags
}

output "db_instance_id" {
  value = aws_db_instance.this.id
}

output "db_endpoint" {
  value = aws_db_instance.this.endpoint
}

output "db_resource_id" {
  value       = aws_db_instance.this.resource_id
  sensitive   = false
  description = "DbiResourceId - needed for Performance Insights API calls"
}
```

The tagging enforcement gap — one gotcha worth flagging: Terraform validation on tags only applies when you provision through Terraform. If someone creates an RDS instance through the console, the validation never runs. Pair this module with an AWS Config rule (**required-tags**) that fires a non-compliant finding on any RDS instance missing your mandatory tag keys. You can enforce this organization-wide via AWS Organizations Service Control Policies to make untagged database creation impossible.

40.5 The Production Dashboard: Queries, Views, and Composite Metrics

A useful cost-performance dashboard answers five questions at a glance:

1. Are any instances currently under resource pressure?
2. Which queries are responsible for the most resource consumption in the last 24 hours?
3. Is there any replication lag that would affect read-replica routing decisions?
4. What is the current storage consumption trajectory — will we hit autoscaling in the next 30 days?
5. Are there any cost optimization opportunities visible right now?

PostgreSQL: Consolidated Health View

This view works on RDS for PostgreSQL, Aurora PostgreSQL, Cloud SQL for PostgreSQL, and AlloyDB. It aggregates the most cost-relevant metrics from the PostgreSQL statistics system. “sql – pg_cloud_dba_dashboard.sql – Unified health and cost-attribution view for PostgreSQL on any cloud provider. – Deploy as a view in a dedicated monitoring schema. – Requires: pg_stat_statements extension enabled (add to shared_preload_libraries – via Parameter Group on RDS, or via database flags on Cloud SQL). – On RDS: pg_stat_statements is available but must be enabled in the parameter group. – On Cloud SQL: add cloudsql.enable_pg_stat_statements=on to database flags. – On AlloyDB: enabled by default in pg_stat_statements compatible form.

CREATE SCHEMA IF NOT EXISTS dba_monitor;

– View 1: Top queries by total cost (CPU + I/O composite) CREATE OR REPLACE VIEW dba_monitor.top_queries_by_cost AS WITH query_stats AS (SELECT pss.userid, pss.dbid, pss.queryid, pss.query, pss.calls, pss.total_exec_time, pss.mean_exec_time, pss.stddev_exec_time, pss.rows, pss.shared_blks_hit, pss.shared_blks_read, – Physical reads = storage I/O cost pss.shared_blks_dirtied, pss.shared_blks_written, pss.temp_blks_read, – Temp block usage = work_mem spill to disk pss.temp_blks_written, pss.blk_read_time, pss.blk_write_time, – Buffer hit ratio per query — below 95% indicates I/O pressure CASE WHEN (pss.shared_blks_hit + pss.shared_blks_read) > 0 THEN ROUND((pss.shared_blks_hit::NUMERIC / (pss.shared_blks_hit + pss.shared_blks_read) * 100), 2) ELSE 100.0 END AS buffer_hit_pct, – Total I/O time as percentage of execution time CASE WHEN pss.total_exec_time > 0 THEN ROUND(((pss.blk_read_time + pss.blk_write_time) / pss.total_exec_time * 100), 2) ELSE 0 END AS io_time_pct, – Work mem spill indicator — temp blocks > 0 means sort/hash went to disk CASE WHEN pss.temp_blks_written > 0 THEN ‘SPILLING_TO_DISK’ ELSE ‘IN_MEMORY’ END AS sort_behavior, – Composite cost score (normalized) – Higher = more resource-intensive = higher cloud cost contribution ROUND((pss.total_exec_time / NULLIF(SUM(pss.total_exec_time) OVER (), 0) * 60) + (pss.shared_blks_read / NULLIF(SUM(pss.shared_blks_read) OVER (), 0) * 40), 4) AS composite_cost_score FROM pg_stat_statements pss JOIN pg_database pd ON pss.dbid = pd.oid WHERE pd.datname = current_database() AND pss.calls > 10 – Filter out one-off queries AND pss.mean_exec_time > 1 – Filter sub-millisecond queries) SELECT queryid, LEFT(query, 150) AS query_preview, calls, ROUND(total_exec_time::NUMERIC, 2) AS total_exec_time_ms, ROUND(mean_exec_time::NUMERIC, 2) AS mean_exec_time_ms, ROUND(stddev_exec_time::NUMERIC, 2) AS stddev_exec_time_ms, – High stddev relative to mean = plan instability CASE WHEN stddev_exec_time > mean_exec_time * 0.5 THEN ‘UNSTABLE’ ELSE ‘STABLE’ END AS execution_stability, ROUND(rows::NUMERIC / NUL-

LIF(calls, 0), 0) AS avg_rows_per_call, shared_blks_hit, shared_blks_read, buffer_hit_pct, io_time_pct, temp_blks_written, sort_behavior, composite_cost_score, RANK() OVER (ORDER BY composite_cost_score DESC) AS cost_rank FROM query_stats ORDER BY composite_cost_score DESC LIMIT 25;

- View 2: Table-level bloat and I/O pressure – Bloat causes unnecessary I/O → unnecessary storage reads → unnecessary cloud cost
 CREATE OR REPLACE VIEW dba_monitor.table_io_and_bloat AS SELECT schemaname, relname AS table_name, n_live_tup AS live_rows, n_dead_tup AS dead_rows, – Dead tuple ratio > 10% indicates autovacuum is not keeping up CASE WHEN (n_live_tup + n_dead_tup) > 0 THEN ROUND(n_dead_tup::NUMERIC / (n_live_tup + n_dead_tup) * 100, 2) ELSE 0 END AS dead_tuple_pct, last_vacuum, last_autovacuum, last_analyze, last_autoanalyze, heap_blks_read AS heap_physical_reads, heap_blks_hit AS heap_buffer_hits, CASE WHEN (heap_blks_hit + heap_blks_read) > 0 THEN ROUND(heap_blks_hit::NUMERIC / (heap_blks_hit + heap_blks_read) * 100, 2) ELSE 100 END AS table_buffer_hit_pct, idx_blks_read AS index_physical_reads, idx_blks_hit AS index_buffer_hits, – pg_relation_size requires access but gives actual on-disk bytes pg_size_pretty(pg_total_relation_size(schemaname || ‘ ’ || relname)) AS total_size, pg_total_relation_size(schemaname || ‘ ’ || relname) AS total_size_bytes FROM pg_stat_user_tables pst JOIN pg_statio_user_tables psio USING (schemaname, relname) WHERE pg_total_relation_size(schemaname || ‘ ’ || relname) > 104857600 – > 100 MB ORDER BY total_size_bytes DESC;
- View 3: Replication lag monitoring (read replica routing decisions) – On RDS read replicas, lag > 30 seconds should trigger alerts. – On Aurora, replica lag is typically < 100ms but can spike during DDL. – On Cloud SQL read replicas, lag > 60 seconds is the default alert threshold. CREATE OR REPLACE VIEW dba_monitor.replication_health AS SELECT client_addr AS replica_address, application_name AS replica_name, state, sent_lsn, write_lsn, flush_lsn, replay_lsn, – Bytes of WAL not yet replayed on the standby (sent_lsn - replay_lsn) AS replication_lag_bytes, pg_size_pretty(sent_lsn - replay_lsn) AS replication_lag_pretty, write_lag, flush_lag, replay_lag, sync_state, – Flag replicas that are falling behind threshold CASE WHEN (sent_lsn - replay_lsn) > 536870912 – 512 MB lag THEN ‘CRITICAL’ WHEN (sent_lsn - replay_lsn) > 104857600 – 100 MB lag THEN ‘WARNING’ ELSE ‘HEALTHY’ END AS lag_status FROM pg_stat_replication ORDER BY replication_lag_bytes DESC NULLS LAST;
- View 4: Connection pool pressure – On RDS, max_connections is set by instance class and cannot exceed it. – db.r6g.large: max_connections ~ 683 (formula: LEAST(DBInstanceClassMemory/9531392, 5000)) – If you are within 20% of max_connections, you need RDS Proxy or pgBouncer. CREATE OR REPLACE VIEW dba_monitor.connection_pressure AS SELECT datname AS database_name, numbackends AS active_connections, xact_commit AS committed_transactions, xact_rollback AS rolled_back_transactions, ROUND(xact_rollback::NUMERIC / NULLIF(xact_commit + xact_rollback, 0) * 100, 2) AS rollback_rate_pct, blks_read AS physical_reads, blks_hit AS buffer_hits, ROUND(blks_hit::NUMERIC / NULLIF(blks_hit + blks_read, 0) * 100, 2) AS db_buffer_hit_pct, deadlocks, temp_files, temp_bytes, pg_size_pretty(temp_bytes) AS temp_usage_pretty FROM pg_stat_database WHERE datname = current_database();
- Composite dashboard query: run this to get a single-screen health snapshot – Useful for morning standup checks and on-call handoff summaries SELECT ‘TOP_QUERIES’ AS metric_group, cost_rank::TEXT AS rank, query_preview AS detail, mean_exec_time_ms::TEXT AS value, ‘ms avg’ AS unit, execution_stability AS status FROM dba_monitor.top_queries_by_cost WHERE

```
cost_rank <= 5
```

```
UNION ALL
```

```
SELECT 'REPLICATION', replica_name, state, replication_lag_bytes::TEXT, 'bytes lag',
lag_status FROM dba_monitor.replication_health
```

```
UNION ALL
```

```
SELECT 'TABLE_HEALTH', table_name, 'dead_tuple_pct:' || dead_tuple_pct::TEXT,
table_buffer_hit_pct::TEXT, '% buffer hit', CASE WHEN dead_tuple_pct > 10 THEN
'BLOATED' WHEN table_buffer_hit_pct < 95 THEN 'IO_PRESSURE' ELSE 'HEALTHY'
END FROM dba_monitor.table_io_and_bloat WHERE dead_tuple_pct > 5 OR ta-
ble_buffer_hit_pct < 95 LIMIT 10
```

```
ORDER BY metric_group, rank;
```

****Cloud SQL and AlloyDB: Augmenting with Cloud Monitoring Metrics****

The SQL views above cover internal PostgreSQL telemetry. For external infrastructure metrics - CPU utilization at the VM level, network throughput, and storage saturation - you need Cloud Monitoring queries. On AlloyDB, the separation between the analytics-optimized

```
```bash
```

```
Query Cloud Monitoring for Cloud SQL CPU utilization over the last hour
Useful for correlating database-side stats with infrastructure-side pressure
```

```
PROJECT_ID="my-gcp-project"
INSTANCE_ID="prod-postgres-instance"
START_TIME=$(date -u -d '1 hour ago' '+%Y-%m-%dT%H:%M:%SZ')
END_TIME=$(date -u '+%Y-%m-%dT%H:%M:%SZ')
```

```
Fetch CPU utilization time series
```

```
gcloud monitoring time-series list \
 --filter='metric.type="cloudsql.googleapis.com/database/cpu/utilization" AND resource.labels
 --interval-start-time="${START_TIME}" \
 --interval-end-time="${END_TIME}" \
 --project="${PROJECT_ID}" \
 --format="table(
 points[0].interval.endTime,
 points[0].value.doubleValue
)"
```

```
Fetch storage utilization - approaching max triggers autoresize
```

```
Cost implication: Cloud SQL autoresize adds 25 GB increments automatically
```

```
gcloud monitoring time-series list \
 --filter='metric.type="cloudsql.googleapis.com/database/disk/utilization" AND resource.labels
 --interval-start-time="${START_TIME}" \
 --interval-end-time="${END_TIME}" \
 --project="${PROJECT_ID}" \
```

```
--format="table(
 points[0].interval.endTime,
 points[0].value.doubleValue
)"

Fetch replication lag for read replicas (in seconds)
Alert threshold: > 30 seconds for OLTP, > 300 seconds for analytics replicas
gcloud monitoring time-series list \
 --filter='metric.type="cloudsql.googleapis.com/database/replication/replica_lag" AND resource.type="cloudsql_instance"' \
 --interval-start-time="${START_TIME}" \
 --interval-end-time="${END_TIME}" \
 --project="${PROJECT_ID}" \
 --format="table(
 points[0].interval.endTime,
 points[0].value.doubleValue
)"

Check connections approaching limit
Cloud SQL max_connections defaults vary: 25 for db-f1-micro, 4000 for db-custom-96-624
gcloud monitoring time-series list \
 --filter='metric.type="cloudsql.googleapis.com/database/postgresql/num_backends" AND resource.type="cloudsql_instance"' \
 --interval-start-time="${START_TIME}" \
 --interval-end-time="${END_TIME}" \
 --project="${PROJECT_ID}" \
 --format="table(
 points[0].interval.endTime,
 points[0].value.int64Value
)"

echo ""
echo "Storage cost reminder:"
echo " Cloud SQL SSD: \$0.170/GB/month"
echo " Cloud SQL HDD: \$0.090/GB/month"
echo " AlloyDB: \$0.204/GB/month (but includes columnar + cache)"
echo " Autoresize adds 25 GB increments - review storage monthly"
```

---

## 40.6 Reserved Capacity, Right-Sizing, and the 30-Day Review Cadence

The SQL views and CLI commands in this chapter give you the data. The 30-day review cadence is how you act on it. Cloud database cost optimization is not a one-time project — it is an operational habit, the same way on-premises DBAs ran index maintenance jobs and reviewed wait statistics weekly.

### The Right-Sizing Decision Framework

Right-sizing is determining whether your provisioned instance class matches your actual workload. The failure mode almost every migrating team hits is provisioning based on peak on-premises hardware specifications and never revisiting it. An on-premises server with 256 GB RAM running at 30% utilization becomes a `db.r6g.16xlarge` (\$5.73/hour) in RDS when a `db.r6g.2xlarge` (\$0.96/hour) would handle the actual workload. The delta is approximately \$3,000/month per instance.

### Right-sizing signals and thresholds:

Metric	Downsize Candidate	Correctly Sized	Upsize Needed
Average CPU (30-day)	< 20%	20–70%	Sustained > 75%
FreeableMemory	Always > 4 GB free	1–4 GB free	< 512 MB free
ReadIOPS vs. provisioned	< 40% of baseline	40–80%	Sustained > 85%
DB load vs. vCPU count	< 0.5x vCPU	0.5–0.8x vCPU	> 1x vCPU sustained
Max connections used	< 30% of max	30–70% of max	> 80% of max

When `FreeableMemory` is consistently above 4 GB on a `db.r6g.xlarge` (32 GB RAM), you have headroom to drop to `db.r6g.large` (16 GB RAM). Verify using 30-day CloudWatch data, not 7-day — you need to capture the end-of-month reporting spike, the weekly batch, and any irregular jobs before making the call.

### Aurora Serverless v2 as an Alternative to Right-Sizing

For workloads with genuine variability — development environments, reporting databases that spike on business hours, staging systems — Aurora Serverless v2 eliminates the right-sizing problem by billing per ACU-second. A development Aurora cluster set to min 0.5 ACU, max 4 ACU costs essentially nothing overnight and scales during business hours. The gotcha: Aurora Serverless v2 has a minimum of 0.5 ACU, which still costs approximately \$0.06/hour (\$43/month) even when idle. For a completely idle environment, stopping the cluster entirely (RDS supports stopping instances for up to 7 days) is free.

### The 30-Day Cost and Performance Review Checklist

Run this review on the first Monday of each month across all production database instances:

```
#!/usr/bin/env bash
monthly_rds_cost_review.sh
Run this script monthly to identify cost optimization opportunities.
Requires: AWS CLI configured with read access to CloudWatch, RDS, and Cost Explorer.

set -euo pipefail

REGION="${AWS_DEFAULT_REGION:-us-east-1}"
LOOKBACK_DAYS=30
END_DATE=$(date +%Y-%m-%d)
START_DATE=$(date -d "${LOOKBACK_DAYS} days ago" +%Y-%m-%d)

echo "=====
echo " Monthly RDS Cost and Performance Review"
echo " Period: ${START_DATE} to ${END_DATE}"
echo " Region: ${REGION}"
echo "=====
echo ""
```

```

1. List all RDS instances and their classes
echo "--- Active RDS Instances ---"
aws rds describe-db-instances \
 --region "${REGION}" \
 --query 'DBInstances[?DBInstanceStatus==`available`].{
 ID:DBInstanceIdentifier,
 Class:DBInstanceClass,
 Engine:Engine,
 EngineVersion:EngineVersion,
 Storage:AllocatedStorage,
 StorageType:StorageType,
 MultiAZ:MultiAZ,
 PEnabled:PerformanceInsightsEnabled
 }' \
 --output table

echo ""

2. Check for instances without mandatory tags (compliance gap)
echo "--- Instances Missing Required Tags ---"
aws rds describe-db-instances \
 --region "${REGION}" \
 --query 'DBInstances[?DBInstanceStatus==`available`].{
 ID:DBInstanceIdentifier,
 Tags:TagList
 }' \
 --output json | python3 -c "
import json, sys

REQUIRED_TAGS = {'team', 'application', 'environment', 'cost-center'}
data = json.load(sys.stdin)

missing_found = False
for instance in data:
 tag_keys = {t['Key'] for t in instance.get('Tags', [])}
 missing = REQUIRED_TAGS - tag_keys
 if missing:
 missing_found = True
 print(f" MISSING TAGS: {instance['ID']} - missing: {' '.join(sorted(missing))}")

if not missing_found:
 print(' All instances have required tags. Good.')
"

echo ""

3. Identify instances not covered by Reserved Instances
echo "--- Reserved Instance Coverage Check ---"
Get active reservations
aws rds describe-reserved-db-instances \
 --region "${REGION}" \
 --query 'ReservedDBInstances[?State==`active`].{
 Class:DBInstanceClass,
 Engine:ProductDescription,
 Count:DBInstanceCount,
 ExpirationDate:EndTime
 }' \
 --output table

echo ""
echo " Compare the above reservations against active instances."
echo " Any provisioned instance class WITHOUT a matching reservation"
echo " is running at On-Demand price - typically 40-60% more expensive."

echo ""

4. Check for gp2 storage volumes that should be migrated to gp3
echo "--- Storage Type Audit (gp2 → gp3 migration candidates) ---"
aws rds describe-db-instances \

```



regions, intended as directional guidance rather than quotes:

Configuration	AWS	Azure	GCP
4 vCPU / 32 GB RAM PostgreSQL	db.r6g.xlarge \$0.48/hr	GP2, 4 vCore \$0.514/hr	db-custom-4-26624 \$0.52/hr
16 vCPU / 128 GB RAM PostgreSQL	db.r6g.4xlarge \$1.92/hr	GP2, 16 vCore \$2.056/hr	db-custom-16-106496 \$2.08/hr
Managed HA surcharge	Multi-AZ: ~2x	Zone Redundant: ~1.5x	HA: ~2x
Storage (SSD, per GB/month)	gp3: \$0.115	\$0.115	\$0.170
Read replica 1-year Reserved/Committed	Same class price ~37% discount	Same class price ~33% discount (reserved)	Same class price ~37% discount (1yr commit)

The replication and HA cost difference is notable: Azure SQL Zone Redundant service tier costs approximately 1.5x single-zone, while AWS Multi-AZ and GCP HA both cost roughly 2x. For a budget-constrained environment where Zone Redundant Azure SQL provides sufficient HA, this can be a meaningful saving.

## Key Takeaways

- **Cost and performance are inseparable in cloud databases.** Every query optimization — adding an index, rewriting a join, tuning `work_mem` — has a measurable cost consequence because cloud instances bill continuously for compute and I/O. Build the habit of evaluating performance fixes through both a latency lens and a cost lens simultaneously.
- **Telemetry must be configured explicitly before you need it.** Enhanced Monitoring, Performance Insights, Azure Diagnostic Settings, and Cloud SQL Query Insights are not on by default. Provision them at instance creation time via Terraform or your IaC pipeline — waiting until an incident occurs means you have no historical data when you need it most.
- **Tagging is not administrative overhead — it is financial infrastructure.** An untagged RDS instance is an untraceable cost. Enforce mandatory tag keys (`team`, `application`, `environment`, `cost-center`) through Terraform variable validation, AWS Config rules, and Azure Policy at the organization level. Cost reports without tag coverage are noise.
- **The 30-day review cadence converts visibility into savings.** Run the monthly right-sizing audit consistently. The three highest-ROI actions available to most cloud DBAs are: purchasing Reserved Instances for stable production workloads (37% savings), migrating gp2 volumes to gp3 (equal cost, better performance), and right-sizing instances based on 30-day CloudWatch averages rather than peak projections. Together these routinely reduce database spend by 40–55% in mature cloud environments.
- **A cloud DBA's operational value is measured in both performance SLAs and cost efficiency.** The tools in this book — RDS Parameter Groups, Aurora Serverless scaling, Azure Elastic Pools, AlloyDB columnar acceleration, Spanner's global consistency, DMS

migration pipelines, RDS Proxy connection management — all exist within the constraint of a billing meter. The engineers who master both dimensions, who can tune a query and read a Cost Explorer report with equal fluency, are the ones who earn a permanent seat at the architecture table.

---

*This concludes The Cloud Database Engineer: AWS RDS, Azure SQL, and Google Cloud for Production DBAs.*

*The five books in this series — from the foundational DBA Handbook through AI-powered operations, autonomous systems, vector databases, and now cloud-managed platforms — represent a complete curriculum for the modern database engineer. The craft evolves. The fundamentals persist. Carry both forward.*